

MPI: A Message-Passing Interface Standard

Version 5.1

(Draft)

Unofficial, for comment only

Message Passing Interface Forum

October 9, 2026

1 This document describes the 202X Draft Specification of the Message-Passing Interface
2 (MPI) standard, intended for comment. This is a working document; the MPI Forum
3 continues to work on MPI-5.1 and expects additional changes before MPI-5.1 is finalized. It
4 is not an official version of the standard.

5 This document describes the Message-Passing Interface (MPI) standard, version 5.1.

6 The MPI standard includes point-to-point message-passing, collective communications,
7 group and communicator concepts, process topologies, environmental management, process
8 creation and management, one-sided communications, extended collective operations, ex-
9 ternal interfaces, I/O, some miscellaneous topics, and multiple tool interfaces. Language
10 bindings for C and Fortran are defined.

11 Historically, the evolution of the standard is:

- 12 • MPI-1.0 (May 5, 1994): Initial release.
- 13
- 14 • MPI-1.1 (June 12, 1995): Minor updates and bug fixes.
- 15
- 16 • MPI-1.2 (July 18, 1997): Several clarifications and additions.
- 17
- 18 • MPI-2.0 (July 18, 1997): New functionality and all the clarifications and additions
19 from MPI-1.2.
- 20
- 21 • MPI-1.3 (May 30, 2008): For historical reasons, combining the MPI-1.1, MPI-1.2, and
22 several errata documents into one combined document.
- 23
- 24 • MPI-2.1 (June 23, 2008): Combining the previous documents.
- 25
- 26 • MPI-2.2 (September 4, 2009): Additional clarifications and seven new routines.
- 27
- 28 • MPI-3.0 (September 21, 2012): Extension of MPI-2.2.
- 29
- 30 • MPI-3.1 (June 4, 2015): Clarifications and minor extensions to MPI-3.0.
- 31
- 32 • MPI-4.0 (June 9, 2021): Significant new features beyond MPI-3.1.
- 33
- 34 • MPI-4.1 (November 2, 2023): Clarifications and minor extensions to MPI-4.0.
- 35
- 36 • MPI-5.0 (June 5, 2025): Significant new features beyond MPI-4.1.

37 Comments. Please send comments on MPI to the MPI Forum as follows:

- 38 1. Subscribe to <https://lists.mpi-forum.org/mailman/listinfo/mpi-comments>
- 39 2. Send your comment to: mpi-comments@lists.mpi-forum.org, together with the version
40 of the MPI standard and the page and line numbers on which you are commenting.

41 Your comment will be forwarded to MPI Forum committee members for consideration.
42 Messages sent from an unsubscribed e-mail address will not be considered.

43
44 ©1993, 1994, 1995, 1996, 1997, 2008, 2009, 2012, 2015, 2021, 2023, 2025 University of
45 Tennessee, Knoxville, Tennessee. Permission to copy without fee all or part of this mate-
46 rial is granted, provided the University of Tennessee copyright notice and the title of this
47 document appear, and notice is given that copying is by permission of the University of
48 Tennessee.

202X Draft Specification, XXXX, 202X. This document contains a draft of the MPI specification as of the date of publication. It has not been adopted as an official MPI specification, and is provided for comment only. This document includes updates and new features that will be present in the final MPI-5.1 document.

Version 5.0: June 5, 2025. This version of the MPI-5 standard is a major update and includes significant new functionality. The largest change is the addition of a standard Application Binary Interface (ABI) to allow interoperability of different implementations.

Version 4.1: November 2, 2023. This version of the MPI-4.1 standard contains mostly corrections and clarifications to the MPI-4.0 document. Several routines, the attribute key `MPI_HOST`, and the `mpif.h` Fortran include file are deprecated.

Version 4.0: June 9, 2021. This version of the MPI-4 standard is a major update and includes significant new functionality. The largest changes are the addition of large-count versions of many routines to address the limitations of using an `int` or `INTEGER` for the count parameter, persistent collectives, partitioned communications, an alternative way to initialize MPI, application info assertions, and improvements to the definitions of error handling. In addition, there are a number of smaller improvements and corrections.

Version 3.1: June 4, 2015. This document contains mostly corrections and clarifications to the MPI-3.0 document. The largest change is a correction to the Fortran bindings introduced in MPI-3.0. Additionally, new functions added include routines to manipulate `MPI_Aint` values in a portable manner, nonblocking collective I/O routines, and routines to get the index value by name for `MPI_T` performance and control variables.

Version 3.0: September 21, 2012. Coincident with the development of MPI-2.2, the MPI Forum began discussions of a major extension to MPI. This document contains the MPI-3 standard. This version of the MPI-3 standard contains significant extensions to MPI functionality, including nonblocking collectives, new one-sided communication operations, and Fortran 2008 bindings. Unlike MPI-2.2, this standard is considered a major update to the MPI standard. As with previous versions, new features have been adopted only when there were compelling needs for the users. Some features, however, may have more than a minor impact on existing MPI implementations.

Version 2.2: September 4, 2009. This document contains mostly corrections and clarifications to the MPI-2.1 document. A few extensions have been added; however all correct MPI-2.1 programs are correct MPI-2.2 programs. New features were adopted only when there were compelling needs for users, open source implementations, and minor impact on existing MPI implementations.

Version 2.1: June 23, 2008. This document combines the previous documents MPI-1.3 (May 30, 2008) and MPI-2.0 (July 18, 1997). Certain parts of MPI-2.0, such as some sections of Chapter 4, Miscellany, and Chapter 7, Extended Collective Operations, have been merged into the chapters of MPI-1.3. Additional errata and clarifications collected by the MPI Forum are also included in this document.

1 Version 1.3: May 30, 2008. This document combines the previous documents MPI-1.1
2 (June 12, 1995) and the MPI-1.2 chapter in MPI-2 (July 18, 1997). Additional errata
3 collected by the MPI Forum referring to MPI-1.1 and MPI-1.2 are also included in this
4 document.

5
6 Version 2.0: July 18, 1997. Beginning after the release of MPI-1.1, the MPI Forum began
7 meeting to consider corrections and extensions. MPI-2 has been focused on process creation
8 and management, one-sided communications, extended collective communications, external
9 interfaces and parallel I/O. A miscellany chapter discusses items that do not fit elsewhere,
10 in particular language interoperability.

11
12 Version 1.2: July 18, 1997. The MPI-2 Forum introduced MPI-1.2 as Chapter 3 in the
13 standard “MPI-2: Extensions to the Message-Passing Interface”, July 18, 1997. This section
14 contains clarifications and minor corrections to Version 1.1 of the MPI standard. The only
15 new function in MPI-1.2 is one for identifying to which version of the MPI standard the
16 implementation conforms. There are small differences between MPI-1 and MPI-1.1. There
17 are very few differences between MPI-1.1 and MPI-1.2, but large differences between MPI-1.2
18 and MPI-2.

19
20 Version 1.1: June, 1995. Beginning in March, 1995, the Message-Passing Interface Forum
21 reconvened to correct errors and make clarifications in the MPI document of May 5, 1994,
22 referred to below as Version 1.0. These discussions resulted in Version 1.1. The changes
23 from Version 1.0 are minor. A version of this document with all changes marked is available.

24
25 Version 1.0: May, 1994. The Message-Passing Interface Forum, with participation from
26 over 40 organizations, has been meeting since January 1993 to discuss and define a set of
27 library interface standards for message passing. The Message-Passing Interface Forum is
28 not sanctioned or supported by any official standards organization.

29 The goal of the Message-Passing Interface, simply stated, is to develop a widely used
30 standard for writing message-passing programs. As such the interface should establish a
31 practical, portable, efficient, and flexible standard for message-passing.

32 This is the final report, Version 1.0, of the Message-Passing Interface Forum. This
33 document contains all the technical features proposed for the interface. This copy of the
34 draft was processed by L^AT_EX on May 5, 1994.

Contents

List of Figures	xx
List of Tables	xxii
List of Examples	xxiv
Acknowledgments	xxix
1 Introduction to MPI	1
1.1 Overview and Goals	1
1.2 Background of MPI-1.0	2
1.3 Background of MPI-1.1, MPI-1.2, and MPI-2.0	2
1.4 Background of MPI-1.3 and MPI-2.1	3
1.5 Background of MPI-2.2	4
1.6 Background of MPI-3.0	4
1.7 Background of MPI-3.1	4
1.8 Background of MPI-4.0	4
1.9 Background of MPI-4.1	5
1.10 Background of MPI-5.0	5
1.11 Background of 202X Draft Specification	5
1.12 Who Should Use This Standard?	5
1.13 What Platforms Are Targets for Implementation?	5
1.14 What Is Included in the Standard?	6
1.15 Side-documents	7
1.16 Organization of This Document	7
2 MPI Terms and Conventions	11
2.1 Document Notation	11
2.2 Naming Conventions	11
2.3 Procedure Specification	12
2.4 Semantic Terms	13
2.4.1 MPI Operations	13
2.4.2 MPI Procedures	16
2.4.3 MPI Datatypes	18
2.5 Datatypes	19
2.5.1 Opaque Objects	19
2.5.2 Array Arguments	21
2.5.3 State	21
2.5.4 Named Constants	21
2.5.5 Choice	22
2.5.6 Absolute Addresses and Relative Address Displacements	23
2.5.7 File Offsets	23

2.5.8	Counts	23
2.6	Language Binding	24
2.6.1	Deprecated and Removed Interfaces	24
2.6.2	Fortran Binding Issues	25
2.6.3	C Binding Issues	26
2.6.4	Functions and Macros	27
2.7	Processes	27
2.8	Error Handling	28
2.9	Progress	29
2.10	Implementation Issues	31
2.10.1	Independence of Basic Runtime Routines	31
2.10.2	Interaction with Signals	31
2.11	Examples	32
3	Point-to-Point Communication	33
3.1	Introduction	33
3.2	Blocking Send and Receive Operations	34
3.2.1	Blocking Send	34
3.2.2	Message Data	35
3.2.3	Message Envelope	37
3.2.4	Blocking Receive	38
3.2.5	Return Status	40
3.2.6	Passing MPI_STATUS_IGNORE for Status	44
3.2.7	Blocking Send-Receive	45
3.3	Datatype Matching and Data Conversion	48
3.3.1	Type Matching Rules	48
	Type MPI_CHARACTER	50
3.3.2	Data Conversion	51
3.4	Communication Modes	52
3.5	Semantics of Point-to-Point Communication	57
3.6	Buffer Allocation and Usage	60
3.6.1	Model Implementation of Buffered Mode	70
3.7	Nonblocking Communication	71
3.7.1	Communication Request Objects	72
3.7.2	Communication Initiation	73
3.7.3	Communication Completion	80
3.7.4	Semantics of Nonblocking Communication Operations	84
3.7.5	Multiple Completions	85
3.7.6	Non-Destructive Test of status	92
3.8	Probe and Cancel	95
3.8.1	Probe	96
3.8.2	Matching Probe	99
3.8.3	Matched Receives	101
3.8.4	Cancel	103
3.9	Persistent Communication Requests	105
3.10	Null MPI Processes	112

4	Partitioned Point-to-Point Communication	113
4.1	Introduction	113
4.2	Semantics of Partitioned Point-to-Point Communication	114
4.2.1	Communication Initialization and Starting with Partitioning	116
4.2.2	Communication Completion under Partitioning	120
4.2.3	Semantics of Communications in Partitioned Mode	121
4.3	Partitioned Communication Examples	121
4.3.1	Partition Communication with Threads/Tasks Using OpenMP 4.0 or later	122
4.3.2	Send-only Partitioning Example with Tasks and OpenMP version 4.0 or later	123
4.3.3	Send and Receive Partitioning Example with OpenMP version 4.0 or later	124
5	Datatypes	127
5.1	Derived Datatypes	127
5.1.1	Type Constructors with Explicit Addresses	129
5.1.2	Datatype Constructors	129
5.1.3	Subarray Datatype Constructor	140
5.1.4	Distributed Array Datatype Constructor	143
5.1.5	Address and Size Procedures	148
5.1.6	Lower-Bound and Upper-Bound Markers	151
5.1.7	Extent and Bounds of Datatypes	153
5.1.8	True Extent of Datatypes	154
5.1.9	Commit and Free	156
5.1.10	Duplicating a Datatype	157
5.1.11	Use of General Datatypes in Communication	158
5.1.12	Correct Use of Addresses	161
5.1.13	Decoding a Datatype	162
5.1.14	Examples	170
5.2	Pack and Unpack	177
5.3	Canonical MPI_PACK and MPI_UNPACK	184
6	Collective Communication	189
6.1	Introduction and Overview	189
6.2	Communicator Argument	192
6.2.1	Specifics for Intra-Communicator Collective Operations	192
6.2.2	Applying Collective Operations to Inter-Communicators	193
6.2.3	Specifics for Inter-Communicator Collective Operations	194
6.3	Barrier Synchronization	196
6.4	Broadcast	196
6.4.1	Example using MPI_BCAST	197
6.5	Gather	198
6.5.1	Examples using MPI_GATHER and MPI_GATHERV	202
6.6	Scatter	208
6.6.1	Examples using MPI_SCATTER and MPI_SCATTERV	212
6.7	All-Gather	214
6.7.1	Example using MPI_ALLGATHER	217

6.8	All-to-All Scatter/Gather	218
6.9	Global Reduction Operations	224
6.9.1	Reduce	224
6.9.2	Predefined Reduction Operations	226
6.9.3	Signed Characters and Reductions	229
6.9.4	MINLOC and MAXLOC	229
6.9.5	User-Defined Reduction Operations	235
	Example of User-Defined Reduce	238
6.9.6	All-Reduce	240
6.9.7	MPI Process-Local Reduction	241
6.10	Reduce-Scatter	243
6.10.1	MPI_REDUCE_SCATTER_BLOCK	243
6.10.2	MPI_REDUCE_SCATTER	245
6.11	Scan	246
6.11.1	Inclusive Scan	246
6.11.2	Exclusive Scan	248
6.11.3	Example using MPI_SCAN	249
6.12	Nonblocking Collective Operations	250
6.12.1	Nonblocking Barrier Synchronization	252
6.12.2	Nonblocking Broadcast	253
	Example using MPI_IBCAST	254
6.12.3	Nonblocking Gather	254
6.12.4	Nonblocking Scatter	257
6.12.5	Nonblocking All-Gather	259
6.12.6	Nonblocking All-to-All Scatter/Gather	262
6.12.7	Nonblocking Reduce	266
6.12.8	Nonblocking All-Reduce	267
6.12.9	Nonblocking Reduce-Scatter with Equal Blocks	268
6.12.10	Nonblocking Reduce-Scatter	269
6.12.11	Nonblocking Inclusive Scan	271
6.12.12	Nonblocking Exclusive Scan	272
6.13	Persistent Collective Operations	273
6.13.1	Persistent Barrier Synchronization	274
6.13.2	Persistent Broadcast	275
6.13.3	Persistent Gather	276
6.13.4	Persistent Scatter	278
6.13.5	Persistent All-Gather	281
6.13.6	Persistent All-to-All Scatter/Gather	284
6.13.7	Persistent Reduce	288
6.13.8	Persistent All-Reduce	290
6.13.9	Persistent Reduce-Scatter with Equal Blocks	291
6.13.10	Persistent Reduce-Scatter	292
6.13.11	Persistent Inclusive Scan	293
6.13.12	Persistent Exclusive Scan	294
6.14	Correctness	296

7	Groups, Contexts, Communicators, and Caching	305
7.1	Introduction	305
7.1.1	Features Needed to Support Libraries	305
7.1.2	MPI's Support for Libraries	305
7.2	Basic Concepts	307
7.2.1	Groups	308
7.2.2	Contexts	308
7.2.3	Intra-Communicators	309
7.2.4	Predefined Intra-Communicators	309
7.3	Group Management	310
7.3.1	Group Accessors	310
7.3.2	Group Constructors	312
7.3.3	Group Destructors	318
7.4	Communicator Management	318
7.4.1	Communicator Accessors	319
7.4.2	Communicator Constructors	321
7.4.3	Communicator Destructors	338
7.4.4	Communicator Info	339
7.5	Motivating Examples	342
7.5.1	Current Practice #1	342
7.5.2	Current Practice #2	343
7.5.3	(Approximate) Current Practice #3	343
7.5.4	Communication Safety Example	344
7.5.5	Library Example #1	345
7.5.6	Library Example #2	346
7.6	Inter-Communication	348
7.6.1	Inter-Communicator Accessors	350
7.6.2	Inter-Communicator Operations	352
7.6.3	Inter-Communication Examples	355
	Example 1: Three-Group "Pipeline"	355
	Example 2: Three-Group "Ring"	357
7.7	Caching	358
7.7.1	Functionality	359
7.7.2	Communicators	360
7.7.3	Windows	365
7.7.4	Datatypes	369
7.7.5	Error Class for Invalid Keyval	372
7.7.6	Attributes Example	373
7.8	Naming Objects	374
7.9	Formalizing the Loosely Synchronous Model	379
7.9.1	Basic Statements	379
7.9.2	Models of Execution	379
	Static Communicator Allocation	380
	Dynamic Communicator Allocation	380
	The General Case	380
8	Virtual Topologies for MPI Processes	381
8.1	Introduction	381

8.2	Virtual Topologies	382
8.3	Embedding in MPI	382
8.4	Overview of the Functions	383
8.5	Topology Constructors	384
8.5.1	Cartesian Constructor	384
8.5.2	Cartesian Convenience Function: <code>MPI_DIMS_CREATE</code>	385
8.5.3	Graph Constructor	386
8.5.4	Distributed Graph Constructor	388
8.5.5	Topology Inquiry Functions	394
8.5.6	Cartesian Shift Coordinates	403
8.5.7	Partitioning of Cartesian Structures	404
8.5.8	Low-Level Topology Functions	405
8.6	Neighborhood Collective Communication	407
8.6.1	Neighborhood Gather	408
8.6.2	Neighborhood Alltoall	412
8.7	Nonblocking Neighborhood Communication	419
8.7.1	Nonblocking Neighborhood Gather	419
8.7.2	Nonblocking Neighborhood Alltoall	422
8.8	Persistent Neighborhood Communication	426
8.8.1	Persistent Neighborhood Gather	426
8.8.2	Persistent Neighborhood Alltoall	429
8.9	An Application Example	433
9	MPI Environmental Management	439
9.1	Implementation Information	439
9.1.1	Version Inquiries	439
9.1.2	Environmental Inquiries	440
	Tag Values	441
	IO Rank	441
	Clock Synchronization	442
	Inquire Processor Name	442
	Inquire Hardware Resource Information	443
9.2	Memory Allocation	445
9.3	Error Handling	448
9.3.1	Error Handlers for Communicators	451
9.3.2	Error Handlers for Windows	453
9.3.3	Error Handlers for Files	455
9.3.4	Error Handlers for Sessions	457
9.3.5	Freeing Errorhandlers and Retrieving Error Strings	458
9.4	Error Codes and Classes	459
9.5	Error Classes, Error Codes, and Error Handlers	463
9.5.1	User-Defined Error Classes and Codes	463
9.5.2	Calling Error Handlers	467
9.6	Timers and Synchronization	469
10	The Info Object	471

11 Process Initialization, Creation, and Management	479
11.1 Introduction	479
11.2 The World Model	480
11.2.1 Starting MPI Processes	480
11.2.2 Finalizing MPI	486
11.2.3 Determining Whether MPI Has Been Initialized When Using the World Model	489
11.2.4 Allowing User Functions at MPI Finalization	490
11.3 The Sessions Model	491
11.3.1 Session Creation and Destruction Methods	492
11.3.2 Processes Sets	495
11.3.3 Runtime Query Functions	496
11.3.4 Sessions Model Examples	499
11.4 Common Elements of Both Process Models	504
11.4.1 MPI Functionality that is Always Available	504
11.4.2 Aborting MPI Processes	505
11.4.3 Memory Allocation Info	506
11.5 Portable MPI Process Startup	511
11.6 MPI and Threads	513
11.6.1 General	513
11.6.2 Clarifications	514
11.7 The Dynamic Process Model	516
11.7.1 Starting Processes	517
11.7.2 The Runtime Environment	517
11.8 Process Manager Interface	518
11.8.1 Processes in MPI	518
11.8.2 Starting Processes and Establishing Communication	518
11.8.3 Starting Multiple Executables and Establishing Communication	523
11.8.4 Reserved Keys	526
11.8.5 Spawn Example	527
11.9 Establishing Communication	528
11.9.1 Names, Addresses, Ports, and All That	529
11.9.2 Server Routines	530
11.9.3 Client Routines	532
11.9.4 Name Publishing	533
11.9.5 Reserved Key Values	536
11.9.6 Client/Server Examples	536
11.10 Other Functionality	538
11.10.1 Universe Size	538
11.10.2 Singleton MPI Initialization	539
11.10.3 MPI_APPNUM	540
11.10.4 Releasing Connections	540
11.10.5 Another Way to Establish MPI Communication	542
12 One-Sided Communications	545
12.1 Introduction	545
12.2 Initialization	546
12.2.1 Window Creation	546

12.2.2	Window That Allocates Memory	550
12.2.3	Window That Allocates Shared Memory	552
12.2.4	Window of Dynamically Attached Memory	556
12.2.5	Window Destruction	560
12.2.6	Window Attributes	560
12.2.7	Window Info	562
12.3	Communication Calls	564
12.3.1	Put	565
12.3.2	Get	567
12.3.3	Examples for Communication Calls	568
12.3.4	Accumulate Functions	570
	Accumulate	570
	Get Accumulate	573
	Fetch and Op	575
	Compare and Swap	576
12.3.5	Request-based RMA Communication Operations	577
12.4	Memory Model	584
12.5	Synchronization Calls	585
12.5.1	Fence	588
12.5.2	General Active Target Synchronization	590
12.5.3	Lock	595
12.5.4	Flush and Sync	598
12.5.5	Assertions	600
12.5.6	Miscellaneous Clarifications	602
12.6	Error Handling	602
12.6.1	Error Handlers	602
12.6.2	Error Classes	602
12.7	Semantics and Correctness	602
12.7.1	Atomicity	611
12.7.2	Ordering	611
12.7.3	Progress	612
12.7.4	Registers and Compiler Optimizations	615
12.8	Examples	615
13	Queued Communication	625
13.1	Introduction	625
13.2	Queue Objects	626
13.2.1	Enqueued Request State	626
13.2.2	Queue Creation and Destruction	626
13.2.3	Enqueuing Operations	628
13.2.4	Queue Completion Operations	631
13.2.5	Queue Types	632
13.3	Matched Persistent Operations	632
13.3.1	Partitioned Communication Requests	637
13.4	Concurrency and Progress	637
13.5	Examples	639

14 External Interfaces	643
14.1 Introduction	643
14.2 Generalized Requests	643
14.2.1 Examples	647
14.3 Associating Information with Status	649
15 I/O	653
15.1 Introduction	653
15.1.1 Definitions	653
15.2 File Manipulation	655
15.2.1 Opening a File	655
15.2.2 Closing a File	657
15.2.3 Deleting a File	658
15.2.4 Resizing a File	659
15.2.5 Preallocating Space for a File	660
15.2.6 Querying the Size of a File	660
15.2.7 Querying File Parameters	661
15.2.8 File Info	663
Reserved File Hints	664
15.3 File Views	666
15.4 Data Access	669
15.4.1 Data Access Routines	669
Positioning	669
Synchronism	670
Coordination	671
Data Access Conventions	671
15.4.2 Data Access with Explicit Offsets	672
15.4.3 Data Access with Individual File Pointers	679
15.4.4 Data Access with Shared File Pointers	689
Noncollective Operations	689
Collective Operations	692
Seek	695
15.4.5 Split Collective Data Access Routines	696
15.5 File Interoperability	705
15.5.1 Datatypes for File Interoperability	707
15.5.2 External Data Representation: "external32"	708
15.5.3 User-Defined Data Representations	709
Extent Callback	713
Datarep Conversion Functions	713
15.5.4 Matching Data Representations	716
15.6 Consistency and Semantics	716
15.6.1 File Consistency	716
15.6.2 Random Access vs. Sequential Files	720
15.6.3 Progress	720
15.6.4 Collective File Operations	720
15.6.5 Nonblocking Collective File Operations	720
15.6.6 Type Matching	721
15.6.7 Miscellaneous Clarifications	721

15.6.8	MPI_Offset Type	721
15.6.9	Logical vs. Physical File Layout	722
15.6.10	File Size	722
15.6.11	Examples	723
	Asynchronous I/O	725
15.7	I/O Error Handling	727
15.8	I/O Error Classes	727
15.9	Examples	727
15.9.1	Double Buffering with Split Collective I/O	727
15.9.2	Subarray Filetype Constructor	730
16	Tool Support	733
16.1	Introduction	733
16.2	Profiling Interface	733
16.2.1	Requirements	733
16.2.2	Discussion	734
16.2.3	Logic of the Design	734
16.2.4	MPI Library Implementation	735
16.2.5	Complications	736
	Multiple Counting	736
	Linker Oddities	737
	Fortran Support Methods	737
16.2.6	Multiple Levels of Interception	737
16.2.7	Miscellaneous Control of Profiling	737
16.3	The MPI Tool Information Interface	738
16.3.1	Verbosity Levels	740
16.3.2	Binding MPI Tool Information Interface Variables to MPI Objects	740
16.3.3	Convention for Returning Strings	741
16.3.4	Initialization and Finalization	742
16.3.5	Datatype System	743
16.3.6	Control Variables	745
	Control Variable Query Functions	745
	Handle Allocation and Deallocation	749
	Control Variable Access Functions	750
16.3.7	Performance Variables	752
	Performance Variable Classes	752
	Performance Variable Query Functions	754
	Performance Experiment Sessions	757
	Handle Allocation and Deallocation	757
	Starting and Stopping of Performance Variables	759
	Performance Variable Access Functions	760
16.3.8	Events	764
	Event Sources	764
	Callback Safety Requirements	766
	Event Type Query Functions	768
	Handle Allocation and Deallocation	771
	Handling Dropped Events	775
	Reading Event Data	776

Reading Event Meta Data	778
16.3.9 Variable Categorization	779
Category Query Functions	780
Category Member Query Functions	781
16.3.10 Return Codes for the MPI Tool Information Interface	783
16.3.11 Profiling Interface	784
17 Process Fault Tolerance	787
17.1 Introduction	787
17.2 Failure Notification	788
17.2.1 Revoked Communicator	788
17.2.2 Error Reporting Range	788
17.2.3 Fault Tolerance Errors in Point-to-Point Communication	789
17.2.4 Fault Tolerance Errors in Collective Operations	789
Error Uniformity	790
17.2.5 Files and One-sided Operations	790
17.3 Failure Mitigation Procedures	791
17.3.1 Communicator Procedures	791
17.4 Fault Tolerance Error Codes and Classes	794
17.5 Examples	794
17.5.1 Fault Tolerant Manager/Worker	794
18 Deprecated Interfaces	797
18.1 Deprecated since MPI-2.0	797
18.2 Deprecated since MPI-2.2	800
18.3 Deprecated since MPI-4.0	800
18.4 Deprecated since MPI-4.1	802
19 Removed Interfaces	807
19.1 Removed MPI-1 Bindings	807
19.1.1 Overview	807
19.1.2 Removed MPI-1 Functions	807
19.1.3 Removed MPI-1 Datatypes	807
19.1.4 Removed MPI-1 Constants	807
19.1.5 Removed MPI-1 Callback Prototypes	807
19.2 C++ Bindings	808
20 Semantic Changes and Warnings	809
20.1 Semantic Changes	809
20.1.1 Semantic Changes Starting in MPI-4.0	809
20.2 Additional Warnings	809
20.2.1 Warnings Starting in MPI-4.1	809
20.2.2 Warnings Starting in MPI-4.0	810
21 Language Bindings	811
21.1 Support for Fortran	811
21.1.1 Overview	811
21.1.2 Fortran Support Through the <code>mpi_f08</code> Module	812

21.1.3	Fortran Support Through the <code>mpi</code> Module	815
21.1.4	Fortran Support Through the <code>mpif.h</code> Include File	817
21.1.5	Interface Specifications, Procedure Names, and the Profiling Interface	818
21.1.6	MPI for Different Fortran Standard Versions	823
21.1.7	Requirements on Fortran Compilers	826
21.1.8	Additional Support for Fortran Register-Memory-Synchronization	828
21.1.9	Additional Support for Fortran Numeric Intrinsic Types	829
	Parameterized Datatypes with Specified Precision and Exponent Range	829
	Support for Size-specific MPI Datatypes	833
	Communication With Size-specific Types	836
21.1.10	Problems With Fortran Bindings for MPI	837
21.1.11	Problems Due to Strong Typing	838
21.1.12	Problems Due to Data Copying and Sequence Association with Subscript Triplets	839
21.1.13	Problems Due to Data Copying and Sequence Association with Vector Subscripts	842
21.1.14	Special Constants	842
21.1.15	Fortran Derived Types	843
21.1.16	Optimization Problems, an Overview	844
21.1.17	Problems with Code Movement and Register Optimization	846
	Nonblocking Operations	846
	Persistent Operations	847
	One-sided Communication	847
	MPI_BOTTOM and Combining Independent Variables in Datatypes Solutions	847
	The Fortran ASYNCHRONOUS Attribute	849
	Calling MPI_F_SYNC_REG	850
	A User Defined Routine Instead of MPI_F_SYNC_REG	851
	Module Variables and COMMON Blocks	852
	The (Poorly Performing) Fortran VOLATILE Attribute	852
	The Fortran TARGET Attribute	852
21.1.18	Temporary Data Movement and Temporary Memory Modification	853
21.1.19	Permanent Data Movement	855
21.1.20	Comparison with C	856
21.2	Support for Large Count and Large Byte Displacement	856
21.3	Language Interoperability	858
21.3.1	Introduction	858
21.3.2	Assumptions	858
21.3.3	Initialization	859
	Concerns specific to the World Model	859
	Concerns specific to the Sessions Model	859
	Concerns common to both the World Model and the Sessions Model	859
21.3.4	Transfer of Handles	860
21.3.5	Status	861
21.3.6	MPI Opaque Objects	864
	Datatypes	864

	Callback Functions	866
	Error Handlers	866
	Reduce Operations	866
21.3.7	Attributes	867
21.3.8	Extra-State	870
21.3.9	Constants	871
21.3.10	Interlanguage Communication	871
22	Application Binary Interface (ABI)	873
22.1	Introduction	873
22.2	Implementation Requirements	873
22.2.1	The MPI ABI Header File and Shared Library	875
22.3	The C Application Binary Interface	875
22.3.1	The Status Object	875
22.3.2	Opaque Handles	876
22.3.3	Handle Constants	876
22.3.4	Integer Constants	877
22.3.5	Integer Types	877
22.3.6	Calling Conventions and Binary Representations	878
22.4	The Fortran Application Binary Interface	878
22.4.1	Fortran Type Registration	878
22.4.2	The MPI ABI Fortran Modules and Shared Library	881
22.4.3	The Status Object	882
22.4.4	Integer Constants	882
22.4.5	Handle Serialization	882
22.5	Handle Constants	884
A	Language Bindings Summary	891
A.1	Defined Values and Handles	891
A.1.1	Defined Constants	891
A.1.2	Types	907
A.1.3	Prototype Definitions	908
	C Bindings	908
	Fortran 2008 Bindings with the <code>mpi_f08</code> Module	910
	Fortran Bindings with <code>mpif.h</code> or the <code>mpi</code> Module	912
A.1.4	Deprecated Prototype Definitions	914
A.1.5	String Values	915
	Default Communicator Names	915
	Default Datatype Names	915
	Default Window Names	915
	Reserved Data Representations	915
	Process Set Names	915
	Info Keys	916
	Info Values	917
A.2	Summary of the Semantics of all Op.-Related Routines	918
A.3	C Bindings	926
A.3.1	Point-to-Point Communication C Bindings	926
A.3.2	Partitioned Communication C Bindings	930

A.3.3	Datatypes C Bindings	930
A.3.4	Collective Communication C Bindings	933
A.3.5	Groups, Contexts, Communicators, and Caching C Bindings	941
A.3.6	Virtual Topologies for MPI Processes C Bindings	944
A.3.7	MPI Environmental Management C Bindings	947
A.3.8	The Info Object C Bindings	948
A.3.9	Process Creation and Management C Bindings	949
A.3.10	One-Sided Communications C Bindings	950
A.3.11	External Interfaces C Bindings	953
A.3.12	I/O C Bindings	953
A.3.13	Language Bindings C Bindings	957
A.3.14	Application Binary Interface (ABI) C Bindings	958
A.3.15	Tools / Profiling Interface C Bindings	959
A.3.16	Tools / MPI Tool Information Interface C Bindings	959
A.3.17	Deprecated C Bindings	962
A.4	Fortran 2008 Bindings with the <code>mpi_f08</code> Module	963
A.4.1	Point-to-Point Communication Fortran 2008 Bindings	963
A.4.2	Partitioned Communication Fortran 2008 Bindings	975
A.4.3	Datatypes Fortran 2008 Bindings	976
A.4.4	Collective Communication Fortran 2008 Bindings	983
A.4.5	Groups, Contexts, Communicators, and Caching Fortran 2008 Bindings	1005
A.4.6	Virtual Topologies for MPI Processes Fortran 2008 Bindings	1012
A.4.7	MPI Environmental Management Fortran 2008 Bindings	1021
A.4.8	The Info Object Fortran 2008 Bindings	1024
A.4.9	Process Creation and Management Fortran 2008 Bindings	1025
A.4.10	One-Sided Communications Fortran 2008 Bindings	1028
A.4.11	External Interfaces Fortran 2008 Bindings	1035
A.4.12	I/O Fortran 2008 Bindings	1036
A.4.13	Language Bindings Fortran 2008 Bindings	1049
A.4.14	Application Binary Interface (ABI) Fortran 2008 Bindings	1049
A.4.15	Tools / Profiling Interface Fortran 2008 Bindings	1050
A.4.16	Deprecated Fortran 2008 Bindings	1050
A.5	Fortran Bindings with <code>mpif.h</code> or the <code>mpi</code> Module	1052
A.5.1	Point-to-Point Communication Fortran Bindings	1052
A.5.2	Partitioned Communication Fortran Bindings	1056
A.5.3	Datatypes Fortran Bindings	1056
A.5.4	Collective Communication Fortran Bindings	1059
A.5.5	Groups, Contexts, Communicators, and Caching Fortran Bindings	1064
A.5.6	Virtual Topologies for MPI Processes Fortran Bindings	1068
A.5.7	MPI Environmental Management Fortran Bindings	1071
A.5.8	The Info Object Fortran Bindings	1074
A.5.9	Process Creation and Management Fortran Bindings	1074
A.5.10	One-Sided Communications Fortran Bindings	1076
A.5.11	External Interfaces Fortran Bindings	1080
A.5.12	I/O Fortran Bindings	1081
A.5.13	Language Bindings Fortran Bindings	1085
A.5.14	Application Binary Interface (ABI) Fortran Bindings	1085

A.5.15	Tools / Profiling Interface Fortran Bindings	1086
A.5.16	Deprecated Fortran Bindings	1086
B	Change-Log	1089
B.1	Changes from Version 5.0 to Version 5.1	1089
B.1.1	Fixes to Errata in Previous Versions of MPI	1089
B.1.2	Changes in MPI-5.1	1089
B.2	Changes from Version 4.1 to Version 5.0	1089
B.2.1	Fixes to Errata in Previous Versions of MPI	1089
B.2.2	Changes in MPI-5.0	1090
B.3	Changes from Version 4.0 to Version 4.1	1090
B.3.1	Fixes to Errata in Previous Versions of MPI	1090
B.3.2	Changes in MPI-4.1	1092
B.4	Changes from Version 3.1 to Version 4.0	1095
B.4.1	Fixes to Errata in Previous Versions of MPI	1095
B.4.2	Changes in MPI-4.0	1095
B.5	Changes from Version 3.0 to Version 3.1	1099
B.5.1	Fixes to Errata in Previous Versions of MPI	1099
B.5.2	Changes in MPI-3.1	1101
B.6	Changes from Version 2.2 to Version 3.0	1101
B.6.1	Fixes to Errata in Previous Versions of MPI	1101
B.6.2	Changes in MPI-3.0	1102
B.7	Changes from Version 2.1 to Version 2.2	1107
B.8	Changes from Version 2.0 to Version 2.1	1110
	Bibliography	1115
	General Index	1121
	Examples Index	1130
	MPI Constant and Predefined Handle Index	1147
	MPI Declarations Index	1155
	MPI Callback Function Prototype Index	1157
	MPI Function Index	1158

List of Figures

2.1	State transition diagram for blocking operations	14
2.2	State transition diagram for nonblocking operations	14
2.3	State transition diagram for persistent operations	14
6.1	Collective communications, an overview	191
6.2	Inter-communicator allgather	195
6.3	Inter-communicator reduce-scatter	195
6.4	Gather example	202
6.5	Gatherv example with strides	203
6.6	Gatherv example, 2-dimensional	204
6.7	Gatherv example, 2-dimensional, subarrays with different sizes	205
6.8	Gatherv example, 2-dimensional, subarrays with different sizes and strides	207
6.9	Scatter example	212
6.10	Scatterv example with strides	213
6.11	Scatterv example with different strides and counts	214
6.12	Race conditions with point-to-point and collective communications	298
6.13	Overlapping communicators example	302
7.1	Inter-communicator creation using <code>MPI_COMM_CREATE</code>	325
7.2	Inter-communicator construction with <code>MPI_COMM_SPLIT</code>	330
7.3	Recursive communicator creation with <code>MPI_COMM_SPLIT_TYPE</code>	336
7.4	Three-group pipeline	356
7.5	Three-group ring	357
8.1	Neighborhood gather communication example	410
8.2	Cartesian neighborhood allgather example for 3 and 1 processes in a dimension	410
8.3	Cartesian neighborhood alltoall example for 3 and 1 MPI processes in a dimension	415
9.1	Diagram for deciding which error handler is invoked depending on the MPI objects associated with the operation and whether the Sessions Model or the World Model is used.	449
11.1	Session handle to communicator	492
11.2	Process set examples	496
12.1	Schematic description of the public/private window operations in the <code>MPI_WIN_SEPARATE</code> memory model for two overlapping windows	585
12.2	Active target communication	587
12.3	Active target communication, with weak synchronization	588
12.4	Passive target communication	589

12.5	Active target communication with several MPI processes and strong synchronization	593
12.6	Symmetric communication	613
12.7	Deadlock situation	613
12.8	No deadlock	614
15.1	Etypes and filetypes	654
15.2	Partitioning a file among parallel processes	654
15.3	Displacements	667
15.4	Example array file layout	730
15.5	Example local array filetype for process 1	731
21.1	Status conversion routines	863

DRAFT

List of Tables

2.1	Deprecated and removed constructs	26
3.1	Predefined MPI datatypes corresponding to Fortran datatypes	35
3.2	Predefined MPI datatypes corresponding to C datatypes	36
3.3	Predefined MPI datatypes corresponding to both C and Fortran datatypes	37
3.4	Predefined MPI datatypes corresponding to C++ datatypes	37
7.1	MPI_COMM_* function behavior (in inter-communication mode)	351
9.1	Error classes (Part 1)	461
9.2	Error classes (Part 2)	462
11.1	List of MPI Functions that can be called at any time within an MPI program, including prior to MPI initialization and following MPI finalization	505
12.1	C types of attribute value argument to MPI_WIN_GET_ATTR and MPI_WIN_SET_ATTR	561
12.2	Error classes in one-sided communication routines	603
15.1	Data access routines	670
15.2	"external32" sizes of predefined datatypes	710
15.3	"external32" sizes of optional datatypes	711
15.4	"external32" sizes of C++ datatypes	711
15.5	I/O error classes	728
16.1	MPI tool information interface verbosity levels	740
16.2	Constants to identify associations of variables	741
16.3	MPI datatypes that can be used by the MPI tool information interface	743
16.4	Scopes for control variables	748
16.5	Hierarchy of safety requirement levels for event callback routines	767
16.6	List of MPI functions that when called from within a callback function may not return MPI_T_ERR_NOT_ACCESSIBLE	768
16.7	Return codes used in procedures of the MPI tool information interface	785
17.1	Fault tolerance error classes	794
19.1	Removed MPI-1 functions and their replacements	807
19.2	Removed MPI-1 datatypes. The indicated routine may be used for changing the lower and upper bound respectively.	808
19.3	Removed MPI-1 constants	808
19.4	Removed MPI-1 callback prototypes and their replacements	808
21.1	Specific Fortran procedure names and related calling conventions	819

21.2	Occurrence of Fortran optimization problems	845
22.1	Predefined MPI datatype categories and instance values, for types without a specified size. All unassigned values are reserved.	884
22.2	Predefined MPI datatype categories and instance values, for types without a specified size. All unassigned values are reserved.	884
22.3	Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.	885
22.4	Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.	885
22.5	Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.	886
22.6	Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.	886
22.7	Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.	886
22.8	Predefined MPI datatype categories and instance values, for types with a specified width. All unassigned values are reserved.	887
22.9	Predefined MPI datatype categories and instance values, for types with a specified width. All unassigned values are reserved.	888
22.10	Predefined MPI_Op categories and instance values. All unassigned values are reserved.	889
22.11	Predefined MPI handle categories and instance values. All unassigned values are reserved.	890

List of Examples

3.1	Hello world	33
3.2	Datatype matching	49
3.3	Datatype matching (erroneous)	49
3.4	Using MPI_BYTE	49
3.5	Fortran CHARACTER	50
3.6	Nonovertaking messages	57
3.7	Message matching	58
3.8	Message exchange	59
3.9	Errant message exchange	59
3.10	Exchange relies on buffering	59
3.11	Attach and detach buffer	68
3.12	Attach and detach communicator-specific buffer	68
3.13	Nonblocking point-to-point	82
3.14	Nonblocking send and receive with request free	83
3.15	Message ordering for nonblocking operations	84
3.16	Progress semantics	84
3.17	Client-server	91
3.18	Client server code with waitsome	91
3.19	Client-server with probe	97
3.20	Client-server (with error)	98
4.1	Partitioned communication	114
4.2	Partitioned communication using threads	122
4.3	Partitioned communication with tasks	123
4.4	Partitioned communication with partial completion	125
5.1	Typemap	129
5.2	Typemap for contiguous	130
5.3	Typemap for vector	131
5.4	Typemap for vector	131
5.5	Typemap for indexed	134
5.6	Typemap for create struct	139
5.7	Datatypes for distributed arrays	147
5.8	Get_address	149
5.9	Typemap of nested datatypes	151
5.10	Type_commit	156
5.11	Matching type with datatypes	159
5.12	Using Get_count and Get_elements	160
5.13	Send/receive of a 3D array	170
5.14	Using indexed datatype	170
5.15	Nested vector datatypes	171

5.16	Transpose with datatypes	171
5.17	Using datatypes with array of structures	172
5.18	Using datatypes with array of structures with absolute addresses	174
5.19	Using datatypes with unions	175
5.20	Decoding a datatype	176
5.21	Using Pack	182
5.22	Pack/Unpack with struct datatype	182
5.23	Pack and Pack_size	183
6.1	Broadcast	198
6.2	Gather	202
6.3	Gather with allocation at the root	202
6.4	Gather with datatype	202
6.5	Gatherv	203
6.6	Gatherv with datatype	203
6.7	Gatherv with datatype	204
6.8	Gatherv with struct datatype	205
6.9	Gatherv with vector datatype	206
6.10	Gather and Gatherv	207
6.11	Scatter	212
6.12	Scatterv	212
6.13	Scatterv with vector datatype	213
6.14	Allgather	217
6.15	Reduction	228
6.16	Reduction of a vector	229
6.17	Retrieving an unnamed predefined value-index handle	232
6.18	Reduction with maxloc	233
6.19	Reduction with maxloc	233
6.20	Reduction with minloc	234
6.21	Reduction with user-defined op	238
6.22	Defining a user function	239
6.23	Allreduce of a vector	241
6.24	User-defined operation with Scan	249
6.25	Ibcast	254
6.26	Erroneous use of Bcast	296
6.27	Erroneous use of Bcast	296
6.28	Erroneous use of Bcast	297
6.29	Nondeterministic use of Bcast	297
6.30	Mixing blocking and nonblocking collective operations	299
6.31	Erroneous matching of collectives	299
6.32	Progress of nonblocking collectives	300
6.33	Erroneous matching of blocking and nonblocking collectives	300
6.34	Mixing collective and point-to-point	301
6.35	Pipelining nonblocking collectives	301
6.36	Overlapping communicators and collectives	301
6.37	Independence of nonblocking operations	302
7.1	Inter-communicator creation	326

7.2	Client-server model	330
7.3	Splitting into NUMANode subcommunicators	333
7.5	Recursive splitting of COMM_WORLD	337
7.6	Parallel output of a message	342
7.7	Message exchange	342
7.8	Collective communication	343
7.9	Using Group_excl	343
7.10	Communication safety	344
7.11	Library Example #1	345
7.12	Library Example #2	346
7.13	Three-Group “Pipeline”	356
7.14	Three-Group “Ring”	357
8.1	Dims create	386
8.2	Graph create	387
8.3	Dist graph creation	393
8.4	Dist_graph_create	394
8.5	Graph creation and neighbors count	400
8.6	Graph creation	400
8.7	Using Cart_shift	404
8.8	Subgroup cart process topology	405
8.9	Neighbor allgather	410
8.10	Neighbor alltoall	414
8.11	Neighborhood collective communication	434
9.1	Splitting into NUMANode subcommunicators	444
9.2	Use of Alloc_mem in Fortran	447
9.3	Use of Alloc_mem in Fortran with Cray pointers	447
9.4	Using Alloc_mem	448
9.5	Using MPI_Wtime	469
11.1	Initializing MPI	481
11.2	mpiexec and environment variables	482
11.3	Rules for finalize	487
11.4	Rules for finalize	487
11.5	Finalize and request free	487
11.6	Finalize and buffer attach	488
11.7	Finalize and cancel	488
11.8	Actions after Finalize	489
11.9	Finalize in the Sessions Model	495
11.10	Creating a communicator using the Sessions Model	499
11.11	Using process set query in group creation	501
11.12	Using process set query in group creation	503
11.13	Requesting and querying memory allocation kinds in the Sessions Model	509
11.14	Using mpiexec, with -n	512
11.15	Using mpiexec, with -host	512
11.16	Using mpiexec, with separate argument lists	512
11.17	Using mpiexec, with -arch	512

11.18	Using <code>mpiexec</code> , with <code>-configfile</code>	512
11.19	Threads and MPI	514
11.20	<code>argv</code> in C and Fortran	520
11.21	Array of <code>argv</code> in C and Fortran	525
11.22	Manager-worker with <code>Comm_spawn</code>	527
11.23	Client-server	536
11.24	Name publishing	537
11.25	Client-server	537
12.1	Using Get with indexed datatype	568
12.2	Using Get	570
12.3	Accumulate in RMA	572
12.4	RMA Start and complete	591
12.5	RMA Lock and unlock	597
12.6	Update location in separate memory model	607
12.7	Update location in unified memory model	608
12.8	Read data in RMA	608
12.9	Read data in RMA (unsafe)	609
12.10	Public and private memory in RMA	609
12.11	Public and private memory in RMA	610
12.12	Active target and local reads in RMA	610
12.13	Deadlock due to synchronization through shared memory	614
12.14	Register and Compiler Optimization	615
12.15	Put with fence	616
12.16	Get with fence	616
12.17	Put with PSCW	616
12.18	Get with PSCW	617
12.19	Double buffer in RMA	617
12.20	Counting semaphore (nonscalable)	618
12.21	Critical region with RMA	619
12.22	Critical region with Compare-and-Swap	619
12.23	Shared memory windows	620
12.24	Requests in RMA@Requests in RMA	620
12.25	Linked list in RMA@Linked list in RMA	621
13.1	Ring exchange with queued communication	640
14.1	User-defined reduce	647
15.1	Decoding <code>amode</code> in Fortran 77	662
15.2	Read to end of file	680
15.3	File pointer update semantics	684
15.4	Erroneous example fragment of concurrent split collective access on a file handle	697
15.5	Consistency by setting atomic mode	723
15.6	Consistency using “sync-barrier-sync”	723
15.7	Erroneous attempt to achieve consistency	724
15.8	Consistency for writing and reading files asynchronously	725
15.9	Overlap computation and output	728

15.10	Local subarray for file output	730
15.11	Local subarray for file output in Fortran 90	730
16.1	Measurement wrapper	735
16.2	Profiling interface, implementation using weak symbols	735
16.3	Profiling interface, implementation using the C macro preprocessor	735
16.5	Listing names of control variables	748
16.6	Reading a control variable	751
16.7	Basic usage of performance variables	762
17.1	Fault Tolerant Manager/Worker	794
21.1	Fortran language bindings	821
21.2	Fortran 2008 measurement wrapper	822
21.3	Fortran choice argument	826
21.4	Fortran 90, with selected KIND	832
21.5	MPI_TYPE_MATCH_SIZE implementation	835
21.6	Fortran 90, with heterogeneous communication (unsafe)	836
21.7	Fortran 90, with heterogeneous MPI I/O (unsafe)	836
21.8	Fortran 90, with a scalar that is not an array	839
21.9	Fortran 90, with a subarray as buffer	839
21.10	Fortran 90, with a subarray as buffer	839
21.11	Fortran 90, with a subarray as buffer	841
21.12	Fortran 90, with scalars as buffer	842
21.13	Fortran 90, with vector subscripts	842
21.14	Fortran 90, with derived types	843
21.15	Fortran 90 register optimization	846
21.16	Fortran 90 register optimization	846
21.17	Fortran 90 register optimization	847
21.18	Fortran 90 register optimization	847
21.19	Protecting nonblocking communication with ASYNCHRONOUS	848
21.20	Fortran 90 register optimization in RMA	851
21.21	Fortran 90 overlapping communication and computation	853
21.22	Fortran 90 overlapping communication and computation	853
21.23	Fortran 90 overlapping communication and computation	853
21.24	Using separated variables	855
21.25	Fortran 90 overlapping communication and computation	856
21.26	C/Fortran handle conversion	861
21.27	C/Fortran handle conversion and absolute addresses	865
21.28	Attributes between languages, set in C	867
21.29	Attributes between languages, set in Fortran	868
21.30	Attributes between languages, set in Fortran	869
21.31	Interlanguage communication	871

Acknowledgements

This document is the product of a number of distinct efforts in five distinct phases: one for each of MPI-1, MPI-2, MPI-3, MPI-4, and MPI-5. This section describes these in historical order, starting with MPI-1. Some efforts, particularly parts of MPI-2, had distinct groups of individuals associated with them, and these efforts are detailed separately.

This document represents the work of many people who have served on the MPI Forum. The meetings have been attended by dozens of people from many parts of the world. It is the hard and dedicated work of this group that has led to the MPI standard.

The technical development was carried out by subgroups, whose work was reviewed by the full committee. During the period of development of the Message-Passing Interface (MPI), many people helped with this effort.

Those who served as primary coordinators in MPI-1.0 and MPI-1.1 are:

- Jack Dongarra, David Walker, Conveners and Meeting Chairs
- Ewing Lusk, Bob Knighten, Minutes
- Marc Snir, William Gropp, Ewing Lusk, Point-to-Point Communication
- Al Geist, Marc Snir, Steve Otto, Collective Communication
- Steve Otto, Editor
- Rolf Hempel, Process Topologies
- Ewing Lusk, Language Binding
- William Gropp, Environmental Management
- James Cownie, Profiling
- Tony Skjellum, Lyndon Clarke, Marc Snir, Richard Littlefield, Mark Sears, Groups, Contexts, and Communicators
- Steven Huss-Lederman, Initial Implementation Subset

The following list includes some of the active participants in the MPI-1.0 and MPI-1.1 process not mentioned above.

Ed Anderson	Robert Babb	Joe Baron	Eric Barszcz
Scott Berryman	Rob Bjornson	Nathan Doss	Anne Elster
Jim Feeney	Vince Fernando	Sam Fineberg	Jon Flower
Daniel Frye	Ian Glendinning	Adam Greenberg	Robert Harrison
Leslie Hart	Tom Haupt	Don Heller	Tom Henderson
Alex Ho	C.T. Howard Ho	Gary Howell	John Kapenga
James Kohl	Susan Krauss	Bob Leary	Arthur Maccabe
Peter Madams	Alan Mainwaring	Oliver McBryan	Phil McKinley
Charles Mosher	Dan Nessett	Peter Pacheco	Howard Palmer
Paul Pierce	Sanjay Ranka	Peter Rigsbee	Arch Robison
Erich Schikuta	Ambuj Singh	Alan Sussman	Robert Tomlinson
Robert G. Voigt	Dennis Weeks	Stephen Wheat	Steve Zenith

The University of Tennessee and Oak Ridge National Laboratory made the draft available by anonymous FTP mail servers and were instrumental in distributing the document.

The work on the MPI-1 standard was supported in part by ARPA and NSF under grant ASC-9310330, the National Science Foundation Science and Technology Center Cooperative Agreement No. CCR-8809615, and by the Commission of the European Community through Esprit project P6643 (PPPE).

MPI-1.2 and MPI-2.0:

Those who served as primary coordinators in MPI-1.2 and MPI-2.0 are:

- Ewing Lusk, Convener and Meeting Chair
- Steve Huss-Lederman, Editor
- Ewing Lusk, Miscellany
- Bill Saphir, Process Creation and Management
- Marc Snir, One-Sided Communications
- William Gropp and Anthony Skjellum, Extended Collective Operations
- Steve Huss-Lederman, External Interfaces
- Bill Nitzberg, I/O
- Andrew Lumsdaine, Bill Saphir, and Jeffrey M. Squyres, Language Bindings
- Anthony Skjellum and Arkady Kanevsky, Real-Time

The following list includes some of the active participants who attended MPI-2 Forum meetings and are not mentioned above.

Greg Astfalk	Robert Babb	Ed Benson	Rajesh Bordawekar	1
Pete Bradley	Peter Brennan	Ron Brightwell	Maciej Brodowicz	2
Eric Brunner	Greg Burns	Margaret Cahir	Pang Chen	3
Ying Chen	Albert Cheng	Yong Cho	Joel Clark	4
Lyndon Clarke	Laurie Costello	Dennis Cottel	Jim Cownie	5
Zhenqian Cui	Suresh Damodaran-Kamal		Raja Daoud	6
Judith Devaney	David DiNucci	Doug Doeffler	Jack Dongarra	7
Terry Dontje	Nathan Doss	Anne Elster	Mark Fallon	8
Karl Feind	Sam Fineberg	Craig Fischberg	Stephen Fleischman	9
Ian Foster	Hubertus Franke	Richard Frost	Al Geist	10
Robert George	David Greenberg	John Hagedorn	Kei Harada	11
Leslie Hart	Shane Hebert	Rolf Hempel	Tom Henderson	12
Alex Ho	Hans-Christian Hoppe	Joefon Jann	Terry Jones	13
Karl Kesselman	Koichi Konishi	Susan Kraus	Steve Kubica	14
Steve Landherr	Mario Lauria	Mark Law	Juan Leon	15
Lloyd Lewins	Ziyang Lu	Bob Madahar	Peter Madams	16
John May	Oliver McBryan	Brian McCandless	Tyce McLarty	17
Thom McMahon	Harish Nag	Nick Nevin	Jarek Nieplocha	18
Ron Oldfield	Peter Ossadnik	Steve Otto	Peter Pacheco	19
Yoonho Park	Perry Partow	Pratap Pattnaik	Elsie Pierce	20
Paul Pierce	Heidi Poxon	Jean-Pierre Prost	Boris Protopopov	21
James Pruyve	Rolf Rabenseifner	Joe Rieken	Peter Rigsbee	22
Tom Robey	Anna Rounbehler	Nobutoshi Sagawa	Arindam Saha	23
Eric Salo	Darren Sanders	Eric Sharakan	Andrew Sherman	24
Fred Shirley	Lance Shuler	A. Gordon Smith	Ian Stockdale	25
David Taylor	Stephen Taylor	Greg Tensa	Rajeev Thakur	26
Marydell Tholburn	Dick Treumann	Simon Tsang	Manuel Ujaldon	27
David Walker	Jerrell Watts	Klaus Wolf	Parkson Wong	28
Dave Wright				29

The MPI Forum also acknowledges and appreciates the valuable input from people via e-mail and in person.

The following institutions supported the MPI-2 effort through time and travel support for the people listed above.

Argonne National Laboratory	35
Bolt, Beranek, and Newman	36
California Institute of Technology	37
Center for Computing Sciences	38
Convex Computer Corporation	39
Cray Research	40
Digital Equipment Corporation	41
Dolphin Interconnect Solutions, Inc.	42
Edinburgh Parallel Computing Centre	43
General Electric Company	44
German National Research Center for Information Technology	45
Hewlett-Packard	46
Hitachi	47
Hughes Aircraft Company	48

1 Intel Corporation
2 International Business Machines
3 Khoral Research
4 Lawrence Livermore National Laboratory
5 Los Alamos National Laboratory
6 MPI Software Technology, Inc.
7 Mississippi State University
8 NEC Corporation
9 National Aeronautics and Space Administration
10 National Energy Research Scientific Computing Center
11 National Institute of Standards and Technology
12 National Oceanic and Atmospheric Administration
13 Oak Ridge National Laboratory
14 The Ohio State University
15 PALLAS GmbH
16 Pacific Northwest National Laboratory
17 Pratt & Whitney
18 San Diego Supercomputer Center
19 Sanders, A Lockheed-Martin Company
20 Sandia National Laboratories
21 Schlumberger
22 Scientific Computing Associates, Inc.
23 Silicon Graphics Incorporated
24 Sky Computers
25 Sun Microsystems Computer Corporation
26 Syracuse University
27 The MITRE Corporation
28 Thinking Machines Corporation
29 United States Navy
30 University of Colorado
31 University of Denver
32 University of Houston
33 University of Illinois
34 University of Maryland
35 University of Notre Dame
36 University of San Francisco
37 University of Stuttgart Computing Center
38 University of Wisconsin

39 MPI-2 operated on a very tight budget (in reality, it had no budget when the first
40 meeting was announced). Many institutions helped the MPI-2 effort by supporting the
41 efforts and travel of the members of the MPI Forum. Direct support was given by NSF and
42 DARPA under NSF contract CDA-9115428 for travel by U.S. academic participants and
43 Esprit under project HPC Standards (21111) for European participants.
44
45
46
47
48

MPI-1.3 and MPI-2.1:

The editors and organizers of the combined documents have been:

- Richard Graham, Convener and Meeting Chair
- Jack Dongarra, Steering Committee
- Al Geist, Steering Committee
- William Gropp, Steering Committee
- Rainer Keller, Merge of MPI-1.3
- Andrew Lumsdaine, Steering Committee
- Ewing Lusk, Steering Committee, MPI-1.1-Errata (Oct. 12, 1998) MPI-2.1-Errata Ballots 1, 2 (May 15, 2002)
- Rolf Rabenseifner, Steering Committee, Merge of MPI-2.1 and MPI-2.1-Errata Ballots 3, 4 (2008)

All chapters have been revisited to achieve a consistent MPI-2.1 text. Those who served as authors for the necessary modifications are:

- William Gropp, Front Matter, Introduction, and Bibliography
- Richard Graham, Point-to-Point Communication
- Adam Moody, Collective Communication
- Richard Treumann, Groups, Contexts, and Communicators
- Jesper Larsson Träff, Process Topologies, Info-Object, and One-Sided Communications
- George Bosilca, Environmental Management
- David Solt, Process Creation and Management
- Bronis R. de Supinski, External Interfaces, and Profiling
- Rajeev Thakur, I/O
- Jeffrey M. Squyres, Language Bindings and MPI-2.1 Secretary
- Rolf Rabenseifner, Deprecated Functions and Annex Change-Log
- Alexander Supalov and Denis Nagorny, Annex Language Bindings

The following list includes some of the active participants who attended MPI-2 Forum meetings and in the e-mail discussions of the errata items and are not mentioned above.

1	Pavan Balaji	Purushotham V. Bangalore	Brian Barrett
2	Richard Barrett	Christian Bell	Robert Blackmore
3	Gil Bloch	Ron Brightwell	Jeffrey Brown
4	Darius Buntinas	Jonathan Carter	Nathan DeBardeleben
5	Terry Dontje	Gabor Dozsa	Edric Ellis
6	Karl Feind	Edgar Gabriel	Patrick Geoffray
7	David Gingold	Dave Goodell	Erez Haba
8	Robert Harrison	Thomas Herault	Steve Hodson
9	Torsten Hoefer	Joshua Hursey	Yann Kalemkarian
10	Matthew Koop	Quincey Koziol	Sameer Kumar
11	Miron Livny	Kannan Narasimhan	Mark Pagel
12	Avneesh Pant	Steve Poole	Howard Pritchard
13	Craig Rasmussen	Hubert Ritzdorf	Rob Ross
14	Tony Skjellum	Brian Smith	Vinod Tipparaju
15	Jesper Larsson Träff	Keith Underwood	

16
17 The MPI Forum also acknowledges and appreciates the valuable input from people via
18 e-mail and in person.

19 The following institutions supported the MPI-2 effort through time and travel support
20 for the people listed above.

21 Argonne National Laboratory
22 Bull
23 Cisco Systems, Inc.
24 Cray Inc.
25 The HDF Group
26 Hewlett-Packard
27 IBM T.J. Watson Research
28 Indiana University
29 Institut National de Recherche en Informatique et Automatique (Inria)
30 Intel Corporation
31 Lawrence Berkeley National Laboratory
32 Lawrence Livermore National Laboratory
33 Los Alamos National Laboratory
34 Mathworks
35 Mellanox Technologies
36 Microsoft
37 Myricom
38 NEC Laboratories Europe, NEC Europe Ltd.
39 Oak Ridge National Laboratory
40 The Ohio State University
41 Pacific Northwest National Laboratory
42 QLogic Corporation
43 Sandia National Laboratories
44 SiCortex
45 Silicon Graphics Incorporated
46 Sun Microsystems, Inc.
47 University of Alabama at Birmingham
48 University of Houston

University of Illinois at Urbana-Champaign
University of Stuttgart, High Performance Computing Center Stuttgart (HLRS)
University of Tennessee, Knoxville
University of Wisconsin

Funding for the MPI Forum meetings was partially supported by award #CCF-0816909 from the National Science Foundation. In addition, the HDF Group provided travel support for one U.S. academic.

MPI-2.2:

All chapters have been revisited to achieve a consistent MPI-2.2 text. Those who served as authors for the necessary modifications are:

- William Gropp, Front Matter, Introduction, and Bibliography; MPI-2.2 Chair.
- Richard Graham, Point-to-Point Communication and Datatypes
- Adam Moody, Collective Communication
- Torsten Hoefler, Collective Communication and Process Topologies
- Richard Treumann, Groups, Contexts, and Communicators
- Jesper Larsson Träff, Process Topologies, Info-Object and One-Sided Communications
- George Bosilca, Datatypes and Environmental Management
- David Solt, Process Creation and Management
- Bronis R. de Supinski, External Interfaces, and Profiling
- Rajeev Thakur, I/O
- Jeffrey M. Squyres, Language Bindings and MPI-2.2 Secretary
- Rolf Rabenseifner, Deprecated Functions, Annex Change-Log, and Annex Language Bindings
- Alexander Supalov, Annex Language Bindings

The following list includes some of the active participants who attended MPI-2 Forum meetings and in the e-mail discussions of the errata items and are not mentioned above.

1	Pavan Balaji	Purushotham V. Bangalore	Brian Barrett
2	Richard Barrett	Christian Bell	Robert Blackmore
3	Gil Bloch	Ron Brightwell	Greg Bronevetsky
4	Jeff Brown	Darius Buntinas	Jonathan Carter
5	Nathan DeBardeleben	Terry Dontje	Gabor Dozsa
6	Edric Ellis	Karl Feind	Edgar Gabriel
7	Patrick Geoffray	Johann George	David Gingold
8	David Goodell	Erez Haba	Robert Harrison
9	Thomas Herault	Marc-André Hermanns	Steve Hodson
10	Joshua Hursey	Yutaka Ishikawa	Bin Jia
11	Hideyuki Jitsumoto	Terry Jones	Yann Kalemkarian
12	Ranier Keller	Matthew Koop	Quincey Koziol
13	Manojkumar Krishnan	Sameer Kumar	Miron Livny
14	Andrew Lumsdaine	Miao Luo	Ewing Lusk
15	Timothy I. Mattox	Kannan Narasimhan	Mark Pagel
16	Avneesh Pant	Steve Poole	Howard Pritchard
17	Craig Rasmussen	Hubert Ritzdorf	Rob Ross
18	Martin Schulz	Pavel Shamis	Galen Shipman
19	Christian Siebert	Anthony Skjellum	Brian Smith
20	Naoki Sueyasu	Vinod Tipparaju	Keith Underwood
21	Rolf Vandevaart	Abhinav Vishnu	Weikuan Yu

22
23 The MPI Forum also acknowledges and appreciates the valuable input from people via
24 e-mail and in person.

25 The following institutions supported the MPI-2.2 effort through time and travel support
26 for the people listed above.

27 Argonne National Laboratory
28 Auburn University
29 Bull
30 Cisco Systems, Inc.
31 Cray Inc.
32 Forschungszentrum Jülich
33 Fujitsu
34 The HDF Group
35 Hewlett-Packard
36 International Business Machines
37 Indiana University
38 Institut National de Recherche en Informatique et Automatique (Inria)
39 Institute for Advanced Science & Engineering Corporation
40 Intel Corporation
41 Lawrence Berkeley National Laboratory
42 Lawrence Livermore National Laboratory
43 Los Alamos National Laboratory
44 Mathworks
45 Mellanox Technologies
46 Microsoft
47 Myricom
48 NEC Corporation

Oak Ridge National Laboratory
 The Ohio State University
 Pacific Northwest National Laboratory
 QLogic Corporation
 RunTime Computing Solutions, LLC
 Sandia National Laboratories
 SiCortex, Inc.
 Silicon Graphics Inc.
 Sun Microsystems, Inc.
 Tokyo Institute of Technology
 University of Alabama at Birmingham
 University of Houston
 University of Illinois at Urbana-Champaign
 University of Stuttgart, High Performance Computing Center Stuttgart (HLRS)
 University of Tennessee, Knoxville
 University of Tokyo
 University of Wisconsin

Funding for the MPI Forum meetings was partially supported by awards #CCF-0816909 and #CCF-1144042 from the National Science Foundation. In addition, the HDF Group provided travel support for one U.S. academic.

MPI-3.0:

MPI-3.0 is a significant effort to extend and modernize the MPI standard.
 The editors and organizers of the MPI-3.0 have been:

- William Gropp, Steering Committee, Front Matter, Introduction, Groups, Contexts, and Communicators, One-Sided Communications, and Bibliography
- Richard Graham, Steering Committee, Point-to-Point Communication, Meeting Convener, and MPI-3.0 Chair
- Torsten Hoefler, Collective Communication, One-Sided Communications, and Process Topologies
- George Bosilca, Datatypes and Environmental Management
- David Solt, Process Creation and Management
- Bronis R. de Supinski, External Interfaces and Tool Support
- Rajeev Thakur, I/O and One-Sided Communications
- Darius Buntinas, Info Object
- Jeffrey M. Squyres, Language Bindings and MPI-3.0 Secretary
- Rolf Rabenseifner, Steering Committee, Terms and Definitions, and Fortran Bindings, Deprecated Functions, Annex Change-Log, and Annex Language Bindings
- Craig Rasmussen, Fortran Bindings

The following list includes some of the active participants who attended MPI-3 Forum meetings or participated in the e-mail discussions and who are not mentioned above.

Tatsuya Abe	Tomoya Adachi	Sadaf Alam
Reinhold Bader	Pavan Balaji	Purushotham V. Bangalore
Brian Barrett	Richard Barrett	Robert Blackmore
Aurélien Bouteiller	Ron Brightwell	Greg Bronevetsky
Jed Brown	Darius Buntinas	Devendar Bureddy
Arno Candel	George Carr	Mohamad Chaarawi
Raghunath Raja Chandrasekar	James Dinan	Terry Dontje
Edgar Gabriel	Balazs Gerofi	Brice Goglin
David Goodell	Manjunath Gorentla	Erez Haba
Jeff Hammond	Thomas Herault	Marc-André Hermanns
Jennifer Herrett-Skjellum	Nathan Hjelm	Atsushi Hori
Joshua Hursey	Marty Itzkowitz	Yutaka Ishikawa
Nysal Jan	Bin Jia	Hideyuki Jitsumoto
Yann Kalemkarian	Krishna Kandalla	Takahiro Kawashima
Chulho Kim	Dries Kimpe	Christof Klausecker
Alice Koniges	Quincey Koziol	Dieter Kranzlmüller
Manojkumar Krishnan	Sameer Kumar	Eric Lantz
Jay Lofstead	Bill Long	Andrew Lumsdaine
Miao Luo	Ewing Lusk	Adam Moody
Nick M. Maclaren	Amith Mamidala	Guillaume Mercier
Scott McMillan	Douglas Miller	Kathryn Mohror
Tim Murray	Tomotake Nakamura	Takeshi Nanri
Steve Oyanagi	Mark Pagel	Swann Perarnau
Sreeram Potluri	Howard Pritchard	Rolf Riesen
Hubert Ritzdorf	Kuninobu Sasaki	Timo Schneider
Martin Schulz	Gilad Shainer	Christian Siebert
Anthony Skjellum	Brian Smith	Marc Snir
Raffaele Giuseppe Solca	Shinji Sumimoto	Alexander Supalov
Sayantan Sur	Masamichi Takagi	Fabian Tillier
Vinod Tipparaju	Jesper Larsson Träff	Richard Treumann
Keith Underwood	Rolf Vandevaart	Anh Vo
Abhinav Vishnu	Min Xie	Enqiang Zhou

The MPI Forum also acknowledges and appreciates the valuable input from people via e-mail and in person.

The MPI Forum also thanks those that provided feedback during the public comment period. In particular, the Forum would like to thank Jeremiah Wilcock for providing detailed comments on the entire draft standard.

The following institutions supported the MPI-3 effort through time and travel support for the people listed above.

Argonne National Laboratory
Bull
Cisco Systems, Inc.
Cray Inc.
CSCS

ETH Zurich	1
Fujitsu Ltd.	2
German Research School for Simulation Sciences	3
The HDF Group	4
Hewlett-Packard	5
International Business Machines	6
IBM India Private Ltd	7
Indiana University	8
Institut National de Recherche en Informatique et Automatique (Inria)	9
Institute for Advanced Science & Engineering Corporation	10
Intel Corporation	11
Lawrence Berkeley National Laboratory	12
Lawrence Livermore National Laboratory	13
Los Alamos National Laboratory	14
Mellanox Technologies, Inc.	15
Microsoft Corporation	16
NEC Corporation	17
National Oceanic and Atmospheric Administration, Global Systems Division	18
NVIDIA Corporation	19
Oak Ridge National Laboratory	20
The Ohio State University	21
Oracle America	22
Platform Computing	23
RIKEN AICS	24
RunTime Computing Solutions, LLC	25
Sandia National Laboratories	26
Technical University of Chemnitz	27
Tokyo Institute of Technology	28
University of Alabama at Birmingham	29
University of Chicago	30
University of Houston	31
University of Illinois at Urbana-Champaign	32
University of Stuttgart, High Performance Computing Center Stuttgart (HLRS)	33
University of Tennessee, Knoxville	34
University of Tokyo	35

Funding for the MPI Forum meetings was partially supported by awards #CCF-0816909 and #CCF-1144042 from the National Science Foundation. In addition, the HDF Group and Sandia National Laboratories provided travel support for one U.S. academic each.

MPI-3.1:

MPI-3.1 is a minor update to the MPI standard.
The editors and organizers of the MPI-3.1 have been:

- Martin Schulz, MPI-3.1 Chair
- William Gropp, Steering Committee, Front Matter, Introduction, One-Sided Communications, and Bibliography; Overall Editor

- Rolf Rabenseifner, Steering Committee, Terms and Definitions, and Fortran Bindings, Deprecated Functions, Annex Change-Log, and Annex Language Bindings
- Richard L. Graham, Steering Committee, Meeting Convener
- Jeffrey M. Squyres, Language Bindings and MPI-3.1 Secretary
- Daniel Holmes, Point-to-Point Communication
- George Bosilca, Datatypes and Environmental Management
- Torsten Hoefler, Collective Communication and Process Topologies
- Pavan Balaji, Groups, Contexts, and Communicators, and External Interfaces
- Jeff Hammond, The Info Object
- David Solt, Process Creation and Management
- Quincey Koziol, I/O
- Kathryn Mohror, Tool Support
- Rajeev Thakur, One-Sided Communications

The following list includes some of the active participants who attended MPI Forum meetings or participated in the e-mail discussions.

Charles Archer	Pavan Balaji	Purushotham V. Bangalore
Brian Barrett	Wesley Bland	Michael Blocksome
George Bosilca	Aurélien Bouteiller	Devendar Bureddy
Yohann Burette	Mohamad Chaarawi	Alexey Cheptsov
James Dinan	Dmitry Durnov	Thomas Francois
Edgar Gabriel	Todd Gamblin	Balazs Gerofi
Paddy Gillies	David Goodell	Manjunath Gorentla Venkata
Richard L. Graham	Ryan E. Grant	William Gropp
Khaled Hamidouche	Jeff Hammond	Amin Hassani
Marc-André Hermanns	Nathan Hjelm	Torsten Hoefler
Daniel Holmes	Atsushi Hori	Yutaka Ishikawa
Hideyuki Jitsumoto	Jithin Jose	Krishna Kandalla
Christos Kavouklis	Takahiro Kawashima	Chulho Kim
Michael Knobloch	Alice Koniges	Quincey Koziol
Sameer Kumar	Joshua Ladd	Ignacio Laguna
Huiwei Lu	Guillaume Mercier	Kathryn Mohror
Adam Moody	Tomotake Nakamura	Takeshi Nanri
Steve Oyanagi	Antonio J. Peña	Sreeram Potluri
Howard Pritchard	Rolf Rabenseifner	Nicholas Radcliffe
Ken Raffenetti	Raghunath Raja	Craig Rasmussen
Davide Rossetti	Kento Sato	Martin Schulz
Sangmin Seo	Christian Siebert	Anthony Skjellum
Brian Smith	David Solt	Jeffrey M. Squyres

Hari Subramoni	Shinji Sumimoto	Alexander Supalov
Bronis R. de Supinski	Sayantana Sur	Masamichi Takagi
Keita Teranishi	Rajeev Thakur	Fabian Tillier
Yuichi Tsujita	Geoffroy Vallée	Rolf vandeVaart
Akshay Venkatesh	Jerome Vienne	Venkat Vishwanath
Anh Vo	Huseyin S. Yildiz	Junchao Zhang
Xin Zhao		

The MPI Forum also acknowledges and appreciates the valuable input from people via e-mail and in person.

The following institutions supported the MPI-3.1 effort through time and travel support for the people listed above.

Argonne National Laboratory
Auburn University
Cisco Systems, Inc.
Cray
EPCC, The University of Edinburgh
ETH Zurich
Forschungszentrum Jülich
Fujitsu
German Research School for Simulation Sciences
The HDF Group
International Business Machines
Institut National de Recherche en Informatique et Automatique (Inria)
Intel Corporation
Kyushu University
Lawrence Berkeley National Laboratory
Lawrence Livermore National Laboratory
Lenovo
Los Alamos National Laboratory
Mellanox Technologies, Inc.
Microsoft Corporation
NEC Corporation
NVIDIA Corporation
Oak Ridge National Laboratory
The Ohio State University
RIKEN AICS
Sandia National Laboratories
Texas Advanced Computing Center
Tokyo Institute of Technology
University of Alabama at Birmingham
University of Houston
University of Illinois at Urbana-Champaign
University of Oregon
University of Stuttgart, High Performance Computing Center Stuttgart (HLRS)
University of Tennessee, Knoxville
University of Tokyo

MPI-4.0:

MPI-4.0 is a major update to the MPI standard.

The editors and organizers of the MPI-4.0 have been:

- Martin Schulz, MPI-4.0 Chair, Info Object, External Interfaces
- Richard Graham, MPI-4.0 Treasurer
- Wesley Bland, MPI-4.0 Secretary, Backward Incompatibilities
- William Gropp, MPI-4.0 Editor, Steering Committee, Front Matter, Introduction, One-Sided Communications, and Bibliography
- Rolf Rabenseifner, Steering Committee, Process Topologies, Deprecated Functions, Removed Interfaces, Annex Language Bindings Summary, and Annex Change-Log
- Purushotham V. Bangalore, Language Bindings
- Claudia Blaas-Schenner, Terms and Conventions
- George Bosilca, Datatypes and Environmental Management
- Ryan E. Grant, Partitioned Communication
- Marc-André Hermanns, Tool Support
- Daniel Holmes, Point-to-Point Communication, Sessions
- Guillaume Mercier, Groups, Contexts, Communicators, Caching
- Howard Pritchard, Process Creation and Management
- Anthony Skjellum, Collective Communication, I/O

As part of the development of MPI-4.0, a number of working groups were established. In some cases, the work for these groups overlapped with multiple chapters. The following describes the major working groups and the leaders of those groups:

Collective Communication, Topology, Communicators: Torsten Hoefer, Andrew Lumsdaine, and Anthony Skjellum

Fault Tolerance: Wesley Bland, Aurélien Bouteiller, and Richard Graham

Hardware-Topologies: Guillaume Mercier

Hybrid & Accelerator: Pavan Balaji and James Dinan

Large Counts: Jeff Hammond

Persistence: Anthony Skjellum

Point to Point Communication: Daniel Holmes and Richard Graham

Remote Memory Access: William Gropp and Rajeev Thakur

Semantic Terms: Purushotham V. Bangalore and Rolf Rabenseifner

Sessions: Daniel Holmes and Howard Pritchard

Tools: Kathryn Mohror and Marc-André Hermanns

The following list includes some of the active participants who attended MPI Forum meetings or participated in the e-mail discussions.

Julien Adam	Abdelhalim Amer	Charles Archer
Ammar Ahmad Awan	Pavan Balaji	Purushotham V. Bangalore
Mohammadreza Bayatpour	Jean-Baptiste Besnard	Claudia Blaas-Schenner
Wesley Bland	Gil Bloch	George Bosilca
Aurélien Bouteiller	Ben Bratu	Alexander Calvert
Nicholas Chaimov	Sourav Chakraborty	Steffen Christgau
Ching-Hsiang Chu	Mikhail Chuvelev	James Clark
Carsten Clauss	Isaias Alberto Comprés Ureña	
Giuseppe Congiu	Brandon Cook	James Custer
Anna Daly	Hoang-Vu Dang	James Dinan
Matthew Dosanjh	Murali Emani	Christian Engelmann
Noah Evans	Ana Gainaru	Esthela Gallardo
Marc Gamell Balmana	Balazs Gerofi	Salvatore Di Girolamo
Brice Goglin	Manjunath Gorentla Venkata	
Richard Graham	Ryan E. Grant	Stanley Graves
William Gropp	Siegmar Gross	Taylor Groves
Yanfei Guo	Khaled Hamidouche	Jeff Hammond
Marc-André Hermanns	Nathan Hjelm	Torsten Hoefler
Daniel Holmes	Atsushi Hori	Josh Hursey
Ilya Ivanov	Julien Jaeger	Emmanuel Jeannot
Sylvain Jauegy	Jithin Jose	Krishna Kandalla
Takahiro Kawashima	Chulho Kim	Michael Knobloch
Alice Koniges	Sameer Kumar	Kim Kyunghun
Ignacio Laguna Peralta	Stefan Lankes	Tonglin Li
Xioyi Lu	Kavitha Madhu	Alexey Malhanov
Ryan Marshall	William Marts	Guillaume Mercier
Ali Mohammed	Kathryn Mohror	Takeshi Nanri
Thomas Naughton	Christoph Niethammer	Takafumi Nose
Lena Oden	Steve Oyanagi	Guillaume Papauré
Ivy Peng	Antonio Peña	Simon Pickartz
Artem Polyakov	Sreeram Potluri	Howard Pritchard
Martina Prugger	Marc Pérache	Rolf Rabenseifner
Nicholas Radcliffe	Ken Raffenetti	Craig Rasmussen
Soren Rasmussen	Hubert Ritzdorf	Sergio Rivas-Gomez
Davide Rossetti	Martin Ruefenacht	Amit Ruhela
Whit Schonbein	Joseph Schuchart	Martin Schulz
Sangmin Seo	Sameh Sharkawi	Sameer Shende
Min Si	Anthony Skjellum	Brian Smith
David Solt	Jeffrey M. Squyres	Srinivas Sridharan
Hari Subramoni	Nawrin Sultana	Shinji Sumimoto
Sayantan Sur	Hugo Taboada	Keita Teranishi

1 Rajeev Thakur Keith Underwood Geoffroy Vallee
2 Akshay Venkatesh Jerome Vienne Anh Vo
3 Justin Wozniak Junchao Zhang Dong Zhong
4 Hui Zhou

5
6
7 The MPI Forum also acknowledges and appreciates the valuable input from people via
8 e-mail and in person.

9 The following institutions supported the MPI-4.0 effort through time and travel support
10 for the people listed above.

11 ATOS
12 Argonne National Laboratory
13 Arm
14 Auburn University
15 Barcelona Supercomputing Center
16 CEA
17 Cisco Systems Inc.
18 Cray Inc.
19 EPCC, The University of Edinburgh
20 ETH Zürich
21 Fujitsu
22 Fulda University of Applied Sciences
23 German Research School for Simulation Sciences
24 Hewlett Packard Enterprise
25 International Business Machines
26 Institut National de Recherche en Informatique et Automatique (Inria)
27 Intel Corporation
28 Jülich Supercomputing Center, Forschungszentrum Jülich
29 KTH Royal Institute of Technology
30 Kyushu University
31 Lawrence Berkeley National Laboratory
32 Lawrence Livermore National Laboratory
33 Lenovo
34 Los Alamos National Laboratory
35 Mellanox Technologies, Inc.
36 Microsoft Corporation
37 NEC Corporation
38 NVIDIA Corporation
39 Oak Ridge National Laboratory
40 PAR-TEC
41 Paratools, Inc.
42 RIKEN AICS (R-CCS as of 2017)
43 RWTH Aachen University
44 Rutgers University
45 Sandia National Laboratories
46 Silicon Graphics, Inc.
47 Technical University of Munich
48 The HDF Group

The Ohio State University
 Texas Advanced Computing Center
 Tokyo Institute of Technology
 University of Alabama at Birmingham
 University of Basel, Switzerland
 University of Houston
 University of Illinois at Urbana-Champaign and the National Center for Supercomputing Applications
 University of Innsbruck
 University of Oregon
 University of Potsdam
 University of Stuttgart, High Performance Computing Center Stuttgart (HLRS)
 University of Tennessee, Chattanooga
 University of Tennessee, Knoxville
 University of Texas at El Paso
 University of Tokyo
 VSC Research Center, TU Wien

MPI-4.1:

MPI-4.1 is a minor update to the MPI standard.

The editors and organizers of the MPI-4.1 have been:

- Martin Schulz, MPI-4.1 Chair, Info Object, External Interfaces
- Brian Smith, MPI-4.1 Treasurer
- Wes Bland, MPI-4.1 Secretary, Semantic Changes and Warnings
- William Gropp, MPI-4.1 Editor, Steering Committee, Front Matter, Introduction, One-Sided Communications, and Bibliography
- Rolf Rabenseifner, Steering Committee, Process Topologies, Deprecated Functions, Removed Interfaces, Annex Language Bindings Summary, and Annex Change-Log
- Purushotham V. Bangalore, Language Bindings
- Claudia Blaas-Schenner, Terms and Conventions
- George Bosilca, Datatypes and Environmental Management
- Ryan E. Grant, Partitioned Communication
- Marc-André Hermanns, Tool Support
- Dan Holmes, Point-to-Point Communication, Sessions
- Guillaume Mercier, Groups, Contexts, Communicators, Caching
- Howard Pritchard, Process Creation and Management
- Anthony Skjellum, Collective Communication, I/O

As part of the development of MPI-4.1, a number of working groups were established or continued from MPI-4.0. In some cases, the work for these groups overlapped with multiple chapters. The following describes the major working groups and the leaders of those groups:

Application Binary Interface (ABI): Jeff Hammond and Quincey Koziol

Collective Communication, Topology, Communicators: Torsten Hoefer, Andrew Lumsdaine, and Anthony Skjellum

Fault Tolerance: Aurélien Bouteiller and Ignacio Laguna

Hardware & Virtual Topologies: Guillaume Mercier

Hybrid & Accelerator: James Dinan

Languages: Martin Ruefenacht and Tony Skjellum

Remote Memory Access: William Gropp, Joseph Schuchart, and Rajeev Thakur

Semantic Terms: Purushotham V. Bangalore and Rolf Rabenseifner

Sessions: Dan Holmes and Howard Pritchard

Tools: Marc-André Hermanns

The following list includes some of the active participants who attended MPI Forum meetings or participated in the e-mail discussions.

Julien Adam	Charles Archer	Christian Nicole Avans
Purushotham V. Bangalore	Wolfgang Bangerth	Jean-Baptiste Besnard
Claudia Blaas-Schenner	Wes Bland	George Bosilca
Aurélien Bouteiller	Emmanuel Brelle	Patrick Bridges
Alex Brooks	Jed Brown	Oliver Thomson Brown
Simon Byrne	Paul Canat	Ludovic Capelli
Ondřej Čertík	Steffen Christgau	Brandon Cook
Alfredo Correa	Pedro Costa	Lisandro Dalcín
Ali Can Demiralp	Bronis de Supinski	James Dinan
Matthew G. F. Dosanjh	Joe Downs	Julien Duprat
Dmitry Durnov	Victor Eijkhout	Ahmed Eleliemy
Kyle Gerard Felker	Edgar Gabriel	Maria J. Garzaran
Florent Germain	Sayan Ghosh	Thomas Gillis
Ryan Grant	William Gropp	Yanfei Guo
Samuel K. Gutierrez	Tobias Haas	Jeff Hammond
Marc-André Hermanns	Thomas Hines	Mark Hoemmen
Dan Holmes	Atsushi Hori	Dominik Huber
Joshua Hursey	Dan Ibanez	Nusrat Islam
Julien Jaeger	Nysal Jan K A	Joachim Jenke
Vivek Kale	Michael Klemm	Michael Knobloch
Quincey Koziol	Donald Kruse	Ignacio Laguna
Pierre Lemarinier	Ben Lynam	W. Pepper Marts
Guillaume Mercier	Jacob Merson	

Pedram Mohammadalizadehbakhtevvari	Ali Mohammed	1
Ali Omar Abdelazim Mohammed	Benson Muite	2
Evelyn Namugwanya	Grace Nansamba	3
Christoph Niethammer	William Okuno	4
Nick Papior	Sri Raj Paul	5
Rolf Rabenseifner	Nick Radcliffe	6
Naveen Ravichandrasekaran	Florian Reynier	7
Martin Ruefenacht	Amit Ruhela	8
Erik Schnetter	Whit Schonbein	9
Thorsten Schütt	Joseph Schuchart	10
Amir Shehata	Nat Shineman	11
Anthony Skjellum	Brian Smith	12
Jeff Squyres	Christopher Subich	13
Hugo Taboada	Daniel Taylor	14
Keita Teranishi	Rajeev Thakur	15
Jesper Larsson Träff	Ehsan Totoni	16
Isaías Alberto Comprés Ureña	Josef Weidendorfer	17
Bill Williams	Andrew Worley	18
	Hui Zhou	19

The MPI Forum also acknowledges and appreciates the valuable input from people via e-mail and in person.

The following institutions supported the MPI-4.1 effort through time and travel support for the people listed above.

Advanced Micro Devices, Inc.	25
Amazon.com, Inc	26
Argonne National Laboratory	27
Atos	28
CEA	29
Cisco Systems Inc.	30
Collis-Holmes Innovations Limited	31
Cornelis Networks	32
EPCC, The University of Edinburgh	33
Forschungszentrum Jülich	34
Fujitsu	35
HLRS, University of Stuttgart	36
Hewlett Packard Enterprise	37
Institut National de Recherche en Informatique et Automatique (Inria)	38
Intel Corporation	39
International Business Machines	40
Kichakato Kizito	41
Lawrence Berkeley National Laboratory	42
Lawrence Livermore National Laboratory	43
Leibniz Supercomputing Centre	44
Los Alamos National Laboratory	45
Meta Platforms Inc.	46
NVIDIA Corporation	47
Oak Ridge National Laboratory	48

1 ParaTools SAS
2 Queen's University
3 RWTH Aachen University
4 Sandia National Laboratory
5 Technical University of Munich
6 Tennessee Technological University
7 Texas Advanced Computing Center
8 The Ohio State University
9 University of Alabama
10 University of Alabama at Birmingham
11 University of Basel
12 University of Illinois Urbana-Champaign
13 University of New Mexico
14 University of Tennessee, Chattanooga
15 University of Tennessee, Knoxville
16 University of Tokyo
17 Université Grenoble Alpes
18 VSC Research Center, TU Wien
19 ZIH, TU Dresden
20

21 MPI-5.0:

22 MPI-5.0 is a major update to the MPI standard.

23 The editors and organizers of the MPI-5.0 have been:

- 24 • Martin Schulz, MPI-5.0 Chair, Info Object, External Interfaces
- 25 • Brian Smith, MPI-5.0 Treasurer
- 26 • Wes Bland, MPI-5.0 Secretary, Deprecated Functions, Removed Interfaces, Semantic
- 27 Changes and Warnings
- 28 • William Gropp, MPI-5.0 Editor, Steering Committee, Front Matter, Introduction,
- 29 One-Sided Communications, and Bibliography
- 30 • Purushotham V. Bangalore, Language Bindings
- 31 • Claudia Blaas-Schenner, Terms and Conventions
- 32 • George Bosilca, Datatypes and Environmental Management
- 33 • Ryan E. Grant, Partitioned Communication
- 34 • Marc-André Hermanns, Tool Support
- 35 • Tobias Haas, Change-Log
- 36 • Jeff Hammond, ABI
- 37 • Dan Holmes, Point-to-Point Communication, Sessions
- 38 • Guillaume Mercier, Groups, Contexts, Communicators, Caching

- Christoph Niethammer, Process Topologies
- Howard Pritchard, Process Creation and Management
- Anthony Skjellum, Collective Communication, I/O

As part of the development of MPI-5.0, a number of working groups were established or continued from MPI-4.1. In some cases, the work for these groups overlapped with multiple chapters. The following describes the major working groups and the leaders of those groups:

Application Binary Interface (ABI): Jeff Hammond and Quincey Koziol

Collective Communication, Topology, Communicators: Anthony Skjellum

Fault Tolerance: Aurélien Bouteiller and Ignacio Laguna

Fortran: Jeff Hammond, Purushotham V. Bangalore, and Anthony Skjellum

Hardware & Virtual Topologies: Guillaume Mercier

Hybrid & Accelerator: James Dinan

I/O: Quincey Koziol

Languages: Purushotham V. Bangalore, Jeff Hammond, and Tony Skjellum

Remote Memory Access: Joseph Schuchart and William Gropp

Sessions: Howard Pritchard and Dan Holmes

Tools: Marc-André Hermanns, Bill Williams and Joachim Jenke

The following list includes some of the active participants who attended MPI Forum meetings or participated in the virtual discussions.

Julien Adam	Purushotham V. Bangalore	Jean-Baptiste Besnard
Claudia Blaas-Schenner	Wes Bland	David Boehme
George Bosilca	Aurélien Bouteiller	Patrick Bridges
Jed Brown	Ludovic Capelli	Pranav Chachara
Jacob Chisholm	Gene Cooperman	Gregor Corbin
Lisandro Dalcin	Anton Daumen	James Dinan
Matthew G. F. Dosanjh	Victor Eijkhout	Ali Farazdaghi
Edgar Gabriel	Maria J. Garzaran	Florent Germain
Ryan E. Grant	William Gropp	Philipp Gschwandtner
Samuel K. Gutierrez	Tobias Haas	Jeff Hammond
Nathan Hanford	Sonja Happ	Marc-André Hermanns
Torsten Hoefler	Dan Holmes	Julien Jaeger
Michael Klemm	Michael Knobloch	Quincey Koziol
Donald Kruse	Ignacio Laguna	W. Pepper Marts
Edric Matwiejew	Guillaume Mercier	Benson Muite
Pedram Mohammadalizadehbakhtevani		Thomas Naughton
Christoph Niethammer	K. A. Nysal Jan	William Okuno

Ted Peters	Howard Pritchard	Joachim Jenke
Rolf Rabenseifner	Ken Raffanetti	Amirreza Rastegari
Naveen Ravichandrasekaran	Florian Reynier	Martin Ruefenacht
Amit Ruhela	Derek Schafer	Erik Schnetter
Martin Schreiber	Joseph Schuchart	Martin Schulz
Nat Shineman	Riley Shipley	Anthony Skjellum
Brian Smith	Marc Snir	AmirHossein Sojoodi
Jeff Squyres	Evan Suggs	Shinji Sumimoto
Hugo Taboada	Yiltan Hassan Temucin	Jesper Larsson Träff
Tim Niklas Uhl	Garrett Weil	Bill Williams
Andrew Worley	Yao Xu	Yuang Yan
Chelne Yu	Rohit Zambre	Hui Zhou

The following institutions supported the MPI-5.0 effort through time and travel support for the people listed above.

Advanced Micro Devices, Inc.
 Amazon.com, Inc
 Argonne National Laboratory
 Atos
 Barcelona Supercomputing Center
 CEA
 Collis-Holmes Innovations Limited
 Cornelis Networks
 EPCC, The University of Edinburgh
 ETH Zürich
 Eviden
 HLRS, University of Stuttgart
 Hewlett Packard Enterprise
 Institut National de Recherche en Informatique et Automatique (Inria)
 Intel Corporation
 International Business Machines
 JSC, Forschungszentrum Jülich
 KAUST
 Karlsruhe Institute of Technology (KIT)
 Kichakato Kizito
 Lawrence Livermore National Laboratory
 Los Alamos National Laboratory
 Meta Platforms Inc.
 Microsoft
 NVIDIA Corporation
 Northeastern University
 Oak Ridge National Laboratory
 ParTec
 ParaTools SAS
 Pawsey Supercomputing Centre
 Queen's University
 RWTH Aachen University

Sandia National Laboratory
Stony Brook University
Technical University of Munich
Tennessee Technological University
Texas Advanced Computing Center
The Ohio State University
University of Alabama
University of Edinburgh
University of Illinois Urbana-Champaign
University of Innsbruck
University of New Mexico
University of Tennessee, Knoxville
University of Tokyo
Université Grenoble Alpes
VSC Research Center, TU Wien
ZIH, TU Dresden

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

DRAFT

DRAFT

Chapter 1

Introduction to MPI

1.1 Overview and Goals

MPI (Message-Passing Interface) is a *message-passing library interface specification*. All parts of this definition are significant. MPI addresses primarily the message-passing parallel programming model, in which data is moved from the address space of one process to that of another process through cooperative operations on each process. Extensions to the “classical” message-passing model are provided in collective operations, remote-memory access operations, dynamic process creation, and parallel I/O. MPI is a *specification*, not an implementation; there are multiple implementations of MPI. This specification is for a *library interface*; MPI is not a language, and all MPI operations are expressed as functions, subroutines, or methods, according to the appropriate language bindings that, for C and Fortran, are part of the MPI standard. The standard has been defined through an open process by a community of parallel computing vendors, computer scientists, and application developers. The next few sections provide an overview of the history of MPI’s development.

The main advantages of establishing a message-passing standard are portability and ease of use. In a distributed memory communication environment in which the higher level routines and/or abstractions are built upon lower level message-passing routines, the benefits of standardization are particularly apparent. Furthermore, the definition of a message-passing standard, such as that proposed here, provides vendors with a clearly defined base set of routines that they can implement efficiently, or in some cases for which they can provide hardware support, thereby enhancing scalability.

The goal of the Message-Passing Interface, simply stated, is to develop a widely used standard for writing message-passing programs. As such the interface should establish a practical, portable, efficient, and flexible standard for message passing.

A complete list of goals follows.

- Design an application programming interface (not necessarily for compilers or a system implementation library).
- Allow efficient communication: Avoid memory-to-memory copying, allow overlap of computation and communication, and offload to communication co-processors, where available.
- Allow for implementations that can be used in a heterogeneous environment.
- Allow convenient C and Fortran bindings for the interface.
- Assume a reliable communication interface: the user need not cope with communication failures. Such failures are dealt with by the underlying communication subsystem.

- Define an interface that can be implemented on many vendor's platforms, with no significant changes in the underlying communication and system software.
- Semantics of the interface should be language independent.
- The interface should be designed to allow for thread safety.

1.2 Background of MPI-1.0

MPI sought to make use of the most attractive features of a number of existing message-passing systems, rather than selecting one of them and adopting it as the standard. Thus, MPI was strongly influenced by work at the IBM T. J. Watson Research Center [3, 4], Intel's NX/2 [58], Express [15], nCUBE's Vertex [54], p4 [10, 11], and PARMACS [7, 12]. Other important contributions have come from Zipcode [62, 63], Chimp [21, 22], PVM [6, 19], Chameleon [32], and PICL [27].

The MPI standardization effort involved about 60 people from 40 organizations mainly from the United States and Europe. Most of the major vendors of concurrent computers were involved in MPI, along with researchers from universities, government laboratories, and industry. The standardization process began with the Workshop on Standards for Message-Passing in a Distributed Memory Environment, sponsored by the Center for Research on Parallel Computing, held April 29–30, 1992, in Williamsburg, Virginia [70]. At this workshop the basic features essential to a standard message-passing interface were discussed, and a working group established to continue the standardization process.

A preliminary draft proposal, known as MPI-1, was put forward by Dongarra, Hempel, Hey, and Walker in November 1992, and a revised version was completed in February 1993 [20]. MPI-1 embodied the main features that were identified at the Williamsburg workshop as being necessary in a message passing standard. Since MPI-1 was primarily intended to promote discussion and “get the ball rolling,” it focused mainly on point-to-point communications. MPI-1 brought to the forefront a number of important standardization issues, but did not include any collective communication routines and was not thread-safe.

In November 1992, a meeting of the MPI working group was held in Minneapolis, at which it was decided to place the standardization process on a more formal footing, and to generally adopt the procedures and organization of the High Performance Fortran Forum. Subcommittees were formed for the major component areas of the standard, and an email discussion service established for each. In addition, the goal of producing a draft MPI standard by the Fall of 1993 was set. To achieve this goal the MPI working group met every 6 weeks for two days throughout the first 9 months of 1993, and presented the draft MPI standard at the Supercomputing 93 conference in November 1993. These meetings and the email discussion together constituted the MPI Forum, membership of which has been open to all members of the high performance computing community.

1.3 Background of MPI-1.1, MPI-1.2, and MPI-2.0

Beginning in March 1995, the MPI Forum began meeting to consider corrections and extensions to the original MPI standard document [24]. The first product of these deliberations was Version 1.1 of the MPI specification, released in June of 1995 [25] (see <http://www.mpi-forum.org> for official MPI document releases). At that time, effort focused in five areas.

1. Further corrections and clarifications for the MPI-1.1 document.
2. Additions to MPI-1.1 that do not significantly change its types of functionality (new datatype constructors, language interoperability, etc.).
3. Completely new types of functionality (dynamic processes, one-sided communication, parallel I/O, etc.) that are what everyone thinks of as “MPI-2 functionality.”
4. Bindings for Fortran 90 and C++. MPI-2 specifies C++ bindings for both MPI-1 and MPI-2 functions, and extensions to the Fortran 77 binding of MPI-1 and MPI-2 to handle Fortran 90 issues.
5. Discussions of areas in which the MPI process and framework seem likely to be useful, but where more discussion and experience are needed before standardization (e.g., zero-copy semantics on shared-memory machines, real-time specifications).

Corrections and clarifications (items of type 1 in the above list) were collected in Chapter 3 of the MPI-2 document: “Version 1.2 of MPI.” That chapter also contains the function for identifying the version number. Additions to MPI-1.1 (items of types 2, 3, and 4 in the above list) are in the remaining chapters of the MPI-2 document, and constitute the specification for MPI-2. Items of type 5 in the above list have been moved to a separate document, the “MPI Journal of Development” (JOD), and are not part of the MPI-2 standard.

This structure makes it easy for users and implementors to understand what level of MPI compliance a given implementation has:

- MPI-1 compliance will mean compliance with MPI-1.3. This is a useful level of compliance. It means that the implementation conforms to the clarifications of MPI-1.1 function behavior given in Chapter 3 of the MPI-2 document. Some implementations may require changes to be MPI-1 compliant.
- MPI-2 compliance will mean compliance with all of MPI-2.1.
- The MPI Journal of Development is not part of the MPI standard.

It is to be emphasized that forward compatibility is preserved. That is, a valid MPI-1.1 program is both a valid MPI-1.3 program and a valid MPI-2.1 program, and a valid MPI-1.3 program is a valid MPI-2.1 program.

1.4 Background of MPI-1.3 and MPI-2.1

After the release of MPI-2.0, the MPI Forum kept working on errata and clarifications for both standard documents (MPI-1.1 and MPI-2.0). The short document “Errata for MPI-1.1” was released October 12, 1998. On July 5, 2001, a first ballot of errata and clarifications for MPI-2.0 was released, and a second ballot was voted on May 22, 2002. Both votes were done electronically. Both ballots were combined into one document: “Errata for MPI-2,” May 15, 2002. This errata process was then interrupted, but the Forum and its e-mail reflectors kept working on new requests for clarification.

Restarting regular work of the MPI Forum was initiated in three meetings, at EuroPVM/MPI’06 in Bonn, at EuroPVM/MPI’07 in Paris, and at SC’07 in Reno. In December 2007, a steering committee started the organization of new MPI Forum meetings at regular 8-weeks intervals. At the January 14–16, 2008 meeting in Chicago, the MPI Forum

decided to combine the existing and future MPI documents to one document for each version of the MPI standard. For technical and historical reasons, this series was started with MPI-1.3. Additional Ballots 3 and 4 solved old questions from the errata list started in 1995 up to new questions from the last years. After all documents (MPI-1.1, MPI-2, Errata for MPI-1.1 (Oct. 12, 1998), and MPI-2.1 Ballots 1–4) were combined into one draft document, for each chapter, a chapter author and review team were defined. They cleaned up the document to achieve a consistent MPI-2.1 document. The final MPI-2.1 standard document was finished in June 2008, and finally released with a second vote in September 2008 in the meeting at Dublin, just before EuroPVM/MPI’08.

1.5 Background of MPI-2.2

MPI-2.2 is a minor update to the MPI-2.1 standard. This version addresses additional errors and ambiguities that were not corrected in the MPI-2.1 standard as well as a small number of extensions to MPI-2.1 that met the following criteria:

- Any correct MPI-2.1 program is a correct MPI-2.2 program.
- Any extension must have significant benefit for users.
- Any extension must not require significant implementation effort. To that end, all such changes are accompanied by an open source implementation.

The discussions of MPI-2.2 proceeded concurrently with the MPI-3 discussions; in some cases, extensions were proposed for MPI-2.2 but were later moved to MPI-3.

1.6 Background of MPI-3.0

MPI-3.0 is a major update to the MPI standard. The updates include the extension of collective operations to include nonblocking versions, extensions to the one-sided operations, and a new Fortran 2008 binding. In addition, the deprecated C++ bindings have been removed, as well as many of the deprecated routines and MPI objects (such as the `MPI_UB` datatype). Any valid MPI-2.2 program not using any of these removed MPI procedures or objects is a valid MPI-3.0 program.

1.7 Background of MPI-3.1

MPI-3.1 is a minor update to the MPI standard. Most of the updates are corrections and clarifications to the standard, especially for the Fortran bindings. New functions added include routines to manipulate `MPI_Aint` values in a portable manner, nonblocking collective I/O routines, and routines to get the index value by name for `MPI_T` performance and control variables. A general index was also added. Any valid MPI-3.0 program is a valid MPI-3.1 program.

1.8 Background of MPI-4.0

MPI-4.0 is a major update to the MPI standard. The largest changes are the addition of large-count versions of many routines to address the limitations of using an `int` or `INTEGER`

for the count parameter, persistent collectives, partitioned communications, an alternative way to initialize MPI, application info assertions, and improvements to the definitions of error handling. In addition, there are a number of smaller improvements and corrections. Any valid MPI-3.1 program is a valid MPI-4.0 program with the exception of semantic changes listed in Chapter 20.

1.9 Background of MPI-4.1

MPI-4.1 is a minor update to the MPI standard. It contains mostly corrections and clarifications to the MPI-4.0 document. Several routines, the attribute key `MPI_HOST`, and the `mpif.h` Fortran include file are deprecated. A new routine provides a way to inquire about the hardware on which the MPI program is running. Any valid MPI-4.0 program is a valid MPI-4.1 program with the exception of semantic changes listed in Chapter 20.

1.10 Background of MPI-5.0

MPI-5.0 is a major update to the MPI standard. The largest change is the addition of a standard Application Binary Interface (ABI) to allow interoperability of different implementations. In addition, there are a number of smaller improvements and corrections. Any valid MPI-4.1 program is a valid MPI-5.0 program.

1.11 Background of 202X Draft Specification

The 202X draft specification is expected to become the MPI-5.1 specification once all features have been merged.

1.12 Who Should Use This Standard?

This standard is intended for use by all those who want to write portable message-passing programs in Fortran and C (and access the C bindings from C++). This includes individual application programmers, developers of software designed to run on parallel machines, and creators of environments and tools. In order to be attractive to this wide audience, the standard must provide a simple, easy-to-use interface for the basic user while not semantically precluding the high-performance message-passing operations available on advanced machines.

1.13 What Platforms Are Targets for Implementation?

The attractiveness of the message-passing paradigm at least partially stems from its wide portability. Programs expressed this way may run on distributed-memory multiprocessors, networks of workstations, and combinations of all of these. In addition, shared-memory implementations, including those for multi-core processors and hybrid architectures, are possible. The paradigm will not be made obsolete by architectures combining the shared- and distributed-memory views, or by increases in network speeds. It thus should be both possible and useful to implement this standard on a great variety of machines, including

those “machines” consisting of collections of other machines, parallel or not, connected by a communication network.

The interface is suitable for use by fully general MIMD (Multiple Instruction, Multiple Data) programs, as well as those written in the more restricted style of SPMD (Single Program, Multiple Data). MPI provides many features intended to improve performance on scalable parallel computers with specialized interprocessor communication hardware. Thus, we expect that native, high-performance implementations of MPI will be provided on such machines. At the same time, implementations of MPI on top of standard Unix interprocessor communication protocols will provide portability to workstation clusters and heterogenous networks of workstations.

1.14 What Is Included in the Standard?

The standard includes:

- Point-to-point communication,
- Partitioned communication,
- Datatypes,
- Collective operations,
- Process groups,
- Communication contexts,
- Virtual Topologies for MPI Processes,
- Environmental management and inquiry,
- The Info object,
- Process initialization, creation, and management,
- One-sided communication,
- External interfaces,
- Parallel file I/O,
- Tool support,
- Process fault tolerance,
- Language bindings for Fortran and C,
- Application Binary Interface, and
- Additional topics in side-documents.

1.15 Side-documents

Side-documents extend and/or modify features, semantics, language bindings, and other aspects covered in this document. Side-documents shall not modify any aspects defined in the MPI Standard without providing a mechanism that explicitly enables these deviations. Execution of a program that does not explicitly enable deviations from the MPI Standard will comply with the MPI Standard, even when using an MPI implementation that implements a side-document that modifies any aspects.

Each side-document is versioned with a scheme that is independent from the MPI Standard version and from other side-documents. All side-documents specify compatibility and interoperability with versions of the MPI Standard and may define interoperability with features and semantics from other side-documents. Side-documents are not required to provide full coverage of all MPI concepts, but shall document which MPI concepts are affected. A compliant implementation is not required to comply with any side-documents. However, if compliance with a particular version of a side-document is claimed, the implementation must comply with the entire side-document. Side-documents will be found at the same location as the MPI Standard [1].

1.16 Organization of This Document

The following is a list of the remaining chapters in this document, along with a brief description of each.

- Chapter 2, [MPI Terms and Conventions](#), explains notational terms and conventions used throughout the MPI document.
- Chapter 3, [Point-to-Point Communication](#), defines the basic, pairwise communication subset of MPI. *Send* and *receive* are found here, along with many associated functions designed to make basic communication powerful and efficient.
- Chapter 4, [Partitioned Point-to-Point Communication](#), defines a method of performing partitioned communication in MPI. Partitioned communication allows multiple contributions of data to be made, potentially, from multiple actors (e.g., threads or tasks) in an MPI process to a single message.
- Chapter 5, [Datatypes](#), defines a method to describe any data layout, e.g., an array of structures.
- Chapter 6, [Collective Communication](#), defines process-group collective communication operations. Well known examples of this are barrier and broadcast over a group of processes (not necessarily all the processes).
- Chapter 7, [Groups, Contexts, Communicators, and Caching](#), shows how groups of processes are formed and manipulated, how unique communication contexts are obtained, and how the two are bound together into a *communicator*.
- Chapter 8, [Virtual Topologies for MPI Processes](#), explains a set of utility functions meant to assist in the mapping of MPI process groups (a linearly ordered set) to richer topological structures such as multi-dimensional grids.

- Chapter 9, [MPI Environmental Management](#), explains how the programmer can manage and make inquiries of the current MPI environment. These functions are needed for the writing of correct, robust programs, and are especially important for the construction of highly-portable message-passing programs.
- Chapter 10, [The Info Object](#), defines an opaque object that is used as input in several MPI routines.
- Chapter 11, [Process Initialization, Creation, and Management](#), defines several approaches to MPI initialization, process creation, and process management while placing minimal restrictions on the execution environment.
- Chapter 12, [One-Sided Communications](#), defines communication routines that can be completed by a single process. These include shared-memory operations (put/get) and remote accumulate operations.
- Chapter 14, [External Interfaces](#), defines routines designed to allow developers to layer on top of MPI.
- Chapter 15, [I/O](#), defines MPI support for parallel I/O.
- Chapter 16, [Tool Support](#), covers interfaces that allow debuggers, performance analyzers, and other tools to obtain data about the operation of MPI processes.
- Chapter 17, [Process Fault Tolerance](#), covers interfaces that allow developers to design applications tolerant to MPI process failures. The definitions and interfaces presented in this chapter describe the behavior of MPI operations after a process failure error has been raised, and provide supplementary interfaces to restore the communication capabilities of MPI.
- Chapter 18, [Deprecated Interfaces](#), describes routines that are kept for reference. However usage of these functions is discouraged, as they may be deleted in future versions of the standard.
- Chapter 19, [Removed Interfaces](#), describes routines and constructs that have been removed from MPI.
- Chapter 20, [Semantic Changes and Warnings](#), describes semantic changes from previous versions of MPI.
- Chapter 21, [Language Bindings](#), discusses Fortran issues and describes language interoperability aspects between C and Fortran.
- Chapter 22, [Application Binary Interface \(ABI\)](#), discusses Application Binary Interface issues and describes interoperability of compiled code.

The Appendices are:

- Annex A, [Language Bindings Summary](#), gives specific syntax in C and Fortran for all MPI functions, constants, and types.
- Annex B, [Change-Log](#), summarizes some changes since the previous version of the standard.

- Several index pages show the locations of [general terms and definitions](#), [examples](#), [constants and predefined handles](#), [declarations of C and Fortran types](#), [callback routine prototypes](#), and all [MPI functions](#).

MPI provides various interfaces to facilitate interoperability of distinct MPI implementations. Among these are the canonical data representation for MPI I/O and for [MPI_PACK_EXTERNAL](#) and [MPI_UNPACK_EXTERNAL](#). The definition of an actual binding of these interfaces that will enable interoperability is outside the scope of this document.

A separate document consists of ideas that were discussed in the MPI Forum during the MPI-2 development and deemed to have value, but were not included in the MPI standard. They are part of the “Journal of Development” (JOD), which was created to capture these ideas and discussions. The JOD is available at <https://www.mpi-forum.org/docs>.

DRAFT

Chapter 2

MPI Terms and Conventions

This chapter explains notational terms and conventions used throughout the MPI document, some of the choices that have been made, and the rationale behind those choices.

2.1 Document Notation

Rationale. Throughout this document, the rationale for the design choices made in the interface specification is set off in this format. Some readers may wish to skip these sections, while readers interested in interface design may want to read them carefully. (*End of rationale.*)

Advice to users. Throughout this document, material aimed at users and that illustrates usage is set off in this format. Some readers may wish to skip these sections, while readers interested in programming in MPI may want to read them carefully. (*End of advice to users.*)

Advice to implementors. Throughout this document, material that is primarily commentary to implementors is set off in this format. Some readers may wish to skip these sections, while readers interested in MPI implementations may want to read them carefully. (*End of advice to implementors.*)

2.2 Naming Conventions

In many cases MPI names for C functions are of the form `MPI_Class_action_subset`. This convention originated with MPI-1. Since MPI-2 an attempt has been made to standardize the names of MPI functions according to the following rules.

1. In C and the Fortran `mpi_f08` module, all routines associated with a particular type of MPI object should be of the form `MPI_Class_action_subset` or, if no subset exists, of the form `MPI_Class_action`. In the Fortran `mpi` module and (deprecated) `mpif.h` file, all routines associated with a particular type of MPI object should be of the form `MPI_CLASS_ACTION_SUBSET` or, if no subset exists, of the form `MPI_CLASS_ACTION`.
2. If the routine is not associated with a class, the name should be of the form `MPI_Action_subset` or `MPI_ACTION_SUBSET` in C and Fortran.
3. The names of certain actions have been standardized. In particular, **create** creates a new object, **get** retrieves information about an object, **set** sets this information, **delete** deletes information, **is** asks whether or not an object has a certain property.

C and Fortran names for some MPI functions (that were defined during the MPI-1 process) violate these rules in several cases. The most common exceptions are the omission of the **Class** name from the routine and the omission of the **Action** where one can be inferred.

2.3 Procedure Specification

MPI procedures are specified using a language-independent notation. The arguments of procedure calls are marked as IN, OUT, or INOUT. The meanings of these are:

IN: the call may use the input value but does not update the argument from the perspective of the caller at any time during the call’s execution,

OUT: the call may update the argument but does not use its input value,

INOUT: the call may both use and update the argument.

There is one special case—if an argument is a handle to an opaque object (these terms are defined in Section 2.5.1), and the object is updated by the procedure call, then the argument is marked INOUT or OUT. It is marked this way even though the handle itself is not modified—we use the INOUT or OUT attribute to denote that what the handle *references* is updated.

Rationale. The definition of MPI tries to avoid, to the largest possible extent, the use of INOUT arguments, because such use is error-prone, especially for scalar arguments. (*End of rationale.*)

MPI’s use of IN, OUT, and INOUT is intended to indicate to the user how an argument is to be used, but does not provide a rigorous classification that can be translated directly into all language bindings (e.g., **INTENT** in Fortran 90 bindings or **const** in C bindings). For instance, the “constant” **MPI_BOTTOM** can usually be passed to OUT buffer arguments. Similarly, **MPI_STATUS_IGNORE** can be passed as the OUT status argument.

A common occurrence for MPI functions is an argument that is used as IN by some processes and OUT by other processes. Such an argument is, syntactically, an INOUT argument and is marked as such, although, semantically, it is not used in one call both for input and for output on a single process.

Another frequent situation arises when an argument value is needed only by a subset of the processes. When an argument is not significant at a process then an arbitrary value can be passed as an argument.

Advice to users. Note that the programmer is still responsible for avoiding undefined behavior in the host language by not passing uninitialized values to MPI procedure calls. (*End of advice to users.*)

Unless specified otherwise, an argument of type OUT or type INOUT cannot be aliased with any other argument passed to an MPI procedure. An example of argument aliasing in C appears below. If we define a C procedure like this,

```
void copyIntBuffer(int *pin, int *pout, int len)
{
    int i;
    for (i=0; i<len; ++i) *pout++ = *pin++;
}
```


then a call to it in the following code fragment has aliased arguments.

```
int a[10];
copyIntBuffer(a, a+3, 7);
```

Although the C language allows this, such usage of MPI procedures is forbidden unless otherwise specified. Note that Fortran prohibits aliasing of arguments.

All MPI functions are first specified in the language-independent notation. Immediately below this, language dependent bindings follow:

- The ISO C version(s) of the function.
- The Fortran version(s) used with `USE mpi_f08`.
- The Fortran version of the same function used with `USE mpi` or (deprecated) `INCLUDE 'mpif.h'`.

Some MPI procedures have two interfaces for a given language support; see Sections 2.5.6 and 2.5.8.

An exception is Section 16.3 “The MPI Tool Information Interface”, which only provides ISO C interfaces.

“Fortran” in this document refers to Fortran 90 or later; see Section 2.6.

The words function, routine, procedure, procedure call, and call are often used as synonyms within this standard.

2.4 Semantic Terms

When discussing MPI procedures the following semantic terms are used. The term **message data buffer** refers to the send/receive buffer used in a communication procedure. The term **file data buffer** refers to the data buffers used by MPI I/O procedures. In this section we use the term **data buffer** and depending on the MPI procedure it will refer to message data buffer or file data buffer. Annex A.2 shows how the terms defined in this section apply to all operation-related MPI procedures.

2.4.1 MPI Operations

MPI operation: An MPI operation is a sequence of steps performed by the MPI library to establish and enable data transfer and/or synchronization. It consists of four stages: initialization, starting, completion, and freeing, and it is implemented as a set of one or more MPI procedures, see Section 2.4.2.

Initialization hands over the argument list to the operation but not the content of the data buffers, if any. The specification of an operation may state that array arguments must not be changed until the operation is freed.

Starting hands over control of the data buffers, if any, to the associated operation.

Note that **initiation** refers to the combination of the initialization and starting stages.

Completion returns control of the content of the data buffers and indicates that output buffers and arguments, if any, have been updated.

Note that an MPI operation is **complete** when the MPI procedure implementing the completion stage returns.



Figure 2.1: State transition diagram for blocking operations



Figure 2.2: State transition diagram for nonblocking operations

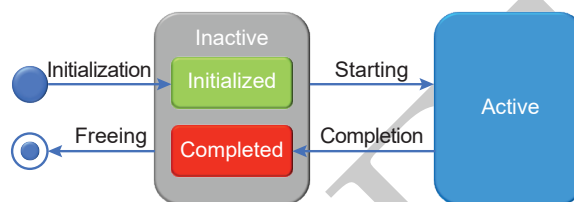


Figure 2.3: State transition diagram for persistent operations

Freeing returns control of the rest of the argument list (e.g., the data buffer address and array arguments).

MPI operations are available in one or more of these forms: blocking, nonblocking, and persistent.

Blocking operation: For a **blocking operation**, all four stages are combined in a single procedure call (as shown in Figure 2.1 and defined in Section 2.4.2).

Nonblocking operation: For a **nonblocking operation**, the initialization and starting stages are combined into a single nonblocking procedure call and the completion and freeing stages are combined into a separate, single procedure call, which can be blocking or nonblocking (as shown in Figure 2.2 and defined in Section 2.4.2).

Persistent operation: For a **persistent operation**, there is a separate procedure for each of the four stages (as shown in Figure 2.3 and defined in Section 2.4.2). Each of these procedures may be blocking or nonblocking.

For a partitioned send operation, an additional call to activate each partition of the send buffer (see Section 4.2.1) is required to finish the starting stage. For a partitioned receive operation, before the operation is complete the user is allowed to access a partition of the output buffer after verifying that it has arrived (see Section 4.2.2).

These four stages lead to the **operation states initialized, started, complete, and freed**. A *started operation* is also named **active**, and the states *initialized* and *complete* are also named **inactive**.

Active communication and I/O operations are also named **pending** operations. Note that a *pending* operation can be a nonblocking or persistent operation that is started and not yet complete (even if the request handle has been freed), or a blocking operation that is not yet complete, such as a receive operation that is waiting for a message to be received.

Additionally, an MPI operation can be collective or noncollective.

Collective operation: A **collective operation** is a set of related operations, one per MPI process in a group or groups of MPI processes. For collective operations the completion stage may or may not finish before all processes in the group have started the operation.

Collective MPI operations are also available as blocking, nonblocking, or persistent operations.

Noncollective operation: **Noncollective operations** are defined as operations that are not collective.

Many MPI operations coordinate activities at multiple MPI processes: the semantics of such an operation require one or more other specific semantically-related operations to be *started* before it is guaranteed that the operation can transition to the *complete* operation state. For example, a receive operation requires a related send operation to be started before the receive can complete; or a collective operation might not complete before such operations are also started in all MPI processes of the respective group.

Enabled: An MPI operation is **enabled** at a particular MPI process when all specific semantically-related operations required to guarantee completion at that MPI process have been started.

Rationale. MPI implementations may include optimizations (for example, automatic buffering) that allow an MPI operation to complete before it is enabled. (*End of rationale.*)

Some MPI operations are **a priori enabled**, i.e., they do not require any other specific semantically-related operation for completion. For example, a buffered send operation completes independently of the related receive operation.

Once an MPI operation is *enabled*, the operation must eventually complete. An operation may already be enabled before it is started. For example, a receive operation is already enabled if it is started after the matching send operation was started.

Rationale. The definition of an operation A being *enabled* is asymmetric: *enabled* includes that all specific semantically-related operations A'_i required to guarantee completion have been started, but does not include that the operation A itself is already started.

Examples:

- A receive is enabled exactly when the related send is started.
- A standard mode send operation is enabled exactly when the related receive is started. If an MPI implementation chooses to use internal buffering, the send operation may be already completed before it is enabled, i.e., the receive is started.
- A synchronous mode send operation is enabled exactly when the related receive is started and must not complete before it is enabled.

- A buffered mode send operation is a priori enabled.
- A ready mode send can be started only when it is already enabled, i.e., the related receive is started.
- For a collective broadcast, the operation at a particular MPI process is enabled exactly when all other MPI processes in the group have started their related broadcast operation.

Specifically, for the set of related operations on a group of MPI processes that constitute a collective operation that may synchronize, the operation on a particular MPI process p is enabled when all other MPI processes $p_i \neq p$ in the group have started their related operation, while the operation on p need not have started yet. (*End of rationale.*)

2.4.2 MPI Procedures

All MPI procedures can either be *local* or *nonlocal*—defined as follows:

Nonlocal procedure: An MPI procedure is **nonlocal** if returning may require, during its execution, some specific semantically-related MPI procedure to be called on another MPI process.

Local procedure: An MPI procedure is **local** if it is not *nonlocal*.

An MPI operation is implemented as a set of one or more MPI procedures. An MPI **operation-related procedure** implements at least a part of a stage of an MPI operation as described in Section 2.4.1. An MPI operation-related procedure may also implement one or more stages of one or several MPI operations. In certain cases, more than one MPI operation-related procedure may be needed to implement a single stage.

There are also other MPI procedures that do not implement any stage of any MPI operation.

The semantics of MPI operation-related procedures are described using two orthogonal (independent) concepts: completeness (depends on which stages are included) and locality. Such procedures can be either incomplete, or completing, or freeing, or completing and freeing based on the status of the associated operation at the time the procedure returns. Also, all such procedures can be described as either blocking or nonblocking, but these latter two terms refer to combinations of the completeness and locality concepts. Additionally, all MPI operation-related procedures can be collective or noncollective.

The following are properties of MPI operation-related procedures:

Initialization procedure: An MPI procedure is an **initialization procedure** if return from the procedure indicates that the associated operation has completed its initialization stage, which implies that the user has handed over control of the argument list (but not contents of the data buffers) to MPI. The user is still allowed to read or modify the contents of the data buffers. If an initializing procedure is not also the freeing procedure of the associated operation (see below) then the user is not permitted to deallocate the data buffers or to modify the array arguments.

Starting procedure: An MPI procedure is a **starting procedure** if return from the procedure indicates that the associated operation has completed its starting stage, which implies that the user has handed over control of the data buffers to MPI. If a starting procedure is not also a completing procedure of the associated operation (see

below) then the user is not permitted to modify input data buffers or to read output data buffers.

Initiation procedure: An MPI procedure is an **initiation procedure** if return from the procedure indicates that both the initialization and the starting stage have completed, which implies control of the entire argument list is handed over to MPI.

Completing procedure: An MPI procedure is called **completing** if return from the procedure indicates that at least one associated operation has finished its completion stage, which implies that the user can rely on the content of the output data buffers and modify the content of input and output data buffers of such operation(s). If a completing procedure is not also a freeing procedure (see below) then the user is not permitted to deallocate the data buffers or to modify the array arguments.

Incomplete procedure: An MPI procedure is called **incomplete** if it is not a completing procedure.

Freeing procedure: An MPI procedure is **freeing** if return from the procedure indicates that at least one associated operation has finished its freeing stage, which implies that the user can reuse all parameters specified when initializing such associated operation(s).

Nonblocking procedure: An MPI procedure is **nonblocking** if it is incomplete and local.

Blocking procedure: An MPI procedure is **blocking** if it is not nonblocking.

Advice to users. Note that for operation-related MPI procedures, in most cases incomplete procedures are local and completing procedures are nonlocal. Exceptions are noted where such procedures are defined. In many cases an additional prefix letter **l** as an abbreviation of the words **incomplete** and **immediate** marks nonblocking procedures in the procedure name.

Some categorization examples are listed below.

Nonblocking procedures:

- incomplete and local: `MPI_ISEND`, `MPI_IRECV`, `MPI_IBCAST`, `MPI_IMPROBE`, `MPI_SEND_INIT`, `MPI_RECV_INIT`, ...

Blocking procedures:

- completing and nonlocal: `MPI_SEND`, `MPI_RECV`, `MPI_BCAST`, ...
- incomplete and nonlocal: `MPI_MPROBE`, `MPI_BCAST_INIT`, ..., `MPI_FILE_{READ|WRITE}_{AT_ALL|ALL|ORDERED}_BEGIN`.
- completing and local: `MPI_BSEND`, `MPI_RSEND`, `MPI_MRECV`.

MPI procedures that are not MPI operation-related:

- `MPI_COMM_RANK`, `MPI_WTIME`, `MPI_PROBE`, `MPI_IPROBE`, ...

(End of advice to users.)

Collective procedure: An MPI procedure is **collective** if all processes in a group or groups of MPI processes need to invoke the procedure.

Initialization procedures of collective operations over the same process group must be executed in the same order by all members of the process group.

An MPI collective procedure is **synchronizing** if it will only return once all processes in the associated group or groups of MPI processes have called the appropriate matching MPI procedure.

The initiation procedures for nonblocking collective operations and the starting procedures for persistent collective operations are local and shall not be synchronizing.

All other procedures for collective operations, such as for blocking collective operations and the initialization procedures for persistent collective operations, may or may not be synchronizing.

Advice to users. Calling any synchronizing function is erroneous when there is no possibility of corresponding calls at all other processes in the associated process group.

Waiting for completion of any collective operation is erroneous when there is no possibility that all other processes in the associated group will be able to start the corresponding operation. (*End of advice to users.*)

Noncollective procedure: Noncollective procedures are defined as procedures that are not collective.

The definition of **local** and **nonlocal** MPI procedures can also be applied to a specific procedure invocation or to procedure calls **under certain constraints**. For example, a call to a completing receive procedure that happens after the related send operation was already started may be described as local, even though the completing receive procedure without the constraint is nonlocal. More generally, a call to any completing procedure that happens after the operation was already *enabled* is local, even if the completing procedure without the constraint is nonlocal. Another example, a call to a blocking collective procedure using a process group of size one is local, even if the blocking collective procedure without the constraint is nonlocal.

2.4.3 MPI Datatypes

For datatypes, the following terms are defined:

predefined: A predefined datatype is a datatype with a predefined (constant) name (such as `MPI_INT`, `MPI_FLOAT_INT`, or `MPI_PACKED`) or a datatype constructed with `MPI_TYPE_CREATE_F90_INTEGER`, `MPI_TYPE_CREATE_F90_REAL`, or `MPI_TYPE_CREATE_F90_COMPLEX`. The former are **named** whereas the latter are **unnamed**.

derived: A derived datatype is any datatype that is not predefined.

portable: A datatype is portable if it is a predefined datatype, or it is derived from a portable datatype using only the type constructors `MPI_TYPE_CONTIGUOUS`, `MPI_TYPE_VECTOR`, `MPI_TYPE_INDEXED`, `MPI_TYPE_CREATE_INDEXED_BLOCK`, `MPI_TYPE_CREATE_SUBARRAY`, `MPI_TYPE_DUP`, and `MPI_TYPE_CREATE_DARRAY`. Such a datatype is portable because all displacements in the datatype are in terms of extents of one predefined

datatype. Therefore, if such a datatype fits a data layout in one memory, it will fit the corresponding data layout in another memory, if the same declarations were used, even if the two systems have different architectures. On the other hand, if a datatype was constructed using `MPI_TYPE_CREATE_HINDEXED`, `MPI_TYPE_CREATE_HINDEXED_BLOCK`, `MPI_TYPE_CREATE_HVECTOR` or `MPI_TYPE_CREATE_STRUCT`, then the datatype contains explicit byte displacements (e.g., providing padding to meet alignment restrictions). These displacements are unlikely to be chosen correctly if they fit data layout on one memory, but are used for data layouts on another process, running on a processor with a different architecture.

equivalent: Two datatypes are equivalent if they appear to have been created with the same sequence of calls (and arguments) and thus have the same typemap. Two equivalent datatypes do not necessarily have the same cached attributes or the same names.

2.5 Datatypes

2.5.1 Opaque Objects

MPI manages **system memory** that is used for buffering messages and for storing internal representations of various MPI objects such as groups, communicators, datatypes, etc. This memory is not directly accessible to the user, and objects stored there are **opaque**: their size and shape is not visible to the user. Opaque objects are accessed via **handles**, which exist in user space. MPI procedures that operate on opaque objects are passed handle arguments to access these objects. In addition to their use by MPI calls for object access, handles can participate in assignments and comparisons.

In Fortran with `USE mpi` or (deprecated) `INCLUDE 'mpif.h'`, all handles have type `INTEGER`. In Fortran with `USE mpi_f08`, and in C, a different handle type is defined for each category of objects. With Fortran `USE mpi_f08`, the handles are defined as Fortran `BIND(C)` derived types that consist of only one element `INTEGER :: MPI_VAL`. The internal handle value is identical to the Fortran `INTEGER` value used in the `mpi` module and (deprecated) `mpif.h`. The operators `.EQ.`, `.NE.`, `==` and `/=` are overloaded to allow the comparison of these handles. The type names are identical to the names in C, except that they are not case sensitive. For example:

```
TYPE, BIND(C) :: MPI_Comm
  INTEGER      :: MPI_VAL
END TYPE MPI_Comm
```

The C types must support the use of the assignment and equality operators.

Advice to implementors. In Fortran, the handle can be an index into a table of opaque objects in a system table; in C it can be such an index or a pointer to the object. (*End of advice to implementors.*)

Rationale. Since the Fortran integer values are equivalent, applications can easily convert MPI handles between all three supported Fortran methods. For example, an integer communicator handle `COMM` can be converted directly into an exactly equivalent `mpi_f08` communicator handle named `comm_f08` by `comm_f08%MPI_VAL=COMM`, and vice versa. The use of the `INTEGER` defined handles and the `BIND(C)` derived type handles

is different: Fortran 2003 (and later) define that BIND(C) derived types can be used within user defined common blocks, but it is up to the rules of the companion C compiler how many numerical storage units are used for these BIND(C) derived type handles. Most compilers use one unit for both, the INTEGER handles and the handles defined as BIND(C) derived types. (*End of rationale.*)

Advice to users. If a user wants to substitute the `mpi` module or the (deprecated) `mpif.h` by the `mpi_f08` module and the application program stores a handle in a Fortran common block then it is necessary to change the Fortran support method in all application routines that use this common block, because the number of numerical storage units of such a handle can be different in the two modules. (*End of advice to users.*)

Opaque objects are allocated and deallocated by calls that are specific to each object type. These are listed in the sections where the objects are described. The calls accept a handle argument of matching type. In an allocate call this is an OUT argument that returns a valid reference to the object. In a call to deallocate this is an INOUT argument that returns with an “invalid handle” value. MPI provides an “invalid handle” constant for each object type. Comparisons to this constant are used to test for validity of the handle.

A call to a deallocate routine invalidates the handle and marks the object for deallocation. The object is not accessible to the user after the call. However, MPI need not deallocate the object immediately. Any operation *pending* (at the time of the deallocate) and *decoupled MPI activity* (see Section 2.9) that involves this object will complete normally; the object will be deallocated afterwards.

An opaque object and its handle are significant only at the process where the object was created and cannot be transferred to another process.

MPI provides certain predefined opaque objects and predefined, static handles to these objects. The user must not free such objects.

Rationale. This design hides the internal representation used for MPI data structures, thus allowing similar calls in C and Fortran. It also avoids conflicts with the typing rules in these languages, and easily allows future extensions of functionality. The mechanism for opaque objects used here loosely follows the POSIX Fortran binding standard.

The explicit separation of handles in user space and objects in system space allows space-reclaiming and deallocation calls to be made at appropriate points in the user program. If the opaque objects were in user space, one would have to be very careful not to go out of scope before any pending operation requiring that object completed. The specified design allows an object to be marked for deallocation, the user program can then go out of scope, and the object itself still persists until any pending operations are complete.

The requirement that handles support assignment/comparison is made since such operations are common. This restricts the domain of possible implementations. The alternative in C would have been to allow handles to have been an arbitrary, opaque type. This would force the introduction of routines to do assignment and comparison, adding complexity, and was therefore ruled out. In Fortran, the handles are defined such that assignment and comparison are available through the operators of the language or overloaded versions of these operators. (*End of rationale.*)

Advice to users. A user may accidentally create a dangling reference by assigning to a handle the value of another handle, and then deallocating the object associated with these handles. Conversely, if a handle variable is deallocated before the associated object is freed, then the object becomes inaccessible (this may occur, for example, if the handle is a local variable within a subroutine, and the subroutine is exited before the associated object is deallocated). It is the user's responsibility to avoid adding or deleting references to opaque objects, except as a result of MPI calls that allocate or deallocate such objects. (*End of advice to users.*)

Advice to implementors. The intended semantics of opaque objects is that opaque objects are separate from one another; each call to allocate such an object copies all the information required for the object. Implementations may avoid excessive copying by substituting referencing for copying. For example, a derived datatype may contain references to its components, rather than copies of its components; a call to `MPI_COMM_GROUP` may return a reference to the group associated with the communicator, rather than a copy of this group. In such cases, the implementation must maintain reference counts, and allocate and deallocate objects in such a way that the visible effect is as if the objects were copied. (*End of advice to implementors.*)

2.5.2 Array Arguments

An MPI call may need an argument that is an array of opaque objects, or an array of handles. The array-of-handles is a regular array with entries that are handles to objects of the same type in consecutive locations in the array. Whenever such an array is used, an additional `len` argument is required to indicate the number of valid entries (unless this number can be derived otherwise). The valid entries are at the beginning of the array; `len` indicates how many of them there are, and need not be the size of the entire array. The same approach is followed for other array arguments. In some cases NULL handles are considered valid entries. When a NULL argument is desired for an array of statuses, one uses `MPI_STATUSES_IGNORE`.

2.5.3 State

MPI procedures use at various places arguments with *state* types. The values of such a datatype are all identified by names, and no operation is defined on them. For example, the `MPI_TYPE_CREATE_SUBARRAY` routine has a state argument `order` with values `MPI_ORDER_C` and `MPI_ORDER_FORTRAN`.

2.5.4 Named Constants

MPI procedures sometimes assign a special meaning to a special value of a basic type argument; e.g., `tag` is an integer-valued argument of point-to-point communication operations, with a special wild-card value, `MPI_ANY_TAG`. Such arguments will have a range of regular values, which is a proper subrange of the range of values of the corresponding basic type; special values (such as `MPI_ANY_TAG`) will be outside the regular range. The range of regular values, such as `tag`, can be queried using environmental inquiry functions, see Chapter 9. The range of other values, such as `source`, depends on values given by other MPI routines (in the case of `source` it is the communicator size).

MPI also provides predefined named constant handles, such as `MPI_COMM_WORLD`.

All named MPI constants, with the exceptions noted below for Fortran, can be used in initialization expressions or assignments. Opaque objects accessed by constant handles are defined and do not change value between MPI initialization (e.g., with `MPI_INIT`) and MPI finalization (e.g., with `MPI_FINALIZE`). The handles themselves are constants and can be also used in initialization expressions or assignments.

In C, all named MPI constants that are described as “integer constant expression” in Section A.1.1 must be implemented as *C integer constant expressions* of the specified integer type. All other MPI constants in C are not required to be *C integer constant expressions* but must be usable in initialization expressions and assignments. Thus, they are not guaranteed to be usable in array declarations or as case-labels in `switch` statements.

In Fortran, all named MPI constants (with the exceptions below) must be declared with the `PARAMETER` attribute.

The constants that cannot be used in initialization expressions or assignments in Fortran are as follows:

```
MPI_BOTTOM
MPI_BUFFER_AUTOMATIC
MPI_STATUS_IGNORE
MPI_STATUSES_IGNORE
MPI_ERRCODES_IGNORE
MPI_IN_PLACE
MPI_ARGV_NULL
MPI_ARGVS_NULL
MPI_UNWEIGHTED
MPI_WEIGHTS_EMPTY
```

Advice to implementors. In Fortran the implementation of these special constants may require the use of language constructs that are outside the Fortran standard. Using special values for the constants (e.g., by defining them through `PARAMETER` statements) is not possible because an implementation cannot distinguish these values from valid data. Typically, these constants are implemented as predefined static variables (e.g., a variable in an MPI-declared `COMMON` block), relying on the fact that the target compiler passes data by address. Inside the subroutine, this address can be extracted by some mechanism outside the Fortran standard (e.g., by Fortran extensions or by implementing the function in C). (*End of advice to implementors.*)

2.5.5 Choice

MPI functions sometimes use arguments with a *choice* (or union) data type. Distinct calls to the same routine may pass by reference actual arguments of different types. The mechanism for providing such arguments will differ from language to language. For Fortran with the (deprecated) include file `mpif.h` or the `mpi` module, the document uses `<type>` to represent a choice variable; with the Fortran `mpi_f08` module, such arguments are declared with the Fortran 2018 syntax `TYPE(*), DIMENSION(..)`; for C, we use `void*`.

Advice to implementors. Implementors can freely choose how to implement choice arguments in the `mpi` module, e.g., with a nonstandard compiler-dependent method that has the quality of the call mechanism in the implicit Fortran interfaces, or with the method defined for the `mpi_f08` module. See details in Section 21.1.1. (*End of advice to implementors.*)

2.5.6 Absolute Addresses and Relative Address Displacements

Some MPI procedures use *address* arguments that represent an *absolute address* in the calling program, or *relative displacement* arguments that represent differences of two absolute addresses. The datatype of such arguments is `MPI_Aint` in C and `INTEGER(KIND=MPI_ADDRESS_KIND)` in Fortran. These types must have the same width and encode address values in the same manner such that address values in one language may be passed directly to another language without conversion. There is the MPI constant `MPI_BOTTOM` to indicate the start of the address range. For retrieving absolute addresses or any calculation with absolute addresses, one should use the routines and functions provided in Section 5.1.5. Section 5.1.12 provides additional rules for the correct use of absolute addresses. For expressions with relative displacements or other usage without absolute addresses, intrinsic operators (e.g., `+`, `-`, `*`) can be used.

Rationale. Byte displacement values need to be large enough to encode any value used for expressing absolute or relative memory addresses. Prior to MPI-4.0, some MPI routines used `int` in C and `INTEGER` in Fortran as the type for *byte displacement* arguments. To avoid breaking backward compatibility, this version of the standard continues to support `int` in C as well as `INTEGER` in Fortran in such routines. In addition, this version of the standard supports using `MPI_Aint` in C (via separate “_c” suffixed procedures) as well as `INTEGER(KIND=MPI_ADDRESS_KIND)` in Fortran (via polymorphic interfaces in newer MPI Fortran bindings (USE `mpi_f08`)) in such routines. See Section 21.2 for a full explanation. (*End of rationale.*)

2.5.7 File Offsets

For I/O there is a need to give the size, displacement, and offset into a file. These quantities can easily be larger than 32 bits, which can be the default size of a Fortran integer. To overcome this, these quantities are declared to be `INTEGER(KIND=MPI_OFFSET_KIND)` in Fortran. In C one uses `MPI_Offset`. These types must have the same width and encode address values in the same manner such that offset values in one language may be passed directly to another language without conversion.

2.5.8 Counts

As described above, MPI defines types (e.g., `MPI_Aint`) to address locations within memory and other types (e.g., `MPI_Offset`) to address locations within files. In addition, some MPI procedures use *count* arguments that represent a number of MPI datatypes on which to operate. Furthermore, *timestamps* in the context of the MPI Tool Information Interface are a count of clock ticks elapsed since some time in the past. At times, one needs a single type that can be used to address locations within either memory or files as well as express *count* values, and that type is `MPI_Count` in C and `INTEGER(KIND=MPI_COUNT_KIND)` in Fortran. These types must have the same width and encode values in the same manner such that count values in one language may be passed directly to another language without conversion. The size of the `MPI_Count` type is determined by the MPI implementation with the restriction that it must be minimally capable of encoding any value that may be stored in a variable of type `int`, `MPI_Aint`, or `MPI_Offset` in C and of type `INTEGER`, `INTEGER(KIND=MPI_ADDRESS_KIND)`, or `INTEGER(KIND=MPI_OFFSET_KIND)` in Fortran. Even

though the `MPI_Count` type is large enough to encode address locations, the `MPI_Count` type shall not be used to represent an *absolute address*.

Rationale. Count values need to be large enough to encode any value used for expressing element counts, strides, offsets, indexes, displacements, typemaps in memory, typemaps in file views, etc. Prior to MPI-4.0, many MPI routines used `int` in C and `INTEGER` in Fortran as the type for *count* arguments. To avoid breaking backward compatibility, this version of the standard continues to support `int` in C as well as `INTEGER` in Fortran in such routines. In addition, this version of the standard supports using `MPI_Count` in C (via separate “_c” suffixed procedures) as well as `INTEGER(KIND=MPI_COUNT_KIND)` in Fortran (via polymorphic interfaces in newer MPI Fortran bindings (USE `mpi_f08`)) in such routines. See Section 21.2 for a full explanation. (*End of rationale.*)

The phrase **large count** refers to the use of `MPI_Count` and `INTEGER(KIND=MPI_COUNT_KIND)` parameter types.

There are cases where `MPI_UNDEFINED` can be returned in a **large count** OUT parameter. Per Table A.1.1 (page 895), the `MPI_UNDEFINED` constant is defined to be a C `int` (or unnamed enum) and a Fortran `INTEGER`. Implementations shall therefore choose the underlying types for `MPI_Count` and `INTEGER(KIND=MPI_COUNT_KIND)` such that they can be compared to `MPI_UNDEFINED`.

Advice to implementors. The comparison of `MPI_UNDEFINED` to an `MPI_Count` or `INTEGER(KIND=MPI_COUNT_KIND)` may need to be via a casting operation. (*End of advice to implementors.*)

2.6 Language Binding

This section defines the rules for MPI language binding in general and for Fortran, and ISO C, in particular. (Note that ANSI C has been replaced by ISO C.) Defined here are various object representations, as well as the naming conventions used for expressing this standard. The actual calling sequences are defined elsewhere.

MPI bindings are for Fortran 90 or later, though they were originally designed to be usable in Fortran 77 environments. With the `mpi_f08` module, two new Fortran features, *assumed type* (i.e., `TYPE(*)`) and *assumed rank* (i.e., `DIMENSION(..)`), are also required, see Section 2.5.5.

Since the word `PARAMETER` is a keyword in the Fortran language, we use the word “argument” to denote the arguments to a subroutine. These are normally referred to as parameters in C, however, we expect that C programmers will understand the word “argument” (which has no specific meaning in C), thus allowing us to avoid unnecessary confusion for Fortran programmers.

Since Fortran is case insensitive, linkers may use either lower case or upper case when resolving Fortran names. Users of case sensitive languages should avoid any prefix of the form “MPI_” and “PMPI_”, where any of the letters are either upper or lower case.

2.6.1 Deprecated and Removed Interfaces

A number of chapters refer to deprecated or replaced MPI constructs. These are constructs that continue to be part of the MPI standard, as documented in Chapter 18, but that users

are recommended not to continue using, since better solutions were provided with newer versions of MPI. For example, the Fortran binding for MPI-1 functions that have address arguments uses `INTEGER`. This is not consistent with the C binding, and causes problems on machines with 32 bit `INTEGER`s and 64 bit addresses. In MPI-2, these functions were given new names with new bindings for the address arguments. The use of the old functions was declared as deprecated. For consistency, here and in a few other cases, new C functions are also provided, even though the new functions are equivalent to the old functions. The old names are deprecated.

Some of the previously deprecated constructs are now removed, as documented in Chapter 19. They may still be provided by an implementation for backwards compatibility, but are not required.

Table 2.1 shows a list of all of the deprecated and removed constructs. Note that some C typedefs and Fortran subroutine names are included in this list; they are the types of callback functions.

2.6.2 Fortran Binding Issues

Originally, MPI-1.1 provided bindings for Fortran 77. These bindings are retained, but they are now interpreted in the context of the Fortran 90 standard. MPI can still be used with most Fortran 77 compilers, as noted below. When the term “Fortran” is used it means Fortran 90 or later; it means Fortran 2008 with TS 29113, which is now an integral part of Fortran 2018 and later if the `mpi_f08` module is used.

All Fortran MPI names have an `MPI_` prefix. Although Fortran is not case sensitive, if the `mpi_f08` module is used, the first character after the `MPI_` prefix is capital and all others are lower case. If the `mpi_f08` module is not used, all characters are capitals. Programs must not declare names, e.g., for variables, subroutines, functions, parameters, derived types, abstract interfaces, or modules, beginning with the prefix `MPI_`. To avoid conflicting with the profiling interface, programs must also avoid subroutines and functions with the prefix `PMPI_`. This is mandated to avoid possible name collisions.

All MPI Fortran subroutines have an error code in the last argument. With `USE mpi_f08`, this last argument is declared as `OPTIONAL`, except for user-defined callback functions (e.g., `COMM_COPY_ATTR_FUNCTION`) and their predefined callbacks (e.g., `MPI_COMM_NULL_COPY_FN`). A few MPI operations that are functions do not have the error code argument. The error code value for successful completion is `MPI_SUCCESS`. Other error codes are implementation dependent; see the error codes in Chapter 9 and Annex A.

Constants representing the maximum length of a string are one smaller in Fortran than in C as discussed in Section 21.3.9.

Handles are represented in Fortran as `INTEGER`s, or as a `BIND(C)` derived type with the `mpi_f08` module; see Section 2.5.1. Binary-valued variables are of type `LOGICAL`.

Array arguments are indexed from one.

The older MPI Fortran bindings—use `mpi` and (deprecated) `mpif.h`—are inconsistent with the Fortran standard in several respects. These inconsistencies, such as register optimization problems, have implications for user codes that are discussed in detail in Section 21.1.16.

The support for large count and displacement in Fortran is only available when using newer MPI Fortran bindings (`USE mpi_f08`). For better readability, all Fortran large count procedure declarations are marked with a comment “`!(_c)`”.

Table 2.1: Deprecated and removed constructs

Deprecated or removed construct	Deprecated since	Removed since	Replacement
<code>MPI_ADDRESS</code>	MPI-2.0	MPI-3.0	<code>MPI_GET_ADDRESS</code>
<code>MPI_TYPE_HINDEXED</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_CREATE_HINDEXED</code>
<code>MPI_TYPE_HVECTOR</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_CREATE_HVECTOR</code>
<code>MPI_TYPE_STRUCT</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_CREATE_STRUCT</code>
<code>MPI_TYPE_EXTENT</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_GET_EXTENT</code>
<code>MPI_TYPE_UB</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_GET_EXTENT</code>
<code>MPI_TYPE_LB</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_GET_EXTENT</code>
<code>MPI_LB¹</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_CREATE_RESIZED</code>
<code>MPI_UB¹</code>	MPI-2.0	MPI-3.0	<code>MPI_TYPE_CREATE_RESIZED</code>
<code>MPI_ERRHANDLER_CREATE</code>	MPI-2.0	MPI-3.0	<code>MPI_COMM_CREATE_ERRHANDLER</code>
<code>MPI_ERRHANDLER_GET</code>	MPI-2.0	MPI-3.0	<code>MPI_COMM_GET_ERRHANDLER</code>
<code>MPI_ERRHANDLER_SET</code>	MPI-2.0	MPI-3.0	<code>MPI_COMM_SET_ERRHANDLER</code>
<code>MPI_Handler_function²</code>	MPI-2.0	MPI-3.0	<code>MPI_Comm_errhandler_function²</code>
<code>MPI_KEYVAL_CREATE</code>	MPI-2.0		<code>MPI_COMM_CREATE_KEYVAL</code>
<code>MPI_KEYVAL_FREE</code>	MPI-2.0		<code>MPI_COMM_FREE_KEYVAL</code>
<code>MPI_DUP_FN³</code>	MPI-2.0		<code>MPI_COMM_DUP_FN³</code>
<code>MPI_NULL_COPY_FN³</code>	MPI-2.0		<code>MPI_COMM_NULL_COPY_FN³</code>
<code>MPI_NULL_DELETE_FN³</code>	MPI-2.0		<code>MPI_COMM_NULL_DELETE_FN³</code>
<code>MPI_Copy_function²</code>	MPI-2.0		<code>MPI_Comm_copy_attr_function²</code>
<code>COPY_FUNCTION²</code>	MPI-2.0		<code>COMM_COPY_ATTR_FUNCTION²</code>
<code>MPI_Delete_function²</code>	MPI-2.0		<code>MPI_Comm_delete_attr_function²</code>
<code>DELETE_FUNCTION²</code>	MPI-2.0		<code>COMM_DELETE_ATTR_FUNCTION²</code>
<code>MPI_ATTR_DELETE</code>	MPI-2.0		<code>MPI_COMM_DELETE_ATTR</code>
<code>MPI_ATTR_GET</code>	MPI-2.0		<code>MPI_COMM_GET_ATTR</code>
<code>MPI_ATTR_PUT</code>	MPI-2.0		<code>MPI_COMM_SET_ATTR</code>
<code>MPI_COMBINER_HVECTOR_INTEGER⁴</code>	-	MPI-3.0	<code>MPI_COMBINER_HVECTOR⁴</code>
<code>MPI_COMBINER_HINDEXED_INTEGER⁴</code>	-	MPI-3.0	<code>MPI_COMBINER_HINDEXED⁴</code>
<code>MPI_COMBINER_STRUCT_INTEGER⁴</code>	-	MPI-3.0	<code>MPI_COMBINER_STRUCT⁴</code>
<code>MPI::...</code>	MPI-2.2	MPI-3.0	C language binding
<code>MPI_CANCEL</code> for send requests	MPI-4.0		no direct replacement
<code>MPI_INFO_GET</code>	MPI-4.0		<code>MPI_INFO_GET_STRING</code>
<code>MPI_INFO_GET_VALUELEN</code>	MPI-4.0		<code>MPI_INFO_GET_STRING</code>
<code>MPI_T_ERR_INVALID_ITEM</code>	MPI-4.0		<code>MPI_T_ERR_INVALID_INDEX</code>
<code>MPI_SIZEOF</code>	MPI-4.0		<code>storage_size()⁵</code> or <code>c_sizeof()</code>
<code>mpif.h</code>	MPI-4.1		<code>mpi</code> module and <code>mpi_f08</code> module
<code>MPI_TYPE_SIZE_X</code>	MPI-4.1		<code>MPI_Type_size_c / !(_c)⁶</code>
<code>MPI_TYPE_GET_EXTENT_X</code>	MPI-4.1		<code>MPI_Type_get_extents_c / !(_c)⁶</code>
<code>MPI_TYPE_GET_TRUE_EXTENT_X</code>	MPI-4.1		<code>MPI_Type_get_true_extents_c / !(_c)⁶</code>
<code>MPI_GET_ELEMENTS_X</code>	MPI-4.1		<code>MPI_Get_elements_c / !(_c)⁶</code>
<code>MPI_STATUS_SET_ELEMENTS_X</code>	MPI-4.1		<code>MPI_Status_set_elements_c / !(_c)⁶</code>
<code>MPI_HOST</code>	MPI-4.1		no direct replacement

¹ Predefined datatype.
² Callback prototype definition.
³ Predefined callback routine.
⁴ Constant.
⁵ Fortran intrinsic. `storage_size()` returns the size in bits instead of bytes; see Section 18.3.
⁶ in C / Fortran with the `mpi_f08` module. No substitute for the `mpi` module and `mpif.h`.
Other entries are regular MPI routines.

2.6.3 C Binding Issues

We use the ISO C declaration format. All MPI names have an `MPI_` prefix, defined constants are in all capital letters, and defined types and functions have one capital letter after the prefix. Programs must not declare names (identifiers), e.g., for variables, functions, constants, types, or macros, beginning with any prefix of the form `MPI_`, where any of the letters are either upper or lower case. To support the profiling interface, programs must not declare functions with names beginning with any prefix of the form `PMPI_`, where any

of the letters are either upper or lower case.

The definition of named constants, function prototypes, and type definitions must be supplied in an include file `mpi.h`.

Almost all C functions return an error code. The successful return value will be `MPI_SUCCESS`, but error codes raised after a failure are implementation dependent.

Type declarations are provided for handles to each category of opaque objects.

Array arguments are indexed from zero.

Logical flags are integers with value 0 meaning “false” and a nonzero value meaning “true.”

Choice arguments are pointers of type `void*`.

2.6.4 Functions and Macros

An implementation is allowed to implement `MPI_AINT_ADD`, `PMPI_AINT_ADD`, `MPI_AINT_DIFF`, and `PMPI_AINT_DIFF`, and no others, as macros in C.

Advice to implementors. Implementors should document which routines are implemented as macros. (*End of advice to implementors.*)

Advice to users. If these routines are implemented as macros, they will not work with the MPI profiling interface. (*End of advice to users.*)

2.7 Processes

An MPI program consists of autonomous processes, executing their own code, in an MIMD style. The codes executed by each process need not be identical. The processes communicate via calls to MPI communication primitives. Typically, each process executes in its own address space, although shared-memory implementations of MPI are possible.

This document specifies the behavior of a parallel program assuming that only MPI calls are used. The interaction of an MPI program with other possible means of communication, I/O, and process management is not specified. Unless otherwise stated in the specification of the standard, MPI places no requirements on the result of its interaction with external mechanisms that provide similar or equivalent functionality. This includes, but is not limited to, interactions with external mechanisms for process control, shared and remote memory access, file system access and control, interprocess communication, process signaling, and terminal I/O. High quality implementations should strive to make the results of such interactions intuitive to users, and attempt to document restrictions where deemed necessary.

Advice to implementors. Implementations that support such additional mechanisms for functionality supported within MPI are expected to document how these interact with MPI. (*End of advice to implementors.*)

The interaction of MPI and threads is defined in Section 11.6.

MPI processes reside in the same **shared memory domain** if it is possible to share a segment of memory between them, i.e., to make a segment of memory (**shared memory segment**) concurrently accessible from all of those MPI processes through load/store accesses. For a group of processes belonging to more than one *shared memory domain* the creation of a subgroup of processes belonging to the same *shared memory domain* is defined in Section 7.4.2.

2.8 Error Handling

MPI provides the user with reliable message transmission. A message sent is always received correctly, and the user does not need to check for transmission errors, time-outs, or other error conditions. In other words, MPI does not provide mechanisms for dealing with **transmission failures** in the communication system. If the MPI implementation is built on an unreliable underlying mechanism, then it is the job of the implementor of the MPI subsystem to insulate the user from this unreliability, and to reflect only unrecoverable transmission failures. Whenever possible, such failures will be reflected as errors in the relevant communication call.

In contrast, MPI defines mechanisms for handling **MPI process failures**, that is, when an MPI process unexpectedly and permanently stops communicating (e.g., a software or hardware crash results in an MPI process terminating unexpectedly). When an MPI process failure happens, the MPI implementation may raise one of the **fault tolerance error** classes as defined in chapter 17 to indicate that the MPI implementation remains in a defined state and can continue to operate as described in that chapter. The support for MPI process fault tolerance is optional and the implementation may raise a different error class if fault tolerance is not available.

Of course, MPI programs may still be erroneous. A **program error** can occur when an MPI call is made with an incorrect argument (nonexisting destination in a send operation, buffer too small in a receive operation, etc.). This type of error would occur in any implementation. In addition, a **resource error** may occur when a program exceeds the amount of available system resources (number of pending messages, system buffers, etc.). The occurrence of this type of error depends on the amount of available resources in the system and the resource allocation mechanism used; this may differ from system to system. A high-quality implementation will provide generous limits on the important resources so as to alleviate the portability problem this represents.

In C and Fortran, almost all MPI calls return a code that indicates successful completion of the operation. Whenever possible, MPI calls return an error code if an error occurred during the call. By default, an error detected during the execution of the MPI library causes the parallel computation to abort, except for file operations. However, MPI provides mechanisms for users to change this default and to handle recoverable errors. The user may specify that no error is fatal, and handle error codes returned by MPI calls by themselves. Also, the user may provide user-defined error-handling routines, which will be invoked whenever an MPI call returns abnormally. The MPI error handling facilities are described in Section 9.3.

Several factors limit the ability of MPI calls to return with meaningful error codes when an error occurs. MPI may not be able to detect some errors; other errors may be too expensive to detect in normal execution mode; some faults (e.g., memory faults) may corrupt the state of the MPI library and its outputs; finally some errors may be “catastrophic” and may prevent MPI from returning control to the caller.

In addition, some errors may be detected in operations that do not refer to an MPI object from which the associated error handler can be obtained. Error handler associations are further described in Section 9.3. In such cases, these errors will be raised on the communicator `MPI_COMM_SELF` when using the World Model (see Section 11.2). When `MPI_COMM_SELF` is not initialized (i.e., before `MPI_INIT` / `MPI_INIT_THREAD`, after `MPI_FINALIZE`, or when using the Sessions Model exclusively) the error raises the **initial error handler** (set during the launch operation, see Section 11.8.4). The Sessions Model

is described in Section 11.3.

Lastly, some errors may be detected after the associated operation has completed locally. An example of such a case arises because of the nature of asynchronous communications: MPI calls may initiate operations that continue asynchronously after the call returned. Thus, the operation may return with a code indicating successful completion, yet later cause an error to be raised. If there is a subsequent call that relates to the same operation (e.g., a call that verifies that an asynchronous operation has completed) then the error argument associated with this call will be used to indicate the nature of the error. In a few cases, the error may occur after all calls that relate to the operation have returned, so that no error value can be used to indicate the nature of the error (e.g., an erroneous program on the receiver in a send with the ready mode).

This document does not specify the state of a computation after an erroneous MPI call has occurred. The desired behavior is that a relevant error code be returned, and the effect of the error be localized to the greatest possible extent. E.g., it is highly desirable that an erroneous receive call will not cause any part of the receiver's memory to be overwritten, beyond the area specified for receiving the message.

Implementations may go beyond this document in supporting in a meaningful manner MPI calls that are defined here to be erroneous. For example, MPI specifies strict type matching rules between matching send and receive operations: it is erroneous to send a floating point variable and receive an integer. Implementations may go beyond these type matching rules, and provide automatic type conversion in such situations. It will be helpful to generate warnings for such nonconforming behavior.

MPI defines a way for users to create new error codes as defined in Section 9.5.

2.9 Progress

MPI communication operations or parallel I/O patterns typically comprise several related operations executed in one or multiple MPI processes. Examples are the point-to-point communications with one MPI process executing a send operation and another (or the same) MPI process executing a receive operation, or all MPI processes of a group executing a collective operation.

Within each MPI process parts of the communication or parallel I/O pattern are executed within the MPI procedure calls that belong to the operation in that MPI process, whereas other parts are **decoupled MPI activities**, i.e., they may be executed within an additional progress thread, offloaded to the network interface controller (NIC), or executed within other MPI procedure calls that are not semantically related to the given communication or parallel I/O pattern.

An MPI procedure invocation is **blocked** if it delays its return until some specific activity or state-change has occurred in another MPI process. An MPI procedure call that is *blocked* can be

- a *nonlocal* MPI procedure call that delays its return until a specific semantically-related MPI call on another MPI process, or
- a *local* MPI procedure call that delays its return until some unspecific MPI call in another MPI process causes a specific state-change in that other MPI process, or
- an MPI finalization procedure (`MPI_FINALIZE` or `MPI_SESSION_FINALIZE`) that delays its return or exit because this MPI finalization must guarantee that all decoupled

MPI activities that are related to that MPI finalization call in the calling MPI process will be executed before this MPI finalization is finished. Note that an MPI finalization procedure may execute attribute deletion callback functions prior to the finalization (see Section 11.2.4); these callback functions may generate additional decoupled MPI activities.

Some examples of a *nonlocal* blocked MPI procedure call:

- `MPI_SSEND` delays its return until the matching receive operation is *started* at the destination MPI process (for example, by a call to `MPI_RECV` or to `MPI_Irecv`).
- `MPI_RECV` delays its return until the matching send operation is *started* at the source MPI process (for example, by a call to `MPI_SEND` or to `MPI_Isend`).

Some examples of a *local* blocked MPI procedure call:

- `MPI_RSEND`, if the message data cannot be entirely buffered, delays its return until the destination MPI process has received the portion of message data that cannot be buffered, which may require one or more unspecific MPI procedure call(s) at the destination MPI process.
- `MPI_RECV`, in case the message was buffered at the sending MPI process (e.g. with `MPI_BSEND`), delays its return until the message is received, which may require one or more unspecific MPI procedure calls at the sending MPI process to send the buffered data.

All MPI processes are required to **guarantee progress**, i.e., all decoupled MPI activities will eventually be executed. This guarantee is required to be provided during

- blocked MPI procedures, and
- repeatedly called MPI test procedures (see below) that return `flag=false`.

The *progress* must be provided independently of whether a decoupled MPI activity belongs to a specific session or to the World Model (see Sections 11.2 and 11.3). Other ways of fulfilling this guarantee are possible and permitted (for example, a dedicated progress thread or off-loading to a network interface controller (NIC)).

MPI test procedures are `MPI_TEST`, `MPI_TESTANY`, `MPI_TESTALL`, `MPI_TESTSOME`, `MPI_Iprobe`, `MPI_Improbe`, `MPI_REQUEST_GET_STATUS`, `MPI_WIN_TEST`, and `MPI_PRRIVED`.

Strong progress is provided by an MPI implementation if all *local* procedures return independently of MPI procedure calls in other MPI processes (operation-related or not). An MPI implementation provides **weak progress** if it does not provide *strong progress*.

Advice to users. The type of *progress* may influence the performance of MPI operations. A correct MPI application must be written under the assumption that only *weak progress* is provided. Every MPI application that is correct under *weak progress* will be correctly executed if *strong progress* is provided. In addition, the MPI standard is designed such that correctness under the assumption of *strong progress* should imply also correctness if only *weak progress* is provided by the implementation. (*End of advice to users.*)

Rationale. MPI does not guarantee progress when using synchronization methods that are not based on MPI procedures. Without guaranteed *strong progress* in MPI this may lead to a *deadlock*, see for example Section 2.7 and Example 12.13 in Section 12.7.3. (*End of rationale.*)

For further rules, see in Section 2.4.2 the definition of *local* MPI procedures, and all references to *progress* in the [general index](#).

2.10 Implementation Issues

There are a number of areas where an MPI implementation may interact with the operating environment and system. While MPI does not mandate that any services (such as signal handling) be provided, it does strongly suggest the behavior to be provided if those services are available. This is an important point in achieving portability across platforms that provide the same set of services.

2.10.1 Independence of Basic Runtime Routines

MPI programs require that library routines that are part of the basic language environment (such as `write` in Fortran and `printf` and `malloc` in ISO C) and are executed after `MPI_INIT` and before `MPI_FINALIZE` operate independently and that their *completion* is independent of the action of other processes in an MPI program.

Note that this in no way prevents the creation of library routines that provide parallel services whose operation is collective. However, the following program is expected to complete in an ISO C environment regardless of the size of `MPI_COMM_WORLD` (assuming that `printf` is available at the executing MPI processes).

```
int commworld_rank;
MPI_Init((void *)0, (void *)0);
MPI_Comm_rank(MPI_COMM_WORLD, &commworld_rank);
if (commworld_rank == 0) printf("Starting program\n");
MPI_Finalize();
```

The corresponding Fortran programs are also expected to complete.

An example of what is *not* required is any particular ordering of the action of these routines when called by several MPI processes. For example, MPI makes neither requirements nor recommendations for the output from the following program (again assuming that I/O is available at the executing MPI processes).

```
MPI_Comm_rank(MPI_COMM_WORLD, &commworld_rank);
printf("Output from MPI process where commworld_rank=%d\n",
      commworld_rank);
```

In addition, calls that fail because of resource exhaustion or other error are not considered a violation of the requirements here (however, they are required to complete, just not to complete successfully).

2.10.2 Interaction with Signals

MPI does not specify the interaction of processes with signals and does not require that MPI be signal safe. The implementation may reserve some signals for its own use. It is required

that the implementation document which signals it uses, and it is strongly recommended that it not use SIGALRM, SIGFPE, or SIGIO. Implementations may also prohibit the use of MPI calls from within signal handlers.

In multithreaded environments, users can avoid conflicts between signals and the MPI library by catching signals only on threads that do not execute MPI calls. High quality single-threaded implementations will be signal safe: an MPI call suspended by a signal will resume and complete normally after the signal is handled.

2.11 Examples

The examples in this document are for illustration purposes only. They are not intended to specify the standard. Many of the examples have been compiled by tools that extract the examples from the source files for the MPI standard. However, the examples have not been carefully checked or verified.

Chapter 3

Point-to-Point Communication

3.1 Introduction

Sending and receiving of *messages* by MPI processes is the basic MPI communication mechanism. The basic point-to-point communication operations are *send* and *receive*. Their use is illustrated in Example 3.1.

Example 3.1. A simple ‘hello world’ example usage of point-to-point communication.

```
#include "mpi.h"
int main(int argc, char *argv[])
{
    char message[20];
    int myrank;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
    if (myrank == 0) /* code for process zero */
    {
        strcpy(message, "Hello, there");
        MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99,
                  MPI_COMM_WORLD);
    }
    else if (myrank == 1) /* code for process one */
    {
        MPI_Recv(message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
        printf("received :%s:\n", message);
    }
    MPI_Finalize();
    return 0;
}
```

In Example 3.1, process zero (`myrank = 0`, strictly ‘the MPI process with rank 0 in communicator `MPI_COMM_WORLD`’) sends a *message* to process one using the *send* operation `MPI_SEND`. The operation specifies a *send buffer* in the sender memory from which the *message data* is taken. In the example above, the send buffer consists of the storage containing the variable `message` in the memory of process zero. The location, size and type of the send buffer are specified by the first three parameters of the send operation. The message sent will contain the 13 characters of this variable. In addition, the send operation associates an *envelope* with the message. This *envelope* specifies the message destination and contains distinguishing information that can be used by the *receive* operation to select a particular message. The last three parameters of the send operation,

along with the rank of the sender, specify the *envelope* for the message sent.

Process one (`myrank = 1`, strictly ‘the MPI process with rank 1 in communicator `MPI_COMM_WORLD`’) receives this message with the *receive* operation `MPI_RECV`. The message to be received is selected according to the value of its *envelope*, and the *message data* is stored into the *receive buffer*. In the example above, the receive buffer consists of the storage containing the string `message` in the memory of process one. The first three parameters of the receive operation specify the location, size and type of the receive buffer. The next three parameters are used for selecting the incoming message. The last parameter is used to return information on the message just received.

Advice to users. Colloquial usage commonly permits references to “rank 0” or “process 0”, which are strictly ambiguous and ideally should be qualified by including the relevant context, for example, the MPI communicator in the case above. (*End of advice to users.*)

The next sections describe the blocking send and receive operations. We discuss send, receive, blocking communication semantics, type matching requirements, type conversion in heterogeneous environments, and more general communication modes. Nonblocking communication is addressed next, followed by probing and cancelling a message, channel-like constructs and send-receive operations, ending with a description of the “dummy” MPI process, `MPI_PROC_NULL`.

3.2 Blocking Send and Receive Operations

3.2.1 Blocking Send

The syntax of the **blocking send** procedure is given below.

`MPI_SEND(buf, count, datatype, dest, tag, comm)`

IN	<code>buf</code>	initial address of send buffer (choice)
IN	<code>count</code>	number of elements in send buffer (nonnegative integer)
IN	<code>datatype</code>	datatype of each send buffer element (handle)
IN	<code>dest</code>	rank of destination (integer)
IN	<code>tag</code>	message tag (integer)
IN	<code>comm</code>	communicator (handle)

C binding

```
int MPI_Send(const void *buf, int count, MPI_Datatype datatype, int dest,
             int tag, MPI_Comm comm)
```

```
int MPI_Send_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
               int dest, int tag, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Send(buf, count, datatype, dest, tag, comm, ierror)
      TYPE(*), DIMENSION(..), INTENT(IN) :: buf
```

Table 3.1: Predefined MPI datatypes corresponding to Fortran datatypes

MPI datatype	Fortran datatype
MPI_INTEGER	INTEGER
MPI_REAL	REAL
MPI_DOUBLE_PRECISION	DOUBLE PRECISION
MPI_COMPLEX	COMPLEX
MPI_LOGICAL	LOGICAL
MPI_CHARACTER	CHARACTER(1)
MPI_BYTE	
MPI_PACKED	

```

INTEGER, INTENT(IN) :: count, dest, tag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Send(buf, count, datatype, dest, tag, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: dest, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
<type> BUF(*)
INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR

```

The blocking semantics of this call are described in Section 3.4.

3.2.2 Message Data

The send buffer specified by the `MPI_SEND` procedure consists of `count` successive entries of the type indicated by `datatype`, starting with the entry at address `buf`. Note that we specify the message length in terms of number of *elements*, not number of *bytes*. The former is machine independent and closer to the application level.

The data part of the message consists of a sequence of `count` values, each of the type indicated by `datatype`. `count` may be zero, in which case the data part of the message is empty. The **basic datatypes** that can be specified for message data values correspond to the basic datatypes of the host language. Possible values of this argument for Fortran and the corresponding Fortran types are listed in Table 3.1. Possible values for this argument for C and the corresponding C types are listed in Table 3.2.

The datatypes `MPI_BYTE` and `MPI_PACKED` do not correspond to a Fortran or C datatype. A value of type `MPI_BYTE` consists of a byte (8 binary digits). A byte is uninterpreted and is different from a character. Different machines may have different representations for characters, or may use more than one byte to represent characters. On

Table 3.2: Predefined MPI datatypes corresponding to C datatypes

MPI datatype	C datatype
MPI_CHAR	char (treated as printable character)
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG_INT	signed long long int
MPI_LONG_LONG (as a synonym)	signed long long int
MPI_SIGNED_CHAR	signed char (treated as integral value)
MPI_UNSIGNED_CHAR	unsigned char (treated as integral value)
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED_LONG_LONG	unsigned long long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_WCHAR	wchar_t (defined in <stddef.h>) (treated as printable character)
MPI_C_BOOL	_Bool
MPI_INT8_T	int8_t
MPI_INT16_T	int16_t
MPI_INT32_T	int32_t
MPI_INT64_T	int64_t
MPI_UINT8_T	uint8_t
MPI_UINT16_T	uint16_t
MPI_UINT32_T	uint32_t
MPI_UINT64_T	uint64_t
MPI_C_COMPLEX	float _Complex
MPI_C_FLOAT_COMPLEX (as a synonym)	float _Complex
MPI_C_DOUBLE_COMPLEX	double _Complex
MPI_C_LONG_DOUBLE_COMPLEX	long double _Complex
MPI_BYTE	
MPI_PACKED	

the other hand, a byte has the same binary value on all machines. The use of the type `MPI_PACKED` is explained in Section 5.2.

MPI requires support of these datatypes, which match the basic datatypes of Fortran and ISO C. Additional MPI datatypes should be provided if the host language has additional datatypes¹: `MPI_DOUBLE_COMPLEX` for double precision complex in Fortran declared to

¹These types, such as `DOUBLE COMPLEX` and `INTEGER*4`, are not specified by any Fortran standard but are

Table 3.3: Predefined MPI datatypes corresponding to both C and Fortran datatypes

MPI datatype	C datatype	Fortran datatype
MPI_AINT	MPI_Aint	INTEGER(KIND=MPI_ADDRESS_KIND)
MPI_OFFSET	MPI_Offset	INTEGER(KIND=MPI_OFFSET_KIND)
MPI_COUNT	MPI_Count	INTEGER(KIND=MPI_COUNT_KIND)

Table 3.4: Predefined MPI datatypes corresponding to C++ datatypes

MPI datatype	C++ datatype
MPI_CXX_BOOL	bool
MPI_CXX_FLOAT_COMPLEX	std::complex<float>
MPI_CXX_DOUBLE_COMPLEX	std::complex<double>
MPI_CXX_LONG_DOUBLE_COMPLEX	std::complex<long double>

be of type `DOUBLE COMPLEX`; `MPI_REAL2`, `MPI_REAL4`, `MPI_REAL8`, and `MPI_REAL16` for Fortran reals, declared to be of type `REAL*2`, `REAL*4`, `REAL*8`, and `REAL*16`, respectively; `MPI_INTEGER1`, `MPI_INTEGER2`, `MPI_INTEGER4`, `MPI_INTEGER8`, and `MPI_INTEGER16` for Fortran integers, declared to be of type `INTEGER*1`, `INTEGER*2`, `INTEGER*4`, `INTEGER*8`, and `INTEGER*16` respectively; `MPI_LOGICAL1`, `MPI_LOGICAL2`, `MPI_LOGICAL4`, `MPI_LOGICAL8`, and `MPI_LOGICAL16` for Fortran integers, declared to be of type `LOGICAL*1`, `LOGICAL*2`, `LOGICAL*4`, `LOGICAL*8`, and `LOGICAL*16` respectively; `MPI_COMPLEX4`, `MPI_COMPLEX8`, `MPI_COMPLEX16`, and `MPI_COMPLEX32` for complex numbers in Fortran declared to be of type `COMPLEX*4`, `COMPLEX*8`, `COMPLEX*16`, and `COMPLEX*32`, respectively; etc.

Rationale. One goal of the design is to allow for MPI to be implemented as a library, with no need for additional preprocessing or compilation. Thus, one cannot assume that a communication call has information on the datatype of variables in the communication buffer; this information must be supplied by an explicit argument. The need for such datatype information will become clear in Section 3.3.2. (*End of rationale.*)

The datatypes `MPI_AINT`, `MPI_OFFSET`, and `MPI_COUNT` correspond to the MPI-defined C types `MPI_Aint`, `MPI_Offset`, and `MPI_Count` and their Fortran equivalents `INTEGER(KIND=MPI_ADDRESS_KIND)`, `INTEGER(KIND=MPI_OFFSET_KIND)`, and `INTEGER(KIND=MPI_COUNT_KIND)`. This is described in Table 3.3. All predefined datatype handles are available in all language bindings. See Sections 21.3.6 and 21.3.10 on page 864 and 871 for information on interlanguage communication with these types.

If there is an accompanying C++ compiler then the datatypes in Table 3.4 are also supported in C and Fortran.

3.2.3 Message Envelope

In addition to the data part, messages carry information that can be used to distinguish messages and selectively receive them. This information consists of a fixed number of fields, which we collectively call the **message envelope**. These fields are

extensions commonly accepted by Fortran compilers.

source
destination
tag
communicator

The *message source* is implicitly determined by the identity of the message sender. The other fields are specified by arguments in the send procedure.

The *message destination* is specified by the **dest** argument.

The integer-valued *message tag* is specified by the **tag** argument. This integer can be used by the program to distinguish different types of messages. The range of valid tag values is $0, \dots, \text{UB}$, where the value of **UB** is implementation dependent. It can be found by querying the value of the attribute **MPI_TAG_UB**, as described in Chapter 9. MPI requires that **UB** be no less than 32767.

The **comm** argument specifies the *communicator* that is used for the send operation. Communicators are explained in Chapter 7; below is a brief summary of their usage.

A communicator specifies the communication context for a communication operation. Each communication context provides a separate “communication universe”: messages are always received within the context they were sent, and messages sent in different contexts do not interfere.

The communicator also specifies the group of MPI processes that share this communication context. This MPI *process group* is ordered and MPI processes are identified by their rank within this group. Thus, the range of valid values for **dest** is $0, \dots, n - 1 \cup \{\text{MPI_PROC_NULL}\}$, where n is the number of MPI processes in the group. (If the communicator is an inter-communicator, then destinations are identified by their rank in the remote group. See Chapter 7.)

An MPI process may have a different rank in each group in which it is a member.

When using the World Model (see Section 11.2), a predefined communicator **MPI_COMM_WORLD** is provided by MPI. It allows communication with all MPI processes that are accessible after MPI initialization and MPI processes are identified by their rank in the group of **MPI_COMM_WORLD**.

Advice to users. Users that are comfortable with the notion of a flat name space for MPI processes, and a single communication context, as offered by most existing communication libraries, need only use the World Model for MPI initialization, and the predefined variable **MPI_COMM_WORLD** as the **comm** argument. This will allow communication with all the MPI processes available at initialization time.

Users may define new communicators, as explained in Chapter 7. Communicators provide an important encapsulation mechanism for libraries and modules. They allow modules to have their own disjoint communication universe and their own MPI process numbering scheme. (*End of advice to users.*)

Advice to implementors. The *message envelope* would normally be encoded by a fixed-length message header. However, the actual encoding is implementation dependent. Some of the information (e.g., source or destination) may be implicit, and need not be explicitly carried by messages. Also, MPI processes may be identified by relative ranks, or absolute ids, etc. (*End of advice to implementors.*)

3.2.4 Blocking Receive

The syntax of the **blocking receive** procedure is given below.

MPI_RECV(buf, count, datatype, source, tag, comm, status)			1
OUT	buf	initial address of receive buffer (choice)	2
IN	count	number of elements in receive buffer (nonnegative integer)	3
IN	datatype	datatype of each receive buffer element (handle)	4
IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)	5
IN	tag	message tag or <code>MPI_ANY_TAG</code> (integer)	6
IN	comm	communicator (handle)	7
OUT	status	status object (status)	8

C binding

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag,
             MPI_Comm comm, MPI_Status *status)
```

```
int MPI_Recv_c(void *buf, MPI_Count count, MPI_Datatype datatype, int source,
               int tag, MPI_Comm comm, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Recv(buf, count, datatype, source, tag, comm, status, ierror)
```

```
TYPE(*), DIMENSION(..) :: buf
INTEGER, INTENT(IN) :: count, source, tag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Recv(buf, count, datatype, source, tag, comm, status, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..) :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: source, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_RECV(BUF, COUNT, DATATYPE, SOURCE, TAG, COMM, STATUS, IERROR)
```

```
<type> BUF(*)
INTEGER COUNT, DATATYPE, SOURCE, TAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR
```

The blocking semantics of this call are described in Section 3.4.

The receive buffer consists of the storage containing `count` consecutive elements of the type specified by `datatype`, starting at address `buf`. The length of the received message must be less than or equal to the length of the receive buffer. An overflow error occurs if all incoming data does not fit, without truncation, into the receive buffer.

If a message that is shorter than the receive buffer arrives, then only those locations corresponding to the (shorter) message are modified.

Advice to users. The `MPI_PROBE` function described in Section 3.8 can be used to receive messages of unknown length. (*End of advice to users.*)

Advice to implementors. Even though no specific behavior is mandated by MPI for *erroneous programs*, the recommended handling of overflow situations is to return in **status** information about the source and tag of the incoming message. The receive procedure will return an error code. High-quality implementations will also ensure that no memory that is outside the receive buffer will ever be overwritten.

In the case of a message shorter than the receive buffer, MPI is quite strict in that it allows no modification of the other locations. A more lenient statement would allow for some optimizations but this is not allowed. The implementation must be ready to end a copy into the receiver memory exactly at the end of the receive buffer, even if it is an odd address. (*End of advice to implementors.*)

The selection of a message by a receive operation is governed by the value of the *message envelope*. A message can be received by a receive operation if its *envelope* matches the **source**, **tag** and **comm** values specified by the receive operation. The receiver may specify a **wildcard** `MPI_ANY_SOURCE` value for **source**, and/or a wildcard `MPI_ANY_TAG` value for **tag**, indicating that any source and/or tag are acceptable. It cannot specify a wildcard value for **comm**. Thus, a message can be received by a receive operation only if it is addressed to the receiving MPI process, has a matching communicator, has matching source unless **source** = `MPI_ANY_SOURCE` in the pattern, and has a matching tag unless **tag** = `MPI_ANY_TAG` in the pattern.

The message tag is specified by the **tag** argument of the receive operation. The argument **source**, if different from `MPI_ANY_SOURCE`, is specified as a rank within the MPI process group associated with that same communicator (remote MPI process group, for inter-communicators). Thus, the range of valid values for the **source** argument is $\{0, \dots, n-1\} \cup \{\text{MPI_ANY_SOURCE}\} \cup \{\text{MPI_PROC_NULL}\}$, where n is the number of MPI processes in this group.

Note the asymmetry between send and receive operations: A receive operation may accept messages from an arbitrary sender, on the other hand, a send operation must specify a unique receiver. This matches a “push” communication mechanism, where data transfer is effected by the sender (rather than a “pull” mechanism, where data transfer is effected by the receiver).

Source = destination is allowed, that is, an MPI process can send a message to itself. However, it is unsafe to do so with the blocking send and receive operations described above, since this may lead to deadlock. See Section 3.5.

Advice to implementors. Message context and other communicator information can be implemented as an additional tag field. It differs from the regular message tag in that wild card matching is not allowed on this field, and that value setting for this field is controlled by communicator manipulation functions. (*End of advice to implementors.*)

The use of **dest** = `MPI_PROC_NULL` or **source** = `MPI_PROC_NULL` to define a “dummy” destination or source in any send or receive call is described in Section 3.10.

3.2.5 Return Status

The source or tag of a received message may not be known if wildcard values were used in the receive operation. Also, if multiple requests are completed by a single MPI function

(see Section 3.7.5), a distinct error code may need to be returned for each request. The information is returned by the `status` argument of `MPI_RECV`. The type of `status` is MPI-defined. Status variables need to be explicitly allocated by the user, that is, they are not system objects.

In C, `status` is a structure that contains three fields named `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR`; the structure may contain additional fields. Thus, `status.MPI_SOURCE`, `status.MPI_TAG`, and `status.MPI_ERROR` contain the source, tag, and error code, respectively, of the received message.

In Fortran with `USE mpi` or (deprecated) `INCLUDE 'mpif.h'`, `status` is an array of `INTEGER`s of size `MPI_STATUS_SIZE`. The constants `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR` are the indices of the entries that store the source, tag, and error fields. Thus, `status(MPI_SOURCE)`, `status(MPI_TAG)`, and `status(MPI_ERROR)` contain, respectively, the source, tag, and error code of the received message.

With Fortran `USE mpi_f08`, `status` is defined as the Fortran `BIND(C)` derived type `TYPE(MPI_Status)` containing three public `INTEGER` fields named `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR`. `TYPE(MPI_Status)` may contain additional, implementation-specific fields. Thus, `status%MPI_SOURCE`, `status%MPI_TAG`, and `status%MPI_ERROR` contain the source, tag, and error code of a received message respectively. Additionally, within both the `mpi` and the `mpi_f08` modules, the constants `MPI_STATUS_SIZE`, `MPI_SOURCE`, `MPI_TAG`, `MPI_ERROR`, and `TYPE(MPI_Status)` are defined to allow conversion between both status representations. Conversion routines are provided in Section 21.3.5.

Rationale. The Fortran `TYPE(MPI_Status)` is defined as a `BIND(C)` derived type so that it can be used at any location where the status integer array representation can be used, e.g., in user defined common blocks. (*End of rationale.*)

Rationale. It is allowed to have the same name (e.g., `MPI_SOURCE`) defined as a constant (e.g., Fortran parameter) and as a field of a derived type. (*End of rationale.*)

In general, message-passing calls do not modify the value of the error code field of status variables. This field may be updated only by the functions in Section 3.7.5 that return multiple statuses. The field is updated if and only if such function returns with an error code of `MPI_ERR_IN_STATUS`.

Rationale. The error field in status is not needed for calls that return only one status, such as `MPI_WAIT`, since that would only duplicate the information returned by the function itself. The current design avoids the additional overhead of setting it, in such cases. The field is needed for calls that return multiple statuses, since each request may have had a different failure. (*End of rationale.*)

The status argument also returns information on the length of the message received. However, this information is not directly available as a field of the status variable and a call to `MPI_GET_COUNT` is required to “decode” this information.

```
1 MPI_GET_COUNT(status, datatype, count)
```

```
2     IN          status          return status of receive operation (status)
3
4     IN          datatype        datatype of each receive buffer entry (handle)
5
6     OUT         count          number of received entries (integer)
```

7 C binding

```
8 int MPI_Get_count(const MPI_Status *status, MPI_Datatype datatype, int *count)
9
10 int MPI_Get_count_c(const MPI_Status *status, MPI_Datatype datatype,
11                    MPI_Count *count)
```

12 Fortran 2008 binding

```
13 MPI_Get_count(status, datatype, count, ierror)
14     TYPE(MPI_Status), INTENT(IN) :: status
15     TYPE(MPI_Datatype), INTENT(IN) :: datatype
16     INTEGER, INTENT(OUT) :: count
17     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
18
19 MPI_Get_count(status, datatype, count, ierror) !(_c)
20     TYPE(MPI_Status), INTENT(IN) :: status
21     TYPE(MPI_Datatype), INTENT(IN) :: datatype
22     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: count
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

24 Fortran binding

```
25 MPI_GET_COUNT(STATUS, DATATYPE, COUNT, IERROR)
26     INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, COUNT, IERROR
```

27 Returns the number of entries received. (Again, we count *entries*, each of type **datatype**,
28 not *bytes*.) The **datatype** argument should match the argument provided by the receive call
29 that set the **status** variable. If the number of entries received exceeds the limits of the **count**
30 parameter, then **MPI_GET_COUNT** sets the value of **count** to **MPI_UNDEFINED**. There are
31 other situations where the value of **count** can be set to **MPI_UNDEFINED**; see Section 5.1.11.

33 *Rationale.* Some message-passing libraries use INOUT **count**, **tag** and **source** arguments,
34 thus using them both to specify the selection criteria for incoming messages and return
35 the actual *envelope* values of the received message. The use of a separate status ar-
36 gument prevents errors that are often attached with INOUT argument (e.g., using the
37 **MPI_ANY_TAG** constant as the tag in a receive). Some libraries use calls that refer
38 implicitly to the “last message received.” This is not thread safe.

39 The **datatype** argument is passed to **MPI_GET_COUNT** so as to improve performance.
40 A message might be received without counting the number of elements it contains,
41 and the count value is often not needed. Also, this allows the same function to be
42 used after a call to **MPI_PROBE** or **MPI_IProbe**. With a status from **MPI_PROBE**
43 or **MPI_IProbe**, the same datatypes are allowed as in a call to **MPI_RECV** to receive
44 this message. (*End of rationale.*)

46 The value returned as the **count** argument of **MPI_GET_COUNT** for a datatype of
47 length zero where zero bytes have been transferred is zero. If the number of bytes transferred
48 is greater than zero, **MPI_UNDEFINED** is returned.

Rationale. Zero-length datatypes may be created in a number of cases. An important case is `MPI_TYPE_CREATE_DARRAY`, where the definition of the particular darray results in an empty block on some MPI process. Programs written in an SPMD style will not check for this special case and may want to use `MPI_GET_COUNT` to check the status. (*End of rationale.*)

Advice to users. The buffer size required for the receive can be affected by data conversions and by the stride of the receive datatype. In most cases, the safest approach is to use the same datatype with `MPI_GET_COUNT` and the receive. (*End of advice to users.*)

All send and receive operations use the `buf`, `count`, `datatype`, `source`, `dest`, `tag`, `comm`, and `status` arguments in the same way as the blocking `MPI_SEND` and `MPI_RECV` procedures described in this section.

While the `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR` status values are directly accessible by the user, for convenience in some contexts, users can also access them via procedure calls, as described below.

`MPI_STATUS_GET_SOURCE(status, source)`

IN	status	status from which to retrieve source rank (status)
OUT	source	rank set in the <code>MPI_SOURCE</code> field (integer)

C binding

```
int MPI_Status_get_source(const MPI_Status *status, int *source)
```

Fortran 2008 binding

```
MPI_Status_get_source(status, source, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
  INTEGER, INTENT(OUT) :: source
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_STATUS_GET_SOURCE(STATUS, SOURCE, IERROR)
  INTEGER STATUS(MPI_STATUS_SIZE), SOURCE, IERROR
```

Returns in `source` the value of the `MPI_SOURCE` field in the `status` object.

`MPI_STATUS_GET_TAG(status, tag)`

IN	status	status from which to retrieve tag (status)
OUT	tag	tag set in the <code>MPI_TAG</code> field (integer)

C binding

```
int MPI_Status_get_tag(const MPI_Status *status, int *tag)
```

Fortran 2008 binding

```
MPI_Status_get_tag(status, tag, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
```

```

1      INTEGER, INTENT(OUT) :: tag
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_STATUS_GET_TAG(STATUS, TAG, IERROR)
5      INTEGER STATUS(MPI_STATUS_SIZE), TAG, IERROR

```

Returns in tag the value in the [MPI_TAG](#) field of the status object.

```

10     MPI_STATUS_GET_ERROR(status, err)

```

11	IN	status	status from which to retrieve error (status)
12	OUT	err	error set in the MPI_ERROR field (integer)

C binding

```

15     int MPI_Status_get_error(const MPI_Status *status, int *err)

```

Fortran 2008 binding

```

18     MPI_Status_get_error(status, err, ierror)
19     TYPE(MPI_Status), INTENT(IN) :: status
20     INTEGER, INTENT(OUT) :: err
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

23     MPI_STATUS_GET_ERROR(STATUS, ERR, IERROR)
24     INTEGER STATUS(MPI_STATUS_SIZE), ERR, IERROR

```

Returns in err the value in the [MPI_ERROR](#) field of the status object.

Procedures for setting these fields in a status object are defined in Section 14.3.

3.2.6 Passing [MPI_STATUS_IGNORE](#) for Status

Every call to [MPI_RECV](#) includes a **status** argument, wherein the system can return details about the message received. There are also a number of other MPI calls where **status** is returned. An object of type `MPI_Status` is not an MPI opaque object; its structure is declared in `mpi.h` and (deprecated) `mpif.h`, and it exists in the user's program. In many cases, application programs are constructed so that it is unnecessary for them to examine the **status** fields. In these cases, it is a waste for the user to allocate a status object, and it is particularly wasteful for the MPI implementation to fill in fields in this object.

To cope with this problem, there are two predefined constants, [MPI_STATUS_IGNORE](#) and [MPI_STATUSES_IGNORE](#), which when passed to a receive, probe, wait, or test function, inform the implementation that the status fields are not to be filled in. Note that [MPI_STATUS_IGNORE](#) is not a special type of `MPI_Status` object; rather, it is a special value for the argument. In C one would expect it to be `NULL`, not the address of a special `MPI_Status`.

[MPI_STATUS_IGNORE](#), and the array version [MPI_STATUSES_IGNORE](#), can be used everywhere a status argument is passed to a receive, wait, or test function.

[MPI_STATUS_IGNORE](#) cannot be used when status is an `IN` argument. Note that in Fortran [MPI_STATUS_IGNORE](#) and [MPI_STATUSES_IGNORE](#) are objects like [MPI_BOTTOM](#) (not usable for initialization or assignment), see Section 2.5.4.

In general, this optimization can apply to all functions for which `status` or an array of `statuses` is an OUT argument. Note that this converts `status` into an INOUT argument. The functions that can be passed `MPI_STATUS_IGNORE` are all the various forms of `MPI_RECV`, `MPI_PROBE`, `MPI_TEST`, and `MPI_WAIT`, as well as `MPI_REQUEST_GET_STATUS`. When an array is passed, as in the `MPI_{TEST|WAIT}{ALL|SOME}` functions, a separate constant, `MPI_STATUSES_IGNORE`, is passed for the array argument. It is possible for an MPI function to return `MPI_ERR_IN_STATUS` even when `MPI_STATUS_IGNORE` or `MPI_STATUSES_IGNORE` has been passed to that function.

`MPI_STATUS_IGNORE` and `MPI_STATUSES_IGNORE` are not required to have the same values in C and Fortran.

It is not allowed to have some of the statuses in an array of statuses for `MPI_{TEST|WAIT}{ALL|SOME}` functions set to `MPI_STATUS_IGNORE`; one either specifies ignoring *all* of the statuses in such a call with `MPI_STATUSES_IGNORE`, or *none* of them by passing normal statuses in all positions in the array of statuses.

3.2.7 Blocking Send-Receive

The **send-receive** operations combine in one operation the sending of a message to one destination and the receiving of another message, from another MPI process. The two (source and destination) are possibly the same. A send-receive operation is very useful for executing a shift operation across a chain of MPI processes. If blocking sends and receives are used for such a shift, then one needs to order the sends and receives correctly (for example, MPI processes with even rank in the communicator send, then receive, MPI processes with odd rank in the communicator receive first, then send) so as to prevent cyclic dependencies that may lead to *deadlock*. When a send-receive operation is used, the communication subsystem takes care of these issues. The send-receive operation can be used in conjunction with the procedures described in Chapter 8 in order to perform shifts on various logical topologies. Also, a send-receive operation is useful for implementing remote procedure calls.

A message sent by a send-receive operation can be received by a regular receive operation or probed by a probe operation; a send-receive operation can receive a message sent by a regular send operation.

`MPI_SENDRECV(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount, recvtype, source, recvtag, comm, status)`

IN	sendbuf	initial address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	type of elements in send buffer (handle)
IN	dest	rank of destination (integer)
IN	sendtag	send tag (integer)
OUT	recvbuf	initial address of receive buffer (choice)
IN	recvcount	number of elements in receive buffer (nonnegative integer)

1	IN	recvtype	type of elements receive buffer element (handle)
2	IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)
3			
4	IN	recvtag	receive tag or <code>MPI_ANY_TAG</code> (integer)
5	IN	comm	communicator (handle)
6	OUT	status	status object (status)
7			

C binding

```

10 int MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
11                 int dest, int sendtag, void *recvbuf, int recvcount,
12                 MPI_Datatype recvtype, int source, int recvtag, MPI_Comm comm,
13                 MPI_Status *status)

```

```

14 int MPI_Sendrecv_c(const void *sendbuf, MPI_Count sendcount,
15                   MPI_Datatype sendtype, int dest, int sendtag, void *recvbuf,
16                   MPI_Count recvcount, MPI_Datatype recvtype, int source,
17                   int recvtag, MPI_Comm comm, MPI_Status *status)

```

Fortran 2008 binding

```

19 MPI_Sendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
20             recvtype, source, recvtag, comm, status, ierror)
21     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
22     INTEGER, INTENT(IN) :: sendcount, dest, sendtag, recvcount, source, recvtag
23     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
24     TYPE(*), DIMENSION(..) :: recvbuf
25     TYPE(MPI_Comm), INTENT(IN) :: comm
26     TYPE(MPI_Status) :: status
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

29 MPI_Sendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
30             recvtype, source, recvtag, comm, status, ierror) !(_c)
31     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
32     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
33     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
34     INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
35     TYPE(*), DIMENSION(..) :: recvbuf
36     TYPE(MPI_Comm), INTENT(IN) :: comm
37     TYPE(MPI_Status) :: status
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

40 MPI_SENDRCV(SENDBUF, SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVBUF, RECVCOUNT,
41             RECVTYPE, SOURCE, RECVTAG, COMM, STATUS, IERROR)
42     <type> SENDBUF(*), RECVBUF(*)
43     INTEGER SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVCOUNT, RECVTYPE, SOURCE,
44             RECVTAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR

```

Execute a blocking send-receive operation. Both send and receive use the same communicator, but possibly different tags. The send buffer and receive buffers must be disjoint, and may have different lengths and datatypes.

The semantics of a send-receive operation is what would be obtained if the caller forked two concurrent threads, one to execute the send, and one to execute the receive, followed by a join of these two threads.

`MPI_SENDRECV_REPLACE(buf, count, datatype, dest, sendtag, source, recvtag, comm, status)`

INOUT	buf	initial address of send and receive buffer (choice)
IN	count	number of elements in send and receive buffer (nonnegative integer)
IN	datatype	type of elements in send and receive buffer (handle)
IN	dest	rank of destination (integer)
IN	sendtag	send message tag (integer)
IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)
IN	recvtag	receive message tag or <code>MPI_ANY_TAG</code> (integer)
IN	comm	communicator (handle)
OUT	status	status object (status)

C binding

```
int MPI_Sendrecv_replace(void *buf, int count, MPI_Datatype datatype, int dest,
                        int sendtag, int source, int recvtag, MPI_Comm comm,
                        MPI_Status *status)
```

```
int MPI_Sendrecv_replace_c(void *buf, MPI_Count count, MPI_Datatype datatype,
                        int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
                        MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Sendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                    comm, status, ierror)
```

```
TYPE(*), DIMENSION(..) :: buf
```

```
INTEGER, INTENT(IN) :: count, dest, sendtag, source, recvtag
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Status) :: status
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Sendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                    comm, status, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..) :: buf
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Status) :: status
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_SENDRECV_REPLACE(BUF, COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG,
                     COMM, STATUS, IERROR)
<type> BUF(*)
INTEGER COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG, COMM,
STATUS(MPI_STATUS_SIZE), IERROR

```

Execute a blocking send and receive. The same buffer is used both for the send and for the receive, so that the message sent is replaced by the message received.

Advice to implementors. Additional intermediate buffering is needed for the “replace” variant. (*End of advice to implementors.*)

3.3 Datatype Matching and Data Conversion

3.3.1 Type Matching Rules

One can think of message transfer as consisting of the following three phases.

1. Data is pulled out of the send buffer and a message is assembled.
2. A message is transferred from sender to receiver.
3. Data is pulled from the incoming message and disassembled into the receive buffer.

Type matching has to be observed at each of these three phases: The type of each variable in the sender buffer has to match the type specified for that entry by the send operation; the type specified by the send operation has to match the type specified by the receive operation; and the type of each variable in the receive buffer has to match the type specified for that entry by the receive operation. A program that fails to observe these three rules is *erroneous*.

To define type matching more precisely, we need to deal with two issues: matching of types of the host language with types specified in communication operations; and matching of types at sender and receiver.

The types of a send and receive match (phase two) if both operations use identical names. That is, `MPI_INTEGER` matches `MPI_INTEGER`, `MPI_REAL` matches `MPI_REAL`, and so on. There is one exception to this rule, discussed in Section 5.2: the type `MPI_PACKED` can match any other type.

The type of a variable in a host program matches the type specified in the communication operation if the datatype name used by that operation corresponds to the basic type of the host program variable. For example, an entry with type name `MPI_INTEGER` matches a Fortran variable of type `INTEGER`. A table giving this correspondence for Fortran and C appears in Section 3.2.2. There are two exceptions to this last rule: an entry with type name `MPI_BYTE` or `MPI_PACKED` can be used to match any byte of storage (on a byte-addressable machine), irrespective of the datatype of the variable that contains this byte. The type `MPI_PACKED` is used to send data that has been explicitly packed, or receive data that will be explicitly unpacked, see Section 5.2. The type `MPI_BYTE` allows one to transfer the binary value of a byte in memory unchanged.

To summarize, the type matching rules fall into the three categories below.

- Communication of typed values (e.g., with datatype different from `MPI_BYTE`), where the datatypes of the corresponding entries in the sender program, in the send call, in the receive call and in the receiver program must all match.
- Communication of untyped values (e.g., of datatype `MPI_BYTE`), where both sender and receiver use the datatype `MPI_BYTE`. In this case, there are no requirements on the types of the corresponding entries in the sender and the receiver programs, nor is it required that they be the same.
- Communication involving packed data, where `MPI_PACKED` is used.

The following examples illustrate the first two cases.

Example 3.2. Sender and receiver specify matching types.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(a(1), 10, MPI_REAL, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(b(1), 15, MPI_REAL, 0, tag, comm, status, ierr)
END IF
```

This code is correct if both `a` and `b` are real arrays of size ≥ 10 . (In Fortran, it might be correct to use this code even if `a` or `b` have size < 10 : e.g., when `a(1)` can be equivalenced to an array with ten reals.)

Example 3.3. Sender and receiver do not specify matching types.

```
! ----- THIS EXAMPLE IS ERRONEOUS -----
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(a(1), 10, MPI_REAL, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(b(1), 40, MPI_BYTE, 0, tag, comm, status, ierr)
END IF
```

This code is *erroneous*, since sender and receiver do not provide matching datatype arguments.

Example 3.4. Sender and receiver specify communication of untyped values.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(a(1), 40, MPI_BYTE, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(b(1), 60, MPI_BYTE, 0, tag, comm, status, ierr)
END IF
```

This code is correct, irrespective of the type and size of `a` and `b` (unless this results in an out of bounds memory access).

Advice to users. If a buffer of type `MPI_BYTE` is passed as an argument to `MPI_SEND`, then MPI will send the data stored at contiguous locations, starting from the address indicated by the `buf` argument. This may have unexpected results when the data layout

is not as a casual user would expect it to be. For example, some Fortran compilers implement variables of type `CHARACTER` as a structure that contains the character length and a pointer to the actual string. In such an environment, sending and receiving a Fortran `CHARACTER` variable using the `MPI_BYTE` type will not have the anticipated result of transferring the character string. For this reason, the user is advised to use typed communication operations whenever possible. (*End of advice to users.*)

Type `MPI_CHARACTER`

The type `MPI_CHARACTER` matches one character of a Fortran variable of type `CHARACTER`, rather than the entire character string stored in the variable. Fortran variables of type `CHARACTER` or substrings are transferred as if they were arrays of characters. This is illustrated in the example below.

Example 3.5. Transfer of Fortran `CHARACTER`s.

```
CHARACTER*10 a
CHARACTER*10 b

CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(a, 5, MPI_CHARACTER, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(b(6:10), 5, MPI_CHARACTER, 0, tag, comm, status, ierr)
END IF
```

The last five characters of string `b` at the MPI process with `rank = 1` are replaced by the first five characters of string `a` at the MPI process with `rank = 0`.

Rationale. The alternative choice would be for `MPI_CHARACTER` to match a character of arbitrary length. This runs into problems.

A Fortran character variable is a constant length string, with no special termination symbol. There is no fixed convention on how to represent characters, and how to store their length. Some compilers pass a character argument to a routine as a pair of arguments, one holding the address of the string and the other holding the length of string. Consider the case of an MPI communication call that is passed a communication buffer with type defined by a derived datatype (Section 5.1). If this communicator buffer contains variables of type `CHARACTER` then the information on their length will not be passed to the MPI routine.

This problem forces us to provide explicit information on character length with the MPI call. One could add a length parameter to the type `MPI_CHARACTER`, but this does not add much convenience and the same functionality can be achieved by defining a suitable derived datatype. (*End of rationale.*)

Advice to implementors. Some compilers pass Fortran `CHARACTER` arguments as a structure with a length and a pointer to the actual string. In such an environment, the MPI call needs to dereference the pointer in order to reach the string. (*End of advice to implementors.*)

3.3.2 Data Conversion

One of the goals of MPI is to support parallel computations across heterogeneous environments. Communication in a heterogeneous environment may require data conversions. We use the following terminology.

type conversion changes the datatype of a value, e.g., by rounding a REAL to an INTEGER.

representation conversion changes the binary representation of a value, e.g., from Hex floating point to IEEE floating point.

The type matching rules imply that MPI communication never entails type conversion. On the other hand, MPI requires that a representation conversion be performed when a typed value is transferred across environments that use different representations for the datatype of this value. MPI does not specify rules for representation conversion. Such conversion is expected to preserve integer, logical and character values, and to convert a floating point value to the nearest value that can be represented on the target system.

Overflow and underflow exceptions may occur during floating point conversions. Conversion of integers or characters may also lead to exceptions when a value that can be represented in one system cannot be represented in the other system. An exception occurring during representation conversion results in a failure of the communication. An error occurs either in the send operation, or the receive operation, or both.

If a value sent in a message is untyped (i.e., of type `MPI_BYTE`), then the binary representation of the byte stored at the receiver is identical to the binary representation of the byte loaded at the sender. This holds true, whether sender and receiver run in the same or in distinct environments. No representation conversion is required. (Note that representation conversion may occur when values of type `MPI_CHARACTER` or `MPI_CHAR` are transferred, for example, from an EBCDIC encoding to an ASCII encoding.)

No conversion need occur when an MPI program executes in a homogeneous system, where all MPI processes run in the same environment.

Consider the three examples, 3.2–3.4. The first program is correct, assuming that `a` and `b` are REAL arrays of size ≥ 10 . If the sender and receiver execute in different environments, then the ten real values that are fetched from the send buffer will be converted to the representation for reals on the receiver site before they are stored in the receive buffer. While the number of real elements fetched from the send buffer equal the number of real elements stored in the receive buffer, the number of bytes stored need not equal the number of bytes loaded. For example, the sender may use a four byte representation and the receiver an eight byte representation for reals.

The second program is *erroneous*, and its behavior is undefined.

The third program is correct. The exact same sequence of forty bytes that were loaded from the send buffer will be stored in the receive buffer, even if sender and receiver run in a different environment. The message sent has exactly the same length (in bytes) and the same binary representation as the message received. If `a` and `b` are of different types, or if they are of the same type but different data representations are used, then the bits stored in the receive buffer may encode values that are different from the values they encoded in the send buffer.

Data representation conversion also applies to the *envelope* of a message: source, destination and tag are all integers that may need to be converted.

Advice to implementors. The current definition does not require messages to carry data type information. Both sender and receiver provide complete data type information. In a heterogeneous environment, one can either use a machine independent encoding such as XDR, or have the receiver convert from the sender representation to its own, or even have the sender do the conversion.

Additional type information might be added to messages in order to allow the system to detect mismatches between datatype at sender and receiver. This might be particularly useful in a slower but safer debug mode. (*End of advice to implementors.*)

MPI requires support for inter-language communication, e.g., if messages are sent using an MPI procedure from the MPI C language interface and received using an MPI procedure from one of the MPI Fortran language interfaces. The behavior is defined in Section 21.3.

3.4 Communication Modes

The send call described in Section 3.2.1 is *blocking*: it does not return until the *message data* and *envelope* have been safely stored away so that the sender is free to modify the send buffer. The message might be copied directly into the matching receive buffer, or it might be copied into a temporary system buffer.

Message buffering decouples the send and receive operations. A blocking send can complete as soon as the message was buffered, even if no matching receive has been executed by the receiver. On the other hand, message buffering can be expensive, as it entails additional memory-to-memory copying, and it requires the allocation of memory for buffering. MPI offers the choice of several **communication modes** that allow one to control the choice of the communication protocol.

The send call described in Section 3.2.1 uses the **standard** communication mode. In this mode, it is up to MPI to decide whether outgoing messages will be buffered. MPI may buffer outgoing messages. In such a case, the send call may complete before a matching receive is invoked. On the other hand, buffer space may be unavailable, or MPI may choose not to buffer outgoing messages, for performance reasons. In this case, the send call will not complete until a matching receive has been *started*, and the data has been moved to the receiver.

Thus, a *standard mode send* can be *started* whether or not a matching receive has been *started*. It may *complete* before a matching receive is *started*. The standard mode send is *nonlocal*: successful completion of the send operation may depend on the occurrence of a matching receive.

Rationale. The reluctance of MPI to mandate whether standard sends are buffering or not stems from the desire to achieve portable programs. Since any system will run out of buffer resources as message sizes are increased, and some implementations may want to provide little buffering, MPI takes the position that correct (and therefore, portable) programs do not rely on system buffering in standard mode. Buffering may improve the performance of a correct program, but it doesn't affect the result of the program. If the user wishes to guarantee a certain amount of buffering, the user-provided buffer system of Section 3.6 should be used, along with the buffered-mode send. (*End of rationale.*)

There are three additional communication modes.

A **buffered** mode send operation can be started whether or not a matching receive has been *started*. It may complete before a matching receive is *started*. However, unlike the standard send, this operation is *local*, and its completion does not depend on the occurrence of a matching receive. Thus, if a send is executed and no matching receive is *started*, then MPI must buffer the outgoing message, so as to allow the send call to complete. An error will occur if there is insufficient buffer space. The amount of available buffer space is controlled by the user—see Section 3.6. Buffer allocation by the user may be required for the buffered mode to be effective.

A send that uses the **synchronous** mode can be started whether or not a matching receive was *started*. However, the send will complete successfully only if a matching receive is *started*, and the receive operation has started to receive the message sent by the synchronous send. Thus, the completion of a synchronous send not only indicates that the send buffer can be reused, but it also indicates that the receiver has reached a certain point in its execution, namely that it has started executing the matching receive. If both sends and receives are blocking operations then the use of the synchronous mode provides synchronous communication semantics: a communication does not complete at either end before both MPI processes rendezvous at the communication. A send executed in this mode is *nonlocal*.

A send that uses the **ready** communication mode may be started *only* if the matching receive is already *started*. Otherwise, the operation is *erroneous* and its outcome is undefined. On some systems, this allows the removal of a hand-shake protocol that is otherwise required and results in improved performance. The completion of the send operation does not depend on the status of a matching receive, and merely indicates that the send buffer can be reused. A send operation that uses the ready mode has the same semantics as a standard send operation, or a synchronous send operation; it is merely that the sender provides additional information to the system (namely that a matching receive is already *started*), that can save some overhead. In a correct program, therefore, a ready send could be replaced by a standard send with no effect on the behavior of the program other than performance.

Three additional send functions are provided for the three additional communication modes. The communication mode is indicated by a one letter prefix: **B** for buffered, **S** for synchronous, and **R** for ready.

MPI_BSEND(buf, count, datatype, dest, tag, comm)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of each send buffer element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)

C binding

```
int MPI_Bsend(const void *buf, int count, MPI_Datatype datatype, int dest,
              int tag, MPI_Comm comm)
```

```

1  int MPI_Bsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
2                  int dest, int tag, MPI_Comm comm)

```

Fortran 2008 binding

```

4  MPI_Bsend(buf, count, datatype, dest, tag, comm, ierror)
5      TYPE(*), DIMENSION(..), INTENT(IN) :: buf
6      INTEGER, INTENT(IN) :: count, dest, tag
7      TYPE(MPI_Datatype), INTENT(IN) :: datatype
8      TYPE(MPI_Comm), INTENT(IN) :: comm
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

11 MPI_Bsend(buf, count, datatype, dest, tag, comm, ierror) !(_c)
12 TYPE(*), DIMENSION(..), INTENT(IN) :: buf
13 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
14 TYPE(MPI_Datatype), INTENT(IN) :: datatype
15 INTEGER, INTENT(IN) :: dest, tag
16 TYPE(MPI_Comm), INTENT(IN) :: comm
17 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

19 MPI_BSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
20 <type> BUF(*)
21 INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR

```

Send in buffered mode.

According to the definitions in Section 2.4.2, `MPI_BSEND` is a completing procedure and the user can re-use all resources given as arguments, including the *message data buffer*. It is also a local procedure because it returns immediately without depending on the execution of any MPI procedure in any other MPI process.

Advice to users. This is one of the exceptions in which a completing and therefore blocking operation-related procedure is local. (*End of advice to users.*)

```

33 MPI_SSEND(buf, count, datatype, dest, tag, comm)

```

34	IN	buf	initial address of send buffer (choice)
35	IN	count	number of elements in send buffer (nonnegative integer)
36			
37	IN	datatype	datatype of each send buffer element (handle)
38	IN	dest	rank of destination (integer)
39	IN	tag	message tag (integer)
40	IN	comm	communicator (handle)
41			
42			

C binding

```

44 int MPI_Ssend(const void *buf, int count, MPI_Datatype datatype, int dest,
45              int tag, MPI_Comm comm)
46
47 int MPI_Ssend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
48                 int dest, int tag, MPI_Comm comm)

```

Fortran 2008 binding

```

MPI_Ssend(buf, count, datatype, dest, tag, comm, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER, INTENT(IN) :: count, dest, tag
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ssend(buf, count, datatype, dest, tag, comm, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, INTENT(IN) :: dest, tag
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
    <type> BUF(*)
    INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR

Send in synchronous mode.

```

```

MPI_RSEND(buf, count, datatype, dest, tag, comm)

```

IN	buf	initial address of send buffer (choice)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of each send buffer element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)

C binding

```

int MPI_Rsend(const void *buf, int count, MPI_Datatype datatype, int dest,
              int tag, MPI_Comm comm)

int MPI_Rsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                int dest, int tag, MPI_Comm comm)

```

Fortran 2008 binding

```

MPI_Rsend(buf, count, datatype, dest, tag, comm, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER, INTENT(IN) :: count, dest, tag
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rsend(buf, count, datatype, dest, tag, comm, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf

```

```

1      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
2      TYPE(MPI_Datatype), INTENT(IN) :: datatype
3      INTEGER, INTENT(IN) :: dest, tag
4      TYPE(MPI_Comm), INTENT(IN) :: comm
5      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

7      MPI_RSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
8      <type> BUF(*)
9      INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR

```

Send in ready mode.

There is only one receive operation, but it matches any of the send modes. The receive procedure described in the last section is *blocking*: it returns only after the receive buffer contains the newly received message. A receive can complete before the matching send has completed (of course, it can complete only after the matching send has started).

In a multithreaded implementation of MPI, the system may de-schedule a thread that is blocked on a send or receive operation, and schedule another thread for execution in the same address space. In such a case it is the user's responsibility not to modify a communication buffer until the communication completes. Otherwise, the outcome of the computation is undefined.

Advice to implementors. Since a synchronous send cannot complete before a matching receive is *started*, one will not normally buffer messages sent by such an operation.

It is recommended to choose buffering over blocking the sender, whenever possible, for standard sends. The programmer can signal a preference for blocking the sender until a matching receive occurs by using the synchronous send mode.

A possible communication protocol for the various communication modes is outlined below.

ready send: The message is sent as soon as possible.

synchronous send: The sender sends a request-to-send message. The receiver stores this request. When a matching receive is *started*, the receiver sends back a permission-to-send message, and the sender now sends the message.

standard send: First protocol may be used for short messages, and second protocol for long messages.

buffered send: The sender copies the message into a buffer and then sends it with a nonblocking send (using the same protocol as for standard send).

Additional control messages might be needed for flow control and error recovery. Of course, there are many other possible protocols.

Ready send can be implemented as a standard send. In this case there will be no performance advantage (or disadvantage) for the use of ready send.

A standard send can be implemented as a synchronous send. In such a case, no data buffering is needed. However, users may expect some buffering.

In a multithreaded environment, the execution of a blocking communication should block only the executing thread, allowing the thread scheduler to de-schedule this thread and schedule another thread for execution. (*End of advice to implementors.*)

3.5 Semantics of Point-to-Point Communication

A valid MPI implementation guarantees certain general properties of point-to-point communication, which are described in this section.

Order. Messages are **nonovertaking**: If a sender sends two messages in succession to the same destination, and both match the same receive, then this operation cannot receive the second message if the first one is still pending. If a receiver posts two receives in succession, and both match the same message, then the second receive operation cannot be satisfied by this message, if the first one is still pending. This requirement facilitates matching of sends to receives. It guarantees that message-passing code is deterministic, if MPI processes are single-threaded and the wildcard `MPI_ANY_SOURCE` is not used in receives. (Some of the calls described later, such as `MPI_CANCEL` or `MPI_WAITANY`, are additional sources of nondeterminism.)

If an MPI process has a single thread of execution, then any two communication operations executed by this MPI process are **ordered**.

Advice to users. The MPI Forum believes the following paragraph is ambiguous and may clarify the meaning in a future version of the MPI Standard. (*End of advice to users.*)

On the other hand, if the MPI process is multithreaded, then the semantics of thread execution may not define a relative order between two send operations executed by two distinct threads. The operations are **logically concurrent**, even if one physically precedes the other. In such a case, the two messages sent can be received in any order. Similarly, if two receive operations that are **logically concurrent** receive two successively sent messages, then the two messages can match the two receives in either order.

Advice to implementors. The MPI Forum believes the previous paragraph is ambiguous and may clarify the meaning in a future version of the MPI Standard. (*End of advice to implementors.*)

Example 3.6. An example of nonovertaking messages.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_BSEND(buf1, count, MPI_REAL, 1, tag, comm, ierr)
  CALL MPI_BSEND(buf2, count, MPI_REAL, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(buf1, count, MPI_REAL, 0, MPI_ANY_TAG, comm, status, &
               ierr)
  CALL MPI_RECV(buf2, count, MPI_REAL, 0, tag, comm, status, ierr)
END IF
```

The message sent by the first send must be received by the first receive, and the message sent by the second send must be received by the second receive.

Progress. If a pair of matching send and receive operations have been initiated, then at least one of these two operations will complete, independently of other actions in the system: the send operation will complete, unless the receive is satisfied by another message,

and completes; the receive operation will complete, unless the message sent is consumed by another matching receive that was *started* at the same destination MPI process.

Example 3.7. An example of two, intertwined matching pairs.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_BSEND(buf1, count, MPI_REAL, 1, tag1, comm, ierr)
  CALL MPI_SSEND(buf2, count, MPI_REAL, 1, tag2, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(buf1, count, MPI_REAL, 0, tag2, comm, status, ierr)
  CALL MPI_RECV(buf2, count, MPI_REAL, 0, tag1, comm, status, ierr)
END IF
```

Both MPI processes invoke their first communication call. Since the first send at the MPI process with `rank = 0` uses the buffered mode, it must complete, irrespective of the state of the other MPI process(es). Since no matching receive is *started*, the message will be copied into buffer space. (If insufficient buffer space is available, then the program will fail.) The second send is then invoked. At that point, a matching pair of send and receive operation is enabled, and both operations must complete. Next, the second receive call is invoked, which will be satisfied by the buffered message. Note that the MPI process with `rank = 1` received the messages in the reverse order they were sent.

Fairness. MPI makes no guarantee of **fairness** in the handling of communication. Suppose that a send is *started*. Then it is possible that the destination MPI process repeatedly posts a receive that matches this send, yet the message is never received, because it is overtaken each time by another message, sent from another source. Similarly, suppose that a receive was *started* by a multithreaded MPI process. Then it is possible that messages that match this receive are repeatedly received, yet the receive is never satisfied, because it is overtaken by other receives *started* at this MPI process (by other executing threads). It is the programmer's responsibility to prevent starvation in such situations.

Resource limitations. Any *pending* communication operation and *decoupled MPI activity* consumes system resources that are limited. Errors may occur when lack of resources prevent the execution of an MPI call. High-quality implementations will use a (small) fixed amount of resources for each *pending* send in the ready or synchronous mode and for each *pending* receive. However, buffer space may be consumed to store messages sent in standard mode, and must be consumed to store messages sent in buffered mode, when no matching receive is available. The amount of space available for buffering will be much smaller than program data memory on many systems. Then, it will be easy to write programs that overrun available buffer space.

MPI allows the user to provide buffer memory for messages sent in the buffered mode. Furthermore, MPI specifies a detailed operational model for the use of this buffer. An MPI implementation is required to do no worse than implied by this model. This allows users to avoid buffer overflows when they use buffered sends. Buffer allocation and use is described in Section 3.6.

A buffered send operation that cannot complete because of a lack of buffer space is *erroneous*. When such a situation is detected, an error is signaled that may cause the program to terminate abnormally. On the other hand, a standard send operation that

cannot complete because of lack of buffer space will merely block, waiting for buffer space to become available or for a matching receive to be *started*. This behavior is preferable in many situations. Consider a situation where a producer repeatedly produces new values and sends them to a consumer. Assume that the producer produces new values faster than the consumer can consume them. If buffered sends are used, then a buffer overflow will result. Additional synchronization has to be added to the program so as to prevent this from occurring. If standard sends are used, then the producer will be automatically throttled, as its send operations will block when buffer space is unavailable.

In some situations, a lack of buffer space leads to deadlock situations. This is illustrated by the examples below.

Example 3.8. An exchange of messages.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(sendbuf, count, MPI_REAL, 1, tag, comm, ierr)
  CALL MPI_RECV(recvbuf, count, MPI_REAL, 1, tag, comm, status, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(recvbuf, count, MPI_REAL, 0, tag, comm, status, ierr)
  CALL MPI_SEND(sendbuf, count, MPI_REAL, 0, tag, comm, ierr)
END IF
```

This program will succeed even if no buffer space for data is available. The standard send operation can be replaced, in this example, with a synchronous send.

Example 3.9. An errant attempt to exchange messages.

```
! ----- THIS EXAMPLE IS ERRONEOUS -----
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_RECV(recvbuf, count, MPI_REAL, 1, tag, comm, status, ierr)
  CALL MPI_SEND(sendbuf, count, MPI_REAL, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_RECV(recvbuf, count, MPI_REAL, 0, tag, comm, status, ierr)
  CALL MPI_SEND(sendbuf, count, MPI_REAL, 0, tag, comm, ierr)
END IF
```

The receive operation of the MPI process with rank = 0 must complete before its send, and can complete only if the matching send of the MPI process with rank = 1 is executed. The receive operation of the MPI process with rank = 1 must complete before its send and can complete only if the matching send of the MPI process with rank = 0 is executed. This program will always deadlock. The same holds for any other send mode.

Example 3.10. An unsafe exchange that relies on MPI to provide sufficient buffering.

```
! ----- THIS EXAMPLE IS ERRONEOUS -----
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(sendbuf, count, MPI_REAL, 1, tag, comm, ierr)
  CALL MPI_RECV(recvbuf, count, MPI_REAL, 1, tag, comm, status, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_SEND(sendbuf, count, MPI_REAL, 0, tag, comm, ierr)
```

```

1  CALL MPI_RECV(recvbuf, count, MPI_REAL, 0, tag, comm, status, ierr)
2  END IF

```

The message sent by each MPI process has to be copied out before the send operation completes and the receive operation starts. For the program to complete, it is necessary that at least one of the two messages sent be buffered. Thus, this program can succeed only if the communication system can buffer at least `count` words of data.

Advice to users. If standard mode send operations are used as in Example 3.10, then a deadlock situation may occur where both MPI processes are blocked because sufficient buffer space is not available. The same will certainly happen, if the synchronous mode is used. If the buffered mode is used, and not enough buffer space is available, then the program will not complete either. However, rather than a deadlock situation, we shall have a buffer overflow error.

A portable program using standard mode send operations should not rely on message buffering for the program to complete without *deadlock*. All sends in such a portable program can be replaced with synchronous mode sends and the program will still run correctly. The buffered send mode can be used for programs that require buffering.

Nonblocking message-passing operations, as described in Section 3.7, can be used to avoid the need for buffering outgoing messages. This can prevent unintentional *serialization* or *deadlock* due to lack of buffer space, and improves performance, by allowing *overlap* of communication with other communication or with computation, and avoiding the overheads of allocating buffers and copying messages into buffers. (*End of advice to users.*)

3.6 Buffer Allocation and Usage

A user may specify up to one buffer per communicator, up to one buffer per session, and up to one buffer per MPI process to be used for buffering messages sent in buffered mode. Buffering is done by the sender.

MPI_COMM_ATTACH_BUFFER(comm, buffer, size)

IN	comm	communicator (handle)
IN	buffer	initial buffer address (choice)
IN	size	buffer size, in bytes (nonnegative integer)

C binding

```
int MPI_Comm_attach_buffer(MPI_Comm comm, void *buffer, int size)
```

```
int MPI_Comm_attach_buffer_c(MPI_Comm comm, void *buffer, MPI_Count size)
```

Fortran 2008 binding

```
MPI_Comm_attach_buffer(comm, buffer, size, ierror)
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
```

```
INTEGER, INTENT(IN) :: size
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```



```

MPI_Comm_attach_buffer(comm, buffer, size, ierror) !(_c)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_ATTACH_BUFFER(COMM, BUFFER, SIZE, IERROR)
    INTEGER COMM, SIZE, IERROR
    <type> BUFFER(*)

```

Provides to MPI a communicator-specific buffer in memory. This is to be used for buffering outgoing messages sent when a buffered mode send is started that uses the communicator `comm`.

If `MPI_BUFFER_AUTOMATIC` is passed as the argument `buffer`, no explicit buffer is attached; rather, automatic buffering is enabled for all buffered mode communication associated with the communicator `comm` (see Section 3.6). Further, if `MPI_BUFFER_AUTOMATIC` is passed as the argument `buffer`, the value of `size` is irrelevant. Note that in Fortran `MPI_BUFFER_AUTOMATIC` is an object like `MPI_BOTTOM` (not usable for initialization or assignment), see Section 2.5.4.

```

MPI_SESSION_ATTACH_BUFFER(session, buffer, size)

```

IN	session	session (handle)
IN	buffer	initial buffer address (choice)
IN	size	buffer size, in bytes (nonnegative integer)

C binding

```

int MPI_Session_attach_buffer(MPI_Session session, void *buffer, int size)
int MPI_Session_attach_buffer_c(MPI_Session session, void *buffer,
    MPI_Count size)

```

Fortran 2008 binding

```

MPI_Session_attach_buffer(session, buffer, size, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
    INTEGER, INTENT(IN) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Session_attach_buffer(session, buffer, size, ierror) !(_c)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SESSION_ATTACH_BUFFER(SESSION, BUFFER, SIZE, IERROR)
    INTEGER SESSION, SIZE, IERROR
    <type> BUFFER(*)

```

Provides to MPI a session-specific buffer in memory. This buffer is to be used for buffering outgoing messages sent when using a communicator that is created from a group that is derived from the session `session`. However, if there is a communicator-specific buffer attached to the particular communicator at the time of the buffered mode send is started, that buffer is used.

If `MPI_BUFFER_AUTOMATIC` is passed as the argument `buffer`, no explicit buffer is attached; rather, automatic buffering is enabled for all buffered mode communication associated with the session `session` that is not explicitly covered by a buffer provided at communicator level (see Section 3.6). Further, if `MPI_BUFFER_AUTOMATIC` is passed as the argument `buffer`, the value of `size` is irrelevant. Note that in Fortran `MPI_BUFFER_AUTOMATIC` is an object like `MPI_BOTTOM` (not usable for initialization or assignment), see Section 2.5.4.

`MPI_BUFFER_ATTACH(buffer, size)`

IN	buffer	initial buffer address (choice)
IN	size	buffer size, in bytes (nonnegative integer)

C binding

```
int MPI_Buffer_attach(void *buffer, int size)
```

```
int MPI_Buffer_attach_c(void *buffer, MPI_Count size)
```

Fortran 2008 binding

```
MPI_Buffer_attach(buffer, size, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
  INTEGER, INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Buffer_attach(buffer, size, ierror) !(_c)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_BUFFER_ATTACH(BUFFER, SIZE, IERROR)
  <type> BUFFER(*)
  INTEGER SIZE, IERROR
```

Provides to MPI an MPI process-specific buffer in memory. This buffer is to be used for buffering outgoing messages sent when using a communicator to which no communicator-specific buffer is attached or which is derived from a session to which no session-specific buffer is attached at the time the buffered mode send is started.

If `MPI_BUFFER_AUTOMATIC` is passed as the argument `buffer`, no explicit buffer is attached; rather, automatic buffering is enabled for all buffered mode communication not explicitly covered by a buffer provided at session or communicator level (see Section 3.6). Further, if `MPI_BUFFER_AUTOMATIC` is passed as the argument `buffer`, the value of `size` is irrelevant. Note that in Fortran `MPI_BUFFER_AUTOMATIC` is an object like `MPI_BOTTOM` (not usable for initialization or assignment), see Section 2.5.4.

Advice to users. The use of a global shared buffer can be problematic when used for communication in different libraries, as the buffer represents a shared resource used

for all buffered mode communication. Further, with the introduction of the Sessions Model, the use of a single shared buffer violates the concept of resource isolation that is intended with MPI Sessions. It is therefore recommended, especially for libraries and programs using the Sessions Model, to use only communicator-specific or session-specific buffers. (*End of advice to users.*)

Any of these buffers are used only for messages sent in buffered mode. Only one MPI process-specific buffer can be attached to an MPI process at a time, only one session-specific buffer can be attached to a session at a time and only one communicator-specific buffer can be attached to a communicator at a time.

If automatic buffering is enabled at any level, no other buffer can be attached at that level.

A particular memory region can only be used in one buffer; reusing buffer space for multiple sessions, communicators and/or the global buffer is erroneous. Further, only one buffer is used for any one communication following the rules above; buffer space is not combined, even if two buffers are directly or indirectly provided to a communicator to be used for buffered sends.

In C, `buffer` is the starting address of a memory region. In Fortran, one can pass the first element of a memory region or a whole array, which must be ‘simply contiguous’ (for ‘simply contiguous,’ see also Section 21.1.12).

`MPI_COMM_DETACH_BUFFER(comm, buffer_addr, size)`

IN	<code>comm</code>	communicator (handle)
OUT	<code>buffer_addr</code>	initial buffer address (choice)
OUT	<code>size</code>	buffer size, in bytes (integer)

C binding

```
int MPI_Comm_detach_buffer(MPI_Comm comm, void *buffer_addr, int *size)
```

```
int MPI_Comm_detach_buffer_c(MPI_Comm comm, void *buffer_addr, MPI_Count *size)
```

Fortran 2008 binding

```
MPI_Comm_detach_buffer(comm, buffer_addr, size, ierror)
```

```
USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(C_PTR), INTENT(OUT) :: buffer_addr
```

```
INTEGER, INTENT(OUT) :: size
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Comm_detach_buffer(comm, buffer_addr, size, ierror) !(_c)
```

```
USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(C_PTR), INTENT(OUT) :: buffer_addr
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_COMM_DETACH_BUFFER(COMM, BUFFER_ADDR, SIZE, IERROR)
    INTEGER COMM, SIZE, IERROR
    <type> BUFFER_ADDR(*)

```

Detach the communicator-specific buffer currently attached to the communicator.

```

MPI_SESSION_DETACH_BUFFER(session, buffer_addr, size)

```

IN	session	session (handle)
OUT	buffer_addr	initial buffer address (choice)
OUT	size	buffer size, in bytes (integer)

C binding

```

int MPI_Session_detach_buffer(MPI_Session session, void *buffer_addr,
                             int *size)

int MPI_Session_detach_buffer_c(MPI_Session session, void *buffer_addr,
                               MPI_Count *size)

```

Fortran 2008 binding

```

MPI_Session_detach_buffer(session, buffer_addr, size, ierror)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(C_PTR), INTENT(OUT) :: buffer_addr
    INTEGER, INTENT(OUT) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Session_detach_buffer(session, buffer_addr, size, ierror) !(_c)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(C_PTR), INTENT(OUT) :: buffer_addr
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SESSION_DETACH_BUFFER(SESSION, BUFFER_ADDR, SIZE, IERROR)
    INTEGER SESSION, SIZE, IERROR
    <type> BUFFER_ADDR(*)

```

Detach the session-specific buffer currently attached to the session.

```

MPI_BUFFER_DETACH(buffer_addr, size)

```

OUT	buffer_addr	initial buffer address (choice)
OUT	size	buffer size, in bytes (integer)

C binding

```

int MPI_Buffer_detach(void *buffer_addr, int *size)

```

```
int MPI_Buffer_detach_c(void *buffer_addr, MPI_Count *size)
```

Fortran 2008 binding

```
MPI_Buffer_detach(buffer_addr, size, ierror)
  USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
  TYPE(C_PTR), INTENT(OUT) :: buffer_addr
  INTEGER, INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Buffer_detach(buffer_addr, size, ierror) !(_c)
  USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
  TYPE(C_PTR), INTENT(OUT) :: buffer_addr
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_BUFFER_DETACH(BUFFER_ADDR, SIZE, IERROR)
  <type> BUFFER_ADDR(*)
  INTEGER SIZE, IERROR
```

Detach the MPI process-specific buffer `buffer` currently attached to MPI.

The procedure calls return the address and the size of the detached buffer. If `MPI_BUFFER_AUTOMATIC` was used in the corresponding attach procedure, then `MPI_BUFFER_AUTOMATIC` is also returned in the detach procedure and the value returned in argument `size` is undefined. In this case, automatic buffering is disabled upon return from the detach procedure. When using Fortran `mpi_f08`, the returned value is identical to `c_loc(MPI_BUFFER_AUTOMATIC)`. Note that `c_loc()` is an intrinsic in the Fortran ISO_C_BINDING module.

Advice to implementors. In Fortran, the implementation of `MPI_BUFFER_AUTOMATIC` must allow the intrinsic `c_loc` to be applied to it. (*End of advice to implementors.*)

These procedures will delay their return until all messages currently in the (explicit or automatic) buffer have been transmitted. Upon return of these procedures, the user may reuse or deallocate the space taken by the buffer.

If the size of the detached buffer cannot be represented in `size`, it is set to `MPI_UNDEFINED`.

The following `MPI_XXX_FLUSH_BUFFER` procedures, as well as `MPI_BUFFER_FLUSH` itself, will not return until all messages currently in the buffer have been transmitted without detaching the buffer.

Rationale. These flush procedures provide the same functionality as an atomic combination of first detaching the buffer and then attaching it again (but without having to actually execute the detaching and the re-attaching of the buffer), but they may be implemented with less internal overhead. (*End of rationale.*)

1 MPI_COMM_FLUSH_BUFFER(comm)

2 IN comm communicator (handle)

4 **C binding**

5 int MPI_Comm_flush_buffer(MPI_Comm comm)

7 **Fortran 2008 binding**

8 MPI_Comm_flush_buffer(comm, ierror)

9 TYPE(MPI_Comm), INTENT(IN) :: comm

10 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

11 **Fortran binding**

12 MPI_COMM_FLUSH_BUFFER(COMM, IERROR)

13 INTEGER COMM, IERROR

14
15 MPI_COMM_FLUSH_BUFFER will not return until all messages currently in the com-
16 municator-specific buffer of the calling MPI process have been transmitted.

18 MPI_SESSION_FLUSH_BUFFER(session)

19 IN session session (handle)

22 **C binding**

23 int MPI_Session_flush_buffer(MPI_Session session)

24 **Fortran 2008 binding**

25 MPI_Session_flush_buffer(session, ierror)

26 TYPE(MPI_Session), INTENT(IN) :: session

27 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

28 **Fortran binding**

29 MPI_SESSION_FLUSH_BUFFER(SESSION, IERROR)

30 INTEGER SESSION, IERROR

31
32 MPI_SESSION_FLUSH_BUFFER will not return until all messages currently in the
33 session-specific buffer of the calling MPI process have been transmitted.

35
36 MPI_BUFFER_FLUSH()

37 **C binding**

38 int MPI_Buffer_flush(void)

39 **Fortran 2008 binding**

40 MPI_Buffer_flush(ierror)

41 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

42 **Fortran binding**

43 MPI_BUFFER_FLUSH(IERROR)

44 INTEGER IERROR

45
46 MPI_BUFFER_FLUSH will not return until all messages currently in the MPI process-
47 specific buffer of the calling MPI process have been transmitted.

For all MPI_XXX_FLUSH_BUFFER procedures, there also exist the following nonblocking variants, which start the respective flush operation. These operations will not complete until all messages currently in the respective buffer of the calling MPI process have been transmitted.

MPI_COMM_IFLUSH_BUFFER(comm, request)

IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Comm_iflush_buffer(MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Comm_iflush_buffer(comm, request, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_IFLUSH_BUFFER(COMM, REQUEST, IERROR)
  INTEGER COMM, REQUEST, IERROR
```

MPI_SESSION_IFLUSH_BUFFER(session, request)

IN	session	session (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Session_iflush_buffer(MPI_Session session, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Session_iflush_buffer(session, request, ierror)
  TYPE(MPI_Session), INTENT(IN) :: session
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SESSION_IFLUSH_BUFFER(SESSION, REQUEST, IERROR)
  INTEGER SESSION, REQUEST, IERROR
```

MPI_BUFFER_IFLUSH(request)

OUT	request	communication request (handle)
-----	---------	--------------------------------

C binding

```
int MPI_Buffer_iflush(MPI_Request *request)
```

Fortran 2008 binding

```

MPI_Buffer_flush(request, ierror)
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_BUFFER_FLUSH(REQUEST, IERROR)
    INTEGER REQUEST, IERROR

```

Example 3.11. Calls to attach and detach buffers.

```

#define BUFFSIZE 10000+MPI_BSEND_OVERHEAD
int size;
char *buff;
MPI_Buffer_attach(malloc(BUFFSIZE), BUFFSIZE);
/* a buffer of 10000 bytes can now be used by MPI_Bsend */
/* on all communicators, assuming only one message at a time is sent */
MPI_Buffer_detach(&buff, &size);
/* Buffer size reduced to zero */
MPI_Buffer_attach(buff, size);
/* Buffer of 10000 bytes available again */

```

Example 3.12. Calls to attach and detach communicator-specific buffers.

```

#define BUFFSIZE1 10000+MPI_BSEND_OVERHEAD
#define BUFFSIZE2 20000+MPI_BSEND_OVERHEAD
int size;
char *buff1, *buff2;
MPI_Comm world_dup;
MPI_Comm_dup(MPI_COMM_WORLD, &world_dup);
MPI_Comm_attach_buffer(MPI_COMM_WORLD, malloc(BUFFSIZE2), BUFFSIZE2);
MPI_Buffer_attach(malloc(BUFFSIZE1), BUFFSIZE1);
/* a buffer of 20000 bytes can now be used by MPI_Bsend for */
/* communication using MPI_COMM_WORLD, assuming only one message */
/* at a time is sent a buffer of 10000 bytes can now be used by */
/* MPI_Bsend for communication using any other communicator, */
/* including world_dup assuming only one message at a time is sent */
MPI_Comm_detach_buffer(MPI_COMM_WORLD, &buff1, &size);
MPI_Buffer_detach(&buff2, &size);
/* Both buffers are detached and no specific or MPI process-specific */
/* buffer can be used for further MPI_Bsend */

```

Advice to users. Even though the C procedures `MPI_Buffer_attach`, `MPI_Session_attach_buffer`, `MPI_Comm_attach_buffer`, `MPI_Buffer_detach`, `MPI_Session_detach_buffer` and `MPI_Comm_detach_buffer` have an argument of type `void*`, these arguments are used differently: a pointer to the buffer is passed to `MPI_Buffer_attach`, `MPI_Session_attach_buffer` and `MPI_Comm_attach_buffer`; the address of the pointer is passed to `MPI_Buffer_detach`, `MPI_Session_detach_buffer` and `MPI_Comm_detach_buffer`, so that this call can return the pointer value. In Fortran with the `mpi` module or (deprecated) `mpif.h`, the type of the `buffer_addr` argument is wrongly defined and the argument is therefore unused. In Fortran with the `mpi_f08`

module, the address of the buffer is returned as `TYPE(C_PTR)`, see also Example 9.2 about the use of `C_PTR` pointers. (*End of advice to users.*)

Rationale. In all cases, arguments are defined to be of type `void*` (rather than `void*` and `void**`, respectively), so as to avoid complex type casts. E.g., in the last two examples, `&buff`, which is of type `char**`, can be passed as argument to `MPI_Buffer_detach`, `MPI_Session_detach_buffer` and `MPI_Comm_detach_buffer` without type casting. If the formal parameter had type `void**` then we would need a type cast before and after each call. (*End of rationale.*)

General semantics of buffered mode sends. The statements made in this section describe the behavior of MPI for buffered-mode sends.

When no MPI process-specific buffer is currently (explicitly) attached and if no automatic buffering is enabled, MPI behaves as if a zero-sized MPI process-specific buffer is (implicitly) attached.

It is erroneous to detach a communicator-specific, session-specific, or MPI process-specific buffer, if no such buffer had been attached using a corresponding attach procedure. This includes attach procedure calls using `MPI_BUFFER_AUTOMATIC` as the buffer argument. It is erroneous to attach a communicator-specific, session-specific, or MPI process-specific buffer, if such buffer had already been attached using a corresponding attach procedure and not yet been detached again. This includes attach procedure calls using `MPI_BUFFER_AUTOMATIC` as the buffer argument. It is erroneous to flush a communicator-specific, session-specific or MPI process-specific buffer, if there is no buffer attached (including automatic buffering).

`MPI_COMM_ATTACH_BUFFER`, `MPI_SESSION_ATTACH_BUFFER`, and `MPI_BUFFER_ATTACH` are local. `MPI_COMM_DETACH_BUFFER`, `MPI_SESSION_DETACH_BUFFER`, `MPI_BUFFER_DETACH`, `MPI_COMM_FLUSH_BUFFER`, `MPI_SESSION_FLUSH_BUFFER`, and `MPI_BUFFER_FLUSH` are nonlocal; they must not return before all buffered messages in their related buffers are transmitted, and they must eventually return when all corresponding receive operations are started (provided that none are cancelled).

Automatic buffering with buffered mode sends. If the buffer used at the time of buffered mode send is set to the buffer address `MPI_BUFFER_AUTOMATIC`, then a buffer of sufficient size is automatically used by the MPI library.

Advice to users. When using automatic buffering, the user relinquishes control over buffer management, including allocation and deallocation decisions and timing, to the MPI library. If explicit control is needed over when and how much buffer space is allocated, automatic buffering must not be used. (*End of advice to users.*)

Advice to implementors. High-quality implementations of an MPI library should strive to support automatic buffering in a balanced fashion, i.e., providing the right balance between memory allocated for send operations and memory available for the end user. (*End of advice to implementors.*)

The flush operations for the communicator-specific, session-specific, and MPI process-specific buffers can also be used for automatic buffering. The flush procedure will not return

until all automatically allocated buffers for the communicator-specific, session-specific, or MPI process-specific buffers, respectively, no longer hold message data and could be deallocated by the MPI library, if it chooses to do so.

Advice to users. With standard mode send, the limitation of needed buffer space is implemented within the MPI library through switching from internal buffering to internal synchronous mode. If the user wants to limit the automatically allocated buffer space for buffered mode send using automatic buffering, the user may call explicitly the appropriate flush procedure to wait until automatically allocated buffers are deallocated. (*End of advice to users.*)

Further rules. In the case of an attached buffer (i.e., not using automatic buffering), the user must provide as much buffering for outgoing messages as would be required if outgoing message data were buffered by the sending MPI process, in the specified buffer space, using a circular, contiguous-space allocation policy. We outline below a model implementation that defines this policy. MPI may provide more buffering, and may use a better buffer allocation algorithm than described below. On the other hand, MPI may signal an error whenever the simple buffering allocator described below would run out of space. MPI must not require more buffer space as described in the model implementation below.

MPI does not provide mechanisms for querying or controlling buffering done by standard mode sends. It is expected that vendors will provide such information for their implementations.

Rationale. There is a wide spectrum of possible implementations of buffered communication operations: buffering can be done at sender, at receiver, or both; buffers can be dedicated to one sender-receiver pair, or be shared by all communication operations; buffering can be done in real or in virtual memory; it can use dedicated memory, or memory shared by other MPI processes; buffer space may be allocated statically or be changed dynamically; etc. It does not seem feasible to provide a portable mechanism for querying or controlling buffering that would be compatible with all these choices, yet provide meaningful information. (*End of rationale.*)

3.6.1 Model Implementation of Buffered Mode

The model implementation uses the packing and unpacking procedures described in Section 5.2 and the nonblocking communication procedures described in Section 3.7.

We assume that a circular queue of pending message entries (PME) is maintained. Each entry contains a communication request handle that identifies a pending nonblocking send, a pointer to the next entry and the packed message data. The entries are stored in successive locations in the buffer. Free space is available between the queue tail and the queue head.

A buffered send call results in the execution of the following algorithm:

- Traverse sequentially the PME queue from head towards the tail, deleting all entries for communication operations that have completed, up to the first entry with an uncompleted request; update queue head to point to that entry.
- Compute the number of bytes, n , needed to store an entry for the new message. An upper bound on n can be computed as follows: A call to the function

`MPI_PACK_SIZE`(count, datatype, comm, size), with the count, datatype and comm arguments used in the `MPI_BSEND` call, returns an upper bound on the amount of space needed to buffer the message data (see Section 5.2). The MPI constant `MPI_BSEND_OVERHEAD` provides an upper bound on the additional space consumed by the entry (e.g., for pointers or *envelope* information).

- Find the next contiguous empty space of n bytes in buffer (space following queue tail, or space at start of buffer if queue tail is too close to end of buffer). If space is not found then raise buffer overflow error.
- Append to end of PME queue in contiguous space the new entry that contains request handle, next pointer and packed message data; `MPI_PACK` is used to pack data.
- Post nonblocking send (standard mode) for packed data.
- Return

3.7 Nonblocking Communication

Nonblocking communication is important both for reasons of correctness and performance. For complex communication patterns, the use of only blocking communication (without buffering) is difficult because the programmer must ensure that each send is matched with a receive in an order that avoids *deadlock*. For communication patterns that are determined only at run time, this is even more difficult. Nonblocking communication can be used to avoid this problem, allowing programmers to express complex and possibly dynamic communication patterns without needing to ensure that all sends and receives are issued in an order that prevents deadlock (see Section 3.5 and the discussion of “safe” programs). Nonblocking communication also allows for the *overlap* of communication with different communication operations, e.g., to prevent the unintentional *serialization* of such operations, and for the *overlap* of communication with computation. Whether an implementation is able to accomplish an effective (from a performance standpoint) overlap of operations depends on the implementation itself and the system on which the implementation is running. Using nonblocking operations *permits* an implementation to overlap communication with computation, but does not require it to do so.

A nonblocking **send start** call *initiates* the send operation, but does not complete it. The send start call can return before the message was copied out of the send buffer. A separate **send complete** call is needed to complete the communication, i.e., to verify that the data has been copied out of the send buffer. With suitable hardware, the transfer of data out of the sender memory may proceed concurrently with computations done at the sender after the send was initiated and before it completed. Similarly, a nonblocking **receive start** call *initiates* the receive operation, but does not complete it. The call can return before a message is stored into the receive buffer. A separate **receive complete** call is needed to complete the receive operation and verify that the data has been received into the receive buffer. With suitable hardware, the transfer of data into the receiver memory may proceed concurrently with computations done after the receive was initiated and before it completed. The use of nonblocking receives may also avoid system buffering and memory-to-memory copying, as information is provided early on the location of the receive buffer.

Nonblocking send start calls can use the same four modes as blocking sends: *standard*, *buffered*, *synchronous*, and *ready*. These carry the same meaning. Sends of all modes, *ready*

excepted, can be started whether a matching receive has been started or not; a nonblocking **ready** send can be started only if the matching receive is already started. In all cases, the send start call is *local*: it returns immediately, irrespective of the status of other MPI processes. If the call causes some system resource to be exhausted, then it will fail and return an error code. High-quality implementations of MPI should ensure that this happens only in “pathological” cases. That is, an MPI implementation should be able to support a large number of *pending* nonblocking operations.

The send-complete call returns no earlier than when all message data has been copied out of the send buffer. It may carry additional meaning, depending on the send mode.

If the send mode is **synchronous**, then the send-complete call is *nonlocal*; the send can complete only if a matching receive has been started and has been matched with the send. Note that a synchronous mode send may complete, if matched by a nonblocking receive, before the receive complete call occurs. (It can complete as soon as the sender “knows” the transfer will complete, but before the receiver “knows” the transfer will complete.)

If the send mode is **buffered**, then the send-complete call is *local*; the send must complete irrespective of the status of a matching receive. If there is no *pending* receive operation, then the message must be buffered.

If the send mode is **standard**, then the send-complete call can be either *local* or *nonlocal*. If the message is buffered, it is permitted for the send to complete before a matching receive is started. On the other hand, it is permitted for the send not to complete until a matching receive has been started and the message has been copied into the receive buffer.

Nonblocking sends can be matched with blocking receives, and vice-versa.

Advice to users. The completion of a send operation may be delayed for standard mode, and must be delayed for synchronous mode, until a matching receive has been started. The use of nonblocking sends in these two cases allows the sender to proceed ahead of the receiver, so that the computation is more tolerant of fluctuations in the speeds of the two MPI processes.

Nonblocking sends in the buffered and ready modes have a more limited impact, e.g., the blocking version of buffered send is capable of completing regardless of when a matching receive call is made. However, separating the start from the completion of these sends still gives some opportunity for optimization within the MPI library. For example, starting a buffered send gives an implementation more flexibility in determining if and how the message is buffered. There are also advantages for both nonblocking buffered and ready modes when data copying can be done concurrently with computation.

The message-passing model implies that communication is initiated by the sender. The communication will generally have lower overhead if a receive is already *started* when the sender initiates the communication (data can be moved directly to the receive buffer, and there is no need to queue a pending send request). However, a receive operation can complete only after the matching send has *started*. The use of nonblocking receives allows one to achieve lower communication overheads without blocking the receiver while it waits for the send. (*End of advice to users.*)

3.7.1 Communication Request Objects

Nonblocking communication operations use opaque **request** objects to identify communication operations and match the operation that initiates the communication with the

operation that terminates it. These are system objects that are accessed via a handle. A request object identifies various properties of a communication operation, such as the send mode, the communication buffer that is associated with it, its context, the tag and destination arguments to be used for a send, or the tag and source arguments to be used for a receive. In addition, this object stores information about the status of the *pending* communication operation.

3.7.2 Communication Initiation

For the functions defined in this section, we use the same naming conventions as for blocking communication: a prefix of B, S, or R is used for *buffered*, *synchronous*, or *ready* mode. In addition, for these functions a prefix of I (for *immediate* and *incomplete*) indicates that the call is nonblocking.

MPI_ISEND(buf, count, datatype, dest, tag, comm, request)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of each send buffer element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Isend(const void *buf, int count, MPI_Datatype datatype, int dest,
             int tag, MPI_Comm comm, MPI_Request *request)

int MPI_Isend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
               int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Isend(buf, count, datatype, dest, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, dest, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Isend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
```

```

1      TYPE(MPI_Request), INTENT(OUT) :: request
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  Fortran binding
5  MPI_ISEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
6      <type> BUF(*)
7      INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
8      Start a standard mode nonblocking send.
9
10
11 MPI_IBSEND(buf, count, datatype, dest, tag, comm, request)
12     IN      buf                initial address of send buffer (choice)
13     IN      count              number of elements in send buffer (nonnegative
14                                integer)
15     IN      datatype           datatype of each send buffer element (handle)
16     IN      dest               rank of destination (integer)
17     IN      tag                message tag (integer)
18     IN      comm               communicator (handle)
19     IN      request            communication request (handle)
20
21
22 C binding
23 int MPI_Ibsend(const void *buf, int count, MPI_Datatype datatype, int dest,
24               int tag, MPI_Comm comm, MPI_Request *request)
25
26 int MPI_Ibsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
27                 int dest, int tag, MPI_Comm comm, MPI_Request *request)
28
29 Fortran 2008 binding
30 MPI_Ibsend(buf, count, datatype, dest, tag, comm, request, ierror)
31     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
32     INTEGER, INTENT(IN) :: count, dest, tag
33     TYPE(MPI_Datatype), INTENT(IN) :: datatype
34     TYPE(MPI_Comm), INTENT(IN) :: comm
35     TYPE(MPI_Request), INTENT(OUT) :: request
36     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Ibsend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
39     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
40     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
41     TYPE(MPI_Datatype), INTENT(IN) :: datatype
42     INTEGER, INTENT(IN) :: dest, tag
43     TYPE(MPI_Comm), INTENT(IN) :: comm
44     TYPE(MPI_Request), INTENT(OUT) :: request
45     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
46
47 Fortran binding
48 MPI_IBSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
49     <type> BUF(*)
50     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR

```

Start a buffered mode nonblocking send.

MPI_ISSEND(buf, count, datatype, dest, tag, comm, request)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of each send buffer element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Issend(const void *buf, int count, MPI_Datatype datatype, int dest,
               int tag, MPI_Comm comm, MPI_Request *request)

int MPI_Issend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                  int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Issend(buf, count, datatype, dest, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, dest, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Issend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ISSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
  <type> BUF(*)
  INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
```

Start a synchronous mode nonblocking send.

```

1 MPI_IRSEND(buf, count, datatype, dest, tag, comm, request)
2     IN      buf                initial address of send buffer (choice)
3
4     IN      count              number of elements in send buffer (nonnegative
5                                integer)
6     IN      datatype           datatype of each send buffer element (handle)
7     IN      dest               rank of destination (integer)
8     IN      tag                message tag (integer)
9     IN      comm               communicator (handle)
10    IN      comm               communicator (handle)
11    OUT     request             communication request (handle)
12

```

C binding

```

14 int MPI_Irsend(const void *buf, int count, MPI_Datatype datatype, int dest,
15               int tag, MPI_Comm comm, MPI_Request *request)
16
17 int MPI_Irsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
18                 int dest, int tag, MPI_Comm comm, MPI_Request *request)
19

```

Fortran 2008 binding

```

20 MPI_Irsend(buf, count, datatype, dest, tag, comm, request, ierror)
21     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
22     INTEGER, INTENT(IN) :: count, dest, tag
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     TYPE(MPI_Comm), INTENT(IN) :: comm
25     TYPE(MPI_Request), INTENT(OUT) :: request
26     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
27
28 MPI_Irsend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
29     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
30     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
31     TYPE(MPI_Datatype), INTENT(IN) :: datatype
32     INTEGER, INTENT(IN) :: dest, tag
33     TYPE(MPI_Comm), INTENT(IN) :: comm
34     TYPE(MPI_Request), INTENT(OUT) :: request
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36

```

Fortran binding

```

37 MPI_IRSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
38     <type> BUF(*)
39     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
40
41     Start a ready mode nonblocking send.
42

```

```

43 MPI_IRECV(buf, count, datatype, source, tag, comm, request)
44
45     OUT     buf                initial address of receive buffer (choice)
46
47     IN      count              number of elements in receive buffer (nonnegative
48                                integer)
49     IN      datatype           datatype of each receive buffer element (handle)

```


IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)	1
IN	tag	message tag or <code>MPI_ANY_TAG</code> (integer)	2
IN	comm	communicator (handle)	3
OUT	request	communication request (handle)	4

C binding

```
int MPI_Irecv(void *buf, int count, MPI_Datatype datatype, int source, int tag,
             MPI_Comm comm, MPI_Request *request)
```

```
int MPI_Irecv_c(void *buf, MPI_Count count, MPI_Datatype datatype, int source,
               int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Irecv(buf, count, datatype, source, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, source, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Irecv(buf, count, datatype, source, tag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: source, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_IRECV(BUF, COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR)
  <type> BUF(*)
  INTEGER COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR
```

Start a nonblocking receive.

```
MPI_ISENDRECV(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount, recvtpe,
              source, recvtag, comm, request)
```

IN	sendbuf	initial address of send buffer (choice)	1
IN	sendcount	number of elements in send buffer (nonnegative integer)	2
IN	sendtype	datatype of each send buffer element (handle)	3
IN	dest	rank of destination (integer)	4
IN	sendtag	send tag (integer)	5
OUT	recvbuf	initial address of receive buffer (choice)	6

1	IN	recvcount	number of elements in receive buffer (nonnegative integer)
2			
3	IN	recvtype	datatype of each receive buffer element (handle)
4	IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)
5			
6	IN	recvtag	receive tag or <code>MPI_ANY_TAG</code> (integer)
7	IN	comm	communicator (handle)
8			
9	OUT	request	communication request (handle)

C binding

```
int MPI_Isendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                 int dest, int sendtag, void *recvbuf, int recvcount,
                 MPI_Datatype recvtype, int source, int recvtag, MPI_Comm comm,
                 MPI_Request *request)
```

```
int MPI_Isendrecv_c(const void *sendbuf, MPI_Count sendcount,
                   MPI_Datatype sendtype, int dest, int sendtag, void *recvbuf,
                   MPI_Count recvcount, MPI_Datatype recvtype, int source,
                   int recvtag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Isendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
              recvtype, source, recvtag, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  INTEGER, INTENT(IN) :: sendcount, dest, sendtag, recvcount, source, recvtag
  TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Isendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
              recvtype, source, recvtag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
  TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
  INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ISENDRECV(SENDBUF, SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVBUF, RECVCOUNT,
              RECVTYPE, SOURCE, RECVTAG, COMM, REQUEST, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVCOUNT, RECVTYPE, SOURCE,
  RECVTAG, COMM, REQUEST, IERROR
```

Initiate a nonblocking communication request for a *send* and *receive* operation.

MPI_ISENDRECV_REPLACE(buf, count, datatype, dest, sendtag, source, recvtag, comm,		1
request)		2
INOUT	buf	3
	initial address of send and receive buffer (choice)	4
IN	count	5
	number of elements in send and receive buffer	6
	(nonnegative integer)	7
IN	datatype	8
	type of elements in send and receive buffer (handle)	9
IN	dest	10
	rank of destination (integer)	11
IN	sendtag	12
	send message tag (integer)	13
IN	source	14
	rank of source or <code>MPI_ANY_SOURCE</code> (integer)	15
IN	recvtag	16
	receive message tag or <code>MPI_ANY_TAG</code> (integer)	17
IN	comm	18
	communicator (handle)	19
OUT	request	20
	communication request (handle)	21

C binding

```

int MPI_Isendrecv_replace(void *buf, int count, MPI_Datatype datatype,
                          int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
                          MPI_Request *request)
int MPI_Isendrecv_replace_c(void *buf, MPI_Count count, MPI_Datatype datatype,
                          int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
                          MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Isendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                      comm, request, ierror)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count, dest, sendtag, source, recvtag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Isendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                      comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ISENDRECV_REPLACE(BUF, COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG,
                      COMM, REQUEST, IERROR)
<type> BUF(*)
INTEGER COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG, COMM, REQUEST,
IERROR

```

Initiate a nonblocking communication request for a *send* and *receive* operation. The same buffer is used both for the send and for the receive, so that the message sent is replaced by the message received.

These calls allocate a communication request object and associate it with the request handle (the argument `request`). The request can be used later to query the status of the communication or wait for its completion.

A nonblocking send call indicates that the system may start copying data out of the send buffer. The sender should not modify any part of the send buffer after a nonblocking send operation is called, until the send completes.

A nonblocking receive call indicates that the system may start writing data into the receive buffer. The receiver should not access any part of the receive buffer after a nonblocking receive operation is called, until the receive completes.

Advice to users. To prevent problems with the argument copying and register optimization done by Fortran compilers, please note the hints in Sections 21.1.10–21.1.20. (*End of advice to users.*)

3.7.3 Communication Completion

The functions `MPI_WAIT` and `MPI_TEST` are used to complete a nonblocking communication. The *completion* of a send operation indicates that the sender is now free to update the send buffer (the send operation itself leaves the content of the send buffer unchanged). It does not indicate that the message has been received, rather, it may have been buffered by the communication subsystem. However, if a *synchronous mode send* was used, the *completion* of the send operation indicates that a matching receive was *initiated*, and that the message will eventually be received by this matching receive.

The *completion* of a receive operation indicates that the receive buffer contains the received message, the receiver is now free to access it, and that the status object is set. It does not indicate that the matching send operation has *completed* (but indicates, of course, that the send was *initiated*).

We shall use the following terminology: A **null handle** is a handle with value `MPI_REQUEST_NULL`. A *persistent communication request* and the handle to it are **inactive** if the request is not associated with any ongoing communication (see Section 3.9). A handle is **active** if it is neither *null* nor *inactive*. An **empty** status is a status that is set to return `tag = MPI_ANY_TAG`, `source = MPI_ANY_SOURCE`, `error = MPI_SUCCESS`, and is also internally configured so that calls to `MPI_GET_COUNT` and `MPI_GET_ELEMENTS` return `count = 0` and `MPI_TEST_CANCELLED` returns false. We set a status variable to *empty* when the value returned by it is not significant. Status is set in this way so as to prevent errors due to accesses of stale information.

The fields in a **status** object returned by a call to `MPI_WAIT`, `MPI_TEST`, or any of the other derived functions (`MPI_{TEST|WAIT}{ALL|SOME|ANY}`), where the `request` corresponds to a send call, are undefined, with two exceptions: The error status field will contain valid information if the wait or test call returned with `MPI_ERR_IN_STATUS`; and the returned status can be queried by the call `MPI_TEST_CANCELLED`.

Error codes belonging to the error class `MPI_ERR_IN_STATUS` should be returned only by the MPI completion functions that take arrays of `MPI_Status`. For the functions that take a single `MPI_Status` argument, the error code is returned by the function, and the value of the `MPI_ERROR` field in the `MPI_Status` argument is undefined (see 3.2.5).

`MPI_WAIT(request, status)`

INOUT	request	request (handle)
OUT	status	status object (status)

C binding

`int MPI_Wait(MPI_Request *request, MPI_Status *status)`

Fortran 2008 binding

```
MPI_Wait(request, status, ierror)
  TYPE(MPI_Request), INTENT(INOUT) :: request
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WAIT(REQUEST, STATUS, IERROR)
  INTEGER REQUEST, STATUS(MPI_STATUS_SIZE), IERROR
```

A call to `MPI_WAIT` returns when the operation identified by `request` is *complete*. If the request is an *active persistent communication request*, it is marked *inactive*. Any other type of request is deallocated and the request handle is set to `MPI_REQUEST_NULL`. `MPI_WAIT` is in general a *nonlocal* procedure. When the operation represented by the `request` is *enabled* then a call to `MPI_WAIT` is a *local* procedure call.

The call returns, in `status`, information on the completed operation. The content of the status object for a receive operation can be accessed as described in Section 3.2.5. The status object for a send operation may be queried by a call to `MPI_TEST_CANCELLED` (see Section 3.8).

One is allowed to call `MPI_WAIT` with a *null* or *inactive* `request` argument. In this case the procedure returns immediately with *empty* `status`.

Advice to users. Successful return of `MPI_WAIT` after a `MPI_IBSEND` implies that the user send buffer can be reused—i.e., data has been sent out or copied into a buffer attached with `MPI_BUFFER_ATTACH`, `MPI_COMM_ATTACH_BUFFER` or `MPI_SESSION_ATTACH_BUFFER`. Further, at this point, we can no longer *cancel* the send (see Section 3.8). If a matching receive is never *started*, then the buffer cannot be freed. This runs somewhat counter to the stated goal of `MPI_CANCEL` (always being able to free program space that was committed to the communication subsystem). (*End of advice to users.*)

Advice to implementors. In a multithreaded environment, a call to `MPI_WAIT` should block only the calling thread, allowing the thread scheduler to schedule another thread for execution. (*End of advice to implementors.*)

`MPI_TEST(request, flag, status)`

INOUT	request	communication request (handle)
OUT	flag	true if operation completed (logical)
OUT	status	status object (status)

C binding

```
int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Test(request, flag, status, ierror)
    TYPE(MPI_Request), INTENT(INOUT) :: request
    LOGICAL, INTENT(OUT) :: flag
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TEST(REQUEST, FLAG, STATUS, IERROR)
    INTEGER REQUEST, STATUS(MPI_STATUS_SIZE), IERROR
    LOGICAL FLAG
```

A call to `MPI_TEST` returns `flag = true` if the operation identified by `request` is *complete*. In such a case, the status object is set to contain information on the completed operation. If the request is an *active persistent communication request*, it is marked as *inactive*. Any other type of request is deallocated and the request handle is set to `MPI_REQUEST_NULL`. The call returns `flag = false` if the operation identified by `request` is not complete. In this case, the value of the status object is undefined. `MPI_TEST` is a *local* procedure.

The return status object for a receive operation carries information that can be accessed as described in Section 3.2.5. The status object for a send operation carries information that can be accessed by a call to `MPI_TEST_CANCELLED` (see Section 3.8).

One is allowed to call `MPI_TEST` with a *null* or *inactive* request argument. In such a case the procedure returns with `flag = true` and *empty* status.

The procedures `MPI_WAIT` and `MPI_TEST` can be used to complete any request-based nonblocking or persistent operation.

Advice to users. The use of the nonblocking `MPI_TEST` call allows the user to schedule alternative activities within a single thread of execution. An event-driven thread scheduler can be emulated with periodic calls to `MPI_TEST`. (*End of advice to users.*)

Example 3.13. Simple usage of nonblocking operations and `MPI_WAIT`.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
    CALL MPI_ISEND(a(1), 10, MPI_REAL, 1, tag, comm, request, ierr)
    ! **** do some computation to mask latency ****
    CALL MPI_WAIT(request, status, ierr)
ELSE IF (rank .EQ. 1) THEN
    CALL MPI_IRECV(a(1), 15, MPI_REAL, 0, tag, comm, request, ierr)
    ! **** do some computation to mask latency ****
    CALL MPI_WAIT(request, status, ierr)
END IF
```

A request object can be *freed* using the following MPI procedure.

`MPI_REQUEST_FREE(request)`

INOUT request communication request (handle)

C binding

```
int MPI_Request_free(MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Request_free(request, ierror)
  TYPE(MPI_Request), INTENT(INOUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REQUEST_FREE(REQUEST, IERROR)
  INTEGER REQUEST, IERROR
```

`MPI_REQUEST_FREE` is a *local* procedure. Upon successful return, `MPI_REQUEST_FREE` sets `request` to `MPI_REQUEST_NULL`. For an *inactive* request representing any type of MPI operation, `MPI_REQUEST_FREE` shall do the *freeing stage* of the associated operation during its execution.

For a request representing a *nonblocking* point-to-point or a *persistent* point-to-point operation, it is permitted (although strongly discouraged) to call `MPI_REQUEST_FREE` when the request is *active*. In this special case, `MPI_REQUEST_FREE` will only mark the request for freeing and MPI will actually do the *freeing stage* of the operation associated with the request later.

The use of this procedure for generalized requests is described in Section 14.2.

Calling `MPI_REQUEST_FREE` with an *active* request representing any other type of MPI operation (e.g., any partitioned operation (see Chapter 4), any collective operation (see Chapter 6), any I/O operation (see Chapter 15), or any request-based RMA operation (see Chapter 12)) is *erroneous*.

Rationale. For point-to-point operations, the `MPI_REQUEST_FREE` mechanism is provided for reasons of performance and convenience on the sending side. (*End of rationale.*)

Advice to users. Once a request is freed by a call to `MPI_REQUEST_FREE`, it is not possible to check for the successful completion of the associated communication with calls to `MPI_WAIT` or `MPI_TEST`. Also, if an error occurs subsequently during the communication, an error code cannot be returned to the user—such an error must be treated as fatal. An active receive request should never be freed as the receiver will have no way to verify that the receive has completed and the receive buffer can be reused. (*End of advice to users.*)

Example 3.14. An example using `MPI_REQUEST_FREE`.

```
CALL MPI_COMM_RANK(MPI_COMM_WORLD, rank, ierr)
IF (rank .EQ. 0) THEN
  DO i=1,n
    CALL MPI_ISEND(outval, 1, MPI_REAL, 1, 0, MPI_COMM_WORLD, req, ierr)
    CALL MPI_REQUEST_FREE(req, ierr)
    CALL MPI_Irecv(inval, 1, MPI_REAL, 1, 0, MPI_COMM_WORLD, req, ierr)
    CALL MPI_WAIT(req, status, ierr)
```



```

1  END DO
2  ELSE IF (rank .EQ. 1) THEN
3    CALL MPI_Irecv(inval, 1, MPI_REAL, 0, 0, MPI_COMM_WORLD, req, ierr)
4    CALL MPI_WAIT(req, status, ierr)
5    DO I=1,n-1
6      CALL MPI_Isend(outval, 1, MPI_REAL, 0, 0, MPI_COMM_WORLD, req, ierr)
7      CALL MPI_REQUEST_FREE(req, ierr)
8      CALL MPI_Irecv(inval, 1, MPI_REAL, 0, 0, MPI_COMM_WORLD, req, ierr)
9      CALL MPI_WAIT(req, status, ierr)
10   END DO
11   CALL MPI_Isend(outval, 1, MPI_REAL, 0, 0, MPI_COMM_WORLD, req, ierr)
12   CALL MPI_WAIT(req, status, ierr)
13 END IF

```

3.7.4 Semantics of Nonblocking Communication Operations

The semantics of nonblocking communication operations are defined by suitably extending the definitions in Section 3.5.

Order. Nonblocking communication operations are **ordered** according to the execution order of the calls that *initiate* the communication. The **nonovertaking** requirement of Section 3.5 is extended to nonblocking communication, with this definition of order being used.

Example 3.15. Message ordering for nonblocking operations.

```

24 CALL MPI_COMM_RANK(comm, rank, ierr)
25 IF (rank .EQ. 0) THEN
26   CALL MPI_Isend(a, 1, MPI_REAL, 1, 0, comm, r1, ierr)
27   CALL MPI_Isend(b, 1, MPI_REAL, 1, 0, comm, r2, ierr)
28 ELSE IF (rank .EQ. 1) THEN
29   CALL MPI_Irecv(a, 1, MPI_REAL, 0, MPI_ANY_TAG, comm, r1, ierr)
30   CALL MPI_Irecv(b, 1, MPI_REAL, 0, 0, comm, r2, ierr)
31 END IF
32 CALL MPI_WAIT(r1, status, ierr)
33 CALL MPI_WAIT(r2, status, ierr)

```

The first send will match the first receive, even if both messages are sent before either receive is executed.

Progress. A call to `MPI_WAIT` that *completes* a receive will eventually terminate and return if a matching send has been *started*, unless the send is satisfied by another receive. In particular, if the matching send is *nonblocking*, then the receive should *complete* even if no call is executed by the sender to *complete* the send. Similarly, a call to `MPI_WAIT` that *completes* a send will eventually return if a matching receive has been *started*, unless the receive is satisfied by another send, and even if no call is executed to *complete* the receive.

Example 3.16. An illustration of progress semantics.

```

45 CALL MPI_COMM_RANK(comm, rank, ierr)
46 IF (rank .EQ. 0) THEN
47   CALL MPI_Ssend(a, 1, MPI_REAL, 1, 0, comm, ierr)

```



```

    CALL MPI_SEND(b, 1, MPI_REAL, 1, 1, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
    CALL MPI_IRECV(a, 1, MPI_REAL, 0, 0, comm, r, ierr)
    CALL MPI_RECV(b, 1, MPI_REAL, 0, 1, comm, status, ierr)
    CALL MPI_WAIT(r, status, ierr)
END IF

```

This code should not deadlock in a correct MPI implementation. The first synchronous send must complete once the matching (nonblocking) receive is *started*, even though the completing wait call has not yet been reached. Thus, the sending MPI process will continue and execute the second send procedure, allowing the receiving MPI process to complete execution.

If an `MPI_TEST` that *completes* a receive is repeatedly called with the same arguments, and a matching send has been *started*, then the call will eventually return `flag = true`, unless the send is satisfied by another receive. If an `MPI_TEST` that *completes* a send is repeatedly called with the same arguments, and a matching receive has been *started*, then the call will eventually return `flag = true`, unless the receive is satisfied by another send. See also Section 2.9 on *progress*.

3.7.5 Multiple Completions

It is convenient to be able to wait for the *completion* of any, some, or all the operations in a list, rather than having to wait for a specific message. A call to `MPI_WAITANY` or `MPI_TESTANY` can be used to wait for the *completion* of one out of several operations. A call to `MPI_WAITALL` or `MPI_TESTALL` can be used to wait for all *pending* operations in a list. A call to `MPI_WAITSOME` or `MPI_TESTSOME` can be used to *complete* all enabled operations in a list.

`MPI_WAITANY(count, array_of_requests, index, status)`

IN	count	list length (nonnegative integer)
INOUT	array_of_requests	array of requests (array of handles)
OUT	index	index of handle for operation that completed (integer)
OUT	status	status object (status)

C binding

```

int MPI_Waitany(int count, MPI_Request array_of_requests[], int *index,
                MPI_Status *status)

```

Fortran 2008 binding

```

MPI_Waitany(count, array_of_requests, index, status, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
  INTEGER, INTENT(OUT) :: index
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WAITANY(COUNT, ARRAY_OF_REQUESTS, INDEX, STATUS, IERROR)
  INTEGER COUNT, ARRAY_OF_REQUESTS(*), INDEX, STATUS(MPI_STATUS_SIZE), IERROR

```

Does not return until one of the operations associated with the *active* requests in the array has *completed*. If more than one operation is enabled and can *complete*, one is arbitrarily chosen. Returns in *index* the index of that request in the array and returns in *status* the status of the completing operation. (The array is indexed from zero in C, and from one in Fortran.) If the request is an *active persistent communication request*, it is marked *inactive*. Any other type of request is deallocated and the request handle is set to `MPI_REQUEST_NULL`.

The `array_of_requests` list may contain *null* or *inactive* handles. If the list contains no *active* handles (list has length zero or all entries are *null* or *inactive*), then the call returns immediately with `index = MPI_UNDEFINED`, and an *empty* status.

The execution of `MPI_WAITANY` with an array containing multiple entries has the same effect as the execution of `MPI_WAIT` with the array entry indicated by the output value of `index` (unless the output value of `index` is `MPI_UNDEFINED`). `MPI_WAITANY` with an array containing one *active* entry is equivalent to `MPI_WAIT`.

```

MPI_TESTANY(count, array_of_requests, index, flag, status)

```

IN	count	list length (nonnegative integer)
INOUT	array_of_requests	array of requests (array of handles)
OUT	index	index of operation that completed or <code>MPI_UNDEFINED</code> if none completed (integer)
OUT	flag	true if one of the operations is complete (logical)
OUT	status	status object (status)

C binding

```

int MPI_Testany(int count, MPI_Request array_of_requests[], int *index,
  int *flag, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_Testany(count, array_of_requests, index, flag, status, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
  INTEGER, INTENT(OUT) :: index
  LOGICAL, INTENT(OUT) :: flag
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TESTANY(COUNT, ARRAY_OF_REQUESTS, INDEX, FLAG, STATUS, IERROR)
  INTEGER COUNT, ARRAY_OF_REQUESTS(*), INDEX, STATUS(MPI_STATUS_SIZE), IERROR
  LOGICAL FLAG

```

Tests for *completion* of either one or none of the operations associated with *active* handles. In the former case, it returns `flag = true`, returns in `index` the index of this request in the array, and returns in `status` the status of that operation. If the request is an *active*

persistent communication request, it is marked as *inactive*. Any other type of request is deallocated and the handle is set to `MPI_REQUEST_NULL`. (The array is indexed from zero in C, and from one in Fortran.) In the latter case (no operation *completed*), it returns `flag = false`, returns a value of `MPI_UNDEFINED` in `index` and `status` is undefined.

The array may contain *null* or *inactive* handles. If the array contains no *active* handles then the call returns *immediately* with `flag = true`, `index = MPI_UNDEFINED`, and an *empty status*.

If the array of requests contains *active* handles then the execution of `MPI_TESTANY` has the same effect as the execution of `MPI_TEST` with each of the *active* handles in the array in some arbitrary order, until one call returns `flag = true`, or all return `flag = false`. In the former case, `index` is set to indicate which array element returned `flag = true` and in the latter case, it is set to `MPI_UNDEFINED`. `MPI_TESTANY` with an array containing one *active* entry is equivalent to `MPI_TEST`.

`MPI_WAITALL(count, array_of_requests, array_of_statuses)`

IN	count	list length (nonnegative integer)
INOUT	array_of_requests	array of requests (array of handles)
OUT	array_of_statuses	array of status objects (array of status)

C binding

```
int MPI_Waitall(int count, MPI_Request array_of_requests[],
               MPI_Status array_of_statuses[])
```

Fortran 2008 binding

```
MPI_Waitall(count, array_of_requests, array_of_statuses, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
  TYPE(MPI_Status) :: array_of_statuses(*)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WAITALL(COUNT, ARRAY_OF_REQUESTS, ARRAY_OF_STATUSES, IERROR)
  INTEGER COUNT, ARRAY_OF_REQUESTS(*), ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *),
  IERROR
```

Does not return until all communication operations associated with *active* handles in the list *complete*, and returns the status of all these operations (this includes the case where no handle in the list is *active*). Both arrays have the same number of valid entries. The *i*-th entry in `array_of_statuses` is set to the return status of the *i*-th operation. *Active persistent requests* are marked *inactive*. Requests of any other type are deallocated and the corresponding handles in the array are set to `MPI_REQUEST_NULL`. The list may contain *null* or *inactive* handles. The call sets to *empty* the status of each such entry.

The error-free execution of `MPI_WAITALL` has the same effect as the execution of `MPI_WAIT` for each of the array elements in some arbitrary order. `MPI_WAITALL` with an array of length one is equivalent to `MPI_WAIT`.

When one or more of the communication operations *completed* by a call to `MPI_WAITALL` fail, it is desirable to return specific information on each communication. The function `MPI_WAITALL` will return in such case the error code `MPI_ERR_IN_STATUS`

and will set the error field of each status to a specific error code. This code will be `MPI_SUCCESS`, if the specific communication *completed*; it will be another specific error code, if it failed; or it can be `MPI_ERR_PENDING` if it has neither failed nor *completed*. The function `MPI_WAITALL` will return `MPI_SUCCESS` if no request had an error, or will return another error code if it failed for other reasons (such as invalid arguments). In such cases, it will not update the error fields of the statuses.

Rationale. This design streamlines error handling in the application. The application code need only test the (single) function result to determine if an error has occurred. It needs to check each individual status only when an error occurred. (*End of rationale.*)

`MPI_TESTALL(count, array_of_requests, flag, array_of_statuses)`

IN	count	list length (nonnegative integer)
INOUT	array_of_requests	array of requests (array of handles)
OUT	flag	true if all of the operations are complete (logical)
OUT	array_of_statuses	array of status objects (array of status)

C binding

```
int MPI_Testall(int count, MPI_Request array_of_requests[], int *flag,
               MPI_Status array_of_statuses[])
```

Fortran 2008 binding

```
MPI_Testall(count, array_of_requests, flag, array_of_statuses, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
  LOGICAL, INTENT(OUT) :: flag
  TYPE(MPI_Status) :: array_of_statuses(*)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TESTALL(COUNT, ARRAY_OF_REQUESTS, FLAG, ARRAY_OF_STATUSES, IERROR)
  INTEGER COUNT, ARRAY_OF_REQUESTS(*), ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *),
  IERROR
  LOGICAL FLAG
```

Returns `flag = true` if all communication operations associated with *active* handles in the array have *completed* (this includes the case where no handle in the list is *active*). In this case, each status entry that corresponds to an *active* request is set to the status of the corresponding operation. *Active persistent requests* are marked *inactive*. Requests of any other type are deallocated and the corresponding handles in the array are set to `MPI_REQUEST_NULL`. Each status entry that corresponds to a *null* or *inactive* handle is set to *empty*.

Otherwise, `flag = false` is returned, no request is modified and the values of the status entries are undefined. This is a *local* procedure.

Errors that occurred during the execution of `MPI_TESTALL` are handled in the same manner as errors in `MPI_WAITALL`.

MPI_WAITSSOME(incount, array_of_requests, outcount, array_of_indices,				1
array_of_statuses)				2
IN	incount	length of array_of_requests (nonnegative integer)		3
INOUT	array_of_requests	array of requests (array of handles)		4
OUT	outcount	number of completed requests (integer)		5
OUT	array_of_indices	array of indices of operations that completed (array		6
		of integers)		7
OUT	array_of_statuses	array of status objects for operations that completed		8
		(array of status)		9
				10
				11

C binding

```
int MPI_Waitssome(int incount, MPI_Request array_of_requests[], int *outcount,
                  int array_of_indices[], MPI_Status array_of_statuses[])
```

Fortran 2008 binding

```
MPI_Waitssome(incount, array_of_requests, outcount, array_of_indices,
              array_of_statuses, ierror)
INTEGER, INTENT(IN) :: incount
TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(incount)
INTEGER, INTENT(OUT) :: outcount, array_of_indices(*)
TYPE(MPI_Status) :: array_of_statuses(*)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WAITSSOME(INCOUNT, ARRAY_OF_REQUESTS, OUTCOUNT, ARRAY_OF_INDICES,
              ARRAY_OF_STATUSES, IERROR)
INTEGER INCOUNT, ARRAY_OF_REQUESTS(*), OUTCOUNT, ARRAY_OF_INDICES(*),
              ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *), IERROR
```

Does not return until at least one of the operations associated with *active* handles in the list have *completed*. Returns in **outcount** the number of requests from the list **array_of_requests** that have *completed*. Returns in the first **outcount** locations of the array **array_of_indices** the indices of these operations (index within the array **array_of_requests**; the array is indexed from zero in C and from one in Fortran). Returns in the first **outcount** locations of the array **array_of_statuses** the status for these *completed* operations. *Completed active persistent requests* are marked as *inactive*. Any other type or request that *completed* is deallocated, and the associated handle is set to **MPI_REQUEST_NULL**.

If the list contains no *active* handles, then the call returns *immediately* with **outcount** = **MPI_UNDEFINED**.

When one or more of the communication operations *completed* by **MPI_WAITSSOME** fails, then it is desirable to return specific information on each communication. The arguments **outcount**, **array_of_indices** and **array_of_statuses** will be adjusted to indicate *completion* of all communication operations that have succeeded or failed. The call will return the error code **MPI_ERR_IN_STATUS** and the error field of each status returned will be set to indicate success or to indicate the specific error that occurred. The call will return **MPI_SUCCESS** if no request resulted in an error, and will return another error code if it failed for other reasons (such as invalid arguments). In such cases, it will not update the error fields of the statuses.

```

1 MPI_TESTSOME(incount, array_of_requests, outcount, array_of_indices,
2             array_of_statuses)
3
4 IN      incount      length of array_of_requests (nonnegative integer)
5 INOUT   array_of_requests  array of requests (array of handles)
6 OUT     outcount     number of completed requests (integer)
7 OUT     array_of_indices  array of indices of operations that completed (array
8                             of integers)
9 OUT     array_of_statuses  array of status objects for operations that completed
10                             (array of status)
11

```

C binding

```

13 int MPI_Testsome(int incount, MPI_Request array_of_requests[], int *outcount,
14                int array_of_indices[], MPI_Status array_of_statuses[])
15

```

Fortran 2008 binding

```

16 MPI_Testsome(incount, array_of_requests, outcount, array_of_indices,
17             array_of_statuses, ierror)
18
19 INTEGER, INTENT(IN) :: incount
20 TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(incount)
21 INTEGER, INTENT(OUT) :: outcount, array_of_indices(*)
22 TYPE(MPI_Status) :: array_of_statuses(*)
23 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24

```

Fortran binding

```

25 MPI_TESTSOME(INCOUNT, ARRAY_OF_REQUESTS, OUTCOUNT, ARRAY_OF_INDICES,
26             ARRAY_OF_STATUSES, IERROR)
27
28 INTEGER INCOUNT, ARRAY_OF_REQUESTS(*), OUTCOUNT, ARRAY_OF_INDICES(*),
29             ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *), IERROR
30

```

This procedure behaves like `MPI_WAITSSOME`, except that it returns *immediately*. If no operation has completed it returns `outcount = 0`. If there is no *active* handle in the list it returns `outcount = MPI_UNDEFINED`.

`MPI_TESTSSOME` is a *local* procedure, which returns *immediately*, whereas `MPI_WAITSSOME` will not return until a communication *completes*, if it was passed a list that contains at least one *active* handle. Both calls fulfill a **fairness requirement**: If a request for a receive repeatedly appears in a list of requests passed to `MPI_WAITSSOME` or `MPI_TESTSSOME`, and a matching send has been *started*, then the receive will eventually succeed, unless the send is satisfied by another receive; and similarly for send requests.

Errors that occur during the execution of `MPI_TESTSSOME` are handled as for `MPI_WAITSSOME`.

Advice to users. The use of `MPI_TESTSSOME` is likely to be more efficient than the use of `MPI_TESTANY`. The former returns information on all *completed* communication operations, with the latter, a new call is required for each communication that completes.

A server with multiple clients can use `MPI_WAITSSOME` so as not to starve any client. Clients send messages to the server with service requests. The server calls `MPI_WAITSSOME` with one receive request for each client, and then handles all receives

that completed. If a call to `MPI_WAITANY` is used instead, then one client could starve while requests from another client always sneak in first. (*End of advice to users.*)

Advice to implementors. `MPI_TESTSOME` should *complete* as many *pending* communication operations of the `array_of_requests` as possible. (*End of advice to implementors.*)

Example 3.17. Client-server code (starvation can occur).

```
CALL MPI_COMM_SIZE(comm, size, ierr)
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .GT. 0) THEN          ! client code
  DO WHILE(.TRUE.)
    CALL MPI_ISEND(a, n, MPI_REAL, 0, tag, comm, request, ierr)
    CALL MPI_WAIT(request, status, ierr)
  END DO
ELSE                            ! rank=0 -- server code
  DO i=1,size-1
    CALL MPI_Irecv(a(1,i), n, MPI_REAL, i, tag, &
                  comm, request_list(i), ierr)
  END DO
  DO WHILE(.TRUE.)
    CALL MPI_WAITANY(size-1, request_list, index, status, ierr)
    CALL DO_SERVICE(a(1,index)) ! handle one message
    CALL MPI_Irecv(a(1, index), n, MPI_REAL, index, tag, &
                  comm, request_list(index), ierr)
  END DO
END IF
```

Example 3.18. Same code, using `MPI_WAITSOME`.

```
CALL MPI_COMM_SIZE(comm, size, ierr)
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .GT. 0) THEN          ! client code
  DO WHILE(.TRUE.)
    CALL MPI_ISEND(a, n, MPI_REAL, 0, tag, comm, request, ierr)
    CALL MPI_WAIT(request, status, ierr)
  END DO
ELSE                            ! rank=0 -- server code
  DO i=1,size-1
    CALL MPI_Irecv(a(1,i), n, MPI_REAL, i, tag, &
                  comm, request_list(i), ierr)
  END DO
  DO WHILE(.TRUE.)
    CALL MPI_WAITSOME(size, request_list, numdone, &
                     indices, statuses, ierr)
    DO i=1,numdone
      CALL DO_SERVICE(a(1, indices(i)))
      CALL MPI_Irecv(a(1, indices(i)), n, MPI_REAL, 0, tag, &
                    comm, request_list(indices(i)), ierr)
    END DO
  END DO
END IF
```


3.7.6 Non-Destructive Test of status

These procedures are useful for accessing the information associated with a request, without *freeing* the request (in case the user is expected to access it later). It allows one to layer libraries more conveniently, since multiple layers of software may access the same *completed* request and extract from it the status information.

MPI_REQUEST_GET_STATUS(request, flag, status)

IN	request	request (handle)
OUT	flag	boolean flag, same as from MPI_TEST (logical)
OUT	status	status object if flag is true (status)

C binding

```
int MPI_Request_get_status(MPI_Request request, int *flag, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Request_get_status(request, flag, status, ierror)
  TYPE(MPI_Request), INTENT(IN) :: request
  LOGICAL, INTENT(OUT) :: flag
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REQUEST_GET_STATUS(REQUEST, FLAG, STATUS, IERROR)
  INTEGER REQUEST, STATUS(MPI_STATUS_SIZE), IERROR
  LOGICAL FLAG
```

Sets `flag = true` if the operation is *complete*, and, if so, returns in `status` the request status. However, unlike `test` or `wait`, it does not deallocate or *inactivate* the request; a subsequent call to `test`, `wait` or `free` must be executed with that request. It sets `flag = false` if the operation is not *complete*.

One is allowed to call **MPI_REQUEST_GET_STATUS** with a *null* or *inactive* request argument. In such a case the procedure returns with `flag = true` and *empty* status.

The *progress* rule for **MPI_TEST**, as described in Section 3.7.4, also applies to **MPI_REQUEST_GET_STATUS**.

MPI_REQUEST_GET_STATUS_ANY(count, array_of_requests, index, flag, status)

IN	count	list length (nonnegative integer)
IN	array_of_requests	array of requests (array of handles)
OUT	index	index of operation that completed or MPI_UNDEFINED if none completed (integer)
OUT	flag	true if one of the operations is complete (logical)
OUT	status	status object if flag is true (status)

C binding

```
int MPI_Request_get_status_any(int count,
```



```
const MPI_Request array_of_requests[], int *index, int *flag,
MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Request_get_status_any(count, array_of_requests, index, flag, status,
ierror)
```

```
INTEGER, INTENT(IN) :: count
TYPE(MPI_Request), INTENT(IN) :: array_of_requests(count)
INTEGER, INTENT(OUT) :: index
LOGICAL, INTENT(OUT) :: flag
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REQUEST_GET_STATUS_ANY(COUNT, ARRAY_OF_REQUESTS, INDEX, FLAG, STATUS,
IERROR)
INTEGER COUNT, ARRAY_OF_REQUESTS(*), INDEX, STATUS(MPI_STATUS_SIZE), IERROR
LOGICAL FLAG
```

Tests for *completion* of either one or none of the operations associated with *active* handles. In the former case, it returns `flag = true`, returns in `index` the index of this request in the array, and returns in `status` the status of that operation. (The array is indexed from zero in C, and from one in Fortran.) In the latter case (no operation *completed*), it returns `flag = false`, returns a value of `MPI_UNDEFINED` in `index` and `status` is undefined.

The array may contain *null* or inactive handles. If the array contains no *active* handles then the call returns *immediately* with `flag = true`, `index = MPI_UNDEFINED`, and an *empty* status.

If the array of requests contains active handles then the execution of `MPI_REQUEST_GET_STATUS_ANY` has the same effect as the execution of `MPI_REQUEST_GET_STATUS` with each of the active array elements in some arbitrary order, until one call returns `flag = true`, or all return `flag = false`. In the former case, `index` is set to indicate which array element returned `flag = true` and in the latter case, it is set to `MPI_UNDEFINED`. `MPI_REQUEST_GET_STATUS_ANY` with an array containing one request is equivalent to `MPI_REQUEST_GET_STATUS`.

```
MPI_REQUEST_GET_STATUS_ALL(count, array_of_requests, flag, array_of_statuses)
```

IN	count	list length (nonnegative integer)
IN	array_of_requests	array of requests (array of handles)
OUT	flag	true if all of the operations are complete (logical)
OUT	array_of_statuses	array of status objects (array of status)

C binding

```
int MPI_Request_get_status_all(int count,
const MPI_Request array_of_requests[], int *flag,
MPI_Status array_of_statuses[])
```

Fortran 2008 binding

```

MPI_Request_get_status_all(count, array_of_requests, flag, array_of_statuses,
                           ierror)
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Request), INTENT(IN) :: array_of_requests(count)
    LOGICAL, INTENT(OUT) :: flag
    TYPE(MPI_Status) :: array_of_statuses(*)
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_REQUEST_GET_STATUS_ALL(COUNT, ARRAY_OF_REQUESTS, FLAG, ARRAY_OF_STATUSES,
                           IERROR)
    INTEGER COUNT, ARRAY_OF_REQUESTS(*), ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *),
    IERROR
    LOGICAL FLAG

```

MPI_REQUEST_GET_STATUS_ALL returns `flag = true` if all communication operations associated with *active* handles in the array have *completed* (this includes the case where all handles in the list are *inactive* or **MPI_REQUEST_NULL**). In this case, each status entry that corresponds to an *active* request is set to the status of the corresponding operation. Unlike `test` or `wait`, it does not deallocate or *inactivate* the requests; a subsequent call to `test`, `wait` or `free` should be executed with each of those requests.

Each status entry that corresponds to a *null* or *inactive* handle is set to *empty*.

Otherwise, `flag = false` is returned and the values of the status entries are undefined.

The *progress* rule for **MPI_TEST**, as described in Section 3.7.4, also applies to **MPI_REQUEST_GET_STATUS_ALL**.

```

MPI_REQUEST_GET_STATUS_SOME(incount, array_of_requests, outcount,
                           array_of_indices, array_of_statuses)

```

IN	<code>incount</code>	length of <code>array_of_requests</code> (nonnegative integer)
IN	<code>array_of_requests</code>	array of requests (array of handles)
OUT	<code>outcount</code>	number of completed requests (integer)
OUT	<code>array_of_indices</code>	array of indices of operations that completed (array of integers)
OUT	<code>array_of_statuses</code>	array of status objects for operations that completed (array of status)

C binding

```

int MPI_Request_get_status_some(int incount,
                               const MPI_Request array_of_requests[], int *outcount,
                               int array_of_indices[], MPI_Status array_of_statuses[])

```

Fortran 2008 binding

```

MPI_Request_get_status_some(incount, array_of_requests, outcount,
                           array_of_indices, array_of_statuses, ierror)
    INTEGER, INTENT(IN) :: incount
    TYPE(MPI_Request), INTENT(IN) :: array_of_requests(incount)
    INTEGER, INTENT(OUT) :: outcount, array_of_indices(*)

```

```

TYPE(MPI_Status) :: array_of_statuses(*)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_REQUEST_GET_STATUS_SOME(INCOUNT, ARRAY_OF_REQUESTS, OUTCOUNT,
    ARRAY_OF_INDICES, ARRAY_OF_STATUSES, IERROR)
INTEGER INCOUNT, ARRAY_OF_REQUESTS(*), OUTCOUNT, ARRAY_OF_INDICES(*),
    ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *), IERROR

```

`MPI_REQUEST_GET_STATUS_SOME` returns in `outcount` the number of requests from the list `array_of_requests` that have *completed*. Returns in the first `outcount` locations of the array `array_of_indices` the indices of these operations within the array `array_of_requests`; the array is indexed from zero in C and from one in Fortran. Returns in the first `outcount` locations of the array `array_of_statuses` the status for these *completed* operations. However, unlike `test` or `wait`, it does not deallocate or *inactivate* any requests in `array_of_requests`; a subsequent call to `test`, `wait` or `free` should be executed with each completed request. If no operation in `array_of_requests` is complete, it returns `outcount = 0`. If all operations in `array_of_requests` are either `MPI_REQUEST_NULL` or *inactive*, `outcount` will be set to `MPI_UNDEFINED`. The *progress* rule for `MPI_TEST`, as described in Section 3.7.4, also applies to `MPI_REQUEST_GET_STATUS_SOME`.

Like `MPI_WAIT_SOME` and `MPI_TEST_SOME`, `MPI_REQUEST_GET_STATUS_SOME` fulfills a **fairness requirement**: If a request for a receive repeatedly appears in a list of requests passed to `MPI_REQUEST_GET_STATUS_SOME`, `MPI_WAIT_SOME`, or `MPI_TEST_SOME` and a matching send has been *started*, then the receive will eventually succeed, unless the send is satisfied by another receive; and similarly for send requests.

Advice to implementors. `MPI_REQUEST_GET_STATUS_SOME` should *complete* as many pending communication operations as possible. (*End of advice to implementors.*)

Advice to users. `MPI_REQUEST_GET_STATUS_ANY`, `MPI_REQUEST_GET_STATUS_SOME`, and `MPI_REQUEST_GET_STATUS_ALL` offer tradeoffs between precision and speed, as do the corresponding `TEST` and `WAIT` functions. The `ANY` variants are fast, but imprecise and unfair. The `ALL` variants will provide all-or-nothing information and/or completion, which can limit their applicability. The `SOME` variants, because of their precision and fairness guarantee, will typically be the slowest on a per-call basis. (*End of advice to users.*)

3.8 Probe and Cancel

The `MPI_PROBE`, `MPI_IPROBE`, `MPI_MPROBE`, and `MPI_IMPROBE` procedures allow incoming messages to be checked for, without actually receiving them. The user can then decide how to receive them, based on the information returned by the **probe** (basically, the information returned by **status**). In particular, the user may allocate memory for the receive buffer, according to the length of the probed message.

The `MPI_CANCEL` procedure allows *pending* communication operations to be **cancelled**. This is required for cleanup. Posting a send or a receive ties up user resources (send or receive buffers), and a *cancel* may be needed to free these resources gracefully.

Cancelling a send request by calling `MPI_CANCEL` is deprecated. *Cancelling* a send-recv request by calling `MPI_CANCEL` is not allowed.

3.8.1 Probe

MPI_IPROBE(source, tag, comm, flag, status)

IN	source	rank of source or MPI_ANY_SOURCE (integer)
IN	tag	message tag or MPI_ANY_TAG (integer)
IN	comm	communicator (handle)
OUT	flag	true if there is a matching message that can be received (logical)
OUT	status	status object (status)

C binding

```
int MPI_Iprobe(int source, int tag, MPI_Comm comm, int *flag,
               MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Iprobe(source, tag, comm, flag, status, ierror)
  INTEGER, INTENT(IN) :: source, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  LOGICAL, INTENT(OUT) :: flag
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_IPROBE(SOURCE, TAG, COMM, FLAG, STATUS, IERROR)
  INTEGER SOURCE, TAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR
  LOGICAL FLAG
```

MPI_IPROBE returns `flag = true` if there is a message that can be received and that matches the pattern specified by the arguments `source`, `tag`, and `comm`. The call matches the same message that would have been received by a call to **MPI_RECV** with the same argument values for `source`, `tag`, `comm`, and `status` executed at the same point in the program, and returns in `status` the same value that would have been returned by **MPI_RECV**. Otherwise, the call returns `flag = false`, and leaves `status` undefined.

If **MPI_IPROBE** returns `flag = true`, then the content of the status object can be subsequently accessed as described in Section 3.2.5 to find the source, tag, and length of the probed message.

MPI_IPROBE is a *local* procedure since its return does not depend on MPI calls in other MPI processes, which is marked with the prefix *I* (for *immediate*).

A subsequent receive executed with the same communicator, and the source and tag returned in `status` by **MPI_IPROBE** will receive the message that was matched by the probe, if no other intervening receive occurs after the probe, and the send is not successfully *cancelled* before the receive. If the receiving MPI process is multithreaded, it is the user's responsibility to ensure that the last condition holds.

The `source` argument of **MPI_IPROBE** can be **MPI_ANY_SOURCE**, and the `tag` argument can be **MPI_ANY_TAG**, so that one can *probe* for *messages* from an arbitrary source and/or with an arbitrary tag. However, a specific communication context must be provided with the `comm` argument.

It is not necessary to receive a message immediately after it has been probed for, and the same message may be probed for several times before it is received.

A probe with `MPI_PROC_NULL` as source returns `flag = true`, and the status object returns `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG`, and `count = 0`; see Section 3.10.

`MPI_PROBE(source, tag, comm, status)`

IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)
IN	tag	message tag or <code>MPI_ANY_TAG</code> (integer)
IN	comm	communicator (handle)
OUT	status	status object (status)

C binding

```
int MPI_Probe(int source, int tag, MPI_Comm comm, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Probe(source, tag, comm, status, ierror)
  INTEGER, INTENT(IN) :: source, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PROBE(SOURCE, TAG, COMM, STATUS, IERROR)
  INTEGER SOURCE, TAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR
```

`MPI_PROBE` behaves like `MPI_IPROBE` except that it is a *nonlocal* call that returns only after a matching message has been found.

The MPI implementations of `MPI_PROBE` and `MPI_IPROBE` need to guarantee *progress*: if a call to `MPI_PROBE` has been issued by an MPI process, and a send that matches the probe has been *initiated* by some MPI process, then the call to `MPI_PROBE` will return, unless the message is received by another concurrent receive operation (that is executed by another thread at the probing MPI process).

Similarly, if an MPI process repeatedly calls `MPI_IPROBE` and a matching message has been issued, then `MPI_IPROBE` will eventually return `flag = true` unless the message is received by another concurrent receive operation or matched by a concurrent *matching probe*. See also Section 2.9 on *progress*.

Example 3.19. Use probe to wait for an incoming message.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
  CALL MPI_SEND(i, 1, MPI_INTEGER, 2, 0, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
  CALL MPI_SEND(x, 1, MPI_REAL, 2, 0, comm, ierr)
ELSE IF (rank .EQ. 2) THEN
  DO i=1,2
    CALL MPI_PROBE(MPI_ANY_SOURCE, 0, comm, status, ierr)
    IF (status(MPI_SOURCE) .EQ. 0) THEN
100      CALL MPI_RECV(i, 1, MPI_INTEGER, 0, 0, comm, status, ierr)
```

```

1      ELSE
2      200      CALL MPI_RECV(x, 1, MPI_REAL, 1, 0, comm, status, ierr)
3      END IF
4      END DO
5      END IF

```

Each message is received with the right type.

Example 3.20. A similar program to the previous example, but now it has a problem.

```

10 ! ----- THIS EXAMPLE IS ERRONEOUS -----
11 CALL MPI_COMM_RANK(comm, rank, ierr)
12 IF (rank .EQ. 0) THEN
13     CALL MPI_SEND(i, 1, MPI_INTEGER, 2, 0, comm, ierr)
14 ELSE IF (rank .EQ. 1) THEN
15     CALL MPI_SEND(x, 1, MPI_REAL, 2, 0, comm, ierr)
16 ELSE IF (rank .EQ. 2) THEN
17     DO i=1,2
18         CALL MPI_PROBE(MPI_ANY_SOURCE, 0, comm, status, ierr)
19         IF (status(MPI_SOURCE) .EQ. 0) THEN
20             100     CALL MPI_RECV(i, 1, MPI_INTEGER, MPI_ANY_SOURCE, &
21                     0, comm, status, ierr)
22             ELSE
23                 200     CALL MPI_RECV(x, 1, MPI_REAL, MPI_ANY_SOURCE, &
24                         0, comm, status, ierr)
25             END IF
26         END DO
27     END IF
28 END IF

```

In Example 3.20, the two receive calls in statements labeled 100 and 200 in Example 3.19 are slightly modified, using `MPI_ANY_SOURCE` as the source argument. The program is now incorrect: the receive operation may receive a message that is distinct from the message probed by the preceding call to `MPI_PROBE`.

Advice to users. In a multithreaded MPI program, `MPI_PROBE` and `MPI_Iprobe` might need special care. If a thread *probes* for a message and then immediately posts a matching receive, the receive may match a message other than that found by the probe since another thread could concurrently receive that original message [34]. `MPI_Mprobe` and `MPI_improbe` solve this problem by matching the incoming message so that it may only be received with `MPI_Mrecv` or `MPI_imrecv` on the corresponding *message handle*. (*End of advice to users.*)

Advice to implementors. A call to `MPI_PROBE` will match the message that would have been received by a call to `MPI_RECV` with the same argument values for source, tag, comm, and status executed at the same point. Suppose that this message has source *s*, tag *t* and communicator *c*. If the tag argument in the probe call has value `MPI_ANY_TAG`, then the message probed will be the earliest pending message from source *s* with communicator *c* and any tag; in any case, the message probed will be the earliest pending message from source *s* with tag *t* and communicator *c* (this is the message that would have been received, so as to preserve message order). This message continues as the earliest pending message from source *s* with tag *t* and communicator *c*, until it is received. A receive operation subsequent to the probe that uses the

same communicator as the probe and uses the tag and source values returned by the probe, must receive this message, unless it has already been received by another receive operation. (*End of advice to implementors.*)

3.8.2 Matching Probe

The function `MPI_PROBE` checks for incoming *messages* without receiving them. Since the list of incoming *messages* is global among the threads of each MPI process, it can be hard to use this functionality in threaded environments [34, 31].

Like `MPI_PROBE` and `MPI_IPROBE`, the **matching probe** operation (`MPI_MPROBE` and `MPI_IMPROBE` procedures) allow incoming *messages* to be queried without actually receiving them, except that `MPI_MPROBE` and `MPI_IMPROBE` provide a mechanism to receive the specific *message* that was matched regardless of other intervening probe or receive operations. This gives the application an opportunity to decide how to receive the message, based on the information returned by the probe. In particular, the user may allocate memory for the receive buffer, according to the length of the probed message.

`MPI_IMPROBE(source, tag, comm, flag, message, status)`

IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)
IN	tag	message tag or <code>MPI_ANY_TAG</code> (integer)
IN	comm	communicator (handle)
OUT	flag	true if there is a matching message that can be received (logical)
OUT	message	returned message (handle)
OUT	status	status object (status)

C binding

```
int MPI_Iprobe(int source, int tag, MPI_Comm comm, int *flag,
               MPI_Message *message, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Iprobe(source, tag, comm, flag, message, status, ierror)
  INTEGER, INTENT(IN) :: source, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  LOGICAL, INTENT(OUT) :: flag
  TYPE(MPI_Message), INTENT(OUT) :: message
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_IMPROBE(SOURCE, TAG, COMM, FLAG, MESSAGE, STATUS, IERROR)
  INTEGER SOURCE, TAG, COMM, MESSAGE, STATUS(MPI_STATUS_SIZE), IERROR
  LOGICAL FLAG
```

`MPI_IMPROBE` returns `flag = true` if there is a message that can be received and that matches the pattern specified by the arguments `source`, `tag`, and `comm`. The call matches the same message that would have been received by a call to `MPI_RECV` with the

same argument values for `source`, `tag`, `comm`, and `status` executed at the same point in the program and returns in `status` the same value that would have been returned by `MPI_RECV`. In addition, it returns in `message` a **message handle** to the matched message. Otherwise, the call returns `flag = false`, and leaves `status` and `message` undefined.

`MPI_IMPROBE` is a *local* procedure. According to the definitions in Section 2.4.2 and in contrast to `MPI_IPROBE`, it is a *nonblocking* procedure because it is the *initialization* of a *matched receive* operation.

A *matched receive* (`MPI_MRECV` or `MPI_IMRECV`) executed with the *message handle* will receive the message that was matched by the *matching probe*. Unlike `MPI_IPROBE`, no other probe or receive operation may match the message returned by `MPI_IMPROBE`. Each *message handle* returned by `MPI_IMPROBE` must be received with either `MPI_MRECV` or `MPI_IMRECV`.

The `source` argument of `MPI_IMPROBE` can be `MPI_ANY_SOURCE`, and the `tag` argument can be `MPI_ANY_TAG`, so that one can *probe* for *messages* from an arbitrary source and/or with an arbitrary tag. However, a specific communication context must be provided with the `comm` argument.

A *synchronous mode send* operation that is matched with `MPI_IMPROBE` or `MPI_MPROBE` will *complete* successfully only if both a *matching receive* is *started* with `MPI_MRECV` or `MPI_IMRECV`, and the *matching receive* operation has *started* to receive the message sent by the *synchronous mode send*.

There is a special **predefined message handle**: `MPI_MESSAGE_NO_PROC`, which is a message that has `MPI_PROC_NULL` as its source. The predefined constant `MPI_MESSAGE_NULL` is the value used for **invalid message handles**.

A *matching probe* with `source = MPI_PROC_NULL` returns `flag = true`, `message = MPI_MESSAGE_NO_PROC`, and the status object returns `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG`, and `count = 0`; see Section 3.10. It is not necessary to call `MPI_MRECV` or `MPI_IMRECV` with `MPI_MESSAGE_NO_PROC`, but it is not *erroneous* to do so.

Rationale. `MPI_MESSAGE_NO_PROC` was chosen instead of `MPI_MESSAGE_PROC_NULL` to avoid possible confusion as another null handle constant. (*End of rationale.*)

`MPI_MPROBE(source, tag, comm, message, status)`

IN	source	rank of source or <code>MPI_ANY_SOURCE</code> (integer)
IN	tag	message tag or <code>MPI_ANY_TAG</code> (integer)
IN	comm	communicator (handle)
OUT	message	returned message (handle)
OUT	status	status object (status)

C binding

```
int MPI_Mprobe(int source, int tag, MPI_Comm comm, MPI_Message *message,
               MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Mprobe(source, tag, comm, message, status, ierror)
  INTEGER, INTENT(IN) :: source, tag
```



```

TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Message), INTENT(OUT) :: message
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_MPROBE(SOURCE, TAG, COMM, MESSAGE, STATUS, IERROR)
  INTEGER SOURCE, TAG, COMM, MESSAGE, STATUS(MPI_STATUS_SIZE), IERROR

```

MPI_MPROBE behaves like **MPI_IMPROBE** except that it is a *blocking* call that returns only after a matching message has been found.

The implementation of **MPI_MPROBE** and **MPI_IMPROBE** needs to guarantee *progress* in the same way as in the case of **MPI_PROBE** and **MPI_IPROBE**. See also Section 2.9 on *progress*.

According to the definitions in Section 2.4.2, **MPI_MPROBE** is *incomplete*. It is also a *nonlocal* procedure.

Advice to users. This is one of the exceptions in which *incomplete* procedures are *nonlocal*. (End of advice to users.)

3.8.3 Matched Receives

The **matched receive** operation (**MPI_MRECV** and **MPI_IMRECV** procedures) receive *messages* that have been previously matched by a *matching probe* operation (Section 3.8.2).

```

MPI_MRECV(buf, count, datatype, message, status)

```

OUT	buf	initial address of receive buffer (choice)
IN	count	number of elements in receive buffer (nonnegative integer)
IN	datatype	datatype of each receive buffer element (handle)
INOUT	message	message (handle)
OUT	status	status object (status)

C binding

```

int MPI_Mrecv(void *buf, int count, MPI_Datatype datatype,
              MPI_Message *message, MPI_Status *status)

int MPI_Mrecv_c(void *buf, MPI_Count count, MPI_Datatype datatype,
                MPI_Message *message, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_Mrecv(buf, count, datatype, message, status, ierror)
  TYPE(*), DIMENSION(..) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Message), INTENT(INOUT) :: message
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1 MPI_Mrecv(buf, count, datatype, message, status, ierror) !(_c)
2   TYPE(*), DIMENSION(..) :: buf
3   INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
4   TYPE(MPI_Datatype), INTENT(IN) :: datatype
5   TYPE(MPI_Message), INTENT(INOUT) :: message
6   TYPE(MPI_Status) :: status
7   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

9 MPI_MRECV(BUF, COUNT, DATATYPE, MESSAGE, STATUS, IERROR)
10 <type> BUF(*)
11 INTEGER COUNT, DATATYPE, MESSAGE, STATUS(MPI_STATUS_SIZE), IERROR

```

This call receives a message matched by a *matching probe* operation (Section 3.8.2).

The *receive buffer* consists of the storage containing `count` consecutive elements of the type specified by `datatype`, starting at address `buf`. The length of the received message must be less than or equal to the length of the receive buffer. An overflow error occurs if all incoming data does not fit, without truncation, into the receive buffer.

If the message is shorter than the receive buffer, then only those locations corresponding to the (shorter) message are modified.

On return from this function, the *message handle* is set to `MPI_MESSAGE_NULL`. All errors that occur during the execution of this operation are handled according to the error handler set for the communicator used in the matching probe call that produced the message handle.

If `MPI_MRECV` is called with `MPI_MESSAGE_NO_PROC` as the message argument, the call returns immediately with the status object set to `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG`, and `count = 0`. This is consistent with the status object produced by a call to `MPI_RECV` or to `MPI_PROBE` with `source = MPI_PROC_NULL` (see Section 3.10). A call to `MPI_MRECV` with `MPI_MESSAGE_NULL` is *erroneous*.

```

30 MPI_IMRECV(buf, count, datatype, message, request)

```

OUT	buf	initial address of receive buffer (choice)
IN	count	number of elements in receive buffer (nonnegative integer)
IN	datatype	datatype of each receive buffer element (handle)
INOUT	message	message (handle)
OUT	request	communication request (handle)

C binding

```

41 int MPI_Imrecv(void *buf, int count, MPI_Datatype datatype,
42               MPI_Message *message, MPI_Request *request)
43
44 int MPI_Imrecv_c(void *buf, MPI_Count count, MPI_Datatype datatype,
45                 MPI_Message *message, MPI_Request *request)

```

Fortran 2008 binding

```

46 MPI_Imrecv(buf, count, datatype, message, request, ierror)
47   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf

```

```

    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Message), INTENT(INOUT) :: message
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Imrecv(buf, count, datatype, message, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Message), INTENT(INOUT) :: message
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IMRECV(BUF, COUNT, DATATYPE, MESSAGE, REQUEST, IERROR)
    <type> BUF(*)
    INTEGER COUNT, DATATYPE, MESSAGE, REQUEST, IERROR

```

MPI_IMRECV is the nonblocking variant of **MPI_MRECV** and starts a nonblocking receive of a matched message. Completion semantics are similar to **MPI_IRECV** as described in Section 3.7.2. On return from this function, the *message handle* is set to **MPI_MESSAGE_NULL**.

If **MPI_IMRECV** is called with **MPI_MESSAGE_NO_PROC** as the message argument, the call returns immediately with a request object that, when completed, will yield a status object set to `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG`, and `count = 0`, as if a receive from **MPI_PROC_NULL** was issued (see Section 3.10). A call to **MPI_IMRECV** with **MPI_MESSAGE_NULL** is *erroneous*.

Advice to implementors. If reception of a matched message is started with **MPI_IMRECV**, then it is possible to *cancel* the returned request with **MPI_CANCEL**. If **MPI_CANCEL** succeeds, the matched message must be found by a subsequent message probe (**MPI_PROBE**, **MPI_IPROBE**, **MPI_MPROBE**, or **MPI_ImPROBE**), received by a subsequent receive operation or *cancelled* by the sender. See Section 3.8.4 for details about **MPI_CANCEL**. The *cancellation* of operations initiated with **MPI_IMRECV** may fail. (*End of advice to implementors.*)

3.8.4 Cancel

```

MPI_CANCEL(request)

```

```

    IN          request          communication request (handle)

```

C binding

```

int MPI_Cancel(MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Cancel(request, ierror)
    TYPE(MPI_Request), INTENT(IN) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```
MPI_CANCEL(REQUEST, IERROR)
  INTEGER REQUEST, IERROR
```

A call to `MPI_CANCEL` marks for *cancellation* a *pending, nonblocking* communication operation (send or receive). *Cancelling* a send request by calling `MPI_CANCEL` is deprecated. The *cancel* call is *local*. It returns *immediately*, possibly before the communication is actually *cancelled*. It is still necessary to call `MPI_REQUEST_FREE`, `MPI_WAIT` or `MPI_TEST` (or any of the derived procedures) with the *cancelled* request as argument after the call to `MPI_CANCEL`. If a communication is marked for *cancellation*, then a `MPI_WAIT` call for that communication is guaranteed to return, irrespective of the activities of other MPI processes (i.e., `MPI_WAIT` behaves as a *local* function); similarly if `MPI_TEST` is repeatedly called for a *cancelled* communication, then `MPI_TEST` will eventually return `flag = true`.

`MPI_CANCEL` can be used to *cancel* a communication that uses a *persistent communication request* (see Section 3.9), in the same way as it is described above for nonblocking operations. *Cancelling* a persistent send request by calling `MPI_CANCEL` is deprecated. A successful *cancellation* *cancels* the *active* communication, but not the request itself. After the call to `MPI_CANCEL` and the subsequent call to `MPI_WAIT` or `MPI_TEST`, the request becomes *inactive* and can be activated for a new communication.

The successful *cancellation* of a *buffered mode send* frees the buffer space occupied by the pending message. *Cancelling* a *buffered mode send* request by calling `MPI_CANCEL` is deprecated.

Either the *cancellation* succeeds, or the communication succeeds, but not both. If a send is marked for *cancellation*, which is deprecated, then it must be the case that either the send *completes* normally, in which case the message sent was received at the destination, or that the send is successfully *cancelled*, in which case no part of the message was received at the destination. Then, any matching receive has to be satisfied by another send. If a receive is marked for *cancellation*, then it must be the case that either the receive *completes* normally, or that the receive is successfully *cancelled*, in which case no part of the receive buffer is altered. Then, any matching send has to be satisfied by another receive.

If the operation has been *cancelled*, then information to that effect will be returned in the status argument of the operation that *completes* the communication.

Rationale. Although the IN request handle parameter should not need to be passed by reference, the C binding has listed the argument type as `MPI_Request*` since MPI-1.0. This function signature therefore cannot be changed without breaking existing MPI applications. (*End of rationale.*)

```
MPI_TEST_CANCELLED(status, flag)
```

IN	status	status object (status)
OUT	flag	true if the operation has been cancelled (logical)

C binding

```
int MPI_Test_cancelled(const MPI_Status *status, int *flag)
```

Fortran 2008 binding

```

MPI_Test_cancelled(status, flag, ierror)
    TYPE(MPI_Status), INTENT(IN) :: status
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TEST_CANCELLED(STATUS, FLAG, IERROR)
    INTEGER STATUS(MPI_STATUS_SIZE), IERROR
    LOGICAL FLAG

```

Returns `flag = true` if the communication associated with the status object was *cancelled* successfully. In such a case, all other fields of `status` (such as `count` or `tag`) are undefined. Returns `flag = false`, otherwise. If a receive operation might be *cancelled* then one should call `MPI_TEST_CANCELLED` first, to check whether the operation was *cancelled*, before checking on the other fields of the return status.

Advice to users. *Cancel* can be an expensive operation that should be used only exceptionally. (*End of advice to users.*)

Advice to implementors. If a send operation uses an “eager” protocol (data is transferred to the receiver before a matching receive is *started*), then the *cancellation* of this send may require communication with the intended receiver in order to free allocated buffers. On some systems this may require an interrupt to the intended receiver. Note that, while communication may be needed to implement `MPI_CANCEL`, this is still a *local* procedure, since its completion does not depend on the code executed by other MPI processes. If processing is required on another MPI process, this should be transparent to the application (hence the need for an interrupt and an interrupt handler). See also Section 2.9 on *progress*. (*End of advice to implementors.*)

3.9 Persistent Communication Requests

Often a communication with the same argument list (with the exception of the buffer contents) is repeatedly executed within the inner loop of a parallel computation. In such a situation, it may be possible to optimize the communication by binding the list of communication arguments to a *persistent communication request* once and then repeatedly using the request to *start* and *complete* operations. In the case of point-to-point communication, the *persistent communication request* thus created can be thought of as a communication port or a “half-channel.” It does not provide the full functionality of a conventional channel, since there is no binding of the send port to the receive port. This construct allows the reduction of the overhead for communication between the MPI process and communication controller, but not of the overhead for communication between one communication controller and another. It is not necessary that messages sent with a persistent point-to-point request be received by a receive operation using a persistent point-to-point request, or vice versa.

There are also persistent collective communication operations defined in Section 6.13 and Section 8.8. The remainder of this section covers the point-to-point persistent *initialization* operations and the start routines, which are used for persistent point-to-point, partitioned point-to-point, and persistent collective communication operations.

A point-to-point **persistent communication request** is created using one of the five following calls. These point-to-point persistent *initialization* calls involve no communication.

MPI_SEND_INIT(buf, count, datatype, dest, tag, comm, request)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements sent (nonnegative integer)
IN	datatype	type of each element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Send_init(const void *buf, int count, MPI_Datatype datatype, int dest,
                 int tag, MPI_Comm comm, MPI_Request *request)
```

```
int MPI_Send_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                   int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Send_init(buf, count, datatype, dest, tag, comm, request, ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
```

```
INTEGER, INTENT(IN) :: count, dest, tag
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Send_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
INTEGER, INTENT(IN) :: dest, tag
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
```

```
<type> BUF(*)
```

```
INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
```

Creates a *persistent communication request* for a *standard mode send* operation.

MPI_BSEND_INIT(buf, count, datatype, dest, tag, comm, request)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements sent (nonnegative integer)
IN	datatype	type of each element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Bsend_init(const void *buf, int count, MPI_Datatype datatype, int dest,
                  int tag, MPI_Comm comm, MPI_Request *request)
```

```
int MPI_Bsend_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                    int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Bsend_init(buf, count, datatype, dest, tag, comm, request, ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
```

```
INTEGER, INTENT(IN) :: count, dest, tag
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Bsend_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
INTEGER, INTENT(IN) :: dest, tag
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_BSEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
```

```
<type> BUF(*)
```

```
INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
```

Creates a *persistent communication request* for a *buffered mode send* operation.

MPI_SSEND_INIT(buf, count, datatype, dest, tag, comm, request)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements sent (nonnegative integer)
IN	datatype	type of each element (handle)
IN	dest	rank of destination (integer)

```

1      IN      tag                message tag (integer)
2      IN      comm              communicator (handle)
3
4      OUT     request            communication request (handle)

```

C binding

```

7  int MPI_Ssend_init(const void *buf, int count, MPI_Datatype datatype, int dest,
8                      int tag, MPI_Comm comm, MPI_Request *request)
9
10 int MPI_Ssend_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
11                      int dest, int tag, MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

12 MPI_Ssend_init(buf, count, datatype, dest, tag, comm, request, ierror)
13     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
14     INTEGER, INTENT(IN) :: count, dest, tag
15     TYPE(MPI_Datatype), INTENT(IN) :: datatype
16     TYPE(MPI_Comm), INTENT(IN) :: comm
17     TYPE(MPI_Request), INTENT(OUT) :: request
18     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
19
20 MPI_Ssend_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
21     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
22     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     INTEGER, INTENT(IN) :: dest, tag
25     TYPE(MPI_Comm), INTENT(IN) :: comm
26     TYPE(MPI_Request), INTENT(OUT) :: request
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

29 MPI_SSEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
30     <type> BUF(*)
31     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
32     Creates a persistent communication request for a synchronous mode send operation.
33
34

```

```

35 MPI_RSEND_INIT(buf, count, datatype, dest, tag, comm, request)

```

```

36     IN      buf                initial address of send buffer (choice)
37
38     IN      count              number of elements sent (nonnegative integer)
39
40     IN      datatype            type of each element (handle)
41
42     IN      dest                rank of destination (integer)
43
44     IN      tag                message tag (integer)
45
46     IN      comm              communicator (handle)
47
48     OUT     request            communication request (handle)

```

C binding

```

47 int MPI_Rsend_init(const void *buf, int count, MPI_Datatype datatype, int dest,
48                   int tag, MPI_Comm comm, MPI_Request *request)

```



```
int MPI_Rsend_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                    int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Rsend_init(buf, count, datatype, dest, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, dest, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rsend_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_RSEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
  <type> BUF(*)
  INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR

  Creates a persistent communication request for a ready mode send operation.
```

```
MPI_RECV_INIT(buf, count, datatype, source, tag, comm, request)
```

OUT	buf	initial address of receive buffer (choice)
IN	count	number of elements received (nonnegative integer)
IN	datatype	type of each element (handle)
IN	source	rank of source or MPI_ANY_SOURCE (integer)
IN	tag	message tag or MPI_ANY_TAG (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Recv_init(void *buf, int count, MPI_Datatype datatype, int source,
                 int tag, MPI_Comm comm, MPI_Request *request)

int MPI_Recv_init_c(void *buf, MPI_Count count, MPI_Datatype datatype,
                   int source, int tag, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Recv_init(buf, count, datatype, source, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, source, tag
```

```

1      TYPE(MPI_Datatype), INTENT(IN) :: datatype
2      TYPE(MPI_Comm), INTENT(IN) :: comm
3      TYPE(MPI_Request), INTENT(OUT) :: request
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6      MPI_Recv_init(buf, count, datatype, source, tag, comm, request, ierror) !(_c)
7      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
8      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
9      TYPE(MPI_Datatype), INTENT(IN) :: datatype
10     INTEGER, INTENT(IN) :: source, tag
11     TYPE(MPI_Comm), INTENT(IN) :: comm
12     TYPE(MPI_Request), INTENT(OUT) :: request
13     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

14 MPI_RECV_INIT(BUF, COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR)
15 <type> BUF(*)
16 INTEGER COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR

```

Creates a *persistent communication request* for a receive operation. The argument `buf` is marked as `OUT` because the user gives permission to write on the receive buffer by passing the argument to `MPI_RECV_INIT`.

A *persistent communication request* is *inactive* after it was created—no active communication is attached to the request.

A communication that uses a *persistent communication request* is *started* by the function `MPI_START`.

```

27 MPI_START(request)
28 INOUT    request      communication request (handle)

```

C binding

```

31 int MPI_Start(MPI_Request *request)

```

Fortran 2008 binding

```

34 MPI_Start(request, ierror)
35 TYPE(MPI_Request), INTENT(INOUT) :: request
36 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

38 MPI_START(REQUEST, IERROR)
39 INTEGER REQUEST, IERROR

```

The argument, `request`, is a handle returned by any of the *initialization* procedures for persistent point-to-point communication (the previous five procedures), or for partitioned point-to-point communication (see Chapter 4), or for persistent collective communication (see Sections 6.13 and 8.8). The associated request should be *inactive*. The request becomes *active* once the call is made.

If the request is for a *ready mode send* operation, then a matching receive operation should be *started* before the call is made. The communication buffer must not be modified after the call, and until the operation *completes*.

The call is *local*, with similar semantics to the nonblocking communication operations described in Section 3.7. That is, a call to `MPI_START` with a request created by `MPI_SEND_INIT` starts a communication in the same manner as a call to `MPI_ISEND`; a call to `MPI_START` with a request created by `MPI_BSEND_INIT` starts a communication in the same manner as a call to `MPI_IBSEND`; and so on.

`MPI_STARTALL(count, array_of_requests)`

IN	count	list length (nonnegative integer)
INOUT	array_of_requests	array of requests (array of handles)

C binding

```
int MPI_Startall(int count, MPI_Request array_of_requests[])
```

Fortran 2008 binding

```
MPI_Startall(count, array_of_requests, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_STARTALL(COUNT, ARRAY_OF_REQUESTS, IERROR)
  INTEGER COUNT, ARRAY_OF_REQUESTS(*), IERROR
```

The execution of `MPI_STARTALL` has the same effect as the execution of `MPI_START` for each of the array elements in some arbitrary order. `MPI_STARTALL` with an array of length one is equivalent to `MPI_START`.

A communication started with a call to `MPI_START` or `MPI_STARTALL` is completed by a call to `MPI_WAIT`, `MPI_TEST`, or one of the derived functions described in Section 3.7.5. The request becomes *inactive* after successful completion of such call. The request is not deallocated and it can be activated anew by an `MPI_START` or `MPI_STARTALL` call.

A *persistent communication request* is deallocated by a call to `MPI_REQUEST_FREE` (Section 3.7.3). The call to `MPI_REQUEST_FREE` can occur at any point in the program after the persistent request was created. However, the request will be deallocated only after it becomes *inactive*. *Active* receive requests should not be *freed*. Otherwise, it will not be possible to check that the receive has *completed*. *Collective* operation requests (defined in Section 6.12 and Section 8.7 for nonblocking collective operations, and Section 6.13 and Section 8.8 for persistent collective operations) must not be *freed* while *active*. It is preferable, in general, to free requests when they are inactive. If this rule is followed, then the functions described in this section will be invoked in a sequence of the form,

Create (Start Complete)* Free

where * indicates zero or more repetitions. If the same *persistent communication request* is used in several concurrent threads, it is the user's responsibility to coordinate calls so that the correct sequence is obeyed.

Inactive persistent requests are not automatically *freed* when the associated communicator is disconnected (via `MPI_COMM_DISCONNECT`, see Section 11.10.4) or the associated World Model or Sessions Model is finalized (via `MPI_FINALIZE`, see Section 11.2.2, or

`MPI_SESSION_FINALIZE`, see Section 11.3.1). In these situations, any further use of the request handle is erroneous. In particular, freeing associated inactive request handles after such a communicator disconnect or finalization is then impossible.

Advice to users. Persistent request handles may bind internal resources such as MPI buffers in shared memory for providing efficient communication. Therefore, it is highly recommended to explicitly free inactive request handles, using `MPI_REQUEST_FREE`, when they are no longer in use, and in particular before freeing or disconnecting the associated communicator with `MPI_COMM_FREE` or `MPI_COMM_DISCONNECT` or finalizing the associated session with `MPI_SESSION_FINALIZE`. (*End of advice to users.*)

A send operation *started* with `MPI_START` can be *matched* with any receive operation and, likewise, a receive operation *started* with `MPI_START` can receive messages generated by any send operation.

Advice to users. To prevent problems with the argument copying and register optimization done by Fortran compilers, please note the hints in Sections 21.1.10–21.1.20. (*End of advice to users.*)

3.10 Null MPI Processes

In many instances, it is convenient to specify a “dummy” source or destination for communication. This simplifies the code that is needed for dealing with boundaries, for example, in the case of a noncircular shift done with calls to send-receive.

The special value `MPI_PROC_NULL` can be used instead of a rank wherever a source or a destination argument is required in a call. A communication with `MPI_PROC_NULL` has no effect. A send to `MPI_PROC_NULL` succeeds and returns as soon as possible. A receive from `MPI_PROC_NULL` succeeds and returns as soon as possible with no modifications to the receive buffer. When a receive with `source = MPI_PROC_NULL` is executed then the status object returns `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG` and `count = 0`. A probe or matching probe with `source = MPI_PROC_NULL` succeeds and returns as soon as possible, and the status object returns `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG` and `count = 0`. A matching probe (cf. Section 3.8.2) with `source = MPI_PROC_NULL` returns `flag = true`, `message = MPI_MESSAGE_NO_PROC`, and the status object returns `source = MPI_PROC_NULL`, `tag = MPI_ANY_TAG`, and `count = 0`.

Chapter 4

Partitioned Point-to-Point Communication

4.1 Introduction

Partitioned communication extends persistent point-to-point communication as defined in Chapter 3. Partitioned communication operations are matched based on the order in which the local initialization calls are performed. Partitioned communication is “partitioned” because it allows for multiple contributions of data to be made, potentially, from multiple actors (e.g., threads or tasks) in an MPI process to a single communication operation.

Advice to users. The techniques of partitioned communication were known as “fine-points” before their adoption into the MPI standard. We refer the interested reader to the original literature describing the design goals, functioning, initial implementation and performance improvements [29, 30]. (*End of advice to users.*)

Partitioned communication operations use a persistent communication style that involves a sequence of start and test or wait operations. For this sequence, partitioned communications use `MPI_START` or `MPI_STARTALL` calls and completion mechanisms (e.g., `MPI_TEST` or `MPI_WAIT`). Partitioned communication is different in three fundamental ways from persistent point-to-point operations in MPI. First, partitioned communication allows additional partitioned test function calls that can expose partial completion of the operation. Second, partitioned communication may perform all of the initialization required to enable data transfer as early as its initialization phase. Third, partitioned communication allows for MPI to be independently notified of multiple contributions from the send-side to a single data buffer of a single MPI message.

Rationale. The rationale behind having different initialization behavior allowed for partitioned communication as opposed to persistent point-to-point communication is to enable flexibility and optimization possibilities in implementations. Buffer setup can occur in the partitioned communication initialization functions (see Section 4.2.1). However, such negotiation can be deferred until data is to be moved between two processes. This means that partitioned communication can lazily negotiate as late as testing for completion of the operation on the first iteration of a sequence of partitioned communication start and test or wait operations. Matching still occurs as if matching happened at the partitioned communication initialization functions as noted in the function descriptions. (*End of rationale.*)

4.2 Semantics of Partitioned Point-to-Point Communication

MPI guarantees certain general properties of partitioned point-to-point communication progress, which are described in this section.

Persistent communications use opaque `MPI_REQUEST` objects as described in Section 3. Partitioned communication uses these same semantics for `MPI_REQUEST` objects.

Partitioned communication provides fine-grained transfers on either or both sides of a send-receive operation described by requests. Persistent communication semantics are ideal for partitioned communication: they provide `MPI_PSEND_INIT` and `MPI_PRECV_INIT` functions that allow partitioned communication setup to occur prior to message transfers. Partitioned communication initialization functions are local. The partitioned communication initialization includes inputs on the number of user-visible partitions on the send-side and receive-side, which may differ. Valid partitioned communication operations must have one or more partitions specified.

Once an `MPI_PSEND_INIT` call has been made, the user may start the operation with a call to a starting procedure and complete the operation with one `MPI_PREADY` call for every send partition followed by a call to a completing procedure. A call to `MPI_PREADY` notifies the MPI library that a specified portion of the data buffer (a specific partition) is ready to be sent. Notification of partial completion can be done via fine-grained `MPI_PARRIVED` calls at the receiver before a final `MPI_TEST/MPI_WAIT` on the request itself; the latter represents overall operation completion upon success. A full set of methods for starting and completing partitioned communication is given in the following sections.

Advice to users. Having a large number of receiver-side partitions can increase overheads as the completion mechanism may need to work with finer-grained notifications. Using a small number of receiver-side partitions *may* provide higher performance.

A large number of sender-side partitions may be aggregated by an MPI implementation, making performance concerns of a large number of sender-side partitions potentially less impactful than receiver-side granularity. (*End of advice to users.*)

Advice to implementors. It is expected that an MPI implementation will attempt to balance latency and aggregation for data transfers for the requested partition counts on the sender-side and receiver-side to allow optimization for different hardware. A high quality implementation may perform significant optimizations to enhance performance in this way; they may, for example, resize the data transfers of the partitions to combine partitions in fractional partition sizes (e.g., 2.5 partitions in a single data transfer). (*End of advice to implementors.*)

Example 4.1 shows a simple partitioned transfer in which the sender-side and receiver-side partitioning is identical in partition count.

Example 4.1. Simple partitioned communication example.

```
#include <stdlib.h>
#include "mpi.h"
#define PARTITIONS 8
#define COUNT 5
int main(int argc, char *argv[])
{
    double message[PARTITIONS*COUNT];
```

```

1  int partitions = PARTITIONS;
2  int source = 0, dest = 1, tag = 1, flag = 0;
3  int myrank, i;
4  MPI_Request request;
5  MPI_Init(&argc, &argv);
6  MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
7  if (myrank == 0)
8  {
9      MPI_Psend_init(message, partitions, COUNT, MPI_DOUBLE, dest, tag,
10                     MPI_COMM_WORLD, MPI_INFO_NULL, &request);
11      MPI_Start(&request);
12      for(i = 0; i < partitions; ++i)
13      {
14          /* compute and fill partition #i, then mark ready: */
15          MPI_Pready(i, request);
16      }
17      while(!flag)
18      {
19          /* do useful work #1 */
20          MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
21          /* do useful work #2 */
22      }
23      MPI_Request_free(&request);
24  }
25  else if (myrank == 1)
26  {
27      MPI_Precv_init(message, partitions, COUNT, MPI_DOUBLE, source, tag,
28                     MPI_COMM_WORLD, MPI_INFO_NULL, &request);
29      MPI_Start(&request);
30      while(!flag)
31      {
32          /* do useful work #1 */
33          MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
34          /* do useful work #2 */
35      }
36      MPI_Request_free(&request);
37  }
38  MPI_Finalize();
39  return 0;
40  }

```

Rationale. Partitioned communication is designed to provide opportunities for MPI implementations to optimize data transfers. MPI is free to choose how many transfers to do within a partitioned communication send independent of how many partitions are reported as ready to MPI through `MPI_PREADY` calls. Aggregation of partitions is permitted but not required. Ordering of partitions is permitted but not required. A naive implementation can simply wait for the entire message buffer to be marked ready before any transfer(s) occur and could wait until the completion function is called on a request before transferring data. However, this modality of communication gives MPI implementations far more flexibility in data movement than nonpartitioned communications. (*End of rationale.*)

4.2.1 Communication Initialization and Starting with Partitioning

Initialization of partitioned communication operations use the initialization calls described below. Subsequent to initialization, `MPI_START/MPI_STARTALL` are used as the first indication to MPI that a message transfer will occur. For send-side operations, neither initializing nor starting the operation enables transfer of any part of the user buffer. Freeing or canceling a partitioned communication request that is active (i.e., initialized and started) and not completed is erroneous. After the partitioned communication operation is started, individual partitions of a message are indicated as ready to be sent by MPI via the `MPI_PREADY` function, described below.

`MPI_PSEND_INIT(buf, partitions, count, datatype, dest, tag, comm, info, request)`

IN	buf	initial address of send buffer (choice)
IN	partitions	number of partitions (nonnegative integer)
IN	count	number of elements sent per partition (nonnegative integer)
IN	datatype	type of each element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Psend_init(const void *buf, int partitions, MPI_Count count,
                  MPI_Datatype datatype, int dest, int tag, MPI_Comm comm,
                  MPI_Info info, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Psend_init(buf, partitions, count, datatype, dest, tag, comm, info,
               request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER, INTENT(IN) :: partitions, dest, tag
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PSEND_INIT(BUF, PARTITIONS, COUNT, DATATYPE, DEST, TAG, COMM, INFO,
               REQUEST, IERROR)
    <type> BUF(*)
    INTEGER PARTITIONS, DATATYPE, DEST, TAG, COMM, INFO, REQUEST, IERROR
    INTEGER(KIND=MPI_COUNT_KIND) COUNT
```


`MPI_PSEND_INIT` creates a partitioned communication request and binds to it all the arguments of a partitioned send operation. Matching follows the same MPI matching rules as for point-to-point communication (see Chapter 3) with communicator, tag, and source dictating message matching. In the event that the communicator, tag, and source do not uniquely identify a message, the order in which partitioned communication *initialization* calls are made is the order in which they will eventually match. This operation can only match with partitioned communication initialization operations, therefore it is required to be matched with a corresponding `MPI_PRECV_INIT` call. Partitioned communication initialization calls are local. It is erroneous to provide a `partitions` value ≤ 0 . Send-side and receive-side buffers must be identical in size.

Advice to implementors. Unlike `MPI_SEND_INIT`, `MPI_PSEND_INIT` can be matched as early as the initialization call. Also, unlike `MPI_SEND_INIT`, `MPI_PSEND_INIT` takes an `info` argument. (*End of advice to implementors.*)

`MPI_PRECV_INIT(buf, partitions, count, datatype, source, tag, comm, info, request)`

IN	buf	initial address of recv buffer (choice)
IN	partitions	number of partitions (nonnegative integer)
IN	count	number of elements received per partition (nonnegative integer)
IN	datatype	type of each element (handle)
IN	source	rank of source (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Precv_init(void *buf, int partitions, MPI_Count count,
                  MPI_Datatype datatype, int source, int tag, MPI_Comm comm,
                  MPI_Info info, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Precv_init(buf, partitions, count, datatype, source, tag, comm, info,
               request, ierror)

TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER, INTENT(IN) :: partitions, source, tag
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_PRECV_INIT(BUF, PARTITIONS, COUNT, DATATYPE, SOURCE, TAG, COMM, INFO,
               REQUEST, IERROR)
<type> BUF(*)
INTEGER PARTITIONS, DATATYPE, SOURCE, TAG, COMM, INFO, REQUEST, IERROR
INTEGER(KIND=MPI_COUNT_KIND) COUNT

```

Rationale. The info argument is provided in order to support per-operation implementation-defined info keys. (*End of rationale.*)

MPI_PRECV_INIT creates a partitioned communication receive request and binds to it all the arguments of a partitioned receive operation. This operation can only match with partitioned communication initialization operations, therefore the MPI library is required to match **MPI_PRECV_INIT** calls only with a corresponding **MPI_PSEND_INIT** call. Matching follows the same MPI matching rules as for point-to-point communication (see Chapter 3) with communicator, tag, and source dictating message matching. In the event that the communicator, tag, and source do not uniquely identify a message, the order in which partitioned communication initialization calls are made is the order in which they will eventually match. Partitioned communication initialization calls are local. That is, **MPI_PRECV_INIT** may return before the operation completes. It is erroneous to provide a partitions value ≤ 0 . Wildcards for source and tag are not allowed.

Advice to implementors. Unlike **MPI_RECV_INIT**, **MPI_PRECV_INIT** may communicate. Also unlike **MPI_RECV_INIT**, **MPI_PRECV_INIT** takes an info argument. (*End of advice to implementors.*)

```
MPI_PREADY(partition, request)
```

IN	partition	partition to mark ready for transfer (nonnegative integer)
INOUT	request	partitioned communication request (handle)

C binding

```
int MPI_Pready(int partition, MPI_Request request)
```

Fortran 2008 binding

```

MPI_Pready(partition, request, ierror)
INTEGER, INTENT(IN) :: partition
TYPE(MPI_Request), INTENT(IN) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_PREADY(PARTITION, REQUEST, IERROR)
INTEGER PARTITION, REQUEST, IERROR

```

MPI_PREADY is a send-side call that indicates that a given partition is ready to be transferred. It is erroneous to use **MPI_PREADY** on any request object that does not correspond to a partitioned send operation. The partitioning is defined by the **MPI_PSEND_INIT** call. Partition numbering starts at zero and ranges to one less than

the number of partitions declared in the `MPI_PSEND_INIT` call. Specifying a partition number that is equal to or larger than the number of partitions is erroneous. After a call to `MPI_START/MPI_STARTALL`, all partitions associated with that operation are inactive. A call to `MPI_PREADY` marks the indicated partition as active. Calling `MPI_PREADY` on an active partition is erroneous.

`MPI_PREADY_RANGE(partition_low, partition_high, request)`

IN	partition_low	lowest partition ready for transfer (nonnegative integer)
IN	partition_high	highest partition ready for transfer (nonnegative integer)
INOUT	request	partitioned communication request (handle)

C binding

```
int MPI_Pready_range(int partition_low, int partition_high,
                    MPI_Request request)
```

Fortran 2008 binding

```
MPI_Pready_range(partition_low, partition_high, request, ierror)
  INTEGER, INTENT(IN) :: partition_low, partition_high
  TYPE(MPI_Request), INTENT(IN) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PREADY_RANGE(PARTITION_LOW, PARTITION_HIGH, REQUEST, IERROR)
  INTEGER PARTITION_LOW, PARTITION_HIGH, REQUEST, IERROR
```

A call to `MPI_PREADY_RANGE` has the same effect as calls to `MPI_PREADY`, executed for `i=partition_low, ..., partition_high`, in some arbitrary order. Calls to `MPI_PREADY_RANGE` follow the same rules as those for `MPI_PREADY` calls.

`MPI_PREADY_LIST(length, array_of_partitions, request)`

IN	length	list length (integer)
IN	array_of_partitions	array of partitions (array of nonnegative integers)
INOUT	request	partitioned communication request (handle)

C binding

```
int MPI_Pready_list(int length, const int array_of_partitions[],
                    MPI_Request request)
```

Fortran 2008 binding

```
MPI_Pready_list(length, array_of_partitions, request, ierror)
  INTEGER, INTENT(IN) :: length, array_of_partitions(length)
  TYPE(MPI_Request), INTENT(IN) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_PREADY_LIST(LENGTH, ARRAY_OF_PARTITIONS, REQUEST, IERROR)
    INTEGER LENGTH, ARRAY_OF_PARTITIONS(*), REQUEST, IERROR

```

A call to `MPI_PREADY_LIST` has the same effect as calls to `MPI_PREADY`, executed for the partitions specified by the elements

$$\text{array_of_partitions}[0], \dots, \text{array_of_partitions}[\text{count} - 1]$$

of the `array_of_partitions`, executed in some arbitrary order. Calls to `MPI_PREADY_LIST` follow the same rules as those for `MPI_PREADY` calls.

4.2.2 Communication Completion under Partitioning

The functions `MPI_WAIT` and `MPI_TEST` (and variants) are used to complete a partitioned communication operation. The completion of a partitioned send operation indicates that the sender is now free to call `MPI_START/MPI_STARTALL` to restart the operation and subsequently `MPI_PREADY`, `MPI_PREADY_RANGE` or `MPI_PREADY_LIST`. Alternatively, the user can safely free the partitioned communication request after the completion of the partitioned operation. For the sending process, completion of the partitioned send operation does not indicate that the partitions of the message have all been received.

The completion of a partitioned receive operation through `MPI_WAIT` or `MPI_TEST` indicates that the receive buffer contains all of the partitions. A function for probing the partial reception of the receive buffer is provided by `MPI_PARRIVED`. The `MPI_PARRIVED` function can be used to determine if the message data for the indicated partition has been received into the receive buffer. Upon success, the receiver becomes free to access the indicated partition (as well as any others that previously completed for that operation).

```

MPI_PARRIVED(request, partition, flag)

```

IN	request	partitioned communication request (handle)
IN	partition	partition to be tested (nonnegative integer)
OUT	flag	true if operation completed on the specified partition, false if not (logical)

C binding

```

int MPI_Parrived(MPI_Request request, int partition, int *flag)

```

Fortran 2008 binding

```

MPI_Parrived(request, partition, flag, ierror)
    TYPE(MPI_Request), INTENT(IN) :: request
    INTEGER, INTENT(IN) :: partition
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_PARRIVED(REQUEST, PARTITION, FLAG, IERROR)
    INTEGER REQUEST, PARTITION, IERROR
    LOGICAL FLAG

```

The function `MPI_PARRIVED` can be used to test partial completion of partitioned receive operations. A call to `MPI_PARRIVED` on an active partitioned communication request returns `flag = true` if the operation identified by `request` for the specified `partition` is complete. The request is not marked as complete/inactive by this procedure. A subsequent call to an MPI completing procedure (e.g., `MPI_TEST/MPI_WAIT`) is required to complete the operation, as described in Chapter 3. `MPI_PARRIVED` may be called multiple times for a partition. `MPI_PARRIVED` may be called with a null or inactive `request` argument. In either case, the operation returns with `flag = true`. Calling `MPI_PARRIVED` on a request that does not correspond to a partitioned receive operation is erroneous.

Repeated calls to `MPI_PARRIVED` with the same `request` and `partition` arguments will eventually return `flag = true` if the corresponding partitioned send operation has been started and all send partitions have been marked as ready. For additional information on MPI *progress* see Sections 2.9 and 3.7.4.

Advice to implementors. A high quality implementation will eventually return `flag = true` from `MPI_PARRIVED` after all of the corresponding `MPI_PREADY` calls have been made for a receive-side partition, even if other send partitions are not yet marked as ready. (*End of advice to implementors.*)

4.2.3 Semantics of Communications in Partitioned Mode

The semantics of nonblocking partitioned communication are defined by suitably extending the definitions in Section 3.5.

Interpretation of count and datatype for partitioned communication. Partitioned communication uses the `count` and `datatype` arguments in the partitioned communication initialization functions to describe a single partition. The argument `partitions` specifies how many equal partitions of a number (`count`) of objects of `datatypes` make up the entire buffer to be transferred in the partitioned communication. As partitioned communication describes many partitions, using absolute displacements in `datatypes` (e.g., `MPI_BOTTOM`) is not supported. Partitions are contiguous in memory, there is no padding in between them. Once a partitioned send operation is started, each partition must be marked as ready using `MPI_PREADY` and the operation must be completed using a completion function, such as `MPI_TEST` or `MPI_WAIT`.

Order. Matching follows the same MPI matching rules as for point-to-point communication (see Chapter 3) with communicator, tag, and source dictating message matching. In the event that the communicator, tag, and source do not uniquely identify the message, the order in which partitioned communication initialization calls are made is the order in which they will eventually match.

4.3 Partitioned Communication Examples

This section provides concrete examples of the utility of partitioned communication in realistic settings.

4.3.1 Partition Communication with Threads/Tasks Using OpenMP 4.0 or later

In the following the equal partitioning on send-side and receive-side Example 4.1 is extended to utilize threads. In this case, the receive-side uses the same number of partitions as the send-side as in the previous example, but this example uses multiple threads on the send-side. Note that the `MPI_PSEND_INIT` and `MPI_PRECV_INIT` functions match each other like in the previous example.

Example 4.2. Equal partitioning on send-side and receive-side using threads.

```

1  #include <stdlib.h>
2  #include "mpi.h"
3  #define NUM_THREADS 8
4  #define PARTITIONS 8
5  #define PARTLENGTH 16
6  int main(int argc, char *argv[]) /* same send/recv partitioning */
7  {
8      double message[PARTITIONS*PARTLENGTH];
9      int partitions = PARTITIONS;
10     int partlength = PARTLENGTH;
11     int count = 1, source = 0, dest = 1, tag = 1, flag = 0;
12     int myrank;
13     int provided;
14     MPI_Request request;
15     MPI_Info info = MPI_INFO_NULL;
16     MPI_Datatype xfer_type;
17     MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
18     if (provided < MPI_THREAD_MULTIPLE)
19         MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
20     MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
21     MPI_Type_contiguous(partlength, MPI_DOUBLE, &xfer_type);
22     MPI_Type_commit(&xfer_type);
23     if (myrank == 0)
24     {
25         MPI_Psend_init(message, partitions, count, xfer_type, dest, tag,
26                        MPI_COMM_WORLD, info, &request);
27         MPI_Start(&request);
28
29         #pragma omp parallel for shared(request) num_threads(NUM_THREADS)
30         for (int i=0; i<partitions; i++)
31         {
32             /* compute and fill partition #i, then mark ready: */
33             MPI_Pready(i, request);
34         }
35         while(!flag)
36         {
37             /* Do useful work */
38             MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
39             /* Do useful work */
40         }
41         MPI_Request_free(&request);
42     }
43     else if (myrank == 1)

```

```

{
    MPI_Precv_init(message, partitions, count, xfer_type, source, tag,
                  MPI_COMM_WORLD, info, &request);
    MPI_Start(&request);
    while(!flag)
    {
        /* Do useful work */
        MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
        /* Do useful work */
    }
    MPI_Request_free(&request);
}
MPI_Finalize();
return 0;
}

```

4.3.2 Send-only Partitioning Example with Tasks and OpenMP version 4.0 or later

The previous example is tailored specifically for send-side partitioning using threads. This is an example where parallel task producers produce input to part of an overall buffer; they complete in any order and contribute to the overall buffer.

Example 4.3. Parallel task producers for partitioned communication using threads.

```

#include <stdlib.h>
#include "mpi.h"
#define NUM_THREADS 8
#define NUM_TASKS 64
#define PARTITIONS NUM_TASKS
#define PARTLENGTH 16
#define MESSAGE_LENGTH PARTITIONS*PARTLENGTH
int main(int argc, char *argv[]) /* send-side partitioning */
{
    double message[MESSAGE_LENGTH];
    int send_partitions = PARTITIONS,
        send_partlength = PARTLENGTH,
        recv_partitions = 1,
        recv_partlength = PARTITIONS*PARTLENGTH;
    int count = 1, source = 0, dest = 1, tag = 1, flag = 0;
    int myrank;
    int provided;
    MPI_Request request;
    MPI_Info info = MPI_INFO_NULL;
    MPI_Datatype send_type;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    if (provided < MPI_THREAD_MULTIPLE)
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
    MPI_Type_contiguous(send_partlength, MPI_DOUBLE, &send_type);
    MPI_Type_commit(&send_type);

    if (myrank == 0)
    {
        MPI_Psend_init(message, send_partitions, count, send_type, dest, tag,
                      MPI_COMM_WORLD, info, &request);
    }
}

```

```

1  MPI_Start(&request);
2
3  #pragma omp parallel shared(request) num_threads(NUM_THREADS)
4  {
5      #pragma omp single
6      {
7          /* single thread creates 64 tasks to be executed by 8 threads */
8          for (int partition_num=0;partition_num<NUM_TASKS;partition_num++)
9          {
10             #pragma omp task firstprivate(partition_num)
11             {
12                 /* compute and fill partition #partition_num, then mark
13                 ready: */
14                 /* buffer is filled in arbitrary order from each task */
15                 MPI_Pready(partition_num, request);
16             } /*end task*/
17         } /* end for */
18     } /* end single */
19 } /* end parallel */
20 while(!flag)
21 {
22     /* Do useful work */
23     MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
24     /* Do useful work */
25 }
26 MPI_Request_free(&request);
27 }
28 else if (myrank == 1)
29 {
30     MPI_Precv_init(message, recv_partitions, recv_partlength, MPI_DOUBLE,
31                    source, tag, MPI_COMM_WORLD, info, &request);
32
33     MPI_Start(&request);
34     while(!flag)
35     {
36         /* Do useful work */
37         MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
38         /* Do useful work */
39     }
40     MPI_Request_free(&request);
41 }
42 MPI_Finalize();
43 return 0;
44 }

```

4.3.3 Send and Receive Partitioning Example with OpenMP version 4.0 or later

This example demonstrates receive-side partial completion notification using more than one partition per receive-side thread. It uses a naive flag based method to test for multiple completed partitions per thread. Note that this means that some threads may be busy polling for completion of assigned partitions when partitions are available to work on that were not assigned to the polling threads in this example. More advanced work stealing methods could be employed for greater efficiency. Like previous examples, it also demonstrates send-side production of input to part of an overall buffer. This example also uses different send-side and receive-side partitioning.

Example 4.4. Partitioned communication receive-side partial completion.

```

1  #include <stdlib.h>
2
3  #include "mpi.h"
4
5  #define NUM_THREADS 64
6
7  #define PARTITIONS NUM_THREADS
8
9  #define PARTLENGTH 16
10
11 #define MESSAGE_LENGTH PARTITIONS*PARTLENGTH
12
13 int main(int argc, char *argv[]) /* send-side partitioning */
14 {
15     double message[MESSAGE_LENGTH];
16     int send_partitions = PARTITIONS,
17         send_partlength = PARTLENGTH,
18         recv_partitions = PARTITIONS*2,
19         recv_partlength = PARTLENGTH/2;
20
21     int source = 0, dest = 1, tag = 1, flag = 0;
22     int myrank;
23     int provided;
24     MPI_Request request;
25     MPI_Info info = MPI_INFO_NULL;
26     MPI_Datatype send_type;
27     MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
28     if (provided < MPI_THREAD_MULTIPLE)
29         MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
30     MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
31     MPI_Type_contiguous(send_partlength, MPI_DOUBLE, &send_type);
32     MPI_Type_commit(&send_type);
33
34     if (myrank == 0)
35     {
36         MPI_Psend_init(message, send_partitions, 1, send_type, dest, tag,
37             MPI_COMM_WORLD, info, &request);
38         MPI_Start(&request);
39         #pragma omp parallel for shared(request) \
40             firstprivate(send_partitions) \
41             num_threads(NUM_THREADS)
42         for (int i=0; i<send_partitions; i++)
43         {
44             /* compute and fill partition #i, then mark ready: */
45             MPI_Pready(i, request);
46         }
47         while(!flag)
48         {
49             /* Do useful work */
50             MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
51             /* Do useful work */
52         }
53         MPI_Request_free(&request);
54     }
55     else if (myrank == 1)
56     {
57         MPI_Precv_init(message, recv_partitions, recv_partlength,

```

```

1      MPI_DOUBLE, source, tag, MPI_COMM_WORLD, info,
2      &request);
3      MPI_Start(&request);
4      #pragma omp parallel for shared(request) \
5          firstprivate(recv_partitions) \
6          num_threads(NUM_THREADS)
7      for (int j=0; j<recv_partitions; j+=2)
8      {
9          int part_flag = 0;
10         int part1_complete = 0;
11         int part2_complete = 0;
12         while(part1_complete == 0 || part2_complete == 0)
13         {
14             /* test partition #j and #j+1 */
15             MPI_Parrived(request, j, &part_flag);
16             if(part_flag && part1_complete == 0)
17             {
18                 part1_complete++;
19                 /* Do work using partition j data */
20             }
21             MPI_Parrived(request, j+1, &part_flag);
22             if(part_flag && part2_complete == 0)
23             {
24                 part2_complete++;
25                 /* Do work using partition j+1 */
26             }
27         }
28     }
29     while(!flag)
30     {
31         /* Do useful work */
32         MPI_Test(&request, &flag, MPI_STATUS_IGNORE);
33         /* Do useful work */
34     }
35     MPI_Request_free(&request);
36 }
37
38
39
40
41
42
43
44
45
46
47
48

```

Chapter 5

Datatypes

Basic datatypes are introduced in Section 3.2.2 and in Section 3.3. In this chapter, this model is extended to describe any data layout. We consider general datatypes that allow one to transfer efficiently heterogeneous and noncontiguous data. We conclude with the description of calls for explicit packing and unpacking of messages.

5.1 Derived Datatypes

Point-to-point communications on buffers containing a sequence of identical basic datatypes is constraining. One often wants to pass messages that contain values with different datatypes (e.g., an integer count, followed by a sequence of real numbers); and one often wants to send noncontiguous data (e.g., a sub-block of a matrix). One solution is to pack noncontiguous data into a contiguous buffer at the sender site and unpack it at the receiver site. This has the disadvantage of requiring additional memory-to-memory copy operations at both sites, even when the communication subsystem has scatter-gather capabilities. Instead, MPI provides mechanisms to specify more general, mixed, and non-contiguous communication buffers. It is up to the implementation to decide whether data should be first packed in a contiguous buffer before being transmitted, or whether it can be collected directly from where it resides.

The general mechanisms provided here allow one to transfer directly, without copying, objects of various shapes and sizes. It is not assumed that the MPI library is cognizant of the objects declared in the host language. Thus, if one wants to transfer a structure, or an array section, it will be necessary to provide in MPI a definition of a communication buffer that mimics the definition of the structure or array section in question. These facilities can be used by library designers to define communication functions that can transfer objects defined in the host language—by decoding their definitions as available in a symbol table or a dope vector. Such higher-level communication functions are not part of MPI.

More general communication buffers are specified by replacing the basic datatypes that have been used so far with derived datatypes that are constructed from basic datatypes using the constructors described in this section. These methods of constructing derived datatypes can be applied recursively.

A **general datatype** is an opaque object that specifies two things:

- A sequence of basic datatypes.
- A sequence of integer (byte) displacements.

The displacements are not required to be positive, distinct, or in increasing order. Therefore, the order of items need not coincide with their order in store, and an item may appear more than once. We call such a pair of sequences (or sequence of pairs) a **type**

map. The sequence of basic datatypes (displacements ignored) is the **type signature** of the datatype.

Let

$$Typemap = \{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

be such a type map, where $type_i$ are basic types, and $disp_i$ are displacements. Let

$$Typesig = \{type_0, \dots, type_{n-1}\}$$

be the associated type signature. This type map, together with a base address **buf**, specifies a communication buffer: the communication buffer that consists of n entries, where the i -th entry is at address $buf + disp_i$ and has type $type_i$. A message assembled from such a communication buffer will consist of n values, of the types defined by *Typesig*.

Most datatype constructors have replication count or block length arguments. Allowed values are nonnegative integers. If the value is zero, no elements are generated in the type map and there is no effect on datatype bounds or extent.

We can use a handle to a general datatype as an argument in a send or receive operation, instead of a basic datatype argument. The operation `MPI_SEND(buf, 1, datatype, ...)` will use the send buffer defined by the base address **buf** and the general datatype associated with **datatype**; it will generate a message with the type signature determined by the **datatype** argument. `MPI_RECV(buf, 1, datatype, ...)` will use the receive buffer defined by the base address **buf** and the general datatype associated with **datatype**.

General datatypes can be used in all send and receive operations. We discuss, in Section 5.1.11, the case where the second argument **count** has value > 1 .

The basic datatypes presented in Section 3.2.2 are particular cases of a general datatype, and are predefined. Thus, `MPI_INT` is a predefined handle to a datatype with type map $\{(int, 0)\}$, with one entry of type `int` and displacement zero. The other basic datatypes are similar.

The **extent** of a datatype is defined to be the span from the first byte to the last byte occupied by entries in this datatype, rounded up to satisfy alignment requirements. That is, if

$$Typemap = \{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

then

$$\begin{aligned} lb(Typemap) &= \min_j disp_j, \\ ub(Typemap) &= \max_j (disp_j + \text{sizeof}(type_j)) + \epsilon, \text{ and} \\ extent(Typemap) &= ub(Typemap) - lb(Typemap). \end{aligned} \tag{5.1}$$

If $type_j$ requires alignment to a byte address that is a multiple of k_j , then ϵ is the least nonnegative increment needed to round $extent(Typemap)$ to the next multiple of $\max_j k_j$. In Fortran, it is implementation dependent whether the MPI implementation computes the alignments k_j according to the alignments used by the compiler in common blocks, SEQUENCE derived types, BIND(C) derived types, or derived types that are neither SEQUENCE nor BIND(C). The complete definition of **extent** is given by Equation 5.1.

Example 5.1. Assume that $Type = \{(double, 0), (char, 8)\}$ (a `double` at displacement zero, followed by a `char` at displacement eight). Assume, furthermore, that doubles have to be strictly aligned at addresses that are multiples of eight. Then, the extent of this datatype is 16 (9 rounded to the next multiple of 8). A datatype that consists of a character immediately followed by a double will also have an extent of 16.

Rationale. The definition of extent is motivated by the assumption that the amount of padding added at the end of each structure in an array of structures is the least needed to fulfill alignment constraints. More explicit control of the extent is provided in Section 5.1.6. Such explicit control is needed in cases where the assumption does not hold, for example, where union types are used. In Fortran, structures can be expressed with several language features, e.g., common blocks, SEQUENCE derived types, or BIND(C) derived types. The compiler may use different alignments, and therefore, it is recommended to use `MPI_TYPE_CREATE_RESIZED` for arrays of structures if an alignment may cause an alignment-gap at the end of a structure as described in Section 5.1.6 and in Section 21.1.15. (*End of rationale.*)

5.1.1 Type Constructors with Explicit Addresses

In Fortran, the procedures `MPI_TYPE_CREATE_HVECTOR`, `MPI_TYPE_CREATE_HINDEXED`, `MPI_TYPE_CREATE_HINDEXED_BLOCK`, `MPI_TYPE_CREATE_STRUCT`, and `MPI_GET_ADDRESS` accept arguments of type `INTEGER(KIND=MPI_ADDRESS_KIND)`, wherever arguments of type `MPI_Aint` are used in C. For Fortran compilers that do not support the Fortran 90 `KIND` notation, and where addresses are 64 bits whereas default `INTEGER`s are 32 bits, these arguments will be of type `INTEGER*8` (assuming the Fortran compiler accepts the common extension of `INTEGER*8` for eight-byte integers).

For the large count versions of three datatype constructors with explicit addresses, `MPI_TYPE_CREATE_HINDEXED`, `MPI_TYPE_CREATE_HINDEXED_BLOCK`, and `MPI_TYPE_CREATE_STRUCT`, absolute addresses shall not be used to specify byte displacements since the parameter is of type `MPI_COUNT` instead of type `MPI_AINT`.

5.1.2 Datatype Constructors

Contiguous. The simplest datatype constructor is `MPI_TYPE_CONTIGUOUS`, which allows replication of a datatype into contiguous locations.

`MPI_TYPE_CONTIGUOUS(count, oldtype, newtype)`

IN	count	replication count (nonnegative integer)
IN	oldtype	old datatype (handle)
OUT	newtype	new datatype (handle)

C binding

`int MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype *newtype)`

```

1  int MPI_Type_contiguous_c(MPI_Count count, MPI_Datatype oldtype,
2      MPI_Datatype *newtype)
3

```

Fortran 2008 binding

```

4  MPI_Type_contiguous(count, oldtype, newtype, ierror)
5      INTEGER, INTENT(IN) :: count
6      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
7      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
8      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
9
10 MPI_Type_contiguous(count, oldtype, newtype, ierror) !(_c)
11     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
12     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
13     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15

```

Fortran binding

```

16 MPI_TYPE_CONTIGUOUS(COUNT, OLDDTYPE, NEWTYPE, IERROR)
17     INTEGER COUNT, OLDDTYPE, NEWTYPE, IERROR
18

```

newtype is the datatype obtained by concatenating *count* copies of *oldtype*. Concatenation is defined using *extent* as the size of the concatenated copies.

Example 5.2. Let *oldtype* have type map $\{(\text{double}, 0), (\text{char}, 8)\}$, with extent 16, and let *count* = 3. The type map of the datatype returned by *newtype* is

$$\{(\text{double}, 0), (\text{char}, 8), (\text{double}, 16), (\text{char}, 24), (\text{double}, 32), (\text{char}, 40)\};$$

i.e., alternating *double* and *char* elements, with displacements 0, 8, 16, 24, 32, 40.

In general, assume that the type map of *oldtype* is

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

with extent *ex*. Then *newtype* has a type map with *count* · *n* entries defined by:

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1}), (type_0, disp_0 + ex), \dots, (type_{n-1}, disp_{n-1} + ex), \\ \dots, (type_0, disp_0 + ex \cdot (count - 1)), \dots, (type_{n-1}, disp_{n-1} + ex \cdot (count - 1))\}.$$

Vector. The procedure `MPI_TYPE_VECTOR` is a more general constructor that allows replication of a datatype into locations that consist of equally spaced blocks. Each block is obtained by concatenating the same number of copies of the old datatype. The spacing between blocks is a multiple of the extent of the old datatype.

`MPI_TYPE_VECTOR(count, blocklength, stride, oldtype, newtype)`

IN	count	number of blocks (nonnegative integer)
IN	blocklength	number of elements in each block (nonnegative integer)
IN	stride	number of elements between start of each block (integer)
IN	oldtype	old datatype (handle)
OUT	newtype	new datatype (handle)

C binding

```
int MPI_Type_vector(int count, int blocklength, int stride,
                   MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_vector_c(MPI_Count count, MPI_Count blocklength, MPI_Count stride,
                     MPI_Datatype oldtype, MPI_Datatype *newtype)
```

Fortran 2008 binding

```
MPI_Type_vector(count, blocklength, stride, oldtype, newtype, ierror)
  INTEGER, INTENT(IN) :: count, blocklength, stride
  TYPE(MPI_Datatype), INTENT(IN) :: oldtype
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Type_vector(count, blocklength, stride, oldtype, newtype, ierror) !(_c)
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength, stride
  TYPE(MPI_Datatype), INTENT(IN) :: oldtype
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_VECTOR(COUNT, BLOCKLENGTH, STRIDE, OLDTYPE, NEWTYPE, IERROR)
  INTEGER COUNT, BLOCKLENGTH, STRIDE, OLDTYPE, NEWTYPE, IERROR
```

Example 5.3. Assume, again, that `oldtype` has type map $\{(\text{double}, 0), (\text{char}, 8)\}$, with extent 16. A call to `MPI_TYPE_VECTOR(2, 3, 4, oldtype, newtype)` will create the datatype with type map,

$$\{(\text{double}, 0), (\text{char}, 8), (\text{double}, 16), (\text{char}, 24), (\text{double}, 32), (\text{char}, 40),$$

$$(\text{double}, 64), (\text{char}, 72), (\text{double}, 80), (\text{char}, 88), (\text{double}, 96), (\text{char}, 104)\}.$$

That is, two blocks with three copies each of the old type, with a stride of 4 elements ($4 \cdot 16$ bytes) between the start of each block.

Example 5.4. A call to `MPI_TYPE_VECTOR(3, 1, -2, oldtype, newtype)` will create the datatype,

$$\{(\text{double}, 0), (\text{char}, 8), (\text{double}, -32), (\text{char}, -24), (\text{double}, -64), (\text{char}, -56)\}.$$

In general, assume that `oldtype` has type map,

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

with extent ex . Let `bl` be the `blocklength`. The newly created datatype has a type map with `count · bl · n` entries:

$$\begin{aligned} &\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1}), \\ &(type_0, disp_0 + ex), \dots, (type_{n-1}, disp_{n-1} + ex), \dots, \\ &(type_0, disp_0 + (bl - 1) \cdot ex), \dots, (type_{n-1}, disp_{n-1} + (bl - 1) \cdot ex), \\ &(type_0, disp_0 + stride \cdot ex), \dots, (type_{n-1}, disp_{n-1} + stride \cdot ex), \dots, \\ &(type_0, disp_0 + (stride + bl - 1) \cdot ex), \dots, (type_{n-1}, disp_{n-1} + (stride + bl - 1) \cdot ex), \dots, \\ &(type_0, disp_0 + stride \cdot (count - 1) \cdot ex), \dots, \\ &(type_{n-1}, disp_{n-1} + stride \cdot (count - 1) \cdot ex), \dots, \\ &(type_0, disp_0 + (stride \cdot (count - 1) + bl - 1) \cdot ex), \dots, \\ &(type_{n-1}, disp_{n-1} + (stride \cdot (count - 1) + bl - 1) \cdot ex)\}. \end{aligned}$$

A call to `MPI_TYPE_CONTIGUOUS(count, oldtype, newtype)` is equivalent to a call to `MPI_TYPE_VECTOR(count, 1, 1, oldtype, newtype)`, or to a call to `MPI_TYPE_VECTOR(1, count, n, oldtype, newtype)`, where `n` is an arbitrary integer value.

Hvector. The procedure `MPI_TYPE_CREATE_HVECTOR` is identical to `MPI_TYPE_VECTOR`, except that `stride` is given in bytes, rather than in elements. The use for both types of vector constructors is illustrated in Section 5.1.14. (H stands for “heterogeneous”).

`MPI_TYPE_CREATE_HVECTOR(count, blocklength, stride, oldtype, newtype)`

IN	count	number of blocks (nonnegative integer)
IN	blocklength	number of elements in each block (nonnegative integer)
IN	stride	number of bytes between start of each block (integer)
IN	oldtype	old datatype (handle)
OUT	newtype	new datatype (handle)

C binding

```
int MPI_Type_create_hvector(int count, int blocklength, MPI_Aint stride,
                           MPI_Datatype oldtype, MPI_Datatype *newtype)

int MPI_Type_create_hvector_c(MPI_Count count, MPI_Count blocklength,
                              MPI_Count stride, MPI_Datatype oldtype, MPI_Datatype *newtype)
```


Fortran 2008 binding

```

MPI_Type_create_hvector(count, blocklength, stride, oldtype, newtype, ierror)
  INTEGER, INTENT(IN) :: count, blocklength
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: stride
  TYPE(MPI_Datatype), INTENT(IN) :: oldtype
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_hvector(count, blocklength, stride, oldtype, newtype, ierror)
  !(_c)
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength, stride
  TYPE(MPI_Datatype), INTENT(IN) :: oldtype
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_CREATE_HVECTOR(COUNT, BLOCKLENGTH, STRIDE, OLDTYPE, NEWTYPE, IERROR)
  INTEGER COUNT, BLOCKLENGTH, OLDTYPE, NEWTYPE, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) STRIDE

```

Assume that `oldtype` has type map,

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

with extent ex . Let bl be the blocklength. The newly created datatype has a type map with $count \cdot bl \cdot n$ entries:

$$\begin{aligned}
&\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1}), \\
&(type_0, disp_0 + ex), \dots, (type_{n-1}, disp_{n-1} + ex), \dots, \\
&(type_0, disp_0 + (bl - 1) \cdot ex), \dots, (type_{n-1}, disp_{n-1} + (bl - 1) \cdot ex), \\
&(type_0, disp_0 + stride), \dots, (type_{n-1}, disp_{n-1} + stride), \dots, \\
&(type_0, disp_0 + stride + (bl - 1) \cdot ex), \dots, \\
&(type_{n-1}, disp_{n-1} + stride + (bl - 1) \cdot ex), \dots, \\
&(type_0, disp_0 + stride \cdot (count - 1)), \dots, (type_{n-1}, disp_{n-1} + stride \cdot (count - 1)), \dots, \\
&(type_0, disp_0 + stride \cdot (count - 1) + (bl - 1) \cdot ex), \dots, \\
&(type_{n-1}, disp_{n-1} + stride \cdot (count - 1) + (bl - 1) \cdot ex)\}.
\end{aligned}$$

Indexed. The procedure `MPI_TYPE_INDEXED` allows replication of an old datatype into a sequence of blocks (each block is a concatenation of the old datatype), where each block can contain a different number of copies and have a different displacement. All block displacements are multiples of the old type extent.

```

1 MPI_TYPE_INDEXED(count, array_of_blocklengths, array_of_displacements, oldtype,
2                   newtype)
3
4     IN          count          number of blocks—also number of entries in
5                                array_of_displacements and array_of_blocklengths
6                                (nonnegative integer)
7
8     IN          array_of_blocklengths  number of elements per block (array of nonnegative
9                                integers)
10
11    IN          array_of_displacements  displacement for each block, in multiples of oldtype
12                                (array of integers)
13
14    IN          oldtype              old datatype (handle)
15
16    OUT         newtype              new datatype (handle)

```

C binding

```

17 int MPI_Type_indexed(int count, const int array_of_blocklengths[],
18                     const int array_of_displacements[], MPI_Datatype oldtype,
19                     MPI_Datatype *newtype)
20
21 int MPI_Type_indexed_c(MPI_Count count,
22                       const MPI_Count array_of_blocklengths[],
23                       const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
24                       MPI_Datatype *newtype)

```

Fortran 2008 binding

```

25 MPI_Type_indexed(count, array_of_blocklengths, array_of_displacements, oldtype,
26                 newtype, ierror)
27
28     INTEGER, INTENT(IN) :: count, array_of_blocklengths(count),
29     array_of_displacements(count)
30
31     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
32
33     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
34
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37 MPI_Type_indexed(count, array_of_blocklengths, array_of_displacements, oldtype,
38                 newtype, ierror) !(_c)
39
40     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count,
41     array_of_blocklengths(count), array_of_displacements(count)
42
43     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
44
45     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
46
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

48 MPI_TYPE_INDEXED(COUNT, ARRAY_OF_BLOCKLENGTHS, ARRAY_OF_DISPLACEMENTS, OLDTYPE,
49                 NEWTYPE, IERROR)
50
51     INTEGER COUNT, ARRAY_OF_BLOCKLENGTHS(*), ARRAY_OF_DISPLACEMENTS(*),
52     OLDTYPE, NEWTYPE, IERROR

```

Example 5.5. Let `oldtype` have type map $\{(double, 0), (char, 8)\}$, with extent 16. Let $B = (3, 1)$ and let $D = (4, 0)$. A call to `MPI_TYPE_INDEXED(2, B, D, oldtype, newtype)` returns a datatype with type map,

```
{(double, 64), (char, 72), (double, 80), (char, 88), (double, 96), (char, 104),
```

```
(double, 0), (char, 8)}.
```

That is, three copies of the old type starting at displacement 64, and one copy starting at displacement 0.

In general, assume that `oldtype` has type map,

```
{(type0, disp0), ..., (typen-1, dispn-1)},
```

with extent `ex`. Let `B` be the `array_of_blocklengths` argument and `D` be the `array_of_displacements` argument. The newly created datatype has $n \cdot \sum_{i=0}^{\text{count}-1} B[i]$ entries:

```
{(type0, disp0 + D[0] · ex), ..., (typen-1, dispn-1 + D[0] · ex), ...,
 (type0, disp0 + (D[0] + B[0] - 1) · ex), ...,
 (typen-1, dispn-1 + (D[0] + B[0] - 1) · ex), ...,
 (type0, disp0 + D[count-1] · ex), ..., (typen-1, dispn-1 + D[count-1] · ex), ...,
 (type0, disp0 + (D[count-1] + B[count-1] - 1) · ex), ...,
 (typen-1, dispn-1 + (D[count-1] + B[count-1] - 1) · ex)}.
```

A call to `MPI_TYPE_VECTOR(count, blocklength, stride, oldtype, newtype)` is equivalent to a call to `MPI_TYPE_INDEXED(count, B, D, oldtype, newtype)` where

```
D[j] = j · stride, j = 0, ..., count - 1,
```

and

```
B[j] = blocklength, j = 0, ..., count - 1.
```

Hindexed. The procedure `MPI_TYPE_CREATE_HINDEXED` is identical to `MPI_TYPE_INDEXED`, except that block displacements in `array_of_displacements` are specified in bytes, rather than in multiples of the `oldtype` extent.

```
MPI_TYPE_CREATE_HINDEXED(count, array_of_blocklengths, array_of_displacements,
                          oldtype, newtype)
```

IN	count	number of blocks—also number of entries in <code>array_of_displacements</code> and <code>array_of_blocklengths</code> (nonnegative integer)
IN	array_of_blocklengths	number of elements in each block (array of nonnegative integers)
IN	array_of_displacements	byte displacement of each block (array of integers)
IN	oldtype	old datatype (handle)
OUT	newtype	new datatype (handle)

C binding

```
int MPI_Type_create_hindexed(int count, const int array_of_blocklengths[],
                             const MPI_Aint array_of_displacements[], MPI_Datatype oldtype,
                             MPI_Datatype *newtype)
```

```

1  int MPI_Type_create_hindexed_c(MPI_Count count,
2                                const MPI_Count array_of_blocklengths[],
3                                const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
4                                MPI_Datatype *newtype)

```

Fortran 2008 binding

```

6  MPI_Type_create_hindexed(count, array_of_blocklengths, array_of_displacements,
7                            oldtype, newtype, ierror)
8
9  INTEGER, INTENT(IN) :: count, array_of_blocklengths(count)
10 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: array_of_displacements(count)
11 TYPE(MPI_Datatype), INTENT(IN) :: oldtype
12 TYPE(MPI_Datatype), INTENT(OUT) :: newtype
13 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
14
15 MPI_Type_create_hindexed(count, array_of_blocklengths, array_of_displacements,
16                            oldtype, newtype, ierror) !(_c)
17 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count,
18     array_of_blocklengths(count), array_of_displacements(count)
19 TYPE(MPI_Datatype), INTENT(IN) :: oldtype
20 TYPE(MPI_Datatype), INTENT(OUT) :: newtype
21 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

22 MPI_TYPE_CREATE_HINDEXED(COUNT, ARRAY_OF_BLOCKLENGTHS, ARRAY_OF_DISPLACEMENTS,
23                           OLDTYPE, NEWTYPE, IERROR)
24
25 INTEGER COUNT, ARRAY_OF_BLOCKLENGTHS(*), OLDTYPE, NEWTYPE, IERROR
26 INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_DISPLACEMENTS(*)

```

Assume that oldtype has type map,

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

with extent ex . Let B be the `array_of_blocklengths` argument and D be the `array_of_displacements` argument. The newly created datatype has a type map with $n \cdot \sum_{i=0}^{count-1} B[i]$ entries:

$$\begin{aligned}
 &\{(type_0, disp_0 + D[0]), \dots, (type_{n-1}, disp_{n-1} + D[0]), \dots, \\
 &(type_0, disp_0 + D[0] + (B[0] - 1) \cdot ex), \dots, \\
 &(type_{n-1}, disp_{n-1} + D[0] + (B[0] - 1) \cdot ex), \dots, \\
 &(type_0, disp_0 + D[count-1]), \dots, (type_{n-1}, disp_{n-1} + D[count-1]), \dots, \\
 &(type_0, disp_0 + D[count-1] + (B[count-1] - 1) \cdot ex), \dots, \\
 &(type_{n-1}, disp_{n-1} + D[count-1] + (B[count-1] - 1) \cdot ex)\}.
 \end{aligned}$$

Indexed_block. This procedure is the same as `MPI_TYPE_INDEXED` except that the blocklength is the same for all blocks. There are many codes using indirect addressing arising from unstructured grids where the blocksize is always 1 (gather/scatter). The following convenience procedure allows for constant blocksize and arbitrary displacements.

MPI_TYPE_CREATE_INDEXED_BLOCK(count, blocklength, array_of_displacements,			1
oldtype, newtype)			2
			3
IN	count	number of blocks—also number of entries in	4
		array_of_displacements (nonnegative integer)	5
IN	blocklength	number of elements in each block (nonnegative	6
		integer)	7
IN	array_of_displacements	array of displacements, in multiples of oldtype (array	8
		of integers)	9
IN	oldtype	old datatype (handle)	10
OUT	newtype	new datatype (handle)	11

C binding

```

int MPI_Type_create_indexed_block(int count, int blocklength,
    const int array_of_displacements[], MPI_Datatype oldtype,
    MPI_Datatype *newtype)
int MPI_Type_create_indexed_block_c(MPI_Count count, MPI_Count blocklength,
    const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
    MPI_Datatype *newtype)

```

Fortran 2008 binding

```

MPI_Type_create_indexed_block(count, blocklength, array_of_displacements,
    oldtype, newtype, ierror)
    INTEGER, INTENT(IN) :: count, blocklength, array_of_displacements(count)
    TYPE(MPI_Datatype), INTENT(IN) :: oldtype
    TYPE(MPI_Datatype), INTENT(OUT) :: newtype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Type_create_indexed_block(count, blocklength, array_of_displacements,
    oldtype, newtype, ierror) !(_c)
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength,
    array_of_displacements(count)
    TYPE(MPI_Datatype), INTENT(IN) :: oldtype
    TYPE(MPI_Datatype), INTENT(OUT) :: newtype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_CREATE_INDEXED_BLOCK(COUNT, BLOCKLENGTH, ARRAY_OF_DISPLACEMENTS,
    OLDTYPE, NEWTYPE, IERROR)
    INTEGER COUNT, BLOCKLENGTH, ARRAY_OF_DISPLACEMENTS(*), OLDTYPE, NEWTYPE,
    IERROR

```

Hindexed_block. The procedure `MPI_TYPE_CREATE_HINDEXED_BLOCK` is identical to `MPI_TYPE_CREATE_INDEXED_BLOCK`, except that block displacements in `array_of_displacements` are specified in bytes, rather than in multiples of the `oldtype` extent.

```

1 MPI_TYPE_CREATE_HINDEXED_BLOCK(count, blocklength, array_of_displacements,
2     oldtype, newtype)
3
4     IN          count          number of blocks—also number of entries in
5                               array_of_displacements (nonnegative integer)
6
7     IN          blocklength    number of elements in each block (nonnegative
8                               integer)
9
10    IN          array_of_displacements  byte displacement of each block (array of integers)
11
12    IN          oldtype         old datatype (handle)
13
14    OUT         newtype         new datatype (handle)

```

C binding

```

13 int MPI_Type_create_hindexed_block(int count, int blocklength,
14     const MPI_Aint array_of_displacements[], MPI_Datatype oldtype,
15     MPI_Datatype *newtype)
16
17 int MPI_Type_create_hindexed_block_c(MPI_Count count, MPI_Count blocklength,
18     const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
19     MPI_Datatype *newtype)

```

Fortran 2008 binding

```

21 MPI_Type_create_hindexed_block(count, blocklength, array_of_displacements,
22     oldtype, newtype, ierror)
23     INTEGER, INTENT(IN) :: count, blocklength
24     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: array_of_displacements(count)
25     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
26     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
28
29 MPI_Type_create_hindexed_block(count, blocklength, array_of_displacements,
30     oldtype, newtype, ierror) !(_c)
31     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength,
32     array_of_displacements(count)
33     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
34     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

37 MPI_TYPE_CREATE_HINDEXED_BLOCK(COUNT, BLOCKLENGTH, ARRAY_OF_DISPLACEMENTS,
38     OLDTYPE, NEWTYPE, IERROR)
39     INTEGER COUNT, BLOCKLENGTH, OLDTYPE, NEWTYPE, IERROR
40     INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_DISPLACEMENTS(*)

```

Struct. `MPI_TYPE_CREATE_STRUCT` is the most general type constructor. It further generalizes `MPI_TYPE_CREATE_HINDEXED` in that it allows each block to consist of replications of different datatypes.

```

MPI_TYPE_CREATE_STRUCT(count, array_of_blocklengths, array_of_displacements,
                        array_of_types, newtype)
IN      count          number of blocks—also number of entries in arrays
                        array_of_types, array_of_displacements, and
                        array_of_blocklengths (nonnegative integer)
IN      array_of_blocklengths  number of elements in each block (array of
                        nonnegative integers)
IN      array_of_displacements  byte displacement of each block (array of integers)
IN      array_of_types        type of elements in each block (array of handles)
OUT     newtype            new datatype (handle)

```

C binding

```

int MPI_Type_create_struct(int count, const int array_of_blocklengths[],
                           const MPI_Aint array_of_displacements[],
                           const MPI_Datatype array_of_types[], MPI_Datatype *newtype)
int MPI_Type_create_struct_c(MPI_Count count,
                             const MPI_Count array_of_blocklengths[],
                             const MPI_Count array_of_displacements[],
                             const MPI_Datatype array_of_types[], MPI_Datatype *newtype)

```

Fortran 2008 binding

```

MPI_Type_create_struct(count, array_of_blocklengths, array_of_displacements,
                       array_of_types, newtype, ierror)
INTEGER, INTENT(IN) :: count, array_of_blocklengths(count)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: array_of_displacements(count)
TYPE(MPI_Datatype), INTENT(IN) :: array_of_types(count)
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Type_create_struct(count, array_of_blocklengths, array_of_displacements,
                       array_of_types, newtype, ierror) !(_c)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count,
array_of_blocklengths(count), array_of_displacements(count)
TYPE(MPI_Datatype), INTENT(IN) :: array_of_types(count)
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_CREATE_STRUCT(COUNT, ARRAY_OF_BLOCKLENGTHS, ARRAY_OF_DISPLACEMENTS,
                       ARRAY_OF_TYPES, NEWTYPE, IERROR)
INTEGER COUNT, ARRAY_OF_BLOCKLENGTHS(*), ARRAY_OF_TYPES(*), NEWTYPE, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_DISPLACEMENTS(*)

```

Example 5.6. Let type1 have type map,

$\{(\text{double}, 0), (\text{char}, 8)\},$

with extent 16. Let $B = (2, 1, 3)$, $D = (0, 16, 26)$, and $T = (\text{MPI_FLOAT}, \text{type1}, \text{MPI_CHAR})$. Then a call to `MPI_TYPE_CREATE_STRUCT(3, B, D, T, newtype)` returns a datatype with

type map,

$$\{(\text{float}, 0), (\text{float}, 4), (\text{double}, 16), (\text{char}, 24), (\text{char}, 26), (\text{char}, 27), (\text{char}, 28)\}.$$

That is, two copies of `MPI_FLOAT` starting at 0, followed by one copy of `type1` starting at 16, followed by three copies of `MPI_CHAR`, starting at 26. In this example, we assume that a float occupies four bytes.

In general, let `T` be the `array_of_types` argument, where `T[i]` is a handle to,

$$\text{typemap}_i = \{(type_0^i, disp_0^i), \dots, (type_{n_i-1}^i, disp_{n_i-1}^i)\},$$

with extent ex_i . Let `B` be the `array_of_blocklength` argument and `D` be the `array_of_displacements` argument. Let `c` be the count argument. Then the newly created datatype has a type map with $\sum_{i=0}^{c-1} B[i] \cdot n_i$ entries:

$$\begin{aligned} &\{(type_0^0, disp_0^0 + D[0]), \dots, (type_{n_0}^0, disp_{n_0}^0 + D[0]), \dots, \\ &(type_0^0, disp_0^0 + D[0] + (B[0] - 1) \cdot ex_0), \dots, (type_{n_0}^0, disp_{n_0}^0 + D[0] + (B[0]-1) \cdot ex_0), \dots, \\ &(type_0^{c-1}, disp_0^{c-1} + D[c-1]), \dots, (type_{n_{c-1}-1}^{c-1}, disp_{n_{c-1}-1}^{c-1} + D[c-1]), \dots, \\ &(type_0^{c-1}, disp_0^{c-1} + D[c-1] + (B[c-1] - 1) \cdot ex_{c-1}), \dots, \\ &(type_{n_{c-1}-1}^{c-1}, disp_{n_{c-1}-1}^{c-1} + D[c-1] + (B[c-1]-1) \cdot ex_{c-1})\}. \end{aligned}$$

A call to `MPI_TYPE_CREATE_HINDEXED(count, B, D, oldtype, newtype)` is equivalent to a call to `MPI_TYPE_CREATE_STRUCT(count, B, D, T, newtype)`, where each entry of `T` is equal to `oldtype`.

5.1.3 Subarray Datatype Constructor

`MPI_TYPE_CREATE_SUBARRAY(ndims, array_of_sizes, array_of_subsizes, array_of_starts, order, oldtype, newtype)`

IN	ndims	number of array dimensions (positive integer)
IN	array_of_sizes	number of elements of type <code>oldtype</code> in each dimension of the full array (array of positive integers)
IN	array_of_subsizes	number of elements of type <code>oldtype</code> in each dimension of the subarray (array of positive integers)
IN	array_of_starts	starting coordinates of the subarray in each dimension (array of nonnegative integers)
IN	order	array storage order flag (state)
IN	oldtype	old datatype (handle)
OUT	newtype	new datatype (handle)

C binding

```
int MPI_Type_create_subarray(int ndims, const int array_of_sizes[],
                             const int array_of_subsizes[], const int array_of_starts[],
                             int order, MPI_Datatype oldtype, MPI_Datatype *newtype)
```



```

int MPI_Type_create_subarray_c(int ndims, const MPI_Count array_of_sizes[],
                             const MPI_Count array_of_subsizes[],
                             const MPI_Count array_of_starts[], int order,
                             MPI_Datatype oldtype, MPI_Datatype *newtype)

```

Fortran 2008 binding

```

MPI_Type_create_subarray(ndims, array_of_sizes, array_of_subsizes,
                        array_of_starts, order, oldtype, newtype, ierror)
INTEGER, INTENT(IN) :: ndims, array_of_sizes(ndims),
                        array_of_subsizes(ndims), array_of_starts(ndims), order
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Type_create_subarray(ndims, array_of_sizes, array_of_subsizes,
                        array_of_starts, order, oldtype, newtype, ierror) !(_c)
INTEGER, INTENT(IN) :: ndims, order
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: array_of_sizes(ndims),
                        array_of_subsizes(ndims), array_of_starts(ndims)
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_CREATE_SUBARRAY(NDIMS, ARRAY_OF_SIZES, ARRAY_OF_SUBSIZES,
                        ARRAY_OF_STARTS, ORDER, OLDTYPE, NEWTYPE, IERROR)
INTEGER NDIMS, ARRAY_OF_SIZES(*), ARRAY_OF_SUBSIZES(*), ARRAY_OF_STARTS(*),
ORDER, OLDTYPE, NEWTYPE, IERROR

```

The subarray type constructor creates an MPI datatype describing an n -dimensional subarray of an n -dimensional array. The subarray may be situated anywhere within the full array, and may be of any nonzero size up to the size of the larger array as long as it is confined within this array. This type constructor facilitates creating filetypes to access arrays distributed in blocks among processes to a single file that contains the global array, see MPI I/O, especially Section 15.1.1.

This type constructor can handle arrays with an arbitrary number of dimensions and works for both C and Fortran ordered matrices (i.e., row-major or column-major). Note that a C program may use Fortran order and a Fortran program may use C order.

The `ndims` parameter specifies the number of dimensions in the full data array and gives the number of elements in `array_of_sizes`, `array_of_subsizes`, and `array_of_starts`.

The number of elements of type `oldtype` in each dimension of the n -dimensional array and the requested subarray are specified by `array_of_sizes` and `array_of_subsizes`, respectively. For any dimension i , it is erroneous to specify `array_of_subsizes[i] < 1` or `array_of_subsizes[i] > array_of_sizes[i]`.

The `array_of_starts` contains the starting coordinates of each dimension of the subarray. Arrays are assumed to be indexed starting from zero. For any dimension i , it is erroneous to specify `array_of_starts[i] < 0` or `array_of_starts[i] > (array_of_sizes[i] - array_of_subsizes[i])`.

Advice to users. In a Fortran program with arrays indexed starting from 1, if the starting coordinate of a particular dimension of the subarray is n , then the entry in `array_of_starts` for that dimension is $n-1$. (*End of advice to users.*)

The `order` argument specifies the storage order for the subarray as well as the full array. It must be set to one of the following:

<code>MPI_ORDER_C</code>	The ordering used by C arrays, (i.e., row-major order).
<code>MPI_ORDER_FORTRAN</code>	The ordering used by Fortran arrays, (i.e., column-major order).

A `ndims`-dimensional subarray (`newtype`) with no extra padding can be defined by the function `Subarray()` as follows:

```
newtype = Subarray(ndims, {size0, size1, ..., sizendims-1},
                  {subsize0, subsize1, ..., subsizendims-1},
                  {start0, start1, ..., startndims-1}, oldtype)
```

Let the typemap of `oldtype` have the form:

$$\{(type_0, disp_0), (type_1, disp_1), \dots, (type_{n-1}, disp_{n-1})\}$$

where $type_i$ is a predefined MPI datatype, and let ex be the extent of `oldtype`. Then we define the `Subarray()` function recursively using the following three equations. Equation 5.2 defines the base step. Equation 5.3 defines the recursion step when `order = MPI_ORDER_FORTRAN`, and Equation 5.4 defines the recursion step when `order = MPI_ORDER_C`. These equations use the conceptual datatypes `lb_marker` and `ub_marker`; see Section 5.1.6 for details.

$$\text{Subarray}(1, \{size_0\}, \{subsize_0\}, \{start_0\}, \quad (5.2)$$

$$\begin{aligned} & \{(type_0, disp_0), (type_1, disp_1), \dots, (type_{n-1}, disp_{n-1})\}) \\ = & \{(lb_marker, 0), \\ & (type_0, disp_0 + start_0 \times ex), \dots, (type_{n-1}, disp_{n-1} + start_0 \times ex), \\ & (type_0, disp_0 + (start_0 + 1) \times ex), \dots, (type_{n-1}, \\ & \quad disp_{n-1} + (start_0 + 1) \times ex), \dots \\ & (type_0, disp_0 + (start_0 + subsize_0 - 1) \times ex), \dots, \\ & (type_{n-1}, disp_{n-1} + (start_0 + subsize_0 - 1) \times ex), \\ & (ub_marker, size_0 \times ex)\} \end{aligned}$$

$$\text{Subarray}(ndims, \{size_0, size_1, \dots, size_{ndims-1}\}, \quad (5.3)$$

$$\begin{aligned} & \{subsize_0, subsize_1, \dots, subsize_{ndims-1}\}, \\ & \{start_0, start_1, \dots, start_{ndims-1}\}, oldtype) \\ = & \text{Subarray}(ndims - 1, \{size_1, size_2, \dots, size_{ndims-1}\}, \\ & \{subsize_1, subsize_2, \dots, subsize_{ndims-1}\}, \\ & \{start_1, start_2, \dots, start_{ndims-1}\}, \\ & \text{Subarray}(1, \{size_0\}, \{subsize_0\}, \{start_0\}, oldtype)) \end{aligned}$$

$$\text{Subarray}(ndims, \{size_0, size_1, \dots, size_{ndims-1}\}, \quad (5.4)$$

$$\{subsize_0, subsize_1, \dots, subsize_{ndims-1}\},$$

```

    {start0, start1, ..., startndims-1}, oldtype)
= Subarray(ndims - 1, {size0, size1, ..., sizendims-2},
    {subsize0, subsize1, ..., subsizendims-2},
    {start0, start1, ..., startndims-2},
    Subarray(1, {sizendims-1}, {subsizendims-1}, {startndims-1}, oldtype))

```

For an example use of `MPI_TYPE_CREATE_SUBARRAY` in the context of I/O see Section 15.9.2.

5.1.4 Distributed Array Datatype Constructor

The distributed array type constructor supports HPF-like [49] data distributions. However, unlike in HPF, the storage order may be specified for C arrays as well as for Fortran arrays.

Advice to users. One can create an HPF-like file view using this type constructor as follows. Complementary filetypes are created by having every process of a group call this constructor with identical arguments (with the exception of `rank` which should be set appropriately). These filetypes (along with identical `disp` and `etype`) are then used to define the view (via `MPI_FILE_SET_VIEW`), see MPI I/O, especially Section 15.1.1 and Section 15.3. Using this view, a collective data access operation (with identical offsets) will yield an HPF-like distribution pattern. (*End of advice to users.*)

```

MPI_TYPE_CREATE_DARRAY(size, rank, ndims, array_of_gsizes, array_of_distribs,
    array_of_dargs, array_of_psize, order, oldtype, newtype)

```

IN	size	size of process group (positive integer)
IN	rank	rank in process group (nonnegative integer)
IN	ndims	number of array dimensions as well as process grid dimensions (positive integer)
IN	array_of_gsizes	number of elements of type <code>oldtype</code> in each dimension of global array (array of positive integers)
IN	array_of_distribs	distribution of array in each dimension (array of states)
IN	array_of_dargs	distribution argument in each dimension (array of positive integers)
IN	array_of_psize	size of process grid in each dimension (array of positive integers)
IN	order	array storage order flag (state)
IN	oldtype	old datatype (handle)
OUT	newtype	new datatype (handle)

C binding

```

int MPI_Type_create_darray(int size, int rank, int ndims,
    const int array_of_gsizes[], const int array_of_distribs[],

```

```

1         const int array_of_dargs[], const int array_of_psizes[],
2         int order, MPI_Datatype oldtype, MPI_Datatype *newtype)
3
4 int MPI_Type_create_darray_c(int size, int rank, int ndims,
5         const MPI_Count array_of_gsizes[], const int array_of_distribs[],
6         const int array_of_dargs[], const int array_of_psizes[],
7         int order, MPI_Datatype oldtype, MPI_Datatype *newtype)

```

Fortran 2008 binding

```

9 MPI_Type_create_darray(size, rank, ndims, array_of_gsizes, array_of_distribs,
10         array_of_dargs, array_of_psizes, order, oldtype, newtype, ierror)
11     INTEGER, INTENT(IN) :: size, rank, ndims, array_of_gsizes(ndims),
12         array_of_distribs(ndims), array_of_dargs(ndims),
13         array_of_psizes(ndims), order
14     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
15     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
16     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18 MPI_Type_create_darray(size, rank, ndims, array_of_gsizes, array_of_distribs,
19         array_of_dargs, array_of_psizes, order, oldtype, newtype, ierror)
20     !(_c)
21     INTEGER, INTENT(IN) :: size, rank, ndims, array_of_distribs(ndims),
22         array_of_dargs(ndims), array_of_psizes(ndims), order
23     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: array_of_gsizes(ndims)
24     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
25     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
26     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

27 MPI_TYPE_CREATE_DARRAY(SIZE, RANK, NDIMS, ARRAY_OF_GSIZES, ARRAY_OF_DISTRIBS,
28         ARRAY_OF_DARGS, ARRAY_OF_PSIZEs, ORDER, OLDTYPE, NEWTYPE, IERROR)
29
30 INTEGER SIZE, RANK, NDIMS, ARRAY_OF_GSIZES(*), ARRAY_OF_DISTRIBS(*),
31         ARRAY_OF_DARGS(*), ARRAY_OF_PSIZEs(*), ORDER, OLDTYPE, NEWTYPE,
32         IERROR

```

`MPI_TYPE_CREATE_DARRAY` can be used to generate the datatypes corresponding to the distribution of an `ndims`-dimensional array of `oldtype` elements onto an `ndims`-dimensional grid of logical processes. Unused dimensions of `array_of_psizes` should be set to 1 (see Example 5.7). For a call to `MPI_TYPE_CREATE_DARRAY` to be correct, the equation $\prod_{i=0}^{ndims-1} array_of_psizes[i] = size$ must be satisfied. The ordering of processes in the process grid is assumed to be row-major, as in the case of virtual Cartesian process topologies.

Advice to users. For both Fortran and C arrays, the ordering of processes in the process grid is assumed to be row-major. This is consistent with the ordering used in virtual Cartesian process topologies in MPI. To create such virtual process topologies, or to find the coordinates of a process in the process grid, etc., users may use the corresponding process topology procedures, see Chapter 8. (*End of advice to users.*)

Each dimension of the array can be distributed in one of three ways:

<code>MPI_DISTRIBUTE_BLOCK</code>	Block distribution.
<code>MPI_DISTRIBUTE_CYCLIC</code>	Cyclic distribution.

`MPI_DISTRIBUTE_NONE` Dimension not distributed.

The constant `MPI_DISTRIBUTE_DFLT_DARG` specifies a default distribution argument. The distribution argument for a dimension that is not distributed is ignored. For any dimension i in which the distribution is `MPI_DISTRIBUTE_BLOCK`, it is erroneous to specify $\text{array_of_dargs}[i] * \text{array_of_psizes}[i] < \text{array_of_gsizes}[i]$.

For example, the HPF layout `ARRAY(CYCLIC(15))` corresponds to `MPI_DISTRIBUTE_CYCLIC` with a distribution argument of 15, and the HPF layout `ARRAY(BLOCK)` corresponds to `MPI_DISTRIBUTE_BLOCK` with a distribution argument of `MPI_DISTRIBUTE_DFLT_DARG`.

The order argument is used as in `MPI_TYPE_CREATE_SUBARRAY` to specify the storage order. Therefore, arrays described by this type constructor may be stored in Fortran (column-major) or C (row-major) order. Valid values for order are `MPI_ORDER_FORTRAN` and `MPI_ORDER_C`.

This routine creates a new MPI datatype with a typemap defined in terms of a function called “cyclic()” (see below).

Without loss of generality, it suffices to define the typemap for the `MPI_DISTRIBUTE_CYCLIC` case where `MPI_DISTRIBUTE_DFLT_DARG` is not used.

`MPI_DISTRIBUTE_BLOCK` and `MPI_DISTRIBUTE_NONE` can be reduced to the `MPI_DISTRIBUTE_CYCLIC` case for dimension i as follows.

`MPI_DISTRIBUTE_BLOCK` with $\text{array_of_dargs}[i]$ equal to `MPI_DISTRIBUTE_DFLT_DARG` is equivalent to `MPI_DISTRIBUTE_CYCLIC` with $\text{array_of_dargs}[i]$ set to

$$(\text{array_of_gsizes}[i] + \text{array_of_psizes}[i] - 1) / \text{array_of_psizes}[i].$$

If $\text{array_of_dargs}[i]$ is not `MPI_DISTRIBUTE_DFLT_DARG`, then `MPI_DISTRIBUTE_BLOCK` and `MPI_DISTRIBUTE_CYCLIC` are equivalent.

`MPI_DISTRIBUTE_NONE` is equivalent to `MPI_DISTRIBUTE_CYCLIC` with $\text{array_of_dargs}[i]$ set to $\text{array_of_gsizes}[i]$.

Finally, `MPI_DISTRIBUTE_CYCLIC` with $\text{array_of_dargs}[i]$ equal to `MPI_DISTRIBUTE_DFLT_DARG` is equivalent to `MPI_DISTRIBUTE_CYCLIC` with $\text{array_of_dargs}[i]$ set to 1.

For `MPI_ORDER_FORTRAN`, an ndims -dimensional distributed array (newtype) is defined by the following code fragment:

```
oldtypes[0] = oldtype;
for (i = 0; i < ndims; i++) {
    oldtypes[i+1] = cyclic(array_of_dargs[i],
                          array_of_gsizes[i],
                          r[i],
                          array_of_psizes[i],
                          oldtypes[i]);
}
newtype = oldtypes[ndims];
```

For `MPI_ORDER_C`, the code is:

```
oldtypes[0] = oldtype;
for (i = 0; i < ndims; i++) {
    oldtypes[i+1] = cyclic(array_of_dargs[ndims - i - 1],
                          array_of_gsizes[ndims - i - 1],
                          r[ndims - i - 1],
                          array_of_psizes[ndims - i - 1],
```

```

1         oldtypes[i]);
2     }
3     newtype = oldtypes[ndims];

```

where $r[i]$ is the position of the process (with rank $rank$) in the process grid at dimension i . The values of $r[i]$ are given by the following code fragment:

```

6     t_rank = rank;
7     t_size = 1;
8     for (i = 0; i < ndims; i++)
9         t_size *= array_of_psize[i];
10    for (i = 0; i < ndims; i++) {
11        t_size = t_size / array_of_psize[i];
12        r[i] = t_rank / t_size;
13        t_rank = t_rank % t_size;
14    }

```

Let the typemap of `oldtype` have the form:

$$\{(type_0, disp_0), (type_1, disp_1), \dots, (type_{n-1}, disp_{n-1})\}$$

where $type_i$ is a predefined MPI datatype, and let ex be the extent of `oldtype`. The following function uses the conceptual datatypes `lb_marker` and `ub_marker`, see Section 5.1.6 for details.

Given the above, the function `cyclic()` is defined as follows:

```

23    cyclic(darg, gsize, r, psize, oldtype)
24    =  {(lb_marker, 0),
25        (type_0, disp_0 + r × darg × ex), ...,
26        (type_{n-1}, disp_{n-1} + r × darg × ex),
27        (type_0, disp_0 + (r × darg + 1) × ex), ...,
28        (type_{n-1}, disp_{n-1} + (r × darg + 1) × ex),
29        ...
30        (type_0, disp_0 + ((r + 1) × darg - 1) × ex), ...,
31        (type_{n-1}, disp_{n-1} + ((r + 1) × darg - 1) × ex),
32        ...
33        (type_0, disp_0 + r × darg × ex + psize × darg × ex), ...,
34        (type_{n-1}, disp_{n-1} + r × darg × ex + psize × darg × ex),
35        (type_0, disp_0 + (r × darg + 1) × ex + psize × darg × ex), ...,
36        (type_{n-1}, disp_{n-1} + (r × darg + 1) × ex + psize × darg × ex),
37        ...
38        (type_0, disp_0 + ((r + 1) × darg - 1) × ex + psize × darg × ex), ...,
39        (type_{n-1}, disp_{n-1} + ((r + 1) × darg - 1) × ex + psize × darg × ex),
40        ...
41        (type_0, disp_0 + r × darg × ex + psize × darg × ex × (count - 1)), ...,
42        (type_{n-1}, disp_{n-1} + r × darg × ex + psize × darg × ex × (count - 1)),
43        (type_0, disp_0 + (r × darg + 1) × ex + psize × darg × ex × (count - 1)), ...,
44        (type_{n-1}, disp_{n-1} + (r × darg + 1) × ex + psize × darg × ex × (count - 1)), ...,
45        (type_0, disp_0 + ((r + 1) × darg - 1) × ex + psize × darg × ex × (count - 1)), ...,
46        (type_{n-1}, disp_{n-1} + ((r + 1) × darg - 1) × ex + psize × darg × ex × (count - 1))}.

```

$$\begin{aligned}
& (type_{n-1}, disp_{n-1} + (r \times darg + 1) \times ex \\
& \quad + psize \times darg \times ex \times (count - 1)), \\
& \dots \\
& (type_0, disp_0 + (r \times darg + darg_{last} - 1) \times ex \\
& \quad + psize \times darg \times ex \times (count - 1)), \dots, \\
& (type_{n-1}, disp_{n-1} + (r \times darg + darg_{last} - 1) \times ex \\
& \quad + psize \times darg \times ex \times (count - 1)), \\
& (ub_marker, gsize \times ex) \}
\end{aligned}$$

where *count* is defined by this code fragment:

```

nblocks = (gsize + (darg - 1)) / darg;
count = nblocks / psize;
left_over = nblocks - count * psize;
if (r < left_over)
    count = count + 1;

```

Here, *nblocks* is the number of blocks that must be distributed among the processors.

Finally, *darg_{last}* is defined by this code fragment:

```

if ((num_in_last_cyclic = gsize % (psize * darg)) == 0)
    darg_last = darg;
else {
    darg_last = num_in_last_cyclic - darg * r;
    if (darg_last > darg)
        darg_last = darg;
    if (darg_last <= 0)
        darg_last = darg;
}

```

Example 5.7. Consider generating the filetypes corresponding to the HPF distribution:

```

<oldtype> FILEARRAY(100, 200, 300)
!HPF$ PROCESSORS PROCESSES(2, 3)
!HPF$ DISTRIBUTE FILEARRAY(CYCLIC(10), *, BLOCK) ONTO PROCESSES

```

This can be achieved by the following Fortran code, assuming there will be six processes attached to the run:

```

ndims = 3
array_of_gsizes(1) = 100
array_of_distribs(1) = MPI_DISTRIBUTE_CYCLIC
array_of_dargs(1) = 10
array_of_gsizes(2) = 200
array_of_distribs(2) = MPI_DISTRIBUTE_NONE
array_of_dargs(2) = 0
array_of_gsizes(3) = 300
array_of_distribs(3) = MPI_DISTRIBUTE_BLOCK
array_of_dargs(3) = MPI_DISTRIBUTE_DFLT_DARG
array_of_psize(1) = 2
array_of_psize(2) = 1
array_of_psize(3) = 3
call MPI_COMM_SIZE(MPI_COMM_WORLD, size, ierr)
call MPI_COMM_RANK(MPI_COMM_WORLD, rank, ierr)

```

```

1  call MPI_TYPE_CREATE_DARRAY(size, rank, ndims, array_of_gsizes, &
2      array_of_distribs, array_of_dargs, array_of_psize, &
3      MPI_ORDER_FORTRAN, oldtype, newtype, ierr)
4

```

5.1.5 Address and Size Procedures

The displacements in a general datatype are relative to some initial buffer address. **Absolute addresses** can be substituted for these displacements: we treat them as displacements relative to “address zero,” the start of the address space. This initial address zero is indicated by the constant `MPI_BOTTOM`. Thus, a datatype can specify the absolute address of the entries in the communication buffer, in which case the `buf` argument is passed the value `MPI_BOTTOM`. Note that in Fortran `MPI_BOTTOM` is not usable for initialization or assignment, see Section 2.5.4.

The address of a location in memory can be found by invoking the procedure `MPI_GET_ADDRESS`. The **relative displacement** between two absolute addresses can be calculated with the procedure `MPI_AINT_DIFF`. A new absolute address as sum of an absolute base address and a relative displacement can be calculated with the procedure `MPI_AINT_ADD`. To ensure portability, arithmetic on absolute addresses should not be performed with the intrinsic operators “-” and “+”. See also Sections 2.5.6 and 5.1.12 on pages 23 and 161.

Rationale. Address sized integer values, i.e., `MPI_Aint` or `INTEGER(KIND=MPI_ADDRESS_KIND)` values, are signed integers, while absolute addresses are unsigned quantities. Direct arithmetic on addresses stored in address sized signed variables can cause overflows, resulting in undefined behavior. (*End of rationale.*)

`MPI_GET_ADDRESS(location, address)`

IN	location	location in caller memory (choice)
OUT	address	address of location (integer)

C binding

```
int MPI_Get_address(const void *location, MPI_Aint *address)
```

Fortran 2008 binding

```

MPI_Get_address(location, address, ierror)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: location
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: address
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GET_ADDRESS(LOCATION, ADDRESS, IERROR)
    <type> LOCATION(*)
    INTEGER(KIND=MPI_ADDRESS_KIND) ADDRESS
    INTEGER IERROR

```

Returns the (byte) address of location.

Rationale. In the `mpi_f08` module, the `location` argument is not defined with `INTENT(IN)` because existing applications may use `MPI_GET_ADDRESS` as a substitute for `MPI_F_SYNC_REG`, which was not defined before MPI-3.0. (*End of rationale.*)

Example 5.8. Using `MPI_GET_ADDRESS` for an array.

```
REAL A(100,100)
INTEGER(KIND=MPI_ADDRESS_KIND) I1, I2, DIFF
CALL MPI_GET_ADDRESS(A(1,1), I1, IERROR)
CALL MPI_GET_ADDRESS(A(10,10), I2, IERROR)
DIFF = MPI_AINT_DIFF(I2, I1)
! The value of DIFF is 909*SIZEOF-REAL); the values of I1 and I2 are
! implementation dependent.
```

Advice to users. C users may be tempted to avoid the usage of `MPI_GET_ADDRESS` and rely on the availability of the address operator `&`. Note, however, that `& cast-expression` is a pointer, not an address. ISO C does not require that the value of a pointer (or the pointer cast to `int`) be the absolute address of the object pointed at—although this is commonly the case. Furthermore, referencing may not have a unique definition on machines with a segmented address space. The use of `MPI_GET_ADDRESS` to “reference” C variables guarantees portability to such machines as well. (*End of advice to users.*)

Advice to users. To prevent problems with the argument copying and register optimization done by Fortran compilers, please note the hints in Sections 21.1.10–21.1.20. (*End of advice to users.*)

To ensure portability, arithmetic on MPI addresses must be performed using the `MPI_AINT_ADD` and `MPI_AINT_DIFF` procedures.

`MPI_AINT_ADD(base, disp)`

IN	base	base address (integer)
IN	disp	displacement (integer)

C binding

`MPI_Aint MPI_Aint_add(MPI_Aint base, MPI_Aint disp)`

Fortran 2008 binding

```
INTEGER(KIND=MPI_ADDRESS_KIND) MPI_Aint_add(base, disp)
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: base, disp
```

Fortran binding

```
INTEGER(KIND=MPI_ADDRESS_KIND) MPI_AINT_ADD(BASE, DISP)
  INTEGER(KIND=MPI_ADDRESS_KIND) BASE, DISP
```

`MPI_AINT_ADD` produces a new `MPI_Aint` value that is equivalent to the sum of the `base` and `disp` arguments, where `base` represents a base address returned by a call to `MPI_GET_ADDRESS` and `disp` represents a signed integer displacement. The resulting address is valid only at the process that generated `base`, and it must correspond to a location

in the same object referenced by `base`, as described in Section 5.1.12. The addition is performed in a manner that results in the correct `MPI_Aint` representation of the output address, as if the process that originally produced `base` had called:

```
MPI_Get_address((char *) base + disp, &result);
```

```
MPI_AINT_DIFF(addr1, addr2)
```

IN	addr1	minuend address (integer)
IN	addr2	subtrahend address (integer)

C binding

```
MPI_Aint MPI_Aint_diff(MPI_Aint addr1, MPI_Aint addr2)
```

Fortran 2008 binding

```
INTEGER(KIND=MPI_ADDRESS_KIND) MPI_Aint_diff(addr1, addr2)
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: addr1, addr2
```

Fortran binding

```
INTEGER(KIND=MPI_ADDRESS_KIND) MPI_AINT_DIFF(ADDR1, ADDR2)
    INTEGER(KIND=MPI_ADDRESS_KIND) ADDR1, ADDR2
```

MPI_AINT_DIFF produces a new `MPI_Aint` value that is equivalent to the difference between `addr1` and `addr2` arguments, where `addr1` and `addr2` represent addresses returned by calls to **MPI_GET_ADDRESS**. The resulting address is valid only at the process that generated `addr1` and `addr2`, and `addr1` and `addr2` must correspond to locations in the same object in the same process, as described in Section 5.1.12. The difference is calculated in a manner that results in the signed difference from `addr1` to `addr2`, as if the process that originally produced the addresses had called `(char *) addr1 - (char *) addr2` on the addresses initially passed to **MPI_GET_ADDRESS**.

The following auxiliary procedures provide useful information on derived datatypes.

```
MPI_TYPE_SIZE(datatype, size)
```

IN	datatype	datatype to get information on (handle)
OUT	size	datatype size (integer)

C binding

```
int MPI_Type_size(MPI_Datatype datatype, int *size)
int MPI_Type_size_c(MPI_Datatype datatype, MPI_Count *size)
```

Fortran 2008 binding

```
MPI_Type_size(datatype, size, ierror)
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, INTENT(OUT) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Type_size(datatype, size, ierror) !(_c)
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```

INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_SIZE(DATATYPE, SIZE, IERROR)
INTEGER DATATYPE, SIZE, IERROR

```

MPI_TYPE_SIZE set the value of *size* to the total size, in bytes, of the entries in the type signature associated with *datatype*; i.e., the total size of the data in a message that would be created with this datatype. Entries that occur multiple times in the datatype are counted with their multiplicity. For both procedures, if the *OUT* parameter cannot express the value to be returned (e.g., if the parameter is too small to hold the output value), it is set to **MPI_UNDEFINED**.

5.1.6 Lower-Bound and Upper-Bound Markers

It is often convenient to define explicitly the lower bound and upper bound of a type map, and override the definition given on page 152. This allows one to define a datatype that has “holes” at its beginning or its end, or a datatype with entries that extend above the upper bound or below the lower bound. Examples of such usage are provided in Section 5.1.14. Also, the user may want to override the alignment rules that are used to compute upper bounds and extents. E.g., a C compiler may allow the user to override default alignment rules for some of the structures within a program. The user has to specify explicitly the bounds of the datatypes that match these structures.

To achieve this, we add two additional conceptual datatypes, **lb_marker** and **ub_marker**, that represent the lower bound and upper bound of a datatype. These conceptual datatypes occupy no space ($\text{extent}(\text{lb_marker}) = \text{extent}(\text{ub_marker}) = 0$). They do not affect the size or count of a datatype, and do not affect the content of a message created with this datatype. However, they do affect the definition of the extent of a datatype and, therefore, affect the outcome of a replication of this datatype by a datatype constructor.

Example 5.9. A call to **MPI_TYPE_CREATE_RESIZED**(MPI_INT, -3, 9, type1) creates a new datatype that has an extent of 9 (from -3 to 5, 5 included), and contains an integer at displacement 0. This is the datatype defined by the typemap $\{(\text{lb_marker}, -3), (\text{int}, 0), (\text{ub_marker}, 6)\}$. If this type is replicated twice by a call to **MPI_TYPE_CONTIGUOUS**(2, type1, type2) then the newly created type can be described by the typemap $\{(\text{lb_marker}, -3), (\text{int}, 0), (\text{int}, 9), (\text{ub_marker}, 15)\}$. (An entry of type **ub_marker** can be deleted if there is another entry of type **ub_marker** with a higher displacement; an entry of type **lb_marker** can be deleted if there is another entry of type **lb_marker** with a lower displacement.)

In general, if

$$\text{Typemap} = \{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

then the **lower bound** of *Typemap* is defined to be

$$lb(\text{Typemap}) = \begin{cases} \min_j disp_j & \text{if no entry has type } lb_marker \\ \min_j \{disp_j \text{ such that } type_j = lb_marker\} & \text{otherwise} \end{cases}$$

Similarly, the **upper bound** of *Typemap* is defined to be

$$ub(\text{Typemap}) = \begin{cases} \max_j (disp_j + \text{sizeof}(type_j)) + \epsilon & \text{if no entry has type } ub_marker \\ \max_j \{disp_j \text{ such that } type_j = ub_marker\} & \text{otherwise} \end{cases}$$

Then

$$\text{extent}(\text{Typemap}) = \text{ub}(\text{Typemap}) - \text{lb}(\text{Typemap})$$

If type_i requires alignment to a byte address that is a multiple of k_i , then ϵ is the least nonnegative increment needed to round $\text{extent}(\text{Typemap})$ to the next multiple of $\max_i k_i$. In Fortran, it is implementation dependent whether the MPI implementation computes the alignments k_i according to the alignments used by the compiler in common blocks, SEQUENCE derived types, BIND(C) derived types, or derived types that are neither SEQUENCE nor BIND(C).

The formal definitions given for the various datatype constructors apply now, with the amended definition of **extent**.

Rationale. Before Fortran 2003, `MPI_TYPE_CREATE_STRUCT` could be applied to Fortran common blocks and SEQUENCE derived types. With Fortran 2003, this list was extended by BIND(C) derived types and MPI implementors have implemented the alignments k_i differently, i.e., some based on the alignments used in SEQUENCE derived types, and others according to BIND(C) derived types. (*End of rationale.*)

Advice to implementors. In Fortran, it is generally recommended to use BIND(C) derived types instead of common blocks or SEQUENCE derived types. Therefore it is recommended to calculate the alignments k_i based on BIND(C) derived types. (*End of advice to implementors.*)

Advice to users. Structures combining different basic datatypes should be defined so that there will be no gaps based on alignment rules. If such a datatype is used to create an array of structures, users should also avoid an alignment-gap at the end of the structure. In MPI communication, the content of such gaps would not be communicated into the receiver's buffer. For example, such an alignment-gap may occur between an odd number of floats or REALs before a double or DOUBLE PRECISION data. Such gaps may be added explicitly to both the structure and the MPI derived datatype handle because the communication of a contiguous derived datatype may be significantly faster than the communication of one that is noncontiguous because of such alignment-gaps.

As an example, instead of

```

TYPE, BIND(C) :: my_data
  REAL, DIMENSION(3) :: x
  ! there may be a gap of the size of one REAL
  ! if the alignment of a DOUBLE PRECISION is
  ! two times the size of a REAL
  DOUBLE PRECISION :: p
END TYPE

```

one should define

```

TYPE, BIND(C) :: my_data
  REAL, DIMENSION(3) :: x
  REAL :: gap1
  DOUBLE PRECISION :: p
END TYPE

```

and also include `gap1` in the matching MPI derived datatype. It is required that all processes in a communication add the same gaps, i.e., defined with the same basic datatype. Both the original and the modified structures are portable, but may have different performance implications for the communication and memory accesses during computation on systems with different alignment values.

In principle, a compiler may define an additional alignment rule for structures, e.g., to use at least 4 or 8 byte alignment, although the content may have a $\max_i k_i$ alignment less than this structure alignment. To maintain portability, users should always resize structure derived datatype handles if used in an array of structures, see the Example in Section 21.1.15. (*End of advice to users.*)

5.1.7 Extent and Bounds of Datatypes

MPI_TYPE_GET_EXTENT(datatype, lb, extent)

IN	datatype	datatype to get information on (handle)
OUT	lb	lower bound of datatype (integer)
OUT	extent	extent of datatype (integer)

C binding

```
int MPI_Type_get_extent(MPI_Datatype datatype, MPI_Aint *lb, MPI_Aint *extent)
```

```
int MPI_Type_get_extent_c(MPI_Datatype datatype, MPI_Count *lb,
    MPI_Count *extent)
```

Fortran 2008 binding

```
MPI_Type_get_extent(datatype, lb, extent, ierror)
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: lb, extent
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Type_get_extent(datatype, lb, extent, ierror) !(_c)
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: lb, extent
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_GET_EXTENT(DATATYPE, LB, EXTENT, IERROR)
```

```
INTEGER DATATYPE, IERROR
```

```
INTEGER(KIND=MPI_ADDRESS_KIND) LB, EXTENT
```

Returns the lower bound and the extent of `datatype` (as defined in Equation 5.1).

If either OUT parameter cannot express the value to be returned (e.g., if the parameter is too small to hold the output value), it is set to `MPI_UNDEFINED`.

MPI allows one to change the extent of a datatype, using lower bound and upper bound markers. This provides control over the stride of successive datatypes that are replicated by datatype constructors, or are replicated by the `count` argument in a send or receive call.

```
1 MPI_TYPE_CREATE_RESIZED(oldtype, lb, extent, newtype)
```

```
2     IN      oldtype          input datatype (handle)
```

```
3     IN      lb              new lower bound of datatype (integer)
```

```
4     IN      extent         new extent of datatype (integer)
```

```
5     OUT     newtype         output datatype (handle)
```

8 C binding

```
9 int MPI_Type_create_resized(MPI_Datatype oldtype, MPI_Aint lb, MPI_Aint extent,  
10 MPI_Datatype *newtype)
```

```
11  
12 int MPI_Type_create_resized_c(MPI_Datatype oldtype, MPI_Count lb,  
13 MPI_Count extent, MPI_Datatype *newtype)
```

14 Fortran 2008 binding

```
15 MPI_Type_create_resized(oldtype, lb, extent, newtype, ierror)  
16 TYPE(MPI_Datatype), INTENT(IN) :: oldtype  
17 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: lb, extent  
18 TYPE(MPI_Datatype), INTENT(OUT) :: newtype  
19 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
20  
21 MPI_Type_create_resized(oldtype, lb, extent, newtype, ierror) !(_c)  
22 TYPE(MPI_Datatype), INTENT(IN) :: oldtype  
23 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: lb, extent  
24 TYPE(MPI_Datatype), INTENT(OUT) :: newtype  
25 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

26 Fortran binding

```
27 MPI_TYPE_CREATE_RESIZED(OLDTYPE, LB, EXTENT, NEWTYPE, IERROR)  
28 INTEGER OLDTYPE, NEWTYPE, IERROR  
29 INTEGER(KIND=MPI_ADDRESS_KIND) LB, EXTENT
```

30
31 Returns in `newtype` a handle to a new datatype that is identical to `oldtype`, except that
32 the lower bound of this new datatype is set to be `lb`, and its upper bound is set to be `lb`
33 + `extent`. Any previous `lb` and `ub` markers are erased, and a new pair of lower bound and
34 upper bound markers are put in the positions indicated by the `lb` and `extent` arguments.
35 This affects the behavior of the datatype when used in communication operations, with
36 `count > 1`, and when used in the construction of new derived datatypes.

38 5.1.8 True Extent of Datatypes

39 Suppose we implement gather (see also Section 6.5) as a spanning tree implemented on
40 top of point-to-point routines. Since the receive buffer is only valid on the root pro-
41 cess, one will need to allocate some temporary space for receiving data on intermediate
42 nodes. However, the datatype extent cannot be used as an estimate of the amount of
43 space that needs to be allocated, if the user has modified the extent, for example by
44 using `MPI_TYPE_CREATE_RESIZED`. The procedure `MPI_TYPE_GET_TRUE_EXTENT`
45 returns the true extent of the datatype.

`MPI_TYPE_GET_TRUE_EXTENT(datatype, true_lb, true_extent)`

IN	<code>datatype</code>	datatype to get information on (handle)
OUT	<code>true_lb</code>	true lower bound of datatype (integer)
OUT	<code>true_extent</code>	true extent of datatype (integer)

C binding

```
int MPI_Type_get_true_extent(MPI_Datatype datatype, MPI_Aint *true_lb,
                             MPI_Aint *true_extent)

int MPI_Type_get_true_extent_c(MPI_Datatype datatype, MPI_Count *true_lb,
                               MPI_Count *true_extent)
```

Fortran 2008 binding

```
MPI_Type_get_true_extent(datatype, true_lb, true_extent, ierror)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: true_lb, true_extent
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_true_extent(datatype, true_lb, true_extent, ierror) !(_c)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: true_lb, true_extent
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_GET_TRUE_EXTENT(DATATYPE, TRUE_LB, TRUE_EXTENT, IERROR)
  INTEGER DATATYPE, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) TRUE_LB, TRUE_EXTENT
```

`true_lb` returns the offset of the lowest unit of storage that is addressed by the datatype, i.e., the lower bound of the corresponding typemap, ignoring explicit lower bound markers. `true_extent` returns the true size of the datatype, i.e., the extent of the corresponding typemap, ignoring explicit lower bound and upper bound markers, and performing no rounding for alignment. If the typemap associated with `datatype` is

$$Typemap = \{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\}$$

Then

$$true_lb(Typemap) = \min_j \{disp_j : type_j \neq lb_marker, ub_marker\},$$

$$true_ub(Typemap) = \max_j \{disp_j + \text{sizeof}(type_j) : type_j \neq lb_marker, ub_marker\},$$

and

$$true_extent(Typemap) = true_ub(Typemap) - true_lb(Typemap).$$

(Readers should compare this with the definitions in Section 5.1.6 and Section 5.1.7, which describe the procedure `MPI_TYPE_GET_EXTENT`.)

The `true_extent` is the minimum number of bytes of memory necessary to hold a datatype, uncompressed.

If either OUT parameter cannot express the value to be returned (e.g., if the parameter is too small to hold the output value), it is set to `MPI_UNDEFINED`.

5.1.9 Commit and Free

A datatype object has to be **committed** before it can be used in a communication. As an argument in datatype constructors, uncommitted and also committed datatypes can be used. There is no need to commit basic datatypes. They are “pre-committed.”

`MPI_TYPE_COMMIT(datatype)`

INOUT datatype datatype that is committed (handle)

C binding

```
int MPI_Type_commit(MPI_Datatype *datatype)
```

Fortran 2008 binding

```
MPI_Type_commit(datatype, ierror)
    TYPE(MPI_Datatype), INTENT(INOUT) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_COMMIT(DATATYPE, IERROR)
    INTEGER DATATYPE, IERROR
```

The commit operation commits the datatype, that is, the formal description of a communication buffer, not the content of that buffer. Thus, after a datatype has been committed, it can be repeatedly reused to communicate the changing content of a buffer or, indeed, the content of different buffers, with different starting addresses.

Advice to implementors. The system may “compile” at commit time an internal representation for the datatype that facilitates communication, e.g., change from a compacted representation to a flat representation of the datatype, and select the most convenient transfer mechanism.

The optimizations chosen during `MPI_TYPE_COMMIT` may no longer be optimal if a session (or the World Model) is initialized or finalized. (*End of advice to implementors.*)

`MPI_TYPE_COMMIT` will accept a committed datatype; in this case, it is equivalent to a no-op.

Example 5.10. The following code fragment gives examples of using `MPI_TYPE_COMMIT`.

```
INTEGER type1, type2
CALL MPI_TYPE_CONTIGUOUS(5, MPI_REAL, type1, ierr)
    ! new type object created
CALL MPI_TYPE_COMMIT(type1, ierr)
    ! now type1 can be used for communication
type2 = type1
    ! type2 can be used for communication
    ! (it is a handle to same object as type1)
CALL MPI_TYPE_VECTOR(3, 5, 4, MPI_REAL, type1, ierr)
    ! new uncommitted type object created
CALL MPI_TYPE_COMMIT(type1, ierr)
    ! now type1 can be used anew for communication
```


MPI_TYPE_FREE(datatype)

INOUT datatype datatype that is freed (handle)

C binding

```
int MPI_Type_free(MPI_Datatype *datatype)
```

Fortran 2008 binding

```
MPI_Type_free(datatype, ierror)
      TYPE(MPI_Datatype), INTENT(INOUT) :: datatype
      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_FREE(DATATYPE, IERROR)
      INTEGER DATATYPE, IERROR
```

Marks the datatype object associated with **datatype** for deallocation and sets **datatype** to **MPI_DATATYPE_NULL**. Any communication that is currently using this datatype will complete normally. Freeing a datatype does not affect any other datatype that was built from the freed datatype. The system behaves as if input datatype arguments to derived datatype constructors are passed by value.

Advice to implementors. The implementation may keep a reference count of active communications that use the datatype, in order to decide when to free it. Also, one may implement constructors of derived datatypes so that they keep pointers to their datatype arguments, rather than copying them. In this case, one needs to keep track of active datatype definition references in order to know when a datatype object can be freed. (*End of advice to implementors.*)

5.1.10 Duplicating a Datatype

MPI_TYPE_DUP(oldtype, newtype)

IN oldtype datatype (handle)
OUT newtype copy of oldtype (handle)

C binding

```
int MPI_Type_dup(MPI_Datatype oldtype, MPI_Datatype *newtype)
```

Fortran 2008 binding

```
MPI_Type_dup(oldtype, newtype, ierror)
      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_DUP(OLDTYPE, NEWTYPE, IERROR)
      INTEGER OLDTYPE, NEWTYPE, IERROR
```

MPI_TYPE_DUP is a type constructor that duplicates the existing **oldtype** with associated key values. For each key value, the respective copy callback function determines the

attribute value associated with this key in the new datatype; one particular action that a copy callback may take is to delete the attribute from the new datatype. Returns in `newtype` a new datatype with exactly the same properties as `oldtype` and any copied cached information, see Section 7.7.4. The new datatype has identical upper bound and lower bound and yields the same net result when fully decoded with the procedures in Section 5.1.13. The `newtype` has the same committed state as the old `oldtype`.

5.1.11 Use of General Datatypes in Communication

Handles to derived datatypes can be passed to a communication call wherever a datatype argument is required. A call of the form `MPI_SEND(buf, count, datatype, ...)`, where `count > 1`, is interpreted as if the call was passed a new datatype that is the concatenation of `count` copies of `datatype`. Thus, `MPI_SEND(buf, count, datatype, dest, tag, comm)` is equivalent to,

```
MPI_TYPE_CONTIGUOUS(count, datatype, newtype)
MPI_TYPE_COMMIT(newtype)
MPI_SEND(buf, 1, newtype, dest, tag, comm)
MPI_TYPE_FREE(newtype).
```

Similar statements apply to all other communication procedures that have a `count` and `datatype` argument.

Suppose that a send operation `MPI_SEND(buf, count, datatype, dest, tag, comm)` is executed, where `datatype` has type map,

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

and extent *extent*. (Explicit lower bound and upper bound markers are not listed in the type map, but they affect the value of *extent*.) The send operation sends $n \cdot \text{count}$ entries, where entry $i \cdot n + j$ is at location $addr_{i,j} = \text{buf} + \text{extent} \cdot i + disp_j$ and has type $type_j$, for $i = 0, \dots, \text{count} - 1$ and $j = 0, \dots, n - 1$. These entries need not be contiguous, nor distinct; their order can be arbitrary.

The variable stored at address $addr_{i,j}$ in the calling program should be of a type that matches $type_j$, where type matching is defined as in Section 3.3.1. The message sent contains $n \cdot \text{count}$ entries, where entry $i \cdot n + j$ has type $type_j$.

Similarly, suppose that a receive operation `MPI_RECV(buf, count, datatype, source, tag, comm, status)` is executed, where `datatype` has type map,

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\},$$

with extent *extent*. (Again, explicit lower bound and upper bound markers are not listed in the type map, but they affect the value of *extent*.) This receive operation receives $n \cdot \text{count}$ entries, where entry $i \cdot n + j$ is at location $\text{buf} + \text{extent} \cdot i + disp_j$ and has type $type_j$. If the incoming message consists of k elements, then we must have $k \leq n \cdot \text{count}$; the $i \cdot n + j$ -th element of the message should have a type that matches $type_j$.

Type matching is defined according to the type signature of the corresponding datatypes, that is, the sequence of basic type components. Type matching does not depend on some aspects of the datatype definition, such as the displacements (layout in memory) or the intermediate types used.

Example 5.11. This example shows that type matching is defined in terms of the basic types that a derived type consists of.

```
...
CALL MPI_TYPE_CONTIGUOUS(2, MPI_REAL, type2, ...)
CALL MPI_TYPE_CONTIGUOUS(4, MPI_REAL, type4, ...)
CALL MPI_TYPE_CONTIGUOUS(2, type2, type22, ...)
...
CALL MPI_SEND(a, 4, MPI_REAL, ...)
CALL MPI_SEND(a, 2, type2, ...)
CALL MPI_SEND(a, 1, type22, ...)
CALL MPI_SEND(a, 1, type4, ...)
...
CALL MPI_RECV(a, 4, MPI_REAL, ...)
CALL MPI_RECV(a, 2, type2, ...)
CALL MPI_RECV(a, 1, type22, ...)
CALL MPI_RECV(a, 1, type4, ...)
```

Each of the sends matches any of the receives.

A datatype may specify overlapping entries. The use of such a datatype in any communication in association with a buffer updated by the operation is erroneous. (This is erroneous even if the actual message received is short enough not to write any entry more than once.)

Suppose that `MPI_RECV(buf, count, datatype, dest, tag, comm, status)` is executed, where `datatype` has type map,

$$\{(type_0, disp_0), \dots, (type_{n-1}, disp_{n-1})\}.$$

The received message need not fill all the receive buffer, nor does it need to fill a number of locations that is a multiple of n . Any number, k , of basic elements can be received, where $0 \leq k \leq \text{count} \cdot n$. The number of basic elements received can be retrieved from `status` using the query procedure `MPI_GET_ELEMENTS`.

`MPI_GET_ELEMENTS(status, datatype, count)`

IN	status	return status of receive operation (status)
IN	datatype	datatype used by receive operation (handle)
OUT	count	number of received basic elements (integer)

C binding

```
int MPI_Get_elements(const MPI_Status *status, MPI_Datatype datatype,
                    int *count)
```

```
int MPI_Get_elements_c(const MPI_Status *status, MPI_Datatype datatype,
                      MPI_Count *count)
```

Fortran 2008 binding

```
MPI_Get_elements(status, datatype, count, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```

1      INTEGER, INTENT(OUT) :: count
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4      MPI_Get_elements(status, datatype, count, ierror) !(_c)
5      TYPE(MPI_Status), INTENT(IN) :: status
6      TYPE(MPI_Datatype), INTENT(IN) :: datatype
7      INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: count
8      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

10     MPI_GET_ELEMENTS(STATUS, DATATYPE, COUNT, IERROR)
11     INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, COUNT, IERROR

```

The `datatype` argument should match the argument provided by the receive call that set the `status` variable. For both procedures, if the `OUT` parameter cannot express the value to be returned (e.g., if the parameter is too small to hold the output value), it is set to `MPI_UNDEFINED`.

The previously defined procedure `MPI_GET_COUNT` (Section 3.2.5), has a different behavior. It returns the number of “top-level entries” received, i.e., the number of “copies” of type `datatype`. In the previous example, `MPI_GET_COUNT` may return any integer value k , where $0 \leq k \leq \text{count}$. If `MPI_GET_COUNT` returns k , then the number of basic elements received (and the value returned by `MPI_GET_ELEMENTS`) is $n \cdot k$. If the number of basic elements received is not a multiple of n , that is, if the receive operation has not received an integral number of `datatype` “copies,” then `MPI_GET_COUNT` sets the value of `count` to `MPI_UNDEFINED`.

Example 5.12. Usage of `MPI_GET_COUNT` and `MPI_GET_ELEMENTS`.

```

26     ...
27     CALL MPI_TYPE_CONTIGUOUS(2, MPI_REAL, Type2, ierr)
28     CALL MPI_TYPE_COMMIT(Type2, ierr)
29     ...
30     CALL MPI_COMM_RANK(comm, rank, ierr)
31     IF (rank .EQ. 0) THEN
32         CALL MPI_SEND(a, 2, MPI_REAL, 1, 0, comm, ierr)
33         CALL MPI_SEND(a, 3, MPI_REAL, 1, 0, comm, ierr)
34     ELSE IF (rank .EQ. 1) THEN
35         CALL MPI_RECV(a, 2, Type2, 0, 0, comm, stat, ierr)
36         CALL MPI_GET_COUNT(stat, Type2, i, ierr)      ! returns i=1
37         CALL MPI_GET_ELEMENTS(stat, Type2, i, ierr)   ! returns i=2
38         CALL MPI_RECV(a, 2, Type2, 0, 0, comm, stat, ierr)
39         CALL MPI_GET_COUNT(stat, Type2, i, ierr)      ! returns i=MPI_UNDEFINED
40         CALL MPI_GET_ELEMENTS(stat, Type2, i, ierr)   ! returns i=3
41     END IF

```

The procedure `MPI_GET_ELEMENTS` can also be used after a probe to find the number of elements in the probed message. Note that the `MPI_GET_COUNT` and `MPI_GET_ELEMENTS` return the same values when they are used with basic datatypes as long as the limits of their respective `count` arguments are not exceeded.

Rationale. The extension given to the definition of `MPI_GET_COUNT` seems natural: one would expect this procedure to return the value of the `count` argument, when the receive buffer is filled. Sometimes `datatype` represents a basic unit of data one wants to transfer, for example, a record in an array of records (structures). One

should be able to find out how many components were received without bothering to divide by the number of elements in each component. However, on other occasions, **datatype** is used to define a complex layout of data in the receiver memory, and does not represent a basic unit of data for transfers. In such cases, one needs to use the procedure **MPI_GET_ELEMENTS**. (*End of rationale.*)

Advice to implementors. The definition implies that a receive cannot change the value of storage outside the entries defined to compose the communication buffer. In particular, the definition implies that padding space in a structure should not be modified when such a structure is copied from one process to another. This would prevent the obvious optimization of copying the structure, together with the padding, as one contiguous block. The implementation is free to do this optimization when it does not impact the outcome of the computation. The user can “force” this optimization by explicitly including padding as part of the message. (*End of advice to implementors.*)

5.1.12 Correct Use of Addresses

Successively declared variables in C or Fortran are not necessarily stored at contiguous locations. Thus, care must be exercised that displacements do not cross from one variable to another. Also, in machines with a segmented address space, addresses are not unique and address arithmetic has some peculiar properties. Thus, the use of **addresses**, that is, displacements relative to the start address **MPI_BOTTOM**, has to be restricted.

Variables belong to the same **sequential storage** if they belong to the same array, to the same **COMMON** block in Fortran, or to the same structure in C. Valid addresses are defined recursively as follows:

1. The procedure **MPI_GET_ADDRESS** returns a valid address, when passed as argument a variable of the calling program.
2. The **buf** argument of a communication procedure evaluates to a valid address, when passed as argument a variable of the calling program.
3. If **v** is a valid address, and **i** is an integer, then **v+i** is a valid address, provided **v** and **v+i** are in the same sequential storage.

A correct program uses only valid addresses to identify the locations of entries in communication buffers. Furthermore, if **u** and **v** are two valid addresses, then the (integer) difference **u – v** can be computed only if both **u** and **v** are in the same sequential storage. No other arithmetic operations can be meaningfully executed on addresses.

The rules above impose no constraints on the use of derived datatypes, as long as they are used to define a communication buffer that is wholly contained within the same sequential storage. However, the construction of a communication buffer that contains variables that are not within the same sequential storage must obey certain restrictions. Basically, a communication buffer with variables that are not within the same sequential storage can be used only by specifying in the communication call **buf = MPI_BOTTOM**, **count = 1**, and using a **datatype** argument where all displacements are valid (absolute) addresses.

Advice to users. It is not expected that MPI implementations will be able to detect erroneous, “out of bound” displacements—unless those overflow the user address space—since the MPI call may not know the extent of the arrays and records in the host program. (*End of advice to users.*)

Advice to implementors. There is no need to distinguish (absolute) addresses and (relative) displacements on a machine with contiguous address space: `MPI_BOTTOM` is zero, and both addresses and displacements are integers. On machines where the distinction is required, addresses are recognized as expressions that involve `MPI_BOTTOM`. (*End of advice to implementors.*)

5.1.13 Decoding a Datatype

MPI datatype objects allow users to specify an arbitrary layout of data in memory. There are several cases where accessing the layout information in opaque datatype objects would be useful. The opaque datatype object has found a number of uses outside MPI. Furthermore, a number of tools wish to display internal information about a datatype. To achieve this, datatype decoding procedures are provided. The two procedures in this section are used together to decode datatypes to recreate the calling sequence used in their initial definition. These can be used to allow a user to determine the type map and type signature of a datatype.

`MPI_TYPE_GET_ENVELOPE(datatype, num_integers, num_addresses, num_large_counts, num_datatypes, combiner)`

IN	datatype	datatype to decode (handle)
OUT	num_integers	number of input integers used in call constructing combiner (nonnegative integer)
OUT	num_addresses	number of input addresses used in call constructing combiner (nonnegative integer)
OUT	num_large_counts	number of input large counts used in call constructing combiner (nonnegative integer, only present for large count variants)
OUT	num_datatypes	number of input datatypes used in call constructing combiner (nonnegative integer)
OUT	combiner	combiner (state)

C binding

```
int MPI_Type_get_envelope(MPI_Datatype datatype, int *num_integers,
                          int *num_addresses, int *num_datatypes, int *combiner)
```

```
int MPI_Type_get_envelope_c(MPI_Datatype datatype, MPI_Count *num_integers,
                            MPI_Count *num_addresses, MPI_Count *num_large_counts,
                            MPI_Count *num_datatypes, int *combiner)
```

Fortran 2008 binding

```
MPI_Type_get_envelope(datatype, num_integers, num_addresses, num_datatypes,
                      combiner, ierror)
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
INTEGER, INTENT(OUT) :: num_integers, num_addresses, num_datatypes,
                      combiner
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```

MPI_Type_get_envelope(datatype, num_integers, num_addresses, num_large_counts,
    num_datatypes, combiner, ierror) !(_c)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: num_integers, num_addresses,
    num_large_counts, num_datatypes
INTEGER, INTENT(OUT) :: combiner
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_GET_ENVELOPE(DATATYPE, NUM_INTEGERS, NUM_ADDRESSES, NUM_DATATYPES,
    COMBINER, IERROR)
INTEGER DATATYPE, NUM_INTEGERS, NUM_ADDRESSES, NUM_DATATYPES, COMBINER,
    IERROR

```

For the given `datatype`, `MPI_TYPE_GET_ENVELOPE` returns information on the number and type of input arguments used in the call that created the `datatype`. The number-of-arguments values returned can be used to provide sufficiently large arrays in the decoding routine `MPI_TYPE_GET_CONTENTS`. This call and the meaning of the returned values is described below. The `combiner` reflects the MPI datatype constructor call that was used in creating `datatype`.

Rationale. By requiring that the `combiner` reflect the constructor used in the creation of the `datatype`, the decoded information can be used to effectively recreate the calling sequence used in the original creation. This is the most useful information and was felt to be reasonable even though it constrains implementations to remember the original constructor sequence even if the internal representation is different.

The decoded information keeps track of datatype duplications. This is important as one needs to distinguish between a predefined datatype and a dup of a predefined datatype. The former is a constant object that cannot be freed, while the latter is a derived datatype that can be freed. (*End of rationale.*)

The list of values that can be returned from `MPI_TYPE_GET_ENVELOPE` in `combiner` (on the left) and the call associated with them (on the right) are as follows:

<code>MPI_COMBINER_NAMED</code>	a named predefined datatype
<code>MPI_COMBINER_DUP</code>	<code>MPI_TYPE_DUP</code>
<code>MPI_COMBINER_CONTIGUOUS</code>	<code>MPI_TYPE_CONTIGUOUS</code>
<code>MPI_COMBINER_VECTOR</code>	<code>MPI_TYPE_VECTOR</code>
<code>MPI_COMBINER_HVECTOR</code>	<code>MPI_TYPE_CREATE_HVECTOR</code>
<code>MPI_COMBINER_INDEXED</code>	<code>MPI_TYPE_INDEXED</code>
<code>MPI_COMBINER_HINDEXED</code>	<code>MPI_TYPE_CREATE_HINDEXED</code>
<code>MPI_COMBINER_INDEXED_BLOCK</code>	<code>MPI_TYPE_CREATE_INDEXED_BLOCK</code>
<code>MPI_COMBINER_HINDEXED_BLOCK</code>	<code>MPI_TYPE_CREATE_HINDEXED_BLOCK</code>
<code>MPI_COMBINER_STRUCT</code>	<code>MPI_TYPE_CREATE_STRUCT</code>
<code>MPI_COMBINER_SUBARRAY</code>	<code>MPI_TYPE_CREATE_SUBARRAY</code>
<code>MPI_COMBINER_DARRAY</code>	<code>MPI_TYPE_CREATE_DARRAY</code>
<code>MPI_COMBINER_F90_REAL</code>	<code>MPI_TYPE_CREATE_F90_REAL</code>
<code>MPI_COMBINER_F90_COMPLEX</code>	<code>MPI_TYPE_CREATE_F90_COMPLEX</code>
<code>MPI_COMBINER_F90_INTEGER</code>	<code>MPI_TYPE_CREATE_F90_INTEGER</code>
<code>MPI_COMBINER_RESIZED</code>	<code>MPI_TYPE_CREATE_RESIZED</code>
<code>MPI_COMBINER_VALUE_INDEX</code>	<code>MPI_TYPE_GET_VALUE_INDEX</code>

If combiner is `MPI_COMBINER_NAMED` then `datatype` is a named predefined datatype. If the `MPI_TYPE_GET_ENVELOPE` variant without `num_large_counts` is invoked with a `datatype` that requires an output value of `num_large_counts > 0`, then an error of class `MPI_ERR_TYPE` is raised.

Rationale. The large count variant of this MPI procedure was added in MPI-4. It contains a new `num_large_counts` parameter. The other variant—the variant that existed before MPI-4—was not changed in order to preserve backwards compatibility. (*End of rationale.*)

The actual arguments used in the creation call for a `datatype` can be obtained using `MPI_TYPE_GET_CONTENTS`.

`MPI_TYPE_GET_ENVELOPE` and `MPI_TYPE_GET_CONTENTS` also support large count types in separate additional MPI procedures in C (suffixed with the “_c”) and interface polymorphism in Fortran when using `USE mpi_f08`.

```
MPI_TYPE_GET_CONTENTS(datatype, max_integers, max_addresses, max_large_counts,
                      max_datatypes, array_of_integers, array_of_addresses,
                      array_of_large_counts, array_of_datatypes)
```

IN	<code>datatype</code>	datatype to decode (handle)
IN	<code>max_integers</code>	number of elements in <code>array_of_integers</code> (nonnegative integer)
IN	<code>max_addresses</code>	number of elements in <code>array_of_addresses</code> (nonnegative integer)
IN	<code>max_large_counts</code>	number of elements in <code>array_of_large_counts</code> (nonnegative integer, only present for large count variants)
IN	<code>max_datatypes</code>	number of elements in <code>array_of_datatypes</code> (nonnegative integer)
OUT	<code>array_of_integers</code>	contains integer arguments used in constructing datatype (array of integers)
OUT	<code>array_of_addresses</code>	contains address arguments used in constructing datatype (array of integers)
OUT	<code>array_of_large_counts</code>	contains large count arguments used in constructing datatype (array of integers, only present for large count variants)
OUT	<code>array_of_datatypes</code>	contains datatype arguments used in constructing datatype (array of handles)

C binding

```
int MPI_Type_get_contents(MPI_Datatype datatype, int max_integers,
                        int max_addresses, int max_datatypes, int array_of_integers[],
                        MPI_Aint array_of_addresses[], MPI_Datatype array_of_datatypes[])

int MPI_Type_get_contents_c(MPI_Datatype datatype, MPI_Count max_integers,
                          MPI_Count max_addresses, MPI_Count max_large_counts,
                          MPI_Count max_datatypes, int array_of_integers[],
```



```

        MPI_Aint array_of_addresses[], MPI_Count array_of_large_counts[],
        MPI_Datatype array_of_datatypes[])

```

Fortran 2008 binding

```

MPI_Type_get_contents(datatype, max_integers, max_addresses, max_datatypes,
    array_of_integers, array_of_addresses, array_of_datatypes,
    ierror)

TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: max_integers, max_addresses, max_datatypes
INTEGER, INTENT(OUT) :: array_of_integers(max_integers)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) ::
    array_of_addresses(max_addresses)
TYPE(MPI_Datatype), INTENT(OUT) :: array_of_datatypes(max_datatypes)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_contents(datatype, max_integers, max_addresses, max_large_counts,
    max_datatypes, array_of_integers, array_of_addresses,
    array_of_large_counts, array_of_datatypes, ierror) !(_c)

TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: max_integers, max_addresses,
    max_large_counts, max_datatypes
INTEGER, INTENT(OUT) :: array_of_integers(max_integers)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) ::
    array_of_addresses(max_addresses)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) ::
    array_of_large_counts(max_large_counts)
TYPE(MPI_Datatype), INTENT(OUT) :: array_of_datatypes(max_datatypes)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_GET_CONTENTS(DATATYPE, MAX_INTEGERS, MAX_ADDRESSES, MAX_DATATYPES,
    ARRAY_OF_INTEGERS, ARRAY_OF_ADDRESSES, ARRAY_OF_DATATYPES,
    IERROR)

INTEGER DATATYPE, MAX_INTEGERS, MAX_ADDRESSES, MAX_DATATYPES,
    ARRAY_OF_INTEGERS(*), ARRAY_OF_DATATYPES(*), IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_ADDRESSES(*)

```

`datatype` must be a predefined unnamed or a derived datatype; the call is erroneous if `datatype` is a predefined named datatype.

The values given for `max_integers`, `max_addresses`, `max_large_counts`, and `max_datatypes` must be at least as large as the value returned in `num_integers`, `num_addresses`, `num_large_counts`, and `num_datatypes`, respectively, in the call `MPI_TYPE_GET_ENVELOPE` for the same `datatype` argument.

Rationale. The arguments `max_integers`, `max_addresses`, `max_large_counts`, and `max_datatypes` allow for error checking in the call. (*End of rationale.*)

If the `MPI_TYPE_GET_CONTENTS` variant without `max_large_counts` is invoked with a `datatype` that requires > 0 values in `array_of_large_counts`, then an error of class `MPI_ERR_TYPE` is raised.

Rationale. The large count variant of this MPI procedure was added in MPI-4. It contains new `max_large_counts` and `array_of_large_counts` parameters. The other variant—the variant that existed before MPI-4—was not changed in order to preserve backwards compatibility. (*End of rationale.*)

The datatypes returned in `array_of_datatypes` are handles to datatype objects that are equivalent to the datatypes used in the original construction call. If these were derived datatypes, then the returned datatypes are new datatype objects, and the user is responsible for freeing these datatypes with `MPI_TYPE_FREE`. If these were predefined datatypes, then the returned datatype is equal to that (constant) predefined datatype and cannot be freed.

The committed state of returned derived datatypes is undefined, i.e., the datatypes may or may not be committed. Furthermore, the content of attributes of returned datatypes is undefined.

Note that `MPI_TYPE_GET_CONTENTS` can be invoked with a `datatype` argument that was constructed using `MPI_TYPE_CREATE_F90_REAL`, `MPI_TYPE_CREATE_F90_INTEGER`, or `MPI_TYPE_CREATE_F90_COMPLEX` (an unnamed predefined datatype). In such a case, an empty `array_of_datatypes` is returned.

Rationale. The definition of datatype equivalence implies that equivalent predefined datatypes are equal. By requiring the same handle for named predefined datatypes, it is possible to use the `==` or `.EQ.` comparison operator to determine the datatype involved. (*End of rationale.*)

Advice to implementors. The datatypes returned in `array_of_datatypes` must appear to the user as if each is an equivalent copy of the datatype used in the type constructor call. Whether this is done by creating a new datatype or via another mechanism such as a reference count mechanism is up to the implementation as long as the semantics are preserved. (*End of advice to implementors.*)

Rationale. The committed state and attributes of the returned datatype is deliberately left vague. The datatype used in the original construction may have been modified since its use in the constructor call. Attributes can be added, removed, or modified as well as having the datatype committed. The semantics given allow for a reference count implementation without having to track these changes. (*End of rationale.*)

In the deprecated datatype constructor calls, the address arguments in Fortran are of type `INTEGER`. In the preferred calls, the address arguments are of type `INTEGER(KIND=MPI_ADDRESS_KIND)`. The call `MPI_TYPE_GET_CONTENTS` returns all addresses in an argument of type `INTEGER(KIND=MPI_ADDRESS_KIND)`. This is true even if the deprecated calls were used. Thus, the location of values returned can be thought of as being returned by the C bindings. It can also be determined by examining the preferred calls for datatype constructors for the deprecated calls that involve addresses.

Rationale. By having all address arguments returned in the `array_of_addresses` argument, the result from a C and Fortran decoding of a `datatype` gives the result in the same argument. It is assumed that an integer of type `INTEGER(KIND=MPI_ADDRESS_KIND)` will be at least as large as the `INTEGER` argument used in datatype construction with the old MPI-1 calls so no loss of information will occur. (*End of rationale.*)

The following defines what values are placed in each entry of the returned arrays depending on the datatype constructor used for `datatype`. It also specifies the size of the arrays needed, which is the values returned by `MPI_TYPE_GET_ENVELOPE`. In Fortran, the following calls were made:

```

PARAMETER (LARGE = 1000)
INTEGER DTYPE, NI, NA, ND, COMBINER, I(LARGE), D(LARGE), IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) A(LARGE)
! CONSTRUCT DATATYPE DTYPE (NOT SHOWN)
CALL MPI_TYPE_GET_ENVELOPE(DTYPE, NI, NA, ND, COMBINER, IERROR)
IF ((NI .GT. LARGE) .OR. (NA .GT. LARGE) .OR. (ND .GT. LARGE)) THEN
  WRITE (*, *) "NI, NA, OR ND = ", NI, NA, ND, &
    " RETURNED BY MPI_TYPE_GET_ENVELOPE IS LARGER THAN LARGE = ", LARGE
  CALL MPI_ABORT(MPI_COMM_WORLD, 99, IERROR)
ENDIF
CALL MPI_TYPE_GET_CONTENTS(DTYPE, NI, NA, ND, I, A, D, IERROR)

```

or in C the analogous calls of:

```

#define LARGE 1000
int ni, na, nd, combiner, i[LARGE];
MPI_Aint a[LARGE];
MPI_Datatype dtype, d[LARGE];
/* construct datatype dtype (not shown) */
MPI_Type_get_envelope(dtype, &ni, &na, &nd, &combiner);
if ((ni > LARGE) || (na > LARGE) || (nd > LARGE)) {
  fprintf(stderr, "ni, na, or nd = %d %d %d returned by ", ni, na, nd);
  fprintf(stderr, "MPI_Type_get_envelope is larger than LARGE = %d\n",
    LARGE);
  MPI_Abort(MPI_COMM_WORLD, 99);
}
MPI_Type_get_contents(dtype, ni, na, nd, i, a, d);

```

The following describes the values of the arguments for each combiner. The lower case name of arguments is used. Also, the descriptions below refer to MPI datatypes created by procedures without large count arguments.

MPI_COMBINER_NAMED the datatype represents a predefined type and therefore it is erroneous to call `MPI_TYPE_GET_CONTENTS`.

MPI_COMBINER_DUP `ni = 0, na = 0, nd = 1`, and

Constructor argument	C	Fortran location
oldtype	d[0]	D(1)

MPI_COMBINER_CONTIGUOUS `ni = 1, na = 0, nd = 1`, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
oldtype	d[0]	D(1)

MPI_COMBINER_VECTOR ni = 3, na = 0, nd = 1, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
blocklength	i[1]	I(2)
stride	i[2]	I(3)
oldtype	d[0]	D(1)

MPI_COMBINER_HVECTOR ni = 2, na = 1, nd = 1, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
blocklength	i[1]	I(2)
stride	a[0]	A(1)
oldtype	d[0]	D(1)

MPI_COMBINER_INDEXED ni = 2*count+1, na = 0, nd = 1, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
array_of_blocklengths	i[1] to i[i[0]]	I(2) to I(I(1)+1)
array_of_displacements	i[i[0]+1] to i[2*i[0]]	I(I(1)+2) to I(2*I(1)+1)
oldtype	d[0]	D(1)

MPI_COMBINER_HINDEXED ni = count+1, na = count, nd = 1, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
array_of_blocklengths	i[1] to i[i[0]]	I(2) to I(I(1)+1)
array_of_displacements	a[0] to a[i[0]-1]	A(1) to A(I(1))
oldtype	d[0]	D(1)

MPI_COMBINER_INDEXED_BLOCK ni = count+2, na = 0, nd = 1, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
blocklength	i[1]	I(2)
array_of_displacements	i[2] to i[i[0]+1]	I(3) to I(I(1)+2)
oldtype	d[0]	D(1)

MPI_COMBINER_HINDEXED_BLOCK ni = 2, na = count, nd = 1, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
blocklength	i[1]	I(2)
array_of_displacements	a[0] to a[i[0]-1]	A(1) to A(I(1))
oldtype	d[0]	D(1)

MPI_COMBINER_STRUCT ni = count+1, na = count, nd = count, and

Constructor argument	C	Fortran location
count	i[0]	I(1)
array_of_blocklengths	i[1] to i[i[0]]	I(2) to I(I(1)+1)
array_of_displacements	a[0] to a[i[0]-1]	A(1) to A(I(1))
array_of_types	d[0] to d[i[0]-1]	D(1) to D(I(1))

MPI_COMBINER_SUBARRAY $ni = 3*ndims+2$, $na = 0$, $nd = 1$, and

Constructor argument	C	Fortran location
ndims	$i[0]$	$I(1)$
array_of_sizes	$i[1]$ to $i[i[0]]$	$I(2)$ to $I(I(1)+1)$
array_of_subsizes	$i[i[0]+1]$ to $i[2*i[0]]$	$I(I(1)+2)$ to $I(2*I(1)+1)$
array_of_starts	$i[2*i[0]+1]$ to $i[3*i[0]]$	$I(2*I(1)+2)$ to $I(3*I(1)+1)$
order	$i[3*i[0]+1]$	$I(3*I(1)+2)$
oldtype	$d[0]$	$D(1)$

MPI_COMBINER_DARRAY $ni = 4*ndims+4$, $na = 0$, $nd = 1$, and

Constructor argument	C	Fortran location
size	$i[0]$	$I(1)$
rank	$i[1]$	$I(2)$
ndims	$i[2]$	$I(3)$
array_of_gsizes	$i[3]$ to $i[i[2]+2]$	$I(4)$ to $I(I(3)+3)$
array_of_distrib	$i[i[2]+3]$ to $i[2*i[2]+2]$	$I(I(3)+4)$ to $I(2*I(3)+3)$
array_of_dargs	$i[2*i[2]+3]$ to $i[3*i[2]+2]$	$I(2*I(3)+4)$ to $I(3*I(3)+3)$
array_of_psize	$i[3*i[2]+3]$ to $i[4*i[2]+2]$	$I(3*I(3)+4)$ to $I(4*I(3)+3)$
order	$i[4*i[2]+3]$	$I(4*I(3)+4)$
oldtype	$d[0]$	$D(1)$

MPI_COMBINER_F90_REAL $ni = 2$, $na = 0$, $nd = 0$, and

Constructor argument	C	Fortran location
p	$i[0]$	$I(1)$
r	$i[1]$	$I(2)$

MPI_COMBINER_F90_COMPLEX $ni = 2$, $na = 0$, $nd = 0$, and

Constructor argument	C	Fortran location
p	$i[0]$	$I(1)$
r	$i[1]$	$I(2)$

MPI_COMBINER_F90_INTEGER $ni = 1$, $na = 0$, $nd = 0$, and

Constructor argument	C	Fortran location
r	$i[0]$	$I(1)$

MPI_COMBINER_RESIZED $ni = 0$, $na = 2$, $nd = 1$, and

Constructor argument	C	Fortran location
lb	$a[0]$	$A(1)$
extent	$a[1]$	$A(2)$
oldtype	$d[0]$	$D(1)$

MPI_COMBINER_VALUE_INDEX $ni = 0$, $na = 0$, $nd = 2$, and

Constructor argument	C	Fortran location
value_type	$d[0]$	$D(1)$
index_type	$d[1]$	$D(2)$

5.1.14 Examples

The following examples illustrate the use of derived datatypes.

Example 5.13. Send and receive a section of a 3D array.

```

REAL a(100,100,100), e(9,9,9)
INTEGER oneslice, twoslice, threeslice, myrank, ierr
INTEGER(KIND=MPI_ADDRESS_KIND) lb, sizeofreal
INTEGER status(MPI_STATUS_SIZE)

! extract the section a(1:17:2, 3:11, 2:10)
! and store it in e(:, :, :).

CALL MPI_COMM_RANK(MPI_COMM_WORLD, myrank, ierr)

CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lb, sizeofreal, ierr)

! create datatype for a 1D section
CALL MPI_TYPE_VECTOR(9, 1, 2, MPI_REAL, oneslice, ierr)

! create datatype for a 2D section
CALL MPI_TYPE_CREATE_HVECTOR(9, 1, 100*sizeofreal, oneslice, &
                             twoslice, ierr)

! create datatype for the entire section
CALL MPI_TYPE_CREATE_HVECTOR(9, 1, 100*100*sizeofreal, twoslice, &
                             threeslice, ierr)

CALL MPI_TYPE_COMMIT(threeslice, ierr)
CALL MPI_SENDRECV(a(1,3,2), 1, threeslice, myrank, 0, e, 9*9*9, &
                  MPI_REAL, myrank, 0, MPI_COMM_WORLD, status, ierr)

```

Example 5.14. Copy the (strictly) lower triangular part of a matrix.

```

REAL a(100,100), b(100,100)
INTEGER disp(100), blocklen(100), ltype, myrank, ierr
INTEGER status(MPI_STATUS_SIZE)

! copy lower triangular part of array a
! onto lower triangular part of array b

CALL MPI_COMM_RANK(MPI_COMM_WORLD, myrank, ierr)

! compute start and size of each column
DO i=1,100
    disp(i) = 100*(i-1) + i
    blocklen(i) = 100-i
END DO

! create datatype for lower triangular part
CALL MPI_TYPE_INDEXED(100, blocklen, disp, MPI_REAL, ltype, ierr)

```

```

CALL MPI_TYPE_COMMIT(ltype, ierr)
CALL MPI_SENDRECV(a, 1, ltype, myrank, 0, b, 1, &
                  ltype, myrank, 0, MPI_COMM_WORLD, status, ierr)

```

Example 5.15. Transpose a matrix.

```

REAL a(100,100), b(100,100)
INTEGER row, xpose, myrank, ierr
INTEGER(KIND=MPI_ADDRESS_KIND) lb, sizeofreal
INTEGER status(MPI_STATUS_SIZE)

! transpose matrix a onto b

CALL MPI_COMM_RANK(MPI_COMM_WORLD, myrank, ierr)

CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lb, sizeofreal, ierr)

! create datatype for one row
CALL MPI_TYPE_VECTOR(100, 1, 100, MPI_REAL, row, ierr)

! create datatype for matrix in row-major order
CALL MPI_TYPE_CREATE_HVECTOR(100, 1, sizeofreal, row, xpose, ierr)

CALL MPI_TYPE_COMMIT(xpose, ierr)

! send matrix in row-major order and receive in column major order
CALL MPI_SENDRECV(a, 1, xpose, myrank, 0, b, 100*100, &
                  MPI_REAL, myrank, 0, MPI_COMM_WORLD, status, ierr)

```

Example 5.16. Another approach to the transpose problem:

```

REAL a(100,100), b(100,100)
INTEGER row, row1
INTEGER(KIND=MPI_ADDRESS_KIND) lb, sizeofreal
INTEGER myrank, ierr
INTEGER status(MPI_STATUS_SIZE)

CALL MPI_COMM_RANK(MPI_COMM_WORLD, myrank, ierr)

! transpose matrix a onto b

CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lb, sizeofreal, ierr)

! create datatype for one row
CALL MPI_TYPE_VECTOR(100, 1, 100, MPI_REAL, row, ierr)

! create datatype for one row, with the extent of one real number
lb = 0
CALL MPI_TYPE_CREATE_RESIZED(row, lb, sizeofreal, row1, ierr)

CALL MPI_TYPE_COMMIT(row1, ierr)

```

```

1
2 ! send 100 rows and receive in column major order
3 CALL MPI_SENDRCV(a, 100, row1, myrank, 0, b, 100*100, &
4                 MPI_REAL, myrank, 0, MPI_COMM_WORLD, status, ierr)
5
6

```

Example 5.17. Use of MPI datatypes to manipulate an array of structures.

```

8 struct Partstruct
9 {
10     int    type;    /* particle type */
11     double d[6];    /* particle coordinates */
12     char   b[7];    /* some additional information */
13 };
14
15 struct Partstruct    particle[1000];
16
17 int                i, dest, tag;
18 MPI_Comm          comm;
19
20 /* build datatype describing structure */
21
22 MPI_Datatype Particlestruct, Particletype;
23 MPI_Datatype type[3] = {MPI_INT, MPI_DOUBLE, MPI_CHAR};
24 int          blocklen[3] = {1, 6, 7};
25 MPI_Aint     disp[3];
26 MPI_Aint     base, lb, sizeofentry;
27
28 /* compute displacements of structure components */
29
30 MPI_Get_address(particle, disp);
31 MPI_Get_address(particle[0].d, disp+1);
32 MPI_Get_address(particle[0].b, disp+2);
33 base = disp[0];
34 for (i=0; i < 3; i++) disp[i] = MPI_Aint_diff(disp[i], base);
35
36 MPI_Type_create_struct(3, blocklen, disp, type, &Particlestruct);
37
38 /* Since the compiler may pad the structure, it is best to explicitly
39    set the extent of the MPI datatype for a structure element using
40    MPI_Type_create_resized */
41
42 /* compute extent of the structure */
43 MPI_Get_address(particle+1, &sizeofentry);
44 sizeofentry = MPI_Aint_diff(sizeofentry, base);
45
46 /* build datatype describing structure */
47 MPI_Type_create_resized(Particlestruct, 0, sizeofentry, &Particletype);
48
49 /* 4.1: send the entire array */

```



```

1
2 MPI_Type_commit(&Particletype);
3 MPI_Send(particle, 1000, Particletype, dest, tag, comm);
4
5
6 /* 4.2: send only the entries of type zero particles,
7    preceded by the number of such entries */
8
9 MPI_Datatype Zparticles; /* datatype describing all particles
10                          with type zero (needs to be recomputed
11                          if types change) */
12
13 MPI_Datatype Ztype;
14
15 int          zdisp[1000];
16 int          zblock[1000], j, k;
17 int          zzbblock[2] = {1,1};
18 MPI_Aint     zzdisp[2];
19 MPI_Datatype zztype[2];
20
21 /* compute displacements of type zero particles */
22 j = 0;
23 for (i=0; i < 1000; i++)
24     if (particle[i].type == 0)
25     {
26         zdisp[j] = i;
27         zblock[j] = 1;
28         j++;
29     }
30
31 /* create datatype for type zero particles */
32 MPI_Type_indexed(j, zblock, zdisp, Particletype, &Zparticles);
33
34 /* prepend particle count */
35 MPI_Get_address(&j, zzdisp);
36 MPI_Get_address(particle, zzdisp+1);
37 zztype[0] = MPI_INT;
38 zztype[1] = Zparticles;
39 MPI_Type_create_struct(2, zzbblock, zzdisp, zztype, &Ztype);
40
41 MPI_Type_commit(&Ztype);
42 MPI_Send(MPI_BOTTOM, 1, Ztype, dest, tag, comm);
43
44
45 /* A probably more efficient way of defining Zparticles */
46
47 /* consecutive particles with index zero are handled as one block */
48 j=0;
49 for (i=0; i < 1000; i++)
50     if (particle[i].type == 0)
51     {
52         for (k=i+1; (k < 1000)&&(particle[k].type == 0); k++);
53         zdisp[j] = i;
54         zblock[j] = k-i;
55     }

```

```

1      j++;
2      i = k;
3  }
4  MPI_Type_indexed(j, zblock, zdisp, Particletype, &Zparticles);
5
6
7  /* 4.3: send the first two coordinates of all entries */
8
9  MPI_Datatype Allpairs;      /* datatype for all pairs of coordinates */
10
11 MPI_Type_get_extent(Particletype, &lb, &sizeofentry);
12
13 /* sizeofentry can also be computed by subtracting the address
14    of particle[0] from the address of particle[1] */
15
16 MPI_Type_create_hvector(1000, 2, sizeofentry, MPI_DOUBLE, &Allpairs);
17 MPI_Type_commit(&Allpairs);
18 MPI_Send(particle[0].d, 1, Allpairs, dest, tag, comm);
19
20 /* an alternative solution to 4.3 */
21
22 MPI_Datatype Twodouble;
23
24 MPI_Type_contiguous(2, MPI_DOUBLE, &Twodouble);
25
26 MPI_Datatype Onepair;      /* datatype for one pair of coordinates, with
27                               the extent of one particle entry */
28
29 MPI_Type_create_resized(Twodouble, 0, sizeofentry, &Onepair );
30 MPI_Type_commit(&Onepair);
31 MPI_Send(particle[0].d, 1000, Onepair, dest, tag, comm);

```

Example 5.18. The same manipulations as in the previous example, but use absolute addresses in datatypes.

```

33 struct Partstruct
34 {
35     int    type;
36     double d[6];
37     char   b[7];
38 };
39
40 struct Partstruct particle[1000];
41
42 /* build datatype describing first array entry */
43
44 MPI_Datatype Particletype;
45 MPI_Datatype type[3] = {MPI_INT, MPI_DOUBLE, MPI_CHAR};
46 int          block[3] = {1, 6, 7};
47 MPI_Aint     disp[3];
48
49 MPI_Get_address(particle, disp);

```

```

MPI_Get_address(particle[0].d, disp+1);
MPI_Get_address(particle[0].b, disp+2);
MPI_Type_create_struct(3, block, disp, type, &Particletype);

/* Particletype describes first array entry -- using absolute
   addresses */

/* 5.1: send the entire array */

MPI_Type_commit(&Particletype);
MPI_Send(MPI_BOTTOM, 1000, Particletype, dest, tag, comm);

/* 5.2: send the entries of type zero,
   preceded by the number of such entries */

MPI_Datatype Zparticles, Ztype;

int      zdisp[1000];
int      zblock[1000], i, j, k;
int      zzbblock[2] = {1,1};
MPI_Datatype zztype[2];
MPI_Aint zzdisp[2];

j=0;
for (i=0; i < 1000; i++)
    if (particle[i].type == 0)
    {
        for (k=i+1; (k < 1000)&&(particle[k].type == 0); k++);
        zdisp[j] = i;
        zblock[j] = k-i;
        j++;
        i = k;
    }
MPI_Type_indexed(j, zblock, zdisp, Particletype, &Zparticles);
/* Zparticles describe particles with type zero, using
   their absolute addresses*/

/* prepend particle count */
MPI_Get_address(&j, zzdisp);
zzdisp[1] = (MPI_Aint)0;
zztype[0] = MPI_INT;
zztype[1] = Zparticles;
MPI_Type_create_struct(2, zzbblock, zzdisp, zztype, &Ztype);

MPI_Type_commit(&Ztype);
MPI_Send(MPI_BOTTOM, 1, Ztype, dest, tag, comm);

```

Example 5.19. This example shows how datatypes can be used to handle unions.

```

union {
    int    ival;

```

```

1      float    fval;
2          } u[1000];
3
4      int      i, utype;
5
6      /* All entries of u have identical type; variable
7         utype keeps track of their current type */
8
9      MPI_Datatype  mpi_utype[2];
10     MPI_Aint      ubase, extent;
11
12     /* compute an MPI datatype for each possible union type;
13        assume values are left-aligned in union storage. */
14
15     MPI_Get_address(u, &ubase);
16     MPI_Get_address(u+1, &extent);
17     extent = MPI_Aint_diff(extent, ubase);
18
19     MPI_Type_create_resized(MPI_INT, 0, extent, &mpi_utype[0]);
20
21     MPI_Type_create_resized(MPI_FLOAT, 0, extent, &mpi_utype[1]);
22
23     for(i=0; i<2; i++) MPI_Type_commit(&mpi_utype[i]);
24
25     /* actual communication */
26     MPI_Send(u, 1000, mpi_utype[utype], dest, tag, comm);

```

Example 5.20. This example shows how a datatype can be decoded. The routine `printdatatype` prints out the elements of the datatype. Note the use of `MPI_Type_free` for datatypes that are not predefined.

```

30     /*
31        Example of decoding a datatype.
32
33        Returns 0 if the datatype is predefined, 1 otherwise
34        */
35     #include <stdio.h>
36     #include <stdlib.h>
37     #include "mpi.h"
38     int printdatatype(MPI_Datatype datatype)
39     {
40         int *array_of_ints;
41         MPI_Aint *array_of_adds;
42         MPI_Datatype *array_of_dtypes;
43         int num_ints, num_adds, num_dtypes, combiner;
44         int i;
45
46         MPI_Type_get_envelope(datatype,
47                               &num_ints, &num_adds, &num_dtypes, &combiner);
48         switch (combiner) {
49             case MPI_COMBINER_NAMED:
50                 printf("Datatype is named:");

```

```

/* To print the specific type, we can match against the
   predefined forms. We can NOT use a switch statement here
   We could also use MPI_TYPE_GET_NAME if we preferred to use
   names that the user may have changed.
*/
if (datatype == MPI_INT)    printf("MPI_INT\n");
else if (datatype == MPI_DOUBLE) printf("MPI_DOUBLE\n");
... else test for other types ...
return 0;
break;
case MPI_COMBINER_STRUCT:
    printf("Datatype is struct containing");
    array_of_ints = (int *)malloc(num_ints * sizeof(int));
    array_of_adds =
        (MPI_Aint *) malloc(num_adds * sizeof(MPI_Aint));
    array_of_dtypes = (MPI_Datatype *)
        malloc(num_dtypes * sizeof(MPI_Datatype));
    MPI_Type_get_contents(datatype, num_ints, num_adds, num_dtypes,
        array_of_ints, array_of_adds, array_of_dtypes);
    printf(" %d datatypes:\n", array_of_ints[0]);
    for (i=0; i<array_of_ints[0]; i++) {
        printf("blocklength %d, displacement %ld, type:\n",
            array_of_ints[i+1], (long)array_of_adds[i]);
        if (printdatatype(array_of_dtypes[i])) {
            /* Note that we free the type ONLY if it
               is not predefined */
            MPI_Type_free(&array_of_dtypes[i]);
        }
    }
    free(array_of_ints);
    free(array_of_adds);
    free(array_of_dtypes);
    break;
    ... other combiner values ...
default:
    printf("Unrecognized combiner type\n");
}
return 1;
}

```

5.2 Pack and Unpack

Some existing communication libraries provide pack/unpack procedures for sending noncontiguous data. In these, the user explicitly packs data into a contiguous buffer before sending it, and unpacks it from a contiguous buffer after receiving it. Derived datatypes, which are described in Section 5.1, allow one, in most cases, to avoid explicit packing and unpacking. The user specifies the layout of the data to be sent or received, and the communication library directly accesses a noncontiguous buffer. The pack/unpack routines are provided for compatibility with previous libraries. Also, they provide some functionality that is not otherwise available in MPI. For instance, a message can be received in several parts, where the receive operation done on a later part may depend on the content of a former part.

Another use is that outgoing messages may be explicitly buffered in user supplied space, thus overriding the system buffering policy. Finally, the availability of pack and unpack operations facilitates the development of additional communication libraries layered on top of MPI.

MPI_PACK(inbuf, incount, datatype, outbuf, outsize, position, comm)

IN	inbuf	input buffer start (choice)
IN	incount	number of input data items (nonnegative integer)
IN	datatype	datatype of each input data item (handle)
OUT	outbuf	output buffer start (choice)
IN	outsize	output buffer size, in bytes (nonnegative integer)
INOUT	position	current position in buffer, in bytes (integer)
IN	comm	communicator for packed message (handle)

C binding

```
int MPI_Pack(const void *inbuf, int incount, MPI_Datatype datatype,
            void *outbuf, int outsize, int *position, MPI_Comm comm)

int MPI_Pack_c(const void *inbuf, MPI_Count incount, MPI_Datatype datatype,
              void *outbuf, MPI_Count outsize, MPI_Count *position,
              MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Pack(inbuf, incount, datatype, outbuf, outsize, position, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
  INTEGER, INTENT(IN) :: incount, outsize
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(*), DIMENSION(..) :: outbuf
  INTEGER, INTENT(INOUT) :: position
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Pack(inbuf, incount, datatype, outbuf, outsize, position, comm, ierror)
  !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incount, outsize
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(*), DIMENSION(..) :: outbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PACK(INBUF, INCOUNT, DATATYPE, OUTBUF, OUTSIZE, POSITION, COMM, IERROR)
  <type> INBUF(*), OUTBUF(*)
  INTEGER INCOUNT, DATATYPE, OUTSIZE, POSITION, COMM, IERROR
```

Packs the message in the send buffer specified by `inbuf`, `incount`, `datatype` into the buffer space specified by `outbuf` and `outsize`. The input buffer can be any communication buffer allowed in `MPI_SEND`. The output buffer is a contiguous storage area containing `outsize` bytes, starting at the address `outbuf` (length is counted in *bytes*, not elements, as if it were a communication buffer for a message of type `MPI_PACKED`).

The input value of `position` is the first location in the output buffer to be used for packing. `position` is incremented by the size of the packed message, and the output value of `position` is the first location in the output buffer following the locations occupied by the packed message. The `comm` argument is the communicator that will be subsequently used for sending the packed message.

MPI_UNPACK(`inbuf`, `insize`, `position`, `outbuf`, `outcount`, `datatype`, `comm`)

IN	<code>inbuf</code>	input buffer start (choice)
IN	<code>insize</code>	size of input buffer, in bytes (nonnegative integer)
INOUT	<code>position</code>	current position in bytes (integer)
OUT	<code>outbuf</code>	output buffer start (choice)
IN	<code>outcount</code>	number of items to be unpacked (integer)
IN	<code>datatype</code>	datatype of each output data item (handle)
IN	<code>comm</code>	communicator for packed message (handle)

C binding

```
int MPI_Unpack(const void *inbuf, int insize, int *position, void *outbuf,
               int outcount, MPI_Datatype datatype, MPI_Comm comm)
```

```
int MPI_Unpack_c(const void *inbuf, MPI_Count insize, MPI_Count *position,
                  void *outbuf, MPI_Count outcount, MPI_Datatype datatype,
                  MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Unpack(inbuf, insize, position, outbuf, outcount, datatype, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
  INTEGER, INTENT(IN) :: insize, outcount
  INTEGER, INTENT(INOUT) :: position
  TYPE(*), DIMENSION(..) :: outbuf
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Unpack(inbuf, insize, position, outbuf, outcount, datatype, comm, ierror)
  !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: insize, outcount
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
  TYPE(*), DIMENSION(..) :: outbuf
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_UNPACK(INBUF, INSIZE, POSITION, OUTBUF, OUTCOUNT, DATATYPE, COMM, IERROR)
  <type> INBUF(*), OUTBUF(*)
  INTEGER INSIZE, POSITION, OUTCOUNT, DATATYPE, COMM, IERROR

```

Unpacks a message into the receive buffer specified by `outbuf`, `outcount`, `datatype` from the buffer space specified by `inbuf` and `insize`. The output buffer can be any communication buffer allowed in `MPI_RECV`. The input buffer is a contiguous storage area containing `insize` bytes, starting at address `inbuf`. The input value of `position` is the first location in the input buffer occupied by the packed message. `position` is incremented by the size of the packed message, so that the output value of `position` is the first location in the input buffer after the locations occupied by the message that was unpacked. `comm` is the communicator used to receive the packed message.

Advice to users. Note the difference between `MPI_RECV` and `MPI_UNPACK`: in `MPI_RECV`, the `count` argument specifies the maximum number of items that can be received. The actual number of items received is determined by the length of the incoming message. In `MPI_UNPACK`, the `count` argument specifies the actual number of items that are unpacked; the “size” of the corresponding message is the increment in `position`. The reason for this change is that the “incoming message size” is not predetermined since the user decides how much to unpack; nor is it easy to determine the “message size” from the number of items to be unpacked. In fact, in a heterogeneous system, this number may not be determined *a priori*. (*End of advice to users.*)

To understand the behavior of pack and unpack, it is convenient to think of the data part of a message as being the sequence obtained by concatenating the successive values sent in that message. The pack operation stores this sequence in the buffer space, as if sending the message to that buffer. The unpack operation retrieves this sequence from buffer space, as if receiving a message from that buffer. (It is helpful to think of internal Fortran files or `sscanf` in C, for a similar function.)

Several messages can be successively packed into one **packing unit**. This is effected by several successive **related** calls to `MPI_PACK`, where the first call provides `position = 0`, and each successive call inputs the value of `position` that was output by the previous call, and the same values for `outbuf`, `outcount` and `comm`. This packing unit now contains the equivalent information that would have been stored in a message by one send call with a send buffer that is the “concatenation” of the individual send buffers.

A packing unit can be sent using type `MPI_PACKED`. Any point-to-point or collective communication operation can be used to move the sequence of bytes that forms the packing unit from one process to another. This packing unit can now be received using any receive operation, with any datatype: the type matching rules are relaxed for messages sent with type `MPI_PACKED`.

A message sent with any type (including `MPI_PACKED`) can be received using the type `MPI_PACKED`. Such a message can then be unpacked by calls to `MPI_UNPACK`.

A packing unit (or a message created by a regular, “typed” send) can be unpacked into several successive messages. This is effected by several successive related calls to `MPI_UNPACK`, where the first call provides `position = 0`, and each successive call inputs the value of `position` that was output by the previous call, and the same values for `inbuf`, `insize` and `comm`.

The concatenation of two packing units is not necessarily a packing unit; nor is a substring of a packing unit necessarily a packing unit. Thus, one cannot concatenate two packing units and then unpack the result as one packing unit; nor can one unpack a substring of a packing unit as a separate packing unit. Each packing unit, that was created by a related sequence of pack calls, or by a regular send, must be unpacked as a unit, by a sequence of related unpack calls.

Rationale. The restriction on “atomic” packing and unpacking of packing units allows the implementation to add at the head of packing units additional information, such as a description of the sender architecture (to be used for type conversion, in a heterogeneous environment) (*End of rationale.*)

The following call allows the user to find out how much space is needed to pack a message and, thus, manage space allocation for buffers.

MPI_PACK_SIZE(incount, datatype, comm, size)

IN	incount	count argument to packing call (nonnegative integer)
IN	datatype	datatype argument to packing call (handle)
IN	comm	communicator argument to packing call (handle)
OUT	size	upper bound on size of packed message, in bytes (nonnegative integer)

C binding

```
int MPI_Pack_size(int incount, MPI_Datatype datatype, MPI_Comm comm, int *size)
```

```
int MPI_Pack_size_c(MPI_Count incount, MPI_Datatype datatype, MPI_Comm comm,
                    MPI_Count *size)
```

Fortran 2008 binding

```
MPI_Pack_size(incount, datatype, comm, size, ierror)
```

```
INTEGER, INTENT(IN) :: incount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Pack_size(incount, datatype, comm, size, ierror) !(_c)
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PACK_SIZE(INCOUNT, DATATYPE, COMM, SIZE, IERROR)
```

```
INTEGER INCOUNT, DATATYPE, COMM, SIZE, IERROR
```

A call to **MPI_PACK_SIZE**(incount, datatype, comm, size) returns in size an upper bound on the increment in position that is effected by a call to **MPI_PACK**(inbuf, incount,

datatype, outbuf, outcount, position, comm). If the packed size of the datatype cannot be expressed by the size parameter, then `MPI_PACK_SIZE` sets the value of size to `MPI_UNDEFINED`.

Rationale. The call returns an upper bound, rather than an exact bound, since the exact amount of space needed to pack the message may depend on the context (e.g., first message packed in a packing unit may take more space). (*End of rationale.*)

Example 5.21. An example using `MPI_PACK`.

```

10  int      position, i, j, a[2];
11  char     buff[1000];
12
13  MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
14  if (myrank == 0)
15  {
16      /* SENDER CODE */
17      position = 0;
18      MPI_Pack(&i, 1, MPI_INT, buff, 1000, &position, MPI_COMM_WORLD);
19      MPI_Pack(&j, 1, MPI_INT, buff, 1000, &position, MPI_COMM_WORLD);
20      MPI_Send(buff, position, MPI_PACKED, 1, 0, MPI_COMM_WORLD);
21  }
22  else /* RECEIVER CODE */
23      MPI_Recv(a, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

Example 5.22. An elaborate example.

```

26  int      position, i = 200;
27  float    a[200];
28  char     buff[1000]; /* larger than or equal to the size returned
29                        from MPI_PACK_SIZE for 1,newtype */
30  MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
31  if (myrank == 0)
32  {
33      /* SENDER CODE */
34      int len[2];
35      MPI_Aint disp[2];
36      MPI_Datatype type[2], newtype;
37
38      /* build datatype for i followed by a[0]...a[i-1] */
39      len[0] = 1;
40      len[1] = i;
41      MPI_Get_address(&i, disp);
42      MPI_Get_address(a, disp+1);
43      type[0] = MPI_INT;
44      type[1] = MPI_FLOAT;
45      MPI_Type_create_struct(2, len, disp, type, &newtype);
46      MPI_Type_commit(&newtype);
47
48      /* Pack i followed by a[0]...a[i-1]*/
49      position = 0;
50      MPI_Pack(MPI_BOTTOM, 1, newtype, buff, 1000, &position,

```

```

        MPI_COMM_WORLD);

    /* Send */
    MPI_Send(buff, position, MPI_PACKED, 1, 0,
             MPI_COMM_WORLD);

/* *****
One can replace the last three lines with
MPI_Send(MPI_BOTTOM, 1, newtype, 1, 0, MPI_COMM_WORLD);
***** */
}
else if (myrank == 1)
{
    /* RECEIVER CODE */
    MPI_Status status;

    /* Receive */
    MPI_Recv(buff, 1000, MPI_PACKED, 0, 0, MPI_COMM_WORLD, &status);

    /* Unpack i */
    position = 0;
    MPI_Unpack(buff, 1000, &position, &i, 1, MPI_INT, MPI_COMM_WORLD);

    /* Unpack a[0]...a[i-1] */
    MPI_Unpack(buff, 1000, &position, a, i, MPI_FLOAT, MPI_COMM_WORLD);
}

```

Example 5.23. Each process sends a count, followed by count characters to the root; the root concatenates all characters into one string.

```

int  count, gsize, counts[64], totalcount, k1, k2, k,
     displs[64], position, concat_pos;
char chr[100], *lbuf, *rbuf, *cbuf;

MPI_Comm_size(comm, &gsize);
MPI_Comm_rank(comm, &myrank);

    /* allocate local pack buffer */
MPI_Pack_size(1, MPI_INT, comm, &k1);
MPI_Pack_size(count, MPI_CHAR, comm, &k2);
k = k1+k2;
lbuf = (char *)malloc(k);

    /* pack count, followed by count characters */
position = 0;
MPI_Pack(&count, 1, MPI_INT, lbuf, k, &position, comm);
MPI_Pack(chr, count, MPI_CHAR, lbuf, k, &position, comm);

if (myrank != root) {
    /* gather at root sizes of all packed messages */
    MPI_Gather(&position, 1, MPI_INT, NULL, 0,
              MPI_DATATYPE_NULL, root, comm);
}

```

```

1
2      /* gather at root packed messages */
3      MPI_Gatherv(lbuf, position, MPI_PACKED, NULL,
4                  NULL, NULL, MPI_DATATYPE_NULL, root, comm);
5
6  } else { /* root code */
7      /* gather sizes of all packed messages */
8      MPI_Gather(&position, 1, MPI_INT, counts, 1,
9                 MPI_INT, root, comm);
10
11     /* gather all packed messages */
12     displs[0] = 0;
13     for (i=1; i < gsize; i++)
14         displs[i] = displs[i-1] + counts[i-1];
15     totalcount = displs[gsz-1] + counts[gsz-1];
16     rbuf = (char *)malloc(totalcount);
17     cbuf = (char *)malloc(totalcount);
18     MPI_Gatherv(lbuf, position, MPI_PACKED, rbuf,
19                 counts, displs, MPI_PACKED, root, comm);
20
21     /* unpack all messages and concatenate strings */
22     concat_pos = 0;
23     for (i=0; i < gsize; i++) {
24         position = 0;
25         MPI_Unpack(rbuf+displs[i], totalcount-displs[i],
26                   &position, &count, 1, MPI_INT, comm);
27         MPI_Unpack(rbuf+displs[i], totalcount-displs[i],
28                   &position, cbuf+concat_pos, count, MPI_CHAR, comm);
29         concat_pos += count;
30     }
31     cbuf[concat_pos] = '\0';
32 }

```

5.3 Canonical MPI_PACK and MPI_UNPACK

These procedures read/write data to/from the buffer in the "external32" data format specified in Section 15.5.2, and calculate the size needed for packing. Their first arguments specify the data format, for future extensibility, but currently the only valid value of the `datarep` argument is "external32".

Advice to users. These procedures could be used, for example, to send typed data in a portable format from one MPI implementation to another. (*End of advice to users.*)

The buffer will contain exactly the packed data, without headers. `MPI_BYTE` should be used to send and receive data that is packed using `MPI_PACK_EXTERNAL`.

Rationale. `MPI_PACK_EXTERNAL` specifies that there is no header on the message and further specifies the exact format of the data. Since `MPI_PACK` may (and is allowed to) use a header, the datatype `MPI_PACKED` cannot be used for data packed with `MPI_PACK_EXTERNAL`. (*End of rationale.*)

MPI_PACK_EXTERNAL(datarep, inbuf, incout, datatype, outbuf, outsize, position)

IN	datarep	data representation (string)
IN	inbuf	input buffer start (choice)
IN	incout	number of input data items (integer)
IN	datatype	datatype of each input data item (handle)
OUT	outbuf	output buffer start (choice)
IN	outsize	output buffer size, in bytes (integer)
INOUT	position	current position in buffer, in bytes (integer)

C binding

```
int MPI_Pack_external(const char datarep[], const void *inbuf, int incout,
                    MPI_Datatype datatype, void *outbuf, MPI_Aint outsize,
                    MPI_Aint *position)
```

```
int MPI_Pack_external_c(const char datarep[], const void *inbuf,
                    MPI_Count incout, MPI_Datatype datatype, void *outbuf,
                    MPI_Count outsize, MPI_Count *position)
```

Fortran 2008 binding

```
MPI_Pack_external(datarep, inbuf, incout, datatype, outbuf, outsize, position,
                  ierror)
```

```
CHARACTER(LEN=*) INTENT(IN) :: datarep
TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
INTEGER, INTENT(IN) :: incout
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(*), DIMENSION(..) :: outbuf
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: outsize
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(INOUT) :: position
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Pack_external(datarep, inbuf, incout, datatype, outbuf, outsize, position,
                  ierror) !(_c)
```

```
CHARACTER(LEN=*) INTENT(IN) :: datarep
TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incout, outsize
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(*), DIMENSION(..) :: outbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PACK_EXTERNAL(DATAREP, INBUF, INCOUNT, DATATYPE, OUTBUF, OUTSIZE, POSITION,
                  IERROR)
```

```
CHARACTER*(*) DATAREP
<type> INBUF(*), OUTBUF(*)
INTEGER INCOUNT, DATATYPE, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) OUTSIZE, POSITION
```

```
1 MPI_UNPACK_EXTERNAL(datarep, inbuf, insize, position, outbuf, outcount, datatype)
```

```
2     IN      datarep      data representation (string)
3
4     IN      inbuf       input buffer start (choice)
5
6     IN      insize      input buffer size, in bytes (integer)
7
8     INOUT   position    current position in buffer, in bytes (integer)
9
10    OUT     outbuf      output buffer start (choice)
11
12    IN      outcount    number of output data items (integer)
13
14    IN      datatype    datatype of output data item (handle)
```

12 C binding

```
13 int MPI_Unpack_external(const char datarep[], const void *inbuf,
14                        MPI_Aint insize, MPI_Aint *position, void *outbuf, int outcount,
15                        MPI_Datatype datatype)
16
```

```
17 int MPI_Unpack_external_c(const char datarep[], const void *inbuf,
18                          MPI_Count insize, MPI_Count *position, void *outbuf,
19                          MPI_Count outcount, MPI_Datatype datatype)
20
```

21 Fortran 2008 binding

```
22 MPI_Unpack_external(datarep, inbuf, insize, position, outbuf, outcount,
23                    datatype, ierror)
```

```
24 CHARACTER(LEN=*), INTENT(IN) :: datarep
25 TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
26 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: insize
27 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(INOUT) :: position
28 TYPE(*), DIMENSION(..) :: outbuf
29 INTEGER, INTENT(IN) :: outcount
30 TYPE(MPI_Datatype), INTENT(IN) :: datatype
31 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
32 MPI_Unpack_external(datarep, inbuf, insize, position, outbuf, outcount,
33                    datatype, ierror) !(_c)
```

```
34 CHARACTER(LEN=*), INTENT(IN) :: datarep
35 TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
36 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: insize, outcount
37 INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
38 TYPE(*), DIMENSION(..) :: outbuf
39 TYPE(MPI_Datatype), INTENT(IN) :: datatype
40 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

41 Fortran binding

```
42 MPI_UNPACK_EXTERNAL(DATAREP, INBUF, INSIZE, POSITION, OUTBUF, OUTCOUNT,
43                    DATATYPE, IERROR)
```

```
44 CHARACTER*(*) DATAREP
45 <type> INBUF(*), OUTBUF(*)
46 INTEGER(KIND=MPI_ADDRESS_KIND) INSIZE, POSITION
47 INTEGER OUTCOUNT, DATATYPE, IERROR
48
```

MPI_PACK_EXTERNAL_SIZE(datarep, incount, datatype, size)

IN	datarep	data representation (string)
IN	incount	number of input data items (integer)
IN	datatype	datatype of each input data item (handle)
OUT	size	output buffer size, in bytes (integer)

C binding

```
int MPI_Pack_external_size(const char datarep[], int incount,
                          MPI_Datatype datatype, MPI_Aint *size)
```

```
int MPI_Pack_external_size_c(const char datarep[], MPI_Count incount,
                             MPI_Datatype datatype, MPI_Count *size)
```

Fortran 2008 binding

```
MPI_Pack_external_size(datarep, incount, datatype, size, ierror)
```

```
  CHARACTER(LEN=*) , INTENT(IN) :: datarep
  INTEGER , INTENT(IN) :: incount
  TYPE(MPI_Datatype) , INTENT(IN) :: datatype
  INTEGER(KIND=MPI_ADDRESS_KIND) , INTENT(OUT) :: size
  INTEGER , OPTIONAL , INTENT(OUT) :: ierror
```

```
MPI_Pack_external_size(datarep, incount, datatype, size, ierror) !(_c)
```

```
  CHARACTER(LEN=*) , INTENT(IN) :: datarep
  INTEGER(KIND=MPI_COUNT_KIND) , INTENT(IN) :: incount
  TYPE(MPI_Datatype) , INTENT(IN) :: datatype
  INTEGER(KIND=MPI_COUNT_KIND) , INTENT(OUT) :: size
  INTEGER , OPTIONAL , INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PACK_EXTERNAL_SIZE(DATAREP, INCOUNT, DATATYPE, SIZE, IERROR)
```

```
  CHARACTER*(*) DATAREP
  INTEGER INCOUNT, DATATYPE, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) SIZE
```

DRAFT

Chapter 6

Collective Communication

6.1 Introduction and Overview

Collective communication is defined as communication that involves a group or groups of MPI processes. The functions of this type provided by MPI are the following:

- `MPI_BARRIER`, `MPI_IBARRIER`, `MPI_BARRIER_INIT`: Barrier synchronization across all members of a group (Section 6.3, Section 6.12.1, and Section 6.13.1).
- `MPI_BCAST`, `MPI_IBCAST`, `MPI_BCAST_INIT`: Broadcast from one member to all members of a group (Section 6.4, Section 6.12.2, and Section 6.13.2). This is shown as “broadcast” in Figure 6.1.
- `MPI_GATHER`, `MPI_IGATHER`, `MPI_GATHER_INIT`, `MPI_GATHERV`, `MPI_IGATHERV`, `MPI_GATHERV_INIT`, : Gather data from all members of a group to one member (Section 6.5, Section 6.12.3, and Section 6.13.3). This is shown as “gather” in Figure 6.1.
- `MPI_SCATTER`, `MPI_ISCATTER`, `MPI_SCATTER_INIT`, `MPI_SCATTERV`, `MPI_ISCATTERV`, `MPI_SCATTERV_INIT`: Scatter data from one member to all members of a group (Section 6.6, Section 6.12.4, and Section 6.13.4). This is shown as “scatter” in Figure 6.1.
- `MPI_ALLGATHER`, `MPI_IALLGATHER`, `MPI_ALLGATHER_INIT`, `MPI_ALLGATHERV`, `MPI_IALLGATHERV`, `MPI_ALLGATHERV_INIT`: A variation on Gather where all members of a group receive the result (Section 6.7, Section 6.12.5, and Section 6.13.5). This is shown as “allgather” in Figure 6.1.
- `MPI_ALLTOALL`, `MPI_IALLTOALL`, `MPI_ALLTOALL_INIT`, `MPI_ALLTOALLV`, `MPI_IALLTOALLV`, `MPI_ALLTOALLV_INIT`, `MPI_ALLTOALLW`, `MPI_IALLTOALLW`, `MPI_ALLTOALLW_INIT`: Scatter/Gather data from all members to all members of a group (also called complete exchange) (Section 6.8, Section 6.12.6, and Section 6.13.6). This is shown as “complete exchange” in Figure 6.1.
- `MPI_ALLREDUCE`, `MPI_IALLREDUCE`, `MPI_ALLREDUCE_INIT`, `MPI_REDUCE`, `MPI_IREDUCE`, `MPI_REDUCE_INIT`: Global reduction operations such as sum, max, min, or user-defined functions, where the result is returned to all members of a group (Section 6.9.6, Section 6.12.8, and Section 6.13.8) and a variation where the result is returned to only one member (Section 6.9, Section 6.12.7, and Section 6.13.7).
- `MPI_REDUCE_SCATTER_BLOCK`, `MPI_IREDUCE_SCATTER_BLOCK`, `MPI_REDUCE_SCATTER_BLOCK_INIT`, `MPI_REDUCE_SCATTER`,

`MPI_IREDUCE_SCATTER`, `MPI_REDUCE_SCATTER_INIT`: A combined reduction and scatter operation (Section 6.10, Section 6.12.9, Section 6.12.10, Section 6.13.9, and Section 6.13.10).

- `MPI_SCAN`, `MPI_ISCAN`, `MPI_SCAN_INIT`, `MPI_EXSCAN`, `MPI_IEXSCAN`, `MPI_EXSCAN_INIT`: Scan across all members of a group (also called prefix) (Section 6.11, Section 6.11.2, Section 6.12.11, Section 6.12.12, Section 6.13.11, and Section 6.13.12).

One of the key arguments in a call to a collective routine is a communicator that defines the group or groups of participating MPI processes and provides a context for the operation. This is discussed further in Section 6.2. The syntax and semantics of the collective operations are defined to be consistent with the syntax and semantics of the point-to-point operations. Thus, general datatypes are allowed and must match between sending and receiving MPI processes as specified in Chapter 5. Several collective routines such as broadcast and gather have a single originating or receiving MPI process. Such an MPI process is called the **root**. Some arguments in the collective functions are specified as “significant only at root,” and are ignored for all participants except the root.

Advice to users. Note that the programmer is still responsible for avoiding undefined behavior in the host language by not passing uninitialized values to MPI procedure calls. (*End of advice to users.*)

The reader is referred to Chapter 5 for information concerning communication buffers, general datatypes and type matching rules, and to Chapter 7 for information on how to define groups and create communicators.

The type-matching conditions for the collective operations are more strict than the corresponding conditions between sender and receiver in point-to-point. Namely, for collective operations, the amount of data sent must exactly match the amount of data specified by the receiver. Different type maps (the layout in memory, see Section 5.1) between sender and receiver are still allowed.

Collective operations can (but are not required to) complete as soon as the caller’s participation in the collective communication is finished. A blocking operation is complete as soon as the call returns. A nonblocking (immediate) call requires a separate completion call (cf. Section 3.7). The completion of a collective operation indicates that the caller is free to modify locations in the communication buffer. It does not indicate that other MPI processes in the group have completed or even started the operation (unless otherwise implied by the description of the operation). Thus, a collective communication operation may, or may not, have the effect of synchronizing all participating MPI processes.

Collective communication calls may use the same communicators as point-to-point communication; MPI guarantees that messages generated on behalf of collective communication calls will not be confused with messages generated by point-to-point communication. The collective operations do not have a message tag argument. A more detailed discussion of correct use of collective routines is found in Section 6.14.

Rationale. The equal-data restriction (on type matching) was made so as to avoid the complexity of providing a facility analogous to the status argument of `MPI_RECV` for discovering the amount of data sent. Some of the collective routines would require an array of status values.

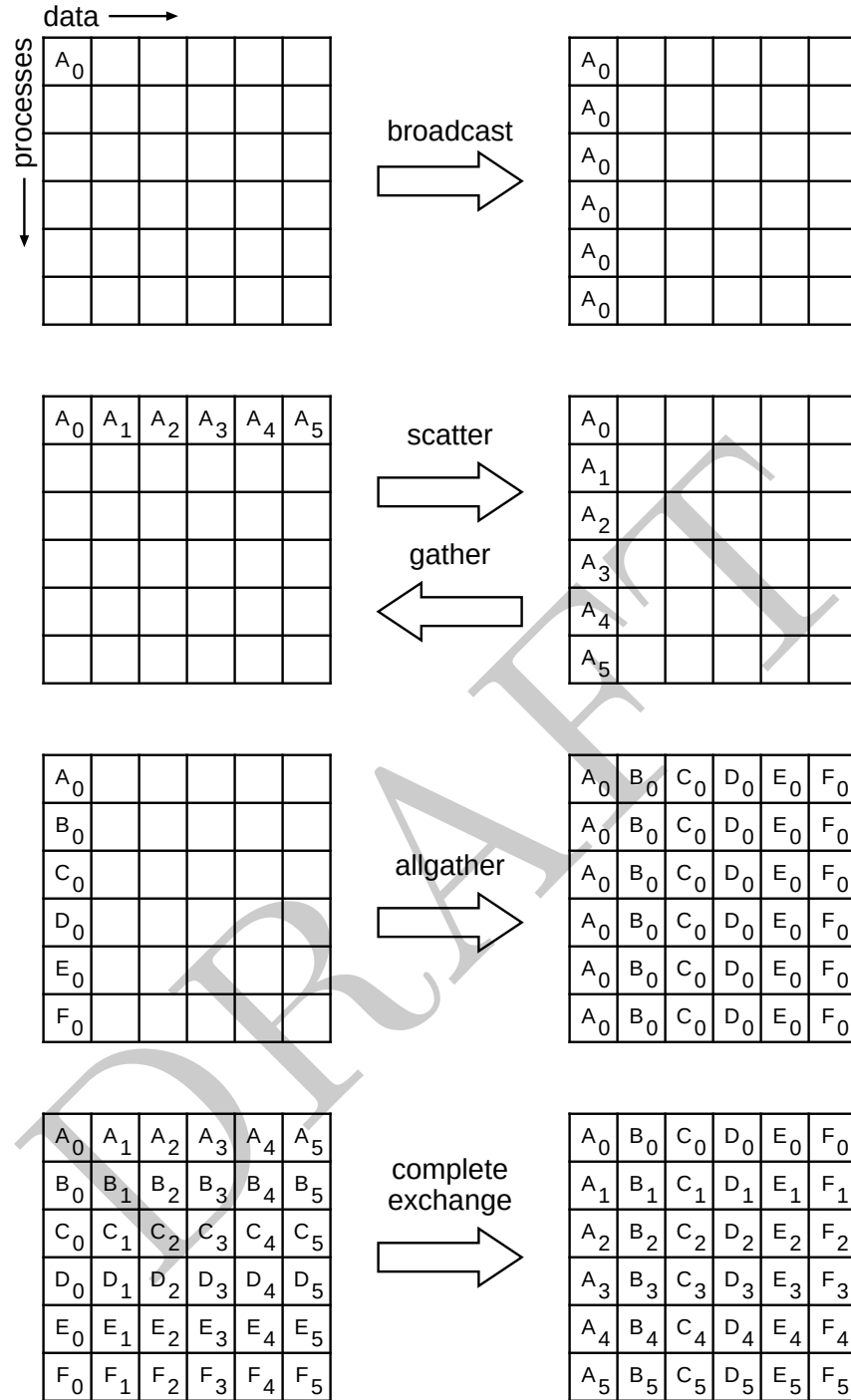


Figure 6.1: Collective move functions illustrated for a group of six MPI processes. In each case, each row of boxes represents data locations in one MPI process. Thus, in the broadcast, initially just the first MPI process contains the data A_0 , but after the broadcast all MPI processes contain it.

The statements about synchronization are made so as to allow a variety of implementations of the collective functions.

(End of rationale.)

Advice to users. It is dangerous to rely on synchronization side-effects of the collective operations for program correctness. For example, even though a particular implementation may provide a broadcast routine with a side-effect of synchronization, the standard does not require this, and a program that relies on this will not be portable.

On the other hand, a correct, portable program must allow for the fact that a collective call *may* be synchronizing. Though one cannot rely on any synchronization side-effect, one must program so as to allow it. These issues are discussed further in Section 6.14. *(End of advice to users.)*

Advice to implementors. While vendors may write optimized collective routines matched to their architectures, a complete library of the collective communication routines can be written entirely using the MPI point-to-point communication functions and a few auxiliary functions. If implementing on top of point-to-point, a hidden, special communicator might be created for the collective operation so as to avoid interference with any on-going point-to-point communication at the time of the collective call. This is discussed further in Section 6.14. *(End of advice to implementors.)*

Many of the descriptions of the collective routines provide illustrations in terms of blocking MPI point-to-point routines. These are intended solely to indicate what data is sent or received by which MPI process. Many of these examples are *not* correct MPI programs; for purposes of simplicity, they often assume infinite buffering.

6.2 Communicator Argument

The key concept of the collective functions is to have a group or groups of participating MPI processes. The routines do not have group identifiers as explicit arguments. Instead, there is a communicator argument. Groups and communicators are discussed in full detail in Chapter 7. For the purposes of this chapter, it is sufficient to know that there are two types of communicators: **intra-communicators** and **inter-communicators**. An intra-communicator can be thought of as an identifier for a single group of MPI processes linked with a context. An inter-communicator identifies two distinct groups of MPI processes linked with a context.

6.2.1 Specifics for Intra-Communicator Collective Operations

All MPI processes in the group identified by the intra-communicator must call the collective routine.

In many cases, collective communication can occur “in place” for intra-communicators, with the output buffer being identical to the input buffer. This is specified by providing a special argument value, `MPI_IN_PLACE`, instead of the send buffer or the receive buffer argument, depending on the operation performed.

Rationale. The “in place” operations are provided to reduce unnecessary memory motion by both the MPI implementation and by the user. Note that while the simple check

of testing whether the send and receive buffers have the same address will work for some cases (e.g., `MPI_ALLREDUCE`), they are inadequate in others (e.g., `MPI_GATHER`, with `root` not equal to zero). Further, Fortran explicitly prohibits aliasing of arguments; the approach of using a special value to denote “in place” operation eliminates that difficulty. (*End of rationale.*)

Advice to users. By allowing the “in place” option, the receive buffer in many of the collective calls becomes a send-and-receive buffer. For this reason, a Fortran binding that includes `INTENT` must mark these as `INOUT`, not `OUT`.

Note that `MPI_IN_PLACE` is a special kind of value; it has the same restrictions on its use that `MPI_BOTTOM` has (not usable in Fortran for initialization or assignment). See Section 2.5.4. (*End of advice to users.*)

6.2.2 Applying Collective Operations to Inter-Communicators

To understand how collective operations apply to inter-communicators, we can view most MPI intra-communicator collective operations as fitting one of the following categories (see, for instance, [61]):

All-To-All All MPI processes contribute to the result. All MPI processes receive the result.

- `MPI_ALLGATHER`, `MPI_IALLGATHER`, `MPI_ALLGATHER_INIT`,
`MPI_ALLGATHERV`, `MPI_IALLGATHERV`, `MPI_ALLGATHERV_INIT`
- `MPI_ALLTOALL`, `MPI_IALLTOALL`, `MPI_ALLTOALL_INIT`, `MPI_ALLTOALLV`,
`MPI_IALLTOALLV`, `MPI_ALLTOALLV_INIT`, `MPI_ALLTOALLW`,
`MPI_IALLTOALLW`, `MPI_ALLTOALLW_INIT`
- `MPI_ALLREDUCE`, `MPI_IALLREDUCE`, `MPI_ALLREDUCE_INIT`,
`MPI_REDUCE_SCATTER_BLOCK`, `MPI_IREDUCE_SCATTER_BLOCK`,
`MPI_REDUCE_SCATTER_BLOCK_INIT`, `MPI_REDUCE_SCATTER`,
`MPI_IREDUCE_SCATTER`, `MPI_REDUCE_SCATTER_INIT`
- `MPI_BARRIER`, `MPI_IBARRIER`, `MPI_BARRIER_INIT`

All-To-One All MPI processes contribute to the result. One MPI process receives the result.

- `MPI_GATHER`, `MPI_IGATHER`, `MPI_GATHER_INIT`, `MPI_GATHERV`,
`MPI_IGATHERV`, `MPI_GATHERV_INIT`
- `MPI_REDUCE`, `MPI_IREDUCE`, `MPI_REDUCE_INIT`,

One-To-All One MPI process contributes to the result. All MPI processes receive the result.

- `MPI_BCAST`, `MPI_IBCAST`, `MPI_BCAST_INIT`
- `MPI_SCATTER`, `MPI_ISCATTER`, `MPI_SCATTER_INIT`, `MPI_SCATTERV`,
`MPI_ISCATTERV`, `MPI_SCATTERV_INIT`

Other: Collective operations that do not fit into one of the above categories.

- `MPI_SCAN`, `MPI_ISCAN`, `MPI_SCAN_INIT`, `MPI_EXSCAN`, `MPI_IEXSCAN`,
`MPI_EXSCAN_INIT`

The application of collective communication to inter-communicators is best described in terms of two groups. For example, an all-to-all `MPI_ALLGATHER` operation can be described as collecting data from all members of one group with the result appearing in all members of the other group (see Figure 6.2). As another example, a one-to-all `MPI_BCAST` operation sends data from one member of one group to all members of the other group. Collective computation operations such as `MPI_REDUCE_SCATTER` have a similar interpretation (see Figure 6.3). For intra-communicators, these two groups are the same. For inter-communicators, these two groups are distinct. For the all-to-all operations, each such operation is described in two phases, so that it has a symmetric, full-duplex behavior.

The following collective operations also apply to inter-communicators:

- `MPI_BARRIER`, `MPI_IBARRIER`, `MPI_BARRIER_INIT`,
- `MPI_BCAST`, `MPI_IBCAST`, `MPI_BCAST_INIT`,
- `MPI_GATHER`, `MPI_IGATHER`, `MPI_GATHER_INIT`, `MPI_GATHERV`,
`MPI_IGATHERV`, `MPI_GATHERV_INIT`,
- `MPI_SCATTER`, `MPI_ISCATTER`, `MPI_SCATTER_INIT`, `MPI_SCATTERV`,
`MPI_ISCATTERV`, `MPI_SCATTERV_INIT`,
- `MPI_ALLGATHER`, `MPI_IALLGATHER`, `MPI_ALLGATHER_INIT`,
`MPI_ALLGATHERV`, `MPI_IALLGATHERV`, `MPI_ALLGATHERV_INIT`,
- `MPI_ALLTOALL`, `MPI_IALLTOALL`, `MPI_ALLTOALL_INIT`, `MPI_ALLTOALLV`,
`MPI_IALLTOALLV`, `MPI_ALLTOALLV_INIT`, `MPI_ALLTOALLW`, `MPI_IALLTOALLW`,
`MPI_ALLTOALLW_INIT`,
- `MPI_ALLREDUCE`, `MPI_IALLREDUCE`, `MPI_ALLREDUCE_INIT`, `MPI_REDUCE`,
`MPI_IREDUCE`, `MPI_REDUCE_INIT`,
- `MPI_REDUCE_SCATTER_BLOCK`, `MPI_IREDUCE_SCATTER_BLOCK`,
`MPI_REDUCE_SCATTER_BLOCK_INIT`, `MPI_REDUCE_SCATTER`,
`MPI_IREDUCE_SCATTER`, `MPI_REDUCE_SCATTER_INIT`.

6.2.3 Specifics for Inter-Communicator Collective Operations

All MPI processes in both groups identified by the inter-communicator must call the collective routine.

Note that the “in place” option for intra-communicators does not apply to inter-communicators since in the inter-communicator case there is no communication from an MPI process to itself.

For inter-communicator collective communication, if the operation is in the All-To-One or One-To-All categories, then the transfer is unidirectional. The direction of the transfer is indicated by a special value of the `root` argument. In this case, for the group containing the root, all MPI processes in the group must call the routine using a special argument for the root. For this, the root uses the special value `MPI_ROOT`; all other MPI processes in the same group as the root use `MPI_PROC_NULL`. All MPI processes in the other group (the group that is the remote group relative to the root) must call the collective routine and

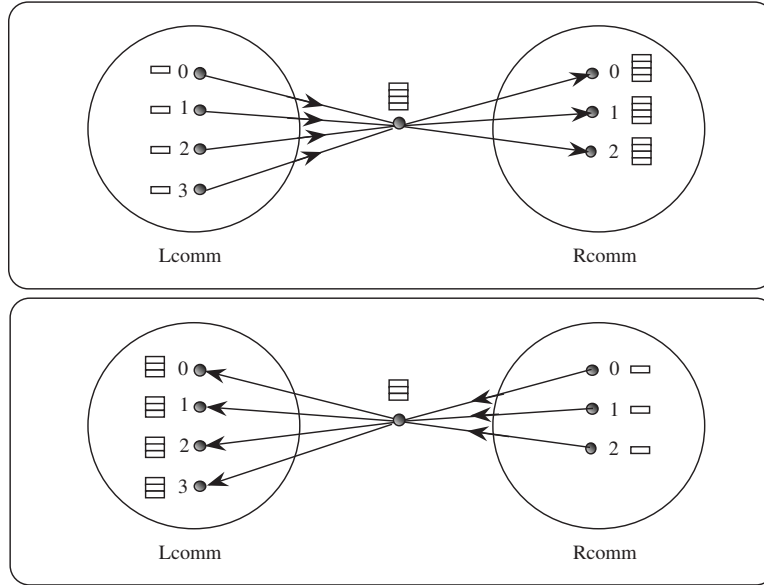


Figure 6.2: Inter-communicator allgather. The focus of data to one MPI process is represented, not mandated by the semantics. The two phases do allgathers in both directions.

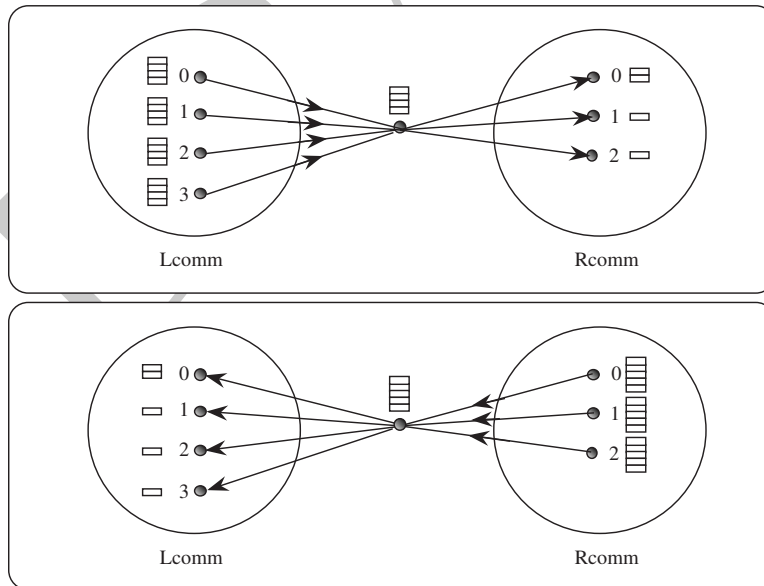


Figure 6.3: Inter-communicator reduce-scatter. The focus of data to one MPI process is represented, not mandated by the semantics. The two phases do reduce-scatters in both directions.

provide the rank of the root. If the operation is in the All-To-All category, then the transfer is bidirectional.

Rationale. Operations in the All-To-One and One-To-All categories are unidirectional by nature, and there is a clear way of specifying direction. Operations in the All-To-All category will often occur as part of an exchange, where it makes sense to communicate in both directions at once. (*End of rationale.*)

6.3 Barrier Synchronization

MPI_BARRIER(comm)

IN comm communicator (handle)

C binding

int MPI_Barrier(MPI_Comm comm)

Fortran 2008 binding

MPI_Barrier(comm, ierror)

TYPE(MPI_Comm), INTENT(IN) :: comm

INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

MPI_BARRIER(COMM, IERROR)

INTEGER COMM, IERROR

If comm is an intra-communicator, **MPI_BARRIER** blocks the caller until all group members have called it. The call returns at any MPI process only after all group members have entered the call.

If comm is an inter-communicator, **MPI_BARRIER** involves two groups. The call returns at MPI processes in one group (group A) of the inter-communicator only after all members of the other group (group B) have entered the call (and vice versa). An MPI process may return from the call before all MPI processes in its own group have entered the call.

6.4 Broadcast

MPI_BCAST(buffer, count, datatype, root, comm)

INOUT buffer starting address of buffer (choice)

IN count number of entries in buffer (nonnegative integer)

IN datatype datatype of buffer (handle)

IN root rank of the root (integer)

IN comm communicator (handle)

C binding

```

int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root,
              MPI_Comm comm)

int MPI_Bcast_c(void *buffer, MPI_Count count, MPI_Datatype datatype, int root,
                MPI_Comm comm)

```

Fortran 2008 binding

```

MPI_Bcast(buffer, count, datatype, root, comm, ierror)
    TYPE(*), DIMENSION(..) :: buffer
    INTEGER, INTENT(IN) :: count, root
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Bcast(buffer, count, datatype, root, comm, ierror) !(_c)
    TYPE(*), DIMENSION(..) :: buffer
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, INTENT(IN) :: root
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_BCAST(BUFFER, COUNT, DATATYPE, ROOT, COMM, IERROR)
    <type> BUFFER(*)
    INTEGER COUNT, DATATYPE, ROOT, COMM, IERROR

```

If `comm` is an intra-communicator, `MPI_BCAST` broadcasts a message from the MPI process with rank `root` to all MPI processes of the group, itself included. It is called by all members of the group using the same arguments for `comm` and `root`. On return, the content of the root's buffer is copied to all other MPI processes.

General, derived datatypes are allowed for `datatype`. The type signature of `count`, `datatype` on any MPI process must be equal to the type signature of `count`, `datatype` at the root. This implies that the amount of data sent must be equal to the amount received, pairwise between each MPI process and the root. `MPI_BCAST` and all other data-movement collective routines make this restriction. Distinct type maps between sender and receiver are still allowed.

The “in place” option is not meaningful here.

If `comm` is an inter-communicator, then the call involves all MPI processes in the inter-communicator, but with one group (group A) defining the root. All MPI processes in the other group (group B) pass the same value in argument `root`, which is the rank of the root in group A. The root passes the value `MPI_ROOT` in `root`. All other MPI processes in group A pass the value `MPI_PROC_NULL` in `root`. Data is broadcast from the root to all MPI processes in group B. The buffer arguments of the MPI processes in group B must be consistent with the buffer argument of the root.

6.4.1 Example using `MPI_BCAST`

The examples in this section use intra-communicators.

Example 6.1. Broadcast 100 ints from MPI process 0 to every MPI process in the group.

```
MPI_Comm comm;
int array[100];
int root=0;
...
MPI_Bcast(array, 100, MPI_INT, root, comm);
```

As in many of our example code fragments, we assume that some of the variables (such as `comm` in the above) have been assigned appropriate values.

6.5 Gather

`MPI_GATHER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)`

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	recvcount	number of elements for any single receive (nonnegative integer, significant only at root)
IN	recvtype	datatype of recv buffer elements (handle, significant only at root)
IN	root	rank of receiving MPI process (integer)
IN	comm	communicator (handle)

C binding

```
int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
              void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
              MPI_Comm comm)
```

```
int MPI_Gather_c(const void *sendbuf, MPI_Count sendcount,
                 MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                 MPI_Datatype recvtype, int root, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
           comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  INTEGER, INTENT(IN) :: sendcount, recvcount, root
  TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
  TYPE(*), DIMENSION(..) :: recvbuf
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```

MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
           comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE, ROOT,
           COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, ROOT, COMM, IERROR

```

If `comm` is an intra-communicator, each MPI process (the root included) sends the contents of its send buffer to the root. The root receives the messages and stores them in rank order. The outcome is *as if* each of the n MPI processes in the group (including the root) had executed a call to

```
MPI_Send(sendbuf, sendcount, sendtype, root , ...),
```

and the root had executed n calls to

```
MPI_Recv(recvbuf+i·recvcount·extent(recvtype), recvcount, recvtype, i,...),
```

where `extent(recvtype)` is the type extent obtained from a call to `MPI_Type_get_extent`.

An alternative description is that the n messages sent by the processes in the group are concatenated in rank order, and the resulting message is received by the root as if by a call to `MPI_RECV(recvbuf, recvcount· $n$ , recvtype, ...)`.

The receive buffer is ignored for all nonroot MPI processes.

General, derived datatypes are allowed for both `sendtype` and `recvtype`. The type signature of `sendcount`, `sendtype` on each MPI process must be equal to the type signature of `recvcount`, `recvtype` at the root. This implies that the amount of data sent must be equal to the amount of data received, pairwise between each MPI process and the root. Distinct type maps between sender and receiver MPI processes are still allowed.

All arguments to the function are significant on the root, while on other MPI processes, only the arguments `sendbuf`, `sendcount`, `sendtype`, `root`, and `comm` are significant. The arguments `root` and `comm` must have identical values on all MPI processes.

The specification of counts and types should not cause any location on the root to be written more than once. Such a call is erroneous.

Note that the `recvcount` argument at the root indicates the number of items it receives from *each* MPI process, not the total number of items it receives.

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` as the value of `sendbuf` at the root. In such a case, `sendcount` and `sendtype` are ignored, and the contribution of the root to the gathered vector is assumed to be already in the correct place in the receive buffer.

If `comm` is an inter-communicator, then the call involves all MPI processes in the inter-communicator, but with one group (group A) defining the root. All MPI processes in the

other group (group B) pass the same value in argument `root`, which is the rank of the root in group A. The root passes the value `MPI_ROOT` in `root`. All other MPI processes in group A pass the value `MPI_PROC_NULL` in `root`. Data is gathered from all MPI processes in group B to the root. The send buffer arguments of the MPI processes in group B must be consistent with the receive buffer argument of the root.

MPI_GATHERV(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs, recvtype, root, comm)

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	recvcunts	nonnegative integer array (of length group size) containing the number of elements that are received from each MPI process (significant only at root)
IN	displs	integer array (of length group size). Entry <i>i</i> specifies the displacement relative to <code>recvbuf</code> at which to place the incoming data from MPI process <i>i</i> (significant only at root)
IN	recvtype	datatype of recv buffer elements (handle, significant only at root)
IN	root	rank of receiving MPI process (integer)
IN	comm	communicator (handle)

C binding

```
int MPI_Gatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, const int recvcunts[], const int displs[],
               MPI_Datatype recvtype, int root, MPI_Comm comm)
```

```
int MPI_Gatherv_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf,
                  const MPI_Count recvcunts[], const MPI_Aint displs[],
                  MPI_Datatype recvtype, int root, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Gatherv(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs,
            recvtype, root, comm, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcunts(*), displs(*), root
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..) :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```

MPI_Gatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
            recvtype, root, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcounts(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
            RECVTYPE, ROOT, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, ROOT,
            COMM, IERROR

```

MPI_GATHERV extends the functionality of **MPI_GATHER** by allowing a varying count of data from each MPI process, since **recvcounts** is now an array. It also allows more flexibility as to where the data is placed on the root, by providing the new argument, **displs**.

If **comm** is an intra-communicator, the outcome is *as if* each MPI process, including the root, sends a message to the root,

```
MPI_Send(sendbuf, sendcount, sendtype, root, ...),
```

and the root executes **n** receives,

```
MPI_Recv(recvbuf+displs[j]·extent(recvtype), recvcounts[j], recvtype, i, ...).
```

The data received from MPI process *j* is placed into **recvbuf** of the root beginning at offset **displs[j]** elements (in terms of the **recvtype**).

The receive buffer is ignored for all nonroot MPI processes.

The type signature implied by **sendcount**, **sendtype** on MPI process *i* must be equal to the type signature implied by **recvcounts[i]**, **recvtype** at the root. This implies that the amount of data sent must be equal to the amount of data received, pairwise between each MPI process and the root. Distinct type maps between sender and receiver are still allowed, as illustrated in Example 6.6.

All arguments to the function are significant on the root, while on other MPI processes, only arguments **sendbuf**, **sendcount**, **sendtype**, **root**, and **comm** are significant. The arguments **root** and **comm** must have identical values on all MPI processes.

The specification of counts, types, and displacements should not cause any location on the root to be written more than once. Such a call is erroneous.

The “in place” option for intra-communicators is specified by passing **MPI_IN_PLACE** as the value of **sendbuf** at the root. In such a case, **sendcount** and **sendtype** are ignored, and the contribution of the root to the gathered vector is assumed to be already in the correct place in the receive buffer.

If **comm** is an inter-communicator, then the call involves all MPI processes in the inter-communicator, but with one group (group A) defining the root. All MPI processes in the

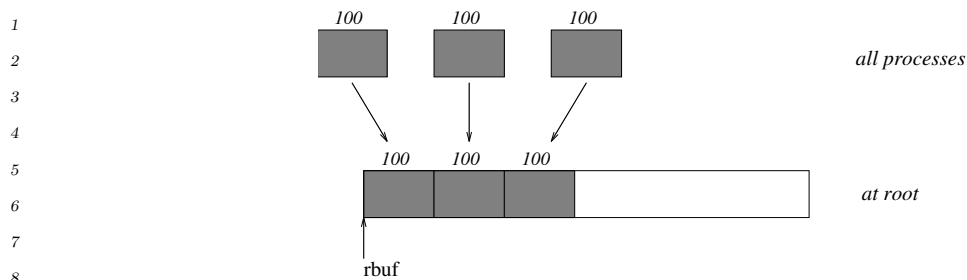


Figure 6.4: The root gathers 100 ints from each MPI process in the group

other group (group B) pass the same value in argument `root`, which is the rank of the root in group A. The root passes the value `MPI_ROOT` in `root`. All other MPI processes in group A pass the value `MPI_PROC_NULL` in `root`. Data is gathered from all MPI processes in group B to the root. The send buffer arguments of the MPI processes in group B must be consistent with the receive buffer argument of the root.

6.5.1 Examples using `MPI_GATHER` and `MPI_GATHERV`

The examples in this section use intra-communicators.

Example 6.2. Gather 100 ints from every MPI process in group to the root. See Figure 6.4.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, *rbuf;
...
MPI_Comm_size(comm, &gsize);
rbuf = (int *)malloc(gsize*100*sizeof(int));
MPI_Gather(sendarray, 100, MPI_INT, rbuf, 100, MPI_INT, root, comm);
```

Example 6.3. Previous example modified—only the root allocates memory for the receive buffer. The argument `rbuf` still must be initialized on all processes.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, myrank, *rbuf = NULL;
...
MPI_Comm_rank(comm, &myrank);
if (myrank == root) {
    MPI_Comm_size(comm, &gsize);
    rbuf = (int *)malloc(gsize*100*sizeof(int));
}
MPI_Gather(sendarray, 100, MPI_INT, rbuf, 100, MPI_INT, root, comm);
```

Example 6.4. Do the same as the previous example, but use a derived datatype. Note that the type cannot be the entire set of `gsize*100` ints since type matching is defined pairwise between the root and each MPI process in the gather.

```
MPI_Comm comm;
```

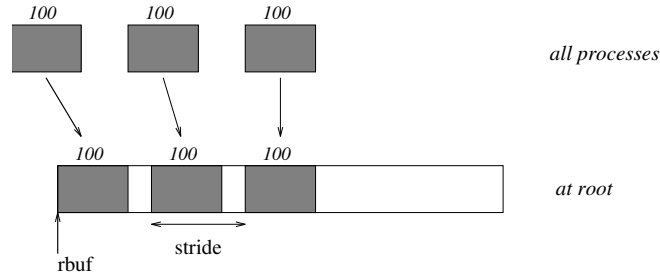


Figure 6.5: The root gathers 100 ints from each MPI process in the group, each set is placed stride ints apart

```
int gsize, sendarray[100];
int root, *rbuf;
MPI_Datatype rtype;
...
MPI_Comm_size(comm, &gsize);
MPI_Type_contiguous(100, MPI_INT, &rtype);
MPI_Type_commit(&rtype);
rbuf = (int *)malloc(gsize*100*sizeof(int));
MPI_Gather(sendarray, 100, MPI_INT, rbuf, 1, rtype, root, comm);
```

Example 6.5. Now have each MPI process send 100 ints to the root, but place each set (of 100) stride ints apart at the receiving end. Use `MPI_GATHERV` and the `displs` argument to achieve this effect. Assume `stride ≥ 100`. See Figure 6.5.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, *rbuf, stride;
int *displs, i, rcounts;
...

MPI_Comm_size(comm, &gsize);
rbuf = (int *)malloc(gsize*stride*sizeof(int));
displs = (int *)malloc(gsize*sizeof(int));
rcounts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    rcounts[i] = 100;
}
MPI_Gatherv(sendarray, 100, MPI_INT, rbuf, rcounts, displs, MPI_INT,
            root, comm);
```

Note that the program is erroneous if `stride < 100`.

Example 6.6. Same as Example 6.5 on the receiving side, but send the 100 ints from the 0th column of a 100×150 int array, in C. See Figure 6.6.

```
MPI_Comm comm;
```

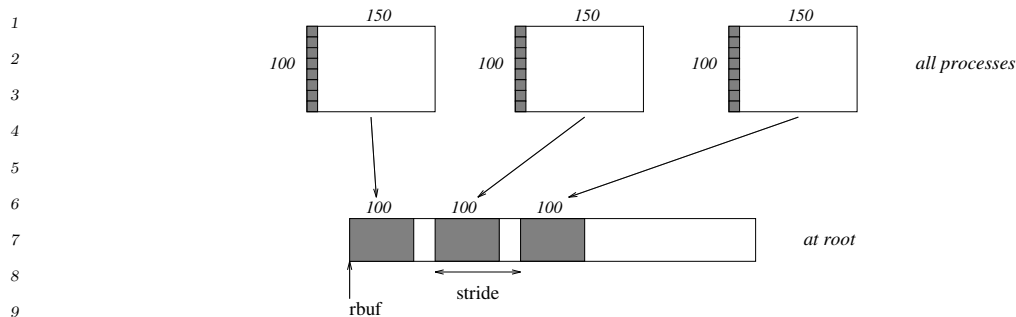


Figure 6.6: The root gathers column 0 of a 100×150 C array, and each set is placed `stride` ints apart

```

14  int gsize, sendarray[100][150];
15  int root, *rbuf, stride;
16  MPI_Datatype stype;
17  int *displs, i, *rcounts;
18
19  ...
20
21  MPI_Comm_size(comm, &gsize);
22  rbuf = (int *)malloc(gsize*stride*sizeof(int));
23  displs = (int *)malloc(gsize*sizeof(int));
24  rcounts = (int *)malloc(gsize*sizeof(int));
25  for (i=0; i<gsize; ++i) {
26      displs[i] = i*stride;
27      rcounts[i] = 100;
28  }
29  /* Create datatype for 1 column of array
30   */
31  MPI_Type_vector(100, 1, 150, MPI_INT, &stype);
32  MPI_Type_commit(&stype);
33  MPI_Gatherv(sendarray, 1, stype, rbuf, rcounts, displs, MPI_INT,
34              root, comm);

```

Example 6.7. MPI process i sends $(100-i)$ ints from the i -th column of a 100×150 int array, in C. It is received into a buffer with stride, as in the previous two examples. See Figure 6.7.

```

38  MPI_Comm comm;
39  int gsize, sendarray[100][150], *sptr;
40  int root, *rbuf, stride, myrank;
41  MPI_Datatype stype;
42  int *displs, i, *rcounts;
43
44  ...
45
46  MPI_Comm_size(comm, &gsize);
47  MPI_Comm_rank(comm, &myrank);
48  rbuf = (int *)malloc(gsize*stride*sizeof(int));

```

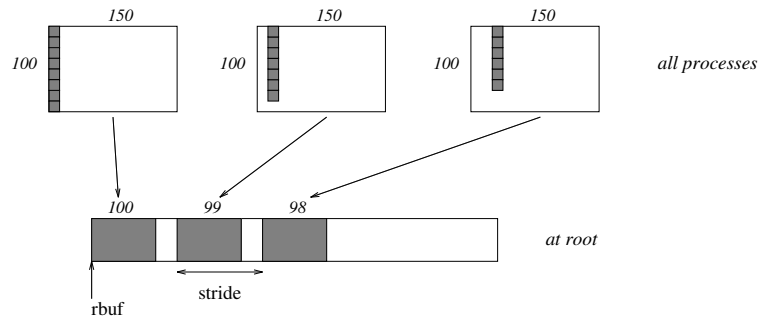



Figure 6.7: The root gathers $100-i$ ints from column i of a 100×150 C array, and each set is placed stride ints apart

```
displs = (int *)malloc(gsize*sizeof(int));
rcounts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    rcounts[i] = 100-i;    /* note change from previous example */
}
/* Create datatype for the column we are sending
*/
MPI_Type_vector(100-myrank, 1, 150, MPI_INT, &stype);
MPI_Type_commit(&stype);
/* sptr is the address of start of "myrank" column
*/
sptr = &sendarray[0][myrank];
MPI_Gatherv(sptr, 1, stype, rbuf, rcounts, displs, MPI_INT,
            root, comm);
```

Note that a different amount of data is received from each MPI process.

Example 6.8. Same as Example 6.7, but done in a different way at the sending end. We create a datatype that causes the correct striding at the sending end so that we read a column of a C array. A similar thing was done in Example 5.16, Section 5.1.14.

```
MPI_Comm comm;
int gsize, sendarray[100][150], *sptr;
int root, *rbuf, stride, myrank;
MPI_Datatype stype;
int *displs, i, *rcounts;
...

MPI_Comm_size(comm, &gsize);
MPI_Comm_rank(comm, &myrank);
rbuf = (int *)malloc(gsize*stride*sizeof(int));
displs = (int *)malloc(gsize*sizeof(int));
rcounts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    rcounts[i] = 100-i;
```

```

1  }
2  /* Create datatype for one int, with extent of entire row
3  */
4  MPI_Type_create_resized(MPI_INT, 0, 150*sizeof(int), &stype);
5  MPI_Type_commit(&stype);
6  sptr = &sendarray[0][myrank];
7  MPI_Gatherv(sptr, 100-myrank, stype, rbuf, rcounts, displs, MPI_INT,
8             root, comm);
9

```

Example 6.9. Same as Example 6.7 at sending side, but at receiving side we make the stride between received blocks vary from block to block. See Figure 6.8.

```

10
11
12
13 MPI_Comm comm;
14 int gsize, sendarray[100][150], *sptr;
15 int root, *rbuf, *stride, myrank, bufsize;
16 MPI_Datatype stype;
17 int *displs, i, *rcounts, offset;
18
19 ...
20 MPI_Comm_size(comm, &gsize);
21 MPI_Comm_rank(comm, &myrank);
22
23 stride = (int *)malloc(gsize*sizeof(int));
24 ...
25 /* stride[i] for i = 0 to gsize-1 is set somehow
26 */
27
28 /* set up displs and rcounts vectors first
29 */
30 displs = (int *)malloc(gsize*sizeof(int));
31 rcounts = (int *)malloc(gsize*sizeof(int));
32 offset = 0;
33 for (i=0; i<gsize; ++i) {
34     displs[i] = offset;
35     offset += stride[i];
36     rcounts[i] = 100-i;
37 }
38 /* the required buffer size for rbuf is now easily obtained
39 */
40 bufsize = displs[gsize-1]+rcounts[gsize-1];
41 rbuf = (int *)malloc(bufsize*sizeof(int));
42 /* Create datatype for the column we are sending
43 */
44 MPI_Type_vector(100-myrank, 1, 150, MPI_INT, &stype);
45 MPI_Type_commit(&stype);
46 sptr = &sendarray[0][myrank];
47 MPI_Gatherv(sptr, 1, stype, rbuf, rcounts, displs, MPI_INT,
48             root, comm);

```

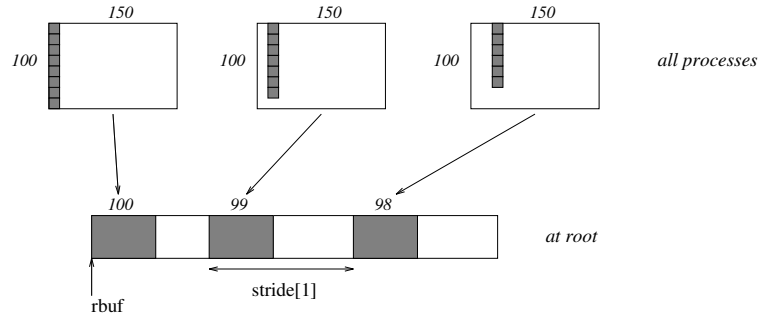


Figure 6.8: The root gathers $100-i$ ints from column i of a 100×150 C array, and each set is placed $\text{stride}[i]$ ints apart (a varying stride)

Example 6.10. MPI process i sends num ints from the i -th column of a 100×150 int array, in C. The complicating factor is that the various values of num are not known to **root**, so a separate gather must first be run to find these out. The data is placed contiguously at the receiving end.

```
MPI_Comm comm;
int gsize, sendarray[100][150], *sptr;
int root, *rbuf, myrank;
MPI_Datatype stype;
int *displs, i, *rcounts, num;

...

MPI_Comm_size(comm, &gsize);
MPI_Comm_rank(comm, &myrank);

/* First, gather nums to root
 */
rcounts = (int *)malloc(gsize*sizeof(int));
MPI_Gather(&num, 1, MPI_INT, rcounts, 1, MPI_INT, root, comm);
/* root now has correct rcounts, using these we set displs[] so that
 * data is placed contiguously (or concatenated) at the receiving end
 */
displs = (int *)malloc(gsize*sizeof(int));
displs[0] = 0;
for (i=1; i<gsize; ++i) {
    displs[i] = displs[i-1]+rcounts[i-1];
}
/* And, create receive buffer
 */
rbuf = (int *)malloc(gsize*(displs[gsize-1]+rcounts[gsize-1])
                    *sizeof(int));

/* Create datatype for one int, with extent of entire row
 */
MPI_Type_create_resized(MPI_INT, 0, 150*sizeof(int), &stype);
MPI_Type_commit(&stype);
sptr = &sendarray[0][myrank];
MPI_Gatherv(sptr, num, stype, rbuf, rcounts, displs, MPI_INT,
```

```
root, comm);
```

6.6 Scatter

MPI_SCATTER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)

IN	sendbuf	address of send buffer (choice, significant only at root)
IN	sendcount	number of elements sent to each MPI process (nonnegative integer, significant only at root)
IN	sendtype	datatype of send buffer elements (handle, significant only at root)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcount	number of elements in receive buffer (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	root	rank of sending MPI process (integer)
IN	comm	communicator (handle)

C binding

```
int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
               MPI_Comm comm)
```

```
int MPI_Scatter_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                  MPI_Datatype recvtype, int root, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: root
```

```

TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SCATTER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, ROOT,
            COMM, IERROR)

```

```

<type> SENDBUF(*), RECVBUF(*)

```

```

INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, IERROR

```

MPI_SCATTER is the inverse operation to **MPI_GATHER**.

If `comm` is an intra-communicator, the outcome is *as if* the root executed `n` send operations,

```

MPI_Send(sendbuf+i·sendcount·extent(sendtype), sendcount, sendtype, i,...),

```

and each MPI process executed a receive,

```

MPI_Recv(recvbuf, recvcount, recvtype, i,...).

```

An alternative description is that the root sends a message with **MPI_Send**(`sendbuf`, `sendcount·n`, `sendtype`, ...). This message is split into `n` equal segments, the i -th segment is sent to the i -th MPI process in the group, and each MPI process receives this message as above.

The send buffer is ignored for all nonroot MPI processes.

The type signature associated with `sendcount`, `sendtype` at the root must be equal to the type signature associated with `recvcount`, `recvtype` at all MPI processes (however, the type maps may be different). This implies that the amount of data sent must be equal to the amount of data received, pairwise between each MPI process and the root. Distinct type maps between sender and receiver are still allowed.

All arguments to the function are significant on the root, while on other MPI processes, only arguments `recvbuf`, `recvcount`, `recvtype`, `root`, and `comm` are significant. The arguments `root` and `comm` must have identical values on all MPI processes.

The specification of counts and types should not cause any location on the root to be read more than once.

Rationale. Though not needed, the last restriction is imposed so as to achieve symmetry with **MPI_GATHER**, where the corresponding restriction (a multiple-write restriction) is necessary. (*End of rationale.*)

The “in place” option for intra-communicators is specified by passing **MPI_IN_PLACE** as the value of `recvbuf` at the root. In such a case, `recvcount` and `recvtype` are ignored, and the root “sends” no data to itself. The scattered vector is still assumed to contain n segments, where n is the group size; the `root`-th segment, which root should “send to itself,” is not moved.

If `comm` is an inter-communicator, then the call involves all MPI processes in the inter-communicator, but with one group (group A) defining the root. All MPI processes in the other group (group B) pass the same value in argument `root`, which is the rank of the root in group A. The root passes the value **MPI_ROOT** in `root`. All other MPI processes in group A pass the value **MPI_PROC_NULL** in `root`. Data is scattered from the root to all MPI

processes in group B. The receive buffer arguments of the MPI processes in group B must be consistent with the send buffer argument of the root.

MPI_SCATTERV(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount, recvttype, root, comm)

IN	sendbuf	address of send buffer (choice, significant only at root)
IN	sendcounts	nonnegative integer array (of length group size) specifying the number of elements to send to each rank (significant only at root)
IN	displs	integer array (of length group size). Entry <i>i</i> specifies the displacement (relative to sendbuf) from which to take the outgoing data to MPI process <i>i</i> (significant only at root)
IN	sendtype	datatype of send buffer elements (handle, significant only at root)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcount	number of elements in receive buffer (nonnegative integer)
IN	recvttype	datatype of receive buffer elements (handle)
IN	root	rank of sending MPI process (integer)
IN	comm	communicator (handle)

C binding

```
int MPI_Scatterv(const void *sendbuf, const int sendcounts[],
                const int displs[], MPI_Datatype sendtype, void *recvbuf,
                int recvcount, MPI_Datatype recvttype, int root, MPI_Comm comm)
```

```
int MPI_Scatterv_c(const void *sendbuf, const MPI_Count sendcounts[],
                  const MPI_Aint displs[], MPI_Datatype sendtype, void *recvbuf,
                  MPI_Count recvcount, MPI_Datatype recvttype, int root,
                  MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Scatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
             recvttype, root, comm, ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcounts(*), displs(*), recvcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvttype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Scatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
             recvttype, root, comm, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcount
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
```

```

TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SCATTERV(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE, RECVBUF, REVCOUNT,
             RECVTYPE, ROOT, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), DISPLS(*), SENDTYPE, REVCOUNT, RECVTYPE, ROOT,
             COMM, IERROR

```

`MPI_SCATTERV` is the inverse operation to `MPI_GATHERV`.

`MPI_SCATTERV` extends the functionality of `MPI_SCATTER` by allowing a varying count of data to be sent to each MPI process, since `sendcounts` is now an array. It also allows more flexibility as to where the data is taken from on the root, by providing an additional argument, `displs`.

If `comm` is an intra-communicator, the outcome is as if the root executed `n` send operations,

```
MPI_Send(sendbuf+displs[i]·extent(sendtype), sendcounts[i], sendtype, i,...),
```

and each MPI process executed a receive,

```
MPI_Recv(recvbuf, recvcoun, recvtype, i,...).
```

The send buffer is ignored for all nonroot MPI processes.

The type signature implied by `sendcount[i]`, `sendtype` at the root must be equal to the type signature implied by `recvcount`, `recvtype` at MPI process `i` (however, the type maps may be different). This implies that the amount of data sent must be equal to the amount of data received, pairwise between each MPI process and the root. Distinct type maps between sender and receiver are still allowed.

All arguments to the function are significant on the root, while on other MPI processes, only arguments `recvbuf`, `recvcount`, `recvtype`, `root`, and `comm` are significant. The arguments `root` and `comm` must have identical values on all MPI processes.

The specification of counts, types, and displacements should not cause any location on the root to be read more than once.

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` as the value of `recvbuf` at the root. In such a case, `recvcount` and `recvtype` are ignored, and root “sends” no data to itself. The scattered vector is still assumed to contain n segments, where n is the group size; the `root`-th segment, which root should “send to itself,” is not moved.

If `comm` is an inter-communicator, then the call involves all MPI processes in the inter-communicator, but with one group (group A) defining the root. All MPI processes in the other group (group B) pass the same value in argument `root`, which is the rank of the root in group A. The root MPI process passes the value `MPI_ROOT` in `root`. All other MPI processes in group A pass the value `MPI_PROC_NULL` in `root`. Data is scattered from the root to all MPI processes in group B. The receive buffer arguments of the MPI processes in group B must be consistent with the send buffer argument of the root.

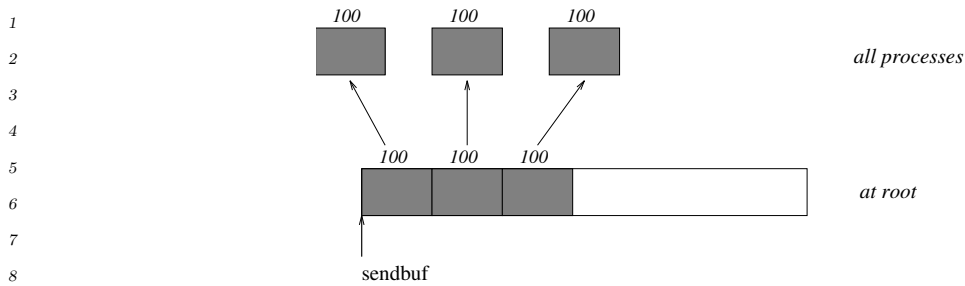


Figure 6.9: The root scatters sets of 100 ints to each MPI process in the group

6.6.1 Examples using `MPI_SCATTER` and `MPI_SCATTERV`

The examples in this section use intra-communicators.

Example 6.11. The reverse of Example 6.2. Scatter sets of 100 ints from the root to each MPI process in the group. See Figure 6.9.

```
MPI_Comm comm;
int gsize,*sendbuf;
int root, rbuf[100];
...
MPI_Comm_size(comm, &gsize);
sendbuf = (int *)malloc(gsize*100*sizeof(int));
...
MPI_Scatter(sendbuf, 100, MPI_INT, rbuf, 100, MPI_INT, root, comm);
```

Example 6.12. The reverse of Example 6.5. The root scatters sets of 100 ints to the other MPI processes, but the sets of 100 are stride ints apart in the sending buffer. Requires use of `MPI_SCATTERV`. Assume $stride \geq 100$. See Figure 6.10.

```
MPI_Comm comm;
int gsize,*sendbuf;
int root, rbuf[100], i, *displs, *counts;
...
MPI_Comm_size(comm, &gsize);
sendbuf = (int *)malloc(gsize*stride*sizeof(int));
...
displs = (int *)malloc(gsize*sizeof(int));
counts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    counts[i] = 100;
}
MPI_Scatterv(sendbuf, counts, displs, MPI_INT, rbuf, 100, MPI_INT,
             root, comm);
```

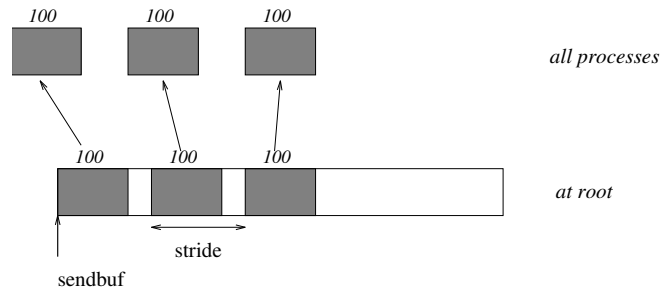



Figure 6.10: The root scatters sets of 100 ints, moving by `stride` ints from send to send in the scatter

Example 6.13. The reverse of Example 6.9. We have a varying stride between blocks at sending (root) end, at the receiving end we receive into the i -th column of a 100×150 C array. See Figure 6.11.

```
MPI_Comm comm;
int gsize, recvarray[100][150], *rptr;
int root, *sendbuf, myrank, *stride;
MPI_Datatype rtype;
int i, *displs, *scounts, offset;
...
MPI_Comm_size(comm, &gsize);
MPI_Comm_rank(comm, &myrank);

stride = (int *)malloc(gsize*sizeof(int));
...
/* stride[i] for i = 0 to gsize-1 is set somehow
 * sendbuf comes from elsewhere
 */
...
displs = (int *)malloc(gsize*sizeof(int));
scount = (int *)malloc(gsize*sizeof(int));
offset = 0;
for (i=0; i<gsize; ++i) {
    displs[i] = offset;
    offset += stride[i];
    scounts[i] = 100 - i;
}
/* Create datatype for the column we are receiving
 */
MPI_Type_vector(100-myrank, 1, 150, MPI_INT, &rtype);
MPI_Type_commit(&rtype);
rptr = &recvarray[0][myrank];
MPI_Scatterv(sendbuf, scounts, displs, MPI_INT, rptr, 1, rtype,
            root, comm);
```

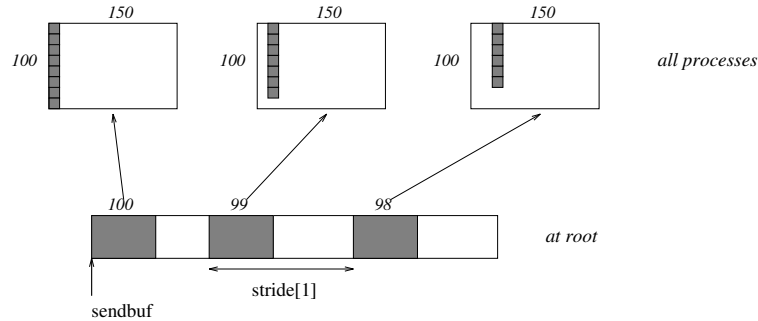


Figure 6.11: The root scatters blocks of $100-i$ ints into column i of a 100×150 C array. At the sending side, the blocks are `stride[i]` ints apart.

6.7 All-Gather

`MPI_ALLGATHER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)`

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcount	number of elements received from any MPI process (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator (handle)

C binding

```
int MPI_Allgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
void *recvbuf, int recvcount, MPI_Datatype recvtype,
MPI_Comm comm)
```

```
int MPI_Allgather_c(const void *sendbuf, MPI_Count sendcount,
MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
MPI_Datatype recvtype, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```

MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
              ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE, COMM,
              IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, IERROR

```

MPI_ALLGATHER can be thought of as **MPI_GATHER**, but where all MPI processes receive the result, instead of just the root. The block of data sent from the *j*-th MPI process is received by every MPI process and placed in the *j*-th block of the buffer *recvbuf*.

The type signature associated with *sendcount*, *sendtype*, at an MPI process must be equal to the type signature associated with *recvcount*, *recvtype* at any other MPI process.

If *comm* is an intra-communicator, the outcome of a call to **MPI_ALLGATHER(...)** is as if all MPI processes executed *n* calls to

```

MPI_Gather (sendbuf , sendcount , sendtype , recvbuf , recvcount ,
              recvtype , root , comm)

```

for *root* = 0, ..., *n*-1. The rules for correct usage of **MPI_ALLGATHER** can be found in the corresponding rules for **MPI_GATHER** (see Section 6.5).

The “in place” option for intra-communicators is specified by passing the value **MPI_IN_PLACE** to the argument *sendbuf* at all MPI processes. *sendcount* and *sendtype* are ignored. Then the input data of each MPI process is assumed to be in the area where that MPI process would receive its own contribution to the receive buffer.

If *comm* is an inter-communicator, then each MPI process of one group (group A) contributes *sendcount* data items; these data are concatenated and the result is stored at each MPI process in the other group (group B). Conversely the concatenation of the contributions of the MPI processes in group B is stored at each MPI process in group A. The send buffer arguments in group A must be consistent with the receive buffer arguments in group B, and vice versa.

Advice to users. In the inter-communicator case, the communication pattern of **MPI_ALLGATHER** need not be symmetric. The number of items sent by MPI processes in group A (as specified by the arguments *sendcount*, *sendtype* in group A and the arguments *recvcount*, *recvtype* in group B), need not equal the number of items sent by MPI processes in group B (as specified by the arguments *sendcount*, *sendtype* in group B and the arguments *recvcount*, *recvtype* in group A). In particular, one can move data in only one direction by specifying *sendcount* = 0 for the communication in the reverse direction. (*End of advice to users.*)

```

1 MPI_ALLGATHERV(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs, recvtype,
2               comm)
3
4     IN      sendbuf      starting address of send buffer (choice)
5
6     IN      sendcount    number of elements in send buffer (nonnegative
7                          integer)
8
9     IN      sendtype     datatype of send buffer elements (handle)
10
11    OUT     recvbuf      address of receive buffer (choice)
12
13    IN      recvcounts    nonnegative integer array (of length group size)
14                          containing the number of elements that are received
15                          from each MPI process
16
17    IN      displs       integer array (of length group size). Entry i specifies
18                          the displacement (relative to recvbuf) at which to
19                          place the incoming data from MPI process i
20
21    IN      recvtype     datatype of receive buffer elements (handle)
22
23    IN      comm         communicator (handle)

```

C binding

```

19 int MPI_Allgatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
20                   void *recvbuf, const int recvcounts[], const int displs[],
21                   MPI_Datatype recvtype, MPI_Comm comm)
22
23 int MPI_Allgatherv_c(const void *sendbuf, MPI_Count sendcount,
24                     MPI_Datatype sendtype, void *recvbuf,
25                     const MPI_Count recvcounts[], const MPI_Aint displs[],
26                     MPI_Datatype recvtype, MPI_Comm comm)

```

Fortran 2008 binding

```

28 MPI_Allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
29               recvtype, comm, ierror)
30
31     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
32     INTEGER, INTENT(IN) :: sendcount, recvcounts(*), displs(*)
33     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
34     TYPE(*), DIMENSION(..) :: recvbuf
35     TYPE(MPI_Comm), INTENT(IN) :: comm
36     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
39               recvtype, comm, ierror) !(_c)
40
41     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
42     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcounts(*)
43     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
44     TYPE(*), DIMENSION(..) :: recvbuf
45     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
46     TYPE(MPI_Comm), INTENT(IN) :: comm
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
               RECVTYPE, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
               IERROR

```

MPI_ALLGATHERV can be thought of as **MPI_GATHERV**, but where all processes receive the result, instead of just the root. The block of data sent from the j -th MPI process is received by every MPI process and placed in the j -th block of the buffer `recvbuf`. These blocks need not all be the same size.

The type signature associated with `sendcount`, `sendtype`, at MPI process j must be equal to the type signature associated with `recvcounts[j]`, `recvtype` at any other MPI process.

If `comm` is an intra-communicator, the outcome is as if all MPI processes executed calls to

```

MPI_Gatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
            recvtype, root, comm);

```

for `root = 0, ..., n-1`. The rules for correct usage of **MPI_ALLGATHERV** can be found in the corresponding rules for **MPI_GATHERV** (see Section 6.5).

The “in place” option for intra-communicators is specified by passing the value **MPI_IN_PLACE** to the argument `sendbuf` at all MPI processes. In such a case, `sendcount` and `sendtype` are ignored, and the input data of each MPI process is assumed to be in the area where that MPI process would receive its own contribution to the receive buffer.

If `comm` is an inter-communicator, then each MPI process of one group (group A) contributes `sendcount` data items; these data are concatenated and the result is stored at each MPI process in the other group (group B). Conversely the concatenation of the contributions of the MPI processes in group B is stored at each MPI process in group A. The send buffer arguments in group A must be consistent with the receive buffer arguments in group B, and vice versa.

6.7.1 Example using MPI_ALLGATHER

The example in this section uses intra-communicators.

Example 6.14. The all-gather version of Example 6.2. Using **MPI_ALLGATHER**, we will gather 100 ints from every MPI process in the group to every MPI process.

```

MPI_Comm comm;
int gsize, sendarray[100];
int *rbuf;
...
MPI_Comm_size(comm, &gsize);
rbuf = (int *)malloc(gsize*100*sizeof(int));
MPI_Allgather(sendarray, 100, MPI_INT, rbuf, 100, MPI_INT, comm);

```

After the call, every MPI process has the group-wide concatenation of the sets of data.

6.8 All-to-All Scatter/Gather

MPI_ALLTOALL(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each MPI process (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcount	number of elements received from any MPI process (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator (handle)

C binding

```
int MPI_Alltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                void *recvbuf, int recvcount, MPI_Datatype recvtype,
                MPI_Comm comm)
```

```
int MPI_Alltoall_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                  MPI_Datatype recvtype, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
             ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
             ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE, COMM,
             IERROR)
```

```
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, IERROR
```

`MPI_ALLTOALL` is an extension of `MPI_ALLGATHER` to the case where each MPI process sends distinct data to each of the receivers. The j -th block sent from MPI process i is received by MPI process j and is placed in the i -th block of `recvbuf`.

The type signature associated with `sendcount`, `sendtype`, at an MPI process must be equal to the type signature associated with `recvcount`, `recvtype` at any other MPI process. This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of MPI processes. As usual, however, the type maps may be different.

If `comm` is an intra-communicator, the outcome is as if each MPI process executed a send to each MPI process (itself included) with a call to,

```
MPI_Send(sendbuf+i·sendcount·extent(sendtype),sendcount,sendtype,i,...),
```

and a receive from every other MPI process with a call to,

```
MPI_Recv(recvbuf+i·recvcount·extent(recvtype),recvcount,recvtype,i,...).
```

All arguments on all MPI processes are significant. The argument `comm` must have identical values on all MPI processes.

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` to the argument `sendbuf` at *all* MPI processes. In such a case, `sendcount` and `sendtype` are ignored. The data to be sent is taken from the `recvbuf` and replaced by the received data. Data sent and received must have the same type map as specified by `recvcount` and `recvtype`.

Rationale. For large `MPI_ALLTOALL` instances, allocating both send and receive buffers may consume too much memory. The “in place” option effectively halves the application memory consumption and is useful in situations where the data to be sent will not be used by the sending MPI process after the `MPI_ALLTOALL` exchange (e.g., in parallel Fast Fourier Transforms). (*End of rationale.*)

Advice to implementors. Users may opt to use the “in place” option in order to conserve memory. Quality MPI implementations should thus strive to minimize system buffering. (*End of advice to implementors.*)

If `comm` is an inter-communicator, then the outcome is as if each MPI process in group A sends a message to each MPI process in group B, and vice versa. The j -th send buffer of MPI process i in group A should be consistent with the i -th receive buffer of MPI process j in group B, and vice versa.

Advice to users. When a complete exchange is executed in the inter-communicator case, then the number of data items sent from MPI processes in group A to MPI processes in group B need not equal the number of items sent in the reverse direction. In particular, one can have unidirectional communication by specifying `sendcount = 0` in the reverse direction. (*End of advice to users.*)

```

1 MPI_ALLTOALLV(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts, rdispls,
2               recvtype, comm)
3
4     IN      sendbuf      starting address of send buffer (choice)
5
6     IN      sendcounts   nonnegative integer array (of length group size)
7                           specifying the number of elements to send to each
8                           rank
9
10    IN      sdispls      integer array (of length group size). Entry j specifies
11                           the displacement (relative to sendbuf) from which to
12                           take the outgoing data destined for MPI process j
13
14    IN      sendtype     datatype of send buffer elements (handle)
15
16    OUT     recvbuf      address of receive buffer (choice)
17
18    IN      recvcounts   nonnegative integer array (of length group size)
19                           specifying the number of elements that can be
20                           received from each rank
21
22    IN      rdispls      integer array (of length group size). Entry i specifies
23                           the displacement (relative to recvbuf) at which to
24                           place the incoming data from MPI process i
25
26    IN      recvtype     datatype of receive buffer elements (handle)
27
28    IN      comm         communicator (handle)

```

C binding

```

29 int MPI_Alltoallv(const void *sendbuf, const int sendcounts[],
30                  const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
31                  const int recvcounts[], const int rdispls[],
32                  MPI_Datatype recvtype, MPI_Comm comm)
33
34 int MPI_Alltoallv_c(const void *sendbuf, const MPI_Count sendcounts[],
35                    const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
36                    const MPI_Count recvcounts[], const MPI_Aint rdispls[],
37                    MPI_Datatype recvtype, MPI_Comm comm)

```

Fortran 2008 binding

```

38 MPI_Alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
39               rdispls, recvtype, comm, ierror)
40
41     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
42     INTEGER, INTENT(IN) :: sendcounts(*), sdispls(*), recvcounts(*), rdispls(*)
43     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
44     TYPE(*), DIMENSION(..) :: recvbuf
45     TYPE(MPI_Comm), INTENT(IN) :: comm
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
47
48 MPI_Alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
49               rdispls, recvtype, comm, ierror) !(_c)
50
51     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
52     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcounts(*)
53     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
54     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
55     TYPE(*), DIMENSION(..) :: recvbuf

```



```

TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF, RECVCOUNTS,
              RDISPLS, RECVTYPE, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, RECVCOUNTS(*), RDISPLS(*),
              RECVTYPE, COMM, IERROR

```

MPI_ALLTOALLV adds flexibility to **MPI_ALLTOALL** in that the location of data for the send is specified by `sdispls` and the location of the placement of the data on the receive side is specified by `rdispls`.

If `comm` is an intra-communicator, then the j -th block sent from MPI process i is received by MPI process j and is placed in the i -th block of `recvbuf`. These blocks need not all have the same size.

The type signature associated with `sendcounts[j]`, `sendtype` at MPI process i must be equal to the type signature associated with `recvcounts[i]`, `recvtype` at MPI process j . This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of MPI processes. Distinct type maps between sender and receiver are still allowed.

The outcome is as if each MPI process sent a message to every other MPI process with,

```
MPI_Send(sendbuf+sdispls[i]*extent(sendtype),sendcounts[i],sendtype,i,...),
```

and received a message from every other MPI process with a call to

```
MPI_Recv(recvbuf+rdispls[i]*extent(recvtype),recvcounts[i],recvtype,i,...).
```

All arguments on all MPI processes are significant. The argument `comm` must have identical values on all MPI processes.

The “in place” option for intra-communicators is specified by passing **MPI_IN_PLACE** to the argument `sendbuf` at *all* MPI processes. In such a case, `sendcounts`, `sdispls` and `sendtype` are ignored. The data to be sent is taken from the `recvbuf` and replaced by the received data. Data sent and received must have the same type map as specified by the `recvcounts` array and the `recvtype`, and is taken from the locations of the receive buffer specified by `rdispls`.

Advice to users. Specifying the “in place” option (which must be given on all MPI processes) implies that the same amount and type of data is sent and received between any two MPI processes in the group of the communicator. Different pairs of MPI processes can exchange different amounts of data. Users must ensure that `recvcounts[j]` and `recvtype` on MPI process i match `recvcounts[i]` and `recvtype` on MPI process j . This symmetric exchange can be useful in applications where the data to be sent will not be used by the sending MPI process after the **MPI_ALLTOALLV** exchange. (*End of advice to users.*)

If `comm` is an inter-communicator, then the outcome is as if each MPI process in group A sends a message to each MPI process in group B, and vice versa. The j -th send buffer of MPI process i in group A should be consistent with the i -th receive buffer of MPI process j in group B, and vice versa.

Rationale. The definitions of `MPI_ALLTOALL` and `MPI_ALLTOALLV` give as much flexibility as one would achieve by specifying `n` independent, point-to-point communications, with two exceptions: all messages use the same datatype, and messages are scattered from (or gathered to) sequential storage. (*End of rationale.*)

Advice to implementors. Although the discussion of collective communication in terms of point-to-point operation implies that each message is transferred directly from sender to receiver, implementations may use a tree communication pattern. Messages can be forwarded by intermediate nodes where they are split (for scatter) or concatenated (for gather), if this is more efficient. (*End of advice to implementors.*)

`MPI_ALLTOALLW(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun-
ts, rdispls,
recvtypes, comm)`

IN	<code>sendbuf</code>	starting address of send buffer (choice)
IN	<code>sendcounts</code>	nonnegative integer array (of length group size) specifying the number of elements to send to each rank
IN	<code>sdispls</code>	integer array (of length group size). Entry <code>j</code> specifies the displacement in bytes (relative to <code>sendbuf</code>) from which to take the outgoing data destined for MPI process <code>j</code> (array of integers)
IN	<code>sendtypes</code>	array of datatypes (of length group size). Entry <code>j</code> specifies the type of data to send to MPI process <code>j</code> (array of handles)
OUT	<code>recvbuf</code>	address of receive buffer (choice)
IN	<code>recvcoun- ts</code>	nonnegative integer array (of length group size) specifying the number of elements that can be received from each rank
IN	<code>rdispls</code>	integer array (of length group size). Entry <code>i</code> specifies the displacement in bytes (relative to <code>recvbuf</code>) at which to place the incoming data from MPI process <code>i</code> (array of integers)
IN	<code>recvtypes</code>	array of datatypes (of length group size). Entry <code>i</code> specifies the type of data received from MPI process <code>i</code> (array of handles)
IN	<code>comm</code>	communicator (handle)

C binding

```
int MPI_Alltoallw(const void *sendbuf, const int sendcounts[],
                  const int sdispls[], const MPI_Datatype sendtypes[],
                  void *recvbuf, const int recvcoun-
                  ts[], const int rdispls[],
                  const MPI_Datatype recvtypes[], MPI_Comm comm)

int MPI_Alltoallw_c(const void *sendbuf, const MPI_Count sendcounts[],
                    const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                    void *recvbuf, const MPI_Count recvcoun-
                    ts[],
```

```
const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun-
ts, rdispls, recvtypes, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcounts(*), sdispls(*), recvcoun-
ts(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun-
ts, rdispls, recvtypes, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcoun-
ts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF, RECV-
COUNTS, RDISPLS, RECVTYPES, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPES(*), REVCOUNTS(*), RDIS-
PLS(*), RECVTYPES(*), COMM, IERROR
```

MPI_ALLTOALLW is the most general form of complete exchange. Like **MPI_TYPE_CREATE_STRUCT**, the most general type constructor, **MPI_ALLTOALLW** allows separate specification of count, displacement and datatype. In addition, to allow maximum flexibility, the displacement of blocks within the send and receive buffers is specified in bytes.

If **comm** is an intra-communicator, then the *j*-th block sent from MPI process *i* is received by MPI process *j* and is placed in the *i*-th block of **recvbuf**. These blocks need not all have the same size.

The type signature associated with **sendcounts**[*j*], **sendtypes**[*j*] at MPI process *i* must be equal to the type signature associated with **recvcoun-**ts[*i*], **recvtypes**[*i*] at MPI process *j*. This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of MPI processes. Distinct type maps between sender and receiver are still allowed.

The outcome is as if each MPI process sent a message to every other MPI process with

```
MPI_Send(sendbuf+sdispls[i],sendcounts[i],sendtypes[i] ,i,...),
```

and received a message from every other MPI process with a call to

```
MPI_Recv(recvbuf+rdispls[i],recvcoun-
ts[i],recvtypes[i] ,i,...).
```

All arguments on all MPI processes are significant. The argument `comm` must describe the same communicator on all MPI processes.

Like for `MPI_ALLTOALLV`, the “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` to the argument `sendbuf` at *all* MPI processes. In such a case, `sendcounts`, `sdispls` and `sendtypes` are ignored. The data to be sent is taken from the `recvbuf` and replaced by the received data. Data sent and received must have the same type map as specified by the `recvcounts` and `recvtypes` arrays, and is taken from the locations of the receive buffer specified by `rdispls`.

If `comm` is an inter-communicator, then the outcome is as if each MPI process in group A sends a message to each MPI process in group B, and vice versa. The *j*-th send buffer of MPI process *i* in group A should be consistent with the *i*-th receive buffer of MPI process *j* in group B, and vice versa.

Rationale. The `MPI_ALLTOALLW` function generalizes several MPI functions by carefully selecting the input arguments. For example, by making all but one MPI process have `sendcounts[i] = 0`, this achieves what one would expect from an `MPI_SCATTERW`, if such a function existed, which is equivalent to an `MPI_SCATTERV` with the blocks not all needing to have the same datatypes. (*End of rationale.*)

6.9 Global Reduction Operations

The functions in this section perform a global reduce operation (for example sum, maximum, and logical and) across all members of a group. The reduction operation can be either one of a predefined list of operations, or a user-defined operation. The global reduction functions come in several flavors: a reduce that returns the result of the reduction to one member of a group, an all-reduce that returns this result to all members of a group, and two scan (parallel prefix) operations. In addition, a reduce-scatter operation combines the functionality of a reduce and of a scatter operation.

6.9.1 Reduce

`MPI_REDUCE(sendbuf, recvbuf, count, datatype, op, root, comm)`

IN	<code>sendbuf</code>	address of send buffer (choice)
OUT	<code>recvbuf</code>	address of receive buffer (choice, significant only at root)
IN	<code>count</code>	number of elements in send buffer (nonnegative integer)
IN	<code>datatype</code>	datatype of elements of send buffer (handle)
IN	<code>op</code>	reduce operation (handle)
IN	<code>root</code>	rank of the root (integer)
IN	<code>comm</code>	communicator (handle)

C binding

```
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
```

```
int MPI_Reduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER, INTENT(IN) :: count, root
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, ROOT, COMM, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER COUNT, DATATYPE, OP, ROOT, COMM, IERROR
```

If `comm` is an intra-communicator, `MPI_REDUCE` combines the elements provided in the input buffer of each MPI process in the group, using the operation `op`, and returns the combined value in the output buffer of the MPI process with rank `root`. The input buffer is defined by the arguments `sendbuf`, `count` and `datatype`; the output buffer is defined by the arguments `recvbuf`, `count` and `datatype`; both have the same number of elements, with the same type. The routine is called by all group members using the same arguments for `count`, `datatype`, `op`, `root` and `comm`. Thus, all MPI processes provide input buffers of the same length, with elements of the same type as the output buffer at the root. Each MPI process can provide one element, or a sequence of elements, in which case the combine operation is executed element-wise on each entry of the sequence. For example, if the operation is `MPI_MAX` and the send buffer contains two elements that are floating point numbers (`count = 2` and `datatype = MPI_FLOAT`), then `recvbuf(1) = global max(sendbuf(1))` and `recvbuf(2) = global max(sendbuf(2))`.

Section 6.9.2, lists the set of predefined operations provided by MPI. That section also enumerates the datatypes to which each operation can be applied.

In addition, users may define their own operations that can be overloaded to operate on several datatypes, either basic or derived. This is further explained in Section 6.9.5.

The operation `op` is always assumed to be associative. All predefined operations are also assumed to be commutative. Users may define operations that are assumed to be associative, but not commutative. The “canonical” evaluation order of a reduction is determined by the ranks of the MPI processes in the group. However, the implementation can take advantage of associativity, or associativity and commutativity in order to change the order of evaluation.

This may change the result of the reduction for operations that are not strictly associative and commutative, such as floating point addition.

Advice to implementors. It is strongly recommended that `MPI_REDUCE` be implemented so that the same result be obtained whenever the function is applied on the same arguments, appearing in the same order. Note that this may prevent optimizations that take advantage of the physical location of ranks. (*End of advice to implementors.*)

Advice to users. Some applications may not be able to ignore the nonassociative nature of floating-point operations or may use user-defined operations (see Section 6.9.5) that require a special reduction order and cannot be treated as associative. Such applications should enforce the order of evaluation explicitly. For example, in the case of operations that require a strict left-to-right (or right-to-left) evaluation order, this could be done by gathering all operands at a single MPI process (e.g., with `MPI_GATHER`), applying the reduction operation in the desired order (e.g., with `MPI_REDUCE_LOCAL`), and if needed, broadcast or scatter the result to the other MPI processes (e.g., with `MPI_BCAST`). (*End of advice to users.*)

The `datatype` argument of `MPI_REDUCE` must be compatible with `op`. Predefined operators work only with the MPI types listed in Section 6.9.2 and Section 6.9.4. Furthermore, the `datatype` and `op` given for predefined operators must be the same on all MPI processes.

Note that it is possible for users to supply different user-defined operations to `MPI_REDUCE` in each MPI process. MPI does not define which operations are used on which operands in this case. User-defined operators may operate on general, derived datatypes. In this case, each argument that the reduce operation is applied to is one element described by such a datatype, which may contain several basic values. This is further explained in Section 6.9.5.

Advice to users. Users should make no assumptions about how `MPI_REDUCE` is implemented. It is safest to ensure that the same function is passed to `MPI_REDUCE` by each MPI process. (*End of advice to users.*)

Overlapping datatypes are permitted in “send” buffers. Overlapping datatypes in “receive” buffers are erroneous and may give unpredictable results.

The “in place” option for intra-communicators is specified by passing the value `MPI_IN_PLACE` to the argument `sendbuf` at the root. In such a case, the input data is taken at the root from the receive buffer, where it will be replaced by the output data.

If `comm` is an inter-communicator, then the call involves all MPI processes in the inter-communicator, but with one group (group A) defining the root. All MPI processes in the other group (group B) pass the same value in argument `root`, which is the rank of the root in group A. The root passes the value `MPI_ROOT` in `root`. All other MPI processes in group A pass the value `MPI_PROC_NULL` in `root`. Only send buffer arguments are significant in group B and only receive buffer arguments are significant at the root.

6.9.2 Predefined Reduction Operations

The following predefined operations are supplied for `MPI_REDUCE` and related functions `MPI_ALLREDUCE`, `MPI_REDUCE_SCATTER_BLOCK`, `MPI_REDUCE_SCATTER`,

`MPI_SCAN`, `MPI_EXSCAN`, all nonblocking variants of those (see Section 6.12), and `MPI_REDUCE_LOCAL`. These operations are invoked by placing the following in `op`.

Name	Meaning
<code>MPI_MAX</code>	maximum
<code>MPI_MIN</code>	minimum
<code>MPI_SUM</code>	sum
<code>MPI_PROD</code>	product
<code>MPI_LAND</code>	logical and
<code>MPI_BAND</code>	bit-wise and
<code>MPI_LOR</code>	logical or
<code>MPI_BOR</code>	bit-wise or
<code>MPI_LXOR</code>	logical exclusive or (xor)
<code>MPI_BXOR</code>	bit-wise exclusive or (xor)
<code>MPI_MAXLOC</code>	max value and location
<code>MPI_MINLOC</code>	min value and location

The two operations `MPI_MINLOC` and `MPI_MAXLOC` are discussed separately in Section 6.9.4. For the other predefined operations, we enumerate below the allowed combinations of `op` and `datatype` arguments. First, define groups of MPI basic datatypes in the following way.

C integer:

`MPI_INT`, `MPI_LONG`, `MPI_SHORT`,
`MPI_UNSIGNED_SHORT`, `MPI_UNSIGNED`,
`MPI_UNSIGNED_LONG`,
`MPI_LONG_LONG_INT`,
`MPI_LONG_LONG` (as synonym),
`MPI_UNSIGNED_LONG_LONG`,
`MPI_SIGNED_CHAR`,
`MPI_UNSIGNED_CHAR`,
`MPI_INT8_T`, `MPI_INT16_T`,
`MPI_INT32_T`, `MPI_INT64_T`,
`MPI_UINT8_T`, `MPI_UINT16_T`,
`MPI_UINT32_T`, and `MPI_UINT64_T`
`MPI_INTEGER`

Fortran integer:

and handles returned from
`MPI_TYPE_CREATE_F90_INTEGER`
and, if available, `MPI_INTEGER1`,
`MPI_INTEGER2`, `MPI_INTEGER4`,
`MPI_INTEGER8`, and `MPI_INTEGER16`

Floating point:

`MPI_FLOAT`, `MPI_DOUBLE`, `MPI_REAL`,
`MPI_DOUBLE_PRECISION`,
`MPI_LONG_DOUBLE`,

and handles returned from

`MPI_TYPE_CREATE_F90_REAL`

and, if available, `MPI_REAL2`,

`MPI_REAL4`, `MPI_REAL8`, and `MPI_REAL16`

Logical:

`MPI_LOGICAL`, `MPI_C_BOOL`,
`MPI_CXX_BOOL`,

Complex:

and, if available, `MPI_LOGICAL1`,
`MPI_LOGICAL2`, `MPI_LOGICAL4`,
`MPI_LOGICAL8`, and `MPI_LOGICAL16`,
`MPI_COMPLEX`, `MPI_C_COMPLEX`,
`MPI_C_FLOAT_COMPLEX` (as synonym),
`MPI_C_DOUBLE_COMPLEX`,
`MPI_C_LONG_DOUBLE_COMPLEX`,
`MPI_CXX_FLOAT_COMPLEX`,
`MPI_CXX_DOUBLE_COMPLEX`,
`MPI_CXX_LONG_DOUBLE_COMPLEX`,
and handles returned from
`MPI_TYPE_CREATE_F90_COMPLEX`
and, if available, `MPI_DOUBLE_COMPLEX`,
`MPI_COMPLEX4`, `MPI_COMPLEX8`,
`MPI_COMPLEX16`, and `MPI_COMPLEX32`
Byte:
`MPI_BYTE`
Multi-language types:
`MPI_AINT`, `MPI_OFFSET`, and `MPI_COUNT`

Now, the valid datatypes for each operation are specified below.

Op	Allowed Types
<code>MPI_MAX</code> , <code>MPI_MIN</code>	C integer, Fortran integer, Floating point, Multi-language types
<code>MPI_SUM</code> , <code>MPI_PROD</code>	C integer, Fortran integer, Floating point, Complex, Multi-language types
<code>MPI_LAND</code> , <code>MPI_LOR</code> , <code>MPI_LXOR</code>	C integer, Logical
<code>MPI_BAND</code> , <code>MPI BOR</code> , <code>MPI_BXOR</code>	C integer, Fortran integer, Byte, Multi-language types

These operations together with all listed datatypes are valid in all supported programming languages, see also Reduce Operations in Section 21.3.6.

The following examples use intra-communicators.

Example 6.15. A routine that computes the dot product of two vectors that are distributed across a group of MPI processes and returns the answer at node zero.

```

SUBROUTINE PAR_BLAS1(m, a, b, c, comm)
USE MPI
REAL a(m), b(m)      ! local slice of array
REAL c                ! result (at node zero)
REAL sum
INTEGER m, comm, i, ierr

! local sum
sum = 0.0
DO i = 1, m
    sum = sum + a(i)*b(i)
END DO

! global sum
CALL MPI_REDUCE(sum, c, 1, MPI_REAL, MPI_SUM, 0, comm, ierr)

```



```

RETURN
END

```

Example 6.16. A routine that computes the product of a vector and an array that are distributed across a group of MPI processes and returns the answer at node zero.

```

SUBROUTINE PAR_BLAS2(m, n, a, b, c, comm)
USE MPI
REAL a(m), b(m,n)      ! local slice of array
REAL c(n)               ! result
REAL sum(n)
INTEGER m, n, comm, i, j, ierr

! local sum
DO j=1,n
  sum(j) = 0.0
  DO i=1,m
    sum(j) = sum(j) + a(i)*b(i,j)
  END DO
END DO

! global sum
CALL MPI_REDUCE(sum, c, n, MPI_REAL, MPI_SUM, 0, comm, ierr)

! return result at node zero (and garbage at the other nodes)
RETURN
END

```

6.9.3 Signed Characters and Reductions

The types `MPI_SIGNED_CHAR` and `MPI_UNSIGNED_CHAR` can be used in reduction operations. `MPI_CHAR`, `MPI_WCHAR`, and `MPI_CHARACTER` (which represent printable characters) cannot be used in reduction operations. In a heterogeneous environment, `MPI_CHAR`, `MPI_WCHAR`, and `MPI_CHARACTER` will be translated so as to preserve the printable character, whereas `MPI_SIGNED_CHAR` and `MPI_UNSIGNED_CHAR` will be translated so as to preserve the integer value.

Advice to users. The types `MPI_CHAR`, `MPI_WCHAR`, and `MPI_CHARACTER` are intended for characters, and so will be translated to preserve the printable representation, rather than the integer value, if sent between machines with different character codes. The types `MPI_SIGNED_CHAR` and `MPI_UNSIGNED_CHAR` should be used in C if the integer value should be preserved. (*End of advice to users.*)

6.9.4 MINLOC and MAXLOC

The operator `MPI_MINLOC` is used to compute a global minimum and also an index attached to the minimum value. `MPI_MAXLOC` similarly computes a global maximum and index. One application of these is to compute a global minimum (maximum) and the rank of the MPI process containing this value.

The operation that defines `MPI_MAXLOC` is:

$$\begin{pmatrix} u \\ i \end{pmatrix} \circ \begin{pmatrix} v \\ j \end{pmatrix} = \begin{pmatrix} w \\ k \end{pmatrix}$$

where

$$w = \max(u, v)$$

and

$$k = \begin{cases} i & \text{if } u > v \\ \min(i, j) & \text{if } u = v \\ j & \text{if } u < v \end{cases}$$

`MPI_MINLOC` is defined similarly:

$$\begin{pmatrix} u \\ i \end{pmatrix} \circ \begin{pmatrix} v \\ j \end{pmatrix} = \begin{pmatrix} w \\ k \end{pmatrix}$$

where

$$w = \min(u, v)$$

and

$$k = \begin{cases} i & \text{if } u < v \\ \min(i, j) & \text{if } u = v \\ j & \text{if } u > v \end{cases}$$

Both operations are associative and commutative. Note that if `MPI_MAXLOC` is applied to reduce a sequence of pairs $(u_0, 0), (u_1, 1), \dots, (u_{n-1}, n-1)$, then the value returned is (u, r) , where $u = \max_i u_i$ and r is the index of the first global maximum in the sequence. Thus, if each MPI process supplies a value and its rank within the group, then a reduce operation with `op = MPI_MAXLOC` will return the maximum value and the rank of the first MPI process with that value. Similarly, `MPI_MINLOC` can be used to return a minimum and its index. More generally, `MPI_MINLOC` computes a *lexicographic minimum*, where elements are ordered according to the first component of each pair, and ties are resolved according to the second component.

The reduce operation is defined to operate on arguments that consist of a pair: value and index. For both Fortran and C, types are provided to describe the pair. The potentially mixed-type nature of such arguments is a problem in older versions of Fortran. The problem is circumvented there by having the MPI-provided type consist of a pair of the same type as value, and coercing the index to this type also. In C, the MPI-provided pair type has distinct types and the index is an integer type. For named predefined pair types in C the index is of type `int`. For unnamed predefined pair types, other integer types are allowed as index instead. To use pair types with distinct value and index in Fortran, these types need to be defined using `BIND(C)` and be equivalent to the corresponding C struct.

In order to use `MPI_MINLOC` and `MPI_MAXLOC` in a reduce operation, one must provide a `datatype` argument that represents a pair (value and index). MPI provides nine such named predefined datatypes as well as the function `MPI_TYPE_GET_VALUE_INDEX` to query named and unnamed predefined types using value type and index type. The operations `MPI_MAXLOC` and `MPI_MINLOC` can be used with each of the following named datatypes.

Fortran:	
Name	Description
MPI_2REAL	pair of REALs
MPI_2DOUBLE_PRECISION	pair of DOUBLE PRECISION variables
MPI_2INTEGER	pair of INTEGERS

C:	
Name	Description
MPI_FLOAT_INT	float and int
MPI_DOUBLE_INT	double and int
MPI_LONG_INT	long and int
MPI_2INT	pair of int
MPI_SHORT_INT	short and int
MPI_LONG_DOUBLE_INT	long double and int

The datatype `MPI_2REAL` is *as if* defined by the following (see Section 5.1).

```
call MPI_Type_contiguous(2, MPI_REAL, MPI_2REAL, ierror)
```

Similar statements apply for `MPI_2INTEGER`, `MPI_2DOUBLE_PRECISION`, and `MPI_2INT`.

The datatype `MPI_SHORT_INT` is *as if* defined by the following sequence of instructions.

```
struct mystruct {
    short val;
    int rank;
};
type[0] = MPI_SHORT;
type[1] = MPI_INT;
disp[0] = 0;
disp[1] = offsetof(struct mystruct, rank);
block[0] = 1;
block[1] = 1;
MPI_Type_create_struct(2, block, disp, type, &MPI_SHORT_INT);
MPI_Type_commit(&MPI_SHORT_INT);
```

Similar statements apply for `MPI_FLOAT_INT`, `MPI_LONG_INT` and `MPI_DOUBLE_INT`.

```
MPI_TYPE_GET_VALUE_INDEX(value_type, index_type, pair_type)
```

IN	value_type	datatype of the value in pair (handle)
IN	index_type	datatype of the index in pair (handle)
OUT	pair_type	datatype of the value-index pair (handle)

C binding

```
int MPI_Type_get_value_index(MPI_Datatype value_type, MPI_Datatype index_type,
    MPI_Datatype *pair_type)
```

Fortran 2008 binding

```
MPI_Type_get_value_index(value_type, index_type, pair_type, ierror)
    TYPE(MPI_Datatype), INTENT(IN) :: value_type, index_type
```

```

1      TYPE(MPI_Datatype), INTENT(OUT) :: pair_type
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_TYPE_GET_VALUE_INDEX(VALUE_TYPE, INDEX_TYPE, PAIR_TYPE, IERROR)
5      INTEGER VALUE_TYPE, INDEX_TYPE, PAIR_TYPE, IERROR

```

`MPI_TYPE_GET_VALUE_INDEX` returns a handle to a predefined datatype suitable for the use with `MPI_MINLOC` and `MPI_MAXLOC` if such a predefined type exists. If the provided combination of `value_type` and `index_type` does not match a predefined pair datatype (named or unnamed), the function will set `pair_type` to `MPI_DATATYPE_NULL` and return `MPI_SUCCESS`. The returned type is not a duplicate. This type cannot be freed. Types supported by the underlying compiler for which the operators `MPI_MIN` and `MPI_MAX` are defined in Section 6.9.2 are acceptable value types. Integer types supported by the underlying compiler are acceptable index types.

Advice to users. Note that a named type handle returned by `MPI_TYPE_GET_VALUE_INDEX` will yield the combiner value `MPI_COMBINER_NAMED` when queried with `MPI_TYPE_GET_ENVELOPE` to ensure backward compatibility to existing behavior, whereas all unnamed type handles returned by `MPI_TYPE_GET_VALUE_INDEX` will yield the combiner value `MPI_COMBINER_VALUE_INDEX` when queried with `MPI_TYPE_GET_ENVELOPE`. There is no observable difference between the named constant value used via its symbol name or via `MPI_TYPE_GET_VALUE_INDEX`. Code evaluating the combiner of type handles returned from `MPI_TYPE_GET_VALUE_INDEX` must therefore handle both `MPI_COMBINER_NAMED` and `MPI_COMBINER_VALUE_INDEX`. (*End of advice to users.*)

Example 6.17. An unnamed predefined value-index type is retrieved for use with the corresponding C struct. If the requested value-index pair does not exist as a predefined type `MPI_DATATYPE_NULL` is returned.

```

31      struct mystruct {
32          double val;
33          uint64_t index;
34      };
35
36      MPI_Datatype dtype;
37      MPI_Type_get_value_index(MPI_DOUBLE, MPI_UINT64_T, &dtype);
38
39      if (dtype == MPI_DATATYPE_NULL) {
40          // Handling for unsupported value-index type
41      }

```

Advice to users. Implementations may apply certain optimizations to operations on compound types with equally sized value and index types. Such optimizations may not be applicable to operations on compound types where value and index type are of different size. (*End of advice to users.*)

The following examples use intra-communicators.

Example 6.18. Each MPI process has an array of 30 doubles, in C. For each of the 30 locations, compute the value and rank of the MPI process containing the largest value.

```

...
/* each MPI process has an array of 30 double: ain[30]
*/
double ain[30], aout[30];
int ind[30];
struct {
    double val;
    int rank;
} in[30], out[30];
int i, myrank, root;

MPI_Comm_rank(comm, &myrank);
for (i=0; i<30; ++i) {
    in[i].val = ain[i];
    in[i].rank = myrank;
}
MPI_Reduce(in, out, 30, MPI_DOUBLE_INT, MPI_MAXLOC, root, comm);
/* At this point, the answer resides on root MPI process
*/
if (myrank == root) {
    /* read ranks out
    */
    for (i=0; i<30; ++i) {
        aout[i] = out[i].val;
        ind[i] = out[i].rank;
    }
}

```

Example 6.19. Same example, in Fortran.

```

...
! each process has an array of 30 double: ain(30)

DOUBLE PRECISION ain(30), aout(30)
INTEGER ind(30)
DOUBLE PRECISION in(2,30), out(2,30)
INTEGER i, myrank, root, ierr

CALL MPI_COMM_RANK(comm, myrank, ierr)
DO i=1,30
    in(1,i) = ain(i)
    in(2,i) = myrank      ! myrank is coerced to a double
END DO

CALL MPI_REDUCE(in, out, 30, MPI_2DOUBLE_PRECISION, MPI_MAXLOC, root,&
               comm, ierr)
! At this point, the answer resides on root MPI process

IF (myrank .EQ. root) THEN

```

```

1      ! read ranks out
2      DO i=1,30
3          aout(i) = out(1,i)
4          ind(i) = out(2,i) ! rank is coerced back to an integer
5      END DO
6  END IF

```

Example 6.20. Each MPI process has a nonempty array of values. Find the minimum global value, the rank of the MPI process that holds it and its index on this MPI process.

```

11  #define LEN 1000
12
13  float val[LEN];          /* local array of values */
14  int count;               /* local number of values */
15  int myrank, minrank, minindex;
16  float minval;
17
18  struct {
19      float value;
20      int index;
21  } in, out;
22
23  /* local minloc */
24  in.value = val[0];
25  in.index = 0;
26  for (i=1; i < count; i++) {
27      if (in.value > val[i]) {
28          in.value = val[i];
29          in.index = i;
30      }
31  }
32  /* global minloc */
33  MPI_Comm_rank(comm, &myrank);
34  in.index = myrank*LEN + in.index;
35  MPI_Reduce(&in, &out, 1, MPI_FLOAT_INT, MPI_MINLOC, root, comm);
36
37  /* At this point, the answer resides on the root */
38  if (myrank == root) {
39      /* read answer out */
40      minval = out.value;
41      minrank = out.index / LEN;
42      minindex = out.index % LEN;
43  }

```

Rationale. The definition of `MPI_MINLOC` and `MPI_MAXLOC` given here has the advantage that it does not require any special-case handling of these two operations: they are handled like any other reduce operation. By assigning a value other than `myrank` to the `in.index` field, a programmer can provide a different definition of `MPI_MAXLOC` and `MPI_MINLOC`, if so desired. The disadvantage is that values and indices have to be first interleaved, and that indices and values have to be coerced to the same type, in Fortran. (*End of rationale.*)

6.9.5 User-Defined Reduction Operations

MPI_OP_CREATE(user_fn, commute, op)

IN	user_fn	user defined function (function)
IN	commute	true if commutative; false otherwise.
OUT	op	operation (handle)

C binding

```
int MPI_Op_create(MPI_User_function *user_fn, int commute, MPI_Op *op)
```

```
int MPI_Op_create_c(MPI_User_function_c *user_fn, int commute, MPI_Op *op)
```

Fortran 2008 binding

```
MPI_Op_create(user_fn, commute, op, ierror)
```

```
  PROCEDURE(MPI_User_function) :: user_fn
```

```
  LOGICAL, INTENT(IN) :: commute
```

```
  TYPE(MPI_Op), INTENT(OUT) :: op
```

```
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Op_create_c(user_fn, commute, op, ierror) !(_c)
```

```
  PROCEDURE(MPI_User_function_c) :: user_fn
```

```
  LOGICAL, INTENT(IN) :: commute
```

```
  TYPE(MPI_Op), INTENT(OUT) :: op
```

```
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_OP_CREATE(USER_FN, COMMUTE, OP, IERROR)
```

```
  EXTERNAL USER_FN
```

```
  LOGICAL COMMUTE
```

```
  INTEGER OP, IERROR
```

MPI_OP_CREATE binds a user-defined reduction operation to an **op** handle that can subsequently be used in **MPI_REDUCE**, **MPI_ALLREDUCE**, **MPI_REDUCE_SCATTER**, **MPI_REDUCE_SCATTER_BLOCK**, **MPI_SCAN**, **MPI_EXSCAN**, all nonblocking and persistent variants of those (see Section 6.12 and Section 6.13), and **MPI_REDUCE_LOCAL**. The user-defined operation is assumed to be associative. If **commute** = **true**, then the operation should be both commutative and associative. If **commute** = **false**, then the order of operands is fixed and is defined to be in ascending, process rank order, beginning with MPI process with rank 0 in the communicator **comm**. The order of evaluation can be changed, talking advantage of the associativity of the operation. If **commute** = **true** then the order of evaluation can be changed, taking advantage of commutativity and associativity.

In Fortran when using **USE mpi_f08**, the large count variant shall be called explicitly as **MPI_Op_create_c** (i.e., with suffix “_c”) because interface polymorphism cannot be used to differentiate between the two different user callback prototypes despite their different type signatures.

The argument **user_fn** is the user-defined function, which must have the following four arguments: **invec**, **inoutvec**, **len**, and **datatype**.

MPI_USER_FUNCTION also supports large count types in separate additional MPI

callback function prototype declarations in C (suffixed with the “_c”) and in Fortran when using `USE mpi_f08`.

The ISO C prototypes for the functions are the following.

```
typedef void MPI_User_function(void *invec, void *inoutvec, int *len,
                               MPI_Datatype *datatype);

typedef void MPI_User_function_c(void *invec, void *inoutvec, MPI_Count *len,
                                 MPI_Datatype *datatype);
```

The Fortran declarations of the user-defined function `user_fn` appear below.

```
ABSTRACT INTERFACE
  SUBROUTINE MPI_User_function(invec, inoutvec, len, datatype)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    TYPE(C_PTR), VALUE :: invec, inoutvec
    INTEGER :: len
    TYPE(MPI_Datatype) :: datatype
  END SUBROUTINE
ABSTRACT INTERFACE
  SUBROUTINE MPI_User_function_c(invec, inoutvec, len, datatype) !(_c)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    TYPE(C_PTR), VALUE :: invec, inoutvec
    INTEGER(KIND=MPI_COUNT_KIND) :: len
    TYPE(MPI_Datatype) :: datatype
  END SUBROUTINE
SUBROUTINE USER_FUNCTION(INVEC, INOUTVEC, LEN, DATATYPE)
  <type> INVEC(LEN), INOUTVEC(LEN)
  INTEGER LEN, DATATYPE
```

The `datatype` argument is a handle to the datatype that was passed into the call to `MPI_REDUCE`. The user reduce function should be written such that the following holds: Let $u[0], \dots, u[\text{len}-1]$ be the `len` elements in the communication buffer described by the arguments `invec`, `len` and `datatype` when the function is invoked; let $v[0], \dots, v[\text{len}-1]$ be `len` elements in the communication buffer described by the arguments `inoutvec`, `len` and `datatype` when the function is invoked; let $w[0], \dots, w[\text{len}-1]$ be `len` elements in the communication buffer described by the arguments `inoutvec`, `len` and `datatype` when the function returns; then $w[i] = u[i] \circ v[i]$, for $i=0, \dots, \text{len}-1$, where $\circ$ is the reduce operation that the function computes.

Informally, we can think of `invec` and `inoutvec` as arrays of `len` elements that `user_fn` is combining. The result of the reduction over-writes values in `inoutvec`, hence the name. Each invocation of the function results in the pointwise evaluation of the reduce operator on `len` elements: i.e., the function returns in `inoutvec[i]` the value `invec[i]  $\circ$  inoutvec[i]`, for $i=0, \dots, \text{count}-1$, where $\circ$ is the combining operation computed by the function.

Rationale. The `len` argument allows `MPI_REDUCE` to avoid calling the function for each element in the input buffer. Rather, the system can choose to apply the function to chunks of input. In C, it is passed in as a reference for reasons of compatibility with Fortran.

By internally comparing the value of the `datatype` argument to known, global handles, it is possible to overload the use of a single user-defined function for several, different datatypes. (*End of rationale.*)

When calling any reduction or prefix scan MPI procedure with a user-defined MPI operator, the type of the `count` parameter in the call to the reduction or prefix scan MPI procedure does not need to be identical to the type of the `len` parameter in the user function associated with the user-defined MPI operator. If the `count` parameter has a type of `int` in C or `INTEGER` in Fortran and the `len` parameter has a type of `MPI_COUNT`, then MPI will perform the appropriate widening type conversion of the `len` parameter. If the `count` parameter has a type of `MPI_COUNT` and the `len` parameter has a type of `int` in C or `INTEGER` in Fortran, then MPI will perform the appropriate narrowing type conversion of the `len` parameter. If this narrowing conversion would result in truncation of the `len` value, then MPI will call the user function multiple times with a sequence of values for `len` that sum to the value of `count`.

General datatypes may be passed to the user function. However, use of datatypes that are not contiguous is likely to lead to inefficiencies.

No MPI communication function may be called inside the user function. `MPI_ABORT` may be called inside the function in case of an error.

Advice to users. Suppose one defines a library of user-defined reduce functions that are overloaded: the `datatype` argument is used to select the right execution path at each invocation, according to the types of the operands. The user-defined reduce function cannot “decode” the `datatype` argument that it is passed, and cannot identify, by itself, the correspondence between the datatype handles and the datatype they represent. This correspondence was established when the datatypes were created. Before the library is used, a library initialization preamble must be executed. This preamble code will define the datatypes that are used by the library, and store handles to these datatypes in global, static variables that are shared by the user code and the library code.

The Fortran version of `MPI_REDUCE` will invoke a user-defined reduce function using the Fortran calling conventions and will pass a Fortran-type datatype argument; the C version will use C calling convention and the C representation of a datatype handle. Users who plan to mix languages should define their reduction functions accordingly. (*End of advice to users.*)

Advice to implementors. We outline below a naive and inefficient implementation of `MPI_REDUCE` not supporting the “in place” option and only valid for intra-communicators.

```
MPI_Comm_size(comm, &groupsize);
MPI_Comm_rank(comm, &rank);
if (rank > 0) {
    MPI_Recv(tempbuf, count, datatype, rank-1,...);
    User_reduce(tempbuf, sendbuf, count, datatype);
}
if (rank < groupsize-1) {
    MPI_Send(sendbuf, count, datatype, rank+1, ...);
}
/* answer now resides in MPI process groupsize-1 ...
 * now send to root
 */
if (rank == root) {
    MPI_Irecv(recvbuf, count, datatype, groupsize-1,..., &req);
```

```

1  }
2  if (rank == groupsize-1) {
3      MPI_Send(sendbuf, count, datatype, root, ...);
4  }
5  if (rank == root) {
6      MPI_Wait(&req, &status);
7  }

```

The reduction computation proceeds, sequentially, from MPI process with rank 0 to MPI process with rank groupsize-1. This order is chosen so as to respect the order of a possibly noncommutative operator defined by the function `User_reduce()`. A more efficient implementation is achieved by taking advantage of associativity and using a logarithmic tree reduction. Commutativity can be used to advantage, for those cases in which the `commute` argument to `MPI_OP_CREATE` is true. Also, the amount of temporary buffer space required can be reduced, and communication can be pipelined with computation, by transferring and reducing the elements in chunks of size `len < count`.

The predefined reduce operations can be implemented as a library of user-defined operations. However, better performance might be achieved if `MPI_REDUCE` handles these functions as a special case. (*End of advice to implementors.*)

MPI_OP_FREE(op)

INOUT op operation (handle)

C binding

int MPI_Op_free(MPI_Op *op)

Fortran 2008 binding

MPI_Op_free(op, ierror)

TYPE(MPI_Op), INTENT(INOUT) :: op

INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

MPI_OP_FREE(OP, IERROR)

INTEGER OP, IERROR

Marks a user-defined reduction operation for deallocation and sets `op` to `MPI_OP_NULL`.

Example of User-Defined Reduce

The example in this section uses an intra-communicator.

Example 6.21. Compute the product of an array of complex numbers, in C.

```

41 typedef struct {
42     double real, imag;
43 } Complex;
44
45 /* the user-defined function
46 */
47
48

```

```

1 void myProd(void *inP, void *inoutP, int *len, MPI_Datatype *dptr)
2 {
3     int i;
4     Complex c;
5     Complex *in = (Complex *)inP, *inout = (Complex *)inoutP;
6
7     for (i=0; i< *len; ++i) {
8         c.real = inout->real*in->real -
9                 inout->imag*in->imag;
10        c.imag = inout->real*in->imag +
11                inout->imag*in->real;
12        *inout = c;
13        inout++;
14    }
15 }
16
17 /* and, to call it...
18 */
19 ...
20
21 /* each MPI process has an array of 100 Complexes
22 */
23 Complex a[100], answer[100];
24 MPI_Op myOp;
25 MPI_Datatype ctype;
26
27 /* explain to MPI how type Complex is defined
28 */
29 MPI_Type_contiguous(2, MPI_DOUBLE, &ctype);
30 MPI_Type_commit(&ctype);
31 /* create the complex-product user-op
32 */
33 MPI_Op_create(myProd, 1, &myOp);
34
35 MPI_Reduce(a, answer, 100, ctype, myOp, root, comm);
36
37 /* At this point, the answer, which consists of 100 Complexes,
38 * resides on root MPI process
39 */

```

Example 6.22. How to use the `mpi_f08` interface of the Fortran `MPI_User_function`.

```

39 subroutine my_user_function(invec, inoutvec, len, dtype) bind(c)
40 use, intrinsic :: iso_c_binding, only : c_ptr, c_f_pointer
41 use mpi_f08
42 type(c_ptr), value :: invec, inoutvec
43 integer :: len
44 type(MPI_Datatype) :: dtype
45 real, pointer :: invec_r(:), inoutvec_r(:)
46 if (dtype == MPI_REAL) then
47     call c_f_pointer(invec, invec_r, (/ len /))
48     call c_f_pointer(inoutvec, inoutvec_r, (/ len /))

```

```

1      inoutvec_r = invec_r + inoutvec_r
2      end if
3  end subroutine

```

6.9.6 All-Reduce

MPI includes a variant of the reduce operations where the result is returned to all MPI processes in a group. MPI requires that all MPI processes from the same group participating in these operations receive identical results.

MPI_ALLREDUCE(sendbuf, recvbuf, count, datatype, op, comm)

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of elements of send buffer (handle)
IN	op	operation (handle)
IN	comm	communicator (handle)

C binding

```

int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)

```

```

int MPI_Allreduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                    MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)

```

Fortran 2008 binding

```

MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm, ierror)

```

```

    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf

```

```

    TYPE(*), DIMENSION(..) :: recvbuf

```

```

    INTEGER, INTENT(IN) :: count

```

```

    TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

    TYPE(MPI_Op), INTENT(IN) :: op

```

```

    TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm, ierror) !(_c)

```

```

    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf

```

```

    TYPE(*), DIMENSION(..) :: recvbuf

```

```

    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count

```

```

    TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

    TYPE(MPI_Op), INTENT(IN) :: op

```

```

    TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```
MPI_ALLREDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, IERROR)
```

```
<type> SENDBUF(*), RECVBUF(*)
```

```
INTEGER COUNT, DATATYPE, OP, COMM, IERROR
```

If `comm` is an intra-communicator, `MPI_ALLREDUCE` behaves the same as `MPI_REDUCE` except that the result appears in the receive buffer of all the group members.

Advice to implementors. The all-reduce operations can be implemented as a reduce, followed by a broadcast. However, a direct implementation can lead to better performance. (*End of advice to implementors.*)

The “in place” option for intra-communicators is specified by passing the value `MPI_IN_PLACE` to the argument `sendbuf` at all processes. In this case, the input data is taken at each MPI process from the receive buffer, where it will be replaced by the output data.

If `comm` is an inter-communicator, then the result of the reduction of the data provided by MPI processes in group A is stored at each MPI process in group B, and vice versa. Both groups should provide `count` and `datatype` arguments that specify the same type signature.

The following example uses an intra-communicator.

Example 6.23. A routine that computes the product of a vector and an array that are distributed across a group of MPI processes and returns the answer at all nodes (see also Example 6.16).

```
SUBROUTINE PAR_BLAS2(m, n, a, b, c, comm)
USE MPI
REAL a(m), b(m,n)      ! local slice of array
REAL c(n)              ! result
REAL sum(n)
INTEGER m, n, comm, i, j, ierr

! local sum
DO j=1,n
  sum(j) = 0.0
  DO i=1,m
    sum(j) = sum(j) + a(i)*b(i,j)
  END DO
END DO

! global sum
CALL MPI_ALLREDUCE(sum, c, n, MPI_REAL, MPI_SUM, comm, ierr)

! return result at all nodes
RETURN
END
```

6.9.7 MPI Process-Local Reduction

The functions in this section are of importance to library implementors who may want to implement special reduction patterns that are otherwise not easily covered by the standard MPI operations.

The following function applies a reduction operator to local arguments.

MPI_REDUCE_LOCAL(inbuf, inoutbuf, count, datatype, op)

IN	inbuf	input buffer (choice)
INOUT	inoutbuf	combined input and output buffer (choice)
IN	count	number of elements in inbuf and inoutbuf buffers (nonnegative integer)
IN	datatype	datatype of elements of inbuf and inoutbuf buffers (handle)
IN	op	operation (handle)

C binding

```
int MPI_Reduce_local(const void *inbuf, void *inoutbuf, int count,
                    MPI_Datatype datatype, MPI_Op op)
```

```
int MPI_Reduce_local_c(const void *inbuf, void *inoutbuf, MPI_Count count,
                      MPI_Datatype datatype, MPI_Op op)
```

Fortran 2008 binding

```
MPI_Reduce_local(inbuf, inoutbuf, count, datatype, op, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
  TYPE(*), DIMENSION(..) :: inoutbuf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_local(inbuf, inoutbuf, count, datatype, op, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
  TYPE(*), DIMENSION(..) :: inoutbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REDUCE_LOCAL(INBUF, INOUTBUF, COUNT, DATATYPE, OP, IERROR)
  <type> INBUF(*), INOUTBUF(*)
  INTEGER COUNT, DATATYPE, OP, IERROR
```

The function applies the operation given by **op** element-wise to the elements of **inbuf** and **inoutbuf** with the result stored element-wise in **inoutbuf**, as explained for user-defined operations in Section 6.9.5. Both **inbuf** and **inoutbuf** (input as well as result) have the same number of elements given by **count** and the same datatype given by **datatype**. The **MPI_IN_PLACE** option is not allowed.

MPI_OP_COMMUTATIVE(op, commute)			1
IN	op	operation (handle)	2
			3
OUT	commute	true if op is commutative, false otherwise (logical)	4
			5

C binding

```
int MPI_Op_commutative(MPI_Op op, int *commute)
```

Fortran 2008 binding

```
MPI_Op_commutative(op, commute, ierror)
  TYPE(MPI_Op), INTENT(IN) :: op
  LOGICAL, INTENT(OUT) :: commute
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_OP_COMMUTATIVE(OP, COMMUTE, IERROR)
  INTEGER OP, IERROR
  LOGICAL COMMUTE
```

Reduction operations can be queried for their commutativity using [MPI_OP_COMMUTATIVE](#).

6.10 Reduce-Scatter

MPI includes variants of the reduce operations where the result is scattered to all MPI processes in a group on return. One variant scatters equal-sized blocks to all MPI processes, while another variant scatters blocks that may vary in size for each MPI process.

6.10.1 MPI_REDUCE_SCATTER_BLOCK

MPI_REDUCE_SCATTER_BLOCK(sendbuf, recvbuf, recvcount, datatype, op, comm)			31
IN	sendbuf	starting address of send buffer (choice)	32
			33
OUT	recvbuf	starting address of receive buffer (choice)	34
			35
IN	recvcount	element count per block (nonnegative integer)	36
IN	datatype	datatype of elements of send and receive buffers (handle)	37
			38
IN	op	operation (handle)	39
IN	comm	communicator (handle)	40
			41

C binding

```
int MPI_Reduce_scatter_block(const void *sendbuf, void *recvbuf, int recvcount,
                             MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Reduce_scatter_block_c(const void *sendbuf, void *recvbuf,
                               MPI_Count recvcount, MPI_Datatype datatype, MPI_Op op,
                               MPI_Comm comm)
```

Fortran 2008 binding

```

1  MPI_Reduce_scatter_block(sendbuf, recvbuf, recvcnt, datatype, op, comm,
2      ierror)
3      TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
4      TYPE(*), DIMENSION(..) :: recvbuf
5      INTEGER, INTENT(IN) :: recvcnt
6      TYPE(MPI_Datatype), INTENT(IN) :: datatype
7      TYPE(MPI_Op), INTENT(IN) :: op
8      TYPE(MPI_Comm), INTENT(IN) :: comm
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_Reduce_scatter_block(sendbuf, recvbuf, recvcnt, datatype, op, comm,
12     ierror) !(_c)
13     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
14     TYPE(*), DIMENSION(..) :: recvbuf
15     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcnt
16     TYPE(MPI_Datatype), INTENT(IN) :: datatype
17     TYPE(MPI_Op), INTENT(IN) :: op
18     TYPE(MPI_Comm), INTENT(IN) :: comm
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20

```

Fortran binding

```

21 MPI_REDUCE_SCATTER_BLOCK(SENDBUF, RECVBUF, RECVCOUNT, DATATYPE, OP, COMM,
22     IERROR)
23     <type> SENDBUF(*), RECVBUF(*)
24     INTEGER RECVCOUNT, DATATYPE, OP, COMM, IERROR
25

```

If `comm` is an intra-communicator, `MPI_REDUCE_SCATTER_BLOCK` first performs a global, element-wise reduction on vectors of `count = n*recvcnt` elements in the send buffers defined by `sendbuf`, `count` and `datatype`, using the operation `op`, where `n` is the number of MPI processes in the group of `comm`. The routine is called by all group members using the same arguments for `recvcnt`, `datatype`, `op` and `comm`. The resulting vector is treated as `n` consecutive blocks of `recvcnt` elements that are scattered to the MPI processes of the group. The *i*-th block is sent to MPI process *i* and stored in the receive buffer defined by `recvbuf`, `recvcnt`, and `datatype`.

Advice to implementors. The `MPI_REDUCE_SCATTER_BLOCK` routine is functionally equivalent to an `MPI_REDUCE` collective operation with `count` equal to `recvcnt*n`, followed by an `MPI_SCATTER` with `sendcount` equal to `recvcnt`. However, a direct implementation may run faster. (*End of advice to implementors.*)

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` in the `sendbuf` argument on *all* MPI processes. In this case, the input data is taken from the receive buffer.

If `comm` is an inter-communicator, then the result of the reduction of the data provided by MPI processes in one group (group A) is scattered among MPI processes in the other group (group B) and vice versa. Within each group, all MPI processes provide the same value for the `recvcnt` argument, and provide input vectors of `count = n*recvcnt` elements stored in the send buffers, where `n` is the size of the group. The number of elements `count` must be the same for the two groups. The resulting vector from the other group is scattered in blocks of `recvcnt` elements among the MPI processes in the group.

Rationale. The last restriction is needed so that the length of the send buffer of one group can be determined by the local `recvcount` argument of the other group. Otherwise, communication is needed to figure out how many elements are reduced. (*End of rationale.*)

6.10.2 MPI_REDUCE_SCATTER

MPI_REDUCE_SCATTER extends the functionality of **MPI_REDUCE_SCATTER_BLOCK** such that the scattered blocks can vary in size. Block sizes are determined by the `recvcounts` array, such that the *i*-th block contains `recvcounts[i]` elements.

MPI_REDUCE_SCATTER(sendbuf, recvbuf, recvcounts, datatype, op, comm)

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcounts	nonnegative integer array (of length group size) specifying the number of elements of the result distributed to each MPI process.
IN	datatype	datatype of elements of send and receive buffers (handle)
IN	op	operation (handle)
IN	comm	communicator (handle)

C binding

```
int MPI_Reduce_scatter(const void *sendbuf, void *recvbuf,
                      const int recvcounts[], MPI_Datatype datatype, MPI_Op op,
                      MPI_Comm comm)

int MPI_Reduce_scatter_c(const void *sendbuf, void *recvbuf,
                        const MPI_Count recvcounts[], MPI_Datatype datatype, MPI_Op op,
                        MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Reduce_scatter(sendbuf, recvbuf, recvcounts, datatype, op, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER, INTENT(IN) :: recvcounts(*)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_scatter(sendbuf, recvbuf, recvcounts, datatype, op, comm, ierror)
  !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcounts(*)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
```

```

1      TYPE(MPI_Comm), INTENT(IN) :: comm
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_REDUCE_SCATTER(SENDBUF, RECVBUF, RECVCOUNTS, DATATYPE, OP, COMM, IERROR)
5      <type> SENDBUF(*), RECVBUF(*)
6      INTEGER REVCOUNTS(*), DATATYPE, OP, COMM, IERROR

```

If `comm` is an intra-communicator, `MPI_REDUCE_SCATTER` first performs a global, element-wise reduction on vectors of `count =  $\sum_{i=0}^{n-1} \text{recvcounts}[i]$` elements in the send buffers defined by `sendbuf`, `count` and `datatype`, using the operation `op`, where `n` is the number of MPI processes in the group of `comm`. The routine is called by all group members using the same arguments for `recvcounts`, `datatype`, `op` and `comm`. The resulting vector is treated as `n` consecutive blocks where the number of elements of the `i`-th block is `recvcounts[i]`. The blocks are scattered to the MPI processes of the group. The `i`-th block is sent to MPI process `i` and stored in the receive buffer defined by `recvbuf`, `recvcounts[i]` and `datatype`.

Advice to implementors. The `MPI_REDUCE_SCATTER` routine is functionally equivalent to an `MPI_REDUCE` collective operation with `count` equal to the sum of `recvcounts[i]` followed by `MPI_SCATTERV` with `sendsizes` equal to `recvcounts`. However, a direct implementation may run faster. (*End of advice to implementors.*)

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` in the `sendbuf` argument. In this case, the input data is taken from the receive buffer. It is not required to specify the “in place” option on all MPI processes, since the MPI processes for which `recvcounts[i] = 0` may not have allocated a receive buffer.

If `comm` is an inter-communicator, then the result of the reduction of the data provided by MPI processes in one group (group A) is scattered among MPI processes in the other group (group B), and vice versa. Within each group, all MPI processes provide the same `recvcounts` argument, and provide input vectors of `count =  $\sum_{i=0}^{n-1} \text{recvcounts}[i]$` elements stored in the send buffers, where `n` is the size of the group. The resulting vector from the other group is scattered in blocks of `recvcounts[i]` elements among the MPI processes in the group. The number of elements `count` must be the same for the two groups.

Rationale. The last restriction is needed so that the length of the send buffer can be determined by the sum of the local `recvcounts` entries. Otherwise, communication is needed to figure out how many elements are reduced. (*End of rationale.*)

6.11 Scan

6.11.1 Inclusive Scan

```

43      MPI_SCAN(sendbuf, recvbuf, count, datatype, op, comm)

```

44	IN	sendbuf	starting address of send buffer (choice)
45			
46	OUT	recvbuf	starting address of receive buffer (choice)
47	IN	count	number of elements in input buffer (nonnegative integer)
48			

IN	datatype	datatype of elements of input buffer (handle)	1
IN	op	operation (handle)	2
IN	comm	communicator (handle)	3
			4
			5

C binding

```

int MPI_Scan(const void *sendbuf, void *recvbuf, int count,
             MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Scan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
               MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)

```

Fortran 2008 binding

```

MPI_Scan(sendbuf, recvbuf, count, datatype, op, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Scan(sendbuf, recvbuf, count, datatype, op, comm, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER COUNT, DATATYPE, OP, COMM, IERROR

```

If `comm` is an intra-communicator, `MPI_SCAN` is used to perform a prefix reduction on data distributed across the group. The operation returns, in the receive buffer of the MPI process with rank `i`, the reduction of the values in the send buffers of MPI processes with ranks `0, ..., i` (inclusive). The routine is called by all group members using the same arguments for `count`, `datatype`, `op` and `comm`, except that for user-defined operations, the same rules apply as for `MPI_REDUCE`. The type of operations supported, their semantics, and the constraints on send and receive buffers are as for `MPI_REDUCE`.

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` in the `sendbuf` argument. In this case, the input data is taken from the receive buffer, and replaced by the output data.

This operation is invalid for inter-communicators.

6.11.2 Exclusive Scan

MPI_EXSCAN(sendbuf, recvbuf, count, datatype, op, comm)

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	count	number of elements in input buffer (nonnegative integer)
IN	datatype	datatype of elements of input buffer (handle)
IN	op	operation (handle)
IN	comm	intra-communicator (handle)

C binding

```
int MPI_Exscan(const void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Exscan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Exscan(sendbuf, recvbuf, count, datatype, op, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Exscan(sendbuf, recvbuf, count, datatype, op, comm, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  TYPE(*), DIMENSION(..) :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_EXSCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER COUNT, DATATYPE, OP, COMM, IERROR
```

If `comm` is an intra-communicator, **MPI_EXSCAN** is used to perform a prefix reduction on data distributed across the group. The value in `recvbuf` on the MPI process with rank 0 is undefined, and `recvbuf` is not significant on that MPI process. The value in `recvbuf` on the MPI process with rank 1 is defined as the value in `sendbuf` on the MPI process with rank 0. For MPI processes with rank $i > 1$, the operation returns, in the receive buffer of the MPI process with rank i , the reduction of the values in the send buffers of MPI processes with

ranks $0, \dots, i-1$ (inclusive). The routine is called by all group members using the same arguments for `count`, `datatype`, `op` and `comm`, except that for user-defined operations, the same rules apply as for `MPI_REDUCE`. The type of operations supported, their semantics, and the constraints on send and receive buffers, are as for `MPI_REDUCE`.

The “in place” option for intra-communicators is specified by passing `MPI_IN_PLACE` in the `sendbuf` argument. In this case, the input data is taken from the receive buffer, and replaced by the output data. The receive buffer on rank 0 is not changed by this operation.

This operation is invalid for inter-communicators.

Rationale. The exclusive scan is more general than the inclusive scan. Any inclusive scan operation can be achieved by using the exclusive scan and then locally combining the local contribution. Note that for noninvertible operations such as `MPI_MAX`, the exclusive scan cannot be computed with the inclusive scan. (*End of rationale.*)

6.11.3 Example using `MPI_SCAN`

The example in this section uses an intra-communicator.

Example 6.24. This example uses a user-defined operation to produce a **segmented scan**. A segmented scan takes, as input, a set of values and a set of logicals, and the logicals delineate the various segments of the scan. For example:

<i>values</i>	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8
<i>logicals</i>	0	0	1	1	1	0	0	1
<i>result</i>	v_1	$v_1 + v_2$	v_3	$v_3 + v_4$	$v_3 + v_4 + v_5$	v_6	$v_6 + v_7$	v_8

The operator that produces this effect is

$$\begin{pmatrix} u \\ i \end{pmatrix} \circ \begin{pmatrix} v \\ j \end{pmatrix} = \begin{pmatrix} w \\ j \end{pmatrix},$$

where

$$w = \begin{cases} u + v & \text{if } i = j \\ v & \text{if } i \neq j \end{cases}.$$

Note that this is a noncommutative operator. C code that implements it is given below.

```
typedef struct {
    double val;
    int log;
} SegScanPair;

/* the user-defined function
*/
void segScan(SegScanPair *in, SegScanPair *inout, int *len,
             MPI_Datatype *dptr)
{
    int i;
    SegScanPair c;

    for (i=0; i< *len; ++i) {
        if (in->log == inout->log)
```

```

1      c.val = in->val + inout->val;
2      else
3          c.val = inout->val;
4      c.log = inout->log;
5      *inout = c;
6      in++; inout++;
7  }
8  }

```

Note that the `inout` argument to the user-defined function corresponds to the right-hand operand of the operator. When using this operator, we must be careful to specify that it is noncommutative, as in the following.

```

12  int i, base;
13  SegScanPair a, answer;
14  MPI_Op      myOp;
15  MPI_Datatype type[2] = {MPI_DOUBLE, MPI_INT};
16  MPI_Aint     disp[2];
17  int          blocklen[2] = {1, 1};
18  MPI_Datatype sspair;

19  /* explain to MPI how type SegScanPair is defined
20   */
21  MPI_Get_address(&a, disp);
22  MPI_Get_address(&a.log, disp+1);
23  base = disp[0];
24  for (i=0; i<2; ++i) disp[i] -= base;
25  MPI_Type_create_struct(2, blocklen, disp, type, &sspair);
26  MPI_Type_commit(&sspair);
27  /* create the segmented-scan user-op
28   */
29  MPI_Op_create(segScan, 0, &myOp);
30  ...
31  MPI_Scan(&a, &answer, 1, sspair, myOp, comm);

```

6.12 Nonblocking Collective Operations

As described in Section 3.7, performance of many applications can be improved by overlapping communication and computation, and many systems enable this. Nonblocking collective operations combine the potential benefits of nonblocking point-to-point operations, to exploit overlap and to avoid synchronization, with the optimized implementation and message scheduling provided by collective operations [35, 39]. One way of doing this would be to perform a blocking collective operation in a separate thread. An alternative mechanism that often leads to better performance (e.g., avoids context switching, scheduler overheads, and thread management) is to use nonblocking collective communication [37].

The nonblocking collective communication model is similar to the model used for nonblocking point-to-point communication. A nonblocking call initiates a collective operation, which must be completed in a separate completion call. Once initiated, the operation may progress independently of any computation or other communication at participating MPI processes. In this manner, nonblocking collective operations can mitigate possible synchronizing effects of collective operations by running them in the “background.” In addition to

enabling communication-computation overlap, nonblocking collective operations can perform collective operations on overlapping communicators, which would lead to deadlocks with blocking operations. Their semantic advantages can also be useful in combination with point-to-point communication.

As in the nonblocking point-to-point case, all calls are local and return immediately, irrespective of the status of other MPI processes. The call initiates the operation, which indicates that the system may start to copy data out of the send buffer and into the receive buffer. Once initiated, all associated send buffers and buffers associated with input arguments (such as arrays of counts, displacements, or datatypes in the vector versions of the collectives) should not be modified, and all associated receive buffers should not be accessed, until the collective operation completes. The call returns a request handle, which must be passed to a completion call.

All completion calls (e.g., `MPI_WAIT`) described in Section 3.7.3 are supported for nonblocking collective operations. Similarly to the blocking case, nonblocking collective operations are considered to be complete when the local part of the operation is finished, i.e., for the caller, the semantics of the operation are guaranteed and all buffers can be safely accessed and modified. Completion does not indicate that other MPI processes have completed or even started the operation (unless otherwise implied by the description of the operation). Completion of a particular nonblocking collective operation also does not indicate completion of any other posted nonblocking collective (or send-receive) operations, whether they are posted before or after the completed operation.

Advice to users. Users should be aware that implementations are allowed, but not required (with exception of `MPI_IBARRIER`), to synchronize MPI processes during the completion of a nonblocking collective operation. (*End of advice to users.*)

Upon returning from a completion call in which a nonblocking collective operation completes, the values of the `MPI_SOURCE` and `MPI_TAG` fields in the associated status object, if any, are undefined. The value of `MPI_ERROR` may be defined, if appropriate, according to the specification in Section 3.2.5. It is valid to mix different request types (i.e., any combination of collective requests, I/O requests, generalized requests, or point-to-point requests) in functions that enable multiple completions (e.g., `MPI_WAITALL`). It is erroneous to call `MPI_REQUEST_FREE` or `MPI_CANCEL` for a request associated with a nonblocking collective operation. Nonblocking collective requests created using the APIs described in this section are not persistent. However, persistent collective requests can be created using persistent collective operations described in Sections 6.13 and 8.8.

Rationale. Freeing an active nonblocking collective request could cause similar problems as discussed for point-to-point requests (see Section 3.7.3). Cancelling a request is not supported because the semantics of this operation are not well-defined. (*End of rationale.*)

Multiple nonblocking collective operations can be outstanding on a single communicator. If the nonblocking call causes some system resource to be exhausted, then it will fail and raise an error. Quality implementations of MPI should ensure that this happens only in pathological cases. That is, an MPI implementation should be able to support a large number of *pending* nonblocking operations.

Unlike point-to-point operations, nonblocking collective operations do not match with blocking collective operations, and collective operations do not have a tag argument. All

MPI processes must call collective operations (blocking and nonblocking) in the same order per communicator. In particular, once a MPI process calls a collective operation, all other MPI processes in the communicator must eventually call the same collective operation, and no other collective operation with the same communicator in between. This is consistent with the ordering rules for blocking collective operations in threaded environments.

Rationale. Matching blocking and nonblocking collective operations is not allowed because the implementation might use different communication algorithms for the two cases. Blocking collective operations may be optimized for minimal time to completion, while nonblocking collective operations may balance time to completion with CPU overhead and asynchronous progress.

The use of tags for collective operations can prevent certain hardware optimizations. (*End of rationale.*)

Advice to users. If program semantics require matching blocking and nonblocking collective operations, then a nonblocking collective operation can be initiated and immediately completed with a blocking wait to emulate blocking behavior. (*End of advice to users.*)

In terms of data movement, each nonblocking collective operation has the same effect as its blocking counterpart for intra-communicators and inter-communicators after completion. Likewise, upon completion, nonblocking collective reduction operations have the same effect as their blocking counterparts, and the same restrictions and recommendations on reduction orders apply.

The use of the “in place” option is allowed exactly as described for the corresponding blocking collective operations. When using the “in place” option, message buffers function as both send and receive buffers. Such buffers should not be modified or accessed until the operation completes.

The *progress* rules for nonblocking collective operations are similar to the progress rules for nonblocking point-to-point operations, refer to Sections 2.9 and 3.7.4.

Advice to implementors. Nonblocking collective operations can be implemented with local execution schedules [38] using nonblocking point-to-point communication and a reserved tag-space. (*End of advice to implementors.*)

6.12.1 Nonblocking Barrier Synchronization

MPI_IBARRIER(comm, request)

IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Ibarrier(MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Ibarrier(comm, request, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
```



```

TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IBARRIER(COMM, REQUEST, IERROR)
    INTEGER COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of `MPI_BARRIER` (see Section 6.3). By calling `MPI_IBARRIER`, an MPI process notifies that it has reached the barrier. The call returns immediately, independent of whether other MPI processes have called `MPI_IBARRIER`. The usual barrier semantics are enforced at the corresponding completion operation (test or wait), which in the intra-communicator case will complete only after all other MPI processes in the communicator have called `MPI_IBARRIER`. In the inter-communicator case, it will complete when all MPI processes in the remote group have called `MPI_IBARRIER`.

Advice to users. A nonblocking barrier can be used to hide latency. Moving independent computations between the `MPI_IBARRIER` and the subsequent completion call can overlap the barrier latency and therefore shorten possible waiting times. The semantic properties are also useful when mixing collective operations and point-to-point messages. (*End of advice to users.*)

6.12.2 Nonblocking Broadcast

```

MPI_IBCAST(buffer, count, datatype, root, comm, request)

```

INOUT	buffer	starting address of buffer (choice)
IN	count	number of entries in buffer (nonnegative integer)
IN	datatype	datatype of buffer (handle)
IN	root	rank of broadcast root (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Ibcast(void *buffer, int count, MPI_Datatype datatype, int root,
               MPI_Comm comm, MPI_Request *request)

```

```

int MPI_Ibcast_c(void *buffer, MPI_Count count, MPI_Datatype datatype,
                 int root, MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Ibcast(buffer, count, datatype, root, comm, request, ierror)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
    INTEGER, INTENT(IN) :: count, root
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1 MPI_Ibcast(buffer, count, datatype, root, comm, request, ierror) !(_c)
2   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
3   INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
4   TYPE(MPI_Datatype), INTENT(IN) :: datatype
5   INTEGER, INTENT(IN) :: root
6   TYPE(MPI_Comm), INTENT(IN) :: comm
7   TYPE(MPI_Request), INTENT(OUT) :: request
8   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

10 MPI_IBCAST(BUFFER, COUNT, DATATYPE, ROOT, COMM, REQUEST, IERROR)
11   <type> BUFFER(*)
12   INTEGER COUNT, DATATYPE, ROOT, COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_BCAST](#) (see Section 6.4).

Example using [MPI_IBCAST](#)

The example in this section uses an intra-communicator.

Example 6.25. Start a broadcast of 100 ints from MPI process 0 to every MPI process in the group, perform some computation on independent data, and then complete the outstanding broadcast operation.

```

23 MPI_Comm comm;
24 int array1[100], array2[100];
25 int root=0;
26 MPI_Request req;
27 ...
28 MPI_Ibcast(array1, 100, MPI_INT, root, comm, &req);
29 compute(array2, 100);
30 MPI_Wait(&req, MPI_STATUS_IGNORE);

```

6.12.3 Nonblocking Gather

```

35 MPI_IGATHER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm,
36             request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	recvcount	number of elements for any single receive (nonnegative integer, significant only at root)
IN	recvtype	datatype of recv buffer elements (handle, significant only at root)
IN	root	rank of receiving MPI process (integer)

IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Igather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
               MPI_Comm comm, MPI_Request *request)

int MPI_Igather_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                  MPI_Datatype recvtype, int root, MPI_Comm comm,
                  MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Igather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcount, root
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Igather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER, INTENT(IN) :: root
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, ROOT,
            COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, REQUEST,
    IERROR

```

This call starts a nonblocking variant of [MPI_GATHER](#) (see Section 6.5).

```

MPI_IGATHERV(sendbuf, sendcount, sendtype, recvbuf, recvcoun, displs, recvtype, root,
            comm, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)

1	IN	sendtype	datatype of send buffer elements (handle)
2	OUT	recvbuf	address of receive buffer (choice, significant only at root)
3			
4	IN	recvcounts	nonnegative integer array (of length group size) containing the number of elements that are received from each MPI process (significant only at root)
5			
6			
7	IN	displs	integer array (of length group size). Entry <i>i</i> specifies the displacement relative to <code>recvbuf</code> at which to place the incoming data from MPI process <i>i</i> (significant only at root)
8			
9			
10			
11	IN	recvtype	datatype of recv buffer elements (handle, significant only at root)
12			
13	IN	root	rank of receiving MPI process (integer)
14	IN	comm	communicator (handle)
15			
16	OUT	request	communication request (handle)
17			

C binding

```

19 int MPI_Igatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
20                 void *recvbuf, const int recvcounts[], const int displs[],
21                 MPI_Datatype recvtype, int root, MPI_Comm comm,
22                 MPI_Request *request)

```

```

23 int MPI_Igatherv_c(const void *sendbuf, MPI_Count sendcount,
24                   MPI_Datatype sendtype, void *recvbuf,
25                   const MPI_Count recvcounts[], const MPI_Aint displs[],
26                   MPI_Datatype recvtype, int root, MPI_Comm comm,
27                   MPI_Request *request)

```

Fortran 2008 binding

```

29 MPI_Igatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
30             recvtype, root, comm, request, ierror)
31     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
32     INTEGER, INTENT(IN) :: sendcount, root
33     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
34     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
35     INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*), displs(*)
36     TYPE(MPI_Comm), INTENT(IN) :: comm
37     TYPE(MPI_Request), INTENT(OUT) :: request
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
39
40 MPI_Igatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
41             recvtype, root, comm, request, ierror) !(_c)
42     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
43     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
44     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
45     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
46     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
47     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
48     INTEGER, INTENT(IN) :: root

```

```

TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
             RECVTYPE, ROOT, COMM, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, ROOT,
             COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_GATHERV](#) (see Section 6.5).

6.12.4 Nonblocking Scatter

```

MPI_ISCATTER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm,
             request)

```

IN	sendbuf	address of send buffer (choice, significant only at root)
IN	sendcount	number of elements sent to each MPI process (nonnegative integer, significant only at root)
IN	sendtype	datatype of send buffer elements (handle, significant only at root)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcount	number of elements in receive buffer (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	root	rank of sending MPI process (integer)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Iscatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
               MPI_Comm comm, MPI_Request *request)

```

```

int MPI_Iscatter_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                  MPI_Datatype recvtype, int root, MPI_Comm comm,
                  MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Iscatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
             comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype

```

```

1      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
2      TYPE(MPI_Comm), INTENT(IN) :: comm
3      TYPE(MPI_Request), INTENT(OUT) :: request
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6      MPI_Iscatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
7                  comm, request, ierror) !(_c)
8      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
9      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
10     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
11     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
12     INTEGER, INTENT(IN) :: root
13     TYPE(MPI_Comm), INTENT(IN) :: comm
14     TYPE(MPI_Request), INTENT(OUT) :: request
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

16 MPI_ISCATTER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, ROOT,
17             COMM, REQUEST, IERROR)
18     <type> SENDBUF(*), RECVBUF(*)
19     INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, REQUEST,
20             IERROR

```

This call starts a nonblocking variant of [MPI_SCATTER](#) (see Section 6.6).

```

25 MPI_ISCATTERV(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount, recvtype, root,
26             comm, request)

```

27	IN	sendbuf	address of send buffer (choice, significant only at root)
28			
29	IN	sendcounts	nonnegative integer array (of length group size) specifying the number of elements to send to each MPI process (significant only at root)
30			
31			
32	IN	displs	integer array (of length group size). Entry i specifies the displacement (relative to <code>sendbuf</code>) from which to take the outgoing data to MPI process i (significant only at root)
33			
34			
35			
36	IN	sendtype	datatype of send buffer elements (handle, significant only at root)
37			
38	OUT	recvbuf	address of receive buffer (choice)
39	IN	recvcount	number of elements in receive buffer (nonnegative integer)
40			
41	IN	recvtype	datatype of receive buffer elements (handle)
42	IN	root	rank of sending MPI process (integer)
43			
44	IN	comm	communicator (handle)
45	OUT	request	communication request (handle)
46			

C binding

```

48 int MPI_Iscatterv(const void *sendbuf, const int sendcounts[],

```

```

const int displs[], MPI_Datatype sendtype, void *recvbuf,
int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm,
MPI_Request *request)

int MPI_Iscatterv_c(const void *sendbuf, const MPI_Count sendcounts[],
const MPI_Aint displs[], MPI_Datatype sendtype, void *recvbuf,
MPI_Count recvcount, MPI_Datatype recvtype, int root,
MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Iscatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
recvtype, root, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), displs(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: recvcount, root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iscatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
recvtype, root, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcount
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ISCATTERV(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE, RECVBUF, RECVCOUNT,
RECVTYPE, ROOT, COMM, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), DISPLS(*), SENDTYPE, RECVCOUNT, RECVTYPE, ROOT,
COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of `MPI_SCATTERV` (see Section 6.6).

6.12.5 Nonblocking All-Gather

```

MPI_IALLGATHER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
request)

```

IN	sendbuf	starting address of send buffer (choice)
----	---------	--

1	IN	sendcount	number of elements in send buffer (nonnegative integer)
2			
3	IN	sendtype	datatype of send buffer elements (handle)
4	OUT	recvbuf	address of receive buffer (choice)
5			
6	IN	recvcount	number of elements received from any MPI process (nonnegative integer)
7			
8	IN	recvtype	datatype of receive buffer elements (handle)
9	IN	comm	communicator (handle)
10			
11	OUT	request	communication request (handle)

C binding

```

13 int MPI_Iallgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
14                  void *recvbuf, int recvcount, MPI_Datatype recvtype,
15                  MPI_Comm comm, MPI_Request *request)
16

```

```

17 int MPI_Iallgather_c(const void *sendbuf, MPI_Count sendcount,
18                    MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
19                    MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
20

```

Fortran 2008 binding

```

21 MPI_Iallgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
22               comm, request, ierror)
23

```

```

24   TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
25   INTEGER, INTENT(IN) :: sendcount, recvcount
26   TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
27   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
28   TYPE(MPI_Comm), INTENT(IN) :: comm
29   TYPE(MPI_Request), INTENT(OUT) :: request
30   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

31 MPI_Iallgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
32               comm, request, ierror) !(_c)

```

```

33   TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
34   INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
35   TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
36   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
37   TYPE(MPI_Comm), INTENT(IN) :: comm
38   TYPE(MPI_Request), INTENT(OUT) :: request
39   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

41 MPI_IALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE,
42               COMM, REQUEST, IERROR)

```

```

43   <type> SENDBUF(*), RECVBUF(*)

```

```

44   INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, REQUEST, IERROR

```

45 This call starts a nonblocking variant of [MPI_ALLGATHER](#) (see Section 6.7).

MPI_IALLGATHERV(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs, recvtpe, comm, request)			1
			2
IN	sendbuf	starting address of send buffer (choice)	3
IN	sendcount	number of elements in send buffer (nonnegative integer)	4
IN	sendtype	datatype of send buffer elements (handle)	5
OUT	recvbuf	address of receive buffer (choice)	6
IN	recvcounts	nonnegative integer array (of length group size) containing the number of elements that are received from each MPI process	7
IN	displs	integer array (of length group size). Entry i specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from MPI process i	8
IN	recvtpe	datatype of receive buffer elements (handle)	9
IN	comm	communicator (handle)	10
OUT	request	communication request (handle)	11

C binding

```

int MPI_Iallgatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
    void *recvbuf, const int recvcounts[], const int displs[],
    MPI_Datatype recvtpe, MPI_Comm comm, MPI_Request *request)
int MPI_Iallgatherv_c(const void *sendbuf, MPI_Count sendcount,
    MPI_Datatype sendtype, void *recvbuf,
    const MPI_Count recvcounts[], const MPI_Aint displs[],
    MPI_Datatype recvtpe, MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Iallgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
    recvtpe, comm, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER, INTENT(IN) :: sendcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*), displs(*)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Iallgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
    recvtpe, comm, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
    TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

1      TYPE(MPI_Request), INTENT(OUT) :: request
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_IALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
5                      RECVTYPE, COMM, REQUEST, IERROR)
6      <type> SENDBUF(*), RECVBUF(*)
7      INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
8                      REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_ALLGATHERV](#) (see Section 6.7).

6.12.6 Nonblocking All-to-All Scatter/Gather

```

15      MPI_IALLTOALL(sendbuf, sendcount, sendtype, recvbuf, recvcoun, recvtype, comm,
16                  request)

```

18	IN	sendbuf	starting address of send buffer (choice)
19	IN	sendcount	number of elements sent to each MPI process (nonnegative integer)
21	IN	sendtype	datatype of send buffer elements (handle)
22	OUT	recvbuf	address of receive buffer (choice)
24	IN	recvcoun	number of elements received from any MPI process (nonnegative integer)
26	IN	recvtype	datatype of receive buffer elements (handle)
27	IN	comm	communicator (handle)
28	OUT	request	communication request (handle)

C binding

```

31      int MPI_Ialltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
32                      void *recvbuf, int recvcoun, MPI_Datatype recvtype,
33                      MPI_Comm comm, MPI_Request *request)
34
35      int MPI_Ialltoall_c(const void *sendbuf, MPI_Count sendcount,
36                      MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcoun,
37                      MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

39      MPI_Ialltoall(sendbuf, sendcount, sendtype, recvbuf, recvcoun, recvtype, comm,
40                  request, ierror)
41      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
42      INTEGER, INTENT(IN) :: sendcount, recvcoun
43      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
44      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
45      TYPE(MPI_Comm), INTENT(IN) :: comm
46      TYPE(MPI_Request), INTENT(OUT) :: request
47      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Ialltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
              request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, COMM,
              REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_ALLTOALL](#) (see Section 6.8).

```

MPI_IALLTOALLV(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts, rdispls,
               recvtype, comm, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	nonnegative integer array (of length group size) specifying the number of elements to send to each MPI process
IN	sdispls	integer array (of length group size). Entry j specifies the displacement (relative to <code>sendbuf</code>) from which to take the outgoing data destined for MPI process j
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcounts	nonnegative integer array (of length group size) specifying the number of elements that can be received from each MPI process
IN	rdispls	integer array (of length group size). Entry i specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from MPI process i
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Ialltoallv(const void *sendbuf, const int sendcounts[],
                  const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
                  const int recvcounts[], const int rdispls[],
                  MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)

int MPI_Ialltoallv_c(const void *sendbuf, const MPI_Count sendcounts[],
                    const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,

```

```

1      const MPI_Count recvcnts[], const MPI_Aint rdispls[],
2      MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
3
4  Fortran 2008 binding
5  MPI_Ialltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcnts,
6      rdispls, recvtype, comm, request, ierror)
7      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
8      INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
9      recvcnts(*), rdispls(*)
10     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
11     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
12     TYPE(MPI_Comm), INTENT(IN) :: comm
13     TYPE(MPI_Request), INTENT(OUT) :: request
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_Ialltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcnts,
17     rdispls, recvtype, comm, request, ierror) !(_c)
18     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
19     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
20     recvcnts(*)
21     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
22     rdispls(*)
23     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
24     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
25     TYPE(MPI_Comm), INTENT(IN) :: comm
26     TYPE(MPI_Request), INTENT(OUT) :: request
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

28 MPI_IALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF, REVCOUNTS,
29     RDISPLS, RECVTYPE, COMM, REQUEST, IERROR)
30     <type> SENDBUF(*), RECVBUF(*)
31     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, REVCOUNTS(*), RDISPLS(*),
32     RECVTYPE, COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_ALLTOALLV](#) (see Section 6.8).

```

36 MPI_IALLTOALLW(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcnts, rdispls,
37     recvtypes, comm, request)

```

39	IN	sendbuf	starting address of send buffer (choice)
40	IN	sendcounts	integer array (of length group size) specifying the number of elements to send to each MPI process (array of nonnegative integers)
41			
42			
43	IN	sdispls	integer array (of length group size). Entry j specifies the displacement in bytes (relative to sendbuf) from which to take the outgoing data destined for MPI process j (array of integers)
44			
45			
46			
47	IN	sendtypes	array of datatypes (of length group size). Entry j specifies the type of data to send to MPI process j (array of handles)
48			

OUT	recvbuf	address of receive buffer (choice)	1
IN	recvcounts	integer array (of length group size) specifying the number of elements that can be received from each MPI process (array of nonnegative integers)	2
IN	rdispls	integer array (of length group size). Entry <i>i</i> specifies the displacement in bytes (relative to <i>recvbuf</i>) at which to place the incoming data from MPI process <i>i</i> (array of integers)	3
IN	recvtypes	array of datatypes (of length group size). Entry <i>i</i> specifies the type of data received from MPI process <i>i</i> (array of handles)	4
IN	comm	communicator (handle)	5
OUT	request	communication request (handle)	6

C binding

```

int MPI_Ialltoallw(const void *sendbuf, const int sendcounts[],
                  const int sdispls[], const MPI_Datatype sendtypes[],
                  void *recvbuf, const int recvcounts[], const int rdispls[],
                  const MPI_Datatype recvtypes[], MPI_Comm comm,
                  MPI_Request *request)
int MPI_Ialltoallw_c(const void *sendbuf, const MPI_Count sendcounts[],
                    const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                    void *recvbuf, const MPI_Count recvcounts[],
                    const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
                    MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Ialltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcounts,
               rdispls, recvtypes, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
recvcounts(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Ialltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcounts,
               rdispls, recvtypes, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
recvcounts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
rdispls(*)
TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

1      TYPE(MPI_Request), INTENT(OUT) :: request
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_IALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF, RECVCOUNTS,
5                     RDISPLS, RECVTYPES, COMM, REQUEST, IERROR)
6      <type> SENDBUF(*), RECVBUF(*)
7      INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPES(*), RECVCOUNTS(*), RDISPLS(*),
8                     RECVTYPES(*), COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_ALLTOALLW](#) (see Section 6.8).

6.12.7 Nonblocking Reduce

```

15     MPI_IREDUCE(sendbuf, recvbuf, count, datatype, op, root, comm, request)

```

17	IN	sendbuf	address of send buffer (choice)
18	OUT	recvbuf	address of receive buffer (choice, significant only at root)
20	IN	count	number of elements in send buffer (nonnegative integer)
22	IN	datatype	datatype of elements of send buffer (handle)
23	IN	op	reduce operation (handle)
25	IN	root	rank of the root (integer)
26	IN	comm	communicator (handle)
27	OUT	request	communication request (handle)

C binding

```

30     int MPI_Ireduce(const void *sendbuf, void *recvbuf, int count,
31                   MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
32                   MPI_Request *request)

```

```

34     int MPI_Ireduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
35                      MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
36                      MPI_Request *request)

```

Fortran 2008 binding

```

38     MPI_Ireduce(sendbuf, recvbuf, count, datatype, op, root, comm, request, ierror)
39     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
40     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
41     INTEGER, INTENT(IN) :: count, root
42     TYPE(MPI_Datatype), INTENT(IN) :: datatype
43     TYPE(MPI_Op), INTENT(IN) :: op
44     TYPE(MPI_Comm), INTENT(IN) :: comm
45     TYPE(MPI_Request), INTENT(OUT) :: request
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Ireduce(sendbuf, recvbuf, count, datatype, op, root, comm, request, ierror)
    !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    INTEGER, INTENT(IN) :: root
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IREDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, ROOT, COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, ROOT, COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_REDUCE](#) (see Section 6.9.1).

Advice to implementors. The implementation is explicitly allowed to use different algorithms for blocking and nonblocking reduction operations that might change the order of evaluation of the operations. However, as for [MPI_REDUCE](#), it is strongly recommended that [MPI_IREDUCE](#) be implemented so that the same result be obtained whenever the function is applied on the same arguments, appearing in the same order. Note that this may prevent optimizations that take advantage of the physical location of MPI processes. (*End of advice to implementors.*)

Advice to users. For operations that are not truly associative, the result delivered upon completion of the nonblocking reduction may not exactly equal the result delivered by the blocking reduction, even when specifying the same arguments in the same order. (*End of advice to users.*)

6.12.8 Nonblocking All-Reduce

```

MPI_IALLREDUCE(sendbuf, recvbuf, count, datatype, op, comm, request)
    IN      sendbuf      starting address of send buffer (choice)
    OUT     recvbuf      starting address of receive buffer (choice)
    IN      count        number of elements in send buffer (nonnegative
                        integer)
    IN      datatype     datatype of elements of send buffer (handle)
    IN      op           operation (handle)
    IN      comm         communicator (handle)
    OUT     request      communication request (handle)

```

C binding

```

int MPI_Iallreduce(const void *sendbuf, void *recvbuf, int count,

```

```

1      MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
2      MPI_Request *request)
3
4  int MPI_Iallreduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
5      MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
6      MPI_Request *request)

```

Fortran 2008 binding

```

8  MPI_Iallreduce(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
9      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
10     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
11     INTEGER, INTENT(IN) :: count
12     TYPE(MPI_Datatype), INTENT(IN) :: datatype
13     TYPE(MPI_Op), INTENT(IN) :: op
14     TYPE(MPI_Comm), INTENT(IN) :: comm
15     TYPE(MPI_Request), INTENT(OUT) :: request
16     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18  MPI_Iallreduce(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
19      !(_c)
20     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
21     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
22     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     TYPE(MPI_Op), INTENT(IN) :: op
25     TYPE(MPI_Comm), INTENT(IN) :: comm
26     TYPE(MPI_Request), INTENT(OUT) :: request
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

29  MPI_IALLREDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, REQUEST, IERROR)
30      <type> SENDBUF(*), RECVBUF(*)
31      INTEGER COUNT, DATATYPE, OP, COMM, REQUEST, IERROR

```

This call starts a nonblocking variant of [MPI_ALLREDUCE](#) (see Section 6.9.6).

6.12.9 Nonblocking Reduce-Scatter with Equal Blocks

```

37  MPI_IREDUCE_SCATTER_BLOCK(sendbuf, recvbuf, recvcnt, datatype, op, comm,
38      request)
39

```

40	IN	sendbuf	starting address of send buffer (choice)
41	OUT	recvbuf	starting address of receive buffer (choice)
42	IN	recvcnt	element count per block (nonnegative integer)
43	IN	datatype	datatype of elements of send and receive buffers (handle)
44	IN	op	operation (handle)
45	IN	comm	communicator (handle)

1	IN	recvcounts	nonnegative integer array specifying the number of
2			elements in result distributed to each MPI process.
3	IN	datatype	datatype of elements of input buffer (handle)
4	IN	op	operation (handle)
5	IN	comm	communicator (handle)
6	OUT	request	communication request (handle)
7			
8			

C binding

```

10 int MPI_Ireduce_scatter(const void *sendbuf, void *recvbuf,
11                        const int recvcounts[], MPI_Datatype datatype, MPI_Op op,
12                        MPI_Comm comm, MPI_Request *request)
13

```

```

14 int MPI_Ireduce_scatter_c(const void *sendbuf, void *recvbuf,
15                          const MPI_Count recvcounts[], MPI_Datatype datatype, MPI_Op op,
16                          MPI_Comm comm, MPI_Request *request)
17

```

Fortran 2008 binding

```

18 MPI_Ireduce_scatter(sendbuf, recvbuf, recvcounts, datatype, op, comm, request,
19                    ierror)
20

```

```

21 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
22 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
23 INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
24 TYPE(MPI_Datatype), INTENT(IN) :: datatype
25 TYPE(MPI_Op), INTENT(IN) :: op
26 TYPE(MPI_Comm), INTENT(IN) :: comm
27 TYPE(MPI_Request), INTENT(OUT) :: request
28 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29

```

```

30 MPI_Ireduce_scatter(sendbuf, recvbuf, recvcounts, datatype, op, comm, request,
31                    ierror) !(_c)
32

```

```

33 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
34 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
35 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
36 TYPE(MPI_Datatype), INTENT(IN) :: datatype
37 TYPE(MPI_Op), INTENT(IN) :: op
38 TYPE(MPI_Comm), INTENT(IN) :: comm
39 TYPE(MPI_Request), INTENT(OUT) :: request
40 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
41

```

Fortran binding

```

42 MPI_IREDUCE_SCATTER(SENDBUF, RECVBUF, RECVCOUNTS, DATATYPE, OP, COMM, REQUEST,
43                    IERROR)
44

```

```

45 <type> SENDBUF(*), RECVBUF(*)
46 INTEGER REVCOUNTS(*), DATATYPE, OP, COMM, REQUEST, IERROR
47

```

48 This call starts a nonblocking variant of [MPI_REDUCE_SCATTER](#) (see Section 6.10.2).
The `array` argument must be identical on all calling MPI processes.

6.12.11 Nonblocking Inclusive Scan

MPI_ISCAN(sendbuf, recvbuf, count, datatype, op, comm, request)

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	count	number of elements in input buffer (nonnegative integer)
IN	datatype	datatype of elements of input buffer (handle)
IN	op	operation (handle)
IN	comm	communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Iscan(const void *sendbuf, void *recvbuf, int count,
             MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
             MPI_Request *request)

int MPI_Iscan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
               MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
               MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Iscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ISCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, REQUEST, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER COUNT, DATATYPE, OP, COMM, REQUEST, IERROR
```

This call starts a nonblocking variant of [MPI_SCAN](#) (see Section 6.11.1).

6.12.12 Nonblocking Exclusive Scan

MPI_IEXSCAN(sendbuf, recvbuf, count, datatype, op, comm, request)

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	count	number of elements in input buffer (nonnegative integer)
IN	datatype	datatype of elements of input buffer (handle)
IN	op	operation (handle)
IN	comm	intra-communicator (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Iexscan(const void *sendbuf, void *recvbuf, int count,
                MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                MPI_Request *request)
```

```
int MPI_Iexscan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                  MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Iexscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iexscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_IEXSCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, REQUEST, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER COUNT, DATATYPE, OP, COMM, REQUEST, IERROR
```

This call starts a nonblocking variant of [MPI_EXSCAN](#) (see Section 6.11.2).

6.13 Persistent Collective Operations

Many parallel computation algorithms involve repetitively executing a collective communication operation with the same arguments each time. As with persistent point-to-point operations (see Section 3.9), persistent collective operations allow the MPI programmer to specify operations that will be reused frequently (with fixed arguments). MPI can be designed to select a more efficient way to perform the collective operation based on the parameters specified when the operation is initialized. This “planned-transfer” approach [53, 42] can offer significant performance benefits for programs with repetitive communication patterns.

In terms of data movement, each persistent collective operation has the same effect as its blocking and nonblocking counterparts for intra-communicators and inter-communicators after completion. Likewise, upon completion, persistent collective reduction operations perform the same operation as their blocking and nonblocking counterparts, and the same restrictions and recommendations on reduction orders apply (see also Section 6.9.1).

Initialization calls for MPI persistent collective operations are nonlocal and follow all the existing rules for collective operations, in particular ordering; programs that do not conform to these restrictions are erroneous. After initialization, all arrays associated with input arguments (such as arrays of counts, displacements, and datatypes in the vector versions of the collectives) must not be modified until the corresponding persistent request is freed with `MPI_REQUEST_FREE`.

According to the definitions in Section 2.4.2, the persistent collective initialization procedures are incomplete. They are also nonlocal procedures because they may or may not return before they are called in all MPI processes of the MPI process group associated with the specified communicator.

Advice to users. This is one of the exceptions in which incomplete procedures are nonlocal and therefore blocking. (*End of advice to users.*)

The `request` argument is an output argument that can be used zero or more times with `MPI_START` or `MPI_STARTALL` in order to start the collective operation. The `request` is initially inactive after the initialization call. Once initialized, persistent collective operations can be started in any order and the order can differ among the MPI processes in the communicator.

Rationale. All ordering requirements that an implementation may need to match up collective operations across the communicator are achieved through the ordering requirements of the initialization functions. This enables out-of-order starts for the persistent operations, and particularly supports their use in `MPI_STARTALL`. (*End of rationale.*)

Advice to implementors. An MPI implementation should do no worse than duplicating the communicator during the initialization function, caching the input arguments, and calling the appropriate nonblocking collective function, using the cached arguments, during `MPI_START`. High-quality implementations should be able to amortize setup costs and further optimize by taking advantage of early-binding, such as efficient and effective pre-allocation of certain resources and algorithm selection. (*End of advice to implementors.*)

A request must be inactive when it is started. Starting the operation makes the request active. Once any MPI process starts a persistent collective operation, it must complete that operation and all other MPI processes in the communicator must eventually start (and complete) the same persistent collective operation. Persistent collective operations cannot be *matched* with blocking or nonblocking collective operations. Completion of a persistent collective operation makes the corresponding request inactive. After starting a persistent collective operation, all associated send buffers must not be modified and all associated receive buffers must not be accessed until the corresponding persistent request is completed.

Completing a persistent collective request, for example using `MPI_TEST` or `MPI_WAIT`, makes it inactive, but does not free the request. This is the same behavior as for persistent point-to-point requests. Inactive persistent collective requests can be freed using `MPI_REQUEST_FREE`. It is erroneous to free an active persistent collective request. Persistent collective operations cannot be canceled; it is erroneous to use `MPI_CANCEL` on a persistent collective request.

For every nonblocking collective communication operation in MPI, there is a corresponding persistent collective operation with the analogous API signature.

The collective persistent API signatures include an info object in order to support optimization hints and other information that may be nonstandard. Persistent collective operations may be optimized during communicator creation or by the initialization operation of an individual persistent collective. Note that communicator-scoped hints should be provided using `MPI_COMM_SET_INFO` while, for operation-scoped hints, they are supplied to the persistent collective communication initialization functions using the `info` argument.

6.13.1 Persistent Barrier Synchronization

`MPI_BARRIER_INIT(comm, info, request)`

IN	<code>comm</code>	communicator (handle)
IN	<code>info</code>	info argument (handle)
OUT	<code>request</code>	communication request (handle)

C binding

```
int MPI_Barrier_init(MPI_Comm comm, MPI_Info info, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Barrier_init(comm, info, request, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_BARRIER_INIT(COMM, INFO, REQUEST, IERROR)
  INTEGER COMM, INFO, REQUEST, IERROR
```

Creates a persistent collective communication request for the barrier operation (see Section 6.3).

6.13.2 Persistent Broadcast

MPI_BCAST_INIT(buffer, count, datatype, root, comm, info, request)

INOUT	buffer	starting address of buffer (choice)
IN	count	number of entries in buffer (nonnegative integer)
IN	datatype	datatype of buffer (handle)
IN	root	rank of broadcast root (integer)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Bcast_init(void *buffer, int count, MPI_Datatype datatype, int root,
                  MPI_Comm comm, MPI_Info info, MPI_Request *request)
```

```
int MPI_Bcast_init_c(void *buffer, MPI_Count count, MPI_Datatype datatype,
                    int root, MPI_Comm comm, MPI_Info info, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Bcast_init(buffer, count, datatype, root, comm, info, request, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
  INTEGER, INTENT(IN) :: count, root
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Bcast_init(buffer, count, datatype, root, comm, info, request, ierror)
  !(_c)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_BCAST_INIT(BUFFER, COUNT, DATATYPE, ROOT, COMM, INFO, REQUEST, IERROR)
  <type> BUFFER(*)
  INTEGER COUNT, DATATYPE, ROOT, COMM, INFO, REQUEST, IERROR
```

Creates a persistent collective communication request for the broadcast operation (see Section 6.4).

6.13.3 Persistent Gather

```

MPI_GATHER_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
                comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	recvcount	number of elements for any single receive (nonnegative integer, significant only at root)
IN	recvtype	datatype of recv buffer elements (handle, significant only at root)
IN	root	rank of receiving MPI process (integer)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Gather_init(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                  void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
                  MPI_Comm comm, MPI_Info info, MPI_Request *request)

int MPI_Gather_init_c(const void *sendbuf, MPI_Count sendcount,
                    MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                    MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
                    MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Gather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                root, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcount, root
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Gather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                root, comm, info, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype

```



```

TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GATHER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE,
                ROOT, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, INFO,
REQUEST, IERROR

```

Creates a persistent collective communication request for the gather operation (see Section 6.5).

```

MPI_GATHERV_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs, recvtype,
                root, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	recvcunts	nonnegative integer array (of length group size) containing the number of elements that are received from each MPI process (significant only at root)
IN	displs	integer array (of length group size). Entry <i>i</i> specifies the displacement relative to <i>recvbuf</i> at which to place the incoming data from MPI process <i>i</i> (significant only at root)
IN	recvtype	datatype of recv buffer elements (handle, significant only at root)
IN	root	rank of receiving MPI process (integer)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Gatherv_init(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                    void *recvbuf, const int recvcunts[], const int displs[],
                    MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
                    MPI_Request *request)

```

```

int MPI_Gatherv_init_c(const void *sendbuf, MPI_Count sendcount,
                      MPI_Datatype sendtype, void *recvbuf,

```

```

1      const MPI_Count recvcunts[], const MPI_Aint displs[],
2      MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
3      MPI_Request *request)
4

```

Fortran 2008 binding

```

5 MPI_Gatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs,
6      recvtype, root, comm, info, request, ierror)
7
8  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
9  INTEGER, INTENT(IN) :: sendcount, root
10 TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
11 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
12 INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcunts(*), displs(*)
13 TYPE(MPI_Comm), INTENT(IN) :: comm
14 TYPE(MPI_Info), INTENT(IN) :: info
15 TYPE(MPI_Request), INTENT(OUT) :: request
16 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18 MPI_Gatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs,
19      recvtype, root, comm, info, request, ierror) !(_c)
20 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
21 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
22 TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
23 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
24 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcunts(*)
25 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
26 INTEGER, INTENT(IN) :: root
27 TYPE(MPI_Comm), INTENT(IN) :: comm
28 TYPE(MPI_Info), INTENT(IN) :: info
29 TYPE(MPI_Request), INTENT(OUT) :: request
30 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

31 MPI_GATHERV_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
32      RECVTYPE, ROOT, COMM, INFO, REQUEST, IERROR)
33
34 <type> SENDBUF(*), RECVBUF(*)
35 INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, ROOT,
36      COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the gatherv operation (see Section 6.5).

6.13.4 Persistent Scatter

```

43 MPI_SCATTER_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcunt, recvtype, root,
44      comm, info, request)
45
46 IN      sendbuf      address of send buffer (choice, significant only at
47                        root)
48 IN      sendcount    number of elements sent to each MPI process
                        (nonnegative integer, significant only at root)

```

IN	sendtype	datatype of send buffer elements (handle, significant only at root)	1
OUT	recvbuf	address of receive buffer (choice)	3
IN	recvcount	number of elements in receive buffer (nonnegative integer)	5
IN	recvtype	datatype of receive buffer elements (handle)	7
IN	root	rank of sending MPI process (integer)	8
IN	comm	communicator (handle)	9
IN	info	info argument (handle)	11
OUT	request	communication request (handle)	12

C binding

```
int MPI_Scatter_init(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                    void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
                    MPI_Comm comm, MPI_Info info, MPI_Request *request)
```

```
int MPI_Scatter_init_c(const void *sendbuf, MPI_Count sendcount,
                      MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                      MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
                      MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Scatter_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                 root, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcount, root
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Scatter_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                 root, comm, info, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER, INTENT(IN) :: root
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_SCATTER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE,
                ROOT, COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, INFO,
    REQUEST, IERROR

```

Creates a persistent collective communication request for the scatter operation (see Section 6.6).

```

MPI_SCATTERV_INIT(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcoun, recvtype,
                root, comm, info, request)

```

IN	sendbuf	address of send buffer (choice, significant only at root)
IN	sendcounts	nonnegative integer array (of length group size) specifying the number of elements to send to each MPI process (significant only at root)
IN	displs	integer array (of length group size). Entry <i>i</i> specifies the displacement (relative to sendbuf) from which to take the outgoing data to MPI process <i>i</i> (significant only at root)
IN	sendtype	datatype of send buffer elements (handle, significant only at root)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	recvcoun	number of elements in receive buffer (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	root	rank of sending MPI process (integer)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Scatterv_init(const void *sendbuf, const int sendcounts[],
                    const int displs[], MPI_Datatype sendtype, void *recvbuf,
                    int recvcoun, MPI_Datatype recvtype, int root, MPI_Comm comm,
                    MPI_Info info, MPI_Request *request)

```

```

int MPI_Scatterv_init_c(const void *sendbuf, const MPI_Count sendcounts[],
                    const MPI_Aint displs[], MPI_Datatype sendtype, void *recvbuf,
                    MPI_Count recvcoun, MPI_Datatype recvtype, int root,
                    MPI_Comm comm, MPI_Info info, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Scatterv_init(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcoun,
                recvtype, root, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf

```

```

INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), displs(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: recvcnt, root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scatterv_init(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcnt,
    recvtype, root, comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcnt
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SCATTERV_INIT(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE, RECVBUF, REVCOUNT,
    RECVTYPE, ROOT, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), DISPLS(*), SENDTYPE, REVCOUNT, RECVTYPE, ROOT,
    COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the scatterv operation (see Section 6.6).

6.13.5 Persistent All-Gather

```

MPI_ALLGATHER_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype, comm,
    info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements in send buffer (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcnt	number of elements received from each MPI process (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator (handle)

```

1      IN      info      info argument (handle)
2
3      OUT      request      communication request (handle)

```

C binding

```

5  int MPI_Allgather_init(const void *sendbuf, int sendcount,
6                          MPI_Datatype sendtype, void *recvbuf, int recvcount,
7                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
8                          MPI_Request *request)
9
10 int MPI_Allgather_init_c(const void *sendbuf, MPI_Count sendcount,
11                          MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
12                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
13                          MPI_Request *request)

```

Fortran 2008 binding

```

15 MPI_Allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
16                   comm, info, request, ierror)
17     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
18     INTEGER, INTENT(IN) :: sendcount, recvcount
19     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
20     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
21     TYPE(MPI_Comm), INTENT(IN) :: comm
22     TYPE(MPI_Info), INTENT(IN) :: info
23     TYPE(MPI_Request), INTENT(OUT) :: request
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26 MPI_Allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
27                   comm, info, request, ierror) !(_c)
28     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
29     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
30     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
31     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
32     TYPE(MPI_Comm), INTENT(IN) :: comm
33     TYPE(MPI_Info), INTENT(IN) :: info
34     TYPE(MPI_Request), INTENT(OUT) :: request
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

37 MPI_ALLGATHER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE,
38                   COMM, INFO, REQUEST, IERROR)
39     <type> SENDBUF(*), RECVBUF(*)
40     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
41     IERROR

```

Creates a persistent collective communication request for the allgather operation (see Section 6.7).

```

46 MPI_ALLGATHERV_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcoun, displs,
47                   recvtype, comm, info, request)
48

```

IN	sendbuf	starting address of send buffer (choice)	1
IN	sendcount	number of elements in send buffer (nonnegative integer)	2
			3
IN	sendtype	datatype of send buffer elements (handle)	4
			5
OUT	recvbuf	address of receive buffer (choice)	6
IN	recvcunts	nonnegative integer array (of length group size) containing the number of elements that are received from each MPI process	7
			8
			9
IN	displs	integer array (of length group size). Entry i specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from MPI process i	10
			11
			12
IN	recvtype	datatype of receive buffer elements (handle)	13
IN	comm	communicator (handle)	14
IN	info	info argument (handle)	15
			16
OUT	request	communication request (handle)	17
			18
C binding			19
<code>int MPI_Allgatherv_init(const void *sendbuf, int sendcount,</code>			20
<code> MPI_Datatype sendtype, void *recvbuf, const int recvcunts[],</code>			21
<code> const int displs[], MPI_Datatype recvtype, MPI_Comm comm,</code>			22
<code> MPI_Info info, MPI_Request *request)</code>			23
			24
<code>int MPI_Allgatherv_init_c(const void *sendbuf, MPI_Count sendcount,</code>			25
<code> MPI_Datatype sendtype, void *recvbuf,</code>			26
<code> const MPI_Count recvcunts[], const MPI_Aint displs[],</code>			27
<code> MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,</code>			28
<code> MPI_Request *request)</code>			29
			30
Fortran 2008 binding			31
<code>MPI_Allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs,</code>			32
<code> recvtype, comm, info, request, ierror)</code>			33
<code> TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf</code>			34
<code> INTEGER, INTENT(IN) :: sendcount</code>			35
<code> TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype</code>			36
<code> TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf</code>			37
<code> INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcunts(*), displs(*)</code>			38
<code> TYPE(MPI_Comm), INTENT(IN) :: comm</code>			39
<code> TYPE(MPI_Info), INTENT(IN) :: info</code>			40
<code> TYPE(MPI_Request), INTENT(OUT) :: request</code>			41
<code> INTEGER, OPTIONAL, INTENT(OUT) :: ierror</code>			42
			43
<code>MPI_Allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs,</code>			44
<code> recvtype, comm, info, request, ierror) !(_c)</code>			45
<code> TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf</code>			46
<code> INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount</code>			47
<code> TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype</code>			48
<code> TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf</code>			
<code> INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcunts(*)</code>			

```

1      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
2      TYPE(MPI_Comm), INTENT(IN) :: comm
3      TYPE(MPI_Info), INTENT(IN) :: info
4      TYPE(MPI_Request), INTENT(OUT) :: request
5      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

7      MPI_ALLGATHERV_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
8                          RECVTYPE, COMM, INFO, REQUEST, IERROR)
9      <type> SENDBUF(*), RECVBUF(*)
10     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
11         INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the allgather operation (see Section 6.7).

6.13.6 Persistent All-to-All Scatter/Gather

```

19     MPI_ALLTOALL_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
20                       info, request)

```

21	IN	sendbuf	starting address of send buffer (choice)
22	IN	sendcount	number of elements sent to each MPI process
23			(nonnegative integer)
24	IN	sendtype	datatype of send buffer elements (handle)
25	OUT	recvbuf	address of receive buffer (choice)
26	IN	recvcount	number of elements received from each MPI process
27			(nonnegative integer)
28	IN	recvtype	datatype of receive buffer elements (handle)
29	IN	comm	communicator (handle)
30	IN	info	info argument (handle)
31	OUT	request	communication request (handle)

C binding

```

36     int MPI_Alltoall_init(const void *sendbuf, int sendcount,
37                          MPI_Datatype sendtype, void *recvbuf, int recvcount,
38                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
39                          MPI_Request *request)
40
41     int MPI_Alltoall_init_c(const void *sendbuf, MPI_Count sendcount,
42                          MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
43                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
44                          MPI_Request *request)

```

Fortran 2008 binding

```

46     MPI_Alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
47                       comm, info, request, ierror)
48     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf

```



```

INTEGER, INTENT(IN) :: sendcount, recvcnt
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype,
                  comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcnt
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding
MPI_ALLTOALL_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE,
                  COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
IERROR

Creates a persistent collective communication request for the alltoall operation (see
Section 6.8).

MPI_ALLTOALLV_INIT(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcnts, rdispls,
                  recvtype, comm, info, request)

IN      sendbuf      starting address of send buffer (choice)
IN      sendcounts    nonnegative integer array (of length group size)
                        specifying the number of elements to send to each
                        MPI process
IN      sdispls       integer array (of length group size). Entry j specifies
                        the displacement (relative to sendbuf) from which to
                        take the outgoing data destined for MPI process j
IN      sendtype      datatype of send buffer elements (handle)
OUT     recvbuf       address of receive buffer (choice)
IN      recvcnts      nonnegative integer array (of length group size)
                        specifying the number of elements that can be
                        received from each MPI process
IN      rdispls       integer array (of length group size). Entry i specifies
                        the displacement (relative to recvbuf) at which to
                        place the incoming data from MPI process i
IN      recvtype      datatype of receive buffer elements (handle)
IN      comm          communicator (handle)

```

```

1      IN      info                      info argument (handle)
2
3      OUT     request                  communication request (handle)

```

C binding

```

5  int MPI_Alltoallv_init(const void *sendbuf, const int sendcounts[],
6                          const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
7                          const int recvcounts[], const int rdispls[],
8                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
9                          MPI_Request *request)
10

```

```

11 int MPI_Alltoallv_init_c(const void *sendbuf, const MPI_Count sendcounts[],
12                          const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
13                          const MPI_Count recvcounts[], const MPI_Aint rdispls[],
14                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
15                          MPI_Request *request)
16

```

Fortran 2008 binding

```

17 MPI_Alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
18                   rdispls, recvtype, comm, info, request, ierror)
19
20 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
21 INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
22   recvcounts(*), rdispls(*)
23 TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
24 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
25 TYPE(MPI_Comm), INTENT(IN) :: comm
26 TYPE(MPI_Info), INTENT(IN) :: info
27 TYPE(MPI_Request), INTENT(OUT) :: request
28 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30 MPI_Alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
31                   rdispls, recvtype, comm, info, request, ierror) !(_c)
32 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
33 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
34   recvcounts(*)
35 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
36   rdispls(*)
37 TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
38 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
39 TYPE(MPI_Comm), INTENT(IN) :: comm
40 TYPE(MPI_Info), INTENT(IN) :: info
41 TYPE(MPI_Request), INTENT(OUT) :: request
42 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

43 MPI_ALLTOALLV_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF, RECVCOUNTS,
44                   RDISPLS, RECVTYPE, COMM, INFO, REQUEST, IERROR)
45
46 <type> SENDBUF(*), RECVBUF(*)
47 INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, RECVCOUNTS(*), RDISPLS(*),
48   RECVTYPE, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the alltoallv operation (see Section 6.8).

MPI_ALLTOALLW_INIT(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcounts, rdispls, recvtypes, comm, info, request)

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	integer array (of length group size) specifying the number of elements to send to each MPI process (array of nonnegative integers)
IN	sdispls	integer array (of length group size). Entry j specifies the displacement in bytes (relative to sendbuf) from which to take the outgoing data destined for MPI process j (array of integers)
IN	sendtypes	array of datatypes (of length group size). Entry j specifies the type of data to send to MPI process j (array of handles)
OUT	recvbuf	address of receive buffer (choice)
IN	recvcounts	integer array (of length group size) specifying the number of elements that can be received from each MPI process (array of nonnegative integers)
IN	rdispls	integer array (of length group size). Entry i specifies the displacement in bytes (relative to recvbuf) at which to place the incoming data from MPI process i (array of integers)
IN	recvtypes	array of datatypes (of length group size). Entry i specifies the type of data received from MPI process i (array of handles)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Alltoallw_init(const void *sendbuf, const int sendcounts[],
                      const int sdispls[], const MPI_Datatype sendtypes[],
                      void *recvbuf, const int recvcounts[], const int rdispls[],
                      const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Info info,
                      MPI_Request *request)
```

```
int MPI_Alltoallw_init_c(const void *sendbuf, const MPI_Count sendcounts[],
                        const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                        void *recvbuf, const MPI_Count recvcounts[],
                        const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
                        MPI_Comm comm, MPI_Info info, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
                   recvcounts, rdispls, recvtypes, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
```

```

1      INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
2          recvcounts(*), rdispls(*)
3      TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
4      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
5      TYPE(MPI_Comm), INTENT(IN) :: comm
6      TYPE(MPI_Info), INTENT(IN) :: info
7      TYPE(MPI_Request), INTENT(OUT) :: request
8      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
9
10     MPI_Alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
11         recvcounts, rdispls, recvtypes, comm, info, request, ierror)
12         !(_c)
13     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
14     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
15         recvcounts(*)
16     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
17         rdispls(*)
18     TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
19     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
20     TYPE(MPI_Comm), INTENT(IN) :: comm
21     TYPE(MPI_Info), INTENT(IN) :: info
22     TYPE(MPI_Request), INTENT(OUT) :: request
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

25     MPI_ALLTOALLW_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
26         RECVCOUNTS, RDISPLS, RECVTYPES, COMM, INFO, REQUEST, IERROR)
27     <type> SENDBUF(*), RECVBUF(*)
28     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPES(*), RECVCOUNTS(*), RDISPLS(*),
29         RECVTYPES(*), COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the alltoallw operation (see Section 6.8).

6.13.7 Persistent Reduce

```

37     MPI_REDUCE_INIT(sendbuf, recvbuf, count, datatype, op, root, comm, info, request)

```

IN	sendbuf	address of send buffer (choice)
OUT	recvbuf	address of receive buffer (choice, significant only at root)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of elements of send buffer (handle)
IN	op	reduce operation (handle)
IN	root	rank of the root (integer)
IN	comm	communicator (handle)

IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Reduce_init(const void *sendbuf, void *recvbuf, int count,
                   MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
                   MPI_Info info, MPI_Request *request)

int MPI_Reduce_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                   MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
                   MPI_Info info, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Reduce_init(sendbuf, recvbuf, count, datatype, op, root, comm, info,
               request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  INTEGER, INTENT(IN) :: count, root
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_init(sendbuf, recvbuf, count, datatype, op, root, comm, info,
               request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Op), INTENT(IN) :: op
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_REDUCE_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, ROOT, COMM, INFO,
               REQUEST, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER COUNT, DATATYPE, OP, ROOT, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the reduce operation (see Section 6.9.1).

6.13.8 Persistent All-Reduce

```
MPI_ALLREDUCE_INIT(sendbuf, recvbuf, count, datatype, op, comm, info, request)
```

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	count	number of elements in send buffer (nonnegative integer)
IN	datatype	datatype of elements of send buffer (handle)
IN	op	operation (handle)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Allreduce_init(const void *sendbuf, void *recvbuf, int count,
                      MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
                      MPI_Request *request)
```

```
int MPI_Allreduce_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                        MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
                        MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Allreduce_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
                  ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Allreduce_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
                  ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ALLREDUCE_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, INFO, REQUEST,
                   IERROR)
```

```
<type> SENDBUF(*), RECVBUF(*)
```

```
INTEGER COUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR
```

Creates a persistent collective communication request for the allreduce operation (see Section 6.9.6).

6.13.9 Persistent Reduce-Scatter with Equal Blocks

```
MPI_REDUCE_SCATTER_BLOCK_INIT(sendbuf, recvbuf, recvcnt, datatype, op, comm,
                               info, request)
```

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcnt	element count per block (nonnegative integer)
IN	datatype	datatype of elements of send and receive buffers (handle)
IN	op	operation (handle)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Reduce_scatter_block_init(const void *sendbuf, void *recvbuf,
                                  int recvcnt, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                                  MPI_Info info, MPI_Request *request)
```

```
int MPI_Reduce_scatter_block_init_c(const void *sendbuf, void *recvbuf,
                                     MPI_Count recvcnt, MPI_Datatype datatype, MPI_Op op,
                                     MPI_Comm comm, MPI_Info info, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Reduce_scatter_block_init(sendbuf, recvbuf, recvcnt, datatype, op, comm,
                              info, request, ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
```

```
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
```

```
INTEGER, INTENT(IN) :: recvcnt
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
TYPE(MPI_Op), INTENT(IN) :: op
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
TYPE(MPI_Info), INTENT(IN) :: info
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```

1 MPI_Reduce_scatter_block_init(sendbuf, recvbuf, recvcount, datatype, op, comm,
2                               info, request, ierror) !(_c)
3     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
5     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcount
6     TYPE(MPI_Datatype), INTENT(IN) :: datatype
7     TYPE(MPI_Op), INTENT(IN) :: op
8     TYPE(MPI_Comm), INTENT(IN) :: comm
9     TYPE(MPI_Info), INTENT(IN) :: info
10    TYPE(MPI_Request), INTENT(OUT) :: request
11    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

13 MPI_REDUCE_SCATTER_BLOCK_INIT(SENDBUF, RECVBUF, RECVCOUNT, DATATYPE, OP, COMM,
14                               INFO, REQUEST, IERROR)
15     <type> SENDBUF(*), RECVBUF(*)
16     INTEGER REVCOUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the reduce-scatter with equal blocks operation (see Section 6.10.1).

6.13.10 Persistent Reduce-Scatter

```

24 MPI_REDUCE_SCATTER_INIT(sendbuf, recvbuf, recvcounts, datatype, op, comm, info,
25                          request)
26
27 IN      sendbuf      starting address of send buffer (choice)
28 OUT    recvbuf      starting address of receive buffer (choice)
29 IN      recvcounts   nonnegative integer array specifying the number of
30                      elements in result distributed to each MPI process.
31                      This array must be identical on all calling MPI
32                      processes.
33 IN      datatype     datatype of elements of input buffer (handle)
34 IN      op           operation (handle)
35 IN      comm         communicator (handle)
36 IN      info         info argument (handle)
37 OUT    request      communication request (handle)

```

C binding

```

41 int MPI_Reduce_scatter_init(const void *sendbuf, void *recvbuf,
42                             const int recvcounts[], MPI_Datatype datatype, MPI_Op op,
43                             MPI_Comm comm, MPI_Info info, MPI_Request *request)
44
45 int MPI_Reduce_scatter_init_c(const void *sendbuf, void *recvbuf,
46                               const MPI_Count recvcounts[], MPI_Datatype datatype, MPI_Op op,
47                               MPI_Comm comm, MPI_Info info, MPI_Request *request)
48

```


Fortran 2008 binding

```

MPI_Reduce_scatter_init(sendbuf, recvbuf, recvcnts, datatype, op, comm, info,
    request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcnts(*)
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_scatter_init(sendbuf, recvbuf, recvcnts, datatype, op, comm, info,
    request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcnts(*)
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_REDUCE_SCATTER_INIT(SENDBUF, RECVBUF, RECVCOUNTS, DATATYPE, OP, COMM, INFO,
    REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER REVCOUNTS(*), DATATYPE, OP, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the reduce-scatter operation (see Section 6.10.2).

6.13.11 Persistent Inclusive Scan

```

MPI_SCAN_INIT(sendbuf, recvbuf, count, datatype, op, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)
IN	count	number of elements in input buffer (nonnegative integer)
IN	datatype	datatype of elements of input buffer (handle)
IN	op	operation (handle)
IN	comm	communicator (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Scan_init(const void *sendbuf, void *recvbuf, int count,
                 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
                 MPI_Request *request)

int MPI_Scan_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
                  MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Scan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
             ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
             ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SCAN_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, INFO, REQUEST,
             IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the inclusive scan operation (see Section 6.11.1).

6.13.12 Persistent Exclusive Scan

```

MPI_EXSCAN_INIT(sendbuf, recvbuf, count, datatype, op, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
OUT	recvbuf	starting address of receive buffer (choice)

IN	count	number of elements in input buffer (nonnegative integer)	1
IN	datatype	datatype of elements of input buffer (handle)	2
IN	op	operation (handle)	3
IN	comm	intra-communicator (handle)	4
IN	info	info argument (handle)	5
OUT	request	communication request (handle)	6

C binding

```

int MPI_Exscan_init(const void *sendbuf, void *recvbuf, int count,
                   MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
                   MPI_Request *request)
int MPI_Exscan_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                   MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
                   MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Exscan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
               ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Exscan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
               ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_EXSCAN_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, INFO, REQUEST,
               IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER COUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the exclusive scan operation (see Section 6.11.2).

6.14 Correctness

A correct, portable program must invoke collective communications so that deadlock will not occur, whether collective communications are synchronizing or not. The following examples illustrate dangerous use of collective routines on intra-communicators.

Example 6.26. The following is erroneous.

```
/* ----- THIS EXAMPLE IS ERRONEOUS ----- */
switch(rank) {
  case 0:
    MPI_Bcast(buf1, count, type, 0, comm);
    MPI_Bcast(buf2, count, type, 1, comm);
    break;
  case 1:
    MPI_Bcast(buf2, count, type, 1, comm);
    MPI_Bcast(buf1, count, type, 0, comm);
    break;
}
```

We assume that the group of `comm` is $\{0,1\}$. Two MPI processes execute two broadcast operations in reverse order. If the operation is synchronizing then a deadlock will occur. Collective operations must be executed in the same order at all members of the communication group.

Example 6.27. The following is erroneous.

```
/* ----- THIS EXAMPLE IS ERRONEOUS ----- */
switch(rank) {
  case 0:
    MPI_Bcast(buf1, count, type, 0, comm0);
    MPI_Bcast(buf2, count, type, 2, comm2);
    break;
  case 1:
    MPI_Bcast(buf1, count, type, 1, comm1);
    MPI_Bcast(buf2, count, type, 0, comm0);
    break;
  case 2:
    MPI_Bcast(buf1, count, type, 2, comm2);
    MPI_Bcast(buf2, count, type, 1, comm1);
    break;
}
```

Assume that the group of `comm0` is $\{0,1\}$, of `comm1` is $\{1,2\}$ and of `comm2` is $\{2,0\}$. If the broadcast is a synchronizing operation, then there is a cyclic dependency: the broadcast in `comm2` completes only after the broadcast in `comm0`; the broadcast in `comm0` completes only after the broadcast in `comm1`; and the broadcast in `comm1` completes only after the broadcast in `comm2`. Thus, the code will deadlock.

Collective operations must be executed in an order so that no cyclic dependencies occur. Nonblocking collective operations can alleviate this issue.

Example 6.28. The following is erroneous.

```

/* ----- THIS EXAMPLE IS ERRONEOUS ----- */
switch(rank) {
    case 0:
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Send(buf2, count, type, 1, tag, comm);
        break;
    case 1:
        MPI_Recv(buf2, count, type, 0, tag, comm, status);
        MPI_Bcast(buf1, count, type, 0, comm);
        break;
}

```

MPI process with rank 0 executes a broadcast, followed by a blocking send operation. MPI process with rank 1 first executes a blocking receive that matches the send, followed by a broadcast call that matches the broadcast of MPI process with rank 0. This program may deadlock. The broadcast call on MPI process with rank 0 *may* block until MPI process with rank 1 executes the matching broadcast call, so that the send is not executed. MPI process with rank 1 will definitely block on the receive and so, in this case, never executes the broadcast.

The relative order of execution of collective operations and point-to-point operations must be such, so that even if the collective operations and the point-to-point operations are synchronizing, no deadlock will occur.

Example 6.29. An unsafe, nondeterministic program.

```

switch(rank) {
    case 0:
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Send(buf2, count, type, 1, tag, comm);
        break;
    case 1:
        MPI_Recv(buf2, count, type, MPI_ANY_SOURCE, tag, comm, status);
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Recv(buf2, count, type, MPI_ANY_SOURCE, tag, comm, status);
        break;
    case 2:
        MPI_Send(buf2, count, type, 1, tag, comm);
        MPI_Bcast(buf1, count, type, 0, comm);
        break;
}

```

All three MPI processes participate in a broadcast. MPI process with rank 0 sends a message to MPI process with rank 1 after the broadcast, and MPI process with rank 2 sends a message to MPI process with rank 1 before the broadcast. MPI process with rank 1 receives before and after the broadcast, with a wildcard source argument.

Two possible executions of this program, with different matchings of sends and receives, are illustrated in Figure 6.12. Note that the second execution has the peculiar effect that a send executed after the broadcast is received at another node before the broadcast. This example illustrates the fact that one should not rely on collective communication functions to have particular synchronization effects. A program that works correctly only when the

first execution occurs (only when broadcast is synchronizing) is erroneous.

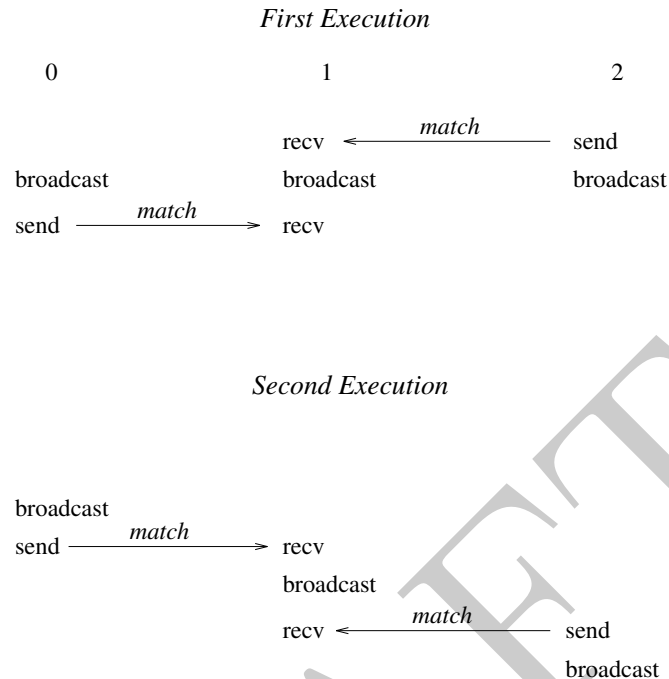


Figure 6.12: A race condition causes nondeterministic matching of sends and receives. One cannot rely on synchronization from a broadcast to make the program deterministic.

Finally, in multithreaded implementations, one can have more than one, concurrently executing, collective communication initialization call at an MPI process. In these situations, it is the user's responsibility to ensure that the same communicator is not used concurrently by two different collective communication initialization calls at the same MPI process. Collective communication initialization calls include all calls for blocking collective operations, all initiation calls for nonblocking collective operations, and all initialization calls for persistent collective operations.

Advice to implementors. Assume that broadcast is implemented using point-to-point MPI communication. Suppose the following two rules are followed.

1. All receives specify their source explicitly (no wildcards).
2. Each MPI process sends all messages that pertain to one collective call before sending any message that pertain to a subsequent collective call.

Then, messages belonging to successive broadcasts cannot be confused, as the order of point-to-point messages is preserved.

It is the implementor's responsibility to ensure that point-to-point messages are not confused with collective messages. One way to accomplish this is, whenever a communicator is created, to also create a "hidden communicator" for collective communication. One could achieve a similar effect more cheaply, for example, by using a hidden tag or context bit to indicate whether the communicator is used for point-to-point or collective communication. (*End of advice to implementors.*)

Example 6.30. Blocking and nonblocking collective operations can be interleaved, i.e., a blocking collective operation can be posted even if there is a nonblocking collective operation outstanding.

```
MPI_Request req;

MPI_Ibarrier(comm, &req);
MPI_Bcast(buf1, count, type, 0, comm);
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Each MPI process starts a nonblocking barrier operation, participates in a blocking broadcast and then waits until every other MPI process started the barrier operation. This effectively turns the broadcast into a synchronizing broadcast with possible communication/communication overlap (**MPI_Bcast** is allowed, but not required to synchronize).

Example 6.31. The starting order of collective operations on a particular communicator defines their matching. The following example shows an erroneous matching of different collective operations on the same communicator.

```
/* ----- THIS EXAMPLE IS ERRONEOUS ----- */
MPI_Request req;
switch(rank) {
    case 0:
        /* erroneous matching */
        MPI_Ibarrier(comm, &req);
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        break;
    case 1:
        /* erroneous matching */
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Ibarrier(comm, &req);
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        break;
}
```

This ordering would match **MPI_Ibarrier** on rank 0 with **MPI_Bcast** on rank 1, which is erroneous and the program behavior is undefined. However, if such an order is required, the user must create different duplicate communicators and perform the operations on them. If started with two MPI processes, the following program would be correct:

```
MPI_Request req;
MPI_Comm dupcomm;
MPI_Comm_dup(comm, &dupcomm);
switch(rank) {
    case 0:
        MPI_Ibarrier(comm, &req);
        MPI_Bcast(buf1, count, type, 0, dupcomm);
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        break;
    case 1:
        MPI_Bcast(buf1, count, type, 0, dupcomm);
        MPI_Ibarrier(comm, &req);
```

```

1      MPI_Wait(&req, MPI_STATUS_IGNORE);
2      break;
3  }

```

Advice to users. The use of different communicators offers some flexibility regarding the matching of nonblocking collective operations. In this sense, communicators could be used as an equivalent to tags. However, communicator construction might induce overheads so that this should be used carefully. (*End of advice to users.*)

Example 6.32. Nonblocking collective operations can rely on the similar progress rules as nonblocking point-to-point operations. Thus, if started with two MPI processes, the following program is a valid MPI program and is guaranteed to terminate:

```

15 MPI_Request req;
16
17 switch(rank) {
18     case 0:
19         MPI_Ibarrier(comm, &req);
20         MPI_Wait(&req, MPI_STATUS_IGNORE);
21         MPI_Send(buf, count, dtype, 1, tag, comm);
22         break;
23     case 1:
24         MPI_Ibarrier(comm, &req);
25         MPI_Recv(buf, count, dtype, 0, tag, comm, MPI_STATUS_IGNORE);
26         MPI_Wait(&req, MPI_STATUS_IGNORE);
27         break;
28 }

```

The MPI library must *progress* the barrier in the MPI_Recv call. Thus, the MPI_Wait call in rank 0 will eventually complete, which enables the matching MPI_Send so all calls eventually return.

Example 6.33. Blocking and nonblocking collective operations do not match. The following example is erroneous.

```

34 /* ----- THIS EXAMPLE IS ERRONEOUS ----- */
35 MPI_Request req;
36
37 switch(rank) {
38     case 0:
39         /* erroneous false matching of Alltoall and Ialltoall */
40         MPI_Ialltoall(sbuf, scnt, stype, rbuf, rcnt, rtype, comm, &req);
41         MPI_Wait(&req, MPI_STATUS_IGNORE);
42         break;
43     case 1:
44         /* erroneous false matching of Alltoall and Ialltoall */
45         MPI_Alltoall(sbuf, scnt, stype, rbuf, rcnt, rtype, comm);
46         break;
47 }

```


Example 6.34. Collective and point-to-point requests can be mixed in functions that enable multiple completions. If started with two MPI processes, the following program is valid.

```
MPI_Request reqs[2];

switch(rank) {
    case 0:
        MPI_Ibarrier(comm, &reqs[0]);
        MPI_Send(buf, count, dtype, 1, tag, comm);
        MPI_Wait(&reqs[0], MPI_STATUS_IGNORE);
        break;
    case 1:
        MPI_Irecv(buf, count, dtype, 0, tag, comm, &reqs[0]);
        MPI_Ibarrier(comm, &reqs[1]);
        MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
        break;
}
```

The MPI_Waitall call returns only after the barrier and the receive completed.

Example 6.35. Multiple nonblocking collective operations can be outstanding on a single communicator and match in order.

```
MPI_Request reqs[3];

compute(buf1);
MPI_Ibcast(buf1, count, type, 0, comm, &reqs[0]);
compute(buf2);
MPI_Ibcast(buf2, count, type, 0, comm, &reqs[1]);
compute(buf3);
MPI_Ibcast(buf3, count, type, 0, comm, &reqs[2]);
MPI_Waitall(3, reqs, MPI_STATUSES_IGNORE);
```

Advice to users. Pipelining and double-buffering techniques can efficiently be used to overlap computation and communication. However, having too many outstanding requests might have a negative impact on performance. (*End of advice to users.*)

Advice to implementors. The use of pipelining may generate many outstanding requests. A high-quality hardware-supported implementation with limited resources should be able to fall back to a software implementation if its resources are exhausted. In this way, the implementation could limit the number of outstanding requests only by the available memory. (*End of advice to implementors.*)

Example 6.36. Nonblocking collective operations can also be used to enable simultaneous collective operations on multiple overlapping communicators (see Figure 6.13). The following example is started with three MPI processes and three communicators. The first communicator comm1 includes ranks 0 and 1, comm2 includes ranks 1 and 2, and comm3 spans ranks 0 and 2. It is not possible to perform a blocking collective operation on all communicators because there exists no deadlock-free order to invoke them. However, nonblocking

collective operations can easily be used to achieve this task.

```

1 MPI_Request reqs[2];
2
3
4 switch(rank) {
5     case 0:
6         MPI_Iallreduce(sbuf1, rbuf1, count, dtype, MPI_SUM, comm1, &reqs[0]);
7         MPI_Iallreduce(sbuf3, rbuf3, count, dtype, MPI_SUM, comm3, &reqs[1]);
8         break;
9     case 1:
10        MPI_Iallreduce(sbuf1, rbuf1, count, dtype, MPI_SUM, comm1, &reqs[0]);
11        MPI_Iallreduce(sbuf2, rbuf2, count, dtype, MPI_SUM, comm2, &reqs[1]);
12        break;
13    case 2:
14        MPI_Iallreduce(sbuf2, rbuf2, count, dtype, MPI_SUM, comm2, &reqs[0]);
15        MPI_Iallreduce(sbuf3, rbuf3, count, dtype, MPI_SUM, comm3, &reqs[1]);
16        break;
17 }
18 MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
19

```

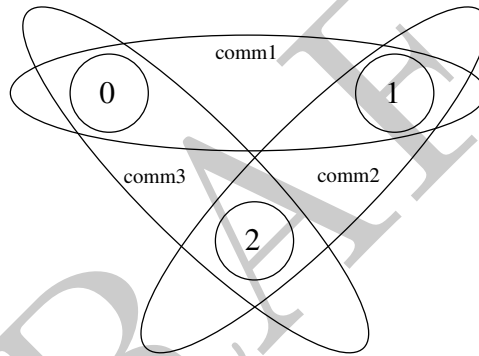


Figure 6.13: Example with overlapping communicators

Advice to users. This method can be useful if overlapping neighboring regions (halo or ghost zones) are used in collective operations. The sequence of the two calls in each MPI process is irrelevant because the two nonblocking operations are performed on different communicators. (*End of advice to users.*)

Example 6.37. The *progress* of multiple outstanding nonblocking collective operations is completely independent.

```

40 MPI_Request reqs[2];
41
42 compute(buf1);
43 MPI_Ibcast(buf1, count, type, 0, comm, &reqs[0]);
44 compute(buf2);
45 MPI_Ibcast(buf2, count, type, 0, comm, &reqs[1]);
46 MPI_Wait(&reqs[1], MPI_STATUS_IGNORE);
47 /* nothing is known about the status of the first bcast here */
48 MPI_Wait(&reqs[0], MPI_STATUS_IGNORE);
49

```

Finishing the second `MPI_IBCAST` is completely independent of the first one. This means that it is not guaranteed that the first broadcast operation is finished or even started after the second one is completed via `reqs[1]`.

DRAFT

DRAFT

Chapter 7

Groups, Contexts, Communicators, and Caching

7.1 Introduction

This chapter introduces MPI features that support the development of parallel libraries. Parallel libraries are needed to encapsulate the distracting complications inherent in parallel implementations of key algorithms. They help to ensure consistent correctness of such procedures, and provide a “higher level” of portability than MPI itself can provide. As such, libraries prevent each programmer from repeating the work of defining consistent data structures, data layouts, and methods that implement key algorithms (such as matrix operations). Since the best libraries come with several variations on parallel systems (different data layouts, different strategies depending on the size of the system or problem, or type of floating point), this too needs to be hidden from the user.

We refer the reader to [5] and [64] for further information on writing libraries in MPI, using the features described in this chapter.

7.1.1 Features Needed to Support Libraries

The key features needed to support the creation of robust parallel libraries are as follows:

- Safe communication space, that guarantees that libraries can communicate as they need to, without conflicting with communication extraneous to the library,
- Group scope for collective operations, that allow libraries to avoid unnecessarily synchronizing uninvolved MPI processes (potentially running unrelated code),
- Abstract naming of MPI processes to allow libraries to describe their communication in terms suitable to their own data structures and algorithms,
- The ability to “adorn” a set of communicating MPI processes with additional user-defined attributes, such as extra collective operations. This mechanism should provide a means for the user or library writer effectively to extend a message-passing notation.

In addition, a unified mechanism or object is needed for conveniently denoting communication context, the group of communicating MPI processes, to house abstract naming of MPI processes, and to store adornments.

7.1.2 MPI’s Support for Libraries

The corresponding concepts that MPI provides, specifically to support robust libraries, are as follows:

- **Contexts** of communication,
- **Groups** of MPI processes,
- **Virtual topologies**,
- **Attribute caching**,
- **Communicators**.

Communicators (see [23, 62, 65]) encapsulate all of these ideas in order to provide the appropriate scope for all communication operations in MPI. Communicators are divided into two kinds: intra-communicators for operations within a single group of MPI processes and inter-communicators for operations between two groups of MPI processes.

Caching. Communicators (see below) provide a “caching” mechanism that allows one to associate new attributes with communicators, on par with MPI built-in features. This can be used by advanced users to adorn communicators further, and by MPI to implement some communicator functions. For example, the virtual-topology functions described in Chapter 8 are likely to be supported this way.

Groups. Groups define an ordered collection of MPI processes, each with a rank, and it is this group that defines the low-level names (ranks) for communication. Thus, groups define a scope for MPI process names in point-to-point communication. In addition, groups define the scope of collective operations. Groups may be manipulated separately from communicators in MPI, but only communicators can be used in communication operations.

Intra-Communicators. The most commonly used means for message-passing in MPI is via intra-communicators. Intra-communicators contain an instance of a group, contexts of communication for both point-to-point and collective communication, and the ability to include virtual topology and other attributes. These features work as follows:

- **Contexts** provide the ability to have separate safe “universes” of message-passing in MPI. A context is akin to an additional tag that differentiates messages. The system manages this differentiation process. The use of separate communication contexts by distinct libraries (or distinct library invocations) insulates communication internal to the library execution from external communication. This allows the invocation of the library even if there are *pending* communication operations or *decoupled MPI activities* on “other” communicators, and avoids the need to synchronize entry or exit into library code. *Pending* communication or *decoupled MPI activities* of point-to-point operations are also guaranteed not to interfere with collective communication operations within a single communicator.
- **Groups** define the participants in the communication (see above) of a communicator.
- A **virtual topology** defines a special mapping of the MPI processes ranks in a group to and from a topology. Special constructors for communicators are defined in Chapter 8 to provide this feature. Intra-communicators as described in this chapter do not have topologies.
- **Attributes** define the local information that the user or library has added to a communicator for later reference.

Advice to users. The practice in many communication libraries is that there is a unique, predefined communication universe that includes all MPI processes available when the parallel program is initiated; the MPI processes are assigned consecutive ranks. Participants in a point-to-point communication are identified by their rank; a collective communication (such as broadcast) always involves all MPI processes. When using the World Model (Section 11.2), this practice can be followed in MPI by using the predefined communicator `MPI_COMM_WORLD`. (*End of advice to users.*)

Inter-Communicators. The discussion has dealt so far with **intra-communication**: communication within a group. MPI also supports **inter-communication**: communication between two nonoverlapping groups. When an application is built by composing several parallel modules, it is convenient to allow one module to communicate with another using local ranks for addressing within the second module. This is especially convenient in a client-server computing paradigm, where either client or server are parallel. The support of inter-communication also provides a mechanism for the extension of MPI to a dynamic model where not all MPI processes are preallocated at initialization time. In such a situation, it becomes necessary to support communication across “universes.” Inter-communication is supported by objects called **inter-communicators**. These objects bind two groups together with communication contexts shared by both groups. For inter-communicators, these features work as follows:

- Contexts provide the ability to have a separate safe “universe” of message-passing between the two groups. A send operation in the local group is always matched by a receive operation in the remote group, and vice versa. The system manages this differentiation process. The use of separate communication contexts by distinct libraries (or distinct library invocations) insulates communication internal to the library execution from external communication. This allows the invocation of the library even if there are *pending* communication operations or *decoupled MPI activities* on “other” communicators, and avoids the need to synchronize entry or exit into library code.
- A local and remote group specify the recipients and destinations for an inter-communicator.
- Virtual topology is undefined for an inter-communicator.
- As before, attributes cache defines the local information that the user or library has added to a communicator for later reference.

MPI provides mechanisms for creating and manipulating inter-communicators. They are used for point-to-point and collective communication in a related manner to intra-communicators. Users who do not need inter-communication in their applications can safely ignore this extension. Users who require inter-communication between overlapping groups must layer this capability on top of MPI.

7.2 Basic Concepts

In this section, we turn to a more formal definition of the concepts introduced above.

7.2.1 Groups

A **group** is an ordered set of MPI process identifiers (henceforth MPI processes); MPI processes are implementation-dependent objects. Each MPI process in a group is associated with an integer **rank**. Ranks are consecutive and start from zero. Groups are represented by opaque **group objects**, and hence cannot be directly transferred from one MPI process to another. A group is used within a communicator to describe the participants in a communication “universe” and to rank such participants (thus giving them unique names within that “universe” of communication).

There is a special pre-defined group: `MPI_GROUP_EMPTY`, which is a group with no members. The predefined constant `MPI_GROUP_NULL` is the value used for invalid group handles.

Advice to users. `MPI_GROUP_EMPTY`, which is a valid handle to an empty group, should not be confused with `MPI_GROUP_NULL`, which in turn is an invalid handle. The former may be used as an argument to group procedures; the latter is not a valid input value for an input argument. (*End of advice to users.*)

Advice to implementors. Simple implementations of MPI will enumerate groups, such as in a table. However, more advanced data structures make sense in order to improve scalability and memory usage with large numbers of MPI processes. Such implementations are possible with MPI. (*End of advice to implementors.*)

7.2.2 Contexts

A **context** is a property of communicators (defined next) that allows partitioning of the communication space. A message sent in one context cannot be received in another context. Furthermore, where permitted, collective operations are independent of *pending* point-to-point operations and *decoupled MPI activities* of point-to-point operations. Contexts are not explicit MPI objects; they appear only as part of the realization of communicators (below).

Advice to implementors. Distinct communicators in the same MPI process have distinct contexts. A context is essentially a system-managed tag (or tags) needed to make a communicator safe for point-to-point and MPI-defined collective communication. Safety means that collective and point-to-point communication within one communicator do not interfere, and that communication over distinct communicators do not interfere.

A possible implementation for a context is as a supplemental tag attached to messages on send and matched on receive. Each intra-communicator stores the value of its two tags (one for point-to-point and one for collective communication). Communicator-generating functions use a collective communication to agree on a new group-wide unique context.

Analogously, in inter-communication, two context tags are stored per communicator, one used by group A to send and group B to receive, and a second used by group B to send and for group A to receive.

Since contexts are not explicit objects, other implementations are also possible. (*End of advice to implementors.*)

7.2.3 Intra-Communicators

Intra-communicators bring together the concepts of group and context. To support implementation-specific optimizations, and application topologies (defined in the next chapter, Chapter 8), communicators may also “cache” additional information (see Section 7.7). MPI communication operations reference communicators to determine the scope and the “communication universe” in which a point-to-point or collective operation is to operate.

Each communicator contains a group of valid participants; this group always includes the local MPI process. The source and destination of a message are identified by MPI process ranks within that group.

For collective communication, the intra-communicator specifies the set of MPI processes that participate in the collective operation (and their order, when significant). Thus, the communicator restricts the “spatial” scope of communication, and provides machine-independent MPI process addressing through ranks.

Intra-communicators are represented by opaque **intra-communicator objects**, and hence cannot be directly transferred from one MPI process to another.

7.2.4 Predefined Intra-Communicators

When using the World Model (Section 11.2) for MPI initialization, an initial intra-communicator `MPI_COMM_WORLD` of all MPI processes the local MPI process can communicate with after initialization (itself included) is defined once `MPI_INIT` or `MPI_INIT_THREAD` has been called. In addition, the communicator `MPI_COMM_SELF` is provided, which includes only the MPI process itself. When using the Sessions Model (Section 11.3) for initialization of MPI resources, `MPI_COMM_WORLD` and `MPI_COMM_SELF` are not valid for use as a communicator. See the discussion concerning use of MPI named constants in 2.5.4 for valid uses of `MPI_COMM_WORLD` and `MPI_COMM_SELF` prior to initialization of MPI. See also the discussion concerning interoperability of the World Model and Sessions Model in Section 11.1.

The predefined constant `MPI_COMM_NULL` is the value used for invalid communicator handles.

In a static-process-model implementation of MPI, all MPI processes that participate in the computation are available after MPI is initialized. For this case, `MPI_COMM_WORLD` is a communicator of all MPI processes available for the computation; this communicator has the same value in all MPI processes. In an implementation of MPI where MPI processes can dynamically join an MPI execution, it may be the case that an MPI process starts an MPI computation without having access to all other MPI processes. In such situations, `MPI_COMM_WORLD` is a communicator incorporating all MPI processes with which the joining MPI process can immediately communicate. Therefore, `MPI_COMM_WORLD` may simultaneously represent disjoint groups in different MPI processes.

All MPI implementations are required to provide the `MPI_COMM_WORLD` communicator. It cannot be deallocated during the life of an MPI process. The group corresponding to this communicator does not appear as a pre-defined constant, but it may be accessed using `MPI_COMM_GROUP` (see below). MPI does not specify the correspondence between the MPI process rank in `MPI_COMM_WORLD` and its (machine-dependent) absolute address. Other implementation-dependent, predefined communicators may also be provided.

7.3 Group Management

This section describes the manipulation of MPI process groups. These operations are local.

7.3.1 Group Accessors

MPI_GROUP_SIZE(group, size)

IN	group	group (handle)
OUT	size	number of MPI processes in the group (integer)

C binding

```
int MPI_Group_size(MPI_Group group, int *size)
```

Fortran 2008 binding

```
MPI_Group_size(group, size, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GROUP_SIZE(GROUP, SIZE, IERROR)
  INTEGER GROUP, SIZE, IERROR
```

MPI_GROUP_RANK(group, rank)

IN	group	group (handle)
OUT	rank	rank of the calling MPI process in group, or MPI_UNDEFINED if the MPI process is not a member (integer)

C binding

```
int MPI_Group_rank(MPI_Group group, int *rank)
```

Fortran 2008 binding

```
MPI_Group_rank(group, rank, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(OUT) :: rank
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GROUP_RANK(GROUP, RANK, IERROR)
  INTEGER GROUP, RANK, IERROR
```

```

MPI_GROUP_TRANSLATE_RANKS(group1, n, ranks1, group2, ranks2)
    IN      group1      group1 (handle)
    IN      n            number of elements in ranks1 and ranks2 arrays
                        (integer)
    IN      ranks1       array of zero or more valid ranks in group1
    IN      group2       group2 (handle)
    OUT     ranks2       array of corresponding ranks in group2,
                        MPI_UNDEFINED when no correspondence exists.

```

C binding

```

int MPI_Group_translate_ranks(MPI_Group group1, int n, const int ranks1[],
                             MPI_Group group2, int ranks2[])

```

Fortran 2008 binding

```

MPI_Group_translate_ranks(group1, n, ranks1, group2, ranks2, ierror)
    TYPE(MPI_Group), INTENT(IN) :: group1, group2
    INTEGER, INTENT(IN) :: n, ranks1(n)
    INTEGER, INTENT(OUT) :: ranks2(n)
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GROUP_TRANSLATE_RANKS(GROUP1, N, RANKS1, GROUP2, RANKS2, IERROR)
    INTEGER GROUP1, N, RANKS1(*), GROUP2, RANKS2(*), IERROR

```

This function is important for determining the relative numbering of the same MPI processes in two different groups. For instance, if one knows the ranks of certain MPI processes in the group of `MPI_COMM_WORLD`, one might want to know their ranks in a subset of that group.

`MPI_PROC_NULL` is a valid rank for input to `MPI_GROUP_TRANSLATE_RANKS`, which returns `MPI_PROC_NULL` as the translated rank.

```

MPI_GROUP_COMPARE(group1, group2, result)
    IN      group1      first group (handle)
    IN      group2      second group (handle)
    OUT     result       result (integer)

```

C binding

```

int MPI_Group_compare(MPI_Group group1, MPI_Group group2, int *result)

```

Fortran 2008 binding

```

MPI_Group_compare(group1, group2, result, ierror)
    TYPE(MPI_Group), INTENT(IN) :: group1, group2
    INTEGER, INTENT(OUT) :: result
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GROUP_COMPARE(GROUP1, GROUP2, RESULT, IERROR)
    INTEGER GROUP1, GROUP2, RESULT, IERROR

```

MPI_IDENT results if the group members and group order are exactly the same in both groups. This happens for instance if `group1` and `group2` are the same handle. MPI_SIMILAR results if the group members are the same but the order is different. MPI_UNEQUAL results otherwise.

7.3.2 Group Constructors

MPI provides two approaches to constructing groups. In the first approach, MPI procedures are provided to subset and superset existing groups. These constructors construct new groups from existing groups. In the second approach, a group is created using a session handle and associated process set. This second approach is available when using the Sessions Model. With both approaches, these are local operations, and distinct groups may be defined on different MPI processes; an MPI process may also define a group that does not include itself. Consistent definitions are required when groups are used as arguments in communicator creation functions. When using the World Model (Section 11.2) for MPI initialization, the base group, upon which all other groups are defined, is the group associated with the initial communicator `MPI_COMM_WORLD` (accessible through the function `MPI_COMM_GROUP`).

Rationale. In what follows, there is no group duplication function analogous to `MPI_COMM_DUP`, defined later in this chapter. There is no need for a group duplicator. A group, once created, can have several references to it by making copies of the handle. The following constructors address the need for subsets and supersets of existing groups. (*End of rationale.*)

Advice to implementors. Each group constructor behaves as if it returned a new group object. When this new group is a copy of an existing group, then one can avoid creating such new objects, using a reference-count mechanism. (*End of advice to implementors.*)

`MPI_COMM_GROUP(comm, group)`

IN	<code>comm</code>	communicator (handle)
OUT	<code>group</code>	group corresponding to <code>comm</code> (handle)

C binding

```
int MPI_Comm_group(MPI_Comm comm, MPI_Group *group)
```

Fortran 2008 binding

```
MPI_Comm_group(comm, group, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Group), INTENT(OUT) :: group
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_GROUP(COMM, GROUP, IERROR)
  INTEGER COMM, GROUP, IERROR
```

`MPI_COMM_GROUP` returns in `group` a handle to the group of `comm`.

```
MPI_GROUP_UNION(group1, group2, newgroup)
```

```
IN      group1      first group (handle)
IN      group2      second group (handle)
OUT     newgroup     union group (handle)
```

C binding

```
int MPI_Group_union(MPI_Group group1, MPI_Group group2, MPI_Group *newgroup)
```

Fortran 2008 binding

```
MPI_Group_union(group1, group2, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group1, group2
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GROUP_UNION(GROUP1, GROUP2, NEWGROUP, IERROR)
  INTEGER GROUP1, GROUP2, NEWGROUP, IERROR
```

```
MPI_GROUP_INTERSECTION(group1, group2, newgroup)
```

```
IN      group1      first group (handle)
IN      group2      second group (handle)
OUT     newgroup     intersection group (handle)
```

C binding

```
int MPI_Group_intersection(MPI_Group group1, MPI_Group group2,
  MPI_Group *newgroup)
```

Fortran 2008 binding

```
MPI_Group_intersection(group1, group2, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group1, group2
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GROUP_INTERSECTION(GROUP1, GROUP2, NEWGROUP, IERROR)
  INTEGER GROUP1, GROUP2, NEWGROUP, IERROR
```

```
MPI_GROUP_DIFFERENCE(group1, group2, newgroup)
```

```
IN      group1      first group (handle)
IN      group2      second group (handle)
OUT     newgroup     difference group (handle)
```

C binding

```
int MPI_Group_difference(MPI_Group group1, MPI_Group group2,
  MPI_Group *newgroup)
```

Fortran 2008 binding

```

MPI_Group_difference(group1, group2, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group1, group2
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GROUP_DIFFERENCE(GROUP1, GROUP2, NEWGROUP, IERROR)
  INTEGER GROUP1, GROUP2, NEWGROUP, IERROR

```

The set-like operations are defined as follows:

union: All elements of the first group (`group1`), followed by all elements of second group (`group2`) not in the first group.

intersect: All elements of the first group that are also in the second group, ordered as in the first group.

difference: All elements of the first group that are not in the second group, ordered as in the first group.

Note that for these operations the order of MPI processes in the output group is determined primarily by order in the first group (if possible) and then, if necessary, by order in the second group. Neither union nor intersection are commutative, but both are associative. The new group can be empty, that is, equal to `MPI_GROUP_EMPTY`.

```

MPI_GROUP_INCL(group, n, ranks, newgroup)

```

IN	group	group (handle)
IN	n	number of elements in array <code>ranks</code> (and size of <code>newgroup</code>) (integer)
IN	ranks	ranks of processes in <code>group</code> to appear in <code>newgroup</code> (array of integers)
OUT	newgroup	new group derived from above, in the order defined by <code>ranks</code> (handle)

C binding

```

int MPI_Group_incl(MPI_Group group, int n, const int ranks[],
  MPI_Group *newgroup)

```

Fortran 2008 binding

```

MPI_Group_incl(group, n, ranks, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: n, ranks(n)
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GROUP_INCL(GROUP, N, RANKS, NEWGROUP, IERROR)
  INTEGER GROUP, N, RANKS(*), NEWGROUP, IERROR

```

The function `MPI_GROUP_INCL` creates a group `newgroup` that consists of the `n` MPI processes in `group` with ranks `ranks[0], ..., ranks[n-1]`; the MPI process with rank `i` in `newgroup` is the MPI process with rank `ranks[i]` in `group`. Each of the `n` elements of `ranks` must be a valid rank in `group` and all elements must be distinct, or else the program is erroneous. If `n = 0`, then `newgroup` is `MPI_GROUP_EMPTY`. This function can, for instance, be used to reorder the elements of a group. See also `MPI_GROUP_COMPARE`.

`MPI_GROUP_EXCL(group, n, ranks, newgroup)`

IN	<code>group</code>	group (handle)
IN	<code>n</code>	number of elements in array <code>ranks</code> (integer)
IN	<code>ranks</code>	array of integer ranks of MPI processes in <code>group</code> not to appear in <code>newgroup</code>
OUT	<code>newgroup</code>	new group derived from above, preserving the order defined by <code>group</code> (handle)

C binding

```
int MPI_Group_excl(MPI_Group group, int n, const int ranks[],
                  MPI_Group *newgroup)
```

Fortran 2008 binding

```
MPI_Group_excl(group, n, ranks, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: n, ranks(n)
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GROUP_EXCL(GROUP, N, RANKS, NEWGROUP, IERROR)
  INTEGER GROUP, N, RANKS(*), NEWGROUP, IERROR
```

The function `MPI_GROUP_EXCL` creates a group of MPI processes `newgroup` that is obtained by deleting from `group` those MPI processes with ranks `ranks[0], ..., ranks[n-1]`. The ordering of MPI processes in `newgroup` is identical to the ordering in `group`. Each of the `n` elements of `ranks` must be a valid rank in `group` and all elements must be distinct; otherwise, the program is erroneous. If `n = 0`, then `newgroup` is identical to `group`.

`MPI_GROUP_RANGE_INCL(group, n, ranges, newgroup)`

IN	<code>group</code>	group (handle)
IN	<code>n</code>	number of triplets in array <code>ranges</code> (integer)
IN	<code>ranges</code>	a one-dimensional array of integer triplets, of the form (first rank, last rank, stride) indicating ranks in <code>group</code> of MPI processes to be included in <code>newgroup</code>
OUT	<code>newgroup</code>	new group derived from above, in the order defined by <code>ranges</code> (handle)

C binding

```
int MPI_Group_range_incl(MPI_Group group, int n, int ranges[][3],
                        MPI_Group *newgroup)
```

Fortran 2008 binding

```
MPI_Group_range_incl(group, n, ranges, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: n, ranges(3, n)
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GROUP_RANGE_INCL(GROUP, N, RANGES, NEWGROUP, IERROR)
  INTEGER GROUP, N, RANGES(3, *), NEWGROUP, IERROR
```

If `ranges` consists of the triplets

$$(first_1, last_1, stride_1), \dots, (first_n, last_n, stride_n)$$

then `newgroup` consists of the sequence of MPI processes in `group` with ranks

$$first_1, first_1 + stride_1, \dots, first_1 + \left\lfloor \frac{last_1 - first_1}{stride_1} \right\rfloor stride_1, \dots,$$

$$first_n, first_n + stride_n, \dots, first_n + \left\lfloor \frac{last_n - first_n}{stride_n} \right\rfloor stride_n.$$

Each computed rank must be a valid rank in `group` and all computed ranks must be distinct, or else the program is erroneous. Note that we may have $first_i > last_i$, and $stride_i$ may be negative, but cannot be zero.

The functionality of this routine is specified to be equivalent to expanding the array of `ranges` to an array of the included ranks and passing the resulting array of ranks and other arguments to `MPI_GROUP_INCL`. A call to `MPI_GROUP_INCL` is equivalent to a call to `MPI_GROUP_RANGE_INCL` with each rank i in `ranks` replaced by the triplet $(i, i, 1)$ in the argument `ranges`.

```
MPI_GROUP_RANGE_EXCL(group, n, ranges, newgroup)
```

IN	group	group (handle)
IN	n	number of triplets in array <code>ranges</code> (integer)
IN	ranges	a one-dimensional array of integer triplets, of the form (first rank, last rank, stride) indicating ranks in <code>group</code> of MPI processes to be excluded from the output group <code>newgroup</code> (array of integers)
OUT	newgroup	new group derived from above, preserving the order in <code>group</code> (handle)

C binding

```
int MPI_Group_range_excl(MPI_Group group, int n, int ranges[][3],
                        MPI_Group *newgroup)
```


Fortran 2008 binding

```

MPI_Group_range_excl(group, n, ranges, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: n, ranges(3, n)
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GROUP_RANGE_EXCL(GROUP, N, RANGES, NEWGROUP, IERROR)
  INTEGER GROUP, N, RANGES(3, *), NEWGROUP, IERROR

```

Each computed rank must be a valid rank in `group` and all computed ranks must be distinct, or else the program is erroneous.

The functionality of this routine is specified to be equivalent to expanding the array of `ranges` to an array of the excluded ranks and passing the resulting array of ranks and other arguments to `MPI_GROUP_EXCL`. A call to `MPI_GROUP_EXCL` is equivalent to a call to `MPI_GROUP_RANGE_EXCL` with each rank `i` in `ranks` replaced by the triplet `(i,i,1)` in the argument `ranges`.

Advice to users. The range operations do not explicitly enumerate ranks, and therefore are more scalable if implemented efficiently. Hence, we recommend MPI programmers to use them whenever possible, as high-quality implementations will take advantage of this fact. (*End of advice to users.*)

Advice to implementors. The range operations should be implemented, if possible, without enumerating the group members, in order to obtain better scalability (time and space). (*End of advice to implementors.*)

```

MPI_GROUP_FROM_SESSION_PSET(session, pset_name, newgroup)

```

IN	session	session (handle)
IN	pset_name	name of process set to use to create the new group (string)
OUT	newgroup	new group derived from supplied session and process set (handle)

C binding

```

int MPI_Group_from_session_pset(MPI_Session session, const char *pset_name,
                               MPI_Group *newgroup)

```

Fortran 2008 binding

```

MPI_Group_from_session_pset(session, pset_name, newgroup, ierror)
  TYPE(MPI_Session), INTENT(IN) :: session
  CHARACTER(LEN=*) , INTENT(IN) :: pset_name
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GROUP_FROM_SESSION_PSET(SESSION, PSET_NAME, NEWGROUP, IERROR)
  INTEGER SESSION, NEWGROUP, IERROR
  CHARACTER*(*) PSET_NAME

```

7.3.3 Group Destructors

INOUT	group	group (handle)
-------	-------	----------------

```
int MPI_Group_free(MPI_Group *group)
```

```
MPI_Group_free(group, ierror)
```

Fortran binding

This operation marks a group object for deallocation. The handle `group` is set to `MPI_GROUP_NULL` by the call. Any on-going operation using this group will complete normally.

Advice to implementors. One can keep a reference count that is incremented for each call to `MPI_COMM_GROUP`, `MPI_COMM_CREATE`, `MPI_COMM_DUP`, `MPI_COMM_IDUP`, `MPI_COMM_DUP_WITH_INFO`, `MPI_COMM_IDUP_WITH_INFO`, `MPI_COMM_SPLIT`, `MPI_COMM_SPLIT_TYPE`, `MPI_COMM_CREATE_GROUP`, `MPI_COMM_CREATE_FROM_GROUP`, `MPI_INTERCOMM_CREATE`, and `MPI_INTERCOMM_CREATE_FROM_GROUPS`, and decremented for each call to `MPI_GROUP_FREE` or `MPI_COMM_FREE`; the group object is ultimately deallocated when the reference count drops to zero. (*End of advice to implementors.*)

7.4 Communicator Management

Advice to implementors. High-quality implementations should amortize the overheads associated with the creation of communicators (for the same group, or subsets thereof) over several calls, by allocating multiple contexts with one collective communication. (*End of advice to implementors.*)

7.4.1 Communicator Accessors

The following are all local operations.

MPI_COMM_SIZE(comm, size)

IN	comm	communicator (handle)
OUT	size	number of MPI processes in the group of comm (integer)

C binding

```
int MPI_Comm_size(MPI_Comm comm, int *size)
```

Fortran 2008 binding

```
MPI_Comm_size(comm, size, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_SIZE(COMM, SIZE, IERROR)
  INTEGER COMM, SIZE, IERROR
```

Rationale. This function is equivalent to accessing the communicator's group with [MPI_COMM_GROUP](#) (see above), computing the size using [MPI_GROUP_SIZE](#), and then freeing the temporary group via [MPI_GROUP_FREE](#). However, this functionality is so commonly used that this shortcut was introduced. (*End of rationale.*)

Advice to users. This function indicates the number of MPI processes involved in a communicator. For [MPI_COMM_WORLD](#), it indicates the total number of MPI processes available unless the number of MPI processes has been changed by using the functions described in Chapter 11; note that the number of MPI processes in [MPI_COMM_WORLD](#) does not change during the life of an MPI program.

This call is often used with the next call to determine the amount of concurrency available for a specific library or program. The following call, [MPI_COMM_RANK](#) indicates the rank of the MPI process that calls it in the range from 0, ..., size-1, where size is the return value of [MPI_COMM_SIZE](#). (*End of advice to users.*)

MPI_COMM_RANK(comm, rank)

IN	comm	communicator (handle)
OUT	rank	rank of the calling MPI process in group of comm (integer)

C binding

```
int MPI_Comm_rank(MPI_Comm comm, int *rank)
```

Fortran 2008 binding

```

MPI_Comm_rank(comm, rank, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(OUT) :: rank
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_RANK(COMM, RANK, IERROR)
    INTEGER COMM, RANK, IERROR

```

Rationale. This function is equivalent to accessing the communicator's group with [MPI_COMM_GROUP](#) (see above), computing the rank using [MPI_GROUP_RANK](#), and then freeing the temporary group via [MPI_GROUP_FREE](#). However, this functionality is so commonly used that this shortcut was introduced. (*End of rationale.*)

Advice to users. This function gives the rank of the MPI process in the particular communicator's group. It is useful, as noted above, in conjunction with [MPI_COMM_SIZE](#).

Many programs will follow the supervisor/executor or manager/worker model, where one MPI process will play a supervisory role while the other MPI processes will play an executory role. In this framework, the two preceding calls are useful for determining the roles of the various MPI processes of a communicator. (*End of advice to users.*)

```

MPI_COMM_COMPARE(comm1, comm2, result)

```

IN	comm1	first communicator (handle)
IN	comm2	second communicator (handle)
OUT	result	result (integer)

C binding

```

int MPI_Comm_compare(MPI_Comm comm1, MPI_Comm comm2, int *result)

```

Fortran 2008 binding

```

MPI_Comm_compare(comm1, comm2, result, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm1, comm2
    INTEGER, INTENT(OUT) :: result
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_COMPARE(COMM1, COMM2, RESULT, IERROR)
    INTEGER COMM1, COMM2, RESULT, IERROR

```

[MPI_IDENT](#) results if and only if comm1 and comm2 are handles for the same object (identical groups and same contexts). [MPI_CONGRUENT](#) results if the underlying groups are identical in constituents and rank order; these communicators differ only by context. [MPI_SIMILAR](#) results if the group members of both communicators are the same but the rank order differs. [MPI_UNEQUAL](#) results otherwise.

7.4.2 Communicator Constructors

The following are collective functions that are invoked by all MPI processes in the group or groups associated with `comm`, with the exception of `MPI_COMM_CREATE_GROUP`, `MPI_COMM_CREATE_FROM_GROUP`, and `MPI_INTERCOMM_CREATE_FROM_GROUPS`. `MPI_COMM_CREATE_GROUP` and `MPI_COMM_CREATE_FROM_GROUP` are invoked only by the MPI processes in the group of the new communicator being constructed. `MPI_INTERCOMM_CREATE_FROM_GROUPS` is invoked by all the MPI processes in the local and remote groups of the new communicator being constructed. See the discussion below for the definition of local and remote groups.

Rationale. Note that, when using the World Model, there is a chicken-and-egg aspect to MPI in that a communicator is needed to create a new communicator. In the World Model, the base communicator for all MPI communicators is predefined outside of MPI, and is `MPI_COMM_WORLD`. The World Model was arrived at after considerable debate, and was chosen to increase “safety” of programs written in MPI. (*End of rationale.*)

This chapter presents the following communicator construction routines: `MPI_COMM_CREATE`, `MPI_COMM_DUP`, `MPI_COMM_IDUP`, `MPI_COMM_DUP_WITH_INFO`, `MPI_COMM_IDUP_WITH_INFO`, `MPI_COMM_SPLIT` and `MPI_COMM_SPLIT_TYPE` can be used to create both intra-communicators and inter-communicators; `MPI_COMM_CREATE_GROUP`, `MPI_COMM_CREATE_FROM_GROUP` and `MPI_INTERCOMM_MERGE` can be used to create intra-communicators; `MPI_INTERCOMM_CREATE` and `MPI_INTERCOMM_CREATE_FROM_GROUPS` can be used to create inter-communicators (see Section 7.6.2).

An intra-communicator involves a single group while an inter-communicator involves two groups. Where the following discussions address inter-communicator semantics, the two groups in an inter-communicator are called the *left* and *right* groups. An MPI process in an inter-communicator is a member of either the left or the right group. From the point of view of that MPI process, the group that the MPI process is a member of is called the *local group*; the other group (relative to that MPI process) is the *remote group*. The left and right group labels give us a way to describe the two groups in an inter-communicator that is not relative to any particular MPI process (as the local and remote groups are).

`MPI_COMM_DUP(comm, newcomm)`

IN	<code>comm</code>	communicator (handle)
OUT	<code>newcomm</code>	copy of <code>comm</code> (handle)

C binding

```
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm)
```

Fortran 2008 binding

```
MPI_Comm_dup(comm, newcomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_COMM_DUP(COMM, NEWCOMM, IERROR)
    INTEGER COMM, NEWCOMM, IERROR

```

MPI_COMM_DUP duplicates the existing communicator `comm` with associated key values, topology information and error handlers. For each key value, the respective copy callback function determines the attribute value associated with this key in the new communicator; one particular action that a copy callback may take is to delete the attribute from the new communicator. **MPI_COMM_DUP** returns in `newcomm` a new communicator with the same group or groups, same topology, same error handlers and any copied cached information, but a new context (see Section 7.7.1). The newly created communicator will have no buffer attached (see Section 3.6).

Advice to users. This operation is used to provide a parallel library with a duplicate communication space that has the same properties as the original communicator. This includes any attributes (see below) and topologies (see Chapter 8). This call is valid even if there are *pending* point-to-point communication operations or *decoupled* MPI activities involving the communicator `comm`. A typical call might involve a **MPI_COMM_DUP** at the beginning of the parallel call, and an **MPI_COMM_FREE** of that duplicated communicator at the end of the call. Other models of communicator management are also possible.

This call applies to both intra- and inter-communicators. (*End of advice to users.*)

Advice to implementors. One need not actually copy the group information, but only add a new reference and increment the reference count. Copy on write can be used for the cached information. (*End of advice to implementors.*)

```

MPI_COMM_DUP_WITH_INFO(comm, info, newcomm)
    IN      comm      communicator (handle)
    IN      info      info object (handle)
    OUT     newcomm    copy of comm (handle)

```

C binding

```
int MPI_Comm_dup_with_info(MPI_Comm comm, MPI_Info info, MPI_Comm *newcomm)
```

Fortran 2008 binding

```

MPI_Comm_dup_with_info(comm, info, newcomm, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Comm), INTENT(OUT) :: newcomm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_DUP_WITH_INFO(COMM, INFO, NEWCOMM, IERROR)
    INTEGER COMM, INFO, NEWCOMM, IERROR

```

MPI_COMM_DUP_WITH_INFO behaves exactly as **MPI_COMM_DUP** except that the hints provided by the argument `info` are associated with the output communicator `newcomm`.

Rationale. It is expected that some hints will only be valid at communicator creation time. However, for legacy reasons, most communicator creation calls do not provide an info argument. One may associate info hints with a duplicate of any communicator at creation time through a call to `MPI_COMM_DUP_WITH_INFO`. (*End of rationale.*)

`MPI_COMM_IDUP(comm, newcomm, request)`

IN	comm	communicator (handle)
OUT	newcomm	copy of comm (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Comm_idup(MPI_Comm comm, MPI_Comm *newcomm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Comm_idup(comm, newcomm, request, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT), ASYNCHRONOUS :: newcomm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_IDUP(COMM, NEWCOMM, REQUEST, IERROR)
  INTEGER COMM, NEWCOMM, REQUEST, IERROR
```

`MPI_COMM_IDUP` is a nonblocking variant of `MPI_COMM_DUP`. With the exception of its nonblocking behavior, the semantics of `MPI_COMM_IDUP` are as if `MPI_COMM_DUP` was executed at the time that `MPI_COMM_IDUP` is called. For example, attributes changed after `MPI_COMM_IDUP` will not be copied to the new communicator. All restrictions and assumptions for nonblocking collective operations (see Section 6.12) apply to `MPI_COMM_IDUP` and the returned request.

It is erroneous to use the communicator `newcomm` as an input argument to other MPI functions before the `MPI_COMM_IDUP` operation completes.

`MPI_COMM_IDUP_WITH_INFO(comm, info, newcomm, request)`

IN	comm	communicator (handle)
IN	info	info object (handle)
OUT	newcomm	copy of comm (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Comm_idup_with_info(MPI_Comm comm, MPI_Info info, MPI_Comm *newcomm,
  MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Comm_idup_with_info(comm, info, newcomm, request, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
```



```

1  TYPE(MPI_Info), INTENT(IN) :: info
2  TYPE(MPI_Comm), INTENT(OUT), ASYNCHRONOUS :: newcomm
3  TYPE(MPI_Request), INTENT(OUT) :: request
4  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

6  MPI_COMM_IDUP_WITH_INFO(COMM, INFO, NEWCOMM, REQUEST, IERROR)
7  INTEGER COMM, INFO, NEWCOMM, REQUEST, IERROR

```

`MPI_COMM_IDUP_WITH_INFO` is a nonblocking variant of `MPI_COMM_DUP_WITH_INFO`. With the exception of its nonblocking behavior, the semantics of `MPI_COMM_IDUP_WITH_INFO` are as if `MPI_COMM_DUP_WITH_INFO` was executed at the time that `MPI_COMM_IDUP_WITH_INFO` is called. For example, attributes or info hints changed after `MPI_COMM_IDUP_WITH_INFO` will not be copied to the new communicator. All restrictions and assumptions for nonblocking collective operations (see Section 6.12) apply to `MPI_COMM_IDUP_WITH_INFO` and the returned request.

It is erroneous to use the communicator `newcomm` as an input argument to other MPI functions before the `MPI_COMM_IDUP_WITH_INFO` operation completes.

Rationale. The `MPI_COMM_IDUP` and `MPI_COMM_IDUP_WITH_INFO` functions are crucial for the development of purely nonblocking libraries (see [41]). (*End of rationale.*)

```

25 MPI_COMM_CREATE(comm, group, newcomm)
26
27 IN      comm      communicator (handle)
28 IN      group     group, which is a subset of the group of comm
29                      (handle)
30 OUT     newcomm    new communicator (handle)

```

C binding

```

33 int MPI_Comm_create(MPI_Comm comm, MPI_Group group, MPI_Comm *newcomm)

```

Fortran 2008 binding

```

36 MPI_Comm_create(comm, group, newcomm, ierror)
37 TYPE(MPI_Comm), INTENT(IN) :: comm
38 TYPE(MPI_Group), INTENT(IN) :: group
39 TYPE(MPI_Comm), INTENT(OUT) :: newcomm
40 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

42 MPI_COMM_CREATE(COMM, GROUP, NEWCOMM, IERROR)
43 INTEGER COMM, GROUP, NEWCOMM, IERROR

```

If `comm` is an intra-communicator, this function returns a new communicator `newcomm` with communication group defined by the `group` argument. No cached information propagates from `comm` to `newcomm` and no virtual topology information is added to the created communicator. Each MPI process must call `MPI_COMM_CREATE` with a `group` argument that

is a subgroup of the group associated with `comm`; this could be `MPI_GROUP_EMPTY`. The MPI processes may specify different values for the `group` argument. If an MPI process calls with a nonempty `group` then all MPI processes in that `group` must call the function with the same `group` as argument, that is the same MPI processes in the same order. Otherwise, the call is erroneous. This implies that the set of groups specified across the MPI processes must be disjoint. If the calling MPI process is a member of the group given as `group` argument, then `newcomm` is a communicator with `group` as its associated group. In the case that an MPI process calls with a `group` to which it does not belong, e.g., `MPI_GROUP_EMPTY`, then `MPI_COMM_NULL` is returned as `newcomm`. The function is collective and must be called by all MPI processes in the group of `comm`.

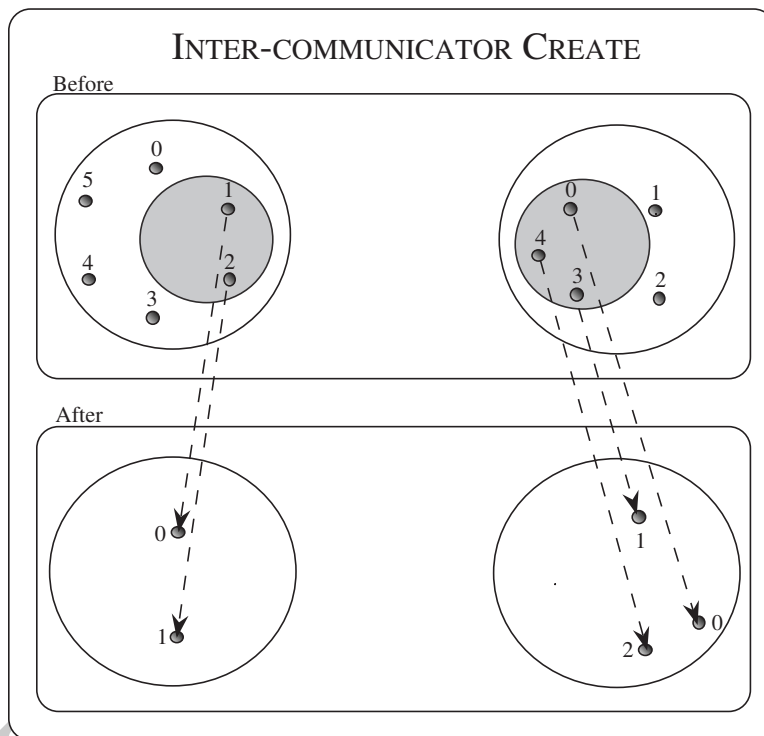


Figure 7.1: Inter-communicator creation using `MPI_COMM_CREATE` extended to inter-communicators. The input groups are those in the grey circle.

Rationale. The interface supports the original mechanism from MPI-1.1, which required the same `group` in all MPI processes of `comm`. It was extended in MPI-2.2 to allow the use of disjoint subgroups in order to allow implementations to eliminate unnecessary communication that `MPI_COMM_SPLIT` would incur when the user already knows the membership of the disjoint subgroups. (*End of rationale.*)

Rationale. The requirement that the entire group of `comm` participate in the call stems from the following considerations:

- It allows the implementation to layer `MPI_COMM_CREATE` on top of regular collective communications.
- It provides additional safety, in particular in the case where partially overlapping groups are used to create new communicators.

- It permits implementations to sometimes avoid communication related to context creation.

(*End of rationale.*)

Advice to users. `MPI_COMM_CREATE` provides a means to subset a group of MPI processes for the purpose of separate MIMD computation, with separate communication space. `newcomm`, which emerges from `MPI_COMM_CREATE`, can be used in subsequent calls to `MPI_COMM_CREATE` (or other communicator constructors) to further subdivide a computation into parallel sub-computations. A more general service is provided by `MPI_COMM_SPLIT`, below. (*End of advice to users.*)

Advice to implementors. When calling `MPI_COMM_DUP`, all MPI processes call with the same `group` (the `group` associated with the communicator). When calling `MPI_COMM_CREATE`, the MPI processes provide the same `group` or disjoint sub-groups. For both calls, it is theoretically possible to agree on a group-wide unique context with no communication. However, local execution of these functions requires use of a larger context name space and reduces error checking. Implementations may strike various compromises between these conflicting goals, such as bulk allocation of multiple contexts in one collective operation.

Important: If new communicators are created without synchronizing the MPI processes involved then the communication system must be able to cope with messages arriving in a context that has not yet been allocated at the receiving MPI process. (*End of advice to implementors.*)

If `comm` is an inter-communicator, then the output communicator is also an inter-communicator where the local group consists only of those MPI processes contained in `group` (see Figure 7.1). The `group` argument should only contain those MPI processes in the local group of the input inter-communicator that are to be a part of `newcomm`. All MPI processes in the same local group of `comm` must specify the same value for `group`, i.e., the same members in the same order. If either `group` does not specify at least one MPI process in the local group of the inter-communicator, or if the calling MPI process is not included in the `group`, `MPI_COMM_NULL` is returned.

Rationale. In the case where either the left or right group is empty, a null communicator is returned instead of an inter-communicator with `MPI_GROUP_EMPTY` because the side with the empty group must return `MPI_COMM_NULL`. (*End of rationale.*)

Example 7.1. Inter-communicator creation.

The following example illustrates how the first node in the left side of an inter-communicator could be joined with all members on the right side of an inter-communicator to form a new inter-communicator.

```
MPI_Comm inter_comm, new_inter_comm;
MPI_Group local_group, group;
int rank = 0; /* rank on left side to include in
               new inter-comm */

/* Construct the original inter-communicator: "inter_comm" */
...
```

```

/* Construct the group of MPI processes to be in new
   inter-communicator */
if (/* I'm on the left side of the inter-communicator */) {
    MPI_Comm_group(inter_comm, &local_group);
    MPI_Group_incl(local_group, 1, &rank, &group);
    MPI_Group_free(&local_group);
}
else
    MPI_Comm_group(inter_comm, &group);

MPI_Comm_create(inter_comm, group, &new_inter_comm);
MPI_Group_free(&group);

```

MPI_COMM_CREATE_GROUP(comm, group, tag, newcomm)

IN	comm	intra-communicator (handle)
IN	group	group, which is a subset of the group of comm (handle)
IN	tag	tag (integer)
OUT	newcomm	new communicator (handle)

C binding

```
int MPI_Comm_create_group(MPI_Comm comm, MPI_Group group, int tag,
                          MPI_Comm *newcomm)
```

Fortran 2008 binding

```

MPI_Comm_create_group(comm, group, tag, newcomm, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Group), INTENT(IN) :: group
    INTEGER, INTENT(IN) :: tag
    TYPE(MPI_Comm), INTENT(OUT) :: newcomm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_CREATE_GROUP(COMM, GROUP, TAG, NEWCOMM, IERROR)
    INTEGER COMM, GROUP, TAG, NEWCOMM, IERROR

```

MPI_COMM_CREATE_GROUP is similar to **MPI_COMM_CREATE**; however, **MPI_COMM_CREATE** must be called by all MPI processes in the group of **comm**, whereas **MPI_COMM_CREATE_GROUP** must be called by all MPI processes in **group**, which is a subgroup of the group of **comm**. In addition, **MPI_COMM_CREATE_GROUP** requires that **comm** is an intra-communicator. **MPI_COMM_CREATE_GROUP** returns a new intra-communicator, **newcomm**, for which the **group** argument defines the communication group. No cached information propagates from **comm** to **newcomm** and no virtual topology information is added to the created communicator. Each MPI process must provide a **group** argument that is a subgroup of the group associated with **comm**; this could be **MPI_GROUP_EMPTY**. If a nonempty group is specified, then all MPI processes in that group must call the function, and each of these MPI processes must provide the same arguments,

including a group that contains the same members with the same ordering. Otherwise the call is erroneous. If the calling MPI process is a member of the group given as the `group` argument, then `newcomm` is a communicator with `group` as its associated group. If the calling MPI process is not a member of `group`, e.g., `group` is `MPI_GROUP_EMPTY`, then the call is a local operation and `MPI_COMM_NULL` is returned as `newcomm`.

Rationale. Functionality similar to `MPI_COMM_CREATE_GROUP` can be implemented through repeated `MPI_INTERCOMM_CREATE` and `MPI_INTERCOMM_MERGE` calls that start with the `MPI_COMM_SELF` communicators at each MPI process in `group` and build up an intra-communicator with group `group` [18]. Such an algorithm requires the creation of many intermediate communicators; `MPI_COMM_CREATE_GROUP` can provide a more efficient implementation that avoids this overhead. (*End of rationale.*)

Advice to users. An inter-communicator can be created collectively over MPI processes in the union of the local and remote groups by creating the local communicator using `MPI_COMM_CREATE_GROUP` and using that communicator as the local communicator argument to `MPI_INTERCOMM_CREATE`. (*End of advice to users.*)

The `tag` argument does not conflict with tags used in point-to-point communication and is not permitted to be a wildcard. If multiple threads at a given MPI process perform concurrent `MPI_COMM_CREATE_GROUP` operations, the user must distinguish these operations by providing different `tag` or `comm` arguments.

Advice to users. `MPI_COMM_CREATE` may provide lower overhead than `MPI_COMM_CREATE_GROUP` because it can take advantage of collective communication on `comm` when constructing `newcomm`. (*End of advice to users.*)

`MPI_COMM_SPLIT(comm, color, key, newcomm)`

IN	<code>comm</code>	communicator (handle)
IN	<code>color</code>	control of subset assignment (integer)
IN	<code>key</code>	control of rank assignment (integer)
OUT	<code>newcomm</code>	new communicator (handle)

C binding

```
int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm *newcomm)
```

Fortran 2008 binding

```
MPI_Comm_split(comm, color, key, newcomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: color, key
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_SPLIT(COMM, COLOR, KEY, NEWCOMM, IERROR)
  INTEGER COMM, COLOR, KEY, NEWCOMM, IERROR
```

This function partitions the group associated with `comm` into disjoint subgroups, one for each value of `color`. Each subgroup contains all MPI processes of the same color. Within each subgroup, the MPI processes are ranked in the order defined by the value of the argument `key`, with ties broken according to their rank in the old group. A new communicator is created for each subgroup and returned in `newcomm`. An MPI process may supply the color value `MPI_UNDEFINED`, in which case `newcomm` returns `MPI_COMM_NULL`. This is a collective call, but each MPI process is permitted to provide different values for `color` and `key`. No cached information propagates from `comm` to `newcomm` and no virtual topology information is added to the created communicators.

With an intra-communicator `comm`, a call to `MPI_COMM_CREATE(comm, group, newcomm)` is equivalent to a call to `MPI_COMM_SPLIT(comm, color, key, newcomm)`, where MPI processes that are members of their `group` argument provide a `color` argument equal to the number of the `group` (based on a unique numbering of all disjoint groups) and a `key` argument equal to their rank in `group`, and all MPI processes that are not members of their `group` argument provide a `color` argument equal to `MPI_UNDEFINED`. The value of `color` must be nonnegative or `MPI_UNDEFINED`.

Advice to users. This is an extremely powerful mechanism for dividing a single communicating group of MPI processes into k subgroups, with k chosen implicitly by the user (by the number of colors asserted over all the MPI processes). Each resulting communicator will be nonoverlapping. Such a division could be useful for defining a hierarchy of computations, such as for multigrid, or linear algebra. For intra-communicators, `MPI_COMM_SPLIT` provides similar capability as `MPI_COMM_CREATE` to split a communicating group into disjoint subgroups. `MPI_COMM_SPLIT` is useful when some MPI processes do not have complete information of the other members in their group, but all MPI processes know (the color of) the group to which they belong. In this case, the MPI implementation discovers the other group members via communication. `MPI_COMM_CREATE` is useful when all MPI processes have complete information of the members of their group. In this case, MPI can avoid the extra communication required to discover group membership. `MPI_COMM_CREATE_GROUP` is useful when all MPI processes in a given group have complete information of the members of their group and synchronization with MPI processes outside the group can be avoided.

Multiple calls to `MPI_COMM_SPLIT` can be used to overcome the requirement that any call have no overlap of the resulting communicators (each MPI process shall belong to only one color per call). In this way, multiple overlapping communication structures can be created. Creative use of the `color` and `key` in such splitting operations is encouraged.

Note that, for a fixed color, the keys need not be unique. It is `MPI_COMM_SPLIT`'s responsibility to sort MPI processes in ascending order according to this key, and to break ties in a consistent way. If all the keys are specified in the same way, then all the MPI processes in a given color will have the relative rank order as they did in their parent group.

(End of advice to users.)

Rationale. `color` is restricted to be nonnegative, so as not to conflict with the value assigned to `MPI_UNDEFINED`. *(End of rationale.)*

The result of `MPI_COMM_SPLIT` on an inter-communicator is that those MPI processes on the left with the same `color` as those MPI processes on the right combine to create a new

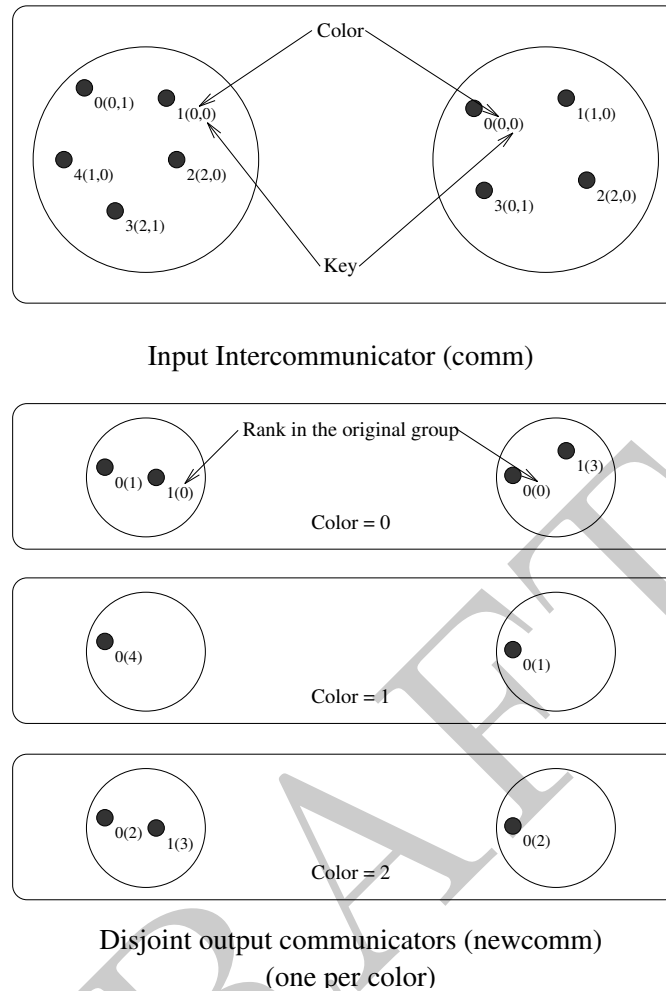


Figure 7.2: Inter-communicator construction achieved by splitting an existing inter-communicator with `MPI_COMM_SPLIT` extended to inter-communicators

inter-communicator. The key argument describes the relative rank of MPI processes on each side of the inter-communicator (see Figure 7.2). For those colors that are specified only on one side of the inter-communicator, `MPI_COMM_NULL` is returned. `MPI_COMM_NULL` is also returned to those MPI processes that specify `MPI_UNDEFINED` as the color.

Advice to users. For inter-communicators, `MPI_COMM_SPLIT` is more general than `MPI_COMM_CREATE`. A single call to `MPI_COMM_SPLIT` can create a set of disjoint inter-communicators, while a call to `MPI_COMM_CREATE` creates only one. (*End of advice to users.*)

Example 7.2. Parallel client-server model.

The following client code illustrates how clients on the left side of an inter-communicator could be assigned to a single server from a pool of servers on the right side of an inter-communicator.

```
/* Client code */
MPI_Comm multiple_server_comm;
```

```

MPI_Comm  single_server_comm;
int        color, rank, num_servers;

/* Create inter-communicator with clients and servers:
   multiple_server_comm */
...

/* Find out the number of servers available */
MPI_Comm_remote_size(multiple_server_comm, &num_servers);

/* Determine my color */
MPI_Comm_rank(multiple_server_comm, &rank);
color = rank % num_servers;

/* Split the inter-communicator */
MPI_Comm_split(multiple_server_comm, color, rank,
               &single_server_comm);

```

The following is the corresponding server code:

```

/* Server code */
MPI_Comm  multiple_client_comm;
MPI_Comm  single_server_comm;
int        rank;

/* Create inter-communicator with clients and servers:
   multiple_client_comm */
...

/* Split the inter-communicator for a single server per group
   of clients */
MPI_Comm_rank(multiple_client_comm, &rank);
MPI_Comm_split(multiple_client_comm, rank, 0,
               &single_server_comm);

```

MPI_COMM_SPLIT_TYPE(comm, split_type, key, info, newcomm)

IN	comm	communicator (handle)
IN	split_type	type of processes to be grouped together (integer)
IN	key	control of rank assignment (integer)
INOUT	info	info argument (handle)
OUT	newcomm	new communicator (handle)

C binding

```

int MPI_Comm_split_type(MPI_Comm comm, int split_type, int key, MPI_Info info,
                        MPI_Comm *newcomm)

```

Fortran 2008 binding

```

MPI_Comm_split_type(comm, split_type, key, info, newcomm, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm

```



```

1      INTEGER, INTENT(IN) :: split_type, key
2      TYPE(MPI_Info), INTENT(IN) :: info
3      TYPE(MPI_Comm), INTENT(OUT) :: newcomm
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

6      MPI_COMM_SPLIT_TYPE(COMM, SPLIT_TYPE, KEY, INFO, NEWCOMM, IERROR)
7      INTEGER COMM, SPLIT_TYPE, KEY, INFO, NEWCOMM, IERROR

```

This function partitions the group associated with `comm` into disjoint subgroups such that each subgroup contains all MPI processes in the same grouping referred to by `split_type`. Within each subgroup, the MPI processes are ranked in the order defined by the value of the argument `key`, with ties broken according to their rank in the old group. A new communicator is created for each subgroup and returned in `newcomm`. This is a collective call. All MPI processes in the group associated with `comm` must provide the same `split_type`, but each MPI process is permitted to provide different values for `key`. An exception to this rule is that an MPI process may supply the type value `MPI_UNDEFINED`, in which case `MPI_COMM_NULL` is returned in `newcomm` for such MPI process. No cached information propagates from `comm` to `newcomm` and no virtual topology information is added to the created communicators.

For `split_type`, the following values are defined by MPI:

MPI_COMM_TYPE_SHARED: all MPI processes in the group of `newcomm` are part of the same *shared memory domain* and can create a *shared memory segment* (e.g., with a successful call to `MPI_WIN_ALLOCATE_SHARED`). This segment can subsequently be used for load/store accesses by all MPI processes in `newcomm`.

Advice to users. Since the location of some of the MPI processes may change during the application execution, the communicators created with the value `MPI_COMM_TYPE_SHARED` before this change may not reflect an actual ability to share memory between MPI processes after this change. (*End of advice to users.*)

MPI_COMM_TYPE_HW_GUIDED: this value specifies that the communicator `comm` is split according to a **hardware resource type** (for example a computing core or an L3 cache) specified by the "mpi_hw_resource_type" info key. Each output communicator `newcomm` corresponds to a single instance of the specified hardware resource type. The MPI processes in the group associated with the output communicator `newcomm` utilize that specific hardware resource type instance, and no other instance of the same hardware resource type.

If an MPI process does not meet the above criteria, then `MPI_COMM_NULL` is returned in `newcomm` for such MPI process.

`MPI_COMM_NULL` is also returned in `newcomm` in the following cases:

- `MPI_INFO_NULL` is provided.
- The info handle does not include the key "mpi_hw_resource_type".
- The MPI implementation neither recognizes nor supports the info key "mpi_hw_resource_type".
- The MPI implementation does not recognize the value associated with the info key "mpi_hw_resource_type".

The MPI implementation will return in the group of the output communicator `newcomm` the largest subset of MPI processes that match the splitting criterion.

The MPI processes in the group associated with `newcomm` are ranked in the order defined by the value of the argument `key` with ties broken according to their rank in the group associated with `comm`.

Advice to users. The set of hardware resources that an MPI process is able to utilize may change during the application execution (e.g., because of the relocation of an MPI process), in which case the communicators created with the value `MPI_COMM_TYPE_HW_GUIDED` before this change may not reflect the utilization of hardware resources of such MPI process at any time after the communicator creation. (*End of advice to users.*)

The user explicitly constrains with the `info` argument the splitting of the input communicator `comm`. To this end, the info key `"mpi_hw_resource_type"` is reserved and its associated value is an implementation-defined string designating the type of the requested hardware. It is strongly recommended that these strings follow the URI format described in Section 9.1.2 (e.g., `"hwloc://NUMANode"`, `"hwloc://Package"` or `"hwloc://L3Cache"`).

The value `"mpi_shared_memory"` is reserved and its use is equivalent to using `MPI_COMM_TYPE_SHARED` for the `split_type` parameter.

Rationale. The value `"mpi_shared_memory"` is defined in order to ensure consistency between the use of `MPI_COMM_TYPE_SHARED` and the use of `MPI_COMM_TYPE_HW_GUIDED`. (*End of rationale.*)

All MPI processes must provide the same value for the info key `"mpi_hw_resource_type"`.

Example 7.3. Splitting `MPI_COMM_WORLD` into NUMANode subcommunicators.

```
MPI_Info info;
MPI_Comm hwcomm;
int rank;

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Info_create(&info);
MPI_Info_set(info, "mpi_hw_resource_type", "hwloc://NUMANode");
MPI_Comm_split_type(MPI_COMM_WORLD,
                    MPI_COMM_TYPE_HW_GUIDED,
                    rank, info, &hwcomm);
```

MPI_COMM_TYPE_RESOURCE_GUIDED: this value specifies that the communicator `comm` is split according to a **hardware resource type** (for example a computing core or an L3 cache) specified by the `"mpi_hw_resource_type"` info key or to a **logical resource type** (for example a process set name, see Section 11.3.2) specified by the `"mpi_pset_name"` info key.

Each output communicator `newcomm` corresponds to a single instance of the specified resource type. The MPI processes in the group associated with the output communicator `newcomm` utilize that specific resource type instance, and no other instance of the same resource type.

If an MPI process does not meet the above criteria, then `MPI_COMM_NULL` is returned in `newcomm` for such process.

`MPI_COMM_NULL` is also returned in `newcomm` in the following cases:

- `MPI_INFO_NULL` is provided.
- The info handle includes neither the key `"mpi_hw_resource_type"` nor the key `"mpi_pset_name"`.
- The MPI implementation neither recognizes nor supports the info keys `"mpi_hw_resource_type"` and `"mpi_pset_name"`.
- The MPI implementation does not recognize the value associated with the info key `"mpi_hw_resource_type"` or `"mpi_pset_name"`.

The MPI implementation will return in the group of the output communicator `newcomm` the largest subset of MPI processes that match the splitting criterion.

Advice to users. The set of resources that an MPI process is able to utilize may change during the application execution (e.g., because of the relocation of an MPI process), in which case the communicators created with the value `MPI_COMM_TYPE_RESOURCE_GUIDED` before this change may not reflect the utilization of resources of such process at any time after the communicator creation. (*End of advice to users.*)

The user explicitly constrains with the `info` argument the splitting of the input communicator `comm`. To this end, the following info keys are reserved and their associated values are implementation-defined strings designating the type of the requested resource. Only one of these info keys can be used in `info` at a time in a call to `MPI_COMM_SPLIT_TYPE`; use of more than one info key is erroneous.

`"mpi_hw_resource_type"` is used to specify the type of a requested hardware resource (e.g., `"hwloc://NUMANode"`, `"hwloc://Package"` or `"hwloc://L3Cache"`). The value `"mpi_shared_memory"` is reserved and its use is equivalent to using `MPI_COMM_TYPE_SHARED` for the `split_type` parameter.

Rationale. The value `"mpi_shared_memory"` is defined in order to ensure consistency between the use of `MPI_COMM_TYPE_SHARED` and the use of `MPI_COMM_TYPE_RESOURCE_GUIDED`. (*End of rationale.*)

All MPI processes in the group of the input communicator `comm` must provide the same info key to perform the splitting action. All MPI processes in the group of the input communicator `comm` must provide the same value for the info key `"mpi_hw_resource_type"`.

`"mpi_pset_name"` is used to specify the type of a requested logical resource through the utilization of a process set name (e.g., `"app://ocean"` or `"app://atmos"`). This process set name must be valid in the session from which the input communicator `comm` is derived. If this input communicator is not derived from a session, then `MPI_COMM_NULL` is returned in `newcomm`.

All MPI processes that are both in the group of the input communicator `comm` and in the process set identified by the given process set name must provide the same info key to perform the splitting action. All MPI processes that are both in the group of the input communicator `comm` and in the process set identified by

the given process set name must provide the same value for the info key "mpi_pset_name".

Example 7.4. Splitting `MPI_COMM_WORLD` into NUMANode subcommunicators.

```
MPI_Info info;
MPI_Comm hwcomm;
int rank;

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Info_create(&info);
MPI_Info_set(info, "mpi_hw_resource_type", "hwloc://NUMANode");
MPI_Comm_split_type(MPI_COMM_WORLD,
                    MPI_COMM_TYPE_RESOURCE_GUIDED,
                    rank, info, &hwcomm);
```

MPI_COMM_TYPE_HW_UNGUIDED: the group of MPI processes associated with `newcomm` must be a *strict* subset of the group associated with `comm` and each `newcomm` corresponds to a single instance of a **hardware resource type** (for example a computing core or an L3 cache).

All MPI processes in the group associated with `comm` that utilize that specific hardware resource type instance—and no other instance of the same hardware resource type—are included in the group of `newcomm`.

If a given MPI process cannot be a member of a communicator that forms such a strict subset, or does not meet the above criteria, then `MPI_COMM_NULL` is returned in `newcomm` for this process.

Advice to implementors. In a high-quality MPI implementation, the number of different new valid communicators `newcomm` produced by this splitting operation should be minimal unless the user provides a key/value pair that modifies this behavior. The sets of hardware resource types used for the splitting operation are implementation-dependent, but should reflect the hardware of the actual system on which the application is currently executing. (*End of advice to implementors.*)

Rationale. If the hardware resources are hierarchically organized, calling this routine several times using as its input communicator `comm` the output communicator `newcomm` of the previous call creates a sequence of `newcomm` communicators in each MPI process, which exposes a hierarchical view of the hardware platform, as shown in Example 7.5. This sequence of returned `newcomm` communicators may differ from the sets of hardware resource types, as shown in the second splitting operation in Figure 7.3. (*End of rationale.*)

Advice to users. Each output communicator `newcomm` can represent a different hardware resource type (see Figure 7.3 for an example). The set of hardware resources an MPI process utilizes may change during the application execution (e.g., because of MPI process relocation), in which case the communicators created with the value `MPI_COMM_TYPE_HW_UNGUIDED` before this change may not reflect the utilization of hardware resources for such MPI process at any time after the communicator creation. (*End of advice to users.*)

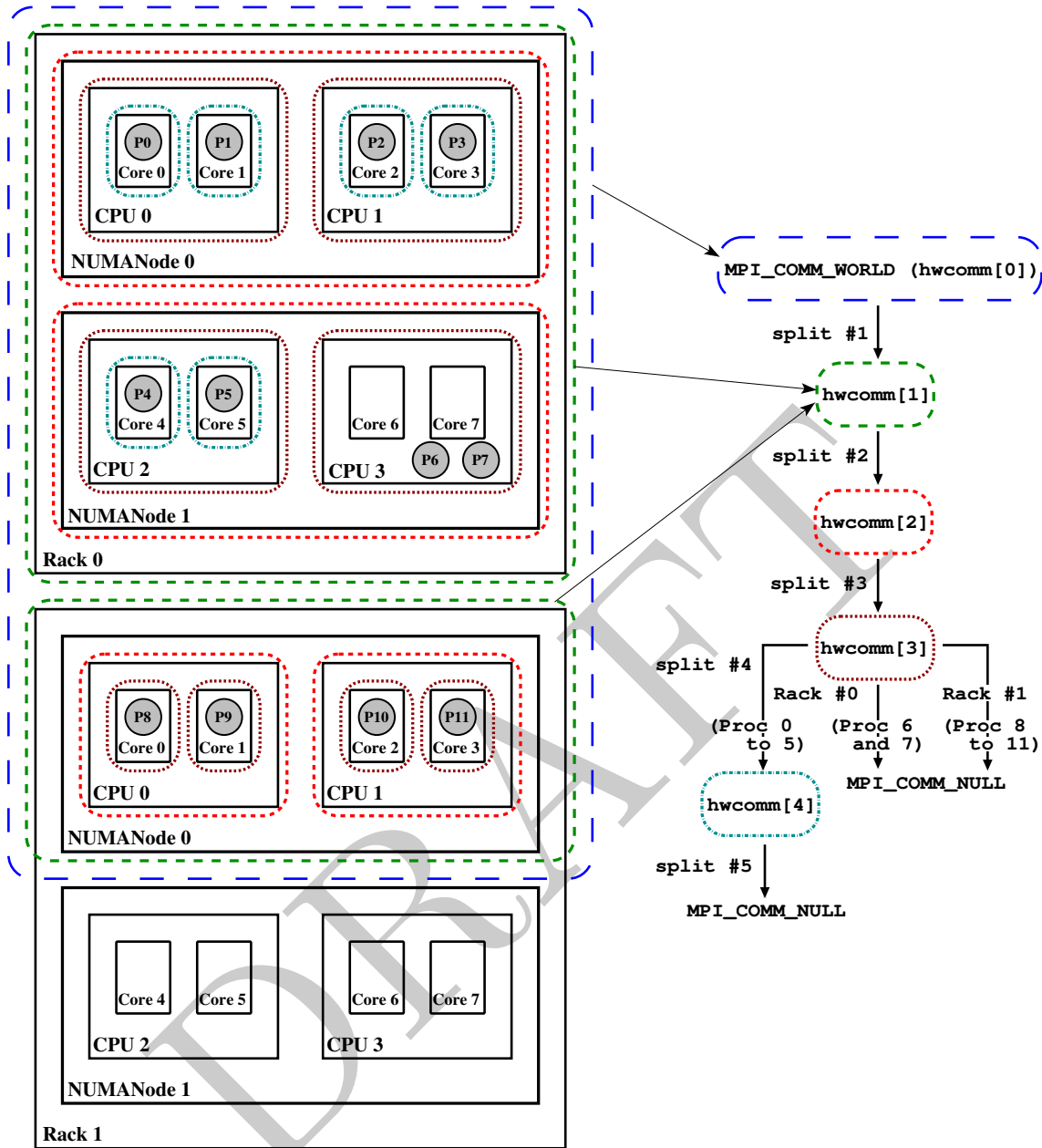


Figure 7.3: Recursive splitting of `MPI_COMM_WORLD` with `MPI_COMM_SPLIT_TYPE` and `MPI_COMM_TYPE_HW_UNGUIDED`. Dashed lines represent communicators whilst solid lines represent hardware resources. MPI processes (P0 to P11) utilize exclusively their respective core, except for P6 and P7, which utilize CPU #3 of Rack #0 and can therefore use Cores #6 and #7 indifferently. The second splitting operation yields two subcommunicators corresponding to NUMANodes in Rack #0 and to CPUs in Rack #1 because Rack #1 features only one NUMANode, which corresponds to the whole portion of the Rack that is included in `MPI_COMM_WORLD` and `hwcomm[1]`. For the first splitting operation, the hardware resource type returned in the info argument is "Rack" on the MPI processes on Rack #0, whereas on Rack #1, it can be either "Rack" or "NUMANode".

If a valid info handle is provided as an argument, the MPI implementation sets the info key "mpi_hw_resource_type" for each MPI process in the group associated with a returned `newcomm` communicator and the info key value is an implementation-defined string that indicates the hardware resource type represented by `newcomm`. The same hardware resource type must be set in all MPI processes in the group associated with `newcomm`.

Example 7.5. Recursive splitting of `MPI_COMM_WORLD`.

```
#define MAX_NUM_LEVELS 32

MPI_Comm hwcomm[MAX_NUM_LEVELS];
int      rank, level_num = 0;

hwcomm[level_num] = MPI_COMM_WORLD;

while((hwcomm[level_num] != MPI_COMM_NULL)
      && (level_num < MAX_NUM_LEVELS-1)){
    MPI_Comm_rank(hwcomm[level_num], &rank);
    MPI_Comm_split_type(hwcomm[level_num],
                        MPI_COMM_TYPE_HW_UNGUIDED,
                        rank,
                        MPI_INFO_NULL,
                        &hwcomm[level_num+1]);
    level_num++;
}
```

Advice to implementors. Implementations can define their own `split_type` values, or use the info argument, to assist in creating communicators that help expose platform-specific information to the application. The concept of hardware-based communicators was first described by Träff [68] for SMP systems. Guided and unguided modes description as well as an implementation path are introduced by Goglin et al. [28]. (*End of advice to implementors.*)

`MPI_COMM_CREATE_FROM_GROUP(group, stringtag, info, errhandler, newcomm)`

IN	group	group (handle)
IN	stringtag	unique identifier for this operation (string)
IN	info	info object (handle)
IN	errhandler	error handler to be attached to new intra-communicator (handle)
OUT	newcomm	new communicator (handle)

C binding

```
int MPI_Comm_create_from_group(MPI_Group group, const char *stringtag,
                               MPI_Info info, MPI_Errhandler errhandler, MPI_Comm *newcomm)
```

Fortran 2008 binding

```

MPI_Comm_create_from_group(group, stringtag, info, errhandler, newcomm, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  CHARACTER(LEN=*), INTENT(IN) :: stringtag
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_CREATE_FROM_GROUP(GROUP, STRINGTAG, INFO, ERRHANDLER, NEWCOMM, IERROR)
  INTEGER GROUP, INFO, ERRHANDLER, NEWCOMM, IERROR
  CHARACTER*(*) STRINGTAG

```

`MPI_COMM_CREATE_FROM_GROUP` is similar to `MPI_COMM_CREATE_GROUP`, except that the set of MPI processes involved in the creation of the new intra-communicator is specified by a `group` argument, rather than the group associated with a pre-existing communicator. If a nonempty `group` is specified, then all MPI processes in that group must call the function and each of these MPI processes must provide the same arguments, including a `group` that contains the same members with the same ordering, and identical `stringtag` value. In the event that `MPI_GROUP_EMPTY` is supplied as the `group` argument, then the call is a local operation and `MPI_COMM_NULL` is returned as `newcomm`. The `stringtag` argument is analogous to the `tag` used for `MPI_COMM_CREATE_GROUP`. If multiple threads at a given MPI process perform concurrent `MPI_COMM_CREATE_FROM_GROUP` operations, the user must distinguish these operations by providing different `stringtag` arguments. The `stringtag` shall not exceed `MPI_MAX_STRINGTAG_LEN` characters in length. For C, this includes space for a null terminating character. `MPI_MAX_STRINGTAG_LEN` shall have a value of at least 63.

The `errhandler` argument specifies an error handler to be attached to the new intra-communicator. Section 9.3 specifies the error handler to be invoked if an error is encountered during the invocation of `MPI_COMM_CREATE_FROM_GROUP`.

The `info` argument provides hints and assertions, possibly MPI implementation dependent, which indicate desired characteristics and guide communicator creation.

Advice to users. The `stringtag` argument is used to distinguish concurrent communicator construction operations issued by different entities. As such, it is important to ensure that this argument is unique for each concurrent call to `MPI_COMM_CREATE_FROM_GROUP`. Reverse domain name notation convention [2] is one approach to constructing unique `stringtag` arguments. See also example 11.10. (*End of advice to users.*)

7.4.3 Communicator Destructors

```

MPI_COMM_FREE(comm)

```

```

  INOUT    comm                      communicator to be destroyed (handle)

```

C binding

```

int MPI_Comm_free(MPI_Comm *comm)

```

Fortran 2008 binding

```

MPI_Comm_free(comm, ierror)
    TYPE(MPI_Comm), INTENT(INOUT) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_FREE(COMM, IERROR)
    INTEGER COMM, IERROR

```

This collective operation marks the communication object for deallocation. Any operations that use the communicator `comm` (whether active or inactive at the time of this procedure call) will continue to work; the object is actually deallocated only if there are no other active references to it. The handle is set to `MPI_COMM_NULL` in the calling MPI process. This call applies to intra- and inter-communicators. The delete callback functions for all cached attributes (see Section 7.7) are called in arbitrary order.

Advice to implementors. Though collective, it is anticipated that this operation will normally be implemented to be local, though a debugging version of an MPI library might choose to synchronize. (*End of advice to implementors.*)

7.4.4 Communicator Info

Hints specified via info (see Chapter 10) allow a user to provide information to direct optimization. Providing hints may enable an implementation to deliver increased performance or minimize use of system resources. As described in Section 10, an implementation is free to ignore all hints; however, applications must comply with any info hints they provide that are used by the MPI implementation (i.e., are returned by a call to `MPI_COMM_GET_INFO`) and that place a restriction on the behavior of the application. Hints are specified on a per communicator basis, in `MPI_COMM_DUP_WITH_INFO`, `MPI_COMM_IDUP_WITH_INFO`, `MPI_COMM_SET_INFO`, `MPI_COMM_SPLIT_TYPE`, `MPI_DIST_GRAPH_CREATE`, and `MPI_DIST_GRAPH_CREATE_ADJACENT`, via the opaque info object. When an info object that specifies a subset of valid hints is passed to `MPI_COMM_SET_INFO`, there will be no effect on previously set or defaulted hints that the info does not specify.

Advice to implementors. It may happen that a program is coded with hints for one system, and later executes on another system that does not support these hints. In general, unsupported hints should simply be ignored. Needless to say, no hint can be mandatory. However, for each hint used by a specific implementation, a default value must be provided when the user does not specify a value for this hint. (*End of advice to implementors.*)

Advice to users. Some optimizations may only be possible when all MPI processes in the group of the communicator provide a given info key with the same value. (*End of advice to users.*)

Info hints are not propagated by MPI from one communicator to another. The following info keys are valid for all communicators.

"mpi_assert_no_any_tag" (boolean, default: "false"): If set to "true", then the implementation may assume that the MPI process will not use the `MPI_ANY_TAG` wildcard on the given communicator.

"mpi_assert_no_any_source" (boolean, default: **"false"**): If set to **"true"**, then the implementation may assume that the MPI process will not use the **MPI_ANY_SOURCE** wildcard on the given communicator.

"mpi_assert_exact_length" (boolean, default: **"false"**): If set to **"true"**, then the implementation may assume that the lengths of messages received by the MPI process are equal to the lengths of the corresponding receive buffers, for point-to-point communication operations on the given communicator.

"mpi_assert_allow_overtaking" (boolean, default: **"false"**): If set to **"true"**, then the implementation may assume that point-to-point communications on the given communicator do not rely on the nonovertaking rule specified in Section 3.5. In other words, the application asserts that send operations are not required to be matched at the receiver in the order in which the send operations were posted by the sender, and receive operations are not required to be matched in the order in which they were posted by the receiver.

Advice to users. Use of the **"mpi_assert_allow_overtaking"** info key can result in nondeterminism in the message matching order. (*End of advice to users.*)

"mpi_assert_strict_persistent_collective_ordering" (boolean, default: **"false"**): If set to **"true"**, then the implementation may assume that all the persistent collective operations are started in the same order across all MPI processes in the group of the communicator. It is required that if this assertion is made on one member of the communicator's group, then it must be made on all members of that communicator's group with the same value.

Advice to users. Use of the **"mpi_assert_strict_persistent_collective_ordering"** may be needed because some optimizations may only be possible on certain systems when strict collective ordering is asserted for the underlying communicator of a persistent collective operation. (*End of advice to users.*)

"mpi_assert_memory_alloc_kinds" (string, not set by default): If set, the implementation may assume that the memory for all communication buffers passed to MPI operations performed by the calling MPI process on the given communicator will use only the memory allocation kinds listed in the value string. See Section 11.4.3.

MPI_COMM_SET_INFO(comm, info)

INOUT	comm	communicator (handle)
IN	info	info object (handle)

C binding

int MPI_Comm_set_info(MPI_Comm comm, MPI_Info info)

Fortran 2008 binding

```

MPI_Comm_set_info(comm, info, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```


Fortran binding

```
MPI_COMM_SET_INFO(COMM, INFO, IERROR)
    INTEGER COMM, INFO, IERROR
```

MPI_COMM_SET_INFO updates the hints of the communicator associated with `comm` using the hints provided in `info`. This operation has no effect on previously set or defaulted hints that are not specified by `info`. It also has no effect on previously set or defaulted hints that are specified by `info`, but are ignored by the MPI implementation in this call to **MPI_COMM_SET_INFO**. **MPI_COMM_SET_INFO** is a collective routine. The `info` object may be different on each MPI process, but any `info` entries that an implementation requires to be the same on all MPI processes must appear with the same value in each MPI process's `info` object.

Advice to users. Some `info` items that an implementation can use when it creates a communicator cannot easily be changed once the communicator has been created. Thus, an implementation may ignore hints issued in this call that it would have accepted in a creation call. An implementation may also be unable to update certain `info` hints in a call to **MPI_COMM_SET_INFO**. **MPI_COMM_GET_INFO** can be used to determine whether updates to existing `info` hints were ignored by the implementation. (*End of advice to users.*)

Advice to users. Setting `info` hints on the predefined communicators **MPI_COMM_WORLD** and **MPI_COMM_SELF** may have unintended effects, as changes to these global objects may affect all components of the application, including libraries and tools. Users must ensure that all components of the application that use a given communicator, including libraries and tools, can comply with any `info` hints associated with that communicator. (*End of advice to users.*)

```
MPI_COMM_GET_INFO(comm, info_used)
```

IN	<code>comm</code>	communicator object (handle)
OUT	<code>info_used</code>	new <code>info</code> object (handle)

C binding

```
int MPI_Comm_get_info(MPI_Comm comm, MPI_Info *info_used)
```

Fortran 2008 binding

```
MPI_Comm_get_info(comm, info_used, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(OUT) :: info_used
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_GET_INFO(COMM, INFO_USED, IERROR)
    INTEGER COMM, INFO_USED, IERROR
```

MPI_COMM_GET_INFO returns a new `info` object containing the hints of the communicator associated with `comm`. The current setting of all hints related to this communicator is returned in `info_used`. An MPI implementation is required to return all hints that are

supported by the implementation and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no key/value pair. The user is responsible for freeing `info_used` via `MPI_INFO_FREE`.

7.5 Motivating Examples

7.5.1 Current Practice #1

Example 7.6. Parallel output of a message

```
int main(int argc, char *argv[])
{
    int me, size;
    ...
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &me);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("MPI process %d size %d\n", me, size);
    ...
    MPI_Finalize();
    return 0;
}
```

Example 7.6 is a do-nothing program that initializes itself, and refers to the “all” communicator, and prints a message. It terminates itself too. This example does not imply that MPI supports `printf`-like communication itself.

Example 7.7. Message exchange (supposing that `size` is even)

```
int main(int argc, char *argv[])
{
    int me, size;
    int SOME_TAG = 0;
    ...
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &me); /* local */
    MPI_Comm_size(MPI_COMM_WORLD, &size); /* local */

    if((me % 2) == 0)
    {
        /* send unless highest-numbered MPI process */
        if((me + 1) < size)
            MPI_Send(..., me + 1, SOME_TAG, MPI_COMM_WORLD);
    }
    else
        MPI_Recv(..., me - 1, SOME_TAG, MPI_COMM_WORLD, &status);
}
```

```

...
MPI_Finalize();
return 0;
}

```

Example 7.7 schematically illustrates message exchanges between “even” and “odd” MPI processes in the “all” communicator.

7.5.2 Current Practice #2

Example 7.8.

```

int main(int argc, char *argv[])
{
    int me, count;
    void *data;
    ...

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &me);

    if(me == 0)
    {
        /* get input, create buffer “data” */
        ...
    }

    MPI_Bcast(data, count, MPI_BYTE, 0, MPI_COMM_WORLD);

    ...
    MPI_Finalize();
    return 0;
}

```

Example 7.8 illustrates the use of a collective communication.

7.5.3 (Approximate) Current Practice #3

Example 7.9.

```

int main(int argc, char *argv[])
{
    int me, count, count2;
    void *send_buf, *recv_buf, *send_buf2, *recv_buf2;
    MPI_Group group_world, grprem;
    MPI_Comm commWorker;
    static int ranks[] = {0};
    ...
    MPI_Init(&argc, &argv);
    MPI_Comm_group(MPI_COMM_WORLD, &group_world);
    MPI_Comm_rank(MPI_COMM_WORLD, &me); /* local */

    MPI_Group_excl(group_world, 1, ranks, &grprem); /* local */
}

```

```

1  MPI_Comm_create(MPI_COMM_WORLD, grpem, &commWorker);
2
3  if(me != 0)
4  {
5      /* compute on worker */
6      ...
7      MPI_Reduce(send_buf, recv_buf, count, MPI_INT, MPI_SUM, 1, commWorker);
8      ...
9      MPI_Comm_free(&commWorker);
10 }
11 /* zero falls through immediately to this reduce, others do later... */
12 MPI_Reduce(send_buf2, recv_buf2, count2,
13             MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
14
15 MPI_Group_free(&group_world);
16 MPI_Group_free(&grpem);
17 MPI_Finalize();
18 return 0;
19 }

```

Example 7.9 illustrates how a group consisting of all but the zeroth MPI process of the “all” group is created, and then how a communicator is formed (`commWorker`) for that new group. The new communicator is used in a collective call, and all MPI processes execute a collective call in the `MPI_COMM_WORLD` context. This example illustrates how the two communicators (that inherently possess distinct contexts) protect communication. That is, communication in `MPI_COMM_WORLD` is insulated from communication in `commWorker`, and vice versa.

In summary, “group safety” is achieved via communicators because distinct contexts within communicators are enforced to be unique on any MPI process.

7.5.4 Communication Safety Example

The following example (7.10) is meant to illustrate “safety” between point-to-point and collective communication. MPI guarantees that a single communicator can do safe point-to-point and collective communication.

Example 7.10.

```

34 #define TAG_ARBITRARY 12345
35 #define SOME_COUNT    50
36
37 int main(int argc, char *argv[])
38 {
39     int me;
40     MPI_Request request[2];
41     MPI_Status status[2];
42     MPI_Group group_world, subgroup;
43     int ranks[] = {2, 4, 6, 8};
44     MPI_Comm the_comm;
45     ...
46     MPI_Init(&argc, &argv);
47     MPI_Comm_group(MPI_COMM_WORLD, &group_world);
48
49     MPI_Group_incl(group_world, 4, ranks, &subgroup); /* local */

```

```

MPI_Group_rank(subgroup, &me);      /* local */

MPI_Comm_create(MPI_COMM_WORLD, subgroup, &the_comm);

if(me != MPI_UNDEFINED)
{
    MPI_Irecv(buff1, count, MPI_DOUBLE, MPI_ANY_SOURCE,
               TAG_ARBITRARY, the_comm, request);
    MPI_Isend(buff2, count, MPI_DOUBLE, (me+1)%4, TAG_ARBITRARY,
               the_comm, request+1);
    for(i = 0; i < SOME_COUNT; i++)
        MPI_Reduce(..., the_comm);
    MPI_Waitall(2, request, status);

    MPI_Comm_free(&the_comm);
}

MPI_Group_free(&group_world);
MPI_Group_free(&subgroup);
MPI_Finalize();
return 0;
}

```

7.5.5 Library Example #1

Example 7.11. First library example
The main program:

```

int main(int argc, char *argv[])
{
    int done = 0;
    user_lib_t *libh_a, *libh_b;
    void *dataset1, *dataset2;
    ...
    MPI_Init(&argc, &argv);
    ...
    init_user_lib(MPI_COMM_WORLD, &libh_a);
    init_user_lib(MPI_COMM_WORLD, &libh_b);
    ...
    user_start_op(libh_a, dataset1);
    user_start_op(libh_b, dataset2);
    ...
    while(!done)
    {
        /* work */
        ...
        MPI_Reduce(..., MPI_COMM_WORLD);
        ...
        /* see if done */
        ...
    }
}

```

```

1  user_end_op(libh_a);
2  user_end_op(libh_b);
3
4  uninit_user_lib(libh_a);
5  uninit_user_lib(libh_b);
6  MPI_Finalize();
7  return 0;
8  }

```

The user library initialization code:

```

10 void init_user_lib(MPI_Comm comm, user_lib_t **handle)
11 {
12     user_lib_t *save;
13
14     user_lib_initsave(&save); /* local */
15     MPI_Comm_dup(comm, &(save->comm));
16
17     /* other inits */
18     ...
19
20     *handle = save;
21 }

```

User start-up code:

```

23 void user_start_op(user_lib_t *handle, void *data)
24 {
25     MPI_Irecv( ..., handle->comm, &(handle->irecv_handle) );
26     MPI_Isend( ..., handle->comm, &(handle->isend_handle) );
27 }

```

User communication clean-up code:

```

29 void user_end_op(user_lib_t *handle)
30 {
31     MPI_Status status;
32     MPI_Wait(&handle->isend_handle, &status);
33     MPI_Wait(&handle->irecv_handle, &status);
34 }

```

User object clean-up code:

```

36 void uninit_user_lib(user_lib_t *handle)
37 {
38     MPI_Comm_free(&(handle->comm));
39     free(handle);
40 }

```

7.5.6 Library Example #2

Example 7.12. Second library example

The main program:

```

47 int main(int argc, char *argv[])
48

```

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
{
    int ma, mb;
    MPI_Group group_world, group_a, group_b;
    MPI_Comm comm_a, comm_b;

    static int list_a[] = {0, 1};
    #if defined(EXAMPLE_2B) || defined(EXAMPLE_2C)
        static int list_b[] = {0, 2, 3};
    #else /* EXAMPLE_2A */
        static int list_b[] = {0, 2};
    #endif
    int size_list_a = sizeof(list_a)/sizeof(int);
    int size_list_b = sizeof(list_b)/sizeof(int);

    ...
    MPI_Init(&argc, &argv);
    MPI_Comm_group(MPI_COMM_WORLD, &group_world);

    MPI_Group_incl(group_world, size_list_a, list_a, &group_a);
    MPI_Group_incl(group_world, size_list_b, list_b, &group_b);

    MPI_Comm_create(MPI_COMM_WORLD, group_a, &comm_a);
    MPI_Comm_create(MPI_COMM_WORLD, group_b, &comm_b);

    if(comm_a != MPI_COMM_NULL)
        MPI_Comm_rank(comm_a, &ma);
    if(comm_b != MPI_COMM_NULL)
        MPI_Comm_rank(comm_b, &mb);

    if(comm_a != MPI_COMM_NULL)
        lib_call(comm_a);

    if(comm_b != MPI_COMM_NULL)
    {
        lib_call(comm_b);
        lib_call(comm_b);
    }

    if(comm_a != MPI_COMM_NULL)
        MPI_Comm_free(&comm_a);
    if(comm_b != MPI_COMM_NULL)
        MPI_Comm_free(&comm_b);
    MPI_Group_free(&group_a);
    MPI_Group_free(&group_b);
    MPI_Group_free(&group_world);
    MPI_Finalize();
    return 0;
}

```

The library:

```

44
45
46
47
48
void lib_call(MPI_Comm comm)
{
    int me, done = 0;

```

```

1  MPI_Status status;
2  MPI_Comm_rank(comm, &me);
3  if(me == 0)
4      while(!done)
5      {
6          MPI_Recv(..., MPI_ANY_SOURCE, MPI_ANY_TAG, comm, &status);
7          ...
8      }
9  else
10 {
11     /* work */
12     MPI_Send(..., 0, ARBITRARY_TAG, comm);
13     ...
14 }
15 #ifdef EXAMPLE_2C
16     /* include (resp, exclude) for safety (resp, no safety): */
17     MPI_Barrier(comm);
18 #endif
19 }

```

The above example is three examples, depending on whether or not one includes rank 3 in `list_b`, and whether or not a synchronizing operation is included in `lib_call`. This example illustrates that, despite contexts, subsequent calls to `lib_call` with the same context need not be safe from one another (colloquially, “back-masking”). Safety is realized if a call to `MPI_Barrier` is added. What this demonstrates is that libraries have to be written carefully, even with contexts. When rank 3 is excluded, then the synchronizing operation is not needed to get safety from back-masking.

Algorithms like “reduce” and “allreduce” have strong enough source selectivity properties so that they are inherently okay (no back-masking), provided that MPI provides basic guarantees. So are multiple calls to a typical tree-broadcast algorithm with the same root or different roots (see [65]). Here we rely on two guarantees of MPI: pairwise ordering of messages between MPI processes in the same context, and source selectivity—deleting either feature removes the guarantee that back-masking cannot be required.

Algorithms that try to do nondeterministic broadcasts or other calls that include wildcard operations will not generally have the good properties of the deterministic implementations of “reduce,” “allreduce,” and “broadcast.” Such algorithms would have to utilize the monotonically increasing tags (within a communicator scope) to keep things straight.

All of the foregoing is a supposition of “collective calls” implemented with point-to-point operations. MPI implementations may or may not implement collective calls using point-to-point operations. These algorithms are used to illustrate the issues of correctness and safety, independent of how MPI implements its collective calls. See also Section 7.9.

7.6 Inter-Communication

This section introduces the concept of inter-communication and describes the portions of MPI that support it. It describes support for writing programs that contain user-level servers.

All communication described thus far has involved communication between MPI processes that are members of the same group. This type of communication is called “**intra-communication**” and the communicator used is called an “**intra-communicator**,” as we

have noted earlier in the chapter.

In modular and multi-disciplinary applications, different MPI process groups execute distinct modules and MPI processes within different modules communicate with one another in a pipeline or a more general module graph. In these applications, the most natural way for an MPI process to specify a target MPI process is by the rank of the target MPI process within the target group. In applications that contain internal user-level servers, each server may be an MPI process group that provides services to one or more clients, and each client may be an MPI process group that uses the services of one or more servers. It is again most natural to specify the target MPI process by rank within the target group in these applications. This type of communication is called “**inter-communication**” and the communicator used is called an “**inter-communicator**,” as introduced earlier.

An **inter-communication** is a point-to-point communication between MPI processes in different groups. The group containing an MPI process that initiates an inter-communication operation is called the “local group,” that is, the sender in a send and the receiver in a receive. The group containing the target MPI process is called the “remote group,” that is, the receiver in a send and the sender in a receive. As in intra-communication, the target MPI process is specified using a (communicator, rank) pair. Unlike intra-communication, the rank is relative to a second, remote group.

All inter-communicator constructors are blocking except for `MPI_COMM_IDUP` and require that the local and remote groups be disjoint.

Advice to users. The groups must be disjoint for several reasons. First, the intent of the inter-communicators is to provide a communicator for communication between disjoint groups. This is reflected in the definition of `MPI_INTERCOMM_MERGE`, which allows the user to control the ranking of the MPI processes in the created intra-communicator; this ranking makes little sense if the groups are not disjoint. In addition, the natural extension of collective operations to inter-communicators makes the most sense when the groups are disjoint. (*End of advice to users.*)

Here is a summary of the properties of inter-communication and inter-communicators:

- The syntax of point-to-point and collective communication is the same for both inter- and intra-communication. The same communicator can be used both for send and for receive operations.
- A target MPI process is addressed by its rank in the remote group, both for sends and for receives.
- Communications using an inter-communicator are guaranteed not to conflict with any communications that use a different communicator.
- A communicator will provide either intra- or inter-communication, never both.

The routine `MPI_COMM_TEST_INTER` may be used to determine if a communicator is an inter- or intra-communicator. Inter-communicators can be used as arguments to some of the other communicator access routines. Inter-communicators cannot be used as input to some of the constructor routines for intra-communicators (for instance, `MPI_CART_CREATE`).

Advice to implementors. For the purpose of point-to-point communication, communicators can be represented in each process by a tuple consisting of:

```

1  group
2  send_context
3
4  receive_context
5
6  source

```

For inter-communicators, *group* describes the remote group, and *source* is the rank of the MPI process in the local group. For intra-communicators, *group* is the communicator group (remote=local), *source* is the rank of the MPI process in this group, and *send context* and *receive context* are identical. A group can be represented by a rank-to-absolute-address translation table.

The inter-communicator cannot be discussed sensibly without considering MPI processes in both the local and remote groups. Imagine an MPI process **P** in group $\mathcal{P}$, which has an inter-communicator C_P , and an MPI process **Q** in group $\mathcal{Q}$, which has an inter-communicator C_Q . Then

- C_P .**group** describes the group $\mathcal{Q}$ and C_Q .**group** describes the group $\mathcal{P}$.
- C_P .**send_context** = C_Q .**receive_context** and the context is unique in $\mathcal{Q}$;
 C_P .**receive_context** = C_Q .**send_context** and this context is unique in $\mathcal{P}$.
- C_P .**source** is rank of **P** in $\mathcal{P}$ and C_Q .**source** is rank of **Q** in $\mathcal{Q}$.

Assume that **P** sends a message to **Q** using the inter-communicator. Then **P** uses the **group** table to find the absolute address of **Q**; **source** and **send_context** are appended to the message.

Assume that **Q** posts a receive with an explicit source argument using the inter-communicator. Then **Q** matches **receive_context** to the message context and source argument to the message source.

The same algorithm is appropriate for intra-communicators as well.

In order to support inter-communicator accessors and constructors, it is necessary to supplement this model with additional structures, that store information about the local communication group, and additional safe contexts. (*End of advice to implementors.*)

7.6.1 Inter-Communicator Accessors

```

37 MPI_COMM_TEST_INTER(comm, flag)

```

```

39   IN      comm      communicator (handle)
40   OUT     flag      true if comm is an inter-communicator (logical)

```

C binding

```

43 int MPI_Comm_test_inter(MPI_Comm comm, int *flag)

```

Fortran 2008 binding

```

46 MPI_Comm_test_inter(comm, flag, ierror)
47   TYPE(MPI_Comm), INTENT(IN) :: comm

```

Table 7.1: MPI_COMM_* function behavior (in inter-communication mode)

MPI_COMM_SIZE	returns the size of the local group.
MPI_COMM_GROUP	returns the local group.
MPI_COMM_RANK	returns the rank in the local group.

LOGICAL, INTENT(OUT) :: flag
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

MPI_COMM_TEST_INTER(COMM, FLAG, IERROR)
 INTEGER COMM, IERROR
 LOGICAL FLAG

This local routine allows the calling MPI process to determine if a communicator is an inter-communicator or an intra-communicator. It returns `true` if it is an inter-communicator, otherwise `false`.

Table 7.1 describes the behavior when an inter-communicator is used as an input argument to the communicator accessors described above under intra-communication. Furthermore, the operation `MPI_COMM_COMPARE` is valid for inter-communicators. Both communicators must be either intra- or inter-communicators, or else `MPI_UNEQUAL` results. Both corresponding local and remote groups must compare correctly to get the results `MPI_CONGRUENT` or `MPI_SIMILAR`. In particular, it is possible for `MPI_SIMILAR` to result because either the local or remote groups were similar but not identical.

The following accessors provide consistent access to the remote group of an inter-communicator. The following are all local operations.

MPI_COMM_REMOTE_SIZE(comm, size)

IN	comm	inter-communicator (handle)
OUT	size	number of MPI processes in the remote group of comm (integer)

C binding

int MPI_Comm_remote_size(MPI_Comm comm, int *size)

Fortran 2008 binding

MPI_Comm_remote_size(comm, size, ierror)
 TYPE(MPI_Comm), INTENT(IN) :: comm
 INTEGER, INTENT(OUT) :: size
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

MPI_COMM_REMOTE_SIZE(COMM, SIZE, IERROR)
 INTEGER COMM, SIZE, IERROR

```

1 MPI_COMM_REMOTE_GROUP(comm, group)
2     IN      comm      inter-communicator (handle)
3
4     OUT     group     remote group corresponding to comm (handle)
5

```

C binding

```

7 int MPI_Comm_remote_group(MPI_Comm comm, MPI_Group *group)
8

```

Fortran 2008 binding

```

9 MPI_Comm_remote_group(comm, group, ierror)
10 TYPE(MPI_Comm), INTENT(IN) :: comm
11 TYPE(MPI_Group), INTENT(OUT) :: group
12 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13

```

Fortran binding

```

15 MPI_COMM_REMOTE_GROUP(COMM, GROUP, IERROR)
16 INTEGER COMM, GROUP, IERROR
17

```

Rationale. Symmetric access to both the local and remote groups of an inter-communicator is important, so this function, as well as [MPI_COMM_REMOTE_SIZE](#) have been provided. (*End of rationale.*)

7.6.2 Inter-Communicator Operations

This section introduces five blocking inter-communicator operations.

[MPI_INTERCOMM_CREATE](#) is used to bind two intra-communicators into an inter-communicator; the function [MPI_INTERCOMM_CREATE_FROM_GROUPS](#) constructs an inter-communicator from two previously defined disjoint groups; the function [MPI_INTERCOMM_MERGE](#) creates an intra-communicator by merging the local and remote groups of an inter-communicator. The functions [MPI_COMM_DUP](#) and [MPI_COMM_FREE](#), introduced previously, duplicate and free an inter-communicator, respectively.

Overlap of local and remote groups that are bound into an inter-communicator is prohibited. If there is overlap, then the program is erroneous and is likely to deadlock.

The function [MPI_INTERCOMM_CREATE](#) can be used to create an inter-communicator from two existing intra-communicators, in the following situation: At least one selected member from each group (the “group leader”) has the ability to communicate with the selected member from the other group; that is, a “peer” communicator exists to which both leaders belong, and each leader knows the rank of the other leader in this peer communicator. Furthermore, members of each group know the rank of their leader.

Construction of an inter-communicator from two intra-communicators requires separate collective operations in the local group and in the remote group, as well as a point-to-point communication between an MPI process in the local group and an MPI process in the remote group.

When using the World Model (Section 11.2), the [MPI_COMM_WORLD](#) communicator (or preferably a dedicated duplicate thereof) can be this peer communicator. For applications that use the Sessions Model, or the spawn or join operations, it may be necessary to first create an intra-communicator to be used as the peer communicator.

The application topology functions described in Chapter 8 do not apply to inter-communicators. Users that require this capability should utilize `MPI_INTERCOMM_MERGE` to build an intra-communicator, then apply the graph or cartesian topology capabilities to that intra-communicator, creating an appropriate topology-oriented intra-communicator. Alternatively, it may be reasonable to devise one's own application topology mechanisms for this case, without loss of generality.

```
MPI_INTERCOMM_CREATE(local_comm, local_leader, peer_comm, remote_leader, tag,
                      newintercomm)
```

IN	<code>local_comm</code>	local intra-communicator (handle)
IN	<code>local_leader</code>	rank of local group leader in <code>local_comm</code> (integer)
IN	<code>peer_comm</code>	“peer” communicator; significant only at the <code>local_leader</code> (handle)
IN	<code>remote_leader</code>	rank of remote group leader in <code>peer_comm</code> ; significant only at the <code>local_leader</code> (integer)
IN	<code>tag</code>	tag (integer)
OUT	<code>newintercomm</code>	new inter-communicator (handle)

C binding

```
int MPI_Intercomm_create(MPI_Comm local_comm, int local_leader,
                        MPI_Comm peer_comm, int remote_leader, int tag,
                        MPI_Comm *newintercomm)
```

Fortran 2008 binding

```
MPI_Intercomm_create(local_comm, local_leader, peer_comm, remote_leader, tag,
                    newintercomm, ierror)
TYPE(MPI_Comm), INTENT(IN) :: local_comm, peer_comm
INTEGER, INTENT(IN) :: local_leader, remote_leader, tag
TYPE(MPI_Comm), INTENT(OUT) :: newintercomm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INTERCOMM_CREATE(LOCAL_COMM, LOCAL_LEADER, PEER_COMM, REMOTE_LEADER, TAG,
                    NEWINTERCOMM, IERROR)
INTEGER LOCAL_COMM, LOCAL_LEADER, PEER_COMM, REMOTE_LEADER, TAG,
NEWINTERCOMM, IERROR
```

This call creates an inter-communicator. It is collective over the union of the local and remote groups. MPI processes should provide identical `local_comm` and `local_leader` arguments within each group. Wildcards are not permitted for `remote_leader`, `local_leader`, and `tag`.

```
MPI_INTERCOMM_CREATE_FROM_GROUPS(local_group, local_leader, remote_group,
                                remote_leader, stringtag, info, errhandler, newintercomm)
```

IN	<code>local_group</code>	local group (handle)
IN	<code>local_leader</code>	rank of local group leader in <code>local_group</code> (integer)

1	IN	remote_group	remote group, significant only at local_leader
2			(handle)
3	IN	remote_leader	rank of remote group leader in remote_group,
4			significant only at local_leader (integer)
5	IN	stringtag	unique identifier for this operation (string)
6	IN	info	info object (handle)
7	IN	errhandler	error handler to be attached to new
8			inter-communicator (handle)
9			
10	OUT	newintercomm	new inter-communicator (handle)

C binding

```

13 int MPI_Intercomm_create_from_groups(MPI_Group local_group, int local_leader,
14                                     MPI_Group remote_group, int remote_leader, const char *stringtag,
15                                     MPI_Info info, MPI_Errhandler errhandler, MPI_Comm *newintercomm)

```

Fortran 2008 binding

```

17 MPI_Intercomm_create_from_groups(local_group, local_leader, remote_group,
18                                 remote_leader, stringtag, info, errhandler, newintercomm, ierror)
19     TYPE(MPI_Group), INTENT(IN) :: local_group, remote_group
20     INTEGER, INTENT(IN) :: local_leader, remote_leader
21     CHARACTER(LEN=*), INTENT(IN) :: stringtag
22     TYPE(MPI_Info), INTENT(IN) :: info
23     TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
24     TYPE(MPI_Comm), INTENT(OUT) :: newintercomm
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

27 MPI_INTERCOMM_CREATE_FROM_GROUPS(LOCAL_GROUP, LOCAL_LEADER, REMOTE_GROUP,
28                                 REMOTE_LEADER, STRINGTAG, INFO, ERRHANDLER, NEWINTERCOMM, IERROR)
29     INTEGER LOCAL_GROUP, LOCAL_LEADER, REMOTE_GROUP, REMOTE_LEADER, INFO,
30     ERRHANDLER, NEWINTERCOMM, IERROR
31     CHARACTER*(*) STRINGTAG

```

This call creates an inter-communicator. Unlike [MPI_INTERCOMM_CREATE](#), this function uses as input previously defined, disjoint local and remote groups. The calling MPI process must be a member of the local group. The call is collective over the union of the local and remote groups. All involved MPI processes shall provide an identical value for the `stringtag` argument. Within each group, all MPI processes shall provide identical `local_group`, `local_leader` arguments. Wildcards are not permitted for the `remote_leader` or `local_leader` arguments. The `stringtag` argument serves the same purpose as the `stringtag` used in the [MPI_COMM_CREATE_FROM_GROUP](#) function; it differentiates concurrent calls in a multithreaded environment. The `stringtag` shall not exceed [MPI_MAX_STRINGTAG_LEN](#) characters in length. For C, this includes space for a null terminating character. [MPI_MAX_STRINGTAG_LEN](#) shall have a value of at least 63. In the event that [MPI_GROUP_EMPTY](#) is supplied as the `local_group` or `remote_group` or both, then the call is a local operation and [MPI_COMM_NULL](#) is returned as the `newintercomm`.

The `errhandler` argument specifies an error handler to be attached to the new inter-communicator. Section 9.3 specifies the error handler to be invoked if an error is encountered during the invocation of [MPI_INTERCOMM_CREATE_FROM_GROUPS](#).

The `info` argument provides hints and assertions, possibly MPI implementation dependent, which indicate desired characteristics and guide communicator creation.

`MPI_INTERCOMM_MERGE(intercomm, high, newintracomm)`

IN	intercomm	inter-communicator (handle)
IN	high	ordering of the local and remote groups in the new intra-communicator (logical)
OUT	newintracomm	new intra-communicator (handle)

C binding

```
int MPI_Intercomm_merge(MPI_Comm intercomm, int high, MPI_Comm *newintracomm)
```

Fortran 2008 binding

```
MPI_Intercomm_merge(intercomm, high, newintracomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: intercomm
  LOGICAL, INTENT(IN) :: high
  TYPE(MPI_Comm), INTENT(OUT) :: newintracomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INTERCOMM_MERGE(INTERCOMM, HIGH, NEWINTRACOMM, IERROR)
  INTEGER INTERCOMM, NEWINTRACOMM, IERROR
  LOGICAL HIGH
```

This function creates an intra-communicator from the union of the two groups that are associated with `intercomm`. All MPI processes should provide the same `high` value within each of the two groups. If MPI processes in one group provided the value `high = false` and MPI processes in the other group provided the value `high = true` then the union orders the “low” group before the “high” group. If all MPI processes provided the same `high` argument then the order of the union is arbitrary. This call is blocking and collective within the union of the two groups.

The error handler on the new inter-communicator in each MPI process is inherited from the communicator that contributes the local group. Note that this can result in different MPI processes in the same communicator having different error handlers.

Advice to implementors. The implementation of `MPI_INTERCOMM_MERGE`, `MPI_COMM_FREE`, and `MPI_COMM_DUP` are similar to the implementation of `MPI_INTERCOMM_CREATE`, except that contexts private to the input inter-communicator are used for communication between group leaders rather than contexts inside a bridge communicator. (*End of advice to implementors.*)

7.6.3 Inter-Communication Examples

Example 1: Three-Group “Pipeline”

As shown in Figure 7.4, groups 0 and 1 communicate. Groups 1 and 2 communicate. Therefore, group 0 requires one inter-communicator, group 1 requires two inter-communicators, and group 2 requires 1 inter-communicator.

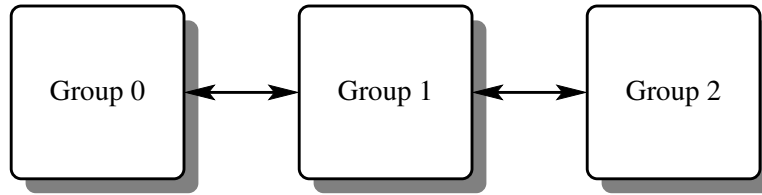


Figure 7.4: Three-group pipeline

Example 7.13.

```

11 int main(int argc, char *argv[])
12 {
13     MPI_Comm    myComm;        /* intra-communicator of local sub-group */
14     MPI_Comm    myFirstComm;   /* inter-communicator */
15     MPI_Comm    mySecondComm; /* second inter-communicator (group 1 only) */
16     int membershipKey;
17     int rank;
18
19     MPI_Init(&argc, &argv);
20     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
21
22     /* User code must generate membershipKey in the range [0, 1, 2] */
23     membershipKey = rank % 3;
24
25     /* Build intra-communicator for local sub-group */
26     MPI_Comm_split(MPI_COMM_WORLD, membershipKey, rank, &myComm);
27
28     /* Build inter-communicators. Tags are hard-coded. */
29     if (membershipKey == 0)
30     {
31         /* Group 0 communicates with group 1. */
32         MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 1,
33                             1, &myFirstComm);
34     }
35     else if (membershipKey == 1)
36     {
37         /* Group 1 communicates with groups 0 and 2. */
38         MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 0,
39                             1, &myFirstComm);
40         MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 2,
41                             12, &mySecondComm);
42     }
43     else if (membershipKey == 2)
44     {
45         /* Group 2 communicates with group 1. */
46         MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 1,
47                             12, &myFirstComm);
48     }
49
50     /* Do work ... */
51
52     switch(membershipKey) /* free communicators appropriately */
53     {
54     case 1:
55         MPI_Comm_free(&mySecondComm);
56     case 0:
57     case 2:
58         MPI_Comm_free(&myFirstComm);
59     }
60 }

```

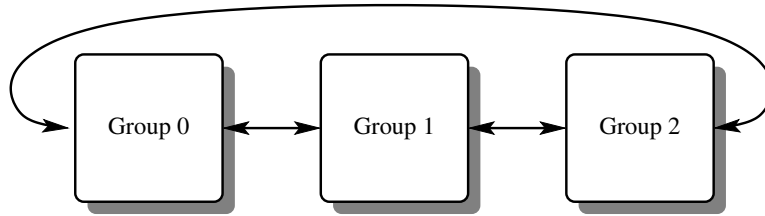



Figure 7.5: Three-group ring

```

    break;
}

MPI_Finalize();
return 0;
}

```

Example 2: Three-Group “Ring”

As shown in Figure 7.5, groups 0 and 1 communicate. Groups 1 and 2 communicate. Groups 0 and 2 communicate. Therefore, each requires two inter-communicators.

Example 7.14.

```

int main(int argc, char *argv[])
{
    MPI_Comm    myComm;        /* intra-communicator of local sub-group */
    MPI_Comm    myFirstComm; /* inter-communicators */
    MPI_Comm    mySecondComm;
    int membershipKey;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    ...

    /* User code must generate membershipKey in the range [0, 1, 2] */
    membershipKey = rank % 3;

    /* Build intra-communicator for local sub-group */
    MPI_Comm_split(MPI_COMM_WORLD, membershipKey, rank, &myComm);

    /* Build inter-communicators. Tags are hard-coded. */
    if (membershipKey == 0)
    {
        /* Group 0 communicates with groups 1 and 2. */
        MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 1,
                           1, &myFirstComm);
        MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 2,
                           2, &mySecondComm);
    }
    else if (membershipKey == 1)
    {
        /* Group 1 communicates with groups 0 and 2. */

```

```

1  MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 0,
2                        1, &myFirstComm);
3  MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 2,
4                        12, &mySecondComm);
5  }
6  else if (membershipKey == 2)
7  {
8      /* Group 2 communicates with groups 0 and 1. */
9      MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 0,
10                          2, &myFirstComm);
11     MPI_Intercomm_create(myComm, 0, MPI_COMM_WORLD, 1,
12                          12, &mySecondComm);
13 }
14
15 /* Do some work ... */
16
17 /* Then free communicators before terminating... */
18 MPI_Comm_free(&myFirstComm);
19 MPI_Comm_free(&mySecondComm);
20 MPI_Comm_free(&myComm);
21 MPI_Finalize();
22 return 0;
23 }

```

7.7 Caching

MPI provides a “caching” facility that allows an application to attach arbitrary pieces of information, called **attributes**, to three kinds of MPI objects: communicators, windows, and datatypes. More precisely, the caching facility allows a portable library to do the following:

- pass information between calls by associating it with an MPI intra- or inter-communicator, window, or datatype,
- quickly retrieve that information, and
- be guaranteed that out-of-date information is never retrieved, even if the object is freed and its handle subsequently reused by MPI.

The caching capabilities, in some form, are required by built-in MPI routines such as collective communication and application topology. Defining an interface to these capabilities as part of the MPI standard is valuable because it permits routines like collective communication and application topologies to be implemented as portable code, and also because it makes MPI more extensible by allowing user-written routines to use standard MPI calling sequences.

Advice to users. The communicator `MPI_COMM_SELF` is a suitable choice for posting MPI process-local attributes, via this attribute-caching mechanism. (*End of advice to users.*)

Rationale. In one extreme one can allow caching on all opaque handles. The other extreme is to only allow it on communicators. Caching has a cost associated with it

and should only be allowed when it is clearly needed and the increased cost is modest. This is the reason that windows and datatypes were added but not other handles. (*End of rationale.*)

One difficulty is the potential for size differences between Fortran integers and C pointers. For this reason, the Fortran versions of these routines use integers of kind `MPI_ADDRESS_KIND`.

Advice to implementors. High-quality implementations should raise an error when a keyval that was created by a call to `MPI_XXX_CREATE_KEYVAL` is used with an object of the wrong type with a call to `MPI_YYY_GET_ATTR`, `MPI_YYY_SET_ATTR`, `MPI_YYY_DELETE_ATTR`, or `MPI_YYY_FREE_KEYVAL`. To do so, it is necessary to maintain, with each keyval, information on the type of the associated user function. (*End of advice to implementors.*)

7.7.1 Functionality

Attributes can be attached to communicators, windows, and datatypes. Attributes are local to the MPI process and specific to the communicator to which they are attached. Attributes are not propagated by MPI from one communicator to another except when the communicator is duplicated using `MPI_COMM_DUP`, `MPI_COMM_IDUP`, `MPI_COMM_DUP_WITH_INFO`, and `MPI_COMM_IDUP_WITH_INFO` (and even then the application must give specific permission through callback functions for the attribute to be copied. Please refer to Section 7.4.2 and Section 7.7.2 for attributes propagation rules).

Advice to users. Attributes in C are of type `void*`. Typically, such an attribute will be a pointer to a structure that contains further information, or a handle to an MPI object. In Fortran, attributes are of type `INTEGER`. Such attribute can be a handle to an MPI object, or just an integer-valued attribute. (*End of advice to users.*)

Advice to implementors. Attributes are scalar values, equal in size to, or larger than a C-language pointer. Attributes can always hold an MPI handle. (*End of advice to implementors.*)

The caching interface defined here requires that attributes be stored by MPI opaquely within a communicator, window, or datatype. Accessor functions include the following:

- obtain a key value (used to identify an attribute); the user specifies “callback” functions by which MPI informs the application when the communicator is destroyed or copied.
- store and retrieve the value of an attribute;

Advice to implementors. Caching and callback functions are only called synchronously, in response to explicit application requests. This avoids problems that result from repeated crossings between user and system space. (This synchronous calling rule is a general property of MPI.)

The choice of key values is under control of MPI. This allows MPI to optimize its implementation of attribute sets. It also avoids conflict between independent modules caching information on the same communicators.

A much smaller interface, consisting of just a callback facility, would allow the entire caching facility to be implemented by portable code. However, with the minimal callback interface, some form of table searching is implied by the need to handle arbitrary communicators. In contrast, the more complete interface defined here permits rapid access to attributes through the use of pointers in communicators (to find the attribute table) and cleverly chosen key values (to retrieve individual attributes). In light of the efficiency “hit” inherent in the minimal interface, the more complete interface defined here is seen to be superior. (*End of advice to implementors.*)

MPI provides the following services related to caching. They are all MPI process local.

7.7.2 Communicators

Functions for caching on communicators are:

```
MPI_COMM_CREATE_KEYVAL(comm_copy_attr_fn, comm_delete_attr_fn,
                        comm_keyval, extra_state)

    IN      comm_copy_attr_fn      copy callback function for comm_keyval (function)
    IN      comm_delete_attr_fn    delete callback function for comm_keyval (function)
    OUT     comm_keyval            key value for future access (integer)
    IN      extra_state            extra state for callback function
```

C binding

```
int MPI_Comm_create_keyval(MPI_Comm_copy_attr_function *comm_copy_attr_fn,
                          MPI_Comm_delete_attr_function *comm_delete_attr_fn,
                          int *comm_keyval, void *extra_state)
```

Fortran 2008 binding

```
MPI_Comm_create_keyval(comm_copy_attr_fn, comm_delete_attr_fn, comm_keyval,
                      extra_state, ierror)

    PROCEDURE(MPI_Comm_copy_attr_function) :: comm_copy_attr_fn
    PROCEDURE(MPI_Comm_delete_attr_function) :: comm_delete_attr_fn
    INTEGER, INTENT(OUT) :: comm_keyval
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_CREATE_KEYVAL(COMM_COPY_ATTR_FN, COMM_DELETE_ATTR_FN, COMM_KEYVAL,
                      EXTRA_STATE, IERROR)

    EXTERNAL COMM_COPY_ATTR_FN, COMM_DELETE_ATTR_FN
    INTEGER COMM_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
```

Generates a new attribute key. Keys are locally unique in an MPI process, and opaque to user, though they are explicitly stored in integers. Once allocated, the key value can be used to associate attributes and access them on any locally defined communicator.

The C callback functions are:

```

typedef int MPI_Comm_copy_attr_function(MPI_Comm oldcomm, int comm_keyval,
    void *extra_state, void *attribute_val_in,
    void *attribute_val_out, int *flag);

```

and

```

typedef int MPI_Comm_delete_attr_function(MPI_Comm comm, int comm_keyval,
    void *attribute_val, void *extra_state);

```

which are the same as the MPI-1.1 calls but with a new name. The old names are deprecated. With the `mpi_f08` module, the Fortran callback functions are:

ABSTRACT INTERFACE

```

SUBROUTINE MPI_Comm_copy_attr_function(oldcomm, comm_keyval, extra_state,
    attribute_val_in, attribute_val_out, flag, ierror)
    TYPE(MPI_Comm) :: oldcomm
    INTEGER :: comm_keyval, ierror
    INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
        attribute_val_out
    LOGICAL :: flag

```

and

ABSTRACT INTERFACE

```

SUBROUTINE MPI_Comm_delete_attr_function(comm, comm_keyval, attribute_val,
    extra_state, ierror)
    TYPE(MPI_Comm) :: comm
    INTEGER :: comm_keyval, ierror
    INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state

```

With the `mpi` module and (deprecated) `mpif.h` include file, the Fortran callback functions are:

```

SUBROUTINE COMM_COPY_ATTR_FUNCTION(OLDCOMM, COMM_KEYVAL, EXTRA_STATE,
    ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT, FLAG, IERROR)
    INTEGER OLDCOMM, COMM_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
        ATTRIBUTE_VAL_OUT
    LOGICAL FLAG

```

and

```

SUBROUTINE COMM_DELETE_ATTR_FUNCTION(COMM, COMM_KEYVAL, ATTRIBUTE_VAL,
    EXTRA_STATE, IERROR)
    INTEGER COMM, COMM_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE

```

The `comm_copy_attr_fn` function is invoked when a communicator is duplicated by `MPI_COMM_DUP`, `MPI_COMM_IDUP`, `MPI_COMM_DUP_WITH_INFO` or `MPI_COMM_IDUP_WITH_INFO`. `comm_copy_attr_fn` should be of type `MPI_Comm_copy_attr_function`. The copy callback function is invoked for each key value in `oldcomm` in arbitrary order. Each call to the copy callback is made with a key value and its corresponding attribute. If it returns `flag = 0` or `.FALSE.`, then the attribute is deleted in the duplicated communicator. Otherwise (`flag = 1` or `.TRUE.`), the new attribute value is set to the value returned in `attribute_val_out`. The function returns `MPI_SUCCESS` on success and an error code on failure (in which case `MPI_COMM_DUP` or `MPI_COMM_IDUP` will fail).

The argument `comm_copy_attr_fn` may be specified as `MPI_COMM_NULL_COPY_FN` or `MPI_COMM_DUP_FN` from either C or Fortran. `MPI_COMM_NULL_COPY_FN` is a function that does nothing other than returning `flag = 0` or `.FALSE.` (depending on whether the keyval was created with a C or Fortran binding to `MPI_COMM_CREATE_KEYVAL`) and `MPI_SUCCESS`. `MPI_COMM_DUP_FN` is a simple copy function that sets `flag = 1` or `.TRUE.`, returns the value of `attribute_val_in` in `attribute_val_out`, and returns `MPI_SUCCESS`. These replace the MPI-1 predefined callbacks `MPI_NULL_COPY_FN` and `MPI_DUP_FN`, whose use is deprecated.

Advice to users. Even though both formal arguments `attribute_val_in` and `attribute_val_out` are of type `void*`, their usage differs. The C copy function is passed by MPI in `attribute_val_in` the *value* of the attribute, and in `attribute_val_out` the *address* of the attribute, so as to allow the function to return the (new) attribute value. The use of type `void*` for both is to avoid messy type casts.

A valid copy function is one that completely duplicates the information by making a full duplicate copy of the data structures implied by an attribute; another might just make another reference to that data structure, while using a reference-count mechanism. Other types of attributes might not copy at all (they might be specific to `oldcomm` only). (*End of advice to users.*)

Advice to implementors. A C interface should be assumed for copy and delete functions associated with key values created in C; a Fortran calling interface should be assumed for key values created in Fortran. (*End of advice to implementors.*)

Analogous to `comm_copy_attr_fn` is a callback deletion function, defined as follows. The `comm_delete_attr_fn` function is invoked when a communicator is deleted by `MPI_COMM_FREE`, `MPI_COMM_DISCONNECT` or when a call is made explicitly to `MPI_COMM_DELETE_ATTR`. `comm_delete_attr_fn` should be of type `MPI_Comm_delete_attr_function`.

This function is called by `MPI_COMM_FREE`, `MPI_COMM_DISCONNECT`, `MPI_COMM_DELETE_ATTR`, and `MPI_COMM_SET_ATTR` to do whatever is needed to remove an attribute. The function returns `MPI_SUCCESS` on success and an error code on failure (in which case `MPI_COMM_FREE` will fail).

The argument `comm_delete_attr_fn` may be specified as `MPI_COMM_NULL_DELETE_FN` from either C or Fortran. `MPI_COMM_NULL_DELETE_FN` is a function that does nothing, other than returning `MPI_SUCCESS`. `MPI_COMM_NULL_DELETE_FN` replaces `MPI_NULL_DELETE_FN`, whose use is deprecated.

If an attribute copy function or attribute delete function returns other than `MPI_SUCCESS`, then the call that caused it to be invoked (for example, `MPI_COMM_FREE`) is erroneous.

The special key value `MPI_KEYVAL_INVALID` is never returned by `MPI_COMM_CREATE_KEYVAL`. Therefore, it can be used for static initialization of key values.

Advice to implementors. The predefined Fortran functions `MPI_COMM_NULL_COPY_FN`, `MPI_COMM_DUP_FN`, and `MPI_COMM_NULL_DELETE_FN` are defined in the `mpi` module (and deprecated

Fortran 2008 binding

```

MPI_Comm_set_attr(comm, comm_keyval, attribute_val, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(IN) :: comm_keyval
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: attribute_val
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_SET_ATTR(COMM, COMM_KEYVAL, ATTRIBUTE_VAL, IERROR)
    INTEGER COMM, COMM_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL

```

This function stores the stipulated attribute value `attribute_val` for subsequent retrieval by `MPI_COMM_GET_ATTR`. If the value is already present, then the outcome is as if `MPI_COMM_DELETE_ATTR` was first called to delete the previous value (and the callback function `comm_delete_attr_fn` was executed), and a new value was next stored. The call is erroneous if there is no key with value `keyval`; in particular `MPI_KEYVAL_INVALID` is an erroneous key value. The call will fail if the `comm_delete_attr_fn` function returned an error code other than `MPI_SUCCESS`.

```

MPI_COMM_GET_ATTR(comm, comm_keyval, attribute_val, flag)

```

IN	comm	communicator to which the attribute is attached (handle)
IN	comm_keyval	key value (integer)
OUT	attribute_val	attribute value, unless <code>flag = false</code>
OUT	flag	false if no attribute is associated with the key (logical)

C binding

```

int MPI_Comm_get_attr(MPI_Comm comm, int comm_keyval, void *attribute_val,
    int *flag)

```

Fortran 2008 binding

```

MPI_Comm_get_attr(comm, comm_keyval, attribute_val, flag, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(IN) :: comm_keyval
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: attribute_val
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_GET_ATTR(COMM, COMM_KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
    INTEGER COMM, COMM_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
    LOGICAL FLAG

```

Retrieves attribute value by key. The call is erroneous if there is no key with value `keyval`. On the other hand, the call is correct if the key value exists, but no attribute is attached on `comm` for that key; in such case, the call returns `flag = false`. In particular `MPI_KEYVAL_INVALID` is an erroneous key value.

Advice to users. The call to `MPI_Comm_set_attr` passes in `attribute_val` the *value* of the attribute; the call to `MPI_Comm_get_attr` passes in `attribute_val` the *address* of the location where the attribute value is to be returned. Thus, if the attribute value itself is a pointer of type `void*`, then the actual `attribute_val` parameter to `MPI_Comm_set_attr` will be of type `void*` and the actual `attribute_val` parameter to `MPI_Comm_get_attr` will be of type `void**`. (*End of advice to users.*)

Rationale. The use of a formal parameter `attribute_val` of type `void*` (rather than `void**`) avoids the messy type casting that would be needed if the attribute value is declared with a type other than `void*`. (*End of rationale.*)

`MPI_COMM_DELETE_ATTR(comm, comm_keyval)`

INOUT	comm	communicator from which the attribute is deleted (handle)
IN	comm_keyval	key value (integer)

C binding

```
int MPI_Comm_delete_attr(MPI_Comm comm, int comm_keyval)
```

Fortran 2008 binding

```
MPI_Comm_delete_attr(comm, comm_keyval, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: comm_keyval
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_DELETE_ATTR(COMM, COMM_KEYVAL, IERROR)
  INTEGER COMM, COMM_KEYVAL, IERROR
```

Delete attribute from cache by key. This function invokes the attribute delete function `comm_delete_attr_fn` specified when the `keyval` was created. The call will fail if the `comm_delete_attr_fn` function returns an error code other than `MPI_SUCCESS`.

Whenever a communicator is replicated using the function `MPI_COMM_DUP`, `MPI_COMM_IDUP`, `MPI_COMM_DUP_WITH_INFO` or `MPI_COMM_IDUP_WITH_INFO`, all call-back copy functions for attributes that are currently set are invoked (in arbitrary order). Whenever a communicator is deleted using the function `MPI_COMM_FREE` all callback delete functions for attributes that are currently set are invoked.

7.7.3 Windows

The functions for caching on windows are:

```

1 MPI_WIN_CREATE_KEYVAL(win_copy_attr_fn, win_delete_attr_fn, win_keyval,
2     extra_state)
3
4     IN        win_copy_attr_fn        copy callback function for win_keyval (function)
5     IN        win_delete_attr_fn      delete callback function for win_keyval (function)
6     OUT       win_keyval              key value for future access (integer)
7     IN        extra_state             extra state for callback function
8
9

```

C binding

```

10 int MPI_Win_create_keyval(MPI_Win_copy_attr_function *win_copy_attr_fn,
11     MPI_Win_delete_attr_function *win_delete_attr_fn,
12     int *win_keyval, void *extra_state)
13

```

Fortran 2008 binding

```

14 MPI_Win_create_keyval(win_copy_attr_fn, win_delete_attr_fn, win_keyval,
15     extra_state, ierror)
16
17     PROCEDURE(MPI_Win_copy_attr_function) :: win_copy_attr_fn
18     PROCEDURE(MPI_Win_delete_attr_function) :: win_delete_attr_fn
19     INTEGER, INTENT(OUT) :: win_keyval
20     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
22

```

Fortran binding

```

23 MPI_WIN_CREATE_KEYVAL(WIN_COPY_ATTR_FN, WIN_DELETE_ATTR_FN, WIN_KEYVAL,
24     EXTRA_STATE, IERROR)
25
26     EXTERNAL WIN_COPY_ATTR_FN, WIN_DELETE_ATTR_FN
27     INTEGER WIN_KEYVAL, IERROR
28     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
29

```

The argument `win_copy_attr_fn` may be specified as `MPI_WIN_NULL_COPY_FN` or `MPI_WIN_DUP_FN` from either C or Fortran. `MPI_WIN_NULL_COPY_FN` is a function that does nothing other than returning `flag = 0` and `MPI_SUCCESS`. `MPI_WIN_DUP_FN` is a simple copy function that sets `flag = 1`, returns the value of `attribute_val_in` in `attribute_val_out`, and returns `MPI_SUCCESS`.

The argument `win_delete_attr_fn` may be specified as `MPI_WIN_NULL_DELETE_FN` from either C or Fortran. `MPI_WIN_NULL_DELETE_FN` is a function that does nothing, other than returning `MPI_SUCCESS`.

The C callback functions are:

```

37 typedef int MPI_Win_copy_attr_function(MPI_Win oldwin, int win_keyval,
38     void *extra_state, void *attribute_val_in,
39     void *attribute_val_out, int *flag);
40

```

and

```

42 typedef int MPI_Win_delete_attr_function(MPI_Win win, int win_keyval,
43     void *attribute_val, void *extra_state);
44

```

With the `mpi_f08` module, the Fortran callback functions are:

```

45 ABSTRACT INTERFACE
46     SUBROUTINE MPI_Win_copy_attr_function(oldwin, win_keyval, extra_state,
47         attribute_val_in, attribute_val_out, flag, ierror)
48     TYPE(MPI_Win) :: oldwin

```

```

    INTEGER :: win_keyval, ierror
    INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
        attribute_val_out
    LOGICAL :: flag
and
ABSTRACT INTERFACE
    SUBROUTINE MPI_Win_delete_attr_function(win, win_keyval, attribute_val,
        extra_state, ierror)
    TYPE(MPI_Win) :: win
    INTEGER :: win_keyval, ierror
    INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
With the mpi module and (deprecated) mpif.h include file, the Fortran callback functions
are:
SUBROUTINE WIN_COPY_ATTR_FUNCTION(OLDWIN, WIN_KEYVAL, EXTRA_STATE,
    ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT, FLAG, IERROR)
    INTEGER OLDWIN, WIN_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
        ATTRIBUTE_VAL_OUT
    LOGICAL FLAG
and
SUBROUTINE WIN_DELETE_ATTR_FUNCTION(WIN, WIN_KEYVAL, ATTRIBUTE_VAL,
    EXTRA_STATE, IERROR)
    INTEGER WIN, WIN_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE
    If an attribute copy function or attribute delete function returns other than
MPI_SUCCESS, then the call that caused it to be invoked (for example, MPI_WIN_FREE),
is erroneous.

MPI_WIN_FREE_KEYVAL(win_keyval)
    INOUT    win_keyval                key value (integer)

C binding
int MPI_Win_free_keyval(int *win_keyval)

Fortran 2008 binding
MPI_Win_free_keyval(win_keyval, ierror)
    INTEGER, INTENT(INOUT) :: win_keyval
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding
MPI_WIN_FREE_KEYVAL(WIN_KEYVAL, IERROR)
    INTEGER WIN_KEYVAL, IERROR

```

```
1 MPI_WIN_SET_ATTR(win, win_keyval, attribute_val)
```

```
2     INOUT    win                window to which attribute will be attached (handle)
```

```
4     IN      win_keyval         key value (integer)
```

```
5     IN      attribute_val      attribute value
```

7 C binding

```
8 int MPI_Win_set_attr(MPI_Win win, int win_keyval, void *attribute_val)
```

10 Fortran 2008 binding

```
11 MPI_Win_set_attr(win, win_keyval, attribute_val, ierror)
```

```
12     TYPE(MPI_Win), INTENT(IN) :: win
```

```
13     INTEGER, INTENT(IN) :: win_keyval
```

```
14     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: attribute_val
```

```
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

16 Fortran binding

```
17 MPI_WIN_SET_ATTR(WIN, WIN_KEYVAL, ATTRIBUTE_VAL, IERROR)
```

```
18     INTEGER WIN, WIN_KEYVAL, IERROR
```

```
19     INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
```

```
22 MPI_WIN_GET_ATTR(win, win_keyval, attribute_val, flag)
```

```
24     IN      win                window to which the attribute is attached (handle)
```

```
25     IN      win_keyval         key value (integer)
```

```
26     OUT     attribute_val      attribute value, unless flag = false
```

```
28     OUT     flag               false if no attribute is associated with the key  
                               (logical)
```

30 C binding

```
31 int MPI_Win_get_attr(MPI_Win win, int win_keyval, void *attribute_val,  
32                      int *flag)
```

34 Fortran 2008 binding

```
35 MPI_Win_get_attr(win, win_keyval, attribute_val, flag, ierror)
```

```
36     TYPE(MPI_Win), INTENT(IN) :: win
```

```
37     INTEGER, INTENT(IN) :: win_keyval
```

```
38     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: attribute_val
```

```
39     LOGICAL, INTENT(OUT) :: flag
```

```
40     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

41 Fortran binding

```
42 MPI_WIN_GET_ATTR(WIN, WIN_KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
```

```
43     INTEGER WIN, WIN_KEYVAL, IERROR
```

```
44     INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
```

```
45     LOGICAL FLAG
```

```

MPI_WIN_DELETE_ATTR(win, win_keyval)
    INOUT    win                window from which the attribute is deleted (handle)
    IN       win_keyval         key value (integer)

```

C binding

```
int MPI_Win_delete_attr(MPI_Win win, int win_keyval)
```

Fortran 2008 binding

```

MPI_Win_delete_attr(win, win_keyval, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, INTENT(IN) :: win_keyval
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_DELETE_ATTR(WIN, WIN_KEYVAL, IERROR)
    INTEGER WIN, WIN_KEYVAL, IERROR

```

7.7.4 Datatypes

The new functions for caching on datatypes are:

```

MPI_TYPE_CREATE_KEYVAL(type_copy_attr_fn, type_delete_attr_fn, type_keyval,
    extra_state)
    IN      type_copy_attr_fn    copy callback function for type_keyval (function)
    IN      type_delete_attr_fn  delete callback function for type_keyval (function)
    OUT     type_keyval          key value for future access (integer)
    IN      extra_state          extra state for callback function

```

C binding

```

int MPI_Type_create_keyval(MPI_Type_copy_attr_function *type_copy_attr_fn,
    MPI_Type_delete_attr_function *type_delete_attr_fn,
    int *type_keyval, void *extra_state)

```

Fortran 2008 binding

```

MPI_Type_create_keyval(type_copy_attr_fn, type_delete_attr_fn, type_keyval,
    extra_state, ierror)
    PROCEDURE(MPI_Type_copy_attr_function) :: type_copy_attr_fn
    PROCEDURE(MPI_Type_delete_attr_function) :: type_delete_attr_fn
    INTEGER, INTENT(OUT) :: type_keyval
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_CREATE_KEYVAL(TYPE_COPY_ATTR_FN, TYPE_DELETE_ATTR_FN, TYPE_KEYVAL,
    EXTRA_STATE, IERROR)
    EXTERNAL TYPE_COPY_ATTR_FN, TYPE_DELETE_ATTR_FN

```

```

1      INTEGER TYPE_KEYVAL, IERROR
2      INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE

```

The argument `type_copy_attr_fn` may be specified as `MPI_TYPE_NULL_COPY_FN` or `MPI_TYPE_DUP_FN` from either C or Fortran. `MPI_TYPE_NULL_COPY_FN` is a function that does nothing other than returning `flag = 0` and `MPI_SUCCESS`.

`MPI_TYPE_DUP_FN` is a simple copy function that sets `flag = 1`, returns the value of `attribute_val_in` in `attribute_val_out`, and returns `MPI_SUCCESS`.

The argument `type_delete_attr_fn` may be specified as `MPI_TYPE_NULL_DELETE_FN` from either C or Fortran. `MPI_TYPE_NULL_DELETE_FN` is a function that does nothing, other than returning `MPI_SUCCESS`.

The C callback functions are:

```

12     typedef int MPI_Type_copy_attr_function(MPI_Datatype oldtype, int type_keyval,
13                                             void *extra_state, void *attribute_val_in,
14                                             void *attribute_val_out, int *flag);
15

```

and

```

17     typedef int MPI_Type_delete_attr_function(MPI_Datatype datatype,
18                                             int type_keyval, void *attribute_val, void *extra_state);

```

With the `mpi_f08` module, the Fortran callback functions are:

```

20     ABSTRACT INTERFACE
21       SUBROUTINE MPI_Type_copy_attr_function(oldtype, type_keyval, extra_state,
22                                             attribute_val_in, attribute_val_out, flag, ierror)
23       TYPE(MPI_Datatype) :: oldtype
24       INTEGER :: type_keyval, ierror
25       INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
26                                             attribute_val_out
27       LOGICAL :: flag
28

```

and

```

29     ABSTRACT INTERFACE
30       SUBROUTINE MPI_Type_delete_attr_function(datatype, type_keyval,
31                                             attribute_val, extra_state, ierror)
32       TYPE(MPI_Datatype) :: datatype
33       INTEGER :: type_keyval, ierror
34       INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
35

```

With the `mpi` module and (deprecated) `mpif.h` include file, the Fortran callback functions are:

```

38     SUBROUTINE TYPE_COPY_ATTR_FUNCTION(OLDTYPE, TYPE_KEYVAL, EXTRA_STATE,
39                                         ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT, FLAG, IERROR)
40       INTEGER OLDTYPE, TYPE_KEYVAL, IERROR
41       INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
42                                         ATTRIBUTE_VAL_OUT
43       LOGICAL FLAG

```

and

```

45     SUBROUTINE TYPE_DELETE_ATTR_FUNCTION(DATATYPE, TYPE_KEYVAL, ATTRIBUTE_VAL,
46                                         EXTRA_STATE, IERROR)
47       INTEGER DATATYPE, TYPE_KEYVAL, IERROR
48       INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE

```



```

1      OUT      flag                                false if no attribute is associated with the key
2                                                    (logical)
3

```

C binding

```

5  int MPI_Type_get_attr(MPI_Datatype datatype, int type_keyval,
6                        void *attribute_val, int *flag)
7

```

Fortran 2008 binding

```

8  MPI_Type_get_attr(datatype, type_keyval, attribute_val, flag, ierror)
9      TYPE(MPI_Datatype), INTENT(IN) :: datatype
10     INTEGER, INTENT(IN) :: type_keyval
11     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: attribute_val
12     LOGICAL, INTENT(OUT) :: flag
13     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
14

```

Fortran binding

```

15 MPI_TYPE_GET_ATTR(DATATYPE, TYPE_KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
16     INTEGER DATATYPE, TYPE_KEYVAL, IERROR
17     INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
18     LOGICAL FLAG
19
20
21

```

```

22 MPI_TYPE_DELETE_ATTR(datatype, type_keyval)
23

```

```

24     INOUT    datatype                                datatype from which the attribute is deleted
25                                                    (handle)
26

```

```

27     IN      type_keyval                            key value (integer)
28
29

```

C binding

```

28 int MPI_Type_delete_attr(MPI_Datatype datatype, int type_keyval)
29

```

Fortran 2008 binding

```

30 MPI_Type_delete_attr(datatype, type_keyval, ierror)
31     TYPE(MPI_Datatype), INTENT(IN) :: datatype
32     INTEGER, INTENT(IN) :: type_keyval
33     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
34
35

```

Fortran binding

```

36 MPI_TYPE_DELETE_ATTR(DATATYPE, TYPE_KEYVAL, IERROR)
37     INTEGER DATATYPE, TYPE_KEYVAL, IERROR
38
39

```

7.7.5 Error Class for Invalid Keyval

Key values for attributes are system-allocated, by `MPI_{XXX}_CREATE_KEYVAL`. Only such values can be passed to the functions that use key values as input arguments. In order to signal that an erroneous key value has been passed to one of these functions, there is a new MPI error class: `MPI_ERR_KEYVAL`. It can be returned by `MPI_ATTR_PUT`, `MPI_ATTR_GET`, `MPI_ATTR_DELETE`, `MPI_KEYVAL_FREE`, `MPI_{XXX}_DELETE_ATTR`, `MPI_{XXX}_SET_ATTR`, `MPI_{XXX}_GET_ATTR`, `MPI_{XXX}_FREE_KEYVAL`, `MPI_COMM_DUP`, `MPI_COMM_IDUP`,

`MPI_COMM_DUP_WITH_INFO`, `MPI_COMM_IDUP_WITH_INFO`,
`MPI_COMM_DISCONNECT`, and `MPI_COMM_FREE`. The last six are included because
`keyval` is an argument to the copy and delete functions for attributes.

7.7.6 Attributes Example

Advice to users. This example shows how to write a collective communication operation
that uses caching to be more efficient after the first call. (*End of advice to users.*)

```

/* key for this module's stuff: */
static int gop_key = MPI_KEYVAL_INVALID;

typedef struct
{
    int ref_count;          /* reference count */
    /* other stuff, whatever else we want */
} gop_stuff_type;

void Efficient_Collective_Op(MPI_Comm comm, ...)
{
    gop_stuff_type *gop_stuff;
    MPI_Group      group;
    int            foundflag;

    MPI_Comm_group(comm, &group);

    if (gop_key == MPI_KEYVAL_INVALID) /* get a key on first call ever */
    {
        if ( ! MPI_Comm_create_keyval(gop_stuff_copier,
                                      gop_stuff_destructor,
                                      &gop_key, NULL)) {
            /* get the key while assigning its copy and delete callback
               behavior. */
        } else
            MPI_Abort(comm, 99);
    }

    MPI_Comm_get_attr(comm, gop_key, &gop_stuff, &foundflag);
    if (foundflag)
    { /* This module has executed in this group before.
       We will use the cached information */
    }
    else
    { /* This is a group that we have not yet cached anything in.
       We will now do so.
       */

        /* First, allocate storage for the stuff we want,
           and initialize the reference count */

        gop_stuff = (gop_stuff_type *) malloc(sizeof(gop_stuff_type));
        if (gop_stuff == NULL) { /* abort on out-of-memory error */ }
    }
}

```

```

1      gop_stuff->ref_count = 1;
2
3      /* Second, fill in *gop_stuff with whatever we want.
4         This part isn't shown here */
5
6      /* Third, store gop_stuff as the attribute value */
7      MPI_Comm_set_attr(comm, gop_key, gop_stuff);
8  }
9  /* Then, in any case, use contents of *gop_stuff
10     to do the global op ... */
11 }
12
13 /* The following routine is called by MPI when a group is freed */
14 int gop_stuff_destructor(MPI_Comm comm, int keyval, void *gop_stuffP,
15                          void *extra)
16 {
17     gop_stuff_type *gop_stuff = (gop_stuff_type *)gop_stuffP;
18     if (keyval != gop_key) { /* abort -- programming error */ }
19
20     /* The group's being freed removes one reference to gop_stuff */
21     gop_stuff->ref_count -= 1;
22
23     /* If no references remain, then free the storage */
24     if (gop_stuff->ref_count == 0) {
25         free((void *)gop_stuff);
26     }
27     return MPI_SUCCESS;
28 }
29
30 /* The following routine is called by MPI when a group is copied */
31 int gop_stuff_copier(MPI_Comm comm, int keyval, void *extra,
32                     void *gop_stuff_inP, void *gop_stuff_outP, int *flag)
33 {
34     gop_stuff_type *gop_stuff_in = (gop_stuff_type *)gop_stuff_inP;
35     gop_stuff_type **gop_stuff_out = (gop_stuff_type **)gop_stuff_outP;
36     if (keyval != gop_key) { /* abort -- programming error */ }
37
38     /* The new group adds one reference to this gop_stuff */
39     gop_stuff_in->ref_count += 1;
40     *gop_stuff_out = gop_stuff_in;
41     return MPI_SUCCESS;
42 }

```

7.8 Naming Objects

There are many occasions on which it would be useful to allow a user to associate a printable identifier with an MPI communicator, window, or datatype, for instance error reporting, debugging, and profiling. The names attached to opaque objects do not propagate when the object is duplicated or copied by MPI routines. For communicators this can be achieved using the following two functions.

```
MPI_COMM_SET_NAME(comm, comm_name)
```

```

    INOUT    comm                communicator whose identifier is to be set (handle)
    IN       comm_name           the character string that is remembered as the name
                                (string)

```

C binding

```
int MPI_Comm_set_name(MPI_Comm comm, const char *comm_name)
```

Fortran 2008 binding

```

MPI_Comm_set_name(comm, comm_name, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    CHARACTER(LEN=*), INTENT(IN) :: comm_name
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_SET_NAME(COMM, COMM_NAME, IERROR)
    INTEGER COMM, IERROR
    CHARACTER*(*) COMM_NAME

```

`MPI_COMM_SET_NAME` allows a user to associate a name string with a communicator. The character string that is passed to `MPI_COMM_SET_NAME` will be saved inside the MPI library (so it can be freed by the caller immediately after the call, or allocated on the stack). Leading spaces in `name` are significant but trailing ones are not.

`MPI_COMM_SET_NAME` is a local (noncollective) operation, which only affects the name of the communicator as seen in the MPI process that made the `MPI_COMM_SET_NAME` call. There is no requirement that the same (or any) name be assigned to a communicator in every MPI process where it exists.

Advice to users. Since `MPI_COMM_SET_NAME` is provided to help debug code, it is sensible to give the same name to a communicator in all of the MPI processes where it exists, to avoid confusion. (*End of advice to users.*)

The length of the name that can be stored is limited to the value of `MPI_MAX_OBJECT_NAME` in Fortran and `MPI_MAX_OBJECT_NAME-1` in C to allow for the null terminator. Attempts to put names longer than this will result in truncation of the name. `MPI_MAX_OBJECT_NAME` must have a value of at least 64.

Advice to users. Under circumstances of store exhaustion an attempt to put a name of any length could fail, therefore the value of `MPI_MAX_OBJECT_NAME` should be viewed only as a strict upper bound on the name length, not a guarantee that setting names of less than this length will always succeed. (*End of advice to users.*)

Advice to implementors. Implementations that pre-allocate a fixed size space for a name should use the length of that allocation as the value of `MPI_MAX_OBJECT_NAME`. Implementations that allocate space for the name from the heap should still define `MPI_MAX_OBJECT_NAME` to be a relatively small value, since the user has to allocate space for a string of up to this size when calling `MPI_COMM_GET_NAME`. (*End of advice to implementors.*)

```

1 MPI_COMM_GET_NAME(comm, comm_name, resultlen)
2   IN      comm      communicator whose name is to be returned
3                  (handle)
4   OUT     comm_name  the name previously stored on the communicator, or
5                  an empty string if no such name exists (string)
6   OUT     resultlen  length of returned name (integer)
7
8

```

C binding

```

9 int MPI_Comm_get_name(MPI_Comm comm, char *comm_name, int *resultlen)
10

```

Fortran 2008 binding

```

11 MPI_Comm_get_name(comm, comm_name, resultlen, ierror)
12   TYPE(MPI_Comm), INTENT(IN) :: comm
13   CHARACTER(LEN=MPI_MAX_OBJECT_NAME), INTENT(OUT) :: comm_name
14   INTEGER, INTENT(OUT) :: resultlen
15   INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16

```

Fortran binding

```

17 MPI_COMM_GET_NAME(COMM, COMM_NAME, RESULTLEN, IERROR)
18   INTEGER COMM, RESULTLEN, IERROR
19   CHARACTER*(*) COMM_NAME
20

```

MPI_COMM_GET_NAME returns the last name that has previously been associated with the given communicator. The name may be set and retrieved from any language. The same name will be returned independent of the language used. `comm_name` should be allocated so that it can hold a resulting string of length `MPI_MAX_OBJECT_NAME` characters. **MPI_COMM_GET_NAME** returns a copy of the set name in `comm_name`.

In C, a null character is additionally stored at `comm_name[resultlen]`. The value of `resultlen` cannot be larger than `MPI_MAX_OBJECT_NAME-1`. In Fortran, `comm_name` is padded on the right with blank characters. The value of `resultlen` cannot be larger than `MPI_MAX_OBJECT_NAME`.

If the user has not associated a name with a communicator, or an error occurs, **MPI_COMM_GET_NAME** will return an empty string (all spaces in Fortran, "" in C). The three predefined communicators will have predefined names associated with them. Thus, the names of `MPI_COMM_WORLD`, `MPI_COMM_SELF`, and the communicator returned by **MPI_COMM_GET_PARENT** (if not `MPI_COMM_NULL`) will have the default of "MPI_COMM_WORLD", "MPI_COMM_SELF", and "MPI_COMM_PARENT". Passing `MPI_COMM_NULL` as `comm` will return the string "MPI_COMM_NULL". The fact that the system may have chosen to give a default name to a communicator does not prevent the user from setting a name on the same communicator; doing this removes the old name and assigns the new one.

Rationale. We provide separate functions for setting and getting the name of a communicator, rather than simply providing a predefined attribute key for the following reasons:

- It is not, in general, possible to store a string as an attribute from Fortran.
- It is not easy to set up the delete function for a string attribute unless it is known to have been allocated from the heap.

- To make the attribute key useful additional code to call `strdup` is necessary. If this is not standardized then users have to write it. This is extra unneeded work that we can easily eliminate.
- The Fortran binding is not trivial to write (it will depend on details of the Fortran compilation system), and will not be portable. Therefore it should be in the library rather than in user code.

(End of rationale.)

Advice to users. The above definition means that it is safe simply to print the string returned by `MPI_COMM_GET_NAME`, as it is always a valid string even if there was no name.

Note that associating a name with a communicator has no effect on the semantics of an MPI program, and will (necessarily) increase the store requirement of the program, since the names must be saved. Therefore there is no requirement that users use these functions to associate names with communicators. However debugging and profiling MPI applications may be made easier if names are associated with communicators, since the debugger or profiler should then be able to present information in a less cryptic manner. *(End of advice to users.)*

The following functions are used for setting and getting names of datatypes. The constant `MPI_MAX_OBJECT_NAME` also applies to these names.

`MPI_TYPE_SET_NAME(datatype, type_name)`

INOUT	<code>datatype</code>	datatype whose identifier is to be set (handle)
IN	<code>type_name</code>	the character string that is remembered as the name (string)

C binding

`int MPI_Type_set_name(MPI_Datatype datatype, const char *type_name)`

Fortran 2008 binding

```
MPI_Type_set_name(datatype, type_name, ierror)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  CHARACTER(LEN=*), INTENT(IN) :: type_name
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_SET_NAME(DATATYPE, TYPE_NAME, IERROR)
  INTEGER DATATYPE, IERROR
  CHARACTER*(*) TYPE_NAME
```

`MPI_TYPE_GET_NAME(datatype, type_name, resultlen)`

IN	<code>datatype</code>	datatype whose name is to be returned (handle)
OUT	<code>type_name</code>	the name previously stored on the datatype, or an empty string if no such name exists (string)

1 OUT resultlen length of returned name (integer)

2

3 C binding

4 int MPI_Type_get_name(MPI_Datatype datatype, char *type_name, int *resultlen)

5

6 Fortran 2008 binding

7 MPI_Type_get_name(datatype, type_name, resultlen, ierror)

8 TYPE(MPI_Datatype), INTENT(IN) :: datatype

9 CHARACTER(LEN=MPI_MAX_OBJECT_NAME), INTENT(OUT) :: type_name

10 INTEGER, INTENT(OUT) :: resultlen

11 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

12 Fortran binding

13 MPI_TYPE_GET_NAME(DATATYPE, TYPE_NAME, RESULTLEN, IERROR)

14 INTEGER DATATYPE, RESULTLEN, IERROR

15 CHARACTER*(*) TYPE_NAME

16

17 Named predefined datatypes have the default names of the datatype name. For exam-
18 ple, `MPI_WCHAR` has the default name of "MPI_WCHAR". Passing `MPI_DATATYPE_NULL`
19 as `datatype` will return the string "MPI_DATATYPE_NULL".

20 The following functions are used for setting and getting names of windows. The con-
21 stant `MPI_MAX_OBJECT_NAME` also applies to these names.

22

23 MPI_WIN_SET_NAME(win, win_name)

24

25 INOUT win window whose identifier is to be set (handle)

26 IN win_name the character string that is remembered as the name
27 (string)

28

29 C binding

30 int MPI_Win_set_name(MPI_Win win, const char *win_name)

31

32 Fortran 2008 binding

33 MPI_Win_set_name(win, win_name, ierror)

34 TYPE(MPI_Win), INTENT(IN) :: win

35 CHARACTER(LEN=*), INTENT(IN) :: win_name

36 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

37 Fortran binding

38 MPI_WIN_SET_NAME(WIN, WIN_NAME, IERROR)

39 INTEGER WIN, IERROR

40 CHARACTER*(*) WIN_NAME

41

42

43

44 MPI_WIN_GET_NAME(win, win_name, resultlen)

45

46 IN win window whose name is to be returned (handle)

47 OUT win_name the name previously stored on the window, or an
48 empty string if no such name exists (string)

49 OUT resultlen length of returned name (integer)

C binding

```
int MPI_Win_get_name(MPI_Win win, char *win_name, int *resultlen)
```

Fortran 2008 binding

```
MPI_Win_get_name(win, win_name, resultlen, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    CHARACTER(LEN=MPI_MAX_OBJECT_NAME), INTENT(OUT) :: win_name
    INTEGER, INTENT(OUT) :: resultlen
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_GET_NAME(WIN, WIN_NAME, RESULTLEN, IERROR)
    INTEGER WIN, RESULTLEN, IERROR
    CHARACTER*(*) WIN_NAME
```

Passing `MPI_WIN_NULL` as win will return the string "MPI_WIN_NULL".

7.9 Formalizing the Loosely Synchronous Model

In this section, we make further statements about the loosely synchronous model, with particular attention to intra-communication.

7.9.1 Basic Statements

When a caller passes a communicator (that contains a context and group) to a callee, that communicator must be free of side effects throughout execution of the subprogram: there should be no active operations on that communicator that might involve the MPI process. This provides one model in which libraries can be written, and work “safely.” For libraries so designated, the callee has permission to do whatever communication it likes with the communicator, and under the above guarantee knows that no other communications will interfere. Since we permit good implementations to create new communicators without synchronization (such as by preallocated contexts on communicators), this does not impose a significant overhead.

This form of safety is analogous to other common computer-science usages, such as passing a descriptor of an array to a library routine. The library routine has every right to expect such a descriptor to be valid and modifiable.

7.9.2 Models of Execution

In the loosely synchronous model, transfer of control to a **parallel procedure** is effected by having each executing MPI process invoke the procedure. The invocation is a collective operation: it is executed by all MPI processes in the execution group, and invocations are similarly ordered at all MPI processes. However, the invocation need not be synchronized.

We say that a parallel procedure is *active* in an MPI process if the MPI process belongs to a group that may collectively execute the procedure, and some member of that group is currently executing the procedure code. If a parallel procedure is active in an MPI process, then this MPI process may be receiving messages pertaining to this procedure, even if it does not currently execute the code of this procedure.

Static Communicator Allocation

This covers the case where, at any point in time, at most one invocation of a parallel procedure can be active at any MPI process, and the group of executing MPI processes is fixed. For example, all invocations of parallel procedures involve all MPI processes, MPI processes are single-threaded, and there are no recursive invocations.

In such a case, a communicator can be statically allocated to each procedure. The static allocation can be done in a preamble, as part of initialization code. If the parallel procedures can be organized into libraries, so that only one procedure of each library can be concurrently active in each processor, then it is sufficient to allocate one communicator per library.

Dynamic Communicator Allocation

Calls of parallel procedures are well-nested if a new parallel procedure is always invoked in a subset of a group executing the same parallel procedure. Thus, MPI processes that execute the same parallel procedure have the same execution stack.

In such a case, a new communicator needs to be dynamically allocated for each new invocation of a parallel procedure. The allocation is done by the caller. A new communicator can be generated by a call to `MPI_COMM_DUP`, if the callee execution group is identical to the caller execution group, or by a call to `MPI_COMM_SPLIT` if the caller execution group is split into several subgroups executing distinct parallel routines. The new communicator is passed as an argument to the invoked routine.

The need for generating a new communicator at each invocation can be alleviated or avoided altogether in some cases: If the execution group is not split, then one can allocate a stack of communicators in a preamble, and next manage the stack in a way that mimics the stack of recursive calls.

One can also take advantage of the well-ordering property of communication to avoid confusing caller and callee communication, even if both use the same communicator. To do so, one needs to abide by the following two rules:

- messages sent before a procedure call (or before a return from the procedure) are also received before the matching call (or return) at the receiving end;
- messages are always selected by source (no use is made of `MPI_ANY_SOURCE`).

The General Case

In the general case, there may be multiple concurrently active invocations of the same parallel procedure within the same group; invocations may not be well-nested. A new communicator needs to be created for each invocation. It is the user's responsibility to make sure that, should two distinct parallel procedures be invoked concurrently on overlapping sets of MPI processes, communicator creation is properly coordinated.

Chapter 8

Virtual Topologies for MPI Processes

8.1 Introduction

This chapter discusses the MPI *virtual topology* mechanism. A *virtual topology* is an extra, optional attribute that one can give to an intra-communicator; *virtual topologies* cannot be added to inter-communicators. A *virtual topology* can provide a convenient naming mechanism for the MPI processes of a group (within a communicator), and additionally, may assist the runtime system in mapping the processes onto hardware.

As stated in Chapter 7, a group in MPI is an ordered set of n process identifiers (henceforth MPI processes). Each MPI process in the group is assigned a rank between 0 and $n-1$. In many parallel applications, a linear assignment of integer ranks to the MPI processes does not adequately reflect the logical communication pattern of the MPI processes (which is usually determined by the underlying problem geometry and the numerical algorithm used). Often the MPI processes are arranged in topological patterns such as two- or three-dimensional grids. More generally, the logical MPI process arrangement is described by a graph. In this chapter we will refer to this logical MPI process arrangement as the *virtual topology*.

A clear distinction must be made between the *virtual topology* and the topology of the underlying, physical hardware. The *virtual topology* can be exploited by the system in the assignment of processes to physical processors, if this helps to improve the communication performance on a given machine. How this mapping is done, however, is outside the scope of MPI. The description of the *virtual topology*, on the other hand, depends only on the application, and is machine-independent. The functions that are described in this chapter deal with machine-independent mapping and communication on *virtual topologies*.

Rationale. Though physical mapping is not discussed, the existence of the *virtual topology* information may be used as advice by the runtime system. There are well-known techniques for mapping grid/torus structures to hardware topologies such as hypercubes or grids. For more complicated graph structures good heuristics often yield nearly optimal results [51]. On the other hand, if there is no way for the user to specify the logical process arrangement as a *virtual topology*, a random mapping is most likely to result. On some machines, this will lead to unnecessary contention in the interconnection network. Some details about predicted and measured performance improvements that result from good process-to-processor mapping on wormhole-routing architectures can be found in [13, 14].

Besides possible performance benefits, the *virtual topology* can function as a convenient, process-naming structure, with significant benefits for program readability and notational power in message-passing programming. (*End of rationale.*)

8.2 Virtual Topologies

The communication pattern of a set of MPI processes can be represented by a graph. The nodes represent MPI processes, and the edges connect MPI processes that communicate with each other. MPI provides message-passing between any pair of MPI processes in a group. There is no requirement for opening a channel explicitly. Therefore, a “missing link” in the user-defined graph of MPI processes does not prevent the corresponding MPI processes from exchanging messages. It means rather that this connection is neglected in the *virtual topology*. This strategy implies that the *virtual topology* gives no convenient way of naming this pathway of communication. Another possible consequence is that an automatic mapping tool (if one exists for the runtime environment) will not take account of this edge when mapping.

Specifying the *virtual topology* in terms of a graph is sufficient for all applications. However, in many applications the graph structure is regular, and the detailed set-up of the graph would be inconvenient for the user and might be less efficient at run time. A large fraction of all parallel applications use MPI process topologies like rings, two- or higher-dimensional grids, or tori. These structures are completely defined by the number of dimensions and the numbers of MPI processes in each coordinate direction. Also, the mapping of grids and tori is generally an easier problem than that of general graphs. Thus, it is desirable to address these cases explicitly.

The coordinates of MPI processes in a Cartesian structure begin their numbering at 0. Row-major numbering is always used for the MPI processes in a Cartesian structure. This means that, for example, for four MPI processes in a (2×2) grid, the relationship between their ranks in the group and their coordinates in the *virtual topology* is as follows:

```
coord (0,0):  rank 0
coord (0,1):  rank 1
coord (1,0):  rank 2
coord (1,1):  rank 3
```

8.3 Embedding in MPI

The support for *virtual topologies* as defined in this chapter is consistent with other parts of MPI, and, whenever possible, makes use of functions that are defined elsewhere. Topology information is associated with communicators. It is added to communicators using the caching mechanism described in Chapter 7.

Information representing a *virtual topology* may be added to a communicator at the time of its creation. If a communicator creation function adds information representing a *virtual topology* to the output communicator it creates, then it either propagates the topology representation from the input communicator to the output communicator, or adds a new topology representation generated from the input parameters that describe a *virtual topology*. The description of every MPI communicator creation function explicitly states how topology information is handled. Communicator creation functions that create new topology representations are described in Section 8.5.

8.4 Overview of the Functions

MPI supports three types of *virtual topology*: **Cartesian**, **graph**, and **distributed graph**. The function `MPI_CART_CREATE` can be used to create Cartesian topologies, the function `MPI_GRAPH_CREATE` can be used to create graph topologies, and the functions `MPI_DIST_GRAPH_CREATE_ADJACENT` and `MPI_DIST_GRAPH_CREATE` can be used to create distributed graph topologies. These topology creation functions are collective. As with other collective calls, the program must be written to work correctly, whether the call synchronizes or not.

The above topology creation functions take as input an existing communicator `comm_old`, which defines the set of MPI processes on which the topology is to be mapped. For `MPI_GRAPH_CREATE` and `MPI_CART_CREATE`, all input arguments must have identical values on all MPI processes of the group of `comm_old`. When calling `MPI_GRAPH_CREATE`, each MPI process specifies all nodes and edges in the graph. In contrast, the functions `MPI_DIST_GRAPH_CREATE_ADJACENT` or `MPI_DIST_GRAPH_CREATE` are used to specify the graph in a distributed fashion, whereby each MPI process only specifies a subset of the edges in the graph such that the entire graph structure is defined collectively across the set of MPI processes. Therefore the MPI processes provide different values for the arguments specifying the graph. However, all MPI processes must give the same value for `reorder` and the `info` argument. In all cases, a new communicator `comm_topol` is created that carries the topological structure as cached information (see Chapter 7). In analogy to function `MPI_COMM_CREATE`, no cached information propagates from `comm_old` to `comm_topol`.

`MPI_CART_CREATE` can be used to describe Cartesian structures of arbitrary dimension. For each coordinate direction one specifies whether the MPI process structure is periodic or not. Note that an n -dimensional hypercube is an n -dimensional torus with two processes per coordinate direction. Thus, special support for hypercube structures is not necessary. The local auxiliary function `MPI_DIMS_CREATE` can be used to compute a balanced distribution of MPI processes among a given number of dimensions.

MPI defines functions to query a communicator for topology information. The function `MPI_TOPO_TEST` is used to query for the type of topology associated with a communicator. Depending on the topology type, different information can be extracted. For a graph topology, the functions `MPI_GRAPHDIMS_GET` and `MPI_GRAPH_GET` retrieve the graph topology information that is associated with the communicator. Additionally, the functions `MPI_GRAPH_NEIGHBORS_COUNT` and `MPI_GRAPH_NEIGHBORS` can be used to obtain the neighbors of an arbitrary node in the graph. For a distributed graph topology, the functions `MPI_DIST_GRAPH_NEIGHBORS_COUNT` and `MPI_DIST_GRAPH_NEIGHBORS` can be used to obtain the neighbors of the calling MPI process. For a Cartesian topology, the function `MPI_CARTDIM_GET` returns the number of dimensions and `MPI_CART_GET` returns the numbers of MPI processes in each dimension and periodicity of the associated Cartesian topology. Additionally, the functions `MPI_CART_RANK` and `MPI_CART_COORDS` translate Cartesian coordinates into a group rank, and vice-versa. The function `MPI_CART_SHIFT` provides the information needed to communicate with neighbors along a Cartesian dimension. All of these query functions are local.

For Cartesian topologies, the function `MPI_CART_SUB` can be used to extract a Cartesian subspace (analogous to `MPI_COMM_SPLIT`). This function is collective over the input communicator's group.

The two additional functions, `MPI_GRAPH_MAP` and `MPI_CART_MAP`, are, in general, not called by the user directly. However, together with the communicator manipulation functions presented in Chapter 7, they are sufficient to implement all other topology functions. Section 8.5.8 outlines such an implementation.

The neighborhood collective communication routines `MPI_NEIGHBOR_ALLGATHER`, `MPI_NEIGHBOR_ALLGATHERV`, `MPI_NEIGHBOR_ALLTOALL`, `MPI_NEIGHBOR_ALLTOALLV`, and `MPI_NEIGHBOR_ALLTOALLW` communicate with the nearest neighbors on the topology associated with the communicator. The nonblocking variants are `MPI_INEIGHBOR_ALLGATHER`, `MPI_INEIGHBOR_ALLGATHERV`, `MPI_INEIGHBOR_ALLTOALL`, `MPI_INEIGHBOR_ALLTOALLV`, and `MPI_INEIGHBOR_ALLTOALLW`.

8.5 Topology Constructors

8.5.1 Cartesian Constructor

`MPI_CART_CREATE(comm_old, ndims, dims, periods, reorder, comm_cart)`

IN	<code>comm_old</code>	input communicator (handle)
IN	<code>ndims</code>	number of dimensions of Cartesian grid (integer)
IN	<code>dims</code>	integer array of size <code>ndims</code> specifying the number of processes in each dimension
IN	<code>periods</code>	logical array of size <code>ndims</code> specifying whether the grid is periodic (<code>true</code>) or not (<code>false</code>) in each dimension
IN	<code>reorder</code>	ranks may be reordered (<code>true</code>) or not (<code>false</code>) (logical)
OUT	<code>comm_cart</code>	new communicator with associated Cartesian topology (handle)

C binding

```
int MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[],
                   const int periods[], int reorder, MPI_Comm *comm_cart)
```

Fortran 2008 binding

```
MPI_Cart_create(comm_old, ndims, dims, periods, reorder, comm_cart, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm_old
  INTEGER, INTENT(IN) :: ndims, dims(ndims)
  LOGICAL, INTENT(IN) :: periods(ndims), reorder
  TYPE(MPI_Comm), INTENT(OUT) :: comm_cart
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_CART_CREATE(COMM_OLD, NDIMS, DIMS, PERIODS, REORDER, COMM_CART, IERROR)
  INTEGER COMM_OLD, NDIMS, DIMS(*), COMM_CART, IERROR
  LOGICAL PERIODS(*), REORDER
```

`MPI_CART_CREATE` returns a handle to a new communicator to which the Cartesian topology information is attached. If `reorder = false` then the rank of each MPI process in the group of the new communicator is identical to its rank in the group of the old communicator.

If `reorder = true` then the procedure may reorder the ranks of the MPI processes (possibly so as to choose a good embedding of the *virtual topology* onto the physical machine). If the total size of the Cartesian grid is smaller than the size of the group of `comm_old`, then some MPI processes return `MPI_COMM_NULL`, in analogy to `MPI_COMM_SPLIT`. If `ndims` is zero then a zero-dimensional Cartesian topology is created. The call is erroneous if it specifies a grid that is larger than the group size or if `ndims` is negative. `MPI_CART_CREATE` will associate information representing a Cartesian topology with the specified number of dimensions, numbers of MPI processes in each coordinate direction, and periodicity with the new communicator.

8.5.2 Cartesian Convenience Function: `MPI_DIMS_CREATE`

For Cartesian topologies, the function `MPI_DIMS_CREATE` helps the user select a balanced distribution of MPI processes per coordinate direction, depending on the number of MPI processes in the group to be balanced and optional constraints that can be specified by the user.

`MPI_DIMS_CREATE`(`nnodes`, `ndims`, `dims`)

IN	<code>nnodes</code>	number of nodes in a grid (integer)
IN	<code>ndims</code>	number of Cartesian dimensions (integer)
INOUT	<code>dims</code>	integer array of size <code>ndims</code> specifying the number of nodes in each dimension

C binding

```
int MPI_Dims_create(int nnodes, int ndims, int dims[])
```

Fortran 2008 binding

```
MPI_Dims_create(nnodes, ndims, dims, ierror)
  INTEGER, INTENT(IN) :: nnodes, ndims
  INTEGER, INTENT(INOUT) :: dims(ndims)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_DIMS_CREATE(NNODES, NDIMS, DIMS, IERROR)
  INTEGER NNODES, NDIMS, DIMS(*), IERROR
```

The entries in the array `dims` are set to describe a Cartesian grid with `ndims` dimensions and a total of `nnodes` nodes. The dimensions are set to be as close to each other as possible, using an appropriate divisibility algorithm. The caller may further constrain the operation of this routine by specifying elements of array `dims`. If `dims[i]` is set to a positive number, the routine will not modify the number of nodes in dimension `i`; only those entries where `dims[i] = 0` are modified by the call.

Negative input values of `dims[i]` are erroneous. An error will occur if `nnodes` is not a multiple of

$$\prod_{i, \text{dims}[i] \neq 0} \text{dims}[i].$$

For `dims[i]` set by the call, `dims[i]` will be ordered in nonincreasing order. Array `dims` is suitable for use as input to routine `MPI_CART_CREATE`. `MPI_DIMS_CREATE` is local. If `ndims` is zero and `nnodes` is one, `MPI_DIMS_CREATE` returns `MPI_SUCCESS`.

Example 8.1. The use of the array argument `dims` in `MPI_DIMS_CREATE`.

dims before call	function call	dims on return
(0,0)	<code>MPI_DIMS_CREATE(6, 2, dims)</code>	(3,2)
(0,0)	<code>MPI_DIMS_CREATE(7, 2, dims)</code>	(7,1)
(0,3,0)	<code>MPI_DIMS_CREATE(6, 3, dims)</code>	(2,3,1)
(0,3,0)	<code>MPI_DIMS_CREATE(7, 3, dims)</code>	erroneous call

8.5.3 Graph Constructor

`MPI_GRAPH_CREATE(comm_old, nnodes, index, edges, reorder, comm_graph)`

IN	<code>comm_old</code>	input communicator (handle)
IN	<code>nnodes</code>	number of nodes in graph (integer)
IN	<code>index</code>	array of integers describing node degrees (see below)
IN	<code>edges</code>	array of integers describing graph edges (see below)
IN	<code>reorder</code>	ranks may be reordered (true) or not (false) (logical)
OUT	<code>comm_graph</code>	new communicator with associated graph topology (handle)

C binding

```
int MPI_Graph_create(MPI_Comm comm_old, int nnodes, const int index[],
                    const int edges[], int reorder, MPI_Comm *comm_graph)
```

Fortran 2008 binding

```
MPI_Graph_create(comm_old, nnodes, index, edges, reorder, comm_graph, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm_old
  INTEGER, INTENT(IN) :: nnodes, index(nnodes), edges(*)
  LOGICAL, INTENT(IN) :: reorder
  TYPE(MPI_Comm), INTENT(OUT) :: comm_graph
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GRAPH_CREATE(COMM_OLD, NNODES, INDEX, EDGES, REORDER, COMM_GRAPH, IERROR)
  INTEGER COMM_OLD, NNODES, INDEX(*), EDGES(*), COMM_GRAPH, IERROR
  LOGICAL REORDER
```

`MPI_GRAPH_CREATE` returns a handle to a new communicator to which the graph topology information is attached. If `reorder = false` then the rank of each MPI process in the group of the new communicator is identical to its rank in the group of the old communicator. If `reorder = true` then the procedure may reorder the ranks of the MPI processes. If the number of nodes in the graph (`nnodes`) is smaller than the size of the group of `comm_old`, then `MPI_COMM_NULL` is returned by some MPI processes, in analogy to `MPI_CART_CREATE` and `MPI_COMM_SPLIT`. If the graph is empty, i.e., `nnodes = 0`, then `MPI_COMM_NULL` is returned in all MPI processes. The call is erroneous if it specifies a graph that is larger than the group size of the input communicator.

The three parameters `nnodes`, `index` and `edges` define the graph structure. `nnodes` is the number of nodes of the graph. The nodes are numbered from 0 to `nnodes-1`. The i -th entry of array `index` stores the total number of neighbors of the first i graph nodes. The lists of neighbors of nodes 0, 1, ..., `nnodes-1` are stored in consecutive locations in array `edges`. The array `edges` is a flattened representation of the edge lists. The total number of entries in `index` is `nnodes` and the total number of entries in `edges` is equal to the number of graph edges.

The definitions of the arguments `nnodes`, `index`, and `edges` are illustrated with the following simple example.

Example 8.2. Specification of the adjacency matrix for `MPI_GRAPH_CREATE`. Assume there are four MPI processes with ranks 0, 1, 2, 3 in the input communicator with the following adjacency matrix:

MPI process	neighbors
0	1, 3
1	0
2	3
3	0, 2

Then, the input arguments are:

```
nnodes = 4
index = 2, 3, 4, 6
edges = 1, 3, 0, 3, 0, 2
```

Thus, in C, `index[0]` is the degree of node zero, and `index[i] - index[i-1]` is the degree of node i , $i=1, \dots, \text{nnodes}-1$; the list of neighbors of node zero is stored in `edges[j]`, for $0 \leq j \leq \text{index}[0] - 1$ and the list of neighbors of node i , $i > 0$, is stored in `edges[j]`, $\text{index}[i-1] \leq j \leq \text{index}[i] - 1$.

In Fortran, `index(1)` is the degree of node zero, and `index(i+1) - index(i)` is the degree of node i , $i=1, \dots, \text{nnodes}-1$; the list of neighbors of node zero is stored in `edges(j)`, for $1 \leq j \leq \text{index}(1)$ and the list of neighbors of node i , $i > 0$, is stored in `edges(j)`, $\text{index}(i)+1 \leq j \leq \text{index}(i+1)$.

A single MPI process is allowed to be defined multiple times in the list of neighbors of an MPI process (i.e., there may be multiple edges between two MPI processes). An MPI process is also allowed to be a neighbor to itself (i.e., a self loop in the graph). The adjacency matrix is allowed to be nonsymmetric.

Advice to users. Performance implications of using multiple edges or a nonsymmetric adjacency matrix are not defined. The definition of a node-neighbor edge does not imply a direction of the communication. (*End of advice to users.*)

Advice to implementors. The following topology information is likely to be stored with a communicator:

- Type of topology (Cartesian/graph)
- For a Cartesian topology:
 1. `ndims` (number of dimensions)

2. `dims` (numbers of MPI processes per coordinate direction)
 3. `periods` (periodicity information)
 4. `own_position` (own position in grid, could also be computed from rank and `dims`)
- For a graph topology:
 1. `index`
 2. `edges`
 which are the arrays defining the graph structure.

For a graph structure the number of nodes is equal to the number of MPI processes in the group. Therefore, the number of nodes does not have to be stored explicitly. An additional zero entry at the start of array `index` simplifies access to the topology information. (*End of advice to implementors.*)

8.5.4 Distributed Graph Constructor

`MPI_GRAPH_CREATE` requires that each MPI process passes the full (global) communication graph to the call. This limits the scalability of this constructor. With the distributed graph interface, the communication graph is specified in a fully distributed fashion. Each MPI process specifies only the part of the communication graph of which it is aware. Typically, this could be the set of MPI processes from which the MPI process will eventually receive or get data, or the set of MPI processes to which the MPI process will send or put data, or some combination of such edges. Two different interfaces can be used to create a distributed graph topology. `MPI_DIST_GRAPH_CREATE_ADJACENT` creates a distributed graph communicator with each MPI process specifying each of its incoming and outgoing (adjacent) edges in the logical communication graph and thus requires minimal communication during creation. `MPI_DIST_GRAPH_CREATE` provides full flexibility such that any MPI process can indicate that communication will occur between any pair of MPI processes in the graph.

To provide better possibilities for optimization by the MPI library, the distributed graph constructors permit weighted communication edges and take an `info` argument that can further influence process reordering or other optimizations performed by the MPI library. For example, hints can be provided on how edge weights are to be interpreted, the quality of the reordering, and/or the time permitted for the MPI library to process the graph.

`MPI_DIST_GRAPH_CREATE_ADJACENT(comm_old, indegree, sources, sourceweights, outdegree, destinations, destweights, info, reorder, comm_dist_graph)`

IN	<code>comm_old</code>	input communicator (handle)
IN	<code>indegree</code>	size of <code>sources</code> and <code>sourceweights</code> arrays (nonnegative integer)
IN	<code>sources</code>	ranks of MPI processes for which the calling process is a destination (array of nonnegative integers)
IN	<code>sourceweights</code>	weights of the edges into the calling MPI process (array of nonnegative integers)
IN	<code>outdegree</code>	size of <code>destinations</code> and <code>destweights</code> arrays (nonnegative integer)

IN	destinations	ranks of MPI processes for which the calling MPI process is a source (array of nonnegative integers)	1
IN	destweights	weights of the edges out of the calling MPI process (array of nonnegative integers)	2
IN	info	hints on optimization and interpretation of weights (handle)	3
IN	reorder	ranks may be reordered (true) or not (false) (logical)	4
OUT	comm_dist_graph	new communicator with associated distributed graph topology (handle)	5

C binding

```

int MPI_Dist_graph_create_adjacent(MPI_Comm comm_old, int indegree,
    const int sources[], const int sourceweights[], int outdegree,
    const int destinations[], const int destweights[], MPI_Info info,
    int reorder, MPI_Comm *comm_dist_graph)

```

Fortran 2008 binding

```

MPI_Dist_graph_create_adjacent(comm_old, indegree, sources, sourceweights,
    outdegree, destinations, destweights, info, reorder,
    comm_dist_graph, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm_old
INTEGER, INTENT(IN) :: indegree, sources(indegree), sourceweights(*),
    outdegree, destinations(outdegree), destweights(*)
TYPE(MPI_Info), INTENT(IN) :: info
LOGICAL, INTENT(IN) :: reorder
TYPE(MPI_Comm), INTENT(OUT) :: comm_dist_graph
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_DIST_GRAPH_CREATE_ADJACENT(COMM_OLD, INDEGREE, SOURCES, SOURCEWEIGHTS,
    OUTDEGREE, DESTINATIONS, DESTWEIGHTS, INFO, REORDER,
    COMM_DIST_GRAPH, IERROR)
INTEGER COMM_OLD, INDEGREE, SOURCES(*), SOURCEWEIGHTS(*), OUTDEGREE,
    DESTINATIONS(*), DESTWEIGHTS(*), INFO, COMM_DIST_GRAPH, IERROR
LOGICAL REORDER

```

`MPI_DIST_GRAPH_CREATE_ADJACENT` returns a handle to a new communicator to which the distributed graph topology information is attached. Each MPI process passes all information about its incoming and outgoing edges in the virtual distributed graph topology. The calling MPI processes must ensure that each edge of the graph is described in the source and in the destination process with the same weights. If there are multiple edges for a given (source,dest) pair, then the sequence of the weights of these edges does not matter. The complete communication topology is the combination of all edges shown in the sources arrays of all MPI processes in comm_old, which must be identical to the combination of all edges shown in the destinations arrays. Source and destination MPI processes must be specified by their rank in the group of comm_old. This allows a fully distributed specification of the communication graph. Isolated MPI processes (i.e., MPI processes with no outgoing or incoming edges, that is, MPI processes that have specified indegree and outdegree as zero and thus do not occur as source or destination in the graph specification) are allowed.

The call creates a new communicator `comm_dist_graph` of distributed graph topology type to which topology information has been attached. The number of MPI processes in `comm_dist_graph` is identical to the number of MPI processes in `comm_old`. The call to `MPI_DIST_GRAPH_CREATE_ADJACENT` is collective.

Weights are specified as nonnegative integers and can be used to influence the process mapping strategy and other internal MPI optimizations. For instance, approximate count arguments of later communication calls along specific edges could be used as their edge weights. Multiplicity of edges can likewise indicate more intense communication between pairs of MPI processes. However, the exact meaning of edge weights is not specified by the MPI standard and is left to the implementation. In C or Fortran, an application can supply the special value `MPI_UNWEIGHTED` for the weight array to indicate that all edges have the same (effectively no) weight. It is erroneous to supply `MPI_UNWEIGHTED` for some but not all MPI processes of `comm_old`. If the graph is weighted but indegree or outdegree is zero, then `MPI_WEIGHTS_EMPTY` or any arbitrary array may be passed to `sourceweights` or `destweights` respectively. Note that `MPI_UNWEIGHTED` and `MPI_WEIGHTS_EMPTY` are not special weight values; rather they are special values for the total array argument. In Fortran, `MPI_UNWEIGHTED` and `MPI_WEIGHTS_EMPTY` are objects like `MPI_BOTTOM` (not usable for initialization or assignment). See Section 2.5.4.

Advice to users. In the case of an empty weights array argument passed while constructing a weighted graph, one should not pass NULL because the value of `MPI_UNWEIGHTED` may be equal to NULL. The value of this argument would then be indistinguishable from `MPI_UNWEIGHTED` to the implementation. In this case `MPI_WEIGHTS_EMPTY` should be used instead. (*End of advice to users.*)

Advice to implementors. It is recommended that `MPI_UNWEIGHTED` not be implemented as NULL. (*End of advice to implementors.*)

Rationale. To ensure backward compatibility, `MPI_UNWEIGHTED` may still be implemented as NULL. See Annex B.6. (*End of rationale.*)

The meaning of the `info` and `reorder` arguments is defined in the description of the following routine.

`MPI_DIST_GRAPH_CREATE(comm_old, n, sources, degrees, destinations, weights, info, reorder, comm_dist_graph)`

IN	<code>comm_old</code>	input communicator (handle)
IN	<code>n</code>	number of source nodes for which this MPI process specifies edges (nonnegative integer)
IN	<code>sources</code>	array containing the <code>n</code> source nodes for which this MPI process specifies edges (array of nonnegative integers)
IN	<code>degrees</code>	array specifying the number of destinations for each source node in the source node array (array of nonnegative integers)
IN	<code>destinations</code>	destination nodes for the source nodes in the source node array (array of nonnegative integers)

IN	weights	weights for source to destination edges (array of nonnegative integers)	1
IN	info	hints on optimization and interpretation of weights (handle)	2
IN	reorder	ranks may be reordered (true) or not (false) (logical)	3
OUT	comm_dist_graph	new communicator with associated distributed graph topology (handle)	4

C binding

```
int MPI_Dist_graph_create(MPI_Comm comm_old, int n, const int sources[],
                        const int degrees[], const int destinations[],
                        const int weights[], MPI_Info info, int reorder,
                        MPI_Comm *comm_dist_graph)
```

Fortran 2008 binding

```
MPI_Dist_graph_create(comm_old, n, sources, degrees, destinations, weights,
                    info, reorder, comm_dist_graph, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm_old
INTEGER, INTENT(IN) :: n, sources(n), degrees(n), destinations(*),
                    weights(*)
TYPE(MPI_Info), INTENT(IN) :: info
LOGICAL, INTENT(IN) :: reorder
TYPE(MPI_Comm), INTENT(OUT) :: comm_dist_graph
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_DIST_GRAPH_CREATE(COMM_OLD, N, SOURCES, DEGREES, DESTINATIONS, WEIGHTS,
                    INFO, REORDER, COMM_DIST_GRAPH, IERROR)
INTEGER COMM_OLD, N, SOURCES(*), DEGREES(*), DESTINATIONS(*), WEIGHTS(*),
                    INFO, COMM_DIST_GRAPH, IERROR
LOGICAL REORDER
```

MPI_DIST_GRAPH_CREATE returns a handle to a new communicator to which the distributed graph topology information is attached. Concretely, each MPI process calls the constructor with a set of directed (source,destination) communication edges as described below. Every MPI process passes an array of n source nodes in the **sources** array. For each source node, a nonnegative number of destination nodes is specified in the **degrees** array. The destination nodes are stored in the corresponding consecutive segment of the **destinations** array. More precisely, if the i -th node in **sources** is s , this specifies **degrees**[i] edges (s,d) with d of the j -th such edge stored in **destinations**[**degrees**[0]+...+**degrees**[$i-1$]+ j]. The weight of this edge is stored in **weights**[**degrees**[0]+...+**degrees**[$i-1$]+ j]. Both the **sources** and the **destinations** arrays may contain the same node more than once, and the order in which nodes are listed as destinations or sources is not significant. Similarly, different processes may specify edges with the same source and destination nodes. Source and destination nodes must be specified by their rank in the group of **comm_old**. Different MPI processes may specify different numbers of source and destination nodes, as well as different source to destination edges. This allows a fully distributed specification of the communication graph. Isolated MPI processes (i.e., MPI processes with no outgoing or incoming edges, that is, MPI processes that do not occur as source or destination node in the graph specification) are allowed.

The call creates a new communicator `comm_dist_graph` of distributed graph topology type to which topology information has been attached. The number of MPI processes in `comm_dist_graph` is identical to the number of MPI processes in `comm_old`. The call to `MPI_DIST_GRAPH_CREATE` is collective.

If `reorder = false`, all MPI processes will have the same rank in `comm_dist_graph` as in `comm_old`. If `reorder = true` then the MPI library is free to remap to other MPI processes (of `comm_old`) in order to improve communication on the edges of the communication graph. The weight associated with each edge is a hint to the MPI library about the amount or intensity of communication on that edge, and may be used to compute a “best” reordering.

Weights are specified as nonnegative integers and can be used to influence the MPI process remapping strategy and other internal MPI optimizations. For instance, approximate count arguments of later communication calls along specific edges could be used as their edge weights. Multiplicity of edges can likewise indicate more intense communication between pairs of MPI processes. However, the exact meaning of edge weights and multiplicity of edges is not specified by the MPI standard and is left to the implementation. In C or Fortran, an application can supply the special value `MPI_UNWEIGHTED` for the weight array to indicate that all edges have the same (effectively no) weight. It is erroneous to supply `MPI_UNWEIGHTED` for some but not all MPI processes of `comm_old`. If the graph is weighted but `n = 0`, then `MPI_WEIGHTS_EMPTY` or any arbitrary array may be passed to weights. Note that `MPI_UNWEIGHTED` and `MPI_WEIGHTS_EMPTY` are not special weight values; rather they are special values for the total array argument. In Fortran, `MPI_UNWEIGHTED` and `MPI_WEIGHTS_EMPTY` are objects like `MPI_BOTTOM` (not usable for initialization or assignment). See Section 2.5.4.

Advice to users. In the case of an empty weights array argument passed while constructing a weighted graph, one should not pass NULL because the value of `MPI_UNWEIGHTED` may be equal to NULL. The value of this argument would then be indistinguishable from `MPI_UNWEIGHTED` to the implementation. `MPI_WEIGHTS_EMPTY` should be used instead. (*End of advice to users.*)

Advice to implementors. It is recommended that `MPI_UNWEIGHTED` not be implemented as NULL. (*End of advice to implementors.*)

Rationale. To ensure backward compatibility, `MPI_UNWEIGHTED` may still be implemented as NULL. See Annex B.6. (*End of rationale.*)

The meaning of the `weights` argument can be influenced by the `info` argument. The `info` argument can be used to guide the mapping of MPI processes to the hardware; possible options include minimizing the maximum number of edges between processes on different SMP nodes, or minimizing the sum of all such edges. As described in Section 10, an MPI implementation is not obliged to follow specific hints, and it is valid for an MPI implementation not to do any reordering. An MPI implementation may specify more `info` (key,value) pairs. All MPI processes must specify the same set of (key,value) `info` pairs.

Advice to implementors. MPI implementations must document every additionally supported (key,value) `info` pair. `MPI_INFO_NULL` is always valid, and may indicate the default creation of the distributed graph topology to the MPI library.

An implementation does not explicitly need to construct the topology from its distributed parts. However, all MPI processes can construct the full topology from the distributed specification and use this in a call to `MPI_GRAPH_CREATE` to create the topology. This may serve as a reference implementation of the functionality, and may be acceptable for small communicators. However, a scalable high-quality implementation would save the topology graph in a distributed way. (*End of advice to implementors.*)

Example 8.3. Several ways to specify the adjacency matrix for `MPI_DIST_GRAPH_CREATE` and `MPI_DIST_GRAPH_CREATE_ADJACENT`.

As for Example 8.2, assume there are four MPI processes with ranks 0, 1, 2, 3 in the input communicator with the following adjacency matrix and unit edge weights:

MPI process	neighbors
0	1, 3
1	0
2	3
3	0, 2

With `MPI_DIST_GRAPH_CREATE`, this graph could be constructed in many different ways. One way would be that each MPI process specifies its outgoing edges. The arguments per MPI process would be:

MPI process	n	sources	degrees	destinations	weights
0	1	0	2	1,3	1,1
1	1	1	1	0	1
2	1	2	1	3	1
3	1	3	2	0,2	1,1

Another way would be to pass the whole graph on MPI process with rank 0 in the input communicator, which could be done with the following arguments per MPI process:

MPI process	n	sources	degrees	destinations	weights
0	4	0,1,2,3	2,1,1,2	1,3,0,3,0,2	1,1,1,1,1,1
1	0	-	-	-	-
2	0	-	-	-	-
3	0	-	-	-	-

In both cases above, the application could supply `MPI_UNWEIGHTED` instead of explicitly providing identical weights.

`MPI_DIST_GRAPH_CREATE_ADJACENT` could be used to specify this graph using the following arguments:

MPI process	indegree	sources	sourceweights	outdegree	destinations	destweights
0	2	1,3	1,1	2	1,3	1,1
1	1	0	1	1	0	1
2	1	3	1	1	3	1
3	2	0,2	1,1	2	0,2	1,1

Example 8.4. Cartesian grid plus diagonals specified with `MPI_DIST_GRAPH_CREATE`. A two-dimensional $P \times Q$ torus where all MPI processes communicate along the dimensions and along the diagonal edges cannot be modeled with Cartesian topologies, but can easily be captured with `MPI_DIST_GRAPH_CREATE` as shown in the following code. In this example, the communication along the dimensions is twice as heavy as the communication along the diagonals:

```

/*
Input:      dimensions P, Q
Condition: number of MPI processes equal to P*Q
*/
int rank, x, y;
int sources[1], degrees[1];
int destinations[8], weights[8];
MPI_Comm comm_dist_graph;

MPI_Comm_rank(MPI_COMM_WORLD, &rank);

/* get x and y dimension */
y=rank/P; x=rank%P;

/* get my communication partners along x dimension */
destinations[0] = P*y+(x+1)%P; weights[0] = 2;
destinations[1] = P*y+(P+x-1)%P; weights[1] = 2;

/* get my communication partners along y dimension */
destinations[2] = P*((y+1)%Q)+x; weights[2] = 2;
destinations[3] = P*((Q+y-1)%Q)+x; weights[3] = 2;

/* get my communication partners along diagonals */
destinations[4] = P*((y+1)%Q)+(x+1)%P; weights[4] = 1;
destinations[5] = P*((Q+y-1)%Q)+(x+1)%P; weights[5] = 1;
destinations[6] = P*((y+1)%Q)+(P+x-1)%P; weights[6] = 1;
destinations[7] = P*((Q+y-1)%Q)+(P+x-1)%P; weights[7] = 1;

sources[0] = rank;
degrees[0] = 8;
MPI_Dist_graph_create(MPI_COMM_WORLD, 1, sources, degrees, destinations,
                      weights, MPI_INFO_NULL, 1, &comm_dist_graph);

```

8.5.5 Topology Inquiry Functions

If a *virtual topology* has been defined with one of the above functions, then the topology information can be looked up using inquiry functions. They all are local calls.

`MPI_TOPO_TEST(comm, status)`

IN	comm	communicator (handle)
OUT	status	topology type of communicator comm (state)

C binding

```
int MPI_Topo_test(MPI_Comm comm, int *status)
```

Fortran 2008 binding

```
MPI_Topo_test(comm, status, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(OUT) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TOPO_TEST(COMM, STATUS, IERROR)
  INTEGER COMM, STATUS, IERROR
```

The function [MPI_TOPO_TEST](#) returns the type of topology that is associated with a communicator.

The output value `status` is one of the following:

<code>MPI_GRAPH</code>	graph topology
<code>MPI_CART</code>	Cartesian topology
<code>MPI_DIST_GRAPH</code>	distributed graph topology
<code>MPI_UNDEFINED</code>	no topology

```
MPI_GRAPHDIMS_GET(comm, nnodes, nedges)
```

IN	<code>comm</code>	communicator with associated graph topology (handle)
OUT	<code>nnodes</code>	number of nodes in graph (same as number of MPI processes in the group of <code>comm</code>) (integer)
OUT	<code>nedges</code>	number of edges in graph (integer)

C binding

```
int MPI_Graphdims_get(MPI_Comm comm, int *nnodes, int *nedges)
```

Fortran 2008 binding

```
MPI_Graphdims_get(comm, nnodes, nedges, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(OUT) :: nnodes, nedges
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GRAPHDIMS_GET(COMM, NNODES, NEDGES, IERROR)
  INTEGER COMM, NNODES, NEDGES, IERROR
```

The functions [MPI_GRAPHDIMS_GET](#) and [MPI_GRAPH_GET](#) retrieve the graph topology information that is associated with the communicator. The information provided by [MPI_GRAPHDIMS_GET](#) can be used to dimension the vectors `index` and `edges` correctly for the following call to [MPI_GRAPH_GET](#).


```
1 MPI_GRAPH_GET(comm, maxindex, maxedges, index, edges)
```

2	IN	comm	communicator with associated graph topology (handle)
4	IN	maxindex	length of vector index in the calling program (integer)
6	IN	maxedges	length of vector edges in the calling program (integer)
8	OUT	index	array of integers containing the graph structure (for details see the definition of MPI_GRAPH_CREATE)
10	OUT	edges	array of integers containing the graph structure

12 C binding

```
13 int MPI_Graph_get(MPI_Comm comm, int maxindex, int maxedges, int index[],
14                 int edges[])
```

16 Fortran 2008 binding

```
17 MPI_Graph_get(comm, maxindex, maxedges, index, edges, ierror)
18     TYPE(MPI_Comm), INTENT(IN) :: comm
19     INTEGER, INTENT(IN) :: maxindex, maxedges
20     INTEGER, INTENT(OUT) :: index(maxindex), edges(maxedges)
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

22 Fortran binding

```
23 MPI_GRAPH_GET(COMM, MAXINDEX, MAXEDGES, INDEX, EDGES, IERROR)
24     INTEGER COMM, MAXINDEX, MAXEDGES, INDEX(*), EDGES(*), IERROR
```

```
27 MPI_CARTDIM_GET(comm, ndims)
```

29	IN	comm	communicator with associated Cartesian topology (handle)
31	OUT	ndims	number of dimensions of the Cartesian structure (integer)

34 C binding

```
35 int MPI_Cartdim_get(MPI_Comm comm, int *ndims)
```

36 Fortran 2008 binding

```
37 MPI_Cartdim_get(comm, ndims, ierror)
38     TYPE(MPI_Comm), INTENT(IN) :: comm
39     INTEGER, INTENT(OUT) :: ndims
40     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

42 Fortran binding

```
43 MPI_CARTDIM_GET(COMM, NDIMS, IERROR)
44     INTEGER COMM, NDIMS, IERROR
```

45 The functions [MPI_CARTDIM_GET](#) and [MPI_CART_GET](#) return the Cartesian topology information that is associated with the communicator. If `comm` is associated with a zero-dimensional Cartesian topology, [MPI_CARTDIM_GET](#) returns `ndims = 0` and [MPI_CART_GET](#) will keep all output arguments unchanged.

MPI_CART_GET(comm, maxdims, dims, periods, coords)

IN	comm	communicator with associated Cartesian topology (handle)
IN	maxdims	length of vectors dims , periods , and coords in the calling program (integer)
OUT	dims	number of MPI processes for each Cartesian dimension (array of integers)
OUT	periods	periodicity (true/false) for each Cartesian dimension (array of logicals)
OUT	coords	coordinates of calling MPI process in Cartesian structure (array of integers)

C binding

```
int MPI_Cart_get(MPI_Comm comm, int maxdims, int dims[], int periods[],
                int coords[])
```

Fortran 2008 binding

```
MPI_Cart_get(comm, maxdims, dims, periods, coords, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: maxdims
  INTEGER, INTENT(OUT) :: dims(maxdims), coords(maxdims)
  LOGICAL, INTENT(OUT) :: periods(maxdims)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_CART_GET(COMM, MAXDIMS, DIMS, PERIODS, COORDS, IERROR)
  INTEGER COMM, MAXDIMS, DIMS(*), COORDS(*), IERROR
  LOGICAL PERIODS(*)
```

If **maxdims** in a call to **MPI_CART_GET** is less than the number of dimensions of the Cartesian topology associated with the communicator **comm**, the outcome is unspecified.

MPI_CART_RANK(comm, coords, rank)

IN	comm	communicator with associated Cartesian topology (handle)
IN	coords	integer array (of size ndims) specifying the Cartesian coordinates of an MPI process
OUT	rank	rank of specified MPI process within group of comm (integer)

C binding

```
int MPI_Cart_rank(MPI_Comm comm, const int coords[], int *rank)
```

Fortran 2008 binding

```
MPI_Cart_rank(comm, coords, rank, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: coords(*)
  INTEGER, INTENT(OUT) :: rank
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```

MPI_CART_RANK(COMM, COORDS, RANK, IERROR)
  INTEGER COMM, COORDS(*), RANK, IERROR

```

For a communicator with an associated Cartesian topology, the function **MPI_CART_RANK** translates the logical coordinates of an MPI process to the corresponding rank in the group of the communicator. For dimension i with `periods(i) = true`, if the coordinate, `coords(i)`, is out of range, that is, `coords(i) < 0` or `coords(i) ≥ dims(i)`, it is shifted back to the interval $0 ≤ \text{coords}(i) < \text{dims}(i)$ automatically. Out-of-range coordinates are erroneous for nonperiodic dimensions.

If `comm` is associated with a zero-dimensional Cartesian topology, `coords` is not significant and 0 is returned in `rank`.

```

MPI_CART_COORDS(comm, rank, maxdims, coords)

```

IN	comm	communicator with associated Cartesian topology (handle)
IN	rank	rank of an MPI process within group of comm (integer)
IN	maxdims	length of vector <code>coords</code> in the calling program (integer)
OUT	coords	coordinates of the MPI process with the rank <code>rank</code> in Cartesian structure (array of integers)

C binding

```

int MPI_Cart_coords(MPI_Comm comm, int rank, int maxdims, int coords[])

```

Fortran 2008 binding

```

MPI_Cart_coords(comm, rank, maxdims, coords, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: rank, maxdims
  INTEGER, INTENT(OUT) :: coords(maxdims)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_CART_COORDS(COMM, RANK, MAXDIMS, COORDS, IERROR)
  INTEGER COMM, RANK, MAXDIMS, COORDS(*), IERROR

```

The inverse mapping, rank-to-coordinates translation is provided by **MPI_CART_COORDS**. If `comm` is associated with a zero-dimensional Cartesian topology, `coords` will be unchanged. If `maxdims` is less than the number of dimensions of the Cartesian topology associated with the communicator `comm`, the outcome is unspecified.

```

MPI_GRAPH_NEIGHBORS_COUNT(comm, rank, nneighbors)

```

IN	comm	communicator with associated graph topology (handle)
IN	rank	rank of MPI process in group of comm (integer)
OUT	nneighbors	number of neighbors of specified MPI process (integer)

C binding

```
int MPI_Graph_neighbors_count(MPI_Comm comm, int rank, int *nneighbors)
```

Fortran 2008 binding

```
MPI_Graph_neighbors_count(comm, rank, nneighbors, ierror)
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
INTEGER, INTENT(IN) :: rank
```

```
INTEGER, INTENT(OUT) :: nneighbors
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GRAPH_NEIGHBORS_COUNT(COMM, RANK, NNEIGHBORS, IERROR)
```

```
INTEGER COMM, RANK, NNEIGHBORS, IERROR
```

```
MPI_GRAPH_NEIGHBORS(comm, rank, maxneighbors, neighbors)
```

IN	comm	communicator with associated graph topology (handle)
IN	rank	rank of MPI process in group of comm (integer)
IN	maxneighbors	size of array neighbors (integer)
OUT	neighbors	ranks of MPI processes that are neighbors to specified MPI process (array of integers)

C binding

```
int MPI_Graph_neighbors(MPI_Comm comm, int rank, int maxneighbors,  
                        int neighbors[])
```

Fortran 2008 binding

```
MPI_Graph_neighbors(comm, rank, maxneighbors, neighbors, ierror)
```

```
TYPE(MPI_Comm), INTENT(IN) :: comm
```

```
INTEGER, INTENT(IN) :: rank, maxneighbors
```

```
INTEGER, INTENT(OUT) :: neighbors(maxneighbors)
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GRAPH_NEIGHBORS(COMM, RANK, MAXNEIGHBORS, NEIGHBORS, IERROR)
```

```
INTEGER COMM, RANK, MAXNEIGHBORS, NEIGHBORS(*), IERROR
```

`MPI_GRAPH_NEIGHBORS_COUNT` and `MPI_GRAPH_NEIGHBORS` provide adjacency information for a graph topology. The returned count and array of neighbors for the queried rank will both include *all* neighbors and reflect the same edge ordering as was specified by the original call to `MPI_GRAPH_CREATE`. Specifically, `MPI_GRAPH_NEIGHBORS_COUNT` and `MPI_GRAPH_NEIGHBORS` will return values based on the original index and edges array passed to `MPI_GRAPH_CREATE` (for the purpose of this example, we assume that `index[-1]` is zero):

- The number of neighbors (`nneighbors`) returned from `MPI_GRAPH_NEIGHBORS_COUNT` will be `(index[rank] - index[rank-1])`.
- The neighbors array returned from `MPI_GRAPH_NEIGHBORS` will be `edges[index[rank-1]]` through `edges[index[rank]-1]`.

Example 8.5. Inquiry of graph topology information.

Assume there are four MPI processes with ranks 0, 1, 2, 3 in the input communicator with the following adjacency matrix (note that some neighbors are listed multiple times):

MPI process	neighbors
0	1, 1, 3
1	0, 0
2	3
3	0, 2, 2

Thus, the input arguments to `MPI_GRAPH_CREATE` are:

```
nnodes = 4
index = 3, 5, 6, 9
edges = 1, 1, 3, 0, 0, 3, 0, 2, 2
```

Therefore, calling `MPI_GRAPH_NEIGHBORS_COUNT` and `MPI_GRAPH_NEIGHBORS` for each of the four MPI processes will return:

Input rank	Count	Neighbors
0	3	1, 1, 3
1	2	0, 0
2	1	3
3	3	0, 2, 2

Example 8.6. Using a communicator with an associated graph topology that represents a shuffle-exchange network.

Suppose that `comm` is a communicator with a shuffle-exchange topology. The group has 2^n members. Each MPI process is labeled by $a_1, \dots, a_n$ with $a_i \in \{0, 1\}$, and has three neighbors: $\text{exchange}(a_1, \dots, a_n) = a_1, \dots, a_{n-1}, \bar{a}_n$ ($\bar{a} = 1 - a$), $\text{shuffle}(a_1, \dots, a_n) = a_2, \dots, a_n, a_1$, and $\text{unshuffle}(a_1, \dots, a_n) = a_n, a_1, \dots, a_{n-1}$. The graph adjacency list is illustrated below for $n = 3$.

node	exchange neighbors(1)	shuffle neighbors(2)	unshuffle neighbors(3)
0 (000)	1	0	0
1 (001)	0	2	4
2 (010)	3	4	1
3 (011)	2	6	5
4 (100)	5	1	2
5 (101)	4	3	6
6 (110)	7	5	3
7 (111)	6	7	7

Suppose that the communicator `comm` has this topology associated with it. The following code fragment cycles through the three types of neighbors and performs an appropriate permutation for each.

```
! assume: each MPI process has stored a real number A.
```

```

! extract neighborhood information
CALL MPI_COMM_RANK(comm, myrank, ierr)
CALL MPI_GRAPH_NEIGHBORS(comm, myrank, 3, neighbors, ierr)
! perform exchange permutation
CALL MPI_SENDRECV_REPLACE(A, 1, MPI_REAL, neighbors(1), 0, &
                        neighbors(1), 0, comm, status, ierr)
! perform shuffle permutation
CALL MPI_SENDRECV_REPLACE(A, 1, MPI_REAL, neighbors(2), 0, &
                        neighbors(3), 0, comm, status, ierr)
! perform unshuffle permutation
CALL MPI_SENDRECV_REPLACE(A, 1, MPI_REAL, neighbors(3), 0, &
                        neighbors(2), 0, comm, status, ierr)

```

`MPI_DIST_GRAPH_NEIGHBORS_COUNT` and `MPI_DIST_GRAPH_NEIGHBORS` provide adjacency information for a distributed graph topology.

`MPI_DIST_GRAPH_NEIGHBORS_COUNT(comm, indegree, outdegree, weighted)`

IN	comm	communicator with associated distributed graph topology (handle)
OUT	indegree	number of edges into this MPI process (nonnegative integer)
OUT	outdegree	number of edges out of this MPI process (nonnegative integer)
OUT	weighted	false if <code>MPI_UNWEIGHTED</code> was supplied during creation, true otherwise (logical)

C binding

```

int MPI_Dist_graph_neighbors_count(MPI_Comm comm, int *indegree,
                                   int *outdegree, int *weighted)

```

Fortran 2008 binding

```

MPI_Dist_graph_neighbors_count(comm, indegree, outdegree, weighted, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(OUT) :: indegree, outdegree
  LOGICAL, INTENT(OUT) :: weighted
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_DIST_GRAPH_NEIGHBORS_COUNT(COMM, INDEGREE, OUTDEGREE, WEIGHTED, IERROR)
  INTEGER COMM, INDEGREE, OUTDEGREE, IERROR
  LOGICAL WEIGHTED

```

`MPI_DIST_GRAPH_NEIGHBORS(comm, maxindegree, sources, sourceweights, maxoutdegree, destinations, destweights)`

IN	comm	communicator with associated distributed graph topology (handle)
IN	maxindegree	size of sources and sourceweights arrays (nonnegative integer)

1	OUT	sources	ranks of MPI processes for which the calling MPI
2			process is a destination (array of nonnegative
3			integers)
4	OUT	sourceweights	weights of the edges into the calling MPI process
5			(array of nonnegative integers)
6	IN	maxoutdegree	size of destinations and destweights arrays
7			(nonnegative integer)
8	OUT	destinations	ranks of MPI processes for which the calling MPI
9			process is a source (array of nonnegative integers)
10	OUT	destweights	weights of the edges out of the calling MPI process
11			(array of nonnegative integers)

C binding

```

14 int MPI_Dist_graph_neighbors(MPI_Comm comm, int maxindegree, int sources[],
15                             int sourceweights[], int maxoutdegree, int destinations[],
16                             int destweights[])

```

Fortran 2008 binding

```

18 MPI_Dist_graph_neighbors(comm, maxindegree, sources, sourceweights,
19                         maxoutdegree, destinations, destweights, ierror)
20
21 TYPE(MPI_Comm), INTENT(IN) :: comm
22 INTEGER, INTENT(IN) :: maxindegree, maxoutdegree
23 INTEGER, INTENT(OUT) :: sources(maxindegree), destinations(maxoutdegree)
24 INTEGER :: sourceweights(*), destweights(*)
25 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

27 MPI_DIST_GRAPH_NEIGHBORS(COMM, MAXINDEGREE, SOURCES, SOURCEWEIGHTS,
28                          MAXOUTDEGREE, DESTINATIONS, DESTWEIGHTS, IERROR)
29 INTEGER COMM, MAXINDEGREE, SOURCES(*), SOURCEWEIGHTS(*), MAXOUTDEGREE,
30 DESTINATIONS(*), DESTWEIGHTS(*), IERROR

```

These calls are local. The number of edges into and out of the MPI process returned by `MPI_DIST_GRAPH_NEIGHBORS_COUNT` are the total number of such edges given in the call to `MPI_DIST_GRAPH_CREATE_ADJACENT` or `MPI_DIST_GRAPH_CREATE` (potentially by MPI processes other than the calling MPI process in the case of `MPI_DIST_GRAPH_CREATE`). Multiply-defined edges are all counted and returned by `MPI_DIST_GRAPH_NEIGHBORS` in some order. If `MPI_UNWEIGHTED` is supplied for `sourceweights` or `destweights` or both, or if `MPI_UNWEIGHTED` was supplied during the construction of the graph then no weight information is returned in that array or those arrays. If the communicator was created with `MPI_DIST_GRAPH_CREATE_ADJACENT` then for each MPI process in `comm`, the order of the values in `sources` and `destinations` is identical to the input that was used by the MPI process with the same rank in `comm_old` in the creation call. If the communicator was created with `MPI_DIST_GRAPH_CREATE` then the only requirement on the order of values in `sources` and `destinations` is that two calls to the routine with same input argument `comm` will return the same sequence of edges. If `maxindegree` or `maxoutdegree` is smaller than the numbers returned by `MPI_DIST_GRAPH_NEIGHBORS_COUNT`, then only the first part of the full list is returned.

Advice to implementors. Since the query calls are defined to be local, each MPI process needs to store the list of its neighbors with incoming and outgoing edges. Communication is required at the collective `MPI_DIST_GRAPH_CREATE` call in order to compute the neighbor lists for each MPI process from the distributed graph specification. (*End of advice to implementors.*)

8.5.6 Cartesian Shift Coordinates

If the MPI process topology is a Cartesian structure, an `MPI_SENDRECV` operation may be used along a coordinate direction to perform a shift of data. As input, `MPI_SENDRECV` takes the rank of a source MPI process for the receive, and the rank of a destination MPI process for the send. If the function `MPI_CART_SHIFT` is called for a communicator with an associated Cartesian topology, it provides the calling MPI process with the above identifiers, which then can be passed to `MPI_SENDRECV`. The user specifies the coordinate direction and the size of the step (positive or negative, but not zero). The function is local.

`MPI_CART_SHIFT(comm, direction, disp, rank_source, rank_dest)`

IN	comm	communicator with associated Cartesian topology (handle)
IN	direction	coordinate dimension of shift (integer)
IN	disp	displacement (> 0: upwards shift, < 0: downwards shift) (integer)
OUT	rank_source	rank of source MPI process (integer)
OUT	rank_dest	rank of destination MPI process (integer)

C binding

```
int MPI_Cart_shift(MPI_Comm comm, int direction, int disp, int *rank_source,
                  int *rank_dest)
```

Fortran 2008 binding

```
MPI_Cart_shift(comm, direction, disp, rank_source, rank_dest, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: direction, disp
  INTEGER, INTENT(OUT) :: rank_source, rank_dest
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_CART_SHIFT(COMM, DIRECTION, DISP, RANK_SOURCE, RANK_DEST, IERROR)
  INTEGER COMM, DIRECTION, DISP, RANK_SOURCE, RANK_DEST, IERROR
```

The `direction` argument indicates the coordinate dimension to be traversed by the shift. The dimensions are numbered from 0 to `ndims-1`, where `ndims` is the number of dimensions.

Depending on the periodicity of the Cartesian topology in the specified coordinate direction, `MPI_CART_SHIFT` provides the identifiers for a circular or an end-off shift. In the case of an end-off shift, the value `MPI_PROC_NULL` is returned in `rank_source` or `rank_dest`, indicating that the source or the destination for the shift is out of range.

It is erroneous to call `MPI_CART_SHIFT` with a `direction` that is either negative or greater than or equal to the number of dimensions in the Cartesian communicator. This

implies that it is erroneous to call `MPI_CART_SHIFT` with a `comm` that is associated with a zero-dimensional Cartesian topology.

Example 8.7. Using `MPI_CART_SHIFT` for a Cartesian topology.

The communicator, `comm`, has a two-dimensional, periodic, Cartesian topology associated with it. A two-dimensional array of REALs is stored one element per MPI process, in variable `A`. One wishes to skew this array, by shifting column `i` (vertically, i.e., along the column) by `i` steps.

```
...
! find MPI process rank
CALL MPI_COMM_RANK(comm, rank, ierr)
! find Cartesian coordinates
CALL MPI_CART_COORDS(comm, rank, maxdims, coords, ierr)
! compute shift source and destination
CALL MPI_CART_SHIFT(comm, 0, coords(2), source, dest, ierr)
! skew array
CALL MPI_SENDRECV_REPLACE(A, 1, MPI_REAL, dest, 0, source, 0, comm, &
                          status, ierr)
```

Advice to users. In Fortran, the dimension indicated by `DIRECTION = i` has `DIMS(i+1)` nodes, where `DIMS` is the array that was used to create the grid. In C, the dimension indicated by `direction = i` is the dimension specified by `dims[i]`. (*End of advice to users.*)

8.5.7 Partitioning of Cartesian Structures

`MPI_CART_SUB(comm, remain_dims, newcomm)`

IN	<code>comm</code>	communicator with associated Cartesian topology (handle)
IN	<code>remain_dims</code>	the <i>i</i> -th entry of <code>remain_dims</code> specifies whether the <i>i</i> -th dimension is kept in the subgrid (<code>true</code>) or is dropped (<code>false</code>) (array of logicals)
OUT	<code>newcomm</code>	new communicator with associated Cartesian topology containing the subgrid that includes the calling MPI process (handle)

C binding

```
int MPI_Cart_sub(MPI_Comm comm, const int remain_dims[], MPI_Comm *newcomm)
```

Fortran 2008 binding

```
MPI_Cart_sub(comm, remain_dims, newcomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  LOGICAL, INTENT(IN) :: remain_dims(*)
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```


Fortran binding

```

MPI_CART_SUB(COMM, REMAIN_DIMS, NEWCOMM, IERROR)
    INTEGER COMM, NEWCOMM, IERROR
    LOGICAL REMAIN_DIMS(*)

```

MPI_CART_SUB can be used to partition the group associated with a communicator that has an associated Cartesian topology into subgroups that form lower-dimensional Cartesian subgrids, and to create for each subgroup a communicator with the associated subgrid Cartesian topology. The topologies of the new communicators describe the subgrids. The number of dimensions of the subgrids is the number of remaining dimensions, i.e., the number of true values in `remain_dims`. The numbers of MPI processes in each coordinate direction of the subgrids are the remaining numbers of MPI processes in each coordinate direction of the grid associated with the original communicator, i.e., the values of the original grid dimensions for which the corresponding entry in `remain_dims` is true. The periodicity for the remaining dimensions in the new communicator is preserved from the original communicator. If all entries in `remain_dims` are false or `comm` is already associated with a zero-dimensional Cartesian topology then `newcomm` is associated with a zero-dimensional Cartesian topology. (This function is closely related to **MPI_COMM_SPLIT**.)

Example 8.8. Creation of nonoverlapping Cartesian subcommunicators with **MPI_CART_SUB**.

Assume that `MPI_Cart_create(..., comm)` has defined a $(2 \times 3 \times 4)$ grid. Let `remain_dims = (true, false, true)`. Then a call to

```
MPI_Cart_sub(comm, remain_dims, &newcomm);
```

will create three communicators each with eight MPI processes in a 2×4 Cartesian topology. If `remain_dims = (false, false, true)` then the call to

```
MPI_Cart_sub(comm, remain_dims, &newcomm);
```

will create six nonoverlapping communicators, each with four MPI processes, in a one-dimensional Cartesian topology.

8.5.8 Low-Level Topology Functions

The two additional functions introduced in this section can be used to implement all other topology functions. In general they will not be called by the user directly, except when creating additional *virtual topology* capabilities other than those provided by MPI. The two calls are both local.

```
MPI_CART_MAP(comm, ndims, dims, periods, newrank)
```

IN	comm	input communicator (handle)
IN	ndims	number of dimensions of Cartesian structure (integer)
IN	dims	integer array of size <code>ndims</code> specifying the number of processes in each coordinate direction
IN	periods	logical array of size <code>ndims</code> specifying the periodicity specification in each coordinate direction

OUT newrank reordered rank of the calling MPI process;
 MPI_UNDEFINED if calling MPI process does not
 belong to grid (integer)

C binding

```
int MPI_Cart_map(MPI_Comm comm, int ndims, const int dims[],
                 const int periods[], int *newrank)
```

Fortran 2008 binding

```
MPI_Cart_map(comm, ndims, dims, periods, newrank, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: ndims, dims(ndims)
  LOGICAL, INTENT(IN) :: periods(ndims)
  INTEGER, INTENT(OUT) :: newrank
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_CART_MAP(COMM, NDIMS, DIMS, PERIODS, NEWRANK, IERROR)
  INTEGER COMM, NDIMS, DIMS(*), NEWRANK, IERROR
  LOGICAL PERIODS(*)
```

MPI_CART_MAP computes an “optimal” placement for the calling MPI process on the physical machine. A possible implementation of this function is to always return the rank of the calling MPI process, that is, not to perform any reordering.

Advice to implementors. The function **MPI_CART_CREATE**(comm, ndims, dims, periods, reorder, comm_cart), with reorder = true can be implemented by calling **MPI_CART_MAP**(comm, ndims, dims, periods, newrank), then calling **MPI_COMM_SPLIT**(comm, color, key, comm_cart), with color = 0 if newrank ≠ MPI_UNDEFINED, color = MPI_UNDEFINED otherwise, and key = newrank. If ndims is zero then a zero-dimensional Cartesian topology is created.

The function **MPI_CART_SUB**(comm, remain_dims, comm_new) can be implemented by a call to **MPI_COMM_SPLIT**(comm, color, key, comm_new), using a single number encoding of the lost dimensions as color and a single number encoding of the preserved dimensions as key.

All other Cartesian topology functions can be implemented locally, using the topology information that is cached with the communicator. (*End of advice to implementors.*)

The corresponding function for graph structures is as follows.

```
MPI_GRAPH_MAP(comm, nnodes, index, edges, newrank)
```

IN comm input communicator (handle)

IN nnodes number of graph nodes (integer)

IN index integer array specifying the graph structure (for
 details see the definition of **MPI_GRAPH_CREATE**)

IN edges integer array specifying the graph structure

OUT newrank reordered rank of the calling MPI process;
 MPI_UNDEFINED if the calling MPI process does
 not belong to graph (integer)

C binding

```
int MPI_Graph_map(MPI_Comm comm, int nnodes, const int index[],
                  const int edges[], int *newrank)
```

Fortran 2008 binding

```
MPI_Graph_map(comm, nnodes, index, edges, newrank, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: nnodes, index(nnodes), edges(*)
  INTEGER, INTENT(OUT) :: newrank
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GRAPH_MAP(COMM, NNODES, INDEX, EDGES, NEWRANK, IERROR)
  INTEGER COMM, NNODES, INDEX(*), EDGES(*), NEWRANK, IERROR
```

Advice to implementors. The function `MPI_GRAPH_CREATE(comm, nnodes, index, edges, reorder, comm_graph)`, with `reorder = true` can be implemented by calling `MPI_GRAPH_MAP(comm, nnodes, index, edges, newrank)`, then calling `MPI_COMM_SPLIT(comm, color, key, comm_graph)`, with `color = 0` if `newrank ≠ MPI_UNDEFINED`, `color = MPI_UNDEFINED` otherwise, and `key = newrank`.

All other graph topology functions can be implemented locally, using the topology information that is cached with the communicator. (*End of advice to implementors.*)

8.6 Neighborhood Collective Communication on Virtual Topologies

Virtual topologies specify a communication graph, but they implement no communication function themselves. Many applications require sparse nearest neighbor communications that can be expressed as graph topologies. We now describe several collective operations that perform communication along the edges of a graph representing a *virtual topology*. All of these functions are collective; i.e., they must be called by all MPI processes in the specified communicator. See Section 6 for an overview of other dense (global) collective communication operations and the semantics of collective operations.

If the graph was created with `MPI_DIST_GRAPH_CREATE_ADJACENT` with `sources` and `destinations` containing `0, ..., n-1`, where `n` is the number of MPI processes in the group of `comm_old` (i.e., the graph is fully connected and also includes an edge from each node to itself), then the sparse neighborhood communication routine performs the same data exchange as the corresponding dense (fully-connected) collective operation. In the case of a Cartesian communicator, only nearest neighbor communication is provided, corresponding to `rank_source` and `rank_dest` in `MPI_CART_SHIFT` with input `disp = 1`.

Rationale. Neighborhood collective communications enable communication on a *virtual topology*. This high-level specification of data exchange among neighboring MPI processes enables optimizations in the MPI library because the communication pattern is known statically (the topology). Thus, the implementation can compute optimized message schedules during creation of the topology [40]. This functionality can significantly simplify the implementation of neighbor exchanges [36]. (*End of rationale.*)

For a distributed graph topology, created with `MPI_DIST_GRAPH_CREATE`, the sequence of neighbors in the send and receive buffers at each MPI process is defined as the

sequence returned by `MPI_DIST_GRAPH_NEIGHBORS` for destinations and sources, respectively. For a general graph topology, created with `MPI_GRAPH_CREATE`, the use of neighborhood collective communication is restricted to adjacency matrices, where the number of edges between any two MPI processes is defined to be the same for both MPI processes (i.e., with a symmetric adjacency matrix). In this case, the order of neighbors in the send and receive buffers is defined as the sequence of neighbors as returned by `MPI_GRAPH_NEIGHBORS`. Note that graph topologies should generally be replaced by the distributed graph topologies.

For a Cartesian topology, created with `MPI_CART_CREATE`, the sequence of neighbors in the send and receive buffers at each MPI process is defined by the order of the dimensions, first the neighbor in the negative direction and then in the positive direction with displacement 1. The numbers of sources and destinations in the communication routines are $2 \cdot \text{ndims}$ with `ndims` defined in `MPI_CART_CREATE`. If a neighbor does not exist, i.e., at the border of a Cartesian topology in the case of a nonperiodic virtual grid dimension (i.e., `periods[...] = false`), then this neighbor is defined to be `MPI_PROC_NULL`.

If a neighbor in any of the functions is `MPI_PROC_NULL`, then the neighborhood collective communication behaves like a point-to-point communication with `MPI_PROC_NULL` in this direction. That is, the buffer is still part of the sequence of neighbors but it is neither communicated nor updated.

8.6.1 Neighborhood Gather

In the neighborhood gather operation, each MPI process i gathers data items from each MPI process j if an edge (j, i) exists in the topology graph, and each MPI process i sends the same data items to all MPI processes j where an edge (i, j) exists. The send buffer is sent to each neighboring MPI process and the l -th block in the receive buffer is received from the l -th neighbor.

`MPI_NEIGHBOR_ALLGATHER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)`

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcount	number of elements received from each neighbor (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)

C binding

```
int MPI_Neighbor_allgather(const void *sendbuf, int sendcount,
                          MPI_Datatype sendtype, void *recvbuf, int recvcount,
                          MPI_Datatype recvtype, MPI_Comm comm)
```

```
int MPI_Neighbor_allgather_c(const void *sendbuf, MPI_Count sendcount,
    MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
    MPI_Datatype recvtype, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Neighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..) :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..) :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_NEIGHBOR_ALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT,
    RECVTYPE, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, IERROR
```

The `MPI_NEIGHBOR_ALLGATHER` procedure supports Cartesian communicators, graph communicators, and distributed graph communicators as described in Section 8.6. If `comm` is a distributed graph communicator, the outcome is as if each MPI process executed sends to each of its outgoing neighbors and receives from each of its incoming neighbors:

```
MPI_Dist_graph_neighbors_count(comm, &indegree, &outdegree, &weighted);
int *srcs=(int*)malloc(indegree*sizeof(int));
int *dsts=(int*)malloc(outdegree*sizeof(int));
MPI_Dist_graph_neighbors(comm, indegree, srcs, MPI_UNWEIGHTED,
    outdegree, dsts, MPI_UNWEIGHTED);

int k;

/* assume sendbuf and recvbuf are of type (char*) */
for(k=0; k<outdegree; ++k)
    MPI_Isend(sendbuf, sendcount, sendtype, dsts[k],...);

for(k=0; k<indegree; ++k)
    MPI_Irecv(recvbuf+k*recvcount*extent(recvtype), recvcount, recvtype,
        srcs[k],...);

MPI_Waitall(...);
```

Figure 8.1 shows the neighborhood gather communication of one MPI process with outgoing neighbors $d_0 \dots d_3$ and incoming neighbors $s_0 \dots s_5$. The MPI process will send

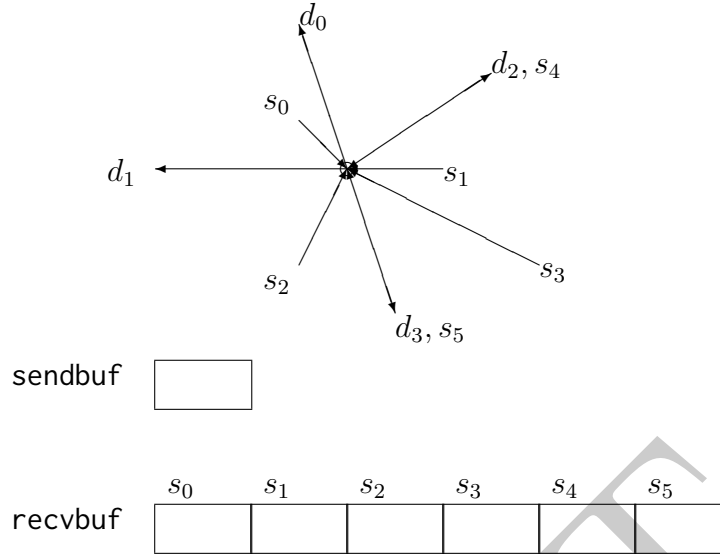


Figure 8.1: Neighborhood gather communication example

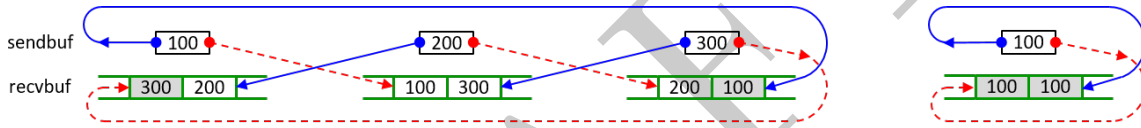


Figure 8.2: Cartesian neighborhood allgather example for 3 and 1 processes in a dimension

its **sendbuf** to all four **destinations** (outgoing neighbors) and it will receive the contribution from all six **sources** (incoming neighbors) into separate locations of its receive buffer.

All arguments are significant on all MPI processes and the argument **comm** must have identical values on all MPI processes.

The type signature associated with **sendcount**, **sendtype** at an MPI process must be equal to the type signature associated with **recvcount**, **recvtype** at all other MPI processes. This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of communicating MPI processes. Distinct type maps between sender and receiver are still allowed.

Rationale. For optimization reasons, the same type signature is required independently of whether the topology graph is connected or not. (*End of rationale.*)

The “in place” option is not meaningful for this operation.

Example 8.9. Buffer usage of **MPI_NEIGHBOR_ALLGATHER** in the case of a Cartesian virtual topology.

On a Cartesian virtual topology, the buffer usage in a given direction **d** with **dims[d]=3** and **1**, respectively during creation of the communicator is described in Figure 8.2.

The figure may apply to any (or multiple) directions in the Cartesian topology. The grey buffers are required in all cases but are only accessed if during creation of the communicator, **periods[d]** was defined as nonzero (in C) or **.TRUE.** (in Fortran).

The vector variant of `MPI_NEIGHBOR_ALLGATHER` allows one to gather different numbers of elements from each neighbor.

`MPI_NEIGHBOR_ALLGATHERV(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs, recvtype, comm)`

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcounts	nonnegative integer array (of length indegree) containing the number of elements that are received from each neighbor
IN	displs	integer array (of length indegree). Entry <i>i</i> specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor <i>i</i>
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)

C binding

```
int MPI_Neighbor_allgatherv(const void *sendbuf, int sendcount,
    MPI_Datatype sendtype, void *recvbuf, const int recvcounts[],
    const int displs[], MPI_Datatype recvtype, MPI_Comm comm)

int MPI_Neighbor_allgatherv_c(const void *sendbuf, MPI_Count sendcount,
    MPI_Datatype sendtype, void *recvbuf,
    const MPI_Count recvcounts[], const MPI_Aint displs[],
    MPI_Datatype recvtype, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Neighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts,
    displs, recvtype, comm, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcounts(*), displs(*)
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..) :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts,
    displs, recvtype, comm, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcounts(*)
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..) :: recvbuf
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
```



```

1      TYPE(MPI_Comm), INTENT(IN) :: comm
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_NEIGHBOR_ALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS,
5                               DISPLS, RECVTYPE, COMM, IERROR)
6      <type> SENDBUF(*), RECVBUF(*)
7      INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
8      IERROR

```

The `MPI_NEIGHBOR_ALLGATHERV` procedure supports Cartesian communicators, graph communicators, and distributed graph communicators as described in Section 8.6. If `comm` is a distributed graph communicator, the outcome is as if each MPI process executed sends to each of its outgoing neighbors and receives from each of its incoming neighbors:

```

14      MPI_Dist_graph_neighbors_count(comm, &indegree, &outdegree, &weighted);
15      int *srcs=(int*)malloc(indegree*sizeof(int));
16      int *dsts=(int*)malloc(outdegree*sizeof(int));
17      MPI_Dist_graph_neighbors(comm, indegree, srcs, MPI_UNWEIGHTED,
18                               outdegree, dsts, MPI_UNWEIGHTED);
19      int k;
20
21      /* assume sendbuf and recvbuf are of type (char*) */
22      for(k=0; k<outdegree; ++k)
23          MPI_Isend(sendbuf, sendcount, sendtype, dsts[k],...);
24
25      for(k=0; k<indegree; ++k)
26          MPI_Irecv(recvbuf+displs[k]*extent(recvtype), recvcounts[k], recvtype,
27                    srcs[k],...);
28      MPI_Waitall(...);

```

The type signature associated with `sendcount`, `sendtype` at MPI process j must be equal to the type signature associated with `recvcounts[l]`, `recvtype` at any other MPI process with `srcs[l]=j`. This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of communicating MPI processes. Distinct type maps between sender and receiver are still allowed. The data received from the l -th neighbor is placed into `recvbuf` beginning at offset `displs[l]` elements (in terms of the `recvtype`).

The “in place” option is not meaningful for this operation.

All arguments are significant on all MPI processes and the argument `comm` must have identical values on all MPI processes.

8.6.2 Neighborhood Alltoall

In the neighborhood alltoall operation, each MPI process i receives data items from each MPI process j if an edge (j, i) exists in the topology graph or Cartesian topology. Similarly, each MPI process i sends data items to all MPI processes j where an edge (i, j) exists. This call is more general than `MPI_NEIGHBOR_ALLGATHER` in that different data items can be sent to each neighbor. The k -th block in send buffer is sent to the k -th neighboring MPI process and the l -th block in the receive buffer is received from the l -th neighbor.


```
MPI_NEIGHBOR_ALLTOALL(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                        comm)
```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcount	number of elements received from each neighbor (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)

C binding

```
int MPI_Neighbor_alltoall(const void *sendbuf, int sendcount,
                          MPI_Datatype sendtype, void *recvbuf, int recvcount,
                          MPI_Datatype recvtype, MPI_Comm comm)

int MPI_Neighbor_alltoall_c(const void *sendbuf, MPI_Count sendcount,
                             MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                             MPI_Datatype recvtype, MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Neighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount,
                      recvtype, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  INTEGER, INTENT(IN) :: sendcount, recvcount
  TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
  TYPE(*), DIMENSION(..) :: recvbuf
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount,
                      recvtype, comm, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
  TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
  TYPE(*), DIMENSION(..) :: recvbuf
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_NEIGHBOR_ALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
                      RECVTYPE, COMM, IERROR)
  <type> SENDBUF(*), RECVBUF(*)
  INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, IERROR
```

The `MPI_NEIGHBOR_ALLTOALL` procedure supports Cartesian communicators, graph communicators, and distributed graph communicators as described in Section 8.6. If `comm`

is a distributed graph communicator, the outcome is as if each MPI process executed sends to each of its outgoing neighbors and receives from each of its incoming neighbors:

```

1  MPI_Dist_graph_neighbors_count(comm, &indegree, &outdegree, &weighted);
2
3  MPI_Dist_graph_neighbors_count(comm, &indegree, &outdegree, &weighted);
4  int *srcs=(int*)malloc(indegree*sizeof(int));
5  int *dsts=(int*)malloc(outdegree*sizeof(int));
6  MPI_Dist_graph_neighbors(comm, indegree, srcs, MPI_UNWEIGHTED,
7                          outdegree, dsts, MPI_UNWEIGHTED);
8
9  int k;
10
11 /* assume sendbuf and recvbuf are of type (char*) */
12 for(k=0; k<outdegree; ++k)
13     MPI_Isend(sendbuf+k*sendcount*extent(sendtype), sendcount, sendtype,
14             dsts[k],...);
15
16 for(k=0; k<indegree; ++k)
17     MPI_Irecv(recvbuf+k*recvcount*extent(recvtype), recvcount, recvtype,
18             srcs[k],...);
19
20 MPI_Waitall(...);

```

The type signature associated with `sendcount`, `sendtype` at an MPI process must be equal to the type signature associated with `recvcount`, `recvtype` at any other MPI process. This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of communicating MPI processes. Distinct type maps between sender and receiver are still allowed.

The “in place” option is not meaningful for this operation.

All arguments are significant on all MPI processes and the argument `comm` must have identical values on all MPI processes.

Example 8.10. Buffer usage of `MPI_NEIGHBOR_ALLTOALL` in the case of a Cartesian virtual topology.

For a halo communication on a Cartesian grid, the buffer usage in a given direction `d` with `dims[d]=3` and 1, respectively during creation of the communicator is described in Figure 8.3. The figure may apply to any (or multiple) directions in the Cartesian topology. The grey buffers are required in all cases but are only accessed if during creation of the communicator, `periods[d]` was defined as nonzero (in C) or `.TRUE.` (in Fortran).

If `sendbuf` and `recvbuf` are declared as `(char *)` and contain a sequence of buffers each described by `sendcount`, `sendtype` and `recvbuf`, `recvtype`, then after `MPI_NEIGHBOR_ALLTOALL` on a Cartesian communicator returned, the content of the `recvbuf` is as if the following code is executed:

```

38 MPI_Cartdim_get(comm, &ndims);
39 MPI_Type_get_extent(sendtype, &send_lb, &send_extent);
40 MPI_Type_get_extent(recvtype, &recv_lb, &recv_extent);
41 for( /*direction*/ d=0; d < ndims; d++) {
42     MPI_Cart_shift(comm, /*direction*/ d, /*disp*/ 1, &rank_source, &rank_dest);
43     MPI_Sendrecv(sendbuf+(d*2+0)*sendcount*send_extent,
44                 sendcount, sendtype, rank_source, /*sendtag*/ d*2,
45                 recvbuf+(d*2+1)*recvcount*recv_extent,
46                 recvcount, recvtype, rank_dest, /*recvtag*/ d*2,
47                 comm, &status); /*communication in direction of displacement -1*/
48     MPI_Sendrecv(sendbuf+(d*2+1)*sendcount*send_extent,
49                 sendcount, sendtype, rank_dest, /*sendtag*/ d*2+1,

```

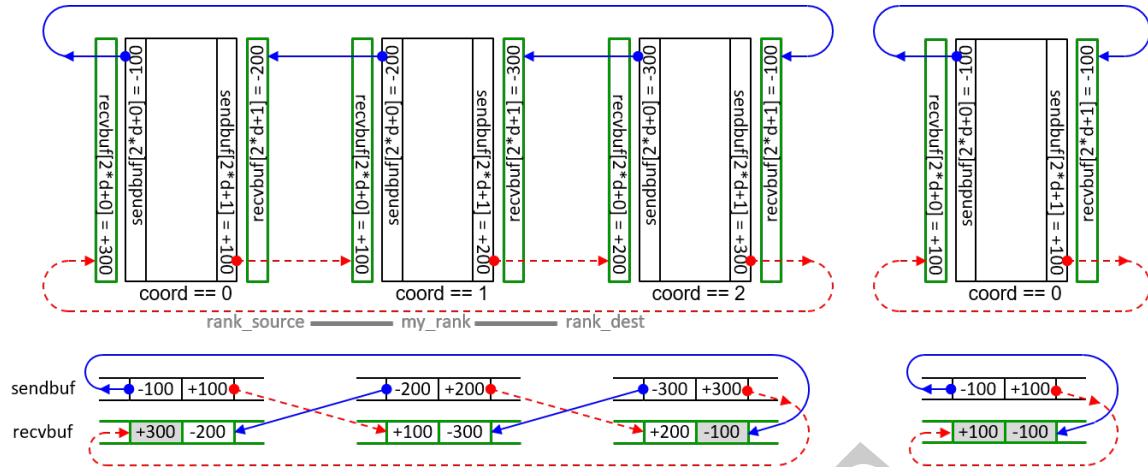


Figure 8.3: Cartesian neighborhood alltoall example for 3 and 1 MPI processes in a dimension

```

    recvbuf+(d*2+0)*recvcount*recv_extent,
    recvcount,recvtype,rank_source,/*recvtag*/d*2+1,
    comm,&status);/*communication in direction of displacement +1*/
}

```

The first call to `MPI_Sendrecv` implements the solid arrows' communication pattern in each diagram of Figure 8.3, whereas the second call is for the dashed arrows' pattern.

Advice to implementors. For a Cartesian topology, if the grid in a direction `d` is periodic and `dims[d]` is equal to 1 or 2, then `rank_source` and `rank_dest` are identical, but still all `ndims` send and `ndims` receive operations use different buffers. If in this case, the two send and receive operations per direction or of all directions are internally parallelized, then the several send and receive operations for the same sender-receiver MPI process pair shall be initiated in the same sequence on sender and receiver side or they shall be distinguished by different tags. The code above shows a valid sequence of operations and tags. (*End of advice to implementors.*)

The vector variant of `MPI_NEIGHBOR_ALLTOALL` allows sending/receiving different numbers of elements to and from each neighbor.

`MPI_NEIGHBOR_ALLTOALLV`(`sendbuf`, `sendcounts`, `sdispls`, `sendtype`, `recvbuf`, `recvcounts`, `rdispls`, `recvtype`, `comm`)

IN	<code>sendbuf</code>	starting address of send buffer (choice)
IN	<code>sendcounts</code>	nonnegative integer array (of length outdegree) specifying the number of elements to send to each neighbor
IN	<code>sdispls</code>	integer array (of length outdegree). Entry <code>j</code> specifies the displacement (relative to <code>sendbuf</code>) from which to send the outgoing data to neighbor <code>j</code>
IN	<code>sendtype</code>	datatype of send buffer elements (handle)

1	OUT	recvbuf	starting address of receive buffer (choice)
2	IN	recvcounts	nonnegative integer array (of length indegree)
3			specifying the number of elements that are received
4			from each neighbor
5	IN	rdispls	integer array (of length indegree). Entry <i>i</i> specifies
6			the displacement (relative to <code>recvbuf</code>) at which to
7			place the incoming data from neighbor <i>i</i>
8	IN	recvtype	datatype of receive buffer elements (handle)
9	IN	comm	communicator with associated virtual topology
10			(handle)

C binding

```

13 int MPI_Neighbor_alltoallv(const void *sendbuf, const int sendcounts[],
14                           const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
15                           const int recvcounts[], const int rdispls[],
16                           MPI_Datatype recvtype, MPI_Comm comm)
17
18 int MPI_Neighbor_alltoallv_c(const void *sendbuf, const MPI_Count sendcounts[],
19                             const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
20                             const MPI_Count recvcounts[], const MPI_Aint rdispls[],
21                             MPI_Datatype recvtype, MPI_Comm comm)

```

Fortran 2008 binding

```

23 MPI_Neighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
24                       recvcounts, rdispls, recvtype, comm, ierror)
25     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
26     INTEGER, INTENT(IN) :: sendcounts(*), sdispls(*), recvcounts(*), rdispls(*)
27     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
28     TYPE(*), DIMENSION(..) :: recvbuf
29     TYPE(MPI_Comm), INTENT(IN) :: comm
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_Neighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
33                       recvcounts, rdispls, recvtype, comm, ierror) !(_c)
34     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
35     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcounts(*)
36     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
37     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
38     TYPE(*), DIMENSION(..) :: recvbuf
39     TYPE(MPI_Comm), INTENT(IN) :: comm
40     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

42 MPI_NEIGHBOR_ALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF,
43                       RECVCOUNTS, RDISPLS, RECVTYPE, COMM, IERROR)
44     <type> SENDBUF(*), RECVBUF(*)
45     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, RECVCOUNTS(*), RDISPLS(*),
46     RECVTYPE, COMM, IERROR

```

The `MPI_NEIGHBOR_ALLTOALLV` procedure supports Cartesian communicators, graph communicators, and distributed graph communicators as described in Section 8.6. If

`comm` is a distributed graph communicator, the outcome is as if each MPI process executed sends to each of its outgoing neighbors and receives from each of its incoming neighbors:

```

MPI_Dist_graph_neighbors_count(comm, &indegree, &outdegree, &weighted);
int *srcs=(int*)malloc(indegree*sizeof(int));
int *dsts=(int*)malloc(outdegree*sizeof(int));
MPI_Dist_graph_neighbors(comm, indegree, srcs, MPI_UNWEIGHTED,
                          outdegree, dsts, MPI_UNWEIGHTED);

int k;

/* assume sendbuf and recvbuf are of type (char*) */
for(k=0; k<outdegree; ++k)
    MPI_Isend(sendbuf+sdispls[k]*extent(sendtype), sendcounts[k],
              sendtype, dsts[k],...);

for(k=0; k<indegree; ++k)
    MPI_Irecv(recvbuf+rdispls[k]*extent(recvtype), recvcounts[k],
              recvtype, srcs[k],...);

MPI_Waitall(...);

```

The type signature associated with `sendcounts[k]`, `sendtype` with `dsts[k]=j` at MPI process i must be equal to the type signature associated with `recvcounts[l]`, `recvtype` with `srcs[l]=i` at MPI process j . This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of communicating MPI processes. Distinct type maps between sender and receiver are still allowed. The data in the `sendbuf` beginning at offset `sdispls[k]` elements (in terms of the `sendtype`) is sent to the k -th outgoing neighbor. The data received from the l -th incoming neighbor is placed into `recvbuf` beginning at offset `rdispls[l]` elements (in terms of the `recvtype`).

The “in place” option is not meaningful for this operation.

All arguments are significant on all MPI processes and the argument `comm` must have identical values on all MPI processes.

MPI_NEIGHBOR_ALLTOALLW allows one to send and receive with different datatypes to and from each neighbor.

MPI_NEIGHBOR_ALLTOALLW(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcounts, rdispls, recvtypes, comm)

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	nonnegative integer array (of length outdegree) specifying the number of elements to send to each neighbor
IN	sdispls	integer array (of length outdegree). Entry j specifies the displacement in bytes (relative to <code>sendbuf</code>) from which to take the outgoing data destined for neighbor j (array of integers)
IN	sendtypes	array of datatypes (of length outdegree). Entry j specifies the type of data to send to neighbor j (array of handles)
OUT	recvbuf	starting address of receive buffer (choice)

1	IN	recvcounts	nonnegative integer array (of length indegree) specifying the number of elements that are received from each neighbor
2			
3			
4	IN	rdispls	integer array (of length indegree). Entry i specifies the displacement in bytes (relative to recvbuf) at which to place the incoming data from neighbor i (array of integers)
5			
6			
7			
8	IN	recvtypes	array of datatypes (of length indegree). Entry i specifies the type of data received from neighbor i (array of handles)
9			
10			
11	IN	comm	communicator with associated virtual topology (handle)
12			

C binding

```

14 int MPI_Neighbor_alltoallw(const void *sendbuf, const int sendcounts[],
15     const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
16     void *recvbuf, const int recvcounts[], const MPI_Aint rdispls[],
17     const MPI_Datatype recvtypes[], MPI_Comm comm)
18
19 int MPI_Neighbor_alltoallw_c(const void *sendbuf, const MPI_Count sendcounts[],
20     const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
21     void *recvbuf, const MPI_Count recvcounts[],
22     const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
23     MPI_Comm comm)

```

Fortran 2008 binding

```

25 MPI_Neighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
26     recvcounts, rdispls, recvtypes, comm, ierror)
27     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
28     INTEGER, INTENT(IN) :: sendcounts(*), recvcounts(*)
29     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
30     TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
31     TYPE(*), DIMENSION(..) :: recvbuf
32     TYPE(MPI_Comm), INTENT(IN) :: comm
33     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
34
35 MPI_Neighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
36     recvcounts, rdispls, recvtypes, comm, ierror) !(_c)
37     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
38     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcounts(*)
39     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
40     TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
41     TYPE(*), DIMENSION(..) :: recvbuf
42     TYPE(MPI_Comm), INTENT(IN) :: comm
43     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

45 MPI_NEIGHBOR_ALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
46     RECVCOUNTS, RDISPLS, RECVTYPES, COMM, IERROR)
47     <type> SENDBUF(*), RECVBUF(*)
48

```

```

INTEGER SENDCOUNTS(*), SENDTYPES(*), RECVCOUNTS(*), RECVTYPES(*), COMM,
        IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) SDISPLS(*), RDISPLS(*)

```

The `MPI_NEIGHBOR_ALLTOALLW` procedure supports Cartesian communicators, graph communicators, and distributed graph communicators as described in Section 8.6. If `comm` is a distributed graph communicator, the outcome is as if each MPI process executed sends to each of its outgoing neighbors and receives from each of its incoming neighbors:

```

MPI_Dist_graph_neighbors_count(comm, &indegree, &outdegree, &weighted);
int *srcs=(int*)malloc(indegree*sizeof(int));
int *dsts=(int*)malloc(outdegree*sizeof(int));
MPI_Dist_graph_neighbors(comm, indegree, srcs, MPI_UNWEIGHTED,
                        outdegree, dsts, MPI_UNWEIGHTED);

int k;

/* assume sendbuf and recvbuf are of type (char*) */
for(k=0; k<outdegree; ++k)
    MPI_Isend(sendbuf+sdispls[k], sendcounts[k], sendtypes[k],
              dsts[k],...);

for(k=0; k<indegree; ++k)
    MPI_Irecv(recvbuf+rdispls[k], recvcounts[k], recvtypes[k],
              srcs[k],...);

MPI_Waitall(...);

```

The type signature associated with `sendcounts[k]`, `sendtypes[k]` with `dsts[k]=j` at MPI process i must be equal to the type signature associated with `recvcounts[l]`, `recvtypes[l]` with `srcs[l]=i` at MPI process j . This implies that the amount of data sent must be equal to the amount of data received, pairwise between every pair of communicating MPI processes. Distinct type maps between sender and receiver are still allowed.

The “in place” option is not meaningful for this operation.

All arguments are significant on all MPI processes and the argument `comm` must have identical values on all MPI processes.

8.7 Nonblocking Neighborhood Communication on Process Topologies

Nonblocking variants of the neighborhood collective operations allow relaxed synchronization and overlapping of computation and communication. The semantics are similar to nonblocking collective operations as described in Section 6.12.

8.7.1 Nonblocking Neighborhood Gather

```

MPI_INEIGHBOR_ALLGATHER(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                        comm, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)

1	IN	sendtype	datatype of send buffer elements (handle)
2	OUT	recvbuf	starting address of receive buffer (choice)
3			
4	IN	recvcount	number of elements received from each neighbor (nonnegative integer)
5			
6	IN	recvtype	datatype of receive buffer elements (handle)
7	IN	comm	communicator with associated virtual topology (handle)
8			
9	OUT	request	communication request (handle)
10			

11 C binding

```

12 int MPI_Ineighbor_allgather(const void *sendbuf, int sendcount,
13                             MPI_Datatype sendtype, void *recvbuf, int recvcount,
14                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
15
16 int MPI_Ineighbor_allgather_c(const void *sendbuf, MPI_Count sendcount,
17                               MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
18                               MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)

```

19 Fortran 2008 binding

```

20 MPI_Ineighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
21                         recvtype, comm, request, ierror)
22     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
23     INTEGER, INTENT(IN) :: sendcount, recvcount
24     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
25     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
26     TYPE(MPI_Comm), INTENT(IN) :: comm
27     TYPE(MPI_Request), INTENT(OUT) :: request
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30 MPI_Ineighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
31                         recvtype, comm, request, ierror) !(_c)
32     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
33     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
34     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
35     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
36     TYPE(MPI_Comm), INTENT(IN) :: comm
37     TYPE(MPI_Request), INTENT(OUT) :: request
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

39 Fortran binding

```

40 MPI_INEIGHBOR_ALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
41                         RECVTYPE, COMM, REQUEST, IERROR)
42     <type> SENDBUF(*), RECVBUF(*)
43     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, REQUEST, IERROR
44
45     MPI_INEIGHBOR_ALLGATHER starts a nonblocking variant of
46     MPI_NEIGHBOR_ALLGATHER.

```


`MPI_INEIGHBOR_ALLGATHERV(sendbuf, sendcount, sendtype, recvbuf, recvcnts, displs,
recvtype, comm, request)`

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcnts	nonnegative integer array (of length indegree) containing the number of elements that are received from each neighbor
IN	displs	integer array (of length indegree). Entry i specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor i
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)
OUT	request	communication request (handle)

C binding

```
int MPI_Ineighbor_allgatherv(const void *sendbuf, int sendcount,
    MPI_Datatype sendtype, void *recvbuf, const int recvcnts[],
    const int displs[], MPI_Datatype recvtype, MPI_Comm comm,
    MPI_Request *request)
```

```
int MPI_Ineighbor_allgatherv_c(const void *sendbuf, MPI_Count sendcount,
    MPI_Datatype sendtype, void *recvbuf,
    const MPI_Count recvcnts[], const MPI_Aint displs[],
    MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Ineighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcnts,
    displs, recvtype, comm, request, ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcnts(*), displs(*)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Ineighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcnts,
    displs, recvtype, comm, request, ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcnts(*)
```

```

1      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
2      TYPE(MPI_Comm), INTENT(IN) :: comm
3      TYPE(MPI_Request), INTENT(OUT) :: request
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

6      MPI_INEIGHBOR_ALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS,
7                               DISPLS, RECVTYPE, COMM, REQUEST, IERROR)
8      <type> SENDBUF(*), RECVBUF(*)
9      INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
10      REQUEST, IERROR

```

12 MPI_INEIGHBOR_ALLGATHERV starts a nonblocking variant of
13 MPI_NEIGHBOR_ALLGATHERV.

14

8.7.2 Nonblocking Neighborhood Alltoall

16

17

```

18      MPI_INEIGHBOR_ALLTOALL(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
19                             comm, request)

```

20

21	IN	sendbuf	starting address of send buffer (choice)
22	IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
23			
24	IN	sendtype	datatype of send buffer elements (handle)
25	OUT	recvbuf	starting address of receive buffer (choice)
26			
27	IN	recvcount	number of elements received from each neighbor (nonnegative integer)
28			
29	IN	recvtype	datatype of receive buffer elements (handle)
30	IN	comm	communicator with associated virtual topology (handle)
31			
32	OUT	request	communication request (handle)
33			

34

C binding

```

35      int MPI_Ineighbor_alltoall(const void *sendbuf, int sendcount,
36                               MPI_Datatype sendtype, void *recvbuf, int recvcount,
37                               MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
38
39      int MPI_Ineighbor_alltoall_c(const void *sendbuf, MPI_Count sendcount,
40                                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
41                                  MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)

```

42

Fortran 2008 binding

```

43      MPI_Ineighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount,
44                             recvtype, comm, request, ierror)
45      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
46      INTEGER, INTENT(IN) :: sendcount, recvcount
47      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
48      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf

```

```

TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Ineighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_INEIGHBOR_ALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT,
    RECVTYPE, COMM, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, REQUEST, IERROR

```

MPI_INEIGHBOR_ALLTOALL starts a nonblocking variant of [MPI_NEIGHBOR_ALLTOALL](#).

```

MPI_INEIGHBOR_ALLTOALLV(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
    rdispls, recvtype, comm, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	nonnegative integer array (of length outdegree) specifying the number of elements to send to each neighbor
IN	sdispls	integer array (of length outdegree). Entry j specifies the displacement (relative to <code>sendbuf</code>) from which send the outgoing data to neighbor j
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcounts	nonnegative integer array (of length indegree) specifying the number of elements that are received from each neighbor
IN	rdispls	integer array (of length indegree). Entry i specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor i
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Ineighbor_alltoallv(const void *sendbuf, const int sendcounts[],
    const int sdispls[], MPI_Datatype sendtype, void *recvbuf,

```

```

1      const int recvcounts[], const int rdispls[],
2      MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
3
4  int MPI_Ineighbor_alltoallv_c(const void *sendbuf,
5      const MPI_Count sendcounts[], const MPI_Aint sdispls[],
6      MPI_Datatype sendtype, void *recvbuf,
7      const MPI_Count recvcounts[], const MPI_Aint rdispls[],
8      MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)

```

Fortran 2008 binding

```

10 MPI_Ineighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
11     recvcounts, rdispls, recvtype, comm, request, ierror)
12     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
13     INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
14     recvcounts(*), rdispls(*)
15     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
16     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
17     TYPE(MPI_Comm), INTENT(IN) :: comm
18     TYPE(MPI_Request), INTENT(OUT) :: request
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21 MPI_Ineighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
22     recvcounts, rdispls, recvtype, comm, request, ierror) !(_c)
23     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
24     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
25     recvcounts(*)
26     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
27     rdispls(*)
28     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
29     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
30     TYPE(MPI_Comm), INTENT(IN) :: comm
31     TYPE(MPI_Request), INTENT(OUT) :: request
32     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

34 MPI_INEIGHBOR_ALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF,
35     RECVCOUNTS, RDISPLS, RECVTYPE, COMM, REQUEST, IERROR)
36     <type> SENDBUF(*), RECVBUF(*)
37     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, REVCOUNTS(*), RDISPLS(*),
38     RECVTYPE, COMM, REQUEST, IERROR

```

MPI_INEIGHBOR_ALLTOALLV starts a nonblocking variant of
[MPI_NEIGHBOR_ALLTOALLV](#).

```

43 MPI_INEIGHBOR_ALLTOALLW(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
44     recvcounts, rdispls, recvtypes, comm, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	nonnegative integer array (of length outdegree) specifying the number of elements to send to each neighbor

IN	sdispls	integer array (of length outdegree). Entry j specifies the displacement in bytes (relative to <code>sendbuf</code>) from which to take the outgoing data destined for neighbor j (array of integers)	1 2 3 4
IN	sendtypes	array of datatypes (of length outdegree). Entry j specifies the type of data to send to neighbor j (array of handles)	5 6 7
OUT	recvbuf	starting address of receive buffer (choice)	8
IN	recvcounts	nonnegative integer array (of length indegree) specifying the number of elements that are received from each neighbor	9 10 11
IN	rdispls	integer array (of length indegree). Entry i specifies the displacement in bytes (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor i (array of integers)	12 13 14 15
IN	recvtypes	array of datatypes (of length indegree). Entry i specifies the type of data received from neighbor i (array of handles)	16 17 18
IN	comm	communicator with associated virtual topology (handle)	19 20
OUT	request	communication request (handle)	21 22
C binding			23
<pre>int MPI_Ineighbor_alltoallw(const void *sendbuf, const int sendcounts[], const MPI_Aint sdispls[], const MPI_Datatype sendtypes[], void *recvbuf, const int recvcounts[], const MPI_Aint rdispls[], const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Request *request)</pre>			24 25 26 27 28
<pre>int MPI_Ineighbor_alltoallw_c(const void *sendbuf, const MPI_Count sendcounts[], const MPI_Aint sdispls[], const MPI_Datatype sendtypes[], void *recvbuf, const MPI_Count recvcounts[], const MPI_Aint rdispls[], const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Request *request)</pre>			29 30 31 32 33 34 35
Fortran 2008 binding			36
<pre>MPI_Ineighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcounts, rdispls, recvtypes, comm, request, ierror) TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), recvcounts(*) INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*), rdispls(*) TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*) TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf TYPE(MPI_Comm), INTENT(IN) :: comm TYPE(MPI_Request), INTENT(OUT) :: request INTEGER, OPTIONAL, INTENT(OUT) :: ierror</pre>			37 38 39 40 41 42 43 44 45 46 47 48

```

1 MPI_Ineighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
2   recvcounts, rdispls, recvtypes, comm, request, ierror) !(_c)
3   TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4   INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
5   recvcounts(*)
6   INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
7   rdispls(*)
8   TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
9   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
10  TYPE(MPI_Comm), INTENT(IN) :: comm
11  TYPE(MPI_Request), INTENT(OUT) :: request
12  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

14 MPI_INEIGHBOR_ALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
15   RECVCOUNTS, RDISPLS, RECVTYPES, COMM, REQUEST, IERROR)
16   <type> SENDBUF(*), RECVBUF(*)
17   INTEGER SENDCOUNTS(*), SENDTYPES(*), RECVCOUNTS(*), RECVTYPES(*), COMM,
18   REQUEST, IERROR
19   INTEGER(KIND=MPI_ADDRESS_KIND) SDISPLS(*), RDISPLS(*)

```

MPI_INEIGHBOR_ALLTOALLW starts a nonblocking variant of [MPI_NEIGHBOR_ALLTOALLW](#).

8.8 Persistent Neighborhood Communication on Process Topologies

Persistent variants of the neighborhood collective operations can offer significant performance benefits for programs with repetitive communication patterns. The semantics are similar to persistent collective operations as described in [Section 6.13](#).

8.8.1 Persistent Neighborhood Gather

```

34 MPI_NEIGHBOR_ALLGATHER_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcount,
35   recvtype, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcount	number of elements received from each neighbor (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)
IN	info	info argument (handle)

```

OUT      request      communication request (handle)
1
2
3
C binding
4
int MPI_Neighbor_allgather_init(const void *sendbuf, int sendcount,
5
    MPI_Datatype sendtype, void *recvbuf, int recvcount,
6
    MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
7
    MPI_Request *request)
8
int MPI_Neighbor_allgather_init_c(const void *sendbuf, MPI_Count sendcount,
9
    MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
10
    MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
11
    MPI_Request *request)
12
Fortran 2008 binding
13
MPI_Neighbor_allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
14
    recvtype, comm, info, request, ierror)
15
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
16
    INTEGER, INTENT(IN) :: sendcount, recvcount
17
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
18
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
19
    TYPE(MPI_Comm), INTENT(IN) :: comm
20
    TYPE(MPI_Info), INTENT(IN) :: info
21
    TYPE(MPI_Request), INTENT(OUT) :: request
22
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24
MPI_Neighbor_allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
25
    recvtype, comm, info, request, ierror) !(_c)
26
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
27
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
28
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
29
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
30
    TYPE(MPI_Comm), INTENT(IN) :: comm
31
    TYPE(MPI_Info), INTENT(IN) :: info
32
    TYPE(MPI_Request), INTENT(OUT) :: request
33
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
34
Fortran binding
35
MPI_NEIGHBOR_ALLGATHER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT,
36
    RECVTYPE, COMM, INFO, REQUEST, IERROR)
37
    <type> SENDBUF(*), RECVBUF(*)
38
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
39
    IERROR
40
41
    Creates a persistent collective communication request for the neighborhood allgather
42
    operation.
43
44
MPI_NEIGHBOR_ALLGATHERV_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcoun
45
    ts, displs, recvtype, comm, info, request)
46
47
IN      sendbuf      starting address of send buffer (choice)
48

```

1	IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
2			
3	IN	sendtype	datatype of send buffer elements (handle)
4	OUT	recvbuf	starting address of receive buffer (choice)
5			
6	IN	recvcounts	nonnegative integer array (of length indegree) containing the number of elements that are received from each neighbor
7			
8			
9	IN	displs	integer array (of length indegree). Entry <i>i</i> specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor <i>i</i>
10			
11			
12	IN	recvtype	datatype of receive buffer elements (handle)
13	IN	comm	communicator with associated virtual topology (handle)
14			
15	IN	info	info argument (handle)
16	OUT	request	communication request (handle)
17			

C binding

```

19 int MPI_Neighbor_allgatherv_init(const void *sendbuf, int sendcount,
20                                MPI_Datatype sendtype, void *recvbuf, const int recvcounts[],
21                                const int displs[], MPI_Datatype recvtype, MPI_Comm comm,
22                                MPI_Info info, MPI_Request *request)
23
24 int MPI_Neighbor_allgatherv_init_c(const void *sendbuf, MPI_Count sendcount,
25                                   MPI_Datatype sendtype, void *recvbuf,
26                                   const MPI_Count recvcounts[], const MPI_Aint displs[],
27                                   MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
28                                   MPI_Request *request)

```

Fortran 2008 binding

```

30 MPI_Neighbor_allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcounts,
31                               displs, recvtype, comm, info, request, ierror)
32   TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
33   INTEGER, INTENT(IN) :: sendcount, displs(*)
34   TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
35   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
36   INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
37   TYPE(MPI_Comm), INTENT(IN) :: comm
38   TYPE(MPI_Info), INTENT(IN) :: info
39   TYPE(MPI_Request), INTENT(OUT) :: request
40   INTEGER, OPTIONAL, INTENT(OUT) :: ierror
41
42 MPI_Neighbor_allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcounts,
43                               displs, recvtype, comm, info, request, ierror) !(_c)
44   TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
45   INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
46   TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
47   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
48   INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)

```



```

INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_NEIGHBOR_ALLGATHERV_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS,
    DISPLS, RECVTYPE, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
    INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the neighborhood allgather operation.

8.8.2 Persistent Neighborhood Alltoall

```

MPI_NEIGHBOR_ALLTOALL_INIT(sendbuf, sendcount, sendtype, recvbuf, recvcoun,
    recvtype, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcount	number of elements sent to each neighbor (nonnegative integer)
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcoun	number of elements received from each neighbor (nonnegative integer)
IN	recvtype	datatype of receive buffer elements (handle)
IN	comm	communicator with associated virtual topology (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Neighbor_alltoall_init(const void *sendbuf, int sendcount,
    MPI_Datatype sendtype, void *recvbuf, int recvcoun,
    MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
    MPI_Request *request)

int MPI_Neighbor_alltoall_init_c(const void *sendbuf, MPI_Count sendcount,
    MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcoun,
    MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
    MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Neighbor_alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER, INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, info, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_NEIGHBOR_ALLTOALL_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
    RECVTYPE, COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
    IERROR

```

Creates a persistent collective communication request for the neighborhood alltoall operation.

```

MPI_NEIGHBOR_ALLTOALLV_INIT(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
    recvcounsts, rdispls, recvtype, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	nonnegative integer array (of length outdegree) specifying the number of elements to send to each neighbor
IN	sdispls	integer array (of length outdegree). Entry j specifies the displacement (relative to sendbuf) from which send the outgoing data to neighbor j
IN	sendtype	datatype of send buffer elements (handle)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcounsts	nonnegative integer array (of length indegree) specifying the number of elements that are received from each neighbor

IN	rdispls	integer array (of length indegree). Entry i specifies the displacement (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor i	1 2 3
IN	recvtype	datatype of receive buffer elements (handle)	4
IN	comm	communicator with associated virtual topology (handle)	5 6
IN	info	info argument (handle)	7
OUT	request	communication request (handle)	8 9

C binding

```

int MPI_Neighbor_alltoallv_init(const void *sendbuf, const int sendcounts[],
                                const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
                                const int recvcounts[], const int rdispls[],
                                MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
                                MPI_Request *request)

```

```

int MPI_Neighbor_alltoallv_init_c(const void *sendbuf,
                                   const MPI_Count sendcounts[], const MPI_Aint sdispls[],
                                   MPI_Datatype sendtype, void *recvbuf,
                                   const MPI_Count recvcounts[], const MPI_Aint rdispls[],
                                   MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
                                   MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Neighbor_alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
                             recvcounts, rdispls, recvtype, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
        recvcounts(*), rdispls(*)
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
                             recvcounts, rdispls, recvtype, comm, info, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
        recvcounts(*)
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
        rdispls(*)
    TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_NEIGHBOR_ALLTOALLV_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF,
    RECVCOUNTS, RDISPLS, RECVTYPE, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, RECVCOUNTS(*), RDISPLS(*),
    RECVTYPE, COMM, INFO, REQUEST, IERROR

```

Creates a persistent collective communication request for the neighborhood alltoallv operation.

```

MPI_NEIGHBOR_ALLTOALLW_INIT(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
    recvcounsts, rdispls, recvtypes, comm, info, request)

```

IN	sendbuf	starting address of send buffer (choice)
IN	sendcounts	nonnegative integer array (of length outdegree) specifying the number of elements to send to each neighbor
IN	sdispls	integer array (of length outdegree). Entry j specifies the displacement in bytes (relative to <code>sendbuf</code>) from which to take the outgoing data destined for neighbor j (array of integers)
IN	sendtypes	array of datatypes (of length outdegree). Entry j specifies the type of data to send to neighbor j (array of handles)
OUT	recvbuf	starting address of receive buffer (choice)
IN	recvcounsts	nonnegative integer array (of length indegree) specifying the number of elements that are received from each neighbor
IN	rdispls	integer array (of length indegree). Entry i specifies the displacement in bytes (relative to <code>recvbuf</code>) at which to place the incoming data from neighbor i (array of integers)
IN	recvtypes	array of datatypes (of length indegree). Entry i specifies the type of data received from neighbor i (array of handles)
IN	comm	communicator with associated virtual topology (handle)
IN	info	info argument (handle)
OUT	request	communication request (handle)

C binding

```

int MPI_Neighbor_alltoallw_init(const void *sendbuf, const int sendcounts[],
    const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
    void *recvbuf, const int recvcounsts[], const MPI_Aint rdispls[],
    const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Info info,
    MPI_Request *request)

int MPI_Neighbor_alltoallw_init_c(const void *sendbuf,
    const MPI_Count sendcounts[], const MPI_Aint sdispls[],

```

```

const MPI_Datatype sendtypes[], void *recvbuf,
const MPI_Count recvcunts[], const MPI_Aint rdispls[],
const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Info info,
MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Neighbor_alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
    recvcunts, rdispls, recvtypes, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), recvcunts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
    rdispls(*)
TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
    recvcunts, rdispls, recvtypes, comm, info, request, ierror)
!(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
    recvcunts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
    rdispls(*)
TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_NEIGHBOR_ALLTOALLW_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
    RECVCOUNTS, RDISPLS, RECVTYPES, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SENDTYPES(*), RECVCOUNTS(*), RECVTYPES(*), COMM,
    INFO, REQUEST, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) SDISPLS(*), RDISPLS(*)

```

Creates a persistent collective communication request for the neighborhood alltoallw operation.

8.9 An Application Example

Example 8.11. Neighborhood collective communication in a Cartesian virtual topology. The example in Listings 8.1–8.4 shows how the grid definition and inquiry functions can be used in an application program. A partial differential equation, for instance the Poisson equation, is to be solved on a rectangular domain. First, the MPI processes organize themselves in a two-dimensional structure. Each MPI process then inquires about the ranks of its neighbors in the four directions (up, down, right, left). The numerical problem is solved by an iterative method, the details of which are hidden in the subroutine `relax`. In each relaxation step each MPI process computes new values for the solution grid function at the points `u(1:100,1:100)` owned by the MPI process. Then the values at inter-process boundaries have to be exchanged with neighboring MPI processes. For example, the newly calculated values in `u(1,1:100)` must be sent into the halo cells `u(101,1:100)` of the left-hand neighbor with coordinates `(own_coord(1)-1,own_coord(2))`.

Listing 8.1: Set-up of MPI process structure for two-dimensional parallel Poisson solver

```

15  INTEGER ndims, num_neigh
16  LOGICAL reorder
17  PARAMETER (ndims=2, num_neigh=4, reorder=.true.)
18  INTEGER comm, comm_size, comm_cart, dims(ndims), ierr
19  INTEGER neigh_rank(num_neigh), own_coords(ndims), i, j, it
20  LOGICAL periods(ndims)
21  REAL u(0:101,0:101), f(0:101,0:101)
22  DATA dims / ndims * 0 /
23  comm = MPI_COMM_WORLD
24  CALL MPI_COMM_SIZE(comm, comm_size, ierr)
25  ! Set MPI process grid size and periodicity
26  CALL MPI_DIMS_CREATE(comm_size, ndims, dims, ierr)
27  periods(1) = .TRUE.
28  periods(2) = .TRUE.
29  ! Create a grid structure in WORLD group and inquire about own position
30  CALL MPI_CART_CREATE(comm, ndims, dims, periods, reorder, &
31  comm_cart, ierr)
32  CALL MPI_CART_GET(comm_cart, ndims, dims, periods, own_coords, ierr)
33  i = own_coords(1)
34  j = own_coords(2)
35  ! Look up the ranks for the neighbors. Own MPI process coordinates are (i,j).
36  ! Neighbors are (i-1,j), (i+1,j), (i,j-1), (i,j+1) modulo (dims(1),dims(2))
37  CALL MPI_CART_SHIFT(comm_cart, 0,1, neigh_rank(1), neigh_rank(2), ierr)
38  CALL MPI_CART_SHIFT(comm_cart, 1,1, neigh_rank(3), neigh_rank(4), ierr)
39  ! Initialize the grid functions and start the iteration
40  CALL init(u, f)
41  DO it=1,100
42  CALL relax(u, f)
43  ! Exchange data with neighbor processes
44  CALL exchange(u, comm_cart, neigh_rank, num_neigh)
45  END DO
46  CALL output(u)

```

Listing 8.2: Communication routine with local data copying and sparse neighborhood alltoall

```

47  SUBROUTINE exchange(u, comm_cart, neigh_rank, num_neigh)
48  USE MPI
49  REAL u(0:101,0:101)
50  INTEGER comm_cart, num_neigh, neigh_rank(num_neigh)
51  REAL sndbuf(100,num_neigh), rcvbuf(100,num_neigh)
52  INTEGER ierr

```

```

sndbuf(1:100,1) = u( 1,1:100)
sndbuf(1:100,2) = u(100,1:100)
sndbuf(1:100,3) = u(1:100, 1)
sndbuf(1:100,4) = u(1:100,100)
CALL MPI_NEIGHBOR_ALLTOALL(sndbuf, 100, MPI_REAL, rcvbuf, 100, MPI_REAL, &
                           comm_cart, ierr)
! instead of
! CALL MPI_IRECV(rcvbuf(1,1),100,MPI_REAL, neigh_rank(1),..., rq(1), ierr)
! CALL MPI_ISEND(sndbuf(1,2),100,MPI_REAL, neigh_rank(2),..., rq(2), ierr)
! Always pairing a receive from rank_source with a send to rank_dest
! of the same direction in MPI_CART_SHIFT!
! CALL MPI_IRECV(rcvbuf(1,2),100,MPI_REAL, neigh_rank(2),..., rq(3), ierr)
! CALL MPI_ISEND(sndbuf(1,1),100,MPI_REAL, neigh_rank(1),..., rq(4), ierr)
! CALL MPI_IRECV(rcvbuf(1,3),100,MPI_REAL, neigh_rank(3),..., rq(5), ierr)
! CALL MPI_ISEND(sndbuf(1,4),100,MPI_REAL, neigh_rank(4),..., rq(6), ierr)
! CALL MPI_IRECV(rcvbuf(1,4),100,MPI_REAL, neigh_rank(4),..., rq(7), ierr)
! CALL MPI_ISEND(sndbuf(1,3),100,MPI_REAL, neigh_rank(3),..., rq(8), ierr)
! Of course, one can first start all four IRECV and then all four ISEND,
! Or vice versa, but both in the sequence shown above. Otherwise, the
! matching would be wrong for 2 or only 1 MPI processes in a direction.
! CALL MPI_WAITALL(2*num_neigh, rq, statuses, ierr)
u( 0,1:100) = rcvbuf(1:100,1)
u(101,1:100) = rcvbuf(1:100,2)
u(1:100, 0) = rcvbuf(1:100,3)
u(1:100,101) = rcvbuf(1:100,4)
END

```

Listing 8.3: Communication routine with sparse neighborhood alltoallw and without local data copying

```

SUBROUTINE exchange(u, comm_cart, neigh_rank, num_neigh)
USE MPI
IMPLICIT NONE
REAL u(0:101,0:101)
INTEGER comm_cart, num_neigh, neigh_rank(num_neigh)
INTEGER sndcounts(num_neigh), sndtypes(num_neigh)
INTEGER rcvcounts(num_neigh), rcvtypes(num_neigh)
INTEGER(KIND=MPI_ADDRESS_KIND) lb, sizeofreal
INTEGER(KIND=MPI_ADDRESS_KIND) sdispls(num_neigh), rdispls(num_neigh)
INTEGER type_vec, ierr
! The following initialization need to be done only once
! before the first call of exchange.
CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lb, sizeofreal, ierr)
CALL MPI_TYPE_VECTOR(100, 1, 102, MPI_REAL, type_vec, ierr)
CALL MPI_TYPE_COMMIT(type_vec, ierr)
sndtypes(1:2) = type_vec
sndcounts(1:2) = 1
sndtypes(3:4) = MPI_REAL
sndcounts(3:4) = 100
rcvtypes = sndtypes
rcvcounts = sndcounts
sdispls(1) = ( 1 + 1*102) * sizeofreal ! first element of u( 1, 1:100)
sdispls(2) = (100 + 1*102) * sizeofreal ! first element of u(100, 1:100)
sdispls(3) = ( 1 + 1*102) * sizeofreal ! first element of u( 1:100, 1 )
sdispls(4) = ( 1 + 100*102) * sizeofreal ! first element of u( 1:100,100 )
rdispls(1) = ( 0 + 1*102) * sizeofreal ! first element of u( 0, 1:100)
rdispls(2) = (101 + 1*102) * sizeofreal ! first element of u(101, 1:100)
rdispls(3) = ( 1 + 0*102) * sizeofreal ! first element of u( 1:100, 0 )

```

```

1  rdispls(4) = ( 1 + 101*102) * sizeofreal ! first element of u( 1:100,101 )
2  ! the following communication has to be done in each call of exchange
3  CALL MPI_NEIGHBOR_ALLTOALLW(u, sndcounts, sdispls, sndtypes, &
4      u, rcvcounts, rdispls, rcvtypes, &
5      comm_cart, ierr)
6  ! The following finalizing need to be done only once
7  ! after the last call of exchange.
8  CALL MPI_TYPE_FREE(type_vec, ierr)
9  END

```

Listing 8.4: Two-dimensional parallel Poisson solver with persistent sparse neighborhood alltoallw and without local data copying

```

12  INTEGER ndims, num_neigh
13  LOGICAL reorder
14  PARAMETER (ndims=2, num_neigh=4, reorder=.true.)
15  INTEGER comm, comm_size, comm_cart, dims(ndims), it, ierr
16  LOGICAL periods(ndims)
17  REAL u(0:101,0:101), f(0:101,0:101)
18  DATA dims / ndims * 0 /
19  INTEGER sndcounts(num_neigh), sndtypes(num_neigh)
20  INTEGER rcvcounts(num_neigh), rcvtypes(num_neigh)
21  INTEGER(KIND=MPI_ADDRESS_KIND) lb, sizeofreal
22  INTEGER(KIND=MPI_ADDRESS_KIND) sdispls(num_neigh), rdispls(num_neigh)
23  INTEGER type_vec, request, info, status(MPI_STATUS_SIZE)
24  comm = MPI_COMM_WORLD
25  CALL MPI_COMM_SIZE(comm, comm_size, ierr)
26  ! Set MPI process grid size and periodicity
27  CALL MPI_DIMS_CREATE(comm_size, ndims, dims, ierr)
28  periods(1) = .TRUE.
29  periods(2) = .TRUE.
30  ! Create a grid structure in WORLD group
31  CALL MPI_CART_CREATE(comm, ndims, dims, periods, reorder, &
32      comm_cart, ierr)
33  ! Create datatypes for the neighborhood communication
34  !
35  ! Insert code from example in Listing 8.3 to create and initialize
36  ! sndcounts, sdispls, sndtypes, rcvcounts, rdispls, and rcvtypes
37  !
38  ! Initialize the neighborhood alltoallw operation
39  info = MPI_INFO_NULL
40  CALL MPI_NEIGHBOR_ALLTOALLW_INIT(u, sndcounts, sdispls, sndtypes, &
41      u, rcvcounts, rdispls, rcvtypes, &
42      comm_cart, info, request, ierr)
43  ! Initialize the grid functions and start the iteration
44  CALL init(u, f)
45  DO it=1,100
46  ! Start data exchange with neighbor processes
47  CALL MPI_START(request, ierr)
48  ! Compute inner cells
49  CALL relax_inner(u, f)
50  ! Check on completion of neighbor exchange
51  CALL MPI_WAIT(request, status, ierr)
52  ! Compute edge cells

```



```
    CALL relax_edges(u, f)
END DO
CALL output(u)
CALL MPI_REQUEST_FREE(request, ierr)
CALL MPI_TYPE_FREE(type_vec, ierr)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

DRAFT

DRAFT

Chapter 9

MPI Environmental Management

This chapter discusses routines for getting and, where appropriate, setting various parameters that relate to the MPI implementation and the execution environment (such as error handling). The procedures for entering and leaving the MPI execution environment are also described here.

9.1 Implementation Information

9.1.1 Version Inquiries

In order to cope with changes to the MPI standard, there are both compile-time and run-time ways to determine which version of the standard is in use in the environment one is using.

The “version” will be represented by two separate integers, for the version and subversion: In C,

```
#define MPI_VERSION 5
#define MPI_SUBVERSION 0
```

in Fortran,

```
INTEGER :: MPI_VERSION, MPI_SUBVERSION
PARAMETER (MPI_VERSION = 5)
PARAMETER (MPI_SUBVERSION = 0)
```

For runtime determination,

`MPI_GET_VERSION(version, subversion)`

OUT	version	version number (integer)
OUT	subversion	subversion number (integer)

C binding

```
int MPI_Get_version(int *version, int *subversion)
```

Fortran 2008 binding

```
MPI_Get_version(version, subversion, ierror)
  INTEGER, INTENT(OUT) :: version, subversion
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GET_VERSION(VERSION, SUBVERSION, IERROR)
  INTEGER VERSION, SUBVERSION, IERROR
```

`MPI_GET_VERSION` can be called at any time in an MPI program. This function must always be thread-safe, as defined in Section 11.6. Valid (`MPI_VERSION`, `MPI_SUBVERSION`) pairs in this and previous versions of the MPI standard are (5,0), (4,1), (4,0), (3,1), (3,0), (2,2), (2,1), (2,0), and (1,2).

`MPI_GET_LIBRARY_VERSION(version, resultlen)`

OUT	version	version number (string)
OUT	resultlen	Length (in printable characters) of the result returned in version (integer)

C binding

```
int MPI_Get_library_version(char *version, int *resultlen)
```

Fortran 2008 binding

```
MPI_Get_library_version(version, resultlen, ierror)
  CHARACTER(LEN=MPI_MAX_LIBRARY_VERSION_STRING), INTENT(OUT) :: version
  INTEGER, INTENT(OUT) :: resultlen
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GET_LIBRARY_VERSION(VERSION, RESULTLEN, IERROR)
  CHARACTER*(*) VERSION
  INTEGER RESULTLEN, IERROR
```

This routine returns a string representing the version of the MPI library. The version argument is a character string for maximum flexibility.

Advice to implementors. An implementation of MPI should return a different string for every change to its source code or build that could be visible to the user. (*End of advice to implementors.*)

The argument `version` must represent storage that is `MPI_MAX_LIBRARY_VERSION_STRING` characters long. `MPI_GET_LIBRARY_VERSION` may write up to this many characters into `version`.

The number of characters actually written is returned in the output argument, `resultlen`. In C, a null character is additionally stored at `version[resultlen]`. The value of `resultlen` cannot be larger than `MPI_MAX_LIBRARY_VERSION_STRING - 1`. In Fortran, `version` is padded on the right with blank characters. The value of `resultlen` cannot be larger than `MPI_MAX_LIBRARY_VERSION_STRING`.

`MPI_GET_LIBRARY_VERSION` can be called at any time in an MPI program. This function must always be thread-safe, as defined in Section 11.6.

9.1.2 Environmental Inquiries

When using the World Model (Section 11.2), a set of attributes that describe the execution environment is attached to the communicator `MPI_COMM_WORLD` when MPI is initialized. The values of these attributes can be inquired by using the function `MPI_COMM_GET_ATTR` described in Section 7.7 and in Section 21.3.7. It is erroneous to delete these attributes, free their keys, or change their values.

The list of predefined attribute keys include

MPI_TAG_UB: Upper bound for tag value.

MPI_IO: Rank of an MPI process that has regular I/O facilities (possibly the rank of the calling MPI process). MPI processes in the same communicator may return different values for this parameter.

MPI_WTIME_IS_GLOBAL: Boolean variable that indicates whether clocks are synchronized.

When using the Sessions Model (Section 11.3), only the **MPI_TAG_UB** attribute is available. Vendors may add implementation-specific parameters (such as node number, real memory size, virtual memory size, etc.)

These predefined attributes do not change value between MPI initialization (**MPI_INIT**) and MPI completion (**MPI_FINALIZE**), and cannot be updated or deleted by users.

Advice to users. Note that in the C binding, the value returned by these attributes is a *pointer* to an `int` containing the requested value. (*End of advice to users.*)

The required parameter values are discussed in more detail below:

Tag Values

Tag values range from 0 to the value returned for **MPI_TAG_UB**, inclusive. These values are guaranteed to be unchanging during the execution of an MPI program. In addition, the tag upper bound value must be *at least* 32767. An MPI implementation is free to make the value of **MPI_TAG_UB** larger than this; for example, the value $2^{30} - 1$ is also a valid value for **MPI_TAG_UB**.

In the Sessions Model, the attribute **MPI_TAG_UB** is attached to all communicators created by **MPI_COMM_CREATE_FROM_GROUP** and **MPI_INTERCOMM_CREATE_FROM_GROUPS**, with the same value on all MPI processes in the communicator. In the World Model, the attribute **MPI_TAG_UB** has the same value on all MPI processes of **MPI_COMM_WORLD**.

IO Rank

The value returned for **MPI_IO** is the rank of an MPI process that can provide language-standard I/O facilities. For Fortran, this means that all of the Fortran I/O operations are supported (e.g., `OPEN`, `REWIND`, `WRITE`). For C, this means that all of the ISO C I/O operations are supported (e.g., `fopen`, `fprintf`, `lseek`).

If every MPI process can provide language-standard I/O, then the value **MPI_ANY_SOURCE** will be returned. Otherwise, if the calling MPI process can provide language-standard I/O its rank in the group of the communicator will be returned. Otherwise, if some MPI process can provide language-standard I/O then the rank of one such MPI process in the group of the communicator will be returned. The same value need not be returned by all MPI processes. If no MPI process can provide language-standard I/O, then the value **MPI_PROC_NULL** will be returned.

Advice to users. Note that input is not collective, and this attribute does *not* indicate which MPI process can or does provide input. (*End of advice to users.*)

Clock Synchronization

The value returned for `MPI_WTIME_IS_GLOBAL` is 1 if clocks at all MPI processes in `MPI_COMM_WORLD` are synchronized, 0 otherwise. A collection of clocks is considered synchronized if explicit effort has been taken to synchronize them. The expectation is that the variation in time, as measured by calls to `MPI_WTIME`, will be less than one half the round-trip time for an MPI message of length zero. If time is measured at an MPI process just before a send and at another MPI process just after a matching receive, the second time should be always higher than the first one.

The attribute `MPI_WTIME_IS_GLOBAL` need not be present when the clocks are not synchronized (however, the attribute key `MPI_WTIME_IS_GLOBAL` is always valid). This attribute may be associated with communicators other than `MPI_COMM_WORLD`.

The attribute `MPI_WTIME_IS_GLOBAL` has the same value on all MPI processes of `MPI_COMM_WORLD`.

Inquire Processor Name

`MPI_GET_PROCESSOR_NAME(name, resultlen)`

OUT	name	A unique specifier for the actual (as opposed to virtual) node.
OUT	resultlen	Length (in printable characters) of the result returned in name

C binding

```
int MPI_Get_processor_name(char *name, int *resultlen)
```

Fortran 2008 binding

```
MPI_Get_processor_name(name, resultlen, ierror)
  CHARACTER(LEN=MPI_MAX_PROCESSOR_NAME), INTENT(OUT) :: name
  INTEGER, INTENT(OUT) :: resultlen
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_GET_PROCESSOR_NAME(NAME, RESULTLEN, IERROR)
  CHARACTER*(*) NAME
  INTEGER RESULTLEN, IERROR
```

This routine returns the name of the processor on which it was called at the moment of the call. The name is a character string for maximum flexibility. From this value it must be possible to identify a specific piece of hardware; possible values include “processor 9 in rack 4 of mpp.cs.org” and “231” (where 231 is the actual processor number in the running homogeneous system). The argument `name` must represent storage that is at least `MPI_MAX_PROCESSOR_NAME` characters long. `MPI_GET_PROCESSOR_NAME` may write up to this many characters into `name`.

The number of characters actually written is returned in the output argument, `resultlen`. In C, a null character is additionally stored at `name[resultlen]`. The value of `resultlen` cannot be larger than `MPI_MAX_PROCESSOR_NAME-1`. In Fortran, `name` is padded on the right with blank characters. The value of `resultlen` cannot be larger than `MPI_MAX_PROCESSOR_NAME`.

Rationale. This function allows MPI implementations that do process migration to return the current processor. Note that nothing in MPI *requires* or defines process migration; this definition of `MPI_GET_PROCESSOR_NAME` simply allows such an implementation. (*End of rationale.*)

Advice to users. The user must provide at least `MPI_MAX_PROCESSOR_NAME` space to write the processor name—processor names can be this long. The user should examine the output argument, `resultlen`, to determine the actual length of the name. (*End of advice to users.*)

Inquire Hardware Resource Information

`MPI_GET_HW_RESOURCE_INFO(hw_info)`

OUT `hw_info` info object created (handle)

C binding

`int MPI_Get_hw_resource_info(MPI_Info *hw_info)`

Fortran 2008 binding

`MPI_Get_hw_resource_info(hw_info, ierror)`

TYPE(MPI_Info), INTENT(OUT) :: `hw_info`

INTEGER, OPTIONAL, INTENT(OUT) :: `ierror`

Fortran binding

`MPI_GET_HW_RESOURCE_INFO(HW_INFO, IERROR)`

INTEGER HW_INFO, IERROR

`MPI_GET_HW_RESOURCE_INFO` is a local procedure that returns an info object containing information pertaining to the hardware platform on which the calling MPI process is executing at the moment of the call. This information is stored as (key,value) pairs where each key is the name of a hardware resource type and its value is set to "true" if the calling MPI process is restricted to a single instance of a hardware resource of that type and "false" otherwise. The order in which the keys are stored in `hw_info` is unspecified. This procedure will return different information for MPI processes that are restricted to different hardware resources. Otherwise, info objects with identical (key, value) pairs are returned. The user is responsible for freeing `hw_info` via `MPI_INFO_FREE`.

Advice to users. The information returned in the info object might reflect the "hardware" resources presented to the application by a virtualized environment and may be restricted by access permissions or other constraints like environment variables and OS settings. (*End of advice to users.*)

The keys stored in the `hw_info` object have a *Uniform Resource Identifier* (URI) format. The first part of the URI indicates the key provider and the second part conforms to the format used by this key provider. The key provider "mpi://" is reserved for exclusive use by the MPI standard.

Advice to implementors. Key provider names could be derived from MPI implementation names (e.g., "mpich://", "openmpi://"), from names of external libraries or pieces of

software (e.g., "hwloc://", "pmix://"), from names of programming or execution models (e.g., "openmp://"), from resource manager names (e.g., "slurm://") or from hardware vendor names. (*End of advice to implementors.*)

Advice to users. Users should be cautious when using such keys because comparisons between different providers may not be always meaningful or relevant. Also, the same hardware resource can be listed by multiple providers under different names.

One provider could convey types that represent individual hardware resource instances—for example, "provider_1://core/FF53C8A9" or "provider_1://numanode/2"—while another provider could provide types that represent categories or locations of hardware resources—for example, "provider_2://core" or "provider_2://numanode".

It is anticipated that types that represent categories or locations will be more useful for `MPI_COMM_SPLIT_TYPE` than types that represent individual resources. (*End of advice to users.*)

Advice to users. The keys stored in the info object returned by this procedure can be used in `MPI_COMM_SPLIT_TYPE` with the `split_type` value `MPI_COMM_TYPE_HW_GUIDED` or `MPI_COMM_TYPE_RESOURCE_GUIDED` as key *values* for the info key "mpi_hw_resource_type". (*End of advice to users.*)

Subsequent calls to `MPI_GET_HW_RESOURCE_INFO` may return different information throughout the execution of the program because an MPI process can be relocated (e.g., migrated or have its hardware restrictions changed).

Example 9.1. Splitting `MPI_COMM_WORLD` into subcommunicators according to NUMANode from the hwloc provider.

```

MPI_Info hw_info;
MPI_Comm hw_comm;
int      nb_keys  = 0, flag = 0;
int      is_found = 0, is_restricted = 0;
int      valuelen = 6; // max length between "false" and "true" + 1
char     *value    = calloc(valuelen, sizeof(char));
char     *hw_type  = calloc((MPI_MAX_INFO_KEY+1), sizeof(char));

MPI_Get_hw_resource_info(&hw_info);

MPI_Info_get_nkeys(hw_info, &nb_keys);
for(int index = 0 ; index < nb_keys ; index++){
    MPI_Info_get_nthkey(hw_info, index, hw_type);
    MPI_Info_get_string(hw_info, hw_type, &valuelen, value, &flag);
    if(strcmp(hw_type, "hwloc://NUMANode") == 0){
        is_found = 1;
        if(strcmp(value, "true") == 0)
            is_restricted = 1;
        break; // Resource of type NUMANode found
    }
}

// The calling MPI process is restricted to a resource
// of the chosen type (NUMANode)

```



```

1  if(is_found && is_restricted){
2      MPI_Info split_info;
3      int rank;
4
5      MPI_Info_create(&split_info);
6
7      // hw_type now serves as value for the "mpi_hw_resource_type" key
8      MPI_Info_set(split_info, "mpi_hw_resource_type", hw_type);
9
10     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11     MPI_Comm_split_type(MPI_COMM_WORLD, MPI_COMM_TYPE_RESOURCE_GUIDED,
12                         rank, split_info, &hw_comm);
13
14     // Check and use hw_comm from this point if it's a valid
15     // communicator or different from MPI_COMM_SELF or MPI_COMM_WORLD.
16 } else {
17     // If resource is not found or not restricted to it,
18     // the calling MPI process does not participate to the call
19     // hence the use of MPI_UNDEFINED as split_type and
20     // MPI_COMM_NULL is produced as output communicator
21
22     MPI_Comm_split_type(MPI_COMM_WORLD, MPI_UNDEFINED,
23                         -1, MPI_INFO_NULL, &hw_comm);
24 }

```

9.2 Memory Allocation

In some systems, message-passing and remote-memory-access (RMA) operations run faster when accessing specially allocated memory (e.g., memory that is shared by the other processes in the communicating group on an SMP). MPI provides a mechanism for allocating and freeing such special memory. The use of such memory for message-passing or RMA is not mandatory, and this memory can be used without restrictions as any other dynamically allocated memory. However, implementations may restrict the use of some RMA functionality as defined in Section 12.5.3.

MPI_ALLOC_MEM(size, info, baseptr)

IN	size	size of memory segment in bytes (nonnegative integer)
IN	info	info argument (handle)
OUT	baseptr	pointer to beginning of memory segment allocated

C binding

`int MPI_Alloc_mem(MPI_Aint size, MPI_Info info, void *baseptr)`

Fortran 2008 binding

`MPI_Alloc_mem(size, info, baseptr, ierror)`

USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR

INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size

```

1      TYPE(MPI_Info), INTENT(IN) :: info
2      TYPE(C_PTR), INTENT(OUT) :: baseptr
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

5      MPI_ALLOC_MEM(SIZE, INFO, BASEPTR, IERROR)
6      INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR
7      INTEGER INFO, IERROR

```

If the Fortran compiler provides `TYPE(C_PTR)`, then the following generic interface must be provided in the `mpi` module and should be provided in the (deprecated) `mpif.h` include file through overloading, i.e., with the same routine name as the routine with `INTEGER(KIND=MPI_ADDRESS_KIND) BASEPTR`, but with a different specific procedure name:

```

13     INTERFACE MPI_ALLOC_MEM
14         SUBROUTINE MPI_ALLOC_MEM(SIZE, INFO, BASEPTR, IERROR)
15             IMPORT :: MPI_ADDRESS_KIND
16             INTEGER :: INFO, IERROR
17             INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
18         END SUBROUTINE
19         SUBROUTINE MPI_ALLOC_MEM_CPTR(SIZE, INFO, BASEPTR, IERROR)
20             USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
21             IMPORT :: MPI_ADDRESS_KIND
22             INTEGER :: INFO, IERROR
23             INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
24             TYPE(C_PTR) :: BASEPTR
25         END SUBROUTINE
26     END INTERFACE

```

The base procedure name of this overloaded function is `MPI_ALLOC_MEM_CPTR`. The implied specific procedure names are described in Section 21.1.5.

By default, the allocated memory shall be aligned to at least the alignment required for load/store accesses of any datatype corresponding to a predefined MPI datatype. The `info` argument may be used to specify a desired alternative minimum alignment in bytes for the allocated memory by setting the value of the key `"mpi_minimum_memory_alignment"` to an integral number equal to a power of two. An implementation may ignore values smaller than the default required alignment. The `info` argument can also be used to provide directives that control the desired location of the allocated memory. Such a directive does not affect the semantics of the call. The corresponding `info` values are implementation-dependent. A null directive value of `info = MPI_INFO_NULL` is always valid.

The function `MPI_ALLOC_MEM` may raise an error of class `MPI_ERR_NO_MEM` to indicate it failed because memory is exhausted.

```

42     MPI_FREE_MEM(base)

```

44	IN	base	initial address of memory segment allocated by
45			<code>MPI_ALLOC_MEM</code> (choice)

C binding

```

48     int MPI_Free_mem(void *base)

```

Fortran 2008 binding

```

MPI_Free_mem(base, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: base
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FREE_MEM(BASE, IERROR)
    <type> BASE(*)
    INTEGER IERROR

```

The function `MPI_FREE_MEM` may raise an error of class `MPI_ERR_BASE` to indicate an invalid base argument.

Rationale. The C bindings of `MPI_ALLOC_MEM` and `MPI_FREE_MEM` are similar to the bindings for the `malloc` and `free` C library calls: a call to `MPI_Alloc_mem(..., &base)` should be paired with a call to `MPI_Free_mem(base)` (one less level of indirection). Both arguments are declared to be of same type `void*` so as to facilitate type casting. The Fortran binding is consistent with the C bindings: the Fortran `MPI_ALLOC_MEM` call returns in `baseptr` the `TYPE(C_PTR)` pointer or the (integer valued) address of the allocated memory. The `base` argument of `MPI_FREE_MEM` is a choice argument, which passes (a reference to) the variable stored at that location. (*End of rationale.*)

Advice to implementors. If `MPI_ALLOC_MEM` allocates special memory, then a design similar to the design of C `malloc` and `free` functions has to be used, in order to find out the size of a memory segment, when the segment is freed. If no special memory is used, `MPI_ALLOC_MEM` simply invokes `malloc`, and `MPI_FREE_MEM` invokes `free`.

A call to `MPI_ALLOC_MEM` can be used in shared memory systems to allocate memory in a shared memory segment. (*End of advice to implementors.*)

Example 9.2. Example use of `MPI_ALLOC_MEM` in Fortran with `TYPE(C_PTR)` pointers. We assume 4-byte REALs.

```

USE mpi_f08      ! or USE mpi      (not guaranteed with INCLUDE 'mpif.h')
USE, INTRINSIC :: ISO_C_BINDING
TYPE(C_PTR) :: p
REAL, DIMENSION(:,:), POINTER :: a      ! no memory is allocated
INTEGER, DIMENSION(2) :: shape
INTEGER(KIND=MPI_ADDRESS_KIND) :: size
shape = (/100,100/)
size = 4 * shape(1) * shape(2)          ! assuming 4 bytes per REAL
CALL MPI_ALLOC_MEM(size, MPI_INFO_NULL, p, ierr) ! memory is allocated and
CALL C_F_POINTER(p, a, shape) ! intrinsic ! now accessible via a(i,j)
...                                     ! in ISO_C_BINDING
a(3,5) = 2.71
...
CALL MPI_FREE_MEM(a, ierr)              ! memory is freed

```

Example 9.3. Example use of `MPI_ALLOC_MEM` in Fortran with nonstandard **Cray-pointers**. We assume 4-byte REALs, and assume that these pointers are address-sized.

```

1  REAL A
2  POINTER (P, A(100,100)) ! no memory is allocated
3  INTEGER(KIND=MPI_ADDRESS_KIND) SIZE
4  SIZE = 4*100*100
5  CALL MPI_ALLOC_MEM(SIZE, MPI_INFO_NULL, P, IERR)
6  ! memory is allocated
7  ...
8  A(3,5) = 2.71
9  ...
10 CALL MPI_FREE_MEM(A, IERR) ! memory is freed

```

This code is not Fortran 77 or Fortran 90 code. Some compilers may not support this code or need a special option, e.g., the GNU gFortran compiler needs `-fcray-pointer`.

Advice to implementors. Some compilers map Cray-pointers to address-sized integers, some to `TYPE(C_PTR)` pointers (e.g., Cray Fortran, version 7.3.3). From the user's viewpoint, this mapping is irrelevant because Examples 9.3 should work correctly with an MPI-3.0 (or later) library if Cray-pointers are available. (*End of advice to implementors.*)

Example 9.4. Same example, in C.

```

21 float (*f)[100][100];
22 /* no memory is allocated */
23 MPI_Alloc_mem(sizeof(float)*100*100, MPI_INFO_NULL, &f);
24 /* memory allocated */
25 ...
26 (*f)[5][3] = 2.71;
27 ...
28 MPI_Free_mem(f);

```

9.3 Error Handling

An MPI implementation may be unable or choose not to handle some failures that occur during MPI calls. These can include failures that generate exceptions or traps, such as floating point errors or access violations. The set of failures that are handled by MPI is implementation-dependent. Each such failure causes an error to be raised.

The above text takes precedence over any text on error handling within this document. Specifically, text that states that errors *will* be handled should be read as *may* be handled. More background information about how MPI treats errors can be found in Section 2.8.

A user can associate error handlers to four types of objects: communicators, windows, files, and sessions. The specified error handling routine will be used for any error that occurs during an MPI procedure or an operation that refers to the respective object. Figure 9.1 presents a diagram of the error handler that is invoked in different situations. When the MPI procedure or operation refers to a communicator, window, or file, the error handler for that object will be invoked; otherwise, if the procedure or operation refers to a session, the error handler for the session will be invoked. Some MPI procedures have indirect references to these objects. For example, in a procedure that takes a request handle as a parameter, an error during the corresponding operation is raised on the communicator, window, or file

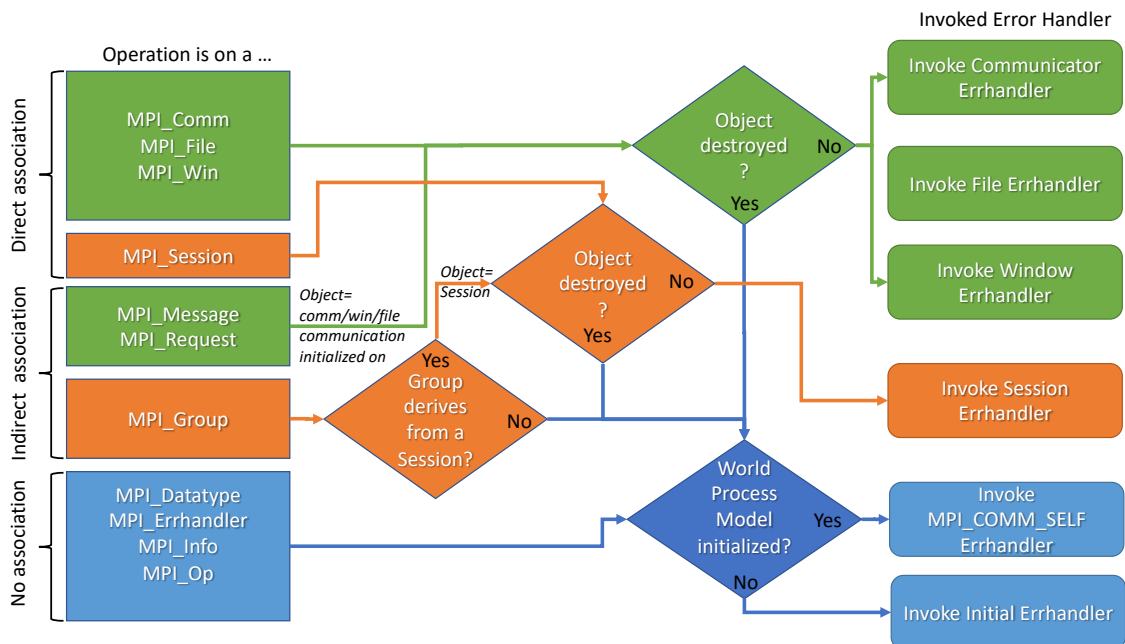


Figure 9.1: Diagram for deciding which error handler is invoked depending on the MPI objects associated with the operation and whether the Sessions Model or the World Model is used.

on which the request has been initialized. Similarly, a group contains a reference to the session from which it was derived, and procedures on groups invoke the error handler from that session. The referenced object may have been destroyed before an error is raised (e.g., a procedure on a group derived from a session that has been finalized), in this case, the associated error handler for the object cannot be obtained.

MPI procedures that do not refer to an MPI object from which the associated error handler can be obtained, directly or indirectly, are considered to be attached to the communicator `MPI_COMM_SELF` when using the World Model (see Section 11.2). When `MPI_COMM_SELF` is not initialized (i.e., before `MPI_INIT` / `MPI_INIT_THREAD`, after `MPI_FINALIZE`, or when using the Sessions Model exclusively) raising an error invokes the initial error handler (set during the launch operation, see Section 11.8.4). The attachment of error handlers to objects is purely local: different processes may attach different error handlers to corresponding objects.

Several predefined error handlers are available in MPI:

MPI_ERRORS_ARE_FATAL: The handler, when called, causes the program to abort all connected MPI processes. This is similar to calling `MPI_ABORT` using a communicator containing all connected processes with an implementation-specific value as the `errorcode` argument.

MPI_ERRORS_ABORT: The handler, when called, is invoked on a communicator in a manner similar to calling `MPI_ABORT` on that communicator. If the error handler is invoked on an window or file, it is similar to calling `MPI_ABORT` using a communicator containing the group of MPI processes associated with the window or file, respectively. If the error handler is invoked on a session, the operation aborts only the local MPI

process. In all cases, the value that would be provided as the `errorcode` argument to `MPI_ABORT` is implementation-specific.

MPI_ERRORS_RETURN: The handler has no effect other than returning the error code to the user.

Advice to implementors. The implementation-specific error information resulting from `MPI_ERRORS_ARE_FATAL` and `MPI_ERRORS_ABORT` provided to the invoking environment should be meaningful to the end-user, for example a predefined error class. (*End of advice to implementors.*)

Implementations may provide additional predefined error handlers and programmers can code their own error handlers.

Unless otherwise requested, the error handler `MPI_ERRORS_ARE_FATAL` is set as the default initial error handler and associated with predefined communicators. Thus, if the user chooses not to control error handling, every error that MPI handles is treated as fatal. Since (almost) all MPI calls return an error code, a user may choose to handle errors in its main code, by testing the return code of MPI calls and executing a suitable recovery code when the call was not successful. In this case, the error handler `MPI_ERRORS_RETURN` will be used. Usually it is more convenient and more efficient not to test for errors after each MPI call, and have such error handled by a nontrivial MPI error handler. Note that unlike predefined communicators, windows and files do not inherit from the initial error handler, as defined in Sections 12.6 and 15.7 respectively.

When an error is raised, MPI will provide the user information about that error using an error code. Some errors might prevent MPI from completing further API calls successfully and those functions will continue to report errors until the cause of the error is corrected or the user terminates the application. The user can make the determination of whether or not to attempt to continue when handling such an error.

Advice to users. For example, users may be unable to correct errors corresponding to some error classes, such as `MPI_ERR_INTERRN`. Such errors may cause subsequent MPI calls to complete in error. (*End of advice to users.*)

If the MPI error raised is one of the process failure error classes, as specified in Chapter 17, then that chapter defines the state of MPI.

Advice to implementors. A high-quality implementation will, to the greatest possible extent, circumscribe the impact of an error, so that normal processing can continue after an error handler was invoked. The implementation documentation will provide information on the possible effect of each class of errors and available recovery actions. (*End of advice to implementors.*)

An MPI error handler is an opaque object, which is accessed by a handle. MPI calls are provided to create new error handlers, to associate error handlers with objects, and to test which error handler is associated with an object. C has distinct typedefs for user defined error handling callback functions that accept communicator, file, window, and session arguments. In Fortran there are four user routines.

An error handler object is created by a call to `MPI_XXX_CREATE_ERRHANDLER`, where XXX is, respectively, COMM, WIN, FILE, or SESSION.

An error handler is attached to a communicator, window, file, or session by a call to `MPI_XXX_SET_ERRHANDLER`. The error handler must be either a predefined error handler, or an error handler that was created by a call to `MPI_XXX_CREATE_ERRHANDLER`, with matching `XXX`. An error handler can also be attached to a session using the `errorhandler` argument to `MPI_SESSION_INIT`. The predefined error handlers `MPI_ERRORS_RETURN` and `MPI_ERRORS_ARE_FATAL` can be attached to communicators, windows, files, or sessions.

The error handler currently associated with a communicator, window, file, or session can be retrieved by a call to `MPI_XXX_GET_ERRHANDLER`.

The MPI function `MPI_ERRHANDLER_FREE` can be used to free an error handler that was created by a call to `MPI_XXX_CREATE_ERRHANDLER`.

`MPI_XXX_GET_ERRHANDLER` behave as if a new error handler object is created. That is, once the error handler is no longer needed, `MPI_ERRHANDLER_FREE` should be called with the error handler returned from `MPI_XXX_GET_ERRHANDLER` to mark the error handler for deallocation. This provides behavior similar to that of `MPI_COMM_GROUP` and `MPI_GROUP_FREE`.

Advice to implementors. High-quality implementations should raise an error when an error handler that was created by a call to `MPI_XXX_CREATE_ERRHANDLER` is attached to an object of the wrong type with a call to `MPI_YYY_SET_ERRHANDLER`. To do so, it is necessary to maintain, with each error handler, information on the typedef of the associated user function. (*End of advice to implementors.*)

The syntax for these calls is given below.

9.3.1 Error Handlers for Communicators

`MPI_COMM_CREATE_ERRHANDLER(comm_errhandler_fn, errhandler)`

IN	<code>comm_errhandler_fn</code>	user defined error handling procedure (function)
OUT	<code>errhandler</code>	MPI error handler (handle)

C binding

```
int MPI_Comm_create_errhandler(
    MPI_Comm_errhandler_function *comm_errhandler_fn,
    MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_Comm_create_errhandler(comm_errhandler_fn, errhandler, ierror)
  PROCEDURE(MPI_Comm_errhandler_function) :: comm_errhandler_fn
  TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_CREATE_ERRHANDLER(COMM_ERRHANDLER_FN, ERRHANDLER, IERROR)
  EXTERNAL COMM_ERRHANDLER_FN
  INTEGER ERRHANDLER, IERROR
```


Creates an error handler that can be attached to communicators. The user routine should be, in C, a function of type `MPI_Comm_errhandler_function`, which is defined as

```
typedef void MPI_Comm_errhandler_function(MPI_Comm *comm, int *error_code,
    ... );
```

The first argument is the communicator in use. The second is the error code to be returned by the MPI routine that raised the error. If the routine would have returned `MPI_ERR_IN_STATUS`, it is the error code returned in the status for the request that caused the error handler to be invoked. The remaining arguments are “varargs” arguments whose number and meaning is implementation-dependent. An implementation should clearly document these arguments. Addresses are used so that the handler may be written in Fortran. With the Fortran `mpi_f08` module, the user routine `comm_errhandler_fn` should be of the form:

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Comm_errhandler_function(comm, error_code)
    TYPE(MPI_Comm) :: comm
    INTEGER :: error_code
```

With the Fortran `mpi` module and (deprecated) `mpif.h` include file, the user routine `COMM_ERRHANDLER_FN` should be of the form:

```
SUBROUTINE COMM_ERRHANDLER_FUNCTION(COMM, ERROR_CODE)
  INTEGER COMM, ERROR_CODE
```

Rationale. The variable argument list is provided because it provides an ISO-standard hook for providing additional information to the error handler; without this hook, ISO C prohibits additional arguments. (*End of rationale.*)

```
MPI_COMM_SET_ERRHANDLER(comm, errhandler)
```

INOUT	comm	communicator (handle)
IN	errhandler	new error handler for communicator (handle)

C binding

```
int MPI_Comm_set_errhandler(MPI_Comm comm, MPI_Errhandler errhandler)
```

Fortran 2008 binding

```
MPI_Comm_set_errhandler(comm, errhandler, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_SET_ERRHANDLER(COMM, ERRHANDLER, IERROR)
  INTEGER COMM, ERRHANDLER, IERROR
```

Attaches a new error handler to a communicator. The error handler must be either a predefined error handler, or an error handler created by a call to `MPI_COMM_CREATE_ERRHANDLER`.

Advice to users. A newly created communicator inherits the error handler that is associated with the “parent” communicator. In the World Model, the user can specify an error handler for all communicators by associating this handler with the predefined communicators (i.e., `MPI_COMM_WORLD` and `MPI_COMM_SELF`) before creating other communicators. (*End of advice to users.*)

`MPI_COMM_GET_ERRHANDLER(comm, errhandler)`

IN	comm	communicator (handle)
OUT	errhandler	error handler currently associated with communicator (handle)

C binding

`int MPI_Comm_get_errhandler(MPI_Comm comm, MPI_Errhandler *errhandler)`

Fortran 2008 binding

`MPI_Comm_get_errhandler(comm, errhandler, ierror)`
 TYPE(MPI_Comm), INTENT(IN) :: comm
 TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

`MPI_COMM_GET_ERRHANDLER(COMM, ERRHANDLER, IERROR)`
 INTEGER COMM, ERRHANDLER, IERROR

Retrieves the error handler currently associated with a communicator. For example, a library function may register at its entry point the current error handler for a communicator, set its own private error handler for this communicator, and restore before exiting the previous error handler.

9.3.2 Error Handlers for Windows

`MPI_WIN_CREATE_ERRHANDLER(win_errhandler_fn, errhandler)`

IN	win_errhandler_fn	user defined error handling procedure (function)
OUT	errhandler	MPI error handler (handle)

C binding

`int MPI_Win_create_errhandler(MPI_Win_errhandler_function *win_errhandler_fn, MPI_Errhandler *errhandler)`

Fortran 2008 binding

`MPI_Win_create_errhandler(win_errhandler_fn, errhandler, ierror)`
 PROCEDURE(MPI_Win_errhandler_function) :: win_errhandler_fn
 TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

```

MPI_WIN_CREATE_ERRHANDLER(WIN_ERRHANDLER_FN, ERRHANDLER, IERROR)
    EXTERNAL WIN_ERRHANDLER_FN
    INTEGER ERRHANDLER, IERROR

```

Creates an error handler that can be attached to a window object. The user routine should be, in C, a function of type `MPI_Win_errhandler_function`, which is defined as `typedef void MPI_Win_errhandler_function(MPI_Win *win, int *error_code, ...);`

The first argument is the window in use, the second is the error code to be returned. The remaining arguments are “varargs” arguments whose number and meaning is implementation-dependent. An implementation should clearly document these arguments. With the Fortran `mpi_f08` module, the user routine `win_errhandler_fn` should be of the form:

```

ABSTRACT INTERFACE
    SUBROUTINE MPI_Win_errhandler_function(win, error_code)
        TYPE(MPI_Win) :: win
        INTEGER :: error_code

```

With the Fortran `mpi` module and (deprecated) `mpif.h` include file, the user routine `WIN_ERRHANDLER_FN` should be of the form:

```

SUBROUTINE WIN_ERRHANDLER_FUNCTION(WIN, ERROR_CODE)
    INTEGER WIN, ERROR_CODE

```

```

MPI_WIN_SET_ERRHANDLER(win, errhandler)

```

INOUT	win	window object (handle)
IN	errhandler	new error handler for window (handle)

C binding

```

int MPI_Win_set_errhandler(MPI_Win win, MPI_Errhandler errhandler)

```

Fortran 2008 binding

```

MPI_Win_set_errhandler(win, errhandler, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_SET_ERRHANDLER(WIN, ERRHANDLER, IERROR)
    INTEGER WIN, ERRHANDLER, IERROR

```

Attaches a new error handler to a window. The error handler must be either a pre-defined error handler, or an error handler created by a call to

[`MPI\_WIN\_CREATE\_ERRHANDLER`](#).

```

MPI_WIN_GET_ERRHANDLER(win, errhandler)

```

IN	win	window object (handle)
OUT	errhandler	error handler currently associated with window (handle)

C binding

```
int MPI_Win_get_errhandler(MPI_Win win, MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_Win_get_errhandler(win, errhandler, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_GET_ERRHANDLER(WIN, ERRHANDLER, IERROR)
    INTEGER WIN, ERRHANDLER, IERROR
```

Retrieves the error handler currently associated with a window.

9.3.3 Error Handlers for Files

```
MPI_FILE_CREATE_ERRHANDLER(file_errhandler_fn, errhandler)
```

IN	file_errhandler_fn	user defined error handling procedure (function)
OUT	errhandler	MPI error handler (handle)

C binding

```
int MPI_File_create_errhandler(
    MPI_File_errhandler_function *file_errhandler_fn,
    MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_File_create_errhandler(file_errhandler_fn, errhandler, ierror)
    PROCEDURE(MPI_File_errhandler_function) :: file_errhandler_fn
    TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_CREATE_ERRHANDLER(FILE_ERRHANDLER_FN, ERRHANDLER, IERROR)
    EXTERNAL FILE_ERRHANDLER_FN
    INTEGER ERRHANDLER, IERROR
```

Creates an error handler that can be attached to a file object. The user routine should be, in C, a function of type `MPI_File_errhandler_function`, which is defined as

```
typedef void MPI_File_errhandler_function(MPI_File *file, int *error_code,
    . . . );
```

The first argument is the file in use, the second is the error code to be returned. The remaining arguments are “varargs” arguments whose number and meaning is implementation-dependent. An implementation should clearly document these arguments.

With the Fortran `mpi_f08` module, the user routine `file_errhandler_fn` should be of the form:

```
ABSTRACT INTERFACE
    SUBROUTINE MPI_File_errhandler_function(file, error_code)
        TYPE(MPI_File) :: file
        INTEGER :: error_code
```

With the Fortran `mpi` module and (deprecated) `mpif.h` include file, the user routine `FILE_ERRHANDLER_FN` should be of the form:

```
SUBROUTINE FILE_ERRHANDLER_FUNCTION(FILE, ERROR_CODE)
    INTEGER FILE, ERROR_CODE
```

```
MPI_FILE_SET_ERRHANDLER(file, errhandler)
```

```
    INOUT    file                file (handle)
```

```
    IN       errhandler          new error handler for file (handle)
```

C binding

```
int MPI_File_set_errhandler(MPI_File file, MPI_Errhandler errhandler)
```

Fortran 2008 binding

```
MPI_File_set_errhandler(file, errhandler, ierror)
```

```
    TYPE(MPI_File), INTENT(IN) :: file
```

```
    TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
```

```
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SET_ERRHANDLER(FILE, ERRHANDLER, IERROR)
```

```
    INTEGER FILE, ERRHANDLER, IERROR
```

Attaches a new error handler to a file. The error handler must be either a predefined error handler, or an error handler created by a call to [MPI_FILE_CREATE_ERRHANDLER](#).

```
MPI_FILE_GET_ERRHANDLER(file, errhandler)
```

```
    IN       file                file (handle)
```

```
    OUT      errhandler          error handler currently associated with file (handle)
```

C binding

```
int MPI_File_get_errhandler(MPI_File file, MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_File_get_errhandler(file, errhandler, ierror)
```

```
    TYPE(MPI_File), INTENT(IN) :: file
```

```
    TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
```

```
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_ERRHANDLER(FILE, ERRHANDLER, IERROR)
```

```
    INTEGER FILE, ERRHANDLER, IERROR
```

Retrieves the error handler currently associated with a file.

9.3.4 Error Handlers for Sessions

MPI_SESSION_CREATE_ERRHANDLER(session_errhandler_fn, errhandler)

IN	session_errhandler_fn	user defined error handling procedure (function)
OUT	errhandler	MPI error handler (handle)

C binding

```
int MPI_Session_create_errhandler(
    MPI_Session_errhandler_function *session_errhandler_fn,
    MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_Session_create_errhandler(session_errhandler_fn, errhandler, ierror)
  PROCEDURE(MPI_Session_errhandler_function) :: session_errhandler_fn
  TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SESSION_CREATE_ERRHANDLER(SESSION_ERRHANDLER_FN, ERRHANDLER, IERROR)
  EXTERNAL SESSION_ERRHANDLER_FN
  INTEGER ERRHANDLER, IERROR
```

Creates an error handler that can be attached to a session object. In C, the `session_errhandler_fn` argument should be a function of type `MPI_Session_errhandler_function`, which is defined as

```
typedef void MPI_Session_errhandler_function(MPI_Session *session,
    int *error_code, ...);
```

The first argument is the session in use, the second is the error code to be returned. The remaining arguments are “varargs” arguments whose number and meaning is implementation-dependent. An implementation should clearly document these arguments. With the Fortran `mpi_f08` module, the `session_errhandler_fn` argument should be of the form:

```
ABSTRACT INTERFACE
  SUBROUTINE MPI_Session_errhandler_function(session, error_code)
    TYPE(MPI_Session) :: session
    INTEGER :: error_code
```

With the Fortran `mpi` module and (deprecated) `mpif.h` include file, the `SESSION_ERRHANDLER_FN` argument should be of the form:

```
SUBROUTINE SESSION_ERRHANDLER_FUNCTION(SESSION, ERROR_CODE)
  INTEGER SESSION, ERROR_CODE
```

MPI_SESSION_SET_ERRHANDLER(session, errhandler)

INOUT	session	session (handle)
IN	errhandler	new error handler for session (handle)

C binding

```
int MPI_Session_set_errhandler(MPI_Session session, MPI_Errhandler errhandler)
```

Fortran 2008 binding

```
MPI_Session_set_errhandler(session, errhandler, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SESSION_SET_ERRHANDLER(SESSION, ERRHANDLER, IERROR)
    INTEGER SESSION, ERRHANDLER, IERROR
```

Attaches a new error handler to a session. The error handler must be either a pre-defined error handler, or an error handler created by a call to [MPI_SESSION_CREATE_ERRHANDLER](#).

```
MPI_SESSION_GET_ERRHANDLER(session, errhandler)
```

IN	session	session (handle)
OUT	errhandler	error handler currently associated with session (handle)

C binding

```
int MPI_Session_get_errhandler(MPI_Session session, MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_Session_get_errhandler(session, errhandler, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SESSION_GET_ERRHANDLER(SESSION, ERRHANDLER, IERROR)
    INTEGER SESSION, ERRHANDLER, IERROR
```

Retrieves the error handler currently associated with a session.

9.3.5 Freeing Errorhandlers and Retrieving Error Strings

```
MPI_ERRHANDLER_FREE(errhandler)
```

INOUT	errhandler	MPI error handler (handle)
-------	------------	----------------------------

C binding

```
int MPI_Errhandler_free(MPI_Errhandler *errhandler)
```

Fortran 2008 binding

```
MPI_Errhandler_free(errhandler, ierror)
    TYPE(MPI_Errhandler), INTENT(INOUT) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ERRHANDLER_FREE(ERRHANDLER, IERROR)
    INTEGER ERRHANDLER, IERROR
```

Marks the error handler associated with `errhandler` for deallocation and sets `errhandler` to `MPI_ERRHANDLER_NULL`. The error handler will be deallocated after all the objects associated with it (communicator, window, or file) have been deallocated.

```
MPI_ERROR_STRING(errorcode, string, resultlen)
```

IN	<code>errorcode</code>	Error code returned by an MPI routine
OUT	<code>string</code>	Text that corresponds to the <code>errorcode</code>
OUT	<code>resultlen</code>	Length (in printable characters) of the result returned in <code>string</code>

C binding

```
int MPI_Error_string(int errorcode, char *string, int *resultlen)
```

Fortran 2008 binding

```
MPI_Error_string(errorcode, string, resultlen, ierror)
    INTEGER, INTENT(IN) :: errorcode
    CHARACTER(LEN=MPI_MAX_ERROR_STRING), INTENT(OUT) :: string
    INTEGER, INTENT(OUT) :: resultlen
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ERROR_STRING(ERRORCODE, STRING, RESULTLEN, IERROR)
    INTEGER ERRORCODE, RESULTLEN, IERROR
    CHARACTER*(*) STRING
```

Returns the error string associated with an error code or class. The argument `string` must represent storage that is at least `MPI_MAX_ERROR_STRING` characters long. The number of characters actually written is returned in the output argument, `resultlen`. This function must always be thread-safe, as defined in Section 11.6. It is one of the few routines that may be called before MPI is initialized or after MPI is finalized.

Rationale. The form of this function was chosen to make the Fortran and C bindings similar. A version that returns a pointer to a string has two difficulties. First, the return string must be statically allocated and different for each error message (allowing the pointers returned by successive calls to `MPI_ERROR_STRING` to point to the correct message). Second, in Fortran, a function declared as returning `CHARACTER*(*)` can not be referenced in, for example, a `PRINT` statement. (*End of rationale.*)

9.4 Error Codes and Classes

The error codes returned by MPI are left entirely to the implementation (with the exception of `MPI_SUCCESS`). This is done to allow an implementation to provide as much information as possible in the error code (for use with `MPI_ERROR_STRING`).

All MPI function calls shall return `MPI_SUCCESS` if and only if the specification of that function has been fulfilled at the point of return. For multiple completion functions,

if the function returns `MPI_ERR_IN_STATUS`, the error code in each status object shall be set to `MPI_SUCCESS` if and only if the specification of the operation represented by the corresponding `MPI_Request` has been fulfilled at the point of return.

When an operation raises an error, it may not satisfy its specification (for example, a synchronizing operation may not have synchronized) and the content of the output buffers, targeted memory, or output parameters is undefined. However, a valid error code shall always be set when an operation raises an error, whether in the return value, error field in the status object, or element in an array of error codes.

To make it possible for an application to interpret an error code, the routine `MPI_ERROR_CLASS` converts any error code into one of a small set of standard error codes, called **error classes**. Valid error classes are shown in Table 9.1 and Table 9.2.

The error classes are a subset of the error codes: an MPI function may return an error class number; and the function `MPI_ERROR_STRING` can be used to compute the error string associated with an error class. The values defined for MPI error classes are valid MPI error codes.

The error codes satisfy,

$$0 = \text{MPI_SUCCESS} < \text{MPI_ERR_}\dots \leq \text{MPI_ERR_LASTCODE}.$$

Rationale. The difference between `MPI_ERR_UNKNOWN` and `MPI_ERR_OTHER` is that `MPI_ERROR_STRING` can return useful information about `MPI_ERR_OTHER`.

Note that `MPI_SUCCESS` = 0 is necessary to be consistent with C practice; the separation of error classes and error codes allows us to define the error classes this way. Having a known `LASTCODE` is often a nice sanity check as well. (*End of rationale.*)

Advice to implementors. Note that all `MPI_T_` return codes, which must have the prefix `MPI_T_ERR_`, are also required to satisfy

$$0 = \text{MPI_SUCCESS} < \text{MPI_T_ERR_XXX} \leq \text{MPI_ERR_LASTCODE}.$$

as described in Section 16.3.10. (*End of advice to implementors.*)

`MPI_ERROR_CLASS(errorcode, errorclass)`

IN	errorcode	Error code returned by an MPI routine
OUT	errorclass	Error class associated with errorcode

C binding

```
int MPI_Error_class(int errorcode, int *errorclass)
```

Fortran 2008 binding

```
MPI_Error_class(errorcode, errorclass, ierror)
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, INTENT(OUT) :: errorclass
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ERROR_CLASS(ERRORCODE, ERRORCLASS, IERROR)
  INTEGER ERRORCODE, ERRORCLASS, IERROR
```


Table 9.1: Error classes (Part 1)

MPI_SUCCESS	No error
MPI_ERR_ACCESS	Permission denied
MPI_ERR_AMODE	Error related to the <code>amode</code> passed to <code>MPI_FILE_OPEN</code>
MPI_ERR_ARG	Invalid argument of some other kind
MPI_ERR_ASSERT	Invalid assertion argument
MPI_ERR_BAD_FILE	Invalid file name (e.g., path name too long)
MPI_ERR_BASE	Invalid base passed to <code>MPI_FREE_MEM</code>
MPI_ERR_BUFFER	Invalid buffer pointer argument
MPI_ERR_COMM	Invalid communicator argument
MPI_ERR_CONVERSION	An error occurred in a user supplied data conversion function
MPI_ERR_COUNT	Invalid count argument
MPI_ERR_DIMS	Invalid dimension argument
MPI_ERR_DISP	Invalid displacement argument
MPI_ERR_DUP_DATAREP	Conversion functions could not be registered because a data representation identifier that was already defined was passed to <code>MPI_REGISTER_DATAREP</code>
MPI_ERR_ERRHANDLER	Invalid error handler argument
MPI_ERR_FILE	Invalid file handle argument
MPI_ERR_FILE_EXISTS	File exists
MPI_ERR_FILE_IN_USE	File operation could not be completed, as the file is currently open by some process
MPI_ERR_GROUP	Invalid group argument
MPI_ERR_INFO	Invalid info argument
MPI_ERR_INFO_KEY	Key longer than <code>MPI_MAX_INFO_KEY</code>
MPI_ERR_INFO_NOKEY	Invalid key passed to <code>MPI_INFO_DELETE</code>
MPI_ERR_INFO_VALUE	Value longer than <code>MPI_MAX_INFO_VAL</code>
MPI_ERR_IN_STATUS	Error code is in status
MPI_ERR_INTERN	Internal MPI (implementation) error
MPI_ERR_IO	Other I/O error
MPI_ERR_KEYVAL	Invalid keyval argument
MPI_ERR_LOCKTYPE	Invalid locktype argument
MPI_ERR_NAME	Invalid service name passed to <code>MPI_LOOKUP_NAME</code>
MPI_ERR_NO_MEM	<code>MPI_ALLOC_MEM</code> failed because memory is exhausted
MPI_ERR_NO_SPACE	Not enough space
MPI_ERR_NO_SUCH_FILE	File does not exist
MPI_ERR_NOT_SAME	Collective argument not identical on all processes, or collective routines called in a different order by different processes

Table 9.2: Error classes (Part 2)

MPI_ERR_OP	Invalid operation argument
MPI_ERR_OTHER	Known error not in this list
MPI_ERR_PENDING	Pending request
MPI_ERR_PORT	Invalid port name passed to MPI_COMM_CONNECT
MPI_ERR_PROC_ABORTED	Operation failed because a peer process has aborted
MPI_ERR_PROC_FAILED	Operation could not complete due to process failure (a fail-stop failure)
MPI_ERR_PROC_FAILED_PENDING	Non-blocking or persistent operation could not complete due to process failure, but request is still active
MPI_ERR_QUEUE	Invalid queue argument, or invalid queue usage
MPI_ERR_QUOTA	Quota exceeded
MPI_ERR_RANK	Invalid rank argument
MPI_ERR_READ_ONLY	Read-only file or file system
MPI_ERR_REQUEST	Invalid request argument
MPI_ERR_REVOKED	Communication object used in the operation has been revoked
MPI_ERR_RMA_ATTACH	Memory cannot be attached (e.g., because of resource exhaustion)
MPI_ERR_RMA_CONFLICT	Conflicting accesses to window
MPI_ERR_RMA_FLAVOR	Passed window has the wrong flavor for the called function
MPI_ERR_RMA_RANGE	Target memory is not part of the window (in the case of a window created with MPI_WIN_CREATE_DYNAMIC , target memory is not attached)
MPI_ERR_RMA_SHARED	Memory cannot be shared (e.g., some process in the group of the specified communicator cannot expose shared memory)
MPI_ERR_RMA_SYNC	Wrong synchronization of RMA calls
MPI_ERR_ROOT	Invalid root argument
MPI_ERR_SERVICE	Invalid service name passed to MPI_UNPUBLISH_NAME
MPI_ERR_SESSION	Invalid session argument
MPI_ERR_SIZE	Invalid size argument
MPI_ERR_SPAWN	Error in spawning processes
MPI_ERR_TAG	Invalid tag argument
MPI_ERR_TOPOLOGY	Invalid topology argument
MPI_ERR_TRUNCATE	Message truncated on receive
MPI_ERR_TYPE	Invalid datatype argument
MPI_ERR_UNKNOWN	Unknown error
MPI_ERR_UNSUPPORTED_DATAREP	Unsupported datarep passed to MPI_FILE_SET_VIEW
MPI_ERR_UNSUPPORTED_OPERATION	Unsupported operation, such as seeking on a file that supports sequential access only
MPI_ERR_VALUE_TOO_LARGE	Value is too large to store
MPI_ERR_WIN	Invalid window argument
MPI_ERR_LASTCODE	Last error code

The function `MPI_ERROR_CLASS` maps each standard error code (error class) onto itself.

This function must always be thread-safe, as defined in Section 11.6. It is one of the few routines that may be called before MPI is initialized or after MPI is finalized.

9.5 Error Classes, Error Codes, and Error Handlers

Users may want to write a layered library on top of an existing MPI implementation, and this library may have its own set of error codes and classes. An example of such a library is an I/O library based on MPI, see Chapter 15. For this purpose, functions are needed to:

1. add a new error class and remove previously added user-defined error classes;
2. associate error codes with this error class, so that `MPI_ERROR_CLASS` works;
3. associate strings with these error codes, so that `MPI_ERROR_STRING` works;
4. remove such associations;
5. invoke the error handler associated with a communicator, window, file, or session object.

Several procedures are provided to do this. They are all local.

9.5.1 User-Defined Error Classes and Codes

The procedures that add and remove error classes, codes, or strings are thread-safe, as defined in Section 11.6. They are some of the few MPI procedures that may be called before MPI is initialized or after MPI is finalized, as defined in Section 11.4.1.

Advice to users. Note that despite the procedures being thread-safe, some concurrent calls can result in undefined behavior. Notably, the rules mandating that a call adding an error class/code/string must precede a call that removes that error class/code/string apply even when the procedures are called from different threads. Calling the procedures with different input values for the class/code parameters is always thread-safe. (*End of advice to users.*)

`MPI_ADD_ERROR_CLASS(errorclass)`

OUT errorclass value for the new error class (integer)

C binding

`int MPI_Add_error_class(int *errorclass)`

Fortran 2008 binding

`MPI_Add_error_class(errorclass, ierror)`
 INTEGER, INTENT(OUT) :: errorclass
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

```

MPI_ADD_ERROR_CLASS(ERRORCLASS, IERROR)
    INTEGER ERRORCLASS, IERROR

```

Creates a new error class and returns the value for it.

Rationale. To avoid conflicts with existing error codes and classes, the value is set by the implementation and not by the user. (*End of rationale.*)

Advice to users. Since a call to `MPI_ADD_ERROR_CLASS` is local, the same `errorclass` may not be returned on all processes that make this call. Thus, it is not safe to assume that registering a new error on a set of processes at the same time will yield the same `errorclass` on all of the processes. Getting the “same” error on multiple processes may not cause the same value of error code to be generated. (*End of advice to users.*)

The value of `MPI_ERR_LASTCODE` is a constant value and is not affected by new user-defined error codes and classes. Instead, when using the World Model (Section 11.2), a predefined attribute key `MPI_LASTUSED` is associated with `MPI_COMM_WORLD`. The attribute value corresponding to this key is the current maximum error class including the user-defined ones. This is a local value and may be different on different processes. The value returned by this key is always greater than or equal to `MPI_ERR_LASTCODE`.

Advice to users. The value returned by the key `MPI_LASTUSED` will not change unless the user calls a procedure to explicitly add or remove an error class/code. In a multithreaded environment, the user must take extra care in assuming this value has not changed. Note that error codes and error classes are not necessarily dense. A user may not assume that each error class below `MPI_LASTUSED` is valid. (*End of advice to users.*)

```

MPI_REMOVE_ERROR_CLASS(errorclass)

```

```

    IN          errorclass          value for the error class to remove (integer)

```

C binding

```

int MPI_Remove_error_class(int errorclass)

```

Fortran 2008 binding

```

MPI_Remove_error_class(errorclass, ierror)
    INTEGER, INTENT(IN) :: errorclass
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_REMOVE_ERROR_CLASS(ERRORCLASS, IERROR)
    INTEGER ERRORCLASS, IERROR

```

Removes a user-created error class.

The value of the predefined attribute key `MPI_LASTUSED` associated with `MPI_COMM_WORLD` is updated to reflect the maximum error class value. Note that there may be unused error classes that have a smaller value than `MPI_LASTUSED`.

It is erroneous to call `MPI_REMOVE_ERROR_CLASS` with a value for `errorclass` that was not added by a call to `MPI_ADD_ERROR_CLASS`. Once an `errorclass` is removed by calling `MPI_REMOVE_ERROR_CLASS`, it is erroneous to remove it again without first obtaining the value from another call to `MPI_ADD_ERROR_CLASS`. It is erroneous to remove an error class when its associated error codes have not been removed before.

`MPI_ADD_ERROR_CODE(errorclass, errorcode)`

IN	<code>errorclass</code>	error class (integer)
OUT	<code>errorcode</code>	new error code to be associated with <code>errorclass</code> (integer)

C binding

```
int MPI_Add_error_code(int errorclass, int *errorcode)
```

Fortran 2008 binding

```
MPI_Add_error_code(errorclass, errorcode, ierror)
  INTEGER, INTENT(IN) :: errorclass
  INTEGER, INTENT(OUT) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ADD_ERROR_CODE(ERRORCLASS, ERRORCODE, IERROR)
  INTEGER ERRORCLASS, ERRORCODE, IERROR
```

Creates new error code associated with `errorclass` and returns its value in `errorcode`.

Rationale. To avoid conflicts with existing error codes and classes, the value of the new error code is set by the implementation and not by the user. (*End of rationale.*)

`MPI_REMOVE_ERROR_CODE(errorcode)`

IN	<code>errorcode</code>	error code to be removed (integer)
----	------------------------	------------------------------------

C binding

```
int MPI_Remove_error_code(int errorcode)
```

Fortran 2008 binding

```
MPI_Remove_error_code(errorcode, ierror)
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REMOVE_ERROR_CODE(ERRORCODE, IERROR)
  INTEGER ERRORCODE, IERROR
```

Removes a user-created error code and all its associations with any error class.

It is erroneous to call `MPI_REMOVE_ERROR_CODE` with a value for `errorcode` that was not added by a call to `MPI_ADD_ERROR_CODE`. Once an `errorcode` is removed by

calling `MPI_REMOVE_ERROR_CODE`, it is erroneous to remove it again without first obtaining the value from another call to `MPI_ADD_ERROR_CODE`. It is erroneous to remove an error code when its associated error string has not been removed before.

`MPI_ADD_ERROR_STRING(errorcode, string)`

IN	errorcode	error code or class (integer)
IN	string	text corresponding to errorcode (string)

C binding

```
int MPI_Add_error_string(int errorcode, const char *string)
```

Fortran 2008 binding

```
MPI_Add_error_string(errorcode, string, ierror)
  INTEGER, INTENT(IN) :: errorcode
  CHARACTER(LEN=*), INTENT(IN) :: string
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ADD_ERROR_STRING(ERRORCODE, STRING, IERROR)
  INTEGER ERRORCODE, IERROR
  CHARACTER*(*) STRING
```

Associates a user-defined error string with an error code or class. The string must be no more than `MPI_MAX_ERROR_STRING` characters long. The length of the string is as defined in the calling language. The length of the string does not include the null terminator in C. Trailing blanks will be stripped in Fortran. Calling `MPI_ADD_ERROR_STRING` for an errorcode that already has a string will replace the old string with the new string. It is erroneous to call `MPI_ADD_ERROR_STRING` for an error code or class with a value $\leq$ `MPI_ERR_LASTCODE`.

If `MPI_ERROR_STRING` is called when no string has been set, it will return a empty string (all spaces in Fortran, "" in C).

`MPI_REMOVE_ERROR_STRING(errorcode)`

IN	errorcode	error code or class (integer)
----	-----------	-------------------------------

C binding

```
int MPI_Remove_error_string(int errorcode)
```

Fortran 2008 binding

```
MPI_Remove_error_string(errorcode, ierror)
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_REMOVE_ERROR_STRING(ERRORCODE, IERROR)
  INTEGER ERRORCODE, IERROR
```

Removes a user-defined association of an error string with an error code or class.

It is erroneous to call `MPI_REMOVE_ERROR_STRING` with a value for `errorcode` that does not have an error string added by a call to `MPI_ADD_ERROR_STRING`.

9.5.2 Calling Error Handlers

Section 9.3 describes the methods for creating and associating error handlers with communicators, files, windows, and sessions. Error handlers can be invoked implicitly when errors are raised during MPI operations, but can also be called by the user.

`MPI_COMM_CALL_ERRHANDLER(comm, errorcode)`

IN	comm	communicator with error handler (handle)
IN	errorcode	error code (integer)

C binding

```
int MPI_Comm_call_errhandler(MPI_Comm comm, int errorcode)
```

Fortran 2008 binding

```
MPI_Comm_call_errhandler(comm, errorcode, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_CALL_ERRHANDLER(COMM, ERRORCODE, IERROR)
  INTEGER COMM, ERRORCODE, IERROR
```

This function invokes the error handler assigned to the communicator with the error code supplied. This function returns `MPI_SUCCESS` in C and the same value in `IERROR` if the error handler was successfully called (assuming the process is not aborted and the error handler returns).

`MPI_WIN_CALL_ERRHANDLER(win, errorcode)`

IN	win	window with error handler (handle)
IN	errorcode	error code (integer)

C binding

```
int MPI_Win_call_errhandler(MPI_Win win, int errorcode)
```

Fortran 2008 binding

```
MPI_Win_call_errhandler(win, errorcode, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_CALL_ERRHANDLER(WIN, ERRORCODE, IERROR)
  INTEGER WIN, ERRORCODE, IERROR
```

This function invokes the error handler assigned to the window with the error code supplied. This function returns `MPI_SUCCESS` in C and the same value in `IERROR` if the error handler was successfully called (assuming the process is not aborted and the error handler returns).

Advice to users. The error handler `MPI_ERRORS_ARE_FATAL` is always associated with a window when it is created. (*End of advice to users.*)

MPI_FILE_CALL_ERRHANDLER(fh, errorcode)

IN	fh	file with error handler (handle)
IN	errorcode	error code (integer)

C binding

```
int MPI_File_call_errhandler(MPI_File fh, int errorcode)
```

Fortran 2008 binding

```
MPI_File_call_errhandler(fh, errorcode, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_CALL_ERRHANDLER(FH, ERRORCODE, IERROR)
  INTEGER FH, ERRORCODE, IERROR
```

This function invokes the error handler assigned to the file with the error code supplied. This function returns `MPI_SUCCESS` in C and the same value in `IERROR` if the error handler was successfully called (assuming the process is not aborted and the error handler returns).

Advice to users. The default error handler for files is `MPI_ERRORS_RETURN`. (*End of advice to users.*)

MPI_SESSION_CALL_ERRHANDLER(session, errorcode)

IN	session	session with error handler (handle)
IN	errorcode	error code (integer)

C binding

```
int MPI_Session_call_errhandler(MPI_Session session, int errorcode)
```

Fortran 2008 binding

```
MPI_Session_call_errhandler(session, errorcode, ierror)
  TYPE(MPI_Session), INTENT(IN) :: session
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```


Fortran binding

```
MPI_SESSION_CALL_ERRHANDLER(SESSION, ERRORCODE, IERROR)
    INTEGER SESSION, ERRORCODE, IERROR
```

This function invokes the error handler assigned to the session with the error code supplied. This function returns `MPI_SUCCESS` in C and the same value in `IERROR` if the error handler was successfully called (assuming the process is not aborted and the error handler returns).

Advice to users. Users are warned that handlers should not be called recursively with `MPI_COMM_CALL_ERRHANDLER`, `MPI_FILE_CALL_ERRHANDLER`, `MPI_WIN_CALL_ERRHANDLER`, or `MPI_SESSION_CALL_ERRHANDLER`. Doing this can create a situation where an infinite recursion is created. This can occur if `MPI_COMM_CALL_ERRHANDLER`, `MPI_FILE_CALL_ERRHANDLER`, `MPI_WIN_CALL_ERRHANDLER`, or `MPI_SESSION_CALL_ERRHANDLER` is called inside an error handler.

Error codes and classes are associated with a process. As a result, they may be used in any error handler. Error handlers should be prepared to deal with any error code they are given. Furthermore, it is good practice to only call an error handler with the appropriate error codes. For example, file errors would normally be sent to the file error handler. (*End of advice to users.*)

9.6 Timers and Synchronization

MPI defines a timer. A timer is specified even though it is not “message-passing,” because timing parallel programs is important in “performance debugging” and because existing timers (both in POSIX 1003.1-1988 and 1003.4D 14.1 and in Fortran 90) are either inconvenient or do not provide adequate access to high resolution timers.

```
MPI_WTIME()
```

C binding

```
double MPI_Wtime(void)
```

Fortran 2008 binding

```
DOUBLE PRECISION MPI_Wtime()
```

Fortran binding

```
DOUBLE PRECISION MPI_WTIME()
```

`MPI_WTIME` returns a floating-point number of seconds, representing elapsed wall-clock time since some time in the past. The “time in the past” is guaranteed not to change during the life of the process. The user is responsible for converting large numbers of seconds to other units if they are preferred. This function is portable (it returns seconds, not “ticks”), and it allows high-resolution. One would use it like this:

Example 9.5.

```
{
    double starttime, endtime;
```

```
1      starttime = MPI_Wtime();
2      ... stuff to be timed ...
3      endtime   = MPI_Wtime();
4      printf("That took %f seconds\n", endtime-starttime);
5  }
```

The times returned are local to the node that called them. There is no requirement that different nodes return “the same time.” (But see also the discussion of `MPI_WTIME_IS_GLOBAL` in Section 9.1.2).

MPI_WTICK()

C binding

```
double MPI_Wtick(void)
```

Fortran 2008 binding

```
DOUBLE PRECISION MPI_Wtick()
```

Fortran binding

```
DOUBLE PRECISION MPI_WTICK()
```

`MPI_WTICK` returns the resolution of `MPI_WTIME` in seconds. That is, it returns, as a double precision value, the number of seconds between successive clock ticks. For example, if the clock is implemented by the hardware as a counter that is incremented every millisecond, the value returned by `MPI_WTICK` should be (10^{-3}) .

Chapter 10

The Info Object

Many of the procedures in MPI take an argument `info`. `info` is an opaque object with a handle of type `MPI_Info` in C and Fortran with the `mpi_f08` module, and `INTEGER` in Fortran with the `mpi` module or the (deprecated) `mpif.h` include file. It stores an unordered set of (key,value) pairs (both key and value are strings). A key can have only one value. MPI reserves several keys and requires that if an implementation uses a reserved key, it must provide the specified functionality. An implementation is not required to support these keys and may support any others not reserved by MPI.

Some info hints allow the MPI library to restrict its support for certain operations in order to improve performance or resource utilization. If an application provides such an info hint, it must be compatible with any changes in the behavior of the MPI library that are allowed by the info hint.

An implementation must support info objects as caches for arbitrary (key,value) pairs, regardless of whether it recognizes the key. Each procedure that takes hints in the form of an `MPI_Info` must be prepared to ignore any key it does not recognize. This description of info objects does not attempt to define how a particular procedure should react if it recognizes a key but not the associated value. `MPI_INFO_GET_NKEYS`, `MPI_INFO_GET_NTHKEY`, and `MPI_INFO_GET_STRING` must retain all (key,value) pairs so that layered functionality can also use the Info object.

Keys have an implementation-defined maximum length of `MPI_MAX_INFO_KEY-1`, where `MPI_MAX_INFO_KEY` is at least 33 and at most 256. Values have an implementation-defined maximum length of `MPI_MAX_INFO_VAL`. In Fortran, leading and trailing spaces are stripped from both. Returned values will never be larger than these maximum lengths. Both key and value are case sensitive.

Rationale. Keys have a maximum length because the set of known keys will always be finite and known to the implementation and because there is no reason for keys to be complex. The small maximum size allows applications to declare keys of size `MPI_MAX_INFO_KEY`. The limitation on value sizes is so that an implementation is not forced to deal with arbitrarily long strings. (*End of rationale.*)

Advice to users. `MPI_MAX_INFO_VAL` might be very large, so it might not be wise to declare a string of that size. (*End of advice to users.*)

When `info` is used as an argument to any MPI procedure, it is interpreted before that procedure returns, so that it may be read, modified, or freed immediately after return. Changes to an info object after return from a procedure do not affect that interpretation.

Rationale. Prior to MPI-4.0, the above statement was restricted to nonblocking MPI procedures. For simplicity this restriction was removed, as it currently applies to

all MPI procedures that use info arguments. Note, this has to be revisited for new procedures added in the future, e.g., for future procedures that could return an info argument to be filled in after the return from the procedure. (*End of rationale.*)

When the descriptions refer to a key or value as being a boolean, an integer, or a list, they mean the string representation of these types. An implementation may define its own rules for how info value strings are converted to other types, but to ensure portability, every implementation must support the following representations. Valid values for a boolean must include the strings "true" and "false" (all lowercase). For integers, valid values must include string representations of decimal values of integers that are within the range of a standard integer type in the program. (However it is possible that not every integer is a valid value for a given key.) On positive numbers, + signs are optional. No space may appear between a + or - sign and the leading digit of a number. For comma separated lists, the string must contain valid elements separated by commas. Leading and trailing spaces are stripped automatically from the types of info values described above and for each element of a comma separated list. These rules apply to all info values of these types. Implementations are free to specify a different interpretation for values of other info keys.

MPI_INFO_CREATE(info)

OUT	info	info object created (handle)
-----	------	------------------------------

C binding

```
int MPI_Info_create(MPI_Info *info)
```

Fortran 2008 binding

```
MPI_Info_create(info, ierror)
  TYPE(MPI_Info), INTENT(OUT) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INFO_CREATE(INFO, IERROR)
  INTEGER INFO, IERROR
```

MPI_INFO_CREATE creates a new info object. The newly created object contains no key/value pairs.

MPI_INFO_SET(info, key, value)

INOUT	info	info object (handle)
IN	key	key (string)
IN	value	value (string)

C binding

```
int MPI_Info_set(MPI_Info info, const char *key, const char *value)
```

Fortran 2008 binding

```
MPI_Info_set(info, key, value, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
```

```

    CHARACTER(LEN=*), INTENT(IN) :: key, value
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_INFO_SET(INFO, KEY, VALUE, IERROR)
    INTEGER INFO, IERROR
    CHARACTER*(*) KEY, VALUE

```

MPI_INFO_SET adds the (key,value) pair to *info*, and overrides the value if a value for the same key was previously set. *key* and *value* are null-terminated strings in C. In Fortran, leading and trailing spaces in *key* and *value* are stripped. If either *key* or *value* are longer than the respective maximum length, the call raises an error of class `MPI_ERR_INFO_KEY` or `MPI_ERR_INFO_VALUE`, respectively.

```

MPI_INFO_DELETE(info, key)

```

INOUT	info	info object (handle)
IN	key	key (string)

C binding

```

int MPI_Info_delete(MPI_Info info, const char *key)

```

Fortran 2008 binding

```

MPI_Info_delete(info, key, ierror)
    TYPE(MPI_Info), INTENT(IN) :: info
    CHARACTER(LEN=*), INTENT(IN) :: key
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_INFO_DELETE(INFO, KEY, IERROR)
    INTEGER INFO, IERROR
    CHARACTER*(*) KEY

```

MPI_INFO_DELETE deletes a (key,value) pair from *info*. If *key* is not defined in *info*, the call raises an error of class `MPI_ERR_INFO_NOKEY`.

```

MPI_INFO_GET_STRING(info, key, buflen, value, flag)

```

IN	info	info object (handle)
IN	key	key (string)
INOUT	buflen	length of buffer (integer)
OUT	value	value (string)
OUT	flag	true if key is defined, false otherwise (logical)

C binding

```

int MPI_Info_get_string(MPI_Info info, const char *key, int *buflen,
    char *value, int *flag)

```

Fortran 2008 binding

```

MPI_Info_get_string(info, key, buflen, value, flag, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  CHARACTER(LEN=*), INTENT(IN) :: key
  INTEGER, INTENT(INOUT) :: buflen
  CHARACTER(LEN=*), INTENT(OUT) :: value
  LOGICAL, INTENT(OUT) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_INFO_GET_STRING(INFO, KEY, BUFLen, VALUE, FLAG, IERROR)
  INTEGER INFO, BUFLen, IERROR
  CHARACTER*(*) KEY, VALUE
  LOGICAL FLAG

```

This procedure retrieves the value associated with `key` from `info`, if any. If such a key exists in `info`, it sets `flag` to `true` and returns the value in `value`, otherwise it sets `flag` to `false` and leaves `value` unchanged. `buflen` on input is the size of the provided buffer, `value`, for the output of `buflen` it is the size of the buffer needed to store the value string. If the `buflen` passed into the procedure is less than the actual size needed to store the value string (including null terminator in C), the value is truncated. On return, the value of `buflen` will be set to the required buffer size to hold the value string. If `buflen` is set to 0, `value` is not changed. In C, `buflen` includes the required space for the null terminator. In C, this procedure returns a null terminated string in all cases where the `buflen` input value is greater than 0.

If `key` is larger than `MPI_MAX_INFO_KEY`, the call is erroneous.

Advice to users. The `MPI_INFO_GET_STRING` procedure can be used to obtain the size of the required buffer for a value string by setting the `buflen` to 0. The returned `buflen` can then be used to allocate memory before calling `MPI_INFO_GET_STRING` again to obtain the value string. (*End of advice to users.*)

```

MPI_INFO_GET_NKEYS(info, nkeys)

```

IN	info	info object (handle)
OUT	nkeys	number of defined keys (integer)

C binding

```

int MPI_Info_get_nkeys(MPI_Info info, int *nkeys)

```

Fortran 2008 binding

```

MPI_Info_get_nkeys(info, nkeys, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, INTENT(OUT) :: nkeys
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_INFO_GET_NKEYS(INFO, NKEYS, IERROR)
  INTEGER INFO, NKEYS, IERROR

```

[MPI_INFO_GET_NKEYS](#) returns the number of currently defined keys in `info`.

`MPI_INFO_GET_NTHKEY(info, n, key)`

IN	<code>info</code>	info object (handle)
IN	<code>n</code>	key number (integer)
OUT	<code>key</code>	key (string)

C binding

```
int MPI_Info_get_nthkey(MPI_Info info, int n, char *key)
```

Fortran 2008 binding

```
MPI_Info_get_nthkey(info, n, key, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, INTENT(IN) :: n
  CHARACTER(LEN=*), INTENT(OUT) :: key
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INFO_GET_NTHKEY(INFO, N, KEY, IERROR)
  INTEGER INFO, N, IERROR
  CHARACTER*(*) KEY
```

This procedure returns the n th defined key in `info`. Keys are numbered $0 \dots N - 1$ where N is the value returned by [MPI_INFO_GET_NKEYS](#). All keys between 0 and $N - 1$ are guaranteed to be defined. The number of a given key does not change as long as `info` is not modified with [MPI_INFO_SET](#) or [MPI_INFO_DELETE](#).

`MPI_INFO_DUP(info, newinfo)`

IN	<code>info</code>	info object (handle)
OUT	<code>newinfo</code>	info object created (handle)

C binding

```
int MPI_Info_dup(MPI_Info info, MPI_Info *newinfo)
```

Fortran 2008 binding

```
MPI_Info_dup(info, newinfo, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Info), INTENT(OUT) :: newinfo
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INFO_DUP(INFO, NEWINFO, IERROR)
  INTEGER INFO, NEWINFO, IERROR
```

[MPI_INFO_DUP](#) duplicates an existing `info` object, creating a new object, with the same (key,value) pairs and the same ordering of keys.

```

1 MPI_INFO_FREE(info)
2     INOUT    info                      info object (handle)
3
4

```

C binding

```

5 int MPI_Info_free(MPI_Info *info)
6

```

Fortran 2008 binding

```

7 MPI_Info_free(info, ierror)
8     TYPE(MPI_Info), INTENT(INOUT) :: info
9     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11

```

Fortran binding

```

12 MPI_INFO_FREE(INFO, IERROR)
13     INTEGER INFO, IERROR
14

```

This procedure frees info and sets it to MPI_INFO_NULL.

```

15
16
17 MPI_INFO_CREATE_ENV(info)
18

```

```

19     OUT      info                      info object (handle)
20
21

```

C binding

```

22 int MPI_Info_create_env(int argc, char *argv[], MPI_Info *info)
23

```

Fortran 2008 binding

```

24 MPI_Info_create_env(info, ierror)
25     TYPE(MPI_Info), INTENT(OUT) :: info
26     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
27
28

```

Fortran binding

```

29 MPI_INFO_CREATE_ENV(INFO, IERROR)
30     INTEGER INFO, IERROR
31

```

This procedure creates an output object info with the same construction as [MPI_INFO_ENV](#) as created during [MPI_INIT](#) or [MPI_INIT_THREAD](#) when the same arguments are used. This construction is described in Section 11.2.1; however, this procedure can be called when not using the World Model, e.g., when using the Sessions Model. This object is not a direct copy or alias of the [MPI_INFO_ENV](#) object and could contain different values based on the input arguments and other sources. Multiple calls to this procedure that are given the same input arguments will produce info objects consistent with the definition of [MPI_INFO_ENV](#). The version for ISO C accepts the `argc` and `argv` that are provided by the arguments to `main` or 0 for `argc` and NULL for `argv`. The user is responsible for freeing the info object via [MPI_INFO_FREE](#). This procedure is local.

This procedure must always be thread-safe, as defined in Section 11.6. It is one of the few procedures that may be called before MPI is initialized or after MPI is finalized.

Advice to users.

In some circumstances (e.g., when passing 0 to `argc` and NULL to `argv` in C or in Fortran where such arguments do not exist), the info object may not be populated or may be populated incompletely because this procedure is local and the implementation may

not be able to determine the correct values. Note that this could result in different values in the resulting info object at different MPI processes.

(End of advice to users.)

DRAFT

DRAFT

Chapter 11

Process Initialization, Creation, and Management

11.1 Introduction

MPI is primarily concerned with communication rather than process or resource management. However, it is necessary to address these issues to some degree in order to define a useful framework for communication. This chapter presents a set of MPI interfaces that allows for several approaches to MPI initialization and process management while placing minimal restrictions on the execution environment.

One goal of MPI is to achieve *source code portability*. By this we mean that a program written using MPI and complying with the relevant language standards is portable as written, and must not require any source code changes when moved from one system to another. This explicitly does *not* say anything about how an MPI program is started or launched from the command line, nor what the user must do to set up the environment in which an MPI program will run. However, an implementation may require some setup or initialization procedure to be performed before the complete set of MPI routines may be called.

To this end, MPI presents two models for **MPI process initialization**. In the World Model, an initial set of processes is created that are related by their membership in a common `MPI_COMM_WORLD` (see Section 11.2) communicator. In the Sessions Model (Section 11.3), an initial set of processes is also created, but the application must explicitly manage the creation of MPI groups, and hence MPI communicators. `MPI_COMM_WORLD` is only valid for use as a communicator in the World Model, i.e., after a successful call to `MPI_INIT` or `MPI_INIT_THREAD` and before a call to `MPI_FINALIZE`. An application can employ both of these Process Models concurrently. In multi-component MPI applications, for example, a component such as a library can make use of the Sessions Model to instantiate MPI resources without impacting the rest of the application.

The Dynamic Process Model (see Section 11.7), provides for the creation and management of additional processes after an MPI application has been started. A major impetus for the Dynamic Process Model comes from the PVM [26] research effort. This work has provided a wealth of experience with process management and resource control that illustrates their benefits and potential pitfalls.

In developing the Dynamic Process Model, the MPI Forum decided not to address resource control because it was not able to design a portable interface that would be appropriate for the broad spectrum of existing and potential resource and process controllers. MPI assumes that resource control is provided externally.

Process management functionality is included in MPI to enable its use in classes of message-passing applications requiring process control. These include task farms, serial

applications with parallel modules, and problems that require a run-time assessment of the number and type of processes that should be started.

The following goals are central to the design of MPI process management:

- The MPI process model must apply to the vast majority of current parallel environments.
- MPI must not take over operating system responsibilities. It should instead provide a clean interface between an application and system software.
- MPI must guarantee communication determinism in the presence of dynamic processes, i.e., dynamic process management must not introduce unavoidable race conditions.
- MPI must not contain features that compromise performance.

The Dynamic Process Model addresses these issues in two ways. First, MPI remains primarily a communication library. It does not manage the parallel environment in which a parallel program executes, though it provides a minimal interface between an application and external resource and process managers.

Second, MPI maintains a consistent concept of a communicator, regardless of how its members came into existence. A communicator is never changed once created, and it is always created using deterministic collective operations.

11.2 The World Model

11.2.1 Starting MPI Processes

When using the World Model, MPI is initialized by calling either `MPI_INIT` or `MPI_INIT_THREAD`.

`MPI_INIT()`

C binding

```
int MPI_Init(int *argc, char ***argv)
```

Fortran 2008 binding

```
MPI_Init(ierr)
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierr
```

Fortran binding

```
MPI_INIT(IERROR)
```

```
INTEGER IERROR
```

In the World Model, an MPI program must contain exactly one call to an MPI initialization routine: `MPI_INIT` or `MPI_INIT_THREAD`. `MPI_COMM_WORLD` and `MPI_COMM_SELF` are not valid for use as communicators prior to invocation of `MPI_INIT` or `MPI_INIT_THREAD`. Subsequent calls to either of these initialization routines are erroneous. A subset of MPI functions may be invoked before MPI initialization routines are called, see Section 11.4.1. The procedures `MPI_INIT` and `MPI_INIT_THREAD` accept either the `argc` and `argv` that are provided by the arguments to `main` or `NULL`.

Example 11.1. Initializing MPI using `MPI_INIT`

```

1  int main(int argc, char *argv[])
2  {
3      MPI_Init(&argc, &argv);
4
5      /* parse arguments */
6      /* main program */
7
8      MPI_Finalize();    /* see below */
9      return 0;
10 }

```

The Fortran version takes only `IERROR`.

Conforming implementations of MPI are required to allow applications to pass `NULL` for both the `argc` and `argv` arguments of `main` in C.

Failures may disrupt the execution of the program before or during MPI initialization. A high-quality implementation shall not deadlock during MPI initialization, even in the presence of failures. Except for functions with the `MPI_T_` prefix, failures in MPI operations prior to or during MPI initialization are reported by invoking the initial error handler. Users can use the "mpi_initial_errhandler" info key during the launch of MPI processes (e.g., `MPI_COMM_SPAWN`, `MPI_COMM_SPAWN_MULTIPLE`, or `mpiexec`) to set a nonfatal initial error handler before MPI initialization. When the initial error handler is set to `MPI_ERRORS_ABORT`, raising an error before or during initialization aborts the local MPI process (i.e., it is similar to calling `MPI_ABORT` on `MPI_COMM_SELF`). An implementation may not always be capable of determining, before MPI initialization, what constitutes the local MPI process, or the set of connected processes. In this case, errors before initialization may cause a different set of MPI processes to abort than specified. During MPI initialization, the initial error handler is associated with `MPI_COMM_WORLD`, `MPI_COMM_SELF`, and the communicator returned by `MPI_COMM_GET_PARENT` (if any).

Advice to implementors. Some failures may leave MPI in an undefined state, or raise an error before the error handling capabilities are fully operational, in which cases the implementation may be incapable of providing the desired error handling behavior. Of note, in some implementations, the notion of an MPI process is not clearly established in the early stages of MPI initialization (for example, when the implementation considers threads that called `MPI_INIT` as independent MPI processes); in this case, before MPI is initialized, the `MPI_ERRORS_ABORT` error handler may abort what would have become multiple MPI processes.

When a failure occurs during MPI initialization, the implementation may decide to return `MPI_SUCCESS` from the MPI initialization function instead of raising an error. It is recommended that an implementation masks an initialization error only when it expects that later MPI calls will result in well-specified behavior (i.e., barring additional failures, either the outcome of any call will be correct, or the call will raise an appropriate error). For example, it may be difficult for an implementation to avoid unspecified behavior when the group of `MPI_COMM_WORLD` does not contain the same set of MPI processes at all members of the communicator, or if the communicator returned from `MPI_COMM_GET_PARENT` was not initialized correctly. (*End of advice to implementors.*)

After MPI is initialized, the application can access information about the execution environment by querying the predefined info object `MPI_INFO_ENV`. The following keys are predefined for this object, corresponding to the arguments of `MPI_COMM_SPAWN` or of `mpiexec`:

"command": Name of program executed.

"argv": Space separated arguments to command.

"maxprocs": Maximum number of MPI processes to start.

"mpi_initial_errhandler": Name of the initial errhandler.

"mpi_memory_alloc_kinds": Requested memory allocation kinds (see Section 11.4.3).

"soft": Allowed values for number of processors.

"host": Hostname.

"arch": Architecture name.

"wdir": Working directory of the MPI process.

"file": Value is the name of a file in which additional information is specified.

"thread_level": Requested level of thread support, if requested before the program started execution.

Note that all values are strings. Thus, the maximum number of processes is represented by a string such as "1024" and the requested level is represented by a string such as "MPI_THREAD_SINGLE".

Advice to users. If one of the "argv" arguments contains a space, there is no way to tell from the value of the "argv" info key whether a space is part of the argument or is separating different arguments. (*End of advice to users.*)

The info object `MPI_INFO_ENV` need not contain a (key,value) pair for each of these predefined keys; the set of (key,value) pairs provided is implementation-dependent. Implementations may provide additional, implementation specific, (key,value) pairs.

In cases where the MPI processes were started with `MPI_COMM_SPAWN_MULTIPLE` or, equivalently, with a startup mechanism that supports multiple process specifications, then the values stored in the info object `MPI_INFO_ENV` at a process are those values that affect the local MPI process.

Example 11.2. If MPI is started with a call to

```
mpiexec -n 5 -arch x86_64 ocean : -n 10 -arch power9 atmos
```

Then the first 5 processes will have in their `MPI_INFO_ENV` object the pairs (command, ocean), (maxprocs, 5), and (arch, x86_64). The next 10 processes will have in `MPI_INFO_ENV` (command, atmos), (maxprocs, 10), and (arch, power9).

Advice to users. The values passed in `MPI_INFO_ENV` are the values of the arguments passed to the mechanism that started the MPI execution—not the actual value provided. Thus, the value associated with "maxprocs" is the number of MPI processes requested; it can be larger than the actual number of processes obtained, if the `soft` option was used. (*End of advice to users.*)

Advice to implementors. High-quality implementations will provide a (key,value) pair for each parameter that can be passed to the command that starts an MPI program. (*End of advice to implementors.*)

The following function may be used to initialize MPI, and to initialize the MPI thread environment, instead of `MPI_INIT`.

`MPI_INIT_THREAD(required, provided)`

IN	required	desired level of thread support (integer)
OUT	provided	provided level of thread support (integer)

C binding

`int MPI_Init_thread(int *argc, char ***argv, int required, int *provided)`

Fortran 2008 binding

`MPI_Init_thread(required, provided, ierror)`
 INTEGER, INTENT(IN) :: required
 INTEGER, INTENT(OUT) :: provided
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

`MPI_INIT_THREAD(REQUIRED, PROVIDED, IERROR)`
 INTEGER REQUIRED, PROVIDED, IERROR

This call initializes MPI in the same way that a call to `MPI_INIT` would. In addition, it initializes the thread environment. The argument `required` is used to specify the desired level of thread support. The possible values are listed in increasing order of thread support.

MPI_THREAD_SINGLE: Only one thread will execute.

MPI_THREAD_FUNNELED: The process may be multithreaded, but the application must ensure that only the main thread makes MPI calls (for the definition of main thread, see `MPI_IS_THREAD_MAIN` on page 485).

MPI_THREAD_SERIALIZED: The process may be multithreaded, and multiple threads may make MPI calls, but only one at a time: MPI calls are not made concurrently from two distinct threads (all MPI calls are "serialized").

MPI_THREAD_MULTIPLE: Multiple threads may call MPI, with no restrictions.

These values are monotonic; i.e., `MPI_THREAD_SINGLE < MPI_THREAD_FUNNELED < MPI_THREAD_SERIALIZED < MPI_THREAD_MULTIPLE`.

Different processes in `MPI_COMM_WORLD` may require different levels of thread support.

The call returns in `provided` information about the actual level of thread support that will be provided by MPI. It can be one of the four values listed above.

The level(s) of thread support that can be provided by `MPI_INIT_THREAD` will depend on the implementation, and may depend on information provided by the user before the program started to execute (e.g., with arguments to `mpiexec`). If possible, the call will return `provided = required`. Failing this, the call will return the least supported level such that `provided > required` (thus providing a stronger level of support than required by the user). Finally, if the user requirement cannot be satisfied, then the call will return in `provided` the highest supported level.

A **thread compliant** MPI implementation will be able to return `provided = MPI_THREAD_MULTIPLE`. Such an implementation may always return `provided = MPI_THREAD_MULTIPLE`, irrespective of the value of `required`.

An MPI library that is not thread compliant must always return `provided = MPI_THREAD_SINGLE`, even if `MPI_INIT_THREAD` is called on a multithreaded process. The library should also return correct values for the MPI calls that can be executed before initialization, even if multiple threads have been spawned.

Rationale. Such code is erroneous, but if the MPI initialization is performed by a library, the error cannot be detected until `MPI_INIT_THREAD` is called. The requirements in the previous paragraph ensure that the error can be properly detected. (*End of rationale.*)

A call to `MPI_INIT` has the same effect as a call to `MPI_INIT_THREAD` with a `required = MPI_THREAD_SINGLE`.

Vendors may provide (implementation dependent) means to specify the level(s) of thread support available when the MPI program is started, e.g., with arguments to `mpiexec`. This will affect the outcome of calls to `MPI_INIT` and `MPI_INIT_THREAD`. Suppose, for example, that an MPI program has been started so that only `MPI_THREAD_MULTIPLE` is available. Then `MPI_INIT_THREAD` will return `provided = MPI_THREAD_MULTIPLE`, irrespective of the value of `required`; a call to `MPI_INIT` will also initialize the MPI thread support level to `MPI_THREAD_MULTIPLE`. Suppose, instead, that an MPI program has been started so that all four levels of thread support are available. Then, a call to `MPI_INIT_THREAD` will return `provided = required`; alternatively, a call to `MPI_INIT` will initialize the MPI thread support level to `MPI_THREAD_SINGLE`.

Rationale. Various optimizations are possible when MPI code is executed single-threaded, or is executed on multiple threads, but not concurrently: mutual exclusion code may be omitted. Furthermore, if only one thread executes, then the MPI library can use library functions that are not thread safe, without risking conflicts with user threads. Also, the model of one communication thread, multiple computation threads fits many applications well, e.g., if the process code is a sequential Fortran/C program with MPI calls that has been parallelized by a compiler for execution on an SMP node, in a cluster of SMPs, then the process computation is multithreaded, but MPI calls will likely execute on a single thread.

The design accommodates a static specification of the thread support level, for environments that require static binding of libraries, and for compatibility for current multithreaded MPI codes. (*End of rationale.*)

Advice to implementors. If `provided` is not `MPI_THREAD_SINGLE` then the MPI library should not invoke C or Fortran library calls that are not thread safe, e.g., in an environment where `malloc` is not thread safe, then `malloc` should not be used by the MPI library.

Some implementors may want to use different MPI libraries for different levels of thread support. They can do so using dynamic linking and selecting which library will be linked when `MPI_INIT_THREAD` is invoked. If this is not possible, then optimizations for lower levels of thread support will occur only when the level of thread support required is specified at link time.

Note that `required` need not be the same value on all processes of `MPI_COMM_WORLD`. (*End of advice to implementors.*)

As with `MPI_INIT`, discussed in Section 11.2.1, the version for ISO C accepts the `argc` and `argv` that are provided by the arguments to `main` or `NULL` for both arguments.

The following function can be used to query the current level of thread support.

`MPI_QUERY_THREAD(provided)`

OUT `provided` provided level of thread support (integer)

C binding

```
int MPI_Query_thread(int *provided)
```

Fortran 2008 binding

```
MPI_Query_thread(provided, ierror)
  INTEGER, INTENT(OUT) :: provided
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_QUERY_THREAD(PROVIDED, IERROR)
  INTEGER PROVIDED, IERROR
```

The call returns in `provided` the current level of thread support, which will be the value returned in `provided` by `MPI_INIT_THREAD`, if MPI was initialized by a call to `MPI_INIT_THREAD`. This function is only applicable when using the World Model to initialize MPI. In the case of applications using both the World Model and the Sessions Model, this function only returns the thread support level returned in `provided` by `MPI_INIT_THREAD`.

`MPI_IS_THREAD_MAIN(flag)`

OUT `flag` true if calling thread is main thread, false otherwise (logical)

C binding

```
int MPI_Is_thread_main(int *flag)
```

Fortran 2008 binding

```

MPI_Is_thread_main(flag, ierror)
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_IS_THREAD_MAIN(FLAG, IERROR)
    LOGICAL FLAG
    INTEGER IERROR

```

This function can be called by a thread to determine if it is the main thread (the thread that called `MPI_INIT` or `MPI_INIT_THREAD`). This function is only applicable when using the World Model to initialize MPI. In the case of applications using both the World Model and the Sessions Model, the behavior of this procedure is the same as if the application were only using the World Model.

All routines listed in this section must be supported by all MPI implementations.

Rationale. MPI libraries are required to provide these calls even if they do not support threads, so that portable code that contains invocations to these functions can link correctly. `MPI_INIT` continues to be supported so as to provide compatibility with current MPI codes. (*End of rationale.*)

Advice to users. It is possible to spawn threads before MPI is initialized, but `MPI_COMM_WORLD` and `MPI_COMM_SELF` cannot be used until the World Model is active, i.e., until `MPI_INIT_THREAD` is invoked by one thread (which, thereby, becomes the main thread). In particular, it is possible to enter the MPI execution with a multithreaded process.

In the World Model, the level of thread support provided is a global property of the MPI process that can be specified only once, when MPI is initialized on that process (or before). Portable third party libraries have to be written so as to accommodate any provided level of thread support. Otherwise, their usage will be restricted to specific level(s) of thread support. If such a library can run only with specific level(s) of thread support, e.g., only with `MPI_THREAD_MULTIPLE`, then `MPI_QUERY_THREAD` can be used to check whether the user initialized MPI to the correct level of thread support. (*End of advice to users.*)

11.2.2 Finalizing MPI

```

MPI_FINALIZE()

```

C binding

```

int MPI_Finalize(void)

```

Fortran 2008 binding

```

MPI_Finalize(ierror)
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FINALIZE(IERROR)
    INTEGER IERROR

```

This routine cleans up all MPI state associated with the World Model. If an MPI program that initializes the World Model terminates normally (i.e., not due to a call to `MPI_ABORT` or an unrecoverable error) then each process must call `MPI_FINALIZE` before it exits.

Before an MPI process invokes `MPI_FINALIZE`, the process must perform all MPI calls needed to complete its involvement in MPI communications associated with the World Model. It must locally complete all MPI operations that it initiated and must execute matching calls needed to complete MPI communications initiated by other processes. For example, if the process executed a nonblocking send, it must eventually call `MPI_WAIT`, `MPI_TEST`, `MPI_REQUEST_FREE`, or any derived function; if the process is the target of a send, then it must post the matching receive; if it is part of a group executing a collective operation, then it must have completed its participation in the operation. This means that before calling `MPI_FINALIZE`, all message handles associated with the World Model must be received (with `MPI_MRECV` or derived procedures) and all request handles associated with the World Model must be freed in the case of nonblocking operations, and must be inactive or freed in the case of persistent or partitioned operations (i.e., by calling one of the procedures `MPI_{TEST|WAIT}{|ANY|SOME|ALL}` or `MPI_REQUEST_FREE`).

The call to `MPI_FINALIZE` does not clean up MPI state associated with objects created using `MPI_SESSION_INIT` and other Sessions Model methods, nor objects created using the communicator returned by `MPI_COMM_GET_PARENT`. See Sections 11.3 and 11.8.

The call to `MPI_FINALIZE` does not free objects created by MPI calls; these objects are freed using `MPI_XXX_FREE`, `MPI_COMM_DISCONNECT`, or `MPI_FILE_CLOSE` calls.

Once `MPI_FINALIZE` returns, no MPI procedure may be called in the World Model (not even `MPI_INIT`, or freeing objects created within the World Model), except for those listed in Section 11.4.1.

`MPI_FINALIZE` is collective over all connected processes. If no processes were spawned, accepted or connected then this means over `MPI_COMM_WORLD`; otherwise it is collective over the union of all processes that have been and continue to be connected, as explained in Section 11.10.4.

The following examples illustrate these rules.

Example 11.3. The following code is correct

Process 0	Process 1
<code>MPI_Init();</code>	<code>MPI_Init();</code>
<code>MPI_Send(dest=1);</code>	<code>MPI_Recv(src=0);</code>
<code>MPI_Finalize();</code>	<code>MPI_Finalize();</code>

Example 11.4. Without a matching receive, the program is erroneous

Process 0	Process 1
<code>MPI_Init();</code>	<code>MPI_Init();</code>
<code>MPI_Send(dest=1);</code>	
<code>MPI_Finalize();</code>	<code>MPI_Finalize();</code>

Example 11.5. This program is correct: Process 0 calls `MPI_Finalize` after it has executed the MPI calls that complete the send operation. Likewise, process 1 executes the MPI call

that completes the matching receive operation before it calls `MPI_Finalize`.

Process 0	Process 1
<code>MPI_Init();</code>	<code>MPI_Init();</code>
<code>MPI_Isend(dest=1);</code>	<code>MPI_Recv(src=0);</code>
<code>MPI_Request_free();</code>	<code>MPI_Finalize();</code>
<code>MPI_Finalize();</code>	<code>exit();</code>
<code>exit();</code>	

Example 11.6. This program is correct. The attached buffer is a resource allocated by the user, not by MPI; it is available to the user after MPI is finalized.

Process 0	Process 1
<code>MPI_Init();</code>	<code>MPI_Init();</code>
<code>buffer = malloc(1000000);</code>	<code>MPI_Recv(src=0);</code>
<code>MPI_Buffer_attach();</code>	<code>MPI_Finalize();</code>
<code>MPI_Send(dest=1);</code>	<code>exit();</code>
<code>MPI_Finalize();</code>	
<code>free(buffer);</code>	
<code>exit();</code>	

Example 11.7. This program is correct. The cancel operation must succeed, since the send cannot complete normally. The wait operation, after the call to `MPI_Cancel`, is local—no matching MPI call is required on process 1. Cancelling a send request by calling `MPI_CANCEL` is deprecated.

Process 0	Process 1
<code>MPI_Issend(dest=1);</code>	<code>MPI_Finalize();</code>
<code>MPI_Cancel();</code>	
<code>MPI_Wait();</code>	
<code>MPI_Finalize();</code>	

Advice to implementors. Even though a process has executed all MPI calls needed to complete the communications it is involved with, such communication may not yet be completed from the viewpoint of the underlying MPI system. For example, a blocking send may have returned, even though the data is still buffered at the sender in an MPI buffer; an MPI process may receive a cancel request for a message it has completed receiving. The MPI implementation must ensure that a process has completed any involvement in MPI communication before `MPI_FINALIZE` returns. Thus, if a process exits after the call to `MPI_FINALIZE`, this will not cause an ongoing communication to fail. The MPI implementation should also complete freeing all objects marked for deletion by MPI calls that freed them. See also Section 2.9 on *progress*. (*End of advice to implementors.*)

Failures may disrupt MPI operations during and after MPI finalization. A high-quality implementation shall not deadlock in MPI finalization, even in the presence of failures. The normal rules for MPI error handling continue to apply. After `MPI_COMM_SELF` has been “freed” (see Section 11.2.4), errors that are not associated with a communicator, window, or file raise the initial error handler (set during the launch operation, see Section 11.8.4).

Although it is not required that all processes return from `MPI_FINALIZE`, it is required that, when it has not failed or aborted, at least the MPI process that was assigned rank 0 in `MPI_COMM_WORLD` returns, so that users can know that the MPI portion of the computation is over. In addition, in a POSIX environment, users may desire to supply an exit code for each process that returns from `MPI_FINALIZE`.

Note that a failure may terminate the MPI process that was assigned rank 0 in `MPI_COMM_WORLD`, in which case it is possible that no MPI process returns from `MPI_FINALIZE`.

Advice to users. Applications that handle errors are encouraged to implement all rank-specific code before the call to `MPI_FINALIZE`. In Example 11.8, the process with rank 0 in `MPI_COMM_WORLD` may have been terminated before, during, or after the call to `MPI_FINALIZE`, possibly leading to the code after `MPI_FINALIZE` never being executed. (*End of advice to users.*)

Example 11.8. The following illustrates the use of requiring that at least one process return and that it be known that process 0 is one of the processes that return. One wants code like the following to work no matter how many processes return.

```
...
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
...
MPI_Finalize();
if (myrank == 0) {
    resultfile = fopen("outfile", "w");
    dump_results(resultfile);
    fclose(resultfile);
}
exit(0);
```

11.2.3 Determining Whether MPI Has Been Initialized When Using the World Model

One of the goals of MPI is to allow for layered libraries. A library using the World Model needs to know if MPI has been initialized using either `MPI_INIT` or `MPI_INIT_THREAD`. In MPI the function `MPI_INITIALIZED` is provided to tell if MPI had been initialized using the World Model. In the World Model, once MPI has been finalized it cannot be restarted. A library needs to be able to determine this to act accordingly. To achieve this, the function `MPI_FINALIZED` is needed.

`MPI_INITIALIZED(flag)`

OUT flag

Flag is true if `MPI_INIT` or `MPI_INIT_THREAD` has been called and false otherwise (logical)

C binding

`int MPI_Initialized(int *flag)`

Fortran 2008 binding

```

MPI_Initialized(flag, ierror)
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_INITIALIZED(FLAG, IERROR)
    LOGICAL FLAG
    INTEGER IERROR

```

This routine may be used to determine whether `MPI_INIT` or `MPI_INIT_THREAD` has been called. `MPI_INITIALIZED` returns true if the calling process has called either of these MPI procedures. It is valid to call `MPI_INITIALIZED` before `MPI_INIT` or `MPI_INIT_THREAD` and after `MPI_FINALIZE`. Whether `MPI_FINALIZE` has been called does not affect the behavior of `MPI_INITIALIZED`. This function must always be thread-safe, as defined in Section 11.6. This function returns false for applications using the Sessions Model exclusively.

MPI_FINALIZED(flag)

```

OUT      flag

```

true if `MPI_FINALIZE` has been called and false otherwise. (logical)

C binding

```

int MPI_Finalized(int *flag)

```

Fortran 2008 binding

```

MPI_Finalized(flag, ierror)
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FINALIZED(FLAG, IERROR)
    LOGICAL FLAG
    INTEGER IERROR

```

This routine returns true if `MPI_FINALIZE` has completed. It is valid to call `MPI_FINALIZED` before `MPI_INIT` or `MPI_INIT_THREAD` and after `MPI_FINALIZE`. This function must always be thread-safe, as defined in Section 11.6.

11.2.4 Allowing User Functions at MPI Finalization

In the context of the World Model, there are times in which it would be convenient to have actions happen when an MPI process finalizes MPI. For example, a routine may do initializations that are useful until the MPI job (or that part of the job that is being terminated in the case of dynamically created processes) finalizes MPI. This can be accomplished in MPI by attaching an attribute to `MPI_COMM_SELF` with a callback function. When `MPI_FINALIZE` is called, it will first execute the equivalent of an `MPI_COMM_FREE` on `MPI_COMM_SELF`. This will cause the delete callback function to be executed on all keys associated with `MPI_COMM_SELF`, in the reverse order that they were set on `MPI_COMM_SELF`. If no key has been attached to `MPI_COMM_SELF`, then no callback is invoked. The “freeing” of

`MPI_COMM_SELF` occurs before any other parts of MPI are affected. Thus, for example, calling `MPI_FINALIZED` will return `false` in any of these callback functions. Once done with `MPI_COMM_SELF`, the order and rest of the actions taken by `MPI_FINALIZE` is not specified.

Advice to implementors. Since attributes can be added from any supported language, the MPI implementation needs to remember the creating language so the correct callback is made. Implementations that use the attribute delete callback on `MPI_COMM_SELF` internally should register their internal callbacks before returning from `MPI_INIT` or `MPI_INIT_THREAD`, so that libraries or applications will not have portions of the MPI implementation shut down before the application-level callbacks are made. (*End of advice to implementors.*)

11.3 The Sessions Model

There are a number of limitations with the World Model described in the preceding section. Among these are the following: MPI cannot be initialized from different application components without *a priori* knowledge or coordination; MPI cannot be initialized more than once; and MPI cannot be reinitialized after `MPI_FINALIZE` has been called. This section describes an alternative approach to MPI initialization—the Sessions Model. With this approach, an MPI application, or components of the application, can instantiate MPI resources for the specific communication needs of this component. `MPI_COMM_WORLD` is not valid for use as a communicator. `MPI_INFO_ENV` is not valid for use as an info object when only using the Sessions Model. As described in Section 11.2.1, MPI must be initialized using the World Model to use this info object. Note that an application may employ both the Sessions Model and World Model concurrently (see Section 11.1).

In the Sessions Model, MPI resources can be allocated and freed multiple times in an MPI process.

As shown in Figure 11.1, when using the Sessions Model, an MPI process instantiates an **MPI Session handle**, which can be used to query the runtime system about characteristics of the job within which the process is running, as well as other system resources. Using this information, the MPI process can then create an MPI Group based on application requirements and available resources, which in turn can be used to create an MPI Communicator, Window, or File. By judicious creation of communicators, an application only needs to allocate MPI resources based on its communication requirements. Although there are existing MPI interfaces for creating communicators that can, in principle, allow for resource optimizations within an MPI implementation, this can only be done following initialization of MPI.

For multithreaded applications, the Sessions Model provides fine-grain control of the thread support level for MPI objects. It is possible to specify different thread support levels when creating different *MPI Session handles*. Thus different components of an application can use different thread support levels.

The Sessions Model introduces a concept of isolation. MPI objects derived from different *MPI Session handles* shall not be intermixed with each other in a single MPI procedure call. MPI objects derived from the Sessions Model shall not be intermixed in a single MPI procedure call with MPI objects derived from the World Model. MPI objects derived from the Sessions Model shall not be intermixed in a single MPI procedure call with MPI objects

derived from the communicator obtained from a call to `MPI_COMM_GET_PARENT` or `MPI_COMM_JOIN`.

This restriction does not apply to generalized requests (Section 14.2) as such requests are not associated directly with communicators or other MPI objects. Note however, the Sessions Model does not otherwise change the semantics or behavior of MPI objects.

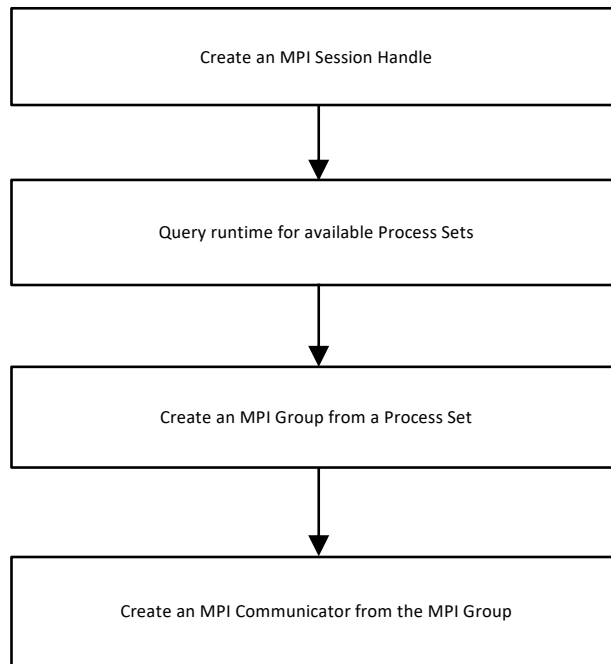


Figure 11.1: Steps to creating an MPI Communicator from an MPI Session handle.

11.3.1 Session Creation and Destruction Methods

`MPI_SESSION_INIT`(info, errhandler, session)

IN	info	info object to specify thread support level and MPI implementation specific resources (handle)
IN	errhandler	error handler to invoke in the event that an error is encountered during this function call (handle)
OUT	session	new session (handle)

C binding

```
int MPI_Session_init(MPI_Info info, MPI_Errhandler errhandler,
                    MPI_Session *session)
```


Fortran 2008 binding

```

MPI_Session_init(info, errhandler, session, ierror)
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
    TYPE(MPI_Session), INTENT(OUT) :: session
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SESSION_INIT(INFO, ERRHANDLER, SESSION, IERROR)
    INTEGER INFO, ERRHANDLER, SESSION, IERROR

```

`MPI_SESSION_INIT` is a local procedure.

The `info` argument is used to request MPI functionality requirements and possible MPI implementation specific capabilities. The following `info` keys are predefined:

"thread_level": Used to request the thread support level required for MPI objects derived from the Session. Allowed values are "MPI_THREAD_SINGLE", "MPI_THREAD_FUNNELED", "MPI_THREAD_SERIALIZED", and "MPI_THREAD_MULTIPLE". Note that the thread support value is specified by a string rather than the integer values supplied to `MPI_INIT_THREAD`. The thread support level actually provided by the MPI implementation can be determined via a subsequent call to `MPI_SESSION_GET_INFO` to return the `info` object associated with the Session. The default thread support level is MPI implementation dependent.

"mpi_memory_alloc_kinds": Used to request support for memory allocation kinds to be used by the calling MPI process on MPI objects derived from the Session. See Section 11.4.3. A value for this `info` key can also be supplied as an argument to an MPI startup mechanism as described in Section 11.5.

The `errhandler` argument specifies an error handler to invoke in the event that the Session instantiation call encounters an error. The error handler shall be either a pre-defined error handler (see Section 9.3) or one created using `MPI_SESSION_CREATE_ERRHANDLER`. Session instantiation is intended to be a lightweight operation. An MPI process may instantiate multiple Sessions. `MPI_SESSION_INIT` is always thread safe; multiple threads within an application may invoke it concurrently.

Advice to users. Requesting "MPI_THREAD_SINGLE" thread support level is generally not recommended, because this will conflict with other components of an application requesting higher levels of thread support. (*End of advice to users.*)

Advice to implementors. Owing to the restrictions of the `MPI_THREAD_SINGLE` thread support level, implementors are discouraged from making this the default thread support level for Sessions. (*End of advice to implementors.*)

```

MPI_SESSION_FINALIZE(session)

```

```

    INOUT    session          session to be finalized (handle)

```

C binding

```

int MPI_Session_finalize(MPI_Session *session)

```

Fortran 2008 binding

```

MPI_Session_finalize(session, ierror)
    TYPE(MPI_Session), INTENT(INOUT) :: session
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SESSION_FINALIZE(SESSION, IERROR)
    INTEGER SESSION, IERROR

```

This routine cleans up all MPI state associated with the supplied `session`. Every instantiated Session must be finalized using `MPI_SESSION_FINALIZE`. The handle `session` is set to `MPI_SESSION_NULL` by the call.

Before an MPI process invokes `MPI_SESSION_FINALIZE`, the process must perform all MPI calls needed to complete its involvement in MPI communications: it must locally complete all MPI operations that it initiated and it must execute matching calls needed to complete MPI communications initiated by other processes. This means that before calling `MPI_SESSION_FINALIZE`, all message handles associated with this session must be received (with `MPI_MRECV` or derived procedures) and all request handles associated with this session must be freed in the case of nonblocking operations, and must be inactive or freed in the case of persistent or partitioned operations (i.e., by calling one of the procedures `MPI_{TEST|WAIT}{|ANY|SOME|ALL}` or `MPI_REQUEST_FREE`).

The call to `MPI_SESSION_FINALIZE` does not free objects created by MPI calls; these objects are freed using `MPI_XXX_FREE`, `MPI_COMM_DISCONNECT`, or `MPI_FILE_CLOSE` calls.

Once `MPI_SESSION_FINALIZE` returns, no MPI procedure may be called in the Sessions Model that are related to this session (not even freeing objects that are derived from this session), except for those listed in Section 11.4.1.

Advice to users. Opaque objects and their handles may bind internal resources. Therefore, it is highly recommended to explicitly free the handles associated with this session before finalizing it. Such associated handles can be group, communicator, window, file, message, and request handles, whereas datatype, operation (e.g., for reductions), error handler, and info handles exist independently of the World Model or a session in the Sessions Model. In addition, if attributes are cached on such an opaque object (see Section 7.7), then the delete callback functions are only invoked when the object is explicitly freed (or disconnected). (*End of advice to users.*)

Most handles that exist independently from the World Model or a session in the Sessions Model, e.g., datatype handles, can be created only while MPI is initialized. For example, a datatype handle that was created when one particular session existed can be used in any other session (or in the World Model), even if the second session was initialized after the first session had already been finalized and no other session existed in between. See Section 11.4.1 for handle creation procedures that do not require that MPI is initialized.

`MPI_SESSION_FINALIZE` may be synchronizing on any or all of the groups associated with communicators, windows, or files derived from the session and not disconnected, freed, or closed, respectively, before the call to the `MPI_SESSION_FINALIZE` procedure.

`MPI_SESSION_FINALIZE` behaves as if all such synchronizations occur concurrently. As `MPI_COMM_FREE` may mark a communicator for freeing later, `MPI_SESSION_FINALIZE` may be synchronizing on the group associated with a communicator that is only freed (with `MPI_COMM_FREE`) rather than disconnected (with `MPI_COMM_DISCONNECT`).

Rationale. This rule is similar to the rule that `MPI_FINALIZE` is collective (see Section 11.2.2), but does not require that `MPI_SESSION_FINALIZE` be collective over all connected MPI processes. It also allows for cases where some MPI processes may have derived a set of communicators using a different number of session handles. See Example 11.9. (*End of rationale.*)

Advice to implementors. This rule also allows for the completion of communications the MPI process is involved with that may not yet be completed from the viewpoint of the underlying MPI system. See Section 2.9 on *progress* and the advice to implementors at the end of Section 11.2.2. (*End of advice to implementors.*)

Advice to implementors. An MPI implementation should be able to implement the semantics of `MPI_SESSION_FINALIZE` as a *local* procedure, provided an application frees all MPI windows, closes all MPI files, and uses `MPI_COMM_DISCONNECT` to free all MPI communicators associated with a session prior to invoking `MPI_SESSION_FINALIZE` on the corresponding session handle. (*End of advice to implementors.*)

Example 11.9. Three MPI processes are connected with 2 communicators (indicated by the = symbols), derived from one session handle in process X but from two separate session handles in both process Y and Z.

process-X	process-Y	process-Z	Remarks
			sesX, sesYA, sesYB, sesZA and sesZB are session handles.
(sesX)=====	(sesYA)=====	(sesZA)	communicator_1 and
(sesX)=====	(sesYB)=====	(sesZB)	communicator_2 are derived
			from them.
SF(sesX)	SF(sesYA)	SF(sesZA)	SF = MPI_SESSION_FINALIZE
	SF(sesYB)	SF(sesZB)	

Process X has only to finalize its one session handle, whereas the other two MPI processes have to call `MPI_SESSION_FINALIZE` twice in the same sequence with respect to the communicators derived from the session handles. Specifically, both process Y and process Z shall call `MPI_SESSION_FINALIZE` for the session from which `communicator_1` was derived before calling the `MPI_SESSION_FINALIZE` for the session from which `communicator_2` was derived, or vice versa (i.e., both shall finalize the session for `communicator_2` first then finalize the session for `communicator_1`). The call `SF(sesX)` in process X may not return until both `SF(sesYA)` and `SF(sesYB)` are called in processes Y and Z.

11.3.2 Processes Sets

Process sets are the mechanism for MPI applications to query the runtime. Process sets are identified by process set names. Process set names have a *Uniform Resource Identifier* (URI) format. Two process set names are mandated: "mpi://WORLD" and "mpi://SELF". Additional process set names may be defined, for example, "mpix://UNIVERSE" and "hwloc://L3Cache" may be defined by the MPI implementation. The "mpi://" namespace is reserved for exclusive use by the MPI standard. Figure 11.2 depicts process sets that the

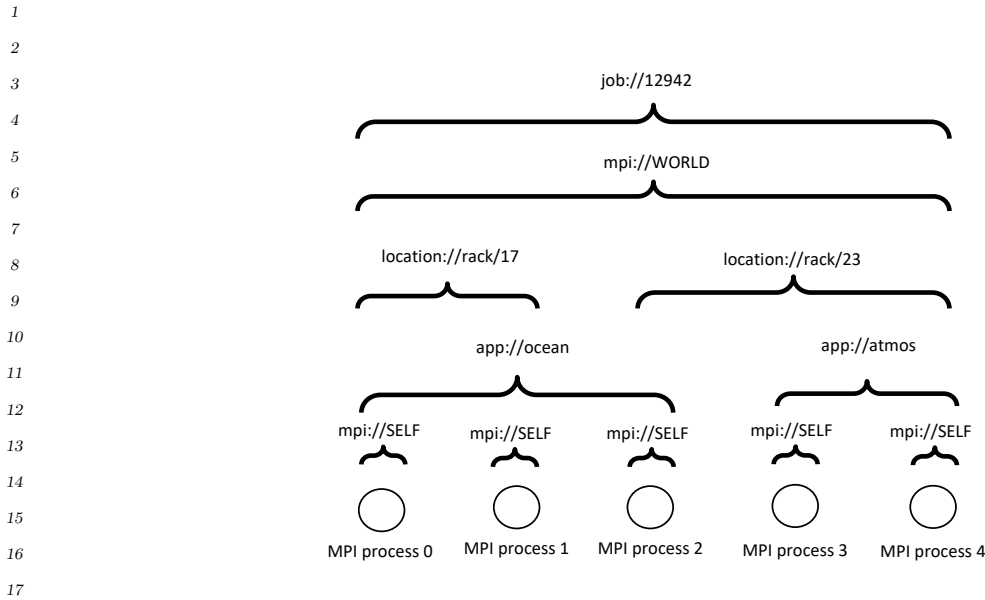


Figure 11.2: Examples of process sets. Illustrated are the two mandated process sets—`"mpi://WORLD"` and `"mpi://SELF"`—along with several optional ones that a runtime could define. In this example, `MPI_SESSION_GET_NUM_PSETS` would return five at each MPI process.

runtime could associate with an instance of an MPI job. In this example, the two mandated process sets are defined, in addition to optional, implementation specific ones.

Mechanisms for defining process sets and how system resources are assigned to these sets is considered to be implementation dependent.

A process set caches (key,value) pairs that are accessible to the application via an `MPI_Info` object. The `"mpi_size"` key is mandatory for all process sets.

11.3.3 Runtime Query Functions

`MPI_SESSION_GET_NUM_PSETS(session, info, npset_names)`

IN	session	session (handle)
IN	info	info object (handle)
OUT	npset_names	number of available process sets (nonnegative integer)

C binding

```
int MPI_Session_get_num_psets(MPI_Session session, MPI_Info info,
                              int *npset_names)
```

Fortran 2008 binding

```

MPI_Session_get_num_psets(session, info, npset_names, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, INTENT(OUT) :: npset_names
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_SESSION_GET_NUM_PSETS(SESSION, INFO, NPSET_NAMES, IERROR)
    INTEGER SESSION, INFO, NPSET_NAMES, IERROR

```

This function is used to query the runtime for the number of available process sets in which the calling MPI process is a member. An MPI implementation is allowed to increase the number of available process sets during the execution of an MPI application when new process sets become available. However, MPI implementations are not allowed to change the index of a particular process set name, or to change the name of the process set at a particular index, or to delete a process set name once it has been added. When a process set becomes invalid, for example, when some processes become unreachable due to failures in the communication system, subsequent usage of the process set name should raise an error. For example, creating an MPI_Group from such a process set might succeed because it is a local operation, but creating an MPI_Comm from that group and attempting collective communication should raise an error.

Advice to implementors. It is anticipated that an MPI implementation may be relying on an external runtime system to provide process sets. Such runtime systems may have the ability to dynamically create process sets during the course of application execution. Requiring the number of process sets returned by `MPI_SESSION_GET_NUM_PSETS` to be constant over the course of application execution would prevent an application from taking advantage of such capabilities. (*End of advice to implementors.*)

```

MPI_SESSION_GET_NTH_PSET(session, info, n, pset_len, pset_name)

```

IN	session	session (handle)
IN	info	info object (handle)
IN	n	index of the desired process set name (integer)
INOUT	pset_len	length of the pset_name argument (integer)
OUT	pset_name	name of the nth process set (string)

C binding

```

int MPI_Session_get_nth_pset(MPI_Session session, MPI_Info info, int n,
    int *pset_len, char *pset_name)

```

Fortran 2008 binding

```

MPI_Session_get_nth_pset(session, info, n, pset_len, pset_name, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, INTENT(IN) :: n

```

```

1      INTEGER, INTENT(INOUT) :: pset_len
2      CHARACTER(LEN=*), INTENT(OUT) :: pset_name
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

5      MPI_SESSION_GET_NTH_PSET(SESSION, INFO, N, PSET_LEN, PSET_NAME, IERROR)
6      INTEGER SESSION, INFO, N, PSET_LEN, IERROR
7      CHARACTER*(*) PSET_NAME

```

This function returns the name of the *n*th process set in the supplied `pset_name` buffer. `pset_len` is the size of the buffer needed to store the *n*th process set name. If the `pset_len` passed into the function is less than the actual buffer size needed for the process set name, then the string value returned in `pset_name` is truncated. If `pset_len` is set to 0, `pset_name` is not changed. On return, the value of `pset_len` will be set to the required buffer size to hold the process set name. In C, `pset_len` includes the required space for the null terminator. In C, this function returns a null terminated string in all cases where the `pset_len` input value is greater than 0.

If two MPI processes get the same process set name, then the intersection of the two process sets shall either be the empty set or identical to the union of the two process sets.

After a successful call to `MPI_SESSION_GET_NTH_PSET`, subsequent calls to routines that query information about the same process set name and same session handle must return the same information. An MPI implementation is not allowed to alter any of the returned process set names.

Process set names have an implementation-defined maximum length of `MPI_MAX_PSET_NAME_LEN` characters. `MPI_MAX_PSET_NAME_LEN` shall have a value of at least 63.

Advice to users. `MPI_MAX_PSET_NAME_LEN` might be very large, so it might not be wise to declare a string of that size. Users are encouraged to use `MPI_SESSION_GET_NTH_PSET` both for obtaining the length of a `pset_name` and the process set name. (*End of advice to users.*)

```

33      MPI_SESSION_GET_INFO(session, info_used)

```

IN	session	session (handle)
OUT	info_used	see explanation below (handle)

C binding

```

39      int MPI_Session_get_info(MPI_Session session, MPI_Info *info_used)

```

Fortran 2008 binding

```

42      MPI_Session_get_info(session, info_used, ierror)
43      TYPE(MPI_Session), INTENT(IN) :: session
44      TYPE(MPI_Info), INTENT(OUT) :: info_used
45      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

47      MPI_SESSION_GET_INFO(SESSION, INFO_USED, IERROR)
48      INTEGER SESSION, INFO_USED, IERROR

```

MPI_SESSION_GET_INFO returns a new info object containing the hints of the MPI Session associated with `session`. The current setting of all hints related to this MPI Session is returned in `info_used`. An MPI implementation is required to return all hints that are supported by the implementation and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no key/value pair. The user is responsible for freeing `info_used` via **MPI_INFO_FREE**.

MPI_SESSION_GET_PSET_INFO(`session`, `pset_name`, `info`)

IN	<code>session</code>	session (handle)
IN	<code>pset_name</code>	name of process set (string)
OUT	<code>info</code>	info object containing information about the given process set (handle)

C binding

```
int MPI_Session_get_pset_info(MPI_Session session, const char *pset_name,
                             MPI_Info *info)
```

Fortran 2008 binding

```
MPI_Session_get_pset_info(session, pset_name, info, ierror)
  TYPE(MPI_Session), INTENT(IN) :: session
  CHARACTER(LEN=*), INTENT(IN) :: pset_name
  TYPE(MPI_Info), INTENT(OUT) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SESSION_GET_PSET_INFO(SESSION, PSET_NAME, INFO, IERROR)
  INTEGER SESSION, INFO, IERROR
  CHARACTER*(*) PSET_NAME
```

This function is used to query properties of a specific process set. The returned `info` object can be queried with existing MPI info object query functions. One key/value pair must be defined, "mpi_size". The value of the "mpi_size" key specifies the number of MPI processes in the process set. The user is responsible for freeing the returned `MPI_Info` object.

11.3.4 Sessions Model Examples

This section presents several examples of how to use MPI Sessions to create MPI Groups and MPI Communicators.

Example 11.10. Example illustrating creation of an MPI communicator using the Sessions Model.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "mpi.h"

static MPI_Session lib_shandle = MPI_SESSION_NULL;
```

```

1  static MPI_Comm lib_comm = MPI_COMM_NULL;
2
3  int library_foo_init(void)
4  {
5      int rc, flag, valuelen;
6      int ret = 0;
7      const char pset_name[] = "mpi://WORLD";
8      const char mt_key[] = "thread_level";
9      const char mt_value[] = "MPI_THREAD_MULTIPLE";
10     char out_value[100]; /* large enough */
11     MPI_Group wgroup = MPI_GROUP_NULL;
12     MPI_Info sinfo = MPI_INFO_NULL;
13     MPI_Info tinfo = MPI_INFO_NULL;
14
15     MPI_Info_create(&sinfo);
16     MPI_Info_set(sinfo, mt_key, mt_value);
17     rc = MPI_Session_init(sinfo, MPI_ERRORS_RETURN,
18                          &lib_shandle);
19     if (rc != MPI_SUCCESS) {
20         ret = -1;
21         goto fn_exit;
22     }
23
24     /*
25      * check we got thread support level foo library needs
26      */
27     rc = MPI_Session_get_info(lib_shandle, &tinfo);
28     if (rc != MPI_SUCCESS) {
29         ret = -1;
30         goto fn_exit;
31     }
32
33     valuelen = sizeof(out_value);
34     MPI_Info_get_string(tinfo, mt_key, &valuelen,
35                        out_value, &flag);
36     if (0 == flag) {
37         printf("Could not find key %s\n", mt_key);
38         ret = -1;
39         goto fn_exit;
40     }
41
42     if (strcmp(out_value, mt_value)) {
43         printf("Did not get thread multiple support, got %s\n",
44                out_value);
45         ret = -1;
46         goto fn_exit;
47     }
48
49     /*
50      * create a group from the WORLD process set
51      */
52     rc = MPI_Group_from_session_pset(lib_shandle,
53                                     pset_name,

```



```

                                &wgroup);
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

    if (rc != MPI_SUCCESS) {
        ret = -1;
        goto fn_exit;
    }

    /*
     * get a communicator
     */
    rc = MPI_Comm_create_from_group(wgroup,
                                    "org.mpi-forum.mpi-v4_0.example-ex11_10",
                                    MPI_INFO_NULL,
                                    MPI_ERRORS_RETURN,
                                    &lib_comm);

    if (rc != MPI_SUCCESS) {
        ret = -1;
        goto fn_exit;
    }

    /*
     * release unused resources
     */

fn_exit:
    if (wgroup != MPI_GROUP_NULL) {
        MPI_Group_free(&wgroup);
    }

    if (sinfo != MPI_INFO_NULL) {
        MPI_Info_free(&sinfo);
    }

    if (tinfo != MPI_INFO_NULL) {
        MPI_Info_free(&tinfo);
    }

    if (ret != 0) {
        MPI_Session_finalize(&lib_shandle);
    }

    return ret;
}

```

Example 11.10 shows how the predefined "mpi://WORLD" process set can be used to first create a local MPI group and then subsequently to create an MPI communicator from this group.

Example 11.11. This example illustrates the use of Process Set query functions to select a Process Set to use for MPI Group creation.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

1  #include "mpi.h"
2
3  int main(int argc, char *argv[])
4  {
5      int i, n_psets, psetlen, rc, ret;
6      int valuelen;
7      int flag = 0;
8      char *pset_name = NULL;
9      char *info_val = NULL;
10     MPI_Session shandle = MPI_SESSION_NULL;
11     MPI_Info sinfo = MPI_INFO_NULL;
12     MPI_Group pgroup = MPI_GROUP_NULL;
13
14     if (argc < 2) {
15         fprintf(stderr, "A process set name fragment is required\n");
16         return EXIT_FAILURE;
17     }
18
19     rc = MPI_Session_init(MPI_INFO_NULL, MPI_ERRORS_RETURN, &shandle);
20     if (rc != MPI_SUCCESS) {
21         fprintf(stderr, "Could not initialize session, bailing out\n");
22         return EXIT_FAILURE;
23     }
24
25     MPI_Session_get_num_psets(shandle, MPI_INFO_NULL, &n_psets);
26
27     for (i=0, pset_name=NULL; i<n_psets; i++) {
28         psetlen = 0;
29         MPI_Session_get_nth_pset(shandle, MPI_INFO_NULL, i,
30                                 &psetlen, NULL);
31         pset_name = (char *)malloc(sizeof(char) * psetlen);
32         MPI_Session_get_nth_pset(shandle, MPI_INFO_NULL, i,
33                                 &psetlen, pset_name);
34         if (strstr(pset_name, argv[1]) != NULL) break;
35
36         free(pset_name);
37         pset_name = NULL;
38     }
39
40     if (pset_name == NULL) {
41         fprintf(stderr, "Unable to find matching process set\n");
42         return EXIT_FAILURE;
43     }
44
45     /*
46      * get instance of an info object for this Session
47      */
48
49     MPI_Session_get_pset_info(shandle, pset_name, &sinfo);
50     valuelen = 0;
51     MPI_Info_get_string(sinfo, "mpi_size", &valuelen, NULL, &flag);
52     if (flag) {
53         info_val = (char *)malloc(valuelen);
54         MPI_Info_get_string(sinfo, "mpi_size", &valuelen, info_val, &flag);
55         free(info_val);
56     }
57
58 }

```

```

/*
 * create a group from the process set
 */

rc = MPI_Group_from_session_pset(shandle, pset_name,
                                &pgroup);
ret = (rc == MPI_SUCCESS) ? 0 : EXIT_FAILURE;

free(pset_name);
if (pgroup != MPI_GROUP_NULL) {
    MPI_Group_free(&pgroup);
}
MPI_Info_free(&sinfo);
MPI_Session_finalize(&shandle);

fprintf(stderr, "Test completed ret = %d\n", ret);
return ret;
}

```

Example 11.11 illustrates several aspects of the Sessions Model. First, the default error handler can be specified when instantiating a Session instance. Second, there must be at least two process sets associated with a Session. Third, the example illustrates use of the Sessions info object and the one required key: "mpi_size".

Example 11.12. A Fortran 2008 example illustrating how to obtain information about available process sets, create an MPI Group from a process set, and subsequently create an MPI Communicator.

```

PROGRAM MAIN
  USE mpi_f08
  IMPLICIT NONE
  INTEGER :: pset_len, ierror, n_psets
  CHARACTER(LEN=:), ALLOCATABLE :: pset_name
  TYPE(MPI_Session) :: shandle
  TYPE(MPI_Group) :: pgroup
  TYPE(MPI_Comm) :: pcomm

  CALL MPI_Session_init(MPI_INFO_NULL, MPI_ERRORS_RETURN, &
                        shandle, ierror)
  IF (ierror .NE. MPI_SUCCESS) THEN
    WRITE(*,*) "MPI_Session_init failed"
    ERROR STOP
  END IF

  CALL MPI_Session_get_num_psets(shandle, MPI_INFO_NULL, n_psets)
  IF (n_psets .LT. 2) THEN
    WRITE(*,*) "MPI_Session_get_num_psets didn't return at least 2 psets"
    ERROR STOP
  END IF

  !
  ! Just get the second pset's length and name
  ! Note that index values are zero-based, even in Fortran
  !

  pset_len = 0

```

```

1  CALL MPI_Session_get_nth_pset(shandle, MPI_INFO_NULL, 1,      &
2                                pset_len, pset_name)
3  ALLOCATE(CHARACTER(LEN=pset_len)::pset_name)
4  CALL MPI_Session_get_nth_pset(shandle, MPI_INFO_NULL, 1,      &
5                                pset_len, pset_name)
6
7  !
8  !   create a group from the pset
9  !
10 CALL MPI_Group_from_session_pset(shandle, pset_name, pgroup)
11 !
12 !   free the buffer used for the pset name
13 !
14 DEALLOCATE(pset_name)
15
16 !
17 !   create a MPI communicator from the group
18 !
19 CALL MPI_Comm_create_from_group(pgroup, "session_example",      &
20                                MPI_INFO_NULL,                    &
21                                MPI_ERRORS_RETURN,                &
22                                pcomm)
23
24 CALL MPI_Barrier(pcomm, ierror)
25 IF (ierror .NE. MPI_SUCCESS) THEN
26   WRITE(*,*) "Barrier call on communicator failed"
27   ERROR STOP
28 END IF
29
30 CALL MPI_Comm_free(pcomm)
31 CALL MPI_Group_free(pgroup)
32 CALL MPI_Session_finalize(shandle, ierror)
33
34 END PROGRAM MAIN

```

Note in this example that the call to `MPI_SESSION_FINALIZE` may block in order to ensure that the calling MPI process has completed its involvement in the preceding `MPI_BARRIER` operation. If `MPI_COMM_DISCONNECT` had been used instead of `MPI_COMM_FREE`, the example would have blocked in `MPI_COMM_DISCONNECT` rather than `MPI_SESSION_FINALIZE`.

11.4 Common Elements of Both Process Models

11.4.1 MPI Functionality that is Always Available

Some MPI functions may be invoked at any time, including prior to calling `MPI_INIT` or `MPI_SESSION_INIT`, and following MPI finalization, independent of whether the World Model, Sessions Model, or both are used. These functions can be called concurrently by multiple threads within an MPI Process. Table 11.1 lists the applicable MPI functions.

In addition to the functions listed in Table 11.1, any function with the prefix `MPI_T_` (within the constraints for functions with this prefix listed in Section 16.3.4) may also be called prior to MPI initialization and after MPI finalization.

Table 11.1: List of MPI Functions that can be called at any time within an MPI program, including prior to MPI initialization and following MPI finalization

MPI_INITIALIZED
MPI_FINALIZED
MPI_GET_VERSION
MPI_GET_LIBRARY_VERSION
MPI_ABI_GET_VERSION
MPI_ABI_GET_INFO
MPI_INFO_CREATE
MPI_INFO_CREATE_ENV
MPI_INFO_SET
MPI_INFO_DELETE
MPI_INFO_GET_STRING
MPI_INFO_GET_NKEYS
MPI_INFO_GET_NTHKEY
MPI_INFO_DUP
MPI_INFO_FREE
MPI_INFO_F2C
MPI_INFO_C2F
MPI_SESSION_CREATE_ERRHANDLER
MPI_SESSION_CALL_ERRHANDLER
MPI_ERRHANDLER_FREE
MPI_ERRHANDLER_F2C
MPI_ERRHANDLER_C2F
MPI_ERROR_STRING
MPI_ERROR_CLASS
MPI_ADD_ERROR_CLASS
MPI_REMOVE_ERROR_CLASS
MPI_ADD_ERROR_CODE
MPI_REMOVE_ERROR_CODE
MPI_ADD_ERROR_STRING
MPI_REMOVE_ERROR_STRING

11.4.2 Aborting MPI Processes

MPI_ABORT(comm, errorcode)

IN	comm	communicator of MPI processes to abort (handle)
IN	errorcode	error code to return to invoking environment (integer)

C binding

int MPI_Abort(MPI_Comm comm, int errorcode)

Fortran 2008 binding

```

MPI_Abort(comm, errorcode, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(IN) :: errorcode
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ABORT(COMM, ERRORCODE, IERROR)
    INTEGER COMM, ERRORCODE, IERROR

```

This routine makes a “best attempt” to abort all MPI processes in the group of `comm`. This function does not require that the invoking environment take any action with the error code. However, a Unix or POSIX environment should handle this as a return `errorcode` from the main program.

It may not be possible for an MPI implementation to abort only the processes represented by `comm` if this is a subset of the processes. In this case, the MPI implementation should attempt to abort all the connected processes but should not abort any unconnected processes. When using the World Model, and if no processes were spawned, accepted, or connected then this has the effect of aborting all the processes associated with `MPI_COMM_WORLD`. In the case of the Sessions Model, if an MPI process has instantiated multiple sessions, the union of the process sets in these sessions are considered connected processes. Thus invoking `MPI_ABORT` on a communicator derived from one of these sessions will result in all MPI processes in this union being aborted.

Advice to implementors. After aborting a subset of processes, a high-quality implementation should be able to provide error handling for communicators, windows, and files involving both aborted and nonaborted processes. As an example, if the user changes the error handler for `MPI_COMM_WORLD` to `MPI_ERRORS_RETURN` or a custom error handler, when a subset of `MPI_COMM_WORLD` is aborted, the remaining processes in `MPI_COMM_WORLD` should be able to continue communicating with each other and receive an appropriate error code when attempting communication with an aborted process (e.g., an error of class `MPI_ERR_PROC_ABORTED`). A high-quality implementation should support equivalent behavior for communicators derived from sessions. (*End of advice to implementors.*)

Advice to users. Whether the `errorcode` is returned from the executable or from the MPI process startup mechanism (e.g., `mpiexec`), is an aspect of quality of the MPI library but not mandatory. (*End of advice to users.*)

Advice to implementors. Where possible, a high-quality implementation will try to return the `errorcode` from the MPI process startup mechanism (e.g. `mpiexec` or `singleton init`). (*End of advice to implementors.*)

11.4.3 Memory Allocation Info

Computing systems contain memory with different properties, including differences in performance, persistence, access permissions, or access mode. These distinct memories are generally allocated using distinct mechanisms and are referred to as memory allocation kinds that are named according to the method of allocation. The following info keys can be used to request or query the memory allocation kinds supported by the MPI library and

to assert application usage of memory allocation kinds with respect to specific MPI objects, as shown in Example 11.13.

"mpi_memory_alloc_kinds" (string, default: "mpi,system"): A comma-separated list of memory allocation kinds. When defaulted, the value returned must, at minimum, contain the kinds specified in "default" and may contain additional implementation-defined kinds. Different sessions may return different default values.

When set on the input info object in a call to `MPI_SESSION_INIT`, `MPI_COMM Spawn`, or `MPI_COMM Spawn Multiple`, or when supplied as an argument to an MPI startup mechanism, this info key requests support for the specified memory allocation kinds.

When returned by MPI, this info key indicates the memory allocation kinds supported by the MPI library on the given session, MPI object, or objects derived from the World Model. This info key does not affect the kind of memory allocated by MPI, e.g., in a call to `MPI_ALLOC_MEM` or `MPI_WIN_ALLOCATE`. A value corresponding to the empty string represents no memory allocation kinds.

"mpi_assert_memory_alloc_kinds" (string, not set by default): A comma separated list of memory allocation kinds that the calling MPI process will use with the given MPI object. A value corresponding to the empty string represents no memory allocation kinds.

The "mpi_memory_alloc_kinds" info key is used both for requesting and querying support for memory allocation kinds from the MPI library.

When supplied to `MPI_SESSION_INIT`, this info key requests support for memory allocation kinds for all objects that will be derived from the new session. This info hint can also be supplied through an argument to an MPI startup mechanism. In the Sessions Model, this behaves as though the "mpi_memory_alloc_kinds" info key with the given value was supplied in the info argument in calls to `MPI_SESSION_INIT`. A value of "mpi_memory_alloc_kinds" supplied in the info argument to `MPI_SESSION_INIT` takes precedence over a value supplied as an argument to an MPI startup mechanism.

In the World Model, an info hint passed to an MPI startup mechanism requests support for memory allocation kinds for all objects derived from the World Model. This info hint can also be supplied to `MPI_COMM Spawn` or `MPI_COMM Spawn Multiple` in the World Model. This requests support for memory allocation kinds for all objects derived from the World Model in the spawned MPI process or MPI processes.

When returned by `MPI_SESSION_GET_INFO`, this info key indicates the memory allocation kinds supported by the MPI library on the given session. When returned by a call to `MPI_COMM_GET_INFO` on `MPI_COMM_WORLD` or `MPI_COMM_SELF`, this info key indicates the memory allocation kinds supported by the MPI library for all objects derived from the World Model. The value of "mpi_memory_alloc_kinds" on `MPI_COMM_WORLD` and `MPI_COMM_SELF` cannot be updated or deleted between MPI initialization (`MPI_INIT`) and MPI finalization (`MPI_FINALIZE`) of the World Model.

If "mpi_memory_alloc_kinds" was supplied during session creation, then the value of the corresponding key in the info object returned by `MPI_SESSION_GET_INFO` must include all requested memory allocation kinds that are supported. The substrings that indicate support for these memory allocation kinds must be identical to those supplied by the user.

MPI may also return additional memory allocation kinds that were not requested by the user. The order of the memory allocation kinds returned through this info key is undefined.

Rationale. MPI libraries may have implementation-specific mechanisms (e.g., environment variables) that control the supported memory allocation kinds. Allowing implementations to return additional memory allocation kinds provides for compatibility with such mechanisms. (*End of rationale.*)

The "mpi_memory_alloc_kinds" info key must also be contained in the info object returned by `MPI_COMM_GET_INFO`, `MPI_WIN_GET_INFO`, and `MPI_FILE_GET_INFO`. If the communicator, window, or file is derived from the World Model, the value of the "mpi_memory_alloc_kinds" info key must be identical to the value of the "mpi_memory_alloc_kinds" info key in the info object returned by a call to `MPI_COMM_GET_INFO` on `MPI_COMM_WORLD` or `MPI_COMM_SELF` unless the user has asserted that support for memory allocation kinds can be restricted by setting "mpi_assert_memory_alloc_kinds" on that communicator, window, or file. If the communicator, window, or file is derived from the Sessions Model, the value of this info key must be identical to the value of this info key in the info object returned by `MPI_SESSION_GET_INFO` for that session unless the user has asserted that support for memory allocation kinds can be restricted by setting "mpi_assert_memory_alloc_kinds" on that communicator, window, or file.

When the user sets the "mpi_assert_memory_alloc_kinds" info key on the input info object for communicator creation, including via `MPI_COMM_SPAWN` or `MPI_COMM_SPAWN_MULTIPLE`, window creation, or file creation the implementation may assume that the memory for all communication buffers passed to MPI operations performed by the calling MPI process on the newly created MPI object will use only the memory allocation kinds listed in the value string. If the MPI library does not support one or more of the allocation kinds associated with the "mpi_assert_memory_alloc_kinds" info key, it will ignore this info key. When an MPI library recognizes this info key, the value returned when querying this info key (e.g., through a call to `MPI_COMM_GET_INFO`) must be identical to the value supplied by the user. It is erroneous to pass a communication buffer with an unsupported memory allocation kind to an MPI routine.

Memory allocation kind strings are comma separated lists that follow the rules specified in Section 10. Each element in the list is a memory allocation kind that is formatted as the name of the kind, followed by an optional colon separated list of restrictors. Whitespace is not permitted within the list of restrictors. For example, "kind_a:restrictor_1,kind_b:restrictor_1:restrictor_2,...".

Within a memory allocation kind string, a given kind may be listed more than once with different restrictors, e.g., "kind_a:restrictor_1,kind_a:restrictor_2". A given kind may also be listed more than once with fewer restrictors, e.g., "kind_a,kind_a:restrictor_1". A memory allocation kind with no restrictors indicates an unrestricted memory allocation kind. Each instance of a kind in the memory allocation kind string indicates a separate and potentially overlapping memory allocation kind. The following memory allocation kinds and restrictors are defined by MPI. This list may be extended by MPI side documents and implementations.

- "system": Memory allocated by standard operating system allocators. When support for the "system" memory allocation kind is requested by the user, it must be provided by the MPI library.

- "mpi": Memory allocated by the MPI library. When support for the "mpi" memory allocation kind is requested by the user, it must be provided by the MPI library.

Restrictors for the "mpi" memory allocation kind:

- "alloc_mem": Memory allocated by a call to `MPI_ALLOC_MEM`
- "win_allocate": Memory allocated by a call to `MPI_WIN_ALLOCATE`
- "win_allocate_shared": Memory allocated by a call to `MPI_WIN_ALLOCATE_SHARED`

Example 11.13. This example demonstrates the usage of memory allocation kinds info keys with the Sessions Model. It shows how support for additional memory allocation kinds can be requested, how supported memory allocation kinds can be queried, how to parse the list of supported memory allocation kinds, and how to assert that a subset of supported memory allocation kinds are used with operations on a specific communicator.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int gpu_aware = 0, len = 0, flag = 0;
    MPI_Info info;
    MPI_Session session;
    MPI_Group wgroup;
    MPI_Comm system_comm, gpu_comm = MPI_COMM_NULL;

    MPI_Info_create(&info);
    MPI_Info_set(info, "mpi_memory_alloc_kinds", "system,gpu:device");
    MPI_Session_init(info, MPI_ERRORS_ARE_FATAL, &session);
    MPI_Info_free(&info);

    MPI_Session_get_info(session, &info);
    MPI_Info_get_string(info, "mpi_memory_alloc_kinds",
                        &len, NULL, &flag);

    if (flag) {
        char *val, *valptr, *kind;

        val = valptr = (char *) malloc(len);
        if (NULL == val) return 1;

        MPI_Info_get_string(info, "mpi_memory_alloc_kinds",
                            &len, val, &flag);

        while ((kind = strsep(&val, ",")) != NULL) {
            if (strcasecmp(kind, "gpu:device") == 0) {
                gpu_aware = 1;
                break;
            }
        }
    }
}
```

```

1      free(valptr);
2  }
3
4  MPI_Info_free(&info);
5
6  MPI_Group_from_session_pset(session, "mpi://WORLD" , &wgroup);
7
8  // Create a communicator for operations on system memory
9  MPI_Info_create(&info);
10 MPI_Info_set(info, "mpi_assert_memory_alloc_kinds", "system");
11 MPI_Comm_create_from_group(wgroup,
12     "org.mpi-forum.example.mem-alloc-kind-usage.system",
13     info, MPI_ERRORS_ABORT, &system_comm);
14
15 MPI_Info_free(&info);
16
17 // Check if all processes have GPU support
18 MPI_Allreduce(MPI_IN_PLACE, &gpu_aware, 1, MPI_INT, MPI_LAND,
19     system_comm);
20
21 // Create a communicator for operations that use GPU buffers.
22 // Note, the "gpu" memory allocation kind is provided as an example
23 // and is not one of the memory allocation kinds defined by the MPI
24 // standard.
25 if (gpu_aware) {
26     MPI_Info_create(&info);
27     MPI_Info_set(info, "mpi_assert_memory_alloc_kinds",
28         "gpu:device");
29     MPI_Comm_create_from_group(wgroup,
30         "org.mpi-forum.example.mem-alloc-kind-usage.gpu",
31         info, MPI_ERRORS_ABORT, &gpu_comm);
32     MPI_Info_free(&info);
33 }
34 else {
35     printf("Warning: GPU alloc kind not supported\n");
36 }
37
38 MPI_Group_free(&wgroup);
39
40 // Perform communication using gpu_comm if it's available.
41 // Otherwise, copy data to a system buffer and use system_comm.
42
43 if (gpu_comm != MPI_COMM_NULL) MPI_Comm_disconnect(&gpu_comm);
44 MPI_Comm_disconnect(&system_comm);
45
46 MPI_Session_finalize(&session);
47
48 return 0;
49 }

```

11.5 Portable MPI Process Startup

A number of implementations of MPI provide a startup command for MPI programs that is of the form `mpirun`

```
mpirun <mpirun arguments> <program> <program arguments>
```

Separating the command to start the program from the program itself provides flexibility, particularly for network and heterogeneous implementations. For example, the startup script need not run on one of the machines that will be executing the MPI program itself.

Having a standard startup mechanism also extends the portability of MPI programs one step further, to the command lines and scripts that manage them. For example, a validation suite script that runs hundreds of programs can be a portable script if it is written using such a standard startup mechanism. In order that the “standard” command not be confused with existing practice, which is not standard and not portable among implementations, instead of `mpirun` MPI specifies `mpiexec`.

While a standardized startup mechanism improves the usability of MPI, the range of environments is so diverse (e.g., there may not even be a command line interface) that MPI cannot mandate such a mechanism. Instead, MPI specifies an `mpiexec` startup command and recommends but does not require it, as advice to implementors. However, if an implementation does provide a command called `mpiexec`, it must be of the form described below.

It is suggested that

```
mpiexec -n <numprocs> <program>
```

be at least one way to start `<program>` with an initial set of `<numprocs>` processes, which will be accessible as the process set named “`mpi://WORLD`” in the Sessions Model and/or used to form the group associated with the built-in communicator, `MPI_COMM_WORLD` in the World Model. Other arguments to `mpiexec` may be implementation-dependent.

Advice to implementors. Implementors, if they do provide a special startup command for MPI programs, are advised to give it the following form. The syntax is chosen in order that `mpiexec` be able to be viewed as a command-line version of `MPI_COMM_SPAWN` (See Section 11.8.4).

Analogous to `MPI_COMM_SPAWN`, we have

```
mpiexec -n          <maxprocs>
      -soft          <      >
      -host          <      >
      -arch          <      >
      -wdir          <      >
      -path          <      >
      -file          <      >
      -initial-errhandler <      >
      -memory-alloc-kinds <      >
      ...
      <command line>
```

for the case where a single command line for the application program and its arguments will suffice. See Section 11.8.4 for the meanings of these arguments. For the case corresponding to `MPI_COMM_SPAWN_MULTIPLE` there are two possible formats:

Form A:

```
mpiexec { <above arguments> } : { ... } : { ... } : ... : { ... }
```

As with `MPI_COMM_SPAWN`, all the arguments are optional. (Even the `-n x` argument is optional; the default is implementation dependent. It might be 1, it might be taken from an environment variable, or it might be specified at compile time.) The names and meanings of the arguments are taken from the keys in the info argument to `MPI_COMM_SPAWN`. There may be other, implementation-dependent arguments as well.

Note that Form A, though convenient to type, prevents colons from being program arguments. Therefore an alternate, file-based form is allowed:

Form B:

```
mpiexec -configfile <filename>
```

where the lines of `<filename>` are of the form separated by the colons in Form A. Lines beginning with '#' are comments, and lines may be continued by terminating the partial line with '\'.

Example 11.14. Start 16 instances of `myprog` on the current or default machine:

```
mpiexec -n 16 myprog
```

Example 11.15. Start 10 instances of `myprog` on the machine called `ferrari`:

```
mpiexec -n 10 -host ferrari myprog
```

Example 11.16. Start 3 instances of the same program `myprog` with different command-line arguments:

```
mpiexec -n 1 myprog infile1 : -n 1 myprog infile2 : -n 1 myprog infile3
```

Example 11.17. Start 5 instances of the `ocean` program on `x86_64` hosts and 10 instances of the `atmos` program on `Power9` hosts (Form B):

```
mpiexec -n 5 -arch x86_64 ocean : -n 10 -arch power9 atmos
```

It is assumed that the implementation in this case has a method for choosing hosts of the appropriate type. Their ranks in `MPI_COMM_WORLD` are in the order specified.

Example 11.18. Start the `ocean` program on five Suns and the `atmos` program on 10 RS/6000's (Form B):

```
mpiexec -configfile myfile
```

where myfile contains

```
-n 5 -arch sun    ocean
-n 10 -arch rs6000 atmos
```

(End of advice to implementors.)

11.6 MPI and Threads

This section specifies the interaction between MPI calls and threads. Although thread compliance is not required, the standard specifies how threads are to work if they are provided. The section lists minimal requirements for **thread compliant** MPI implementations and defines functions that can be used for initializing the thread environment. MPI may be implemented in environments where threads are not supported or perform poorly. Therefore, MPI implementations are not required to be thread compliant as defined in this section. Regardless of whether or not the MPI implementation is thread compliant, a subset of MPI functions must always be thread safe. A complete list of such MPI functions is given in Table 11.1. When a thread is executing one of these routines, if another concurrently running thread also makes an MPI call, the outcome will be as if the calls executed in some order.

This section generally assumes a thread package similar to POSIX threads [45], but the syntax and semantics of thread calls are not specified here—these are beyond the scope of this document.

11.6.1 General

In a thread-compliant implementation, an MPI process is a process that may be multithreaded. Each thread can issue MPI calls; however, threads are not separately addressable: the rank argument in a send or receive call identifies an MPI process, not a thread. A message sent to an MPI process can be received by any thread in this MPI process.

Rationale. This model corresponds to the POSIX model of interprocess communication: the fact that a process is multithreaded, rather than single-threaded, does not affect the external interface of this process. MPI implementations in which MPI ‘processes’ are POSIX threads inside a single POSIX process are not thread-compliant by this definition (indeed, their “processes” are single-threaded). *(End of rationale.)*

Advice to users. It is the user’s responsibility to prevent races when threads within the same application post conflicting communication calls. The user can make sure that two threads in the same process will not issue conflicting communication calls by using distinct communicators at each thread. *(End of advice to users.)*

The two main requirements for a thread-compliant implementation are listed below.

1. All MPI calls are **thread-safe**, i.e., two concurrently running threads may make MPI calls and the outcome will be as if the calls executed in some order, even if their execution is interleaved.
2. Blocking MPI calls will block the calling thread only, allowing another thread to execute, if available. The calling thread will be blocked until the event on which it is waiting occurs. Once the blocked communication is enabled and can proceed, then

the call will complete and the thread will be marked runnable, within a finite time. A blocked thread will not prevent *progress* of other runnable threads on the same process, and will not prevent them from executing MPI calls.

Example 11.19. Process 0 consists of two threads. The first thread executes a blocking send call `MPI_Send(buff1, count, type, 0, 0, comm)`, whereas the second thread executes a blocking receive call `MPI_Recv(buff2, count, type, 0, 0, comm, &status)`, i.e., the first thread sends a message that is received by the second thread. This communication should always succeed. According to the first requirement, the execution will correspond to some interleaving of the two calls. According to the second requirement, a call can only block the calling thread and cannot prevent progress of the other thread. If the send call went ahead of the receive call, then the sending thread may block, but this will not prevent the receiving thread from executing. Thus, the receive call will occur. Once both calls occur, the communication is enabled and both calls will complete. On the other hand, a single-threaded process that posts a send, followed by a matching receive, may deadlock. The progress requirement for multithreaded implementations is stronger, as a blocked call cannot prevent progress in other threads.

Advice to implementors. MPI calls can be made thread-safe by executing only one at a time, e.g., by protecting MPI code with one process-global lock. However, blocked operations cannot hold the lock, as this would prevent progress of other threads in the process. The lock is held only for the duration of an atomic, locally-completing suboperation such as posting a send or completing a send, and is released in between. Finer locks can provide more concurrency, at the expense of higher locking overheads. Concurrency can also be achieved by having some of the MPI protocol executed by separate server threads. (*End of advice to implementors.*)

11.6.2 Clarifications

Initialization and Completion. When using the World Model, the call to `MPI_FINALIZE` should occur on the same thread that initialized MPI. We call this thread the **main thread**. The call should occur only after all process threads have completed their MPI calls, and have no *pending* communication or I/O operations.

Rationale. This constraint simplifies implementation. (*End of rationale.*)

Threads and the Sessions Model. The Sessions Model provides a finer-grain approach to controlling the interaction between MPI calls and threads. When using this model, the desired level of thread support is specified at Session initialization time. See Section 11.3. Thus it is possible for communicators and other MPI objects derived from one Session to provide a different level of thread support than those created from another Session for which a different level of thread support was requested. Depending on the level of thread support requested at Session initialization time, different threads in a MPI process can make concurrent calls to MPI when using MPI objects derived from different *session handles*. Note that the requested and provided level of thread support when creating a Session may influence the granted level of thread support in a subsequent invocation of `MPI_SESSION_INIT`. Likewise, if the application at some point calls

`MPI_INIT_THREAD`, the requested and granted level of thread support may influence the granted level of thread support for subsequent calls to `MPI_SESSION_INIT`. Similarly, if the application calls `MPI_INIT_THREAD` after a call to `MPI_SESSION_INIT`, the level of thread support returned from `MPI_INIT_THREAD` may be similarly influenced by the requested level of thread support in the prior call to `MPI_SESSION_INIT`.

In addition, if an MPI application is only using the Sessions Model, the provided thread support level returned by `MPI_QUERY_THREAD` is the same as that returned prior to invocation of `MPI_INIT_THREAD` or `MPI_INIT`. If the application also used the World Model in some component of the application, `MPI_QUERY_THREAD` will return the level of thread support returned by the original call to `MPI_INIT_THREAD`.

Multiple threads completing the same request. A program in which two threads block, waiting on the same request, is erroneous. Similarly, the same request cannot appear in the array of requests of two concurrent `MPI_{WAIT|TEST}_{ANY|SOME|ALL}` calls. In MPI, a request can only be completed once. Any combination of wait or test that violates this rule is erroneous.

Rationale. This restriction is consistent with the view that a multithreaded execution corresponds to an interleaving of the MPI calls. In a single threaded implementation, once a wait is posted on a request the request handle will be nullified before it is possible to post a second wait on the same handle. With threads, an `MPI_WAIT_{ANY|SOME|ALL}` may be blocked without having nullified its request(s) so it becomes the user's responsibility to avoid using the same request in an `MPI_WAIT` on another thread. This constraint also simplifies implementation, as only one thread will be blocked on any communication or I/O event. (*End of rationale.*)

Probe. A receive call that uses source and tag values returned by a preceding call to `MPI_PROBE` or `MPI_IPROBE` will receive the message matched by the probe call only if there was no other matching receive after the probe and before that receive. In a multithreaded environment, it is up to the user to enforce this condition using suitable mutual exclusion logic. This can be enforced by making sure that each communicator is used by only one thread on each process. Alternatively, `MPI_MPROBE` or `MPI_IMPROBE` can be used.

Collective calls. Matching of collective calls on a communicator, window, or file handle is done according to the order in which the calls are issued at each process. If concurrent threads issue such calls on the same communicator, window or file handle, it is up to the user to make sure the calls are correctly ordered, using interthread synchronization.

Advice to users. With three concurrent threads in each MPI process of a communicator `comm`, it is allowed that thread A in each MPI process calls a collective operation on `comm`, thread B calls a file operation on an existing file handle that was formerly opened on `comm`, and thread C invokes one-sided operations on an existing window handle that was also formerly created on `comm`. (*End of advice to users.*)

Rationale. As specified in `MPI_FILE_OPEN` and `MPI_WIN_CREATE`, a file handle and a window handle inherit only the group of processes of the underlying communicator,

but not the communicator itself. Accesses to communicators, window handles and file handles cannot affect one another. (*End of rationale.*)

Advice to implementors. If the implementation of file or window operations internally uses MPI communication then a duplicated communicator may be cached on the file or window object. (*End of advice to implementors.*)

Error handlers. An error handler does not necessarily execute in the context of the thread that made the error-raising MPI call; the error handler may be executed by a thread that is distinct from the thread that will return the error code.

Rationale. The MPI implementation may be multithreaded, so that part of the communication protocol may execute on a thread that is distinct from the thread that made the MPI call. The design allows the error handler to be executed on the thread where the error is raised. (*End of rationale.*)

Interaction with signals and cancellations. The outcome is undefined if a thread that executes an MPI call is cancelled (by another thread), or if a thread catches a signal while executing an MPI call. However, a thread of an MPI process may terminate, and may catch signals or be cancelled by another thread when not executing MPI calls.

Rationale. Few C library functions are signal safe, and many have cancellation points—points at which the thread executing them may be cancelled. The above restriction simplifies implementation (no need for the MPI library to be “async-cancel-safe” or “async-signal-safe”). (*End of rationale.*)

Advice to users. Users can catch signals in separate, non-MPI threads (e.g., by masking signals on MPI calling threads, and unmasking them in one or more non-MPI threads). A good programming practice is to have a distinct thread blocked in a call to `sigwait` for each user expected signal that may occur. Users must not catch signals used by the MPI implementation; as each MPI implementation is required to document the signals used internally, users can avoid these signals. (*End of advice to users.*)

Advice to implementors. The MPI library should not invoke library calls that are not thread safe, if multiple threads execute. (*End of advice to implementors.*)

11.7 The Dynamic Process Model

The dynamic process model allows for the creation and cooperative termination of processes after an MPI application has started. It provides a mechanism to establish communication between the newly created processes and the existing MPI application. It also provides a mechanism to establish communication between two existing MPI applications, even when one did not “start” the other.

The MPI procedures described in this section require the World Model, meaning that `MPI_INIT` or `MPI_INIT_THREAD` has been used to initialize MPI.

11.7.1 Starting Processes

MPI applications may start new processes through an interface to an external process manager.

`MPI_COMM Spawn` starts MPI processes and establishes communication with them, returning an inter-communicator. `MPI_COMM Spawn Multiple` starts several different binaries (or the same binary with different arguments), placing them in the same `MPI_COMM_WORLD` and returning an inter-communicator.

MPI uses the group abstraction to represent processes. A process is identified by a (group, rank) pair.

11.7.2 The Runtime Environment

The `MPI_COMM Spawn` and `MPI_COMM Spawn Multiple` routines provide an interface between MPI and the *runtime environment* of an MPI application. The difficulty is that there is an enormous range of runtime environments and application requirements, and MPI must not be tailored to any particular one.

MPI assumes, implicitly, the existence of an environment in which an application runs. It does not provide “operating system” services, such as a general ability to query what processes are running, to kill arbitrary processes, to find out properties of the runtime environment (how many processors, how much memory, etc.). Complex interaction of an MPI application with its runtime environment should be done through an environment-specific API.

At some low level, MPI must be able to interact with the runtime system, but the interaction is not visible at the application level and the details of the interaction are not specified by the MPI standard.

In many cases, it is impossible to keep environment-specific information out of the MPI interface without seriously compromising MPI functionality. To permit applications to take advantage of environment-specific functionality, many MPI routines take an `info` argument that allows an application to specify environment-specific information. There is a tradeoff between functionality and portability: applications that make use of environment-specific `info` are not portable.

MPI does not require the existence of an underlying “virtual machine” model, in which there is a consistent global view of an MPI application and an implicit “operating system” managing resources and processes. For instance, MPI processes spawned by one MPI process may not be visible to another; additional hosts added to the runtime environment by one MPI process may not be visible in another MPI process; MPI processes spawned by different processes may not be automatically distributed over available resources.

Interaction between MPI and the runtime environment is limited to the following areas:

- A process may start new processes with `MPI_COMM Spawn` and `MPI_COMM Spawn Multiple`.
- When a process spawns a child process, it may optionally use an `info` argument to tell the runtime environment where or how to start the process. This extra information may be opaque to MPI.
- An attribute `MPI_UNIVERSE_SIZE` (See Section 11.10.1) on `MPI_COMM_WORLD` tells a program how “large” the initial runtime environment is, namely how many processes

can usefully be started in all. One can subtract the size of `MPI_COMM_WORLD` from this value to find out how many processes might usefully be started in addition to those already running.

11.8 Process Manager Interface

11.8.1 Processes in MPI

A process is represented in MPI by a (group, rank) pair. A (group, rank) pair specifies a unique process but a process does not determine a unique (group, rank) pair, since a process may belong to several groups.

11.8.2 Starting Processes and Establishing Communication

The following routine starts a number of MPI processes and establishes communication with them, returning an inter-communicator.

Advice to users. It is possible in MPI to start an SPMD or MPMD application with a fixed number of processes after initialization by first starting one process and having that process start its siblings with `MPI_COMM_SPAWN`. This practice is discouraged primarily for reasons of performance. If possible, it is preferable to start all processes at once, as a single MPI application. (*End of advice to users.*)

`MPI_COMM_SPAWN(command, argv, maxprocs, info, root, comm, intercomm,
array_of_errcodes)`

IN	command	name of program to be spawned (string, significant only at root)
IN	argv	arguments to command (array of strings, significant only at root)
IN	maxprocs	maximum number of processes to start (integer, significant only at root)
IN	info	a set of key-value pairs telling the runtime system where and how to start the processes (handle, significant only at root)
IN	root	rank of process in which previous arguments are examined (integer)
IN	comm	intra-communicator containing group of spawning processes (handle)
OUT	intercomm	inter-communicator between original group and the newly spawned group (handle)
OUT	array_of_errcodes	one code per process (array of integers)

C binding

```
int MPI_Comm_spawn(const char *command, char *argv[], int maxprocs,
                  MPI_Info info, int root, MPI_Comm comm, MPI_Comm *intercomm,
                  int array_of_errcodes[])
```

Fortran 2008 binding

```

MPI_Comm_spawn(command, argv, maxprocs, info, root, comm, intercomm,
               array_of_errcodes, ierror)
CHARACTER(LEN=*), INTENT(IN) :: command, argv(*)
INTEGER, INTENT(IN) :: maxprocs, root
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Comm), INTENT(OUT) :: intercomm
INTEGER :: array_of_errcodes(*)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_SPAWN(COMMAND, ARGV, MAXPROCS, INFO, ROOT, COMM, INTERCOMM,
               ARRAY_OF_ERRCODES, IERROR)
CHARACTER*(*) COMMAND, ARGV(*)
INTEGER MAXPROCS, INFO, ROOT, COMM, INTERCOMM, ARRAY_OF_ERRCODES(*), IERROR

```

MPI_COMM_SPAWN tries to start `maxprocs` identical copies of the MPI program specified by `command`, establishing communication with them and returning an inter-communicator. The spawned processes are referred to as children. The children have their own **MPI_COMM_WORLD**, which is separate from that of the parents. **MPI_COMM_SPAWN** is collective over `comm`, and also may not return until **MPI_INIT** has been called in the children. Similarly, **MPI_INIT** in the children may not return until all parents have called **MPI_COMM_SPAWN**. In this sense, **MPI_COMM_SPAWN** in the parents and **MPI_INIT** in the children form a collective operation over the union of parent and child processes. The inter-communicator returned by **MPI_COMM_SPAWN** contains the parent processes in the local group and the child processes in the remote group. The ordering of processes in the local and remote groups is the same as the ordering of the group of the `comm` in the parents and of **MPI_COMM_WORLD** of the children, respectively. This inter-communicator can be obtained in the children through the function **MPI_COMM_GET_PARENT**.

Advice to users. An implementation may automatically establish communication before **MPI_INIT** is called by the children. Thus, completion of **MPI_COMM_SPAWN** in the parent does not necessarily mean that **MPI_INIT** has been called in the children (although the returned inter-communicator can be used immediately). (*End of advice to users.*)

The arguments are:

command: The `command` argument is a string containing the name of a program to be spawned. The string is null-terminated in C. In Fortran, leading and trailing spaces are stripped. MPI does not specify how to find the executable or how the working directory is determined. These rules are implementation-dependent and should be appropriate for the runtime environment.

Advice to implementors. The implementation should use a natural rule for finding executables and determining working directories. For instance, a homogeneous system with a global file system might look first in the working directory of the spawning process, or might search the directories in a `PATH` environment variable as do Unix shells. An implementation should document its rules for finding executables and determining working directories, and a high-quality implementation

should give the user some control over these rules. (*End of advice to implementors.*)

If the program named in `command` does not call `MPI_INIT`, but instead forks a process that calls `MPI_INIT`, the results are undefined. Implementations may allow this case to work but are not required to.

Advice to users. MPI does not say what happens if the program you start is a shell script and that shell script starts a program that calls `MPI_INIT`. Though some implementations may allow you to do this, they may also have restrictions, such as requiring that arguments supplied to the shell script be supplied to the program, or requiring that certain parts of the environment not be changed. (*End of advice to users.*)

argv: `argv` is an array of strings containing arguments that are passed to the program. The first element of `argv` is the first argument passed to `command`, not, as is conventional in some contexts, the command itself. The argument list is terminated by NULL in C and an empty string in Fortran. In Fortran, leading and trailing spaces are always stripped, so that a string consisting of all spaces is considered an empty string. The constant `MPI_ARGV_NULL` may be used in C and Fortran to indicate an empty argument list. In C this constant is the same as NULL.

Example 11.20. Examples of `argv` in C and Fortran

To run the program “ocean” with arguments “-gridfile” and “ocean1.grd” in C:

```
char command[] = "ocean";
char *argv[] = {"-gridfile", "ocean1.grd", NULL};
MPI_Comm_spawn(command, argv, ...);
```

or, if not everything is known at compile time:

```
char *command;
char **argv;
command = "ocean";
argv=(char **)malloc(3 * sizeof(char *));
argv[0] = "-gridfile";
argv[1] = "ocean1.grd";
argv[2] = NULL;
MPI_Comm_spawn(command, argv, ...);
```

In Fortran:

```
CHARACTER*25 command, argv(3)
command = 'ocean'
argv(1) = '-gridfile'
argv(2) = 'ocean1.grd'
argv(3) = ' '
call MPI_COMM_SPAWN(command, argv, ...)
```

Arguments are supplied to the program if this is allowed by the operating system. In C, the `MPI_COMM_SPAWN` argument `argv` differs from the `argv` argument of `main` in two respects. First, it is shifted by one element. Specifically, `argv[0]` of `main` is provided by the implementation and conventionally contains the name of the program (given by `command`). `argv[1]` of `main` corresponds to `argv[0]` in `MPI_COMM_SPAWN`, `argv[2]`

of `main` to `argv[1]` of `MPI_COMM_SPAWN`, etc. Passing an `argv` of `MPI_ARGV_NULL` to `MPI_COMM_SPAWN` results in `main` receiving `argc` of 1 and an `argv` whose element 0 is (conventionally) the name of the program. Second, `argv` of `MPI_COMM_SPAWN` must be null-terminated, so that its length can be determined.

If a Fortran implementation supplies routines that allow a program to obtain its arguments, the arguments may be available through that mechanism. In C, if the operating system does not support arguments appearing in `argv` of `main()`, the MPI implementation may add the arguments to the `argv` that is passed to `MPI_INIT`.

maxprocs: MPI tries to spawn `maxprocs` processes. If it is unable to spawn `maxprocs` processes, it raises an error of class `MPI_ERR_SPAWN`.

An implementation may allow the `info` argument to change the default behavior, such that if the implementation is unable to spawn all `maxprocs` processes, it may spawn a smaller number of processes instead of raising an error. In principle, the `info` argument may specify an arbitrary set $\{m_i : 0 \leq m_i \leq \text{maxprocs}\}$ of allowed values for the number of processes spawned. The set $\{m_i\}$ does not necessarily include the value `maxprocs`. If an implementation is able to spawn one of these allowed numbers of processes, `MPI_COMM_SPAWN` returns successfully and the number of spawned processes, m , is given by the size of the remote group of `intercomm`. If m is less than `maxproc`, reasons why the other processes were not spawned are given in `array_of_errcodes` as described below. If it is not possible to spawn one of the allowed numbers of processes, `MPI_COMM_SPAWN` raises an error of class `MPI_ERR_SPAWN`.

A spawn call with the default behavior is called *hard*. A spawn call for which fewer than `maxprocs` processes may be returned is called *soft*. See Section 11.8.4 for more information on the *soft* key for `info`.

Advice to users. By default, requests are hard and MPI errors are fatal. This means that by default there will be a fatal error if MPI cannot spawn all the requested processes. If you want the behavior “spawn as many processes as possible, up to N ,” you should do a soft spawn, where the set of allowed values $\{m_i\}$ is $\{0, \dots, N\}$. However, this is not completely portable, as implementations are not required to support soft spawning. (*End of advice to users.*)

info: The `info` argument to all of the routines in this chapter is an opaque handle of type `MPI_Info` in C and Fortran with the `mpi_f08` module and `INTEGER` in Fortran with the `mpi` module or the include file `mpif.h` (deprecated). It is a container for a number of user-specified (key,value) pairs. `key` and `value` are strings (null-terminated `char*` in C, `character*(*)` in Fortran). Routines to create and manipulate the `info` argument are described in Chapter 10.

For the `SPAWN` calls, `info` provides additional (and possibly implementation-dependent) instructions to MPI and the runtime system on how to start processes. An application may pass `MPI_INFO_NULL` in C or Fortran. Portable programs not requiring detailed control over process locations should use `MPI_INFO_NULL`.

MPI does not specify the content of the `info` argument, except to reserve a number of special key values (see Section 11.8.4). The `info` argument is quite flexible and could even be used, for example, to specify the executable and its command-line arguments. In this case the `command` argument to `MPI_COMM_SPAWN` could be empty. The

ability to do this follows from the fact that MPI does not specify how an executable is found, and the `info` argument can tell the runtime system where to “find” the executable "" (empty string). Of course, a program that does this will not be portable across MPI implementations.

root: All arguments before the `root` argument are examined only on the process whose rank in `comm` is equal to `root`. The value of these arguments on other processes is ignored.

array_of_errcodes: The `array_of_errcodes` is an array of length `maxprocs` in which MPI reports the status of each process that MPI was requested to start. If all `maxprocs` processes were spawned, `array_of_errcodes` is filled in with the value `MPI_SUCCESS`. If only m ($0 \leq m < \text{maxprocs}$) processes are spawned, m of the entries will contain `MPI_SUCCESS` and the rest will contain an implementation-specific error code indicating the reason MPI could not start the process. MPI does not specify which entries correspond to failed processes. An implementation may, for instance, fill in error codes in one-to-one correspondence with a detailed specification in the `info` argument. These error codes all belong to the error class `MPI_ERR_SPAWN` if there was no error in the argument list. In C or Fortran, an application may pass `MPI_ERRCODES_IGNORE` if it is not interested in the error codes.

Advice to implementors. `MPI_ERRCODES_IGNORE` in Fortran is a special type of constant, like `MPI_BOTTOM`. See the discussion in Section 2.5.4. (*End of advice to implementors.*)

`MPI_COMM_GET_PARENT(parent)`

OUT parent the parent communicator (handle)

C binding

`int MPI_Comm_get_parent(MPI_Comm *parent)`

Fortran 2008 binding

`MPI_Comm_get_parent(parent, ierror)`
 TYPE(MPI_Comm), INTENT(OUT) :: parent
 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

`MPI_COMM_GET_PARENT(PARENT, IERROR)`
 INTEGER PARENT, IERROR

If a process was started with `MPI_COMM_SPAWN` or `MPI_COMM_SPAWN_MULTIPLE`, `MPI_COMM_GET_PARENT` returns the “parent” inter-communicator of the current process. This parent inter-communicator is created implicitly inside of `MPI_INIT` and is the same inter-communicator returned by `SPAWN` in the parents.

If the process was not spawned, `MPI_COMM_GET_PARENT` returns `MPI_COMM_NULL`.

After the parent communicator is freed or disconnected, `MPI_COMM_GET_PARENT` returns `MPI_COMM_NULL`.

Advice to users. `MPI_COMM_GET_PARENT` returns a handle to a single inter-communicator. Calling `MPI_COMM_GET_PARENT` a second time returns a handle to

the same inter-communicator. Freeing the handle with `MPI_COMM_DISCONNECT` or `MPI_COMM_FREE` will cause other references to the inter-communicator to become invalid (dangling). Note that calling `MPI_COMM_FREE` on the parent communicator is not useful. (*End of advice to users.*)

Rationale. The desire of the Forum was to create a constant `MPI_COMM_PARENT` similar to `MPI_COMM_WORLD`. Unfortunately such a constant cannot be used (syntactically) as an argument to `MPI_COMM_DISCONNECT`, which is explicitly allowed. (*End of rationale.*)

11.8.3 Starting Multiple Executables and Establishing Communication

While `MPI_COMM_SPAWN` is sufficient for most cases, it does not allow the spawning of multiple binaries, or of the same binary with multiple sets of arguments. The following routine spawns multiple binaries or the same binary with multiple sets of arguments, establishing communication with them and placing them in the same `MPI_COMM_WORLD`.

```
MPI_COMM_SPAWN_MULTIPLE(count, array_of_commands, array_of_argv,
                        array_of_maxprocs, array_of_info, root, comm, intercomm,
                        array_of_errcodes)
```

IN	count	number of commands (positive integer, significant only at root)
IN	array_of_commands	programs to be executed (array of strings, significant only at root)
IN	array_of_argv	arguments for commands (array of array of strings, significant only at root)
IN	array_of_maxprocs	maximum number of processes to start for each command (array of integers, significant only at root)
IN	array_of_info	info objects telling the runtime system where and how to start processes (array of handles, significant only at root)
IN	root	rank of process in which previous arguments are examined (integer)
IN	comm	intra-communicator containing group of spawning processes (handle)
OUT	intercomm	inter-communicator between original group and the newly spawned group (handle)
OUT	array_of_errcodes	one error code per process (array of integers)

C binding

```
int MPI_Comm_spawn_multiple(int count, char *array_of_commands[],
                          char **array_of_argv[], const int array_of_maxprocs[],
                          const MPI_Info array_of_info[], int root, MPI_Comm comm,
                          MPI_Comm *intercomm, int array_of_errcodes[])
```

Fortran 2008 binding

```
MPI_Comm_spawn_multiple(count, array_of_commands, array_of_argv,
```

```

1         array_of_maxprocs, array_of_info, root, comm, intercomm,
2         array_of_errcodes, ierror)
3     INTEGER, INTENT(IN) :: count, array_of_maxprocs(*), root
4     CHARACTER(LEN=*) , INTENT(IN) :: array_of_commands(*),
5         array_of_argv(count, *)
6     TYPE(MPI_Info), INTENT(IN) :: array_of_info(*)
7     TYPE(MPI_Comm), INTENT(IN) :: comm
8     TYPE(MPI_Comm), INTENT(OUT) :: intercomm
9     INTEGER :: array_of_errcodes(*)
10    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

12 MPI_COMM_SPAWN_MULTIPLE(COUNT, ARRAY_OF_COMMANDS, ARRAY_OF_ARGV,
13     ARRAY_OF_MAXPROCS, ARRAY_OF_INFO, ROOT, COMM, INTERCOMM,
14     ARRAY_OF_ERRCODES, IERROR)
15     INTEGER COUNT, ARRAY_OF_MAXPROCS(*), ARRAY_OF_INFO(*), ROOT, COMM,
16     INTERCOMM, ARRAY_OF_ERRCODES(*), IERROR
17     CHARACTER*(*) ARRAY_OF_COMMANDS(*), ARRAY_OF_ARGV(COUNT, *)

```

`MPI_COMM_SPAWN_MULTIPLE` is identical to `MPI_COMM_SPAWN` except that there are multiple executable specifications. The first argument, `count`, gives the number of specifications. Each of the next four arguments are simply arrays of the corresponding arguments in `MPI_COMM_SPAWN`. For the Fortran version of `array_of_argv`, the element `array_of_argv(i,j)` is the `j`-th argument to command number `i`.

Rationale. This may seem backwards to Fortran programmers who are familiar with Fortran's column-major ordering. However, it is necessary to do it this way to allow `MPI_COMM_SPAWN` to sort out arguments. Note that the leading dimension of `array_of_argv` *must* be the same as `count`. Also note that Fortran rules for sequence association allow a different value in the first dimension; in this case, the sequence of array elements is interpreted by `MPI_COMM_SPAWN_MULTIPLE` as if the sequence is stored in an array defined with the first dimension set to `count`. This Fortran feature allows an implementor to define `MPI_ARGVS_NULL` (see below) with fixed dimensions, e.g., (1,1), or only with one dimension, e.g., (1). (*End of rationale.*)

Advice to users. The argument `count` is interpreted by MPI only at the root, as is `array_of_argv`. Since the leading dimension of `array_of_argv` is `count`, a nonpositive value of `count` at a nonroot node could theoretically cause a runtime bounds check error, even though `array_of_argv` should be ignored by the subroutine. If this happens, you should explicitly supply a reasonable value of `count` on the nonroot nodes. (*End of advice to users.*)

In any language, an application may use the constant `MPI_ARGVS_NULL` (which is likely to be `(char ***)0` in C) to specify that no arguments should be passed to any commands. The effect of setting individual elements of `array_of_argv` to `MPI_ARGV_NULL` is not defined. To specify arguments for some commands but not others, the commands without arguments should have a corresponding `argv` whose first element is null (`(char *)0` in C and empty string in Fortran). In Fortran at nonroot processes, the `count` argument must be set to a value that is consistent with the provided `array_of_argv` although the content of these arguments has no meaning for this operation.

All of the spawned processes have the same `MPI_COMM_WORLD`. Their ranks in `MPI_COMM_WORLD` correspond directly to the order in which the commands are specified in `MPI_COMM Spawn Multiple`. Assume that m_1 MPI processes are generated by the first command, m_2 by the second, etc. The MPI processes corresponding to the first command have ranks $0, 1, \dots, m_1 - 1$ in `MPI_COMM_WORLD`. The MPI processes in the second command have ranks $m_1, m_1 + 1, \dots, m_1 + m_2 - 1$ in `MPI_COMM_WORLD`. The MPI processes in the third have ranks $m_1 + m_2, m_1 + m_2 + 1, \dots, m_1 + m_2 + m_3 - 1$ in `MPI_COMM_WORLD`, etc.

Advice to users. Calling `MPI_COMM Spawn` multiple times would create many sets of children with different `MPI_COMM_WORLD`s whereas `MPI_COMM Spawn Multiple` creates children with a single `MPI_COMM_WORLD`, so the two methods are not completely equivalent. There are also two performance-related reasons why, if you need to spawn multiple executables, you may want to use `MPI_COMM Spawn Multiple` instead of calling `MPI_COMM Spawn` several times. First, spawning several things at once may be faster than spawning them sequentially. Second, in some implementations, communication between processes spawned at the same time may be faster than communication between processes spawned separately. (*End of advice to users.*)

The `array_of_errcodes` argument is a 1-dimensional array of size $\sum_{i=1}^{count} n_i$, where n_i is the i -th element of `array_of_maxprocs`. Command number i corresponds to the n_i contiguous slots in this array from element $\sum_{j=1}^{i-1} n_j$ to $[\sum_{j=1}^i n_j] - 1$. Error codes are treated the same as with `MPI_COMM Spawn`.

Example 11.21. Examples of `array_of_argv` in C and Fortran

To run the program “ocean” with arguments “-gridfile” and “ocean1.grd” and the program “atmos” with argument “atmos.grd” in C:

```
char *array_of_commands[2] = {"ocean", "atmos"};
char **array_of_argv[2];
char *argv0[] = {"-gridfile", "ocean1.grd", (char *)0};
char *argv1[] = {"atmos.grd", (char *)0};
array_of_argv[0] = argv0;
array_of_argv[1] = argv1;
MPI_Comm_spawn_multiple(2, array_of_commands, array_of_argv, ...);
```

Here is how you do it in Fortran:

```
CHARACTER*25 commands(2), array_of_argv(2, 3)
commands(1) = 'ocean'
array_of_argv(1, 1) = '-gridfile'
array_of_argv(1, 2) = 'ocean1.grd'
array_of_argv(1, 3) = ' '

commands(2) = 'atmos'
array_of_argv(2, 1) = 'atmos.grd'
array_of_argv(2, 2) = ' '

call MPI_COMM_SPAWN_MULTIPLE(2, commands, array_of_argv, ...)
```

11.8.4 Reserved Keys

The following keys are reserved. An implementation is not required to interpret these keys, but if it does interpret the key, it must provide the functionality described.

"host": Value is a hostname. The format of the hostname is determined by the implementation.

"arch": Value is an architecture name. Valid architecture names and what they mean are determined by the implementation.

"wdir": Value is the name of a directory on a machine on which the spawned process(es) execute(s). This directory is made the working directory of the executing process(es). The format of the directory name is determined by the implementation.

"path": Value is a directory or set of directories where the implementation should look for the executable. The format of "path" is determined by the implementation.

"file": Value is the name of a file in which additional information is specified. The format of the filename and internal format of the file are determined by the implementation.

"mpi_initial_errhandler": Value is the name of an errhandler that will be set as the initial error handler. The "mpi_initial_errhandler" key can take the case insensitive values "mpi_errors_are_fatal", "mpi_errors_abort", and "mpi_errors_return" representing the pre-defined MPI error handlers ([MPI_ERRORS_ARE_FATAL](#)—the default, [MPI_ERRORS_ABORT](#), and [MPI_ERRORS_RETURN](#), respectively). Other, nonstandard values may be supported by the implementation, which should document the resultant behavior.

"mpi_memory_alloc_kinds" (string, default: "mpi,system"): Value is a comma separated list of memory allocation kinds. Support for these memory allocation kinds is requested from the MPI library (see Section 11.4.3).

"soft": Value specifies a set of numbers that are allowed values for the number of processes that [MPI_COMM_SPAWN](#) (et al.) may create. The format of the value is a comma-separated list of Fortran-90 triplets each of which specifies a set of integers and that together specify the set formed by the union of these sets. Negative values in this set and values greater than `maxprocs` are ignored. MPI will spawn the largest number of processes it can, consistent with some number in the set. The order in which triplets are given is not significant.

By Fortran-90 triplets, we mean:

1. `a` means a
2. `a:b` means $a, a + 1, a + 2, \dots, b$
3. `a:b:c` means $a, a + c, a + 2c, \dots, a + ck$, where for $c > 0$, k is the largest integer for which $a + ck \leq b$ and for $c < 0$, k is the largest integer for which $a + ck \geq b$. If $b > a$ then c must be positive. If $b < a$ then c must be negative.

Examples:

1. `a:b` gives a range between a and b
2. `0:N` gives full "soft" functionality

3. 1,2,4,8,16,32,64,128,256,512,1024,2048,4096 allows a power-of-two number of processes.
4. 2:10000:2 allows an even number of processes up to a maximum of 10,000 processes.
5. 2:10:2,7 allows 2, 4, 6, 7, 8, or 10 processes.

11.8.5 Spawn Example

Example 11.22. Manager-worker Example Using `MPI_COMM_SPAWN`

```

/* manager */
#include <stdio.h>
#include "mpi.h"
int main(int argc, char *argv[])
{
    int world_size, universe_size, *universe_sizep, flag;
    MPI_Comm everyone;          /* inter-communicator */
    char worker_program[100];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    if (world_size != 1)    error("Top heavy with management");

    MPI_Comm_get_attr(MPI_COMM_WORLD, MPI_UNIVERSE_SIZE,
                      &universe_sizep, &flag);
    if (!flag) {
        printf("This MPI does not support UNIVERSE_SIZE.\n"
              "How many processes total?");
        scanf("%d", &universe_size);
    } else universe_size = *universe_sizep;
    if (universe_size == 1) error("No room to start workers");

    /*
     * Now spawn the workers. Note that there is a run-time determination
     * of what type of worker to spawn, and presumably this calculation
     * must be done at run time and cannot be calculated before starting
     * the program. If everything is known when the application is
     * first started, it is generally better to start them all at once
     * in a single MPI_COMM_WORLD.
     */

    choose_worker_program(worker_program);
    MPI_Comm_spawn(worker_program, MPI_ARGV_NULL, universe_size-1,
                  MPI_INFO_NULL, 0, MPI_COMM_SELF, &everyone,
                  MPI_ERRCODES_IGNORE);

    /*
     * Parallel code here. The communicator "everyone" can be used
     * to communicate with the spawned processes, which have ranks 0,..
     * MPI_UNIVERSE_SIZE-1 in the remote group of the inter-communicator
     * "everyone".
     */
}

```

```

1
2     MPI_Finalize();
3     return 0;
4 }
5
6 /* worker */
7
8 #include "mpi.h"
9 int main(int argc, char *argv[])
10 {
11     int size;
12     MPI_Comm parent;
13     MPI_Init(&argc, &argv);
14     MPI_Comm_get_parent(&parent);
15     if (parent == MPI_COMM_NULL) error("No parent!");
16     MPI_Comm_remote_size(parent, &size);
17     if (size != 1) error("Something's wrong with the parent");
18
19     /*
20      * Parallel code here.
21      * The manager is represented as the process with rank 0 in (the
22      * remote group of) the parent communicator. If the workers need
23      * to communicate among themselves, they can use MPI_COMM_WORLD.
24      */
25
26     MPI_Finalize();
27     return 0;
28 }

```

11.9 Establishing Communication

This section provides functions that establish communication between two sets of MPI processes that do not share a communicator.

Some situations in which these functions are useful are:

1. Two parts of an application that are started independently need to communicate.
2. A visualization tool wants to attach to a running process.
3. A server wants to accept connections from multiple clients. Both clients and server may be parallel programs.

In each of these situations, MPI must establish communication channels where none existed before, and there is no parent/child relationship. The routines described in this section establish communication between the two sets of processes by creating an MPI inter-communicator, where the two groups of the inter-communicator are the original sets of processes.

Establishing contact between two groups of processes that do not share an existing communicator is a collective but asymmetric process. One group of processes indicates its willingness to accept connections from other groups of processes. We will call this group the (parallel) *server*, even if this is not a client/server type of application. The other group connects to the server; we will call it the (parallel) *client*.

Advice to users. While the names *client* and *server* are used throughout this section, MPI does not guarantee the traditional robustness of client/server systems. The functionality described in this section is intended to allow two cooperating parts of the same application to communicate with one another. For instance, a client that gets a segmentation fault and dies, or one that does not participate in a collective operation may cause a server to crash or hang. (*End of advice to users.*)

11.9.1 Names, Addresses, Ports, and All That

Almost all of the complexity in MPI client/server routines addresses the question “how does the client find out how to contact the server?” The difficulty, of course, is that there is no existing communication channel between them, yet they must somehow agree on a rendezvous point where they will establish communication.

Agreeing on a rendezvous point always involves a third party. The third party may itself provide the rendezvous point or may communicate rendezvous information from server to client. Complicating matters might be the fact that it is not important to the client which particular server it contacts, only that it be able to get in touch with one that can handle its request.

Ideally, MPI can accommodate a wide variety of run-time systems while retaining the ability to write simple, portable code. The following should be compatible with MPI:

- The server resides at a well-known internet address host:port.
- The server prints out an address to the terminal; the user gives this address to the client program.
- The server places the address information on a nameserver, where it can be retrieved with an agreed-upon name.
- The server to which the client connects is actually a broker, acting as a middleman between the client and the real server.

MPI does not require a nameserver, so not all implementations will be able to support all of the above scenarios. However, MPI provides an optional nameserver interface, and is compatible with external name servers.

A `port_name` is a *system-supplied* string that encodes a low-level network address at which a server can be contacted. Typically this is an IP address and a port number, but an implementation is free to use any protocol. The server establishes a `port_name` with the `MPI_OPEN_PORT` routine. It accepts a connection to a given port with `MPI_COMM_ACCEPT`. A client uses `port_name` to connect to the server.

By itself, the `port_name` mechanism is completely portable, but it may be clumsy to use because of the necessity to communicate `port_name` to the client. It would be more convenient if a server could specify that it be known by an *application-supplied* `service_name` so that the client could connect to that `service_name` without knowing the `port_name`.

An MPI implementation may allow the server to publish a (`port_name`, `service_name`) pair with `MPI_PUBLISH_NAME` and the client to retrieve the port name from the service name with `MPI_LOOKUP_NAME`. This allows three levels of portability, with increasing levels of functionality.

1. Applications that do not rely on the ability to publish names are the most portable. Typically the `port_name` must be transferred “by hand” from server to client.

2. Applications that use the `MPI_PUBLISH_NAME` mechanism are completely portable among implementations that provide this service. To be portable among all implementations, these applications should have a fall-back mechanism that can be used when names are not published.
3. Applications may ignore MPI's name publishing functionality and use their own mechanism (possibly system-supplied) to publish names. This allows arbitrary flexibility but is not portable.

11.9.2 Server Routines

A server makes itself available with two routines. First it must call `MPI_OPEN_PORT` to establish a port at which it may be contacted. Secondly it must call `MPI_COMM_ACCEPT` to accept connections from clients.

`MPI_OPEN_PORT(info, port_name)`

IN	info	implementation-specific information on how to establish an address (handle)
OUT	port_name	newly established port (string)

C binding

```
int MPI_Open_port(MPI_Info info, char *port_name)
```

Fortran 2008 binding

```
MPI_Open_port(info, port_name, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  CHARACTER(LEN=MPI_MAX_PORT_NAME), INTENT(OUT) :: port_name
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_OPEN_PORT(INFO, PORT_NAME, IERROR)
  INTEGER INFO, IERROR
  CHARACTER*(*) PORT_NAME
```

This function establishes a network address, encoded in the `port_name` string, at which the server will be able to accept connections from clients. `port_name` is supplied by the system, possibly using information in the `info` argument.

MPI copies a system-supplied port name into `port_name`. `port_name` identifies the newly opened port and can be used by a client to contact the server. The maximum size of the string that may be supplied by the system is `MPI_MAX_PORT_NAME`.

Advice to users. The system copies the port name into `port_name`. The application must pass a buffer of sufficient size to hold this value. (*End of advice to users.*)

`port_name` is essentially a network address. It is unique within the communication universe to which it belongs (determined by the implementation), and may be used by any client within that communication universe. For instance, if it is an internet (host:port) address, it will be unique on the internet. If it is a low level switch address on an IBM SP, it will be unique to that SP.

Advice to implementors. These examples are not meant to constrain implementations. A `port_name` could, for instance, contain a user name or the name of a batch job, as long as it is unique within some well-defined communication domain. The larger the communication domain, the more useful MPI's client/server functionality will be. (*End of advice to implementors.*)

The precise form of the address is implementation defined. For instance, an internet address may be a host name or IP address, or anything that the implementation can decode into an IP address. A port name may be reused after it is freed with `MPI_CLOSE_PORT` and released by the system.

Advice to implementors. Since the user may type in `port_name` by hand, it is useful to choose a form that is easily readable and does not have embedded spaces. (*End of advice to implementors.*)

`info` may be used to tell the implementation how to establish the address. It may, and usually will, be `MPI_INFO_NULL` in order to get the implementation defaults.

`MPI_CLOSE_PORT(port_name)`

IN	<code>port_name</code>	a port (string)
----	------------------------	-----------------

C binding

```
int MPI_Close_port(const char *port_name)
```

Fortran 2008 binding

```
MPI_Close_port(port_name, ierror)
  CHARACTER(LEN=*) INTENT(IN) :: port_name
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_CLOSE_PORT(PORT_NAME, IERROR)
  CHARACTER*(*) PORT_NAME
  INTEGER IERROR
```

This function releases the network address represented by `port_name`.

`MPI_COMM_ACCEPT(port_name, info, root, comm, newcomm)`

IN	<code>port_name</code>	port name (string, significant only at root)
IN	<code>info</code>	implementation-dependent information (handle, significant only at root)
IN	<code>root</code>	rank of root in <code>comm</code> (integer)
IN	<code>comm</code>	intra-communicator over which call is collective (handle)
OUT	<code>newcomm</code>	inter-communicator with client as remote group (handle)

C binding

```
int MPI_Comm_accept(const char *port_name, MPI_Info info, int root,
  MPI_Comm comm, MPI_Comm *newcomm)
```

Fortran 2008 binding

```

MPI_Comm_accept(port_name, info, root, comm, newcomm, ierror)
  CHARACTER(LEN=*), INTENT(IN) :: port_name
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_ACCEPT(PORT_NAME, INFO, ROOT, COMM, NEWCOMM, IERROR)
  CHARACTER*(*) PORT_NAME
  INTEGER INFO, ROOT, COMM, NEWCOMM, IERROR

```

MPI_COMM_ACCEPT establishes communication with a client. It is collective over the calling communicator. It returns an inter-communicator that allows communication with the client.

The `port_name` must have been established through a call to **MPI_OPEN_PORT**.

`info` can be used to provide directives that may influence the behavior of the call to **MPI_COMM_ACCEPT**.

11.9.3 Client Routines

There is only one routine on the client side.

```

MPI_COMM_CONNECT(port_name, info, root, comm, newcomm)

```

IN	<code>port_name</code>	network address (string, significant only at root)
IN	<code>info</code>	implementation-dependent information (handle, significant only at root)
IN	<code>root</code>	rank of root in <code>comm</code> (integer)
IN	<code>comm</code>	intra-communicator over which call is collective (handle)
OUT	<code>newcomm</code>	inter-communicator with server as remote group (handle)

C binding

```

int MPI_Comm_connect(const char *port_name, MPI_Info info, int root,
                    MPI_Comm comm, MPI_Comm *newcomm)

```

Fortran 2008 binding

```

MPI_Comm_connect(port_name, info, root, comm, newcomm, ierror)
  CHARACTER(LEN=*), INTENT(IN) :: port_name
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```


Fortran binding

```

MPI_COMM_CONNECT(PORT_NAME, INFO, ROOT, COMM, NEWCOMM, IERROR)
  CHARACTER*(*) PORT_NAME
  INTEGER INFO, ROOT, COMM, NEWCOMM, IERROR

```

This routine establishes communication with a server specified by `port_name`. It is collective over the calling communicator and returns an inter-communicator in which the remote group participated in an `MPI_COMM_ACCEPT`.

If the named port does not exist (or has been closed), `MPI_COMM_CONNECT` raises an error of class `MPI_ERR_PORT`.

If the port exists, but does not have a pending `MPI_COMM_ACCEPT`, the connection attempt will eventually time out after an implementation-defined time, or succeed when the server calls `MPI_COMM_ACCEPT`. In the case of a time out, `MPI_COMM_CONNECT` raises an error of class `MPI_ERR_PORT`.

Advice to implementors. The time out period may be arbitrarily short or long. However, a high-quality implementation will try to queue connection attempts so that a server can handle simultaneous requests from several clients. A high-quality implementation may also provide a mechanism, through the `info` arguments to `MPI_OPEN_PORT`, `MPI_COMM_ACCEPT`, and/or `MPI_COMM_CONNECT`, for the user to control timeout and queuing behavior. (*End of advice to implementors.*)

MPI provides no guarantee of fairness in servicing connection attempts. That is, connection attempts are not necessarily satisfied in the order they were initiated and competition from other connection attempts may prevent a particular connection attempt from being satisfied.

`port_name` is the address of the server. It must be the same as the name returned by `MPI_OPEN_PORT` on the server. Some freedom is allowed here. If there are equivalent forms of `port_name`, an implementation may accept them as well. For instance, if `port_name` is (hostname:port), an implementation may accept (ip_address:port) as well.

11.9.4 Name Publishing

The routines in this section provide a mechanism for publishing names. A (`service_name`, `port_name`) pair is published by the server, and may be retrieved by a client using the `service_name` only. An MPI implementation defines the *scope* of the `service_name`, that is, the domain over which the `service_name` can be retrieved. If the domain is the empty set, that is, if no client can retrieve the information, then we say that name publishing is not supported. Implementations should document how the scope is determined. High-quality implementations will give some control to users through the `info` arguments to name publishing functions. Examples are given in the descriptions of individual functions.

```

MPI_PUBLISH_NAME(service_name, info, port_name)

```

IN	<code>service_name</code>	a service name to associate with the port (string)
IN	<code>info</code>	implementation-specific information (handle)
IN	<code>port_name</code>	a port name (string)

C binding

```
int MPI_Publish_name(const char *service_name, MPI_Info info,
                    const char *port_name)
```

Fortran 2008 binding

```
MPI_Publish_name(service_name, info, port_name, ierror)
  CHARACTER(LEN=*) INTENT(IN) :: service_name, port_name
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_PUBLISH_NAME(SERVICE_NAME, INFO, PORT_NAME, IERROR)
  CHARACTER*(*) SERVICE_NAME, PORT_NAME
  INTEGER INFO, IERROR
```

This routine publishes the pair (`port_name`, `service_name`) so that an application may retrieve a system-supplied `port_name` using a well-known `service_name`.

The implementation must define the *scope* of a published service name, that is, the domain over which the service name is unique, and conversely, the domain over which the (`port_name`, `service_name`) pair may be retrieved. For instance, a service name may be unique to a job (where job is defined by a distributed operating system or batch scheduler), unique to a machine, or unique to a Kerberos realm. The scope may depend on the `info` argument to `MPI_PUBLISH_NAME`.

MPI permits publishing more than one `service_name` for a single `port_name`. On the other hand, if `service_name` has already been published within the scope determined by `info`, the behavior of `MPI_PUBLISH_NAME` is undefined. An MPI implementation may, through a mechanism in the `info` argument to `MPI_PUBLISH_NAME`, provide a way to allow multiple servers with the same service in the same scope. In this case, an implementation-defined policy will determine which of several port names is returned by `MPI_LOOKUP_NAME`.

Note that while `service_name` has a limited scope, determined by the implementation, `port_name` always has global scope within the communication universe used by the implementation (i.e., it is globally unique).

`port_name` should be the name of a port established by `MPI_OPEN_PORT` and not yet released by `MPI_CLOSE_PORT`. If it is not, the result is undefined.

Advice to implementors. In some cases, an MPI implementation may use a name service that a user can also access directly. In this case, a name published by MPI could easily conflict with a name published by a user. In order to avoid such conflicts, MPI implementations should mangle service names so that they are unlikely to conflict with user code that makes use of the same service. Such name mangling will of course be completely transparent to the user.

The following situation is problematic but unavoidable, if we want to allow implementations to use nameservers. Suppose there are multiple instances of “ocean” running on a machine. If the scope of a service name is confined to a job, then multiple oceans can coexist. If an implementation provides site-wide scope, however, multiple instances are not possible as all calls to `MPI_PUBLISH_NAME` after the first may fail. There is no universal solution to this.

To handle these situations, a high-quality implementation should make it possible to limit the domain over which names are published. (*End of advice to implementors.*)

```

MPI_UNPUBLISH_NAME(service_name, info, port_name)
IN      service_name      a service name (string)
IN      info              implementation-specific information (handle)
IN      port_name         a port name (string)

```

C binding

```

int MPI_Unpublish_name(const char *service_name, MPI_Info info,
                      const char *port_name)

```

Fortran 2008 binding

```

MPI_Unpublish_name(service_name, info, port_name, ierror)
  CHARACTER(LEN=*), INTENT(IN) :: service_name, port_name
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_UNPUBLISH_NAME(SERVICE_NAME, INFO, PORT_NAME, IERROR)
  CHARACTER*(*) SERVICE_NAME, PORT_NAME
  INTEGER INFO, IERROR

```

This routine unpublishes a service name that has been previously published. Attempting to unpublish a name that has not been published or has already been unpublished is erroneous and is indicated by the error class `MPI_ERR_SERVICE`.

All published names must be unpublished before the corresponding port is closed and before the publishing process exits. The behavior of `MPI_UNPUBLISH_NAME` is implementation dependent when a process tries to unpublish a name that it did not publish.

If the `info` argument was used with `MPI_PUBLISH_NAME` to tell the implementation how to publish names, the implementation may require that `info` passed to `MPI_UNPUBLISH_NAME` contain information to tell the implementation how to unpublish a name.

```

MPI_LOOKUP_NAME(service_name, info, port_name)
IN      service_name      a service name (string)
IN      info              implementation-specific information (handle)
OUT     port_name         a port name (string)

```

C binding

```

int MPI_Lookup_name(const char *service_name, MPI_Info info, char *port_name)

```

Fortran 2008 binding

```

MPI_Lookup_name(service_name, info, port_name, ierror)
  CHARACTER(LEN=*), INTENT(IN) :: service_name
  TYPE(MPI_Info), INTENT(IN) :: info
  CHARACTER(LEN=MPI_MAX_PORT_NAME), INTENT(OUT) :: port_name
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_LOOKUP_NAME(SERVICE_NAME, INFO, PORT_NAME, IERROR)
  CHARACTER*(*) SERVICE_NAME, PORT_NAME
  INTEGER INFO, IERROR

```

This function retrieves a `port_name` published by `MPI_PUBLISH_NAME` with `service_name`. If `service_name` has not been published, it raises an error in the error class `MPI_ERR_NAME`. The application must supply a `port_name` buffer large enough to hold the largest possible port name (see discussion above under `MPI_OPEN_PORT`).

If an implementation allows multiple entries with the same `service_name` within the same scope, a particular `port_name` is chosen in a way determined by the implementation.

If the `info` argument was used with `MPI_PUBLISH_NAME` to tell the implementation how to publish names, a similar `info` argument may be required for `MPI_LOOKUP_NAME`.

11.9.5 Reserved Key Values

The following key values are reserved. An implementation is not required to interpret these key values, but if it does interpret the key value, it must provide the functionality described.

"ip_address": Value contains IP address at which to establish a port. If the address is not a valid IP address of the host on which the `MPI_OPEN_PORT` call is made, the results are undefined. (Reserved for `MPI_OPEN_PORT` only).

"ip_port": Value contains IP port number at which to establish a port. (Reserved for `MPI_OPEN_PORT` only).

11.9.6 Client/Server Examples**Example 11.23.** Printing Port Name Example—Completely Portable.

The following example shows the simplest way to use the client/server interface. It does not use service names at all.

On the server side:

```

char myport[MPI_MAX_PORT_NAME];
MPI_Comm intercomm;
/* ... */
MPI_Open_port(MPI_INFO_NULL, myport);
printf("port name is: %s\n", myport);

MPI_Comm_accept(myport, MPI_INFO_NULL, 0, MPI_COMM_SELF, &intercomm);
/* do something with intercomm */

```

The server prints out the port name to the terminal and the user must type it in when starting up the client (assuming the MPI implementation supports stdin such that this works). On the client side:

```

MPI_Comm intercomm;
char name[MPI_MAX_PORT_NAME];
printf("enter port name: ");
gets(name);
MPI_Comm_connect(name, MPI_INFO_NULL, 0, MPI_COMM_SELF, &intercomm);

```

Example 11.24. Ocean/Atmosphere—Relies on Name Publishing

In this example, the “ocean” application is the “server” side of a coupled ocean-atmosphere climate model. It assumes that the MPI implementation publishes names.

```

char port_name[MPI_MAX_PORT_NAME];
MPI_Comm intercomm;
/* ... */
MPI_Open_port(MPI_INFO_NULL, port_name);
MPI_Publish_name("ocean", MPI_INFO_NULL, port_name);

MPI_Comm_accept(port_name, MPI_INFO_NULL, 0, MPI_COMM_SELF,
                &intercomm);
/* do something with intercomm */
MPI_Unpublish_name("ocean", MPI_INFO_NULL, port_name);

```

On the client side:

```

MPI_Lookup_name("ocean", MPI_INFO_NULL, port_name);
MPI_Comm_connect(port_name, MPI_INFO_NULL, 0, MPI_COMM_SELF,
                 &intercomm);

```

Example 11.25. Simple Client-Server Example

This is a simple example; the server accepts only a single connection at a time and serves that connection until the client requests to be disconnected. The server is a single process. Here is the server. It accepts a single connection and then processes data until it receives a message with tag 1. A message with tag 0 tells the server to exit.

```

#include "mpi.h"
int main(int argc, char *argv[])
{
    MPI_Comm client;
    MPI_Status status;
    char port_name[MPI_MAX_PORT_NAME];
    double buf[MAX_DATA];
    int size, again;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size != 1) error(FATAL, "Server too big");
    MPI_Open_port(MPI_INFO_NULL, port_name);
    printf("server available at %s\n", port_name);
    while (1) {
        MPI_Comm_accept(port_name, MPI_INFO_NULL, 0, MPI_COMM_WORLD,
                        &client);

        again = 1;
        while (again) {
            MPI_Recv(buf, MAX_DATA, MPI_DOUBLE,
                    MPI_ANY_SOURCE, MPI_ANY_TAG, client, &status);
            switch (status.MPI_TAG) {
                case 0: MPI_Comm_free(&client);
                        MPI_Close_port(port_name);
                        MPI_Finalize();
            }
        }
    }
}

```

```

1         return 0;
2     case 1: MPI_Comm_disconnect(&client);
3             again = 0;
4             break;
5     case 2: /* do something */
6             ...
7     default:
8             /* Unexpected message type */
9             MPI_Abort(MPI_COMM_WORLD, 1);
10    }
11 }
12 }

```

Here is the client.

```

14 #include "mpi.h"
15 int main(int argc, char *argv[])
16 {
17     MPI_Comm server;
18     int done = 0;
19     double buf[MAX_DATA];
20     char port_name[MPI_MAX_PORT_NAME];
21
22     MPI_Init(&argc, &argv);
23     strcpy(port_name, argv[1]); /* assume server's name is cmd-line arg */
24
25     MPI_Comm_connect(port_name, MPI_INFO_NULL, 0, MPI_COMM_WORLD,
26                     &server);
27
28     while (!done) {
29         tag = 2; /* Action to perform */
30         MPI_Send(buf, n, MPI_DOUBLE, 0, tag, server);
31         /* etc */
32     }
33     MPI_Send(buf, 0, MPI_DOUBLE, 0, 1, server);
34     MPI_Comm_disconnect(&server);
35     MPI_Finalize();
36     return 0;
37 }

```

11.10 Other Functionality

11.10.1 Universe Size

Many “dynamic” MPI applications are expected to exist in a static runtime environment, in which resources have been allocated before the application is run. When running one of these quasi-static applications, the user (or possibly a batch system) will usually specify a number of processes to start and a total number of processes that are expected. An application simply needs to know how many slots there are, i.e., how many processes it should spawn.

MPI provides an attribute on `MPI_COMM_WORLD`, `MPI_UNIVERSE_SIZE`, that allows

the application to obtain this information in a portable manner. This attribute indicates the total number of processes that are expected. In Fortran, the attribute is the integer value. In C, the attribute is a pointer to the integer value. An application typically subtracts the size of `MPI_COMM_WORLD` from `MPI_UNIVERSE_SIZE` to find out how many processes it should spawn. `MPI_UNIVERSE_SIZE` is initialized in `MPI_INIT` and is not changed by MPI. If defined, it has the same value on all processes of `MPI_COMM_WORLD`. `MPI_UNIVERSE_SIZE` is determined by the application startup mechanism in a way not specified by MPI. (The size of `MPI_COMM_WORLD` is another example of such a parameter.)

Possibilities for how `MPI_UNIVERSE_SIZE` might be set include:

- A `-universe_size` argument to a program that starts MPI processes.
- Automatic interaction with a batch scheduler to figure out how many processors have been allocated to an application.
- An environment variable set by the user.
- Extra information passed to `MPI_COMM_SPAWN` through the `info` argument.

An implementation must document how `MPI_UNIVERSE_SIZE` is set. An implementation may not support the ability to set `MPI_UNIVERSE_SIZE`, in which case the attribute `MPI_UNIVERSE_SIZE` is not set.

`MPI_UNIVERSE_SIZE` is a recommendation, not necessarily a hard limit. For instance, some implementations may allow an application to spawn 50 processes per processor, if they are requested. However, it is likely that the user only wants to spawn one process per processor.

`MPI_UNIVERSE_SIZE` is assumed to have been specified when an application was started, and is in essence a portable mechanism to allow the user to pass to the application (through the MPI process startup mechanism, such as `mpirun`) a piece of critical runtime information. Note that no interaction with the runtime environment is required. If the runtime environment changes size while an application is running, `MPI_UNIVERSE_SIZE` is not updated, and the application must find out about the change through direct communication with the runtime system.

11.10.2 Singleton MPI Initialization

A high-quality implementation will allow any process (including those not started with a “parallel application” mechanism) to become an MPI process by calling `MPI_INIT`, `MPI_INIT_THREAD`, or `MPI_SESSION_INIT`. Such a process can then connect to other MPI processes using the `MPI_COMM_ACCEPT` and `MPI_COMM_CONNECT` routines, or spawn other MPI processes. MPI does not mandate this behavior, but strongly encourages it where technically feasible.

Advice to implementors. Special coordination is required to start MPI processes belonging to the same `MPI_COMM_WORLD` in the case of the World Model, or the same “`mpi://WORLD`” process set in the Sessions Model. The processes must be started at the “same” time, they must have a mechanism to establish communication, etc. Either the user or the operating system must take special steps beyond simply starting processes.

Considering the World Model, when an application enters `MPI_INIT`, clearly it must be able to determine if these special steps were taken. If a process enters `MPI_INIT` and

determines that no special steps were taken (i.e., it has not been given the information to form an `MPI_COMM_WORLD` with other processes) it succeeds and forms a singleton MPI program, that is, one in which `MPI_COMM_WORLD` has size 1.

In some implementations, MPI may not be able to function without an “MPI environment.” For example, MPI may require that daemons be running or MPI may not be able to work at all on the front-end of an MPP. In this case, an MPI implementation may either

1. Create the environment (e.g., start a daemon) or
2. Raise an error if it cannot create the environment and the environment has not been started independently.

A high-quality implementation will try to create a singleton MPI process and not raise an error. (*End of advice to implementors.*)

11.10.3 `MPI_APPNUM`

There is a predefined attribute `MPI_APPNUM` of `MPI_COMM_WORLD`. In Fortran, the attribute is an integer value. In C, the attribute is a pointer to an integer value. If a process was spawned with `MPI_COMM Spawn_MULTIPLE`, `MPI_APPNUM` is the command number that generated the current process. Numbering starts from zero. If a process was spawned with `MPI_COMM Spawn`, it will have `MPI_APPNUM` equal to zero.

Additionally, if the process was not started by a spawn call, but by an implementation-specific startup mechanism that can handle multiple process specifications, `MPI_APPNUM` should be set to the number of the corresponding process specification. In particular, if it is started with

```
mpirexec spec0 [: spec1 : spec2 : ...]
```

`MPI_APPNUM` should be set to the number of the corresponding specification.

If an application was not spawned with `MPI_COMM Spawn` or `MPI_COMM Spawn_MULTIPLE`, and `MPI_APPNUM` does not make sense in the context of the implementation-specific startup mechanism, `MPI_APPNUM` is not set.

MPI implementations may optionally provide a mechanism to override the value of `MPI_APPNUM` through the info argument. MPI reserves the following key for all `Spawn` calls.

"appnum": Value contains an integer that overrides the default value for `MPI_APPNUM` in the child.

Rationale. When a single application is started, it is able to figure out how many processes there are by looking at the size of `MPI_COMM_WORLD`. An application consisting of multiple SPMD sub-applications has no way to find out how many sub-applications there are and to which sub-application the process belongs. While there are ways to figure it out in special cases, there is no general mechanism. `MPI_APPNUM` provides such a general mechanism. (*End of rationale.*)

11.10.4 Releasing Connections

Before a client and a server connect, they are independent MPI applications. An error in one does not affect the other. After establishing a connection with `MPI_COMM_Connect`

and `MPI_COMM_ACCEPT`, an error in one may affect the other. It is desirable for a client and a server to be able to disconnect, so that an error in one will not affect the other. Similarly, it might be desirable for a parent and child to disconnect, so that errors in the child do not affect the parent, or vice-versa.

- Two processes are **connected** if there is a communication path (direct or indirect) between them. More precisely:
 1. Two processes are connected if
 - a) they both belong to the same communicator (inter- or intra-, including `MPI_COMM_WORLD`) *or*
 - b) they have previously belonged to a communicator that was freed with `MPI_COMM_FREE` instead of `MPI_COMM_DISCONNECT` *or*
 - c) they both belong to the group of the same window or file handle.
 2. If A is connected to B and B to C, then A is connected to C.
- Two processes are **disconnected** (also **independent**) if they are not connected.
- By the above definitions, connectivity is a transitive property, and divides the universe of MPI processes into disconnected (independent) sets (equivalence classes) of processes.
- Processes that are connected, but do not share the same `MPI_COMM_WORLD`, may become disconnected (independent) if the communication path between them is broken by using `MPI_COMM_DISCONNECT`.

The following additional rules apply to MPI routines in other chapters:

- `MPI_FINALIZE` is collective over a set of connected processes.
- `MPI_ABORT` does not abort independent processes. It may abort all processes in the caller's `MPI_COMM_WORLD` (ignoring its `comm` argument). Additionally, it may abort connected processes as well, though it makes a “best attempt” to abort only the processes in `comm`.
- If a process terminates without calling `MPI_FINALIZE`, independent processes are not affected but the effect on connected processes is not defined.

Advice to implementors. In practice, it may be difficult to distinguish between an MPI process failure and an erroneous program that terminates without calling an MPI finalization function: an implementation that defines semantics for process failure management may have to exhibit the behavior defined for MPI process failures with such erroneous programs. A high-quality implementation should exhibit a different behavior for erroneous programs and MPI process failures. (*End of advice to implementors.*)

`MPI_COMM_DISCONNECT(comm)`

INOUT `comm` communicator (handle)

C binding

`int MPI_Comm_disconnect(MPI_Comm *comm)`

Fortran 2008 binding

```

MPI_Comm_disconnect(comm, ierror)
    TYPE(MPI_Comm), INTENT(INOUT) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_DISCONNECT(COMM, IERROR)
    INTEGER COMM, IERROR

```

This function waits for all *decoupled MPI activities* on `comm` to complete internally, deallocates the communicator object, and sets the handle to `MPI_COMM_NULL`. It is a collective operation.

It may not be called with the communicator `MPI_COMM_WORLD` or `MPI_COMM_SELF`. `MPI_COMM_DISCONNECT` may be called only if all communication is complete and matched, so that buffered data can be delivered to its destination. This requirement is the same as for `MPI_FINALIZE`. This means that before calling `MPI_COMM_DISCONNECT`, all request handles associated with `comm` must be freed in the case of nonblocking operations, and must be inactive or freed in the case of persistent or partitioned operations (i.e., by calling one of the procedures `MPI_{TEST|WAIT}{|ANY|SOME|ALL}` or `MPI_REQUEST_FREE`).

`MPI_COMM_DISCONNECT` has the same effect as `MPI_COMM_FREE`, except that it waits for *decoupled MPI activities* on `comm` to finish internally, disallows any further use of derived inactive persistent requests, and enables the guarantee about the behavior of disconnected processes. The *decoupled MPI activities* also include any communication that is needed to complete a nonblocking or persistent operation on `comm` that was freed with `MPI_REQUEST_FREE`. After calling `MPI_COMM_DISCONNECT`, freeing or starting an inactive persistent request handle for a communication operation on `comm` is erroneous.

Advice to users. To disconnect two processes you may need to call `MPI_COMM_DISCONNECT`, `MPI_WIN_FREE`, and `MPI_FILE_CLOSE` to remove all communication paths between the two processes. Note that it may be necessary to disconnect several communicators (or to free several windows or files) before two processes are completely independent. (*End of advice to users.*)

Rationale. It would be nice to be able to use `MPI_COMM_FREE` instead, but that procedure explicitly does not wait for *decoupled MPI activities* to complete, and it does not disallow freeing or starting of related inactive (but not yet freed) persistent request handles. (*End of rationale.*)

11.10.5 Another Way to Establish MPI Communication

```

MPI_COMM_JOIN(fd, intercomm)

```

IN	fd	socket file descriptor
OUT	intercomm	new inter-communicator (handle)

C binding

```

int MPI_Comm_join(int fd, MPI_Comm *intercomm)

```

Fortran 2008 binding

```

MPI_Comm_join(fd, intercomm, ierror)
    INTEGER, INTENT(IN) :: fd
    TYPE(MPI_Comm), INTENT(OUT) :: intercomm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_JOIN(FD, INTERCOMM, IERROR)
    INTEGER FD, INTERCOMM, IERROR

```

MPI_COMM_JOIN is intended for MPI implementations that exist in an environment supporting the Berkeley Socket interface [52, 57]. Implementations that exist in an environment not supporting Berkeley Sockets should provide the entry point for **MPI_COMM_JOIN** and should return **MPI_COMM_NULL**.

This call creates an inter-communicator from the union of two MPI processes that are connected by a socket. **MPI_COMM_JOIN** should normally succeed if the local and remote processes have access to the same implementation-defined MPI communication universe.

Advice to users. An MPI implementation may require a specific communication medium for MPI communication, such as a shared memory segment or a special switch. In this case, it may not be possible for two processes to successfully join even if there is a socket connecting them and they are using the same MPI implementation. (*End of advice to users.*)

Advice to implementors. A high-quality implementation will attempt to establish communication over a slow medium if its preferred one is not available. If implementations do not do this, they must document why they cannot do MPI communication over the medium used by the socket (especially if the socket is a TCP connection). (*End of advice to implementors.*)

fd is a file descriptor representing a socket of type **SOCK_STREAM** (a two-way reliable byte-stream connection). Nonblocking I/O and asynchronous notification via **SIGIO** must not be enabled for the socket. The socket must be in a connected state. The socket must be quiescent when **MPI_COMM_JOIN** is called (see below). It is the responsibility of the application to create the socket using standard socket API calls.

MPI_COMM_JOIN must be called by the process at each end of the socket. It does not return until both processes have called **MPI_COMM_JOIN**. The two processes are referred to as the local and remote processes.

MPI only uses the socket to bootstrap the creation of the inter-communicator. Upon return from **MPI_COMM_JOIN**, the file descriptor will be open and quiescent (see below).

If MPI is unable to create an inter-communicator, but is able to leave the socket in its original state, with no pending communication, it succeeds and sets **intercomm** to **MPI_COMM_NULL**.

The socket must be quiescent before **MPI_COMM_JOIN** is called and after **MPI_COMM_JOIN** returns. More specifically, on entry to **MPI_COMM_JOIN**, a read on the socket will not read any data that was written to the socket before the remote process called **MPI_COMM_JOIN**. On exit from **MPI_COMM_JOIN**, a read will not read any data that was written to the socket before the remote process returned from **MPI_COMM_JOIN**. It is the responsibility of the application to ensure the first condition,

and the responsibility of the MPI implementation to ensure the second. In a multithreaded application, the application must ensure that one thread does not access the socket while another is calling `MPI_COMM_JOIN`, or call `MPI_COMM_JOIN` concurrently.

Advice to implementors. MPI is free to use any available communication path(s) for MPI messages in the new communicator; the socket is only used for the initial handshaking. *(End of advice to implementors.)*

`MPI_COMM_JOIN` uses non-MPI communication to do its work. The interaction of non-MPI communication with pending MPI communication is not defined. Therefore, the result of calling `MPI_COMM_JOIN` on two connected processes (see Section 11.10.4 for the definition of connected) is undefined.

The returned communicator may be used to establish MPI communication with additional processes, through the usual MPI communicator creation mechanisms.

Chapter 12

One-Sided Communications

12.1 Introduction

Remote Memory Access (RMA) extends the communication mechanisms of MPI by allowing one MPI process to specify all communication parameters, both for the sending side and for the receiving side. This mode of communication facilitates the coding of some applications with dynamically changing data access patterns where the data distribution is fixed or slowly changing. In such a case, each MPI process can compute what data it needs to access or to update at other MPI processes. However, the programmer may not be able to easily determine which data in an MPI process may need to be accessed or to be updated by operations initiated by a different MPI process, and may not even know which MPI processes may perform such updates. Thus, the transfer parameters are all available only on one side. Regular send/receive communication requires matching operations by sender and receiver. In order to issue the matching operations, an application needs to distribute the transfer parameters. This distribution may require all MPI processes to participate in a time-consuming global computation, or to poll for potential communication requests to receive and upon which to act periodically. The use of RMA communication operations avoids the need for global computations or explicit polling. A generic example of this nature is the execution of an assignment of the form $A = B(\text{map})$, where `map` is a permutation vector, and `A`, `B`, and `map` are distributed in the same manner.

Message-passing communication achieves two effects: *communication* of data from sender to receiver and *synchronization* of sender with receiver. The RMA design separates these two functions. The following communication calls are provided:

- Remote write: `MPI_PUT`, `MPI_RPUT`
- Remote read: `MPI_GET`, `MPI_RGET`
- Remote update: `MPI_ACCUMULATE`, `MPI_RACCUMULATE`
- Remote read and update: `MPI_GET_ACCUMULATE`, `MPI_RGET_ACCUMULATE`, and `MPI_FETCH_AND_OP`
- Remote atomic swap: `MPI_COMPARE_AND_SWAP`

This chapter refers to an operations set that includes all remote update, remote read and update, and remote atomic swap operations as “accumulate” operations.

MPI supports two fundamentally different *memory models*: *separate* and *unified*. The separate model makes no assumption about memory consistency and is highly portable. This model is similar to that of weakly coherent memory systems: the user must impose correct ordering of memory accesses through synchronization calls. The unified model can

exploit cache-coherent hardware and hardware-accelerated, one-sided operations that are commonly available in high-performance systems. The two different models are discussed in detail in Section 12.4. Both models support several synchronization calls to support different synchronization styles.

The design of the RMA functions allows implementors to take advantage of fast or asynchronous communication mechanisms provided by various platforms, such as coherent or noncoherent shared memory, DMA engines, hardware-supported put/get operations, and communication coprocessors. The most frequently used RMA communication mechanisms can be layered on top of message-passing. However, certain RMA functions might need support for asynchronous communication agents in software (handlers, threads, etc.) in a distributed memory environment.

We shall denote by **origin** or *origin process* the MPI process that calls an RMA procedure, and by **target** or *target process* the MPI process whose memory is accessed. Thus, in a put operation, `source = origin` and `destination = target`; in a get operation, `source = target` and `destination = origin`.

12.2 Initialization

MPI provides the following window initialization functions: `MPI_WIN_CREATE`, `MPI_WIN_ALLOCATE`, `MPI_WIN_ALLOCATE_SHARED`, and `MPI_WIN_CREATE_DYNAMIC`, which are collective over the group of an intra-communicator. `MPI_WIN_CREATE` allows each MPI process to specify a “window” in its memory that is made available for accesses by other MPI processes. The call returns an opaque object that represents the group of MPI processes that own and access the set of windows, and the attributes of each window, as specified by the initialization call. `MPI_WIN_ALLOCATE` and `MPI_WIN_ALLOCATE_SHARED` differ from `MPI_WIN_CREATE` in that the user does not pass allocated memory; instead `MPI_WIN_ALLOCATE` and `MPI_WIN_ALLOCATE_SHARED` return a pointer to memory allocated by the MPI implementation. `MPI_WIN_ALLOCATE_SHARED` differs from `MPI_WIN_ALLOCATE` in that the allocated memory is guaranteed to be accessible from all MPI processes in the window’s group with direct load/store accesses. Some restrictions may apply to the specified communicator. `MPI_WIN_CREATE_DYNAMIC` creates a window that allows the user to dynamically control which memory is exposed by the window.

12.2.1 Window Creation

`MPI_WIN_CREATE(base, size, disp_unit, info, comm, win)`

IN	base	initial address of window (choice)
IN	size	size of window in bytes (nonnegative integer)
IN	disp_unit	local unit size for displacements, in bytes (positive integer)
IN	info	info argument (handle)
IN	comm	intra-communicator (handle)
OUT	win	window object (handle)

C binding

```
int MPI_Win_create(void *base, MPI_Aint size, int disp_unit, MPI_Info info,
                  MPI_Comm comm, MPI_Win *win)
```

```
int MPI_Win_create_c(void *base, MPI_Aint size, MPI_Aint disp_unit,
                    MPI_Info info, MPI_Comm comm, MPI_Win *win)
```

Fortran 2008 binding

```
MPI_Win_create(base, size, disp_unit, info, comm, win, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
  INTEGER, INTENT(IN) :: disp_unit
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Win), INTENT(OUT) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_create(base, size, disp_unit, info, comm, win, ierror) !(_c)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size, disp_unit
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Win), INTENT(OUT) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_CREATE(BASE, SIZE, DISP_UNIT, INFO, COMM, WIN, IERROR)
  <type> BASE(*)
  INTEGER(KIND=MPI_ADDRESS_KIND) SIZE
  INTEGER DISP_UNIT, INFO, COMM, WIN, IERROR
```

This procedure is collective over the group of `comm`. It returns a handle to a window that can be used by the MPI processes in this group to perform RMA operations. Each MPI process specifies a window of existing memory that it exposes to RMA accesses by any MPI processes in the group of `comm`. The window consists of `size` bytes, starting at address `base`. In C, `base` is the starting address of a memory region. In Fortran, one can pass the first element of a memory region or a whole array, which must be ‘simply contiguous’ (for ‘simply contiguous,’ see also Section 21.1.12). An MPI process may elect to expose no memory by specifying `size = 0`.

The displacement unit argument is provided to facilitate address arithmetic in RMA operations: the target displacement argument of an RMA operation is scaled by the factor `disp_unit` specified by the target process, at window creation.

Rationale. The window size is specified using an address-sized integer, rather than a basic integer type, to allow windows that span more memory than can be described with a basic integer type. (*End of rationale.*)

Advice to users. Common choices for `disp_unit` are 1 (no scaling), and (in C syntax) `sizeof(type)`, for a window that consists of an array of elements of type `type`. The latter choice will allow one to use array indices in RMA calls, and have those scaled correctly to byte displacements, even in a heterogeneous environment. (*End of advice to users.*)

The `info` argument provides optimization hints to the runtime about the expected usage pattern of the window. The following info keys are predefined:

"no_locks" (boolean, default: **"false"**): if set to **"true"**, then the implementation may assume that passive target synchronization (i.e., `MPI_WIN_LOCK`, `MPI_WIN_LOCK_ALL`) will not be used on the given window. This implies that this window is not used for 3-party communication, and RMA can be implemented with no (less) asynchronous agent activity at this MPI process.

"accumulate_ordering" (string, default **"rar,raw,war,waw"**): controls the ordering of accumulate operations at the target. See Section 12.7.2 for details.

"accumulate_ops" (string, default: **"same_op_no_op"**): if set to **"same_op"**, the implementation will assume that all concurrent accumulate calls to the same target address will use the same operator. If set to **"same_op_no_op"**, then the implementation will assume that all concurrent accumulate calls to the same target address will use the same operator or `MPI_NO_OP`. This can eliminate the need to protect access for certain operators where the hardware can guarantee atomicity.

"mpi_accumulate_granularity" (integer, default **0**): provides a hint to implementations about the desired synchronization granularity for accumulate operations, i.e., the size of memory ranges in bytes for which the implementation should acquire a synchronization primitive to ensure atomicity of updates. If the specified granularity is not divisible by the size of the type used in an accumulate operation, it should be treated as if it was the next multiple of the element size. For example, a granularity of **"1"** byte should be treated as **"8"** in an accumulate operation using `MPI_UINT64_T`. By default, this info key is set to **"0"**, which leaves the choice of synchronization granularity to the implementation. If specified, all MPI processes in the group of a window must supply the same value.

Advice to users. Small synchronization granularities may provide improved latencies for accumulate operations with few elements and potentially increase concurrency of updates, at the cost of lower throughput. For example, a value matching the size of a type involved in an accumulate operation may enable implementations to use atomic memory operations instead of mutual exclusion devices. Larger synchronization granularities may yield higher throughput of accumulate operation with large numbers of elements due to lower synchronization costs, potentially at the expense of higher latency for accumulate operations with few elements, e.g., if atomic memory operations are not employed. By dividing larger accumulate operations into smaller segments, concurrent accumulate operations to the same window memory may update different segments in parallel. (*End of advice to users.*)

Advice to implementors. Implementations are encouraged to avoid mutual exclusion devices in cases where the granularity is small enough to warrant the use of atomic memory operations. For larger granularities, implementations should use this info value as a hint to partition the window memory into zones of mutual exclusion to enable segmentation of large accumulate operations. (*End of advice to implementors.*)

"same_size" (boolean, default: **"false"**): if set to **"true"**, then the implementation may assume that the argument **size** is identical on all MPI processes, and that all MPI processes have provided this info key with the same value.

"same_disp_unit" (boolean, default: **"false"**): if set to **"true"**, then the implementation may assume that the argument **disp_unit** is identical on all MPI processes, and that all MPI processes have provided this info key with the same value.

"mpi_assert_memory_alloc_kinds" (string, not set by default): If set, the implementation may assume that the memory for all communication buffers passed to MPI operations performed by the calling MPI process on the given window will use only the memory allocation kinds listed in the value string. See Section 11.4.3. This info hint also applies to the window buffer provided in a call to [MPI_WIN_CREATE](#) or [MPI_WIN_ATTACH](#). It does not apply to the memory allocated in a call to [MPI_WIN_ALLOCATE](#) or [MPI_WIN_ALLOCATE_SHARED](#).

Advice to users. The info query mechanism described in Section 12.2.7 can be used to query the specified info arguments for windows that have been passed to a library. It is recommended that libraries check attached info keys for each passed window. (*End of advice to users.*)

The various MPI processes in the group of **comm** may specify completely different target windows, in location, size, displacement units, and info arguments. As long as all the get, put and accumulate accesses to a particular MPI process fit their specific target window this should pose no problem. The same area in memory may appear in multiple windows, each associated with a different window object. However, concurrent communications to distinct, overlapping windows may lead to undefined results.

Implementations may make the memory provided by the user available for load/store accesses by MPI processes in the same *shared memory domain*. A communicator of such MPI processes can be constructed as described in Section 7.4.2 using [MPI_COMM_SPLIT_TYPE](#). Pointers to access a *shared memory segment* can be queried using [MPI_WIN_SHARED_QUERY](#).

Rationale. The reason for specifying the memory that may be accessed from another MPI process in an RMA operation is to permit the programmer to specify what memory can be a target of RMA operations and for the implementation to enforce that specification. For example, with this definition, a server MPI process can safely allow a client MPI process to use RMA operations, knowing that (under the assumption that the MPI implementation does enforce the specified limits on the exposed memory) an error in the client cannot affect any memory other than what was explicitly exposed. (*End of rationale.*)

Advice to users. A window can be created in any part of the MPI process memory. However, on some systems, the performance of windows in memory allocated by [MPI_ALLOC_MEM](#) (Section 9.2) will be better. Also, on some systems, performance is improved when window boundaries are aligned at “natural” boundaries (word, double-word, cache line, page frame, etc.). (*End of advice to users.*)

Advice to implementors. In cases where RMA operations use different mechanisms in different memory areas (e.g., load/store accesses in a *shared memory segment*, and

an asynchronous handler in private memory), the `MPI_WIN_CREATE` call needs to figure out which type of memory is used for the window. To do so, MPI maintains, internally, the list of memory segments allocated by `MPI_ALLOC_MEM`, or by other, implementation-specific, mechanisms, together with information on the type of memory segment allocated. When a call to `MPI_WIN_CREATE` occurs, then MPI checks which segment contains each window, and decides, accordingly, which mechanism to use for RMA operations.

Vendors may provide additional, implementation-specific mechanisms to allocate or to specify memory regions that are preferable for use in one-sided communication. In particular, such mechanisms can be used to place static variables into such preferred regions.

Implementors should document any performance impact of window alignment. (*End of advice to implementors.*)

12.2.2 Window That Allocates Memory

`MPI_WIN_ALLOCATE(size, disp_unit, info, comm, baseptr, win)`

IN	size	size of window in bytes (nonnegative integer)
IN	disp_unit	local unit size for displacements, in bytes (positive integer)
IN	info	info argument (handle)
IN	comm	intra-communicator (handle)
OUT	baseptr	initial address of window (choice)
OUT	win	window object (handle)

C binding

```
int MPI_Win_allocate(MPI_Aint size, int disp_unit, MPI_Info info,
    MPI_Comm comm, void *baseptr, MPI_Win *win)
```

```
int MPI_Win_allocate_c(MPI_Aint size, MPI_Aint disp_unit, MPI_Info info,
    MPI_Comm comm, void *baseptr, MPI_Win *win)
```

Fortran 2008 binding

```
MPI_Win_allocate(size, disp_unit, info, comm, baseptr, win, ierror)
```

```
USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
INTEGER, INTENT(IN) :: disp_unit
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(C_PTR), INTENT(OUT) :: baseptr
TYPE(MPI_Win), INTENT(OUT) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Win_allocate(size, disp_unit, info, comm, baseptr, win, ierror) !(_c)
USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size, disp_unit
```

```

TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(C_PTR), INTENT(OUT) :: baseptr
TYPE(MPI_Win), INTENT(OUT) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_ALLOCATE(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, WIN, IERROR)
  INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR
  INTEGER DISP_UNIT, INFO, COMM, WIN, IERROR

```

This procedure is collective over the group of `comm`. On each MPI process, it allocates memory of at least `size` bytes and returns a pointer to it along with a handle to a new window that can be used by all MPI processes in the group of `comm` to perform RMA operations. The returned memory consists of `size` bytes local to each MPI process, starting at address `baseptr` and is associated with the window as if the user called `MPI_WIN_CREATE` on existing memory. The size argument may be different at each MPI process and `size = 0` is valid; however, a library might allocate and expose more memory in order to create a fast, globally symmetric allocation. The discussion of and rationales for `MPI_ALLOC_MEM` and `MPI_FREE_MEM` in Section 9.2 also apply to `MPI_WIN_ALLOCATE`; in particular, see the rationale in Section 9.2 for an explanation of the type used for `baseptr`.

Implementations may make allocated memory available for load/store accesses by MPI processes in the same *shared memory domain*. A communicator of such MPI processes can be constructed as described in Section 7.4.2 using `MPI_COMM_SPLIT_TYPE`. Pointers to access a *shared memory segment* can be queried using `MPI_WIN_SHARED_QUERY`. If *shared memory* is available it is not guaranteed to be *contiguous* (see Section 12.2.3).

The `info` argument can be used to specify hints similar to the `info` argument for `MPI_WIN_CREATE` and `MPI_ALLOC_MEM`.

The default memory alignment requirements and the "mpi_minimum_memory_alignment" info key described for `MPI_ALLOC_MEM` in Section 9.2 apply to all MPI processes with nonzero `size` argument.

If the Fortran compiler provides `TYPE(C_PTR)`, then the following generic interface must be provided in the `mpi` module and should be provided in the (deprecated) `mpif.h` include file through overloading, i.e., with the same routine name as the routine with `INTEGER(KIND=MPI_ADDRESS_KIND) BASEPTR`, but with a different specific procedure name:

```

INTERFACE MPI_WIN_ALLOCATE
  SUBROUTINE MPI_WIN_ALLOCATE(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, &
    WIN, IERROR)
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
  END SUBROUTINE
  SUBROUTINE MPI_WIN_ALLOCATE_CPTR(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, &
    WIN, IERROR)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
  END SUBROUTINE

```

```

1      TYPE(C_PTR) :: BASEPTR
2      END SUBROUTINE
3  END INTERFACE
4

```

The base procedure name of this overloaded function is `MPI_WIN_ALLOCATE_CPTR`. The implied specific procedure names are described in Section 21.1.5.

Rationale. By allocating (potentially aligned) memory instead of allowing the user to pass in an arbitrary buffer, this call can improve the performance for systems with remote direct memory access. This also permits the collective allocation of memory and supports what is sometimes called the “symmetric allocation” model that can be more scalable (for example, the implementation can arrange to return an address for the allocated memory that is the same on all MPI processes). (*End of rationale.*)

12.2.3 Window That Allocates Shared Memory

`MPI_WIN_ALLOCATE_SHARED(size, disp_unit, info, comm, baseptr, win)`

IN	size	size of local window in bytes (nonnegative integer)
IN	disp_unit	local unit size for displacements, in bytes (positive integer)
IN	info	info argument (handle)
IN	comm	intra-communicator (handle)
OUT	baseptr	address of local allocated window segment (choice)
OUT	win	window object (handle)

C binding

```

30 int MPI_Win_allocate_shared(MPI_Aint size, int disp_unit, MPI_Info info,
31                             MPI_Comm comm, void *baseptr, MPI_Win *win)
32
33 int MPI_Win_allocate_shared_c(MPI_Aint size, MPI_Aint disp_unit, MPI_Info info,
34                             MPI_Comm comm, void *baseptr, MPI_Win *win)
35

```

Fortran 2008 binding

```

36 MPI_Win_allocate_shared(size, disp_unit, info, comm, baseptr, win, ierror)
37     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
38     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
39     INTEGER, INTENT(IN) :: disp_unit
40     TYPE(MPI_Info), INTENT(IN) :: info
41     TYPE(MPI_Comm), INTENT(IN) :: comm
42     TYPE(C_PTR), INTENT(OUT) :: baseptr
43     TYPE(MPI_Win), INTENT(OUT) :: win
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46 MPI_Win_allocate_shared(size, disp_unit, info, comm, baseptr, win, ierror)
47     !(_c)
48     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
49     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size, disp_unit
50

```

```

TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(C_PTR), INTENT(OUT) :: baseptr
TYPE(MPI_Win), INTENT(OUT) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_ALLOCATE_SHARED(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, WIN, IERROR)
  INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR
  INTEGER DISP_UNIT, INFO, COMM, WIN, IERROR

```

This procedure is collective over the group of `comm`. On each MPI process, it allocates memory of at least `size` bytes that is shared among all MPI processes in `comm`, and returns a pointer to the locally allocated segment in `baseptr` that can be used for load/store accesses on the calling MPI process. The locally allocated memory can be the target of load/store accesses by remote MPI processes; the base pointers for other MPI processes can be queried using the function `MPI_WIN_SHARED_QUERY`. The call also returns a handle to a new window that can be used by all MPI processes in `comm` to perform RMA operations. The size argument may be different at each MPI process and `size = 0` is valid. It is the user's responsibility to ensure that the communicator `comm` represents a group of MPI processes that are in the same *shared memory domain*, i.e., that they can create a *shared memory segment* that can be accessed by all MPI processes in the group. The discussions of rationales for `MPI_ALLOC_MEM` and `MPI_FREE_MEM` in Section 9.2 also apply to `MPI_WIN_ALLOCATE_SHARED`; in particular, see the rationale in Section 9.2 for an explanation of the type used for `baseptr`. The allocated memory is *contiguous across MPI processes in rank order* unless the info key "alloc_shared_noncontig" is specified. Contiguous across MPI processes in rank order means that the first address in the memory segment of MPI process i is consecutive with the last address in the memory segment of MPI process $i - 1$. This may enable the user to calculate remote address offsets with local information only.

The `info` argument can be used to specify hints similar to the `info` argument for `MPI_WIN_CREATE`, `MPI_WIN_ALLOCATE`, and `MPI_ALLOC_MEM`. The additional info key "alloc_shared_noncontig" allows the library to optimize the layout of the *shared memory segments* in memory.

Advice to users. If the info key "alloc_shared_noncontig" is not set to "true", the allocation strategy is to allocate *contiguous memory* across MPI process ranks. This may limit the performance on some architectures because it does not allow the implementation to modify the data layout (e.g., padding to reduce access latency). (*End of advice to users.*)

Advice to implementors. If the user sets the info key "alloc_shared_noncontig" to "true", the implementation can allocate the memory requested by each MPI process in a location that is close to this MPI process. This can be achieved by padding or allocating memory in special memory segments. Both techniques may make the address space across consecutive ranks *noncontiguous*. (*End of advice to implementors.*)

For *contiguous shared memory* allocations, the default alignment requirements outlined for `MPI_ALLOC_MEM` in Section 9.2 and the "mpi_minimum_memory_alignment" info key apply to the start of the *contiguous memory* that is returned in `baseptr` to the first MPI

process with nonzero `size` argument. For noncontiguous memory allocations, the default alignment requirements and the `"mpi_minimum_memory_alignment"` info key apply to all MPI processes with nonzero `size` argument.

Advice to users. If the info key `"alloc_shared_noncontig"` is not set to `"true"` (or ignored by the MPI implementation), the alignment of the memory returned in `baseptr` to all but the first MPI process with nonzero `size` argument depends on the value of the `size` argument provided by other MPI processes. It is thus the user's responsibility to control the alignment of contiguous memory allocated for these MPI processes by ensuring that each MPI process provides a `size` argument that is an integral multiple of the alignment required for the application. (*End of advice to users.*)

The consistency of load/store accesses from/to the shared memory as observed by the user program depends on the architecture. For details on how to create a consistent view see the description of [MPI_WIN_SHARED_QUERY](#).

If the Fortran compiler provides `TYPE(C_PTR)`, then the following generic interface must be provided in the `mpi` module and should be provided in the (deprecated) `mpif.h` include file through overloading, i.e., with the same routine name as the routine with `INTEGER(KIND=MPI_ADDRESS_KIND) BASEPTR`, but with a different specific procedure name:

```

INTERFACE MPI_WIN_ALLOCATE_SHARED
  SUBROUTINE MPI_WIN_ALLOCATE_SHARED(SIZE, DISP_UNIT, INFO, COMM, &
    BASEPTR, WIN, IERROR)
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
  END SUBROUTINE
  SUBROUTINE MPI_WIN_ALLOCATE_SHARED_CPTR(SIZE, DISP_UNIT, INFO, COMM, &
    BASEPTR, WIN, IERROR)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
    TYPE(C_PTR) :: BASEPTR
  END SUBROUTINE
END INTERFACE

```

The base procedure name of this overloaded function is `MPI_WIN_ALLOCATE_SHARED_CPTR`. The implied specific procedure names are described in Section [21.1.5](#).

`MPI_WIN_SHARED_QUERY(win, rank, size, disp_unit, baseptr)`

IN	win	shared memory window (handle)
IN	rank	rank in the group of window win or MPI_PROC_NULL (nonnegative integer)
OUT	size	size of the window segment (nonnegative integer)
OUT	disp_unit	local unit size for displacements, in bytes (positive integer)

MPI_WIN_CREATE. The potential for multiple memory regions in windows created through **MPI_WIN_CREATE_DYNAMIC** means that these windows cannot be used as input for **MPI_WIN_SHARED_QUERY**. (*End of rationale.*)

Advice to users. For windows allocated using **MPI_WIN_ALLOCATE** or **MPI_WIN_CREATE**, the group of MPI processes for which the implementation may provide shared memory can be determined using **MPI_COMM_SPLIT_TYPE** described in Section 7.4.2. (*End of advice to users.*)

The consistency of load/store accesses from/to the *shared memory* as observed by the user program depends on the architecture. A consistent view can be created in the *unified memory model* (see Section 12.4) by utilizing the window synchronization functions (see Section 12.5) or explicitly completing outstanding store accesses (e.g., by calling **MPI_WIN_FLUSH**). MPI does not define the semantics for accessing *shared window memory* in the *separate memory model*.

If the Fortran compiler provides **TYPE(C_PTR)**, then the following generic interface must be provided in the **mpi** module and should be provided in the (deprecated) **mpif.h** include file through overloading, i.e., with the same routine name as the routine with **INTEGER(KIND=MPI_ADDRESS_KIND) BASEPTR**, but with a different specific procedure name:

```

INTERFACE MPI_WIN_SHARED_QUERY
  SUBROUTINE MPI_WIN_SHARED_QUERY(WIN, RANK, SIZE, DISP_UNIT, &
    BASEPTR, IERROR)
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: WIN, RANK, DISP_UNIT, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
  END SUBROUTINE
  SUBROUTINE MPI_WIN_SHARED_QUERY_CPTR(WIN, RANK, SIZE, DISP_UNIT, &
    BASEPTR, IERROR)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: WIN, RANK, DISP_UNIT, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
    TYPE(C_PTR) :: BASEPTR
  END SUBROUTINE
END INTERFACE

```

The base procedure name of this overloaded function is **MPI_WIN_SHARED_QUERY_CPTR**. The implied specific procedure names are described in Section 21.1.5.

12.2.4 Window of Dynamically Attached Memory

The previously described window creation procedures require the user to identify the local memory that may be a target of RMA calls at the time the window is created. This has advantages for both the programmer (only this memory can be updated by one-sided operations and provides greater safety) and the MPI implementation (special steps may be taken to make one-sided access to such memory more efficient). However, consider implementing a modifiable linked list using RMA operations; as new items are added to

the list, memory must be allocated. In a C or C++ program, this memory is typically allocated using `malloc` or `new` respectively. With the previously described window creation procedures, the programmer must create a window with a predefined amount of memory and then implement routines for allocating memory from within the window's memory. In addition, there is no easy way to handle the situation where the predefined amount of memory turns out to be inadequate. To support this model, the routine `MPI_WIN_CREATE_DYNAMIC` creates a window that makes it possible to expose memory without remote synchronization. It must be used in combination with the local routines `MPI_WIN_ATTACH` and `MPI_WIN_DETACH`.

`MPI_WIN_CREATE_DYNAMIC`(info, comm, win)

IN	info	info argument (handle)
IN	comm	intra-communicator (handle)
OUT	win	window object (handle)

C binding

```
int MPI_Win_create_dynamic(MPI_Info info, MPI_Comm comm, MPI_Win *win)
```

Fortran 2008 binding

```
MPI_Win_create_dynamic(info, comm, win, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Win), INTENT(OUT) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_CREATE_DYNAMIC(INFO, COMM, WIN, IERROR)
  INTEGER INFO, COMM, WIN, IERROR
```

This procedure is collective over the group of `comm`. It returns a window `win` without memory attached. Existing MPI process memory can be attached as described below. This procedure returns a handle to a new window that can be used by MPI processes in the group of `comm` to perform RMA operations on attached memory. Because this window has special properties, it will sometimes be referred to as a **dynamic** window.

The `info` argument can be used to specify hints similar to the `info` argument for `MPI_WIN_CREATE`.

In the case of a window created with `MPI_WIN_CREATE_DYNAMIC`, the `target_disp` for all RMA functions is the address at the target; i.e., the effective `window_base` is `MPI_BOTTOM` and the `disp_unit` is one. For dynamic windows, the `target_disp` argument to RMA communication operations is not restricted to nonnegative values. Users should use `MPI_GET_ADDRESS` at the target process to determine the address of a target memory location and communicate this address to the origin process.

Advice to users. Users are cautioned that displacement arithmetic can overflow in variables of type `MPI_Aint` and result in unexpected values on some platforms. The `MPI_AINT_ADD` and `MPI_AINT_DIFF` functions can be used to safely perform address arithmetic with `MPI_Aint` displacements. (*End of advice to users.*)

Advice to implementors. In environments with heterogeneous data representations, care must be exercised in communicating addresses between MPI processes. For example, it is possible that an address valid at the target MPI process (for example, a 64-bit pointer) cannot be expressed as an address at the origin (for example, the origin uses 32-bit pointers). For this reason, a portable MPI implementation should ensure that the type `MPI_AINT` (see Table 3.3) is able to store addresses from any MPI process. *(End of advice to implementors.)*

Memory at the target cannot be accessed with this window until that memory has been attached using the function `MPI_WIN_ATTACH`. That is, in addition to using `MPI_WIN_CREATE_DYNAMIC` to create an MPI window, the user must use `MPI_WIN_ATTACH` before any local memory may be the target of an MPI RMA operation. Only memory that is currently accessible may be attached.

```
MPI_WIN_ATTACH(win, base, size)
```

IN	win	window object (handle)
IN	base	initial address of memory to be attached (choice)
IN	size	size of memory to be attached in bytes (nonnegative integer)

C binding

```
int MPI_Win_attach(MPI_Win win, void *base, MPI_Aint size)
```

Fortran 2008 binding

```
MPI_Win_attach(win, base, size, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_ATTACH(WIN, BASE, SIZE, IERROR)
  INTEGER WIN, IERROR
  <type> BASE(*)
  INTEGER(KIND=MPI_ADDRESS_KIND) SIZE
```

Attaches a local memory region beginning at `base` for remote access within the given window. The memory region specified must not contain any part that is already attached to the window `win`, that is, attaching overlapping memory concurrently within the same window is erroneous. The argument `win` must be a window that was created with `MPI_WIN_CREATE_DYNAMIC`. The local memory region attached to the window consists of `size` bytes, starting at address `base`. In C, `base` is the starting address of a memory region. In Fortran, one can pass the first element of a memory region or a whole array, which must be ‘simply contiguous’ (for ‘simply contiguous,’ see Section 21.1.12). Multiple (but nonoverlapping) memory regions may be attached to the same window.

Rationale. Requiring that memory be explicitly attached before it is exposed to one-sided access by other MPI processes can simplify implementations and improve performance. The ability to make memory available for RMA operations without requiring a

collective `MPI_WIN_CREATE` call is needed for some one-sided programming models.
(*End of rationale.*)

Advice to users. Attaching memory to a window may require the use of scarce resources; thus, attaching large regions of memory is not recommended in portable programs. Attaching memory to a window may fail if sufficient resources are not available; this is similar to the behavior of `MPI_ALLOC_MEM`.

The user is also responsible for ensuring that `MPI_WIN_ATTACH` at the target has returned before an MPI process attempts to target that memory with an MPI RMA operation.

Performing an RMA operation on memory that has not been attached to a window created with `MPI_WIN_CREATE_DYNAMIC` is erroneous. (*End of advice to users.*)

Advice to implementors. A high-quality implementation will attempt to make as much memory available for attaching as possible. Any limitations should be documented by the implementor. (*End of advice to implementors.*)

`MPI_WIN_ATTACH` is a local procedure that is not collective. Memory may be detached with the procedure `MPI_WIN_DETACH`. After memory has been detached, it may not be the target of an MPI RMA operation on that window (unless the memory is re-attached with `MPI_WIN_ATTACH`).

`MPI_WIN_DETACH(win, base)`

IN	win	window object (handle)
IN	base	initial address of memory to be detached (choice)

C binding

```
int MPI_Win_detach(MPI_Win win, const void *base)
```

Fortran 2008 binding

```
MPI_Win_detach(win, base, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_DETACH(WIN, BASE, IERROR)
  INTEGER WIN, IERROR
  <type> BASE(*)
```

Detaches a previously attached memory region beginning at `base`. The arguments `base` and `win` must match the arguments passed to a previous call to `MPI_WIN_ATTACH`. `MPI_WIN_DETACH` is a local procedure that is not collective.

Advice to users. Detaching memory may permit the implementation to make more efficient use of special memory or provide memory that may be needed by a subsequent `MPI_WIN_ATTACH`. Users are encouraged to detach memory that is no longer needed. Memory should be detached before it is freed by the user. (*End of advice to users.*)

Memory becomes detached when the associated dynamic memory window is freed, see Section 12.2.5.

12.2.5 Window Destruction

MPI_WIN_FREE(win)

INOUT win window object (handle)

C binding

```
int MPI_Win_free(MPI_Win *win)
```

Fortran 2008 binding

```
MPI_Win_free(win, ierror)
      TYPE(MPI_Win), INTENT(INOUT) :: win
      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_FREE(WIN, IERROR)
      INTEGER WIN, IERROR
```

Frees the window object win and returns a null handle (equal to MPI_WIN_NULL). This procedure is collective over the group associated with win. **MPI_WIN_FREE** can be invoked by an MPI process only after it has completed its involvement in RMA communications on window win: e.g., the MPI process has called **MPI_WIN_FENCE**, or called **MPI_WIN_WAIT** to match a previous call to **MPI_WIN_POST**, called **MPI_WIN_COMPLETE** to match a previous call to **MPI_WIN_START**, or called **MPI_WIN_UNLOCK** to match a previous call to **MPI_WIN_LOCK**. The memory associated with windows created by a call to **MPI_WIN_CREATE** may be freed after the call returns. If the window was created with **MPI_WIN_ALLOCATE**, **MPI_WIN_FREE** will free the window memory that was allocated in **MPI_WIN_ALLOCATE**. If the window was created with **MPI_WIN_ALLOCATE_SHARED**, **MPI_WIN_FREE** will free the window memory that was allocated in **MPI_WIN_ALLOCATE_SHARED**.

Freeing a window that was created with a call to **MPI_WIN_CREATE_DYNAMIC** detaches all associated memory; i.e., it has the same effect as if all attached memory was detached by calls to **MPI_WIN_DETACH**.

MPI_WIN_FREE is required to delay its return until all accesses to the local window using passive target synchronization have completed. Therefore, it is synchronizing unless the window was created with the "no_locks" info key set to "true".

12.2.6 Window Attributes

The following attributes are cached with a window when the window is created.

MPI_WIN_BASE	window base address.
MPI_WIN_SIZE	window size, in bytes.
MPI_WIN_DISP_UNIT	displacement unit associated with the window.
MPI_WIN_CREATE_FLAVOR	how the window was created.
MPI_WIN_MODEL	memory model for window.

Table 12.1: C types of attribute value argument to `MPI_WIN_GET_ATTR` and `MPI_WIN_SET_ATTR`

Attribute	C Type
<code>MPI_WIN_BASE</code>	<code>void *</code>
<code>MPI_WIN_SIZE</code>	<code>MPI_Aint *</code>
<code>MPI_WIN_DISP_UNIT</code>	<code>int *</code>
<code>MPI_WIN_CREATE_FLAVOR</code>	<code>int *</code>
<code>MPI_WIN_MODEL</code>	<code>int *</code>

In C, calls such as `MPI_Win_get_attr(win, MPI_WIN_BASE, &base, &flag)`, `MPI_Win_get_attr(win, MPI_WIN_SIZE, &size, &flag)`, `MPI_Win_get_attr(win, MPI_WIN_DISP_UNIT, &disp_unit, &flag)`, `MPI_Win_get_attr(win, MPI_WIN_CREATE_FLAVOR, &create_flavor, &flag)`, and `MPI_Win_get_attr(win, MPI_WIN_MODEL, &memory_model, &flag)` will return in `base` a pointer to the start of the window `win` and in `size`, `disp_unit`, `create_flavor`, and `memory_model` pointers to the size of the window, the displacement unit of the window, the flavor of the window, and the memory model of the window, respectively. A detailed listing of the type of the pointer in the attribute value argument to `MPI_WIN_GET_ATTR` and `MPI_WIN_SET_ATTR` is shown in Table 12.1.

In Fortran, calls such as `MPI_WIN_GET_ATTR(win, MPI_WIN_BASE, base, flag, ierror)`, `MPI_WIN_GET_ATTR(win, MPI_WIN_SIZE, size, flag, ierror)`, `MPI_WIN_GET_ATTR(win, MPI_WIN_DISP_UNIT, disp_unit, flag, ierror)`, `MPI_WIN_GET_ATTR(win, MPI_WIN_CREATE_FLAVOR, create_flavor, flag, ierror)`, and `MPI_WIN_GET_ATTR(win, MPI_WIN_MODEL, memory_model, flag, ierror)` will return in `base`, `size`, `disp_unit`, `create_flavor`, and `memory_model` the (integer representation of) the base address of the window, the size of the window, the displacement unit of the window, the flavor of the window, and the memory model of the window, respectively.

The values of `create_flavor` are

<code>MPI_WIN_FLAVOR_CREATE</code>	Window was created with <code>MPI_WIN_CREATE</code> .
<code>MPI_WIN_FLAVOR_ALLOCATE</code>	Window was created with <code>MPI_WIN_ALLOCATE</code> .
<code>MPI_WIN_FLAVOR_DYNAMIC</code>	Window was created with <code>MPI_WIN_CREATE_DYNAMIC</code> .
<code>MPI_WIN_FLAVOR_SHARED</code>	Window was created with <code>MPI_WIN_ALLOCATE_SHARED</code> .

The values of `memory_model` are `MPI_WIN_SEPARATE` and `MPI_WIN_UNIFIED`. The meaning of these is described in Section 12.4.

In the case of windows created with `MPI_WIN_CREATE_DYNAMIC`, the base address is `MPI_BOTTOM` and the size is 0. In C, pointers are returned, and in Fortran, the values are returned, for the respective attributes. (The window attribute access functions are defined in Section 7.7.3.) The value returned for an attribute on a window is constant over the lifetime of the window.

The other “window attribute,” namely the group of MPI processes attached to the window, can be retrieved using the call below.

```
1 MPI_WIN_GET_GROUP(win, group)
```

```
2     IN      win                window object (handle)
3
4     OUT     group              group of MPI processes that share access to the
5                                window (handle)
```

6 C binding

```
7 int MPI_Win_get_group(MPI_Win win, MPI_Group *group)
```

9 Fortran 2008 binding

```
10 MPI_Win_get_group(win, group, ierror)
11     TYPE(MPI_Win), INTENT(IN) :: win
12     TYPE(MPI_Group), INTENT(OUT) :: group
13     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

14 Fortran binding

```
15 MPI_WIN_GET_GROUP(WIN, GROUP, IERROR)
16     INTEGER WIN, GROUP, IERROR
```

18 **MPI_WIN_GET_GROUP** returns in **group** a duplicate of the group of the communicator
19 used to create the window associated with **win**.

21 12.2.7 Window Info

22 Hints specified via info (see Chapter 10) allow a user to provide information to direct
23 optimization. Providing hints may enable an implementation to deliver increased perfor-
24 mance or use system resources more efficiently. As described in Chapter 10, an imple-
25 mentation is free to ignore all hints; however, applications must comply with any info
26 hints they provide that are used by the MPI implementation (i.e., are returned by a
27 call to **MPI_WIN_GET_INFO**) and that place a restriction on the behavior of the ap-
28 plication. Hints are specified on a per window basis, in window creation functions and
29 **MPI_WIN_SET_INFO**, via the opaque info object. When an info object that specifies a
30 subset of valid hints is passed to **MPI_WIN_SET_INFO** there will be no effect on previously
31 set or default hints that the info does not specify.

33 *Advice to implementors.* It may happen that a program is coded with hints for one
34 system, and later executes on another system that does not support these hints. In
35 general, unsupported hints should simply be ignored. Needless to say, no hint can be
36 mandatory. However, for each hint used by a specific implementation, a default value
37 must be provided when the user does not specify a value for the hint. (*End of advice*
38 *to implementors.*)

```
42 MPI_WIN_SET_INFO(win, info)
```

```
43     INOUT   win                window object (handle)
44
45     IN      info              info argument (handle)
```

47 C binding

```
48 int MPI_Win_set_info(MPI_Win win, MPI_Info info)
```

Fortran 2008 binding

```

MPI_Win_set_info(win, info, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_SET_INFO(WIN, INFO, IERROR)
    INTEGER WIN, INFO, IERROR

```

MPI_WIN_SET_INFO updates the hints of the window associated with `win` using the hints provided in `info`. This operation has no effect on previously set or defaulted hints that are not specified by `info`. It also has no effect on previously set or defaulted hints that are specified by `info`, but are ignored by the MPI implementation in this call to **MPI_WIN_SET_INFO**. The procedure is collective over the group of `win`. The entries in the `info` object may be different on each MPI process, but any `info` entries that an implementation requires to be the same on all MPI processes must appear with the same value in each MPI process's `info` object.

Advice to users. Some `info` items that an implementation can use when it creates a window cannot easily be changed once the window has been created. Thus, an implementation may ignore hints issued in this call that it would have accepted in a creation call. An implementation may also be unable to update certain `info` hints in a call to **MPI_WIN_SET_INFO**. **MPI_WIN_GET_INFO** can be used to determine whether `info` changes were ignored by the implementation. (*End of advice to users.*)

```

MPI_WIN_GET_INFO(win, info_used)
    IN      win      window object (handle)
    OUT     info_used new info object (handle)

```

C binding

```

int MPI_Win_get_info(MPI_Win win, MPI_Info *info_used)

```

Fortran 2008 binding

```

MPI_Win_get_info(win, info_used, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Info), INTENT(OUT) :: info_used
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_GET_INFO(WIN, INFO_USED, IERROR)
    INTEGER WIN, INFO_USED, IERROR

```

MPI_WIN_GET_INFO returns a new `info` object containing the hints of the window associated with `win`. The current setting of all hints related to this window is returned in `info_used`. An MPI implementation is required to return all hints that are supported by the implementation and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created `info` object is returned that contains no key/value pair. The user is responsible for freeing `info_used` via **MPI_INFO_FREE**.

12.3 Communication Calls

MPI supports the following RMA communication calls: `MPI_PUT` and `MPI_RPUT` transfer data from the caller memory (origin) to the target memory; `MPI_GET` and `MPI_RGET` transfer data from the target memory to the caller memory; `MPI_ACCUMULATE` and `MPI_RACCUMULATE` perform element-wise atomic updates of locations in the target memory, e.g., by adding to these locations values sent from the caller memory; `MPI_GET_ACCUMULATE`, `MPI_RGET_ACCUMULATE`, and `MPI_FETCH_AND_OP` perform element-wise atomic read-modify-write updates and return each value before the update; and `MPI_COMPARE_AND_SWAP` performs a remote atomic compare and swap operation. These procedures are *nonblocking*. The operation is completed, at the origin or both the origin and the target, when a subsequent *synchronization* procedure is called by the origin on the involved window object. These synchronization procedures are described in Section 12.5. RMA communication operations can also be completed with calls to flush procedures; see Section 12.5.4 for details. Request-based operations `MPI_RPUT`, `MPI_RGET`, `MPI_RACCUMULATE`, and `MPI_RGET_ACCUMULATE` can be completed at the origin by using the MPI test or wait procedures described in Section 3.7.3.

The local communication buffer of an RMA operation should not be updated after the operation started and until the operation completes at the origin. The local communication buffer of a get operation should not be accessed after the operation started and until the operation completes at the origin.

Two concurrent accesses are called conflicting if one of the two is a put operation, exactly one of them is an accumulate operation, or one of them is a get operation and the other is a local store access. The outcome of conflicting accesses to the same memory location is undefined; if a location is updated by a put or accumulate operation, then the outcome of loads or other RMA operations is undefined until the updating operation has completed at the target. There is one exception to this rule; namely, the same location can be updated by several concurrent accumulate operations, the outcome being as if these updates occurred in some order. In addition, the outcome of concurrent load/store accesses and RMA updates to the same memory location is undefined. These restrictions are described in more detail in Section 12.7.

The calls use general datatype arguments to specify communication buffers at the origin and at the target. Thus, a transfer operation may also gather data at the source and scatter it at the destination. However, all arguments specifying both communication buffers are provided by the caller.

For all RMA communication operations, the target process may be identical with the origin process; i.e., an MPI process may use an RMA operation to move data in its memory.

Rationale. The choice of supporting “self-communication” is the same as for message-passing. It simplifies some coding, and is very useful with accumulate operations, to allow atomic updates of local variables. (*End of rationale.*)

`MPI_PROC_NULL` is a valid target rank in all MPI RMA communication calls. The effect is the same as for `MPI_PROC_NULL` in MPI point-to-point communication. After any RMA operation with rank `MPI_PROC_NULL`, it is still necessary to close the RMA epoch with the synchronization method that opened the epoch.

12.3.1 Put

The execution of a put operation is similar to the execution of a send by the origin process and a matching receive by the target process. The obvious difference is that all arguments are provided by one call—the call executed by the origin process.

```
MPI_PUT(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win)
```

IN	origin_addr	initial address of origin buffer (choice)
IN	origin_count	number of entries in origin buffer (nonnegative integer)
IN	origin_datatype	datatype of each entry in origin buffer (handle)
IN	target_rank	rank of target (nonnegative integer)
IN	target_disp	displacement from start of window to target buffer (nonnegative integer)
IN	target_count	number of entries in target buffer (nonnegative integer)
IN	target_datatype	datatype of each entry in target buffer (handle)
IN	win	window used for communication (handle)

C binding

```
int MPI_Put(const void *origin_addr, int origin_count,
            MPI_Datatype origin_datatype, int target_rank,
            MPI_Aint target_disp, int target_count,
            MPI_Datatype target_datatype, MPI_Win win)

int MPI_Put_c(const void *origin_addr, MPI_Count origin_count,
              MPI_Datatype origin_datatype, int target_rank,
              MPI_Aint target_disp, MPI_Count target_count,
              MPI_Datatype target_datatype, MPI_Win win)
```

Fortran 2008 binding

```
MPI_Put(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
    INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Put(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER, INTENT(IN) :: target_rank
```

```

1      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
2      TYPE(MPI_Win), INTENT(IN) :: win
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

5      MPI_PUT(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
6              TARGET_COUNT, TARGET_DATATYPE, WIN, IERROR)
7      <type> ORIGIN_ADDR(*)
8      INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
9              TARGET_DATATYPE, WIN, IERROR
10     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

Transfers `origin_count` successive entries of the type specified by the `origin_datatype`, starting at address `origin_addr` on the origin process, to the target process specified by the `win`, `target_rank` pair. The data are written in the target buffer at address `target_addr = window_base + target_disp × disp_unit`, where `window_base` and `disp_unit` are the base address and window displacement unit specified at window initialization, by the target process.

The target buffer is specified by the arguments `target_count` and `target_datatype`.

The data transfer is the same as that which would occur if the origin process executed a send operation with arguments `origin_addr`, `origin_count`, `origin_datatype`, `target_rank`, `tag`, `comm`, and the target process executed a receive operation with arguments `target_addr`, `target_count`, `target_datatype`, `source`, `tag`, `comm`, where `target_addr` is the target buffer address computed as explained above, the values of `tag` are arbitrary valid matching tag values, and `comm` is a communicator for the group of `win`.

The communication must satisfy the same constraints as for a similar message-passing communication. The `target_datatype` may not specify overlapping entries in the target buffer. The message sent must fit, without truncation, in the target buffer. Furthermore, the target buffer must fit in the target window or in attached memory in a dynamic window.

The `target_datatype` argument is a handle to a datatype object defined at the origin process. However, this object is interpreted at the target process: the outcome is as if the target datatype object was defined at the target process by the same sequence of calls used to define it at the origin process. The target datatype must contain only relative displacements, not absolute addresses. The same holds for get and accumulate operations.

Advice to users. The `target_datatype` argument is a handle to a datatype object that is defined at the origin process, even though it defines a data layout in the target process memory. This causes no problems in a homogeneous environment, or in a heterogeneous environment if only portable datatypes are used (portable datatypes are defined in Section 2.4).

The performance of a put transfer can be significantly affected, on some systems, by the choice of window location and the shape and location of the origin and target buffer: transfers to a target window in memory allocated by `MPI_ALLOC_MEM` or `MPI_WIN_ALLOCATE` may be much faster on shared memory systems; transfers from contiguous buffers will be faster on most, if not all, systems; the alignment of the communication buffers may also impact performance. (*End of advice to users.*)

Advice to implementors. A high-quality implementation will attempt to prevent remote accesses to memory outside the window that was exposed by the MPI process. This

is important both for debugging purposes and for protection with client-server codes that use RMA. That is, a high-quality implementation will check, if possible, window bounds on each RMA call, and raise an error at the origin call if an out-of-bound situation occurs. Note that the condition can be checked at the origin. Of course, the added safety achieved by such checks has to be weighed against the added cost of such checks. (*End of advice to implementors.*)

12.3.2 Get

MPI_GET(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
target_count, target_datatype, win)

OUT	origin_addr	initial address of origin buffer (choice)
IN	origin_count	number of entries in origin buffer (nonnegative integer)
IN	origin_datatype	datatype of each entry in origin buffer (handle)
IN	target_rank	rank of target (nonnegative integer)
IN	target_disp	displacement from window start to the beginning of the target buffer (nonnegative integer)
IN	target_count	number of entries in target buffer (nonnegative integer)
IN	target_datatype	datatype of each entry in target buffer (handle)
IN	win	window used for communication (handle)

C binding

```
int MPI_Get(void *origin_addr, int origin_count, MPI_Datatype origin_datatype,
            int target_rank, MPI_Aint target_disp, int target_count,
            MPI_Datatype target_datatype, MPI_Win win)

int MPI_Get_c(void *origin_addr, MPI_Count origin_count,
              MPI_Datatype origin_datatype, int target_rank,
              MPI_Aint target_disp, MPI_Count target_count,
              MPI_Datatype target_datatype, MPI_Win win)
```

Fortran 2008 binding

```
MPI_Get(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, ierror)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
    INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, ierror) !(_c)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
```

```

1      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
2      TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
3      INTEGER, INTENT(IN) :: target_rank
4      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
5      TYPE(MPI_Win), INTENT(IN) :: win
6      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

8      MPI_GET(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
9              TARGET_COUNT, TARGET_DATATYPE, WIN, IERROR)
10     <type> ORIGIN_ADDR(*)
11     INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
12           TARGET_DATATYPE, WIN, IERROR
13     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

Similar to `MPI_PUT`, except that the direction of data transfer is reversed. Data are copied from the target memory to the origin. The `origin_datatype` may not specify overlapping entries in the origin buffer. The target buffer must be contained within the target window or within attached memory in a dynamic window, and the copied data must fit, without truncation, in the origin buffer.

12.3.3 Examples for Communication Calls

These examples show the use of the `MPI_GET` procedure. As all MPI RMA communication procedures are nonblocking, the associated operations must be completed by subsequent calls to synchronization procedures. In the following example, completion is accomplished with the routine `MPI_WIN_FENCE`, introduced in Section 12.5.

Example 12.1. We show how to implement the generic indirect assignment $A = B(\text{map})$, where A , B , and map have the same distribution, and map is a permutation. To simplify, we assume a block distribution with equal size blocks.

```

30      SUBROUTINE MAPVALS(A, B, map, m, comm, p)
31      USE MPI
32      INTEGER m, map(m), comm, p
33      REAL A(m), B(m)
34
35      INTEGER otype(p), oindex(m), & ! used to construct origin datatypes
36             ttype(p), tindex(m), & ! used to construct target datatypes
37             count(p), total(p), &
38             disp_int, win, ierr, i, j, k
39      INTEGER(KIND=MPI_ADDRESS_KIND) lowerbound, size, realextent, disp_aint
40
41      ! This part does the work that depends on the locations of B.
42      ! Can be reused while this does not change
43
44      CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lowerbound, realextent, ierr)
45      disp_int = realextent
46      size = m * realextent
47      CALL MPI_WIN_CREATE(B, size, disp_int, MPI_INFO_NULL, &
48                          comm, win, ierr)
49
50      ! This part does the work that depends on the value of map and
51      ! the locations of the arrays.

```

```

! Can be reused while these do not change
! Compute number of entries to be received from each process

DO i=1,p
    count(i) = 0
END DO
DO i=1,m
    j = map(i)/m+1
    count(j) = count(j)+1
END DO

total(1) = 0
DO i=2,p
    total(i) = total(i-1) + count(i-1)
END DO

DO i=1,p
    count(i) = 0
END DO

! compute origin and target indices of entries.
! entry i at current process is received from location
! k at process (j-1), where map(i) = (j-1)*m + (k-1),
! j = 1..p and k = 1..m

DO i=1,m
    j = map(i)/m+1
    k = MOD(map(i),m)+1
    count(j) = count(j)+1
    oindex(total(j) + count(j)) = i
    tindex(total(j) + count(j)) = k
END DO

! create origin and target datatypes for each get operation
DO i=1,p
    CALL MPI_TYPE_CREATE_INDEXED_BLOCK(count(i), 1, &
                                     oindex(total(i)+1:total(i)+count(i)), &
                                     MPI_REAL, otype(i), ierr)
    CALL MPI_TYPE_COMMIT(otype(i), ierr)
    CALL MPI_TYPE_CREATE_INDEXED_BLOCK(count(i), 1, &
                                     tindex(total(i)+1:total(i)+count(i)), &
                                     MPI_REAL, ttype(i), ierr)
    CALL MPI_TYPE_COMMIT(ttype(i), ierr)
END DO

! this part does the assignment itself
CALL MPI_WIN_FENCE(0, win, ierr)
disp_aint = 0
DO i=1,p
    CALL MPI_GET(A, 1, otype(i), i-1, disp_aint, 1, ttype(i), win, ierr)
END DO
CALL MPI_WIN_FENCE(0, win, ierr)

CALL MPI_WIN_FREE(win, ierr)
DO i=1,p
    CALL MPI_TYPE_FREE(otype(i), ierr)

```

```

1      CALL MPI_TYPE_FREE(ttype(i), ierr)
2  END DO
3  RETURN
4  END

```

Example 12.2. A simpler version can be written that does not require that a datatype be built for the target buffer. One then needs a separate get operation for each entry, as illustrated below. This code is much simpler, but usually much less efficient, for large arrays.

```

10  SUBROUTINE MAPVALS(A, B, map, m, comm, p)
11  USE MPI
12  INTEGER m, map(m), comm, p
13  REAL A(m), B(m)
14  INTEGER disp_int, i, j, win, ierr
15  INTEGER(KIND=MPI_ADDRESS_KIND) lowerbound, size, realexent, disp_aint
16
17  CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lowerbound, realexent, ierr)
18  disp_int = realexent
19  size = m * realexent
20  CALL MPI_WIN_CREATE(B, size, disp_int, MPI_INFO_NULL, &
21                      comm, win, ierr)
22
23  CALL MPI_WIN_FENCE(0, win, ierr)
24  DO i=1,m
25      j = map(i)/m
26      disp_aint = MOD(map(i),m)
27      CALL MPI_GET(A(i), 1, MPI_REAL, j, disp_aint, 1, MPI_REAL, win, ierr)
28  END DO
29  CALL MPI_WIN_FENCE(0, win, ierr)
30  CALL MPI_WIN_FREE(win, ierr)
31  RETURN
32  END

```

12.3.4 Accumulate Functions

It is often useful in a put operation to combine the data moved to the target process with the data that resides at that MPI process, rather than replacing it. This will allow, for example, the accumulation of a sum by having all involved MPI processes add their contributions to the sum variable in the memory of one MPI process. The accumulate functions have slightly different semantics with respect to overlapping data accesses than the put and get functions; see Section 12.7 for details.

Accumulate

```

MPI_ACCUMULATE(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
               target_count, target_datatype, op, win)

```

IN	origin_addr	initial address of buffer (choice)
IN	origin_count	number of entries in buffer (nonnegative integer)

IN	origin_datatype	datatype of each entry (handle)	1
IN	target_rank	rank of target (nonnegative integer)	2
IN	target_disp	displacement from start of window to beginning of target buffer (nonnegative integer)	3
IN	target_count	number of entries in target buffer (nonnegative integer)	4
IN	target_datatype	datatype of each entry in target buffer (handle)	5
IN	op	accumulate operator (handle)	6
IN	win	window object (handle)	7

C binding

```

int MPI_Accumulate(const void *origin_addr, int origin_count,
                  MPI_Datatype origin_datatype, int target_rank,
                  MPI_Aint target_disp, int target_count,
                  MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
int MPI_Accumulate_c(const void *origin_addr, MPI_Count origin_count,
                    MPI_Datatype origin_datatype, int target_rank,
                    MPI_Aint target_disp, MPI_Count target_count,
                    MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)

```

Fortran 2008 binding

```

MPI_Accumulate(origin_addr, origin_count, origin_datatype, target_rank,
               target_disp, target_count, target_datatype, op, win, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Accumulate(origin_addr, origin_count, origin_datatype, target_rank,
               target_disp, target_count, target_datatype, op, win, ierror)
!(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK,
               TARGET_DISP, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, IERROR)
<type> ORIGIN_ADDR(*)

```

```

1      INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
2              TARGET_DATATYPE, OP, WIN, IERROR
3      INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

Accumulate the contents of the origin buffer (as defined by `origin_addr`, `origin_count`, and `origin_datatype`) to the buffer specified by arguments `target_count` and `target_datatype`, at offset `target_disp`, in the target window specified by `target_rank` and `win`, using the operator `op`. This is like `MPI_PUT` except that data is combined into the target area instead of overwriting it.

Any of the predefined operators for `MPI_REDUCE` can be used. User-defined operators cannot be used. For example, if `op` is `MPI_SUM`, each element of the origin buffer is added to the corresponding element in the target, replacing the former value in the target.

Each datatype argument must be a predefined datatype or a derived datatype, where all basic components are of the same predefined datatype. Both datatype arguments must be constructed from the same predefined datatype. The operator `op` applies to elements of that predefined type. The parameter `target_datatype` must not specify overlapping entries, and the target buffer must fit in the target window.

An additional predefined operator, `MPI_REPLACE`, is defined. It corresponds to the associative function $f(a, b) = b$; i.e., the current value in the target memory is replaced by the value supplied by the origin.

`MPI_REPLACE` can be used only in `MPI_ACCUMULATE`, `MPI_RACCUMULATE`, `MPI_GET_ACCUMULATE`, `MPI_FETCH_AND_OP`, and `MPI_RGET_ACCUMULATE`, but not in collective reduction operations such as `MPI_REDUCE`.

Advice to users. `MPI_PUT` can be considered a special case of `MPI_ACCUMULATE` with the operator `MPI_REPLACE`. Note, however, that `MPI_PUT` and `MPI_ACCUMULATE` have different constraints on concurrent updates. (*End of advice to users.*)

Example 12.3. We want to compute $B(j) = \sum_{\text{map}(i)=j} A(i)$. The arrays `A`, `B`, and `map` are distributed in the same manner. We write the simple version.

```

31      SUBROUTINE SUM(A, B, map, m, comm, p)
32      USE MPI
33      INTEGER m, map(m), comm, p, win, ierr, disp_int, i, j
34      REAL A(m), B(m)
35      INTEGER(KIND=MPI_ADDRESS_KIND) lowerbound, size, realexent, disp_aint
36
37      CALL MPI_TYPE_GET_EXTENT(MPI_REAL, lowerbound, realexent, ierr)
38      size = m * realexent
39      disp_int = realexent
40      CALL MPI_WIN_CREATE(B, size, disp_int, MPI_INFO_NULL, &
41                          comm, win, ierr)
42
43      CALL MPI_WIN_FENCE(0, win, ierr)
44      DO i=1,m
45          j = map(i)/m
46          disp_aint = MOD(map(i),m)
47          CALL MPI_ACCUMULATE(A(i), 1, MPI_REAL, j, disp_aint, 1, MPI_REAL, &
48                              MPI_SUM, win, ierr)
49      END DO

```



```

CALL MPI_WIN_FENCE(0, win, ierr)

CALL MPI_WIN_FREE(win, ierr)
RETURN
END

```

This code is identical to the code in Example 12.2, except that a call to `MPI_GET` has been replaced by a call to `MPI_ACCUMULATE`. (Note that, if `map` is one-to-one, the code computes $B = A(\text{map}^{-1})$, which is the reverse assignment to the one computed in that previous example.) In a similar manner, we can replace in Example 12.1, the call to `get` by a call to `accumulate`, thus performing the computation with only one communication between any two MPI processes.

Get Accumulate

It is often useful to have fetch-and-accumulate semantics such that the remote data is returned to the caller before the sent data is accumulated into the remote data. The `get` and `accumulate` steps are executed atomically for each basic element in the datatype (see Section 12.7 for details). The predefined operator `MPI_REPLACE` provides fetch-and-set behavior.

```

MPI_GET_ACCUMULATE(origin_addr, origin_count, origin_datatype, result_addr,
                   result_count, result_datatype, target_rank, target_disp, target_count,
                   target_datatype, op, win)

```

IN	origin_addr	initial address of buffer (choice)
IN	origin_count	number of entries in origin buffer (nonnegative integer)
IN	origin_datatype	datatype of each entry in origin buffer (handle)
OUT	result_addr	initial address of result buffer (choice)
IN	result_count	number of entries in result buffer (nonnegative integer)
IN	result_datatype	datatype of each entry in result buffer (handle)
IN	target_rank	rank of target (nonnegative integer)
IN	target_disp	displacement from start of window to beginning of target buffer (nonnegative integer)
IN	target_count	number of entries in target buffer (nonnegative integer)
IN	target_datatype	datatype of each entry in target buffer (handle)
IN	op	accumulate operator (handle)
IN	win	window object (handle)

C binding

```

int MPI_Get_accumulate(const void *origin_addr, int origin_count,
                      MPI_Datatype origin_datatype, void *result_addr,
                      int result_count, MPI_Datatype result_datatype, int target_rank,

```

```

1      MPI_Aint target_disp, int target_count,
2      MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
3
4  int MPI_Get_accumulate_c(const void *origin_addr, MPI_Count origin_count,
5      MPI_Datatype origin_datatype, void *result_addr,
6      MPI_Count result_count, MPI_Datatype result_datatype,
7      int target_rank, MPI_Aint target_disp, MPI_Count target_count,
8      MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)

```

Fortran 2008 binding

```

10 MPI_Get_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
11     result_count, result_datatype, target_rank, target_disp,
12     target_count, target_datatype, op, win, ierror)
13 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
14 INTEGER, INTENT(IN) :: origin_count, result_count, target_rank,
15     target_count
16 TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
17     target_datatype
18 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
19 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
20 TYPE(MPI_Op), INTENT(IN) :: op
21 TYPE(MPI_Win), INTENT(IN) :: win
22 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24 MPI_Get_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
25     result_count, result_datatype, target_rank, target_disp,
26     target_count, target_datatype, op, win, ierror) !(_c)
27 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
28 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, result_count,
29     target_count
30 TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
31     target_datatype
32 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
33 INTEGER, INTENT(IN) :: target_rank
34 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
35 TYPE(MPI_Op), INTENT(IN) :: op
36 TYPE(MPI_Win), INTENT(IN) :: win
37 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

39 MPI_GET_ACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_ADDR,
40     RESULT_COUNT, RESULT_DATATYPE, TARGET_RANK, TARGET_DISP,
41     TARGET_COUNT, TARGET_DATATYPE, OP, WIN, IERROR)
42 <type> ORIGIN_ADDR(*), RESULT_ADDR(*)
43 INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_COUNT, RESULT_DATATYPE,
44     TARGET_RANK, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, IERROR
45 INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

Accumulate `origin_count` elements of type `origin_datatype` from the origin buffer (`origin_addr`) to the buffer at offset `target_disp`, in the target window specified by `target_rank` and `win`, using the operator `op` and return in the result buffer `result_addr` the content

of the target buffer before the accumulation, specified by `target_disp`, `target_count`, and `target_datatype`. The data transferred from origin to target must fit, without truncation, in the target buffer. Likewise, the data copied from target to origin must fit, without truncation, in the result buffer.

The origin and result buffers (`origin_addr` and `result_addr`) must be disjoint. Each datatype argument must be a predefined datatype or a derived datatype where all basic components are of the same predefined datatype. All datatype arguments must be constructed from the same predefined datatype. The operator `op` applies to elements of that predefined type. `target_datatype` must not specify overlapping entries, and the target buffer must fit in the target window or in attached memory in a dynamic window. The operation is executed atomically for each basic datatype; see Section 12.7 for details.

Any of the predefined operators for `MPI_REDUCE`, as well as `MPI_NO_OP` or `MPI_REPLACE` can be specified as `op`. User-defined functions cannot be used. An additional predefined operator, `MPI_NO_OP`, is defined. It corresponds to the associative function $f(a, b) = a$; i.e., the current value in the target memory is returned in the result buffer at the origin and the target buffer is not updated. If `MPI_NO_OP` is specified as the operator, the `origin_addr`, `origin_count`, and `origin_datatype` arguments are ignored. `MPI_NO_OP` can be used only in `MPI_GET_ACCUMULATE`, `MPI_RGET_ACCUMULATE`, and `MPI_FETCH_AND_OP`. `MPI_NO_OP` cannot be used in `MPI_ACCUMULATE`, `MPI_RACCUMULATE`, or collective reduction operations, such as `MPI_REDUCE` and others.

Advice to users. `MPI_GET` is similar to `MPI_GET_ACCUMULATE`, with the operator `MPI_NO_OP`. Note, however, that `MPI_GET` and `MPI_GET_ACCUMULATE` have different constraints on concurrent updates. (*End of advice to users.*)

Fetch and Op

The generic functionality of `MPI_GET_ACCUMULATE` might limit the performance of fetch-and-increment or fetch-and-add calls that might be supported by special hardware operations. `MPI_FETCH_AND_OP` thus allows for a fast implementation of a commonly used subset of the functionality of `MPI_GET_ACCUMULATE`.

`MPI_FETCH_AND_OP(origin_addr, result_addr, datatype, target_rank, target_disp, op, win)`

IN	<code>origin_addr</code>	initial address of buffer (choice)
OUT	<code>result_addr</code>	initial address of result buffer (choice)
IN	<code>datatype</code>	datatype of the entry in origin, result, and target buffers (handle)
IN	<code>target_rank</code>	rank of target (nonnegative integer)
IN	<code>target_disp</code>	displacement from start of window to beginning of target buffer (nonnegative integer)
IN	<code>op</code>	accumulate operator (handle)
IN	<code>win</code>	window object (handle)

C binding

`int MPI_Fetch_and_op(const void *origin_addr, void *result_addr,`

```

1      MPI_Datatype datatype, int target_rank, MPI_Aint target_disp,
2      MPI_Op op, MPI_Win win)
3

```

Fortran 2008 binding

```

4 MPI_Fetch_and_op(origin_addr, result_addr, datatype, target_rank, target_disp,
5     op, win, ierror)
6     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
7     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
8     TYPE(MPI_Datatype), INTENT(IN) :: datatype
9     INTEGER, INTENT(IN) :: target_rank
10    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
11    TYPE(MPI_Op), INTENT(IN) :: op
12    TYPE(MPI_Win), INTENT(IN) :: win
13    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
14

```

Fortran binding

```

15 MPI_FETCH_AND_OP(ORIGIN_ADDR, RESULT_ADDR, DATATYPE, TARGET_RANK, TARGET_DISP,
16     OP, WIN, IERROR)
17     <type> ORIGIN_ADDR(*), RESULT_ADDR(*)
18     INTEGER DATATYPE, TARGET_RANK, OP, WIN, IERROR
19     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP
20

```

Accumulate one element of type `datatype` from the origin buffer `origin_addr` to the buffer at offset `target_disp`, in the target window specified by `target_rank` and `win`, using the operator `op` and return in the result buffer `result_addr` the content of the target buffer before the accumulation.

The origin and result buffers (`origin_addr` and `result_addr`) must be disjoint. Any of the predefined operators for `MPI_REDUCE`, as well as `MPI_NO_OP` or `MPI_REPLACE`, can be specified as `op`; user-defined functions cannot be used. The `datatype` argument must be a predefined datatype. The operation is executed atomically.

Compare and Swap

Another useful operation is an atomic compare and swap where the value at the origin is compared to the value at the target, which is atomically replaced by a third value only if the values at origin and target are equal.

```

36 MPI_COMPARE_AND_SWAP(origin_addr, compare_addr, result_addr, datatype,
37     target_rank, target_disp, win)
38
39     IN      origin_addr      initial address of buffer (choice)
40     IN      compare_addr     initial address of compare buffer (choice)
41     OUT     result_addr      initial address of result buffer (choice)
42     IN      datatype         datatype of the element in all buffers (handle)
43     IN      target_rank      rank of target (nonnegative integer)
44     IN      target_disp      displacement from start of window to beginning of
45                               target buffer (nonnegative integer)
46     IN      win              window object (handle)
47
48

```

C binding

```
int MPI_Compare_and_swap(const void *origin_addr, const void *compare_addr,
                        void *result_addr, MPI_Datatype datatype, int target_rank,
                        MPI_Aint target_disp, MPI_Win win)
```

Fortran 2008 binding

```
MPI_Compare_and_swap(origin_addr, compare_addr, result_addr, datatype,
                    target_rank, target_disp, win, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr,
                    compare_addr
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, INTENT(IN) :: target_rank
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMPARE_AND_SWAP(ORIGIN_ADDR, COMPARE_ADDR, RESULT_ADDR, DATATYPE,
                    TARGET_RANK, TARGET_DISP, WIN, IERROR)
    <type> ORIGIN_ADDR(*), COMPARE_ADDR(*), RESULT_ADDR(*)
    INTEGER DATATYPE, TARGET_RANK, WIN, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP
```

This function compares one element of type `datatype` in the compare buffer `compare_addr` with the buffer at offset `target_disp` in the target window specified by `target_rank` and `win` and replaces the value at the target with the value in the origin buffer `origin_addr` if the compare buffer and the target buffer are identical. The original value at the target is returned in the buffer `result_addr`. The parameter `datatype` must belong to one of the following categories of predefined datatypes: C integer, Fortran integer, Logical, Multi-language types, or Byte as specified in Section 6.9.2. The origin and result buffers (`origin_addr` and `result_addr`) must be disjoint.

12.3.5 Request-based RMA Communication Operations

Request-based RMA communication operations allow the user to associate a request handle with the RMA operations and test or wait for the completion of these requests using the functions described in Section 3.7.3. Request-based RMA operations are only valid within a passive target epoch (see Section 12.5).

Upon returning from a completion call in which an RMA operation completes, all fields of the status object, if any, and the results of status query functions (e.g., `MPI_GET_COUNT`) are undefined with the exception of `MPI_ERROR` if appropriate (see Section 3.2.5). It is valid to mix different request types (e.g., any combination of RMA requests, collective requests, I/O requests, generalized requests, or point-to-point requests) in functions that enable multiple completions (e.g., `MPI_WAITALL`). It is erroneous to call `MPI_REQUEST_FREE` or `MPI_CANCEL` for a request associated with an RMA operation. RMA requests are not persistent.

The closing of the epoch, or explicit bulk synchronization using `MPI_WIN_FLUSH`, `MPI_WIN_FLUSH_ALL`, `MPI_WIN_FLUSH_LOCAL`, or `MPI_WIN_FLUSH_LOCAL_ALL`,

also indicates completion of request-based RMA operations on the specified window. However, users must still wait or test on the request handle to allow the MPI implementation to release any resources associated with these requests.

```

MPI_RPUT(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, request)
    IN      origin_addr      initial address of origin buffer (choice)
    IN      origin_count     number of entries in origin buffer (nonnegative
                              integer)
    IN      origin_datatype  datatype of each entry in origin buffer (handle)
    IN      target_rank      rank of target (nonnegative integer)
    IN      target_disp      displacement from start of window to target buffer
                              (nonnegative integer)
    IN      target_count     number of entries in target buffer (nonnegative
                              integer)
    IN      target_datatype  datatype of each entry in target buffer (handle)
    IN      win              window used for communication (handle)
    OUT     request          RMA request (handle)

```

C binding

```

int MPI_Rput(const void *origin_addr, int origin_count,
             MPI_Datatype origin_datatype, int target_rank,
             MPI_Aint target_disp, int target_count,
             MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)

int MPI_Rput_c(const void *origin_addr, MPI_Count origin_count,
              MPI_Datatype origin_datatype, int target_rank,
              MPI_Aint target_disp, MPI_Count target_count,
              MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Rput(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, request, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
    INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rput(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER, INTENT(IN) :: target_rank

```

```

INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_RPUT(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
        TARGET_COUNT, TARGET_DATATYPE, WIN, REQUEST, IERROR)
<type> ORIGIN_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
        TARGET_DATATYPE, WIN, REQUEST, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

MPI_RPUT is similar to **MPI_PUT** (Section 12.3.1), except that it allocates a communication request object and associates it with the request handle (the argument **request**). The completion of the operation at the origin (i.e., after the corresponding test or wait) indicates that the sender is now free to update the locations in the origin buffer. It does not indicate that the data is available at the target window. If remote completion is required, **MPI_WIN_FLUSH**, **MPI_WIN_FLUSH_ALL**, **MPI_WIN_UNLOCK**, or **MPI_WIN_UNLOCK_ALL** can be used.

```

MPI_RGET(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, request)

```

OUT	origin_addr	initial address of origin buffer (choice)
IN	origin_count	number of entries in origin buffer (nonnegative integer)
IN	origin_datatype	datatype of each entry in origin buffer (handle)
IN	target_rank	rank of target (nonnegative integer)
IN	target_disp	displacement from window start to the beginning of the target buffer (nonnegative integer)
IN	target_count	number of entries in target buffer (nonnegative integer)
IN	target_datatype	datatype of each entry in target buffer (handle)
IN	win	window used for communication (handle)
OUT	request	RMA request (handle)

C binding

```

int MPI_Rget(void *origin_addr, int origin_count, MPI_Datatype origin_datatype,
            int target_rank, MPI_Aint target_disp, int target_count,
            MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)

int MPI_Rget_c(void *origin_addr, MPI_Count origin_count,
              MPI_Datatype origin_datatype, int target_rank,
              MPI_Aint target_disp, MPI_Count target_count,
              MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Rget(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, request, ierror)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
    INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rget(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, request, ierror) !(_c)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
    TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
    INTEGER, INTENT(IN) :: target_rank
    INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
    TYPE(MPI_Win), INTENT(IN) :: win
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_RGET(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
        TARGET_COUNT, TARGET_DATATYPE, WIN, REQUEST, IERROR)
    <type> ORIGIN_ADDR(*)
    INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
        TARGET_DATATYPE, WIN, REQUEST, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

MPI_RGET is similar to **MPI_GET** (Section 12.3.2), except that it allocates a communication request object and associates it with the request handle (the argument **request**) that can be used to wait or test for completion of the operation at the origin, which indicates that the data is available in the origin buffer. If **origin_addr** points to memory attached to a window, then the data becomes available in the private copy of this window.

```

MPI_RACCUMULATE(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, op, win, request)

```

IN	origin_addr	initial address of buffer (choice)
IN	origin_count	number of entries in buffer (nonnegative integer)
IN	origin_datatype	datatype of each entry in origin buffer (handle)
IN	target_rank	rank of target (nonnegative integer)
IN	target_disp	displacement from start of window to beginning of target buffer (nonnegative integer)
IN	target_count	number of entries in target buffer (nonnegative integer)
IN	target_datatype	datatype of each entry in target buffer (handle)

IN	op	accumulate operator (handle)
IN	win	window object (handle)
OUT	request	RMA request (handle)

C binding

```
int MPI_Raccumulate(const void *origin_addr, int origin_count,
    MPI_Datatype origin_datatype, int target_rank,
    MPI_Aint target_disp, int target_count,
    MPI_Datatype target_datatype, MPI_Op op, MPI_Win win,
    MPI_Request *request)
```

```
int MPI_Raccumulate_c(const void *origin_addr, MPI_Count origin_count,
    MPI_Datatype origin_datatype, int target_rank,
    MPI_Aint target_disp, MPI_Count target_count,
    MPI_Datatype target_datatype, MPI_Op op, MPI_Win win,
    MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Raccumulate(origin_addr, origin_count, origin_datatype, target_rank,
    target_disp, target_count, target_datatype, op, win, request,
    ierror)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Raccumulate(origin_addr, origin_count, origin_datatype, target_rank,
    target_disp, target_count, target_datatype, op, win, request,
    ierror) !(_c)
```

```
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_RACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK,
    TARGET_DISP, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, REQUEST,
    IERROR)
```

```
<type> ORIGIN_ADDR(*)
```

```

1      INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
2          TARGET_DATATYPE, OP, WIN, REQUEST, IERROR
3      INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

`MPI_RACCUMULATE` is similar to `MPI_ACCUMULATE` (Section 12.3.4), except that it allocates a communication request object and associates it with the request handle (the argument `request`) that can be used to wait or test for completion. The completion of the operation at the origin (i.e., after the corresponding test or wait) indicates that the origin buffer is free to be updated. It does not indicate that the operation has completed at the target window.

```

12 MPI_RGET_ACCUMULATE(origin_addr, origin_count, origin_datatype, result_addr,
13     result_count, result_datatype, target_rank, target_disp, target_count,
14     target_datatype, op, win, request)

```

15	IN	origin_addr	initial address of buffer (choice)
16	IN	origin_count	number of entries in origin buffer (nonnegative integer)
17	IN	origin_datatype	datatype of each entry in origin buffer (handle)
18	OUT	result_addr	initial address of result buffer (choice)
19	IN	result_count	number of entries in result buffer (nonnegative integer)
20	IN	result_datatype	datatype of entries in result buffer (handle)
21	IN	target_rank	rank of target (nonnegative integer)
22	IN	target_disp	displacement from start of window to beginning of target buffer (nonnegative integer)
23	IN	target_count	number of entries in target buffer (nonnegative integer)
24	IN	target_datatype	datatype of each entry in target buffer (handle)
25	IN	op	accumulate operator (handle)
26	IN	win	window object (handle)
27	OUT	request	RMA request (handle)

C binding

```

37 int MPI_Rget_accumulate(const void *origin_addr, int origin_count,
38     MPI_Datatype origin_datatype, void *result_addr,
39     int result_count, MPI_Datatype result_datatype, int target_rank,
40     MPI_Aint target_disp, int target_count,
41     MPI_Datatype target_datatype, MPI_Op op, MPI_Win win,
42     MPI_Request *request)
43
44 int MPI_Rget_accumulate_c(const void *origin_addr, MPI_Count origin_count,
45     MPI_Datatype origin_datatype, void *result_addr,
46     MPI_Count result_count, MPI_Datatype result_datatype,
47     int target_rank, MPI_Aint target_disp, MPI_Count target_count,
48

```

```

        MPI_Datatype target_datatype, MPI_Op op, MPI_Win win,
        MPI_Request *request)

```

Fortran 2008 binding

```

MPI_Rget_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
    result_count, result_datatype, target_rank, target_disp,
    target_count, target_datatype, op, win, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, result_count, target_rank,
    target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
    target_datatype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rget_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
    result_count, result_datatype, target_rank, target_disp,
    target_count, target_datatype, op, win, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, result_count,
    target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
    target_datatype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_RGET_ACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_ADDR,
    RESULT_COUNT, RESULT_DATATYPE, TARGET_RANK, TARGET_DISP,
    TARGET_COUNT, TARGET_DATATYPE, OP, WIN, REQUEST, IERROR)
<type> ORIGIN_ADDR(*), RESULT_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_COUNT, RESULT_DATATYPE,
    TARGET_RANK, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, REQUEST,
    IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```

MPI_RGET_ACCUMULATE is similar to **MPI_GET_ACCUMULATE** (Section 12.3.4), except that it allocates a communication request object and associates it with the request handle (the argument *request*) that can be used to wait or test for completion. The completion of the operation at the origin (i.e., after the corresponding test or wait) indicates that the data is available in the result buffer and the origin buffer is free to be updated. It

does not indicate that the operation has been completed at the target window.

12.4 Memory Model

The memory semantics of RMA are best understood by using the concept of *public* and *private* window copies. We assume that systems have a public memory region that is addressable by all MPI processes (e.g., the shared memory in shared memory machines or the exposed main memory in distributed memory machines). In addition, most machines have fast private buffers (e.g., transparent caches or explicit communication buffers) local to each MPI process where copies of data elements from the main memory can be stored for faster access. Such buffers are either coherent, i.e., all updates to main memory are reflected in all private copies consistently, or noncoherent, i.e., conflicting accesses to main memory need to be synchronized and updated in all private copies explicitly. Coherent systems allow direct updates to remote memory without any participation of the remote side. Noncoherent systems, however, need to call RMA functions in order to reflect updates to the public window in their private memory. Thus, in coherent memory, the public and the private window are identical while they remain logically separate in the noncoherent case. MPI thus differentiates between two **memory models** called **RMA unified**, if public and private window are logically identical, and **RMA separate**, otherwise.

In the RMA separate model, there is only one instance of each variable in MPI process memory, but a distinct *public* copy of the variable for each window that contains it. A load accesses the instance in MPI process memory (this includes MPI sends). A local store accesses and updates the instance in MPI process memory (this includes MPI receives), but the update may affect other public copies of the same locations. A get on a window accesses the public copy of that window. A put or accumulate on a window accesses and updates the public copy of that window, but the update may affect the private copy of the same locations in MPI process memory, and public copies of other overlapping windows. This is illustrated in Figure 12.1.

In the RMA unified model, public and private copies are identical and updates via put or accumulate operations are eventually observed by load accesses without additional RMA procedure calls. A store access to a window is eventually visible to remote get or accumulate operations without additional RMA procedure calls. These stronger semantics of the RMA unified model allow the user to omit some synchronization calls and potentially improve performance.

Advice to users. If accesses in the RMA unified model are not synchronized (with locks or flushes, see Section 12.5.3), load/store accesses might observe changes to the memory while they are in progress. The order in which data is written is not specified unless further synchronization is used. This might lead to inconsistent views on memory and programs that assume that a transfer is complete by only checking parts of the message are erroneous. (*End of advice to users.*)

The memory model for a particular RMA window can be determined by accessing the attribute `MPI_WIN_MODEL`. If the memory model is the unified model, the value of this attribute is `MPI_WIN_UNIFIED`; otherwise, the value is `MPI_WIN_SEPARATE`.

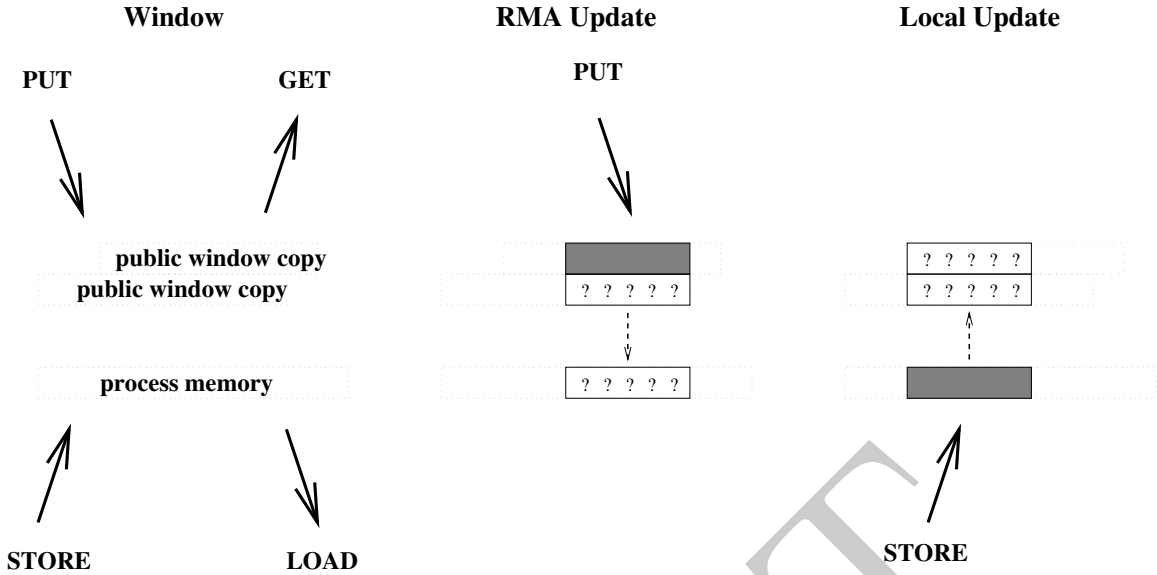


Figure 12.1: Schematic description of the public/private window operations in the `MPI_WIN_SEPARATE` memory model for two overlapping windows

12.5 Synchronization Calls

RMA communications fall in two categories:

active target communication, where data is moved from the memory of one MPI process to the memory of another, and both are explicitly involved in the communication. This communication pattern is similar to message passing, except that all the data transfer arguments are provided by the origin process, and the target process only participates in the synchronization.

passive target communication, where data is moved from the memory of one MPI process to the memory of another, and only the origin process is explicitly involved in the transfer. Thus, two origin processes may communicate by accessing the same location in a target window. The MPI process that owns the target window may be distinct from the two communicating MPI processes, in which case it does not participate explicitly in the communication. This communication paradigm is closest to a shared memory model, where shared data can be accessed by all MPI processes, irrespective of location.

RMA communication calls with argument `win` must occur at an origin process only within an **access epoch** for `win`. Such an epoch is opened with an RMA synchronization call on `win`; it proceeds with zero or more RMA communication calls (e.g., `MPI_PUT`, `MPI_GET` or `MPI_ACCUMULATE`) on `win`; it is closed with another synchronization call on `win`. This allows users to amortize one synchronization with multiple data transfers and provide implementors more flexibility in the implementation of RMA operations.

Distinct access epochs for `win` at the same MPI process must be disjoint. On the other hand, epochs pertaining to different `win` arguments may overlap. Load/store accesses or other MPI calls may also occur during an epoch.

In active target communication, a target window can be accessed by RMA operations only within an **exposure epoch**. Such an epoch is opened and closed by RMA synchronization calls executed by the target process. Distinct exposure epochs at an MPI process on

the same window must be disjoint, but such an exposure epoch may overlap with exposure epochs on other windows or with access epochs for the same or other window arguments. There is a one-to-one matching between access epochs at origin processes and exposure epochs on target processes: RMA operations issued by an origin process for a target window will access that target window during the same exposure epoch if and only if they were issued during the same access epoch.

In passive target communication the target process does not execute RMA synchronization calls, and there is no concept of an exposure epoch.

MPI provides three synchronization mechanisms:

1. The `MPI_WIN_FENCE` collective synchronization call supports a simple synchronization pattern that is often used in parallel computations: namely a loosely-synchronous model, where global computation phases alternate with global communication phases. This mechanism is most useful for loosely synchronous algorithms where the graph of communicating MPI processes changes very frequently, or where each MPI process communicates with many others.

This call is used for active target communication. An access epoch at an origin process or an exposure epoch at a target process is opened and closed by calls to `MPI_WIN_FENCE`. An origin process can access windows at all target processes in the group of `win` during such an access epoch, and the local window can be accessed by all MPI processes in the group of `win` during such an exposure epoch.

2. The four functions `MPI_WIN_START`, `MPI_WIN_COMPLETE`, `MPI_WIN_POST`, and `MPI_WIN_WAIT` can be used to restrict synchronization to the minimum: only pairs of communicating MPI processes synchronize, and they do so only when a synchronization is needed to order RMA accesses to a window correctly with respect to local accesses to that same window. This mechanism may be more efficient when each MPI process communicates with few (logical) neighbors, and the communication graph is fixed or changes infrequently.

These calls are used for active target communication. An access epoch is opened at the origin process with a call to `MPI_WIN_START` and is closed by a call to `MPI_WIN_COMPLETE`. The start call has a group argument that specifies the group of target processes for that epoch. An exposure epoch is opened at the target process by a call to `MPI_WIN_POST` and is closed by a call to `MPI_WIN_WAIT`. The post call has a group argument that specifies the set of origin processes for that epoch.

3. Finally, *shared lock* access is provided by the functions `MPI_WIN_LOCK`, `MPI_WIN_LOCK_ALL`, `MPI_WIN_UNLOCK`, and `MPI_WIN_UNLOCK_ALL`. `MPI_WIN_LOCK` and `MPI_WIN_UNLOCK` also provide *exclusive lock* capability. Lock synchronization is useful for MPI applications that emulate a shared memory model via MPI calls; e.g., in a “bulletin board” model, where MPI processes can, at random times, access or update different parts of the bulletin board.

These four calls provide passive target communication. An access epoch is opened by a call to `MPI_WIN_LOCK` or `MPI_WIN_LOCK_ALL` and closed by a call to `MPI_WIN_UNLOCK` or `MPI_WIN_UNLOCK_ALL`, respectively.

Figure 12.2 illustrates the general synchronization pattern for active target communication. The synchronization between `post` and `start` ensures that the put operation of the

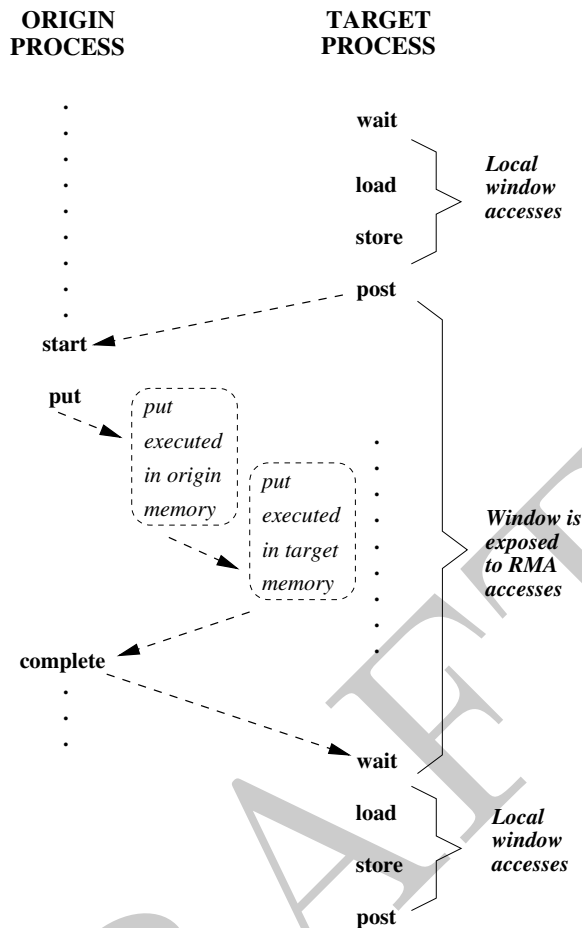


Figure 12.2: Active target communication. Dashed arrows represent synchronizations (ordering of events).

origin process does not start until the target process exposes the window (with the `post` call); the target process will expose the window only after preceding local accesses to the window have completed. The synchronization between `complete` and `wait` ensures that the `put` operation of the origin process completes at the origin and the target before the window is unexposed (with the `wait` call). The target process will execute subsequent local accesses to the target window only after the `wait` returned.

Figure 12.2 shows operations occurring in the natural temporal order implied by the synchronizations: the `post` occurs before the matching `start`, and `complete` occurs before the matching `wait`. However, such **strong synchronization** is more than needed for correct ordering of window accesses. The semantics of MPI calls allow **weak synchronization**, as illustrated in Figure 12.3. The access to the target window is delayed until the window is exposed, after the `post`. However the `start` may return before the exposure epoch opens at the target. Similarly, the `put` and `complete` calls may also return before the exposure epoch opens at the target, if `put` data is buffered by the implementation. The synchronization calls correctly order window accesses, but do not necessarily synchronize other operations. This weaker synchronization semantic allows for more efficient implementations.

Figure 12.4 illustrates the general synchronization pattern for passive target communication. The first origin process communicates data to the second origin process, through

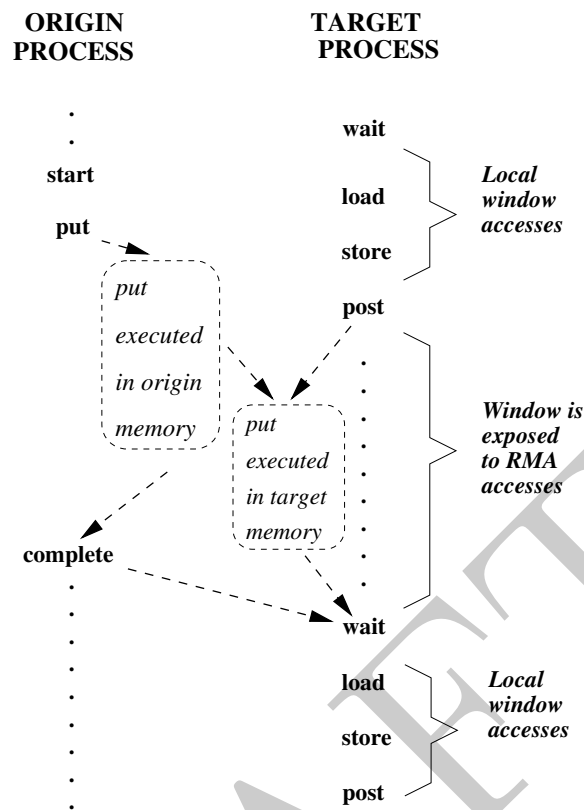


Figure 12.3: Active target communication, with weak synchronization. Dashed arrows represent synchronizations (ordering of events).

the memory of the target process; the target process is not explicitly involved in the communication. The `lock` and `unlock` calls ensure that the two RMA accesses do not occur concurrently. However, they do *not* ensure that the `put` by origin 1 will precede the `get` by origin 2.

Rationale. RMA does not define fine-grained mutexes in memory (only logical coarse-grained window locks). MPI provides the primitives (compare and swap, accumulate, send/receive, etc.) needed to implement high-level synchronization operations. (*End of rationale.*)

12.5.1 Fence

MPI_WIN_FENCE(assert, win)

IN	assert	program assertion (integer)
IN	win	window object (handle)

C binding

```
int MPI_Win_fence(int assert, MPI_Win win)
```

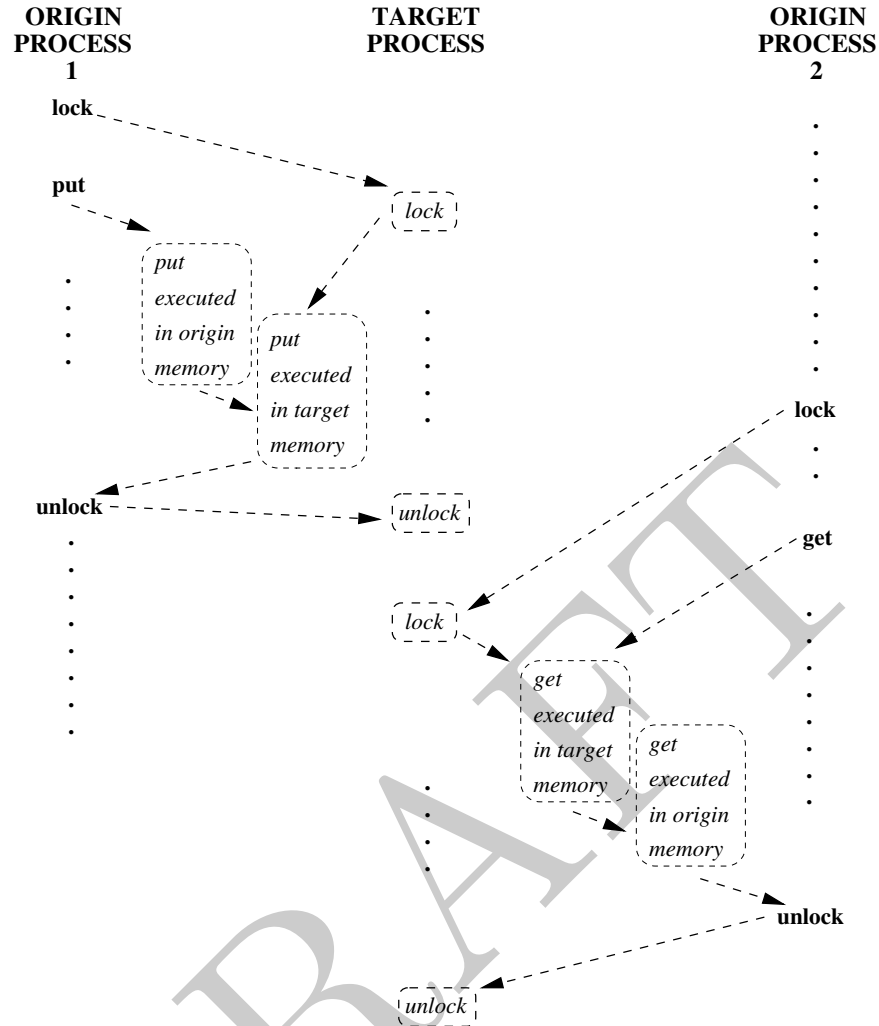



Figure 12.4: Passive target communication. Dashed arrows represent synchronizations (ordering of events).

Fortran 2008 binding

```
MPI_Win_fence(assert, win, ierror)
  INTEGER, INTENT(IN) :: assert
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_FENCE(ASSERT, WIN, IERROR)
  INTEGER ASSERT, WIN, IERROR
```

MPI_WIN_FENCE synchronizes RMA communication operations on win. The procedure is collective over the group of win. All RMA operations on win originating at a given origin process and started before the fence call will complete at that MPI process before the fence call returns. They will be completed at their target before the fence call returns at the target. Store accesses to shared-memory of win will become visible before the fence call returns at the target. RMA operations on win started by an origin process after the fence

call returns will access their target window only after `MPI_WIN_FENCE` has been called by the target process.

The call closes an RMA access epoch if it was preceded by another fence call and the local MPI process initiated any RMA communication operations on win between these two calls. The call closes an RMA exposure epoch if it was preceded by another fence call and the local window was the target of RMA accesses between these two calls. The call opens an RMA access epoch if it is followed by another fence call and by RMA communication calls issued between these two fence calls. The call opens an exposure epoch if it is followed by another fence call and the local window is the target of RMA accesses between these two fence calls. Thus, the fence call is equivalent to calls to a subset of `post`, `start`, `complete`, `wait`.

A call to `MPI_WIN_FENCE` is usually synchronizing. However, a call to `MPI_WIN_FENCE` that is known not to close any epoch (in particular, a call with the `MPI_MODE_NOPRECEDE` assert set) is not necessarily synchronizing.

The `assert` argument is used to provide assertions on the context of the call that may be used for various optimizations. This is described in Section 12.5.5. A value of `assert = 0` is always valid.

Advice to users. Calls to `MPI_WIN_FENCE` should both precede and follow calls to RMA communication procedures that are synchronized with fence calls. (*End of advice to users.*)

12.5.2 General Active Target Synchronization

`MPI_WIN_START(group, assert, win)`

IN	group	group of target processes (handle)
IN	assert	program assertion (integer)
IN	win	window object (handle)

C binding

`int MPI_Win_start(MPI_Group group, int assert, MPI_Win win)`

Fortran 2008 binding

```

MPI_Win_start(group, assert, win, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: assert
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_START(GROUP, ASSERT, WIN, IERROR)
  INTEGER GROUP, ASSERT, WIN, IERROR

```

Opens an RMA access epoch for win. RMA calls issued on win during this epoch must access only windows at MPI processes in group. Each MPI process in group must issue a matching call to `MPI_WIN_POST`. RMA accesses to each target window will be delayed, if necessary, until the target process executed the matching call to

MPI_WIN_POST. **MPI_WIN_START** is allowed to delay its return until the corresponding calls to **MPI_WIN_POST** have occurred, but is not required to.

The **assert** argument is used to provide assertions on the context of the call that may be used for various optimizations. This is described in Section 12.5.5. A value of **assert** = 0 is always valid.

MPI_WIN_COMPLETE(win)

IN win window object (handle)

C binding

```
int MPI_Win_complete(MPI_Win win)
```

Fortran 2008 binding

```
MPI_Win_complete(win, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_COMPLETE(WIN, IERROR)
  INTEGER WIN, IERROR
```

Closes an RMA access epoch on **win** opened by a call to **MPI_WIN_START**. All RMA communication operations initiated on **win** during this epoch will have completed at the origin when the call returns. All updates to shared memory in **win** through load/store accesses executed during this epoch will be visible at the target when the call returns.

MPI_WIN_COMPLETE enforces completion of preceding RMA operations and visibility of load/store accesses at the origin, but not at the target. A put or accumulate operation may not have completed at the target when it has completed at the origin.

Consider the sequence of calls in the example below.

Example 12.4. Use of **MPI_WIN_START** and **MPI_WIN_COMPLETE**.

```
MPI_Win_start(group, flag, win);
MPI_Put(..., win);
MPI_Win_complete(win);
```

The call to **MPI_WIN_COMPLETE** does not return until the put operation has completed at the origin; and the target window will be accessed by the put operation only after the call to **MPI_WIN_START** has matched a call to **MPI_WIN_POST** by the target process.

Advice to implementors. The semantics described above still leave much choice to implementors. The return from the call to **MPI_WIN_START** can block until the matching call to **MPI_WIN_POST** occurs at all target processes. One can also have implementations where the call to **MPI_WIN_START** returns immediately, but the call to **MPI_WIN_COMPLETE** delays its return until the call to **MPI_WIN_POST** occurred; or implementations where all three calls can complete before any target process has called **MPI_WIN_POST**—the data put must be buffered, in this last case, so as to allow the put to complete at the origin ahead of its completion at the target. However, once the call to **MPI_WIN_POST** is issued, the sequence above must complete, without further dependencies. (*End of advice to implementors.*)

Advice to users. In order to ensure a portable deadlock free program, users must assume that `MPI_WIN_START` may delay its return until the corresponding call to `MPI_WIN_POST` has occurred. (*End of advice to users.*)

`MPI_WIN_POST(group, assert, win)`

IN	group	group of origin processes (handle)
IN	assert	program assertion (integer)
IN	win	window object (handle)

C binding

`int MPI_Win_post(MPI_Group group, int assert, MPI_Win win)`

Fortran 2008 binding

```
MPI_Win_post(group, assert, win, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: assert
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_POST(GROUP, ASSERT, WIN, IERROR)
  INTEGER GROUP, ASSERT, WIN, IERROR
```

Opens an RMA exposure epoch for the local window associated with `win`. Only MPI processes in `group` may access the window with RMA calls on `win` during this epoch. Each MPI process in `group` must issue a matching call to `MPI_WIN_START`. `MPI_WIN_POST` is a *local* procedure.

`MPI_WIN_WAIT(win)`

IN	win	window object (handle)
----	-----	------------------------

C binding

`int MPI_Win_wait(MPI_Win win)`

Fortran 2008 binding

```
MPI_Win_wait(win, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_WAIT(WIN, IERROR)
  INTEGER WIN, IERROR
```

Closes an RMA exposure epoch opened by a call to `MPI_WIN_POST` on `win`. This call matches calls to `MPI_WIN_COMPLETE` on `win` issued by each of the origin processes that were granted access to the window during this epoch. The call to `MPI_WIN_WAIT` will return only after all matching calls to `MPI_WIN_COMPLETE` have occurred. This

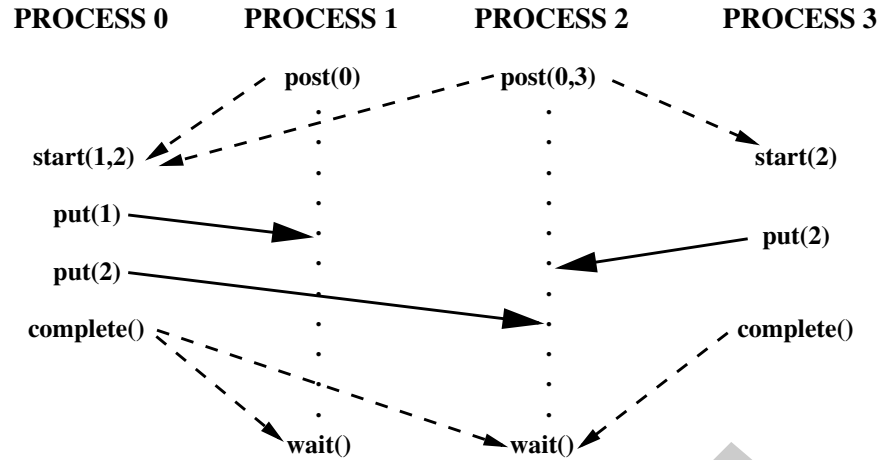


Figure 12.5: Active target communication, with strong synchronization. Dashed arrows represent synchronizations and solid arrows represent data transfer.

guarantees that all these origin processes have completed their RMA operations and shared-memory load/store accesses have become visible on the local window. When the call returns, all these RMA accesses will have completed at the target window.

Figure 12.5 illustrates the use of these four functions. Process 0 puts data in the windows of processes 1 and 2 and process 3 puts data in the window of process 2. Each start call lists the ranks of the MPI processes whose windows will be accessed; each post call lists the ranks of the MPI processes that access the local window. The figure illustrates a possible timing for the events, assuming strong synchronization; in a weak synchronization, the start, put or complete calls may occur ahead of the matching post calls.

MPI_WIN_TEST(win, flag)

IN	win	window object (handle)
OUT	flag	success flag (logical)

C binding

```
int MPI_Win_test(MPI_Win win, int *flag)
```

Fortran 2008 binding

```
MPI_Win_test(win, flag, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  LOGICAL, INTENT(OUT) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_TEST(WIN, FLAG, IERROR)
  INTEGER WIN, IERROR
  LOGICAL FLAG
```

MPI_WIN_TEST is a local procedure. Repeated calls to **MPI_WIN_TEST** with the same `win` argument will eventually return `flag = true` once all accesses to the local window by the group to which it was exposed by the corresponding call to **MPI_WIN_POST** have

been completed as indicated by matching `MPI_WIN_COMPLETE` calls, and `flag = false` otherwise. In the former case `MPI_WIN_WAIT` would have returned immediately. The effect of return of `MPI_WIN_TEST` with `flag = true` is the same as the effect of a return of `MPI_WIN_WAIT`. If `flag = false` is returned, then the call has no visible effect.

`MPI_WIN_TEST` should be called only where `MPI_WIN_WAIT` can be called. Once the call has returned `flag = true`, it must not be called again, until the window is posted again.

Assume that window `win` is associated with a “hidden” communicator `wincomm`, used for communication by the MPI processes in the group of `win`. The rules for matching of post and start calls and for matching complete and wait calls can be derived from the rules for matching sends and receives, by considering the following (partial) model implementation.

`MPI_WIN_POST(group,0,win)` initiates a nonblocking send with tag `tag0` to each MPI process in `group`, using `wincomm`.

`MPI_WIN_START(group,0,win)` initiates a nonblocking receive with tag `tag0` from each MPI process in `group`, using `wincomm`. An RMA access to a target process is delayed until the receive from that MPI process is completed.

`MPI_WIN_COMPLETE(win)` initiates a nonblocking send with tag `tag1` to each MPI process in the group of the preceding start call.

`MPI_WIN_WAIT(win)` initiates a nonblocking receive with tag `tag1` from each MPI process in the group of the preceding post call. Wait for the completion of all receives.

No races can occur in a correct program: each of the sends matches a unique receive, and vice versa.

Rationale. The design for general active target synchronization requires the user to provide complete information on the communication pattern, at each end of a communication link: each origin specifies a list of targets, and each target specifies a list of origins. This provides maximum flexibility (hence, efficiency) for the implementor: each synchronization can be initiated by either side, since each “knows” the identity of the other. This also provides maximum protection from possible races. On the other hand, the design requires more information than RMA needs: in general, it is sufficient for the origin to know the rank of the target, but not vice versa. Users that want more “anonymous” communication will be required to use the fence or lock mechanisms. (*End of rationale.*)

Advice to users. Assume a communication pattern that is represented by a directed graph $G = \langle V, E \rangle$, where $V = \{0, \dots, n-1\}$ and $ij \in E$ if origin process i accesses the window at target process j . Then each MPI process i issues a call to `MPI_WIN_POST(ingroupi, ...)`, followed by a call to `MPI_WIN_START(outgroupi, ...)`, where $outgroup_i = \{j : ij \in E\}$ and $ingroup_i = \{j : ji \in E\}$. A call is a no-op, and can be skipped, if the `group` argument is empty. After the communications calls, each MPI process that issued a start will issue a complete. Finally, each MPI process that issued a post will issue a wait.

Note that each MPI process may call with a `group` argument that has different members. (*End of advice to users.*)

12.5.3 Lock

Locks are used to protect accesses to the locked target window effected by RMA calls issued between the lock and unlock calls, and to protect load/store accesses to a locked local or shared memory window executed between the lock and unlock calls. Accesses that are protected by an **exclusive lock** (acquired using `MPI_LOCK_EXCLUSIVE`) will not be concurrent at the window site with other accesses to the same window that are lock protected. Accesses that are protected by a **shared lock** (acquired using `MPI_LOCK_SHARED`) will not be concurrent at the window site with accesses protected by an exclusive lock to the same window.

`MPI_WIN_LOCK(lock_type, rank, assert, win)`

IN	lock_type	either <code>MPI_LOCK_EXCLUSIVE</code> or <code>MPI_LOCK_SHARED</code> (state)
IN	rank	rank of target MPI process in the group of the window win (nonnegative integer)
IN	assert	program assertion (integer)
IN	win	window object (handle)

C binding

```
int MPI_Win_lock(int lock_type, int rank, int assert, MPI_Win win)
```

Fortran 2008 binding

```
MPI_Win_lock(lock_type, rank, assert, win, ierror)
  INTEGER, INTENT(IN) :: lock_type, rank, assert
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_LOCK(LOCK_TYPE, RANK, ASSERT, WIN, IERROR)
  INTEGER LOCK_TYPE, RANK, ASSERT, WIN, IERROR
```

Opens an RMA access epoch. The window at the MPI process with a rank of rank in the group of win can be accessed by RMA operations on win during that epoch. Multiple RMA access epochs (with calls to `MPI_WIN_LOCK`) can occur simultaneously; however, each access epoch must target a different MPI process.

`MPI_WIN_LOCK_ALL(assert, win)`

IN	assert	program assertion (integer)
IN	win	window object (handle)

C binding

```
int MPI_Win_lock_all(int assert, MPI_Win win)
```

Fortran 2008 binding

```
MPI_Win_lock_all(assert, win, ierror)
  INTEGER, INTENT(IN) :: assert
```

```

1      TYPE(MPI_Win), INTENT(IN) :: win
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

4      MPI_WIN_LOCK_ALL(ASSERT, WIN, IERROR)
5      INTEGER ASSERT, WIN, IERROR

```

Opens an RMA access epoch to all MPI processes in win, with a lock type of `MPI_LOCK_SHARED`. During the epoch, the calling MPI process can access the window memory on all MPI processes in win by using RMA operations. A window locked with `MPI_WIN_LOCK_ALL` must be unlocked with `MPI_WIN_UNLOCK_ALL`. This routine is not collective—the ALL refers to a lock on all members of the group of the window.

Advice to users. There may be additional overheads associated with using `MPI_WIN_LOCK` and `MPI_WIN_LOCK_ALL` concurrently on the same window. These overheads could be avoided by specifying the assertion `MPI_MODE_NOCHECK` when possible (see Section 12.5.5). (*End of advice to users.*)

```

19     MPI_WIN_UNLOCK(rank, win)

```

<pre> 21 IN rank 22 23 IN win </pre>	<pre> rank of target MPI process in the group of the window win (nonnegative integer) window object (handle) </pre>
--	---

C binding

```

26     int MPI_Win_unlock(int rank, MPI_Win win)

```

Fortran 2008 binding

```

28     MPI_Win_unlock(rank, win, ierror)
29     INTEGER, INTENT(IN) :: rank
30     TYPE(MPI_Win), INTENT(IN) :: win
31     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

34     MPI_WIN_UNLOCK(RANK, WIN, IERROR)
35     INTEGER RANK, WIN, IERROR

```

Closes an RMA access epoch opened by a call to `MPI_WIN_LOCK` on window win. RMA operations issued during this period will have completed both at the origin and at the target when the call returns.

```

41     MPI_WIN_UNLOCK_ALL(win)

```

<pre> 42 IN win </pre>	<pre> window object (handle) </pre>
---	-------------------------------------

C binding

```

46     int MPI_Win_unlock_all(MPI_Win win)

```


Fortran 2008 binding

```

MPI_Win_unlock_all(win, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_UNLOCK_ALL(WIN, IERROR)
    INTEGER WIN, IERROR

```

Closes a shared RMA access epoch opened by a call to `MPI_WIN_LOCK_ALL` on window `win`. RMA operations issued during this epoch will have completed both at the origin and at the target when the call returns.

It is erroneous to have a window locked and exposed (in an exposure epoch) concurrently. For example, an MPI process may not call `MPI_WIN_LOCK` to lock a target window if the target process has called `MPI_WIN_POST` and has not yet called `MPI_WIN_WAIT`; it is erroneous to call `MPI_WIN_POST` while the local window is locked.

Rationale. An alternative is to require MPI to enforce mutual exclusion between exposure epochs and locking periods. However, this would entail additional overheads when locks or active target synchronization do not interact in support of those rare interactions between the two mechanisms. The programming style that we encourage here is that a set of windows is used with only one synchronization mechanism at a time, with shifts from one mechanism to another being rare and involving global synchronization. (*End of rationale.*)

Advice to users. Users need to use explicit synchronization code in order to enforce mutual exclusion between locking periods and exposure epochs on a window. (*End of advice to users.*)

Implementors may restrict the use of RMA communication that is synchronized by lock calls to windows in memory allocated by `MPI_ALLOC_MEM` (Section 9.2), `MPI_WIN_ALLOCATE` (Section 12.2.2), `MPI_WIN_ALLOCATE_SHARED` (Section 12.2.3), or attached with `MPI_WIN_ATTACH` (Section 12.2.4). Locks can be used portably only in such memory.

Rationale. The implementation of passive target communication between processes in different *shared memory domains* may require an asynchronous software agent. Such an agent can be implemented more easily, and can achieve better performance, if restricted to specially allocated memory. It can be avoided altogether if *shared memory* is used. It seems natural to impose restrictions that allow the use of shared memory for RMA communication in shared memory machines.

(*End of rationale.*)

Consider the sequence of calls in the example below.

Example 12.5. Use of `MPI_WIN_LOCK` and `MPI_WIN_UNLOCK`.

```

MPI_Win_lock(MPI_LOCK_EXCLUSIVE, rank, assert, win);
MPI_Put(..., rank, ..., win);
MPI_Win_unlock(rank, win);

```

The call to `MPI_WIN_UNLOCK` will not return until the put transfer has completed at the origin and at the target.

Advice to implementors. The semantics described above still leave much freedom to implementors. Return from the call to `MPI_WIN_LOCK` may be delayed until an exclusive lock on the window is acquired; or, the first two calls may return immediately, while return from `MPI_WIN_UNLOCK` is delayed until a lock is acquired—the update of the target window is then postponed until the call to `MPI_WIN_UNLOCK` occurs. However, if the call to `MPI_WIN_LOCK` is used to lock a window accessible via load/store accesses (i.e., a local window or a window at an MPI process for which a pointer to shared memory can be obtained via `MPI_WIN_SHARED_QUERY`), then the call must not return before the lock is acquired, since the lock may protect load/store accesses to the window issued after the lock call returns. (*End of advice to implementors.*)

Advice to users. In order to ensure a portable deadlock free program, a user must assume that `MPI_WIN_LOCK` may delay its return until the desired lock on the window has been acquired. (*End of advice to users.*)

12.5.4 Flush and Sync

All flush and sync functions can be called only within passive target epochs.

`MPI_WIN_FLUSH(rank, win)`

IN	rank	rank of target MPI process in the group of the window win (nonnegative integer)
IN	win	window object (handle)

C binding

`int MPI_Win_flush(int rank, MPI_Win win)`

Fortran 2008 binding

```
MPI_Win_flush(rank, win, ierror)
  INTEGER, INTENT(IN) :: rank
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_WIN_FLUSH(RANK, WIN, IERROR)
  INTEGER RANK, WIN, IERROR
```

All outstanding RMA operations on win initiated by the MPI process calling this procedure to the target with rank in the group of the specified window will have completed when `MPI_WIN_FLUSH` returns. The operations are completed both at the origin and at the target.

Fortran 2008 binding

```

MPI_Win_flush_local_all(win, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_FLUSH_LOCAL_ALL(WIN, IERROR)
    INTEGER WIN, IERROR

```

All RMA operations initiated by the MPI process calling this procedure to any target on the specified window prior to this call will have completed at the origin when `MPI_WIN_FLUSH_LOCAL_ALL` returns.

MPI_WIN_SYNC(win)

```

IN      win      window object (handle)

```

C binding

```

int MPI_Win_sync(MPI_Win win)

```

Fortran 2008 binding

```

MPI_Win_sync(win, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_WIN_SYNC(WIN, IERROR)
    INTEGER WIN, IERROR

```

For windows in the separate memory model, a call to `MPI_WIN_SYNC` synchronizes the private and public window copies of `win` at the calling MPI process, as described in Section 12.7.

In the unified memory model, `MPI_WIN_SYNC` may be used to order load and store accesses to shared memory and to ensure visibility of store updates in shared memory for other threads and MPI processes.

A call to `MPI_WIN_SYNC` does not open or close an epoch and does not complete any pending RMA operations. A call to `MPI_WIN_SYNC` does not guarantee *progress* of any pending MPI operation.

12.5.5 Assertions

The `assert` argument in the calls `MPI_WIN_POST`, `MPI_WIN_START`, `MPI_WIN_FENCE`, `MPI_WIN_LOCK`, and `MPI_WIN_LOCK_ALL` is used to provide assertions on the context of the call that may be used to optimize performance. The `assert` argument does not change program semantics if it provides correct information on the program—it is erroneous to provide incorrect information. Users may always provide `assert = 0` to indicate a general case where no guarantees are made.

Advice to users. Many implementations may not take advantage of the information in `assert`; some of the information is relevant only for noncoherent shared memory machines. Users should consult their implementation's manual to find which information

is useful on each system. On the other hand, applications that provide correct assertions whenever applicable are portable and will take advantage of assertion specific optimizations whenever available. (*End of advice to users.*)

Advice to implementors. Implementations can always ignore the `assert` argument. Implementors should document which `assert` values are significant on their implementation. (*End of advice to implementors.*)

`assert` is the bit vector OR of zero or more of the following integer constants: `MPI_MODE_NOCHECK`, `MPI_MODE_NOSTORE`, `MPI_MODE_NOPUT`, `MPI_MODE_NOPRECEDE`, and `MPI_MODE_NOSUCCEED`. The significant options are listed below for each synchronization procedure.

Advice to users. C/C++ users can use bit vector OR (`|`) to combine these constants; Fortran 90 users can use the bit vector IOR intrinsic. Alternatively, Fortran users can portably use integer addition to OR the constants (each constant should appear at most once in the addition). (*End of advice to users.*)

MPI_WIN_START:

MPI_MODE_NOCHECK: the matching calls to `MPI_WIN_POST` have already completed on all target processes when the call to `MPI_WIN_START` is made. This option can be specified in a start call if and only if it is specified in each matching post call. This is similar to the optimization of “ready-send” that may save a handshake when the handshake is implicit in the code. However, ready-send is matched by a regular receive, whereas both start and post must specify the `MPI_MODE_NOCHECK` option.

MPI_WIN_POST:

MPI_MODE_NOCHECK: the matching calls to `MPI_WIN_START` have not yet occurred on any origin processes when the call to `MPI_WIN_POST` is made. This option can be specified by a post call if and only if it is specified by each matching start call.

MPI_MODE_NOSTORE: the local window was not updated by stores (or get or receive operations) since the last synchronization. This may avoid the need for cache synchronization during the post call.

MPI_MODE_NOPUT: the local window will not be updated by put or accumulate operations after the post call, until the ensuing (wait) synchronization. This may avoid the need for cache synchronization during the wait call.

MPI_WIN_FENCE:

MPI_MODE_NOSTORE: the local window was not updated by stores (or get or receive operations) since the last synchronization.

MPI_MODE_NOPUT: the local window will not be updated by put or accumulate operations after the fence call, until the ensuing (fence) synchronization.

MPI_MODE_NOPRECEDE: the fence does not complete any sequence of RMA operations initiated by the calling MPI process. If this assertion is given by any MPI process in the group of the window, then it must be given by all MPI processes in the group.

MPI_MODE_NOSUCCEED: the fence does not start any sequence of RMA operations initiated by the calling MPI process. If the assertion is given by any MPI process in the group of the window, then it must be given by all MPI processes in the group.

MPI_WIN_LOCK, MPI_WIN_LOCK_ALL:

MPI_MODE_NOCHECK: no other MPI process holds, or will attempt to acquire, a conflicting lock, while the calling MPI process holds the window lock. This is useful when mutual exclusion is achieved by other means, but the coherence operations that may be attached to the lock and unlock calls are still required.

Advice to users. The **MPI_MODE_NOSTORE** and **MPI_MODE_NOPRECEDE** options provide information on what happened *before* the call; the **MPI_MODE_NOPUT** and **MPI_MODE_NOSUCCEED** options provide information on what will happen *after* the call. (*End of advice to users.*)

12.5.6 Miscellaneous Clarifications

Once an RMA procedure call returns, it is safe to free any opaque objects passed as arguments to that procedure. For example, the **datatype** argument of an **MPI_PUT** call can be freed as soon as the call returns, even though the communication may not be complete.

As in message-passing, datatypes must be committed before they can be used in RMA communication.

12.6 Error Handling

12.6.1 Error Handlers

Errors occurring during calls to routines that create MPI windows (e.g., **MPI_WIN_CREATE**) cause an error to be raised on the communicator provided to that procedure call. All other RMA calls have an input window argument on which errors will be raised if they occur.

The error handler **MPI_ERRORS_ARE_FATAL** is associated with the window during its creation. Users may change this default by explicitly associating a new error handler with the window (see Section 9.3).

12.6.2 Error Classes

The error classes for one-sided communication are defined in Table 12.2. RMA routines may (and almost certainly will) use other MPI error classes, such as **MPI_ERR_OP** or **MPI_ERR_RANK**.

12.7 Semantics and Correctness

The following rules specify the latest point in the execution of the application an operation must complete at the origin or the target. The update initiated by a call to **MPI_GET** in the origin process memory is visible when the get operation is complete at the origin (or earlier); the update initiated by a call to **MPI_PUT** or an accumulate procedure in the public copy of the target window is visible when the put or accumulate operation has completed at the

Table 12.2: Error classes in one-sided communication routines

MPI_ERR_WIN	invalid win argument
MPI_ERR_BASE	invalid base argument
MPI_ERR_SIZE	invalid size argument
MPI_ERR_DISP	invalid disp argument
MPI_ERR_LOCKTYPE	invalid locktype argument
MPI_ERR_ASSERT	invalid assert argument
MPI_ERR_RMA_CONFLICT	conflicting accesses to window
MPI_ERR_RMA_SYNC	invalid synchronization of RMA calls
MPI_ERR_RMA_RANGE	target memory is not part of the window (in the case of a window created with <code>MPI_WIN_CREATE_DYNAMIC</code> , target memory is not attached)
MPI_ERR_RMA_ATTACH	memory cannot be attached (e.g., because of resource exhaustion)
MPI_ERR_RMA_SHARED	memory cannot be shared (e.g., some MPI process in the group of the specified communicator cannot expose <i>shared memory</i>)
MPI_ERR_RMA_FLAVOR	passed window has the wrong flavor for the called function

target (or earlier). The rules also specify the latest point at which an update of one window copy becomes visible in another overlapping copy.

1. An RMA operation is completed at the origin by the ensuing call to `MPI_WIN_COMPLETE`, `MPI_WIN_FENCE`, `MPI_WIN_FLUSH`, `MPI_WIN_FLUSH_ALL`, `MPI_WIN_FLUSH_LOCAL`, `MPI_WIN_FLUSH_LOCAL_ALL`, `MPI_WIN_UNLOCK`, or `MPI_WIN_UNLOCK_ALL` that synchronizes this access at the origin.
2. If an RMA operation is completed at the origin by a call to `MPI_WIN_FENCE` then the operation is completed at the target by the matching call to `MPI_WIN_FENCE` by the target process.
3. If an RMA operation is completed at the origin by a call to `MPI_WIN_COMPLETE` then the operation is completed at the target by the matching call to `MPI_WIN_WAIT` by the target process.
4. If an RMA operation is completed at the origin by a call to `MPI_WIN_UNLOCK` or `MPI_WIN_FLUSH` (with `rank=target`), `MPI_WIN_UNLOCK_ALL`, or `MPI_WIN_FLUSH_ALL`, then the operation is completed at the target by that same call.
5. An update of a location in a private window copy in MPI process memory becomes visible in the public window copy at the latest when an ensuing call to `MPI_WIN_POST`, `MPI_WIN_FENCE`, `MPI_WIN_UNLOCK`, `MPI_WIN_UNLOCK_ALL`, or `MPI_WIN_SYNC` is executed on that window by the window owner. In the RMA

unified memory model, an update of a location in a private window in MPI process memory becomes visible without additional RMA calls.

6. An update by a put or accumulate operation to a public window copy becomes visible in the private copy in MPI process memory at the latest when an ensuing call to `MPI_WIN_WAIT`, `MPI_WIN_FENCE`, `MPI_WIN_LOCK`, `MPI_WIN_LOCK_ALL`, or `MPI_WIN_SYNC` is executed on that window by the window owner. In the RMA unified memory model, an update by a put or accumulate operation to a public window copy eventually becomes visible in the private copy in MPI process memory without additional RMA calls.

The `MPI_WIN_FENCE` or `MPI_WIN_WAIT` call that completes the transfer from public copy to private copy (Rule 6) is the same call that completes the put or accumulate operation in the window copy (Rule 2, Rule 3). If a put or accumulate access was synchronized with a lock, then the update of the public window copy is complete as soon as the updating origin process executed `MPI_WIN_UNLOCK` or `MPI_WIN_UNLOCK_ALL`. In the RMA separate memory model, the update of a private copy in the target process memory may be delayed until the target process executes a synchronization call on that window (Rule 6). Thus, updates to target process memory can always be delayed in the RMA separate memory model until the target process executes a suitable synchronization call, while they must complete in the RMA unified model without additional synchronization calls. If fence or post-start-complete-wait synchronization is used, updates to a public window copy can be delayed in both memory models until the window owner executes a synchronization call. When passive target synchronization is used, it is necessary to update the public window copy even if the window owner does not execute any related synchronization call.

The rules above also define, by implication, when an update to a public window copy becomes visible in another overlapping public window copy. Consider, for example, two overlapping windows, `win1` and `win2`. A call to `MPI_WIN_FENCE` on `win1` by the window owner makes visible in the target process memory previous updates to window `win1` by origin processes. A subsequent call to `MPI_WIN_FENCE` on `win2` makes these updates visible in the public copy of `win2`.

The behavior of some MPI RMA operations may be *undefined* in certain situations. For example, the result of several origin processes performing concurrent put operations to the same target location is undefined. In addition, the result of a single origin process performing multiple put operations to the same target location within the same access epoch is also undefined. The result at the target may have all of the data from one of the put operations (the “last” one, in some sense), some bytes from each of the operations, or something else. In MPI-2, such operations were *erroneous*. That meant that an MPI implementation was permitted to raise an error. Thus, user programs or tools that used MPI RMA could not portably permit such operations, even if the application code could function correctly with such an undefined result. Starting with MPI-3, these operations are not erroneous, but do not have a defined behavior.

Rationale. As discussed in [8], requiring operations such as overlapping puts to be erroneous makes it difficult to use MPI RMA to implement programming models—such as Unified Parallel C (UPC) or SHMEM—that permit these operations. Further, while MPI-2 defined these operations as erroneous, the MPI Forum is unaware of any implementation that enforces this rule, as it would require significant overhead. Thus,

relaxing this condition does not impact existing implementations or applications. (*End of rationale.*)

Advice to implementors. Overlapping accesses are undefined. However, to assist users in debugging code, implementations may wish to provide a mode in which such operations are detected and reported to the user. Note, however, that starting with MPI-3, such operations must not raise an error. (*End of advice to implementors.*)

A program with a well-defined outcome in the `MPI_WIN_SEPARATE` memory model must obey the following rules.

- S1. A location in a window must not be accessed with load/store accesses once an update to that location has started, until the update becomes visible in the private window copy in target process memory.
- S2. A location in a window must not be accessed as a target of an RMA operation once an update to that location has started, until the update becomes visible in the public window copy. There is one exception to this rule, in the case where the same variable is updated by two concurrent accumulates with the same predefined datatype, on the same window. Additional restrictions on the operation apply, see the info key "accumulate_ops" in Section 12.2.1.
- S3. A put or accumulate must not access a target window once a store or a put or accumulate update to another (overlapping) window has started on a location in the target window, until the update becomes visible in the public copy of the window. Conversely, a store to MPI process memory to a location in a window must not start once a put or accumulate update to that target window has started, until the put or accumulate update becomes visible in target process memory. In both cases, the restriction applies to operations even if they access disjoint locations in the window.

Rationale. The last constraint on correct RMA accesses may seem unduly restrictive, as it forbids concurrent accesses to nonoverlapping locations in a window. The reason for this constraint is that, on some architectures, explicit coherence restoring operations may be needed at synchronization points. A different operation may be needed for locations that were updated by stores and for locations that were remotely updated by put or accumulate operations. Without this constraint, the MPI library would have to track precisely which locations in a window were updated by a put or accumulate operation. The additional overhead of maintaining such information is considered prohibitive. (*End of rationale.*)

Note that `MPI_WIN_SYNC` may be used within a passive target epoch to synchronize the private and public window copies (that is, updates to one are made visible to the other).

In the `MPI_WIN_UNIFIED` memory model, the rules are simpler because the public and private windows are the same. However, there are restrictions to avoid concurrent access to the same memory locations by different MPI processes. The rules that a program with a well-defined outcome must obey in this case are:

- U1. A location in a window must not be accessed with load/store accesses once an update to that location has started, until the update is complete, subject to the special case laid out in Rule 2.

- U2. Accessing a location in the window that is also the target of a remote update is valid (not erroneous) but the precise result will depend on the behavior of the implementation. Updates from an origin process will appear in the memory of the target, but there are no atomicity or ordering guarantees if more than one byte is updated. Updates are stable in the sense that once data appears in the memory of the target, the data remains until replaced by another update. This permits polling on a location for a change from zero to nonzero or for a particular value, but not polling and comparing the relative magnitude of values. Users are cautioned that polling on one memory location and then accessing a different memory location has defined behavior only if the other rules given here and in this chapter are followed.

Advice to users. Some compiler optimizations can result in code that maintains the sequential semantics of the program, but violates this rule by introducing temporary values into locations in memory. Most compilers only apply such transformations under very high levels of optimization and users should be aware that such aggressive optimization may produce unexpected results. (*End of advice to users.*)

- U3. Updating a location in the window with a store access that is also the target of a remote read (but not update) is valid (not erroneous) but the precise result will depend on the behavior of the implementation. Store updates will appear in memory, but there are no atomicity or ordering guarantees if more than one byte is updated. Updates are stable in the sense that once data appears in memory, the data remains until replaced by another update. This permits updates to memory with store accesses without requiring an RMA epoch. Users are cautioned that remote accesses to a window that is updated by the local MPI process has defined behavior only if the other rules given here and elsewhere in this chapter are followed.
- U4. A location in a window must not be accessed as a target of an RMA operation once an update to that location has started and until the update completes at the target. There is one exception to this rule: in the case where the same location is updated by two concurrent accumulates with the same predefined datatype on the same window. Additional restrictions on the operation apply; see the info key "accumulate_ops" in Section 12.2.1.
- U5. A put or accumulate must not access a target window once a store, put, or accumulate update to another (overlapping) target window has started on the same location in the target window and until the update completes at the target window. Conversely, a store access to a location in a window must not be executed once a put or accumulate update to the same location in that target window has started and until the put or accumulate update completes at the target.

Advice to users. In the unified memory model, in the case where the window is in *shared memory*, `MPI_WIN_SYNC` can be used to order store accesses and make store updates to the window visible to other MPI processes and threads. Use of this routine is necessary to ensure portable behavior when point-to-point, collective, or *shared memory* synchronization is used in place of an RMA synchronization routine.

`MPI_WIN_SYNC` should be called by both the reader and the writer of a shared memory variable between any non-RMA synchronization and access to that variable, as shown

in Example 12.23. The calls to `MPI_WIN_SYNC` can be replaced by language level memory synchronization operations, if available. (*End of advice to users.*)

A program that violates these rules has undefined behavior.

Advice to users. A user can write correct programs by following the following rules:

fence: During each period between fence calls, each window is either updated by put or accumulate operation, or updated by stores, but not both. Locations updated by put or accumulate operations should not be accessed during the same period (with the exception of concurrent updates to the same location by accumulate operations). Locations accessed by get operations should not be updated during the same period.

post-start-complete-wait: A window should not be updated with store accesses while posted if it is being updated by put or accumulate operations. Locations updated by put or accumulate operations should not be accessed while the window is posted (with the exception of concurrent updates to the same location by accumulate operations). Locations accessed by get operations should not be updated while the window is posted.

With the post-start synchronization, the target process can tell the origin process that its window is now ready for RMA access; with the complete-wait synchronization, the origin process can tell the target process that it has finished its RMA accesses to the window.

lock: Updates to the window are protected by *exclusive locks* if they may conflict. Nonconflicting accesses (such as read-only accesses or accumulate accesses) are protected by *shared locks*, both for load/store accesses and for RMA accesses.

changing window or synchronization mode: One can change synchronization mode, or change the window used to access a location that belongs to two overlapping windows, when the MPI process memory and the window copy are guaranteed to have the same values. This is true for an MPI process after it has returned from `MPI_WIN_FENCE`, if RMA accesses to the window are synchronized with fences; after it has returned from `MPI_WIN_WAIT`, if the accesses are synchronized with post-start-complete-wait; it is true at the origin and target after the origin returned from a call to `MPI_WIN_UNLOCK` or `MPI_WIN_UNLOCK_ALL` if the accesses are synchronized with locks.

In addition, an origin process should not access the local buffer of a get operation until the operation is complete, and should not update the local buffer of a put or accumulate operation until that operation is complete.

The RMA synchronization operations define when updates are guaranteed to become visible in public and private windows. Updates may become visible earlier, but such behavior is implementation dependent. (*End of advice to users.*)

The following examples illustrate these semantics.

Example 12.6. The following example demonstrates updating a memory location inside a window for the separate memory model, according to Rule 5. The `MPI_WIN_LOCK` and `MPI_WIN_UNLOCK` calls around the store to X in process B are necessary to ensure

consistency between the public and private copies of the window.

Process A	Process B
	window location X
	MPI_Win_lock (EXCLUSIVE, B)
	store X /* local update to private copy of B */
	MPI_Win_unlock (B)
	/* now visible in public window copy */
MPI_Barrier	MPI_Barrier
MPI_Win_lock (EXCLUSIVE, B)	
MPI_Get (X) /* ok, read from public window */	
MPI_Win_unlock (B)	

Example 12.7. In the RMA unified model, although the public and private copies of the windows are synchronized, caution must be used when combining load/store accesses with multi-process synchronization. Although the following example appears correct, the compiler or hardware may delay the store to X after the barrier, possibly resulting in the **MPI_GET** returning an incorrect value of X.

Process A	Process B
	window location X
	store X /* update to private & public copy of B */
MPI_Barrier	MPI_Barrier
MPI_Win_lock_all	
MPI_Get (X) /* ok, read from window */	
MPI_Win_flush_local (B)	
/* read value in X */	
MPI_Win_unlock_all	

MPI_BARRIER provides process synchronization, but not memory synchronization. The example could potentially be made safe through the use of compiler- and hardware-specific notations to ensure the store to X occurs before process B enters the **MPI_BARRIER**. The use of one-sided synchronization calls, as shown in Example 12.6, also ensures the correct result.

Example 12.8. The following example demonstrates the reading of a memory location updated by an origin process (Rule 6) in the RMA separate memory model. Although the call to **MPI_WIN_UNLOCK** on process A and the **MPI_BARRIER** ensure that the public copy on process B reflects the updated value of X, the call to **MPI_WIN_LOCK** by process B is necessary to synchronize the private copy with the public copy.

Process A	Process B
	window location X
MPI_Win_lock (EXCLUSIVE, B)	
MPI_Put (X) /* update to public window */	

```
MPI_Win_unlock(B)
```

```
MPI_Barrier
```

```
MPI_Barrier
```

```
MPI_Win_lock(EXCLUSIVE, B)
```

```
/* now visible in private copy of B */
```

```
load X
```

```
MPI_Win_unlock(B)
```

Note that in this example, the barrier is not critical to the semantic correctness. The use of *exclusive locks* guarantees no other MPI process will modify the public copy after `MPI_WIN_LOCK` synchronizes the private and public copies. A polling implementation looking for changes in `X` on process B would be semantically correct. The barrier is required to ensure that process A completes the put operation at the target before process B executes the load of `X`.

Example 12.9. Similar to Example 12.7, the following example is unsafe even in the unified model, because the load of `X` cannot be guaranteed to occur after the `MPI_BARRIER`. While Process B does not need to explicitly synchronize the public and private copies through `MPI_WIN_LOCK` as the `MPI_PUT` will update both the public and private copies of the window, the scheduling of the load could result in old values of `X` being returned. Compiler and hardware specific notations could ensure the load occurs after the data is updated, or explicit one-sided synchronization calls can be used to ensure the proper result.

```
Process A
```

```
Process B
```

```
window location X
```

```
MPI_Win_lock_all
```

```
MPI_Put(X) /* update to window */
```

```
MPI_Win_flush(B)
```

```
MPI_Barrier
```

```
MPI_Barrier
```

```
load X /* may return an obsolete value */
```

```
MPI_Win_unlock_all
```

Example 12.10. The following example further clarifies Rule 5. `MPI_WIN_LOCK` and `MPI_WIN_LOCK_ALL` do *not* update the public copy of a window with changes to the private copy. Therefore, there is no guarantee that process A in the following sequence will see the value of `X` as updated by the store by process B before the lock.

```
Process A
```

```
Process B
```

```
window location X
```

```
store X /* update to private copy of B */
```

```
MPI_Win_lock(SHARED, B)
```

```
MPI_Barrier
```

```
MPI_Barrier
```

```
MPI_Win_lock(SHARED, B)
```

```
MPI_Get(X) /* X may be the X before the store */
```

```
MPI_Win_unlock(B)
```

```
MPI_Win_unlock(B)
```

```
1      /* update on X now visible in public window */
```

2
3 The addition of a call to `MPI_WIN_SYNC` before the call to `MPI_BARRIER` by process B
4 would guarantee process A would see the updated value of X, as the public copy of the
5 window would be explicitly synchronized with the private copy.
6

7
8 **Example 12.11.** Similar to the previous example, Rule 5 can have unexpected implications
9 for general active target synchronization with the RMA separate memory model. It is *not*
10 guaranteed that process B reads the value of X as per the local update by process A, because
11 neither the call to `MPI_WIN_WAIT` nor the call to `MPI_WIN_COMPLETE` by process A
12 ensure visibility in the public window copy.

Process A	Process B
window location X	
window location Y	
store Y	
<code>MPI_Win_post(A, B) /* Y visible in public window */</code>	
<code>MPI_Win_start(A)</code>	<code>MPI_Win_start(A)</code>
store X /* update to private window */	
<code>MPI_Win_complete</code>	<code>MPI_Win_complete</code>
<code>MPI_Win_wait</code>	
<code>/* update on X may not yet be visible in the public window copy */</code>	
<code>MPI_Barrier</code>	<code>MPI_Barrier</code>
	<code>MPI_Win_lock(EXCLUSIVE, A)</code>
	<code>MPI_Get(X) /* may return an obsolete value */</code>
	<code>MPI_Get(Y)</code>
	<code>MPI_Win_unlock(A)</code>

22
23
24
25
26
27
28
29
30
31 To allow process B to read the value of X stored by A, the local store must be replaced by
32 a local put operation that updates the public window copy. Note that by this replacement
33 X may become visible in the private copy of process A only after the `MPI_WIN_WAIT` call
34 in process A. The update to Y made before the `MPI_WIN_POST` call is visible in the public
35 window after the `MPI_WIN_POST` call and therefore process B will read the proper value
36 of Y. The get of Y could be moved to the epoch opened by `MPI_WIN_START`, and process
37 B would still get the value stored by process A.
38

39
40 **Example 12.12.** The following example demonstrates the interaction of general active
41 target synchronization with load accesses in the RMA separate memory model. Rules 5 and 6
42 do *not* guarantee that the private copy of X at process B has been updated before the load
43 access is executed.
44

Process A	Process B
	window location X
<code>MPI_Win_lock(EXCLUSIVE, B)</code>	

```

MPI_Put(X) /* update to public window */
MPI_Win_unlock(B)

MPI_Barrier                                MPI_Barrier

                                           MPI_Win_post(B)
                                           MPI_Win_start(B)

                                           load X /* access to private window */
                                           /* may return an obsolete value */

                                           MPI_Win_complete
                                           MPI_Win_wait

```

To ensure that the value put by process A is read, the load access must be replaced with a get operation, or must be placed after the call to [MPI_WIN_WAIT](#).

12.7.1 Atomicity

The outcome of concurrent accumulate operations to the same location with the same predefined datatype is as if the accumulate operations were done at that location in some serial order. Additional restrictions on the operation apply; see the info key "accumulate_ops" in Section 12.2.1. Concurrent accumulate operations with different origin and target pairs are not ordered. Thus, there is no guarantee of atomicity beyond element-wise atomicity. The effect of this lack of atomicity is limited: The previous correctness conditions imply that a location updated by an accumulate operation cannot be accessed by a load access or an RMA operation other than another accumulate operation until the accumulate operation has completed (at the target). Different interleavings can lead to different results only to the extent that computer arithmetics are not truly associative or commutative. The outcome of accumulate operations with overlapping types of different sizes or target displacements is undefined.

12.7.2 Ordering

Accumulate operations enable element-wise atomic read and write to window memory locations. MPI specifies ordering between accumulate operations from an origin process to the same (or overlapping) memory locations at a target process on a per-datatype granularity. The default ordering is strict ordering, which guarantees that overlapping updates from the same origin to a remote location are committed in program order and that reads (e.g., with [MPI_GET_ACCUMULATE](#)) and writes (e.g., with [MPI_ACCUMULATE](#)) are executed and committed in program order. Ordering only applies to operations originating at the same origin that access overlapping target memory regions. MPI does not provide any guarantees for accesses or updates from different origin processes to overlapping target memory regions.

The default strict ordering may incur a significant performance penalty. MPI specifies the info key "accumulate_ordering" to allow relaxation of the ordering semantics when specified to any window creation function. The values for this key are as follows. If set to "none", then no ordering will be guaranteed for accumulate operations. This was the behavior for RMA in MPI-2 but has *not* been the default since MPI-3. The key can be set to a comma-separated list of required access orderings at the target. Allowed values in the comma-separated list

are "rar", "war", "raw", and "waw" for read-after-read, write-after-read, read-after-write, and write-after-write ordering, respectively. These indicate whether operations of the specified type complete in the order they were issued. For example, "raw" means that any writes must complete at the target before subsequent reads. These ordering requirements apply only to operations issued by the same origin process and targeting the same target process. The default value for "accumulate_ordering" is "rar,raw,war,waw", which implies that writes complete at the target in the order in which they were issued, reads complete at the target before any writes that are issued after the reads, and writes complete at the target before any reads that are issued after the writes. Any subset of these four orderings can be specified. For example, if only read-after-read and write-after-write ordering is required, then the value of the "accumulate_ordering" key could be set to "rar,waw". The order of values is not significant.

Note that the above ordering semantics apply only to accumulate operations, not to put and get operations. Put and get operations within an epoch are unordered.

12.7.3 Progress

One-sided communication has the same progress requirements as point-to-point communication: once a communication is enabled it is guaranteed to complete. RMA calls must have local semantics, except when required for synchronization with other RMA calls.

There is some fuzziness in the definition of the time when an RMA communication becomes enabled. This fuzziness provides to the implementor more flexibility than with point-to-point communication. Access to a target window becomes enabled once the corresponding synchronization (such as `MPI_WIN_FENCE` or `MPI_WIN_POST`) has executed. On the origin process, an RMA communication operation may become enabled as soon as the corresponding put, get or accumulate call has occurred, or as late as when the ensuing synchronization call is issued. Once the operation is enabled both at the origin and at the target, the operation must complete.

Consider the code fragment in Example 12.4. Some of the calls may have to delay their return until the target window has been posted. However, if the target window is posted, then the code fragment must complete. The data transfer may start as soon as the put call occurs, but may be delayed until the ensuing complete call occurs.

Consider the code fragment in Example 12.5. Some of the calls may delay their return until the lock is acquired if another MPI process holds a conflicting lock. However, if no conflicting lock is held, then the code fragment must complete.

Consider the code illustrated in Figure 12.6. Each MPI process updates the window of the other MPI process using a put operation, then accesses its own window. The post calls are local. Once the post calls occur, RMA access to the windows is enabled, so that each MPI process should complete the sequence of start-put-complete. Once these are done, the wait calls should complete at both MPI processes. Thus, this communication should not deadlock, irrespective of the amount of data transferred.

Assume, in the last example, that the order of the post and start calls is reversed at each MPI process. Then, the code may deadlock, as each MPI process may not return from the start call, waiting for the matching post to occur. Similarly, the program will deadlock if the order of the complete and wait calls is reversed at each MPI process.

The following two examples illustrate the fact that the synchronization between complete and wait is not symmetric: the wait call returns only once the complete occurs, but not vice versa. Consider the code illustrated in Figure 12.7. This code will deadlock: the wait of process 1 completes only once process 0 calls complete, and the receive of process 0

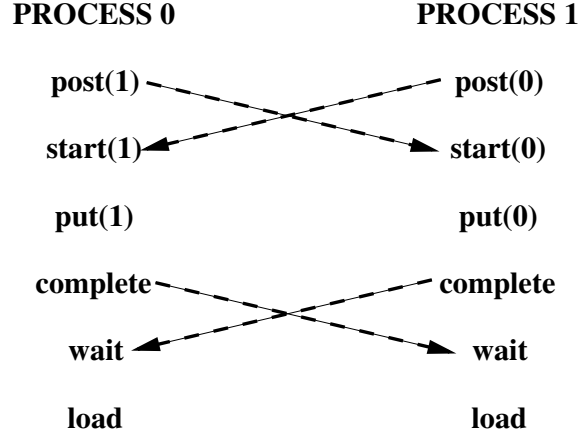


Figure 12.6: Symmetric communication

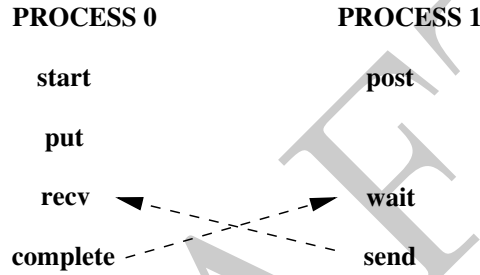


Figure 12.7: Deadlock situation

completes once process 1 calls send. Consider, on the other hand, the code illustrated in Figure 12.8. This code will not deadlock. Once process 1 calls post, then the sequence start-put-complete on process 0 can proceed. Process 0 will reach the send call, allowing the receive call of process 1 to return.

Rationale. MPI implementations must guarantee that an MPI process makes *progress* on all enabled communications it participates in, while blocked on an MPI call. This is true for send-receive communication and applies to RMA communication as well. Thus, in the example in Figure 12.8, the put and complete calls of process 0 should complete while process 1 is waiting for the receive operation to complete. This may require the involvement of process 1, e.g., to transfer the data.

A similar issue is whether such progress must occur while an MPI process is busy computing, or blocked in a non-MPI call. Suppose that in the last example the send-receive pair is replaced by a write-to-socket/read-from-socket pair. Then MPI does not specify whether deadlock is avoided. Suppose that the blocking receive of process 1 is replaced by a very long compute loop. Then, according to one interpretation of the MPI standard, process 0 must return from the complete call after a bounded delay, even if process 1 does not reach any MPI call in this period of time. According to another interpretation, the complete call may block until process 1 reaches the wait call, or reaches another MPI call. The qualitative behavior is the same, under both interpretations, unless an MPI process is caught in an infinite compute loop, in which case the difference may not matter. However, the quantitative expectations are differ-

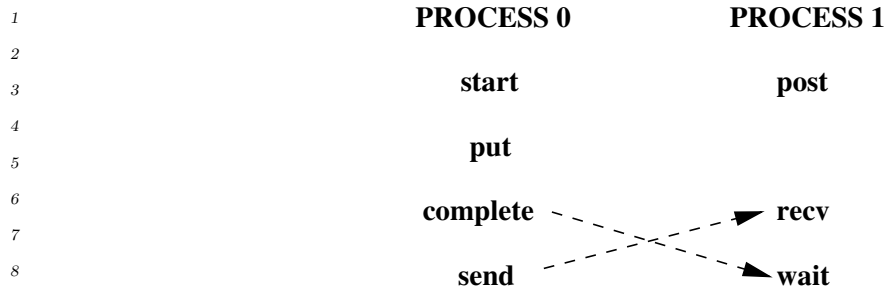


Figure 12.8: No deadlock

ent. Different MPI implementations reflect these different interpretations. While this ambiguity is unfortunate, the MPI Forum decided not to define which interpretation of the standard is the correct one, since the issue is contentious. See also Section 2.9 on *progress*. (*End of rationale*.)

The use of shared memory loads and/or stores for synchronizing purposes between MPI processes does not guarantee progress, and therefore a *deadlock* may occur if an MPI implementation does not provide *strong progress*, as shown in Example 12.13.

Example 12.13. Possible *deadlock* due to the use of a shared memory variable for synchronization.

`comm_sm` shall be a shared memory communicator (e.g., returned from a call to `MPI_COMM_SPLIT_TYPE` with `split_type=MPI_COMM_TYPE_SHARED`) with at least two MPI processes. `win_sm` is a shared memory window with the `AckInRank0` as window portion in MPI process with rank 0. The ranks in `comm_sm` and `win_sm` should be the same. According to Section 12.7 rules U2 and U3, a volatile store to `AckInRank0` will be visible in the other MPI process without further RMA calls.

<pre> int volatile_load(int *addr) {return *((volatile int *)addr);} void volatile_store(int *addr, int val) {*((volatile int *)addr) = val;} Process with rank 0 MPI_Win_shared_query(win, /*rank=*/ 0, ..., AckInRank0); volatile_store(AckInRank0, 0); MPI_Win_fence(win_sm) MPI_Buffer_attach(myHugeBuffer,...); MPI_Bsend(myHugeMessage, ..., /*rank=*/ 1,..., comm_sm); sleep(10); // to guarantee that // the while-loop starts // after rank 1 is // blocked in MPI_Recv while(volatile_load(AckInRank0)!=222) /*empty polling loop*/; MPI_Buffer_detach(&pTemp, &size); // deadlock </pre>	<pre> Process with rank 1 MPI_Win_shared_query(win, /*rank=*/ 0, ..., AckInRank0); MPI_Win_fence(win_sm) sleep(5); // to ensure // that the MPI_Bsend // in rank 0 returned MPI_Recv(&myHugeMessage, ... /*rank=*/ 0, ..., comm_sm, ...); volatile_store(AckInRank0,222); // deadlock </pre>
--	--

While the call to `MPI_Recv` in the MPI process with rank 1 delays its return (until an unspecified MPI procedure call in the MPI process with rank 0 happens to send the buffered data), the subsequent statement cannot change the value of the shared window buffer `AckInRank0`. As long as this value is not changed, the while loop in the MPI process with rank 0 will continue and therefore the next MPI procedure call (`MPI_Buffer_detach`) cannot happen, which is then a *deadlock*.

Note that both communication patterns (A) `BSEND-RECV-DETACH` and (B) the shared memory store/load for synchronization purpose, can be in different software layers and each layer would work properly, but the combination of (A) and (B) can cause the *deadlock*.

12.7.4 Registers and Compiler Optimizations

Advice to users. All the material in this section is an advice to users. (*End of advice to users.*)

A coherence problem exists between variables kept in registers and the memory values of these variables. An RMA call may access a variable in memory (or cache), while the up-to-date value of this variable is in register. A get will not return the latest variable value, and a put may be overwritten when the register is stored back in memory. Note that these issues are unrelated to the RMA memory model; that is, these issues apply even if the memory model is `MPI_WIN_UNIFIED`.

The problem is illustrated in Example 12.14.

Example 12.14.

In this example, variable `buff` is allocated in the register `reg_A` and therefore `ccc` will have the old value of `buff` and not the new value 777.

Source of Process 1	Source of Process 2	Executed in Process 2
<code>bbbb = 777</code>	<code>buff = 999</code>	<code>reg_A:=999</code>
<code>call MPI_WIN_FENCE</code>	<code>call MPI_WIN_FENCE</code>	
<code>call MPI_PUT(bbbb</code> into <code>buff</code> of process 2)		<code>stop appl.thread</code> <code>buff:=777</code> in PUT handler <code>continue</code> <code>appl.thread</code>
<code>call MPI_WIN_FENCE</code>	<code>call MPI_WIN_FENCE</code>	
	<code>ccc = buff</code>	<code>ccc:=reg_A</code>

This problem, which also afflicts in some cases send/receive communication, is discussed more at length in Section 21.1.16.

Programs written in C avoid this problem, because of the semantics of C. Many Fortran compilers will avoid this problem, without disabling compiler optimizations. However, in order to avoid register coherence problems in a completely portable manner, users should restrict their use of RMA windows to variables stored in modules or `COMMON` blocks. To prevent problems with the argument copying and register optimization done by Fortran compilers, please note the hints in Sections 21.1.10–21.1.20. In Section 21.1.17, [Solutions to The \(Poorly Performing\) Fortran VOLATILE Attribute](#) discuss several solutions for the problem in this example.

12.8 Examples

Example 12.15. The following example shows a generic loosely synchronous, iterative code, using `MPI_WIN_FENCE` for synchronization. The window at each MPI process consists of array A, which contains the origin and target buffers of the put operations.

```
...
while (!converged(A)) {
    update(A);
    MPI_Win_fence(MPI_MODE_NOPRECEDE, win);
    for(i=0; i < toneighbors; i++)
        MPI_Put(&frombuf[i], 1, fromtype[i], toneighbor[i],
                todisp[i], 1, totype[i], win);
    MPI_Win_fence((MPI_MODE_NOSTORE | MPI_MODE_NOSUCCEED), win);
}
```

The same code could be written with get rather than put. Note that, during the communication phase, each window is concurrently read (as origin buffer of puts) and written (as target buffer of puts). This is OK, provided that there is no overlap between the target buffer of a put and another communication buffer.

Example 12.16. Same generic example, with more computation/communication overlap. We assume that the update phase is broken into two subphases: the first, where the “boundary,” which is involved in communication, is updated, and the second, where the “core,” which neither uses nor provides communicated data, is updated.

```
...
while (!converged(A)) {
    update_boundary(A);
    MPI_Win_fence((MPI_MODE_NOPUT | MPI_MODE_NOPRECEDE), win);
    for(i=0; i < fromneighbors; i++)
        MPI_Get(&tobuf[i], 1, totype[i], fromneighbor[i],
                fromdisp[i], 1, fromtype[i], win);
    update_core(A);
    MPI_Win_fence(MPI_MODE_NOSUCCEED, win);
}
```

The get communication can be concurrent with the core update, since they do not access the same locations, and the local update of the origin buffer by the get operation can be concurrent with the local update of the core by the `update_core` call. In order to get similar overlap with put communication we would need to use separate windows for the core and for the boundary. This is required because we do not allow local stores to be concurrent with puts on the same, or on overlapping, windows.

Example 12.17. Same code as in Example 12.15, rewritten using post-start-complete-wait.

```
...
while (!converged(A)) {
    update(A);
    MPI_Win_post(fromgroup, 0, win);
    MPI_Win_start(togroup, 0, win);
    for(i=0; i < toneighbors; i++)
        MPI_Put(&frombuf[i], 1, fromtype[i], toneighbor[i],
```

```

        todisp[i], 1, totype[i], win);
    MPI_Win_complete(win);
    MPI_Win_wait(win);
}

```

Example 12.18. Same example, with post-start-complete-wait, as in Example 12.16.

```

...
while (!converged(A)) {
    update_boundary(A);
    MPI_Win_post(togroup, MPI_MODE_NOPUT, win);
    MPI_Win_start(fromgroup, 0, win);
    for(i=0; i < fromneighbors; i++)
        MPI_Get(&tobuf[i], 1, totype[i], fromneighbor[i],
                fromdisp[i], 1, fromtype[i], win);
    update_core(A);
    MPI_Win_complete(win);
    MPI_Win_wait(win);
}

```

Example 12.19. A checkerboard, or double buffer communication pattern, that allows more computation/communication overlap. Array A0 is updated using values of array A1, and vice versa. We assume that communication is symmetric: if process A gets data from process B, then process B gets data from process A. Window wini consists of array Ai.

```

...
if (!converged(A0,A1))
    MPI_Win_post(neighbors, (MPI_MODE_NOCHECK | MPI_MODE_NOPUT), win0);
MPI_Barrier(comm0);
/* the barrier is needed because the start call inside the
loop uses the nocheck option */
while (!converged(A0, A1)) {
    /* communication on A0 and computation on A1 */
    update2(A1, A0); /* local update of A1 that depends on A0 (and A1) */
    MPI_Win_start(neighbors, MPI_MODE_NOCHECK, win0);
    for(i=0; i < fromneighbors; i++)
        MPI_Get(&tobuf0[i], 1, totype0[i], neighbor[i],
                fromdisp0[i], 1, fromtype0[i], win0);
    update1(A1); /* local update of A1 that is
                  concurrent with communication that updates A0 */
    MPI_Win_post(neighbors, (MPI_MODE_NOCHECK | MPI_MODE_NOPUT), win1);
    MPI_Win_complete(win0);
    MPI_Win_wait(win0);

    /* communication on A1 and computation on A0 */
    update2(A0, A1); /* local update of A0 that depends on A1 (and A0) */
    MPI_Win_start(neighbors, MPI_MODE_NOCHECK, win1);
    for(i=0; i < fromneighbors; i++)
        MPI_Get(&tobuf1[i], 1, totype1[i], neighbor[i],
                fromdisp1[i], 1, fromtype1[i], win1);
    update1(A0); /* local update of A0 that depends on A0 only,

```

```

1      concurrent with communication that updates A1 */
2      if (!converged(A0,A1))
3          MPI_Win_post(neighbors, (MPI_MODE_NOCHECK | MPI_MODE_NOPUT), win0);
4          MPI_Win_complete(win1);
5          MPI_Win_wait(win1);
6      }

```

An MPI process posts the local window associated with win0 before it completes RMA accesses to the remote windows associated with win1. When the call to `MPI_WIN_WAIT` on win1 returns, then all neighbors of the calling MPI process have posted the windows associated with win0. Conversely, when the call to `MPI_WIN_WAIT` on win0 returns, then all neighbors of the calling MPI process have posted the windows associated with win1. Therefore, the `MPI_MODE_NOCHECK` option can be used with the calls to `MPI_WIN_START`.

Put operations can be used, instead of get operations, if the area of array A0 (resp. A1) used by `update(A1, A0)` (resp. `update(A0, A1)`) is disjoint from the area modified by the RMA operation. On some systems, a put operation may be more efficient than a get operation, as it requires information exchange only in one direction.

In the next several examples, for conciseness, the expression

```
z = MPI_Get_accumulate(...)
```

means to perform a get-accumulate operation with the result buffer (given by `result_addr` in the description of `MPI_GET_ACCUMULATE`) on the left side of the assignment, in this case, `z`. This format is also used with `MPI_COMPARE_AND_SWAP` and `MPI_COMM_SIZE`. Process B... refers to any process other than A.

Example 12.20. The following example implements a naive, nonscalable counting semaphore. The example demonstrates the use of `MPI_WIN_SYNC` to manipulate the public copy of `X`, as well as `MPI_WIN_FLUSH` to complete operations without closing the access epoch opened with `MPI_WIN_LOCK_ALL`. To avoid the rules regarding synchronization of the public and private copies of windows, `MPI_ACCUMULATE` and `MPI_GET_ACCUMULATE` are used to write to or read from the local public copy.

Process A	Process B...
<code>MPI_Win_lock_all</code>	<code>MPI_Win_lock_all</code>
window location <code>X</code>	
<code>X=MPI_Comm_size()</code>	
<code>MPI_Win_sync</code>	
<code>MPI_Barrier</code>	<code>MPI_Barrier</code>
<code>MPI_Accumulate(X, MPI_SUM, -1)</code>	<code>MPI_Accumulate(X, MPI_SUM, -1)</code>
stack variable <code>z</code>	stack variable <code>z</code>
do	do
<code>z = MPI_Get_accumulate(X,</code>	<code>z = MPI_Get_accumulate(X,</code>
<code>MPI_NO_OP, 0)</code>	<code>MPI_NO_OP, 0)</code>
<code>MPI_Win_flush(A)</code>	<code>MPI_Win_flush(A)</code>
while(<code>z!=0</code>)	while(<code>z!=0</code>)
<code>MPI_Win_unlock_all</code>	<code>MPI_Win_unlock_all</code>

Example 12.21. Implementing a critical region between two MPI processes (Peterson's algorithm). Despite their appearance in the following example, `MPI_WIN_LOCK_ALL` and `MPI_WIN_UNLOCK_ALL` are not collective calls, but it is frequently useful to open shared access epochs to all MPI processes from all other MPI processes in a window. Once the access epochs are opened, accumulate operations as well as flush and sync synchronization can be used to read from or write to the public copy of the window.

Process A	Process B
window location X	window location Y
window location T	
 <code>MPI_Win_lock_all</code>	 <code>MPI_Win_lock_all</code>
X=1	Y=1
<code>MPI_Win_sync</code>	<code>MPI_Win_sync</code>
<code>MPI_Barrier</code>	<code>MPI_Barrier</code>
<code>MPI_Accumulate(T, MPI_REPLACE, 1)</code>	<code>MPI_Accumulate(T, MPI_REPLACE, 0)</code>
stack variables t,y	stack variable t,x
t=1	t=0
y= <code>MPI_Get_accumulate(Y,</code> <code>MPI_NO_OP, 0)</code>	x= <code>MPI_Get_accumulate(X,</code> <code>MPI_NO_OP, 0)</code>
while (y==1 && t==1) do	while (x==1 && t==0) do
y= <code>MPI_Get_accumulate(Y,</code> <code>MPI_NO_OP, 0)</code>	x= <code>MPI_Get_accumulate(X,</code> <code>MPI_NO_OP, 0)</code>
t= <code>MPI_Get_accumulate(T,</code> <code>MPI_NO_OP, 0)</code>	t= <code>MPI_Get_accumulate(T,</code> <code>MPI_NO_OP, 0)</code>
<code>MPI_Win_flush_all</code>	<code>MPI_Win_flush(A)</code>
done	done
// critical region	// critical region
<code>MPI_Accumulate(X, MPI_REPLACE, 0)</code>	<code>MPI_Accumulate(Y, MPI_REPLACE, 0)</code>
<code>MPI_Win_unlock_all</code>	<code>MPI_Win_unlock_all</code>

Example 12.22. Implementing a critical region between multiple MPI processes with compare and swap. The call to `MPI_WIN_SYNC` is necessary on Process A after local initialization of A to guarantee the public copy has been updated with the initialization value found in the private copy. It would also be valid to call `MPI_ACCUMULATE` with `MPI_REPLACE` to directly initialize the public copy. A call to `MPI_WIN_FLUSH` would be necessary to assure A in the public copy of Process A had been updated before the barrier.

Process A	Process B...
<code>MPI_Win_lock_all</code>	<code>MPI_Win_lock_all</code>
atomic location A	
A=0	
<code>MPI_Win_sync</code>	
<code>MPI_Barrier</code>	<code>MPI_Barrier</code>
stack variable r = 1	stack variable r = 1
while (r != 0) do	while (r != 0) do
r = <code>MPI_Compare_and_swap(A, 0, 1)</code>	r = <code>MPI_Compare_and_swap(A, 0, 1)</code>
<code>MPI_Win_flush(A)</code>	<code>MPI_Win_flush(A)</code>
done	done
// critical region	// critical region
r = <code>MPI_Compare_and_swap(A, 1, 0)</code>	r = <code>MPI_Compare_and_swap(A, 1, 0)</code>

MPI_Win_unlock_all

MPI_Win_unlock_all

Example 12.23. The following example demonstrates the proper synchronization in the unified memory model when a data transfer is implemented with load and store accesses in the case of windows in *shared memory* (instead of using `MPI_PUT` or `MPI_GET`) and the synchronization between MPI processes is performed using point-to-point communication. The synchronization between MPI processes must be supplemented with a memory synchronization through calls to `MPI_WIN_SYNC`, which act locally as a processor-memory barrier. In Fortran, if `MPI_ASYNC_PROTECTS_NONBLOCKING` is `.FALSE.` or the variable `X` is not declared as `ASYNCHRONOUS`, reordering of the accesses to the variable `X` must be prevented with `MPI_F_SYNC_REG` operations. (No equivalent function is needed in C.)

The variable `X` is contained within a *shared memory window* and `X` corresponds to the same memory location at both processes. The first call to `MPI_WIN_SYNC` performed by process A ensures completion of the load/store accesses issued by process A. The first call to `MPI_WIN_SYNC` performed by process B ensures that process A's updates to `X` are visible to process B. Similarly, the second call to `MPI_WIN_SYNC` on each process ensures correct ordering of the point-to-point communication and thus that the load/store operations on process B have completed before any subsequent load/store accesses to the variable `X` in process A.

Process A	Process B
<code>MPI_WIN_LOCK_ALL (</code>	<code>MPI_WIN_LOCK_ALL (</code>
<code>MPI_MODE_NOCHECK, win)</code>	<code>MPI_MODE_NOCHECK, win)</code>
<code>DO ...</code>	<code>DO ...</code>
<code>X = ...</code>	
<code>MPI_F_SYNC_REG(X)</code>	
<code>MPI_WIN_SYNC(win)</code>	
<code>MPI_SEND</code>	<code>MPI_RECV</code>
	<code>MPI_WIN_SYNC(win)</code>
	<code>MPI_F_SYNC_REG(X)</code>
	<code>print X</code>
	<code>MPI_F_SYNC_REG(X)</code>
	<code>MPI_WIN_SYNC(win)</code>
	<code>MPI_SEND</code>
<code>MPI_RECV</code>	
<code>MPI_WIN_SYNC(win)</code>	
<code>MPI_F_SYNC_REG(X)</code>	
<code>END DO</code>	<code>END DO</code>
<code>MPI_WIN_UNLOCK_ALL(win)</code>	<code>MPI_WIN_UNLOCK_ALL(win)</code>

Example 12.24. The following example shows how request-based operations can be used to overlap communication with computation. Each MPI process fetches, processes, and writes the result for `NSTEPS` chunks of data. Instead of a single buffer, `M` local buffers are used to allow up to `M` communication operations to overlap with computation.


```

1  int      i, j;
2  MPI_Win  win;
3  MPI_Request put_req[M] = { MPI_REQUEST_NULL };
4  MPI_Request get_req;
5  double    *baseptr;
6  double    data[M][N];
7
8  MPI_Win_allocate(NSTEPS*N*sizeof(double), sizeof(double), MPI_INFO_NULL,
9                  MPI_COMM_WORLD, &baseptr, &win);
10
11 MPI_Win_lock_all(0, win);
12
13 for (i = 0; i < NSTEPS; i++) {
14     if (i < M)
15         j=i;
16     else
17         MPI_Waitany(M, put_req, &j, MPI_STATUS_IGNORE);
18
19     MPI_Rget(data[j], N, MPI_DOUBLE, target, i*N, N, MPI_DOUBLE, win,
20             &get_req);
21     MPI_Wait(&get_req, MPI_STATUS_IGNORE);
22     compute(i, data[j], ...);
23     MPI_Rput(data[j], N, MPI_DOUBLE, target, i*N, N, MPI_DOUBLE, win,
24             &put_req[j]);
25 }
26
27 MPI_Waitall(M, put_req, MPI_STATUSES_IGNORE);
28 MPI_Win_unlock_all(win);

```

Example 12.25. The following example constructs a distributed shared linked list using dynamic windows. Initially process 0 creates the head of the list, attaches it to the window, and broadcasts the pointer to all MPI processes. All MPI processes then concurrently append N new elements to the list. When an MPI process attempts to attach its element to the tail of the list it may discover that its tail pointer is stale and it must chase ahead to the new tail before the element can be attached. This example requires some modification to work in an environment where the layout of the structures is different on different MPI processes.

```

36 ...
37 #define NUM_ELEMS 10
38
39 #define LLIST_ELEM_NEXT_RANK ( offsetof(llist_elem_t, next) + \
40                               offsetof(llist_ptr_t, rank) )
41
42 #define LLIST_ELEM_NEXT_DISP ( offsetof(llist_elem_t, next) + \
43                               offsetof(llist_ptr_t, disp) )
44
45 /* Linked list pointer */
46 typedef struct {
47     MPI_Aint disp;
48     int      rank;
49 } llist_ptr_t;
50
51 /* Linked list element */

```

```

1  typedef struct {
2      llist_ptr_t next;
3      int value;
4  } llist_elem_t;
5
6  const llist_ptr_t nil = { (MPI_Aint) MPI_BOTTOM, -1 };
7
8  /* List of locally allocated list elements. */
9  static llist_elem_t **my_elems = NULL;
10 static int my_elems_size = 0;
11 static int my_elems_count = 0;
12
13 /* Allocate a new shared linked list element */
14 MPI_Aint alloc_elem(int value, MPI_Win win) {
15     MPI_Aint disp;
16     llist_elem_t *elem_ptr;
17
18     /* Allocate the new element and register it with the window */
19     MPI_Alloc_mem(sizeof(llist_elem_t), MPI_INFO_NULL, &elem_ptr);
20     elem_ptr->value = value;
21     elem_ptr->next = nil;
22     MPI_Win_attach(win, elem_ptr, sizeof(llist_elem_t));
23
24     /* Add the element to the list of local elements so we can free
25        it later. */
26     if (my_elems_size == my_elems_count) {
27         my_elems_size += 100;
28         my_elems = realloc(my_elems, my_elems_size*sizeof(void*));
29     }
30     my_elems[my_elems_count] = elem_ptr;
31     my_elems_count++;
32
33     MPI_Get_address(elem_ptr, &disp);
34     return disp;
35 }
36
37 int main(int argc, char *argv[]) {
38     int          procid, nproc, i;
39     MPI_Win      llist_win;
40     llist_ptr_t  head_ptr, tail_ptr;
41
42     MPI_Init(&argc, &argv);
43
44     MPI_Comm_rank(MPI_COMM_WORLD, &procid);
45     MPI_Comm_size(MPI_COMM_WORLD, &nproc);
46
47     MPI_Win_create_dynamic(MPI_INFO_NULL, MPI_COMM_WORLD, &llist_win);
48
49     /* Process 0 creates the head node */
50     if (procid == 0)
51         head_ptr.disp = alloc_elem(-1, llist_win);
52
53     /* Broadcast the head pointer to everyone */
54     head_ptr.rank = 0;
55     MPI_Bcast(&head_ptr.disp, 1, MPI_AINT, 0, MPI_COMM_WORLD);
56     tail_ptr = head_ptr;

```

```

/* Lock the window for shared access to all targets */
MPI_Win_lock_all(0, llist_win);

/* All processes concurrently append NUM_ELEMS elements to the list */
for (i = 0; i < NUM_ELEMS; i++) {
    llist_ptr_t new_elem_ptr;
    int success;

    /* Create a new list element and attach it to the window */
    new_elem_ptr.rank = procid;
    new_elem_ptr.disp = alloc_elem(procid, llist_win);

    /* Append the new node to the list. This might take multiple
       attempts if others have already appended and our tail pointer
       is stale. */
    do {
        llist_ptr_t next_tail_ptr = nil;

        MPI_Compare_and_swap((void*) &new_elem_ptr.rank, (void*) &nil.rank,
                              (void*)&next_tail_ptr.rank, MPI_INT, tail_ptr.rank,
                              MPI_Aint_add(tail_ptr.disp, LLIST_ELEM_NEXT_RANK),
                              llist_win);

        MPI_Win_flush(tail_ptr.rank, llist_win);
        success = (next_tail_ptr.rank == nil.rank);

        if (success) {
            MPI_Accumulate(&new_elem_ptr.disp, 1, MPI_AINT, tail_ptr.rank,
                          MPI_Aint_add(tail_ptr.disp, LLIST_ELEM_NEXT_DISP), 1,
                          MPI_AINT, MPI_REPLACE, llist_win);

            MPI_Win_flush(tail_ptr.rank, llist_win);
            tail_ptr = new_elem_ptr;
        } else {
            /* Tail pointer is stale, fetch the displacement. May take
               multiple tries if it is being updated. */
            do {
                MPI_Get_accumulate(NULL, 0, MPI_AINT, &next_tail_ptr.disp,
                                   1, MPI_AINT, tail_ptr.rank,
                                   MPI_Aint_add(tail_ptr.disp, LLIST_ELEM_NEXT_DISP),
                                   1, MPI_AINT, MPI_NO_OP, llist_win);

                MPI_Win_flush(tail_ptr.rank, llist_win);
            } while (next_tail_ptr.disp == nil.disp);
            tail_ptr = next_tail_ptr;
        }
    } while (!success);
}

MPI_Win_unlock_all(llist_win);
MPI_Barrier(MPI_COMM_WORLD);

/* Free all the elements in the list */
for ( ; my_elems_count > 0; my_elems_count--) {
    MPI_Win_detach(llist_win, my_elems[my_elems_count-1]);
    MPI_Free_mem(my_elems[my_elems_count-1]);
}

```

```
1  }  
2  MPI_Win_free(&llist_win);  
3  ...  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48
```

DRAFT

Chapter 13

Queued Communication

13.1 Introduction

MPI programs may need to carefully order communication operations with respect to each other, including specifying when communication operations on different communicators may be initiated and completed concurrently, and when operations may be initiated or completed concurrently. In addition, MPI programs may need to order communication operations with respect to each other and computation in asynchronous programming systems that have specialized execution models. This is particularly important because such systems, including GPU streams, graph programming systems, fine-grain threading models, and asynchronous many-task systems, may have execution semantics that efficiently support only limited computational capabilities; for example, *blocking* operations are often disallowed or strongly discouraged in these systems.

This chapter describes a general MPI API for carefully ordering MPI communication operations with respect to each other and with computation in external asynchronous programming systems. To do so, this chapter defines a `MPI_Queue` object for ordering initiation and completion of persistent communication operations with respect to each other and with an optional (opaque) external execution context. It also introduces a new *matched* version of persistent operations whose semantics are optimized for use on queues and defines their expanded life cycle in comparison with standard persistent operations. The current queue specification permits *only* matched persistent operations and *partitioned persistent communication requests*, which require no matching step of their own, to be enqueued on `MPI_Queue` objects; ordinary, unmatched persistent requests, nonblocking requests, and generalized requests may not. Finally, this chapter carefully defines the matching, ordering, progress, and concurrency semantics of the initiation and completion of enqueued matched MPI persistent communication operations and the exact semantics of enqueued MPI communication operations when no opaque external execution context is provided.

The semantics of enqueued persistent communication operations with respect to computation when an opaque external execution context is provided are outside of the scope of the MPI standard. One-sided (RMA) communication operations cannot be *matched* or enqueued on a *queue*; support for RMA communication on *queues* is outside the scope of this version of the MPI standard. Systems in which this interface is implemented should provide a separate reference document that describes how operations on *queue* objects interact with its programming environment. Such a document may also specify additional system-specific MPI extensions for interacting with enqueued MPI persistent communication operations.

13.2 Queue Objects

A **queue** represents an ordered set of MPI *start* and *wait* operations to execute and order with respect to each other and, optionally, with respect to computations in an external asynchronous programming system. A *queue* includes a provided *queue type* and, if that type supports it, an ordering-context object provided by an external asynchronous programming system. Examples of such ordering-context objects include, but are not limited to, GPU streams. MPI start and wait operations on the same *queue* are sequentially ordered with respect to each other and the execution order defined by the associated external execution context. MPI start and wait operations issued to a *queue* and communication operations issued outside that *queue* or to a different *queue* shall be considered concurrent. Concurrent communication operations may be started or completed by an MPI implementation in any order.

13.2.1 Enqueued Request State

MPI start and wait operations can only be enqueued for a limited set of persistent request types: a *matched persistent request* (see Section 13.3) or a *partitioned persistent communication request* (see Section 13.3.1). When used with queues, these requests carry the following additional state in addition to the *operation states* defined in Section 2.4.2:

enqueued: A request is **enqueued** after `MPI_ENQUEUE_START` or `MPI_ENQUEUE_STARTALL` has been called for it but before the corresponding communication operation has been initiated by its *bound queue* (i.e., before the request becomes *active*).

bound queue: The *queue* on which a request is currently *enqueued* or *active*. A request that is neither *enqueued* nor *active* on any *queue* has no *bound queue*.

enqueue count: The number of `MPI_ENQUEUE_START` and `MPI_ENQUEUE_WAIT` operations enqueued for a request on its *bound queue* that have not yet completed. A request's *enqueue count* is zero exactly when it has no *bound queue*.

A request whose *enqueue count* reaches zero is no longer bound to any *queue* and may subsequently be enqueued on the same or a different *queue*. It is erroneous to enqueue an operation for a request on a *queue* other than its current *bound queue* while its *enqueue count* is nonzero.

13.2.2 Queue Creation and Destruction

`MPI_QUEUE_CREATE(queue, type, external, info)`

OUT	queue	handle to the created queue object (handle)
IN	type	type of the queue object (integer)
IN	external	pointer to the associated opaque external ordering-context object (choice)
IN	info	info object (handle)

C binding

```
int MPI_Queue_create(MPI_Queue *queue, int type, void *external, MPI_Info info)
```

Fortran 2008 binding

```
MPI_Queue_create(queue, type, external, info, ierror)
```

```
    TYPE(MPI_Queue), INTENT(OUT) :: queue
```

```
    INTEGER, INTENT(IN) :: type
```

```
    TYPE(*), DIMENSION(..), INTENT(IN) :: external
```

```
    TYPE(MPI_Info), INTENT(IN) :: info
```

```
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_QUEUE_CREATE(Queue, Type, External, Info, Ierror)
```

```
    INTEGER Queue, Type, Info, Ierror
```

```
    <type> External(*)
```

`MPI_QUEUE_CREATE` creates a **queue object** and returns a handle to it in `queue`. A *queue object* is the concrete handle through which an MPI program refers to an abstract *queue*. The `type` argument identifies the external asynchronous programming system with which the *queue* is associated; with the exception of `MPI_QUEUE_TYPE_DEFAULT`, valid type values and their meanings are defined by the implementation for each supported external programming system. It is erroneous to pass a `type` value not supported by the implementation. The `external` argument is a pointer to the execution context object in that programming system (e.g., a GPU stream handle) with which the operations enqueued on this *queue* are ordered. The interpretation of `external` is determined by `type`. The calling program is responsible for ensuring that the handle passed as `external` remains valid for the lifetime of the *queue object*. The `info` argument provides hints to the implementation about the created *queue*; `MPI_INFO_NULL` may always be passed. This chapter reserves no `info` keys. This chapter deliberately provides no operation to query a *queue*'s `type` or `external` context, and no `info` get/set pair for a *queue* (unlike windows, files, and sessions), so as to minimize the number of new functions this chapter introduces.

`MPI_QUEUE_CREATE` is a *local* operation.

A *queue object* can be *freed* using the following MPI procedure.

```
MPI_QUEUE_FREE(queue)
```

```
    INOUT    queue
```

```
                address of queue to free (handle)
```

C binding

```
int MPI_Queue_free(MPI_Queue *queue)
```

Fortran 2008 binding

```
MPI_Queue_free(queue, ierror)
```

```
    TYPE(MPI_Queue), INTENT(INOUT) :: queue
```

```
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_QUEUE_FREE(Queue, Ierror)
```

```
    INTEGER Queue, Ierror
```

`MPI_QUEUE_FREE` marks the queue object referenced by `queue` for deallocation and releases associated resources. It is erroneous to free a *queue* that has enqueued operations

that have not yet been completed (i.e., a *queue* that is not empty). After the call returns, `queue` is set to `MPI_QUEUE_NULL`.

`MPI_QUEUE_FREE` is a *local* operation.

13.2.3 Enqueuing Operations

`MPI_ENQUEUE_START(queue, request)`

INOUT	<code>queue</code>	address of queue on which to enqueue start (handle)
INOUT	<code>request</code>	inactive matched persistent request to start (handle)

C binding

```
int MPI_Enqueue_start(MPI_Queue *queue, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_Enqueue_start(queue, request, ierror)
  TYPE(MPI_Queue), INTENT(INOUT) :: queue
  TYPE(MPI_Request), INTENT(INOUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ENQUEUE_START(Queue, Request, Ierror)
  INTEGER Queue, Request, Ierror
```

`MPI_ENQUEUE_START` enqueues the start of the *matched* persistent communication request on `queue`. The `queue` is updated to record the enqueued start operation.

The request argument shall be a handle to a *matched persistent request* (see Section 13.3) returned by `MPI_MATCH`, `MPI_MATCHALL`, `MPI_IMATCH`, or `MPI_IMATCHALL` (once the corresponding matching operation has completed), or to a *partitioned persistent request* created by `MPI_PSEND_INIT` or `MPI_PRECV_INIT` (see Section 13.3.1). It is erroneous to pass a *nonblocking* request, a generalized request, an ordinary (unmatched) persistent request, or a *matched persistent request* whose matching operation has not yet completed, as `request`; in this case `MPI_ENQUEUE_START` returns the error class `MPI_ERR_QUEUE` and the request is not enqueued. In order to enqueue the start of request, `request` shall either have no *bound queue*, or already be bound to `queue` with a corresponding `MPI_ENQUEUE_WAIT` already enqueued on `queue` for its current activation (see Section 13.2.1). It is erroneous to enqueue the start of a request that remains bound to a different *queue*.

Once `MPI_ENQUEUE_START` returns, the only MPI operations that may be performed on `request` while its *enqueue count* is nonzero are: another call to `MPI_ENQUEUE_START` on `queue`, once a corresponding `MPI_ENQUEUE_WAIT` has already been enqueued on `queue`; and `MPI_ENQUEUE_WAIT` or `MPI_ENQUEUE_WAITALL` on `queue`. If `request` is a partitioned persistent send or receive request, `MPI_PREADY` and its variants and `MPI_PARRIVED` may additionally be called on it, subject to the restrictions in Section 13.3.1. `MPI_REQUEST_GET_STATUS` and `MPI_REQUEST_FREE` (see Section 3) may also be called on `request` while its *enqueue count* is nonzero.

`MPI_REQUEST_GET_STATUS` returns `flag = false` while `request` is *enqueued* or *active*,

without deallocating or otherwise altering `request`, exactly as it would for any other incomplete operation. As for any other active *persistent request*, calling `MPI_REQUEST_FREE` on `request` while its *enqueue count* is nonzero only marks it for freeing: `request` is set to `MPI_REQUEST_NULL` immediately, while the underlying operation, together with any starts and waits already enqueued for it, continues to execute on its *bound queue* to completion exactly as if it had not been freed, and MPI performs the *freeing stage* once its *enqueue count* would otherwise have reached zero. All other MPI operations on `request`, including `MPI_WAIT`, `MPI_TEST`, and `MPI_CANCEL`, are erroneous while `request`'s *enqueue count* is nonzero.

`MPI_ENQUEUE_START` is a *local* operation.

`MPI_ENQUEUE_STARTALL(queue, count, array of requests)`

INOUT	queue	address of queue on which to enqueue startall (handle)
IN	count	list length (nonnegative integer)
INOUT	array of requests	array of inactive matched persistent requests to start (array of handles)

C binding

```
int MPI_Enqueue_startall(MPI_Queue *queue, int count, MPI_Request array of
                        requests[])
```

Fortran 2008 binding

```
MPI_Enqueue_startall(queue, count, array of requests, ierror)
  TYPE(MPI_Queue), INTENT(INOUT) :: queue
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array of requests(count)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ENQUEUE_STARTALL(Queue, COUNT, ARRAY OF REQUESTS, IERROR)
  INTEGER Queue, COUNT, ARRAY OF REQUESTS(*), IERROR
```

`MPI_ENQUEUE_STARTALL` has the same effect as the execution of `MPI_ENQUEUE_START` for each element of `array of requests` in some arbitrary order. Each request in `array of requests` shall satisfy the preconditions required by `MPI_ENQUEUE_START`. If any request in the array does not satisfy these preconditions, no request in the array is enqueued and the error class `MPI_ERR_QUEUE` is returned. `MPI_ENQUEUE_STARTALL` with an array of length one is equivalent to `MPI_ENQUEUE_START`.

`MPI_ENQUEUE_STARTALL` is a *local* operation.

`MPI_ENQUEUE_WAIT(queue, request, status)`

INOUT	queue	address of queue on which to enqueue wait (handle)
INOUT	request	matched persistent request whose start is pending on queue (handle)
OUT	status	status object for the completed operation (status)

C binding

```
int MPI_Enqueue_wait(MPI_Queue *queue, MPI_Request *request,
                    MPI_Status *status)
```

Fortran 2008 binding

```
MPI_Enqueue_wait(queue, request, status, ierror)
    TYPE(MPI_Queue), INTENT(INOUT) :: queue
    TYPE(MPI_Request), INTENT(INOUT) :: request
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ENQUEUE_WAIT(Queue, Request, Status, Ierror)
    INTEGER Queue, Request, Status(MPI_STATUS_SIZE), Ierror
```

`MPI_ENQUEUE_WAIT` enqueues a wait for the completion of the communication operation identified by `request` on `queue`. Queue execution is blocked until `request` has *completed*.

The `request` argument shall be a handle to a *matched persistent request* whose start has been enqueued on `queue` via `MPI_ENQUEUE_START`. It is erroneous to enqueue a wait for `request` on a different *queue* than the one on which its start was enqueued, or to enqueue a wait for a request whose start has not yet been enqueued.

When the enqueued wait *completes*, `request` is marked *inactive* and `status` is set to reflect the completed operation, with the same content as described for `MPI_WAIT`. If `status` is `MPI_STATUS_IGNORE`, no status information is stored. Because the enqueued request is processed asynchronously, the contents of the `status` argument and the state of the `request` object are undefined until the enqueued wait has completed. An MPI program may determine or ensure the completion of the enqueued wait by calling `MPI_QUEUE_TEST` or `MPI_QUEUE_WAIT` on `queue`, or through another mechanism specific to the associated asynchronous execution system. Once the caller has ensured that the enqueued wait has completed, it may safely examine the `status` argument or call `MPI_REQUEST_FREE` to free the request. When the enqueued wait completes, `request`'s *enqueue count* for `queue` is decremented; once this count reaches zero, `request` is no longer bound to `queue` and may subsequently be enqueued on a different *queue* (see Section 13.2.1).

`MPI_ENQUEUE_WAIT` is a *local* operation.

```
MPI_ENQUEUE_WAITALL(queue, count, array of requests, array of statuses)
```

INOUT	queue	address of queue on which to enqueue waitall (handle)
IN	count	list length (nonnegative integer)
INOUT	array of requests	array of matched persistent requests whose starts are pending on queue (array of handles)
OUT	array of statuses	array of status objects for the completed operations (array of status)

C binding

```
int MPI_Enqueue_waitall(MPI_Queue *queue, int count, MPI_Request array of
                        requests[], MPI_Status array of statuses[])
```

Fortran 2008 binding

```

MPI_Enqueue_waitall(queue, count, array of requests, array of statuses, ierror)
  TYPE(MPI_Queue), INTENT(INOUT) :: queue
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array of requests(count)
  TYPE(MPI_Status) :: array of statuses(count)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ENQUEUE_WAITALL(Queue, COUNT, ARRAY OF REQUESTS, ARRAY OF STATUSES, IERROR)
  INTEGER Queue, COUNT, ARRAY OF REQUESTS(*),
    ARRAY OF STATUSES(MPI_STATUS_SIZE, count), IERROR

```

MPI_ENQUEUE_WAITALL has the same effect as the execution of MPI_ENQUEUE_WAIT for each element of array of requests in some arbitrary order. Each request in array of requests shall satisfy the preconditions required by MPI_ENQUEUE_WAIT. Upon completion of each enqueued wait, array of statuses[i] is set to reflect the completed operation for array of requests[i], with the same content as described for MPI_WAIT. If array of statuses is MPI_STATUSES_IGNORE, no status information is stored. MPI_ENQUEUE_WAITALL with an array of length one is equivalent to MPI_ENQUEUE_WAIT.

MPI_ENQUEUE_WAITALL is a *local* operation.

13.2.4 Queue Completion Operations

```

MPI_QUEUE_TEST(queue, flag)

```

IN	queue	queue to test for completion (handle)
OUT	flag	true if all operations enqueued on queue have completed (logical)

C binding

```

int MPI_Queue_test(MPI_Queue queue, int *flag)

```

Fortran 2008 binding

```

MPI_Queue_test(queue, flag, ierror)
  TYPE(MPI_Queue), INTENT(IN) :: queue
  LOGICAL, INTENT(OUT) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_QUEUE_TEST(Queue, FLAG, IERROR)
  INTEGER Queue, IERROR
  LOGICAL FLAG

```

MPI_QUEUE_TEST sets flag to true if all MPI operations, and, if applicable, all associated operations on the external execution context bound to queue, enqueued on queue have *completed*, and to false otherwise. MPI_QUEUE_TEST does not block the calling thread and does not alter the state of queue or of any request enqueued on it.

MPI_QUEUE_TEST is a *local* operation.

1 MPI_QUEUE_WAIT(queue)

2 INOUT queue

address of queue on which to wait for all enqueued
operations to complete (handle)

5 C binding

6 int MPI_Queue_wait(MPI_Queue *queue)

8 Fortran 2008 binding

9 MPI_Queue_wait(queue, ierror)

10 TYPE(MPI_Queue), INTENT(INOUT) :: queue

11 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

12 Fortran binding

13 MPI_QUEUE_WAIT(Queue, Ierror)

14 INTEGER Queue, Ierror

15
16 MPI_QUEUE_WAIT blocks the calling thread until all operations previously enqueued
17 on *queue* have *completed*. This call synchronizes the calling thread with the external ex-
18 ecution context associated with *queue* and allows the calling thread to determine when all
19 enqueued communication operations have finished. After MPI_QUEUE_WAIT returns, all
20 requests whose waits were enqueued on *queue* are *inactive*, are no longer bound to *queue*,
21 and their associated communication buffers may be safely reused.

22 MPI_QUEUE_WAIT is a *nonlocal* operation.

24 13.2.5 Queue Types

25 The MPI standard defines only a single predefined queue type:

26 MPI_QUEUE_TYPE_DEFAULT. When this *queue* type is used, the associated external ex-
27 ecution environment handle is ignored. When using MPI_QUEUE_TYPE_DEFAULT, calls
28 to MPI_ENQUEUE_START and MPI_ENQUEUE_STARTALL are semantically equivalent
29 to calling, respectively, MPI_START and MPI_STARTALL, and calls to
30 MPI_ENQUEUE_WAIT and MPI_ENQUEUE_WAITALL are semantically equivalent to call-
31 ing, respectively, MPI_WAIT and MPI_WAITALL: both block the calling thread until the
32 corresponding communication operations have completed. Because a *queue* of type
33 MPI_QUEUE_TYPE_DEFAULT therefore never has outstanding enqueued waits once control
34 returns to the calling thread, a subsequent call to MPI_QUEUE_TEST on that *queue* always
35 returns true and a call to MPI_QUEUE_WAIT always returns immediately.

38 13.3 Matched Persistent Operations

39 A *persistent* point-to-point or collective communication request shall be **matched** (via
40 MPI_MATCH or one of its variants) before a **matched persistent request** derived from
41 it may be enqueued on a *queue*. Matching takes an *inactive persistent request* and returns
42 a new, separate *matched persistent request* handle in a distinct request object; the original
43 persistent request handle is left unchanged by a successful matching operation and remains
44 valid for its own, independent use, including being passed to MPI_MATCH or one of its
45 variants again to create another *matched persistent request*. Matching resolves the pairing
46 between all buffers and exchanges the information required to carry out the data transfer, so
47 that subsequent starts enqueued on the *queue* do not require per-operation tag matching,
48

buffer negotiation, or other similar synchronization with the remote MPI process. This separation of matching and other *nonlocal* initialization operations from data movement is essential for use in external execution contexts in which these operations cannot be executed directly. *Partitioned persistent communication requests* do not use these matching operations; see Section 13.3.1.

`MPI_MATCH`, `MPI_IMATCH`, `MPI_MATCHALL`, or `MPI_IMATCHALL` on a persistent point-to-point communication request is a point-to-point operation between two ranks, the sender and receiver ranks of the corresponding point-to-point operations. These functions follow the same MPI matching rules as for point-to-point communication (see Section 3), with communicator, tag, and source dictating message matching, but may only match a persistent send against a persistent receive that is also being matched. `MPI_ANY_SOURCE` and `MPI_ANY_TAG` are permitted as the source and tag of a persistent receive request passed to one of these functions; the resulting *matched persistent request* is thereafter bound to the specific source and tag with which it was matched. In the event that the communicator, tag, and source of persistent point-to-point requests do not uniquely identify a message, the order in which the relevant `MPI_MATCH` function is called is the order in which they are matched. `MPI_MATCHALL` and `MPI_IMATCHALL` do not define an order in which the elements of their request array are matched; a program whose correctness depends on a particular matching order shall disambiguate using the communicator, tag, and source of the individual requests. A single call to `MPI_MATCHALL` or `MPI_IMATCHALL` may mix persistent point-to-point and persistent collective communication requests within the same array of requests, with the same semantics as mixing such requests in a single call to `MPI_STARTALL` or `MPI_WAITALL`.

`MPI_MATCH`, `MPI_IMATCH`, `MPI_MATCHALL`, or `MPI_IMATCHALL` on a persistent collective communication request is a collective operation between all ranks on the communicator associated with the request. These functions may complete only when all persistent collective communication initialization operations associated are complete. Once enqueued, the underlying collective communication operations associated with *matched persistent requests* are subject to the same ordering requirements as any other MPI collective operation: all processes in the associated communicator's group shall enqueue corresponding collective operations in the same order (see Section 13.4).

Once *matched*, a *matched persistent request* retains its match permanently until it is freed via `MPI_REQUEST_FREE`.

As for any other *persistent request* (see Section 3), `MPI_REQUEST_FREE` applied to a *matched persistent request* is a *local* operation: it affects only the local request object and performs no communication with, and requires no coordination from, any remote MPI process. This holds regardless of whether the request's *enqueue count* is zero; see Section 13.2.3 for the resulting semantics when `MPI_REQUEST_FREE` is called while a request is *enqueued* or *active*. Freeing one side of a matched pairing therefore has no defined effect on the corresponding *matched persistent request*, if any, held by the remote process; that request remains valid and may still be enqueued or started. This is the same approach taken by *partitioned persistent communication requests* (Section 13.3.1), whose initialization and freeing are likewise local on each side, with pairing resolved through the ordinary communication protocol rather than through any cross-process bookkeeping that freeing would need to update. A program that frees one side of a matched pairing before that side's communication is ever started, while the remote side's corresponding request is subsequently started, produces the same outcome as failing to start any other matching send or receive: the remote operation does not complete. MPI already requires that programs

ensure that matching sends and receives are started (Section 13.4); this requirement applies equally here.

MPI_MATCH(request, info, newrequest)

IN	request	inactive persistent request to match; unchanged on return (handle)
IN	info	info argument providing optimization hints for the created matched persistent request (handle)
OUT	newrequest	handle to the new matched persistent request (handle)

C binding

```
int MPI_Match(MPI_Request request, MPI_Info info, MPI_Request *newrequest)
```

Fortran 2008 binding

```
MPI_Match(request, info, newrequest, ierror)
  TYPE(MPI_Request), INTENT(IN) :: request
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: newrequest
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_MATCH(REQUEST, INFO, NEWREQUEST, IERROR)
  INTEGER REQUEST, INFO, NEWREQUEST, IERROR
```

MPI_MATCH blocks until the *persistent request* `request` is *matched* with its corresponding *persistent request* on the remote MPI process, and returns a handle to the resulting *matched persistent request* in `newrequest`. The `request` argument shall be a handle to an *inactive persistent request* created by a persistent point-to-point or collective operation. It is erroneous to pass a *nonpersistent*, *nonblocking*, *generalized*, or *partitioned* request to **MPI_MATCH**. After **MPI_MATCH** returns, `request` is unchanged and remains a valid, *inactive persistent request*, and `newrequest` is *inactive* and may be enqueued via **MPI_ENQUEUE_START**. `info` provides hints to the implementation for the created `newrequest`; it may, for example, supply optimization information that could not be passed when `request` was originally created (e.g., via **MPI_SEND_INIT**) or that has since become available. **MPI_INFO_NULL** may always be passed. This chapter reserves no `info` keys.

MPI_MATCH is a *nonlocal* operation.

MPI_IMATCH(tomatch, info, newrequest, matchrequest)

IN	tomatch	inactive persistent request to match; unchanged on return (handle)
IN	info	info argument providing optimization hints for the created matched persistent request (handle)
OUT	newrequest	handle to the new matched persistent request, not usable until matching completes (handle)
OUT	matchrequest	request for the matching operation (handle)

C binding

```
int MPI_IMatch(MPI_Request tomatch, MPI_Info info, MPI_Request *newrequest,
               MPI_Request *matchrequest)
```

Fortran 2008 binding

```
MPI_IMatch(tomatch, info, newrequest, matchrequest, ierror)
  TYPE(MPI_Request), INTENT(IN) :: tomatch
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: newrequest, matchrequest
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_IMATCH(TOMATCH, INFO, NEWREQUEST, MATCHREQUEST, IERROR)
  INTEGER TOMATCH, INFO, NEWREQUEST, MATCHREQUEST, IERROR
```

MPI_IMATCH initiates a *nonblocking* match for the *persistent request* tomatch, immediately returns a handle to the resulting *matched persistent request* in newrequest, and returns a new request handle in matchrequest that may be used to test or wait for the completion of the matching operation. The tomatch argument shall satisfy the same preconditions as the request argument of MPI_MATCH. tomatch is unchanged when MPI_IMATCH returns and remains a valid, *inactive persistent request*. The returned matchrequest is a *nonblocking, nonpersistent* request that cannot be canceled; it is considered complete when the match for newrequest has been resolved. It is erroneous to enqueue newrequest via MPI_ENQUEUE_START until the matching operation identified by matchrequest has completed. info has the same meaning as for MPI_MATCH.

MPI_IMATCH is a *local* operation; completion of the matching operation it initiates is *nonlocal*.

```
MPI_MATCHALL(count, array of requests, info, array of newrequests)
```

IN	count	list length (nonnegative integer)
IN	array of requests	array of inactive persistent requests to match; each is unchanged on return (array of handles)
IN	info	info argument providing optimization hints for the created matched persistent requests (handle)
OUT	array of newrequests	array of handles to the new matched persistent requests (array of handles)

C binding

```
int MPI_Matchall(int count, MPI_Request array of requests[], MPI_Info info,
                 MPI_Request array of newrequests[])
```

Fortran 2008 binding

```
MPI_Matchall(count, array of requests, info, array of newrequests, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(IN) :: array of requests(count)
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Request), INTENT(OUT) :: array of newrequests(count)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_MATCHALL(COUNT, ARRAY OF REQUESTS, INFO, ARRAY OF NEWREQUESTS, IERROR)
      INTEGER COUNT, ARRAY OF REQUESTS(*), INFO, ARRAY OF NEWREQUESTS(*), IERROR
```

`MPI_MATCHALL` has the same effect as calling `MPI_MATCH` on each element of array of requests, in some arbitrary order, storing the resulting handle in the corresponding element of array of newrequests. Each request in array of requests shall satisfy the preconditions required by `MPI_MATCH`. `info` has the same meaning as for `MPI_MATCH` and applies to every element of array of newrequests. `MPI_MATCHALL` with an array of length one is equivalent to `MPI_MATCH`.

`MPI_MATCHALL` is a *nonlocal* operation.

```
MPI_IMATCHALL(count, array of requests, info, array of newrequests, matchrequest)
```

IN	count	list length (nonnegative integer)
IN	array of requests	array of inactive persistent requests to match; each is unchanged on return (array of handles)
IN	info	info argument providing optimization hints for the created matched persistent requests (handle)
OUT	array of newrequests	array of handles to the new matched persistent requests, not usable until matching completes (array of handles)
OUT	matchrequest	request for the matching operation (handle)

C binding

```
int MPI_IMatchall(int count, MPI_Request array of requests[], MPI_Info info,
      MPI_Request array of newrequests[], MPI_Request *matchrequest)
```

Fortran 2008 binding

```
MPI_IMatchall(count, array of requests, info, array of newrequests,
      matchrequest, ierror)
      INTEGER, INTENT(IN) :: count
      TYPE(MPI_Request), INTENT(IN) :: array of requests(count)
      TYPE(MPI_Info), INTENT(IN) :: info
      TYPE(MPI_Request), INTENT(OUT) :: array of newrequests(count), matchrequest
      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_IMATCHALL(COUNT, ARRAY OF REQUESTS, INFO, ARRAY OF NEWREQUESTS,
      MATCHREQUEST, IERROR)
      INTEGER COUNT, ARRAY OF REQUESTS(*), INFO, ARRAY OF NEWREQUESTS(*),
      MATCHREQUEST, IERROR
```

`MPI_IMATCHALL` initiates nonblocking matching for all requests in array of requests, immediately returns handles to the resulting matched persistent requests in array of newrequests, and returns a single new request handle in `matchrequest` that is complete when all requests in the array have been *matched*. Each request in array of requests shall satisfy the preconditions required by `MPI_IMATCH`. `info` has the same meaning as for `MPI_MATCH` and applies to every element of array of newrequests. The returned `matchrequest` is *nonblocking* and *nonpersistent* and cannot be canceled. It is

erroneous to enqueue any element of array of newrequests via `MPI_ENQUEUE_START` until `matchrequest` has completed. `MPI_IMATCHALL` with an array of length one is equivalent to `MPI_IMATCH`.

`MPI_IMATCHALL` is a *local* operation; completion of the matching operations it initiates is *nonlocal*.

13.3.1 Partitioned Communication Requests

Partitioned persistent communication requests do not require matching. A partitioned persistent request created by `MPI_PSEND_INIT` or `MPI_PRECV_INIT` may be enqueued directly via `MPI_ENQUEUE_START` or `MPI_ENQUEUE_STARTALL` without first being passed to `MPI_MATCH` or one of its variants.

`MPI_PREADY` and its variants, and `MPI_PARRIVED`, may be called on a partitioned persistent send or receive request whose start has been enqueued or which is already *active*, notwithstanding the restriction in Section 13.2.3 on other MPI operations while a request's *enqueue count* is nonzero. Because these calls are not themselves enqueued and are not ordered with respect to the *queue*, it is the responsibility of the calling program to ensure that the associated partitioned request has already become *active* (i.e., that its enqueued start has already been initiated by the *queue*) before calling them; calling `MPI_PREADY` or one of its variants, or `MPI_PARRIVED`, on a request that is still only *enqueued* and not yet *active* is erroneous. There is no enqueueable variant of `MPI_PREADY` or its variants.

13.4 Concurrency and Progress

Ordering within a *queue*. Start and wait operations enqueued on a *queue* execute in the order in which they were enqueued. Specifically, an enqueued start does not initiate the underlying communication until all previously enqueued start operations on the same *queue* have initiated and all previously enqueued wait operations have completed; similarly, an enqueued wait does not complete until both the associated communication request has completed and all previously enqueued waits on the same *queue* have completed. Start operations and wait operations enqueued simultaneously (using, e.g., `MPI_ENQUEUE_STARTALL` or `MPI_ENQUEUE_WAITALL`), may initiate or complete the specified requests in any order. How enqueued start and wait operations are ordered with respect to non-MPI operations on the associated asynchronous execution context is separately defined by the relevant asynchronous programming system.

Buffer usage. The send or receive buffer associated with a request is considered in use by MPI from the return of the `MPI_ENQUEUE_START` or `MPI_ENQUEUE_STARTALL` call that enqueues its start, not from the later point at which the *queue* actually initiates the corresponding communication, until the corresponding enqueued wait is known by the calling program to have completed (for example, via `MPI_QUEUE_TEST` or `MPI_QUEUE_WAIT`). It is erroneous for a program to access or modify such a buffer before that point.

Concurrency between *queues*. Operations enqueued on different *queues* may proceed concurrently. The *queue* abstraction is local; *queues* on the same or different MPI processes

have no inherent ordering relationship with respect to one another. An MPI program that requires ordering between operations on two different *queues* shall introduce explicit synchronization, for example by calling `MPI_QUEUE_WAIT` on both *queues*.

Rationale. The serial-execution-context guarantee within a *queue*, combined with the independence between *queues*, allows implementations to assign dedicated network and hardware resources to each *queue* and to omit locking on those resources within the execution of a single *queue*. This is the primary performance motivation for the *queue* abstraction. (*End of rationale.*)

Thread safety. MPI programs that use *queue* objects may use any MPI thread model. A program is not required to serialize its own calls that enqueue operations on the same *queue* object; under `MPI_THREAD_MULTIPLE`, multiple threads may concurrently call `MPI_ENQUEUE_START`, `MPI_ENQUEUE_WAIT`, `MPI_ENQUEUE_STARTALL`, `MPI_ENQUEUE_WAITALL`, `MPI_QUEUE_TEST`, and `MPI_QUEUE_WAIT` on the same *queue*, consistent with MPI's general thread-safety requirements. The relative order in which operations enqueued concurrently by different threads are placed on the *queue* is unspecified; a program that requires a specific relative order between operations issued by different threads shall itself synchronize the issuing threads before enqueueing.

Operations started by external execution contexts shall respect the provided MPI thread model.

Progress of enqueued operations. An MPI implementation is required to guarantee progress on operations enqueued on a *queue*. Progress on a *queue* is guaranteed during any call to `MPI_QUEUE_TEST` or `MPI_QUEUE_WAIT` on that *queue*, and during any *blocked* MPI procedure call or MPI test procedure on the same MPI process (see Section 2.9). Additional mechanisms by which an implementation may guarantee progress include a dedicated progress thread, offloading to a network interface controller, or coordination with the external execution context associated with the *queue*.

Progress on a *queue* is guaranteed to be provided independently of any enqueued operations on other *queues*. An MPI process that has enqueued operations on a *queue* but does not subsequently call `MPI_QUEUE_TEST`, `MPI_QUEUE_WAIT`, or any other MPI procedure may not be guaranteed progress; programs that require communication to complete without calling into MPI on the CPU shall use `MPI_QUEUE_TEST` or `MPI_QUEUE_WAIT` or rely on implementation-specific guarantees provided by the associated external execution context.

Finalization. As with other pending MPI communication (Section 11.2.2), a process shall ensure that all operations it has enqueued on a *queue* have completed, for example via `MPI_QUEUE_WAIT`, before calling `MPI_FINALIZE` or `MPI_SESSION_FINALIZE`. Because `MPI_CANCEL` is *erroneous* on an *enqueued* or *active* request (Section 13.2.3), draining a *queue* through completion is the only way to retire outstanding enqueued operations before finalization.

Error reporting. An error detected during the execution of an operation enqueued on a *queue* is reported through the *error handler* currently associated with the communicator of the *request* for which the error occurred, exactly as it would be for the equivalent operation

issued directly by the calling thread. Because enqueued operations execute asynchronously, this error handler may be invoked from a context other than the thread that enqueued the operation, and at a time later than the call that enqueued it.

Queue usage errors. `MPI_ERR_QUEUE` is the error class returned for the queue- and request-usage errors that this chapter defines as synchronously detected, namely passing an unenqueueable request to `MPI_ENQUEUE_START` or `MPI_ENQUEUE_STARTALL` (Section 13.2.3).

Rationale. For some external execution contexts, such as GPU streams, the execution context itself may drive progress without CPU involvement. The standard does not mandate this capability because it depends on implementation and hardware support, but conforming implementations are encouraged to provide it for contexts where it is feasible. (*End of rationale.*)

Ordering with respect to standard MPI operations. A *matched persistent request* or *partitioned persistent communication request* used with the enqueueing operations in this chapter may also be accessed through standard MPI operations (for example, `MPI_START` or `MPI_STARTALL`) while it has no *bound queue*. The restrictions on which MPI operations may be performed on such a request while its *enqueue count* is nonzero are given in Section 13.2.3 and Section 13.3.1.

Enqueued communication operations interact with standard MPI matching rules in the same way as equivalent operations issued by the calling thread, subject to the restriction, given in Section 13.3, that a *matched persistent request* may only match another request that has itself been matched via `MPI_MATCH` or one of its variants.

Deadlock. Enqueued operations are executed asynchronously with respect to the calling thread, introducing potential sources of deadlock into programs and implementations. In particular:

- An enqueued wait on a *queue* blocks *queue* execution until the corresponding communication has completed. If the matching operation on the remote MPI process is never started, the enqueued wait never completes. MPI already requires that programs ensure that matching sends and receives are started; this requirement applies equally to enqueued operations.
- An external execution context may impose additional constraints on when enqueued operations are eligible for execution. For example, a GPU scheduler that is not given sufficient resources to schedule a stream containing enqueued communication operations may deadlock. Programs should ensure that external execution contexts associated with MPI *queues* are given appropriate scheduling priority.

13.5 Examples

The following example illustrates the use of queued communication to implement a ring exchange across a one-dimensional ring of MPI processes. The example is organized in two phases. In the setup phase (done once before the iteration loop), a *queue* is created and

bound to an external execution context, persistent send and receive requests are created for each neighbor, and all requests are *matched* via `MPI_MATCHALL` to move the matching protocol off the per-iteration critical path. In the iteration phase, starts for the receive and send requests are enqueued on the *queue*, followed by an enqueued wait for all requests. After all iterations, `MPI_QUEUE_WAIT` blocks the calling thread until all enqueued operations have completed.

Example 13.1. Queued data exchange in a one-dimensional ring of MPI processes. Code that interfaces with an external execution environment would include a context for that environment as part of *queue* creation and use an execution environment-specific call to schedule computation at the commented points relative to the MPI communication calls.

```
#include "mpi.h"
#define N 1024
#define NITER 100

int main(int argc, char *argv[])
{
    int rank, size, left, right;
    double send_left[N], send_right[N];
    double recv_left[N], recv_right[N];
    MPI_Queue queue;
    MPI_Request reqs[4];
    MPI_Request matched_reqs[4];
    MPI_Status statuses[4];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    /* Determine neighbors in a 1-D ring topology */
    left = (rank - 1 + size) % size;
    right = (rank + 1) % size;

    /* Create a queue. A system-specific type and external execution
     * context can be used if available. */
    MPI_Queue_create(&queue, MPI_QUEUE_TYPE_DEFAULT, NULL, MPI_INFO_NULL);

    /* Create persistent send and receive requests for ring exchange */
    MPI_Recv_init(recv_left, N, MPI_DOUBLE, left, 0,
                 MPI_COMM_WORLD, &reqs[0]);
    MPI_Recv_init(recv_right, N, MPI_DOUBLE, right, 0,
                 MPI_COMM_WORLD, &reqs[1]);
    MPI_Send_init(send_left, N, MPI_DOUBLE, left, 0,
                 MPI_COMM_WORLD, &reqs[2]);
    MPI_Send_init(send_right, N, MPI_DOUBLE, right, 0,
                 MPI_COMM_WORLD, &reqs[3]);

    /* Match all requests once, removing per-iteration tag matching and
     * buffer negotiation from the communication critical path. reqs[0..3]
     * remain valid, ordinary persistent requests and are freed separately
     * below; the enqueueable requests are returned in matched_reqs[0..3]. */
    MPI_Matchall(4, reqs, MPI_INFO_NULL, matched_reqs);
```

```

1  /* Enqueue the data exchange operations each iteration */
2  for (int iter = 0; iter < NITER; iter++) {
3      MPI_Enqueue_startall(queue, 2, &matched_reqs[0]);
4
5      /* Schedule iteration work needed prior to sending
6       * (e.g., packing send buffers) here */
7      MPI_Enqueue_startall(queue, 2, &matched_reqs[2]);
8
9      /* Schedule work overlapped with communication here */
10
11     /* Enqueue waits for all sends and receives to complete */
12     MPI_Enqueue_waitall(queue, 4, matched_reqs, statuses);
13
14     /* Schedule work that depends on the results of the receive
15      * (e.g., unpacking receive buffers) here */
16 }
17
18 /* Block until all enqueued communication operations complete */
19 MPI_Queue_wait(queue);
20
21 /* Free the matched requests and the original requests they were
22  * derived from, then the queue */
23 for (int i = 0; i < 4; i++) {
24     MPI_Request_free(&matched_reqs[i]);
25     MPI_Request_free(&reqs[i]);
26 }
27 MPI_Queue_free(&queue);
28
29 MPI_Finalize();
30 return 0;
31 }

```

In this example, the setup phase creates four *persistent requests* (two receives and two sends) and matches them all with a single call to `MPI_MATCHALL`, which leaves `reqs[0..3]` unchanged and returns the resulting *matched persistent requests* in `matched_reqs[0..3]`. This call resolves the pairing between sends and receives on both processes and exchanges the information needed for data transfer, so that subsequent enqueued starts do not require per-operation tag matching. Matching is a *nonlocal* operation: both the sending and receiving processes shall call a matching operation for the match to complete. Because `MPI_MATCHALL` does not define an order in which its requests are matched, and all four requests here use the same tag, each rank relies on *source* (left and right) to disambiguate the two receives from one another.

In the iteration loop, `MPI_ENQUEUE_STARTALL` is called first for the receive requests and then for the send requests. Both calls are *local* operations that return immediately after recording the enqueued starts; neither call blocks the calling thread or blocks execution of the *queue*. `MPI_ENQUEUE_WAITALL` enqueues waits for all four requests and is likewise a *local* operation. Execution of the enqueued starts and waits on the *queue* proceeds asynchronously, ordered with respect to any computational operations enqueued on the same external execution context. The `statuses` array is reused, unread, across all `NITER` iterations; this is safe because operations enqueued on a single *queue* execute in order (Sec-

tion 13.4), so each iteration's `MPI_ENQUEUE_WAITALL` finishes writing `statuses` before the next iteration's is enqueued. A program that needs to examine an intermediate iteration's status values, or that shares a status object across more than one *queue*, needs a separate status object per outstanding enqueued wait.

After the loop, `MPI_QUEUE_WAIT` synchronizes the calling thread with the *queue*, blocking until all enqueued operations have completed. Once `MPI_QUEUE_WAIT` returns, all requests are *inactive* and their communication buffers may be safely reused or freed. The cleanup loop frees both `matched_reqs[0..3]` and `reqs[0..3]`: since `MPI_MATCHALL` left the latter unchanged rather than invalidating them, both sets of handles remain live objects that must each be freed via `MPI_REQUEST_FREE`.

DRAFT

Chapter 14

External Interfaces

14.1 Introduction

This chapter contains calls used to create **generalized requests**, which allow users to create new nonblocking operations with an interface similar to what is present in MPI. These calls can be used to layer new functionality on top of MPI. Section 14.3 deals with setting the information found in **status**. This functionality is needed for generalized requests.

14.2 Generalized Requests

The goal of generalized requests is to allow users to define new nonblocking operations. Such an outstanding nonblocking operation is represented by a (generalized) request. A fundamental property of nonblocking operations is that *progress* toward the completion of this operation occurs asynchronously, i.e., concurrently with normal program execution. Typically, this requires execution of code concurrently with the execution of the user code, e.g., in a separate thread or in a signal handler. Operating systems provide a variety of mechanisms in support of concurrent execution. MPI does not attempt to standardize or to replace these mechanisms: it is assumed programmers who wish to define new asynchronous operations will use the mechanisms provided by the underlying operating system. Thus, the calls in this section only provide a means for defining the effect of MPI calls such as `MPI_WAIT` or `MPI_CANCEL` when they apply to generalized requests, and for signaling to MPI the completion of a generalized operation.

Rationale. It is tempting to also define an MPI standard mechanism for achieving concurrent execution of user-defined nonblocking operations. However, it is difficult to define such a mechanism without consideration of the specific mechanisms used in the operating system. The Forum feels that concurrency mechanisms are a proper part of the underlying operating system and should not be standardized by MPI; the MPI standard should only deal with the interaction of such mechanisms with MPI. (*End of rationale.*)

For a regular request, the operation associated with the request is performed by the MPI implementation, and the operation completes without intervention by the application. For a generalized request, the operation associated with the request is performed by the application; therefore, the application must notify MPI through a call to `MPI_GREQUEST_COMPLETE` when the operation completes. MPI maintains the “completion” status of generalized requests. Any other request state has to be maintained by the user.

A new generalized request is started with


```

1 MPI_GREQUEST_START(query_fn, free_fn, cancel_fn, extra_state, request)
2     IN          query_fn          callback function invoked when request status is
3                                   queried (function)
4     IN          free_fn           callback function invoked when request is freed
5                                   (function)
6     IN          cancel_fn         callback function invoked when request is cancelled
7                                   (function)
8     IN          extra_state       extra state
9
10    OUT         request            generalized request (handle)

```

C binding

```

13 int MPI_Grequest_start(MPI_Grequest_query_function *query_fn,
14                        MPI_Grequest_free_function *free_fn,
15                        MPI_Grequest_cancel_function *cancel_fn, void *extra_state,
16                        MPI_Request *request)

```

Fortran 2008 binding

```

18 MPI_Grequest_start(query_fn, free_fn, cancel_fn, extra_state, request, ierror)
19     PROCEDURE(MPI_Grequest_query_function) :: query_fn
20     PROCEDURE(MPI_Grequest_free_function) :: free_fn
21     PROCEDURE(MPI_Grequest_cancel_function) :: cancel_fn
22     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
23     TYPE(MPI_Request), INTENT(OUT) :: request
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

27 MPI_GREQUEST_START(QUERY_FN, FREE_FN, CANCEL_FN, EXTRA_STATE, REQUEST, IERROR)
28     EXTERNAL QUERY_FN, FREE_FN, CANCEL_FN
29     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
30     INTEGER REQUEST, IERROR

```

Advice to users. Note that a generalized request is of the same type as regular requests, in C and Fortran. (*End of advice to users.*)

The call starts a generalized request and returns a handle to it in `request`.

The syntax and meaning of the callback functions are listed below. All callback functions are passed the `extra_state` argument that was associated with the request by the starting call `MPI_GREQUEST_START`; `extra_state` can be used to maintain user-defined state for the request.

In C, the query procedure is

```

41 typedef int MPI_Grequest_query_function(void *extra_state, MPI_Status *status);

```

in Fortran with the `mpi_f08` module

```

44 ABSTRACT INTERFACE
45     SUBROUTINE MPI_Grequest_query_function(extra_state, status, ierror)
46         INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
47         TYPE(MPI_Status) :: status
48         INTEGER :: ierror

```


in Fortran with the `mpi` module and (deprecated) `mpif.h` include file

```

SUBROUTINE GREQUEST_QUERY_FUNCTION(EXTRA_STATE, STATUS, IERROR)
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
  INTEGER STATUS(MPI_STATUS_SIZE), IERROR

```

The `query_fn` function computes the status that should be returned for the generalized request. The status also includes information about successful/unsuccessful cancellation of the request (result to be returned by `MPI_TEST_CANCELLED`).

The `query_fn` callback is invoked by the `MPI_{WAIT|TEST}{|ANY|SOME|ALL}` call that completed the generalized request associated with this callback. The callback function is also invoked by calls to `MPI_REQUEST_GET_STATUS`, if the request is complete when the call occurs. In both cases, the callback is passed a reference to the corresponding status variable passed by the user to the MPI call; the status set by the callback function is returned by the MPI call. If the user provided `MPI_STATUS_IGNORE` or `MPI_STATUSES_IGNORE` to the MPI procedure that causes `query_fn` to be called, then MPI will pass a valid status object to `query_fn`, and this status will be ignored upon return of the callback function. Note that `query_fn` is invoked only after `MPI_GREQUEST_COMPLETE` is called on the request; it may be invoked several times for the same generalized request, e.g., if the user calls `MPI_REQUEST_GET_STATUS` several times for this request. Note also that a call to `MPI_{WAIT|TEST}{SOME|ALL}` may cause multiple invocations of `query_fn` callback functions, one for each generalized request that is completed by the MPI call. The order of these invocations is not specified by MPI.

In C, the free procedure is

```

typedef int MPI_Grequest_free_function(void *extra_state);

```

in Fortran with the `mpi_f08` module

```

ABSTRACT INTERFACE
  SUBROUTINE MPI_Grequest_free_function(extra_state, ierror)
    INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
    INTEGER :: ierror

```

in Fortran with the `mpi` module and (deprecated) `mpif.h` include file

```

SUBROUTINE GREQUEST_FREE_FUNCTION(EXTRA_STATE, IERROR)
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
  INTEGER IERROR

```

The `free_fn` function is invoked to clean up user-allocated resources when the generalized request is freed.

The `free_fn` callback is invoked by the `MPI_{WAIT|TEST}{|ANY|SOME|ALL}` call that completed the generalized request associated with this callback. `free_fn` is invoked after the call to `query_fn` for the same request. However, if the MPI call completed multiple generalized requests, the order in which `free_fn` callback functions are invoked is not specified by MPI.

The `free_fn` callback is also invoked for generalized requests that are freed by a call to `MPI_REQUEST_FREE` (no call to `MPI_{WAIT|TEST}{|ANY|SOME|ALL}` will occur for such a request). In this case, the callback function will be called either in the MPI call `MPI_REQUEST_FREE(request)`, or in the MPI call `MPI_GREQUEST_COMPLETE(request)`, whichever happens last, i.e., in this case the actual freeing code is executed as soon as both calls `MPI_REQUEST_FREE` and `MPI_GREQUEST_COMPLETE` have occurred. The

request is not deallocated until after `free_fn` completes. Note that `free_fn` will be invoked only once per request by a correct program.

Advice to users. Calling `MPI_REQUEST_FREE(request)` will cause the request handle to be set to `MPI_REQUEST_NULL`. This handle to the generalized request is no longer valid. However, user copies of this handle are valid until after `free_fn` completes since MPI does not deallocate the object until then. Since `free_fn` is not called until after `MPI_GREQUEST_COMPLETE`, the user copy of the handle can be used to make this call. Users should note that MPI will deallocate the object after `free_fn` executes. At this point, user copies of the request handle no longer point to a valid request. MPI will not set user copies to `MPI_REQUEST_NULL` in this case, so it is up to the user to avoid accessing this stale handle. This is a special case in which MPI defers deallocating the object until a later time that is known by the user. (*End of advice to users.*)

In C, the cancel procedure is

```
typedef int MPI_Grequest_cancel_function(void *extra_state, int complete);
```

in Fortran with the `mpi_f08` module

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Grequest_cancel_function(extra_state, complete, ierror)
```

```
    INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
```

```
    LOGICAL :: complete
```

```
    INTEGER :: ierror
```

in Fortran with the `mpi` module and (deprecated) `mpif.h` include file

```
SUBROUTINE GREQUEST_CANCEL_FUNCTION(EXTRA_STATE, COMPLETE, IERROR)
```

```
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
```

```
  LOGICAL COMPLETE
```

```
  INTEGER IERROR
```

The `cancel_fn` function is invoked to start the cancelation of a generalized request. It is called by `MPI_CANCEL(request)`. MPI passes `complete = true` to the callback function if `MPI_GREQUEST_COMPLETE` was already called on the request, and `complete = false` otherwise.

All callback functions return an error code. The code is passed back and dealt with as appropriate for the error code by the MPI procedure that invoked the callback function. For example, if error codes are returned then the error code returned by the callback function will be returned by the MPI procedure that invoked the callback function. In the case of an `MPI_{WAIT|TEST}{ANY}` call that invokes both `query_fn` and `free_fn`, the MPI call will return the error code returned by the last callback, namely `free_fn`. If one or more of the requests in a call to `MPI_{WAIT|TEST}{SOME|ALL}` failed, then the MPI call will return `MPI_ERR_IN_STATUS`. In such a case, if the MPI call was passed an array of statuses, then MPI will return in each of the statuses that correspond to a completed generalized request the error code returned by the corresponding invocation of its `free_fn` callback function. However, if the MPI procedure was passed `MPI_STATUSES_IGNORE`, then the individual error codes returned by each callback functions will be lost.

Advice to users. `query_fn` must *not* set the error field of `status` since `query_fn` may be called by `MPI_WAIT` or `MPI_TEST`, in which case the error field of `status` should not change. The MPI library knows the “context” in which `query_fn` is invoked and can

decide correctly when to put the returned error code in the error field of `status`. (*End of advice to users.*)

`MPI_GREQUEST_COMPLETE(request)`

INOUT request generalized request (handle)

C binding

`int MPI_Grequest_complete(MPI_Request request)`

Fortran 2008 binding

`MPI_Grequest_complete(request, ierror)`
 `TYPE(MPI_Request), INTENT(IN) :: request`
 `INTEGER, OPTIONAL, INTENT(OUT) :: ierror`

Fortran binding

`MPI_GREQUEST_COMPLETE(REQUEST, IERROR)`
 `INTEGER REQUEST, IERROR`

The call informs MPI that the operations represented by the generalized request `request` are complete (see definitions in Section 2.4). A call to `MPI_WAIT(request, status)` will return and a call to `MPI_TEST(request, flag, status)` will return `flag = true` only after a call to `MPI_GREQUEST_COMPLETE` has declared that these operations are complete.

MPI imposes no restrictions on the code executed by the callback functions. However, new nonblocking operations should be defined so that the general semantic rules about MPI calls such as `MPI_TEST`, `MPI_REQUEST_FREE`, or `MPI_CANCEL` still hold. Therefore, the callback functions `query_fn`, `free_fn`, or `cancel_fn` should invoke blocking MPI communication calls only if the context is such that these calls are guaranteed to return in finite time. Once `MPI_CANCEL` is invoked, the cancelled operation should complete in finite time, irrespective of the state of other MPI processes (the operation has acquired “local” semantics). It should either succeed, or fail without side-effects. The user should guarantee these properties for newly defined operations.

Advice to implementors. A call to `MPI_GREQUEST_COMPLETE` may unblock a blocked user process/thread. The MPI library should ensure that the blocked user computation will resume. (*End of advice to implementors.*)

14.2.1 Examples

Example 14.1. This example shows the code for a user-defined reduce operation on an `int` using a binary tree: each nonroot MPI process receives two messages, sums them, and sends them up. We assume that no status is returned and that the operation cannot be cancelled.

```
typedef struct {
    MPI_Comm comm;
    int tag;
    int root;
    int valin;
    int *valout;
```

```

1  MPI_Request request;
2  } ARGS;
3
4
5  int myreduce(MPI_Comm comm, int tag, int root,
6              int valin, int *valout, MPI_Request *request)
7  {
8      ARGS *args;
9      pthread_t thread;
10
11     /* start request */
12     MPI_Grequest_start(query_fn, free_fn, cancel_fn, NULL, request);
13
14     args = (ARGS*)malloc(sizeof(ARGS));
15     args->comm = comm;
16     args->tag = tag;
17     args->root = root;
18     args->valin = valin;
19     args->valout = valout;
20     args->request = *request;
21
22     /* spawn thread to handle request */
23     /* The availability of the pthread_create call is system dependent */
24     pthread_create(&thread, NULL, reduce_thread, args);
25
26     return MPI_SUCCESS;
27 }
28
29 /* thread code */
30 void* reduce_thread(void *ptr)
31 {
32     int lchild, rchild, parent, lval, rval, val;
33     MPI_Request req[2];
34     ARGS *args;
35
36     args = (ARGS*)ptr;
37
38     /* compute left and right child and parent in tree; set
39        to MPI_PROC_NULL if does not exist */
40     /* code not shown */
41     ...
42
43     MPI_Irecv(&lval, 1, MPI_INT, lchild, args->tag, args->comm, &req[0]);
44     MPI_Irecv(&rval, 1, MPI_INT, rchild, args->tag, args->comm, &req[1]);
45     MPI_Waitall(2, req, MPI_STATUSES_IGNORE);
46     val = lval + args->valin + rval;
47     MPI_Send(&val, 1, MPI_INT, parent, args->tag, args->comm);
48     if (parent == MPI_PROC_NULL) *(args->valout) = val;
49     MPI_Grequest_complete((args->request));
50     free(ptr);
51     return(NULL);
52 }

```

```

1  int query_fn(void *extra_state, MPI_Status *status)
2  {
3      /* always send just one int */
4      MPI_Status_set_elements(status, MPI_INT, 1);
5      /* can never cancel so always true */
6      MPI_Status_set_cancelled(status, 0);
7      /* choose not to return a value for this */
8      status->MPI_SOURCE = MPI_UNDEFINED;
9      /* tag has no meaning for this generalized request */
10     status->MPI_TAG = MPI_UNDEFINED;
11     /* this generalized request never fails */
12     return MPI_SUCCESS;
13 }
14
15 int free_fn(void *extra_state)
16 {
17     /* this generalized request does not need to do any freeing */
18     /* as a result it never fails here */
19     return MPI_SUCCESS;
20 }
21
22 int cancel_fn(void *extra_state, int complete)
23 {
24     /* This generalized request does not support cancelling.
25      * Abort if not already done.
26      * If done then treat as if cancel failed.*/
27     if (!complete) {
28         fprintf(stderr,
29             "Cannot cancel generalized request - aborting program\n");
30         MPI_Abort(MPI_COMM_WORLD, 99);
31     }
32     return MPI_SUCCESS;
33 }

```

14.3 Associating Information with Status

MPI supports several different types of requests besides those for point-to-point operations, this includes MPI calls for I/O and generalized requests. It is desirable to allow these calls to use the same request mechanism, which allows one to wait or test on different types of requests. However, `MPI_{TEST|WAIT}{|ANY|SOME|ALL}` returns a status with information about the request. With the generalization of requests, one needs to define what information will be returned in the status object.

Each MPI call fills in the appropriate fields in the status object. Any unused field will have an undefined value. A call to `MPI_{TEST|WAIT}{|ANY|SOME|ALL}` can modify any of the fields in the status object. Specifically, it can modify fields that are undefined. The fields with meaningful values for a given request are defined in the respective sections.

Generalized requests raise additional considerations. Here, the user provides the functions to deal with the request. Unlike other MPI calls, the user needs to provide the

information to be returned in the status. The status argument is provided directly to the callback function where the status needs to be set. Users can directly set the values in 3 of the 5 status values. The count and cancel fields are opaque. To overcome this, these calls are provided:

```
MPI_STATUS_SET_ELEMENTS(status, datatype, count)
```

INOUT	status	status with which to associate count (status)
IN	datatype	datatype associated with count (handle)
IN	count	number of elements to associate with status (integer)

C binding

```
int MPI_Status_set_elements(MPI_Status *status, MPI_Datatype datatype,
                           int count)
```

```
int MPI_Status_set_elements_c(MPI_Status *status, MPI_Datatype datatype,
                              MPI_Count count)
```

Fortran 2008 binding

```
MPI_Status_set_elements(status, datatype, count, ierror)
```

```
TYPE(MPI_Status), INTENT(INOUT) :: status
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: count
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Status_set_elements(status, datatype, count, ierror) !(_c)
```

```
TYPE(MPI_Status), INTENT(INOUT) :: status
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_STATUS_SET_ELEMENTS(STATUS, DATATYPE, COUNT, IERROR)
```

```
INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, COUNT, IERROR
```

This procedure modifies the opaque part of status so calls to [MPI_GET_ELEMENTS](#) will return count. Calls to [MPI_GET_COUNT](#) will return a compatible value.

Rationale. The number of elements is set instead of the count because the former can deal with a non-integer number of datatypes. (*End of rationale.*)

A subsequent call to [MPI_GET_COUNT](#) or [MPI_GET_ELEMENTS](#) must use a **datatype** argument that has the same type signature as the **datatype** argument that was used in the call to [MPI_STATUS_SET_ELEMENTS](#).

Rationale. The requirement of matching type signatures for these calls is similar to the restriction that holds when **count** is set by a receive operation: in that case, calls to [MPI_GET_COUNT](#) and [MPI_GET_ELEMENTS](#) must use a **datatype** with the same signature as the datatype used in the receive call. (*End of rationale.*)

`MPI_STATUS_SET_CANCELLED(status, flag)`

INOUT	status	status with which to associate cancel flag (status)
IN	flag	if true, indicates request was cancelled (logical)

C binding

`int MPI_Status_set_cancelled(MPI_Status *status, int flag)`

Fortran 2008 binding

```
MPI_Status_set_cancelled(status, flag, ierror)
  TYPE(MPI_Status), INTENT(INOUT) :: status
  LOGICAL, INTENT(IN) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_STATUS_SET_CANCELLED(STATUS, FLAG, IERROR)
  INTEGER STATUS(MPI_STATUS_SIZE), IERROR
  LOGICAL FLAG
```

If flag is set to true then a subsequent call to `MPI_TEST_CANCELLED` will also return flag = true, otherwise it will return false.

While the `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR` status values are directly accessible by the user, for convenience in some contexts, users can also modify them via the procedure calls described below. Procedures for querying these fields from a status object are defined in Section 3.2.5.

`MPI_STATUS_SET_SOURCE(status, source)`

INOUT	status	status with which to associate source rank (status)
IN	source	rank to set in the <code>MPI_SOURCE</code> field (integer)

C binding

`int MPI_Status_set_source(MPI_Status *status, int source)`

Fortran 2008 binding

```
MPI_Status_set_source(status, source, ierror)
  TYPE(MPI_Status), INTENT(INOUT) :: status
  INTEGER, INTENT(IN) :: source
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_STATUS_SET_SOURCE(STATUS, SOURCE, IERROR)
  INTEGER STATUS(MPI_STATUS_SIZE), SOURCE, IERROR
```

Set the `MPI_SOURCE` field in the status object to the provided source argument.

1 MPI_STATUS_SET_TAG(status, tag)

2 INOUT status status with which to associate tag (status)

3 IN tag tag to set in the [MPI_TAG](#) field (integer)

5
6 **C binding**

7 int MPI_Status_set_tag(MPI_Status *status, int tag)

8 **Fortran 2008 binding**

9 MPI_Status_set_tag(status, tag, ierror)

10 TYPE(MPI_Status), INTENT(INOUT) :: status

11 INTEGER, INTENT(IN) :: tag

12 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

13 **Fortran binding**

14 MPI_STATUS_SET_TAG(STATUS, TAG, IERROR)

15 INTEGER STATUS(MPI_STATUS_SIZE), TAG, IERROR

16 Set the [MPI_TAG](#) field in the status object to the provided tag argument.

19 MPI_STATUS_SET_ERROR(status, err)

20 INOUT status status with which to associate error (status)

21 IN err error to set in the [MPI_ERROR](#) field (integer)

24 **C binding**

25 int MPI_Status_set_error(MPI_Status *status, int err)

26 **Fortran 2008 binding**

27 MPI_Status_set_error(status, err, ierror)

28 TYPE(MPI_Status), INTENT(INOUT) :: status

29 INTEGER, INTENT(IN) :: err

30 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

31 **Fortran binding**

32 MPI_STATUS_SET_ERROR(STATUS, ERR, IERROR)

33 INTEGER STATUS(MPI_STATUS_SIZE), ERR, IERROR

34 Set the [MPI_ERROR](#) field in the status object to the provided err error code.

37 *Rationale.* These functions exist for convenience when using MPI from languages other than C and Fortran, where having a function in the MPI library with a known API reduces the need for utility code written in C. (*End of rationale.*)

40 *Advice to users.* Users are advised not to reuse the status fields for values other than those for which they were intended. Doing so may lead to unexpected results when using the status object. For example, calling [MPI_GET_ELEMENTS](#) may cause an error if the value is out of range or it may be impossible to detect such an error. The `extra_state` argument provided with a generalized request can be used to return information that does not logically belong in status. Furthermore, modifying the values in a status set internally by MPI, e.g., [MPI_RECV](#), may lead to unpredictable results and is strongly discouraged. (*End of advice to users.*)

Chapter 15

I/O

15.1 Introduction

POSIX provides a model of a widely portable file system, but the portability and optimization needed for parallel I/O cannot be achieved with the POSIX interface.

The significant optimizations required for efficiency (e.g., grouping [55], collective buffering [9, 17, 56, 60, 67], and disk-directed I/O [50]) can only be implemented if the parallel I/O system provides a high-level interface supporting partitioning of file data among processes and a collective interface supporting complete transfers of global data structures between process memories and files. In addition, further efficiencies can be gained via support for asynchronous I/O, strided accesses, and control over physical file layout on storage devices (disks). The I/O environment described in this chapter provides these facilities.

Instead of defining I/O access modes to express the common patterns for accessing a shared file (broadcast, reduction, scatter, gather), we chose another approach in which data partitioning is expressed using derived datatypes. Compared to a limited set of predefined access patterns, this approach has the advantage of added flexibility and expressiveness.

15.1.1 Definitions

MPI file: An MPI file is an ordered collection of typed data items. MPI supports random or sequential access to any integral set of these items. A file is opened collectively by a group of processes. All collective I/O calls on a file are collective over this group.

displacement: A file *displacement* is an absolute byte position relative to the beginning of a file. The displacement defines the location where a *view* begins. Note that a “file displacement” is distinct from a “typemap displacement.”

etype: An *etype* (*elementary datatype*) is the unit of data access and positioning. It can be any MPI predefined or derived datatype. Derived etypes can be constructed using any of the MPI datatype constructor routines, provided all resulting typemap displacements are nonnegative and monotonically increasing. Data access is performed in etype units, reading or writing whole data items of type etype. Offsets are expressed as a count of etypes; file pointers point to the beginning of etypes. Depending on context, the term “etype” is used to describe one of three aspects of an elementary datatype: a particular MPI type, a data item of that type, or the extent of that type.

filetype: A *filetype* is the basis for partitioning a file among processes and defines a template for accessing the file. A filetype is either a single etype or a derived MPI datatype constructed from multiple instances of the same etype. In addition, the extent of any hole in the filetype must be a multiple of the etype’s extent. The displacements in the

typemap of the filetype are not required to be distinct, but they must be nonnegative and monotonically nondecreasing.

view: A *view* defines the current set of data visible and accessible from an open file as an ordered set of etypes. Each process has its own view of the file, defined by three quantities: a displacement, an etype, and a filetype. The pattern described by a filetype is repeated, beginning at the displacement, to define the view. The pattern of repetition is defined to be the same pattern that `MPI_TYPE_CONTIGUOUS` would produce if it were passed the filetype and an arbitrarily large count. Figure 15.1 shows how the tiling works; note that the filetype in this example must have explicit lower and upper bounds set in order for the initial and final holes to be repeated in the view. Views can be changed by the user during program execution. The default view is a linear byte stream (displacement is zero, etype and filetype equal to `MPI_BYTE`).

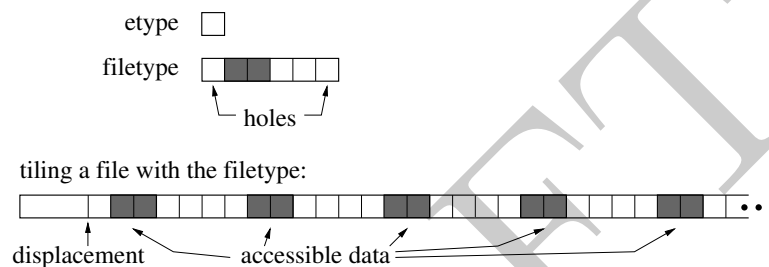


Figure 15.1: Etypes and filetypes

A group of processes can use complementary views to achieve a global data distribution, such as a scatter/gather pattern (see Figure 15.2).



Figure 15.2: Partitioning a file among parallel processes

offset: An *offset* is a position in a file relative to the current view, expressed as a count of etypes. Holes in the view's filetype are skipped when calculating this position. Offset 0 is the location of the first etype visible in the view (after skipping the displacement and any initial holes in the view). For example, an offset of 2 for Process 1 in Figure 15.2 is the position of the eighth etype in the file after the displacement. An “explicit offset” is an offset that is used as an argument in explicit data access routines.

file size and end of file: The *size* of an MPI file is measured in bytes from the beginning of the file. A newly created file has a size of zero bytes. Using the size as an absolute displacement gives the position of the byte immediately following the last byte in the

file. For any given view, the *end of file* is the offset of the first etype accessible in the current view starting after the last byte in the file.

file pointer: A *file pointer* is an implicit offset maintained by MPI. “Individual file pointers” are file pointers that are local to each process that opened the file. A “shared file pointer” is a file pointer that is shared by the group of processes that opened the file.

file handle: A *file handle* is an opaque object created by `MPI_FILE_OPEN` and freed by `MPI_FILE_CLOSE`. All operations on an open file reference the file through the file handle.

15.2 File Manipulation

15.2.1 Opening a File

`MPI_FILE_OPEN(comm, filename, amode, info, fh)`

IN	comm	communicator (handle)
IN	filename	name of file to open (string)
IN	amode	file access mode (integer)
IN	info	info object (handle)
OUT	fh	new file handle (handle)

C binding

```
int MPI_File_open(MPI_Comm comm, const char *filename, int amode,
                  MPI_Info info, MPI_File *fh)
```

Fortran 2008 binding

```
MPI_File_open(comm, filename, amode, info, fh, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  CHARACTER(LEN=*), INTENT(IN) :: filename
  INTEGER, INTENT(IN) :: amode
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_File), INTENT(OUT) :: fh
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_OPEN(COMM, FILENAME, AMODE, INFO, FH, IERROR)
  INTEGER COMM, AMODE, INFO, FH, IERROR
  CHARACTER*(*) FILENAME
```

`MPI_FILE_OPEN` opens the file identified by the file name `filename` on all processes in the `comm` communicator group. `MPI_FILE_OPEN` is a collective routine: all processes must provide the same value for `amode`, and all processes must provide `filenames` that reference the same file. (Values for `info` may vary.) `comm` must be an intra-communicator; it is erroneous to pass an inter-communicator to `MPI_FILE_OPEN`. Errors in `MPI_FILE_OPEN` are raised using the default file error handler (see Section 15.7). When using the World Model (Section 11.1), a process can open a file independently of other processes by using

the `MPI_COMM_SELF` communicator. Applications using the Sessions Model (Section 11.3) can achieve the same result using communicators created from the `"mpi://SELF"` process set. The file handle returned, `fh`, can subsequently be used to access the file until the file is closed using `MPI_FILE_CLOSE`. Before calling `MPI_FINALIZE`, the user is required to close (via `MPI_FILE_CLOSE`) all files that were opened with `MPI_FILE_OPEN`. Note that the communicator `comm` is unaffected by calls to `MPI_FILE_OPEN` and continues to be usable in all MPI routines (e.g., `MPI_SEND`). Furthermore, the use of `comm` will not interfere with I/O behavior.

The format for specifying the file name in the `filename` argument is implementation dependent and must be documented by the implementation.

Advice to implementors. An implementation may require that `filename` include a string or strings specifying additional information about the file. Examples include the type of filesystem (e.g., a prefix of `ufs:`), a remote hostname (e.g., a prefix of `machine.univ.edu:`), or a file password (e.g., a suffix of `/PASSWORD=SECRET`). (*End of advice to implementors.*)

Advice to users. On some implementations of MPI, the file namespace may not be identical from all processes of all applications. For example, `"/tmp/foo"` may denote different files on different processes, or a single file may have many names, dependent on process location. The user is responsible for ensuring that a single file is referenced by the `filename` argument, as it may be impossible for an implementation to detect this type of namespace error. (*End of advice to users.*)

Initially, all processes view the file as a linear byte stream, and each process views data in its own native representation (no data representation conversion is performed). (POSIX files are linear byte streams in the native representation.) The file view can be changed via the `MPI_FILE_SET_VIEW` routine.

The following access modes are supported (specified in `amode`, a bit vector created by OR with one or more of the following integer constants):

<code>MPI_MODE_RDONLY</code>	read only
<code>MPI_MODE_RDWR</code>	reading and writing
<code>MPI_MODE_WRONLY</code>	write only
<code>MPI_MODE_CREATE</code>	create the file if it does not exist
<code>MPI_MODE_EXCL</code>	error if creating file that already exists
<code>MPI_MODE_DELETE_ON_CLOSE</code>	delete file on close
<code>MPI_MODE_UNIQUE_OPEN</code>	file will not be opened concurrently elsewhere
<code>MPI_MODE_SEQUENTIAL</code>	file will only be accessed sequentially
<code>MPI_MODE_APPEND</code>	set initial position of all file pointers to end of file

Advice to users. C users can use bit vector OR (`|`) to combine these constants; Fortran 90 users can use the bit vector `IOR` intrinsic. Fortran 77 users can use (nonportably) bit vector `IOR` on systems that support it. Alternatively, Fortran users can portably use integer addition to OR the constants (each constant should appear at most once in the addition.). (*End of advice to users.*)

Advice to implementors. The values of these constants must be defined such that the bitwise OR and the sum of any distinct set of these constants is equivalent. (*End of advice to implementors.*)

Fortran 2008 binding

```

MPI_File_close(fh, ierror)
    TYPE(MPI_File), INTENT(INOUT) :: fh
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_CLOSE(FH, IERROR)
    INTEGER FH, IERROR

```

`MPI_FILE_CLOSE` first synchronizes file state (equivalent to performing an `MPI_FILE_SYNC`), then closes the file associated with `fh`. The file is deleted if it was opened with access mode `MPI_MODE_DELETE_ON_CLOSE` (equivalent to performing an `MPI_FILE_DELETE`). `MPI_FILE_CLOSE` is a collective routine over the group associated with the file.

Advice to users. If the file is deleted on close, and there are other processes currently accessing the file, the status of the file and the behavior of future accesses by these processes are implementation dependent. (*End of advice to users.*)

The user is responsible for ensuring that all outstanding nonblocking requests and split collective operations associated with `fh` made by a process have completed before that process calls `MPI_FILE_CLOSE`.

The `MPI_FILE_CLOSE` routine deallocates the file handle object and sets `fh` to `MPI_FILE_NULL`.

15.2.3 Deleting a File

```

MPI_FILE_DELETE(filename, info)
    IN      filename      name of file to delete (string)
    IN      info           info object (handle)

```

C binding

```

int MPI_File_delete(const char *filename, MPI_Info info)

```

Fortran 2008 binding

```

MPI_File_delete(filename, info, ierror)
    CHARACTER(LEN=*) , INTENT(IN) :: filename
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_DELETE(FILENAME, INFO, IERROR)
    CHARACTER*(*) FILENAME
    INTEGER INFO, IERROR

```

`MPI_FILE_DELETE` deletes the file identified by the file name `filename`. If the file does not exist, `MPI_FILE_DELETE` raises an error of class `MPI_ERR_NO_SUCH_FILE`.

The `info` argument can be used to provide information regarding file system specifics (see Section 15.2.8). The constant `MPI_INFO_NULL` refers to the null info, and can be used when no info needs to be specified.

If a process currently has the file open, the behavior of any access to the file (as well as the behavior of any outstanding accesses) is implementation dependent. In addition, whether an open file is deleted or not is also implementation dependent. If the file is not deleted, an error in the class `MPI_ERR_FILE_IN_USE` or `MPI_ERR_ACCESS` will be raised. Errors are raised using the default file error handler (see Section 15.7).

15.2.4 Resizing a File

`MPI_FILE_SET_SIZE(fh, size)`

INOUT	<code>fh</code>	file handle (handle)
IN	<code>size</code>	size to truncate or expand file (integer)

C binding

```
int MPI_File_set_size(MPI_File fh, MPI_Offset size)
```

Fortran 2008 binding

```
MPI_File_set_size(fh, size, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SET_SIZE(FH, SIZE, IERROR)
  INTEGER FH, IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) SIZE
```

`MPI_FILE_SET_SIZE` resizes the file associated with the file handle `fh`. `size` is measured in bytes from the beginning of the file. `MPI_FILE_SET_SIZE` is collective over the group associated with the file; all processes in the group must pass identical values for `size`.

If `size` is smaller than the current file size, the file is truncated at the position defined by `size`. The implementation is free to deallocate file blocks located beyond this position.

If `size` is larger than the current file size, the file size becomes `size`. Regions of the file that have been previously written are unaffected. The values of data in the new regions in the file (those locations with displacements between old file size and `size`) are undefined. It is implementation dependent whether the `MPI_FILE_SET_SIZE` routine allocates file space—use `MPI_FILE_PREALLOCATE` to force file space to be reserved.

`MPI_FILE_SET_SIZE` does not affect the individual file pointers or the shared file pointer. If `MPI_MODE_SEQUENTIAL` mode was specified when the file was opened, it is erroneous to call this routine.

Advice to users. It is possible for the file pointers to point beyond the end of file after a `MPI_FILE_SET_SIZE` operation truncates a file. This is valid, and equivalent to seeking beyond the current end of file. (*End of advice to users.*)

All nonblocking requests and split collective operations on `fh` must be completed before calling `MPI_FILE_SET_SIZE`. Otherwise, calling `MPI_FILE_SET_SIZE` is erroneous. As far as consistency semantics are concerned, `MPI_FILE_SET_SIZE` is a write operation that conflicts with operations that access bytes at displacements between the old and new file sizes (see Section 15.6.1).

15.2.5 Preallocating Space for a File

`MPI_FILE_PREALLOCATE(fh, size)`

INOUT	<code>fh</code>	file handle (handle)
IN	<code>size</code>	size to preallocate file (integer)

C binding

```
int MPI_File_preallocate(MPI_File fh, MPI_Offset size)
```

Fortran 2008 binding

```
MPI_File_preallocate(fh, size, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_PREALLOCATE(FH, SIZE, IERROR)
    INTEGER FH, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) SIZE
```

`MPI_FILE_PREALLOCATE` ensures that storage space is allocated for the first `size` bytes of the file associated with `fh`. `MPI_FILE_PREALLOCATE` is collective over the group associated with the file; all processes in the group must pass identical values for `size`. Regions of the file that have previously been written are unaffected. For newly allocated regions of the file, `MPI_FILE_PREALLOCATE` has the same effect as writing undefined data. If `size` is larger than the current file size, the file size increases to `size`. If `size` is less than or equal to the current file size, the file size is unchanged.

The treatment of file pointers, nonblocking accesses, and file consistency is the same as with `MPI_FILE_SET_SIZE`. If `MPI_MODE_SEQUENTIAL` mode was specified when the file was opened, it is erroneous to call this routine.

Advice to users. In some implementations, file preallocation may be time-consuming.
(*End of advice to users.*)

15.2.6 Querying the Size of a File

`MPI_FILE_GET_SIZE(fh, size)`

IN	<code>fh</code>	file handle (handle)
OUT	<code>size</code>	size of the file in bytes (integer)

C binding

```
int MPI_File_get_size(MPI_File fh, MPI_Offset *size)
```

Fortran 2008 binding

```
MPI_File_get_size(fh, size, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_SIZE(FH, SIZE, IERROR)
  INTEGER FH, IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) SIZE
```

[MPI_FILE_GET_SIZE](#) returns, in `size`, the current size in bytes of the file associated with the file handle `fh`. As far as consistency semantics are concerned, [MPI_FILE_GET_SIZE](#) is a data access operation (see Section 15.6.1).

15.2.7 Querying File Parameters

```
MPI_FILE_GET_GROUP(fh, group)
```

IN	fh	file handle (handle)
OUT	group	group that opened the file (handle)

C binding

```
int MPI_File_get_group(MPI_File fh, MPI_Group *group)
```

Fortran 2008 binding

```
MPI_File_get_group(fh, group, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(MPI_Group), INTENT(OUT) :: group
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_GROUP(FH, GROUP, IERROR)
  INTEGER FH, GROUP, IERROR
```

[MPI_FILE_GET_GROUP](#) returns a duplicate of the group of the communicator used to open the file associated with `fh`. The group is returned in `group`. The user is responsible for freeing `group`.

```
MPI_FILE_GET_AMODE(fh, amode)
```

IN	fh	file handle (handle)
OUT	amode	file access mode used to open the file (integer)

C binding

```
int MPI_File_get_amode(MPI_File fh, int *amode)
```

Fortran 2008 binding

```

MPI_File_get_amode(fh, amode, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER, INTENT(OUT) :: amode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_GET_AMODE(FH, AMODE, IERROR)
  INTEGER FH, AMODE, IERROR

```

MPI_FILE_GET_AMODE returns, in `amode`, the access mode of the file associated with `fh`.

Example 15.1. In Fortran 77, decoding an `amode` bit vector will require a routine such as the following:

```

SUBROUTINE BIT_QUERY(TEST_BIT, MAX_BIT, AMODE, BIT_FOUND)
!
!  TEST IF THE INPUT TEST_BIT IS SET IN THE INPUT AMODE
!  IF SET, RETURN 1 IN BIT_FOUND, 0 OTHERWISE
!
  INTEGER TEST_BIT, AMODE, BIT_FOUND, CP_AMODE, HIFOUND
  INTEGER L, LBIT, MATCHER, MAX_BIT
  BIT_FOUND = 0
  CP_AMODE = AMODE
100 CONTINUE
  LBIT = 0
  HIFOUND = 0
  DO L = MAX_BIT, 0, -1
    MATCHER = 2**L
    IF (CP_AMODE .GE. MATCHER .AND. HIFOUND .EQ. 0) THEN
      HIFOUND = 1
      LBIT = MATCHER
      CP_AMODE = CP_AMODE - MATCHER
    END IF
  END DO
  IF (HIFOUND .EQ. 1 .AND. LBIT .EQ. TEST_BIT) BIT_FOUND = 1
  IF (BIT_FOUND .EQ. 0 .AND. HIFOUND .EQ. 1 .AND. &
    CP_AMODE .GT. 0) GO TO 100
END

```

This routine could be called successively to decode `amode`, one bit at a time. For example, the following code fragment would check for **MPI_MODE_RDONLY**.

```

CALL BIT_QUERY(MPI_MODE_RDONLY, 30, AMODE, BIT_FOUND)
IF (BIT_FOUND .EQ. 1) THEN
  PRINT *, ' FOUND READ-ONLY BIT IN AMODE=', AMODE
ELSE
  PRINT *, ' READ-ONLY BIT NOT FOUND IN AMODE=', AMODE
END IF

```

15.2.8 File Info

Hints specified via `info` (see Chapter 10) allow a user to provide information, such as file access patterns and file system specifics to direct optimization. Providing hints may enable an implementation to deliver increased I/O performance or minimize the use of system resources. As described in Chapter 10, an implementation is free to ignore all hints; however, applications must comply with any info hints they provide that are used by the MPI implementation (i.e., are returned by a call to `MPI_FILE_GET_INFO`) and that place a restriction on the behavior of the application. Hints are specified on a per file basis, in `MPI_FILE_OPEN`, `MPI_FILE_DELETE`, `MPI_FILE_SET_VIEW`, and `MPI_FILE_SET_INFO`, via the opaque info object. When an info object that specifies a subset of valid hints is passed to `MPI_FILE_SET_VIEW` or `MPI_FILE_SET_INFO`, there will be no effect on previously set or defaulted hints that info does not specify.

Advice to implementors. It may happen that a program is coded with hints for one system, and later executes on another system that does not support these hints. In general, unsupported hints should simply be ignored.

However, for each hint used by a specific implementation, a default value must be provided when the user does not specify a value for this hint. (*End of advice to implementors.*)

`MPI_FILE_SET_INFO(fh, info)`

INOUT	fh	file handle (handle)
IN	info	info object (handle)

C binding

```
int MPI_File_set_info(MPI_File fh, MPI_Info info)
```

Fortran 2008 binding

```
MPI_File_set_info(fh, info, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SET_INFO(FH, INFO, IERROR)
  INTEGER FH, INFO, IERROR
```

`MPI_FILE_SET_INFO` updates the hints of the file associated with `fh` using the hints provided in `info`. This operation has no effect on previously set or defaulted hints that are not specified by `info`. It also has no effect on previously set or defaulted hints that are specified by `info`, but are ignored by the MPI implementation in this call to `MPI_FILE_SET_INFO`. `MPI_FILE_SET_INFO` is a collective routine. The info object may be different on each process, but any info entries that an implementation requires to be the same on all processes must appear with the same value in each process's info object.

Advice to users. Many info items that an implementation can use when it creates or opens a file cannot easily be changed once the file has been created or opened. Thus,

an implementation may ignore hints issued in this call that it would have accepted in an open call. An implementation may also be unable to update certain info hints in a call to `MPI_FILE_SET_VIEW` or `MPI_FILE_SET_INFO`. `MPI_FILE_GET_INFO` can be used to determine whether info changes were ignored by the implementation. (*End of advice to users.*)

`MPI_FILE_GET_INFO(fh, info_used)`

IN	fh	file handle (handle)
OUT	info_used	new info object (handle)

C binding

`int MPI_File_get_info(MPI_File fh, MPI_Info *info_used)`

Fortran 2008 binding

```
MPI_File_get_info(fh, info_used, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(MPI_Info), INTENT(OUT) :: info_used
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_INFO(FH, INFO_USED, IERROR)
    INTEGER FH, INFO_USED, IERROR
```

`MPI_FILE_GET_INFO` returns a new info object containing the hints of the file associated with `fh`. The current setting of all hints related to this file is returned in `info_used`. An MPI implementation is required to return all hints that are supported by the implementation and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no (key,value) pairs. The user is responsible for freeing `info_used` via `MPI_INFO_FREE`.

Reserved File Hints

Some potentially useful hints (info key values) are outlined below. The following key values are reserved. An implementation is not required to interpret these key values, but if it does interpret the key value, it must provide the functionality described. (For more details on “info,” see Chapter 10.)

These hints mainly affect access patterns and the layout of data on parallel I/O devices. For each hint name introduced, we describe the purpose of the hint, and the type of the hint value. The “[**SAME**]” annotation specifies that the hint values provided by all participating processes must be identical; otherwise the program is erroneous. In addition, some hints are context dependent, and are only used by an implementation at specific times (e.g., “file_perm” is only useful during file creation).

“access_style” (comma separated list of strings): This hint specifies the manner in which the file will be accessed until the file is closed or until the “access_style” key value is altered. The hint value is a comma separated list of the following:

"read_once", "write_once", "read_mostly", "write_mostly", "sequential", "reverse_sequential", and "random".

"collective_buffering" (boolean) [SAME]: This hint specifies whether the application is expected to benefit from collective buffering. Collective buffering is an optimization performed on collective accesses. Accesses to the file are performed on behalf of all processes in the group by a number of target nodes. These target nodes coalesce small requests into large disk accesses. Valid values for this key are "true" and "false". Collective buffering parameters are further directed via additional hints: "cb_block_size", "cb_buffer_size", and "cb_nodes".

"cb_block_size" (integer) [SAME]: This hint specifies the block size to be used for collective buffering file access. **Target nodes** access data in chunks of this size. The chunks are distributed among target nodes in a round-robin (cyclic) pattern.

"cb_buffer_size" (integer) [SAME]: This hint specifies the total buffer space that can be used for collective buffering on each target node, usually a multiple of "cb_block_size".

"cb_nodes" (integer) [SAME]: This hint specifies the number of target nodes to be used for collective buffering.

"chunked" (comma separated list of integers) [SAME]: This hint specifies that the file consists of a multidimensional array that is often accessed by subarrays. The value for this hint is a comma separated list of array dimensions, starting from the most significant one (for an array stored in row-major order, as in C, the most significant dimension is the first one; for an array stored in column-major order, as in Fortran, the most significant dimension is the last one, and array dimensions should be reversed).

"chunked_item" (comma separated list of integers) [SAME]: This hint specifies the size of each array entry, in bytes.

"chunked_size" (comma separated list of integers) [SAME]: This hint specifies the dimensions of the subarrays. This is a comma separated list of array dimensions, starting from the most significant one.

"filename" (string): This hint specifies the file name used when the file was opened. If the implementation is capable of returning the file name of an open file, it will be returned using this key by `MPI_FILE_GET_INFO`. This key is ignored when passed to `MPI_FILE_OPEN`, `MPI_FILE_SET_VIEW`, `MPI_FILE_SET_INFO`, and `MPI_FILE_DELETE`.

"file_perm" (string) [SAME]: This hint specifies the file permissions to use for file creation. Setting this hint is only useful when passed to `MPI_FILE_OPEN` with an `amode` that includes `MPI_MODE_CREATE`. The set of valid values for this key is implementation dependent.

"io_node_list" (comma separated list of strings) [SAME]: This hint specifies the list of I/O devices that should be used to store the file. This hint is most relevant when the file is created.

"nb_proc" (integer) [SAME]: This hint specifies the number of parallel processes that will typically be assigned to access this file. This hint is most relevant when the file is created.

"num_io_nodes" (integer) [SAME]: This hint specifies the number of I/O devices in the system. This hint is most relevant when the file is created.

"striping_factor" (integer) [SAME]: This hint specifies the number of I/O devices that the file should be striped across, and is relevant only when the file is created.

"striping_unit" (integer) [SAME]: This hint specifies the suggested striping unit to be used for this file. The striping unit is the amount of consecutive data assigned to one I/O device before progressing to the next device, when striping across a number of devices. It is expressed in bytes. This hint is relevant only when the file is created.

"mpi_assert_memory_alloc_kinds" (string, not set by default): If set, the implementation may assume that the memory for all data buffers passed to MPI operations performed by the calling MPI process on the given file will use only the memory allocation kinds listed in the value string. See Section 11.4.3.

15.3 File Views

MPI_FILE_SET_VIEW(fh, disp, etype, filetype, datarep, info)

INOUT	fh	file handle (handle)
IN	disp	displacement (integer)
IN	etype	elementary datatype (handle)
IN	filetype	filetype (handle)
IN	datarep	data representation (string)
IN	info	info object (handle)

C binding

```
int MPI_File_set_view(MPI_File fh, MPI_Offset disp, MPI_Datatype etype,
    MPI_Datatype filetype, const char *datarep, MPI_Info info)
```

Fortran 2008 binding

```
MPI_File_set_view(fh, disp, etype, filetype, datarep, info, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: disp
    TYPE(MPI_Datatype), INTENT(IN) :: etype, filetype
    CHARACTER(LEN=*), INTENT(IN) :: datarep
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SET_VIEW(FH, DISP, ETYPE, FILETYPE, DATAREP, INFO, IERROR)
    INTEGER FH, ETYPE, FILETYPE, INFO, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) DISP
    CHARACTER*(*) DATAREP
```

The **MPI_FILE_SET_VIEW** routine changes the process's view of the data in the file. The start of the view is set to **disp**; the type of data is set to **etype**; the distribution of data

to processes is set to `filetype`; and the representation of data in the file is set to `datarep`. In addition, `MPI_FILE_SET_VIEW` resets the individual file pointers and the shared file pointer to zero. `MPI_FILE_SET_VIEW` is collective; the values for `datarep` and the extent of `etype` in the file data representation must be identical on all processes in the group; values for `disp`, `filetype`, and `info` may vary. The datatypes passed in `etype` and `filetype` must be committed.

The `etype` always specifies the data layout in the file. If `etype` is a portable datatype (see Section 2.4), the extent of `etype` is computed by scaling any displacements in the datatype to match the file data representation. If `etype` is not a portable datatype, no scaling is done when computing the extent of `etype`. The user must be careful when using nonportable `etypes` in heterogeneous environments; see Section 15.5.1 for further details.

If `MPI_MODE_SEQUENTIAL` mode was specified when the file was opened, the special displacement `MPI_DISPLACEMENT_CURRENT` must be passed in `disp`. This sets the displacement to the current position of the shared file pointer. `MPI_DISPLACEMENT_CURRENT` is invalid unless the `amode` for the file has `MPI_MODE_SEQUENTIAL` set.

Rationale. For some sequential files, such as those corresponding to magnetic tapes or streaming network connections, the *displacement* may not be meaningful. `MPI_DISPLACEMENT_CURRENT` allows the view to be changed for these types of files. (*End of rationale.*)

Advice to implementors. It is expected that a call to `MPI_FILE_SET_VIEW` will immediately follow `MPI_FILE_OPEN` in numerous instances. A high-quality implementation will ensure that this behavior is efficient. (*End of advice to implementors.*)

The `disp` displacement argument specifies the position (absolute offset in bytes from the beginning of the file) where the view begins.

Advice to users. `disp` can be used to skip headers or when the file includes a sequence of data segments that are to be accessed in different patterns (see Figure 15.3). Separate views, each using a different displacement and `filetype`, can be used to access each segment.

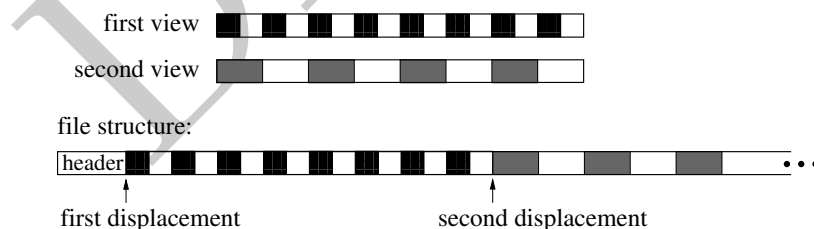


Figure 15.3: Displacements

(*End of advice to users.*)

An *etype* (*elementary* datatype) is the unit of data access and positioning. It can be any MPI predefined or derived datatype. Derived `etypes` can be constructed by using any of the MPI datatype constructor routines, provided all resulting typemap displacements are nonnegative and monotonically increasing. Data access is performed in `etype` units, reading or writing whole data items of type `etype`. Offsets are expressed as a count of `etypes`; file pointers point to the beginning of `etypes`.

Advice to users. In order to ensure interoperability in a heterogeneous environment, additional restrictions must be observed when constructing the **etype** (see Section 15.5).
(End of advice to users.)

A filetype is either a single etype or a derived MPI datatype constructed from multiple instances of the same etype. In addition, the extent of any hole in the filetype must be a multiple of the etype’s extent. These displacements are not required to be distinct, but they cannot be negative, and they must be monotonically nondecreasing.

If the file is opened for writing, neither the etype nor the filetype is permitted to contain overlapping regions. This restriction is equivalent to the “datatype used in a receive cannot specify overlapping regions” restriction for communication. Note that filetypes from different processes may still overlap each other.

If a filetype has holes in it, then the data in the holes is inaccessible to the calling process. However, the **disp**, **etype**, and **filetype** arguments can be changed via future calls to **MPI_FILE_SET_VIEW** to access a different part of the file.

It is erroneous to use absolute addresses in the construction of the etype and filetype.

The **info** argument is used to provide information regarding file access patterns and file system specifics to direct optimization (see Section 15.2.8). The constant **MPI_INFO_NULL** refers to the null info and can be used when no info needs to be specified.

The **datarep** argument is a string that specifies the representation of data in the file. See the file interoperability section (Section 15.5) for details and a discussion of valid values.

The user is responsible for ensuring that all nonblocking requests and split collective operations on **fh** have been completed before calling **MPI_FILE_SET_VIEW**—otherwise, the call to **MPI_FILE_SET_VIEW** is erroneous.

MPI_FILE_GET_VIEW(fh, disp, etype, filetype, datarep)

IN	fh	file handle (handle)
OUT	disp	displacement (integer)
OUT	etype	elementary datatype (handle)
OUT	filetype	filetype (handle)
OUT	datarep	data representation (string)

C binding

```
int MPI_File_get_view(MPI_File fh, MPI_Offset *disp, MPI_Datatype *etype,
                     MPI_Datatype *filetype, char *datarep)
```

Fortran 2008 binding

```
MPI_File_get_view(fh, disp, etype, filetype, datarep, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: disp
  TYPE(MPI_Datatype), INTENT(OUT) :: etype, filetype
  CHARACTER(LEN=*), INTENT(OUT) :: datarep
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_VIEW(FH, DISP, ETYPE, FILETYPE, DATAREP, IERROR)
  INTEGER FH, ETYPE, FILETYPE, IERROR
```



```

INTEGER(KIND=MPI_OFFSET_KIND) DISP
CHARACTER*(*) DATAREP

```

`MPI_FILE_GET_VIEW` returns the process's view of the data in the file. The current value of the displacement is returned in `disp`. The `etype` and `filetype` are new datatypes with typemaps equal to the typemaps of the current `etype` and `filetype`, respectively.

The data representation is returned in `datarep`. The user is responsible for ensuring that `datarep` is large enough to hold the returned data representation string. The length of a data representation string is limited to the value of `MPI_MAX_DATAREP_STRING`.

In addition, if a portable datatype was used to set the current view, then the corresponding datatype returned by `MPI_FILE_GET_VIEW` is also a portable datatype. If `etype` or `filetype` are derived datatypes, the user is responsible for freeing them. The `etype` and `filetype` returned are both in a committed state.

15.4 Data Access

15.4.1 Data Access Routines

Data is moved between files and processes by issuing read and write calls. There are three orthogonal aspects to data access: **positioning** (explicit offset vs. implicit file pointer), **synchronism** (blocking vs. nonblocking and split collective), and **coordination** (noncollective vs. collective). The following combinations of these data access routines, including two types of file pointers (individual and shared) are provided in Table 15.1.

POSIX `read()/fread()` and `write()/fwrite()` are blocking, noncollective operations and use individual file pointers. The MPI counterparts are `MPI_FILE_READ` and `MPI_FILE_WRITE`.

Implementations of data access routines may buffer data to improve performance. This does not affect reads, as the data is always available in the user's buffer after a read operation completes. For writes, however, the `MPI_FILE_SYNC` routine provides the only guarantee that data has been transferred to the storage device.

Positioning

MPI provides three types of positioning for data access routines: **explicit offsets**, **individual file pointers**, and **shared file pointers**. The different positioning methods may be mixed within the same program and do not affect each other.

The data access routines that accept explicit offsets contain `_AT` in their name (e.g., `MPI_FILE_WRITE_AT`). Explicit offset operations perform data access at the file position given directly as an argument—no file pointer is used nor updated. Note that this is not equivalent to an atomic seek-and-read or seek-and-write operation, as no “seek” is issued. Operations with explicit offsets are described in Section 15.4.2.

The names of the individual file pointer routines contain no positional qualifier (e.g., `MPI_FILE_WRITE`). Operations with individual file pointers are described in Section 15.4.3. The data access routines that use shared file pointers contain `_SHARED` or `_ORDERED` in their name (e.g., `MPI_FILE_WRITE_SHARED`). Operations with shared file pointers are described in Section 15.4.4.

The main semantic issues with MPI-maintained file pointers are how and when they are updated by I/O operations. In general, each I/O operation leaves the file pointer pointing to the next data item after the last one that is accessed by the operation. In a nonblocking or

Table 15.1: Data access routines

positioning	synchronism	coordination	
		noncollective	collective
explicit offsets	<i>blocking</i>	MPI_FILE_READ_AT MPI_FILE_WRITE_AT	MPI_FILE_READ_AT_ALL MPI_FILE_WRITE_AT_ALL
	<i>nonblocking</i>	MPI_FILE_IREAD_AT MPI_FILE_IWRITE_AT	MPI_FILE_IREAD_AT_ALL MPI_FILE_IWRITE_AT_ALL
	<i>split collective</i>	N/A	MPI_FILE_READ_AT_ALL_BEGIN MPI_FILE_READ_AT_ALL_END MPI_FILE_WRITE_AT_ALL_BEGIN MPI_FILE_WRITE_AT_ALL_END
individual file pointers	<i>blocking</i>	MPI_FILE_READ MPI_FILE_WRITE	MPI_FILE_READ_ALL MPI_FILE_WRITE_ALL
	<i>nonblocking</i>	MPI_FILE_IREAD MPI_FILE_IWRITE	MPI_FILE_IREAD_ALL MPI_FILE_IWRITE_ALL
	<i>split collective</i>	N/A	MPI_FILE_READ_ALL_BEGIN MPI_FILE_READ_ALL_END MPI_FILE_WRITE_ALL_BEGIN MPI_FILE_WRITE_ALL_END
shared file pointer	<i>blocking</i>	MPI_FILE_READ_SHARED MPI_FILE_WRITE_SHARED	MPI_FILE_READ_ORDERED MPI_FILE_WRITE_ORDERED
	<i>nonblocking</i>	MPI_FILE_IREAD_SHARED MPI_FILE_IWRITE_SHARED	N/A
	<i>split collective</i>	N/A	MPI_FILE_READ_ORDERED_BEGIN MPI_FILE_READ_ORDERED_END MPI_FILE_WRITE_ORDERED_BEGIN MPI_FILE_WRITE_ORDERED_END

split collective operation, the pointer is updated by the call that initiates the I/O, possibly before the access completes.

More formally,

$$new_file_offset = old_file_offset + \frac{elements(datatype)}{elements(etype)} \times count$$

where *count* is the number of *datatype* items to be accessed, *elements(X)* is the number of predefined datatypes in the typemap of *X*, and *old_file_offset* is the value of the implicit offset before the call. The file position, *new_file_offset*, is in terms of a count of etypes relative to the current view.

Synchronism

MPI supports blocking and nonblocking I/O routines.

A *blocking* I/O call will not return until the I/O request is completed.

A *nonblocking* I/O call initiates an I/O operation, but does not wait for it to complete. Given suitable hardware, this allows the transfer of data out of and into the user's buffer to proceed concurrently with computation. A separate *request complete* call ([MPI_WAIT](#), [MPI_TEST](#), or any of their variants) is needed to complete the I/O request, i.e., to confirm that the data has been read or written and that it is safe for the user to reuse the buffer. The nonblocking versions of the routines are named `MPI_FILE_IXXX`, where the *I* stands for immediate.

It is erroneous to access the local buffer of a nonblocking data access operation, or to use that buffer as the source or target of other communications, between the initiation and completion of the operation.

The split collective routines support a restricted form of “nonblocking” operations for collective data access (see Section 15.4.5).

Coordination

Every noncollective data access routine `MPI_FILE_XXX` has a collective counterpart. For most routines, this counterpart is `MPI_FILE_XXX_ALL` or a pair of `MPI_FILE_XXX_BEGIN` and `MPI_FILE_XXX_END`. The counterparts to the `MPI_FILE_XXX_SHARED` routines are `MPI_FILE_XXX_ORDERED`.

The completion of a noncollective call only depends on the activity of the calling process. However, the completion of a collective call (which must be called by all members of the process group) may depend on the activity of the other processes participating in the collective call. See Section 15.6.4 for rules on semantics of collective calls.

Collective operations may perform much better than their noncollective counterparts, as global data accesses have significant potential for automatic optimization.

Data Access Conventions

Data is moved between files and processes by calling read and write routines. Read routines move data from a file into memory. Write routines move data from memory into a file. The file is designated by a file handle, `fh`. The location of the file data is specified by an offset into the current view. The data in memory is specified by a triple: `buf`, `count`, and `datatype`. Upon completion, the amount of data accessed by the calling process is returned in a `status`.

An offset designates the starting position in the file for an access. The offset is always in `etype` units relative to the current view. Explicit offset routines pass `offset` as an argument (negative values are erroneous). The file pointer routines use implicit offsets maintained by MPI.

A data access routine attempts to transfer (read or write) `count` data items of type `datatype` between the user’s buffer `buf` and the file. The `datatype` passed to the routine must be a committed datatype. The layout of data in memory corresponding to `buf`, `count`, `datatype` is interpreted the same way as in MPI communication functions; see Section 3.2.2 and Section 5.1.11. The data is accessed from those parts of the file specified by the current view (Section 15.3). The type signature of `datatype` must match the type signature of some number of contiguous copies of the `etype` of the current view. As in a receive, it is erroneous to specify a `datatype` for reading that contains overlapping regions (areas of memory that would be stored into more than once).

The nonblocking data access routines indicate that MPI can start a data access and associate a request handle, `request`, with the I/O operation. Nonblocking operations are completed via `MPI_TEST`, `MPI_WAIT`, or any of their variants.

Data access operations, when completed, return the amount of data accessed in `status`.

Advice to users. To prevent problems with the argument copying and register optimization done by Fortran compilers, please note the hints in Sections 21.1.10–21.1.20. (*End of advice to users.*)

For blocking routines, `status` is returned directly. For nonblocking routines and split collective routines, `status` is returned when the operation is completed. The number of `datatype` entries and predefined elements accessed by the calling process can be extracted from `status` by using `MPI_GET_COUNT` and `MPI_GET_ELEMENTS`, respectively. The interpretation of the `MPI_ERROR` field is the same as for other operations—normally undefined, but meaningful if an MPI routine returns `MPI_ERR_IN_STATUS`. The user can pass (in C and Fortran) `MPI_STATUS_IGNORE` in the `status` argument if the return value of this argument is not needed. The `status` can be passed to `MPI_TEST_CANCELLED` to determine if the operation was cancelled. All other fields of `status` are undefined.

When reading, a program can detect the end of the file by noting that the amount of data read is less than the amount requested. Writing past the end of the file increases the file size. The amount of data accessed will be the amount requested, unless an error is raised (or a read reaches the end of the file).

15.4.2 Data Access with Explicit Offsets

If `MPI_MODE_SEQUENTIAL` mode was specified when the file was opened, it is erroneous to call the routines in this section.

`MPI_FILE_READ_AT(fh, offset, buf, count, datatype, status)`

IN	fh	file handle (handle)
IN	offset	file offset (integer)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```
int MPI_File_read_at(MPI_File fh, MPI_Offset offset, void *buf, int count,
                    MPI_Datatype datatype, MPI_Status *status)
```

```
int MPI_File_read_at_c(MPI_File fh, MPI_Offset offset, void *buf,
                    MPI_Count count, MPI_Datatype datatype, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_File_read_at(fh, offset, buf, count, datatype, status, ierror)
```

```
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
TYPE(*), DIMENSION(..) :: buf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_File_read_at(fh, offset, buf, count, datatype, status, ierror) !(_c)
```

```
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
```

```

TYPE(*), DIMENSION(..) :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_AT(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
  INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
  <type> BUF(*)

```

MPI_FILE_READ_AT reads a file beginning at the position specified by **offset**.

MPI_FILE_READ_AT_ALL(fh, offset, buf, count, datatype, status)

IN	fh	file handle (handle)
IN	offset	file offset (integer)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```

int MPI_File_read_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_File_read_at_all_c(MPI_File fh, MPI_Offset offset, void *buf,
    MPI_Count count, MPI_Datatype datatype, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_File_read_at_all(fh, offset, buf, count, datatype, status, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_at_all(fh, offset, buf, count, datatype, status, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
  INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
  <type> BUF(*)

```

`MPI_FILE_READ_AT_ALL` is a collective version of the blocking `MPI_FILE_READ_AT` interface.

```

MPI_FILE_WRITE_AT(fh, offset, buf, count, datatype, status)

```

INOUT	fh	file handle (handle)
IN	offset	file offset (integer)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```

int MPI_File_write_at(MPI_File fh, MPI_Offset offset, const void *buf,
  int count, MPI_Datatype datatype, MPI_Status *status)

int MPI_File_write_at_c(MPI_File fh, MPI_Offset offset, const void *buf,
  MPI_Count count, MPI_Datatype datatype, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_File_write_at(fh, offset, buf, count, datatype, status, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write_at(fh, offset, buf, count, datatype, status, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_WRITE_AT(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
  INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
  <type> BUF(*)

```

`MPI_FILE_WRITE_AT` writes a file beginning at the position specified by `offset`.

`MPI_FILE_WRITE_AT_ALL(fh, offset, buf, count, datatype, status)`

INOUT	fh	file handle (handle)
IN	offset	file offset (integer)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```
int MPI_File_write_at_all(MPI_File fh, MPI_Offset offset, const void *buf,
                        int count, MPI_Datatype datatype, MPI_Status *status)
```

```
int MPI_File_write_at_all_c(MPI_File fh, MPI_Offset offset, const void *buf,
                        MPI_Count count, MPI_Datatype datatype, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_File_write_at_all(fh, offset, buf, count, datatype, status, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write_at_all(fh, offset, buf, count, datatype, status, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_WRITE_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
  INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
  <type> BUF(*)
```

`MPI_FILE_WRITE_AT_ALL` is a collective version of the blocking `MPI_FILE_WRITE_AT` interface.


```

1 MPI_FILE_IREAD_AT(fh, offset, buf, count, datatype, request)
2     IN      fh                      file handle (handle)
3
4     IN      offset                  file offset (integer)
5
6     OUT     buf                     initial address of buffer (choice)
7
8     IN      count                   number of elements in buffer (integer)
9
10    IN      datatype                 datatype of each buffer element (handle)
11
12    OUT     request                  request object (handle)

```

C binding

```

13 int MPI_File_iread_at(MPI_File fh, MPI_Offset offset, void *buf, int count,
14                      MPI_Datatype datatype, MPI_Request *request)
15
16 int MPI_File_iread_at_c(MPI_File fh, MPI_Offset offset, void *buf,
17                      MPI_Count count, MPI_Datatype datatype, MPI_Request *request)

```

Fortran 2008 binding

```

18 MPI_File_iread_at(fh, offset, buf, count, datatype, request, ierror)
19     TYPE(MPI_File), INTENT(IN) :: fh
20     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
21     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
22     INTEGER, INTENT(IN) :: count
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     TYPE(MPI_Request), INTENT(OUT) :: request
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26
27 MPI_File_iread_at(fh, offset, buf, count, datatype, request, ierror) !(_c)
28     TYPE(MPI_File), INTENT(IN) :: fh
29     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
30     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
31     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
32     TYPE(MPI_Datatype), INTENT(IN) :: datatype
33     TYPE(MPI_Request), INTENT(OUT) :: request
34     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

35 MPI_FILE_IREAD_AT(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
36     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
37     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
38     <type> BUF(*)

```

[MPI_FILE_IREAD_AT](#) is a nonblocking version of the [MPI_FILE_READ_AT](#) interface.

```

42 MPI_FILE_IREAD_AT_ALL(fh, offset, buf, count, datatype, request)
43
44     IN      fh                      file handle (handle)
45
46     IN      offset                  file offset (integer)
47
48     OUT     buf                     initial address of buffer (choice)
49
50     IN      count                   number of elements in buffer (integer)

```


IN	datatype	datatype of each buffer element (handle)
OUT	request	request object (handle)

C binding

```
int MPI_File_iread_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count,
    MPI_Datatype datatype, MPI_Request *request)
```

```
int MPI_File_iread_at_all_c(MPI_File fh, MPI_Offset offset, void *buf,
    MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_File_iread_at_all(fh, offset, buf, count, datatype, request, ierror)
```

```
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_File_iread_at_all(fh, offset, buf, count, datatype, request, ierror) !(_c)
```

```
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_IREAD_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
```

```
    INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
    <type> BUF(*)
```

[MPI_FILE_IREAD_AT_ALL](#) is a nonblocking version of [MPI_FILE_READ_AT_ALL](#). See Section 15.6.5 for semantics of nonblocking collective file operations.

```
MPI_FILE_IWRITE_AT(fh, offset, buf, count, datatype, request)
```

INOUT	fh	file handle (handle)
IN	offset	file offset (integer)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	request	request object (handle)

C binding

```

1 int MPI_File_iwrite_at(MPI_File fh, MPI_Offset offset, const void *buf,
2     int count, MPI_Datatype datatype, MPI_Request *request)
3
4 int MPI_File_iwrite_at_c(MPI_File fh, MPI_Offset offset, const void *buf,
5     MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
6

```

Fortran 2008 binding

```

7 MPI_File_iwrite_at(fh, offset, buf, count, datatype, request, ierror)
8     TYPE(MPI_File), INTENT(IN) :: fh
9     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
10    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
11    INTEGER, INTENT(IN) :: count
12    TYPE(MPI_Datatype), INTENT(IN) :: datatype
13    TYPE(MPI_Request), INTENT(OUT) :: request
14    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_File_iwrite_at(fh, offset, buf, count, datatype, request, ierror) !(_c)
17     TYPE(MPI_File), INTENT(IN) :: fh
18     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
19     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
20     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
21     TYPE(MPI_Datatype), INTENT(IN) :: datatype
22     TYPE(MPI_Request), INTENT(OUT) :: request
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24

```

Fortran binding

```

25 MPI_FILE_IWRITE_AT(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
26     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
27     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
28     <type> BUF(*)
29

```

[MPI_FILE_IWRITE_AT](#) is a nonblocking version of the [MPI_FILE_WRITE_AT](#) interface.

```

30
31
32
33 MPI_FILE_IWRITE_AT_ALL(fh, offset, buf, count, datatype, request)
34

```

35	INOUT	fh	file handle (handle)
36	IN	offset	file offset (integer)
37	IN	buf	initial address of buffer (choice)
38	IN	count	number of elements in buffer (integer)
39	IN	datatype	datatype of each buffer element (handle)
40	OUT	request	request object (handle)

C binding

```

41
42
43
44 int MPI_File_iwrite_at_all(MPI_File fh, MPI_Offset offset, const void *buf,
45     int count, MPI_Datatype datatype, MPI_Request *request)
46
47 int MPI_File_iwrite_at_all_c(MPI_File fh, MPI_Offset offset, const void *buf,
48     MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
49

```

Fortran 2008 binding

```

MPI_File_iwrite_at_all(fh, offset, buf, count, datatype, request, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iwrite_at_all(fh, offset, buf, count, datatype, request, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_IWRITE_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
    INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
    <type> BUF(*)

```

[MPI_FILE_IWRITE_AT_ALL](#) is a nonblocking version of [MPI_FILE_WRITE_AT_ALL](#).

15.4.3 Data Access with Individual File Pointers

MPI maintains one individual file pointer per process per file handle. The current value of this pointer implicitly specifies the offset in the data access routines described in this section. These routines only use and update the individual file pointers maintained by MPI. The shared file pointer is not used nor updated.

The individual file pointer routines have the same semantics as the data access with explicit offset routines described in Section 15.4.2, with the following modification:

- the `offset` is defined to be the current value of the MPI-maintained individual file pointer.

After an individual file pointer operation is initiated, the individual file pointer is updated to point to the next etype after the last one that will be accessed. The file pointer is updated relative to the current view of the file.

If [MPI_MODE_SEQUENTIAL](#) mode was specified when the file was opened, it is erroneous to call the routines in this section, with the exception of [MPI_FILE_GET_BYTE_OFFSET](#).

```

MPI_FILE_READ(fh, buf, count, datatype, status)

```

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)

```

1      IN      datatype      datatype of each buffer element (handle)
2      OUT     status        status object (status)
3
4

```

C binding

```

5      int MPI_File_read(MPI_File fh, void *buf, int count, MPI_Datatype datatype,
6                          MPI_Status *status)
7
8      int MPI_File_read_c(MPI_File fh, void *buf, MPI_Count count,
9                          MPI_Datatype datatype, MPI_Status *status)
10

```

Fortran 2008 binding

```

11     MPI_File_read(fh, buf, count, datatype, status, ierror)
12     TYPE(MPI_File), INTENT(IN) :: fh
13     TYPE(*), DIMENSION(..) :: buf
14     INTEGER, INTENT(IN) :: count
15     TYPE(MPI_Datatype), INTENT(IN) :: datatype
16     TYPE(MPI_Status) :: status
17     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
18
19     MPI_File_read(fh, buf, count, datatype, status, ierror) !(_c)
20     TYPE(MPI_File), INTENT(IN) :: fh
21     TYPE(*), DIMENSION(..) :: buf
22     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     TYPE(MPI_Status) :: status
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26

```

Fortran binding

```

27     MPI_FILE_READ(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
28     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
29     <type> BUF(*)
30

```

MPI_FILE_READ reads a file using the individual file pointer.

Example 15.2. The following Fortran code fragment is an example of reading a file until the end of file is reached:

```

34     ! Read a pre-existing input file until all data has been read.
35     ! Call routine "process_input" if all requested data is read.
36     ! The Fortran 90 "exit" statement exits the loop.
37
38     integer    bufsize, numread, totprocessed, status(MPI_STATUS_SIZE)
39     parameter (bufsize=100)
40     real       localbuffer(bufsize)
41     integer(kind=MPI_OFFSET_KIND) zero
42
43     zero = 0
44
45     call MPI_FILE_OPEN(MPI_COMM_WORLD, "myoldfile", &
46                        MPI_MODE_RDONLY, MPI_INFO_NULL, myfh, ierr)
47     call MPI_FILE_SET_VIEW(myfh, zero, MPI_REAL, MPI_REAL, 'native', &
48                        MPI_INFO_NULL, ierr)
49     totprocessed = 0
50

```

```

do
  call MPI_FILE_READ(myfh, localbuffer, bufsize, MPI_REAL, &
                    status, ierr)
  call MPI_GET_COUNT(status, MPI_REAL, numread, ierr)
  call process_input(localbuffer, numread)
  totprocessed = totprocessed + numread
  if (numread < bufsize) exit
end do

write(6, 1001) numread, bufsize, totprocessed
1001 format("No more data: read", I3, "and expected", I3, &
           "Processed total of", I6, "before terminating job.")

call MPI_FILE_CLOSE(myfh, ierr)

```

MPI_FILE_READ_ALL(fh, buf, count, datatype, status)

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```
int MPI_File_read_all(MPI_File fh, void *buf, int count, MPI_Datatype datatype,
                     MPI_Status *status)
```

```
int MPI_File_read_all_c(MPI_File fh, void *buf, MPI_Count count,
                       MPI_Datatype datatype, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_File_read_all(fh, buf, count, datatype, status, ierror)
```

```

TYPE(MPI_File), INTENT(IN) :: fh
TYPE(*), DIMENSION(..) :: buf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```
MPI_File_read_all(fh, buf, count, datatype, status, ierror) !(_c)
```

```

TYPE(MPI_File), INTENT(IN) :: fh
TYPE(*), DIMENSION(..) :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_ALL(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

```

`MPI_FILE_READ_ALL` is a collective version of the blocking `MPI_FILE_READ` interface.

```

MPI_FILE_WRITE(fh, buf, count, datatype, status)

```

INOUT	fh	file handle (handle)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```

int MPI_File_write(MPI_File fh, const void *buf, int count,
    MPI_Datatype datatype, MPI_Status *status)

int MPI_File_write_c(MPI_File fh, const void *buf, MPI_Count count,
    MPI_Datatype datatype, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_File_write(fh, buf, count, datatype, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write(fh, buf, count, datatype, status, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_WRITE(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

```

`MPI_FILE_WRITE` writes a file using the individual file pointer.

MPI_FILE_WRITE_ALL(fh, buf, count, datatype, status)

INOUT	fh	file handle (handle)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```
int MPI_File_write_all(MPI_File fh, const void *buf, int count,
                      MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write_all_c(MPI_File fh, const void *buf, MPI_Count count,
                      MPI_Datatype datatype, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_File_write_all(fh, buf, count, datatype, status, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_all(fh, buf, count, datatype, status, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_WRITE_ALL(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
  INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
  <type> BUF(*)
```

MPI_FILE_WRITE_ALL is a collective version of the blocking **MPI_FILE_WRITE** interface.

MPI_FILE_IREAD(fh, buf, count, datatype, request)

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	request	request object (handle)

C binding

```
int MPI_File_iread(MPI_File fh, void *buf, int count, MPI_Datatype datatype,
                  MPI_Request *request)
```

```
int MPI_File_iread_c(MPI_File fh, void *buf, MPI_Count count,
                    MPI_Datatype datatype, MPI_Request *request)
```

Fortran 2008 binding

```
MPI_File_iread(fh, buf, count, datatype, request, ierror)
```

```
TYPE(MPI_File), INTENT(IN) :: fh
```

```
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

```
INTEGER, INTENT(IN) :: count
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_File_iread(fh, buf, count, datatype, request, ierror) !(_c)
```

```
TYPE(MPI_File), INTENT(IN) :: fh
```

```
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

```
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
```

```
TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
TYPE(MPI_Request), INTENT(OUT) :: request
```

```
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_IREAD(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
```

```
INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
```

```
<type> BUF(*)
```

MPI_FILE_IREAD is a nonblocking version of the **MPI_FILE_READ** interface.

Example 15.3. The following Fortran code fragment illustrates file pointer update semantics:

```
! Read the first twenty reals in a file into two local
! buffers. Note that when the first MPI_FILE_IREAD returns,
! the file pointer has been updated to point to the
! eleventh real in the file.

integer bufsize, req1, req2
integer, dimension(MPI_STATUS_SIZE) :: status1, status2
parameter (bufsize=10)
real buf1(bufsize), buf2(bufsize)
integer(kind=MPI_OFFSET_KIND) zero

zero = 0
call MPI_FILE_OPEN(MPI_COMM_WORLD, 'myoldfile', &
                  MPI_MODE_RDONLY, MPI_INFO_NULL, myfh, ierr)
call MPI_FILE_SET_VIEW(myfh, zero, MPI_REAL, MPI_REAL, 'native', &
                  MPI_INFO_NULL, ierr)
call MPI_FILE_IREAD(myfh, buf1, bufsize, MPI_REAL, &
                  req1, ierr)
call MPI_FILE_IREAD(myfh, buf2, bufsize, MPI_REAL, &
                  req2, ierr)
```



```

call MPI_WAIT(req1, status1, ierr)
call MPI_WAIT(req2, status2, ierr)

call MPI_FILE_CLOSE(myfh, ierr)

```

MPI_FILE_IREAD_ALL(fh, buf, count, datatype, request)

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	request	request object (handle)

C binding

```

int MPI_File_iread_all(MPI_File fh, void *buf, int count,
                      MPI_Datatype datatype, MPI_Request *request)

int MPI_File_iread_all_c(MPI_File fh, void *buf, MPI_Count count,
                      MPI_Datatype datatype, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_File_iread_all(fh, buf, count, datatype, request, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iread_all(fh, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_IREAD_ALL(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
  INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
  <type> BUF(*)

```

MPI_FILE_IREAD_ALL is a nonblocking version of **MPI_FILE_READ_ALL**.

```

1 MPI_FILE_IWRITE(fh, buf, count, datatype, request)
2     INOUT    fh                file handle (handle)
3
4     IN       buf                initial address of buffer (choice)
5
6     IN       count              number of elements in buffer (integer)
7
8     IN       datatype            datatype of each buffer element (handle)
9
10    OUT      request             request object (handle)

```

C binding

```

11 int MPI_File_iwrite(MPI_File fh, const void *buf, int count,
12                    MPI_Datatype datatype, MPI_Request *request)
13
14 int MPI_File_iwrite_c(MPI_File fh, const void *buf, MPI_Count count,
15                      MPI_Datatype datatype, MPI_Request *request)

```

Fortran 2008 binding

```

16 MPI_File_iwrite(fh, buf, count, datatype, request, ierror)
17     TYPE(MPI_File), INTENT(IN) :: fh
18     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
19     INTEGER, INTENT(IN) :: count
20     TYPE(MPI_Datatype), INTENT(IN) :: datatype
21     TYPE(MPI_Request), INTENT(OUT) :: request
22     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24 MPI_File_iwrite(fh, buf, count, datatype, request, ierror) !(_c)
25     TYPE(MPI_File), INTENT(IN) :: fh
26     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
27     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
28     TYPE(MPI_Datatype), INTENT(IN) :: datatype
29     TYPE(MPI_Request), INTENT(OUT) :: request
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

33 MPI_FILE_IWRITE(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
34     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
35     <type> BUF(*)
36
37     MPI_FILE_IWRITE is a nonblocking version of MPI_FILE_WRITE.

```

```

39 MPI_FILE_IWRITE_ALL(fh, buf, count, datatype, request)
40     INOUT    fh                file handle (handle)
41
42     IN       buf                initial address of buffer (choice)
43
44     IN       count              number of elements in buffer (integer)
45
46     IN       datatype            datatype of each buffer element (handle)
47
48     OUT      request             request object (handle)

```

C binding

```

int MPI_File_iwrite_all(MPI_File fh, const void *buf, int count,
                        MPI_Datatype datatype, MPI_Request *request)

int MPI_File_iwrite_all_c(MPI_File fh, const void *buf, MPI_Count count,
                        MPI_Datatype datatype, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_File_iwrite_all(fh, buf, count, datatype, request, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iwrite_all(fh, buf, count, datatype, request, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_IWRITE_ALL(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
    INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
    <type> BUF(*)

    MPI_FILE_IWRITE_ALL is a nonblocking version of MPI_FILE_WRITE_ALL.

```

```

MPI_FILE_SEEK(fh, offset, whence)

```

INOUT	fh	file handle (handle)
IN	offset	file offset (integer)
IN	whence	update mode (state)

C binding

```

int MPI_File_seek(MPI_File fh, MPI_Offset offset, int whence)

```

Fortran 2008 binding

```

MPI_File_seek(fh, offset, whence, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    INTEGER, INTENT(IN) :: whence
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_SEEK(FH, OFFSET, WHENCE, IERROR)
    INTEGER FH, WHENCE, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET

```

`MPI_FILE_SEEK` updates the individual file pointer according to whence, which has the following possible values:

<code>MPI_SEEK_SET</code>	the pointer is set to <code>offset</code>
<code>MPI_SEEK_CUR</code>	the pointer is set to the current pointer position plus <code>offset</code>
<code>MPI_SEEK_END</code>	the pointer is set to the end of file plus <code>offset</code>

The `offset` can be negative, which allows seeking backwards. It is erroneous to seek to a negative position in the view.

`MPI_FILE_GET_POSITION(fh, offset)`

IN	fh	file handle (handle)
OUT	offset	offset of individual pointer (integer)

C binding

```
int MPI_File_get_position(MPI_File fh, MPI_Offset *offset)
```

Fortran 2008 binding

```
MPI_File_get_position(fh, offset, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: offset
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_POSITION(FH, OFFSET, IERROR)
  INTEGER FH, IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
```

`MPI_FILE_GET_POSITION` returns, in `offset`, the current position of the individual file pointer in etype units relative to the current view.

Advice to users. The `offset` can be used in a future call to `MPI_FILE_SEEK` using `whence = MPI_SEEK_SET` to return to the current position. To set the displacement to the current file pointer position, first convert `offset` into an absolute byte position using `MPI_FILE_GET_BYTE_OFFSET`, then call `MPI_FILE_SET_VIEW` with the resulting displacement. (*End of advice to users.*)

`MPI_FILE_GET_BYTE_OFFSET(fh, offset, disp)`

IN	fh	file handle (handle)
IN	offset	offset (integer)
OUT	disp	absolute byte position of offset (integer)

C binding

```
int MPI_File_get_byte_offset(MPI_File fh, MPI_Offset offset, MPI_Offset *disp)
```

Fortran 2008 binding

```
MPI_File_get_byte_offset(fh, offset, disp, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
```

```

INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: disp
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_GET_BYTE_OFFSET(FH, OFFSET, DISP, IERROR)
  INTEGER FH, IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET, DISP

```

MPI_FILE_GET_BYTE_OFFSET converts a view-relative offset into an absolute byte position. The absolute byte position (from the beginning of the file) of *offset* relative to the current view of *fh* is returned in *disp*.

15.4.4 Data Access with Shared File Pointers

MPI maintains exactly one shared file pointer per collective **MPI_FILE_OPEN** (shared among processes in the communicator group). The current value of this pointer implicitly specifies the offset in the data access routines described in this section. These routines only use and update the shared file pointer maintained by MPI. The individual file pointers are not used nor updated.

The shared file pointer routines have the same semantics as the data access with explicit offset routines described in Section 15.4.2, with the following modifications:

- the *offset* is defined to be the current value of the MPI-maintained shared file pointer,
- the effect of multiple calls to shared file pointer routines is defined to behave as if the calls were serialized, and
- the use of shared file pointer routines is erroneous unless all processes use the same file view.

For the noncollective shared file pointer routines, the serialization ordering is not deterministic. The user needs to use other synchronization means to enforce a specific order.

After a shared file pointer operation is initiated, the shared file pointer is updated to point to the next etype after the last one that will be accessed. The file pointer is updated relative to the current view of the file.

Noncollective Operations

```

MPI_FILE_READ_SHARED(fh, buf, count, datatype, status)

```

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```

int MPI_File_read_shared(MPI_File fh, void *buf, int count,
  MPI_Datatype datatype, MPI_Status *status)

```

```

1  int MPI_File_read_shared_c(MPI_File fh, void *buf, MPI_Count count,
2      MPI_Datatype datatype, MPI_Status *status)
3

```

Fortran 2008 binding

```

4  MPI_File_read_shared(fh, buf, count, datatype, status, ierror)
5      TYPE(MPI_File), INTENT(IN) :: fh
6      TYPE(*), DIMENSION(..) :: buf
7      INTEGER, INTENT(IN) :: count
8      TYPE(MPI_Datatype), INTENT(IN) :: datatype
9      TYPE(MPI_Status) :: status
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12 MPI_File_read_shared(fh, buf, count, datatype, status, ierror) !(_c)
13     TYPE(MPI_File), INTENT(IN) :: fh
14     TYPE(*), DIMENSION(..) :: buf
15     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
16     TYPE(MPI_Datatype), INTENT(IN) :: datatype
17     TYPE(MPI_Status) :: status
18     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
19

```

Fortran binding

```

20 MPI_FILE_READ_SHARED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
21     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
22     <type> BUF(*)
23

```

MPI_FILE_READ_SHARED reads a file using the shared file pointer.

```

24
25
26 MPI_FILE_WRITE_SHARED(fh, buf, count, datatype, status)
27
28     INOUT    fh                file handle (handle)
29     IN       buf               initial address of buffer (choice)
30     IN       count             number of elements in buffer (integer)
31     IN       datatype          datatype of each buffer element (handle)
32     OUT      status            status object (status)
33
34

```

C binding

```

35
36 int MPI_File_write_shared(MPI_File fh, const void *buf, int count,
37     MPI_Datatype datatype, MPI_Status *status)
38
39 int MPI_File_write_shared_c(MPI_File fh, const void *buf, MPI_Count count,
40     MPI_Datatype datatype, MPI_Status *status)
41

```

Fortran 2008 binding

```

42 MPI_File_write_shared(fh, buf, count, datatype, status, ierror)
43     TYPE(MPI_File), INTENT(IN) :: fh
44     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
45     INTEGER, INTENT(IN) :: count
46     TYPE(MPI_Datatype), INTENT(IN) :: datatype
47     TYPE(MPI_Status) :: status
48     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_File_write_shared(fh, buf, count, datatype, status, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_WRITE_SHARED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

```

MPI_FILE_WRITE_SHARED writes a file using the shared file pointer.

```

MPI_FILE_IREAD_SHARED(fh, buf, count, datatype, request)
    INOUT    fh                file handle (handle)
    OUT      buf                initial address of buffer (choice)
    IN       count              number of elements in buffer (integer)
    IN       datatype            datatype of each buffer element (handle)
    OUT      request             request object (handle)

```

C binding

```

int MPI_File_iread_shared(MPI_File fh, void *buf, int count,
    MPI_Datatype datatype, MPI_Request *request)
int MPI_File_iread_shared_c(MPI_File fh, void *buf, MPI_Count count,
    MPI_Datatype datatype, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_File_iread_shared(fh, buf, count, datatype, request, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_iread_shared(fh, buf, count, datatype, request, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_IREAD_SHARED(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
  INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
  <type> BUF(*)

```

`MPI_FILE_IREAD_SHARED` is a nonblocking version of `MPI_FILE_READ_SHARED`.

```

MPI_FILE_IWRITE_SHARED(fh, buf, count, datatype, request)

```

INOUT	fh	file handle (handle)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	request	request object (handle)

C binding

```

int MPI_File_iwrite_shared(MPI_File fh, const void *buf, int count,
                           MPI_Datatype datatype, MPI_Request *request)

int MPI_File_iwrite_shared_c(MPI_File fh, const void *buf, MPI_Count count,
                             MPI_Datatype datatype, MPI_Request *request)

```

Fortran 2008 binding

```

MPI_File_iwrite_shared(fh, buf, count, datatype, request, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iwrite_shared(fh, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_IWRITE_SHARED(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
  INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
  <type> BUF(*)

```

`MPI_FILE_IWRITE_SHARED` is a nonblocking version of the `MPI_FILE_WRITE_SHARED` interface.

Collective Operations

The semantics of collective access using a shared file pointer are that the accesses to the file will be in the order determined by the ranks of the processes within the group. For each

process in the group, the location in the file at which data is accessed is the position at which the shared file pointer would be after all processes with ranks in the group less than that of this process had accessed their data. In addition, in order to prevent subsequent shared offset accesses by the same processes from interfering with this collective access, the call might return only after all the processes within the group have initiated their accesses. When the call returns, the shared file pointer points to the next etype accessible, according to the file view used by all processes, after the last etype requested.

Advice to users. There may be some programs in which all processes in the group need to access the file using the shared file pointer, but the program may not *require* that data be accessed in order of process rank. In such programs, using the shared ordered routines (e.g., `MPI_FILE_WRITE_ORDERED` rather than `MPI_FILE_WRITE_SHARED`) may enable an implementation to optimize access, improving performance. (*End of advice to users.*)

Advice to implementors. Accesses to the data requested by all processes do not have to be serialized. Once all processes have issued their requests, locations within the file for all accesses can be computed, and accesses can proceed independently from each other, possibly in parallel. (*End of advice to implementors.*)

`MPI_FILE_READ_ORDERED(fh, buf, count, datatype, status)`

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)
OUT	status	status object (status)

C binding

```
int MPI_File_read_ordered(MPI_File fh, void *buf, int count,
                          MPI_Datatype datatype, MPI_Status *status)

int MPI_File_read_ordered_c(MPI_File fh, void *buf, MPI_Count count,
                            MPI_Datatype datatype, MPI_Status *status)
```

Fortran 2008 binding

```
MPI_File_read_ordered(fh, buf, count, datatype, status, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_ordered(fh, buf, count, datatype, status, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..) :: buf
```

```

1      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
2      TYPE(MPI_Datatype), INTENT(IN) :: datatype
3      TYPE(MPI_Status) :: status
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

6      MPI_FILE_READ_ORDERED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
7      INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
8      <type> BUF(*)

```

10 [MPI_FILE_READ_ORDERED](#) is a collective version of the [MPI_FILE_READ_SHARED](#)
11 interface.

```

13      MPI_FILE_WRITE_ORDERED(fh, buf, count, datatype, status)

```

15	INOUT	fh	file handle (handle)
16	IN	buf	initial address of buffer (choice)
17	IN	count	number of elements in buffer (integer)
18	IN	datatype	datatype of each buffer element (handle)
19	OUT	status	status object (status)

C binding

```

23      int MPI_File_write_ordered(MPI_File fh, const void *buf, int count,
24                                MPI_Datatype datatype, MPI_Status *status)
25
26      int MPI_File_write_ordered_c(MPI_File fh, const void *buf, MPI_Count count,
27                                MPI_Datatype datatype, MPI_Status *status)

```

Fortran 2008 binding

```

29      MPI_File_write_ordered(fh, buf, count, datatype, status, ierror)
30      TYPE(MPI_File), INTENT(IN) :: fh
31      TYPE(*), DIMENSION(..), INTENT(IN) :: buf
32      INTEGER, INTENT(IN) :: count
33      TYPE(MPI_Datatype), INTENT(IN) :: datatype
34      TYPE(MPI_Status) :: status
35      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37      MPI_File_write_ordered(fh, buf, count, datatype, status, ierror) !(_c)
38      TYPE(MPI_File), INTENT(IN) :: fh
39      TYPE(*), DIMENSION(..), INTENT(IN) :: buf
40      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
41      TYPE(MPI_Datatype), INTENT(IN) :: datatype
42      TYPE(MPI_Status) :: status
43      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

45      MPI_FILE_WRITE_ORDERED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
46      INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
47      <type> BUF(*)

```

`MPI_FILE_WRITE_ORDERED` is a collective version of the `MPI_FILE_WRITE_SHARED` interface.

Seek

If `MPI_MODE_SEQUENTIAL` mode was specified when the file was opened, it is erroneous to call the following two routines (`MPI_FILE_SEEK_SHARED` and `MPI_FILE_GET_POSITION_SHARED`).

`MPI_FILE_SEEK_SHARED(fh, offset, whence)`

INOUT	fh	file handle (handle)
IN	offset	file offset (integer)
IN	whence	update mode (state)

C binding

`int MPI_File_seek_shared(MPI_File fh, MPI_Offset offset, int whence)`

Fortran 2008 binding

```
MPI_File_seek_shared(fh, offset, whence, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  INTEGER, INTENT(IN) :: whence
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SEEK_SHARED(FH, OFFSET, WHENCE, IERROR)
  INTEGER FH, WHENCE, IERROR
  INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
```

`MPI_FILE_SEEK_SHARED` updates the shared file pointer according to `whence`, which has the following possible values:

<code>MPI_SEEK_SET</code>	the pointer is set to <code>offset</code>
<code>MPI_SEEK_CUR</code>	the pointer is set to the current pointer position plus <code>offset</code>
<code>MPI_SEEK_END</code>	the pointer is set to the end of file plus <code>offset</code>

`MPI_FILE_SEEK_SHARED` is collective; all the processes in the communicator group associated with the file handle `fh` must call `MPI_FILE_SEEK_SHARED` with the same values for `offset` and `whence`.

The `offset` can be negative, which allows seeking backwards. It is erroneous to seek to a negative position in the view.

`MPI_FILE_GET_POSITION_SHARED(fh, offset)`

IN	fh	file handle (handle)
OUT	offset	offset of shared pointer (integer)

C binding

`int MPI_File_get_position_shared(MPI_File fh, MPI_Offset *offset)`

Fortran 2008 binding

```

MPI_File_get_position_shared(fh, offset, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: offset
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_GET_POSITION_SHARED(FH, OFFSET, IERROR)
    INTEGER FH, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET

```

`MPI_FILE_GET_POSITION_SHARED` returns, in `offset`, the current position of the shared file pointer in type units relative to the current view.

Advice to users. The `offset` can be used in a future call to `MPI_FILE_SEEK_SHARED` using `whence = MPI_SEEK_SET` to return to the current position. To set the displacement to the current file pointer position, first convert `offset` into an absolute byte position using `MPI_FILE_GET_BYTE_OFFSET`, then call `MPI_FILE_SET_VIEW` with the resulting displacement. (*End of advice to users.*)

15.4.5 Split Collective Data Access Routines

MPI provides a restricted form of “nonblocking collective” I/O operations for all data accesses using split collective data access routines. These routines are referred to as “split” collective routines, because a single collective operation is split in two: a begin routine and an end routine. The begin routine begins the operation, much like a nonblocking data access (e.g., `MPI_FILE_IREAD`). The end routine completes the operation, much like the matching test or wait (e.g., `MPI_WAIT`). As with nonblocking data access operations, the user must not use the buffer passed to a begin routine while the routine is outstanding; the operation must be completed with an end routine before it is safe to free buffers, etc.

Split collective data access operations on a file handle `fh` are subject to the semantic rules given below.

- On any MPI process, each file handle may have at most one active split collective operation at any time.
- Begin calls are collective over the group of processes that participated in the collective open and follow the ordering rules for collective calls.
- End calls are collective over the group of processes that participated in the collective open and follow the ordering rules for collective calls. Each end call matches the preceding begin call for the same collective operation. When an “end” call is made, exactly one unmatched “begin” call for the same operation must precede it.
- An implementation is free to implement any split collective data access routine using the corresponding blocking collective routine when either the begin call (e.g., `MPI_FILE_READ_ALL_BEGIN`) or the end call (e.g., `MPI_FILE_READ_ALL_END`) is issued. The begin and end calls are provided to allow the user and MPI implementation to optimize the collective operation.

According to the definitions in Section 2.4.2, the begin procedures are incomplete. They are also nonlocal procedures because they may or may not return before they are called in all MPI processes of the process group.

Advice to users. This is one of the exceptions in which incomplete procedures are nonlocal and therefore blocking. (*End of advice to users.*)

- Split collective operations do not match the corresponding regular collective operation. For example, in a single collective read operation, an `MPI_FILE_READ_ALL` on one process does not match an `MPI_FILE_READ_ALL_BEGIN/`
`MPI_FILE_READ_ALL_END` pair on another process.
- Split collective routines must specify a buffer in both the begin and end routines. By specifying the buffer that receives data in the end routine, we can avoid the problems described in [Problems with Code Movement and Register Optimization](#), Section 21.1.17, but not all of the problems, such as those described in Sections 21.1.12, 21.1.13, and 21.1.16.
- No collective I/O operations are permitted on a file handle concurrently with a split collective access on that file handle (i.e., between the begin and end of the access). That is, the following example is erroneous.

Example 15.4. Erroneous example fragment of concurrent split collective access on a file handle:

```
MPI_File_read_all_begin(fh, ...);
...
MPI_File_read_all(fh, ...);
...
MPI_File_read_all_end(fh, ...);
```

- In a multithreaded implementation, any split collective begin and end operation called by a process must be called from the same thread. This restriction is made to simplify the implementation in the multithreaded case. (Note that we have already disallowed having two threads begin a split collective operation on the same file handle since only one split collective operation can be active on a file handle at any time.)

The arguments for these routines have the same meaning as for the equivalent collective versions (e.g., the argument definitions for `MPI_FILE_READ_ALL_BEGIN` and `MPI_FILE_READ_ALL_END` are equivalent to the arguments for `MPI_FILE_READ_ALL`). The begin routine (e.g., `MPI_FILE_READ_ALL_BEGIN`) begins a split collective operation that, when completed with the matching end routine (i.e., `MPI_FILE_READ_ALL_END`) produces the result as defined for the equivalent collective routine (i.e., `MPI_FILE_READ_ALL`).

For the purpose of consistency semantics (Section 15.6.1), a matched pair of split collective data access operations (e.g., `MPI_FILE_READ_ALL_BEGIN` and `MPI_FILE_READ_ALL_END`) compose a single data access.

```
1 MPI_FILE_READ_AT_ALL_BEGIN(fh, offset, buf, count, datatype)
```

```
2     IN      fh                      file handle (handle)
```

```
3     IN      offset                  file offset (integer)
```

```
4     OUT     buf                     initial address of buffer (choice)
```

```
5     IN      count                   number of elements in buffer (integer)
```

```
6     IN      datatype                datatype of each buffer element (handle)
```

10 C binding

```
11 int MPI_File_read_at_all_begin(MPI_File fh, MPI_Offset offset, void *buf,  
12                               int count, MPI_Datatype datatype)
```

```
13 int MPI_File_read_at_all_begin_c(MPI_File fh, MPI_Offset offset, void *buf,  
14                                 MPI_Count count, MPI_Datatype datatype)
```

16 Fortran 2008 binding

```
17 MPI_File_read_at_all_begin(fh, offset, buf, count, datatype, ierror)
```

```
18     TYPE(MPI_File), INTENT(IN) :: fh
```

```
19     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
```

```
20     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

```
21     INTEGER, INTENT(IN) :: count
```

```
22     TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
24 MPI_File_read_at_all_begin(fh, offset, buf, count, datatype, ierror) !(_c)
```

```
25     TYPE(MPI_File), INTENT(IN) :: fh
```

```
26     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
```

```
27     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

```
28     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
```

```
29     TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

32 Fortran binding

```
33 MPI_FILE_READ_AT_ALL_BEGIN(FH, OFFSET, BUF, COUNT, DATATYPE, IERROR)
```

```
34     INTEGER FH, COUNT, DATATYPE, IERROR
```

```
35     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
```

```
36     <type> BUF(*)
```

```
39 MPI_FILE_READ_AT_ALL_END(fh, buf, status)
```

```
40     IN      fh                      file handle (handle)
```

```
41     OUT     buf                     initial address of buffer (choice)
```

```
42     OUT     status                  status object (status)
```

45 C binding

```
46 int MPI_File_read_at_all_end(MPI_File fh, void *buf, MPI_Status *status)
```

Fortran 2008 binding

```

MPI_File_read_at_all_end(fh, buf, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_AT_ALL_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

```

```

MPI_FILE_WRITE_AT_ALL_BEGIN(fh, offset, buf, count, datatype)

```

INOUT	fh	file handle (handle)
IN	offset	file offset (integer)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)

C binding

```

int MPI_File_write_at_all_begin(MPI_File fh, MPI_Offset offset,
    const void *buf, int count, MPI_Datatype datatype)
int MPI_File_write_at_all_begin_c(MPI_File fh, MPI_Offset offset,
    const void *buf, MPI_Count count, MPI_Datatype datatype)

```

Fortran 2008 binding

```

MPI_File_write_at_all_begin(fh, offset, buf, count, datatype, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_at_all_begin(fh, offset, buf, count, datatype, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_WRITE_AT_ALL_BEGIN(FH, OFFSET, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
    <type> BUF(*)

```

```
1 MPI_FILE_WRITE_AT_ALL_END(fh, buf, status)
```

```
2     INOUT    fh                file handle (handle)
```

```
4     IN      buf                initial address of buffer (choice)
```

```
5     OUT     status             status object (status)
```

7 C binding

```
8 int MPI_File_write_at_all_end(MPI_File fh, const void *buf, MPI_Status *status)
```

10 Fortran 2008 binding

```
11 MPI_File_write_at_all_end(fh, buf, status, ierror)
```

```
12     TYPE(MPI_File), INTENT(IN) :: fh
```

```
13     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
```

```
14     TYPE(MPI_Status) :: status
```

```
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

16 Fortran binding

```
17 MPI_FILE_WRITE_AT_ALL_END(FH, BUF, STATUS, IERROR)
```

```
18     INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
```

```
19     <type> BUF(*)
```

```
22 MPI_FILE_READ_ALL_BEGIN(fh, buf, count, datatype)
```

```
23     INOUT    fh                file handle (handle)
```

```
24     OUT     buf                initial address of buffer (choice)
```

```
26     IN      count              number of elements in buffer (integer)
```

```
28     IN      datatype           datatype of each buffer element (handle)
```

30 C binding

```
31 int MPI_File_read_all_begin(MPI_File fh, void *buf, int count,  
32     MPI_Datatype datatype)
```

```
33 int MPI_File_read_all_begin_c(MPI_File fh, void *buf, MPI_Count count,  
34     MPI_Datatype datatype)
```

36 Fortran 2008 binding

```
37 MPI_File_read_all_begin(fh, buf, count, datatype, ierror)
```

```
38     TYPE(MPI_File), INTENT(IN) :: fh
```

```
39     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

```
40     INTEGER, INTENT(IN) :: count
```

```
41     TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
42     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
43 MPI_File_read_all_begin(fh, buf, count, datatype, ierror) !(_c)
```

```
44     TYPE(MPI_File), INTENT(IN) :: fh
```

```
45     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

```
46     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
```

```
47     TYPE(MPI_Datatype), INTENT(IN) :: datatype
```

```
48     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```


Fortran binding

```

MPI_FILE_READ_ALL_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    <type> BUF(*)

```

```

MPI_FILE_READ_ALL_END(fh, buf, status)

```

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
OUT	status	status object (status)

C binding

```

int MPI_File_read_all_end(MPI_File fh, void *buf, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_File_read_all_end(fh, buf, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_ALL_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

```

```

MPI_FILE_WRITE_ALL_BEGIN(fh, buf, count, datatype)

```

INOUT	fh	file handle (handle)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)

C binding

```

int MPI_File_write_all_begin(MPI_File fh, const void *buf, int count,
    MPI_Datatype datatype)

int MPI_File_write_all_begin_c(MPI_File fh, const void *buf, MPI_Count count,
    MPI_Datatype datatype)

```

Fortran 2008 binding

```

MPI_File_write_all_begin(fh, buf, count, datatype, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1 MPI_File_write_all_begin(fh, buf, count, datatype, ierror) !(_c)
2     TYPE(MPI_File), INTENT(IN) :: fh
3     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
4     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
5     TYPE(MPI_Datatype), INTENT(IN) :: datatype
6     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

8 MPI_FILE_WRITE_ALL_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
9     INTEGER FH, COUNT, DATATYPE, IERROR
10     <type> BUF(*)

```

```

12
13 MPI_FILE_WRITE_ALL_END(fh, buf, status)

```

```

15     INOUT    fh                file handle (handle)
16     IN       buf              initial address of buffer (choice)
17     OUT      status           status object (status)

```

C binding

```

20 int MPI_File_write_all_end(MPI_File fh, const void *buf, MPI_Status *status)

```

Fortran 2008 binding

```

22 MPI_File_write_all_end(fh, buf, status, ierror)
23     TYPE(MPI_File), INTENT(IN) :: fh
24     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
25     TYPE(MPI_Status) :: status
26     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

28 MPI_FILE_WRITE_ALL_END(FH, BUF, STATUS, IERROR)
29     INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
30     <type> BUF(*)

```

```

32
33
34 MPI_FILE_READ_ORDERED_BEGIN(fh, buf, count, datatype)

```

```

36     INOUT    fh                file handle (handle)
37     OUT      buf              initial address of buffer (choice)
38     IN       count            number of elements in buffer (integer)
39     IN       datatype         datatype of each buffer element (handle)

```

C binding

```

42 int MPI_File_read_ordered_begin(MPI_File fh, void *buf, int count,
43                               MPI_Datatype datatype)
44
45 int MPI_File_read_ordered_begin_c(MPI_File fh, void *buf, MPI_Count count,
46                                  MPI_Datatype datatype)

```

Fortran 2008 binding

```

MPI_File_read_ordered_begin(fh, buf, count, datatype, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_ordered_begin(fh, buf, count, datatype, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_ORDERED_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    <type> BUF(*)

```

```

MPI_FILE_READ_ORDERED_END(fh, buf, status)

```

INOUT	fh	file handle (handle)
OUT	buf	initial address of buffer (choice)
OUT	status	status object (status)

C binding

```

int MPI_File_read_ordered_end(MPI_File fh, void *buf, MPI_Status *status)

```

Fortran 2008 binding

```

MPI_File_read_ordered_end(fh, buf, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_FILE_READ_ORDERED_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

```

```

MPI_FILE_WRITE_ORDERED_BEGIN(fh, buf, count, datatype)

```

INOUT	fh	file handle (handle)
IN	buf	initial address of buffer (choice)
IN	count	number of elements in buffer (integer)
IN	datatype	datatype of each buffer element (handle)

C binding

```
int MPI_File_write_ordered_begin(MPI_File fh, const void *buf, int count,
                                MPI_Datatype datatype)
```

```
int MPI_File_write_ordered_begin_c(MPI_File fh, const void *buf,
                                    MPI_Count count, MPI_Datatype datatype)
```

Fortran 2008 binding

```
MPI_File_write_ordered_begin(fh, buf, count, datatype, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write_ordered_begin(fh, buf, count, datatype, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_WRITE_ORDERED_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    <type> BUF(*)
```

```
MPI_FILE_WRITE_ORDERED_END(fh, buf, status)
```

INOUT	fh	file handle (handle)
IN	buf	initial address of buffer (choice)
OUT	status	status object (status)

C binding

```
int MPI_File_write_ordered_end(MPI_File fh, const void *buf,
                                MPI_Status *status)
```

Fortran 2008 binding

```
MPI_File_write_ordered_end(fh, buf, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_WRITE_ORDERED_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)
```

15.5 File Interoperability

At the most basic level, file interoperability is the ability to read the information previously written to a file—not just the bits of data, but the actual information the bits represent. MPI guarantees full interoperability within a single MPI environment, and supports increased interoperability outside that environment through the external data representation (Section 15.5.2) as well as the data conversion functions (Section 15.5.3).

Interoperability within a single MPI environment (which could be considered “operability”) ensures that file data written by one MPI process can be read by any other MPI process, subject to the consistency constraints (see Section 15.6.1), provided that it would have been possible to start the two processes simultaneously and have them reside in a single `MPI_COMM_WORLD`. Furthermore, both processes must see the same data values at every absolute byte offset in the file for which data was written.

This single environment file interoperability implies that file data is accessible regardless of the number of processes.

There are three aspects to file interoperability:

- transferring the bits,
- converting between different file structures, and
- converting between different machine representations.

The first two aspects of file interoperability are beyond the scope of this standard, as both are highly machine dependent. However, transferring the bits of a file into and out of the MPI environment (e.g., by writing a file to tape) is required to be supported by all MPI implementations. In particular, an implementation must specify how familiar operations similar to POSIX `cp`, `rm`, and `mv` can be performed on the file. Furthermore, it is expected that the facility provided maintains the correspondence between absolute byte offsets (e.g., after possible file structure conversion, the data bits at byte offset 102 in the MPI environment are at byte offset 102 outside the MPI environment). As an example, a simple off-line conversion utility that transfers and converts files between the native file system and the MPI environment would suffice, provided it maintained the offset coherence mentioned above. In a high-quality implementation of MPI, users will be able to manipulate MPI files using the same or similar tools that the native file system offers for manipulating its files.

The remaining aspect of file interoperability, converting between different machine representations, is supported by the typing information specified in the `etype` and `filetype`. This facility allows the information in files to be shared between any two applications, regardless of whether they use MPI, and regardless of the machine architectures on which they run.

MPI supports multiple data representations: “native”, “internal”, and “external32”. An implementation may support additional data representations. MPI also supports user-defined data representations (see Section 15.5.3). The “native” and “internal” data representations are implementation dependent, while the “external32” representation is common to all MPI implementations and facilitates file interoperability. The data representation is specified in the `datarep` argument to `MPI_FILE_SET_VIEW`.

Advice to users. MPI is not guaranteed to retain knowledge of what data representation was used when a file is written. Therefore, to correctly retrieve file data, an MPI

application is responsible for specifying the same data representation as was used to create the file. (*End of advice to users.*)

"native": Data in this representation is stored in a file exactly as it is in memory. The advantage of this data representation is that data precision and I/O performance are not lost in type conversions with a purely homogeneous environment. The disadvantage is the loss of transparent interoperability within a heterogeneous MPI environment.

Advice to users. This data representation should only be used in a homogeneous MPI environment, or when the MPI application is capable of performing the datatype conversions itself. (*End of advice to users.*)

Advice to implementors. When implementing read and write operations on top of MPI message-passing, the message data should be typed as `MPI_BYTE` to ensure that the message routines do not perform any type conversions on the data. (*End of advice to implementors.*)

"internal": This data representation can be used for I/O operations in a homogeneous or heterogeneous environment; the implementation will perform type conversions if necessary. The implementation is free to store data in any format of its choice, with the restriction that it will maintain constant extents for all predefined datatypes in any one file. The environment in which the resulting file can be reused is implementation-defined and must be documented by the implementation.

Rationale. This data representation allows the implementation to perform I/O efficiently in a heterogeneous environment, though with implementation-defined restrictions on how the file can be reused. (*End of rationale.*)

Advice to implementors. Since "external32" is a superset of the functionality provided by "internal", an implementation may choose to implement "internal" as "external32". (*End of advice to implementors.*)

"external32": This data representation states that read and write operations convert all data from and to the "external32" representation defined in Section 15.5.2. The data conversion rules for communication also apply to these conversions (see Section 3.3.2). The data on the storage medium is always in this canonical representation, and the data in memory is always in the local process's native representation.

This data representation has several advantages. First, all processes reading the file in a heterogeneous MPI environment will automatically have the data converted to their respective native representations. Second, the file can be exported from one MPI environment and imported into any other MPI environment with the guarantee that the second environment will be able to read all the data in the file.

The disadvantage of this data representation is that data precision and I/O performance may be lost in datatype conversions.

Advice to implementors. When implementing read and write operations on top of MPI message-passing, the message data should be converted to and from the "external32" representation in the client, and sent as type `MPI_BYTE`. This will avoid possible double datatype conversions and the associated further loss of precision and performance. (*End of advice to implementors.*)

15.5.1 Datatypes for File Interoperability

If the file data representation is other than "native", care must be taken in constructing etypes and filetypes. Any of the datatype constructor functions may be used; however, for those functions that accept displacements in bytes, the displacements must be specified in terms of their values in the file for the file data representation being used. MPI will interpret these byte displacements as is; no scaling will be done. The function `MPI_FILE_GET_TYPE_EXTENT` can be used to calculate the extents of datatypes in the file. For etypes and filetypes that are portable datatypes (see Section 2.4), MPI will scale any displacements in the datatypes to match the file data representation. Datatypes passed as arguments to read/write routines specify the data layout in memory; therefore, they must always be constructed using displacements corresponding to displacements in memory.

Advice to users. One can logically think of the file as if it were stored in the memory of a file server. The `etype` and `filetype` are interpreted as if they were defined at this file server, by the same sequence of calls used to define them at the calling process. If the data representation is "native", then this logical file server runs on the same architecture as the calling process, so that these types define the same data layout on the file as they would define in the memory of the calling process. If the `etype` and `filetype` are portable datatypes, then the data layout defined in the file is the same as would be defined in the calling process memory, up to a scaling factor. The routine `MPI_FILE_GET_TYPE_EXTENT` can be used to calculate this scaling factor. Thus, two equivalent, portable datatypes will define the same data layout in the file, even in a heterogeneous environment with "internal", "external32", or user defined data representations. Otherwise, the `etype` and `filetype` must be constructed so that their typemap and extent are the same on any architecture. This can be achieved if they have an explicit upper bound and lower bound (defined using `MPI_TYPE_CREATE_RESIZED`). This condition must also be fulfilled by any datatype that is used in the construction of the `etype` and `filetype`, if this datatype is replicated contiguously, either explicitly, by a call to `MPI_TYPE_CONTIGUOUS`, or implicitly, by a `blocklength` argument that is greater than one. If an `etype` or `filetype` is not portable, and has a typemap or extent that is architecture dependent, then the data layout specified by it on a file is implementation dependent.

File data representations other than "native" may be different from corresponding data representations in memory. Therefore, for these file data representations, it is important not to use hardwired byte offsets for file positioning, including the initial displacement that specifies the view. When a portable datatype (see Section 2.4) is used in a data access operation, any holes in the datatype are scaled to match the data representation. However, note that this technique only works when all the processes that created the file view build their etypes from the same predefined datatypes. For example, if one process uses an `etype` built from `MPI_INT` and another uses an `etype` built from `MPI_FLOAT`, the resulting views may be nonportable because the relative sizes of these types may differ from one data representation to another. (*End of advice to users.*)

`MPI_FILE_GET_TYPE_EXTENT(fh, datatype, extent)`

IN fh file handle (handle)

```

1      IN      datatype      datatype (handle)
2
3      OUT      extent      datatype extent (integer)

```

C binding

```

5  int MPI_File_get_type_extent(MPI_File fh, MPI_Datatype datatype,
6                               MPI_Aint *extent)
7
8  int MPI_File_get_type_extent_c(MPI_File fh, MPI_Datatype datatype,
9                               MPI_Count *extent)

```

Fortran 2008 binding

```

11 MPI_File_get_type_extent(fh, datatype, extent, ierror)
12     TYPE(MPI_File), INTENT(IN) :: fh
13     TYPE(MPI_Datatype), INTENT(IN) :: datatype
14     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: extent
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17 MPI_File_get_type_extent(fh, datatype, extent, ierror) !(_c)
18     TYPE(MPI_File), INTENT(IN) :: fh
19     TYPE(MPI_Datatype), INTENT(IN) :: datatype
20     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: extent
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

23 MPI_FILE_GET_TYPE_EXTENT(FH, DATATYPE, EXTENT, IERROR)
24     INTEGER FH, DATATYPE, IERROR
25     INTEGER(KIND=MPI_ADDRESS_KIND) EXTENT

```

Returns the extent of `datatype` in the file `fh`. This extent will be the same for all processes accessing the file `fh`. If the current view uses a user-defined data representation (see Section 15.5.3), MPI uses the `dtype_file_extent_fn` callback to calculate the extent.

If the datatype extent cannot be represented in `extent`, it is set to `MPI_UNDEFINED`.

Advice to implementors. In the case of user-defined data representations, the extent of a derived datatype can be calculated by first determining the extents of the predefined datatypes in this derived datatype using `dtype_file_extent_fn` (see Section 15.5.3). (*End of advice to implementors.*)

15.5.2 External Data Representation: "external32"

All MPI implementations are required to support the data representation defined in this section. Support of optional datatypes (e.g., `MPI_INTEGER2`) is not required.

All floating point values are in big-endian IEEE format [43] of the appropriate size. Floating point values are represented by one of three IEEE formats. These are the IEEE “Single (binary32),” “Double (binary64),” and “Double Extended (binary128)” formats, requiring 4, 8, and 16 bytes of storage, respectively. For the IEEE “Double Extended (binary128)” formats, MPI specifies a format width of 16 bytes, with 15 exponent bits, bias = +16383, 112 fraction bits, and an encoding analogous to the “Double (binary64)” format. All integral values are in two’s complement big-endian format. Big-endian means most significant byte at lowest address byte. For C `_Bool`, Fortran `LOGICAL`, and C++ `bool`, 0 implies false and nonzero implies true. C float `_Complex`, double `_Complex`, and

long double `_Complex`, Fortran `COMPLEX` and `DOUBLE COMPLEX`, and other complex types are represented by a pair of floating point format values for the real and imaginary components. Characters are in ISO 8859-1 format [44]. Wide characters (of type `MPI_WCHAR`) are in Unicode format [69].

All signed numerals (e.g., `MPI_INT`, `MPI_REAL`) have the sign bit at the most significant bit. `MPI_COMPLEX` and `MPI_DOUBLE_COMPLEX` have the sign bit of the real and imaginary parts at the most significant bit of each part.

According to IEEE specifications [43], the “NaN” (not a number) is system dependent. It should not be interpreted within MPI as anything other than “NaN.”

Advice to implementors. The MPI treatment of “NaN” is similar to the approach used in XDR [66]. (*End of advice to implementors.*)

All data is byte aligned, regardless of type. All data items are stored contiguously in the file (if the file view is contiguous).

Advice to implementors. All bytes of `LOGICAL` and `bool` must be checked to determine the value. (*End of advice to implementors.*)

Advice to users. The type `MPI_PACKED` is treated as bytes and is not converted. The user should be aware that `MPI_PACK` has the option of placing a header in the beginning of the pack buffer. (*End of advice to users.*)

The sizes of the predefined datatypes returned from `MPI_TYPE_CREATE_F90_REAL`, `MPI_TYPE_CREATE_F90_COMPLEX`, and `MPI_TYPE_CREATE_F90_INTEGER` are defined in Section 21.1.9, page 833.

Advice to implementors. When converting a larger size integer to a smaller size integer, only the least significant bytes are moved. Care must be taken to preserve the sign bit value. This allows no conversion errors if the data range is within the range of the smaller size integer. (*End of advice to implementors.*)

Tables 15.2, 15.3, and 15.4 specify the sizes of predefined, optional, and C++ datatypes in “external32” format, respectively.

15.5.3 User-Defined Data Representations

There are two situations that cannot be handled by the required representations:

1. a user wants to write a file in a representation unknown to the implementation, and
2. a user wants to read a file written in a representation unknown to the implementation.

User-defined data representations allow the user to insert a third party converter into the I/O stream to do the data representation conversion.

Table 15.2: "external32" sizes of predefined datatypes

Predefined Type	Length
MPI_PACKED	1
MPI_BYTE	1
MPI_CHAR	1
MPI_UNSIGNED_CHAR	1
MPI_SIGNED_CHAR	1
MPI_WCHAR	2
MPI_SHORT	2
MPI_UNSIGNED_SHORT	2
MPI_INT	4
MPI_LONG	4
MPI_UNSIGNED	4
MPI_UNSIGNED_LONG	4
MPI_LONG_LONG_INT	8
MPI_UNSIGNED_LONG_LONG	8
MPI_FLOAT	4
MPI_DOUBLE	8
MPI_LONG_DOUBLE	16
MPI_C_BOOL	1
MPI_INT8_T	1
MPI_INT16_T	2
MPI_INT32_T	4
MPI_INT64_T	8
MPI_UINT8_T	1
MPI_UINT16_T	2
MPI_UINT32_T	4
MPI_UINT64_T	8
MPI_AINT	8
MPI_COUNT	8
MPI_OFFSET	8
MPI_C_COMPLEX	2*4
MPI_C_FLOAT_COMPLEX	2*4
MPI_C_DOUBLE_COMPLEX	2*8
MPI_C_LONG_DOUBLE_COMPLEX	2*16
MPI_CHARACTER	1
MPI_LOGICAL	4
MPI_INTEGER	4
MPI_REAL	4
MPI_DOUBLE_PRECISION	8
MPI_COMPLEX	2*4
MPI_DOUBLE_COMPLEX	2*8
MPI_CXX_BOOL	1
MPI_CXX_FLOAT_COMPLEX	2*4
MPI_CXX_DOUBLE_COMPLEX	2*8
MPI_CXX_LONG_DOUBLE_COMPLEX	2*16

Table 15.3: "external32" sizes of optional datatypes

Predefined Type	Length
MPI_INTEGER1	1
MPI_INTEGER2	2
MPI_INTEGER4	4
MPI_INTEGER8	8
MPI_INTEGER16	16
MPI_LOGICAL1	1
MPI_LOGICAL2	2
MPI_LOGICAL4	4
MPI_LOGICAL8	8
MPI_LOGICAL16	16
MPI_REAL2	2
MPI_REAL4	4
MPI_REAL8	8
MPI_REAL16	16
MPI_COMPLEX4	2*2
MPI_COMPLEX8	2*4
MPI_COMPLEX16	2*8
MPI_COMPLEX32	2*16

Table 15.4: "external32" sizes of C++ datatypes

C++ Types	Length
MPI_CXX_BOOL	1
MPI_CXX_FLOAT_COMPLEX	2*4
MPI_CXX_DOUBLE_COMPLEX	2*8
MPI_CXX_LONG_DOUBLE_COMPLEX	2*16

MPI_REGISTER_DATAREP(datarep, read_conversion_fn, write_conversion_fn,
dtype_file_extent_fn, extra_state)

IN	datarep	data representation identifier (string)
IN	read_conversion_fn	function invoked to convert from file representation to native representation (function)
IN	write_conversion_fn	function invoked to convert from native representation to file representation (function)
IN	dtype_file_extent_fn	function invoked to get the extent of a datatype as represented in the file (function)
IN	extra_state	extra state

C binding

```
int MPI_Register_datarep(const char *datarep,
    MPI_Datarep_conversion_function *read_conversion_fn,
    MPI_Datarep_conversion_function *write_conversion_fn,
```

```

1      MPI_Daterep_extent_function *dtype_file_extent_fn,
2      void *extra_state)
3
4  int MPI_Register_daterep_c(const char *daterep,
5      MPI_Daterep_conversion_function_c *read_conversion_fn,
6      MPI_Daterep_conversion_function_c *write_conversion_fn,
7      MPI_Daterep_extent_function *dtype_file_extent_fn,
8      void *extra_state)

```

Fortran 2008 binding

```

10 MPI_Register_daterep(daterep, read_conversion_fn, write_conversion_fn,
11     dtype_file_extent_fn, extra_state, ierror)
12 CHARACTER(LEN=*), INTENT(IN) :: daterep
13 PROCEDURE(MPI_Daterep_conversion_function) :: read_conversion_fn,
14     write_conversion_fn
15 PROCEDURE(MPI_Daterep_extent_function) :: dtype_file_extent_fn
16 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
17 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
18
19 MPI_Register_daterep_c(daterep, read_conversion_fn, write_conversion_fn,
20     dtype_file_extent_fn, extra_state, ierror) !(_c)
21 CHARACTER(LEN=*), INTENT(IN) :: daterep
22 PROCEDURE(MPI_Daterep_conversion_function_c) :: read_conversion_fn,
23     write_conversion_fn
24 PROCEDURE(MPI_Daterep_extent_function) :: dtype_file_extent_fn
25 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
26 INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

28 MPI_REGISTER_DATAREP(DATAREP, READ_CONVERSION_FN, WRITE_CONVERSION_FN,
29     DTYPE_FILE_EXTENT_FN, EXTRA_STATE, IERROR)
30 CHARACTER*(*) DATAREP
31 EXTERNAL READ_CONVERSION_FN, WRITE_CONVERSION_FN, DTYPE_FILE_EXTENT_FN
32 INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
33 INTEGER IERROR

```

The call associates `read_conversion_fn`, `write_conversion_fn`, and `dtype_file_extent_fn` with the data representation identifier `daterep`. `daterep` can then be used as an argument to [MPI_FILE_SET_VIEW](#), causing subsequent data access operations to call the conversion functions to convert all data items accessed between file data representation and native representation. [MPI_REGISTER_DATAREP](#) is a local operation and only registers the data representation for the calling MPI process. If `daterep` is already defined, an error in the error class `MPI_ERR_DUP_DATAREP` is raised using the default file error handler (see Section 15.7). The length of a data representation string is limited to the value of [MPI_MAX_DATAREP_STRING](#). [MPI_MAX_DATAREP_STRING](#) must have a value of at least 64. No routines are provided to delete data representations and free the associated resources; it is not expected that an application will generate them in significant numbers.

Extent Callback

```

typedef int MPI_Datarep_extent_function(MPI_Datatype datatype,
                                         MPI_Aint *extent, void *extra_state);

ABSTRACT INTERFACE
  SUBROUTINE MPI_Datarep_extent_function(datatype, extent, extra_state, ierror)
    TYPE(MPI_Datatype) :: datatype
    INTEGER(KIND=MPI_ADDRESS_KIND) :: extent, extra_state
    INTEGER :: ierror

SUBROUTINE DATAREP_EXTENT_FUNCTION(DATATYPE, EXTENT, EXTRA_STATE, IERROR)
  INTEGER DATATYPE, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTENT, EXTRA_STATE

```

The function `dtype_file_extent_fn` must return, in `file_extent`, the number of bytes required to store `datatype` in the file representation. The function is passed, in `extra_state`, the argument that was passed to the `MPI_REGISTER_DATAREP` call. MPI will only call this routine with predefined datatypes employed by the user.

Rationale. This callback does not have a large count variant because it is anticipated that large counts will not be required to represent the extent output value. (*End of rationale.*)

`MPI_Datarep_conversion_function` also supports large count types in separate additional MPI procedures in C (suffixed with the “_c”) and multiple abstract interfaces in Fortran when using `USE mpi_f08`.

If the extent cannot be represented in `extent`, the callback function shall set `extent` to `MPI_UNDEFINED`. The MPI implementation will then raise an error of class `MPI_ERR_VALUE_TOO_LARGE`.

Datarep Conversion Functions

```

typedef int MPI_Datarep_conversion_function(void *userbuf,
                                             MPI_Datatype datatype, int count, void *filebuf,
                                             MPI_Offset position, void *extra_state);

typedef int MPI_Datarep_conversion_function_c(void *userbuf,
                                             MPI_Datatype datatype, MPI_Count count, void *filebuf,
                                             MPI_Offset position, void *extra_state);

ABSTRACT INTERFACE
  SUBROUTINE MPI_Datarep_conversion_function(userbuf, datatype, count, filebuf,
                                             position, extra_state, ierror)
    USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
    TYPE(C_PTR), VALUE :: userbuf, filebuf
    TYPE(MPI_Datatype) :: datatype
    INTEGER :: count, ierror
    INTEGER(KIND=MPI_OFFSET_KIND) :: position
    INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state

ABSTRACT INTERFACE

```

```

1  SUBROUTINE MPI_Datarep_conversion_function_c(userbuf, datatype, count,
2      filebuf, position, extra_state, ierror) !(_c)
3      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
4      TYPE(C_PTR), VALUE :: userbuf, filebuf
5      TYPE(MPI_Datatype) :: datatype
6      INTEGER(KIND=MPI_COUNT_KIND) :: count
7      INTEGER(KIND=MPI_OFFSET_KIND) :: position
8      INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
9      INTEGER :: ierror
10
11  SUBROUTINE DATAREP_CONVERSION_FUNCTION(USERBUF, DATATYPE, COUNT, FILEBUF,
12      POSITION, EXTRA_STATE, IERROR)
13      <TYPE> USERBUF(*), FILEBUF(*)
14      INTEGER DATATYPE, COUNT, IERROR
15      INTEGER(KIND=MPI_OFFSET_KIND) POSITION
16      INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE

```

The function `read_conversion_fn` must convert from file data representation to native representation. Before calling this routine, MPI allocates and fills `filebuf` with `count` contiguous data items. The type of each data item matches the corresponding entry for the predefined `datatype` in the type signature of `datatype`. The function is passed, in `extra_state`, the argument that was passed to the `MPI_REGISTER_DATAREP` call. The function must copy all `count` data items from `filebuf` to `userbuf` in the distribution described by `datatype`, converting each data item from file representation to native representation. `datatype` will be equivalent to the `datatype` that the user passed to the read function. If the size of `datatype` is less than the size of the `count` data items, the conversion function must treat `datatype` as being contiguously tiled over the `userbuf`. The conversion function must begin storing converted data at the location in `userbuf` specified by `position` into the (tiled) `datatype`.

Advice to users. Although the conversion functions have similarities to `MPI_PACK` and `MPI_UNPACK`, one should note the differences in the use of the arguments `count` and `position`. In the conversion functions, `count` is a count of data items (i.e., count of typemap entries of `datatype`), and `position` is an index into this typemap. In `MPI_PACK`, `incount` refers to the number of whole `datatypes`, and `position` is a number of bytes. (*End of advice to users.*)

Advice to implementors. A converted read operation could be implemented as follows:

1. Get file extent of all data items
2. Allocate a `filebuf` large enough to hold all `count` data items
3. Read data from file into `filebuf`
4. Call `read_conversion_fn` to convert data and place it into `userbuf`
5. Deallocate `filebuf`

(*End of advice to implementors.*)

If MPI cannot allocate a buffer large enough to hold all the data to be converted from a read operation, it may call the conversion function repeatedly using the same `datatype` and `userbuf`, and reading successive chunks of data to be converted in `filebuf`. For the first

call (and in the case when all the data to be converted fits into `filebuf`), MPI will call the function with `position` set to zero. Data converted during this call will be stored in the `userbuf` according to the first `count` data items in `datatype`. Then in subsequent calls to the conversion function, MPI will increment the value in `position` by the `count` of items converted in the previous call, and the `userbuf` pointer will be unchanged.

Rationale. Passing the conversion function a position and one datatype for the transfer allows the conversion function to decode the datatype only once and cache an internal representation of it on the datatype. Then on subsequent calls, the conversion function can use the `position` to quickly find its place in the datatype and continue storing converted data where it left off at the end of the previous call. (*End of rationale.*)

Advice to users. Although the conversion function may usefully cache an internal representation on the datatype, it should not cache any state information specific to an ongoing conversion operation, since it is possible for the same datatype to be used concurrently in multiple conversion operations. (*End of advice to users.*)

The function `write_conversion_fn` must convert from native representation to file data representation. Before calling this routine, MPI allocates `filebuf` of a size large enough to hold `count` contiguous data items. The type of each data item matches the corresponding entry for the predefined datatype in the type signature of `datatype`. The function must copy `count` data items from `userbuf` in the distribution described by `datatype`, to a contiguous distribution in `filebuf`, converting each data item from native representation to file representation. If the size of `datatype` is less than the size of `count` data items, the conversion function must treat `datatype` as being contiguously tiled over the `userbuf`.

The function must begin copying at the location in `userbuf` specified by `position` into the (tiled) `datatype`. `datatype` will be equivalent to the datatype that the user passed to the write function. The function is passed, in `extra_state`, the argument that was passed to the `MPI_REGISTER_DATAREP` call.

The predefined constant `MPI_CONVERSION_FN_NULL` may be used as either `write_conversion_fn` or `read_conversion_fn` in bindings of `MPI_REGISTER_DATAREP` without large counts in these conversion callbacks, whereas the constant `MPI_CONVERSION_FN_NULL_C` can be used in the large count version (i.e., `MPI_Register_datarep_c`). In either of these cases, MPI will not attempt to invoke `write_conversion_fn` or `read_conversion_fn`, respectively, but will perform the requested data access using the native data representation.

An MPI implementation must ensure that all data accessed is converted, either by using a `filebuf` large enough to hold all the requested data items or else by making repeated calls to the conversion function with the same `datatype` argument and appropriate values for `position`.

An implementation will only invoke the callback routines in this section (`read_conversion_fn`, `write_conversion_fn`, and `dtype_file_extent_fn`) when one of the read or write routines in Section 15.4 or `MPI_FILE_GET_TYPE_EXTENT` is called by the user. `dtype_file_extent_fn` will only be passed predefined datatypes employed by the user. The conversion functions will only be passed datatypes equivalent to those that the user has passed to one of the routines noted above.

The conversion functions must be reentrant. User defined data representations are restricted to use byte alignment for all types. Furthermore, it is erroneous for the conversion functions to call any collective routines or to free `datatype`.

The conversion functions should return an error code. If the returned error code has a value other than `MPI_SUCCESS`, the implementation will raise an error in the class `MPI_ERR_CONVERSION`.

15.5.4 Matching Data Representations

It is the user's responsibility to ensure that the data representation used to read data from a file is *compatible* with the data representation that was used to write that data to the file.

In general, using the same data representation name when writing and reading a file does not guarantee that the representation is compatible. Similarly, using different representation names on two different implementations may yield compatible representations.

Compatibility can be obtained when "external32" representation is used, although precision may be lost and the performance may be less than when "native" representation is used. Compatibility is guaranteed using "external32" provided at least one of the following conditions is met.

- The data access routines directly use types enumerated in Section 15.5.2, that are supported by all implementations participating in the I/O. The predefined type used to write a data item must also be used to read a data item.
- In the case of Fortran 90 programs, the programs participating in the data accesses obtain compatible datatypes using MPI routines that specify precision and/or range (Section 21.1.9).
- For any given data item, the programs participating in the data accesses use compatible predefined types to write and read the data item.

User-defined data representations may be used to provide an implementation compatibility with another implementation's "native" or "internal" representation.

Advice to users. Section 21.1.9 defines routines that support the use of matching datatypes in heterogeneous environments and contains examples illustrating their use. (*End of advice to users.*)

15.6 Consistency and Semantics

15.6.1 File Consistency

Consistency semantics define the outcome of multiple accesses to a single file. All file accesses in MPI are relative to a specific file handle created from a collective open. MPI provides three levels of consistency: sequential consistency among all accesses using a single file handle, sequential consistency among all accesses using file handles created from a single collective open with atomic mode enabled, and user-imposed consistency among accesses other than the above. Sequential consistency means the behavior of a set of operations will be as if the operations were performed in some serial order consistent with program order; each access appears atomic, although the exact ordering of accesses is unspecified. User-imposed consistency may be obtained using program order and calls to `MPI_FILE_SYNC`.

Let FH_1 be the set of file handles created from one particular collective open of the file FOO , and FH_2 be the set of file handles created from a different collective open of

FOO. Note that nothing restrictive is said about FH_1 and FH_2 : the sizes of FH_1 and FH_2 may be different, the groups of processes used for each open may or may not intersect, the file handles in FH_1 may be destroyed before those in FH_2 are created, etc. Consider the following three cases: a single file handle (e.g., $fh_1 \in FH_1$), two file handles created from a single collective open (e.g., $fh_{1a} \in FH_1$ and $fh_{1b} \in FH_1$), and two file handles from different collective opens (e.g., $fh_1 \in FH_1$ and $fh_2 \in FH_2$).

For the purpose of consistency semantics, a matched pair (Section 15.4.5) of split collective data access operations (e.g., `MPI_FILE_READ_ALL_BEGIN` and `MPI_FILE_READ_ALL_END`) compose a single data access operation. Similarly, a non-blocking data access routine (e.g., `MPI_FILE_IREAD`) and the routine that completes the request (e.g., `MPI_WAIT`) also compose a single data access operation. For all cases below, these data access operations are subject to the same constraints as blocking data access operations.

Advice to users. For an `MPI_FILE_IREAD` and `MPI_WAIT` pair, the operation begins when `MPI_FILE_IREAD` is called and ends when `MPI_WAIT` returns. (*End of advice to users.*)

Assume that A_1 and A_2 are two data access operations. Let D_1 (D_2) be the set of absolute byte displacements of every byte accessed in A_1 (A_2). The two data accesses **overlap** if $D_1 \cap D_2 \neq \emptyset$. The two data accesses **conflict** if they overlap and at least one is a write access.

Let SEQ_{fh} be a sequence of file operations on a single file handle, bracketed by `MPI_FILE_SYNC`s on that file handle. (Both opening and closing a file implicitly perform an `MPI_FILE_SYNC`.) SEQ_{fh} is a “write sequence” if any of the data access operations in the sequence are writes or if any of the file manipulation operations in the sequence change the state of the file (e.g., `MPI_FILE_SET_SIZE` or `MPI_FILE_PREALLOCATE`). Given two sequences, SEQ_1 and SEQ_2 , we say they are not **concurrent** if one sequence is guaranteed to completely precede the other (temporally).

The requirements for guaranteeing sequential consistency among all accesses to a particular file are divided into the three cases given below. If any of these requirements are not met, then the value of all data in that file is implementation dependent.

Case 1: $fh_1 \in FH_1$. All operations on fh_1 are sequentially consistent if atomic mode is set. If nonatomic mode is set, then all operations on fh_1 are sequentially consistent if they are either nonconcurrent, nonconflicting, or both.

Case 2: $fh_{1a} \in FH_1$ and $fh_{1b} \in FH_1$. Assume A_1 is a data access operation using fh_{1a} , and A_2 is a data access operation using fh_{1b} . If for any access A_1 , there is no access A_2 that conflicts with A_1 , then MPI guarantees sequential consistency.

However, unlike POSIX semantics, the default MPI semantics for conflicting accesses do not guarantee sequential consistency. If A_1 and A_2 conflict, sequential consistency can be guaranteed by either enabling atomic mode via the `MPI_FILE_SET_ATOMICITY` routine, or meeting the condition described in Case 3 below.

Case 3: $fh_1 \in FH_1$ and $fh_2 \in FH_2$. Consider access to a single file using file handles from distinct collective opens. In order to guarantee sequential consistency, `MPI_FILE_SYNC` must be used (both opening and closing a file implicitly perform an `MPI_FILE_SYNC`).

Sequential consistency is guaranteed among accesses to a single file if for any write sequence SEQ_1 to the file, there is no sequence SEQ_2 to the file that is *concurrent* with SEQ_1 . To guarantee sequential consistency when there are write sequences, `MPI_FILE_SYNC` must be used together with a mechanism that guarantees nonconcurrency of the sequences.

See the examples in Section 15.6.11 for further clarification of some of these consistency semantics.

`MPI_FILE_SET_ATOMICITY(fh, flag)`

INOUT	fh	file handle (handle)
IN	flag	true to set atomic mode, false to set nonatomic mode (logical)

C binding

```
int MPI_File_set_atomicity(MPI_File fh, int flag)
```

Fortran 2008 binding

```
MPI_File_set_atomicity(fh, flag, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  LOGICAL, INTENT(IN) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SET_ATOMICITY(FH, FLAG, IERROR)
  INTEGER FH, IERROR
  LOGICAL FLAG
```

Let FH be the set of file handles created by one collective open. The consistency semantics for data access operations using FH is set by collectively calling `MPI_FILE_SET_ATOMICITY` on FH . `MPI_FILE_SET_ATOMICITY` is collective; all processes in the group must pass identical values for fh and $flag$. If $flag$ is true, atomic mode is set; if $flag$ is false, nonatomic mode is set.

Changing the consistency semantics for an open file only affects new data accesses. All completed data accesses are guaranteed to abide by the consistency semantics in effect during their execution. Nonblocking data accesses and split collective operations that have not completed (e.g., via `MPI_WAIT`) are only guaranteed to abide by nonatomic mode consistency semantics.

Advice to implementors. Since the semantics guaranteed by atomic mode are stronger than those guaranteed by nonatomic mode, an implementation is free to adhere to the more stringent atomic mode semantics for outstanding requests. (*End of advice to implementors.*)

MPI_FILE_GET_ATOMICITY(fh, flag)

IN	fh	file handle (handle)
OUT	flag	true if atomic mode, false if nonatomic mode (logical)

C binding

```
int MPI_File_get_atomicity(MPI_File fh, int *flag)
```

Fortran 2008 binding

```
MPI_File_get_atomicity(fh, flag, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  LOGICAL, INTENT(OUT) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_GET_ATOMICITY(FH, FLAG, IERROR)
  INTEGER FH, IERROR
  LOGICAL FLAG
```

MPI_FILE_GET_ATOMICITY returns the current consistency semantics for data access operations on the set of file handles created by one collective open. If **flag** is true, atomic mode is enabled; if **flag** is false, nonatomic mode is enabled.

MPI_FILE_SYNC(fh)

INOUT	fh	file handle (handle)
-------	----	----------------------

C binding

```
int MPI_File_sync(MPI_File fh)
```

Fortran 2008 binding

```
MPI_File_sync(fh, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_FILE_SYNC(FH, IERROR)
  INTEGER FH, IERROR
```

Calling **MPI_FILE_SYNC** with **fh** causes all previous writes to **fh** by the calling process to be transferred to the storage device. If other processes have made updates to the storage device, then all such updates become visible to subsequent reads of **fh** by the calling process. **MPI_FILE_SYNC** may be necessary to ensure sequential consistency in certain cases (see above).

MPI_FILE_SYNC is a collective operation.

The user is responsible for ensuring that all nonblocking requests and split collective operations on **fh** have been completed before calling **MPI_FILE_SYNC**—otherwise, the call to **MPI_FILE_SYNC** is erroneous.

15.6.2 Random Access vs. Sequential Files

MPI distinguishes ordinary random access files from sequential stream files, such as pipes and tape files. Sequential stream files must be opened with the `MPI_MODE_SEQUENTIAL` flag set in the `amode`. For these files, the only permitted data access operations are shared file pointer reads and writes. Filetypes and etypes with holes are erroneous. In addition, the notion of file pointer is not meaningful; therefore, calls to `MPI_FILE_SEEK_SHARED` and `MPI_FILE_GET_POSITION_SHARED` are erroneous, and the pointer update rules specified for the data access routines do not apply. The amount of data accessed by a data access operation will be the amount requested unless the end of file is reached or an error is raised.

Rationale. This implies that reading on a pipe will always wait until the requested amount of data is available or until the process writing to the pipe has issued an end of file. (*End of rationale.*)

Finally, for some sequential files, such as those corresponding to magnetic tapes or streaming network connections, writes to the file may be destructive. In other words, a write may act as a truncate (a `MPI_FILE_SET_SIZE` with size set to the current position) followed by the write.

15.6.3 Progress

The *progress* rules of MPI are both a promise to users and a set of constraints on implementors. In cases where the progress rules restrict possible implementation choices more than the interface specification alone, the progress rules take precedence.

All blocking routines must complete in finite time unless an exceptional condition (such as resource exhaustion) causes an error.

Nonblocking data access routines inherit the following progress rule from nonblocking point-to-point communication: a nonblocking write is equivalent to a nonblocking send for which a receive is eventually posted, and a nonblocking read is equivalent to a nonblocking receive for which a send is eventually posted.

Finally, an implementation is free to delay progress of collective routines until all processes in the group associated with the collective call have invoked the routine. Once all processes in the group have invoked the routine, the progress rule of the equivalent noncollective routine must be followed.

15.6.4 Collective File Operations

Collective file operations are subject to the same restrictions as collective communication operations. For a complete discussion, please refer to the semantics set forth in Section 6.14.

Collective file operations are collective over a duplicate of the communicator used to open the file—this duplicate communicator is implicitly specified via the file handle argument. Different processes can pass different values for other arguments of a collective routine unless specified otherwise.

15.6.5 Nonblocking Collective File Operations

Nonblocking collective file operations are defined only for data access routines with explicit offsets and individual file pointers but not with shared file pointers.

Nonblocking collective file operations are subject to the same restrictions as blocking collective I/O operations. All processes belonging to the group of the communicator that was used to open the file must call collective I/O operations (blocking and nonblocking) in the same order. This is consistent with the ordering rules for collective operations in threaded environments. For a complete discussion, please refer to the semantics set forth in Section 6.14.

Nonblocking collective I/O operations do not match with blocking collective I/O operations. Multiple nonblocking collective I/O operations can be outstanding on a single file handle. High quality MPI implementations should be able to support a large number of *pending* nonblocking I/O operations.

All nonblocking collective I/O calls are local. The call initiates the operation that may progress independently of any communication, computation, or I/O. The call returns a request handle, which must be passed to a completion call. Input buffers should not be modified and output buffers should not be accessed before the completion call returns. The same *progress* rules described for nonblocking collective operations apply for nonblocking collective I/O operations. For a complete discussion, please refer to the semantics set forth in Section 6.12.

15.6.6 Type Matching

The type matching rules for I/O mimic the type matching rules for communication with one exception: if `etype` is `MPI_BYTE`, then this matches any `datatype` in a data access operation. In general, the `etype` of data items written must match the `etype` used to read the items, and for each data access operation, the current `etype` must also match the type declaration of the data access buffer.

Advice to users. In most cases, use of `MPI_BYTE` as a wild card will defeat the file interoperability features of MPI. File interoperability can only perform automatic conversion between heterogeneous data representations when the exact datatypes accessed are explicitly specified. (*End of advice to users.*)

15.6.7 Miscellaneous Clarifications

Once an I/O routine completes, it is safe to free any opaque objects passed as arguments to that routine. For example, the `comm` and `info` used in an `MPI_FILE_OPEN`, or the `etype` and `filetype` used in an `MPI_FILE_SET_VIEW`, can be freed without affecting access to the file. Note that for nonblocking routines and split collective operations, the operation must be completed before it is safe to reuse data buffers passed as arguments.

As in communication, datatypes must be committed before they can be used in file manipulation or data access operations. For example, the `etype` and `filetype` must be committed before calling `MPI_FILE_SET_VIEW`, and the `datatype` must be committed before calling `MPI_FILE_READ` or `MPI_FILE_WRITE`.

15.6.8 MPI_Offset Type

`MPI_Offset` is an integer type of size sufficient to represent the size (in bytes) of the largest file supported by MPI. Displacements and offsets are always specified as values of type `MPI_Offset`.

In Fortran, the corresponding integer is an integer with kind parameter `MPI_OFFSET_KIND`, which is defined in the `mpi_f08` module, the `mpi` module and the `mpif.h` include file.

In Fortran 77 environments that do not support KIND parameters, `MPI_Offset` arguments should be declared as an INTEGER of suitable size. The language interoperability implications for `MPI_Offset` are similar to those for addresses (see Section 21.3).

15.6.9 Logical vs. Physical File Layout

MPI specifies how the data should be laid out in a virtual file structure (the view), not how that file structure is to be stored on one or more disks. Specification of the physical file structure was avoided because it is expected that the mapping of files to disks will be system specific, and any specific control over file layout would therefore restrict program portability. However, there are still cases where some information may be necessary to optimize file layout. This information can be provided as *hints* specified via `info` when a file is created (see Section 15.2.8).

15.6.10 File Size

The size of a file may be increased by writing to the file after the current end of file. The size may also be changed by calling *MPI size changing* routines, such as `MPI_FILE_SET_SIZE`. A call to a size changing routine does not necessarily change the file size. For example, calling `MPI_FILE_PREALLOCATE` with a size less than the current size does not change the size.

Consider a set of bytes that has been written to a file since the most recent call to a size changing routine, or since `MPI_FILE_OPEN` if no such routine has been called. Let the *high byte* be the byte in that set with the largest displacement. The file size is the larger of

- One plus the displacement of the high byte.
- The size immediately after the size changing routine, or `MPI_FILE_OPEN`, returned.

When applying consistency semantics, calls to `MPI_FILE_SET_SIZE` and `MPI_FILE_PREALLOCATE` are considered writes to the file (which conflict with operations that access bytes at displacements between the old and new file sizes), and `MPI_FILE_GET_SIZE` is considered a read of the file (which overlaps with all accesses to the file).

Advice to users. Any sequence of operations containing the collective routines `MPI_FILE_SET_SIZE` and `MPI_FILE_PREALLOCATE` is a write sequence. As such, sequential consistency in nonatomic mode is not guaranteed unless the conditions in Section 15.6.1 are satisfied. (*End of advice to users.*)

File pointer update semantics (i.e., file pointers are updated by the amount accessed) are only guaranteed if file size changes are sequentially consistent.

Advice to users. Consider the following example. Given two operations made by separate processes to a file containing 100 bytes: an `MPI_FILE_READ` of 10 bytes and an `MPI_FILE_SET_SIZE` to 0 bytes. If the user does not enforce sequential consistency between these two operations, the file pointer may be updated by the amount requested (10 bytes) even if the amount accessed is zero bytes. (*End of advice to users.*)

15.6.11 Examples

The examples in this section illustrate the application of the MPI consistency and semantics guarantees. These address

- conflicting accesses on file handles obtained from a single collective open, and
- all accesses on file handles obtained from two separate collective opens.

The simplest way to achieve consistency for conflicting accesses is to obtain sequential consistency by setting atomic mode.

Example 15.5. For the code below, process 1 will read either 0 or 10 integers. If the latter, every element of `b` will be 5. If nonatomic mode is set, the results of the read are undefined.

```
/* Process 0 */
int i, a[10];
int TRUE = 1;

for (i=0; i<10; i++)
    a[i] = 5;

MPI_File_open(MPI_COMM_WORLD, "workfile",
               MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &fh0);
MPI_File_set_view(fh0, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_File_set_atomicity(fh0, TRUE);
MPI_File_write_at(fh0, 0, a, 10, MPI_INT, &status);
/* MPI_Barrier(MPI_COMM_WORLD); */

/* Process 1 */
int b[10];
int TRUE = 1;
MPI_File_open(MPI_COMM_WORLD, "workfile",
               MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &fh1);
MPI_File_set_view(fh1, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_File_set_atomicity(fh1, TRUE);
/* MPI_Barrier(MPI_COMM_WORLD); */
MPI_File_read_at(fh1, 0, b, 10, MPI_INT, &status);
```

A user may guarantee that the write on process 0 precedes the read on process 1 by imposing temporal order with, for example, calls to [MPI_BARRIER](#).

Advice to users. Routines other than [MPI_BARRIER](#) may be used to impose temporal order. In the example above, process 0 could use [MPI_SEND](#) to send a 0 byte message, received by process 1 using [MPI_RECV](#). (*End of advice to users.*)

Example 15.6. Alternatively, a user can impose consistency with nonatomic mode set:

```
/* Process 0 */
int i, a[10];
for (i=0; i<10; i++)
```



```

1      a[i] = 5;
2
3      MPI_File_open(MPI_COMM_WORLD, "workfile",
4                    MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &fh0);
5      MPI_File_set_view(fh0, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
6      MPI_File_write_at(fh0, 0, a, 10, MPI_INT, &status );
7      MPI_File_sync(fh0);
8      MPI_Barrier(MPI_COMM_WORLD);
9      MPI_File_sync(fh0);
10
11  /* Process 1 */
12
13  int  b[10];
14  MPI_File_open(MPI_COMM_WORLD, "workfile",
15                MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &fh1);
16  MPI_File_set_view(fh1, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
17  MPI_File_sync(fh1);
18  MPI_Barrier(MPI_COMM_WORLD);
19  MPI_File_sync(fh1);
20  MPI_File_read_at(fh1, 0, b, 10, MPI_INT, &status);

```

The “sync-barrier-sync” construct is required because:

- The barrier ensures that the write on process 0 occurs before the read on process 1.
- The first sync guarantees that the data written by all processes is transferred to the storage device.
- The second sync guarantees that all data that has been transferred to the storage device is visible to all processes. (This does not affect process 0 in this example.)

Example 15.7. The following program represents an erroneous attempt to achieve consistency by eliminating the apparently superfluous second “sync” call for each process.

```

31  /* ----- THIS EXAMPLE IS ERRONEOUS ----- */
32  /* Process 0 */
33
34  int  i, a[10];
35  for (i=0; i<10; i++)
36      a[i] = 5;
37
38  MPI_File_open(MPI_COMM_WORLD, "workfile",
39                MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &fh0);
40  MPI_File_set_view(fh0, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
41  MPI_File_write_at(fh0, 0, a, 10, MPI_INT, &status);
42  MPI_File_sync(fh0);
43  MPI_Barrier(MPI_COMM_WORLD);
44
45  /* Process 1 */
46
47  int  b[10];
48  MPI_File_open(MPI_COMM_WORLD, "workfile",
49                MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &fh1);

```



```

MPI_File_set_view(fh1, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_Barrier(MPI_COMM_WORLD);
MPI_File_sync(fh1);
MPI_File_read_at(fh1, 0, b, 10, MPI_INT, &status);

/* ----- THIS EXAMPLE IS ERRONEOUS ----- */

```

The above program also violates the MPI rule against out-of-order collective operations and will deadlock for implementations in which `MPI_FILE_SYNC` blocks.

Advice to users. Some implementations may choose to implement `MPI_FILE_SYNC` as a temporally synchronizing function. When using such an implementation, the “sync-barrier-sync” construct above can be replaced by a single “sync.” The results of using such code with an implementation for which `MPI_FILE_SYNC` is not temporally synchronizing is undefined. (*End of advice to users.*)

Asynchronous I/O

The behavior of asynchronous I/O operations is determined by applying the rules specified above for synchronous I/O operations.

Example 15.8. The following examples all access a pre-existing file “myfile.” Word 10 in myfile initially contains the integer 2. Each example writes and reads word 10. First consider the following code fragment:

```

int a = 4, b, TRUE=1;
MPI_File_open(MPI_COMM_WORLD, "myfile",
              MPI_MODE_RDWR, MPI_INFO_NULL, &fh);
MPI_File_set_view(fh, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
/* MPI_File_set_atomicity(fh, TRUE); Use this to set atomic mode. */
MPI_File_iwrite_at(fh, 10, &a, 1, MPI_INT, &reqs[0]);
MPI_File_iread_at(fh, 10, &b, 1, MPI_INT, &reqs[1]);
MPI_Waitall(2, reqs, statuses);

```

For asynchronous data access operations, MPI specifies that the access occurs at any time between the call to the asynchronous data access routine and the return from the corresponding request complete routine. Thus, executing either the read before the write, or the write before the read is consistent with program order. If atomic mode is set, then MPI guarantees sequential consistency, and the program will read either 2 or 4 into b. If atomic mode is not set, then sequential consistency is not guaranteed and the program may read something other than 2 or 4 due to the conflicting data access.

Similarly, the following code fragment does not order file accesses:

```

int a = 4, b;
MPI_File_open(MPI_COMM_WORLD, "myfile",
              MPI_MODE_RDWR, MPI_INFO_NULL, &fh);
MPI_File_set_view(fh, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
/* MPI_File_set_atomicity(fh, TRUE); Use this to set atomic mode. */
MPI_File_iwrite_at(fh, 10, &a, 1, MPI_INT, &reqs[0]);
MPI_File_iread_at(fh, 10, &b, 1, MPI_INT, &reqs[1]);
MPI_Wait(&reqs[0], &status);
MPI_Wait(&reqs[1], &status);

```

If atomic mode is set, either 2 or 4 will be read into b. Again, MPI does not guarantee sequential consistency in nonatomic mode.

On the other hand, the following code fragment:

```
int a = 4, b;
MPI_File_open(MPI_COMM_WORLD, "myfile",
               MPI_MODE_RDWR, MPI_INFO_NULL, &fh);
MPI_File_set_view(fh, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_File_irewrite_at(fh, 10, &a, 1, MPI_INT, &reqs[0]);
MPI_Wait(&reqs[0], &status);
MPI_File_iread_at(fh, 10, &b, 1, MPI_INT, &reqs[1]);
MPI_Wait(&reqs[1], &status);
```

defines the same ordering as:

```
int a = 4, b;
MPI_File_open(MPI_COMM_WORLD, "myfile",
               MPI_MODE_RDWR, MPI_INFO_NULL, &fh);
MPI_File_set_view(fh, 0, MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_File_write_at(fh, 10, &a, 1, MPI_INT, &status );
MPI_File_read_at(fh, 10, &b, 1, MPI_INT, &status );
```

Since

- nonconcurrent operations on a single file handle are sequentially consistent, and
- the program fragments specify an order for the operations,

MPI guarantees that both program fragments will read the value 4 into b. There is no need to set atomic mode for this example.

Similar considerations apply to conflicting accesses of the form:

```
MPI_File_irewrite_all(fh,...);
MPI_File_iread_all(fh,...);
MPI_Waitall(...);
```

In addition, as mentioned in Section 15.6.5, nonblocking collective I/O operations have to be called in the same order on the file handle by all processes.

Similar considerations apply to conflicting accesses of the form:

```
MPI_File_write_all_begin(fh,...);
MPI_File_iread(fh,...);
MPI_Wait(...);
MPI_File_write_all_end(fh,...);
```

Recall that constraints governing consistency and semantics are not relevant to the following:

```
MPI_File_write_all_begin(fh,...);
MPI_File_read_all_begin(fh,...);
MPI_File_read_all_end(fh,...);
MPI_File_write_all_end(fh,...);
```

since split collective operations on the same file handle may not overlap (see Section 15.4.5).

15.7 I/O Error Handling

By default, communication errors are fatal—`MPI_ERRORS_ARE_FATAL` is the default error handler associated with `MPI_COMM_WORLD`. I/O errors are usually less catastrophic (e.g., “file not found”) than communication errors, and common practice is to catch these errors and continue executing. For this reason, MPI provides additional error facilities for I/O.

Advice to users. MPI does not specify the state of a computation after an erroneous MPI call has occurred. A high-quality implementation will support the I/O error handling facilities, allowing users to write programs using common practice for I/O. (*End of advice to users.*)

Like communicators, each file handle has an error handler associated with it. The MPI I/O error handling routines are defined in Section 9.3.

When MPI calls a user-defined error handler resulting from an error on a particular file handle, the first two arguments passed to the file error handler are the file handle and the error code. For I/O errors that are not associated with a valid file handle (e.g., in `MPI_FILE_OPEN` or `MPI_FILE_DELETE`), the first argument passed to the error handler is `MPI_FILE_NULL`.

I/O error handling differs from communication error handling in another important aspect. By default, the error handler for file handles is `MPI_ERRORS_RETURN`. The **default file error** handler has two purposes: when a new file handle is created (by `MPI_FILE_OPEN`), the error handler for the new file handle is initially set to the default file error handler, and I/O routines that have no valid file handle on which to raise an error (e.g., `MPI_FILE_OPEN` or `MPI_FILE_DELETE`) use the default file error handler. The default file error handler can be changed by specifying `MPI_FILE_NULL` as the `fh` argument to `MPI_FILE_SET_ERRHANDLER`. The current value of the default file error handler can be determined by passing `MPI_FILE_NULL` as the `fh` argument to `MPI_FILE_GET_ERRHANDLER`.

Rationale. For communication, the default error handler is inherited from `MPI_COMM_WORLD` when using the World Model. In I/O, there is no analogous “root” file handle from which default properties can be inherited. Rather than invent a new global file handle, the default file error handler is manipulated as if it were attached to `MPI_FILE_NULL`. (*End of rationale.*)

15.8 I/O Error Classes

The implementation dependent error codes returned by the I/O routines can be used to obtain the matching error classes, as defined in Table 15.5.

In addition, calls to routines in this chapter may raise errors in other MPI classes, such as `MPI_ERR_TYPE`.

15.9 Examples

15.9.1 Double Buffering with Split Collective I/O

Table 15.5: I/O error classes

<code>MPI_ERR_FILE</code>	Invalid file handle
<code>MPI_ERR_NOT_SAME</code>	Collective argument not identical on all processes, or collective routines called in a different order by different processes
<code>MPI_ERR_AMODE</code>	Error related to the <code>amode</code> passed to <code>MPI_FILE_OPEN</code>
<code>MPI_ERR_UNSUPPORTED_DATAREP</code>	Unsupported <code>datarep</code> passed to <code>MPI_FILE_SET_VIEW</code>
<code>MPI_ERR_UNSUPPORTED_OPERATION</code>	Unsupported operation, such as seeking on a file that supports sequential access only
<code>MPI_ERR_NO_SUCH_FILE</code>	File does not exist
<code>MPI_ERR_FILE_EXISTS</code>	File exists
<code>MPI_ERR_BAD_FILE</code>	Invalid file name (e.g., path name too long)
<code>MPI_ERR_ACCESS</code>	Permission denied
<code>MPI_ERR_NO_SPACE</code>	Not enough space
<code>MPI_ERR_QUOTA</code>	Quota exceeded
<code>MPI_ERR_READ_ONLY</code>	Read-only file or file system
<code>MPI_ERR_FILE_IN_USE</code>	File operation could not be completed, as the file is currently opened by some process
<code>MPI_ERR_DUP_DATAREP</code>	Conversion functions could not be registered because a data representation identifier that was already defined was passed to <code>MPI_REGISTER_DATAREP</code>
<code>MPI_ERR_CONVERSION</code>	An error occurred in a user supplied data conversion function.
<code>MPI_ERR_IO</code>	Other I/O error

Example 15.9. This example shows how to overlap computation and output. The computation is performed by the function `compute_buffer()`.

```

/*=====
*
* Function:          double_buffer
*
* Synopsis:
*   void double_buffer(
*       MPI_File fh,                ** IN
*       MPI_Datatype buftype,       ** IN
*       int bufcount                ** IN
*   )
*
* Description:
*   Performs the steps to overlap computation with a collective write
*   by using a double-buffering technique.
*
* Parameters:
*   fh                previously opened MPI file handle
*   buftype            MPI datatype for memory layout
*                       (Assumes a compatible view has been set on fh)
*/

```

```

*      bufcount      # buftype elements to transfer
*-----*/
1
2
3
/* this macro switches which buffer "x" is pointing to */
4
#define TOGGLE_PTR(x) (((x)==(buffer1)) ? (x=buffer2) : (x=buffer1))
5
void double_buffer(MPI_File fh, MPI_Datatype buftype, int bufcount)
6
{
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
    MPI_Status status;      /* status for MPI calls */
    float *buffer1, *buffer2; /* buffers to hold results */
    float *compute_buf_ptr;   /* destination buffer */
                             /* for computing */
    float *write_buf_ptr;     /* source for writing */
    int done;                 /* determines when to quit */

    /* buffer initialization */
    buffer1 = (float *) malloc(bufcount*sizeof(float));
    buffer2 = (float *) malloc(bufcount*sizeof(float));
    compute_buf_ptr = buffer1; /* initially point to buffer1 */
    write_buf_ptr = buffer1;   /* initially point to buffer1 */

    /* DOUBLE-BUFFER prolog:
     *   compute buffer1; then initiate writing buffer1 to disk
     */
    compute_buffer(compute_buf_ptr, bufcount, &done);
    MPI_File_write_all_begin(fh, write_buf_ptr, bufcount, buftype);

    /* DOUBLE-BUFFER steady state:
     *   Overlap writing old results from buffer pointed to by write_buf_ptr
     *   with computing new results into buffer pointed to by compute_buf_ptr.
     *
     *   There is always one write-buffer and one compute-buffer in use
     *   during steady state.
     */
    while (!done) {
        TOGGLE_PTR(compute_buf_ptr);
        compute_buffer(compute_buf_ptr, bufcount, &done);
        MPI_File_write_all_end(fh, write_buf_ptr, &status);
        TOGGLE_PTR(write_buf_ptr);
        MPI_File_write_all_begin(fh, write_buf_ptr, bufcount, buftype);
    }

    /* DOUBLE-BUFFER epilog:
     *   wait for final write to complete.
     */
    MPI_File_write_all_end(fh, write_buf_ptr, &status);

    /* buffer cleanup */
    free(buffer1);
    free(buffer2);
}

```

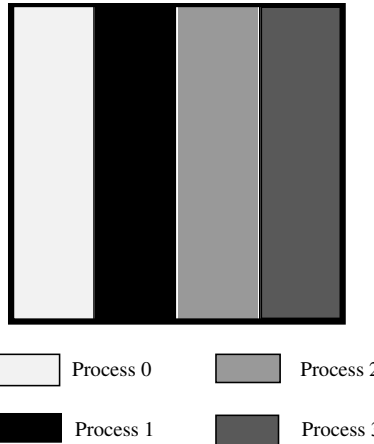


Figure 15.4: Example array file layout

15.9.2 Subarray Filetype Constructor

Example 15.10. Assume we are writing out a 100×100 2D array of double precision floating point numbers that is distributed among 4 processes such that each process has a block of 25 columns (e.g., process 0 has columns 0–24, process 1 has columns 25–49, etc.; see Figure 15.4). To create the filetypes for each process one could use the following C program (see Section 5.1.3):

```
double subarray[100][25];
MPI_Datatype filetype;
int sizes[2], subsizes[2], starts[2];
int rank;

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
sizes[0]=100; sizes[1]=100;
subsizes[0]=100; subsizes[1]=25;
starts[0]=0; starts[1]=rank*subsizes[1];

MPI_Type_create_subarray(2, sizes, subsizes, starts, MPI_ORDER_C,
MPI_DOUBLE, &filetype);
```

Or, equivalently in Fortran:

Example 15.11. Writing out a 100×100 2D array of double precision floating point numbers that is distributed among 4 processes such that each process has a block of 25 columns (e.g., process 0 has columns 0–24, process 1 has columns 25–49, etc.; see Figure 15.4).

```
double precision subarray(100,25)
integer filetype, rank, ierror
integer sizes(2), subsizes(2), starts(2)

call MPI_COMM_RANK(MPI_COMM_WORLD, rank, ierror)
sizes(1) = 100
sizes(2) = 100
subsizes(1) = 100
```

```
subsizes(2) = 25
starts(1)   = 0
starts(2)   = rank*subsizes(2)

call MPI_TYPE_CREATE_SUBARRAY(2, sizes, subsizes, starts, &
                             MPI_ORDER_FORTRAN, MPI_DOUBLE_PRECISION, &
                             filetype, ierror)
```

The generated filetype will then describe the portion of the file contained within the process's subarray with holes for the space taken by the other processes. Figure 15.5 shows the filetype created for process 1.

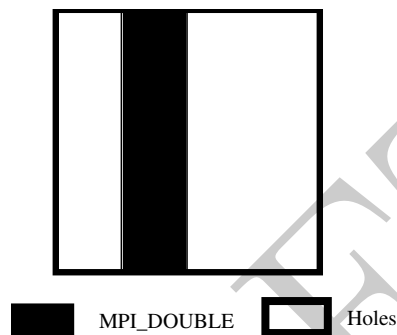


Figure 15.5: Example local array filetype for process 1

DRAFT

Chapter 16

Tool Support

16.1 Introduction

This chapter discusses interfaces that allow debuggers, performance analyzers, and other tools to extract information about the behavior of MPI processes. Specifically, this chapter defines both the MPI profiling interface (Section 16.2), which supports the transparent interception and inspection of MPI calls, and the MPI tool information interface (Section 16.3), which supports the inspection and manipulation of MPI control and performance variables, as well as the registration of callbacks for MPI library events. The interfaces described in this chapter are all defined in the context of an MPI process, i.e., are callable from the same code that invokes other MPI functions.

16.2 Profiling Interface

16.2.1 Requirements

To meet the requirements for the MPI profiling interface, an implementation of the MPI functions *must*

1. provide a mechanism through which all of the MPI defined functions, except those allowed as macros (See Section 2.6.4), may be accessed with a name shift. This requires, in C and Fortran, an alternate entry point name, with the prefix `PMPI_` for each MPI function in each provided language binding and language support method. For routines implemented as macros, it is still required that the `PMPI_` version be supplied and work as expected, but it is not possible to replace the `MPI_` version with a user-defined version at link time.

For Fortran, the different support methods cause several specific procedure names. Therefore, several profiling routines (with these specific procedure names) are needed for each Fortran MPI routine, as described in Section 21.1.5.

2. ensure that those MPI functions that are not replaced may still be linked into an executable image without causing name clashes.
3. document the implementation of different language bindings of the MPI interface if they are layered on top of each other, so that the profiler developer knows whether to implement the profile interface for each binding, or to economize by implementing it only for the lowest level routines.
4. where the implementation of different language bindings is done through a layered approach (e.g., the Fortran binding is a set of “wrapper” functions that call the C

implementation), ensure that these wrapper functions are separable from the rest of the library.

This separability is necessary to allow a separate profiling library to be correctly implemented, since (at least with Unix linker semantics) the profiling library must contain these wrapper functions if it is to perform as expected. This requirement allows the person who builds the profiling library to extract these functions from the original MPI library and add them into the profiling library without bringing along any other unnecessary code.

5. provide a no-op routine `MPI_PCONTROL` in the MPI library.

16.2.2 Discussion

The objective of the MPI profiling interface is to ensure that it is relatively easy for authors of profiling (and other similar) tools to interface their codes to MPI implementations on different machines.

Since MPI is a machine independent standard with many different implementations, it is unreasonable to expect that the authors of profiling tools for MPI will have access to the source code that implements MPI on any particular machine. It is therefore necessary to provide a mechanism by which the implementors of such tools can collect whatever performance information they wish *without* access to the underlying implementation.

We believe that having such an interface is important if MPI is to be attractive to end users, since the availability of many different tools will be a significant factor in attracting users to the MPI standard.

The profiling interface is just that, an interface. It says *nothing* about the way in which it is used. There is therefore no attempt to lay down what information is collected through the interface, or how the collected information is saved, filtered, or displayed.

While the initial impetus for the development of this interface arose from the desire to permit the implementation of profiling tools, it is clear that an interface like that specified may also prove useful for other purposes, such as “internetworking” multiple MPI implementations. Since all that is defined is an interface, there is no objection to it being used wherever it is useful.

As the issues being addressed here are intimately tied up with the way in which executable images are built, which may differ greatly on different machines, the examples given below should be treated solely as one way of implementing the objective of the MPI profiling interface. The actual requirements made of an implementation are those detailed in the Requirements section above, the whole of the rest of this section is only present as justification and discussion of the logic for those requirements.

Examples 16.1, 16.2, 16.3, and 16.4 show ways in which an implementation could be constructed to meet the requirements on a Unix system (there may be others that would be equally valid).

16.2.3 Logic of the Design

Provided that an MPI implementation meets the requirements above, it is possible for the implementor of the profiling system to intercept the MPI calls that are made by the user program. The profiling system implementor can then collect any required information

before calling the underlying MPI implementation (through its name shifted entry points) to achieve the desired effects.

Example 16.1. A wrapper to accumulate the total amount of data sent by the `MPI_SEND` function, along with the total elapsed time spent in the function.

```
static int totalBytes = 0;
static double totalTime = 0.0;

int MPI_Send(const void* buffer, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
{
    double tstart = MPI_Wtime();      /* Pass on all arguments */
    int size;
    int result = PMPI_Send(buffer, count, datatype, dest, tag, comm);

    totalTime += MPI_Wtime() - tstart; /* Compute time */

    MPI_Type_size(datatype, &size);   /* and size */
    totalBytes += count*size;

    return result;
}
```

16.2.4 MPI Library Implementation

If the MPI library is implemented in C on a Unix system, then there are various options, including the two presented here, for supporting the name-shift requirement. The choice between these two options depends partly on whether the linker and compiler support weak symbols.

If the compiler and linker support weak external symbols, then only a single library is required as the following example shows:

Example 16.2. Library implementation using weak symbols.

```
#pragma weak MPI_Example = PMPI_Example

int PMPI_Example(/* appropriate args */)
{
    /* Useful content */
}
```

The effect of this `#pragma` is to define the external symbol `MPI_Example` as a weak definition. This means that the linker will not complain if there is another definition of the symbol (for instance in the profiling library); however if no other definition exists, then the linker will use the weak definition.

In the absence of weak symbols then one possible solution would be to use the C macro preprocessor as the following example shows:

Example 16.3. Library implementation using C pre-processor macros.

```
#ifdef PROFILELIB
```

```

1  #   ifdef __STDC__
2  #       define FUNCTION(name) P##name
3  #   else
4  #       define FUNCTION(name) P/**/name
5  #   endif
6  #else
7  #       define FUNCTION(name) name
8  #endif

```

Each of the user visible functions in the library would then be declared thus

```

10 int FUNCTION(MPI_Example)(/* appropriate args */)
11 {
12     /* Useful content */
13 }

```

The same source file can then be compiled to produce both versions of the library, depending on the state of the PROFILELIB macro symbol.

It is required that the standard MPI library be built in such a way that the inclusion of MPI functions can be achieved one at a time. This may mean that each external function must reside in its own compilation unit. This is necessary so that the author of the profiling library need only define those MPI functions that need to be intercepted, references to any others being fulfilled by the normal MPI library.

Example 16.4.

The following example shows a potential link step when using the profiling interface.

```

25 % cc ... -lmyprof -lpmpi -lmpi

```

Here libmyprof.a contains the profiler functions that intercept some of the MPI functions, libpmpi.a contains the “name shifted” MPI functions, and libmpi.a contains the normal definitions of the MPI functions.

16.2.5 Complications

Multiple Counting

Since parts of the MPI library may themselves be implemented using more basic MPI functions (e.g., a portable implementation of the collective operations implemented using point-to-point communications), there is potential for profiling functions to be called from within an MPI function that was called from a profiling function. This could lead to “double counting” of the time spent in the inner routine. Since this effect could actually be useful under some circumstances (e.g., it might allow one to answer the question “How much time is spent in the point-to-point routines when they are called from collective functions?”), we have decided not to enforce any restrictions on the author of the MPI library that would overcome this. Therefore the author of the profiling library should be aware of this problem, and guard against it. In a single-threaded world this is easily achieved through use of a static variable in the profiling code that remembers if you are already inside a profiling routine. It becomes more complex in a multithreaded environment (as does the meaning of the times recorded).

Linker Oddities

The Unix linker traditionally operates in one pass: the effect of this is that functions from libraries are only included in the image if they are needed at the time the library is scanned. When combined with weak symbols, or multiple definitions of the same function, this can cause odd (and unexpected) effects.

Consider, for instance, an implementation of MPI in which the Fortran binding is achieved by using wrapper functions on top of the C implementation. The author of the profile library then assumes that it is reasonable only to provide profile functions for the C binding, since Fortran will eventually call these, and the cost of the wrappers is assumed to be small. However, if the wrapper functions are not in the profiling library, then none of the profiled entry points will be undefined when the profiling library is called. Therefore none of the profiling code will be included in the image. When the standard MPI library is scanned, the Fortran wrappers will be resolved, and will also pull in the base versions of the MPI functions. The overall effect is that the code will link successfully, but will not be profiled.

To overcome this we must ensure that the Fortran wrapper functions are included in the profiling version of the library. We ensure that this is possible by requiring that these be separable from the rest of the base MPI library. This allows them to be copied out of the base library and into the profiling one using a tool such as `ar`.

Fortran Support Methods

The different Fortran support methods and possible options for the support of subarrays (depending on whether the compiler can support `TYPE(*)`, `DIMENSION(..)` choice buffers) imply different specific procedure names for the same Fortran MPI routine. The rules and implications for the profiling interface are described in Section 21.1.5.

16.2.6 Multiple Levels of Interception

The scheme given here does not directly support the nesting of profiling functions, since it provides only a single alternative name for each MPI function. Consideration was given to an implementation that would allow multiple levels of call interception, however we were unable to construct an implementation of this that did not have the following disadvantages

- assuming a particular implementation language, and
- imposing a run time cost even when no profiling was taking place.

Since one of the objectives of MPI is to permit efficient, low latency implementations, and it is not the business of a standard to require a particular implementation language, we decided to accept the scheme outlined above.

Note, however, that it is possible to use the scheme above to implement a multi-level system, since the function called by the user may call many different profiling functions before calling the underlying MPI function. This capability has been demonstrated in the P^NMPI tool infrastructure [59].

16.2.7 Miscellaneous Control of Profiling

There is a clear requirement for the user code to be able to control the profiler dynamically at run time. This capability is normally used for (at least) the purposes of

- Enabling and disabling profiling depending on the state of the calculation.
- Flushing trace buffers at noncritical points in the calculation.
- Adding user events to a trace file.

These requirements are met by use of `MPI_PCONTROL`.

`MPI_PCONTROL(level, ...)`

IN level Profiling level (integer)

C binding

`int MPI_Pcontrol(const int level, ...)`

Fortran 2008 binding

`MPI_Pcontrol(level)`
 INTEGER, INTENT(IN) :: level

Fortran binding

`MPI_PCONTROL(LEVEL)`
 INTEGER LEVEL

MPI libraries themselves make no use of this routine, and simply return immediately to the user code. However the presence of calls to this routine allows a profiling package to be explicitly called by the user.

Since MPI has no control of the implementation of the profiling code, we are unable to specify precisely the semantics that will be provided by calls to `MPI_PCONTROL`. This vagueness extends to the number of arguments to the function, and their datatypes.

However to provide some level of portability of user codes to different profiling libraries, we request the following meanings for certain values of level.

- level=0 Profiling is disabled.
- level=1 Profiling is enabled at a normal default level of detail.
- level=2 Profile buffers are flushed, which may be a no-op in some profilers.
- All other values of level have profile library defined effects and additional arguments.

We also request that the default state after MPI has been initialized is for profiling to be enabled at the normal default level. (i.e., as if `MPI_PCONTROL` had just been called with the argument 1). This allows users to link with a profiling library and to obtain profile output without having to modify their source code at all.

The provision of `MPI_PCONTROL` as a no-op in the standard MPI library supports the collection of more detailed profiling information with source code that can still link against the standard MPI library.

16.3 The MPI Tool Information Interface

MPI implementations often use internal variables to control their behavior and performance and rely on internal events for their implementation. Understanding and manipulating these

variables and tracking these events can provide a more efficient execution environment or improve performance for many applications. This section describes the MPI tool information interface, which provides a mechanism for MPI implementors to expose variables, each of which represents a particular property, setting, or performance measurement from within the MPI implementation, as well as expose events that can be tracked by tools. The interface is split into three parts: the first part provides information about, and supports the setting of, control variables through which the MPI implementation tunes its configuration. The second part provides access to performance variables that can provide insight into internal performance information of the MPI implementation. The third part enables tools to query available events within an MPI implementation and register callbacks for them.

To avoid restrictions on the MPI implementation, the MPI tool information interface allows the implementation to specify which control variables, performance variables, and events exist. Additionally, the user of the MPI tool information interface can obtain meta-data about each available variable or event, such as its datatype, and a textual description. The MPI tool information interface provides the necessary routines to find all variables and events that exist in a particular MPI implementation; to query their properties; to retrieve descriptions about their meaning; to access and, if appropriate, to alter their values; and (in case of events) set callbacks triggered by them.

Variables, events, and categories across connected MPI processes with equivalent names are required to have the same meaning (see the definition of “equivalent” as related to strings in Section 16.3.3). Furthermore, enumerations with equivalent names across connected MPI processes are required to have the same meaning, but are allowed to comprise different enumeration items. Enumeration items that have equivalent names across connected MPI processes in enumerations with the same meaning must also have the same meaning. In order for variables and categories to have the same meaning, routines in the tools information interface that return details for those variables and categories have requirements on what parameters must be identical. These requirements are specified in their respective sections.

Rationale. The intent of requiring the same meaning for entities with equivalent names is to enforce consistency across connected MPI processes. For example, variables describing the number of packets sent on different types of network devices should have different names to reflect their potentially different meanings. (*End of rationale.*)

The MPI tool information interface can be used independently from the MPI communication functionality. In particular, the routines of this interface can be called before MPI is initialized and after MPI is finalized. In order to support this behavior cleanly, the MPI tool information interface uses separate initialization and finalization routines. All identifiers used in the MPI tool information interface have the prefix `MPI_T_`.

On success, all MPI tool information interface routines return `MPI_SUCCESS`, otherwise they return an appropriate and unique return code indicating the reason why the call was not successfully completed. Details on return codes can be found in Section 16.3.10. However, unsuccessful calls to the MPI tool information interface are not fatal and do not impact the execution of subsequent MPI routines.

Since the MPI tool information interface primarily focuses on tools and support libraries, MPI implementations are only required to provide C bindings for functions and constants introduced in this section. Except where otherwise noted, all conventions and principles governing the C bindings of the MPI API also apply to the MPI tool information interface, which is available by including the `mpi.h` header file. All routines in this interface have local semantics.

Table 16.1: MPI tool information interface verbosity levels

<code>MPI_T_VERBOSITY_USER_BASIC</code>	Basic information of interest to users
<code>MPI_T_VERBOSITY_USER_DETAIL</code>	Detailed information of interest to users
<code>MPI_T_VERBOSITY_USER_ALL</code>	All remaining information of interest to users
<code>MPI_T_VERBOSITY_TUNER_BASIC</code>	Basic information required for tuning
<code>MPI_T_VERBOSITY_TUNER_DETAIL</code>	Detailed information required for tuning
<code>MPI_T_VERBOSITY_TUNER_ALL</code>	All remaining information required for tuning
<code>MPI_T_VERBOSITY_MPIDEV_BASIC</code>	Basic information for MPI implementors
<code>MPI_T_VERBOSITY_MPIDEV_DETAIL</code>	Detailed information for MPI implementors
<code>MPI_T_VERBOSITY_MPIDEV_ALL</code>	All remaining information for MPI implementors

Advice to users. The number and type of control variables, performance variables, and events can vary between MPI implementations, platforms and different builds of the same implementation on the same platform as well as between runs. Hence, any application relying on a particular variable will not be portable. Further, there is no guarantee that the number of variables and variable indices are the same across connected MPI processes.

This interface is primarily intended for performance monitoring tools, support tools, and libraries controlling the application's environment. When maximum portability is desired, application programmers should either avoid using the MPI tool information interface or avoid being dependent on the existence of a particular control or performance variable or of a particular event. (*End of advice to users.*)

16.3.1 Verbosity Levels

The MPI tool information interface provides access to internal configuration and performance information through a set of control and performance variables defined by the MPI implementation. Since some implementations may export a large number of variables, variables are classified by a verbosity level that categorizes both their intended audience (end users, performance tuners or MPI implementors) and a relative measure of level of detail (basic, detailed or all). These verbosity levels are described by a single integer. Table 16.1 lists the constants for all possible verbosity levels. The values of the constants are monotonic in the order listed in the table; i.e., `MPI_T_VERBOSITY_USER_BASIC` < `MPI_T_VERBOSITY_USER_DETAIL` < ... < `MPI_T_VERBOSITY_MPIDEV_ALL`.

16.3.2 Binding MPI Tool Information Interface Variables to MPI Objects

Each MPI tool information interface variable provides access to a particular control setting or performance property of the MPI implementation. A variable may refer to a specific MPI object such as a communicator, datatype, or one-sided communication window, or the variable may refer more generally to the MPI environment of the process. Except for the last case, the variable must be bound to exactly one MPI object before it can be used. Table 16.2 lists all MPI object types to which an MPI tool information interface variable can be bound, together with the matching constant that MPI tool information interface routines return to identify the object type. It is erroneous to bind a control variable, performance variable, or

event to a handle that would not be valid to use as an input argument to another MPI call (excluding calls to the MPI Tool Information Interface) at the same point of execution.

Table 16.2: Constants to identify associations of variables

Constant	MPI object
MPI_T_BIND_NO_OBJECT	N/A; applies globally to entire MPI process
MPI_T_BIND_MPI_COMM	MPI communicator
MPI_T_BIND_MPI_DATATYPE	MPI datatype
MPI_T_BIND_MPI_ERRHANDLER	MPI error handler
MPI_T_BIND_MPI_FILE	MPI file handle
MPI_T_BIND_MPI_GROUP	MPI group
MPI_T_BIND_MPI_OP	MPI reduction operator
MPI_T_BIND_MPI_REQUEST	MPI request
MPI_T_BIND_MPI_WIN	MPI window
MPI_T_BIND_MPI_MESSAGE	MPI message object
MPI_T_BIND_MPI_INFO	MPI info object
MPI_T_BIND_MPI_SESSION	MPI session

Rationale. Some variables have meanings tied to a specific MPI object. Examples include the number of send or receive operations that use a particular datatype, the number of times a particular error handler has been called, or the communication protocol and “eager limit” used for a particular communicator. Creating a new MPI tool information interface variable for each MPI object would cause the number of variables to grow without bound, since they cannot be reused to avoid naming conflicts. By associating MPI tool information interface variables with a specific MPI object, the MPI implementation only must specify and maintain a single variable, which can then be applied to as many MPI objects of the respective type as created during the program’s execution. (*End of rationale.*)

16.3.3 Convention for Returning Strings

Several MPI tool information interface functions return one or more strings. These functions have two arguments for each string to be returned: an OUT parameter that identifies a pointer to the buffer in which the string will be returned, and an INOUT parameter to pass the length of the buffer. The user is responsible for the memory allocation of the buffer and must pass the size of the buffer (n) as the length argument. Let n be the length value specified to the function. On return, the function writes at most $n - 1$ of the string’s characters into the buffer, followed by a null terminator. If the returned string’s length is greater than or equal to n , the string will be truncated to $n - 1$ characters. In this case, the length of the string plus one (for the terminating null character) is returned in the length argument. If the user passes the null pointer as the buffer argument or passes 0 as the length argument, the function does not return the string and only returns the length of the string plus one in the length argument. If the user passes the null pointer as the length argument, the buffer argument is ignored and nothing is returned.

MPI implementations behave as if they have an internal character array that is copied to the output character array supplied by the user. Such output strings are only defined

to be equivalent if their notional source-internal character arrays are identical (up to and including the null terminator), even if the output string is truncated due to a small input length parameter n .

16.3.4 Initialization and Finalization

The MPI tool information interface requires a separate set of initialization and finalization routines.

MPI_T_INIT_THREAD(required, provided)

IN	required	desired level of thread support (integer)
OUT	provided	provided level of thread support (integer)

C binding

```
int MPI_T_init_thread(int required, int *provided)
```

All programs or tools that use the MPI tool information interface must initialize the MPI tool information interface in the processes that will use the interface before calling any other of its routines. A user can initialize the MPI tool information interface by calling **MPI_T_INIT_THREAD**, which can be called multiple times. In addition, this routine initializes the thread environment for all routines in the MPI tool information interface. Calling this routine when the MPI tool information interface is already initialized has no effect beyond increasing the reference count of how often the interface has been initialized. The argument **required** is used to specify the desired level of thread support. The possible values and their semantics are identical to the ones that can be used with **MPI_INIT_THREAD** listed in Section 11.6. The call returns in **provided** information about the actual level of thread support that will be provided by the MPI implementation for calls to MPI tool information interface routines. It can be one of the four values listed in Section 11.6.

The MPI specification does not require all MPI processes to exist before MPI is initialized. If the MPI tool information interface is used before initialization of MPI, the user is responsible for ensuring that the MPI tool information interface is initialized on all processes it is used in. Processes created by the MPI implementation during initialization inherit the status of the MPI tool information interface (whether it is initialized or not as well as all active sessions and handles) from the process from which they are created.

Processes created at runtime as a result of calls to MPI's dynamic process management require their own initialization before they can use the MPI tool information interface.

Advice to users. If **MPI_T_INIT_THREAD** is called before **MPI_INIT_THREAD**, the requested and provided thread level for **MPI_T_INIT_THREAD** may influence the behavior and return value of **MPI_INIT_THREAD**. The same is true for the reverse order. Likewise, when using the Sessions Model (Section 11.3), the requested and provided thread level for **MPI_T_INIT_THREAD** may influence the behavior and return values of **MPI_SESSION_INIT** (see Section 11.3), with the same being true for the reverse order. (*End of advice to users.*)

Advice to implementors. MPI implementations should strive to make as many control or performance variables available before MPI initialization (instead of adding them

during initialization) to allow tools the most flexibility. In particular, control variables should be available before MPI initialization if their value cannot be changed after MPI initialization. *(End of advice to implementors.)*

MPI_T_FINALIZE()

C binding

```
int MPI_T_finalize(void)
```

This routine finalizes the use of the MPI tool information interface and may be called as often as the corresponding `MPI_T_INIT_THREAD` routine up to the current point of execution. Calling it more times returns a corresponding return code. As long as the number of calls to `MPI_T_FINALIZE` is smaller than the number of calls to `MPI_T_INIT_THREAD` up to the current point of execution, the MPI tool information interface remains initialized and calls to its routines are permissible. Further, additional calls to `MPI_T_INIT_THREAD` after one or more calls to `MPI_T_FINALIZE` are permissible.

Once `MPI_T_FINALIZE` is called the same number of times as the routine `MPI_T_INIT_THREAD` up to the current point of execution, the MPI tool information interface is no longer initialized. The user can reinitialize the interface by a subsequent call to `MPI_T_INIT_THREAD`.

At the end of the program execution, unless `MPI_ABORT` is called, an application must have called `MPI_T_INIT_THREAD` and `MPI_T_FINALIZE` an equal number of times.

16.3.5 Datatype System

All variables managed through the MPI tool information interface represent their values through typed buffers of a given length and type using an MPI datatype (similar to regular send/receive buffers). Since the initialization of the MPI tool information interface is separate from the initialization of MPI, MPI tool information interface routines can be called before MPI initialization. Consequently, these routines can also use MPI datatypes before MPI initialization. Therefore, within the context of the MPI tool information interface, it is permissible to use a subset of MPI datatypes as specified below before MPI initialization.

Table 16.3: MPI datatypes that can be used by the MPI tool information interface

```
MPI_INT
MPI_INT32_T
MPI_INT64_T
MPI_UNSIGNED
MPI_UNSIGNED_LONG
MPI_UNSIGNED_LONG_LONG
MPI_UINT32_T
MPI_UINT64_T
MPI_COUNT
MPI_CHAR
MPI_DOUBLE
```

Rationale. The MPI tool information interface relies mainly on unsigned datatypes for integer values since most variables are expected to represent counters or resource sizes. `MPI_INT` is provided for additional flexibility and is expected to be used mainly for control variables and enumeration types (see below).

Providing all basic datatypes, in particular providing all signed and unsigned variants of integer types, would lead to a larger number of types, which tools need to interpret. This would cause unnecessary complexity in the implementation of tools based on the MPI tool information interface. (*End of rationale.*)

The MPI tool information interface only relies on a subset of the basic MPI datatypes and does not use any derived MPI datatypes. Table 16.3 lists all MPI datatypes that can be returned by the MPI tool information interface to represent its variables.

The use of the datatype `MPI_CHAR` in the MPI tool information interface implies a null-terminated character array, i.e., a string in the C language. If a variable has type `MPI_CHAR`, the value of the count parameter returned by `MPI_T_CVAR_HANDLE_ALLOC` and `MPI_T_PVAR_HANDLE_ALLOC` must be large enough to include any valid value, including its terminating null character. The contents of returned `MPI_CHAR` arrays are only defined from index 0 through the location of the first null character.

Rationale. The MPI tool information interface requires a significantly simpler type system than MPI itself. Therefore, only its required subset must be present before MPI initialization and MPI implementations do not need to initialize the complete MPI datatype system. (*End of rationale.*)

For variables of type `MPI_INT`, an MPI implementation can provide additional information by associating names with a fixed number of values. We refer to this information in the following as an enumeration. In this case, the respective calls that provide additional metadata for each control or performance variable, i.e., `MPI_T_CVAR_GET_INFO` (Section 16.3.6), `MPI_T_PVAR_GET_INFO` (Section 16.3.7), and `MPI_T_EVENT_GET_INFO` (Section 16.3.8), return a handle of type `MPI_T_enum` that can be passed to the following functions to extract additional information. Thus, the MPI implementation can describe variables with a fixed set of values that each represents a particular state. Each enumeration type can have N different values, with a fixed N that can be queried using `MPI_T_ENUM_GET_INFO`.

`MPI_T_ENUM_GET_INFO(enumtype, num, name, name_len)`

IN	enumtype	enumeration to be queried (handle)
OUT	num	number of discrete values represented by this enumeration (integer)
OUT	name	buffer to return the string containing the name of the enumeration item (string)
INOUT	name_len	length of the string and/or buffer for name (integer)

C binding

```
int MPI_T_enum_get_info(MPI_T_enum enumtype, int *num, char *name,
                        int *name_len)
```

If `enumtype` is a valid enumeration, this routine returns the number of items represented by this enumeration type as well as its name. N must be greater than 0, i.e., the enumeration must represent at least one value.

The arguments `name` and `name_len` are used to return the name of the enumeration as described in Section 16.3.3.

The routine is required to return a name of at least length one. This name must be unique with respect to all other names for enumerations that the MPI implementation uses.

Names associated with individual values in each enumeration `enumtype` can be queried using `MPI_T_ENUM_GET_ITEM`.

`MPI_T_ENUM_GET_ITEM(enumtype, index, value, name, name_len)`

IN	<code>enumtype</code>	enumeration to be queried (handle)
IN	<code>index</code>	number of the value to be queried in this enumeration (integer)
OUT	<code>value</code>	variable value (integer)
OUT	<code>name</code>	buffer to return the string containing the name of the enumeration item (string)
INOUT	<code>name_len</code>	length of the string and/or buffer for name (integer)

C binding

```
int MPI_T_enum_get_item(MPI_T_enum enumtype, int index, int *value, char *name,
                        int *name_len)
```

The arguments `name` and `name_len` are used to return the name of the enumeration item as described in Section 16.3.3.

If completed successfully, the routine returns the name/value pair that describes the enumeration at the specified index. The call is further required to return a name of at least length one. This name must be unique with respect to all other names of items for the same enumeration.

16.3.6 Control Variables

The routines described in this section of the MPI tool information interface specification focus on the ability to list, query, and possibly set control variables exposed by the MPI implementation. These variables can typically be used by the user to fine tune properties and configuration settings of the MPI implementation. On many systems, such variables can be set using environment variables, although other configuration mechanisms may be available, such as configuration files or central configuration registries. A typical example that is available in several existing MPI implementations is the ability to specify an “eager limit,” i.e., an upper bound on the size of messages sent or received using an eager protocol.

Control Variable Query Functions

An MPI implementation exports a set of N control variables through the MPI tool information interface. If N is zero, then the MPI implementation does not export any control variables, otherwise the provided control variables are indexed from 0 to $N - 1$. This index number is used in subsequent calls to identify the individual variables.

An MPI implementation is allowed to increase the number of control variables during the execution of an MPI application when new variables become available through dynamic loading. However, MPI implementations are not allowed to change the index of a control variable or to delete a variable once it has been added to the set. When a variable becomes inactive, e.g., through dynamic unloading, accessing its value should return a corresponding return code.

Advice to users. While the MPI tool information interface guarantees that indices or variable properties do not change during a particular run of an MPI program, it does not provide a similar guarantee between runs. (*End of advice to users.*)

The following function can be used to query the number of control variables, `num_cvar`:

```
MPI_T_CVAR_GET_NUM(num_cvar)
```

OUT	num_cvar	returns number of control variables (integer)
-----	----------	---

C binding

```
int MPI_T_cvar_get_num(int *num_cvar)
```

The function `MPI_T_CVAR_GET_INFO` provides access to additional information for each variable.

```
MPI_T_CVAR_GET_INFO(cvar_index, name, name_len, verbosity, datatype, enumtype,
                    desc, desc_len, bind, scope)
```

IN	cvar_index	index of the control variable to be queried, value between 0 and <code>num_cvar - 1</code> (integer)
OUT	name	buffer to return the string containing the name of the control variable (string)
INOUT	name_len	length of the string and/or buffer for <code>name</code> (integer)
OUT	verbosity	verbosity level of this variable (integer)
OUT	datatype	MPI datatype of the information stored in the control variable (handle)
OUT	enumtype	optional descriptor for enumeration information (handle)
OUT	desc	buffer to return the string containing a description of the control variable (string)
INOUT	desc_len	length of the string and/or buffer for <code>desc</code> (integer)
OUT	bind	type of MPI object to which this variable must be bound (integer)
OUT	scope	scope of when changes to this variable are possible (integer)

C binding

```
int MPI_T_cvar_get_info(int cvar_index, char *name, int *name_len,
                      int *verbosity, MPI_Datatype *datatype, MPI_T_enum *enumtype,
                      char *desc, int *desc_len, int *bind, int *scope)
```

After a successful call to `MPI_T_CVAR_GET_INFO` for a particular variable, subsequent calls to this routine that query information about the same variable must return the same information. An MPI implementation is not allowed to alter any of the returned values.

If any OUT parameter to `MPI_T_CVAR_GET_INFO` is a NULL pointer, the implementation will ignore the parameter and not return a value for the parameter.

The arguments `name` and `name_len` are used to return the name of the control variable as described in Section 16.3.3.

If completed successfully, the routine is required to return a name of at least length one. The name must be unique with respect to all other names for control variables used by the MPI implementation.

The argument `verbosity` returns the verbosity level of the variable (see Section 16.3.1).

The argument `datatype` returns the MPI datatype that is used to represent the control variable.

If the variable is of type `MPI_INT`, MPI can optionally specify an enumeration for the values represented by this variable and return it in `enumtype`. In this case, MPI returns an enumeration identifier, which can then be used to gather more information as described in Section 16.3.5. Otherwise, `enumtype` is set to `MPI_T_ENUM_NULL`. If the datatype is not `MPI_INT` or the argument `enumtype` is the null pointer, no enumeration type is returned.

The arguments `desc` and `desc_len` are used to return a description of the control variable as described in Section 16.3.3.

Returning a description is optional. If an MPI implementation does not return a description, the first character for `desc` must be set to the null character and `desc_len` must be set to one at the return of this call.

The parameter `bind` returns the type of the MPI object to which the variable must be bound or the value `MPI_T_BIND_NO_OBJECT` (see Section 16.3.2).

The scope of a variable determines whether changing a variable's value is either local to the MPI process or must be done by the user across multiple connected MPI processes. The latter is further split into variables that require changes in a group of MPI processes and those that require collective changes among all connected MPI processes. Both cases can require variables on all participating MPI processes either to be set to consistent (but potentially different) values or to equal values. The description provided with the variable must contain an explanation about the requirements and/or restrictions for setting the particular variable.

On successful return from `MPI_T_CVAR_GET_INFO`, the argument `scope` will be set to one of the constants listed in Table 16.4.

If the name of a control variable is equivalent across connected MPI processes, the following OUT parameters must be identical: `verbosity`, `datatype`, `enumtype`, `bind`, and `scope`. The returned description must be equivalent.

Advice to users. The `scope` of a variable only indicates if a variable might be changeable; it is not a guarantee that it can be changed at any time. (*End of advice to users.*)

`MPI_T_CVAR_GET_INDEX(name, cvar_index)`

IN	<code>name</code>	name of the control variable (string)
OUT	<code>cvar_index</code>	index of the control variable (integer)

Table 16.4: Scopes for control variables

Scope Constant	Description
MPI_T_SCOPE_CONSTANT	read-only, value is constant
MPI_T_SCOPE_READONLY	read-only, cannot be written, but can change
MPI_T_SCOPE_LOCAL	may be writeable, writing only affects the calling MPI process
MPI_T_SCOPE_GROUP	may be writeable, must be set to consistent values across a group of connected MPI processes
MPI_T_SCOPE_GROUP_EQ	may be writeable, must be set to the same value across a group of connected MPI processes
MPI_T_SCOPE_ALL	may be writeable, must be set to consistent values across all connected MPI processes
MPI_T_SCOPE_ALL_EQ	may be writeable, must be set to the same value across all connected MPI processes

C binding

```
int MPI_T_cvar_get_index(const char *name, int *cvar_index)
```

MPI_T_CVAR_GET_INDEX is a function for retrieving the index of a control variable given a known variable name. The `name` parameter is provided by the caller, and `cvar_index` is returned by the MPI implementation. The `name` parameter is a string terminated with a null character.

This routine returns **MPI_SUCCESS** on success and returns **MPI_T_ERR_INVALID_NAME** if `name` does not match the name of any control variable provided by the implementation at the time of the call.

Rationale. This routine is provided to enable fast retrieval of control variables by a tool, assuming it knows the name of the variable for which it is looking. The number of variables exposed by the implementation can change over time, so it is not possible for the tool to simply iterate over the list of variables once at initialization. Although using MPI implementation specific variable names is not portable across MPI implementations, tool developers may choose to take this route for lower overhead at runtime because the tool will not have to iterate over the entire set of variables to find a specific one. (*End of rationale.*)

Example 16.5. Querying and printing the names of all available control variables.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int i, err, num, namelen, bind, verbose, scope;
    int threadsupport;
    char name[100];

    MPI_Datatype datatype;
```



```

err=MPI_T_init_thread(MPI_THREAD_SINGLE, &threadsupport);
if (err!=MPI_SUCCESS)
    return err;

err=MPI_T_cvar_get_num(&num);
if (err!=MPI_SUCCESS)
    return err;

for (i=0; i<num; i++) {
    namelen=100;
    err=MPI_T_cvar_get_info(i, name, &namelen,
                           &verbose, &datatype, NULL,
                           NULL, NULL, /*no description */
                           &bind, &scope);
    if (err!=MPI_SUCCESS && err!=MPI_T_ERR_INVALID_INDEX)
        return err;
    printf("Var %i: %s\n", i, name);
}

err=MPI_T_finalize();
if (err!=MPI_SUCCESS)
    return 1;
else
    return 0;
}

```

Handle Allocation and Deallocation

Before reading or writing the value of a variable, a user must first allocate a handle of type `MPI_T_cvar_handle` for the variable by binding it to an MPI object (see also Section 16.3.2).

Rationale. Handles used in the MPI tool information interface are distinct from handles used in the remaining parts of the MPI standard because they must be usable before MPI is initialized and after MPI is finalized. Further, accessing handles, in particular for performance variables, can be time critical and having a separate handle space enables optimizations. (*End of rationale.*)

`MPI_T_CVAR_HANDLE_ALLOC(cvar_index, obj_handle, handle, count)`

IN	<code>cvar_index</code>	index of control variable for which handle is to be allocated (index)
IN	<code>obj_handle</code>	reference to a handle of the MPI object to which this variable is supposed to be bound (pointer)
OUT	<code>handle</code>	allocated handle (handle)
OUT	<code>count</code>	number of elements used to represent this variable (integer)

C binding

```

int MPI_T_cvar_handle_alloc(int cvar_index, void *obj_handle,
                           MPI_T_cvar_handle *handle, int *count)

```

This routine binds the control variable specified by the argument `index` to an MPI object. The object is passed in the argument `obj_handle` as an address to a local variable that stores the object's handle. The argument `obj_handle` is ignored if the `MPI_T_CVAR_GET_INFO` call for this control variable returned `MPI_T_BIND_NO_OBJECT` in the argument `bind`. The handle allocated to reference the variable is returned in the argument `handle`. Upon successful return, `count` contains the number of elements (of the datatype returned by a previous `MPI_T_CVAR_GET_INFO` call) used to represent this variable.

Advice to users. The `count` can be different based on the MPI object to which the control variable was bound. For example, variables bound to communicators could have a count that matches the size of the communicator.

It is not portable to pass references to predefined MPI object handles, such as `MPI_COMM_WORLD` to this routine, since their implementation depends on the MPI library. Instead, such object handles should be stored in a local variable and the address of this local variable should be passed into `MPI_T_CVAR_HANDLE_ALLOC`. (*End of advice to users.*)

The value of `cvar_index` should be in the range from 0 to `num_cvar - 1`, where `num_cvar` is the number of available control variables as determined from a prior call to `MPI_T_CVAR_GET_NUM`. The type of the MPI object it references must be consistent with the type returned in the `bind` argument in a prior call to `MPI_T_CVAR_GET_INFO`.

`MPI_T_CVAR_HANDLE_FREE(handle)`

INOUT	handle	handle to be freed (handle)
-------	--------	-----------------------------

C binding

`int MPI_T_cvar_handle_free(MPI_T_cvar_handle *handle)`

When a handle is no longer needed, a user of the MPI tool information interface should call `MPI_T_CVAR_HANDLE_FREE` to free the handle and the associated resources in the MPI implementation. On a successful return, MPI sets the handle to `MPI_T_CVAR_HANDLE_NULL`.

Control Variable Access Functions

`MPI_T_CVAR_READ(handle, buf)`

IN	handle	handle to the control variable to be read (handle)
OUT	buf	initial address of storage location for variable value (choice)

C binding

`int MPI_T_cvar_read(MPI_T_cvar_handle handle, void *buf)`

This routine queries the value of a control variable identified by the argument `handle` and stores the result in the buffer identified by the parameter `buf`. The user must ensure that the

buffer is of the appropriate size to hold the entire value of the control variable (based on the returned datatype and count from prior corresponding calls to `MPI_T_CVAR_GET_INFO` and `MPI_T_CVAR_HANDLE_ALLOC`, respectively).

`MPI_T_CVAR_WRITE(handle, buf)`

INOUT	handle	handle to the control variable to be written (handle)
IN	buf	initial address of storage location for variable value (choice)

C binding

`int MPI_T_cvar_write(MPI_T_cvar_handle handle, const void *buf)`

This routine sets the value of the control variable identified by the argument `handle` to the data stored in the buffer identified by the parameter `buf`. The user must ensure that the buffer is of the appropriate size to hold the entire value of the control variable (based on the returned datatype and count from prior corresponding calls to `MPI_T_CVAR_GET_INFO` and `MPI_T_CVAR_HANDLE_ALLOC`, respectively).

If the variable has a global scope (as returned by a prior corresponding `MPI_T_CVAR_GET_INFO` call), any write call to this variable must be issued by the user in all connected (as defined in Section 11.10.4) MPI processes. If the variable has group scope, any write call to this variable must be issued by the user in all MPI processes in the group, which must be described by the MPI implementation in the description returned by the call to `MPI_T_CVAR_GET_INFO`.

In both cases, the user must ensure that the writes in all participating MPI processes are consistent. If the scope is either `MPI_T_SCOPE_ALL_EQ` or `MPI_T_SCOPE_GROUP_EQ` this means that the variable in all connected MPI processes or MPI processes of the group, respectively, must be set to the same value.

If it is not possible to change the variable at the time the call is made, the function returns either `MPI_T_ERR_CVAR_SET_NOT_NOW`, if there may be a later time at which the variable could be set, or `MPI_T_ERR_CVAR_SET_NEVER`, if the variable cannot be set for the remainder of the application's execution.

Example 16.6. Reading the value of a control variable.

```
int getValue_int_comm(int index, MPI_Comm comm, int *val) {
    int err, count;
    MPI_T_cvar_handle handle;

    /* This example assumes that the variable index */
    /* can be bound to a communicator */

    err=MPI_T_cvar_handle_alloc(index, &comm, &handle, &count);
    if (err!=MPI_SUCCESS)
        return err;

    /* The following assumes that the variable is */
    /* represented by a single integer */

    err=MPI_T_cvar_read(handle, val);
```

```

1  if (err!=MPI_SUCCESS)
2      return err;
3
4  err=MPI_T_cvar_handle_free(&handle);
5  return err;
6  }

```

16.3.7 Performance Variables

The following section focuses on the ability to list and to query performance variables provided by the MPI implementation. Performance variables provide insight into MPI implementation-specific internals and can represent information such as the state of the MPI implementation (e.g., waiting blocked, receiving, not active), aggregated timing data for submodules, or queue sizes and lengths.

Rationale. The interface for performance variables is separate from the interface for control variables, since performance variables have different requirements and parameters. By keeping them separate, the interface provides cleaner semantics and allows for more performance optimization opportunities. (*End of rationale.*)

Some performance variables and classes refer to **events**. In general, such events describe state transitions within software or hardware related to the performance of an MPI application. The events offered through the callback-driven event-notification interface described in Section 16.3.8 also refer to such state transitions; however, the set of state transitions referred to by performance variables and events as described in Section 16.3.8 may not be identical.

Performance Variable Classes

Each performance variable is associated with a class that describes its basic semantics, possible datatypes, basic behavior, its starting value, whether it can overflow, and when and how an MPI implementation can change the variable's value. The starting value is the value that is assigned to the variable the first time that it is used or whenever it is reset.

Advice to users. If a performance variable belongs to a class that can overflow, it is up to the user to protect against this overflow, e.g., by frequently reading and resetting the variable value. (*End of advice to users.*)

Advice to implementors. MPI implementations should use large enough datatypes for each performance variable to avoid overflows under normal circumstances. (*End of advice to implementors.*)

The classes are defined by the following constants:

MPI_T_PVAR_CLASS_STATE: A performance variable in this class represents a set of discrete states. Variables of this class are represented by **MPI_INT** and can be set by the MPI implementation at any time. Variables of this type should be described further using an enumeration, as discussed in Section 16.3.5. The starting value is the current state of the implementation at the time that the starting value is set. MPI implementations must ensure that variables of this class cannot overflow.

MPI_T_PVAR_CLASS_LEVEL: A performance variable in this class represents a value that describes the utilization level of a resource. The value of a variable of this class can change at any time to match the current utilization level of the resource. Values returned from variables in this class are nonnegative and represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`, `MPI_DOUBLE`. The starting value is the current utilization level of the resource at the time that the starting value is set. MPI implementations must ensure that variables of this class cannot overflow.

MPI_T_PVAR_CLASS_SIZE: A performance variable in this class represents a value that is the size of a resource. Values returned from variables in this class are nonnegative and represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`, `MPI_DOUBLE`. The starting value is the current size of the resource at the time that the starting value is set. MPI implementations must ensure that variables of this class cannot overflow.

MPI_T_PVAR_CLASS_PERCENTAGE: The value of a performance variable in this class represents the percentage utilization of a finite resource. The value of a variable of this class can change at any time to match the current utilization level of the resource. It will be returned as an `MPI_DOUBLE` datatype. The value must always be between 0.0 (resource not used at all) and 1.0 (resource completely used). The starting value is the current percentage utilization level of the resource at the time that the starting value is set. MPI implementations must ensure that variables of this class cannot overflow.

MPI_T_PVAR_CLASS_HIGHWATERMARK: A performance variable in this class represents a value that describes the maximum observed utilization of a resource. The value of a variable of this class is nonnegative and grows monotonically from the initialization or reset of the variable. It can be represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`, `MPI_DOUBLE`. The starting value is the current utilization level of the resource at the time that the variable is started or reset. MPI implementations must ensure that variables of this class cannot overflow.

MPI_T_PVAR_CLASS_LOWWATERMARK: A performance variable in this class represents a value that describes the minimum observed utilization of a resource. The value of a variable of this class is nonnegative and decreases monotonically from the initialization or reset of the variable. It can be represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`, `MPI_DOUBLE`. The starting value is the current utilization level of the resource at the time that the variable is started or reset. MPI implementations must ensure that variables of this class cannot overflow.

MPI_T_PVAR_CLASS_COUNTER: A performance variable in this class counts the number of occurrences of a specific event (e.g., the number of memory allocations within an MPI library). The value of a variable of this class increases monotonically from the initialization or reset of the performance variable by one for each specific event that is observed. Values must be nonnegative and represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`. The starting value for variables of this class is 0. Variables of this class can overflow.

MPI_T_PVAR_CLASS_AGGREGATE: The value of a performance variable in this class is an aggregated value that represents a sum of arguments processed during a specific event (e.g., the amount of memory allocated by all memory allocations). This class is similar to the counter class, but instead of counting individual events, the value can be incremented by arbitrary amounts. The value of a variable of this class increases monotonically from the initialization or reset of the performance variable. It must be nonnegative and represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`, `MPI_DOUBLE`. The starting value for variables of this class is 0. Variables of this class can overflow.

MPI_T_PVAR_CLASS_TIMER: The value of a performance variable in this class represents the aggregated time that the MPI implementation spends executing a particular event, type of event, or section of the MPI library. This class has the same basic semantics as `MPI_T_PVAR_CLASS_AGGREGATE`, but explicitly records a timing value. The value of a variable of this class increases monotonically from the initialization or reset of the performance variable. It must be nonnegative and represented by one of the following datatypes: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`, `MPI_UNSIGNED_LONG_LONG`, `MPI_DOUBLE`. The starting value for variables of this class is 0. If the type `MPI_DOUBLE` is used, the units that represent time in this datatype must match the units used by `MPI_WTIME`. Otherwise, the time units should be documented, e.g., in the description returned by `MPI_T_PVAR_GET_INFO`. Variables of this class can overflow.

MPI_T_PVAR_CLASS_GENERIC: This class can be used to describe a variable that does not fit into any of the other classes. For variables in this class, the starting value is variable-specific and implementation-defined.

Performance Variable Query Functions

An MPI implementation exports a set of N performance variables through the MPI tool information interface. If N is zero, then the MPI implementation does not export any performance variables; otherwise the provided performance variables are indexed from 0 to $N - 1$. This index number is used in subsequent calls to identify the individual variables.

An MPI implementation is allowed to increase the number of performance variables during the execution of an MPI application when new variables become available through dynamic loading. However, MPI implementations are not allowed to change the index of a performance variable or to delete a variable once it has been added to the set. When a variable becomes inactive, e.g., through dynamic unloading, accessing its value should return a corresponding return code.

The following function can be used to query the number of performance variables, `num_pvar`:

```
MPI_T_PVAR_GET_NUM(num_pvar)
```

```
OUT      num_pvar          returns number of performance variables (integer)
```

C binding

```
int MPI_T_pvar_get_num(int *num_pvar)
```

The function `MPI_T_PVAR_GET_INFO` provides access to additional information for each variable.

```
MPI_T_PVAR_GET_INFO(pvar_index, name, name_len, verbosity, var_class, datatype,
                    enumtype, desc, desc_len, bind, readonly, continuous, atomic)
```

IN	<code>pvar_index</code>	index of the performance variable to be queried between 0 and <code>num_pvar - 1</code> (integer)
OUT	<code>name</code>	buffer to return the string containing the name of the performance variable (string)
INOUT	<code>name_len</code>	length of the string and/or buffer for <code>name</code> (integer)
OUT	<code>verbosity</code>	verbosity level of this variable (integer)
OUT	<code>var_class</code>	class of performance variable (integer)
OUT	<code>datatype</code>	MPI datatype of the information stored in the performance variable (handle)
OUT	<code>enumtype</code>	optional descriptor for enumeration information (handle)
OUT	<code>desc</code>	buffer to return the string containing a description of the performance variable (string)
INOUT	<code>desc_len</code>	length of the string and/or buffer for <code>desc</code> (integer)
OUT	<code>bind</code>	type of MPI object to which this variable must be bound (integer)
OUT	<code>readonly</code>	flag indicating whether the variable can be written/reset (integer)
OUT	<code>continuous</code>	flag indicating whether the variable can be started and stopped or is continuously active (integer)
OUT	<code>atomic</code>	flag indicating whether the variable can be atomically read and reset (integer)

C binding

```
int MPI_T_pvar_get_info(int pvar_index, char *name, int *name_len,
                       int *verbosity, int *var_class, MPI_Datatype *datatype,
                       MPI_T_enum *enumtype, char *desc, int *desc_len, int *bind,
                       int *readonly, int *continuous, int *atomic)
```

After a successful call to `MPI_T_PVAR_GET_INFO` for a particular variable, subsequent calls to this routine that query information about the same variable must return the same information. An MPI implementation is not allowed to alter any of the returned values.

If any OUT parameter to `MPI_T_PVAR_GET_INFO` is a NULL pointer, the implementation will ignore the parameter and not return a value for the parameter.

The arguments `name` and `name_len` are used to return the name of the performance variable as described in Section 16.3.3. If completed successfully, the routine is required to return a name of at least length one.

The argument `verbosity` returns the verbosity level of the variable (see Section 16.3.1).

The class of the performance variable is returned in the parameter `var_class`. The class must be one of the constants defined in Section 16.3.7.

The combination of the name and the class of the performance variable must be unique with respect to all other names for performance variables used by the MPI implementation.

Advice to implementors. Groups of variables that belong closely together, but have different classes, can have the same name. This choice is useful, e.g., to refer to multiple variables that describe a single resource (like the level, the total size, as well as high- and low-water marks). (*End of advice to implementors.*)

The argument `datatype` returns the MPI datatype that is used to represent the performance variable.

If the variable is of type `MPI_INT`, MPI can optionally specify an enumeration for the values represented by this variable and return it in `enumtype`. In this case, MPI returns an enumeration identifier, which can then be used to gather more information as described in Section 16.3.5. Otherwise, `enumtype` is set to `MPI_T_ENUM_NULL`. If the datatype is not `MPI_INT` or the argument `enumtype` is the null pointer, no enumeration type is returned.

Returning a description is optional. If an MPI implementation does not return a description, the first character for `desc` must be set to the null character and `desc_len` must be set to one at the return from this function.

The parameter `bind` returns the type of the MPI object to which the variable must be bound or the value `MPI_T_BIND_NO_OBJECT` (see Section 16.3.2).

Upon return, the argument `readonly` is set to zero if the variable can be written or reset by the user. It is set to one if the variable can only be read.

Upon return, the argument `continuous` is set to zero if the variable can be started and stopped by the user, i.e., it is possible for the user to control if and when the value of a variable is updated. It is set to one if the variable is always active and cannot be controlled by the user.

Upon return, the argument `atomic` is set to zero if the variable cannot be read and reset atomically. Only variables for which the call sets `atomic` to one can be used in a call to `MPI_T_PVAR_READRESET`.

If a performance variable has an equivalent name and has the same class across connected MPI processes, the following OUT parameters must be identical: `verbosity`, `varclass`, `datatype`, `enumtype`, `bind`, `readonly`, `continuous`, and `atomic`. The returned description must be equivalent.

`MPI_T_PVAR_GET_INDEX(name, var_class, pvar_index)`

IN	<code>name</code>	the name of the performance variable (string)
IN	<code>var_class</code>	the class of the performance variable (integer)
OUT	<code>pvar_index</code>	the index of the performance variable (integer)

C binding

`int MPI_T_pvar_get_index(const char *name, int var_class, int *pvar_index)`

`MPI_T_PVAR_GET_INDEX` is a function for retrieving the index of a performance variable given a known variable name and class. The `name` and `var_class` parameters are provided by the caller, and `pvar_index` is returned by the MPI implementation. The `name` parameter is a string terminated with a null character.

This routine returns `MPI_SUCCESS` on success and returns `MPI_T_ERR_INVALID_NAME` if `name` does not match the name of any performance variable of the specified `var_class` provided by the implementation at the time of the call.

Rationale. This routine is provided to enable fast retrieval of performance variables by a tool, assuming it knows the name of the variable for which it is looking. The number of variables exposed by the implementation can change over time, so it is not possible for the tool to simply iterate over the list of variables once at initialization. Although using MPI implementation specific variable names is not portable across MPI implementations, tool developers may choose to take this route for lower overhead at runtime because the tool will not have to iterate over the entire set of variables to find a specific one. (*End of rationale.*)

Performance Experiment Sessions

Within a single program, multiple components can use the MPI tool information interface. To avoid collisions with respect to accesses to performance variables, users of the MPI tool information interface must first create a performance experiment session. Subsequent calls that access performance variables can then be made within the context of this performance experiment session. Starting, stopping, reading, writing, or resetting a variable in one performance experiment session shall not influence whether a variable is started, stopped, read, written, or reset in another performance experiment session.

`MPI_T_PVAR_SESSION_CREATE(pe_session)`

OUT `pe_session` identifier of performance experiment session (handle)

C binding

`int MPI_T_pvar_session_create(MPI_T_pvar_session *pe_session)`

This call creates a new performance experiment session for accessing performance variables and returns a handle for this performance experiment session in the argument `pe_session` of type `MPI_T_pvar_session`.

`MPI_T_PVAR_SESSION_FREE(pe_session)`

INOUT `pe_session` identifier of performance experiment session (handle)

C binding

`int MPI_T_pvar_session_free(MPI_T_pvar_session *pe_session)`

This call frees an existing performance experiment session. Calls to the MPI tool information interface can no longer be made within the context of a performance experiment session after it is freed. On a successful return, MPI sets the performance experiment session identifier to `MPI_T_PVAR_SESSION_NULL`.

Handle Allocation and Deallocation

Before using a performance variable, a user must first allocate a handle of type `MPI_T_pvar_handle` for the variable by binding it to an MPI object (see also Section 16.3.2).

```

1 MPI_T_PVAR_HANDLE_ALLOC(pe_session, pvar_index, obj_handle, handle, count)
2
3     INOUT    pe_session          identifier of performance experiment session (handle)
4     IN       pvar_index         index of performance variable for which handle is to
5                                     be allocated (integer)
6     IN       obj_handle         reference to a handle of the MPI object to which this
7                                     variable is supposed to be bound (pointer)
8     OUT      handle             allocated handle (handle)
9     OUT      count              number of elements used to represent this variable
10                                     (integer)

```

C binding

```

12 int MPI_T_pvar_handle_alloc(MPI_T_pvar_session pe_session, int pvar_index,
13                             void *obj_handle, MPI_T_pvar_handle *handle, int *count)
14

```

This routine binds the performance variable specified by the argument `index` to an MPI object in the performance experiment session identified by the parameter `pe_session`. The object is passed in the argument `obj_handle` as an address to a local variable that stores the object's handle. The argument `obj_handle` is ignored if the `MPI_T_PVAR_GET_INFO` call for this performance variable returned `MPI_T_BIND_NO_OBJECT` in the argument `bind`. The handle allocated to reference the variable is returned in the argument `handle`. Upon successful return, `count` contains the number of elements (of the datatype returned by a previous `MPI_T_PVAR_GET_INFO` call) used to represent this variable.

Advice to users. The `count` can be different based on the MPI object to which the performance variable was bound. For example, variables bound to communicators could have a count that matches the size of the communicator.

It is not portable to pass references to predefined MPI object handles, such as `MPI_COMM_WORLD`, to this routine, since their implementation depends on the MPI library. Instead, such an object handle should be stored in a local variable and the address of this local variable should be passed into `MPI_T_PVAR_HANDLE_ALLOC`. (*End of advice to users.*)

The value of `index` should be in the range from 0 to `num_pvar - 1`, where `num_pvar` is the number of available performance variables as determined from a prior call to `MPI_T_PVAR_GET_NUM`. The type of the MPI object it references must be consistent with the type returned in the `bind` argument in a prior call to `MPI_T_PVAR_GET_INFO`.

For all routines in the rest of this section that take both `handle` and `pe_session` as IN or INOUT arguments, if the `handle` argument passed in is not associated with the `pe_session` argument, `MPI_T_ERR_INVALID_HANDLE` is returned.

```

41 MPI_T_PVAR_HANDLE_FREE(pe_session, handle)
42
43     INOUT    pe_session          identifier of performance experiment session (handle)
44     INOUT    handle             handle to be freed (handle)
45

```

C binding

```

46 int MPI_T_pvar_handle_free(MPI_T_pvar_session pe_session,
47                             MPI_T_pvar_handle *handle)
48

```

When a handle is no longer needed, a user of the MPI tool information interface should call `MPI_T_PVAR_HANDLE_FREE` to free the handle in the performance experiment session identified by the parameter `pe_session` and the associated resources in the MPI implementation. On a successful return, MPI sets the handle to `MPI_T_PVAR_HANDLE_NULL`.

Starting and Stopping of Performance Variables

Performance variables that have the continuous flag set during the query procedure are continuously updated once a handle has been allocated. Such variables may be queried at any time, but they cannot be started or stopped by the user. All other variables are in a stopped state after their handle has been allocated; their values are not updated until they have been started by the user.

`MPI_T_PVAR_START(pe_session, handle)`

IN	<code>pe_session</code>	identifier of performance experiment session (handle)
INOUT	<code>handle</code>	handle of a performance variable (handle)

C binding

`int MPI_T_pvar_start(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle)`

This function starts the performance variable with the handle identified by the parameter `handle` in the performance experiment session identified by the parameter `pe_session`.

If the constant `MPI_T_PVAR_ALL_HANDLES` is passed in `handle`, the MPI implementation attempts to start all variables within the performance experiment session identified by the parameter `pe_session` for which handles have been allocated. In this case, the routine returns `MPI_SUCCESS` if all variables are started successfully (even if there are no noncontinuous variables to be started), otherwise `MPI_T_ERR_PVAR_NO_STARTSTOP` is returned. Continuous variables and variables that are already started are ignored when `MPI_T_PVAR_ALL_HANDLES` is specified.

`MPI_T_PVAR_STOP(pe_session, handle)`

IN	<code>pe_session</code>	identifier of performance experiment session (handle)
INOUT	<code>handle</code>	handle of a performance variable (handle)

C binding

`int MPI_T_pvar_stop(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle)`

This function stops the performance variable with the handle identified by the parameter `handle` in the performance experiment session identified by the parameter `pe_session`.

If the constant `MPI_T_PVAR_ALL_HANDLES` is passed in `handle`, the MPI implementation attempts to stop all variables within the performance experiment session identified by the parameter `pe_session` for which handles have been allocated. In this case, the routine returns `MPI_SUCCESS` if all variables are stopped successfully (even if there are no noncontinuous variables to be stopped), otherwise `MPI_T_ERR_PVAR_NO_STARTSTOP` is returned. Continuous variables and variables that are already stopped are ignored when `MPI_T_PVAR_ALL_HANDLES` is specified.

Performance Variable Access Functions

MPI_T_PVAR_READ(pe_session, handle, buf)

IN	pe_session	identifier of performance experiment session (handle)
IN	handle	handle of a performance variable (handle)
OUT	buf	initial address of storage location for variable value (choice)

C binding

```
int MPI_T_pvar_read(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle,
void *buf)
```

The `MPI_T_PVAR_READ` call queries the value of the performance variable with the handle `handle` in the performance experiment session identified by the parameter `pe_session` and stores the result in the buffer identified by the parameter `buf`. The user must ensure that the buffer is of the appropriate size to hold the entire value of the performance variable (based on the datatype and count returned by the corresponding previous calls to `MPI_T_PVAR_GET_INFO` and `MPI_T_PVAR_HANDLE_ALLOC`, respectively).

The constant `MPI_T_PVAR_ALL_HANDLES` cannot be used as an argument for the function `MPI_T_PVAR_READ`.

MPI_T_PVAR_WRITE(pe_session, handle, buf)

IN	pe_session	identifier of performance experiment session (handle)
INOUT	handle	handle of a performance variable (handle)
IN	buf	initial address of storage location for variable value (choice)

C binding

```
int MPI_T_pvar_write(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle,
const void *buf)
```

The `MPI_T_PVAR_WRITE` call attempts to write the value of the performance variable with the handle identified by the parameter `handle` in the performance experiment session identified by the parameter `pe_session`. The value to be written is passed in the buffer identified by the parameter `buf`. The user must ensure that the buffer is of the appropriate size to hold the entire value of the performance variable (based on the datatype and count returned by the corresponding previous calls to `MPI_T_PVAR_GET_INFO` and `MPI_T_PVAR_HANDLE_ALLOC`, respectively).

If it is not possible to change the variable, the function returns `MPI_T_ERR_PVAR_NO_WRITE`.

The constant `MPI_T_PVAR_ALL_HANDLES` cannot be used as an argument for the function `MPI_T_PVAR_WRITE`.

`MPI_T_PVAR_RESET(pe_session, handle)`

IN	<code>pe_session</code>	identifier of performance experiment session (handle)
INOUT	<code>handle</code>	handle of a performance variable (handle)

C binding

```
int MPI_T_pvar_reset(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle)
```

The `MPI_T_PVAR_RESET` call sets the performance variable with the handle identified by the parameter `handle` to its starting value specified in Section 16.3.7. If it is not possible to change the variable, the function returns `MPI_T_ERR_PVAR_NO_WRITE`.

If the constant `MPI_T_PVAR_ALL_HANDLES` is passed in `handle`, the MPI implementation attempts to reset all variables within the performance experiment session identified by the parameter `pe_session` for which handles have been allocated. In this case, the routine returns `MPI_SUCCESS` if all variables are reset successfully (even if there are no valid handles or all are read-only), otherwise `MPI_T_ERR_PVAR_NO_WRITE` is returned. Read-only variables are ignored when `MPI_T_PVAR_ALL_HANDLES` is specified.

`MPI_T_PVAR_READRESET(pe_session, handle, buf)`

IN	<code>pe_session</code>	identifier of performance experiment session (handle)
INOUT	<code>handle</code>	handle of a performance variable (handle)
OUT	<code>buf</code>	initial address of storage location for variable value (choice)

C binding

```
int MPI_T_pvar_readreset(MPI_T_pvar_session pe_session,
                        MPI_T_pvar_handle handle, void *buf)
```

This call atomically combines the functionality of `MPI_T_PVAR_READ` and `MPI_T_PVAR_RESET` with the same semantics as if these two calls were called separately. If the variable cannot be read and reset atomically, this routine returns `MPI_T_ERR_PVAR_NO_ATOMIC`.

The constant `MPI_T_PVAR_ALL_HANDLES` cannot be used as an argument for the function `MPI_T_PVAR_READRESET`.

Advice to implementors. Sampling-based tools rely on the ability to call the MPI tool information interface, in particular routines to start, stop, read, write, and reset performance variables, from any program context, including asynchronous contexts such as signal handlers. MPI implementations should strive, if possible in their particular environment, to enable these usage scenarios for all or a subset of the routines mentioned above. If implementing only a subset, the read, write, and reset routines are typically the most critical for sampling-based tools. An MPI implementation should clearly document any restrictions on the program contexts in which the MPI tool information interface can be used. Restrictions might include guaranteeing usage outside of all signals or outside a specific set of signals. Any restrictions could be documented, for example, through the description returned by `MPI_T_PVAR_GET_INFO`. (*End of advice to implementors.*)

Rationale. All routines to read, to write or to reset performance variables require the performance experiment session argument. This requirement keeps the interface consistent and allows the use of `MPI_T_PVAR_ALL_HANDLES` where appropriate. Further, this opens up additional performance optimizations for the implementation of handles.
(*End of rationale.*)

Example 16.7. Detecting receives with long unexpected message queues.

The following example shows a sample tool to identify receive operations that occur during times with long message queues. This example assumes that the MPI implementation exports a variable with the name “`MPI_T_UMQ_LENGTH`” to represent the current length of the unexpected message queue. The tool is implemented as a PMPI tool using the MPI profiling interface.

The tool consists of three parts: (1) the initialization (by intercepting the call to `MPI_INIT`), (2) the test for long unexpected message queues (by intercepting calls to `MPI_RECV`), and (3) the clean-up phase (by intercepting the call to `MPI_FINALIZE`). To capture all receives, the example would have to be extended to have similar wrappers for all receive operations.

Part 1—Initialization: During initialization, the tool searches for the variable and, once the right index is found, allocates a performance experiment session and a handle for the variable with the found index, and starts the performance variable.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <mpi.h>

/* Global variables for the tool */
static MPI_T_pvar_session pe_session;
static MPI_T_pvar_handle handle;

int MPI_Init(int *argc, char ***argv ) {
    int err, num, i, index, namelen, verbosity;
    int var_class, bind, threadsup;
    int readonly, continuous, atomic, count;
    char name[18];

    MPI_Comm comm;
    MPI_Datatype datatype;
    MPI_T_enum enumtype;

    err=PMPI_Init(argc, argv);
    if (err!=MPI_SUCCESS)
        return err;

    err=PMPI_T_init_thread(MPI_THREAD_SINGLE, &threadsup);
    if (err!=MPI_SUCCESS)
        return err;

    err=PMPI_T_pvar_get_num(&num);
    if (err!=MPI_SUCCESS)
        return err;

    index=-1;
```

```

1  i=0;
2  while ((i<num) && (index<0) && (err==MPI_SUCCESS)) {
3      /* Pass a buffer that is at least one character longer than */
4      /* the name of the variable being searched for to avoid */
5      /* finding variables that have a name that has a prefix */
6      /* equal to the name of the variable being searched. */
7      namelen=18;
8      err=PMPI_T_pvar_get_info(i, name, &namelen, &verbosity,
9                              &var_class, &datatype, &enumtype,
10                             NULL, NULL, &bind,&readonly,
11                             &continuous, &atomic);
12      if (strcmp(name,"MPI_T_UMQ_LENGTH")==0) index=i;
13      i++;
14  }
15  if (err!=MPI_SUCCESS)
16      return err;
17
18  /* this could be handled in a more flexible way for a generic tool */
19  assert(index>=0);
20  assert(var_class==MPI_T_PVAR_CLASS_LEVEL);
21  assert(datatype==MPI_INT);
22  assert(bind==MPI_T_BIND_MPI_COMM);
23
24  /* Create a session */
25  err=PMPI_T_pvar_session_create(&pe_session);
26  if (err!=MPI_SUCCESS) return err;
27
28  /* Get a handle and bind to MPI_COMM_WORLD */
29  comm=MPI_COMM_WORLD;
30  err=PMPI_T_pvar_handle_alloc(pe_session, index, &comm, &handle,
31                              &count);
32  if (err!=MPI_SUCCESS) return err;
33
34  /* this could be handled in a more flexible way for a generic tool */
35  assert(count==1);
36
37  /* Start variable */
38  err=PMPI_T_pvar_start(pe_session, handle);
39  if (err!=MPI_SUCCESS) return err;
40
41  return MPI_SUCCESS;
42 }

```

Part 2—Testing the Queue Lengths During Receives: During every receive operation, the tool reads the unexpected queue length through the matching performance variable and compares it against a predefined threshold.

```

40 #define THRESHOLD 5
41
42 int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source,
43             int tag, MPI_Comm comm, MPI_Status *status)
44 {
45     int value, err;
46
47     if (comm==MPI_COMM_WORLD) {
48         err=PMPI_T_pvar_read(pe_session, handle, &value);
49         if ((err==MPI_SUCCESS) && (value>THRESHOLD))

```

```

1      {
2          /* tool identified receive called with long UMQ */
3          /* execute tool functionality, */
4          /* e.g., gather and print call stack */
5      }
6
7      return PMPI_Recv(buf, count, datatype, source, tag, comm, status);
8  }

```

Part 3—Termination: In the wrapper for `MPI_FINALIZE`, the MPI tool information interface is finalized.

```

12  int MPI_Finalize(void)
13  {
14      int err;
15
16      err=PMPI_T_pvar_handle_free(pe_session, &handle);
17      err=PMPI_T_pvar_session_free(&pe_session);
18      err=PMPI_T_finalize();
19      return PMPI_Finalize();
20  }

```

16.3.8 Events

During the execution of an MPI application, the MPI implementation can raise *events* of a specific type to inform the user of a state change in the implementation. **Event types** describe specific state changes within the MPI implementation. In comparison to aggregate performance variables, events provide per-instance information on such state changes. The MPI implementation is said to **raise an event** when it invokes a callback function previously registered by the user for the corresponding event type. Each callback invocation for a specific event instance has a timestamp associated with it, which can be queried by the user, describing the time when the event was observed by the implementation. This decouples the observation of the state change from the communication of this information to the user. A timestamp in this context is a count of clock ticks elapsed since some time in the past and represented as a variable of type `MPI_Count`.

Event Sources

As a means to manage multiple state changes to be observed concurrently by different parts of the software and hardware system, the event interface of the MPI Tool Information Interface uses the concept of *sources*. A source in this context is a concept describing the logical entity raising the event. A source may or may not directly represent a concrete part of the software or hardware system. This concept is used primarily to describe partial ordering of events across different components where total ordering cannot necessarily be determined or is too costly to enforce.

The following function can be used to query the number of event sources, `num_sources`:

`MPI_T_SOURCE_GET_NUM(num_sources)`

OUT `num_sources` returns number of event sources (integer)

C binding

```
int MPI_T_source_get_num(int *num_sources)
```

The number of available event sources can be queried with a call to `MPI_T_SOURCE_GET_NUM`. An MPI implementation is allowed to increase the number of sources during the execution of an MPI process. However, MPI implementations are not allowed to change the index of an event source or to delete an event source once it has been made visible to the user (e.g., if new event sources become available via dynamic loading of additional components in the MPI implementation).

`MPI_T_SOURCE_GET_INFO(source_index, name, name_len, desc, desc_len, ordering, ticks_per_second, max_ticks, info)`

IN	<code>source_index</code>	index of the source to be queried between 0 and <code>num_sources - 1</code> (integer)
OUT	<code>name</code>	buffer to return the string containing the name of the source (string)
INOUT	<code>name_len</code>	length of the string and/or buffer for <code>name</code> (integer)
OUT	<code>desc</code>	buffer to return the string containing the description of the source (string)
INOUT	<code>desc_len</code>	length of the string and/or buffer for <code>desc</code> (integer)
OUT	<code>ordering</code>	flag indicating chronological ordering guarantees given by the source (integer)
OUT	<code>ticks_per_second</code>	the number of ticks per second for the timer of this source (integer)
OUT	<code>max_ticks</code>	the maximum count of ticks reported by this source before overflow occurs (integer)
OUT	<code>info</code>	optional info object (handle)

C binding

```
int MPI_T_source_get_info(int source_index, char *name, int *name_len,
    char *desc, int *desc_len, MPI_T_source_order *ordering,
    MPI_Count *ticks_per_second, MPI_Count *max_ticks,
    MPI_Info *info)
```

A call to `MPI_T_SOURCE_GET_INFO` returns additional information on the source identified by the `source_index` argument.

The arguments `name` and `name_len` are used to return the name of the source as described in Section 16.3.3.

The arguments `desc` and `desc_len` are used to return the description of the source as described in Section 16.3.3.

The `ordering` argument is of type `MPI_T_source_order` and returns whether event callbacks of this source will be invoked in chronological order, i.e., the timestamps reported

by `MPI_T_EVENT_GET_TIMESTAMP` of subsequent events of the same source are monotonically increasing. The value of `ordering` can be `MPI_T_SOURCE_ORDERED` or `MPI_T_SOURCE_UNORDERED`.

The `ticks_per_seconds` argument returns the number of ticks elapsed in one second for the timer used for the specific source.

The `max_ticks` argument returns the largest number of ticks reported by this source as a timestamp before the value overflows.

Advice to users. As the size of `MPI_Count` is defined in relation to the types `MPI_Aint` and `MPI_Offset`, the effective size of `MPI_Count` may lead to overflows of the timestamp values reported. Users can use the argument `max_ticks` to mitigate resulting problems. (*End of advice to users.*)

MPI can optionally return an info object containing the default hints set for this source. If the argument to `info` provided by the user is the NULL pointer, this argument is ignored, otherwise an MPI implementation is required to return all hints that are supported by the implementation for this source and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no key/value pair. The user is responsible for freeing `info` via `MPI_INFO_FREE`.

`MPI_T_SOURCE_GET_TIMESTAMP(source_index, timestamp)`

IN	<code>source_index</code>	index of the source (integer)
OUT	<code>timestamp</code>	current timestamp from specified source (integer)

C binding

`int MPI_T_source_get_timestamp(int source_index, MPI_Count *timestamp)`

To enable proper query of a reference timestamp for a specific source, a user can obtain a current timestamp using `MPI_T_SOURCE_GET_TIMESTAMP`. The argument `source_index` identifies the index of the source to query. The call returns `MPI_SUCCESS` and a current timestamp in the argument `timestamp` if the source supports ad-hoc generation of timestamps. The call returns `MPI_T_ERR_INVALID_INDEX` if the index does not identify a valid source. The call returns `MPI_T_ERR_NOT_SUPPORTED` if the source does not support the ad-hoc generation of timestamps.

Callback Safety Requirements

The actions a user is allowed to perform inside a callback function may vary with its execution context. As the user has no control over the execution context of specific callback function invocations, MPI provides a way to communicate this information using callback safety levels.

Table 16.5 provides the hierarchy of callback safety requirements levels within user-defined callback functions. The MPI implementation provides the safety requirement as an argument to the callback when it is invoked.

The level of `MPI_T_CB_REQUIRE_NONE` is the lowest level and does not impose any restrictions on the callback function.

Table 16.5: Hierarchy of safety requirement levels for event callback routines

Safety Requirement
<code>MPI_T_CB_REQUIRE_NONE</code>
<code>MPI_T_CB_REQUIRE_MPI_RESTRICTED</code>
<code>MPI_T_CB_REQUIRE_THREAD_SAFE</code>
<code>MPI_T_CB_REQUIRE_ASYNC_SIGNAL_SAFE</code>

The level of `MPI_T_CB_REQUIRE_MPI_RESTRICTED` restricts the set of MPI functions that can be called from inside the callback to all functions with the prefix `MPI_T` as well as `MPI_WTICK` and `MPI_WTIME`.

Advice to users. While some MPI functions are safe to be called inside a callback function used in the MPI tool information interface—which may in some implementations be issued from asynchronous contexts such as signal handlers—this does not imply that those MPI functions are generally safe to be called in asynchronous contexts such as signal handlers. (*End of advice to users.*)

The level of `MPI_T_CB_REQUIRE_THREAD_SAFE` includes all the limitations of `MPI_T_CB_REQUIRE_MPI_RESTRICTED` and additionally requires the callback to be reentrant and thread-safe. This means the callback must allow its execution to be interrupted by or happen concurrently with any other callback including itself.

The level of `MPI_T_CB_REQUIRE_ASYNC_SIGNAL_SAFE` includes all the limitations of `MPI_T_CB_REQUIRE_THREAD_SAFE` and additionally requires the callback to meet the safety requirements needed to support invocations from asynchronous contexts, such as signal handlers.

Advice to users. It is always safe to assume the highest restrictions for a callback invocation (i.e., `MPI_T_CB_REQUIRE_ASYNC_SIGNAL_SAFE`). By evaluating the specific requirements at runtime, a tool may obtain more freedom of action within the callback. (*End of advice to users.*)

Advice to implementors. A high-quality implementation will strive to set callback safety requirements to the most permissive level for a given callback invocation. (*End of advice to implementors.*)

All functions with the prefix `MPI_T`, except those listed in Table 16.6, may return the return code `MPI_T_ERR_NOT_ACCESSIBLE` to indicate that the user may not access this function at this time. The functions (and their respective PMPI versions) listed in Table 16.6 are exceptions to this rule and shall not return `MPI_T_ERR_NOT_ACCESSIBLE`.

Rationale. A call may be implemented in a way that is not safe for all execution contexts of a callback function, e.g., inside a signal handler. An MPI implementation therefore needs a way to communicate its inability to perform a certain action due to the execution context of a callback invocation. (*End of rationale.*)

Advice to implementors. A high-quality implementation shall not return `MPI_T_ERR_NOT_ACCESSIBLE` except where absolutely necessary. (*End of advice to implementors.*)

Table 16.6: List of MPI functions that when called from within a callback function may not return `MPI_T_ERR_NOT_ACCESSIBLE`

```

MPI_T_EVENT_COPY
MPI_T_EVENT_GET_SOURCE
MPI_T_EVENT_GET_TIMESTAMP
MPI_T_EVENT_READ
MPI_T_PVAR_READ
MPI_T_PVAR_READRESET
MPI_T_PVAR_RESET
MPI_T_PVAR_START
MPI_T_PVAR_STOP
MPI_T_PVAR_WRITE
MPI_T_SOURCE_GET_TIMESTAMP

```

Advice to users. Users intercepting calls into the MPI tool information interface using the PMPI interface must ensure that the safety requirements for the calling context are met. This means that users may have to implement the wrapper with the highest safety level used by the MPI implementation. (*End of advice to users.*)

Event Type Query Functions

An MPI implementation exports a set of N event types through the MPI tool information interface. If N is zero, then the MPI implementation does not export any event types; otherwise, the provided event types are indexed from 0 to $N - 1$. This index number is used in subsequent calls to identify a specific event type.

An MPI implementation is allowed to increase the number of event types during the execution of an MPI process. However, MPI implementations are not allowed to change the index of an event type or to delete an event type once it has been made visible to the user (e.g., if new event types become available via dynamic loading of additional components in the MPI implementation).

The following function can be used to query the number of event types, `num_events`:

```
MPI_T_EVENT_GET_NUM(num_events)
```

OUT `num_events` returns number of event types (integer)

C binding

```
int MPI_T_event_get_num(int *num_events)
```

The function `MPI_T_EVENT_GET_INFO` provides access to additional information about a specific event type.

```

MPI_T_EVENT_GET_INFO(event_index, name, name_len, verbosity, array_of_datatypes,
array_of_displacements, num_elements, enumtype, info, desc, desc_len,
bind)

```

IN	event_index	index of the event type to be queried between 0 and num_events - 1 (integer)	1
OUT	name	buffer to return the string containing the name of the event type (string)	2
INOUT	name_len	length of the string and/or buffer for name (integer)	3
OUT	verbosity	verbosity level of this event type (integer)	4
OUT	array_of_datatypes	array of MPI basic datatypes used to encode the event data (array of handles)	5
OUT	array_of_displacements	array of byte displacements of the elements in the event buffer (array of nonnegative integers)	6
INOUT	num_elements	length of array_of_datatypes and array_of_displacements arrays (nonnegative integer)	7
OUT	enumtype	optional descriptor for enumeration information (handle)	8
OUT	info	optional info object (handle)	9
OUT	desc	buffer to return the string containing a description of the event type (string)	10
INOUT	desc_len	length of the string and/or buffer for desc (integer)	11
OUT	bind	type of MPI object to which an event of this type must be bound (integer)	12

C binding

```

int MPI_T_event_get_info(int event_index, char *name, int *name_len,
                        int *verbosity, MPI_Datatype array_of_datatypes[],
                        MPI_Aint array_of_displacements[], int *num_elements,
                        MPI_T_enum *enumtype, MPI_Info *info, char *desc, int *desc_len,
                        int *bind)

```

After a successful call to `MPI_T_EVENT_GET_INFO` for a particular event type, subsequent calls to this routine that query information about the same event type must return the same information. If any INOUT or OUT argument to `MPI_T_EVENT_GET_INFO` is a NULL pointer, the implementation will ignore the argument and not return a value for the specific argument.

The arguments `name` and `name_len` are used to return the name of the event type as described in Section 16.3.3. If completed successfully, the routine is required to return a name of at least length one. The name of the event type must be unique with respect to all other names for event types used by the MPI implementation.

The argument `verbosity` returns the verbosity level of the event type (see Section 16.3.1).

The argument `array_of_datatypes` returns an array of MPI datatype handles that describe the elements returned for an instance of the event type with index `event_index`. The event data can either be queried element by element with `MPI_T_EVENT_READ` or copied into a contiguous event buffer with `MPI_T_EVENT_COPY`. For the latter case, the argument `array_of_displacements` returns an array of byte displacements in the event buffer in ascending order starting with zero.

The user is responsible for the memory allocation for the `array_of_datatypes` and `array_of_displacements` arrays. The number of elements in each array is supplied by the user in `num_elements`. If the number of elements used by the event type is larger than the value

of `num_elements` provided by the user, the number of datatype handles and displacements returned in the corresponding arrays is truncated to the value of `num_elements` passed in by the user. If the user passes the NULL pointer for `array_of_datatypes` or `array_of_displacements`, the respective arguments are ignored. Unless the user passes the NULL pointer for `num_elements`, the function returns the number of elements required for this event type. If the user passes the NULL pointer for `num_elements`, the arguments `num_elements`, `array_of_datatypes`, and `array_of_displacements` are ignored.

MPI can optionally return an enumeration identifier in the `enumtype` argument, describing the individual elements in the `array_of_datatypes` argument. Otherwise, `enumtype` is set to `MPI_T_ENUM_NULL`. If the argument to `enumtype` provided by the user is the NULL pointer, no enumeration type is returned.

MPI can optionally return an info object containing the default hints set for a registration handle for this event type. If the argument to `info` provided by the user is the NULL pointer, this argument is ignored, otherwise an MPI implementation is required to return all hints that are supported by the implementation for a registration handle for this event type and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no key/value pair. The user is responsible for freeing `info` via `MPI_INFO_FREE`.

The arguments `desc` and `desc_len` are used to return the description of the event type as described in Section 16.3.3. Returning a description is optional. If an MPI implementation does not return a description, the first character for `desc` must be set to the null character and `desc_len` must be set to one at the return from this function.

The parameter `bind` returns the type of the MPI object to which the event type must be bound or the value `MPI_T_BIND_NO_OBJECT` (see Section 16.3.2).

If an event type has an equivalent name across connected MPI processes, the following OUT parameters must be identical: `verbosity`, `array_of_datatypes`, `num_elements`, `enumtype`, and `bind`. The returned description must be equivalent. As the argument `array_of_displacements` is process dependent, it may differ across connected MPI processes.

This routine returns `MPI_SUCCESS` on success and returns `MPI_T_ERR_INVALID_INDEX` if `event_index` does not match a valid event type index provided by the implementation at the time of the call.

`MPI_T_EVENT_GET_INDEX(name, event_index)`

IN	<code>name</code>	name of the event type (string)
OUT	<code>event_index</code>	index of the event type (integer)

C binding

`int MPI_T_event_get_index(const char *name, int *event_index)`

`MPI_T_EVENT_GET_INDEX` returns the index of an event type identified by a known event type name. The `name` parameter is provided by the caller, and `event_index` is returned by the MPI implementation. The `name` parameter is a string terminated with a null character.

This routine returns `MPI_SUCCESS` on success and returns `MPI_T_ERR_INVALID_NAME` if `name` does not match the name of any event type provided by the implementation at the time of the call.

Rationale. This routine is provided to enable fast retrieval of an event index by a tool, assuming it knows the name of the event type for which it is looking. The number of event types exposed by the implementation can change over time, so it is not possible for the tool to simply iterate over the list of event types once at initialization. Although using MPI implementation specific event type names is not portable across MPI implementations, tool developers may choose to take this route for lower overhead at runtime because the tool will not have to iterate over the entire set of event types to find a specific one. (*End of rationale.*)

Handle Allocation and Deallocation

Before the MPI implementation calls a callback function on the occurrence of a specific event, the user needs to register a callback function to be called for that event type and obtain a handle of type `MPI_T_event_registration`.

`MPI_T_EVENT_HANDLE_ALLOC(event_index, obj_handle, info, event_registration)`

IN	<code>event_index</code>	index of event type for which the registration handle is to be allocated (integer)
IN	<code>obj_handle</code>	reference to a handle of the MPI object to which this event is supposed to be bound (pointer)
IN	<code>info</code>	info object (handle)
OUT	<code>event_registration</code>	event registration (handle)

C binding

```
int MPI_T_event_handle_alloc(int event_index, void *obj_handle, MPI_Info info,
                             MPI_T_event_registration *event_registration)
```

`MPI_T_EVENT_HANDLE_ALLOC` creates a **registration handle** for the event type identified by `event_index`. Furthermore, if required by the event type, the registration handle is bound to the object referred to by the argument `obj_handle`. The argument `obj_handle` is ignored if the `MPI_T_EVENT_GET_INFO` call for this event type returned `MPI_T_BIND_NO_OBJECT` in the argument `bind`. The user can pass hints for the handle allocation to the MPI implementation via the `info` argument. The allocated event-registration handle is returned in the argument `event_registration`.

`MPI_T_EVENT_HANDLE_SET_INFO(event_registration, info)`

INOUT	<code>event_registration</code>	event registration (handle)
IN	<code>info</code>	info object (handle)

C binding

```
int MPI_T_event_handle_set_info(MPI_T_event_registration event_registration,
                                MPI_Info info)
```

`MPI_T_EVENT_HANDLE_SET_INFO` updates the hints of the event-registration handle associated with `event_registration` using the hints provided in `info`. A call to this procedure has no effect on previously set or defaulted hints that are not specified by `info`. It also

has no effect on previously set or defaulted hints that are specified by `info`, but are ignored by the MPI implementation in this call to `MPI_T_EVENT_HANDLE_SET_INFO`.

Advice to users. Some info items that an implementation can use when it creates an event-registration handle cannot easily be changed once the registration handle is created. Thus, an implementation may ignore hints issued in this call that it would have accepted in a handle allocation call. An implementation may also be unable to update certain info hints in a call to `MPI_T_EVENT_HANDLE_SET_INFO`. `MPI_T_EVENT_HANDLE_GET_INFO` can be used to determine whether info changes were ignored by the implementation. (*End of advice to users.*)

`MPI_T_EVENT_HANDLE_GET_INFO(event_registration, info_used)`

IN	<code>event_registration</code>	event registration (handle)
OUT	<code>info_used</code>	info object (handle)

C binding

```
int MPI_T_event_handle_get_info(MPI_T_event_registration event_registration,
                               MPI_Info *info_used)
```

`MPI_T_EVENT_HANDLE_GET_INFO` returns a new info object containing the hints of the event-registration handle associated with `event_registration`. The current setting of all hints related to this registration handle is returned in `info_used`. An MPI implementation is required to return all hints that are supported by the implementation and have default values specified; any user-supplied hints that were not ignored by the implementation; and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no key/value pairs. The user is responsible for freeing `info_used` via `MPI_INFO_FREE`.

`MPI_T_EVENT_REGISTER_CALLBACK(event_registration, cb_safety, info, user_data, event_cb_function)`

INOUT	<code>event_registration</code>	event registration (handle)
IN	<code>cb_safety</code>	maximum callback safety level (integer)
IN	<code>info</code>	info object (handle)
IN	<code>user_data</code>	pointer to a user-controlled buffer
IN	<code>event_cb_function</code>	pointer to user-defined callback function (function)

C binding

```
int MPI_T_event_register_callback(MPI_T_event_registration event_registration,
                                  MPI_T_cb_safety cb_safety, MPI_Info info, void *user_data,
                                  MPI_T_event_cb_function event_cb_function)
```

`MPI_T_EVENT_REGISTER_CALLBACK` associates a user-defined function pointed to by `event_cb_function` with an allocated event-registration handle. The maximum callback safety level supported by the callback function is passed in the argument `cb_safety`. The

safety levels are defined in Table 16.5. A user can register multiple callback functions for a given event-registration handle, potentially specifying one for each callback safety level. Registering a callback function for a specific callback safety level overwrites any previously-registered callback function pointer and info object associated with the event registration for the specific callback safety level. If `event_cb_function` is the NULL pointer, an existing association of a callback function for that callback safety level is removed.

When an event is triggered, the implementation will select from all registered callbacks the callback with the lowest safety level valid in the context in which the callback is invoked. In situations where the required callback safety level exceeds the highest level for which a callback function is registered for a given registration handle, the event instance is dropped.

At callback invocation time, the implementation passes the pointer to a user-defined memory region specified during callback registration with the argument `user_data`.

The user can pass hints for the registration of the specified callback function to the MPI implementation via the `info` argument.

Advice to users. As event instances can be raised as soon as the registration handle is associated with the first callback function, the callback function with the highest callback safety guarantees should be registered before any further registrations for lower callback safety guarantees, to avoid dropped events due to insufficient callback safety guarantees. (*End of advice to users.*)

The callback function passed to `MPI_T_EVENT_REGISTER_CALLBACK` in the argument `event_cb_function` needs to have the following type:

```
typedef void MPI_T_event_cb_function(MPI_T_event_instance event_instance,
                                     MPI_T_event_registration event_registration,
                                     MPI_T_cb_safety cb_safety, void *user_data);
```

The argument `event_instance` corresponds to a handle for the opaque event-instance object of type `MPI_T_event_instance`. This handle is only valid inside the corresponding invocation of the function to which it is passed. The argument `event_registration` corresponds to the event-registration handle returned by `MPI_T_EVENT_HANDLE_ALLOC` for the user function to the same event type and bound object combination. The handle can be used to identify the specific event registration information, such as event type and bound object, or even to deallocate the handle from within the callback invocation. The argument `cb_safety` describes the safety requirements the callback function must fulfill in the current invocation. The argument `user_data` is the pointer to user-allocated memory that was passed to the MPI implementation during callback registration.

```
MPI_T_EVENT_CALLBACK_SET_INFO(event_registration, cb_safety, info)
```

INOUT	<code>event_registration</code>	event registration (handle)
IN	<code>cb_safety</code>	callback safety level (integer)
IN	<code>info</code>	info object (handle)

C binding

```
int MPI_T_event_callback_set_info(MPI_T_event_registration event_registration,
                                  MPI_T_cb_safety cb_safety, MPI_Info info)
```

`MPI_T_EVENT_CALLBACK_SET_INFO` updates the hints of the callback function registered for the callback safety level specified by `cb_safety` of the event-registration handle associated with `event_registration` using the hints provided in `info`. A call to this procedure has no effect on previously set or defaulted hints that are not specified by `info`. It also has no effect on previously set or defaulted hints that are specified by `info`, but are ignored by the MPI implementation in this call to `MPI_T_EVENT_CALLBACK_SET_INFO`.

```
MPI_T_EVENT_CALLBACK_GET_INFO(event_registration, cb_safety, info_used)
```

IN	<code>event_registration</code>	event registration (handle)
IN	<code>cb_safety</code>	callback safety level (integer)
OUT	<code>info_used</code>	info object (handle)

C binding

```
int MPI_T_event_callback_get_info(MPI_T_event_registration event_registration,
                                MPI_T_cb_safety cb_safety, MPI_Info *info_used)
```

`MPI_T_EVENT_CALLBACK_GET_INFO` returns a new info object containing the hints of the callback function registered for the callback safety level specified by `cb_safety` of the event-registration handle associated with `event_registration`. The current set of all hints related to this callback safety level of the event-registration handle is returned in `info_used`. An MPI implementation is required to return all hints that are supported by the implementation and have default values specified, any user-supplied hints that were not ignored by the implementation, and any additional hints that were set by the implementation. If no such hints exist, a handle to a newly created info object is returned that contains no key/value pairs. The user is responsible for freeing `info_used` via `MPI_INFO_FREE`.

To stop the MPI implementation from raising events for a specific registration, a user needs to free the corresponding event-registration handle.

```
MPI_T_EVENT_HANDLE_FREE(event_registration, user_data, free_cb_function)
```

INOUT	<code>event_registration</code>	event registration (handle)
IN	<code>user_data</code>	pointer to a user-controlled buffer
IN	<code>free_cb_function</code>	pointer to user-defined callback function (function)

C binding

```
int MPI_T_event_handle_free(MPI_T_event_registration event_registration,
                            void *user_data, MPI_T_event_free_cb_function free_cb_function)
```

`MPI_T_EVENT_HANDLE_FREE` returns `MPI_SUCCESS` when deallocation of the handle was initiated successfully and returns `MPI_T_ERR_INVALID_HANDLE` if `event_registration` does not match a valid allocated event-registration handle at the time of the call. The callback function `free_cb_function` is called by the MPI implementation, when it is able to guarantee that no further event instances for the corresponding event-registration handle will be raised. If the pointer to `free_cb_function` is the NULL pointer, no user function is invoked after successful deallocation of the event registration handle. The

pointer to user-controlled memory provided in the `user_data` argument will be passed to the function provided in the `free_cb_function` on invocation.

Advice to users. A free-callback function associated with a registration handle should always be prepared to postpone any pending actions, should the provided callback safety requirements exceed those required by the pending actions. (*End of advice to users.*)

The callback function passed to `MPI_T_EVENT_HANDLE_FREE` in the argument `free_cb_function` needs to have the following type:

```
typedef void MPI_T_event_free_cb_function(
    MPI_T_event_registration event_registration,
    MPI_T_cb_safety cb_safety, void *user_data);
```

Handling Dropped Events

Events may occur at times when the MPI implementation cannot invoke the user function corresponding to a matching event handle. An implementation is allowed to buffer such events and delay the callback invocation. If an event occurs at times when the corresponding callback function cannot be called and the corresponding data cannot be buffered, or no callback function meeting the required callback safety level is registered, the event data may be dropped. To discover such data loss, the user can set a handler function for a specific event-registration handle.

`MPI_T_EVENT_SET_DROPPED_HANDLER(event_registration, dropped_cb_function)`

INOUT	<code>event_registration</code>	valid event registration (handle)
IN	<code>dropped_cb_function</code>	pointer to user-defined callback function (function)

C binding

```
int MPI_T_event_set_dropped_handler(
    MPI_T_event_registration event_registration,
    MPI_T_event_dropped_cb_function dropped_cb_function)
```

`MPI_T_EVENT_SET_DROPPED_HANDLER` registers the function `dropped_cb_function` to be called by the MPI implementation when event information is dropped for the registration handle specified in `event_registration`. Subsequent calls to `MPI_T_EVENT_SET_DROPPED_HANDLER` with the same registration handle will replace previously-registered callback functions for that registration handle. If the pointer to `dropped_cb_function` is the NULL pointer, no data loss is recorded or reported until a new valid callback function is registered.

Advice to users. The invocation of the dropped handler callback function may not necessarily occur close to the time the event was actually lost. (*End of advice to users.*)

The callback function passed to `MPI_T_EVENT_SET_DROPPED_HANDLER` in the argument `dropped_cb_function` needs to have the following type:

```

1 typedef void MPI_T_event_dropped_cb_function(MPI_Count count,
2       MPI_T_event_registration event_registration, int source_index,
3       MPI_T_cb_safety cb_safety, void *user_data);
4

```

The argument `event_registration` corresponds to the event registration handle to which the dropped data corresponds. The argument `count` provides a best effort estimation of the number of invocations to a registered event callback corresponding to `event_registration` that were not executed since the registration of the dropped-callback handler or the last invocation of a registered dropped-callback handler. If the number of dropped events observed by the implementation exceeds the limit of `count`, an implementation shall set `count` to the maximum possible value for the type of `count`. The `source_index` provides the index of the source that dropped the corresponding event information. The argument `cb_safety` describes the safety requirements the callback function must fulfill in the current invocation. The possible values for `cb_safety` are described in Table 16.5. The argument `user_data` is the pointer to user-allocated memory that was passed to the MPI implementation during callback registration. If no event callback is registered for safety requirement levels that an implementation uses to invoke the dropped handler callback function for a specific event, the corresponding dropped handler callback function will not be invoked.

Advice to users. A callback function for dropped events associated with a registration handle should always be prepared to postpone any pending actions, should the provided callback safety requirements exceed those required by the pending actions. (*End of advice to users.*)

Advice to implementors. A high-quality implementation should strive to find a good balance between timely notification, completeness of information, and the freedom of action for a tool when invoking the callback function for dropped events associated with a registration handle. (*End of advice to implementors.*)

If dropped event notifications have been observed for a specific source since the last event notification of that source, the corresponding dropped handler callback function must be called before other events are raised for that source. This means in a sequence of five events E1 to E5 from the same source, where E3 and E4 were dropped, any handler function set through `MPI_T_EVENT_SET_DROPPED_HANDLER` for event-registration handles associated with E3 or E4 must be called before E5 is raised.

Reading Event Data

In event callbacks, the parameter `event_instance` provides access to the per-instance event data, i.e., the data encoded by the specific event type for this instance. The user can obtain event data as well as event meta data, such as a time stamp and the source, by providing this handle to the respective query functions. The event-instance handle is invalid beyond the scope of the current invocation of the callback function to which it is provided.

The callback function argument `event_registration` identifies the registration handle that was used to register the callback function.

The callback function argument `cb_safety` indicates the requirements for the specific callback invocation. The value is one of the safety requirements levels described in Table 16.5. The argument `user_data` passes the pointer provided by the user during callback registration back to the function call.

Advice to users. Depending on the registered event and usage of MPI by the application, a callback function may be invoked with high frequency. Users should therefore strive to minimize the amount of work done inside callback functions. Furthermore, the time spent in a callback function may influence the capability of an implementation to buffer events; long execution times may lead to an increased number of dropped events. (*End of advice to users.*)

MPI provides the following function calls to access data of a specific event instance and its corresponding meta data (such as its time and source).

MPI_T_EVENT_READ(event_instance, element_index, buffer)

IN	event_instance	event-instance handle provided to the callback function (handle)
IN	element_index	index into the array of datatypes of the item to be queried (integer)
OUT	buffer	pointer to a memory location to store the item data (choice)

C binding

```
int MPI_T_event_read(MPI_T_event_instance event_instance, int element_index,
                    void *buffer)
```

MPI_T_EVENT_READ allows users to copy one element of the event data to a user-specified buffer at a time.

The `event_instance` argument identifies the event instance to query. It is erroneous to provide any other event-instance handle to the call than the one passed by the MPI implementation to the callback function in which the data is read. The `buffer` argument must point to a memory location the MPI implementation can copy the element of the event data to identified by `element_index`.

MPI_T_EVENT_COPY(event_instance, buffer)

IN	event_instance	event instance provided to the callback function (handle)
OUT	buffer	user-allocated buffer for event data (choice)

C binding

```
int MPI_T_event_copy(MPI_T_event_instance event_instance, void *buffer)
```

MPI_T_EVENT_COPY copies the event data as a whole into the user-provided `buffer`. The user must ensure that the buffer is of at least the size of the extent of the event type, which can be computed from the type and displacement information returned by the corresponding call to **MPI_T_EVENT_GET_INFO**. The data may include padding bytes between individual elements of the event data in the buffer. A user can reconstruct the location and size of the data contained in the buffer through the information returned by **MPI_T_EVENT_GET_INFO**.

Advice to implementors. An implementation should strive to use an appropriately compact representation when copying event instance data to a user buffer via

`MPI_T_EVENT_COPY` to reduce the amount of memory required for the user buffer.
(*End of advice to implementors.*)

Reading Event Meta Data

Additional to the specific event data encoded by each event type, supplemental information available across all event types can be queried.

`MPI_T_EVENT_GET_TIMESTAMP(event_instance, event_timestamp)`

IN	<code>event_instance</code>	event instance provided to the callback function (handle)
OUT	<code>event_timestamp</code>	timestamp the event was observed (integer)

C binding

```
int MPI_T_event_get_timestamp(MPI_T_event_instance event_instance,
                             MPI_Count *event_timestamp)
```

`MPI_T_EVENT_GET_TIMESTAMP` returns the timestamp of when the event was initially observed by the implementation. The `event_instance` argument identifies the event instance to query. It is erroneous to provide any other handle to the call than the one passed by the MPI implementation to the callback function in which the timestamp is read.

Advice to users. An MPI implementation may postpone the call to the user's callback function. In this case, the call to `MPI_T_EVENT_GET_TIMESTAMP` may yield a timestamp in the past that is closer to the time the event was initially observed, as opposed to a timestamp captured during callback function invocation. (*End of advice to users.*)

Advice to implementors. A high-quality implementation will return a timestamp as close as possible to the earliest time the event was observed by the MPI implementation. (*End of advice to implementors.*)

An event may be raised from different components acting as event sources in the MPI implementation. A source in this context is an abstract concept that helps to define partial ordering of raised events, as each source provides its own ordering guarantees. A source describes the entity that raises the event, rather than the origin of the data.

To identify the source of an event instance, the user can query the index of the source within the corresponding event callback function invocation.

Advice to implementors. An excessive number of event sources may negatively impact performance of a tool due to per-source overhead in event handling. (*End of advice to implementors.*)

`MPI_T_EVENT_GET_SOURCE(event_instance, source_index)`

IN	<code>event_instance</code>	event instance provided to the callback function (handle)
OUT	<code>source_index</code>	index identifying the source (integer)

C binding

```
int MPI_T_event_get_source(MPI_T_event_instance event_instance,
                          int *source_index)
```

The `event_instance` argument identifies the event instance to query. It is erroneous to provide any other event-instance handle to the call than the one passed by the MPI implementation to the callback function in which the source is queried.

The `source_index` argument returns the index of the source of the event instance. It can be used to query more information on the source using [MPI_T_SOURCE_GET_INFO](#).

Rationale. Event callback function invocations are associated with a source to enable chronological processing of events on the tool side, when required, while retaining low overhead on the side of the MPI implementation. (*End of rationale.*)

16.3.9 Variable Categorization

MPI implementations can optionally group performance and control variables into categories to express logical relationships between various variables. For example, an MPI implementation could group all control and performance variables that refer to message transfers in the MPI implementation and thereby distinguish them from variables that refer to local resources such as memory allocations or other interactions with the operating system.

Categories can also contain other categories to form a hierarchical grouping. Categories can never include themselves, either directly or transitively within other included categories. Expanding on the example above, this allows MPI to refine the grouping of variables referring to message transfers into variables to control and to monitor message queues, message matching activities and communication protocols. Each of these groups of variables would be represented by a separate category and these categories would then be listed in a single category representing variables for message transfers.

The category information may be queried in a fashion similar to the mechanism for querying variable information. The MPI implementation exports a set of N categories via the MPI tool information interface. If $N = 0$, then the MPI implementation does not export any categories, otherwise the provided categories are indexed from 0 to $N - 1$. This index number is used in subsequent calls to functions of the MPI tool information interface to identify the individual categories.

An MPI implementation is permitted to increase the number of categories during the execution of an MPI program when new categories become available through dynamic loading. However, MPI implementations are not allowed to change the index of a category or delete it once it has been added to the set.

Similarly, MPI implementations are allowed to add variables to categories, but they are not allowed to remove variables from categories or change the order in which they are returned.

Category Query Functions

The following function can be used to query the number of categories, `num_cat`.

```
MPI_T_CATEGORY_GET_NUM(num_cat)
```

OUT	<code>num_cat</code>	current number of categories (integer)
-----	----------------------	--

C binding

```
int MPI_T_category_get_num(int *num_cat)
```

Individual category information can then be queried by calling the following function:

```
MPI_T_CATEGORY_GET_INFO(cat_index, name, name_len, desc, desc_len, num_cvars,
                        num_pvars, num_categories)
```

IN	<code>cat_index</code>	index of the category to be queried (integer)
OUT	<code>name</code>	buffer to return the string containing the name of the category (string)
INOUT	<code>name_len</code>	length of the string and/or buffer for name (integer)
OUT	<code>desc</code>	buffer to return the string containing the description of the category (string)
INOUT	<code>desc_len</code>	length of the string and/or buffer for desc (integer)
OUT	<code>num_cvars</code>	number of control variables in the category (integer)
OUT	<code>num_pvars</code>	number of performance variables in the category (integer)
OUT	<code>num_categories</code>	number of categories contained in the category (integer)

C binding

```
int MPI_T_category_get_info(int cat_index, char *name, int *name_len,
                          char *desc, int *desc_len, int *num_cvars, int *num_pvars,
                          int *num_categories)
```

The arguments `name` and `name_len` are used to return the name of the category as described in Section 16.3.3.

The routine is required to return a name of at least length one. This name must be unique with respect to all other names for categories used by the MPI implementation.

If any OUT parameter to `MPI_T_CATEGORY_GET_INFO` is the NULL pointer, the implementation will ignore the parameter and not return a value for the parameter.

The arguments `desc` and `desc_len` are used to return the description of the category as described in Section 16.3.3.

Returning a description is optional. If an MPI implementation decides not to return a description, the first character for `desc` must be set to the null character and `desc_len` must be set to one at the return of this call.

The function returns the number of control variables, performance variables and other categories contained in the queried category in the arguments `num_cvars`, `num_pvars`, and `num_categories`, respectively.

If the name of a category is equivalent across connected MPI processes, then the returned description must be equivalent.

`MPI_T_CATEGORY_GET_NUM_EVENTS(cat_index, num_events)`

IN	<code>cat_index</code>	index of the category to be queried (integer)
OUT	<code>num_events</code>	number of event types in the category (integer)

C binding

```
int MPI_T_category_get_num_events(int cat_index, int *num_events)
```

`MPI_T_CATEGORY_GET_NUM_EVENTS` returns the number of event types contained in the queried category.

`MPI_T_CATEGORY_GET_INDEX(name, cat_index)`

IN	<code>name</code>	the name of the category (string)
OUT	<code>cat_index</code>	the index of the category (integer)

C binding

```
int MPI_T_category_get_index(const char *name, int *cat_index)
```

`MPI_T_CATEGORY_GET_INDEX` is a function for retrieving the index of a category given a known category name. The `name` parameter is provided by the caller, and `cat_index` is returned by the MPI implementation. The `name` parameter is a string terminated with a null character.

This routine returns `MPI_SUCCESS` on success and returns `MPI_T_ERR_INVALID_NAME` if `name` does not match the name of any category provided by the implementation at the time of the call.

Rationale. This routine is provided to enable fast retrieval of a category index by a tool, assuming it knows the name of the category for which it is looking. The number of categories exposed by the implementation can change over time, so it is not possible for the tool to simply iterate over the list of categories once at initialization. Although using MPI implementation specific category names is not portable across MPI implementations, tool developers may choose to take this route for lower overhead at runtime because the tool will not have to iterate over the entire set of categories to find a specific one. (*End of rationale.*)

Category Member Query Functions

`MPI_T_CATEGORY_GET_CVARS(cat_index, len, indices)`

IN	<code>cat_index</code>	index of the category to be queried, in the range from 0 to <code>num_cat</code> - 1 (integer)
IN	<code>len</code>	the length of the indices array (integer)
OUT	<code>indices</code>	an integer array of size <code>len</code> , indicating control variable indices (array of integers)

C binding

```
int MPI_T_category_get_cvars(int cat_index, int len, int indices[])
```

MPI_T_CATEGORY_GET_CVARS can be used to query which control variables are contained in a particular category. A category contains zero or more control variables.

```
MPI_T_CATEGORY_GET_PVARS(cat_index, len, indices)
```

IN	cat_index	index of the category to be queried, in the range from 0 to num_cat - 1 (integer)
IN	len	the length of the indices array (integer)
OUT	indices	an integer array of size len, indicating performance variable indices (array of integers)

C binding

```
int MPI_T_category_get_pvars(int cat_index, int len, int indices[])
```

MPI_T_CATEGORY_GET_PVARS can be used to query which performance variables are contained in a particular category. A category contains zero or more performance variables.

```
MPI_T_CATEGORY_GET_EVENTS(cat_index, len, indices)
```

IN	cat_index	index of the category to be queried, in the range from 0 to num_cat - 1 (integer)
IN	len	the length of the indices array (integer)
OUT	indices	an integer array of size len, indicating event type indices (array of integers)

C binding

```
int MPI_T_category_get_events(int cat_index, int len, int indices[])
```

MPI_T_CATEGORY_GET_EVENTS can be used to query which event types are contained in a particular category. A category contains zero or more event types.

```
MPI_T_CATEGORY_GET_CATEGORIES(cat_index, len, indices)
```

IN	cat_index	index of the category to be queried, in the range from 0 to num_cat - 1 (integer)
IN	len	the length of the indices array (integer)
OUT	indices	an integer array of size len, indicating category indices (array of integers)

C binding

```
int MPI_T_category_get_categories(int cat_index, int len, int indices[])
```

MPI_T_CATEGORY_GET_CATEGORIES can be used to query which other categories are contained in a particular category. A category contains zero or more other categories.

As mentioned above, MPI implementations can grow the number of categories as well as the number of variables or other categories within a category. In order to allow users of the MPI tool information interface to check quickly whether new categories have been added or new variables or categories have been added to a category, MPI maintains an update number that is monotonically increasing during the execution and is returned by the following function:

`MPI_T_CATEGORY_CHANGED(update_number)`

OUT `update_number` update number (integer)

C binding

`int MPI_T_category_changed(int *update_number)`

If two calls to this routine return the same update number, it is guaranteed that the category information has not changed between the two calls. If the update number retrieved from the second call is higher, then some categories have been added or expanded. If the number of changes to categories exceeds the limit of `update_number`, an implementation shall set `update_number` to the maximum possible value for the type of `update_number`.

The index values returned in indices by `MPI_T_CATEGORY_GET_CVARS`, `MPI_T_CATEGORY_GET_PVARS`, `MPI_T_CATEGORY_GET_EVENTS`, and `MPI_T_CATEGORY_GET_CATEGORIES` can be used as input to `MPI_T_CVAR_GET_INFO`, `MPI_T_PVAR_GET_INFO`, `MPI_T_EVENT_GET_INFO`, and `MPI_T_CATEGORY_GET_INFO`, respectively.

The user is responsible for allocating the arrays passed into the functions `MPI_T_CATEGORY_GET_CVARS`, `MPI_T_CATEGORY_GET_PVARS`, `MPI_T_CATEGORY_GET_EVENTS`, and `MPI_T_CATEGORY_GET_CATEGORIES`. Starting from array index 0, each function writes up to `len` elements into the array. If the category contains more than `len` elements, the function returns an arbitrary subset of size `len`. Otherwise, the entire set of elements is returned in the beginning entries of the array, and any remaining array entries are not modified.

16.3.10 Return Codes for the MPI Tool Information Interface

All procedures defined as part of the MPI tool information interface return an integer *return code* (see Table 16.7) to indicate whether the function was completed successfully or was aborted. For the former case, the value `MPI_SUCCESS` is returned. In the latter case, the return code indicates the reason for not completing the routine. Regardless of whether the return code is `MPI_SUCCESS` or indicates that the procedure abnormally terminated, the MPI process continues normal execution and does not invoke any MPI error handler. The MPI implementation is not required to check all user-provided parameters; if a user passes invalid parameter values to any routine, the behavior of the implementation is undefined.

All return codes with the prefix `MPI_T_ERR_` must be unique values and cannot overlap with any error codes or error classes returned by the MPI implementation. They must also satisfy

$$0 = \text{MPI_SUCCESS} < \text{MPI_T_ERR_XXX} \leq \text{MPI_ERR_LASTCODE}.$$

16.3.11 Profiling Interface

All requirements for the profiling interface, as described in Section 16.2, also apply to the MPI tool information interface. All rules, guidelines, and recommendations from Section 16.2 apply equally to procedures defined as part of the MPI tool information interface.

DRAFT

Table 16.7: Return codes used in procedures of the MPI tool information interface

Return Code	Description
Return Codes for All Procedures in the MPI Tool Information Interface	
MPI_SUCCESS	Call completed successfully
MPI_T_ERR_INVALID	Invalid or bad parameter value(s)
MPI_T_ERR_MEMORY	Out of memory
MPI_T_ERR_NOT_INITIALIZED	Interface not initialized
MPI_T_ERR_CANNOT_INIT	Interface not in the state to be initialized
MPI_T_ERR_NOT_ACCESSIBLE	Requested functionality not accessible
Return Codes for Datatype Procedures: MPI_T_ENUM_XXX	
MPI_T_ERR_INVALID_INDEX	The enumeration index is invalid
Return Codes for Variable, Category, and Event Query Procedures: MPI_T_XXX_GET_XXX	
MPI_T_ERR_INVALID_INDEX	The variable or category index is invalid
MPI_T_ERR_INVALID_NAME	The variable or category name is invalid
Return Codes for Handle Procedures: MPI_T_XXX_{ALLOC FREE}	
MPI_T_ERR_INVALID_INDEX	The variable index is invalid
MPI_T_ERR_INVALID_HANDLE	The handle is invalid
MPI_T_ERR_OUT_OF_HANDLES	No more handles available
Return Codes for Performance Experiment Session Procedures: MPI_T_PVAR_SESSION_XXX	
MPI_T_ERR_OUT_OF_SESSIONS	No more sessions available
MPI_T_ERR_INVALID_SESSION	Session argument is not a valid session
Return Codes for Control Variable Access Procedures: MPI_T_CVAR_{READ WRITE}	
MPI_T_ERR_CVAR_SET_NOT_NOW	Variable cannot be set at this moment
MPI_T_ERR_CVAR_SET_NEVER	Variable cannot be set until end of execution
MPI_T_ERR_INVALID_HANDLE	The handle is invalid
Return Codes for Performance Variable Access and Control Procedures:	
MPI_T_PVAR_{START STOP READ WRITE RESET READREST}	
MPI_T_ERR_INVALID_HANDLE	The handle is invalid
MPI_T_ERR_INVALID_SESSION	Performance experiment session argument is invalid
MPI_T_ERR_PVAR_NO_STARTSTOP	Variable cannot be started or stopped (for MPI_T_PVAR_START and MPI_T_PVAR_STOP)
MPI_T_ERR_PVAR_NO_WRITE	Variable cannot be written or reset (for MPI_T_PVAR_WRITE and MPI_T_PVAR_RESET)
MPI_T_ERR_PVAR_NO_ATOMIC	Variable cannot be read and written atomically (for MPI_T_PVAR_READRESET)
Return Codes for Source Procedures: MPI_T_SOURCE_XXX	
MPI_T_ERR_INVALID_INDEX	The source index is invalid
MPI_T_ERR_NOT_SUPPORTED	Requested functionality not supported
Return Codes for Category Procedures: MPI_T_CATEGORY_XXX	
MPI_T_ERR_INVALID_INDEX	The category index is invalid

DRAFT

Chapter 17

Process Fault Tolerance

17.1 Introduction

In distributed systems with numerous or complex components, a serious risk is that a component fault manifests as a process failure that disrupts the normal execution of a long running application. A process failure is a common outcome for many hardware, network, or software faults that cause a process to crash; it can be more formally defined as a fail-stop failure: the affected MPI process unexpectedly and permanently stops communicating. This chapter introduces MPI features that support the development of applications, libraries, and programming languages that can tolerate MPI process failures. The primary goal is to specify error classes and interfaces that permit users to continue simple MPI communication (e.g., some point-to-point patterns) after failures have impacted the execution and rebuild MPI objects (e.g., communicators, etc.) as needed to restore the full capability of MPI to carry out communication operations (like collective communications). This specification does not include mechanisms to restore the application data lost due to process failures. The literature is rich with diverse fault tolerance techniques that the users may employ at their discretion, including checkpoint-restart, algorithmic dataset recovery, and continuation ignoring failed MPI processes. All these fault tolerance approaches benefit from, and often require, the definitions and interfaces specified in this chapter in order to resume communicating after a failure.

The expected behavior of MPI in the case of an MPI process failure is defined by the following statements: any MPI operation that involves a failed process must eventually either succeed or raise an MPI error (see Section 17.2); an MPI operation that does not involve a failed process will complete normally, unless interrupted by the user through provided functionality. If an application needs comprehensive knowledge of failures, it can use the interfaces defined in Section 17.3 to explicitly propagate the notification of locally detected failures, or set communicators in specific modes that enforce such propagation, see Section 17.2.2.

Some usage patterns on reliable machines do not require fault tolerance. An MPI implementation that does not tolerate process failures must never raise a *fault tolerance error* (as listed in Section 17.4). Applications using the interfaces defined in this chapter must be portable across MPI implementations (including those which do not provide fault tolerance). If an MPI implementation does not provide fault tolerance features, it may exhibit undefined behavior after a process failure at any MPI process.

Advice to users. The MPI standard does not specify transparent process recovery upon MPI process failure. In particular, restoring the lost dataset, spawning spare processes or taking other recovery actions are the responsibility of the user.

Many of the operations and semantics described in this chapter are applicable only when

the MPI application has replaced the default error handler `MPI_ERRORS_ARE_FATAL` on the communicators it uses. (*End of advice to users.*)

17.2 Failure Notification

All MPI procedures must eventually return (possibly by raising one of the process failure error classes listed in Section 17.4) even if they involve a failed MPI process.

Advice to implementors. As long as an implementation can successfully complete some operations during a completing procedure, it may choose to delay raising an error. Another valid implementation might choose to raise an error as quickly as possible. (*End of advice to implementors.*)

When an operation raises a process failure error it may not satisfy its specification (like any other error, see 9.4).

17.2.1 Revoked Communicator

A communicator is **revoked** at a given MPI process either when `MPI_COMM_REVOKE` is locally called on it, or when any MPI operation on the communicator raises an error of class `MPI_ERR_REVOKED` at that process. Once a communicator has been revoked at an MPI process, all subsequent non-local operations on that communicator are considered local and must complete by raising an error of class `MPI_ERR_REVOKED` at that MPI process.

Advice to implementors. An implementation may detect that a communicator should become revoked when completing procedures can still complete some operations successfully. The implementation may delay raising the `MPI_ERR_REVOKED` error to a later operation, but it cannot delay revoking the communicator indefinitely and should favor raising the error promptly. (*End of advice to implementors.*)

17.2.2 Error Reporting Range

By default, process failure errors are raised only during communication operations in which a failed process is involved, but users can control the range of MPI processes whose failure cause MPI operations to raise errors on their communicators by setting the following values to the info key "mpi_error_range" on their communicators:

"operation" A process fault tolerance error shall be raised only if the operation involves a failed MPI process (e.g., a point-to-point message between two non-failed MPI processes shall not raise a process fault tolerance error). An MPI process is considered involved in an operation (for the purpose of this definition) if its failure may prevent it from calling a matching procedure for the operation, that is, an MPI process is involved in a communication if any of the following is true: the process is in the group(s) over which the operation is collective; the process is a destination or a specified or a matched source in a point-to-point communication; or the process belongs to the source group in an `MPI_ANY_SOURCE` receive operation.

"group" The failure of any MPI process in the group of the communicator causes the communicator to become revoked (as if `MPI_COMM_REVOKE` had been called on the communicator).

"universe" The failure of any MPI process in the MPI universe causes the communicator to become revoked (as if `MPI_COMM_REVOKE` had been called on the communicator).

When the info key "mpi_error_range" is not set, it is equivalent to having the key set to "operation".

17.2.3 Fault Tolerance Errors in Point-to-Point Communication

An MPI point-to-point communication raises errors of the following classes to notify users that the communication could not complete successfully because of the failure of at least one involved MPI process:

- `MPI_ERR_REVOKED` indicates that the communicator is revoked.
- `MPI_ERR_PROC_FAILED_PENDING` indicates, for a nonblocking or persistent communication, that the communication is a receive operation from `MPI_ANY_SOURCE` and no send operation has matched, yet a potential sending MPI process has failed. The request remains active.
- In all other cases, the operation raises an error of class `MPI_ERR_PROC_FAILED` to indicate that the failure prevents the operation from following its failure-free specification. If there is a request representing a point-to-point communication, it is completed.

17.2.4 Fault Tolerance Errors in Collective Operations

A collective operation raises errors of the following classes to notify users that the communication could not complete successfully because of the failure of at least one involved MPI process:

- `MPI_ERR_REVOKED` indicates that the communicator is revoked.
- `MPI_ERR_PROC_FAILED` indicates that the failure prevents the operation from following its failure-free specification. If there is a request representing a collective communication, it is completed.

Advice to users.

Depending on how the collective operation is implemented and when an MPI process failure occurs, some participating MPI processes may raise an error while other MPI processes return successfully from the same collective operation. For example, in `MPI_BCAST`, the root process may succeed before a failed MPI process disrupts the operation, resulting in some other MPI processes raising an error.

(End of advice to users.)

Advice to users.

Note that communicator creation procedures (e.g., `MPI_COMM_DUP` or `MPI_COMM_SPLIT`) are collective operations. As such, if a failure happened during the procedure, an error might be raised at some MPI processes while others succeed and obtain a new communicator handle. Although it is valid to communicate between MPI processes that succeeded in creating the new communicator handle, the user is responsible for ensuring a consistent view of the communicator creation, if needed.

(End of advice to users.)

After an MPI process failure, `MPI_COMM_FREE` (as with all other collective operations) may not complete successfully at all MPI processes. For any MPI process that receives the return code `MPI_SUCCESS`, the behavior is defined in Section 7.4.3. If an MPI process raises a process failure error (classes `MPI_ERR_PROC_FAILED` or `MPI_ERR_REVOKED`), the communicator handle `comm` is set to `MPI_COMM_NULL`; however, the implementation makes no guarantee about the success or failure of the `MPI_COMM_FREE` operation, locally or remotely.

Advice to users. Users are encouraged to call `MPI_COMM_FREE` on communicators they do not wish to use anymore, even when they contain failed MPI processes. Although the operation may raise a fault tolerance error and not synchronize properly, this gives a high quality implementation an opportunity to release local resources and memory consumed by the object. (*End of advice to users.*)

Error Uniformity

As noted above, by default, collective communication do not enforce uniformity in error raising across MPI processes. Despite the performance advantages that non-uniformity offers, a common usage pattern in applications is to transform non-uniform error raising into a uniform behavior across all MPI processes of the group.

Users can set collective operations on a communicator to enforce uniform error raising by setting the following values in the info key "mpi_error_uniform" on the communicator:

"local" Process fault tolerance errors are raised to indicate that an MPI process failure prevents from guaranteeing the specified behavior at this process for the collective communication operation (e.g., the output buffer contains invalid data). Other MPI processes may have satisfied their specification (e.g., the output buffer is valid at that process) and may have returned `MPI_SUCCESS`.

"coll" Process fault tolerance errors are raised to indicate that an MPI process failure prevents from guaranteeing the specified behavior at any process for the collective communication operation. Non-synchronizing collective communication become synchronizing.

"construct" Process fault tolerance errors are raised to indicate that an MPI process failure prevents from guaranteeing the specified behavior at any process for the collective MPI communication context constructor/destructor operation (e.g., `MPI_COMM_DUP` could not create a new communicator at any process in the group of the communicator). Non-synchronizing context constructor/destructor operations become synchronizing. Collective communication that do not create or free a communication context are not impacted.

When the info key "mpi_error_uniform" is not set on the communicator, it is equivalent to having the key set to "local". The info key "mpi_error_uniform" must be set to the same value at all MPI processes for a given communicator.

17.2.5 Files and One-sided Operations

Advice to users. Support for fault tolerance with MPI Files and RMA Windows is not defined by the MPI standard. (*End of advice to users.*)

17.3 Failure Mitigation Procedures

17.3.1 Communicator Procedures

Process failure notification is not global in MPI. MPI processes that do not call operations involving a failed MPI process are possibly never notified of its failure (see Section 17.2). If a notification must be propagated, MPI provides a procedure to revoke a communicator at all members.

MPI_COMM_REVOKE(comm)

IN comm communicator (handle)

C binding

```
int MPI_Comm_revoke(MPI_Comm comm)
```

Fortran 2008 binding

```
MPI_Comm_revoke(comm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_REVOKE(COMM, IERROR)
  INTEGER COMM, IERROR
```

This procedure notifies all MPI processes in the groups (local and remote) associated with the communicator `comm` that this communicator is revoked. The revocation of a communicator by any MPI process completes non-local MPI operations on `comm` at all MPI processes by raising an error of class [MPI_ERR_REVOKED](#). This procedure is not collective and therefore does not have a matching call on remote MPI processes. All non-failed MPI processes belonging to `comm` will be notified of the revocation despite failures.

MPI_COMM_IS_REVOKED(comm, flag)

IN comm communicator (handle)
OUT flag true if the communicator is revoked (logical)

C binding

```
int MPI_Comm_is_revoked(MPI_Comm comm, int *flag)
```

Fortran 2008 binding

```
MPI_Comm_is_revoked(comm, flag, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  LOGICAL, INTENT(OUT) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_IS_REVOKED(COMM, FLAG, IERROR)
  INTEGER COMM, IERROR
  LOGICAL FLAG
```

Returns `flag = true` if the communicator associated with the handle `comm` is revoked at the calling process. It returns `flag = false` otherwise. The operation is local.

Advice to users. In a multithreaded application, a thread calling `MPI_COMM_IS_REVOKED` may return `flag = true` before the operation that raises the first exception of class `MPI_ERR_REVOKED` has completed in a concurrent thread. (*End of advice to users.*)

`MPI_COMM_GET_FAILED(comm, failedgrp)`

IN	<code>comm</code>	communicator (handle)
OUT	<code>failedgrp</code>	group of failed processes (handle)

C binding

```
int MPI_Comm_get_failed(MPI_Comm comm, MPI_Group *failedgrp)
```

Fortran 2008 binding

```
MPI_Comm_get_failed(comm, failedgrp, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Group), INTENT(OUT) :: failedgrp
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_COMM_GET_FAILED(COMM, FAILEDGRP, IERROR)
  INTEGER COMM, FAILEDGRP, IERROR
```

This local procedure returns the group `failedgrp` of MPI processes from the communicator `comm` that are known to have failed by the calling MPI process. Any MPI process whose failure caused a procedure on `comm` to raise a process failure error at this MPI process must appear in that group. The `failedgrp` can be empty, that is, equal to `MPI_GROUP_EMPTY`.

When an MPI process is a member of `failedgrp`, it is also a member, with the same rank, in any group produced by a subsequent call to `MPI_COMM_GET_FAILED` on `comm` at the same calling process.

Advice to users. MPI makes no assumption about asynchronous progress of the failure detection. A valid MPI implementation may choose to update the group of locally known failed MPI processes only when it enters a procedure that must raise a fault tolerance error.

It is possible that only the calling MPI process has detected the reported failure. (*End of advice to users.*)

```

MPI_COMM_ACK_FAILED(comm, num_to_ack, num_acked)
    IN      comm      communicator (handle)
    IN      num_to_ack Maximum number of process failures to acknowledge
                        (integer)
    OUT     num_acked  Number of process failures acknowledged (integer)

```

C binding

```
int MPI_Comm_ack_failed(MPI_Comm comm, int num_to_ack, int *num_acked)
```

Fortran 2008 binding

```

MPI_Comm_ack_failed(comm, num_to_ack, num_acked, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(IN) :: num_to_ack
    INTEGER, INTENT(OUT) :: num_acked
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_COMM_ACK_FAILED(COMM, NUM_TO_ACK, NUM_ACKED, IERROR)
    INTEGER COMM, NUM_TO_ACK, NUM_ACKED, IERROR

```

This local procedure is used to **acknowledge** locally notified failures on `comm`. The operation acknowledges the first `num_to_ack` process failures on `comm`, that is, it acknowledges the failure of members with a rank lower than `num_to_ack` in the group that would be produced by a concurrent call to [MPI_COMM_GET_FAILED](#) on the same `comm`.

The operation also sets the value of `num_acked` to the current number of acknowledged process failures in `comm`, that is, a process failure has been acknowledged on `comm` if and only if the rank of the process is lower than `num_acked` in the group that would be produced by a subsequent call to [MPI_COMM_GET_FAILED](#) on the same `comm`.

`num_acked` can be larger than `num_to_ack` when process failures have been acknowledged in a prior call to [MPI_COMM_ACK_FAILED](#).

After an MPI process failure is acknowledged on `comm`, unmatched [MPI_ANY_SOURCE](#) receive operations on the same `comm` that would have raised an error of class [MPI_ERR_PROC_FAILED_PENDING](#) (see Section 17.2.3) proceed without further raising errors due to this acknowledged failure.

Advice to users. One may query, without side effect, for the number of currently acknowledged process failures in `comm` by supplying 0 in `num_to_ack`. Conversely, one may unconditionally acknowledge all currently known process failures in `comm` by supplying the size of the group of `comm` in `num_to_ack`. Note that the number of acknowledged MPI processes, as returned in `num_acked`, can be smaller or larger than the value supplied in `num_to_ack`; It is however never larger than the size of the group returned by a subsequent call to [MPI_COMM_GET_FAILED](#).

Calling [MPI_COMM_ACK_FAILED](#) on a communicator with failed MPI processes has no effect on collective operations. If a collective operation would raise an error due to the communicator containing a failed process (as defined in Section 17.2.4), it can continue to raise an error even after the failure has been acknowledged. (*End of advice to users.*)

17.4 Fault Tolerance Error Codes and Classes

Among the error classes defined in Section 9.4, the following are **fault tolerance error** classes:

<code>MPI_ERR_PROC_FAILED</code>	The operation could not complete because of an MPI process failure (a fail-stop failure).
<code>MPI_ERR_PROC_FAILED_PENDING</code>	The operation was interrupted by an MPI process failure (a fail-stop failure). The request is still pending and the operation may be completed later.
<code>MPI_ERR_REVOKED</code>	The communication object used in the operation has been revoked.

Table 17.1: Fault tolerance error classes

17.5 Examples

17.5.1 Fault Tolerant Manager/Worker

The example below presents a manager code that handles worker failures by discarding failed worker MPI processes and resubmitting the work to the remaining workers. It demonstrates the different failure cases that may occur with receive operations using `MPI_ANY_SOURCE` as discussed in Section 17.2.3.

Example 17.1. Fault Tolerant Manager Example

```

int manager(void)
{
    MPI_Comm_set_errhandler(comm, MPI_ERRORS_RETURN);
    MPI_Comm_size(comm, &size);
    MPI_Comm_group(comm, &gcomm);

    /* ... submit the initial work requests ... */

    /* Progress engine: Get answers, send new requests,
       and handle process failures */
    MPI_Irecv(buffer, 1, MPI_INT, MPI_ANY_SOURCE, tag, comm, &req);
    while( (active_workers > 0) && work_available ) {
        rc = MPI_Wait(&req, &status);
        if( MPI_SUCCESS == rc ) {
            /* ... process the answer and update work_available ... */
        }
        else {
            MPI_Error_class(rc, &ec);
            if( (MPI_ERR_PROC_FAILED == ec) ||
                (MPI_ERR_PROC_FAILED_PENDING == ec) ) {
                /* We ack the full size of comm, so we will ack
                 * unconditionally. Variable gsize will contain all

```

```

    * currently known failures. */
    MPI_Comm_ack_failed(comm, size, &gsize);

    /* ... find the lost work and requeue it ... */
    MPI_Comm_get_failed(comm, &g);
    granks = (int*)calloc(active_workers-gsize-1,
                          sizeof(int));
    cranks = (int*)calloc(active_workers-gsize-1,
                          sizeof(int));
    for(i = active_workers; i < gsize; i++)
        granks[i-active_workers] = i;
    MPI_Group_translate_ranks(g, gsize, granks,
                             gcomm, cranks);
    /* iterate over newly failed procs */
    for(i = active_workers; i < gsize; i++) {
        /* resubmit the work */
    }
    free(cranks); free(granks);
    MPI_Group_free(&g);

    active_workers = size - gsize - 1;

    /* no need to repost: the request is still pending */
    if( ec == MPI_ERR_PROC_FAILED_PENDING )
        continue;
    }
}
/* get ready to receive more notifications from workers */
MPI_Irecv(buffer, 1, MPI_INT, MPI_ANY_SOURCE, tag, comm, &req);
}
/* ... cancel request and cleanup ... */
}

```

DRAFT

Chapter 18

Deprecated Interfaces

18.1 Deprecated since MPI-2.0

The following function is deprecated and is superseded by [MPI_COMM_CREATE_KEYVAL](#) in MPI-2.0. The language independent definition of the deprecated function is the same as that of the new function, except for the function name and a different behavior in the C/Fortran language interoperability, see Section 21.3.7. The language bindings are modified.

MPI_KEYVAL_CREATE(copy_fn, delete_fn, keyval, extra_state)

IN	copy_fn	Copy callback function for keyval
IN	delete_fn	Delete callback function for keyval
OUT	keyval	key value for future access (integer)
IN	extra_state	Extra state for callback functions

C binding

```
int MPI_Keyval_create(MPI_Copy_function *copy_fn,  
                     MPI_Delete_function *delete_fn, int *keyval, void *extra_state)
```

For this routine, an interface within the `mpi_f08` module was never defined.

Fortran binding

```
MPI_KEYVAL_CREATE(COPY_FN, DELETE_FN, KEYVAL, EXTRA_STATE, IERROR)  
EXTERNAL COPY_FN, DELETE_FN  
INTEGER KEYVAL, EXTRA_STATE, IERROR
```

The `copy_fn` function is invoked when a communicator is duplicated by [MPI_COMM_DUP](#). `copy_fn` should be of type `MPI_Copy_function`, which is defined as follows:

```
typedef int MPI_Copy_function(MPI_Comm oldcomm, int keyval, void *extra_state,  
                             void *attribute_val_in, void *attribute_val_out, int *flag);
```

A Fortran declaration for such a function is as follows:

For this routine, an interface within the `mpi_f08` module was never defined.

```
SUBROUTINE COPY_FUNCTION(OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,  
                        ATTRIBUTE_VAL_OUT, FLAG, IERR)  
    INTEGER OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT,  
           IERR  
    LOGICAL FLAG
```

`copy_fn` may be specified as `MPI_NULL_COPY_FN` or `MPI_DUP_FN` from either C or Fortran; `MPI_NULL_COPY_FN` is a function that does nothing other than return `flag = 0` and `MPI_SUCCESS`. `MPI_DUP_FN` is a simple-minded copy function that sets `flag = 1`, returns the value of `attribute_val_in` in `attribute_val_out`, and returns `MPI_SUCCESS`. Note that `MPI_NULL_COPY_FN` and `MPI_DUP_FN` are also deprecated.

Analogous to `copy_fn` is a callback deletion function, defined as follows. The `delete_fn` function is invoked when a communicator is deleted by `MPI_COMM_FREE` or when a call is made explicitly to `MPI_ATTR_DELETE`. `delete_fn` should be of type `MPI_Delete_function`, which is defined as follows:

```
typedef int MPI_Delete_function(MPI_Comm comm, int keyval, void *attribute_val,
                               void *extra_state);
```

A Fortran declaration for such a function is as follows:
For this routine, an interface within the `mpi_f08` module was never defined.

```
SUBROUTINE DELETE_FUNCTION(COMM, KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERR)
  INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERR
```

`delete_fn` may be specified as `MPI_NULL_DELETE_FN` from either C or Fortran; `MPI_NULL_DELETE_FN` is a function that does nothing other than return `MPI_SUCCESS`. Note that `MPI_NULL_DELETE_FN` is also deprecated.

The following function is deprecated and is superseded by `MPI_COMM_FREE_KEYVAL` in MPI-2.0. The language independent definition of the deprecated function is the same as the new function, except for the function name. The language bindings are modified.

```
MPI_KEYVAL_FREE(keyval)
  INOUT  keyval      Frees the integer key value (integer)
```

C binding

```
int MPI_Keyval_free(int *keyval)
```

For this routine, an interface within the `mpi_f08` module was never defined.

Fortran binding

```
MPI_KEYVAL_FREE(KEYVAL, IERROR)
  INTEGER KEYVAL, IERROR
```

The following function is deprecated and is superseded by `MPI_COMM_SET_ATTR` in MPI-2.0. The language independent definition of the deprecated function is the same as the new function, except for the function name. The language bindings are modified.

```
MPI_ATTR_PUT(comm, keyval, attribute_val)
  INOUT  comm      communicator to which attribute will be attached
                        (handle)
  IN      keyval    key value, as returned by MPI_KEYVAL_CREATE
                        (integer)
```

IN attribute_val attribute value

C binding

```
int MPI_Attr_put(MPI_Comm comm, int keyval, void *attribute_val)
```

For this routine, an interface within the `mpi_f08` module was never defined.

Fortran binding

```
MPI_ATTR_PUT(COMM, KEYVAL, ATTRIBUTE_VAL, IERROR)
  INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, IERROR
```

The following function is deprecated and is superseded by [MPI_COMM_GET_ATTR](#) in MPI-2.0. The language independent definition of the deprecated function is the same as the new function, except for the function name. The language bindings are modified.

```
MPI_ATTR_GET(comm, keyval, attribute_val, flag)
```

IN	comm	communicator to which attribute is attached (handle)
IN	keyval	key value (integer)
OUT	attribute_val	attribute value, unless flag = false
OUT	flag	true if an attribute value was extracted; false if no attribute is associated with the key

C binding

```
int MPI_Attr_get(MPI_Comm comm, int keyval, void *attribute_val, int *flag)
```

For this routine, an interface within the `mpi_f08` module was never defined.

Fortran binding

```
MPI_ATTR_GET(COMM, KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
  INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, IERROR
  LOGICAL FLAG
```

The following function is deprecated and is superseded by [MPI_COMM_DELETE_ATTR](#) in MPI-2.0. The language independent definition of the deprecated function is the same as the new function, except for the function name. The language bindings are modified.

```
MPI_ATTR_DELETE(comm, keyval)
```

INOUT	comm	communicator to which attribute is attached (handle)
IN	keyval	The key value of the deleted attribute (integer)

C binding

```
int MPI_Attr_delete(MPI_Comm comm, int keyval)
```

For this routine, an interface within the `mpi_f08` module was never defined.

Fortran binding

```
MPI_ATTR_DELETE(COMM, KEYVAL, IERROR)
    INTEGER COMM, KEYVAL, IERROR
```

18.2 Deprecated since MPI-2.2

The entire set of C++ language bindings was deprecated as of MPI-2.2 and removed in MPI-3.0. See Chapter 19, [Removed Interfaces](#) for more information.

The following function typedefs have been deprecated and are superseded by new names. Other than the typedef names, the function signatures are exactly the same; the names were updated to match conventions of other function typedef names.

Deprecated Name	New Name
MPI_Comm_errhandler_fn	MPI_Comm_errhandler_function
MPI_File_errhandler_fn	MPI_File_errhandler_function
MPI_Win_errhandler_fn	MPI_Win_errhandler_function

18.3 Deprecated since MPI-4.0

Cancelling a send request by calling [MPI_CANCEL](#) has been deprecated and may be removed in a future version of the MPI specification.

The following function is deprecated and is superseded by the new [MPI_INFO_GET_STRING](#) call in MPI-4.0.

```
MPI_INFO_GET(info, key, valuelen, value, flag)
```

IN	info	info object (handle)
IN	key	key (string)
IN	valuelen	length of value associated with key (integer)
OUT	value	value (string)
OUT	flag	true if key defined, false if not (logical)

C binding

```
int MPI_Info_get(MPI_Info info, const char *key, int valuelen, char *value,
    int *flag)
```

Fortran 2008 binding

```
MPI_Info_get(info, key, valuelen, value, flag, ierror)
    TYPE(MPI_Info), INTENT(IN) :: info
    CHARACTER(LEN=*), INTENT(IN) :: key
    INTEGER, INTENT(IN) :: valuelen
    CHARACTER(LEN=valuelen), INTENT(OUT) :: value
    LOGICAL, INTENT(OUT) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INFO_GET(INFO, KEY, VALUELEN, VALUE, FLAG, IERROR)
```

```
    INTEGER INFO, VALUELEN, IERROR
```

```
    CHARACTER*(*) KEY, VALUE
```

```
    LOGICAL FLAG
```

This function retrieves the value associated with key in a previous call to [MPI_INFO_SET](#). If such a key exists, it sets flag to true and returns the value in value, otherwise it sets flag to false and leaves value unchanged. valuelen is the number of characters available in value. If it is less than the actual size of the value, the value is truncated. In C, valuelen should be one less than the amount of allocated space to allow for the null terminator.

If key is larger than [MPI_MAX_INFO_KEY](#), the call is erroneous.

The function [MPI_INFO_GET](#) is allowed to be called at any time, following the description for MPI functionality that is always available in Section [11.4.1](#).

The following function is deprecated and is superseded by the new [MPI_INFO_GET_STRING](#) call in MPI-4.0.

```
MPI_INFO_GET_VALUELEN(info, key, valuelen, flag)
```

```
IN      info          info object (handle)
```

```
IN      key           key (string)
```

```
OUT     valuelen      length of value associated with key (integer)
```

```
OUT     flag          true if key defined, false if not (logical)
```

C binding

```
int MPI_Info_get_valuelen(MPI_Info info, const char *key, int *valuelen,
                          int *flag)
```

Fortran 2008 binding

```
MPI_Info_get_valuelen(info, key, valuelen, flag, ierror)
```

```
    TYPE(MPI_Info), INTENT(IN) :: info
```

```
    CHARACTER(LEN=*), INTENT(IN) :: key
```

```
    INTEGER, INTENT(OUT) :: valuelen
```

```
    LOGICAL, INTENT(OUT) :: flag
```

```
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_INFO_GET_VALUELEN(INFO, KEY, VALUELEN, FLAG, IERROR)
```

```
    INTEGER INFO, VALUELEN, IERROR
```

```
    CHARACTER*(*) KEY
```

```
    LOGICAL FLAG
```

Retrieves the length of the value associated with key. If key is defined, valuelen is set to the length of its associated value and flag is set to true. If key is not defined, valuelen is not touched and flag is set to false. The length returned in C does not include the end-of-string character.

If key is larger than [MPI_MAX_INFO_KEY](#), the call is erroneous.

The function `MPI_INFO_GET_VALUELEN` is allowed to be called at any time, following the description for MPI functionality that is always available in Section 11.4.1.

The following return code has been deprecated and is superseded by a new name in MPI-4.0.

Deprecated Name	Replacement Name
<code>MPI_T_ERR_INVALID_ITEM</code>	<code>MPI_T_ERR_INVALID_INDEX</code>

The following Fortran subroutines are deprecated because the Fortran language `storage_size()` and `c_sizeof()` intrinsic functions provide similar functionality. Note that while `MPI_SIZEOF` and `c_sizeof()` return the size in bytes, `storage_size()` provides the size in bits.

`MPI_SIZEOF(x, size)`

IN	x	a Fortran variable of numeric intrinsic type (choice)
OUT	size	size of machine representation of that type (integer)

Fortran 2008 binding

```
MPI_Sizeof(x, size, ierror)
  TYPE(*), DIMENSION(..) :: x
  INTEGER, INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_SIZEOF(X, SIZE, IERROR)
  <type> X
  INTEGER SIZE, IERROR
```

This function returns the size in bytes of the machine representation of the given variable. It is a generic Fortran routine and has a Fortran binding only.

Advice to users. This function is similar to the C `sizeof` operator but behaves slightly differently. If given an array argument, it returns the size of the base element, not the size of the whole array. (*End of advice to users.*)

Rationale. This function is not available in other languages because it would not be useful. (*End of rationale.*)

18.4 Deprecated since MPI-4.1

The use of the `mpif.h` include file has been deprecated. Information supporting the transition to `USE mpi` or `USE mpi_f08` is provided in Section 21.1.4.

The predefined attribute key `MPI_HOST` for `MPI_COMM_WORLD` when using the World Model is deprecated.

`MPI_HOST`: Host process rank, if such exists, `MPI_PROC_NULL`, otherwise.

Host Rank

The value returned for `MPI_HOST` gets the rank of the *HOST* process in the group associated with communicator `MPI_COMM_WORLD`, if there is such. `MPI_PROC_NULL` is returned if there is no host. MPI does not specify what it means for a process to be a *HOST*, nor does it require that a *HOST* exists.

The attribute `MPI_HOST` has the same value on all processes of `MPI_COMM_WORLD`.

Environmental inquiry keys

C type: `const int` (or unnamed enum)

Fortran type: `INTEGER`

`MPI_HOST`

All `MPI_XXX_X` procedures have been deprecated and may be removed in a future version of the MPI specification. In the case of their C binding and their Fortran binding through the `mpi_f08` module, they are superseded by the large count and large byte displacement bindings of their counterpart in the form of `MPI_XXX`.

`MPI_TYPE_SIZE_X(datatype, size)`

IN	<code>datatype</code>	datatype to get information on (handle)
OUT	<code>size</code>	datatype size (integer)

C binding

`int MPI_Type_size_x(MPI_Datatype datatype, MPI_Count *size)`

Fortran 2008 binding

`MPI_Type_size_x(datatype, size, ierror)`
`TYPE(MPI_Datatype), INTENT(IN) :: datatype`
`INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size`
`INTEGER, OPTIONAL, INTENT(OUT) :: ierror`

Fortran binding

`MPI_TYPE_SIZE_X(DATATYPE, SIZE, IERROR)`
`INTEGER DATATYPE, IERROR`
`INTEGER(KIND=MPI_COUNT_KIND) SIZE`

The description of `MPI_TYPE_SIZE` is applicable to this deprecated `MPI_TYPE_SIZE_X` accordingly, see Section 5.1.5.

`MPI_TYPE_GET_EXTENT_X(datatype, lb, extent)`

IN	<code>datatype</code>	datatype to get information on (handle)
OUT	<code>lb</code>	lower bound of datatype (integer)
OUT	<code>extent</code>	extent of datatype (integer)

C binding

`int MPI_Type_get_extent_x(MPI_Datatype datatype, MPI_Count *lb, MPI_Count *extent)`

Fortran 2008 binding

```

MPI_Type_get_extent_x(datatype, lb, extent, ierror)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: lb, extent
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_GET_EXTENT_X(DATATYPE, LB, EXTENT, IERROR)
  INTEGER DATATYPE, IERROR
  INTEGER(KIND=MPI_COUNT_KIND) LB, EXTENT

```

The description of [MPI_TYPE_GET_EXTENT](#) is applicable to this deprecated [MPI_TYPE_GET_EXTENT_X](#) accordingly, see Section 5.1.7.

```

MPI_TYPE_GET_TRUE_EXTENT_X(datatype, true_lb, true_extent)

```

IN	datatype	datatype to get information on (handle)
OUT	true_lb	true lower bound of datatype (integer)
OUT	true_extent	true extent of datatype (integer)

C binding

```

int MPI_Type_get_true_extent_x(MPI_Datatype datatype, MPI_Count *true_lb,
                               MPI_Count *true_extent)

```

Fortran 2008 binding

```

MPI_Type_get_true_extent_x(datatype, true_lb, true_extent, ierror)
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: true_lb, true_extent
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_GET_TRUE_EXTENT_X(DATATYPE, TRUE_LB, TRUE_EXTENT, IERROR)
  INTEGER DATATYPE, IERROR
  INTEGER(KIND=MPI_COUNT_KIND) TRUE_LB, TRUE_EXTENT

```

The description of [MPI_TYPE_GET_TRUE_EXTENT](#) is applicable to this deprecated [MPI_TYPE_GET_TRUE_EXTENT_X](#) accordingly, see Section 5.1.8.

```

MPI_GET_ELEMENTS_X(status, datatype, count)

```

IN	status	return status of receive operation (status)
IN	datatype	datatype used by receive operation (handle)
OUT	count	number of received basic elements (integer)

C binding

```

int MPI_Get_elements_x(const MPI_Status *status, MPI_Datatype datatype,
                      MPI_Count *count)

```


Fortran 2008 binding

```

MPI_Get_elements_x(status, datatype, count, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: count
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_GET_ELEMENTS_X(STATUS, DATATYPE, COUNT, IERROR)
  INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, IERROR
  INTEGER(KIND=MPI_COUNT_KIND) COUNT

```

The description of [MPI_GET_ELEMENTS](#) is applicable to this deprecated `MPI_GET_ELEMENTS_X` accordingly, see Section 5.1.11.

```

MPI_STATUS_SET_ELEMENTS_X(status, datatype, count)

```

INOUT	status	status with which to associate count (status)
IN	datatype	datatype associated with count (handle)
IN	count	number of elements to associate with status (integer)

C binding

```

int MPI_Status_set_elements_x(MPI_Status *status, MPI_Datatype datatype,
                             MPI_Count count)

```

Fortran 2008 binding

```

MPI_Status_set_elements_x(status, datatype, count, ierror)
  TYPE(MPI_Status), INTENT(INOUT) :: status
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_STATUS_SET_ELEMENTS_X(STATUS, DATATYPE, COUNT, IERROR)
  INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, IERROR
  INTEGER(KIND=MPI_COUNT_KIND) COUNT

```

The description of [MPI_STATUS_SET_ELEMENTS](#) is applicable to this deprecated `MPI_STATUS_SET_ELEMENTS_X` accordingly, see Section 14.3.

DRAFT

Chapter 19

Removed Interfaces

19.1 Removed MPI-1 Bindings

19.1.1 Overview

The following MPI-1 bindings were deprecated as of MPI-2 and were removed in MPI-3. They may be provided by an implementation for backwards compatibility, but are not required. Removal of these bindings affects all language-specific definitions thereof. Only the language-neutral bindings are listed when possible.

19.1.2 Removed MPI-1 Functions

Table 19.1 shows the removed MPI-1 functions and their replacements.

Table 19.1: Removed MPI-1 functions and their replacements

Removed	MPI-2 Replacement
MPI_ADDRESS	MPI_GET_ADDRESS
MPI_ERRHANDLER_CREATE	MPI_COMM_CREATE_ERRHANDLER
MPI_ERRHANDLER_GET	MPI_COMM_GET_ERRHANDLER
MPI_ERRHANDLER_SET	MPI_COMM_SET_ERRHANDLER
MPI_TYPE_EXTENT	MPI_TYPE_GET_EXTENT
MPI_TYPE_HINDEXED	MPI_TYPE_CREATE_HINDEXED
MPI_TYPE_HVECTOR	MPI_TYPE_CREATE_HVECTOR
MPI_TYPE_LB	MPI_TYPE_GET_EXTENT
MPI_TYPE_STRUCT	MPI_TYPE_CREATE_STRUCT
MPI_TYPE_UB	MPI_TYPE_GET_EXTENT

19.1.3 Removed MPI-1 Datatypes

Table 19.2 shows the removed MPI-1 datatypes and their replacements.

19.1.4 Removed MPI-1 Constants

Table 19.3 shows the removed MPI-1 constants. There are no replacements.

19.1.5 Removed MPI-1 Callback Prototypes

Table 19.4 shows the removed MPI-1 callback prototypes and their replacements.

Table 19.2: Removed MPI-1 datatypes. The indicated routine may be used for changing the lower and upper bound respectively.

Removed	MPI-2 Replacement
MPI_LB	MPI_TYPE_CREATE_RESIZED
MPI_UB	MPI_TYPE_CREATE_RESIZED

Table 19.3: Removed MPI-1 constants

Removed MPI-1 Constants
C type: <code>const int</code> (or unnamed enum)
Fortran type: <code>INTEGER</code>
<code>MPI_COMBINER_HINDEXED_INTEGER</code>
<code>MPI_COMBINER_HVECTOR_INTEGER</code>
<code>MPI_COMBINER_STRUCT_INTEGER</code>

Table 19.4: Removed MPI-1 callback prototypes and their replacements

Removed	MPI-2 Replacement
<code>MPI_Handler_function</code>	MPI_Comm_errhandler_function

19.2 C++ Bindings

The C++ bindings were deprecated as of MPI-2.2. The C++ bindings were removed in MPI-3.0. The namespace is still reserved, however, and bindings may only be provided by an implementation as described in the MPI-2.2 standard.

Chapter 20

Semantic Changes and Warnings

This chapter lists semantic changes that have been introduced into the MPI standard as well as warnings that could potentially impact program behavior. In addition to those listed here, Chapter 19 also lists changes and backward incompatibilities caused by removing interfaces. Unlike Chapter 19, the changes in this chapter did not go through a deprecation process.

20.1 Semantic Changes

This section describes semantics that have changed in a way that would potentially cause an MPI program to behave differently when using this version of the MPI standard without changing the program's code.

20.1.1 Semantic Changes Starting in MPI-4.0

- `MPI_COMM_DUP` and `MPI_COMM_IDUP` no longer propagate info hints from the input communicator to the output communicator. This behavior can be achieved using `MPI_COMM_DUP_WITH_INFO` and `MPI_COMM_IDUP_WITH_INFO`.
- The default communicator where errors are raised when not involving a communicator, window, or file was changed from `MPI_COMM_WORLD` to `MPI_COMM_SELF`.

20.2 Additional Warnings

This section describes additional changes that could potentially cause a program that relies on the semantics described in a previous version of the MPI standard to behave differently than with this version of MPI. The changes in this section are limited in scope and unlikely to impact most programs.

20.2.1 Warnings Starting in MPI-4.1

Implementations are no longer allowed to implement `MPI_WTICK`, `PMPI_WTICK`, `MPI_WTIME`, and `PMPI_WTIME` as well as handle conversion functions as macros (Sections 9.6 and 21.3.4). This should not impact applications but may require changes in some implementations.

20.2.2 Warnings Starting in MPI-4.0

The limit for length of MPI identifiers was removed. Prior to MPI-4.0, MPI identifiers were limited to 30 characters (31 with the profiling interface). This limitation was initially introduced to avoid exceeding the limit on some compilation systems.

Rationale. For Fortran, this limit was already relaxed for the Fortran specific function names, see Section 21.1.5, and the Fortran language specification 2003 requires support for a minimum of 63 characters for internal and external identifiers. Starting with the ISO/IEC 9899:1999 C programming language standard, support for a minimum of 63 characters is required for internal identifiers, but only 31 characters are required to be significant for external identifiers. At the time of the release of MPI-4.0, most or nearly all compilers allow external identifiers longer than 31 characters. Therefore, the restriction is removed. (*End of rationale.*)

Advice to users. This affects users only if they store MPI identifiers into fixed sized strings. (*End of advice to users.*)

Chapter 21

Language Bindings

21.1 Support for Fortran

21.1.1 Overview

The Fortran MPI language bindings have been designed to be compatible with the Fortran 90 standard with additional features from Fortran 2018 [48]. In previous versions of this document, references were made to Fortran 2003 and Fortran 2008 [46] with TS 29113 [47]; where appropriate, the specific features of Fortran 2018 that MPI requires will be noted explicitly.

Rationale. Fortran 90 contains numerous features designed to make it a more “modern” language than Fortran 77. It seems natural that MPI should be able to take advantage of these new features with a set of bindings tailored to Fortran 90. In Fortran 2008 with TS 29113 and later Fortran 2018, the major new language features used are the ASYNCHRONOUS attribute to protect nonblocking MPI operations, and assumed-type and assumed-rank dummy arguments for choice buffer arguments. Further requirements for compiler support are listed in Section 21.1.7. (*End of rationale.*)

MPI defines three methods of Fortran support:

1. **USE mpi_f08:** This method is described in Section 21.1.2. It requires compile-time argument checking with unique MPI handle types and provides techniques to fully solve the optimization problems with nonblocking calls. This is the only Fortran support method that is consistent with the Fortran standard (Fortran 2008 with TS 29113 and later Fortran 2018). This method is highly recommended for all MPI applications.
2. **USE mpi:** This method is described in Section 21.1.3 and requires compile-time argument checking. Handles are defined as INTEGER. This Fortran support method is inconsistent with the Fortran standard, and its use is therefore not recommended. It exists only for backwards compatibility.
3. **INCLUDE 'mpif.h':** This method is described in Section 21.1.4. The use of the include file `mpif.h` has been strongly discouraged starting with MPI-3.0 and deprecated with MPI-4.1, because this method neither guarantees compile-time argument checking nor provides sufficient techniques to solve the optimization problems with nonblocking calls, and is therefore inconsistent with the Fortran standard. It exists only for backwards compatibility with legacy MPI applications.

MPI implementations providing a Fortran interface must provide one or both of the following:

- The USE `mpi_f08` Fortran support method.
- The USE `mpi` and INCLUDE `'mpif.h'` Fortran support methods.

Section 21.1.6 describes restrictions if the compiler does not support all the needed features.

Application subroutines and functions may use either one of the modules or the (deprecated) `mpif.h` include file. An implementation may require the use of one of the modules to prevent type mismatch errors.

Advice to users. Users are advised to utilize one of the MPI modules even if `mpif.h` enforces type checking on a particular system. Using a module provides several potential advantages over using an include file; the `mpi_f08` module offers the most robust and complete Fortran support. (*End of advice to users.*)

In a single application, it must be possible to link together routines that USE `mpi_f08`, USE `mpi`, and INCLUDE `'mpif.h'`.

The LOGICAL constant `MPI_SUBARRAYS_SUPPORTED` is set to `.TRUE.` if all buffer choice arguments are defined in explicit interfaces with assumed-type and assumed-rank [48]; otherwise it is set to `.FALSE.` The LOGICAL constant `MPI_ASYNC_PROTECTS_NONBLOCKING` is set to `.TRUE.` if the `ASYNCHRONOUS` attribute was added to the choice buffer arguments of all nonblocking interfaces **and** the underlying Fortran compiler supports the `ASYNCHRONOUS` attribute for MPI communication (as part of TS 29113, which has been superseded by Fortran 2018), otherwise it is set to `.FALSE.` These constants exist for each Fortran support method, but not in the C header file. The values may be different for each Fortran support method. All other constants and the integer values of handles must be the same for each Fortran support method.

Sections 21.1.2 through 21.1.4 define the Fortran support methods. The Fortran interfaces of each MPI routine are shorthands. Section 21.1.5 defines the corresponding full interface specification together with the specific procedure names and implications for the profiling interface. Section 21.1.6 describes the implementation of the MPI routines for different versions of the Fortran standard. Section 21.1.7 summarizes major requirements for MPI implementations with Fortran support. Section 21.1.8 and Section 21.1.9 describe additional functionality that is part of the Fortran support. `MPI_F_SYNC_REG` is needed for one of the methods to prevent register optimization problems. A set of functions provides additional support for Fortran intrinsic numeric types, including parameterized types: `MPI_TYPE_MATCH_SIZE`, `MPI_TYPE_CREATE_F90_INTEGER`, `MPI_TYPE_CREATE_F90_REAL` and `MPI_TYPE_CREATE_F90_COMPLEX`. In the context of MPI, parameterized types are Fortran intrinsic types that are specified using `KIND` type parameters. Sections 21.1.10 through 21.1.19 give an overview and details on known problems when using Fortran together with MPI; Section 21.1.20 compares the Fortran problems with those in C.

21.1.2 Fortran Support Through the `mpi_f08` Module

An MPI implementation providing a Fortran interface must provide a module named `mpi_f08` that can be used in a Fortran program. Section 21.1.6 describes restrictions if the compiler does not support all the needed features. Within all MPI function specifications, the first of the set of two Fortran routine interface specifications is provided by this module. This module must:

- Define all named MPI constants.
- Declare MPI functions that return a value.
- Provide explicit interfaces according to the Fortran routine interface specifications. This module therefore guarantees compile-time argument checking for all arguments that are not `TYPE(*)`, with the following exception:

Only one Fortran interface is defined for functions that are deprecated as of MPI-3.0. This interface must be provided as an explicit interface according to the rules defined for the `mpi` module, see Section 21.1.3.

Advice to users. It is strongly recommended that developers substitute calls to deprecated routines when upgrading from the (deprecated) `mpif.h` or the `mpi` module to the `mpi_f08` module. (*End of advice to users.*)

- Define the derived type `MPI_Status`, and define all MPI handles with uniquely named handle types (instead of `INTEGER` handles, as in the `mpi` module). This is reflected in the first Fortran binding in each MPI function definition throughout this document (except for the deprecated routines).
- Overload the operators `.EQ.` and `.NE.` to allow the comparison of these MPI handles with `.EQ.`, `.NE.`, `==` and `/=`.
- Use the `ASYNCHRONOUS` attribute to protect the buffers of nonblocking operations, and set the `LOGICAL` constant `MPI_ASYNC_PROTECTS_NONBLOCKING` to `.TRUE.` if the underlying Fortran compiler supports the `ASYNCHRONOUS` attribute for MPI communication (as part of TS 29113). See Section 21.1.6 for older compiler versions.
- Set the `LOGICAL` constant `MPI_SUBARRAYS_SUPPORTED` to `.TRUE.` and declare choice buffers using the Fortran 2018 features assumed-type and assumed-rank, i.e., `TYPE(*)`, `DIMENSION(...)` in all nonblocking, split collective and persistent communication routines, if the underlying Fortran compiler supports it. With this, noncontiguous sub-arrays can be used as buffers in nonblocking routines.

Rationale. In all blocking routines, i.e., if the choice-buffer is not declared as `ASYNCHRONOUS`, the Fortran 2018 feature is not needed for the support of noncontiguous buffers because the compiler can pass the buffer by in-and-out-copy through a contiguous scratch array. (*End of rationale.*)

- Set the `MPI_SUBARRAYS_SUPPORTED` constant to `.FALSE.` and declare choice buffers with a compiler-dependent mechanism that overrides type checking if the underlying Fortran compiler does not support the Fortran 2018 assumed-type and assumed-rank notation. In this case, the use of noncontiguous sub-arrays as buffers in nonblocking calls may be invalid. See Section 21.1.6 for details.
- Declare each argument with an `INTENT` of `IN`, `OUT`, or `INOUT` as defined in this standard.

Rationale. For these definitions in the `mpi_f08` bindings, in most cases, `INTENT(IN)` is used if the C interface uses call-by-value. For all buffer arguments and for `OUT` and `INOUT` dummy arguments that allow one of the nonordinary Fortran constants (see `MPI_BOTTOM`, etc. in Section 2.5.4) as input, an `INTENT` is not specified. (*End of rationale.*)

Advice to users. If a dummy argument is declared with `INTENT(OUT)`, then the Fortran standard stipulates that the actual argument becomes undefined upon invocation of the MPI routine, i.e., it may be overwritten by some other values, e.g. zeros; according to [46], 12.5.2.4 Ordinary dummy variables, Paragraph 17: “If a dummy argument has `INTENT(OUT)`, the actual argument becomes undefined at the time the association is established, except [...]”. For example, if the dummy argument is an assumed-size array and the actual argument is a strided array, the call may be implemented with copy-in and copy-out of the argument. In the case of `INTENT(OUT)` the copy-in may be suppressed by the optimization and the routine starts execution using an array of undefined values. If the routine stores fewer elements into the dummy argument than is provided in the actual argument, then the remaining locations are overwritten with these undefined values. See also both advices to implementors in Section 21.1.3. (*End of advice to users.*)

- Declare all `ierror` output arguments as `OPTIONAL`, except for user-defined callback functions (e.g., of type `MPI_Comm_copy_attr_function` or `COMM_COPY_ATTR_FUNCTION`) and predefined callbacks (e.g., `MPI_COMM_NULL_COPY_FN`).

Rationale. For user-defined callback functions (e.g., of type `MPI_Comm_copy_attr_function` or `COMM_COPY_ATTR_FUNCTION`) and their predefined callbacks (e.g., `MPI_COMM_NULL_COPY_FN`), the `ierror` argument is not optional. The MPI library must always call these routines with an actual `ierror` argument. Therefore, these user-defined functions need not check whether the MPI library calls these routines with or without an actual `ierror` output argument. (*End of rationale.*)

The MPI Fortran bindings in the `mpi_f08` module are designed based on the Fortran 2008 standard [46] together with the Technical Specification “TS 29113 Further Interoperability with C” [47] of the ISO/IEC JTC1/SC22/WG5 (Fortran) working group, which is now integrated in Fortran 2018 standard [48].

Rationale. The features in TS 29113 on further interoperability with C were decided on by ISO/IEC JTC1/SC22/WG5 and designed by PL22.3 (formerly J3) to support a higher level of integration between Fortran-specific features and C than was provided in the Fortran 2008 standard; part of this design is based on requirements from the MPI Forum to support MPI-3.0. These features became part of Fortran 2018 [48], so references to TS 29113 are obsolete, except insofar as to specify a particular feature set from Fortran 2018 or minimal requirements to a compiler.

Fortran 2018 contains the following language features that are needed for the MPI bindings in the `mpi_f08` module: assumed-type and assumed-rank. It is important that any possible actual argument can be used for such dummy arguments, e.g., scalars, arrays, assumed-shape arrays, assumed-size arrays, allocatable arrays, and with any element type, e.g., `REAL`, `CHARACTER*5`, `CHARACTER*(*)`, sequence derived types, or `BIND(C)` derived types. Especially for backward compatibility reasons, it is important that any possible actual argument in an implicit interface implementation of a choice buffer dummy argument (e.g., with the deprecated `mpif.h` without argument-checking) can be used in an implementation with assumed-type and assumed-rank argument in an explicit interface (e.g., with the `mpi_f08` module).

A further feature useful for MPI is the extension of the semantics of the `ASYNCHRONOUS` attribute: In F2003 and F2008, this attribute could be used only to protect buffers of Fortran asynchronous I/O. With TS 29113 and now Fortran 2018, this attribute also covers asynchronous communication occurring within library routines written in C.

The MPI Forum hereby wishes to acknowledge this important effort by the Fortran PL22.3 and WG5 committee. (*End of rationale.*)

21.1.3 Fortran Support Through the `mpi` Module

An MPI implementation providing a Fortran interface must provide a module named `mpi` that can be used in a Fortran program. Within all MPI function specifications, the second of the set of two Fortran routine interface specifications is provided by this module. This module must:

- Define all named MPI constants.
- Declare MPI functions that return a value.
- Provide explicit interfaces according to the Fortran routine interface specifications. This module therefore guarantees compile-time argument checking and allows positional and keyword-based argument lists. If an implementation is paired with a compiler that either does not support `TYPE(*)`, `DIMENSION(..)` from Fortran 2018, or is otherwise unable to ignore the types of choice buffers, then the implementation must provide explicit interfaces only for MPI routines with no choice buffer arguments. See Section 21.1.6 for more details.
- Define all MPI handles as type `INTEGER`.
- Define the derived type `MPI_Status` and all named handle types that are used in the `mpi_f08` module. For these named handle types, overload the operators `.EQ.` and `.NE.` to allow handle comparison via the `.EQ.`, `.NE.`, `==` and `/=` operators.

Rationale. They are needed only when the application converts old-style `INTEGER` handles into new-style handles with a named type. (*End of rationale.*)

- A high quality MPI implementation may enhance the interface by using the `ASYNCHRONOUS` attribute in the same way as in the `mpi_f08` module if it is supported by the underlying compiler.
- Set the LOGICAL constant `MPI_ASYNC_PROTECTS_NONBLOCKING` to `.TRUE.` if the `ASYNCHRONOUS` attribute is used in all nonblocking interfaces **and** the underlying Fortran compiler supports the `ASYNCHRONOUS` attribute for MPI communication (as part of Fortran 2018), otherwise to `.FALSE.`

Advice to users. For an MPI implementation that fully supports nonblocking calls with the `ASYNCHRONOUS` attribute for choice buffers, an existing MPI-2.2 application may fail to compile even if it compiled and executed with expected results with an MPI-2.2 implementation. One reason may be that the application uses “contiguous” but not “simply contiguous” `ASYNCHRONOUS` arrays as actual arguments for choice buffers of nonblocking routines, e.g., by using subscript triplets with stride one or specifying `(1:n)`

for a whole dimension instead of using (:). This should be fixed to fulfill the Fortran constraints for ASYNCHRONOUS dummy arguments. This is not considered a violation of backward compatibility because existing applications can not use the ASYNCHRONOUS attribute to protect nonblocking calls. Another reason may be that the application does not conform either to the MPI standard or to the Fortran standard, typically because the program forces the compiler to perform copy-in/out for a choice buffer argument in a nonblocking MPI call. This is also not a violation of backward compatibility because the application itself is nonconforming. See Section 21.1.12 for more details. (*End of advice to users.*)

- A high quality MPI implementation may enhance the interface by using TYPE(*), DIMENSION(..) choice buffer dummy arguments instead of using nonstandardized extensions such as !\$PRAGMA IGNORE_TKR or a set of overloaded functions as described by M. Hennecke in [33], if the compiler supports this Fortran 2018 language feature. See Section 21.1.6 for further details.
- Set the LOGICAL constant MPI_SUBARRAYS_SUPPORTED to .TRUE. if all choice buffer arguments in all nonblocking, split collective and persistent communication routines are declared with TYPE(*), DIMENSION(..), otherwise set it to .FALSE.. When MPI_SUBARRAYS_SUPPORTED is defined as .TRUE., noncontiguous sub-arrays can be used as buffers in nonblocking routines.
- Set the MPI_SUBARRAYS_SUPPORTED constant to .FALSE. and declare choice buffers with a compiler-dependent mechanism that overrides type checking if the underlying Fortran compiler does not support the Fortran 2018 assumed-type and assumed-rank features. In this case, the use of noncontiguous sub-arrays in nonblocking calls may be disallowed. See Section 21.1.6 for details.

An MPI implementation may provide other features in the mpi module that enhance the usability of MPI while maintaining adherence to the standard. For example, it may provide INTENT information in these interface blocks.

Advice to implementors. The appropriate INTENT may be different from what is given in the MPI language-neutral bindings. Implementations must choose INTENT so that the function adheres to the MPI standard, e.g., by defining the INTENT as provided in the mpi_f08 bindings. (*End of advice to implementors.*)

Rationale. The intent given by the MPI generic interface is not precisely defined and does not in all cases correspond to the correct Fortran INTENT. For instance, receiving into a buffer specified by a datatype with absolute addresses may require associating MPI_BOTTOM with a dummy OUT argument. Moreover, “constants” such as MPI_BOTTOM and MPI_STATUS_IGNORE are not constants as defined by Fortran, but “special addresses” used in a nonstandard way. Finally, the MPI-1 generic intent was changed in several places in MPI-2. For instance, MPI_IN_PLACE changes the intent of an OUT argument to be INOUT. (*End of rationale.*)

Advice to implementors. The Fortran 2008 standard illustrates in its Note 5.17 that “INTENT(OUT)” means that the value of the argument after invoking the procedure is entirely the result of executing that procedure. If an argument should retain its

value rather than being redefined, `INTENT(INOUT)` should be used rather than `INTENT(OUT)`, even if there is no explicit reference to the value of the dummy argument. Furthermore, `INTENT(INOUT)` is not equivalent to omitting the `INTENT` attribute, because `INTENT(INOUT)` always requires that the associated actual argument is definable.” Applications that include the (deprecated) `mpif.h` may not expect that `INTENT(OUT)` is used. In particular, output array arguments are expected to keep their content as long as the MPI routine does not modify them. To keep this behavior, it is recommended that implementations not use `INTENT(OUT)` in the `mpi` module and the (deprecated) `mpif.h` include file, even though `INTENT(OUT)` is specified in an interface description of the `mpi_f08` module. (*End of advice to implementors.*)

21.1.4 Fortran Support Through the `mpif.h` Include File

The use of the `mpif.h` include file has been deprecated in MPI-4.1.

An MPI implementation providing a Fortran interface must provide an include file named `mpif.h` that can be used in a Fortran program. Within all MPI function specifications, the second of the set of two Fortran routine interface specifications is supported by this include file. This include file must:

- Define all named MPI constants.
- Declare MPI functions that return a value.
- Define all handles as `INTEGER`.
- Be valid and equivalent for both fixed and free source form.

For each MPI routine, an implementation can choose to use an implicit or explicit interface for the second Fortran binding (in deprecated routines, the first one may be omitted).

- Set the LOGICAL constants `MPI_SUBARRAYS_SUPPORTED` and `MPI_ASYNC_PROTECTS_NONBLOCKING` according to the same rules as for the `mpi` module. In the case of implicit interfaces for choice buffer or nonblocking routines, the constants must be set to `.FALSE..`

Advice to users. Instead of using `mpif.h`, the use of the `mpi_f08` or `mpi` module is strongly encouraged for the following reasons:

- Most `mpif.h` implementations do not include compile-time argument checking.
- Therefore, many bugs in MPI applications remain undetected at compile-time, such as:
 - Missing `ierror` as last argument in most Fortran bindings.
 - Declaration of a `status` as an `INTEGER` variable instead of an `INTEGER` array with size `MPI_STATUS_SIZE`.
 - Incorrect argument positions; e.g., interchanging the `count` and `datatype` arguments.
 - Passing incorrect MPI handles; e.g., passing a datatype instead of a communicator.

- The migration from `mpif.h` to the `mpi` module should be relatively straightforward (i.e., substituting `INCLUDE 'mpif.h'` after an `implicit` statement by `use mpi` before that `implicit` statement) as long as the application syntax is correct.
- Migrating portable and correctly written applications to the `mpi` module is not expected to be difficult. No compile or runtime problems should occur because an `mpif.h` include file was always allowed to provide explicit Fortran interfaces.

(End of advice to users.)

21.1.5 Interface Specifications, Procedure Names, and the Profiling Interface

The Fortran interface specification of each MPI routine specifies the routine name that must be called by the application program, and the names and types of the dummy arguments together with additional attributes. The Fortran standard allows a given Fortran interface to be implemented with several methods, e.g., within or outside of a module, with or without `BIND(C)`, or the buffers with or without Fortran 2018 (as successor of Fortran 2008 with TS 29113). Such implementation decisions imply different binary interfaces and different specific procedure names. The requirements for several implementation schemes together with the rules for the specific procedure names and its implications for the profiling interface are specified within this section, but not the implementation details.

Rationale. When this section was originally introduced in MPI-3.0, the major goals for the three Fortran support methods were:

- Portable implementation of the wrappers from the MPI Fortran interfaces to the MPI routines in C.
- Binary backward compatible implementation path when switching `MPI_SUBARRAYS_SUPPORTED` from `.FALSE.` to `.TRUE.`
- The Fortran PMPI interface need not be backward compatible, but a method must be included that a tools layer can use to examine the MPI library about the specific procedure names and interfaces used.
- No performance drawbacks.
- Consistency between all three Fortran support methods.
- Consistent with Fortran 2018.

The design expected that all dummy arguments in the MPI Fortran interfaces are interoperable with C according to Fortran 2018. This expectation was not fulfilled. The `LOGICAL` arguments are not interoperable with C, mainly because the internal representations for `.FALSE.` and `.TRUE.` are compiler dependent. The provided interface was mainly based on `BIND(C)` interfaces and therefore inconsistent with Fortran. To be consistent with Fortran, the `BIND(C)` had to be removed from the callback procedure interfaces and the predefined callbacks, e.g., `MPI_COMM_DUP_FN`. Non-`BIND(C)` procedures are also not interoperable with C, and therefore the `BIND(C)` had to be removed from all routines with `PROCEDURE` arguments, e.g., from `MPI_OP_CREATE`.

Therefore, this section was rewritten as an erratum to MPI-3.0. (End of rationale.)

A Fortran call to an MPI routine shall result in a call to a procedure with one of the specific procedure names and calling conventions, as described in Table 21.1. Case is not significant in the names.

Table 21.1: Specific Fortran procedure names and related calling conventions. `MPI_ISEND` is used as an example. For routines without choice buffers, only 1A and 2A apply.

No.	Specific procedure name	Calling convention
1A	<code>MPI_Isend_f08</code>	Fortran interface and arguments, as in Annex A.4, except that in routines with a choice buffer dummy argument, this dummy argument is implemented with nonstandard extensions like <code>!\$PRAGMA IGNORE_TKR</code> , which provides a call-by-reference argument without type, kind, and dimension checking.
1B	<code>MPI_Isend_f08ts</code>	Fortran interface and arguments, as in Annex A.4, but only for routines with one or more choice buffer dummy arguments; these dummy arguments are implemented with <code>TYPE(*)</code> , <code>DIMENSION(..)</code> .
2A	<code>MPI_ISEND</code>	Fortran interface and arguments, as in Annex A.5, except that in routines with a choice buffer dummy argument, this dummy argument is implemented with nonstandard extensions like <code>!\$PRAGMA IGNORE_TKR</code> , which provides a call-by-reference argument without type, kind, and dimension checking.
2B	<code>MPI_ISEND_FTS</code>	Fortran interface and arguments, as in Annex A.5, but only for routines with one or more choice buffer dummy arguments; these dummy arguments are implemented with <code>TYPE(*)</code> , <code>DIMENSION(..)</code> . In the (deprecated) <code>mpif.h</code> only, the postfix “_FTS” for <code>MPI_NEIGHBOR_ALLGATHERV_INIT</code> , <code>MPI_NEIGHBOR_ALLTOALLV_INIT</code> , and <code>MPI_NEIGHBOR_ALLTOALLW_INIT</code> is shortened to “_F”.

Note that for the deprecated routines in Section 18.1, which are reported only in Annex A.5, scheme 2A is utilized in the `mpi` module and (deprecated) `mpif.h`, and also in the `mpi_f08` module.

To set `MPI_SUBARRAYS_SUPPORTED` to `.TRUE.` within a Fortran support method, it is required that all nonblocking and split-collective routines with buffer arguments are implemented according to 1B and 2B, i.e., with `MPI_Xxxx_f08ts` in the `mpi_f08` module, and with `MPI_XXXX_FTS` in the `mpi` module and the (deprecated) `mpif.h` include file.

The `mpi` and `mpi_f08` modules and the (deprecated) `mpif.h` include file will each correspond to exactly one implementation scheme from Table 21.1. However, the MPI library may contain multiple implementation schemes from Table 21.1.

Advice to implementors. This may be desirable for backwards binary compatibility in the scope of a single MPI implementation, for example. (*End of advice to implementors.*)

Rationale. After a compiler provides the facilities `TYPE(*)`, `DIMENSION(..)` from For-

tran 2018, it is possible to change the bindings within a Fortran support method to support subarrays without recompiling the complete application provided that the previous interfaces with their specific procedure names are still included in the library. Of course, only recompiled routines can benefit from the added facilities. There is no binary compatibility conflict because each interface uses its own specific procedure names and all interfaces use the same constants (except the value of `MPI_SUBARRAYS_SUPPORTED` and `MPI_ASYNC_PROTECTS_NONBLOCKING`) and type definitions. After a compiler also ensures that buffer arguments of nonblocking MPI operations can be protected through the `ASYNCHRONOUS` attribute, and the procedure declarations in the `mpi_f08` and `mpi` module and the (deprecated) `mpif.h` include file declare choice buffers with the `ASYNCHRONOUS` attribute, then the value of `MPI_ASYNC_PROTECTS_NONBLOCKING` can be switched to `.TRUE.` in the module definition and include file. (*End of rationale.*)

Advice to users. Partial recompilation of user applications when upgrading MPI implementations is a highly complex and subtle topic. Users are strongly advised to consult their MPI implementation's documentation to see exactly what is—and what is not—supported. (*End of advice to users.*)

Within the `mpi_f08` and `mpi` modules and (deprecated) `mpif.h` include file, for all MPI procedures, a second procedure with the same calling conventions shall be supplied, except that the name is modified by prefixing with the letter “P”, e.g., `PMPI_Isend`. The specific procedure names for these `PMPI_Xxxx` procedures must be different from the specific procedure names for the `MPI_Xxxx` procedures and are not specified by this standard.

A user-written or middleware profiling routine should provide the same specific Fortran procedure names and calling conventions, and therefore can interpose itself as the MPI library routine. The profiling routine can internally call the matching `PMPI` routine with any of its existing bindings, except for routines that have callback routine dummy arguments, choice buffer arguments, or that are attribute caching routines (`MPI_{COMM|WIN|TYPE}_{SET|GET}_ATTR`). In this case, the profiling software should invoke the corresponding `PMPI` routine using the same Fortran support method as used in the calling application program, because the C, `mpi_f08` and `mpi` callback prototypes are different or the meaning of the choice buffer or `attribute_val` arguments are different.

Advice to users. Although for each support method and MPI routine (e.g., `MPI_ISEND` in `mpi_f08`), multiple routines may need to be provided to intercept the specific procedures in the MPI library (e.g., `MPI_Isend_f08` and `MPI_Isend_f08ts`), each profiling routine itself uses only one support method (e.g., `mpi_f08`) and calls the real MPI routine through the one `PMPI` routine defined in this support method (i.e., `PMPI_Isend` in this example). (*End of advice to users.*)

Advice to implementors. If all of the following conditions are fulfilled:

- the handles in the `mpi_f08` module occupy one Fortran numerical storage unit (same as an `INTEGER` handle),
- the internal argument passing mechanism used to pass an actual `ierror` argument to a nonoptional `ierror` dummy argument is binary compatible to passing an actual `ierror` argument to an `ierror` dummy argument that is declared as `OPTIONAL`,
- the internal argument passing mechanism for `ASYNCHRONOUS` and non-`ASYNCHRONOUS` arguments is the same,

- the internal routine call mechanism is the same for the Fortran and the C compilers for which the MPI library is compiled, and
- the compiler does not provide the appropriate features from Fortran 2018,

then the implementor may use the same internal routine implementations for all Fortran support methods but with several different specific procedure names. If the accompanying Fortran compiler supports Fortran 2018 or at least Fortran 2008 with TS 29113, then the new routines are needed only for routines with choice buffer arguments. (*End of advice to implementors.*)

Advice to implementors. In the (deprecated) Fortran support method `mpif.h`, compile-time argument checking can be also implemented for all routines. For `mpif.h`, the argument names are not specified through the MPI standard, i.e., only positional argument lists are defined, and not key-word based lists. Due to the rule that `mpif.h` must be valid for fixed and free source form, the subroutine declaration is restricted to one line with 72 characters. To keep the argument lists short, each argument name can be shortened to a minimum of one character. With this, the three longest subroutine declaration statements are

```
SUBROUTINE PMPI_DIST_GRAPH_CREATE_ADJACENT(a,b,c,d,e,f,g,h,i,j,k)
SUBROUTINE PMPI_NEIGHBOR_ALLTOALLW_INIT(a,b,c,d,e,f,g,h,i,j,k,l)
SUBROUTINE PMPI_NEIGHBOR_ALLTOALLV_INIT(a,b,c,d,e,f,g,h,i,j,k,l)
```

with 71 and 70 characters each. With buffers implemented with Fortran 2018 (or TS 29113), the specific procedure names have an additional postfix. Some of the longest of such interface definitions are

```
INTERFACE PMPI_NEIGHBOR_ALLTOALLW_INIT
SUBROUTINE PMPI_NEIGHBOR_ALLTOALLW_INIT_F(a,b,c,d,e,f,g,h,i,j,j,k)
INTERFACE PMPI_NEIGHBOR_ALLGATHERV_INIT
SUBROUTINE PMPI_NEIGHBOR_ALLGATHERV_INIT_F(a,b,c,d,e,f,g,h,i,j,k)
INTERFACE PMPI_RGET_ACCUMULATE
SUBROUTINE PMPI_RGET_ACCUMULATE_FTS(a,b,c,d,e,f,g,h,i,j,k,l,m,n)
```

with 72, 71, and 70 characters. In principle, continuation lines would be possible in `mpif.h` (spaces in columns 73–131, & in column 132, and in column 6 of the continuation line) but this would not be valid if the source line length is extended with a compiler flag to 132 characters. Column 133 is also not available for the continuation character because lines longer than 132 characters are invalid with some compilers by default.

If an implementation applies the rules of Table 21.1 also for the PMPI interface, then the longest specific procedure name is `PMPI_Reduce_scatter_block_init_c_f08ts` with 38 characters in the `mpi_f08` module.

Example 21.1. The interface specifications for `MPI_Comm_rank` together with the specific procedure names can be implemented in the `mpi_f08` and `mpi` modules like this:

```
MODULE mpi_f08
  TYPE, BIND(C) :: MPI_Comm
    INTEGER :: MPI_VAL
  END TYPE MPI_Comm
  ...
```

```

1  INTERFACE MPI_Comm_rank ! (as defined in Chapter 6)
2  SUBROUTINE MPI_Comm_rank_f08(comm, rank, ierror)
3  IMPORT :: MPI_Comm
4  TYPE(MPI_Comm),      INTENT(IN)  :: comm
5  INTEGER,              INTENT(OUT) :: rank
6  INTEGER, OPTIONAL,   INTENT(OUT) :: ierror
7  END SUBROUTINE
8  END INTERFACE
9  END MODULE mpi_f08
10
11 MODULE mpi
12   INTERFACE MPI_Comm_rank ! (as defined in Chapter 6)
13     SUBROUTINE MPI_Comm_rank(comm, rank, ierror)
14       INTEGER, INTENT(IN) :: comm ! The INTENT may be added although
15       INTEGER, INTENT(OUT) :: rank ! it is not defined in the
16       INTEGER, INTENT(OUT) :: ierror ! official routine definition.
17     END SUBROUTINE
18   END INTERFACE
19 END MODULE mpi

```

And if interfaces are provided in `mpif.h`, they might look like this (outside of any module and in fixed source format):

```

20 ! 23456789012345678901234567890123456789012345678901234567890123456789012
21   INTERFACE MPI_Comm_rank ! (as defined in Chapter 6)
22     SUBROUTINE MPI_Comm_rank(comm, rank, ierror)
23       INTEGER, INTENT(IN) :: comm ! The argument names may be
24       INTEGER, INTENT(OUT) :: rank ! shortened so that the
25       INTEGER, INTENT(OUT) :: ierror ! subroutine line fits to the
26     END SUBROUTINE
27   END INTERFACE

```

(End of advice to implementors.)

Advice to users.

Example 21.2. Illustration of how a user-written or middleware profiling routine can be implemented:

```

33 SUBROUTINE MPI_Isend_f08ts(buf, count, datatype, dest, tag, comm, request, ierror)
34   USE :: mpi_f08, my_noname => MPI_Isend_f08ts
35   TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
36   INTEGER,              INTENT(IN)      :: count, dest, tag
37   TYPE(MPI_Datatype),   INTENT(IN)      :: datatype
38   TYPE(MPI_Comm),       INTENT(IN)      :: comm
39   TYPE(MPI_Request),    INTENT(OUT)     :: request
40   INTEGER, OPTIONAL,   INTENT(OUT)     :: ierror
41   ! ... some code for the begin of profiling
42   call PMPI_Isend (buf, count, datatype, dest, tag, comm, request, ierror)
43   ! ... some code for the end of profiling
44 END SUBROUTINE MPI_Isend_f08ts

```

Note that this routine is used to intercept the existing specific procedure name `MPI_Isend_f08ts` in the MPI library. This routine must not be part of a module. This routine itself calls `PMPI_Isend`. The `USE` of the `mpi_f08` module is needed for definitions of handle types and the interface for `PMPI_Isend`. However, this module also contains an interface definition for the specific procedure name `MPI_Isend_f08ts` that conflicts

with the definition of this profiling routine (i.e., the name is doubly defined). Therefore, the USE here specifically excludes the interface from the module by renaming the unused routine name in the `mpi_f08` module into “`my_noname`” in the scope of this routine.

(End of advice to users.)

Advice to users. The PMPI interface allows intercepting MPI routines. For example, an additional `MPI_ISEND` profiling wrapper can be provided that is called by the application and internally calls `PMPI_ISEND`. There are two typical use cases: a profiling layer that is developed independently from the application and the MPI library, and profiling routines that are part of the application and have access to the application data. With MPI-3.0, new Fortran interfaces and implementation schemes were introduced that have several implications on how Fortran MPI routines are internally implemented and optimized. For profiling layers, these schemes imply that several internal interfaces with different specific procedure names may need to be intercepted, as shown in Example 21.2. Therefore, for wrapper routines that are part of a Fortran application, it may be more convenient to make the name shift within the application, i.e., to substitute the call to the MPI routine (e.g., `MPI_ISEND`) by a call to a user-written profiling wrapper with a new name (e.g., `X_MPI_ISEND`) and to call the Fortran `MPI_ISEND` from this wrapper, instead of using the PMPI interface. *(End of advice to users.)*

Advice to implementors. An implementation that provides a Fortran interface must provide a combination of MPI library and module or include file that uses the specific procedure names as described in Table 21.1 so that the MPI Fortran routines are interceptable as described above. *(End of advice to implementors.)*

21.1.6 MPI for Different Fortran Standard Versions

This section describes which Fortran interface functionality can be provided for different versions of the Fortran standard.

- *For Fortran 77* with some extensions:
 - MPI identifiers may be up to 30 characters (31 with the profiling interface).
 - MPI identifiers may contain underscores after the first character.
 - An MPI subroutine with a choice argument may be called with different argument types.
 - Although not required by the MPI standard, the `INCLUDE` statement should be available for including `mpif.h` into the user application source code.

Only MPI-1.1, MPI-1.2, and MPI-1.3 can be implemented. The use of absolute addresses from `MPI_ADDRESS` and `MPI_BOTTOM` may cause problems if an address does not fit into the memory space provided by an `INTEGER`. (In MPI-2.0 this problem is solved with `MPI_GET_ADDRESS`, but not for Fortran 77.)

- *For Fortran 90:*

The major additional features that are needed from Fortran 90 are:

 - The `MODULE` and `INTERFACE` concept.
 - The `KIND=` and `SELECTED_XXX_KIND` concept.

- Fortran derived `TYPE`s and the `SEQUENCE` attribute.
- The `OPTIONAL` attribute for dummy arguments.
- Cray pointers, which are a nonstandard compiler extension, are needed for the use of `MPI_ALLOC_MEM`.

With these features, MPI-1.1 – MPI-2.2 can be implemented without restrictions. MPI-3.0 and later can be implemented with some restrictions. The Fortran support methods are abbreviated with `S1` = the `mpi_f08` module, `S2` = the `mpi` module, and `S3` = the `mpif.f` include file. If not stated otherwise, restrictions exist for each method that prevent implementing the complete semantics of MPI.

- `MPI_SUBARRAYS_SUPPORTED` equals `.FALSE.`, i.e., subscript triplets and non-contiguous subarrays cannot be used as buffers in nonblocking routines, RMA, or split-collective I/O.
- `S1`, `S2`, and `S3` can be implemented, but for `S1`, only a preliminary implementation is possible.
- In this preliminary interface of `S1`, the following changes are necessary:
 - * `TYPE(*)`, `DIMENSION(..)` is substituted by nonstandardized extensions like `!$PRAGMA IGNORE_TKR`.
 - * The `ASYNCHRONOUS` attribute is omitted.
 - * `PROCEDURE(...)` callback declarations are substituted by `EXTERNAL`.
- The specific procedure names are specified in Section 21.1.5.
- Due to the rules specified in Section 21.1.5, choice buffer declarations should be implemented only with nonstandardized extensions like `!$PRAGMA IGNORE_TKR` (as long as F2008 with TS 29113 or Fortran 2018 is not available).

In `S2` and `S3`: Without such extensions, routines with choice buffers should be provided with an implicit interface, instead of overloading with a different MPI function for each possible buffer type (as mentioned in Section 21.1.11). Such overloading would also imply restrictions for passing Fortran derived types as choice buffer, see also Section 21.1.15.

Only in `S1`: The implicit interfaces for routines with choice buffer arguments imply that the `ierror` argument cannot be defined as `OPTIONAL`. For this reason, it is recommended not to provide the `mpi_f08` module if such an extension is not available.

- The `ASYNCHRONOUS` attribute can **not** be used in applications to protect buffers in nonblocking MPI calls (`S1`–`S3`).
- The `TYPE(C_PTR)` binding of the `MPI_ALLOC_MEM` and `MPI_WIN_ALLOCATE` routines is not available.
- In `S1` and `S2`, the definition of the handle types (e.g., `TYPE(MPI_Comm)` and the status type `TYPE(MPI_Status)` must be modified: The `SEQUENCE` attribute must be used instead of `BIND(C)` (which is not available in Fortran 90/95). This restriction implies that the application must be fully recompiled if one switches to an MPI library for Fortran 2003 and later because the internal memory size of the handles may have changed. For this reason, an implementor may choose not

to provide the `mpi_f08` module for Fortran 90 compilers. In this case, the `mpi_f08` handle types and all routines, constants and types related to `TYPE(MPI_Status)` (see Section 21.3.5) are also not available in the `mpi` module and `mpif.h`.

- *For Fortran 95:*

The quality of the MPI interface and the restrictions are the same as with Fortran 90.

- *For Fortran 2003:*

The major features that are needed from Fortran 2003 are:

- Interoperability with C, i.e.,
 - * `BIND(C)` derived types.
 - * The `ISO_C_BINDING` intrinsic type `C_PTR` and routine `C_F_POINTER`.
- The ability to define an `ABSTRACT INTERFACE` and to use it for `PROCEDURE` dummy arguments.
- The ability to overload the operators `.EQ.` and `.NE.` to allow the comparison of derived types (used in MPI-3.0 and later for MPI handles).
- The `ASYNCHRONOUS` attribute is available to protect Fortran asynchronous I/O. This feature is not yet used by MPI, but it is the basis for the enhancement for MPI communication in the TS 29113.

With these features (but still without the features of TS 29113), MPI-1.1 – MPI-2.2 can be implemented without restrictions, but with one enhancement:

- The user application can use `TYPE(C_PTR)` together with `MPI_ALLOC_MEM` as long as `MPI_ALLOC_MEM` is defined with an implicit interface because a `C_PTR` and an `INTEGER(KIND=MPI_ADDRESS_KIND)` argument must both map to a `void *` argument.

MPI-3.0 and later can be implemented with the following restrictions:

- `MPI_SUBARRAYS_SUPPORTED` equals `.FALSE.`
- For `S1`, only a preliminary implementation is possible. The following changes are necessary:
 - * `TYPE(*)`, `DIMENSION(..)` is substituted by nonstandardized extensions like `!$PRAGMA IGNORE_TKR`.
- The specific procedure names are specified in Section 21.1.5.
- With `S1`, the `ASYNCHRONOUS` is required as specified in the second Fortran interfaces. With `S2` and `S3` the implementation can also add this attribute if explicit interfaces are used.
- The `ASYNCHRONOUS` Fortran attribute can be used in applications to *try to* protect buffers in nonblocking MPI calls, but the protection can work only if the compiler is able to protect asynchronous Fortran I/O and makes no difference between such asynchronous Fortran I/O and MPI communication.
- The `TYPE(C_PTR)` binding of the `MPI_ALLOC_MEM`, `MPI_WIN_ALLOCATE`, `MPI_WIN_ALLOCATE_SHARED`, and `MPI_WIN_SHARED_QUERY` routines can be used only for Fortran types that are C compatible.

- The same restriction as for Fortran 90 applies if nonstandardized extensions like `!$PRAGMA IGNORE_TKR` are not available.
- *For Fortran 2008 with TS 29113 and later and
For Fortran 2003 with TS 29113:*
The major features that are needed from TS 29113 are:
 - `TYPE(*)`, `DIMENSION(..)` is available.
 - The `ASYNCHRONOUS` attribute is extended to protect also nonblocking MPI communication.
 - The array dummy argument of the `ISO_C_BINDING` intrinsic `C_F_POINTER` is not restricted to Fortran types for which a corresponding type in C exists.

Using these features, MPI-3.0 and later can be implemented without any restrictions.

- With S1, `MPI_SUBARRAYS_SUPPORTED` equals `.TRUE.`. The `ASYNCHRONOUS` attribute can be used to protect buffers in nonblocking MPI calls. The `TYPE(C_PTR)` binding of the `MPI_ALLOC_MEM`, `MPI_WIN_ALLOCATE`, `MPI_WIN_ALLOCATE_SHARED`, and `MPI_WIN_SHARED_QUERY` routines can be used for any Fortran type.
- With S2 and S3, the value of `MPI_SUBARRAYS_SUPPORTED` is implementation dependent. A high quality implementation will also provide `MPI_SUBARRAYS_SUPPORTED` set to `.TRUE.` and will use the `ASYNCHRONOUS` attribute in the same way as in S1.
- If nonstandardized extensions like `!$PRAGMA IGNORE_TKR` are not available then S2 must be implemented with `TYPE(*)`, `DIMENSION(..)`.

Advice to implementors. If `MPI_SUBARRAYS_SUPPORTED=.FALSE.`, the choice argument may be implemented with an explicit interface using compiler directives.

Example 21.3. Use of typical compiler directives to disable type, kind, and rank checks for choice buffer arguments.

```

INTERFACE
  SUBROUTINE MPI_...(buf, ...)
    !$DEC$ ATTRIBUTES NO_ARG_CHECK :: buf
    !$PRAGMA IGNORE_TKR buf
    !$DIR$ IGNORE_TKR buf
    !$IBM* IGNORE_TKR buf
    REAL, DIMENSION(*) :: buf
    ... ! declarations of the other arguments
  END SUBROUTINE
END INTERFACE

```

(End of advice to implementors.)

21.1.7 Requirements on Fortran Compilers

MPI-3.0 (and later) compliant Fortran bindings are not only a property of the MPI library itself, but rather a property of an MPI library together with the Fortran compiler suite for which it is compiled.

Advice to users. Users must take appropriate steps to ensure that proper options are specified to compilers. MPI libraries must document these options. Some MPI libraries are shipped together with special compilation scripts (e.g., `mpif90`, `mpicc`) that set these options automatically. (*End of advice to users.*)

An MPI library together with the Fortran compiler suite is only compliant with MPI-3.0 (and later), as referred by `MPI_GET_VERSION`, if all the solutions described in Sections 21.1.11 through 21.1.19 work correctly. Based on this rule, major requirements for all three Fortran support methods (i.e., the `mpi_f08` and `mpi` modules, and `mpif.h`) are:

- The language features assumed-type and assumed-rank from Fortran 2008 TS 29113 [47] are available. This is required only for `mpi_f08`. As long as this requirement is not supported by the compiler, it is valid to build an MPI library that implements the `mpi_f08` module with `MPI_SUBARRAYS_SUPPORTED` set to `.FALSE..`
- “Simply contiguous” arrays and scalars must be passed to choice buffer dummy arguments of nonblocking routines with call by reference. This is needed only if one of the support methods does not use the `ASYNCHRONOUS` attribute. See Section 21.1.12 for more details.
- `SEQUENCE` and `BIND(C)` derived types are valid as actual arguments passed to choice buffer dummy arguments, and, in the case of `MPI_SUBARRAYS_SUPPORTED` set to `.FALSE..`, they are passed with call by reference, and passed by descriptor in the case of `.TRUE..`
- All actual arguments that are allowed for a dummy argument in an implicitly defined and separately compiled Fortran routine with the given compiler (e.g., `CHARACTER(LEN=*)` strings and array of strings) must also be valid for choice buffer dummy arguments with all Fortran support methods.
- The array dummy argument of the `ISO_C_BINDING` intrinsic module procedure `C_F_POINTER` is not restricted to Fortran types for which a corresponding type in C exists.
- The Fortran compiler shall not provide `TYPE(*)` unless the `ASYNCHRONOUS` attribute protects MPI communication as described in TS 29113. Specifically, the TS 29113 must be implemented as a whole.

The following rules are required at least as long as the compiler does not provide the extension of the `ASYNCHRONOUS` attribute as part of TS 29113 and there still exists a Fortran support method with `MPI_ASYNC_PROTECTS_NONBLOCKING` set to `.FALSE..` Observation of these rules by the MPI application developer is especially recommended for backward compatibility of existing applications that use the `mpi` module or the (deprecated) `mpif.h` include file. The rules are as follows:

- Separately compiled empty Fortran routines with implicit interfaces and separately compiled empty C routines with `BIND(C)` Fortran interfaces (e.g., `MPI_F_SYNC_REG` on page 850 and Section 21.1.8, and `DD` on page 851) solve the problems described in Section 21.1.17.

Advice to implementors. Some of these rules are already part of the Fortran 2003 standard, some of these requirements require the Fortran TS 29113 [47], and some of these requirements for MPI are beyond the scope of TS 29113. (*End of advice to implementors.*)

21.1.8 Additional Support for Fortran Register-Memory-Synchronization

As described in Section 21.1.17, a dummy call may be necessary to tell the compiler that registers are to be flushed for a given buffer or that accesses to a buffer may not be moved across a given point in the execution sequence. Only a Fortran binding exists for this call.

MPI_F_SYNC_REG(buf)

INOUT buf initial address of buffer (choice)

Fortran 2008 binding

```
MPI_F_sync_reg(buf)
```

```
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
```

Fortran binding

MPI_F_SYNC_REG(BUF)

<type> BUF(*)

This routine has no executable statements. It must be compiled in the MPI library in such a manner that a Fortran compiler cannot detect in the module that the routine has an empty body. It is used only to force the compiler to flush a cached register value of a variable or buffer back to memory (when necessary), or to invalidate the register value.

Rationale. This function is not available in other languages because it would not be useful. This routine has no `ierror` return argument because there is no operation that can fail. (*End of rationale.*)

Advice to implementors. This routine can be bound to a C routine to minimize the risk that the Fortran compiler can learn that this routine is empty (and that the call to this routine can be removed as part of an optimization). However, it is explicitly allowed to implement this routine within the `mpi_f08` module according to the definition for the `mpi` module or `mpif.h` to circumvent the overhead of building the internal dope vector to handle the assumed-type, assumed-rank argument. (*End of advice to implementors.*)

Rationale. This routine is not defined with `TYPE(*)`, `DIMENSION(*)`, i.e., assumed size instead of assumed rank, because this would restrict the usability to “simply contiguous” arrays and would require overloading with another interface for scalar arguments. (*End of rationale.*)

Advice to users. If only a part of an array (e.g., defined by a subscript triplet) is used in a nonblocking routine, it is recommended to pass the whole array to `MPI_F_SYNC_REG` anyway to minimize the overhead of this no-operation call. Note that this routine need not be called if `MPI_ASYNC_PROTECTS_NONBLOCKING` is `.TRUE.` and the application fully uses the facilities of `ASYNCHRONOUS` arrays. (*End of advice to users.*)

21.1.9 Additional Support for Fortran Numeric Intrinsic Types

MPI provides a small number of named datatypes that correspond to named intrinsic types supported by C and Fortran. These include `MPI_INTEGER`, `MPI_REAL`, `MPI_INT`, `MPI_DOUBLE`, etc., as well as the optional types `MPI_REAL4`, `MPI_REAL8`, etc. There is a one-to-one correspondence between language declarations and MPI types.

Fortran (starting with Fortran 90) provides so-called `KIND`-parameterized types. These types are declared using an intrinsic type (one of `INTEGER`, `REAL`, `COMPLEX`, `LOGICAL`, and `CHARACTER`) with an optional integer `KIND` parameter that selects from among one or more variants. The specific meaning of different `KIND` values themselves are implementation dependent and not specified by the language. Fortran provides the `KIND` selection functions `selected_real_kind` for `REAL` and `COMPLEX` types, and `selected_int_kind` for `INTEGER` types that allow users to declare variables with a minimum precision or number of digits. These functions provide a portable way to declare `KIND`-parameterized `REAL`, `COMPLEX`, and `INTEGER` variables in Fortran. This scheme is backward compatible with Fortran 77. `REAL` and `INTEGER` Fortran variables have a default `KIND` if none is specified. Fortran `DOUBLE PRECISION` variables are of intrinsic type `REAL` with a nondefault `KIND`. The following two declarations are equivalent:

```
double precision x
real(KIND(0.0d0)) x
```

MPI provides two orthogonal methods for handling communication buffers of numeric intrinsic types. The first method (see the following section) can be used when variables have been declared in a portable way—using default `KIND` or using `KIND` parameters obtained with the `selected_int_kind` or `selected_real_kind` functions. With this method, MPI automatically selects the correct data size (e.g., 4 or 8 bytes) and provides representation conversion in heterogeneous environments. The second method (see “[Support for Size-specific MPI Datatypes](#)” on page 833) gives the user complete control over communication by exposing machine representations.

Parameterized Datatypes with Specified Precision and Exponent Range

MPI provides named datatypes corresponding to standard Fortran 77 numeric types: `MPI_INTEGER`, `MPI_COMPLEX`, `MPI_REAL`, `MPI_DOUBLE_PRECISION` and `MPI_DOUBLE_COMPLEX`. MPI automatically selects the correct data size and provides representation conversion in heterogeneous environments. The mechanism described in this section extends this model to support portable parameterized numeric types.

The model for supporting portable parameterized types is as follows. Real variables are declared (perhaps indirectly) using `selected_real_kind(p, r)` to determine the `KIND` parameter, where `p` is decimal digits of precision and `r` is an exponent range. Implicitly MPI maintains a two-dimensional array of predefined MPI datatypes `D(p, r)`. `D(p, r)` is defined for each value of `(p, r)` supported by the compiler, including pairs for which one value is unspecified. Attempting to access an element of the array with an index `(p, r)` not supported by the compiler is erroneous. MPI implicitly maintains a similar array of `COMPLEX` datatypes. For integers, there is a similar implicit array related to `selected_int_kind` and indexed by the requested number of digits `r`. Note that the predefined datatypes contained in these implicit arrays are not the same as the named MPI datatypes `MPI_REAL`, etc., but a new set.

Advice to implementors. The above description is for explanatory purposes only. It is not expected that implementations will have such internal arrays. (*End of advice to implementors.*)

Advice to users. `selected_real_kind()` maps a large number of `(p, r)` pairs to a much smaller number of `KIND` parameters supported by the compiler. `KIND` parameters are not specified by the language and are not portable. From the language point of view intrinsic types of the same base type and `KIND` parameter are of the same type. In order to allow interoperability in a heterogeneous environment, MPI is more stringent. The corresponding MPI datatypes match if and only if they have the same `(p, r)` value (`REAL` and `COMPLEX`) or `r` value (`INTEGER`). Thus MPI has many more datatypes than there are fundamental language types. (*End of advice to users.*)

MPI_TYPE_CREATE_F90_REAL(p, r, newtype)

IN	<code>p</code>	precision, in decimal digits (integer)
IN	<code>r</code>	decimal exponent range (integer)
OUT	<code>newtype</code>	the requested MPI datatype (handle)

C binding

```
int MPI_Type_create_f90_real(int p, int r, MPI_Datatype *newtype)
```

Fortran 2008 binding

```
MPI_Type_create_f90_real(p, r, newtype, ierror)
  INTEGER, INTENT(IN) :: p, r
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_CREATE_F90_REAL(P, R, NEWTYPE, IERROR)
  INTEGER P, R, NEWTYPE, IERROR
```

This function returns a predefined MPI datatype that matches a `REAL` variable of `KIND selected_real_kind(p, r)`. In the model described above it returns a handle for the element `D(p, r)`. Either `p` or `r` may be omitted from calls to `selected_real_kind(p, r)` (but not both). Analogously, either `p` or `r` may be set to `MPI_UNDEFINED`. In communication, an

MPI datatype A returned by `MPI_TYPE_CREATE_F90_REAL` matches a datatype B if and only if B was returned by `MPI_TYPE_CREATE_F90_REAL` called with the same values for `p` and `r` or B is a duplicate of such a datatype. Restrictions on using the returned datatype with the "external32" data representation are given on page 833.

It is erroneous to supply values for `p` and `r` not supported by the compiler.

`MPI_TYPE_CREATE_F90_COMPLEX(p, r, newtype)`

IN	<code>p</code>	precision, in decimal digits (integer)
IN	<code>r</code>	decimal exponent range (integer)
OUT	<code>newtype</code>	the requested MPI datatype (handle)

C binding

```
int MPI_Type_create_f90_complex(int p, int r, MPI_Datatype *newtype)
```

Fortran 2008 binding

```
MPI_Type_create_f90_complex(p, r, newtype, ierror)
  INTEGER, INTENT(IN) :: p, r
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_CREATE_F90_COMPLEX(P, R, NEWTYPE, IERROR)
  INTEGER P, R, NEWTYPE, IERROR
```

This function returns a predefined MPI datatype that matches a `COMPLEX` variable of `KIND selected_real_kind(p, r)`. Either `p` or `r` may be omitted from calls to `selected_real_kind(p, r)` (but not both). Analogously, either `p` or `r` may be set to `MPI_UNDEFINED`. Matching rules for datatypes created by this function are analogous to the matching rules for datatypes created by `MPI_TYPE_CREATE_F90_REAL`. Restrictions on using the returned datatype with the "external32" data representation are given on page 833.

It is erroneous to supply values for `p` and `r` not supported by the compiler.

`MPI_TYPE_CREATE_F90_INTEGER(r, newtype)`

IN	<code>r</code>	decimal exponent range, i.e., number of decimal digits (integer)
OUT	<code>newtype</code>	the requested MPI datatype (handle)

C binding

```
int MPI_Type_create_f90_integer(int r, MPI_Datatype *newtype)
```

Fortran 2008 binding

```
MPI_Type_create_f90_integer(r, newtype, ierror)
  INTEGER, INTENT(IN) :: r
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_TYPE_CREATE_F90_INTEGER(R, NEWTYPE, IERROR)
    INTEGER R, NEWTYPE, IERROR
```

This function returns a predefined MPI datatype that matches an INTEGER variable of KIND `selected_int_kind(r)`. Matching rules for datatypes created by this function are analogous to the matching rules for datatypes created by `MPI_TYPE_CREATE_F90_REAL`. Restrictions on using the returned datatype with the "external32" data representation are given on page 833.

It is erroneous to supply a value for `r` that is not supported by the compiler.

Example 21.4. Fortran selected integer and real kind buffers in MPI communications.

```
integer      longtype, quadtype
integer, parameter :: long = selected_int_kind(15)
integer(long) ii(10)
real(selected_real_kind(30)) x(10)
call MPI_TYPE_CREATE_F90_INTEGER(15, longtype, ierror)
call MPI_TYPE_CREATE_F90_REAL(30, MPI_UNDEFINED, quadtype, ierror)
...

call MPI_SEND(ii, 10, longtype, ...)
call MPI_SEND(x, 10, quadtype, ...)
```

Advice to users. The datatypes returned by the procedures in Example 21.4 are predefined datatypes. They cannot be freed; they do not need to be committed; they can be used with predefined reduction operations. There are two situations in which they behave differently syntactically, but not semantically, from the MPI named predefined datatypes.

1. `MPI_TYPE_GET_ENVELOPE` returns special combinators that allow a program to retrieve the values of `p` and `r`.
2. Because the datatypes are not named, they cannot be used as compile-time initializers or otherwise accessed before a call to one of the `MPI_TYPE_CREATE_F90_XXX` routines.

If a variable was declared specifying a nondefault KIND value that was not obtained with `selected_real_kind()` or `selected_int_kind()`, the only way to obtain a matching MPI datatype is to use the size-based mechanism described in the next section. (*End of advice to users.*)

Advice to implementors. An application may often repeat a call to `MPI_TYPE_CREATE_F90_XXX` with the same combination of `(XXX,p,r)`. The application is not allowed to free the returned predefined, unnamed datatype handles. To prevent the creation of a potentially huge amount of handles, a high quality MPI implementation should return the same datatype handle for the same `(REAL/COMPLEX/INTEGER,p,r)` combination. Checking for the combination `(p,r)` in the preceding call to `MPI_TYPE_CREATE_F90_XXX` and using a hash table to find formerly generated handles should limit the overhead of finding a previously generated datatype with same combination of `(XXX,p,r)`. (*End of advice to implementors.*)

Rationale. The `MPI_TYPE_CREATE_F90_REAL/COMPLEX/INTEGER` interface needs as input the original range and precision values to be able to define useful and compiler-independent external (Section 15.5.2) or user-defined (Section 15.5.3) data representations, and in order to be able to perform automatic and efficient data conversions in a heterogeneous environment. (*End of rationale.*)

We now specify how the datatypes described in this section behave when used with the "external32" external data representation described in Section 15.5.2.

The "external32" representation specifies data formats for integer and floating point values. Integer values are represented in two's complement big-endian format. Floating point values are represented by one of three IEEE formats. These are the IEEE "Single," "Double," and "Double Extended" formats, requiring 4, 8, and 16 bytes of storage, respectively. For the IEEE "Double Extended" formats, MPI specifies a Format Width of 16 bytes, with 15 exponent bits, bias = +10383, 112 fraction bits, and an encoding analogous to the "Double" format.

The "external32" representations of the datatypes returned by `MPI_TYPE_CREATE_F90_REAL/COMPLEX/INTEGER` are given by the following rules.

For `MPI_TYPE_CREATE_F90_REAL`:

```

if      (p > 33) or (r > 4931) then  external32 representation
                                     is undefined
else if (p > 15) or (r > 307) then  external32_size = 16
else if (p > 6) or (r > 37) then  external32_size = 8
else                                external32_size = 4

```

For `MPI_TYPE_CREATE_F90_COMPLEX`: twice the size as for `MPI_TYPE_CREATE_F90_REAL`.

For `MPI_TYPE_CREATE_F90_INTEGER`:

```

if      (r > 38) then  external32 representation is undefined
else if (r > 18) then  external32_size = 16
else if (r > 9) then   external32_size = 8
else if (r > 4) then   external32_size = 4
else if (r > 2) then   external32_size = 2
else                  external32_size = 1

```

If the "external32" representation of a datatype is undefined, the result of using the datatype directly or indirectly (i.e., as part of another datatype or through a duplicated datatype) in operations that require the "external32" representation is undefined. These operations include `MPI_PACK_EXTERNAL`, `MPI_UNPACK_EXTERNAL`, and many `MPI_FILE` functions, when the "external32" data representation is used. The ranges for which the "external32" representation is undefined are reserved for future standardization.

Support for Size-specific MPI Datatypes

MPI provides named datatypes corresponding to optional Fortran 77 numeric types that contain explicit byte lengths—`MPI_REAL4`, `MPI_INTEGER8`, etc. This section describes a mechanism that generalizes this model to support all Fortran numeric intrinsic types.

We assume that for each **typeclass** (integer, real, complex) and each word size **n** there is a unique machine representation. For every pair (**typeclass**, **n**) supported by a compiler,

MPI must provide a named size-specific datatype. The name of this datatype is of the form `MPI_<TYPECLASS><n>` in C and Fortran where `<TYPECLASS>` is one of `REAL`, `INTEGER`, or `COMPLEX`, and `<n>` is the length in bytes of the machine representation. This datatype locally matches all variables of type (**typeclass**, **n**) in Fortran. The list of names for such types includes:

```

MPI_REAL4
MPI_REAL8
MPI_REAL16
MPI_COMPLEX8
MPI_COMPLEX16
MPI_COMPLEX32
MPI_INTEGER1
MPI_INTEGER2
MPI_INTEGER4
MPI_INTEGER8
MPI_INTEGER16
MPI_LOGICAL1
MPI_LOGICAL2
MPI_LOGICAL4
MPI_LOGICAL8
MPI_LOGICAL16

```

One datatype is required for each representation supported by the Fortran compiler.

Rationale. Particularly for the longer floating-point types, C and Fortran may use different representations. For example, a Fortran compiler may define a 16-byte `REAL` type with 33 decimal digits of precision while a C compiler may define a 16-byte long double type that implements an 80-bit (10 byte) extended precision floating point value. Both of these types are 16 bytes long, but they are not interoperable. Thus, these types are defined by Fortran, even though C may define types of the same length. (*End of rationale.*)

To be backward compatible with the interpretation of these types in MPI-1, we assume that the nonstandard declarations `REAL*n`, `INTEGER*n`, `LOGICAL*n`, always create a variable whose representation is of size **n**. These datatypes may also be used for variables declared with `KIND=INT8/16/32/64` or `KIND=REAL32/64/128`, which are defined in the `ISO_FORTRAN_ENV` intrinsic module. Note that the MPI datatypes and the `REAL*n`, `INTEGER*n`, `LOGICAL*n` declarations count bytes whereas the Fortran `KIND` values count bits. All these datatypes are predefined.

The following function allows a user to obtain a size-specific MPI datatype for any intrinsic Fortran type.

`MPI_TYPE_MATCH_SIZE(typeclass, size, datatype)`

IN	typeclass	generic type specifier (integer)
IN	size	size, in bytes, of representation (integer)
OUT	datatype	datatype with correct type, size (handle)

C binding

```
int MPI_Type_match_size(int typeclass, int size, MPI_Datatype *datatype)
```

Fortran 2008 binding

```

MPI_Type_match_size(typeclass, size, datatype, ierror)
  INTEGER, INTENT(IN) :: typeclass, size
  TYPE(MPI_Datatype), INTENT(OUT) :: datatype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_TYPE_MATCH_SIZE(TYPECLASS, SIZE, DATATYPE, IERROR)
  INTEGER TYPECLASS, SIZE, DATATYPE, IERROR

```

typeclass is one of MPI_TYPECLASS_REAL, MPI_TYPECLASS_INTEGER, and MPI_TYPECLASS_COMPLEX, corresponding to the desired **typeclass**. The function returns an MPI datatype matching a local variable of type (**typeclass**, **size**).

This function returns a reference (handle) to one of the predefined named datatypes, not a duplicate. This type cannot be freed. **MPI_TYPE_MATCH_SIZE** can be used to obtain a size-specific type that matches a Fortran numeric intrinsic type by first calling **storage_size()** in order to compute the variable size in bits, dividing it by eight, and then calling **MPI_TYPE_MATCH_SIZE** to find a suitable datatype. In C, one can use the C operator **sizeof()** (which returns the size in bytes) instead of **storage_size()** (which returns the size in bits). In addition, for variables of default kind the variable's size can be computed by a call to **MPI_TYPE_GET_EXTENT**, if the typeclass is known. It is erroneous to specify a size not supported by the compiler.

Rationale. This is a convenience function. Without it, it can be tedious to find the correct named type. See note to implementors below. (*End of rationale.*)

Advice to implementors. This function could be implemented as a series of tests.

Example 21.5. Example of an implementation of MPI_TYPE_MATCH_SIZE.

```

int MPI_Type_match_size(int typeclass, int size, MPI_Datatype *rtype)
{
  switch(typeclass) {
    case MPI_TYPECLASS_REAL: switch(size) {
      case 4: *rtype = MPI_REAL4; return MPI_SUCCESS;
      case 8: *rtype = MPI_REAL8; return MPI_SUCCESS;
      default: error(...);
    }
    case MPI_TYPECLASS_INTEGER: switch(size) {
      case 4: *rtype = MPI_INTEGER4; return MPI_SUCCESS;
      case 8: *rtype = MPI_INTEGER8; return MPI_SUCCESS;
      default: error(...);
    }
    ... etc. ...
  }

  return MPI_SUCCESS;
}

```

(*End of advice to implementors.*)

Communication With Size-specific Types

The usual type matching rules apply to size-specific datatypes: a value sent with datatype `MPI_<TYPECLASS><n>` can be received with this same datatype on another MPI process. Most modern computers use two's complement for integers and IEEE format for floating point. Thus, communication using these size-specific datatypes will not entail loss of precision or truncation errors.

Advice to users. Care is required when communicating in a heterogeneous environment. Consider the following code:

Example 21.6. Unsafe heterogeneous communication due to the use of `MPI_TYPE_MATCH_SIZE`.

```
real(selected_real_kind(5)) x(100)
size = storage_size(x) / 8
call MPI_TYPE_MATCH_SIZE(MPI_TYPECLASS_REAL, size, xtype, ierror)
if (myrank .eq. 0) then
    ... initialize x ...
    call MPI_SEND(x, xtype, 100, 1, ...)
else if (myrank .eq. 1) then
    call MPI_RECV(x, xtype, 100, 0, ...)
endif
```

This may not work in a heterogeneous environment if the value of `size` is not the same on the MPI processes with ranks 0 and 1. There should be no problem in a homogeneous environment. To communicate in a heterogeneous environment, there are at least four options. The first is to declare variables of default type and use the MPI datatypes for these types, e.g., declare a variable of type `REAL` and use `MPI_REAL`. The second is to use `selected_real_kind` or `selected_int_kind` and with the functions of the previous section. The third is to declare a variable that is known to be the same size on all architectures (e.g., `selected_real_kind(12)` on almost all compilers will result in an 8-byte representation). The fourth is to carefully check representation size before communication. This may require explicit conversion to a variable of size that can be communicated and handshaking between sender and receiver to agree on a size.

Note finally that using the "external32" representation for I/O requires explicit attention to the representation sizes. Consider the following code:

Example 21.7. Unsafe heterogeneous MPI file I/O due to the use of `MPI_TYPE_MATCH_SIZE`.

```
real(selected_real_kind(5)) x(100)
size = storage_size(x) / 8
call MPI_TYPE_MATCH_SIZE(MPI_TYPECLASS_REAL, size, xtype, ierror)

if (myrank .eq. 0) then
    call MPI_FILE_OPEN(MPI_COMM_SELF, 'foo', &
                     MPI_MODE_CREATE+MPI_MODE_WRONLY, &
                     MPI_INFO_NULL, fh, ierror)
    call MPI_FILE_SET_VIEW(fh, zero, xtype, xtype, 'external32', &
                          MPI_INFO_NULL, ierror)
```



```

    call MPI_FILE_WRITE(fh, x, 100, xtype, status, ierror)
    call MPI_FILE_CLOSE(fh, ierror)
endif

call MPI_BARRIER(MPI_COMM_WORLD, ierror)

if (myrank .eq. 1) then
    call MPI_FILE_OPEN(MPI_COMM_SELF, 'foo', MPI_MODE_RDONLY, &
        MPI_INFO_NULL, fh, ierror)
    call MPI_FILE_SET_VIEW(fh, zero, xtype, xtype, 'external32', &
        MPI_INFO_NULL, ierror)
    call MPI_FILE_READ(fh, x, 100, xtype, status, ierror)
    call MPI_FILE_CLOSE(fh, ierror)
endif

```

If the MPI processes with ranks 0 and 1 are on different machines, this code may not work as expected if the size is different on the two machines. (*End of advice to users.*)

21.1.10 Problems With Fortran Bindings for MPI

This section discusses a number of problems that may arise when using MPI in a Fortran program. It is intended as advice to users, and clarifies how MPI interacts with Fortran. It is intended to clarify, not add to, this standard.

As noted in the original MPI specification, the interface violates the Fortran standard in several ways. While these may cause few problems for Fortran 77 programs, they become more significant for Fortran 90 programs, so that users must exercise care when using new Fortran 90 features. With Fortran 2008 and the semantics defined in TS 29113, most violations are resolved, and this is hinted at in an addendum to each item. The violations were originally adopted and have been retained because they are important for the usability of MPI. The rest of this section describes the potential problems in detail.

The following MPI features are inconsistent with Fortran 90 and Fortran 77.

1. An MPI subroutine with a choice argument may be called with different argument types. When using the `mpi_f08` module together with a compiler that supports Fortran 2008 with TS 29113, this problem is resolved.
2. An MPI subroutine with an assumed-size dummy argument may be passed an actual scalar argument. This is only solved for choice buffers through the use of `DIMENSION(...)`.
3. Nonblocking and split-collective MPI routines assume that actual arguments are passed by address or descriptor and that arguments and the associated data are not copied on entrance to or exit from the subroutine. This problem is solved with the use of the `ASYNCHRONOUS` attribute.
4. An MPI implementation may read or modify user data (e.g., communication buffers used by nonblocking communications) concurrently with a user program that is executing outside of MPI calls. This problem is resolved by relying on the extended semantics of the `ASYNCHRONOUS` attribute as specified in TS 29113.

5. Several named “constants,” such as `MPI_BOTTOM`, `MPI_IN_PLACE`, `MPI_STATUS_IGNORE`, `MPI_STATUSES_IGNORE`, `MPI_ERRCODES_IGNORE`, `MPI_UNWEIGHTED`, `MPI_WEIGHTS_EMPTY`, `MPI_ARGV_NULL`, and `MPI_ARGVS_NULL` are not ordinary Fortran constants and require a special implementation. See Section 2.5.4 for more information.
6. The memory allocation routine `MPI_ALLOC_MEM` cannot be used from Fortran 77/90/95 without a language extension (for example, Cray pointers) that allows the allocated memory to be associated with a Fortran variable. Therefore, address sized integers were used in MPI-2.0 – MPI-2.2. In Fortran 2003, `TYPE(C_PTR)` entities were added, which allow a standard-conforming implementation of the semantics of `MPI_ALLOC_MEM`. In MPI-3.0 and later, `MPI_ALLOC_MEM` has an additional, overloaded interface to support this language feature. The use of Cray pointers is deprecated. The `mpi_f08` module only supports `TYPE(C_PTR)` pointers.

Additionally, MPI is inconsistent with Fortran 77 in a number of ways, as noted below.

- MPI identifiers exceed 6 characters.
- MPI identifiers may contain underscores after the first character.
- MPI requires an include file, `mpif.h` (deprecated). On systems that do not support include files, the implementation should specify the values of named constants.
- Many routines in MPI have `KIND`-parameterized integers (e.g., `MPI_ADDRESS_KIND` and `MPI_OFFSET_KIND`) that hold address information. On systems that do not support Fortran 90-style parameterized types, `INTEGER*8` or `INTEGER` should be used instead.

MPI-1 contained several routines that take address-sized information as input or return address-sized information as output. In C such arguments were of type `MPI_Aint` and in Fortran of type `INTEGER`. On machines where integers are smaller than addresses, these routines can lose information. In MPI-2 the use of these functions has been deprecated and they have been replaced by routines taking `INTEGER` arguments of `KIND=MPI_ADDRESS_KIND`. A number of MPI-2 functions also take `INTEGER` arguments of nondefault `KIND`. See Section 2.6 and Section 5.1.1 for more information.

Sections 21.1.11 through 21.1.19 describe several problems in detail that concern the interaction of MPI and Fortran as well as their solutions. Some of these solutions require special capabilities from the compilers. Major requirements are summarized in Section 21.1.7.

21.1.11 Problems Due to Strong Typing

All MPI functions with choice arguments associate actual arguments of different Fortran datatypes with the same dummy argument. This is not allowed by Fortran 77, and in Fortran 90, it is technically only allowed if the function is overloaded with a different function for each type (see also Section 21.1.6). In C, the use of `void*` formal arguments avoids these problems. Similar to C, with Fortran 2008 with TS 29113 (and later) together with the `mpi_f08` module, the problem is avoided by declaring choice arguments with `TYPE(*)`, `DIMENSION(..)`, i.e., as assumed-type and assumed-rank dummy arguments.

Using `INCLUDE 'mpif.h'` (deprecated), the following code fragment is technically invalid and may generate a compile-time error.

```

integer i(5)
real    x(5)
...
call mpi_send(x, 5, MPI_REAL, ...)
call mpi_send(i, 5, MPI_INTEGER, ...)

```

In practice, it is rare for compilers to do more than issue a warning. When using either the `mpi_f08` or `mpi` module, the problem is usually resolved through the assumed-type and assumed-rank declarations of the dummy arguments, or with a compiler-dependent mechanism that overrides type checking for choice arguments.

It is also technically invalid in Fortran to pass a scalar actual argument to an array dummy argument that is not a choice buffer argument. Thus, when using the `mpi_f08` or `mpi` module, the following code fragment usually generates an error since the `dims` and `periods` arguments to `MPI_CART_CREATE` are declared as assumed size arrays `INTEGER :: DIMS(*)` and `LOGICAL :: PERIODS(*)`.

Example 21.8. It is erroneous to pass a variable instead of an array with one element.

```

! ----- THIS EXAMPLE IS ERRONEOUS -----
USE mpi_f08      ! or  USE mpi
INTEGER size
CALL MPI_Cart_create(comm_old, 1, size, .TRUE., .TRUE., comm_cart, ierror)

```

Although this is a nonconforming MPI call, compiler warnings are not expected (but may occur) when using `INCLUDE 'mpif.h'` (deprecated) and this include file does not use Fortran explicit interfaces.

21.1.12 Problems Due to Data Copying and Sequence Association with Subscript Triplets

Arrays with subscript **triplets** describe Fortran subarrays with or without strides, e.g.,

Example 21.9. Fortran subarrays as actual buffer in MPI procedures.

```

REAL a(100,100,100)
CALL MPI_Send(a(11:17, 12:99:3, 1:100), 7*30*100, MPI_REAL, ...)

```

The handling of subscript triplets depends on the value of the constant `MPI_SUBARRAYS_SUPPORTED`:

- If `MPI_SUBARRAYS_SUPPORTED` equals `.TRUE.:`

Choice buffer arguments are declared as `TYPE(*)`, `DIMENSION(..)`. For example, consider the following code fragment:

Example 21.10. Fortran subarrays without restrictions if `MPI_SUBARRAYS_SUPPORTED` equals `.TRUE..`

```

REAL s(100), r(100)
CALL MPI_Isend(s(1:100:5), 3, MPI_REAL, ..., rq, ierror)
CALL MPI_Wait(rq, status, ierror)
CALL MPI_Irecv(r(1:100:5), 3, MPI_REAL, ..., rq, ierror)
CALL MPI_Wait(rq, status, ierror)

```

In this case, the individual elements `s(1)`, `s(6)`, and `s(11)` are sent between the start of `MPI_ISEND` and the end of `MPI_WAIT` even though the compiled code will not copy `s(1:100:5)` to a real contiguous temporary scratch buffer. Instead, the compiled code will pass a descriptor to `MPI_ISEND` that allows MPI to operate directly on `s(1)`, `s(6)`, `s(11)`, ..., `s(96)`. The called `MPI_ISEND` routine will take only the first three of these elements due to the type signature “3, `MPI_REAL`”.

All nonblocking MPI functions (e.g., `MPI_ISEND`, `MPI_PUT`, `MPI_FILE_WRITE_ALL_BEGIN`) behave as if *the user-specified elements of choice buffers are copied to a contiguous scratch buffer in the MPI runtime environment*. All datatype descriptions (in the example above, “3, `MPI_REAL`”) read and store data from and to this virtual contiguous scratch buffer. Displacements in MPI derived datatypes are relative to the beginning of this virtual contiguous scratch buffer. Upon completion of a nonblocking receive operation (e.g., when `MPI_WAIT` on a corresponding `MPI_Request` returns), it is as if the received data has been copied from the virtual contiguous scratch buffer back to the noncontiguous application buffer. In the example above, `r(1)`, `r(6)`, and `r(11)` are guaranteed to be defined with the received data when `MPI_WAIT` returns.

Note that the above definition does not supercede restrictions about buffers used with nonblocking operations (e.g., those specified in Section 3.7.2).

Advice to implementors. The Fortran descriptor for `TYPE(*)`, `DIMENSION(..)` arguments contains enough information that, if desired, the MPI library can make a real contiguous copy of noncontiguous user buffers when the nonblocking operation is started, and release this buffer not before the nonblocking communication has completed (e.g., the `MPI_WAIT` routine). Efficient implementations may avoid such additional memory-to-memory data copying. (*End of advice to implementors.*)

Rationale. If `MPI_SUBARRAYS_SUPPORTED` equals `.TRUE.`, non-contiguous buffers are handled inside the MPI library instead of by the compiler through argument association conventions. Therefore, the scope of MPI library scratch buffers can be from the beginning of a nonblocking operation until the completion of the operation although beginning and completion are implemented in different routines. (*End of rationale.*)

- If `MPI_SUBARRAYS_SUPPORTED` equals `.FALSE.`:

In this case, the use of Fortran arrays with subscript triplets as actual choice buffer arguments in any nonblocking MPI operation (which also includes persistent request, and split collectives) may cause undefined behavior. They may, however, be used in blocking MPI operations.

Implicit in MPI is the idea of a contiguous chunk of memory accessible through a linear address space. MPI copies data to and from this memory. An MPI program specifies the location of data by providing memory addresses and offsets. In the C language, sequence association rules plus pointers provide all the necessary low-level structure.

In Fortran, array data is not necessarily stored contiguously. For example, the array section `A(1:N:2)` involves only the elements of `A` with indices 1, 3, 5, The same is

true for a pointer array whose target is such a section. Most compilers ensure that an array that is a dummy argument is held in contiguous memory if it is declared with an explicit shape (e.g., `B(N)`) or is of assumed size (e.g., `B(*)`). If necessary, they do this by making a copy of the array into contiguous memory.¹

Because MPI dummy buffer arguments are assumed-size arrays if `MPI_SUBARRAYS_SUPPORTED` equals `.FALSE.`, this leads to a serious problem for a nonblocking call: the compiler copies the temporary array back on return but MPI continues to copy data to the memory that held it. For example, consider the following code fragment:

Example 21.11. Fortran subarrays cannot be used if `MPI_SUBARRAYS_SUPPORTED` equals `.FALSE.`.

```
!-- THIS EXAMPLE IS ERRONEOUS if MPI_SUBARRAYS_SUPPORTED==.FALSE. --
real a(100)
call MPI_IRECV(a(1:100:2), MPI_REAL, 50, ...)
```

Since the first dummy argument to `MPI_IRECV` is an assumed-size array (`<type> buf(*)`), the array section `a(1:100:2)` is copied to a temporary before being passed to `MPI_IRECV`, so that it is contiguous in memory. `MPI_IRECV` returns immediately, and data is copied from the temporary back into the array `a`. Sometime later, MPI may write to the address of the deallocated temporary. Copying is also a problem for `MPI_ISEND` since the temporary array may be deallocated before the data has all been sent from it.

Most Fortran 90 compilers do not make a copy if the actual argument is the whole of an explicit-shape or assumed-size array or is a “simply contiguous” section such as `A(1:N)` of such an array. (“Simply contiguous” is defined in the next paragraph.) Also, many compilers treat allocatable arrays the same as they treat explicit-shape arrays in this regard (though we know of one that does not). However, the same is not true for assumed-shape and pointer arrays; since they may be discontinuous, copying is often done. It is this copying that causes problems for MPI as described in the previous paragraph.

According to the Fortran 2008 Standard, Section 6.5.4, a “simply contiguous” array section is

name ([:,]... [<subscript>]:<subscript>] [,<subscript>]...)

That is, there are zero or more dimensions that are selected in full, then one dimension selected without a stride, then zero or more dimensions that are selected with a simple subscript. The compiler can detect from analyzing the source code that the array is contiguous. Examples are

`A(1:N)`, `A(:,N)`, `A(:,1:N,1)`, `A(1:6,N)`, `A(:, :, 1:N)`

Because of Fortran’s column-major ordering, where the first index varies fastest, a “simply contiguous” section of a contiguous array will also be contiguous.

The same problem can occur with a scalar argument. A compiler may make a copy of scalar dummy arguments within a called procedure when passed as an actual ar-

¹Technically, the Fortran standard is worded to allow noncontiguous storage of any array data, unless the dummy argument has the `CONTIGUOUS` attribute.

argument to a choice buffer routine. That this can cause a problem is illustrated by Example 21.12.

Example 21.12. Problem with scalar arguments.

```

real :: a
call user1(a,rq)
call MPI_WAIT(rq,status,ierr)
write (*,*) a

subroutine user1(buf,request)
call MPI_IRECV(buf,...,request,...)
end

```

If `a` is copied, `MPI_IRECV` will alter the copy when it completes the communication and will not alter `a` itself.

Note that copying will almost certainly occur for an argument that is a nontrivial expression (one with at least one operator or function call), a section that does not select a contiguous part of its parent (e.g., `A(1:n:2)`), a pointer whose target is such a section, or an assumed-shape array that is (directly or indirectly) associated with such a section.

If a compiler option exists that inhibits copying of arguments, in either the calling or called procedure, this must be employed.

If a compiler makes copies in the calling procedure of arguments that are explicit-shape or assumed-size arrays, “simply contiguous” array sections of such arrays, or scalars, and if no compiler option exists to inhibit such copying, then the compiler cannot be used for applications that use `MPI_GET_ADDRESS`, or any nonblocking MPI routine. If a compiler copies scalar arguments in the called procedure and there is no compiler option to inhibit this, then this compiler cannot be used for applications that use memory references across subroutine calls as in the example above.

21.1.13 Problems Due to Data Copying and Sequence Association with Vector Subscripts

Fortran arrays with **vector** subscripts describe subarrays containing a possibly irregular set of elements

Example 21.13. Fortran irregular subarrays through using vector subscripts.

```

REAL a(100)
CALL MPI_Send(A((/7,9,23,81,82/)), 5, MPI_REAL, ...)

```

Fortran arrays with a vector subscript must not be used as actual choice buffer arguments in any nonblocking or split collective MPI operations. They may, however, be used in blocking MPI operations.

21.1.14 Special Constants

MPI requires a number of special “constants” that cannot be implemented as normal Fortran constants, e.g., `MPI_BOTTOM`. The complete list can be found in Section 2.5.4. In C, these

are implemented as constant pointers, usually as NULL and are used where the function prototype calls for a pointer to a variable, not the variable itself.

In Fortran, using special values for the constants (e.g., by defining them through parameter statements) is not possible because an implementation cannot distinguish these values from valid data. Typically these constants are implemented as predefined static variables (e.g., a variable in an MPI-declared COMMON block), relying on the fact that the target compiler passes data by address. Inside the subroutine, the address of the actual choice buffer argument can be compared with the address of such a predefined static variable.

These special constants also cause an exception with the usage of Fortran INTENT: with USE mpi_f08, the attributes INTENT(IN), INTENT(OUT), and INTENT(INOUT) are used in the Fortran interface. In most cases, INTENT(IN) is used if the C interface uses call-by-value. For all buffer arguments and for dummy arguments that may be modified and allow one of these special constants as input, an INTENT is not specified.

21.1.15 Fortran Derived Types

MPI supports passing Fortran entities of BIND(C) and SEQUENCE derived types to choice dummy arguments, provided no type component has the ALLOCATABLE or POINTER attribute.

The following code fragment shows some possible ways to send scalars or arrays of interoperable derived types in Fortran. Example 21.14 assumes that all data is passed by address.

Example 21.14. Fortran array of derived Fortran types: the struct MPI derived type should be resized.

```

type, BIND(C) :: mytype
  integer :: i
  real :: x
  double precision :: d
  logical :: l
end type mytype

type(mytype) :: foo, fooarr(5)
integer :: blocklen(4), dtype(4)
integer(KIND=MPI_ADDRESS_KIND) :: disp(4), base, lb, extent

call MPI_GET_ADDRESS(foo%i, disp(1), ierr)
call MPI_GET_ADDRESS(foo%x, disp(2), ierr)
call MPI_GET_ADDRESS(foo%d, disp(3), ierr)
call MPI_GET_ADDRESS(foo%l, disp(4), ierr)

base = disp(1)
disp(1) = disp(1) - base
disp(2) = disp(2) - base
disp(3) = disp(3) - base
disp(4) = disp(4) - base

blocklen(1) = 1
blocklen(2) = 1
blocklen(3) = 1
blocklen(4) = 1

```



```

1  dtype(1) = MPI_INTEGER
2  dtype(2) = MPI_REAL
3  dtype(3) = MPI_DOUBLE_PRECISION
4  dtype(4) = MPI_LOGICAL
5
6  call MPI_TYPE_CREATE_STRUCT(4, blocklen, disp, dtype, newtype, ierr)
7  call MPI_TYPE_COMMIT(newtype, ierr)
8
9  call MPI_SEND(foo%i, 1, newtype, dest, tag, comm, ierr)
10 ! or
11 call MPI_SEND(foo, 1, newtype, dest, tag, comm, ierr)
12 ! expects that base == address(foo%i) == address(foo)
13
14 call MPI_GET_ADDRESS(fooarr(1), disp(1), ierr)
15 call MPI_GET_ADDRESS(fooarr(2), disp(2), ierr)
16 extent = disp(2) - disp(1)
17 lb = 0
18 call MPI_TYPE_CREATE_RESIZED(newtype, lb, extent, newarrtype, ierr)
19 call MPI_TYPE_COMMIT(newarrtype, ierr)
20
21 call MPI_SEND(fooarr, 5, newarrtype, dest, tag, comm, ierr)

```

Using the derived type variable `foo` instead of its first basic type element `foo%i` may be impossible if the MPI library implements choice buffer arguments through overloading instead of using `TYPE(*)`, `DIMENSION(..)`, or through a nonstandardized extension such as `!$PRAGMA IGNORE_TKR`; see Section 21.1.6.

To use a derived type in an array requires a correct extent of the datatype handle to take care of the alignment rules applied by the compiler. These alignment rules may imply that there are gaps between the components of a derived type, and also between the subsequent elements of an array of a derived type. The extent of an interoperable derived type (i.e., defined with `BIND(C)`) and a `SEQUENCE` derived type with the same content may be different because C and Fortran may apply different alignment rules. As recommended in the advice to users in Section 5.1.6, one should add an additional fifth structure element with one numerical storage unit at the end of this structure to force in most cases that the array of structures is contiguous. Even with such an additional element, one should keep this resizing due to the special alignment rules that can be used by the compiler for structures, as also mentioned in this advice.

Using the extended semantics defined in TS 29113, it is also possible to use entities or derived types without either the `BIND(C)` or the `SEQUENCE` attribute as choice buffer arguments; some additional constraints must be observed, e.g., no `ALLOCATABLE` or `POINTER` type components may exist. In this case, the `base` address in the example must be changed to become the address of `foo` instead of `foo%i`, because the Fortran compiler may rearrange type components or add padding. Sending the structure `foo` should then also be performed by providing it (and not `foo%i`) as actual argument for `MPI_Send`.

21.1.16 Optimization Problems, an Overview

MPI provides operations that may be hidden from the user code and run concurrently with it, accessing the same memory as user code. Examples include the data transfer for an `MPI_IRECV`. The optimizer of a compiler will assume that it can recognize periods

Table 21.2: Occurrence of Fortran optimization problems in several usage areas

Optimization ...	... may cause a problem in following usage areas			
	Nonbl.	1-sided	Split	Bottom
Code movement and register optimization	yes	yes	no	yes
Temporary data movement	yes	yes	yes	no
Permanent data movement	yes	yes	yes	yes

when a copy of a variable can be kept in a register without reloading from or storing to memory. When the user code is working with a register copy of some variable while the hidden operation reads or writes the memory copy, problems occur. These problems are independent of the Fortran support method; i.e., they occur with the `mpi_f08` module, the `mpi` module, and the (deprecated) `mpif.h` include file.

This section shows four problematic usage areas (the abbreviations in parentheses are used in the table below):

- Use of nonblocking routines or persistent requests (*Nonbl.*).
- Use of one-sided routines (*1-sided*).
- Use of MPI parallel file I/O split collective operations (*Split*).
- Use of `MPI_BOTTOM` together with absolute displacements in MPI datatypes, or relative displacements between two variables in such datatypes (*Bottom*).

The following compiler optimization strategies (valid for serial code) may cause problems in MPI applications:

- Code movement and register optimization problems; see Section 21.1.17.
- Temporary data movement and temporary memory modifications; see Section 21.1.18.
- Permanent data movement (e.g., through garbage collection); see Section 21.1.19.

Table 21.2 shows the only usage areas where these optimization problems may occur.

The solutions in the following sections are based on compromises:

- to minimize the burden for the application programmer, e.g., as shown in Sections Solutions through The (Poorly Performing) Fortran `VOLATILE` Attribute on pages 848–852,
- to minimize the drawbacks on compiler based optimization, and
- to minimize the requirements defined in Section 21.1.7.

21.1.17 Problems with Code Movement and Register Optimization

Nonblocking Operations

If a variable is local to a Fortran subroutine (i.e., not in a module or a COMMON block), the compiler will assume that it cannot be modified by a called subroutine unless it is an actual argument of the call. In the most common linkage convention, the subroutine is expected to save and restore certain registers. Thus, the optimizer will assume that a register that held a valid copy of such a variable before the call will still hold a valid copy on return.

Example 21.15. Fortran 90 register optimization—extreme.

Source	compiled as	or compiled as
REAL :: buf, b1	REAL :: buf, b1	REAL :: buf, b1
call MPI_IRECV (buf,..req)	call MPI_IRECV (buf,..req)	call MPI_IRECV (buf,..req)
	register = buf	b1 = buf
call MPI_WAIT (req,..)	call MPI_WAIT (req,..)	call MPI_WAIT (req,..)
b1 = buf	b1 = register	

Example 21.15 shows extreme, but allowed, possibilities. `MPI_WAIT` on a concurrent thread modifies `buf` between the invocation of `MPI_IRECV` and the completion of `MPI_WAIT`. But the compiler cannot see any possibility that `buf` can be changed after `MPI_IRECV` has returned, and may schedule the load of `buf` earlier than typed in the source. The compiler has no reason to avoid using a register to hold `buf` across the call to `MPI_WAIT`. It also may reorder the instructions as illustrated in the rightmost column.

Example 21.16. Similar example with `MPI_ISEND`

Source	compiled as	with a possible MPI-internal execution sequence
REAL :: buf, copy	REAL :: buf, copy	REAL :: buf, copy
buf = val	buf = val	buf = val
call MPI_ISEND (buf,..req)	call MPI_ISEND (buf,..req)	addr = &buf
copy = buf	copy = buf	copy = buf
	buf = val_overwrite	buf = val_overwrite
call MPI_WAIT (req,..)	call MPI_WAIT (req,..)	call send (*addr) ! within
		! MPI_WAIT
buf = val_overwrite		

Due to valid compiler code movement optimizations in Example 21.16, the content of `buf` may already have been overwritten by the compiler when the content of `buf` is sent. The code movement is permitted because the compiler cannot detect a possible access to `buf` in `MPI_WAIT` (or in a second thread between the start of `MPI_ISEND` and the end of `MPI_WAIT`).

Such register optimization is based on moving code; here, the access to `buf` was moved from after `MPI_WAIT` to before `MPI_WAIT`. Note that code movement may also occur across subroutine boundaries when subroutines or functions are inlined.

This register optimization/code movement problem for nonblocking operations does not occur with MPI parallel file I/O split collective operations, because in the `MPI_XXX_BEGIN` and `MPI_XXX_END` calls, the same buffer has to be provided as an actual argument. The register optimization / code movement problem for `MPI_BOTTOM` and derived MPI datatypes may occur in each blocking and nonblocking communication call, as well as in each parallel file I/O operation.

Persistent Operations

With persistent requests, the buffer argument is hidden from the `MPI_START` and `MPI_STARTALL` calls, i.e., the Fortran compiler may move buffer accesses across the `MPI_START` or `MPI_STARTALL` call, similar to the `MPI_WAIT` call as described in the Nonblocking Operations subsection in Section 21.1.17.

One-sided Communication

An example with instruction reordering due to register optimization can be found in Section 12.7.4.

`MPI_BOTTOM` and Combining Independent Variables in Datatypes

This section is only relevant if the MPI program uses a buffer argument to an `MPI_SEND`, `MPI_RECV`, etc., that hides the actual variables involved in the communication. `MPI_BOTTOM` with an MPI_Datatype containing *absolute addresses* is one example. Creating a datatype that uses one variable as an anchor and brings along others by using `MPI_GET_ADDRESS` to determine their offsets from the anchor is another. The anchor variable would be the only one referenced in the call. Also attention must be paid if MPI operations are used that run in parallel with the user's application.

Example 21.17 shows what Fortran compilers are allowed to do.

Example 21.17. Fortran 90 register optimization.

This source	can be compiled as
<code>call MPI_GET_ADDRESS(buf,bufaddr, ierror)</code>	<code>call MPI_GET_ADDRESS(buf,...)</code>
<code>call MPI_TYPE_CREATE_STRUCT(1,1, bufaddr, MPI_REAL, dtype, ierror)</code>	<code>call MPI_TYPE_CREATE_STRUCT(...)</code>
<code>call MPI_TYPE_COMMIT(dtype, ierror)</code>	<code>call MPI_TYPE_COMMIT(...)</code>
<code>val_old = buf</code>	<code>register = buf</code>
	<code>val_old = register</code>
<code>call MPI_RECV(MPI_BOTTOM,1,dtype,...)</code>	<code>call MPI_RECV(MPI_BOTTOM,...)</code>
<code>val_new = buf</code>	<code>val_new = register</code>

In Example 21.17, the compiler does not invalidate the register because it cannot see that `MPI_RECV` changes the value of `buf`. The access to `buf` is hidden by the use of `MPI_GET_ADDRESS` and `MPI_BOTTOM`.

Example 21.18. Similar example with `MPI_SEND`

This source	can be compiled as
<code>! buf contains val_old</code>	<code>! buf contains val_old</code>
<code>buf = val_new</code>	
<code>call MPI_SEND(MPI_BOTTOM,1,dtype,...)</code>	<code>call MPI_SEND(...)</code>
<code>! with buf as a displacement in dtype</code>	<code>! i.e., val_old is sent</code>
	<code>!</code>
	<code>! buf=val_new is moved to here</code>
	<code>! and detected as dead code</code>
	<code>! and therefore removed</code>
	<code>!</code>

```
1  buf = val_overwrite          buf = val_overwrite
2
```

In Example 21.18, several successive assignments to the same variable `buf` can be combined in a way such that only the last assignment is executed. “Successive” means that no interfering load access to this variable occurs between the assignments. The compiler cannot detect that the call to `MPI_SEND` statement is interfering because the load access to `buf` is hidden by the usage of `MPI_BOTTOM`.

Solutions

The following sections show in detail how the problems with code movement and register optimization can be portably solved. Application writers can partially or fully avoid these compiler optimization problems by using one or more of the special Fortran declarations with the send and receive buffers used in nonblocking operations, or in operations in which `MPI_BOTTOM` is used, or if datatype handles that combine several variables are used:

- Use of the Fortran `ASYNCHRONOUS` attribute.
- Use of the helper routine `MPI_F_SYNC_REG`, or an equivalent user-written dummy routine.
- Declare the buffer as a Fortran module variable or within a Fortran common block.
- Use of the Fortran `VOLATILE` attribute.

Example 21.19. Protecting nonblocking communication with the `ASYNCHRONOUS` attribute.

```
26  USE mpi_f08
27  REAL, ASYNCHRONOUS :: b(0:101) ! elements 0 and 101 are halo cells
28  REAL :: bnew(0:101)           ! elements 1 and 100 are newly computed
29  TYPE(MPI_Request) :: req(4)
30  INTEGER :: left, right, i
31  CALL MPI_Cart_shift(...,left,right,...)
32  CALL MPI_Irecv(b( 0), ..., left, ..., req(1), ...)
33  CALL MPI_Irecv(b(101), ..., right, ..., req(2), ...)
34  CALL MPI_Isend(b( 1), ..., left, ..., req(3), ...)
35  CALL MPI_Isend(b(100), ..., right, ..., req(4), ...)
36
37  #ifdef WITHOUT_OVERLAPPING_COMMUNICATION_AND_COMPUTATION
38  ! Case (a)
39  CALL MPI_Waitall(4, req, ...)
40  DO i=1,100 ! compute all new local data
41    bnew(i) = function(b(i-1), b(i), b(i+1))
42  END DO
43  #endif
44
45  #ifdef WITH_OVERLAPPING_COMMUNICATION_AND_COMPUTATION
46  ! Case (b)
47  DO i=2,99 ! compute only elements for which halo data is not needed
48    bnew(i) = function(b(i-1), b(i), b(i+1))
49  END DO
50  CALL MPI_Waitall(4, req, ...)
```

```

i=1 ! compute leftmost element
  bnew(i) = function(b(i-1), b(i), b(i+1))
i=100 ! compute rightmost element
  bnew(i) = function(b(i-1), b(i), b(i+1))
#endif

```

Each of these methods solves the problems of code movement and register optimization, but may incur various degrees of performance impact, and may not be usable in every application context. These methods may not be guaranteed by the Fortran standard, but they must be guaranteed by a MPI-3.0 (and later) compliant MPI library and associated compiler suite according to the requirements listed in Section 21.1.7. The performance impact of using `MPI_F_SYNC_REG` is expected to be low, that of using module variables or the `ASYNCHRONOUS` attribute is expected to be low to medium, and that of using the `VOLATILE` attribute is expected to be high or very high. Note that there is one attribute that cannot be used for this purpose: the Fortran `TARGET` attribute does not solve code movement problems in MPI applications.

The Fortran ASYNCHRONOUS Attribute

Declaring an actual buffer argument with the `ASYNCHRONOUS` Fortran attribute in a scoping unit (or `BLOCK`) informs the compiler that any statement in the scoping unit may be executed while the buffer is affected by a pending asynchronous Fortran input/output operation (since Fortran 2003) or by an asynchronous communication (TS 29113 extension). Without the extensions specified in TS 29113, a Fortran compiler may totally ignore this attribute if the Fortran compiler implements asynchronous Fortran input/output operations with blocking I/O. The `ASYNCHRONOUS` attribute protects the buffer accesses from optimizations through code movements across routine calls, and the buffer itself from temporary and permanent data movements. If the choice buffer dummy argument of a nonblocking MPI routine is declared with `ASYNCHRONOUS` (which is mandatory for the `mpi_f08` module, with allowable exceptions listed in Section 21.1.6), then the compiler has to guarantee call by reference and should report a compile-time error if call by reference is impossible, e.g., if vector subscripts are used. The `MPI_ASYNC_PROTECTS_NONBLOCKING` is set to `.TRUE.` if both the protection of the actual buffer argument through `ASYNCHRONOUS` according to the TS 29113 extension and the declaration of the dummy argument with `ASYNCHRONOUS` in the Fortran support method is guaranteed for all nonblocking routines, otherwise it is set to `.FALSE.`

The `ASYNCHRONOUS` attribute has some restrictions. Section 5.4.2 of the TS 29113 specifies:

“Asynchronous communication for a Fortran variable occurs through the action of procedures defined by means other than Fortran. It is initiated by execution of an asynchronous communication initiation procedure and completed by execution of an asynchronous communication completion procedure. Between the execution of the initiation and completion procedures, any variable of which any part is associated with any part of the asynchronous communication variable is a **pending communication affector**. Whether a procedure is an asynchronous communication initiation or completion procedure is processor dependent.

Asynchronous communication is either input communication or output com-

munication. For input communication, a *pending communication affector* shall not be referenced, become defined, become undefined, become associated with a dummy argument that has the VALUE attribute, or have its pointer association status changed. For output communication, a *pending communication affector* shall not be redefined, become undefined, or have its pointer association status changed.”

In Example 21.19 Case (a) on page 848, the read accesses to **b** within `function(b(i-1), b(i), b(i+1))` cannot be moved by compiler optimizations to before the wait call because **b** was declared as `ASYNCHRONOUS`. Note that only the elements 0, 1, 100, and 101 of **b** are involved in asynchronous communication but by definition, the total variable **b** is the *pending communication affector* and is usable for input and output asynchronous communication between the `MPI_IXXX` routines and `MPI_Waitall`. Case (a) works fine because the read accesses to **b** occur after the communication has completed.

In Case (b), the read accesses to `b(1:100)` in the loop `i=2,99` are read accesses to a *pending communication affector* while input communication (i.e., the two `MPI_Irecv` calls) is *pending*. This is a contradiction to the rule that *for input communication, a pending communication affector shall not be referenced*. The problem can be solved by using separate variables for the halos and the inner array, or by splitting a common array into disjoint subarrays that are passed through different dummy arguments into a subroutine, as shown in Example 21.24.

If one does not overlap communication and computation on the same variable, then all optimization problems can be solved through the `ASYNCHRONOUS` attribute.

The problems with `MPI_BOTTOM`, as shown in Example 21.17 and Example 21.18, can also be solved by declaring the buffer `buf` with the `ASYNCHRONOUS` attribute.

In some MPI routines, a buffer dummy argument is defined as `ASYNCHRONOUS` to guarantee passing by reference, provided that the actual argument is also defined as `ASYNCHRONOUS`.

Calling `MPI_F_SYNC_REG`

The compiler may be prevented from moving a reference to a buffer across a call to an MPI subroutine by surrounding the call by calls to an external subroutine with the buffer as an actual argument. The MPI library provides the `MPI_F_SYNC_REG` routine for this purpose; see Section 21.1.8.

- The problems illustrated by the Examples 21.15 and 21.16 can be solved by calling `MPI_F_SYNC_REG(buf)` once immediately after `MPI_WAIT`.

Example 21.15 can be solved with

Example 21.16 can be solved with

```
call MPI_Irecv(buf, .. req)

call MPI_WAIT(req, ..)
call MPI_F_SYNC_REG(buf)
b1 = buf
```

```
buf = val
call MPI_ISend(buf, .. req)
copy = buf
call MPI_WAIT(req, ..)
call MPI_F_SYNC_REG(buf)
buf = val_overwrite
```

The call to `MPI_F_SYNC_REG` prevents moving the last line before the `MPI_WAIT` call. Further calls to `MPI_F_SYNC_REG` are not needed because it is still correct if the additional read access `copy=buf` is moved below `MPI_WAIT` and before `buf=val_overwrite`.

- The problems illustrated by the Examples 21.17 and 21.18 can be solved with two additional `MPI_F_SYNC_REG` statements; one directly before `MPI_RECV/MPI_SEND`, and one directly after this communication operation.

Example 21.17 can be solved with

```
call MPI_F_SYNC_REG(buf)
call MPI_RECV(MPI_BOTTOM, ...)
call MPI_F_SYNC_REG(buf)
```

Example 21.18 can be solved with

```
call MPI_F_SYNC_REG(buf)
call MPI_SEND(MPI_BOTTOM, ...)
call MPI_F_SYNC_REG(buf)
```

The first call to `MPI_F_SYNC_REG` is needed to finish all load and store references to `buf` prior to `MPI_RECV/MPI_SEND`; the second call is needed to assure that any subsequent access to `buf` is not moved before `MPI_RECV/MPI_SEND`.

- In the Example 12.14 in Section 12.7.4, two asynchronous accesses must be protected: in Process 1, the access to `bbbb` must be protected similar to Example 21.15, i.e., a call to `MPI_F_SYNC_REG` is needed after the second `MPI_WIN_FENCE` to guarantee that further accesses to `bbbb` are not moved ahead of the call to `MPI_WIN_FENCE`. In Process 2, both calls to `MPI_WIN_FENCE` together act as a communication call with `MPI_BOTTOM` as the buffer. That is, before the first fence and after the second fence, a call to `MPI_F_SYNC_REG` is needed to guarantee that accesses to `buff` are not moved after or ahead of the calls to `MPI_WIN_FENCE`. Using `MPI_GET` instead of `MPI_PUT`, the same calls to `MPI_F_SYNC_REG` are necessary.

Example 21.20. Solution for the Fortran register optimization problems with one-sided communication in Example 12.14.

Source of Process 1

```
bbbb = 777

call MPI_WIN_FENCE
call MPI_PUT(bbbb
into buff of process 2)

call MPI_WIN_FENCE
call MPI_F_SYNC_REG(bbbb)
```

Source of Process 2

```
buff = 999
call MPI_F_SYNC_REG(buff)
call MPI_WIN_FENCE

call MPI_WIN_FENCE
call MPI_F_SYNC_REG(buff)
ccc = buff
```

- The temporary memory modification problem, (see Example 21.21), can **not** be solved with this method.

A User Defined Routine Instead of `MPI_F_SYNC_REG`

Instead of `MPI_F_SYNC_REG`, one can also use a user defined external subroutine, which is separately compiled:

```
subroutine DD(buf)
  integer buf
end
```

Note that if the `INTENT` is declared in an explicit interface for the external subroutine, it must be `OUT` or `INOUT`. The subroutine itself may have an empty body, but the compiler does not know this and has to assume that the buffer may be altered. For example, a call to `MPI_RECV` with `MPI_BOTTOM` as buffer might be replaced by


```

1  call DD(buf)
2  call MPI_RECV(MPI_BOTTOM, ...)
3  call DD(buf)

```

Such a user-defined routine was introduced in MPI-2.0 and is still included here to document such usage in existing application programs although new applications should prefer `MPI_F_SYNC_REG` or one of the other possibilities. In an existing application, calls to such a user-written routine should be substituted by a call to `MPI_F_SYNC_REG` because the user-written routine may not be implemented in accordance with the rules specified in Section 21.1.7.

Module Variables and COMMON Blocks

An alternative to the previously mentioned methods is to put the buffer or variable into a module or a common block and access it through a `USE` or `COMMON` statement in each scope where it is referenced, defined or appears as an actual argument in a call to an MPI routine. The compiler will then have to assume that the MPI procedure may alter the buffer or variable, provided that the compiler cannot infer that the MPI procedure does not reference the module or common block.

- This method solves problems of instruction reordering, code movement, and register optimization related to nonblocking and one-sided communication, or related to the usage of `MPI_BOTTOM` and derived datatype handles.
- Unfortunately, this method does **not** solve problems caused by asynchronous accesses between the start and end of a nonblocking or one-sided communication. Specifically, problems caused by temporary memory modifications are not solved.

The (Poorly Performing) Fortran VOLATILE Attribute

The `VOLATILE` attribute gives the buffer or variable the properties needed to avoid register optimization or code movement problems, but it may inhibit optimization of any code containing references or definitions of the buffer or variable. On many modern systems, the performance impact will be large because not only register, but also cache optimizations will not be applied. Therefore, use of the `VOLATILE` attribute to enforce correct execution of MPI programs is discouraged.

The Fortran TARGET Attribute

The `TARGET` attribute does not solve the code movement problem because it is not specified for the choice buffer dummy arguments of nonblocking routines. If the compiler detects that the application program specifies the `TARGET` attribute for an actual buffer argument used in the call to a nonblocking routine, the compiler may ignore this attribute if no pointer reference to this buffer exists.

Rationale. The Fortran standardization body decided to extend the `ASYNCHRONOUS` attribute within the TS 29113 to protect buffers in nonblocking calls from all kinds of optimization, instead of extending the `TARGET` attribute. (*End of rationale.*)

21.1.18 Temporary Data Movement and Temporary Memory Modification

The compiler is allowed to temporarily modify data in memory. Normally, this problem may occur only when overlapping communication and computation, as in Example 21.19, Case (b) on page 848. Example 21.21 also shows a possibility that could be problematic.

Example 21.21. Overlapping Communication and Computation.

```

USE mpi_f08
REAL :: buf(100,100)
CALL MPI_Irecv(buf(1,1:100),..., req,...)
DO j=1,100
  DO i=2,100
    buf(i,j)=...
  END DO
END DO
CALL MPI_Wait(req,...)

```

Example 21.22. The compiler may substitute the nested loops through loop fusion.

```

REAL :: buf(100,100), buf_1dim(10000)
EQUIVALENCE (buf(1,1), buf_1dim(1))
CALL MPI_Irecv(buf(1,1:100),..., req,...)
tmp(1:100) = buf(1,1:100)
DO j=1,10000
  buf_1dim(h)=...
END DO
buf(1,1:100) = tmp(1:100)
CALL MPI_Wait(req,...)

```

Example 21.23. Another optimization is based on the usage of a separate memory storage area, e.g., in a GPU.

```

REAL :: buf(100,100), local_buf(100,100)
CALL MPI_Irecv(buf(1,1:100),..., req,...)
local_buf = buf
DO j=1,100
  DO i=2,100
    local_buf(i,j)=...
  END DO
END DO
buf = local_buf ! may overwrite asynchronously received
                ! data in buf(1,1:100)
CALL MPI_Wait(req,...)

```

In the compiler-generated, possible optimization in Example 21.22, `buf(100,100)` from Example 21.21 is equivalenced with the 1-dimensional array `buf_1dim(10000)`. The non-blocking receive may asynchronously receive the data in the boundary `buf(1,1:100)` while the fused loop is temporarily using this part of the buffer. When the `tmp` data is written back to `buf`, the previous data of `buf(1,1:100)` is restored and the received data is lost. The principle behind this optimization is that the receive buffer data `buf(1,1:100)` was temporarily moved

1 to tmp.

2 Example 21.23 shows a second possible optimization. The whole array is temporarily
3 moved to local_buf.

4 When storing local_buf back to the original location buf, then this implies overwriting
5 the section of buf that serves as a receive buffer in the nonblocking MPI call, i.e., this
6 storing back of local_buf is therefore likely to interfere with asynchronously received data
7 in buf(1,1:100).

8 Note that this problem may also occur:

- 9
- 10 • With the local buffer at the origin process, between an RMA communication call and
11 the ensuing synchronization call; see Chapter 12.
- 12
- 13 • With the window buffer at the target process between two ensuing RMA synchroniza-
14 tion calls.
- 15 • With the local buffer in MPI parallel file I/O split collective operations between the
16 MPI_XXX_BEGIN and MPI_XXX_END calls; see Section 15.4.5.
- 17

18 As already mentioned in Section The Fortran ASYNCHRONOUS Attribute on page 849 of
19 Section 21.1.17, the ASYNCHRONOUS attribute can prevent compiler optimization with tem-
20 porary data movement, but only if the receive buffer and the local references are separated
21 into different variables, as shown in Example 21.24 and in Example 21.25.

22 Note also that the methods

- 23 • calling MPI_F_SYNC_REG (or such a user-defined routine),
- 24 • using module variables and COMMON blocks, and
- 25 • the TARGET attribute
- 26
- 27

28 cannot be used to prevent such temporary data movement. These methods influence com-
29 piler optimization when library routines are called. They cannot prevent the optimizations
30 of the code fragments shown in Examples 21.21 and 21.22.

31 Note also that compiler optimization with temporary data movement should **not** be
32 prevented by declaring buf as VOLATILE because the VOLATILE implies that all accesses to
33 any storage unit (word) of buf must be directly done in the main memory exactly in the
34 sequence defined by the application program. The VOLATILE attribute prevents all register
35 and cache optimizations. Therefore, VOLATILE may cause a huge performance degradation.

36 Instead of solving the problem, it is better to **prevent** the problem: when overlapping
37 communication and computation, the nonblocking communication (or nonblocking or split
38 collective I/O) and the computation should be executed **on different variables**, and the
39 communication should be *protected* with the ASYNCHRONOUS attribute. In this case, the
40 temporary memory modifications are done only on the variables used in the computation
41 and cannot have any side effect on the data used in the nonblocking MPI operations.
42

43 *Rationale.* This is a strong restriction for application programs. To weaken this re-
44 striction, a new or modified asynchronous feature in the Fortran language would be
45 necessary: an asynchronous attribute that can be used on parts of an array and to-
46 gether with asynchronous operations outside the scope of Fortran. If such a feature
47 becomes available in a future edition of the Fortran standard, then this restriction also
48 may be weakened in a later version of the MPI standard. (*End of rationale.*)

In Example 21.24 (which is a solution for the problem shown in Example 21.19 and in Example 21.25 (which is a solution for the problem shown in Example 21.23), the array is split into inner and halo part and both disjoint parts are passed to a subroutine `separated_sections`. This routine overlaps the receiving of the halo data and the calculations on the inner part of the array. In a second step, the whole array is used to do the calculation on the elements where inner+halo is needed. Note that the halo and the inner area are strided arrays. Those can be used in nonblocking communication only with a Fortran 2018 (or TS 29113) based MPI library.

21.1.19 Permanent Data Movement

A Fortran compiler may implement permanent data movement during the execution of a Fortran program. This would require that pointers to such data are appropriately updated. An implementation with automatic garbage collection is one use case. Such permanent data movement is in conflict with MPI in several areas:

- MPI datatype handles with absolute addresses in combination with `MPI_BOTTOM`.
- All nonblocking MPI operations if the internally used pointers to the buffers are not updated by the Fortran runtime, or if within an MPI process, the data movement is executed in parallel with the MPI operation.

This problem can be also solved by using the `ASYNCHRONOUS` attribute for such buffers. This MPI standard requires that the problems with permanent data movement do not occur by imposing suitable restrictions on the MPI library together with the compiler used; see Section 21.1.7.

Example 21.24. Using separated variables for overlapping communication and computation to allow the protection of nonblocking communication with the `ASYNCHRONOUS` attribute.

```

USE mpi_f08
REAL :: b(0:101)      ! elements 0 and 101 are halo cells
REAL :: bnew(0:101)   ! elements 1 and 100 are newly computed
INTEGER :: i
CALL separated_sections(b(0), b(1:100), b(101), bnew(0:101))
i=1 ! compute leftmost element
  bnew(i) = function(b(i-1), b(i), b(i+1))
i=100 ! compute rightmost element
  bnew(i) = function(b(i-1), b(i), b(i+1))
END

SUBROUTINE separated_sections(b_lefthalo, b_inner, b_righthalo, bnew)
USE mpi_f08
REAL, ASYNCHRONOUS :: b_lefthalo(0:0), b_inner(1:100), b_righthalo(101:101)
REAL :: bnew(0:101) ! elements 1 and 100 are newly computed
TYPE(MPI_Request) :: req(4)
INTEGER :: left, right, i
CALL MPI_Cart_shift(..., left, right, ...)
CALL MPI_Irecv(b_lefthalo( 0), ..., left, ..., req(1), ...)
CALL MPI_Irecv(b_righthalo(101), ..., right, ..., req(2), ...)
! b_lefthalo and b_righthalo is written asynchronously.
! There is no other concurrent access to b_lefthalo and b_righthalo.
CALL MPI_Isend(b_inner( 1), ..., left, ..., req(3), ...)
CALL MPI_Isend(b_inner(100), ..., right, ..., req(4), ...)

```

```

1
2 DO i=2,99 ! compute only elements for which halo data is not needed
3   bnew(i) = function(b_inner(i-1), b_inner(i), b_inner(i+1))
4   ! b_inner is read and sent at the same time.
5   ! This is allowed based on the rules for ASYNCHRONOUS.
6 END DO
7 CALL MPI_Waitall(4, req,...)
8 END SUBROUTINE

```

21.1.20 Comparison with C

In C, subroutines that modify variables that are not in the argument list will not cause register optimization problems. This is because taking pointers to storage objects by using the `&` operator and later referencing the objects by indirection on the pointer is an integral part of the language. A C compiler understands the implications, so that the problem should not occur, in general. However, some compilers do offer optional aggressive optimization levels that may not be safe. Problems due to temporary memory modifications can also occur in C. As above, the best advice is to avoid the problem: use different variables for buffers in nonblocking MPI operations and computation that is executed while a nonblocking communication operation is *pending*.

Example 21.25. Protecting GPU optimizations with the ASYNCHRONOUS attribute.

```

22 USE mpi_f08
23 REAL :: buf(100,100)
24 CALL separated_sections(buf(1:1,1:100), buf(2:100,1:100))
25 END
26
27 SUBROUTINE separated_sections(buf_halo, buf_inner)
28 REAL, ASYNCHRONOUS :: buf_halo(1:1,1:100)
29 REAL :: buf_inner(2:100,1:100)
30 REAL :: local_buf(2:100,100)
31
32 CALL MPI_Irecv(buf_halo(1,1:100),..., req,...)
33 local_buf = buf_inner
34 DO j=1,100
35   DO i=2,100
36     local_buf(i,j)=...
37   END DO
38 END DO
39 buf_inner = local_buf ! buf_halo is not touched!!!
40 CALL MPI_Wait(req,...)

```

21.2 Support for Large Count and Large Byte Displacement in MPI Language Bindings

The following types, which were used prior to MPI-4.0, have been deemed too small to hold values that some applications wish to use:

- The C `int` type and the Fortran `INTEGER` type were used for *count* parameters.

- The C `int` type and the Fortran `INTEGER` type were used for some parameters that represent *byte displacement* in memory.
- The C `MPI_Aint` type and the Fortran `INTEGER(KIND=MPI_ADDRESS_KIND)` type were used for some parameters that represent *byte displacement* in files (e.g., in constructors of MPI datatypes that can be used with files).

In order to avoid breaking backwards compatibility, MPI-4.0 and later support larger types via separate additional MPI procedures in C (suffixed with “_c”) and via interface polymorphism in Fortran when using `USE mpi_f08`. For better readability, all Fortran large count procedure declarations are marked with a comment “!(c)”. No polymorphic support for larger types is provided in Fortran when using `mpif.h` and `use mpi`.

For the large count versions of three datatype constructors, `MPI_TYPE_CREATE_HINDEXED`, `MPI_TYPE_CREATE_HINDEXED_BLOCK`, and `MPI_TYPE_CREATE_STRUCT`, absolute addresses shall not be used to specify byte displacements since the parameter is of type `MPI_COUNT` instead of type `MPI_AINT` (see Section 2.5.8).

In addition, the functions `MPI_TYPE_GET_ENVELOPE` and `MPI_TYPE_GET_CONTENTS` also support large count types via *additional parameters* in separate additional MPI procedures in C (suffixed with “_c”) and interface polymorphism in Fortran when using `USE mpi_f08` (see Section 5.1.13).

Further, the callbacks of type `MPI_User_function` and `MPI_Datarep_conversion_function` also support large count types via separate additional callback prototypes in C (suffixed with “_c”) and multiple abstract interfaces in Fortran when using `USE mpi_f08` (see Sections 6.9.5 and 15.5.3, respectively). An additional large count predefined callback function `MPI_CONVERSION_FN_NULL_C` is provided within each of these two language bindings.

In C bindings, for each MPI procedure that had at least one *count* or *byte displacement* parameter that used the `int` and/or `MPI_Aint` types prior to MPI-4.0, an additional MPI procedure is provided, with the same name but suffixed by “_c”. The MPI procedure without the “_c” token has the same name and parameter types as versions prior to MPI-4.0. The “_c” suffixed MPI procedure has `MPI_Count` for all *count* parameters, `MPI_Aint` for parameters that represent *byte displacement* in memory, `MPI_Offset` for parameters that represent *byte displacement* in files, and `MPI_Count` for parameters that may represent *byte displacement* in both memory and files.

In Fortran, when using `USE mpi_f08`, for each MPI procedure that had at least one *count* or *byte displacement* parameter that used the `INTEGER` or `INTEGER(KIND=MPI_ADDRESS_KIND)` types prior to MPI-4.0, a polymorphic interface containing two specific procedures is provided. One of the specific procedures has the same name and dummy parameter types as in versions prior to MPI-4.0. `INTEGER` and/or `INTEGER(KIND=MPI_ADDRESS_KIND)` for *count* and *byte displacement* parameters. The other specific procedure has the same name followed by “_c”, and then suffixed by the token specified in Table 21.1 for `USE mpi_f08`. It also has `INTEGER(KIND=MPI_COUNT_KIND)` for all *count* parameters, `INTEGER(KIND=MPI_ADDRESS_KIND)` for parameters that represent *byte displacement* in memory, `INTEGER(KIND=MPI_OFFSET_KIND)` for parameters that represent *byte displacement* in files, and `INTEGER(KIND=MPI_COUNT_KIND)` for parameters that may represent *byte displacement* in both memory and files (for more details on specific Fortran procedure names and related calling conventions, refer to Table 21.1 in Section 21.1.5). There is one exception: if the type signatures of the two specific procedures are identical (e.g., if

INTEGER(KIND=MPI_COUNT_KIND) is the same type as INTEGER(KIND=MPI_ADDRESS_KIND)), then the implementation shall not provide the “_c” specific procedure.

It is erroneous to directly invoke the “_c” specific procedures in the Fortran `mpi_f08` module with the exception of the following procedures: `MPI_Op_create_c` and `MPI_Register_datarep_c`.

In older Fortran bindings (`mpif.h` (deprecated) and `use mpi`), no new interfaces and no new specific procedures for larger types are provided beyond what existed in MPI-3.1; all MPI procedures have the same types as in the versions prior to MPI-4.0.

21.3 Language Interoperability

21.3.1 Introduction

It is not uncommon for library developers to use one language to develop an application library that may be called by an application program written in a different language. MPI currently supports ISO (previously ANSI) C and Fortran bindings. It should be possible for applications in any of the supported languages to call MPI-related functions in another language.

Moreover, MPI allows the development of client-server code, with MPI communication used between a parallel client and a parallel server. It should be possible to code the server in one language and the clients in another language. To do so, communications should be possible between applications written in different languages.

There are several issues that need to be addressed in order to achieve interoperability.

Initialization: We need to specify how the MPI environment is initialized for all languages.

Interlanguage passing of MPI opaque objects: We need to specify how MPI object handles are passed between languages. We also need to specify what happens when an MPI object is accessed in one language, to retrieve information (e.g., attributes) set in another language.

Interlanguage communication: We need to specify how messages sent in one language can be received in another language.

It is highly desirable that the solution for interlanguage interoperability be extensible to new languages, should MPI bindings be defined for such languages.

21.3.2 Assumptions

We assume that conventions exist for programs written in one language to call routines written in another language. These conventions specify how to link routines in different languages into one program, how to call functions in a different language, how to pass arguments between languages, and the correspondence between basic datatypes in different languages. In general, these conventions will be implementation dependent. Furthermore, not every basic datatype may have a matching type in other languages. For example, C character strings may not be compatible with Fortran CHARACTER variables. However, we assume that a Fortran INTEGER, as well as a (sequence associated) Fortran array of INTEGERS, can be passed to a C program. We also assume that Fortran and C have address-sized integers. This does not mean that the default-size integers are the same size as default-sized pointers, but only that there is some way to hold (and pass) a C address in

a Fortran integer. It is also assumed that `INTEGER(KIND=MPI_OFFSET_KIND)` can be passed from Fortran to C as `MPI_Offset`.

21.3.3 Initialization

Two approaches are available for initializing MPI: the World Model (Section 11.2), and the Sessions Model (Section 11.3).

Concerns specific to the World Model

A call to `MPI_INIT` or `MPI_INIT_THREAD`, from any language, initializes MPI for execution in all languages.

Advice to users. Certain implementations use the (inout) `argc`, `argv` arguments of the C version of `MPI_INIT` in order to propagate values for `argc` and `argv` to all executing MPI processes. Use of the Fortran version of `MPI_INIT` to initialize MPI may result in a loss of this ability. (*End of advice to users.*)

The function `MPI_INITIALIZED` returns the same answer in all languages.

The function `MPI_FINALIZE` finalizes the MPI environments for all languages.

The function `MPI_FINALIZED` returns the same answer in all languages.

The MPI environment is initialized in the same manner for all languages by `MPI_INIT`. E.g., `MPI_COMM_WORLD` carries the same information regardless of language: same MPI processes, same environmental attributes, same error handlers.

Advice to users. The use of several languages in one MPI program may require the use of special options at compile and/or link time. (*End of advice to users.*)

Advice to implementors. Implementations may selectively link language specific MPI libraries only to codes that need them, so as not to increase the size of binaries for codes that use only one language. The MPI initialization code needs to perform initialization for a language only if that language library is loaded. (*End of advice to implementors.*)

Concerns specific to the Sessions Model

A call to `MPI_SESSION_INIT` from any language initializes a session that can be used from all languages.

A call to `MPI_SESSION_FINALIZE` from any language finalizes the session for all languages.

Concerns common to both the World Model and the Sessions Model

The function `MPI_ABORT` kills MPI processes in the group of the supplied communicator, irrespective of the language used by the caller or by the MPI processes killed.

Information can be added to info objects in one language and retrieved in another.

21.3.4 Transfer of Handles

Handles are passed between Fortran and C by using an explicit C wrapper to convert Fortran handles to C handles. There is no direct access to C handles in Fortran.

The type definition `MPI_Fint` is provided in C for an integer of the size that matches a Fortran `INTEGER`; usually, `MPI_Fint` will be equivalent to `int`. With the Fortran `mpi` module or the (deprecated) `mpif.h` include file, a Fortran handle is a Fortran `INTEGER` value that can be used in the following conversion functions. With the Fortran `mpi_f08` module, a Fortran handle is a `BIND(C)` derived type that contains an `INTEGER` component named `MPI_VAL`. This `INTEGER` value can be used in the following conversion functions.

The following functions are provided in C to convert from a Fortran communicator handle (which is an integer) to a C communicator handle, and vice versa.

C binding

```
MPI_Comm MPI_Comm_f2c(MPI_Fint comm)
```

If `comm` is a valid Fortran handle to a communicator, then `MPI_Comm_f2c` returns a valid C handle to that same communicator; if `comm = MPI_COMM_NULL` (Fortran value), then `MPI_Comm_f2c` returns a null C handle; if `comm` is an invalid Fortran handle, then `MPI_Comm_f2c` returns an invalid C handle.

```
MPI_Fint MPI_Comm_c2f(MPI_Comm comm)
```

The function `MPI_Comm_c2f` translates a C communicator handle into a Fortran handle to the same communicator; it maps a null handle into a null handle and an invalid handle into an invalid handle.

Similar functions are provided for the other types of opaque objects.

```
MPI_Datatype MPI_Type_f2c(MPI_Fint datatype)
```

```
MPI_Fint MPI_Type_c2f(MPI_Datatype datatype)
```

```
MPI_Group MPI_Group_f2c(MPI_Fint group)
```

```
MPI_Fint MPI_Group_c2f(MPI_Group group)
```

```
MPI_Request MPI_Request_f2c(MPI_Fint request)
```

```
MPI_Fint MPI_Request_c2f(MPI_Request request)
```

```
MPI_File MPI_File_f2c(MPI_Fint file)
```

```
MPI_Fint MPI_File_c2f(MPI_File file)
```

```
MPI_Win MPI_Win_f2c(MPI_Fint win)
```

```
MPI_Fint MPI_Win_c2f(MPI_Win win)
```

```
MPI_Op MPI_Op_f2c(MPI_Fint op)
```

```
MPI_Fint MPI_Op_c2f(MPI_Op op)
```

```
MPI_Info MPI_Info_f2c(MPI_Fint info)
```

```
MPI_Fint MPI_Info_c2f(MPI_Info info)
```

```
MPI_Errhandler MPI_Errhandler_f2c(MPI_Fint errhandler)
```

```
MPI_Fint MPI_Errhandler_c2f(MPI_Errhandler errhandler)
```

```
MPI_Message MPI_Message_f2c(MPI_Fint message)
```



```

MPI_Fint MPI_Message_c2f(MPI_Message message)
MPI_Session MPI_Session_f2c(MPI_Fint session)
MPI_Fint MPI_Session_c2f(MPI_Session session)

```

Example 21.26. The example below illustrates how the Fortran MPI function `MPI_TYPE_COMMIT` can be implemented by wrapping the C MPI function `MPI_Type_commit` with a C wrapper to do handle conversions. In this example a Fortran-C interface is assumed where a Fortran function is all upper case when referred to from C and arguments are passed by addresses.

```

! FORTRAN PROCEDURE
SUBROUTINE MPI_TYPE_COMMIT(DATATYPE, IERR)
INTEGER :: DATATYPE, IERR
CALL MPI_X_TYPE_COMMIT(DATATYPE, IERR)
RETURN
END

/* C wrapper */

void MPI_X_TYPE_COMMIT(MPI_Fint *f_handle, MPI_Fint *ierr)
{
    MPI_Datatype datatype;

    datatype = MPI_Type_f2c(*f_handle);
    *ierr = (MPI_Fint)MPI_Type_commit(&datatype);
    *f_handle = MPI_Type_c2f(datatype);
    return;
}

```

The same approach can be used for all other MPI functions. The call to `MPI_XXX_f2c` (resp. `MPI_XXX_c2f`) can be omitted when the handle is an OUT (resp. IN) argument, rather than INOUT.

Rationale. The design here provides a convenient solution for the prevalent case, where a C wrapper is used to allow Fortran code to call a C library, or C code to call a Fortran library. The use of C wrappers is much more likely than the use of Fortran wrappers, because it is much more likely that a variable of type `INTEGER` can be passed to C, than a C handle can be passed to Fortran.

Returning the converted value as a function value rather than through the argument list allows the generation of efficient inlined code when these functions are simple (e.g., the identity). The conversion function in the wrapper does not catch an invalid handle argument. Instead, an invalid handle is passed below to the library function, which, presumably, checks its input arguments. (*End of rationale.*)

21.3.5 Status

The following two procedures are provided in C to convert from a Fortran (with the `mpi` module or deprecated `mpif.h`) status (which is an array of integers) to a C status (which is a structure), and vice versa. The conversion occurs on all the information in status, including that which is hidden. That is, no status information is lost in the conversion.

```
1 int MPI_Status_f2c(const MPI_Fint *f_status, MPI_Status *c_status)
```

2 If `f_status` is a valid Fortran status, but not the Fortran value of
 3 `MPI_STATUS_IGNORE` or `MPI_STATUSES_IGNORE`, then `MPI_Status_f2c` returns in `c_status`
 4 a valid C status with the same content. If `f_status` is the Fortran value of
 5 `MPI_STATUS_IGNORE` or `MPI_STATUSES_IGNORE`, or if `f_status` is not a valid Fortran status,
 6 then the call is erroneous.

7 In C, such an `f_status` array can be defined with `MPI_Fint f_status[`
 8 `MPI_F_STATUS_SIZE]`. Within this array, one can use in C the indexes `MPI_F_SOURCE`,
 9 `MPI_F_TAG`, and `MPI_F_ERROR`, to access the same elements as in Fortran with
 10 `MPI_SOURCE`, `MPI_TAG` and `MPI_ERROR`. The C indexes are 1 less than the corresponding
 11 indexes in Fortran due to the different default array start indexes in both languages.

12 The C status has the same source, tag and error code values as the Fortran status,
 13 and returns the same answers when queried for count, elements, and cancellation. The
 14 conversion function may be called with a Fortran status argument that has an undefined
 15 error field, in which case the value of the error field in the C status argument is undefined.

16 Two global variables of type `MPI_Fint*`, `MPI_F_STATUS_IGNORE` and
 17 `MPI_F_STATUSES_IGNORE`, are declared in `mpi.h`. They can be used to test, in C, whether
 18 `f_status` is the Fortran value of `MPI_STATUS_IGNORE` or `MPI_STATUSES_IGNORE` defined
 19 in the `mpi` module or (deprecated) `mpif.h`. These are global variables, not C constant
 20 expressions and cannot be used in places where C requires constant expressions. Their
 21 value is defined only between the calls to `MPI_INIT` and `MPI_FINALIZE` and should not be
 22 changed by user code.

23 To do the conversion in the other direction, we have the following:

```
24 int MPI_Status_c2f(const MPI_Status *c_status, MPI_Fint *f_status)
```

25 This call converts a C status into a Fortran status, and has a behavior similar to
 26 `MPI_Status_f2c`. That is, the value of `c_status` must not be either `MPI_STATUS_IGNORE`
 27 or `MPI_STATUSES_IGNORE`.

30 *Advice to users.* There exists no separate conversion function for arrays of statuses,
 31 since one can simply loop through the array, converting each status with the routines
 32 in Figure 21.1. (*End of advice to users.*)

34 *Rationale.* The handling of `MPI_STATUS_IGNORE` is required in order to layer libraries
 35 with only a C wrapper: if the Fortran call has passed `MPI_STATUS_IGNORE`, then the
 36 C wrapper must handle this correctly. Note that this constant need not have the same
 37 value in Fortran and C. If `MPI_Status_f2c` were to handle `MPI_STATUS_IGNORE`, then
 38 the type of its result would have to be `MPI_Status**`, which was considered an inferior
 39 solution. (*End of rationale.*)

40 Using the `mpi_f08` Fortran module, a status is declared as `TYPE(MPI_Status)`. The C
 41 type `MPI_F08_status` can be used to pass a Fortran `TYPE(MPI_Status)` argument into a C
 42 routine. Figure 21.1 illustrates all status conversion routines. Some are only available in
 43 C, some in both C and the Fortran `mpi` and `mpi_f08` interfaces (but not in the deprecated
 44 `mpif.h` include file).

```
45 int MPI_Status_f082c(const MPI_F08_status *f08_status, MPI_Status *c_status)
```

46 This C routine converts a Fortran `mpi_f08 TYPE(MPI_Status)` into a C `MPI_Status`.

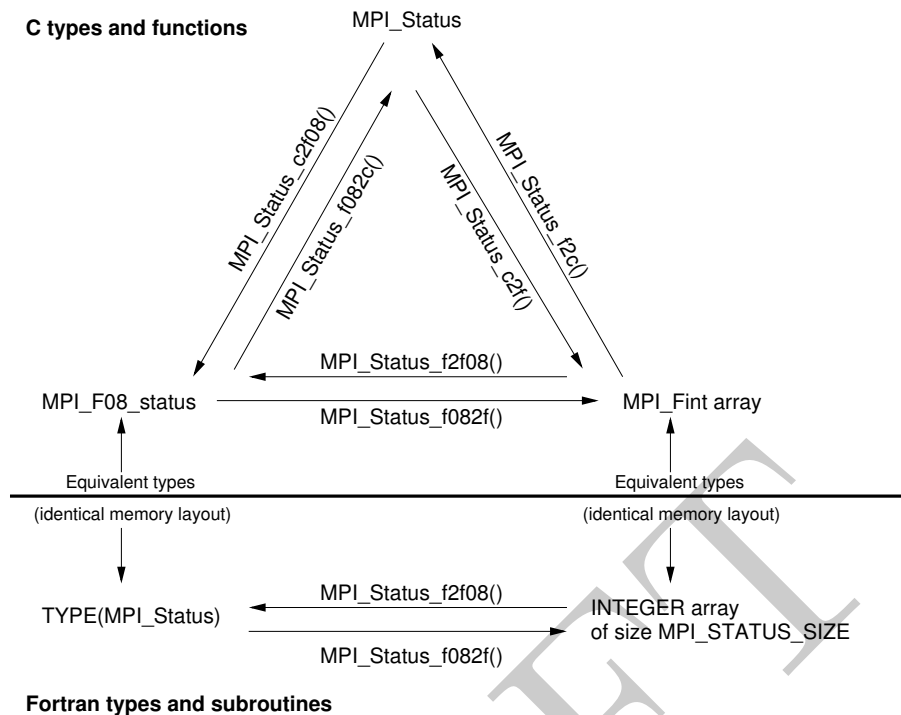


Figure 21.1: Status conversion routines

```
int MPI_Status_c2f08(const MPI_Status *c_status, MPI_F08_status *f08_status)
```

This C routine converts a C MPI_Status into a Fortran mpi_f08 TYPE(MPI_Status). Two global variables of type MPI_F08_status*, MPI_F08_STATUS_IGNORE and MPI_F08_STATUSES_IGNORE are declared in mpi.h. They can be used to test, in C, whether f_status is the Fortran value of MPI_STATUS_IGNORE or MPI_STATUSES_IGNORE defined in the mpi_f08 module. These are global variables, not C constant expressions and cannot be used in places where C requires constant expressions. Their value is defined only between the calls to MPI_INIT and MPI_FINALIZE and should not be changed by user code.

Conversion between the two Fortran versions of a status can be done with:

```
MPI_STATUS_F2F08(f_status, f08_status)
```

IN	f_status	status object declared as array (status)
OUT	f08_status	status object declared as named type (status)

C binding

```
int MPI_Status_f2f08(const MPI_Fint *f_status, MPI_F08_status *f08_status)
```

Fortran 2008 binding

```
MPI_Status_f2f08(f_status, f08_status, ierror)
  INTEGER, INTENT(IN) :: f_status(MPI_STATUS_SIZE)
  TYPE(MPI_Status), INTENT(OUT) :: f08_status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding (the following procedure is not available with mpif.h)

```
MPI_STATUS_F2F08(F_STATUS, F08_STATUS, IERROR)
  INTEGER :: F_STATUS(MPI_STATUS_SIZE), IERROR
  TYPE(MPI_Status) :: F08_STATUS
```

This routine converts a Fortran INTEGER, DIMENSION(MPI_STATUS_SIZE) status array into a Fortran mpi_f08 TYPE(MPI_Status).

```
MPI_STATUS_F082F(f08_status, f_status)
```

```
IN      f08_status      status object declared as named type (status)
OUT     f_status        status object declared as array (status)
```

C binding

```
int MPI_Status_f082f(const MPI_F08_status *f08_status, MPI_Fint *f_status)
```

Fortran 2008 binding

```
MPI_Status_f082f(f08_status, f_status, ierror)
  TYPE(MPI_Status), INTENT(IN) :: f08_status
  INTEGER, INTENT(OUT) :: f_status(MPI_STATUS_SIZE)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding (the following procedure is not available with mpif.h)

```
MPI_STATUS_F082F(F08_STATUS, F_STATUS, IERROR)
  TYPE(MPI_Status) :: F08_STATUS
  INTEGER :: F_STATUS(MPI_STATUS_SIZE), IERROR
```

This routine converts a Fortran mpi_f08 TYPE(MPI_Status) into a Fortran INTEGER, DIMENSION(MPI_STATUS_SIZE) status array.

21.3.6 MPI Opaque Objects

Unless said otherwise, opaque objects are “the same” in all languages: they carry the same information, and have the same meaning in both languages. The mechanism described in the previous section can be used to pass references to MPI objects from language to language. An object created in one language can be accessed, modified or freed in another language.

We examine below in more detail issues that arise for each type of MPI object.

Datatypes

Datatypes encode the same information in all languages. E.g., a datatype accessor like `MPI_TYPE_GET_EXTENT` will return the same information in all languages. If a datatype defined in one language is used for a communication call in another language, then the message sent will be identical to the message that would be sent from the first language: the same communication buffer is accessed, and the same representation conversion is performed, if needed. All predefined datatypes can be used in datatype constructors in any language. If a datatype is committed, it can be used for communication in any language.

The function `MPI_GET_ADDRESS` returns the same value in all languages. Note that we do not require that the constant `MPI_BOTTOM` have the same value in all languages (see Section 21.3.9).

Example 21.27. Absolute addresses and the conversion of datatype handles in a mixed Fortran/C program.

```

1  ! FORTRAN CODE
2
3
4  REAL :: R(5)
5
6  INTEGER :: DTYPE, IERR, AOBLN(1), AOTYPE(1)
7
8  INTEGER(KIND=MPI_ADDRESS_KIND) :: AODISP(1)
9
10 ! create an absolute datatype for array R
11 AOBLN(1) = 5
12 CALL MPI_GET_ADDRESS(R, AODISP(1), IERR)
13 AOTYPE(1) = MPI_REAL
14 CALL MPI_TYPE_CREATE_STRUCT(1, AOBLN, AODISP, AOTYPE, DTYPE, IERR)
15 CALL C_ROUTINE(DTYPE)
16
17 /* C code */
18
19 void C_ROUTINE(MPI_Fint *ftype)
20 {
21     int count = 5;
22     int lens[2] = {1,1};
23     MPI_Aint displs[2];
24     MPI_Datatype types[2], newtype;
25
26     /* create an absolute datatype for buffer that consists */
27     /* of count, followed by R(5) */
28
29     MPI_Get_address(&count, &displs[0]);
30     displs[1] = 0;
31     types[0] = MPI_INT;
32     types[1] = MPI_Type_f2c(*ftype);
33     MPI_Type_create_struct(2, lens, displs, types, &newtype);
34     MPI_Type_commit(&newtype);
35
36     MPI_Send(MPI_BOTTOM, 1, newtype, 1, 0, MPI_COMM_WORLD);
37     /* the message sent contains an int count of 5, followed */
38     /* by the 5 REAL entries of the Fortran array R. */
39 }

```

Advice to implementors. The following implementation can be used: MPI addresses, as returned by `MPI_GET_ADDRESS`, will have the same value in all languages. One obvious choice is that MPI addresses be identical to regular addresses. The address is stored in the datatype, when datatypes with absolute addresses are constructed. When a send or receive operation is performed, then addresses stored in a datatype are interpreted as displacements that are all augmented by a base address. This base address is (the address of) `buf`, or zero, if `buf = MPI_BOTTOM`. Thus, if `MPI_BOTTOM` is zero then a send or receive call with `buf = MPI_BOTTOM` is implemented exactly as a call with a regular buffer argument: in both cases the base address is `buf`. On the other hand, if `MPI_BOTTOM` is not zero, then the implementation has to be slightly different. A test is performed to check whether `buf = MPI_BOTTOM`. If true, then the base address is zero, otherwise it is `buf`. In particular, if `MPI_BOTTOM` does not have the same value in Fortran and C, then an additional test for `buf = MPI_BOTTOM` is

needed in at least one of the languages.

It may be desirable to use a value other than zero for `MPI_BOTTOM` even in C, so as to distinguish it from a NULL pointer. If `MPI_BOTTOM = c` then one can still avoid the test `buf = MPI_BOTTOM`, by using the displacement from `MPI_BOTTOM`, i.e., the regular address $-c$, as the MPI address returned by `MPI_GET_ADDRESS` and stored in absolute datatypes. (*End of advice to implementors.*)

Callback Functions

MPI calls may associate callback functions with MPI objects: error handlers are associated with communicators, files, windows, and sessions; attribute copy and delete functions are associated with attribute keys; reduce operations are associated with operation objects, etc. In a multilanguage environment, a function passed in an MPI call in one language may be invoked by an MPI call in another language. MPI implementations must make sure that such invocation will use the calling convention of the language the function is bound to.

Advice to implementors. Callback functions need to have a language tag. This tag is set when the callback function is passed in by the library function (which is presumably different for each language and language support method), and is used to generate the right calling sequence when the callback function is invoked. (*End of advice to implementors.*)

Advice to users. If a subroutine written in one language or Fortran support method wants to pass a callback routine including the predefined Fortran functions (e.g., `MPI_COMM_NULL_COPY_FN`) to another application routine written in another language or Fortran support method, then it must be guaranteed that both routines use the callback interface definition that is defined for the argument when passing the callback to an MPI routine (e.g., `MPI_COMM_CREATE_KEYVAL`); see also the advice to users on page 363. (*End of advice to users.*)

Error Handlers

Advice to implementors. Error handlers, have, in C, a variable length argument list. It might be useful to provide to the handler information on the language environment where the error occurred. (*End of advice to implementors.*)

Reduce Operations

All predefined named and unnamed datatypes as listed in Section 6.9.2 can be used in the listed predefined operations independent of the programming language from which the MPI routine is called.

Advice to users. Reduce operations receive as one of their arguments the datatype of the operands. Thus, one can define “polymorphic” reduce operations that work for C and Fortran datatypes. (*End of advice to users.*)

21.3.7 Attributes

Attribute keys can be allocated in one language and freed in another. Similarly, attribute values can be set in one language and accessed in another. To achieve this, attribute keys will be allocated in an integer range that is valid all languages. The same holds true for system-defined attribute values (such as `MPI_TAG_UB`, `MPI_WTIME_IS_GLOBAL`, etc.).

Attribute keys declared in one language are associated with copy and delete functions in that language (the functions provided by the `MPI_XXX_CREATE_KEYVAL` call). When a communicator is duplicated, for each attribute, the corresponding copy function is called, using the right calling convention for the language of that function; and similarly, for the delete callback function.

Advice to implementors. This requires that attributes be tagged either as “C” or “Fortran” and that the language tag be checked in order to use the right calling convention for the callback function. (*End of advice to implementors.*)

The attribute manipulation functions described in Section 7.7 defines attributes arguments to be of type `void*` in C, and of type `INTEGER`, in Fortran. On some systems, `INTEGER`s will have 32 bits, while C pointers will have 64 bits. This is a problem if communicator attributes are used to move information from a Fortran caller to a C callee, or vice-versa.

MPI behaves as if it stores, internally, address sized attributes. If Fortran `INTEGER`s are smaller, then the (deprecated) Fortran function `MPI_ATTR_GET` will return the least significant part of the attribute word; the (deprecated) Fortran function `MPI_ATTR_PUT` will set the least significant part of the attribute word, which will be sign extended to the entire word. (These two functions may be invoked explicitly by user code, or implicitly, by attribute copying callback functions.)

As for addresses, new functions are provided that manipulate Fortran address sized attributes, and have the same functionality as the old functions in C. These functions are described in Section 7.7. Users are encouraged to use these new functions.

MPI supports two types of attributes: address-valued (pointer) attributes, and integer-valued attributes. C attribute functions put and get address-valued attributes. Fortran attribute functions put and get integer-valued attributes. When an integer-valued attribute is accessed from C, then `MPI_XXX_get_attr` will return the address of (a pointer to) the integer-valued attribute, which is a pointer to `MPI_Aint` if the attribute was stored with Fortran `MPI_XXX_SET_ATTR`, and a pointer to `int` if it was stored with the deprecated Fortran `MPI_ATTR_PUT`. When an address-valued attribute is accessed from Fortran, then `MPI_XXX_GET_ATTR` will convert the address into an integer and return the result of this conversion. This conversion is lossless if new style attribute functions are used, and an integer of kind `MPI_ADDRESS_KIND` is returned. The conversion may cause truncation if deprecated attribute functions are used. In C, the deprecated routines `MPI_Attr_put` and `MPI_Attr_get` behave identical to `MPI_Comm_set_attr` and `MPI_Comm_get_attr`.

Example 21.28. Setting an attribute in C and reading in C or Fortran.

A. Setting an attribute value in C

```
int set_val = 3;
struct foo set_struct;

/* Set a value that is a pointer to an int */
```



```

1 MPI_Comm_set_attr(MPI_COMM_WORLD, keyval1, &set_val);
2 /* Set a value that is a pointer to a struct */
3 MPI_Comm_set_attr(MPI_COMM_WORLD, keyval2, &set_struct);
4 /* Set an integer value */
5 MPI_Comm_set_attr(MPI_COMM_WORLD, keyval3, (void *) 17);

```

6 B. Reading the attribute value in C

```

7
8 int flag, *get_val;
9 struct foo *get_struct;
10
11 /* Upon successful return, get_val == &set_val
12    (and therefore *get_val == 3) */
13 MPI_Comm_get_attr(MPI_COMM_WORLD, keyval1, &get_val, &flag);
14 /* Upon successful return, get_struct == &set_struct */
15 MPI_Comm_get_attr(MPI_COMM_WORLD, keyval2, &get_struct, &flag);
16 /* Upon successful return, get_val == (void*) 17 */
17 /*      i.e., (MPI_Aint) get_val == 17 */
18 MPI_Comm_get_attr(MPI_COMM_WORLD, keyval3, &get_val, &flag);

```

19 C. Reading the attribute value with (deprecated) Fortran MPI-1 calls

```

20 LOGICAL FLAG
21 INTEGER IERR, GET_VAL, GET_STRUCT
22
23 ! Upon successful return, GET_VAL == &set_val, possibly truncated
24 CALL MPI_ATTR_GET(MPI_COMM_WORLD, KEYVAL1, GET_VAL, FLAG, IERR)
25 ! Upon successful return, GET_STRUCT == &set_struct, possibly truncated
26 CALL MPI_ATTR_GET(MPI_COMM_WORLD, KEYVAL2, GET_STRUCT, FLAG, IERR)
27 ! Upon successful return, GET_VAL == 17
28 CALL MPI_ATTR_GET(MPI_COMM_WORLD, KEYVAL3, GET_VAL, FLAG, IERR)

```

29 D. Reading the attribute value with Fortran MPI-2 calls

```

30 LOGICAL FLAG
31 INTEGER IERR
32 INTEGER(KIND=MPI_ADDRESS_KIND) GET_VAL, GET_STRUCT
33
34 ! Upon successful return, GET_VAL == &set_val
35 CALL MPI_COMM_GET_ATTR(MPI_COMM_WORLD, KEYVAL1, GET_VAL, FLAG, IERR)
36 ! Upon successful return, GET_STRUCT == &set_struct
37 CALL MPI_COMM_GET_ATTR(MPI_COMM_WORLD, KEYVAL2, GET_STRUCT, FLAG, IERR)
38 ! Upon successful return, GET_VAL == 17
39 CALL MPI_COMM_GET_ATTR(MPI_COMM_WORLD, KEYVAL3, GET_VAL, FLAG, IERR)

```

40 **Example 21.29.** Setting an attribute in Fortran and reading in C or Fortran.

41 A. Setting an attribute value with the (deprecated) Fortran MPI-1 call

```

42 INTEGER IERR, VAL
43 VAL = 7
44 CALL MPI_ATTR_PUT(MPI_COMM_WORLD, KEYVAL, VAL, IERR)

```

45 B. Reading the attribute value in C

```

46 int flag;
47 int *value;
48

```



```

/* Upon successful return, value points to internal MPI storage and
   *value == (int) 7 */
MPI_Comm_get_attr(MPI_COMM_WORLD, keyval, &value, &flag);

```

C. Reading the attribute value with (deprecated) Fortran MPI-1 calls

```

LOGICAL FLAG
INTEGER IERR, VALUE

! Upon successful return, VALUE == 7
CALL MPI_ATTR_GET(MPI_COMM_WORLD, KEYVAL, VALUE, FLAG, IERR)

```

D. Reading the attribute value with Fortran MPI-2 calls

```

LOGICAL FLAG
INTEGER IERR
INTEGER(KIND=MPI_ADDRESS_KIND) VALUE

! Upon successful return, VALUE == 7 (sign extended)
CALL MPI_COMM_GET_ATTR(MPI_COMM_WORLD, KEYVAL, VALUE, FLAG, IERR)

```

Example 21.30. Setting an attribute in Fortran and reading in C or Fortran.

A. Setting an attribute value via a Fortran MPI-2 call

```

INTEGER IERR
INTEGER(KIND=MPI_ADDRESS_KIND) VALUE1
INTEGER(KIND=MPI_ADDRESS_KIND) VALUE2
VALUE1 = 42
VALUE2 = INT(2, KIND=MPI_ADDRESS_KIND) ** 40

CALL MPI_COMM_SET_ATTR(MPI_COMM_WORLD, KEYVAL1, VALUE1, IERR)
CALL MPI_COMM_SET_ATTR(MPI_COMM_WORLD, KEYVAL2, VALUE2, IERR)

```

B. Reading the attribute value in C

```

int flag;
MPI_Aint *value1, *value2;

/* Upon successful return, value1 points to internal MPI storage and
   *value1 == 42 */
MPI_Comm_get_attr(MPI_COMM_WORLD, keyval1, &value1, &flag);
/* Upon successful return, value2 points to internal MPI storage and
   *value2 == 2^40 */
MPI_Comm_get_attr(MPI_COMM_WORLD, keyval2, &value2, &flag);

```

C. Reading the attribute value with (deprecated) Fortran MPI-1 calls

```

LOGICAL FLAG
INTEGER IERR, VALUE1, VALUE2

! Upon successful return, VALUE1 == 42
CALL MPI_ATTR_GET(MPI_COMM_WORLD, KEYVAL1, VALUE1, FLAG, IERR)
! Upon successful return, VALUE2 == 2^40, or 0 if truncation
! needed (i.e., the least significant part of the attribute word)
CALL MPI_ATTR_GET(MPI_COMM_WORLD, KEYVAL2, VALUE2, FLAG, IERR)

```

D. Reading the attribute value with Fortran MPI-2 calls

```

1  LOGICAL FLAG
2  INTEGER IERR
3  INTEGER(KIND=MPI_ADDRESS_KIND) VALUE1, VALUE2
4
5
6  ! Upon successful return, VALUE1 == 42
7  CALL MPI_COMM_GET_ATTR(MPI_COMM_WORLD, KEYVAL1, VALUE1, FLAG, IERR)
8  ! Upon successful return, VALUE2 == 2^40
9  CALL MPI_COMM_GET_ATTR(MPI_COMM_WORLD, KEYVAL2, VALUE2, FLAG, IERR)
10

```

The predefined MPI attributes can be integer valued or address-valued. Predefined integer valued attributes, such as `MPI_TAG_UB`, behave as if they were put by a call to the deprecated Fortran routine `MPI_ATTR_PUT`, i.e., in Fortran, `MPI_COMM_GET_ATTR(MPI_COMM_WORLD, MPI_TAG_UB, val, flag, ierr)` will return in `val` the upper bound for tag value; in C, `MPI_Comm_get_attr(MPI_COMM_WORLD, MPI_TAG_UB, &p, &flag)` will return in `p` a pointer to an int containing the upper bound for tag value.

Address-valued predefined attributes, such as `MPI_WIN_BASE` behave as if they were put by a C call, i.e., in Fortran, `MPI_WIN_GET_ATTR(win, MPI_WIN_BASE, val, flag, ierror)` will return in `val` the base address of the window, converted to an integer. In C, `MPI_Win_get_attr(win, MPI_WIN_BASE, &p, &flag)` will return in `p` a pointer to the window base, cast to `(void *)`.

Rationale. The design is consistent with the behavior specified for predefined attributes, and ensures that no information is lost when attributes are passed from language to language. Because the language interoperability for predefined attributes was defined based on `MPI_ATTR_PUT`, this definition is kept for compatibility reasons although the routine itself is now deprecated. (*End of rationale.*)

Advice to implementors. Implementations should tag attributes either as (1) address attributes, (2) as `INTEGER(KIND=MPI_ADDRESS_KIND)` attributes or (3) as `INTEGER` attributes, according to whether they were set in (1) C (with `MPI_Attr_put` or `MPI_XXX_set_attr`), (2) in Fortran with `MPI_XXX_SET_ATTR` or (3) with the deprecated Fortran routine `MPI_ATTR_PUT`. Thus, the right choice can be made when the attribute is retrieved. (*End of advice to implementors.*)

21.3.8 Extra-State

Extra-state should not be modified by the copy or delete callback functions. (This is obvious from the C binding, but not obvious from the Fortran binding). However, these functions may update state that is indirectly accessed via extra-state. E.g., in C, extra-state can be a pointer to a data structure that is modified by the copy or callback functions; in Fortran, extra-state can be an index into an entry in a `COMMON` array that is modified by the copy or callback functions. In a multithreaded environment, users should be aware that distinct threads may invoke the same callback function concurrently: if this function modifies state associated with extra-state, then mutual exclusion code must be used to protect updates and accesses to the shared state.

21.3.9 Constants

MPI constants have the same value in all languages, unless specified otherwise. This does not apply to constant handles (`MPI_INT`, `MPI_COMM_WORLD`, `MPI_ERRORS_RETURN`, `MPI_SUM`, etc.) These handles need to be converted, as explained in Section 21.3.4. Constants that specify maximum lengths of strings (see Section A.1.1 for a listing) have a value one less in Fortran than C since in C the length includes the null terminating character. Thus, these constants represent the amount of space that must be allocated to hold the largest possible such string, rather than the maximum number of printable characters the string could contain.

Advice to users. This definition means that it is safe in C to allocate a buffer to receive a string using a declaration like

```
char name [MPI_MAX_OBJECT_NAME];
```

(*End of advice to users.*)

Also constant “addresses,” i.e., special values for reference arguments that are not handles, such as `MPI_BOTTOM` or `MPI_STATUS_IGNORE` may have different values in different languages.

Rationale. The current MPI standard specifies that `MPI_BOTTOM` can be used in initialization expressions in C, but not in Fortran. Since Fortran does not normally support call by value, then `MPI_BOTTOM` in Fortran must be the name of a predefined static variable, e.g., a variable in an MPI declared COMMON block. On the other hand, in C, it is natural to take `MPI_BOTTOM = 0` (Caveat: Defining `MPI_BOTTOM = 0` implies that NULL pointer cannot be distinguished from `MPI_BOTTOM`; it may be that `MPI_BOTTOM = 1` is better. See the advice to implementors in the *Datatypes* subsection in Section 21.3.6) Requiring that the Fortran and C values be the same will complicate the initialization process. (*End of rationale.*)

21.3.10 Interlanguage Communication

The type matching rules for communication in MPI are not changed: the datatype specification for each item sent should match, in type signature, the datatype specification used to receive this item (unless one of the types is `MPI_PACKED`). Also, the type of a message item should match the type declaration for the corresponding communication buffer location, unless the type is `MPI_BYTE` or `MPI_PACKED`. Interlanguage communication is allowed if it complies with these rules.

Example 21.31. In the example below, a Fortran array is sent from Fortran and received in C.

```
! FORTRAN CODE
SUBROUTINE MYEXAMPLE()
USE mpi_f08
REAL :: R(5)
INTEGER :: IERR, MYRANK, AOBLN(1)
TYPE(MPI_Datatype) :: DTYPE, AOTYPE(1)
INTEGER(KIND=MPI_ADDRESS_KIND) :: AODISP(1)
```

```

1  ! create an absolute datatype for array R
2  AOBLEN(1) = 5
3  CALL MPI_GET_ADDRESS(R, AODISP(1), IERR)
4  AOTYPE(1) = MPI_REAL
5  CALL MPI_TYPE_CREATE_STRUCT(1, AOBLEN, AODISP, AOTYPE, DTYPE, IERR)
6  CALL MPI_TYPE_COMMIT(DTYPE, IERR)
7
8  CALL MPI_COMM_RANK(MPI_COMM_WORLD, MYRANK, IERR)
9  IF (MYRANK .EQ. 0) THEN
10     CALL MPI_SEND(MPI_BOTTOM, 1, DTYPE, 1, 0, MPI_COMM_WORLD, IERR)
11 ELSE
12     CALL C_ROUTINE(DTYPE%MPI_VAL)
13 END IF
14 END SUBROUTINE
15
16 /* C code */
17
18 void C_ROUTINE(MPI_Fint *fhandle)
19 {
20     MPI_Datatype type;
21     MPI_Status status;
22
23     type = MPI_Type_f2c(*fhandle);
24
25     MPI_Recv(MPI_BOTTOM, 1, type, 0, 0, MPI_COMM_WORLD, &status);
26 }

```

MPI implementors may weaken these type matching rules, and allow messages to be sent with Fortran types and received with C types, and vice versa, when those types match. I.e., if the Fortran type `INTEGER` is identical to the C type `int`, then an MPI implementation may allow data to be sent with datatype `MPI_INTEGER` and be received with datatype `MPI_INT`. However, such code is not portable.

Chapter 22

Application Binary Interface (ABI)

22.1 Introduction

The other chapters of the MPI standard specify an Application Programming Interface (API) that defines (amongst other things) a set of opaque handle types and named constants without specifying their memory layout or values, respectively. This allows implementations to choose these according to different types of requirement. However, this flexibility means that different implementations are incompatible from the perspective of compiled applications, because the Application Binary Interface (ABI) is not specified.

This chapter defines an Application Binary Interface (ABI) for MPI, meaning that it specifies the memory layouts of all opaque handle types, the values of integer constants, and other aspects of MPI needed by use cases that require a defined ABI. This standard ABI for MPI exists in parallel with existing implementation ABIs, in order to preserve backwards-compatibility of existing MPI implementations.

A standard ABI for MPI serves many purposes, including support for applications compiled with one implementation of MPI to be executed with another implementation. It is also a necessary requirement for third-party languages that intend to interface with MPI through binary symbol names, rather than direct function calls to the C API. This chapter specifies the ABI of the C API of MPI as well as the implications of the ABI for Fortran implementations.

22.2 Implementation Requirements

Although the ABI is designed to be portable, there are platform and implementation designs where it cannot be supported, or is not useful to support. Furthermore, backwards compatibility of existing implementation ABIs is important to some users, and MPI implementations may continue to support these.

The standard ABI is intended to support systems with the following properties:

- Dynamic shared libraries, which are loaded by the operating system or by the application, are supported.
- The calling convention of C functions and the sizes and alignment requirements of C standard types are known constant properties of the system; if the system possesses any means for changing these, each choice constitutes a different, incompatible system from the perspective of the MPI ABI.
- Addresses can be represented as 32- or 64-bit signed integers and have a direct relationship with C pointers; segmented addressing is not supported.

The following query functions are provided to allow MPI applications, tools, and language bindings to determine whether an implementation provides ABI support and, if so, which version of the ABI is supported. `MPI_ABI_GET_VERSION` and `MPI_ABI_GET_INFO`, can be called at any time in an MPI program. These functions must always be thread-safe, as defined in Section 11.6.

`MPI_ABI_GET_VERSION`(abi_major, abi_minor)

OUT abi_major ABI major version (integer)

OUT abi_minor ABI minor version (integer)

C binding

int MPI_Abi_get_version(int *abi_major, int *abi_minor)

Fortran 2008 binding

MPI_Abi_get_version(abi_major, abi_minor, ierror)

INTEGER, INTENT(OUT) :: abi_major, abi_minor

INTEGER, OPTIONAL, INTENT(OUT) :: ierror

Fortran binding

MPI_ABI_GET_VERSION(ABI_MAJOR, ABI_MINOR, IERROR)

INTEGER ABI_MAJOR, ABI_MINOR, IERROR

`MPI_ABI_GET_VERSION` produces the standard ABI version, if supported. Otherwise, the values of the major and minor version are set to -1 . The ABI version is independent of the MPI specification version. The major and minor version of the ABI associated with MPI-5.0 are 1 and 0.

The ABI version macros `MPI_ABI_VERSION` and `MPI_ABI_SUBVERSION` are present in the MPI header and modules so that applications can check for consistency between the compilation environment and the properties of the implementation at runtime.

```
#define MPI_ABI_VERSION      1
```

```
#define MPI_ABI_SUBVERSION   0
```

```
INTEGER :: MPI_ABI_VERSION, MPI_ABI_SUBVERSION
```

```
PARAMETER (MPI_ABI_VERSION      = 1)
```

```
PARAMETER (MPI_ABI_SUBVERSION = 0)
```

Backwards-compatible changes, such as the addition of new handle types, will increment the minor version. Backwards-incompatible changes will increment the major version. The addition of new functions to the MPI API does not change the ABI version. The existing function `MPI_GET_VERSION` can be used to query the version of the API supported and whether certain functions are present in the MPI library.

`MPI_ABI_GET_INFO`(info)

OUT info ABI details info object (implementation-defined)
(handle)

C binding

int MPI_Abi_get_info(MPI_Info *info)

Fortran 2008 binding

```

MPI_Abi_get_info(info, ierror)
    TYPE(MPI_Info), INTENT(OUT) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ABI_GET_INFO(INFO, IERROR)
    INTEGER INFO, IERROR

```

Implementations may provide additional information related to the ABI. The function `MPI_ABI_GET_INFO` allows the user to query this information via an info object.

The following keys are predefined for this object:

"mpi_aint_size": The size in bytes of `MPI_Aint`.

"mpi_count_size": The size in bytes of `MPI_Count`.

"mpi_offset_size": The size in bytes of `MPI_Offset`.

22.2.1 The MPI ABI Header File and Shared Library

The ABI must be implemented using a header named `mpi.h`. The MPI library that implements the standard ABI must be named `mpi_abi`. The filename for this library may have a platform-specific prefix and/or a platform-specific suffix. For Linux, for example, `lib` and `.so` would be the default prefix and suffix. Implementors are expected to follow platform-specific conventions for dynamic shared library naming and versioning. ABI-compliant implementations must not require more than `mpi_abi` or its versioned variant as the sole direct dependency of the application binary.

Advice to implementors. If an implementation implements its own ABI definition, it must clearly document how users employ one or the other, such as the paths of the aforementioned files and any other options required for their correct use. (*End of advice to implementors.*)

Applications must not mix different ABIs. If implementations provide both the standard ABI and an implementation-specific ABI, applications must compile and link against only one of these.

The API defined in `mpi.h` associated with the standard ABI does not include features of MPI deprecated in MPI-3.1 or earlier. A full list of deprecated features can be found in Table 2.1.

Rationale. If deprecated features are included in the standard ABI, deleting them will cause a backwards-incompatibility issue in the ABI. Removing them from the ABI now makes it straightforward for them to be deleted from MPI in the future. (*End of rationale.*)

22.3 The C Application Binary Interface**22.3.1 The Status Object**

The MPI status object is a struct containing 8 integers: the three public member fields described in Section 3.2.5 and 5 private member fields that are reserved for implementations and must never be directly accessed by applications.

The MPI status object is defined in C as follows:

```
typedef struct {
    int MPI_SOURCE;
    int MPI_TAG;
    int MPI_ERROR;
    int MPI_internal[5];
} MPI_Status;
```

The MPI status object must use exactly eight C int worth of storage.

Advice to implementors. The alignment of the status object may be less than the alignment of a pointer or MPI_Count. Therefore, implementations that store such a value in the status object must take care to access it using a method that does not depend on alignment greater than int. Such methods include memcopy and type-punning. (End of advice to implementors.)

Rationale. This definition provides sufficient space to accommodate implementation-specific information and leads to memory alignment that allows efficient access to arrays of status objects. (End of rationale.)

22.3.2 Opaque Handles

Handles for MPI objects are defined to be incomplete struct pointers, which allows for C compilers to do type-checking, while also satisfying the existing requirements, such as equality comparison.

Rationale. Integer handles do not provide type safety, while struct or union handles fail to satisfy the existing API requirements. (End of rationale.)

The following handle type definitions are part of the MPI ABI:

```
typedef struct MPI_ABI_Comm* MPI_Comm;
typedef struct MPI_ABI_Datatype* MPI_Datatype;
typedef struct MPI_ABI_Errhandler* MPI_Errhandler;
typedef struct MPI_ABI_File* MPI_File;
typedef struct MPI_ABI_Group* MPI_Group;
typedef struct MPI_ABI_Info* MPI_Info;
typedef struct MPI_ABI_Message* MPI_Message;
typedef struct MPI_ABI_Op* MPI_Op;
typedef struct MPI_ABI_Queue* MPI_Queue;
typedef struct MPI_ABI_Request* MPI_Request;
typedef struct MPI_ABI_Session* MPI_Session;
typedef struct MPI_ABI_Win* MPI_Win;
```

22.3.3 Handle Constants

Every handle type has at least one, and often many, predefined constants of that type, e.g., [MPI_REQUEST_NULL](#) for MPI_Request and [MPI_COMM_WORLD](#) for MPI_Comm. The MPI ABI defines handle constants to be compile-time constants, which are specified as integer expressions cast to the appropriate handle type.

Rationale. Link-time constants, while convenient, are not strictly portable. (*End of rationale.*)

All predefined handle constants correspond to integer representations that are unlikely to be valid addresses, and must not be dereferenced. Implementations must ensure that the handles they create for the user are never in the range reserved for predefined handle constants. The MPI ABI reserves values corresponding to the integers 1 to 4095 for predefined handle constants. The definition of these constants is described in Section 22.5. Handle arguments with an integer representation of zero are never valid handles.

Rationale. Many operating systems support a “zero page” that corresponds to the above address range, in which case, implementations will not need to do any runtime checking to ensure the above requirement is satisfied. Ensuring that an integer representation of zero is never a legal handle argument allows the detection of uninitialized data, which may lead to undefined behavior. (*End of rationale.*)

All of the constants are specified in Section A.1.1.

22.3.4 Integer Constants

Integer constants fall into groups, where all constants in each group must have unique values. In cases where integer constants are intended to be combined using bitwise logical expressions, there are additional requirements, specified elsewhere (e.g. Section 15.2.1). A different constraint exists for constants like `MPI_ANY_SOURCE`, which must be negative, because any non-negative integer may be a valid rank.

The MPI ABI reserves all unused values up to 16384 for integer constants, to allow for adding new constants without creating a noncontiguous range. For example, implementations may define their own extensions for `MPI_COMM_SPLIT_TYPE` that use non-standard values of the `split_type` argument – these integer constants must exceed 16384.

22.3.5 Integer Types

MPI defines four special types of integers in C: `MPI_Aint`, `MPI_Offset`, `MPI_Count`, and `MPI_Fint`. The properties of `MPI_Aint` correspond to the C standard integer type `intptr_t`. Thus, `MPI_Aint` should be defined as a C typedef to `intptr_t`. In a compilation environment where `intptr_t` is not available, a type with the same properties must be used.

Essentially all filesystems relevant to MPI use 64-bit addressing so the standard ABI defines `MPI_Offset` to be the C standard integer type `int64_t` (or an equivalent type). On systems where `intptr_t` is 32 or 64 bits, `MPI_Count` is the C standard integer type `int64_t` (or an equivalent type).

`MPI_Fint` is discussed in Section 22.4.

Rationale. Fixing the size of `MPI_Offset` ensures the standard ABI depends only on the address size and thus is unambiguous on each platform. The need for MPI to support offsets greater than 64 bits implies the possibility of a single MPI file of more than 8 exabytes in size, which is not currently practical. The use of 64-bit MPI offsets does not prevent MPI from supporting 128-bit filesystems. (*End of rationale.*)

22.3.6 Calling Conventions and Binary Representations

ABI compatibility means that the binary object code of the MPI implementation, libraries that use MPI, and the main MPI application program can be linked and executed correctly. The type layouts, symbol names, and calling conventions of MPI routines behave as if they have been compiled with the system C compiler toolchain (as determined, in particular, by the system C runtime library).

Advice to users. Libraries and applications that use MPI may be built with any toolchain they wish, as long as they adhere to these conventions when calling MPI routines. Compiler options that change the size or layout of types, or calling conventions, should be avoided. (*End of advice to users.*)

22.4 The Fortran Application Binary Interface

Fortran support for the ABI is more complicated than C, because the sizes of `INTEGER`, `REAL`, and `DOUBLE PRECISION` can be changed with compiler options; there is no ABI constancy even with a single Fortran compiler and platform. This flexibility creates a difficult situation for the MPI C ABI because the functions defined in Section 21.3.4 and Section 21.3.5 in the MPI C API depend on the size of `INTEGER` via `MPI_Fint`. As a result, the functions defined in Section 21.3.4 and Section 21.3.5 as well as `MPI_F08_Status` are not part of this ABI. Instead, to support Fortran and other use cases where obtaining an integer associated with an MPI handle is necessary, new functions are added that do not depend on `MPI_Fint`.

MPI applications can discover the size of Fortran types such as `MPI_INTEGER` and `MPI_REAL` using `MPI_TYPE_SIZE`. Lack of support in the implementation for optional predefined datatypes is indicated when the type size returned is `MPI_UNDEFINED`.

Rationale. Prior to the ABI, optional predefined datatypes were not present in the MPI header and modules. When absent, usage would generate compilation errors. The standard ABI must define a value for all predefined datatypes, including the optional ones. Therefore, the absence of optional datatypes must be detected at runtime. (*End of rationale.*)

22.4.1 Fortran Type Registration

In order to decouple MPI Fortran support from the rest of the implementation, there must be a way to inform the implementation of the properties of MPI Fortran datatypes. With this, it is possible to implement MPI Fortran support on top of the MPI C implementation, without the latter having to know the properties of the Fortran environment.

MPI_ABI_SET_FORTRAN_INFO(info)

IN	info	Fortran ABI details info object (handle)
----	------	--

C binding

```
int MPI_Abi_set_fortran_info(MPI_Info info)
```

Fortran 2008 binding

```

MPI_Abi_set_fortran_info(info, ierror)
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ABI_SET_FORTRAN_INFO(INFO, IERROR)
    INTEGER INFO, IERROR

```

```

MPI_ABI_GET_FORTRAN_INFO(info)

```

```

    OUT      info      Fortran ABI details info object (handle)

```

C binding

```

int MPI_Abi_get_fortran_info(MPI_Info *info)

```

Fortran 2008 binding

```

MPI_Abi_get_fortran_info(info, ierror)
    TYPE(MPI_Info), INTENT(OUT) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

MPI_ABI_GET_FORTRAN_INFO(INFO, IERROR)
    INTEGER INFO, IERROR

```

[MPI_ABI_SET_FORTRAN_INFO](#) allows the application to inform the implementation of the sizes of Fortran types and whether or not optional types are supported by the Fortran compiler. Before setting this information, the application should get this info object using [MPI_ABI_GET_FORTRAN_INFO](#). When [MPI_INFO_NULL](#) is returned, the implementation does not know the properties of the Fortran compiler and they must be set by the application. Only the first call to [MPI_ABI_SET_FORTRAN_INFO](#) affects the state of the MPI library; all subsequent calls will return the error code [MPI_ERR_ABI](#). If a call to [MPI_ABI_SET_FORTRAN_INFO](#) is not successful, the user should call [MPI_ABI_GET_FORTRAN_INFO](#) to determine what Fortran compiler properties were set.

The following keys are predefined for this object:

"mpi_logical_size": The size in bytes of the Fortran default LOGICAL kind.

"mpi_integer_size": The size in bytes of the Fortran default INTEGER kind.

"mpi_real_size": The size in bytes of the Fortran default REAL kind.

"mpi_double_precision_size": The size in bytes of the Fortran DOUBLE PRECISION kind.

"mpi_logical1_supported": (boolean) [MPI_LOGICAL1](#) is supported.

"mpi_logical2_supported": (boolean) [MPI_LOGICAL2](#) is supported.

"mpi_logical4_supported": (boolean) [MPI_LOGICAL4](#) is supported.

"mpi_logical8_supported": (boolean) [MPI_LOGICAL8](#) is supported.

"mpi_logical16_supported": (boolean) [MPI_LOGICAL16](#) is supported.

```

1  "mpi_integer1_supported": (boolean) MPI_INTEGER1 is supported.
2
3  "mpi_integer2_supported": (boolean) MPI_INTEGER2 is supported.
4
5  "mpi_integer4_supported": (boolean) MPI_INTEGER4 is supported.
6
7  "mpi_integer8_supported": (boolean) MPI_INTEGER8 is supported.
8
9  "mpi_integer16_supported": (boolean) MPI_INTEGER16 is supported.
10
11 "mpi_real2_supported": (boolean) MPI_REAL2 is supported.
12
13 "mpi_real4_supported": (boolean) MPI_REAL4 is supported.
14
15 "mpi_real8_supported": (boolean) MPI_REAL8 is supported.
16
17 "mpi_real16_supported": (boolean) MPI_REAL16 is supported.
18
19 "mpi_complex4_supported": (boolean) MPI_COMPLEX4 is supported.
20
21 "mpi_complex8_supported": (boolean) MPI_COMPLEX8 is supported.
22
23 "mpi_complex16_supported": (boolean) MPI_COMPLEX16 is supported.
24
25 "mpi_complex32_supported": (boolean) MPI_COMPLEX32 is supported.
26
27 "mpi_double_complex_supported": (boolean) MPI_DOUBLE_COMPLEX is supported.

```

```

26 MPI_ABI_SET_FORTRAN_BOOLEANS(logical_size, logical_true, logical_false)
27     IN        logical_size        the size of Fortran LOGICAL in bytes (integer)
28     IN        logical_true        the Fortran literal value .TRUE. (logical)
29     IN        logical_false       the Fortran literal value .FALSE. (logical)

```

C binding

```

33 int MPI_Abi_set_fortran_booleans(int logical_size, void *logical_true,
34                                void *logical_false)

```

Fortran 2008 binding

```

36 MPI_Abi_set_fortran_booleans(logical_size, logical_true, logical_false, ierror)
37     INTEGER, INTENT(IN) :: logical_size
38     LOGICAL, INTENT(IN) :: logical_true, logical_false
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

Fortran binding

```

42 MPI_ABI_SET_FORTRAN_BOOLEANS(LOGICAL_SIZE, LOGICAL_TRUE, LOGICAL_FALSE, IERROR)
43     INTEGER LOGICAL_SIZE, IERROR
44     LOGICAL LOGICAL_TRUE, LOGICAL_FALSE

```

```
MPI_ABI_GET_FORTRAN_BOOLEANS(logical_size, logical_true, logical_false, is_set)
```

IN	logical_size	the size of Fortran LOGICAL in bytes (integer)
OUT	logical_true	the Fortran literal value <code>.TRUE.</code> (logical)
OUT	logical_false	the Fortran literal value <code>.FALSE.</code> (logical)
OUT	is_set	flag to indicate whether the logical boolean values were set previously (logical)

C binding

```
int MPI_Abi_get_fortran_booleans(int logical_size, void *logical_true,
                                void *logical_false, int *is_set)
```

Fortran 2008 binding

```
MPI_Abi_get_fortran_booleans(logical_size, logical_true, logical_false, is_set,
                              ierror)
    INTEGER, INTENT(IN) :: logical_size
    LOGICAL, INTENT(OUT) :: logical_true, logical_false, is_set
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

Fortran binding

```
MPI_ABI_GET_FORTRAN_BOOLEANS(LOGICAL_SIZE, LOGICAL_TRUE, LOGICAL_FALSE, IS_SET,
                              IERROR)
    INTEGER LOGICAL_SIZE, IERROR
    LOGICAL LOGICAL_TRUE, LOGICAL_FALSE, IS_SET
```

MPI_ABI_SET_FORTRAN_BOOLEANS allows the application to inform the implementation of the literal values of the Fortran booleans. Boolean literals must be passed directly so that they can be observed by the implementation, since it may not be possible to obtain their literal values directly. This function has a size argument to allow it to be called before **MPI_ABI_SET_FORTRAN_INFO**. Before setting this information, the application should check if the logical values are already set using **MPI_ABI_GET_FORTRAN_BOOLEANS**. When `is_set = false`, the implementation does not know the properties of the Fortran compiler and they must be set by the application. When `is_set = true`, the implementation already knows the literal values of the Fortran booleans and they cannot be set. As with **MPI_ABI_SET_FORTRAN_INFO**, only the first call to this function affects the state of the MPI library; subsequent calls will return the error code **MPI_ERR_ABI**.

Rationale. MPI does not assume that Fortran boolean literals follow the C convention (zero is false and non-zero is true). (*End of rationale.*)

22.4.2 The MPI ABI Fortran Modules and Shared Library

The ABI must be implemented using modules named `mpi` and `mpi_f08`. The MPI library that implements the standard ABI must be named `mpifort_abi` and follow all of the requirements stated in Section 22.2.1 for naming, versioning, and dependencies.

22.4.3 The Status Object

The MPI status object is defined in Fortran as follows:

```
integer, parameter :: MPI_STATUS_SIZE = 8
type, bind(C) :: Status
    integer :: MPI_SOURCE
    integer :: MPI_TAG
    integer :: MPI_ERROR
    integer :: MPI_INTERNAL(5)
end type Status
```

The MPI status object must use exactly eight Fortran INTEGER worth of storage.

The following constants can be specified:

```
#define MPI_F_STATUS_SIZE 8
#define MPI_F_SOURCE      0
#define MPI_F_TAG         1
#define MPI_F_ERROR       2
```

22.4.4 Integer Constants

As specified elsewhere (Section 21.3.9), constants have the same value in all languages, unless specified otherwise.

The constants `MPI_F_STATUS_IGNORE`, `MPI_F_STATUSES_IGNORE`, `MPI_F08_STATUS_IGNORE` and `MPI_F08_STATUSES_IGNORE` are not specified in the C header file.

Rationale. Users can obtain the addresses of Fortran sentinels by passing them as arguments to a Fortran function call that is implemented in C. (*End of rationale.*)

22.4.5 Handle Serialization

Section 21.3.4 defines methods for converting handles to integers of the type `MPI_Fint`. In order to provide this functionality without depending on the behavior of the Fortran compiler, new functions that convert handles to and from `int` are necessary. Because these functions depend only on C language features, they are not referred to as C-Fortran conversion functions but handle serialization functions.

In the C ABI, handles are pointers and therefore applications can trivially serialize handles into the type `intptr_t` using a cast, but this does not support use cases where a language integer type is narrower than this.

Rationale. While it is possible to implement this functionality outside of MPI, e.g. using a lookup table or hash function, it is possible to implement more efficiently within an MPI implementation, particularly if the implementation is already using a limited range of values for C handles. An implementation may also store the integer associated with a C handle, in which case the lookup is trivial. (*End of rationale.*)

C binding

```
int MPI_Comm_toint(MPI_Comm comm)
```

The function `MPI_Comm_toint` translates a C communicator handle into a C integer. For all predefined handles, the integer value must be the same as the values listed in Section A. For user-defined handles, the implementation must return the same integer for every call to this function with the same handle, which does not conflict with the reserved range for predefined handles. It is erroneous to call this function with an invalid handle argument.

```
MPI_Comm MPI_Comm_fromint(int comm)
```

The function `MPI_Comm_fromint` translates a C integer to the appropriate C communicator handle. Only an integer obtained from a previous call to `MPI_Comm_toint` may be passed to this function. It is erroneous to pass to this function an integer associated with a handle that has been freed, disconnected, or aborted (or that was derived from a session that has been finalized).

Similar functions are provided for the other types of opaque objects.

```
MPI_Datatype MPI_Type_fromint(int datatype)
```

```
int MPI_Type_toint(MPI_Datatype datatype)
```

```
MPI_Group MPI_Group_fromint(int group)
```

```
int MPI_Group_toint(MPI_Group group)
```

```
MPI_Request MPI_Request_fromint(int request)
```

```
int MPI_Request_toint(MPI_Request request)
```

```
MPI_File MPI_File_fromint(int file)
```

```
int MPI_File_toint(MPI_File file)
```

```
MPI_Win MPI_Win_fromint(int win)
```

```
int MPI_Win_toint(MPI_Win win)
```

```
MPI_Op MPI_Op_fromint(int op)
```

```
int MPI_Op_toint(MPI_Op op)
```

```
MPI_Info MPI_Info_fromint(int info)
```

```
int MPI_Info_toint(MPI_Info info)
```

```
MPI_Errhandler MPI_Errhandler_fromint(int errhandler)
```

```
int MPI_Errhandler_toint(MPI_Errhandler errhandler)
```

```
MPI_Message MPI_Message_fromint(int message)
```

```
int MPI_Message_toint(MPI_Message message)
```

```
MPI_Queue MPI_Queue_fromint(int queue)
```

```
int MPI_Queue_toint(MPI_Queue queue)
```

```
MPI_Session MPI_Session_fromint(int session)
```

```
int MPI_Session_toint(MPI_Session session)
```

Within the context of the ABI, where the layout of the status object is known and representable directly in terms of C int, no serialization functionality is necessary.

Table 22.1: Predefined MPI datatype categories and instance values, for types without a specified size. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00000	000 001 010 011 111	Language-independent types	MPI_DATATYPE_NULL MPI_AINT MPI_COUNT MPI_OFFSET MPI_PACKED

Table 22.2: Predefined MPI datatype categories and instance values, for types without a specified size. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00001	000 001 010 011 100 101 110 111	C integer types	MPI_SHORT MPI_INT MPI_LONG MPI_LONG_LONG MPI_UNSIGNED_SHORT MPI_UNSIGNED MPI_UNSIGNED_LONG MPI_UNSIGNED_LONG_LONG

22.5 Handle Constants

Predefined handle constants are represented as integers in the range 1 to 4095. To make it easy for implementations and tools to decode handle constants, their values are derived from a Huffman code. The Huffman code currently uses the lower 10 bits of the 12 bits allocated above. In the following, we will describe constants in their binary representation, `0b***_***_***_***`, where underscores are added for readability. We count bits from the right starting from zero; the rightmost three bits will be denoted 2 : 0.

Datatypes are the most common type of predefined handle and use the range `0b00_10_*****`. Bits 7 : 3 identify the category and the bits 2 : 0 identify the specific instances thereof. The first set of predefined datatypes are not fixed-sized. While `long` has a fixed size for a given platform ABI, it is not fixed across all platforms. In contrast, types like `int32_t` and `REAL*8` (or its Fortran standard equivalents) have the same size on all platforms. Fixed-sized types allow the implementation to determine the size of the datatype from the predefined handle value itself, without a lookup table.

The Huffman code for fixed-size types encodes the base-2 logarithm of the type size in bits 5 : 3. Implementations can identify datatypes with fixed-size using bit 6.

Reduction operators use the range `0b00_00_001_*****` and the predefined values fall into four categories: arithmetic, bit-wise, logical, and other.

All other predefined handles use the range `0b00_01_*****`. In general, predefined

Table 22.3: Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00010	000 001 010 100 101 110	C/C++ floating-point types	MPI_FLOAT MPI_C_FLOAT_COMPLEX MPI_CXX_FLOAT_COMPLEX MPI_DOUBLE MPI_C_DOUBLE_COMPLEX MPI_CXX_DOUBLE_COMPLEX

Table 22.4: Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00011	000 001 010 011 100 101 110	Fortran types	MPI_LOGICAL MPI_INTEGER MPI_REAL MPI_COMPLEX MPI_DOUBLE_PRECISION MPI_DOUBLE_COMPLEX MPI_CHARACTER

null handles have all zero bits other than those required to detect the handle type.

Table 22.5: Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00100	000 100 101	Long double types	MPI_LONG_DOUBLE MPI_C_LONG_DOUBLE_COMPLEX MPI_CXX_LONG_DOUBLE_COMPLEX

Table 22.6: Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00101	000 001 010 011 100 101	C pair types	MPI_FLOAT_INT MPI_DOUBLE_INT MPI_LONG_INT MPI_2INT MPI_SHORT_INT MPI_LONG_DOUBLE_INT
0010	00110	000 001 010	Fortran pair types	MPI_2REAL MPI_2DOUBLE_PRECISION MPI_2INTEGER

Table 22.7: Predefined MPI datatype categories and instance values, for types without a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	00111	000 001 100	Other C/C++ types	MPI_C_BOOL MPI_CXX_BOOL MPI_WCHAR

Table 22.8: Predefined MPI datatype categories and instance values, for types with a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	01000	000	1-byte C/C++ types	MPI_INT8_T
		001		MPI_UINT8_T
		011		MPI_CHAR
		100		MPI_SIGNED_CHAR
		101		MPI_UNSIGNED_CHAR
		111		MPI_BYTE
0010	01001	000	2-byte C/C++ types	MPI_INT16_T
		001		MPI_UINT16_T
0010	01010	000	4-byte C/C++ types	MPI_INT32_T
		001		MPI_UINT32_T
0010	01011	000	8-byte C/C++ types	MPI_INT64_T
		001		MPI_UINT64_T

Table 22.9: Predefined MPI datatype categories and instance values, for types with a specified width. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0010	11000	000 001	1-byte Fortran types	MPI_LOGICAL1 MPI_INTEGER1
0010	11001	000 001 010	2-byte Fortran types	MPI_LOGICAL2 MPI_INTEGER2 MPI_REAL2
0010	11010	000 001 010 011	4-byte Fortran types	MPI_LOGICAL4 MPI_INTEGER4 MPI_REAL4 MPI_COMPLEX4
0010	11011	000 001 010 011	8-byte Fortran types	MPI_LOGICAL8 MPI_INTEGER8 MPI_REAL8 MPI_COMPLEX8
0010	11100	000 001 010 011	16-byte Fortran types	MPI_LOGICAL16 MPI_INTEGER16 MPI_REAL16 MPI_COMPLEX16
0010	11101	011	32-byte Fortran types	MPI_COMPLEX32

Table 22.10: Predefined MPI_Op categories and instance values. All unassigned values are reserved.

Bits 11 : 5	Bits 4 : 3	Bits 2 : 0	Category	Instance
0000001	00	000 001 010 011 100	Arithmetic operations	MPI_OP_NULL MPI_SUM MPI_MIN MPI_MAX MPI_PROD
0000001	01	000 001 010	Bit operations	MPI_BAND MPI_BOR MPI_BXOR
0000001	10	000 001 010	Logical operations	MPI_LAND MPI_LOR MPI_LXOR
0000001	11	000 001 100 101	Other operations	MPI_MINLOC MPI_MAXLOC MPI_REPLACE MPI_NO_OP

Table 22.11: Predefined MPI handle categories and instance values. All unassigned values are reserved.

Bits 11 : 8	Bits 7 : 3	Bits 2 : 0	Category	Instance
0001	00000	000	Communicators	MPI_COMM_NULL
		001		MPI_COMM_WORLD
		010		MPI_COMM_SELF
0001	00001	000	Group	MPI_GROUP_NULL
		001		MPI_GROUP_EMPTY
0001	00010	000	Windows	MPI_WIN_NULL
		000		MPI_FILE_NULL
0001	00011	000	Files	MPI_SESSION_NULL
		000		MPI_MESSAGE_NULL
0001	00100	000	Sessions	MPI_MESSAGE_NO_PROC
		001		MPI_INFO_NULL
0001	00101	000	Messages	MPI_INFO_ENV
		001		MPI_ERRHANDLER_NULL
0001	00110	000	Info	MPI_ERRORS_ARE_FATAL
		001		MPI_ERRORS_RETURN
0001	01000	000	Errhandlers	MPI_ERRORS_ABORT
		001		MPI_REQUEST_NULL
0001	10000	000	Requests	
		000		

Appendix A

Language Bindings Summary

In this section we summarize the specific bindings for C and Fortran. First we present the constants, type definitions, info values and keys. Then we present the routine prototypes separately for each binding. Listings are alphabetical within chapter.

All ABI values must be cast to the appropriate type if the type of the constant is not a C int or Fortran INTEGER. For example, when `MPI_COMM_WORLD` is said to be 257, the implementation will use `((MPI_Comm)257)` in C.

A.1 Defined Values and Handles

A.1.1 Defined Constants

The C and Fortran names are listed below. Constants described as “integer constant expression” may be implemented as literal integer constants of the specified integer type substituted by the preprocessor or (where possible) as enum members.

Error classes	
C type: integer constant expression of type int Fortran type: INTEGER	ABI value
<code>MPI_SUCCESS</code>	0
<code>MPI_ERR_BUFFER</code>	1
<code>MPI_ERR_COUNT</code>	2
<code>MPI_ERR_TYPE</code>	3
<code>MPI_ERR_TAG</code>	4
<code>MPI_ERR_COMM</code>	5
<code>MPI_ERR_RANK</code>	6
<code>MPI_ERR_REQUEST</code>	7
<code>MPI_ERR_ROOT</code>	8
<code>MPI_ERR_GROUP</code>	9
<code>MPI_ERR_OP</code>	10
<code>MPI_ERR_TOPOLOGY</code>	11
<code>MPI_ERR_DIMS</code>	12
<code>MPI_ERR_ARG</code>	13
<code>MPI_ERR_UNKNOWN</code>	14
<code>MPI_ERR_TRUNCATE</code>	15
<code>MPI_ERR_OTHER</code>	16
<code>MPI_ERR_INTERN</code>	17
<code>MPI_ERR_PENDING</code>	18

(Continued on next page)

Error classes (continued)

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_ERR_IN_STATUS	19
MPI_ERR_ACCESS	20
MPI_ERR_AMODE	21
MPI_ERR_ASSERT	22
MPI_ERR_BAD_FILE	23
MPI_ERR_BASE	24
MPI_ERR_CONVERSION	25
MPI_ERR_DISP	26
MPI_ERR_DUP_DATAREP	27
MPI_ERR_FILE_EXISTS	28
MPI_ERR_FILE_IN_USE	29
MPI_ERR_FILE	30
MPI_ERR_INFO_KEY	31
MPI_ERR_INFO_NOKEY	32
MPI_ERR_INFO_VALUE	33
MPI_ERR_INFO	34
MPI_ERR_IO	35
MPI_ERR_KEYVAL	36
MPI_ERR_LOCKTYPE	37
MPI_ERR_NAME	38
MPI_ERR_NO_MEM	39
MPI_ERR_NOT_SAME	40
MPI_ERR_NO_SPACE	41
MPI_ERR_NO_SUCH_FILE	42
MPI_ERR_PORT	43
MPI_ERR_QUOTA	44
MPI_ERR_READ_ONLY	45
MPI_ERR_RMA_ATTACH	46
MPI_ERR_RMA_CONFLICT	47
MPI_ERR_RMA_RANGE	48
MPI_ERR_RMA_SHARED	49
MPI_ERR_RMA_SYNC	50
MPI_ERR_SERVICE	51
MPI_ERR_SIZE	52
MPI_ERR_SPAWN	53
MPI_ERR_UNSUPPORTED_DATAREP	54
MPI_ERR_UNSUPPORTED_OPERATION	55
MPI_ERR_WIN	56
MPI_ERR_RMA_FLAVOR	57
MPI_ERR_PROC_ABORTED	58
MPI_ERR_VALUE_TOO_LARGE	59
MPI_ERR_SESSION	60
MPI_ERR_ERRHANDLER	61
MPI_ERR_ABI	62
MPI_ERR_PROC_FAILED	63
MPI_ERR_REVOKED	64
MPI_ERR_PROC_FAILED_PENDING	65
MPI_ERR_QUEUE	66

Unofficial Draft for Comment Only

(Continued on next page)

Error classes (continued)

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_T_ERR_CANNOT_INIT	1001
MPI_T_ERR_NOT_ACCESSIBLE	1002
MPI_T_ERR_NOT_INITIALIZED	1003
MPI_T_ERR_NOT_SUPPORTED	1004
MPI_T_ERR_MEMORY	1005
MPI_T_ERR_INVALID	1006
MPI_T_ERR_INVALID_INDEX	1007
MPI_T_ERR_INVALID_ITEM	1008
MPI_T_ERR_INVALID_SESSION	1009
MPI_T_ERR_INVALID_HANDLE	1010
MPI_T_ERR_INVALID_NAME	1011
MPI_T_ERR_OUT_OF_HANDLES	1012
MPI_T_ERR_OUT_OF_SESSIONS	1013
MPI_T_ERR_CVAR_SET_NOT_NOW	1014
MPI_T_ERR_CVAR_SET_NEVER	1015
MPI_T_ERR_PVAR_NO_WRITE	1016
MPI_T_ERR_PVAR_NO_STARTSTOP	1017
MPI_T_ERR_PVAR_NO_ATOMIC	1018
MPI_ERR_LASTCODE	16383

Buffer address constants

C type: void * const Fortran type: (predefined memory location) ¹	ABI value in mpi.h
MPI_BOTTOM	((void*)0)
MPI_IN_PLACE	((void*)1)
MPI_BUFFER_AUTOMATIC	((void*)2)

¹ Note that in Fortran these constants are not usable for initialization expressions or assignment. See Section 2.5.4.

Constants specifying empty or ignored input

C/Fortran name C type / Fortran type ¹	ABI value in <code>mpi.h</code>
<code>MPI_ARGVS_NULL</code> char*** / 2-dim. array of CHARACTER*(*)	0
<code>MPI_ARGV_NULL</code> char** / array of CHARACTER*(*)	0
<code>MPI_ERRCODES_IGNORE</code> int* / INTEGER array	0
<code>MPI_STATUSES_IGNORE</code> MPI_Status* / INTEGER, DIMENSION(MPI_STATUS_SIZE,*) or TYPE(MPI_Status), DIMENSION(*)	0
<code>MPI_STATUS_IGNORE</code> MPI_Status* / INTEGER, DIMENSION(MPI_STATUS_SIZE) or TYPE(MPI_Status)	0
<code>MPI_UNWEIGHTED</code> int* / INTEGER array	10
<code>MPI_WEIGHTS_EMPTY</code> int* / INTEGER array	11

¹ Note that in Fortran these constants are not usable for initialization expressions or assignment. See Section 2.5.4.

Maximum sizes for strings

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
<code>MPI_MAX_DATAREP_STRING</code>	128
<code>MPI_MAX_ERROR_STRING</code>	512
<code>MPI_MAX_INFO_KEY</code>	256
<code>MPI_MAX_INFO_VAL</code>	1024
<code>MPI_MAX_LIBRARY_VERSION_STRING</code>	8192
<code>MPI_MAX_OBJECT_NAME</code>	128
<code>MPI_MAX_PORT_NAME</code>	1024
<code>MPI_MAX_PROCESSOR_NAME</code>	256
<code>MPI_MAX_STRINGTAG_LEN</code>	1024
<code>MPI_MAX_PSET_NAME_LEN</code>	1024

Mode constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_MODE_APPEND	1
MPI_MODE_CREATE	2
MPI_MODE_DELETE_ON_CLOSE	4
MPI_MODE_EXCL	8
MPI_MODE_RDONLY	16
MPI_MODE_RDWR	32
MPI_MODE_SEQUENTIAL	64
MPI_MODE_UNIQUE_OPEN	128
MPI_MODE_WRONLY	256
MPI_MODE_NOCHECK	1024
MPI_MODE_NOPRECEDE	2048
MPI_MODE_NOPUT	4096
MPI_MODE_NOSTORE	8192
MPI_MODE_NOSUCCEED	16384

Assorted constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_ANY_SOURCE	-1
MPI_ANY_TAG	-2
MPI_PROC_NULL	-3
MPI_ROOT	-4
MPI_UNDEFINED	-32766
MPI_BSEND_OVERHEAD	512

Threads constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_THREAD_SINGLE	0
MPI_THREAD_FUNNELED	1024
MPI_THREAD_SERIALIZED	2048
MPI_THREAD_MULTIPLE	4096

File operation constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_ORDER_C	12 (0xC)
MPI_ORDER_FORTRAN	15 (0xF)
MPI_DISTRIBUTE_NONE	16
MPI_DISTRIBUTE_BLOCK	17
MPI_DISTRIBUTE_CYCLIC	18
MPI_DISTRIBUTE_DFLT_DARG	19

Datatype decoding constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI values
MPI_COMBINER_NAMED	101
MPI_COMBINER_DUP	102
MPI_COMBINER_CONTIGUOUS	103
MPI_COMBINER_VECTOR	104
MPI_COMBINER_HVECTOR	105
MPI_COMBINER_INDEXED	106
MPI_COMBINER_HINDEXED	107
MPI_COMBINER_INDEXED_BLOCK	108
MPI_COMBINER_HINDEXED_BLOCK	109
MPI_COMBINER_STRUCT	110
MPI_COMBINER_SUBARRAY	111
MPI_COMBINER_DARRAY	112
MPI_COMBINER_F90_REAL	113
MPI_COMBINER_F90_COMPLEX	114
MPI_COMBINER_F90_INTEGER	115
MPI_COMBINER_RESIZED	116
MPI_COMBINER_VALUE_INDEX	117

F90 datatype matching constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_TYPECLASS_INTEGER	192
MPI_TYPECLASS_REAL	193
MPI_TYPECLASS_COMPLEX	194

Results of communicator and group comparisons

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_IDENT	201
MPI_CONGRUENT	202
MPI_SIMILAR	203
MPI_UNEQUAL	204

Topologies

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_CART	211
MPI_GRAPH	212
MPI_DIST_GRAPH	213

Communicator split type constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_COMM_TYPE_SHARED	221
MPI_COMM_TYPE_HW_UNGUIDED	222
MPI_COMM_TYPE_HW_GUIDED	223
MPI_COMM_TYPE_RESOURCE_GUIDED	224

Window lock type constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_LOCK_EXCLUSIVE	301
MPI_LOCK_SHARED	302

MPI window create flavors

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_WIN_FLAVOR_CREATE	311
MPI_WIN_FLAVOR_ALLOCATE	312
MPI_WIN_FLAVOR_DYNAMIC	313
MPI_WIN_FLAVOR_SHARED	314

MPI window models

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_WIN_UNIFIED	321
MPI_WIN_SEPARATE	322

File positioning constants

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_SEEK_CUR	401
MPI_SEEK_END	402
MPI_SEEK_SET	403

File operation constants

C type: integer constant expression of type MPI_Offset Fortran type: INTEGER(KIND=MPI_OFFSET_KIND)	ABI value
MPI_DISPLACEMENT_CURRENT	-1

Environmental inquiry and predefined attribute keys

C type: integer constant expression of type int Fortran type: INTEGER	ABI value
MPI_KEYVAL_INVALID	0
MPI_TAG_UB	501
MPI_IO	502
MPI_HOST (deprecated)	503
MPI_WTIME_IS_GLOBAL	504
MPI_APPNUM	505
MPI_LASTUSEDPCODE	506
MPI_UNIVERSE_SIZE	507
MPI_WIN_BASE	601
MPI_WIN_DISP_UNIT	602
MPI_WIN_SIZE	603
MPI_WIN_CREATE_FLAVOR	604
MPI_WIN_MODEL	605

Fortran support method specific constants

Fortran type: LOGICAL
MPI_SUBARRAYS_SUPPORTED (Fortran only)
MPI_ASYNC_PROTECTS_NONBLOCKING (Fortran only)

Status array size and reserved index values (Fortran only)

Fortran type: INTEGER	ABI value
MPI_STATUS_SIZE	8
MPI_SOURCE	1
MPI_TAG	2
MPI_ERROR	3

Fortran status array size and reserved index values (C only)

C type: integer constant expression of type int	ABI value
MPI_F_STATUS_SIZE	8
MPI_F_SOURCE	0
MPI_F_TAG	1
MPI_F_ERROR	2

Variable address size (Fortran only)

Fortran type: INTEGER	ABI value (from ISO_C_BINDING)
MPI_ADDRESS_KIND	c_intptr_t
MPI_OFFSET_KIND	c_int64_t
MPI_COUNT_KIND	c_int64_t

Reserved communicators

C type: MPI_Comm Fortran type: INTEGER or TYPE(MPI_Comm)	ABI value
MPI_COMM_NULL	256
MPI_COMM_WORLD	257
MPI_COMM_SELF	258

Named predefined datatypes

C type: MPI_Datatype Fortran type: INTEGER or TYPE(MPI_Datatype)	C types	ABI value
MPI_DATATYPE_NULL		512
MPI_AINT	MPI_Aint	513
MPI_COUNT	MPI_Count	514
MPI_OFFSET	MPI_Offset	515
MPI_PACKED	(any C datatype)	519
MPI_SHORT	signed short	520
MPI_INT	signed int	521
MPI_LONG	signed long	522
MPI_LONG_LONG_INT	signed long long	523
MPI_LONG_LONG (as a synonym)	signed long long	523
MPI_UNSIGNED_SHORT	unsigned short	524
MPI_UNSIGNED	unsigned int	525
MPI_UNSIGNED_LONG	unsigned long	526
MPI_UNSIGNED_LONG_LONG	unsigned long long	527
MPI_FLOAT	float	528
MPI_C_COMPLEX	float _Complex	530
MPI_C_FLOAT_COMPLEX (as a synonym)	float _Complex	530
MPI_DOUBLE	double	532
MPI_C_DOUBLE_COMPLEX	double _Complex	534
MPI_LONG_DOUBLE	long double	544
MPI_C_LONG_DOUBLE_COMPLEX	long double _Complex	548
MPI_C_BOOL	_Bool	568
MPI_WCHAR	wchar_t ^{1,3}	572
MPI_INT8_T	int8_t	576
MPI_UINT8_T	uint8_t	577
MPI_CHAR	char ¹	579
MPI_SIGNED_CHAR	signed char ²	580
MPI_UNSIGNED_CHAR	unsigned char ²	581
MPI_BYTE	(any C datatype)	583
MPI_INT16_T	int16_t	584
MPI_UINT16_T	uint16_t	585
MPI_INT32_T	int32_t	592
MPI_UINT32_T	uint32_t	593
MPI_INT64_T	int64_t	600
MPI_UINT64_T	uint64_t	601

¹ Treated as printable character.² Treated as integral value.³ Defined in <stddef.h>.

Named predefined datatypes

C type: MPI_Datatype Fortran type: INTEGER or TYPE(MPI_Datatype)	Fortran types	ABI value
MPI_LOGICAL	LOGICAL	536
MPI_INTEGER	INTEGER	537
MPI_REAL	REAL	538
MPI_COMPLEX	COMPLEX	539
MPI_DOUBLE_PRECISION	DOUBLE PRECISION	540
MPI_CHARACTER	CHARACTER(1)	542
MPI_AINT	INTEGER(KIND=MPI_ADDRESS_KIND)	513
MPI_COUNT	INTEGER(KIND=MPI_COUNT_KIND)	514
MPI_OFFSET	INTEGER(KIND=MPI_OFFSET_KIND)	515
MPI_BYTE	(any Fortran type)	583
MPI_PACKED	(any Fortran type)	519

Named predefined datatypes¹

C type: MPI_Datatype Fortran type: INTEGER or TYPE(MPI_Datatype)	C++ types	ABI value
MPI_CXX_FLOAT_COMPLEX	std::complex<float>	531
MPI_CXX_DOUBLE_COMPLEX	std::complex<double>	535
MPI_CXX_LONG_DOUBLE_COMPLEX	std::complex<long double>	549
MPI_CXX_BOOL	bool	569

¹ If an accompanying C++ compiler is missing, then the MPI datatypes in this table are not defined.

Optional datatypes (Fortran)

C type: MPI_Datatype Fortran type: INTEGER or TYPE(MPI_Datatype)	Fortran types	ABI value
MPI_DOUBLE_COMPLEX	DOUBLE COMPLEX	541
MPI_LOGICAL1	LOGICAL*1	704
MPI_LOGICAL2	LOGICAL*2	712
MPI_LOGICAL4	LOGICAL*4	720
MPI_LOGICAL8	LOGICAL*8	728
MPI_LOGICAL16	LOGICAL*16	736
MPI_INTEGER1	INTEGER*1	705
MPI_INTEGER2	INTEGER*2	713
MPI_INTEGER4	INTEGER*4	721
MPI_INTEGER8	INTEGER*8	729
MPI_INTEGER16	INTEGER*16	737
MPI_REAL2	REAL*2	714
MPI_REAL4	REAL*4	722
MPI_REAL8	REAL*8	730
MPI_REAL16	REAL*16	738
MPI_COMPLEX4	COMPLEX*4	723
MPI_COMPLEX8	COMPLEX*8	731
MPI_COMPLEX16	COMPLEX*16	739
MPI_COMPLEX32	COMPLEX*32	747

Datatypes for reduction functions (C)

C type: MPI_Datatype Fortran type: INTEGER or TYPE(MPI_Datatype)	ABI value
MPI_FLOAT_INT	552
MPI_DOUBLE_INT	553
MPI_LONG_INT	554
MPI_2INT	555
MPI_SHORT_INT	556
MPI_LONG_DOUBLE_INT	557

Datatypes for reduction functions (Fortran)

C type: MPI_Datatype Fortran type: INTEGER or TYPE(MPI_Datatype)	ABI value
MPI_2REAL	560
MPI_2DOUBLE_PRECISION	561
MPI_2INTEGER	562

Predefined message handles

C type: MPI_Message Fortran type: INTEGER or TYPE(MPI_Message)	ABI value
MPI_MESSAGE_NULL	296
MPI_MESSAGE_NO_PROC	297

Predefined queue handles

C type: MPI_Queue	ABI value
Fortran type: INTEGER or TYPE(MPI_Queue)	
MPI_QUEUE_NULL	312

Predefined error-handling specifiers

C type: MPI_Errhandler	ABI value
Fortran type: INTEGER or TYPE(MPI_Errhandler)	
MPI_ERRHANDLER_NULL	320
MPI_ERRORS_ARE_FATAL	321
MPI_ERRORS_ABORT	322
MPI_ERRORS_RETURN	323

Environmental inquiry info key

C type: MPI_Info	ABI value
Fortran type: INTEGER or TYPE(MPI_Info)	
MPI_INFO_NULL	304
MPI_INFO_ENV	305

Collective operators

C type: MPI_Op	ABI value
Fortran type: INTEGER or TYPE(MPI_Op)	
MPI_OP_NULL	32
MPI_SUM	33
MPI_MIN	34
MPI_MAX	35
MPI_PROD	36
MPI_BAND	40
MPI_BOR	41
MPI_BXOR	42
MPI_LAND	48
MPI_LOR	49
MPI_LXOR	50
MPI_MINLOC	56
MPI_MAXLOC	57
MPI_REPLACE	60
MPI_NO_OP	61

Predefined group handles

C type: MPI_Group	ABI value
Fortran type: INTEGER or TYPE(MPI_Group)	
MPI_GROUP_NULL	264
MPI_GROUP_EMPTY	265

Other predefined handles	
C/Fortran name	ABI value
C type / Fortran type	
MPI_REQUEST_NULL	
MPI_Request / INTEGER or TYPE(MPI_Request)	384
MPI_FILE_NULL	
MPI_File / INTEGER or TYPE(MPI_File)	280
MPI_SESSION_NULL	
MPI_Session / INTEGER or TYPE(MPI_Session)	288
MPI_WIN_NULL	
MPI_Win / INTEGER or TYPE(MPI_Win)	272

DRAFT

Predefined functions		
C/Fortran name		ABI value in <code>mpi.h</code>
C type		
/ Fortran type with <code>mpi</code> module		
/ Fortran type with <code>mpi_f08</code> module		
MPI_COMM_NULL_COPY_FN		
MPI_Comm_copy_attr_function		0
/ COMM_COPY_ATTR_FUNCTION		
/ PROCEDURE(MPI_Comm_copy_attr_function) ¹		
MPI_COMM_DUP_FN		
MPI_Comm_copy_attr_function		1
/ COMM_COPY_ATTR_FUNCTION		
/ PROCEDURE(MPI_Comm_copy_attr_function) ¹		
MPI_COMM_NULL_DELETE_FN		
MPI_Comm_delete_attr_function		0
/ COMM_DELETE_ATTR_FUNCTION		
/ PROCEDURE(MPI_Comm_delete_attr_function) ¹		
MPI_WIN_NULL_COPY_FN		
MPI_Win_copy_attr_function		0
/ WIN_COPY_ATTR_FUNCTION		
/ PROCEDURE(MPI_Win_copy_attr_function) ¹		
MPI_WIN_DUP_FN		
MPI_Win_copy_attr_function		1
/ WIN_COPY_ATTR_FUNCTION		
/ PROCEDURE(MPI_Win_copy_attr_function) ¹		
MPI_WIN_NULL_DELETE_FN		
MPI_Win_delete_attr_function		0
/ WIN_DELETE_ATTR_FUNCTION		
/ PROCEDURE(MPI_Win_delete_attr_function) ¹		
MPI_TYPE_NULL_COPY_FN		
MPI_Type_copy_attr_function		0
/ TYPE_COPY_ATTR_FUNCTION		
/ PROCEDURE(MPI_Type_copy_attr_function) ¹		
MPI_TYPE_DUP_FN		
MPI_Type_copy_attr_function		1
/ TYPE_COPY_ATTR_FUNCTION		
/ PROCEDURE(MPI_Type_copy_attr_function) ¹		
MPI_TYPE_NULL_DELETE_FN		
MPI_Type_delete_attr_function		0
/ TYPE_DELETE_ATTR_FUNCTION		
/ PROCEDURE(MPI_Type_delete_attr_function) ¹		
MPI_CONVERSION_FN_NULL		
MPI_Datarep_conversion_function		0
/ DATAREP_CONVERSION_FUNCTION		
/ PROCEDURE(MPI_Datarep_conversion_function) ¹		
MPI_CONVERSION_FN_NULL_C		
MPI_Datarep_conversion_function_c		0
/ (n/a)		
/ PROCEDURE(MPI_Datarep_conversion_function_c)		

¹ See the advice to implementors (on page 362) and advice to users (on page 363) on the predefined Fortran functions `MPI_COMM_NULL_COPY_FN`, ... in Section 7.7.2.

Deprecated predefined functions

C/Fortran name C type / Fortran type with mpi module	ABI value
MPI_NULL_COPY_FN MPI_Copy_function / COPY_FUNCTION	0
MPI_DUP_FN MPI_Copy_function / COPY_FUNCTION	1
MPI_NULL_DELETE_FN MPI_Delete_function / DELETE_FUNCTION	0

C constants specifying ignored input (no Fortran)

C constant (type: MPI_Fint*)	is equivalent to the Fortran constant
MPI_F_STATUSES_IGNORE	MPI_STATUSES_IGNORE in mpi / mpif.h
MPI_F_STATUS_IGNORE	MPI_STATUS_IGNORE in mpi / mpif.h
C constant (type: MPI_F08_status*)	is equivalent to the Fortran constant
MPI_F08_STATUSES_IGNORE	MPI_STATUSES_IGNORE in mpi_f08
MPI_F08_STATUS_IGNORE	MPI_STATUS_IGNORE in mpi_f08

C preprocessor constants and Fortran parameters

C type: C-preprocessor macro that expands to an int value Fortran type: INTEGER	ABI value
MPI_VERSION	N/A
MPI_SUBVERSION	N/A
MPI_ABI_VERSION	1
MPI_ABI_SUBVERSION	0

The MPI API version constants change with every release of the standard and are thus not constants in the ABI. The MPI ABI subversion will increment with every release of the standard, unless there is a breaking change, in which case the ABI version will increment and the subversion will reset to zero.

Handles used in the MPI tool information interface

Null Handles		
Handle	Type	ABI value
MPI_T_ENUM_NULL	MPI_T_enum	0
MPI_T_CVAR_HANDLE_NULL	MPI_T_cvar_handle	0
MPI_T_PVAR_HANDLE_NULL	MPI_T_pvar_handle	0
MPI_T_PVAR_SESSION_NULL	MPI_T_pvar_session	0
Other Handles		
Handle	Type	ABI value
MPI_T_PVAR_ALL_HANDLES	MPI_T_pvar_handle	1

Verbosity levels in the MPI tool information interface

C type: integer constant expression of type int	ABI value
MPI_T_VERBOSITY_USER_BASIC	0x09
MPI_T_VERBOSITY_USER_DETAIL	0x0a
MPI_T_VERBOSITY_USER_ALL	0x0c
MPI_T_VERBOSITY_TUNER_BASIC	0x11
MPI_T_VERBOSITY_TUNER_DETAIL	0x12
MPI_T_VERBOSITY_TUNER_ALL	0x14
MPI_T_VERBOSITY_MPIDEV_BASIC	0x21
MPI_T_VERBOSITY_MPIDEV_DETAIL	0x22
MPI_T_VERBOSITY_MPIDEV_ALL	0x24

**Constants to identify associations of variables
in the MPI tool information interface**

C type: integer constant expression of type int	ABI value
MPI_T_BIND_NO_OBJECT	1
MPI_T_BIND_MPI_COMM	2
MPI_T_BIND_MPI_DATATYPE	3
MPI_T_BIND_MPI_ERRHANDLER	4
MPI_T_BIND_MPI_FILE	5
MPI_T_BIND_MPI_GROUP	6
MPI_T_BIND_MPI_OP	7
MPI_T_BIND_MPI_REQUEST	8
MPI_T_BIND_MPI_WIN	9
MPI_T_BIND_MPI_MESSAGE	10
MPI_T_BIND_MPI_INFO	11
MPI_T_BIND_MPI_SESSION	12

**Constants describing the scope of a control variable
in the MPI tool information interface**

C type: integer constant expression of type int	ABI value
MPI_T_SCOPE_CONSTANT	1
MPI_T_SCOPE_READONLY	2
MPI_T_SCOPE_LOCAL	3
MPI_T_SCOPE_GROUP	4
MPI_T_SCOPE_GROUP_EQ	5
MPI_T_SCOPE_ALL	6
MPI_T_SCOPE_ALL_EQ	7

**Performance variable classes used by the
MPI tool information interface**

C type: integer constant expression of type int	ABI value
<code>MPI_T_PVAR_CLASS_STATE</code>	1
<code>MPI_T_PVAR_CLASS_LEVEL</code>	2
<code>MPI_T_PVAR_CLASS_SIZE</code>	3
<code>MPI_T_PVAR_CLASS_PERCENTAGE</code>	4
<code>MPI_T_PVAR_CLASS_HIGHWATERMARK</code>	5
<code>MPI_T_PVAR_CLASS_LOWWATERMARK</code>	6
<code>MPI_T_PVAR_CLASS_COUNTER</code>	7
<code>MPI_T_PVAR_CLASS_AGGREGATE</code>	8
<code>MPI_T_PVAR_CLASS_TIMER</code>	9
<code>MPI_T_PVAR_CLASS_GENERIC</code>	10

**Source event ordering guarantees in the
MPI tool information interface**

C type: <code>MPI_T_source_order</code>	ABI value
<code>MPI_T_SOURCE_ORDERED</code>	1
<code>MPI_T_SOURCE_UNORDERED</code>	2

**Callback safety requirement levels used in the
MPI tool information interface**

C type: <code>MPI_T_cb_safety</code>	ABI value
<code>MPI_T_CB_REQUIRE_NONE</code>	0x00
<code>MPI_T_CB_REQUIRE_MPI_RESTRICTED</code>	0x03
<code>MPI_T_CB_REQUIRE_THREAD_SAFE</code>	0x0F
<code>MPI_T_CB_REQUIRE_ASYNC_SIGNAL_SAFE</code>	0x3F

A.1.2 Types

The following are defined C type definitions included in the file `mpi.h`.

```
/* C opaque types */
```

```
MPI_Aint
```

```
MPI_Count
```

```
MPI_Fint
```

```
MPI_Offset
```

```
MPI_Status
```

```
MPI_F08_status
```

```
/* C handles to assorted structures */
```

```
MPI_Comm
```

```
MPI_Datatype
```

```
MPI_Errhandler
```

```
MPI_File
```

```
MPI_Group
```

```
MPI_Info
```

```
MPI_Message
```

```
MPI_Op
```

```

1  MPI_Queue
2  MPI_Request
3  MPI_Session
4  MPI_Win
5
6  /* Types for the MPI_T interface */
7  MPI_T_enum
8  MPI_T_cvar_handle
9  MPI_T_pvar_handle
10 MPI_T_pvar_session
11 MPI_T_event_instance
12 MPI_T_event_registration
13 MPI_T_source_order
14 MPI_T_cb_safety
15
16

```

The following are defined Fortran type definitions included in the `mpi_f08` and `mpi` modules.

```

19 ! Fortran opaque types in the mpi_f08 and mpi modules
20 TYPE(MPI_Status)
21

```

```

22 ! Fortran handles in the mpi_f08 and mpi modules
23 TYPE(MPI_Comm)
24 TYPE(MPI_Datatype)
25 TYPE(MPI_Errhandler)
26 TYPE(MPI_File)
27 TYPE(MPI_Group)
28 TYPE(MPI_Info)
29 TYPE(MPI_Message)
30 TYPE(MPI_Op)
31 TYPE(MPI_Queue)
32 TYPE(MPI_Request)
33 TYPE(MPI_Session)
34 TYPE(MPI_Win)
35

```

A.1.3 Prototype Definitions

C Bindings

The following are defined C typedefs for user-defined functions, also included in the file `mpi.h`.

```

42 /* prototypes for user-defined functions */
43
44 typedef void MPI_User_function(void *invec, void *inoutvec, int *len,
45                               MPI_Datatype *datatype);
46
47 typedef void MPI_User_function_c(void *invec, void *inoutvec, MPI_Count *len,
48                                 MPI_Datatype *datatype);

```



```

typedef int MPI_Comm_copy_attr_function(MPI_Comm oldcomm, int comm_keyval,
    void *extra_state, void *attribute_val_in,
    void *attribute_val_out, int *flag);
typedef int MPI_Comm_delete_attr_function(MPI_Comm comm, int comm_keyval,
    void *attribute_val, void *extra_state);
typedef int MPI_Win_copy_attr_function(MPI_Win oldwin, int win_keyval,
    void *extra_state, void *attribute_val_in,
    void *attribute_val_out, int *flag);
typedef int MPI_Win_delete_attr_function(MPI_Win win, int win_keyval,
    void *attribute_val, void *extra_state);
typedef int MPI_Type_copy_attr_function(MPI_Datatype oldtype, int type_keyval,
    void *extra_state, void *attribute_val_in,
    void *attribute_val_out, int *flag);
typedef int MPI_Type_delete_attr_function(MPI_Datatype datatype,
    int type_keyval, void *attribute_val, void *extra_state);
typedef void MPI_Comm_errhandler_function(MPI_Comm *comm, int *error_code,
    ...);
typedef void MPI_Win_errhandler_function(MPI_Win *win, int *error_code, ...);
typedef void MPI_File_errhandler_function(MPI_File *file, int *error_code,
    ...);
typedef void MPI_Session_errhandler_function(MPI_Session *session,
    int *error_code, ...);
typedef int MPI_Grequest_query_function(void *extra_state, MPI_Status *status);
typedef int MPI_Grequest_free_function(void *extra_state);
typedef int MPI_Grequest_cancel_function(void *extra_state, int complete);
typedef int MPI_Datarep_extents_function(MPI_Datatype datatype,
    MPI_Aint *extents, void *extra_state);
typedef int MPI_Datarep_conversion_function(void *userbuf,
    MPI_Datatype datatype, int count, void *filebuf,
    MPI_Offset position, void *extra_state);
typedef int MPI_Datarep_conversion_function_c(void *userbuf,
    MPI_Datatype datatype, MPI_Count count, void *filebuf,
    MPI_Offset position, void *extra_state);
typedef void MPI_T_event_cb_function(MPI_T_event_instance event_instance,
    MPI_T_event_registration event_registration,
    MPI_T_cb_safety cb_safety, void *user_data);
typedef void MPI_T_event_free_cb_function(
    MPI_T_event_registration event_registration,
    MPI_T_cb_safety cb_safety, void *user_data);
typedef void MPI_T_event_dropped_cb_function(MPI_Count count,
    MPI_T_event_registration event_registration, int source_index,
    MPI_T_cb_safety cb_safety, void *user_data);

```

Fortran 2008 Bindings with the mpi_f08 Module

The callback prototypes when using the Fortran `mpi_f08` module are shown below:

The user-function argument to `MPI_Op_create` and `MPI_Op_create_c` should be declared according to:

ABSTRACT INTERFACE

```
SUBROUTINE MPI_User_function(invec, inoutvec, len, datatype)
  USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
  TYPE(C_PTR), VALUE :: invec, inoutvec
  INTEGER :: len
  TYPE(MPI_Datatype) :: datatype
```

ABSTRACT INTERFACE

```
SUBROUTINE MPI_User_function_c(invec, inoutvec, len, datatype) !(_c)
  USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
  TYPE(C_PTR), VALUE :: invec, inoutvec
  INTEGER(KIND=MPI_COUNT_KIND) :: len
  TYPE(MPI_Datatype) :: datatype
```

The copy and delete function arguments to `MPI_Comm_create_keyval` should be declared according to:

ABSTRACT INTERFACE

```
SUBROUTINE MPI_Comm_copy_attr_function(oldcomm, comm_keyval, extra_state,
  attribute_val_in, attribute_val_out, flag, ierror)
  TYPE(MPI_Comm) :: oldcomm
  INTEGER :: comm_keyval, ierror
  INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
  attribute_val_out
  LOGICAL :: flag
```

ABSTRACT INTERFACE

```
SUBROUTINE MPI_Comm_delete_attr_function(comm, comm_keyval, attribute_val,
  extra_state, ierror)
  TYPE(MPI_Comm) :: comm
  INTEGER :: comm_keyval, ierror
  INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
```

The copy and delete function arguments to `MPI_Win_create_keyval` should be declared according to:

ABSTRACT INTERFACE

```
SUBROUTINE MPI_Win_copy_attr_function(oldwin, win_keyval, extra_state,
  attribute_val_in, attribute_val_out, flag, ierror)
  TYPE(MPI_Win) :: oldwin
  INTEGER :: win_keyval, ierror
  INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
  attribute_val_out
  LOGICAL :: flag
```

ABSTRACT INTERFACE

```
SUBROUTINE MPI_Win_delete_attr_function(win, win_keyval, attribute_val,
  extra_state, ierror)
  TYPE(MPI_Win) :: win
```

```
INTEGER :: win_keyval, ierror
```

```
INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
```

The copy and delete function arguments to `MPI_Type_create_keyval` should be declared according to:

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Type_copy_attr_function(oldtype, type_keyval, extra_state,  
    attribute_val_in, attribute_val_out, flag, ierror)
```

```
  TYPE(MPI_Datatype) :: oldtype
```

```
  INTEGER :: type_keyval, ierror
```

```
  INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,  
    attribute_val_out
```

```
  LOGICAL :: flag
```

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Type_delete_attr_function(datatype, type_keyval,  
    attribute_val, extra_state, ierror)
```

```
  TYPE(MPI_Datatype) :: datatype
```

```
  INTEGER :: type_keyval, ierror
```

```
  INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
```

The handler-function argument to `MPI_Comm_create_errhandler` should be declared like this:

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Comm_errhandler_function(comm, error_code)
```

```
  TYPE(MPI_Comm) :: comm
```

```
  INTEGER :: error_code
```

The handler-function argument to `MPI_Win_create_errhandler` should be declared like this:

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Win_errhandler_function(win, error_code)
```

```
  TYPE(MPI_Win) :: win
```

```
  INTEGER :: error_code
```

The handler-function argument to `MPI_File_create_errhandler` should be declared like this:

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_File_errhandler_function(file, error_code)
```

```
  TYPE(MPI_File) :: file
```

```
  INTEGER :: error_code
```

The handler-function argument to `MPI_Session_create_errhandler` should be declared like this:

```
ABSTRACT INTERFACE
```

```
  SUBROUTINE MPI_Session_errhandler_function(session, error_code)
```

```
  TYPE(MPI_Session) :: session
```

```
  INTEGER :: error_code
```

The query, free, and cancel function arguments to `MPI_Grequest_start` should be declared according to:

```
ABSTRACT INTERFACE
```

```

1      SUBROUTINE MPI_Grequest_query_function(extra_state, status, ierror)
2          INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
3          TYPE(MPI_Status) :: status
4          INTEGER :: ierror
5
6      ABSTRACT INTERFACE
7          SUBROUTINE MPI_Grequest_free_function(extra_state, ierror)
8              INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
9              INTEGER :: ierror
10
11     ABSTRACT INTERFACE
12         SUBROUTINE MPI_Grequest_cancel_function(extra_state, complete, ierror)
13             INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
14             LOGICAL :: complete
15             INTEGER :: ierror
16
17     The extent and conversion function arguments to MPI_Register_datatype and
18     MPI_Register_datatype_c should be declared according to:
19     ABSTRACT INTERFACE
20         SUBROUTINE MPI_Datatype_extent_function(datatype, extent, extra_state, ierror)
21             TYPE(MPI_Datatype) :: datatype
22             INTEGER(KIND=MPI_ADDRESS_KIND) :: extent, extra_state
23             INTEGER :: ierror
24
25     ABSTRACT INTERFACE
26         SUBROUTINE MPI_Datatype_conversion_function(userbuf, datatype, count, filebuf,
27             position, extra_state, ierror)
28             USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
29             TYPE(C_PTR), VALUE :: userbuf, filebuf
30             TYPE(MPI_Datatype) :: datatype
31             INTEGER :: count, ierror
32             INTEGER(KIND=MPI_OFFSET_KIND) :: position
33             INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
34
35     ABSTRACT INTERFACE
36         SUBROUTINE MPI_Datatype_conversion_function_c(userbuf, datatype, count,
37             filebuf, position, extra_state, ierror) !(_c)
38             USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
39             TYPE(C_PTR), VALUE :: userbuf, filebuf
40             TYPE(MPI_Datatype) :: datatype
41             INTEGER(KIND=MPI_COUNT_KIND) :: count
42             INTEGER(KIND=MPI_OFFSET_KIND) :: position
43             INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
44             INTEGER :: ierror

```

Fortran Bindings with mpif.h or the mpi Module

With the Fortran `mpi` module or (deprecated) `mpif.h`, here are examples of how each of the user-defined subroutines should be declared.

The user-function argument to `MPI_OP_CREATE` should be declared like this:

```

SUBROUTINE USER\_FUNCTION(INVEC, INOUTVEC, LEN, DATATYPE)
  <type> INVEC(LEN), INOUTVEC(LEN)
  INTEGER LEN, DATATYPE

```

The copy and delete function arguments to [MPI_COMM_CREATE_KEYVAL](#) should be declared like these:

```

SUBROUTINE COMM\_COPY\_ATTR\_FUNCTION(OLDCOMM, COMM_KEYVAL, EXTRA_STATE,
  ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT, FLAG, IERROR)
  INTEGER OLDCOMM, COMM_KEYVAL, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
  ATTRIBUTE_VAL_OUT
  LOGICAL FLAG

```

```

SUBROUTINE COMM\_DELETE\_ATTR\_FUNCTION(COMM, COMM_KEYVAL, ATTRIBUTE_VAL,
  EXTRA_STATE, IERROR)
  INTEGER COMM, COMM_KEYVAL, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE

```

The copy and delete function arguments to [MPI_WIN_CREATE_KEYVAL](#) should be declared like these:

```

SUBROUTINE WIN\_COPY\_ATTR\_FUNCTION(OLDWIN, WIN_KEYVAL, EXTRA_STATE,
  ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT, FLAG, IERROR)
  INTEGER OLDWIN, WIN_KEYVAL, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
  ATTRIBUTE_VAL_OUT
  LOGICAL FLAG

```

```

SUBROUTINE WIN\_DELETE\_ATTR\_FUNCTION(WIN, WIN_KEYVAL, ATTRIBUTE_VAL,
  EXTRA_STATE, IERROR)
  INTEGER WIN, WIN_KEYVAL, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE

```

The copy and delete function arguments to [MPI_TYPE_CREATE_KEYVAL](#) should be declared like these:

```

SUBROUTINE TYPE\_COPY\_ATTR\_FUNCTION(OLDTYPE, TYPE_KEYVAL, EXTRA_STATE,
  ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT, FLAG, IERROR)
  INTEGER OLDTYPE, TYPE_KEYVAL, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
  ATTRIBUTE_VAL_OUT
  LOGICAL FLAG

```

```

SUBROUTINE TYPE\_DELETE\_ATTR\_FUNCTION(DATATYPE, TYPE_KEYVAL, ATTRIBUTE_VAL,
  EXTRA_STATE, IERROR)
  INTEGER DATATYPE, TYPE_KEYVAL, IERROR
  INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE

```

The handler-function argument to [MPI_COMM_CREATE_ERRHANDLER](#) should be declared like this:

```

SUBROUTINE COMM\_ERRHANDLER\_FUNCTION(COMM, ERROR_CODE)
  INTEGER COMM, ERROR_CODE

```

The handler-function argument to [MPI_WIN_CREATE_ERRHANDLER](#) should be declared like this:

```

1  SUBROUTINE WIN_ERRHANDLER_FUNCTION(WIN, ERROR_CODE)
2      INTEGER WIN, ERROR_CODE

```

The handler-function argument to `MPI_FILE_CREATE_ERRHANDLER` should be declared like this:

```

5  SUBROUTINE FILE_ERRHANDLER_FUNCTION(FILE, ERROR_CODE)
6      INTEGER FILE, ERROR_CODE

```

The handler-function argument to `MPI_SESSION_CREATE_ERRHANDLER` should be declared like this:

```

10 SUBROUTINE SESSION_ERRHANDLER_FUNCTION(SESSION, ERROR_CODE)
11     INTEGER SESSION, ERROR_CODE

```

The query, free, and cancel function arguments to `MPI_GREQUEST_START` should be declared like these:

```

14 SUBROUTINE GREQUEST_QUERY_FUNCTION(EXTRA_STATE, STATUS, IERROR)
15     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
16     INTEGER STATUS(MPI_STATUS_SIZE), IERROR

```

```

18 SUBROUTINE GREQUEST_FREE_FUNCTION(EXTRA_STATE, IERROR)
19     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
20     INTEGER IERROR

```

```

21 SUBROUTINE GREQUEST_CANCEL_FUNCTION(EXTRA_STATE, COMPLETE, IERROR)
22     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
23     LOGICAL COMPLETE
24     INTEGER IERROR

```

The extent and conversion function arguments to `MPI_REGISTER_DATAREP` should be declared like these:

```

27 SUBROUTINE DATAREP_EXTENT_FUNCTION(DATATYPE, EXTENT, EXTRA_STATE, IERROR)
28     INTEGER DATATYPE, IERROR
29     INTEGER(KIND=MPI_ADDRESS_KIND) EXTENT, EXTRA_STATE

```

```

31 SUBROUTINE DATAREP_CONVERSION_FUNCTION(USERBUF, DATATYPE, COUNT, FILEBUF,
32     POSITION, EXTRA_STATE, IERROR)
33     <TYPE> USERBUF(*), FILEBUF(*)
34     INTEGER DATATYPE, COUNT, IERROR
35     INTEGER(KIND=MPI_OFFSET_KIND) POSITION
36     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE

```

A.1.4 Deprecated Prototype Definitions

The following are defined C typedefs for deprecated user-defined functions, also included in the file `mpi.h`.

```

43 /* prototypes for user-defined functions */

```

```

44 typedef int MPI_Copy_function(MPI_Comm oldcomm, int keyval, void *extra_state,
45     void *attribute_val_in, void *attribute_val_out, int *flag);

```

```

47 typedef int MPI_Delete_function(MPI_Comm comm, int keyval, void *attribute_val,
48     void *extra_state);

```

The following are deprecated Fortran user-defined callback subroutine prototypes. The deprecated copy and delete function arguments to `MPI_KEYVAL_CREATE` should be declared like these:

```
SUBROUTINE COPY_FUNCTION(OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,  
    ATTRIBUTE_VAL_OUT, FLAG, IERR)  
    INTEGER OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT,  
    IERR  
    LOGICAL FLAG  
SUBROUTINE DELETE_FUNCTION(COMM, KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERR)  
    INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERR
```

A.1.5 String Values

Default Communicator Names

The following default communicator names are defined by MPI.

```
"MPI_COMM_WORLD"  
"MPI_COMM_SELF"  
"MPI_COMM_PARENT"  
"MPI_COMM_NULL"
```

Default Datatype Names

Named predefined datatypes have the default names of the datatype name. In addition, the following default datatype name is defined by MPI.

```
"MPI_DATATYPE_NULL"
```

Default Window Names

The following default window name is defined by MPI.

```
"MPI_WIN_NULL"
```

Reserved Data Representations

The following data representations are supported by MPI.

```
"native"  
"internal"  
"external32"
```

Process Set Names

Process set name	Comment
"mpi://"	reserved namespace
"mpi://SELF"	mandatory process set name
"mpi://WORLD"	mandatory process set name

Info Keys

The following info keys are reserved. They are strings.

"access_style"
"accumulate_ops"
"accumulate_ordering"
"alloc_shared_noncontig"
"appnum"
"arch"
"argv"
"cb_block_size"
"cb_buffer_size"
"cb_nodes"
"chunked"
"chunked_item"
"chunked_size"
"collective_buffering"
"command"
"file"
"file_perm"
"filename"
"host"
"io_node_list"
"ip_address"
"ip_port"
"maxprocs"
"mpi_accumulate_granularity"
"mpi_aint_size"
"mpi_assert_allow_overtaking"
"mpi_assert_exact_length"
"mpi_assert_memory_alloc_kinds"
"mpi_assert_no_any_source"
"mpi_assert_no_any_tag"
"mpi_complex4_supported"
"mpi_complex8_supported"
"mpi_complex16_supported"
"mpi_complex32_supported"
"mpi_count_size"
"mpi_double_complex_supported"
"mpi_double_precision_size"
"mpi_hw_resource_type"
"mpi_initial_errhandler"
"mpi_integer_size"
"mpi_integer1_supported"
"mpi_integer2_supported"
"mpi_integer4_supported"
"mpi_integer8_supported"
"mpi_integer16_supported"

"mpi_logical_size"	1
"mpi_logical1_supported"	2
"mpi_logical2_supported"	3
"mpi_logical4_supported"	4
"mpi_logical8_supported"	5
"mpi_logical16_supported"	6
"mpi_memory_alloc_kinds"	7
"mpi_minimum_memory_alignment"	8
"mpi_offset_size"	9
"mpi_pset_name"	10
"mpi_size"	11
"mpi_real_size"	12
"mpi_real2_supported"	13
"mpi_real4_supported"	14
"mpi_real8_supported"	15
"mpi_real16_supported"	16
"nb_proc"	17
"no_locks"	18
"num_io_nodes"	19
"path"	20
"same_disp_unit"	21
"same_size"	22
"soft"	23
"striping_factor"	24
"striping_unit"	25
"thread_level"	26
"wdir"	27

Info Values

The following info values are reserved. They are strings.

"alloc_mem"	30
"false"	31
"mpi"	32
"mpi_errors_abort"	33
"mpi_errors_are_fatal"	34
"mpi_errors_return"	35
"mpi_shared_memory"	36
"MPI_THREAD_FUNNELED"	37
"MPI_THREAD_MULTIPLE"	38
"MPI_THREAD_SERIALIZED"	39
"MPI_THREAD_SINGLE"	40
"none"	41
"random"	42
"rar"	43
"raw"	44
"read_mostly"	45
"read_once"	46

```

1  "reverse_sequential"
2  "same_op"
3  "same_op_no_op"
4  "sequential"
5  "system"
6  "true"
7  "war"
8  "waw"
9  "win_allocate"
10 "win_allocate_shared"
11 "write_mostly"
12 "write_once"

```

A.2 Summary of the Semantics of all Operation-Related MPI Procedures

This annex provides the list of MPI procedures that are associated with an MPI operation, or inquiry procedures providing information about an operation.

In many cases, the MPI procedures and their properties are listed under certain constraints, e.g., a call to `MPI_WAIT` that completes either a nonblocking or a persistent operation, or RMA calls in combination with various synchronization methods.

Table Legend:

- **Stages:** i=initialization, s=starting, c=completion, f=freeing. The procedure does at least part of the indicated stage(s).
- **Cpl:** ic=incomplete procedure, c=completing procedure, f=freeing procedure
- **Loc:** l=local procedure, nl=non-local procedure
- *: exceptions, e.g., ic+nl = incomplete+non-local, and c+l = completing+local (both are defined as blocking)
- **Blk:** b=blocking procedure, nb=nonblocking procedure. Note that from a user's view point, this column is only a hint. Relevant is, whether a routine is local or not and which resources are blocked until when. See both previous and last columns.
- ‡: exceptions, e.g., nonblocking procedures without prefix l, or that prefix l only marks immediate return.
- **Op:** part of operation type: b-op = blocking operation, nb-op = nonblocking operation, p-op = persistent operation, pp-op = persistent partitioned operation
- **Collective procedures:**
 - C = all processes of the group must call the procedure
 - sq = in the same sequence
 - S1 = blocking synchronization, i.e., no process shall return from this procedure until all processes on the associated process group called this procedure
 - W1 = the implementation is permitted to do S1 but not required to do S1
 - S2 = start-complete-synchronization, i.e., no process shall complete the associated operation until all processes on the associated process group have called the associated starting procedure

- W2 = the implementation is permitted to do S2 but not required to do S2
- **Blocked resources:** They are blocked after the call until the end of the subsequent stage where this resource is not mentioned further in the table.

Table Remarks:

1. Must not return before the corresponding MPI receive operation is started.
2. Not related to an MPI operation. Prior to MPI-4.0, `MPI_PROBE` and `MPI_IProbe` were also described as blocking and nonblocking. From MPI-4.0 onwards, only non-local and local are used to describe these procedures.
3. Usually, `MPI_WAIT` is non-local, but in this case it is local.
4. In case of a `MPI_(I)BARRIER` on an intra-communicator, the S1/S2 synchronization is required (instead of W1/W2).
5. Collective: all processes must complete, but with the free choice of using `MPI_WAIT` or `MPI_TEST` returning `flag = TRUE`.
6. It also may not return until `MPI_INIT` was called in the children.
7. Addresses are cached on the request handle.
8. One of the rare cases that an incomplete call is non-local and therefore blocking.
9. One shall not free or deallocate the buffer before the operation is freed, that is `MPI_REQUEST_FREE` returned.
10. For `MPI_WAIT` and `MPI_TEST`, see corresponding lines for a) `MPI_BSEND`, or b) `MPI_IBCAST`.
11. The prefix `l` marks only that this procedure returns immediately.
12. One of the exceptions that a completing and therefore blocking operation-related procedure is local.
13. `MPI_(I)MProbe` initializes the operation through generating the message handle whereas `MPI_(I)MRECV` initializes the receive buffer (i.e., two MPI procedures together implement the initialization stage).
14. Nonblocking procedure without an `l` prefix.
15. Initialization stage (“i”) only if `flag = TRUE` is returned else no operation is progressed.
16. Collective: all processes must start, but with the free choice of using `MPI_START` or `MPI_STARTALL` for a given persistent request handle (i.e., if one process starts a persistent request handle then all processes of the associated process group must start their corresponding request handle, and if any process starts then all processes must complete their handles).
17. In a correct MPI program, a call to `MPI_(I)RSEND` requires that the receiver has already started the corresponding receive. Under this assumption, the call to `MPI_RSEND` and the call to `MPI_WAIT` with an (active) ready send request handle are local.
18. Based on their semantics, when called using an intra-communicator, `MPI_ALLGATHER`, `MPI_ALLTOALL`, and their V and W variants, `MPI_ALLREDUCE`, `MPI_REDUCE_SCATTER`, and `MPI_REDUCE_SCATTER_BLOCK` must synchronize (i.e., S1/S2 instead of W1/W2) provided that all counts and the size of all datatypes are larger than zero.
19. `MPI_COMM_FREE` may return before any pending communication has finished and the communicator is deallocated. In contrast, `MPI_COMM_DISCONNECT` waits for pending communication to finish and deallocates the communicator before it returns.

20. The request handle is in the “active” state after `MPI_START`, i.e., `MPI_REQUEST_FREE` is now forbidden. But the starting stage is not yet finished, and the contents of the buffer are not yet “blocked.” An additional `MPI_PREADY` and variants `MPI_PREADY_RANGE` `MPI_PREADY_LIST` are required to activate each partition of the send buffer to finish the starting stage.
21. As part of the completion stage, the user is allowed to read part of the output buffer after returning from `MPI_PARRIVED` with `flag = TRUE` before completing the whole operation with a `MPI_WAIT/MPI_TEST` procedure.
22. It initializes the attached buffer as completely free.
23. It uses the attached buffer and performs all four stages on the send buffer. It occupies the needed part of the attached buffer.
24. It waits until the attached buffer is empty, i.e., all messages have been transmitted, and then releases the attached buffer.
25. Although in case of `flag = TRUE` the operation is completed, a subsequent call to test, wait, or free must be executed for deallocating or inactivating the request handle as final part of the stages c and f. It is listed only in this scenario, but can be used everywhere, where `MPI_TEST` can be called.
26. It frees the request handle. If the related communication operation is still ongoing then the completion and freeing stage can take place after the procedure returned.
27. Cancelling a send request is deprecated.
28. Can also be applied to activ persistent requests.
29. As an exception, `MPI_WAIT` is local and `MPI_TEST` repeatedly called will eventually return `flag=true`. The cancelled send or receive operation is completed and the buffer can be reused. Whether the message is sent out from the buffer or received in the buffer, this part of the completion stage is only executed if a subsequent `MPI_TEST_CANCELLED` for the returned status would return `flag = FALSE`. The freeing stage will be performed only for non-persistent requests.
30. In some cases, more than one MPI procedure may be needed to implement one stage of an MPI one-sided operation. For details on the semantics of one-sided operations, see Chapter 12.
31. Local completion only (at origin).
32. Local completion only (at target).
33. Completion at target and locally at origin.
34. Return from `MPI_WIN_START` and these subsequent procedures at the origin process may be delayed until `MPI_WIN_POST` has been called at the target process (see Section 12.5 and Example 12.4).
35. Return from `MPI_WIN_LOCK` and these subsequent procedures may be delayed until other origin processes have released their lock (see Section 12.5 and Example 12.5).
36. The init and freeing stages and the buffer address of the target window only apply to MPI processes in the role of a target of an RMA operation.
37. The freeing stage applies to operations only and does not apply to any request.
38. The same procedure call may serve different stages for different operations, i.e., the completion of a previous RMA and/or exposure epoch and/or the start of a next RMA and/or exposure epoch.
39. In addition to the completion and freeing of the RMA operations prior the the flush call (stages “c+f”), this call initializes the next RMA epoch (stage “i”).
40. The stages represent the invocation as part of an RMA operation. As collective procedure itself, it is a blocking procedure with all stages.

Procedure	Stages	Cpl	Loc	Blk	Op	Collective C sq S/W	Blocked resources and remarks
Chapter 3: Point-to-Point Communication							
MPI_SEND	i-s-c-f	c+f	nl	b	b-op	-	
MPI_SSEND	i-s-c-f	c+f	nl	b	b-op	-	1)
MPI_RSEND	i-s-c-f	c+f	l*	b	b-op	-	12)* 17)
MPI_BUFFER_ATTACH	i-----		l		b-op	-	attached buffer 22)
MPI_BSEND	i-s-c-f	c+f	l*	b	b-op	-	attached buffer 23) 12)*
MPI_BUFFER_DETACH	-----f	f	nl	b	b-op	-	24)
MPI_RECV	i-s-c-f	c+f	nl	b	b-op	-	
MPI_ISEND, MPI_ISSEND	i-s----	ic	l	nb	nb-op	-	buffer
MPI_IRECV	i-s----	ic	l	nb	nb-op	-	buffer
corresp. MPI_REQUEST_GET_STATUS ret. flag=FALSE	-----		l		nb-op	-	25)
corresp. MPI_REQUEST_GET_STATUS ret. flag=TRUE	-----c--		l		nb-op	-	25)
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op	-	buffer 7)
corresponding MPI_TEST returning flag=TRUE	-----c-f	c+f	l		nb-op	-	
corresponding MPI_WAIT	-----c-f	c+f	nl		nb-op	-	
corresponding MPI_REQUEST_FREE (for active request)	-----		l		nb-op	-	buffer 26)
MPI_IBSEND, MPI_IRSEND	i-s----	ic	l	nb	nb-op	-	buffer
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op	-	buffer 7)
corresponding MPI_TEST returning flag=TRUE	-----c-f	c+f	l		nb-op	-	
corresponding MPI_WAIT	-----c-f	c+f	l*		nb-op	-	3)* 17)
corresponding MPI_REQUEST_FREE (for active request)	-----		l		nb-op	-	buffer 26)
MPI_ISEND, MPI_ISSEND, MPI_IBSEND, MPI_IRSEND	i-s----	ic	l	nb	nb-op	-	buffer 27
MPI_IRECV	i-s----	ic	l	nb	nb-op	-	buffer
corresponding MPI_CANCEL	-----		l		nb-op	-	buffer 28)
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op	-	buffer 7)
corresponding MPI_TEST returning flag=TRUE	-----c-f	c+f	l		nb-op	-	29)
corresponding MPI_WAIT	-----c-f	c+f	l*		nb-op	-	29)
corresponding MPI_TEST_CANCELLED	-----		l		nb-op	-	
MPI_PROBE	-----	c	nl			-	2)
MPI_IPROBE	-----	c	l†			-	2) 11)†
MPI_MPROBE	i-----	ic	nl*	b	b-op	-	message 8)*
MPI_IMPROBE	i-----	ic	l	nb	b-op	-	message 15)
MPI_MRECV of a probed message	i-s-c-f	c+f	l	b	b-op	-	13)
MPI_IRECV of a probed message	i-s----	ic	l	nb	nb-op	-	buffer 13)
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op	-	buffer 7)
corresponding MPI_TEST returning flag=TRUE	-----c-f	c+f	l		nb-op	-	
corresponding MPI_WAIT	-----c-f	c+f	l		nb-op	-	
corresponding MPI_REQUEST_FREE (for active request)	-----		l		nb-op	-	buffer 26)
MPI_[S B R]SEND_INIT, MPI_RECV_INIT	i-----	ic	l	nb†	p-op	-	buffer address 9) 14)†
corresponding MPI_START, MPI_STARTALL	--s----	ic	l	nb†	p-op	-	buffer address+contents 7) 14)†
corresponding MPI_TEST returning flag=FALSE	-----		l		p-op	-	buffer address+contents 7)
corresponding MPI_TEST returning flag=TRUE	-----c--	c	l		p-op	-	buffer address 7) 9)
corresponding MPI_WAIT (for MPI_(B R)SEND_INIT req.)	-----c--	c	l*		p-op	-	buffer address 3)* 7) 9) 17)
corresponding MPI_WAIT (for other request)	-----c--	c	nl		p-op	-	buffer address 7) 9)
corresponding MPI_REQUEST_FREE (for active request)	-----		l		nb-op	-	buffer 26)
corresponding MPI_REQUEST_FREE (for inactive request)	-----f	f	l		p-op	-	
MPI_CANCEL of nonblocking/persistent pt-to-pt			l		p-op	-	
MPI_SENDRECV[_REPLACE]	i-s-c-f	c+f	nl	b	b-op	-	
MPI_ISENDRECV[_REPLACE]	i-s----	ic	l	nb	nb-op	-	buffer
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op	-	buffer 7)
corresponding MPI_TEST returning flag=TRUE	-----c-f	c+f	l		nb-op	-	
corresponding MPI_WAIT	-----c-f	c+f	nl		nb-op	-	
corresponding MPI_REQUEST_FREE (for active request)	-----		l		nb-op	-	buffer 26)
Chapter 4: Partitioned Point-to-Point Communication							
MPI_PSEND_INIT	i-----	ic	l	nb†	pp-op	-	buffer address 9) 14)†
corresponding MPI_START, MPI_STARTALL	--s----	ic	l	nb†	pp-op	-	buffer address 7) 9) 14)† 20)
corresponding MPI_PREADY and variants	--s----	ic	l	nb†	pp-op	-	buffer address+contents 7) 9) 14)† 20)
corresponding MPI_TEST returning flag=FALSE	-----		l		pp-op	-	buffer address+contents 7) 9)
corresponding MPI_TEST returning flag=TRUE	-----c--	c	l		pp-op	-	buffer address 7) 9)
corresponding MPI_WAIT	-----c--	c	nl		pp-op	-	buffer address 7) 9)
corresponding MPI_REQUEST_FREE (for inactive request)	-----f	f	l		pp-op	-	
MPI_PRCV_INIT	i-----	ic	l	nb†	pp-op	-	buffer address 9) 14)†
corresponding MPI_START, MPI_STARTALL	--s----	ic	l	nb†	pp-op	-	buffer address+contents 7) 9) 14)†
MPI_PARRIVED returning flag=TRUE	-----c--	ic	l		pp-op	-	buffer address+contents 7) 9) 21)
MPI_PARRIVED returning flag=FALSE	-----		l		pp-op	-	buffer address+contents 7) 9)
corresponding MPI_TEST returning flag=FALSE	-----		l		pp-op	-	buffer address+contents 7) 9)
corresponding MPI_TEST returning flag=TRUE	-----c--	c	l		pp-op	-	buffer address 7) 9) 21)
corresponding MPI_WAIT	-----c--	c	nl		pp-op	-	buffer address 7) 9) 21)
corresponding MPI_REQUEST_FREE (for inactive request)	-----f	f	l		pp-op	-	

Procedure	Stages	Cpl	Loc	Blk	Op	Collective			Blocked resources and remarks	
Chapter 6: Collective Communication										
MPI_BCAST, MPI_BARRIER, MPI_GATHER, MPI_GATHERV, MPI_SCATTER, MPI_SCATTERV, MPI_ALLGATHER, MPI_ALLGATHERV, MPI_ALLTOALL, MPI_ALLTOALLV, MPI_ALLTOALLW, MPI_REDUCE, MPI_ALLREDUCE, MPI_REDUCE_SCATTER_BLOCK, MPI_REDUCE_SCATTER, MPI_SCAN, MPI_EXSCAN	i-s-c-f	c+f	nl	b	b-op	C	sq	W1	4) 18)	
MPI_IBCAST, MPI_IBARRIER, MPI_IGATHER, MPI_ISCATTER, MPI_IALLGATHER, MPI_IALLTOALL, MPI_IREDUCE, MPI_IALLREDUCE, MPI_IREDUCE_SCATTER_BLOCK, MPI_ISCAN, MPI_IEXSCAN	i-s----	ic	l	nb	nb-op	C	sq		buffer	
MPI_IGATHERV, MPI_ISCATTERV, MPI_IALLGATHERV, MPI_IALLTOALLV, MPI_IALLTOALLW, MPI_IREDUCE_SCATTER	i-s----	ic	l	nb	nb-op	C	sq		buffer, array arguments	
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op				buffer, array arguments 7)	
corresponding MPI_TEST returning flag=TRUE	---c-f	c+f	l		nb-op	C		W2	4) 5) 18)	
corresponding MPI_WAIT	---c-f	c+f	nl		nb-op	C		W2	4) 5) 18)	
MPI_BCAST_INIT, MPI_BARRIER_INIT, MPI_GATHER_INIT, MPI_SCATTER_INIT, MPI_ALLGATHER_INIT, MPI_ALLTOALL_INIT, MPI_REDUCE_INIT, MPI_ALLREDUCE_INIT, MPI_REDUCE_SCATTER_BLOCK_INIT, MPI_SCAN_INIT, MPI_EXSCAN_INIT	i-----	ic	nl*	b	p-op	C	sq	W1	buffer address 8)* 9)	
MPI_GATHERV_INIT, MPI_SCATTERV_INIT, MPI_ALLGATHERV_INIT, MPI_ALLTOALLV_INIT, MPI_ALLTOALLW_INIT, MPI_REDUCE_SCATTER_INIT	i-----	ic	nl*	b	p-op	C	sq	W1	buffer address, array arguments 8)* 9)	
corresponding MPI_START, MPI_STARTALL	--s----	ic	l	nb [†]	p-op	C			buf.addr.+contents 7) 14) [†] 16)	
corresponding MPI_TEST returning flag=FALSE	-----		l		p-op				buf addr+contents & arr-args 7)	
corresponding MPI_TEST returning flag=TRUE	---c--	c	l		p-op	C		W2	buf-addr & arr-args 4) 5) 7) 9) 18)	
corresponding MPI_WAIT	---c--	c	nl		p-op	C		W2	buffer address and array arguments cached on the request handle 4) 5) 7) 9) 18)	
corresponding MPI_REQUEST_FREE	-----f	f	l		p-op					
Chapter 7: Groups, Contexts, Communicators, and Caching										
MPI_COMM_CREATE, MPI_COMM_DUP, MPI_COMM_DUP_WITH_INFO, MPI_COMM_SPLIT, MPI_COMM_SPLIT_TYPE, MPI_COMM_SET_INFO	i-s-c-f	c+f	nl	b	b-op	C	sq	W1	coll. over comm arg.	
MPI_COMM_CREATE_GROUP	i-s-c-f	c+f	nl	b	b-op	C	sq	W1	coll. over group arg.	
MPI_INTERCOMM_CREATE, MPI_INTERCOMM_MERGE	i-s-c-f	c+f	nl	b	b-op	C	sq	W1	coll. over union of local & remote group	
MPI_COMM_IDUP	i-s----	ic	l	nb	nb-op	C	sq		communicator handle	
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op					
corresponding MPI_TEST returning flag=TRUE	---c-f	c+f	l		nb-op	C		W2	5)	
corresponding MPI_WAIT	---c-f	c+f	nl		nb-op	C		W2	5)	
MPI_COMM_FREE	i-s----	ic	nl	b	nb-op	C	sq	W1	19)	
MPI_COMM_DISCONNECT	i-s-c-f	c+f	nl	b	b-op	C	sq	W1	19)	

Procedure	Stages	Cpl	Loc	Blk	Op	Collective C sq S/W	Blocked resources and remarks
Chapter 8: Process Topologies							
MPI_CART_CREATE, MPI_CART_SUB, MPI_GRAPH_CREATE, MPI_DIST_GRAPH_CREATE_ADJACENT, MPI_DIST_GRAPH_CREATE	i-s-c-f	c	nl	b	b-op	C sq W1	coll. over comm arg.
MPI_NEIGHBOR_ALLGATHER, MPI_NEIGHBOR_ALLGATHERV, MPI_NEIGHBOR_ALLTOALL, MPI_NEIGHBOR_ALLTOALLV, MPI_NEIGHBOR_ALLTOALLW	i-s-c-f	c+f	nl	b	b-op	C sq W1	18)
MPI_INEIGHBOR_ALLGATHER, MPI_INEIGHBOR_ALLTOALL	i-s----	ic	l	nb	nb-op	C sq	buffer
MPI_INEIGHBOR_ALLGATHERV, MPI_INEIGHBOR_ALLTOALLV, MPI_INEIGHBOR_ALLTOALLW	i-s----	ic	l	nb	nb-op	C sq	buffer, array arguments
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op		buffer, array arguments 7)
corresponding MPI_TEST returning flag=TRUE	----c-f	c+f	l		nb-op	C W2	5) 18)
corresponding MPI_WAIT	----c-f	c+f	nl		nb-op	C W2	5) 18)
MPI_NEIGHBOR_ALLGATHER_INIT, MPI_NEIGHBOR_ALLTOALL_INIT	i-----	ic	nl*	b	p-op	C sq W1	buffer address 8)* 9)
MPI_NEIGHBOR_ALLGATHERV_INIT, MPI_NEIGHBOR_ALLTOALLV_INIT, MPI_NEIGHBOR_ALLTOALLW_INIT	i-----	ic	nl*	b	p-op	C sq W1	buffer address, array arguments 8)* 9)
corresponding MPI_START, MPI_STARTALL	--s----	ic	l	nb [†]	p-op	C	buf.addr.+contents 7) 14) [†] 16)
corresponding MPI_TEST returning flag=FALSE	-----		l		p-op		buf addr+contents & arr-args 7)
corresponding MPI_TEST returning flag=TRUE	----c--	c	l		p-op	C W2	buf-addr & arr-args 5) 7) 9) 18)
corresponding MPI_WAIT	----c--	c	nl		p-op	C W2	buffer address and array arguments cached on the request handle 5) 7) 9) 18)
corresponding MPI_REQUEST_FREE	-----f	f	l		p-op		
Chapter 11: Process Initialization, Creation, and Management							
MPI_INIT, MPI_INIT_THREAD	i-s-c-f	c+f	nl	b	b-op	C sq W1	collective over MPI_COMM_WORLD
MPI_FINALIZE	i-s-c-f	c+f	nl	b	b-op	C sq W1	collective over all connected processes
MPI_SESSION_INIT	-----		l				2)
MPI_SESSION_FINALIZE	i-s-c-f	c+f	nl	b	b-op	C sq W1	collective over connected processes scoped by the session
MPI_COMM_SPAWN, ... _MULTIPLE	i-s-c-f	c+f	nl	b	b-op	C sq W1	collective over comm 6)
MPI_COMM_ACCEPT, MPI_COMM_CONNECT	i-s-c-f	c+f	nl	b	b-op	C sq W1	collective over comm

Procedure	Stages	Cpl	Loc	Blk	Op	Collective C sq S/W	Blocked resources and remarks
Chapter 12: One-Sided Communication							
Window Allocation and Destruction							
MPI_WIN_CREATE, MPI_WIN_ALLOCATE, MPI_WIN_ALLOCATE_SHARED, MPI_WIN_CREATE_DYNAMIC	i-----		nl			C sq W1	window buffer address 36) 40)
MPI_WIN_SET_INFO	i-----		nl			C sq W1	window buffer address 40)
MPI_WIN_FREE	-----f	f	nl			C sq W1	36) 40)
With Fence Synchronization in the Role of an Origin Process							
MPI_WIN_FENCE	i-----	-	nl			C sq W1	30) 38)
MPI_PUT, MPI_GET, MPI_ACCUMULATE, MPI_GET_ACCUMULATE, MPI_COMPARE_AND_SWAP, MPI_FETCH_AND_OP	i-s----	ic	l	nb	nb-op	-	argument buffer 14) 30)
MPI_WIN_FENCE	---C-f	c+f	nl			C sq W1	31) 38)
With Fence Synchronization in the Role of a Target Process							
MPI_WIN_FENCE	--S----	ic	nl			C sq W1	window buffer address+content 38)
MPI_WIN_FENCE	---C--	c	nl			C sq W1	window buffer address 32) 38)
With General Active Target Synchronization in the Role of an Origin Process							
MPI_WIN_START	i-----		nl			-	30) 34)
MPI_PUT, MPI_GET, MPI_ACCUMULATE, MPI_GET_ACCUMULATE, MPI_COMPARE_AND_SWAP, MPI_FETCH_AND_OP	i-s----	ic	l	nb	nb-op	-	argument buffer 14) 30) 34)
MPI_WIN_COMPLETE	---C-f	c+f	l			-	31) 34)
With General Active Target Synchronization in the Role of a Target Process							
MPI_WIN_POST	--S----	ic	l			-	window buffer address+content
MPI_WIN_TEST returning flag=FALSE	-----		l			-	
MPI_WIN_TEST returning flag=TRUE	---C--	c	l			-	window buffer address 32)
MPI_WIN_WAIT	---C--	c	nl			-	window buffer address 32)
With Lock/Unlock Synchronization in the Role of an Origin Process							
MPI_WIN_LOCK, MPI_WIN_LOCK_ALL	i-----		nl			-	30) 35)
MPI_PUT, MPI_GET, MPI_ACCUMULATE, MPI_GET_ACCUMULATE, MPI_COMPARE_AND_SWAP, MPI_FETCH_AND_OP	i-s----	ic	l	nb	nb-op	-	argument buffer 14) 30) 35)
MPI_RPUT, MPI_RGET, MPI_RACCUMULATE, MPI_RGET_ACCUMULATE	i-s----	ic	l	nb	nb-op	-	argument buffer 14) 30) 35)
corresponding MPI_TEST returning flag=FALSE	-----		l		nb-op	-	argument buffer 7)
corresponding MPI_TEST returning flag=TRUE	---C-f	c+f	l		nb-op	-	30) 31)
corresponding MPI_WAIT	---C-f	c+f	l*		nb-op	-	30) 31) 35)
MPI_WIN_FLUSH_LOCAL, MPI_WIN_FLUSH_LOCAL_ALL	i--C-f	c+f	l			-	30) 31) 35) 37) 39)
MPI_WIN_FLUSH, MPI_WIN_FLUSH_ALL	i--C-f	c+f	l			-	30) 33) 35) 37) 39)
MPI_WIN_UNLOCK, MPI_WIN_UNLOCK_ALL	---C-f	c+f	l			-	30) 33) 35) 37)

Procedure	Stages	Cpl	Loc	Blk	Op	Collective C sq S/W			Blocked resources and remarks
Chapter 14: I/O									
MPI_FILE_READ/WRITE[_AT _SHARED], MPI_FILE_DELETE/SEEK/GET_VIEW	i-s-c-f	c+f	l*	b	b-op	-			12)*
MPI_FILE_READ/WRITE_AT_ALL, MPI_FILE_READ/WRITE_ALL[ORDERED, MPI_FILE_OPEN/CLOSE/SEEK_SHARED, MPI_FILE_PREALLOCATE/SYNC, MPI_FILE_SET_VIEW/SIZE/INFO/ATOMICITY	i-s-c-f	c+f	nl	b	b-op	C	sq	W1	
MPI_FILE_IREAD/IWRITE[_AT _SHARED]	i-s----	ic	l	nb	nb-op	-			buffer 10a)
MPI_FILE_IREAD/IWRITE[_AT]_ALL	i-s----	ic	l	nb	nb-op	C	sq		buffer 10b)
MPI_FILE_READ/WRITE[_AT]_ALL_BEGIN MPI_FILE_READ/WRITE_ORDERED_BEGIN	i-s----	ic	nl*	b	b-op	C	sq	W1	buffer 8)*
MPI_FILE_READ/WRITE[_AT]_ALL_END MPI_FILE_READ/WRITE_ORDERED_END	----c-f	c+f	nl	b	b-op	C	sq	W1	

A.3 C Bindings

A.3.1 Point-to-Point Communication C Bindings

```

1  int MPI_Bsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
2
3      int dest, int tag, MPI_Comm comm)
4
5  int MPI_Bsend_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
6
7      int dest, int tag, MPI_Comm comm, MPI_Request *request)
8
9  int MPI_Bsend_init(const void *buf, int count, MPI_Datatype datatype, int dest,
10
11      int tag, MPI_Comm comm, MPI_Request *request)
12
13  int MPI_Bsend(const void *buf, int count, MPI_Datatype datatype, int dest,
14
15      int tag, MPI_Comm comm)
16
17  int MPI_Buffer_attach_c(void *buffer, MPI_Count size)
18
19  int MPI_Buffer_attach(void *buffer, int size)
20
21  int MPI_Buffer_detach_c(void *buffer_addr, MPI_Count *size)
22
23  int MPI_Buffer_detach(void *buffer_addr, int *size)
24
25  int MPI_Buffer_flush(void)
26
27  int MPI_Buffer_iflush(MPI_Request *request)
28
29  int MPI_Cancel(MPI_Request *request)
30
31  int MPI_Comm_attach_buffer_c(MPI_Comm comm, void *buffer, MPI_Count size)
32
33  int MPI_Comm_attach_buffer(MPI_Comm comm, void *buffer, int size)
34
35  int MPI_Comm_detach_buffer_c(MPI_Comm comm, void *buffer_addr, MPI_Count *size)
36
37  int MPI_Comm_detach_buffer(MPI_Comm comm, void *buffer_addr, int *size)
38
39  int MPI_Comm_flush_buffer(MPI_Comm comm)
40
41  int MPI_Comm_iflush_buffer(MPI_Comm comm, MPI_Request *request)
42
43  int MPI_Get_count_c(const MPI_Status *status, MPI_Datatype datatype,
44
45      MPI_Count *count)
46
47  int MPI_Get_count(const MPI_Status *status, MPI_Datatype datatype, int *count)
48
49  int MPI_Ibsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
50
51      int dest, int tag, MPI_Comm comm, MPI_Request *request)
52
53  int MPI_Ibsend(const void *buf, int count, MPI_Datatype datatype, int dest,
54
55      int tag, MPI_Comm comm, MPI_Request *request)
56
57  int MPI_Improbe(int source, int tag, MPI_Comm comm, int *flag,
58
59      MPI_Message *message, MPI_Status *status)
60
61  int MPI_Imrecv_c(void *buf, MPI_Count count, MPI_Datatype datatype,
62
63      MPI_Message *message, MPI_Request *request)
64
65  int MPI_Imrecv(void *buf, int count, MPI_Datatype datatype,
66
67      MPI_Message *message, MPI_Request *request)

```

```

int MPI_Iprobe(int source, int tag, MPI_Comm comm, int *flag,
               MPI_Status *status)
int MPI_Irecv_c(void *buf, MPI_Count count, MPI_Datatype datatype, int source,
                int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Irecv(void *buf, int count, MPI_Datatype datatype, int source, int tag,
               MPI_Comm comm, MPI_Request *request)
int MPI_Irsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                 int dest, int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Irsend(const void *buf, int count, MPI_Datatype datatype, int dest,
               int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Isend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                int dest, int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Isendrecv_c(const void *sendbuf, MPI_Count sendcount,
                    MPI_Datatype sendtype, int dest, int sendtag, void *recvbuf,
                    MPI_Count recvcount, MPI_Datatype recvtype, int source,
                    int recvtag, MPI_Comm comm, MPI_Request *request)
int MPI_Isendrecv_replace_c(void *buf, MPI_Count count, MPI_Datatype datatype,
                             int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
                             MPI_Request *request)
int MPI_Isendrecv_replace(void *buf, int count, MPI_Datatype datatype,
                           int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
                           MPI_Request *request)
int MPI_Isendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                  int dest, int sendtag, void *recvbuf, int recvcount,
                  MPI_Datatype recvtype, int source, int recvtag, MPI_Comm comm,
                  MPI_Request *request)
int MPI_Isend(const void *buf, int count, MPI_Datatype datatype, int dest,
              int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Issend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                 int dest, int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Issend(const void *buf, int count, MPI_Datatype datatype, int dest,
               int tag, MPI_Comm comm, MPI_Request *request)
int MPI_Mprobe(int source, int tag, MPI_Comm comm, MPI_Message *message,
               MPI_Status *status)
int MPI_Mrecv_c(void *buf, MPI_Count count, MPI_Datatype datatype,
                MPI_Message *message, MPI_Status *status)
int MPI_Mrecv(void *buf, int count, MPI_Datatype datatype,
               MPI_Message *message, MPI_Status *status)
int MPI_Probe(int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Recv_c(void *buf, MPI_Count count, MPI_Datatype datatype, int source,
               int tag, MPI_Comm comm, MPI_Status *status)

```

```

1  int MPI_Recv_init_c(void *buf, MPI_Count count, MPI_Datatype datatype,
2                      int source, int tag, MPI_Comm comm, MPI_Request *request)
3
4  int MPI_Recv_init(void *buf, int count, MPI_Datatype datatype, int source,
5                  int tag, MPI_Comm comm, MPI_Request *request)
6
7  int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag,
8              MPI_Comm comm, MPI_Status *status)
9
10 int MPI_Request_free(MPI_Request *request)
11
12 int MPI_Request_get_status_all(int count,
13                               const MPI_Request array_of_requests[], int *flag,
14                               MPI_Status array_of_statuses[])
15
16 int MPI_Request_get_status_any(int count,
17                               const MPI_Request array_of_requests[], int *index, int *flag,
18                               MPI_Status *status)
19
20 int MPI_Request_get_status_some(int incout,
21                                const MPI_Request array_of_requests[], int *outcount,
22                                int array_of_indices[], MPI_Status array_of_statuses[])
23
24 int MPI_Request_get_status(MPI_Request request, int *flag, MPI_Status *status)
25
26 int MPI_Rsend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
27                int dest, int tag, MPI_Comm comm)
28
29 int MPI_Rsend_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
30                     int dest, int tag, MPI_Comm comm, MPI_Request *request)
31
32 int MPI_Rsend_init(const void *buf, int count, MPI_Datatype datatype, int dest,
33                   int tag, MPI_Comm comm, MPI_Request *request)
34
35 int MPI_Rsend(const void *buf, int count, MPI_Datatype datatype, int dest,
36              int tag, MPI_Comm comm)
37
38 int MPI_Send_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
39               int dest, int tag, MPI_Comm comm)
40
41 int MPI_Send_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
42                    int dest, int tag, MPI_Comm comm, MPI_Request *request)
43
44 int MPI_Send_init(const void *buf, int count, MPI_Datatype datatype, int dest,
45                  int tag, MPI_Comm comm, MPI_Request *request)
46
47 int MPI_Sendrecv_c(const void *sendbuf, MPI_Count sendcount,
48                   MPI_Datatype sendtype, int dest, int sendtag, void *recvbuf,
49                   MPI_Count recvcount, MPI_Datatype recvtype, int source,
50                   int recvtag, MPI_Comm comm, MPI_Status *status)
51
52 int MPI_Sendrecv_replace_c(void *buf, MPI_Count count, MPI_Datatype datatype,
53                            int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
54                            MPI_Status *status)
55
56 int MPI_Sendrecv_replace(void *buf, int count, MPI_Datatype datatype, int dest,
57                          int sendtag, int source, int recvtag, MPI_Comm comm,
58                          MPI_Status *status)

```

```

int MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                 int dest, int sendtag, void *recvbuf, int recvcount,
                 MPI_Datatype recvttype, int source, int recvtag, MPI_Comm comm,
                 MPI_Status *status)
1
2
3
4
5
int MPI_Send(const void *buf, int count, MPI_Datatype datatype, int dest,
             int tag, MPI_Comm comm)
6
7
int MPI_Session_attach_buffer_c(MPI_Session session, void *buffer,
                                MPI_Count size)
8
9
int MPI_Session_attach_buffer(MPI_Session session, void *buffer, int size)
10
11
int MPI_Session_detach_buffer_c(MPI_Session session, void *buffer_addr,
                                MPI_Count *size)
12
13
int MPI_Session_detach_buffer(MPI_Session session, void *buffer_addr,
                              int *size)
14
15
int MPI_Session_flush_buffer(MPI_Session session)
16
17
int MPI_Session_iflush_buffer(MPI_Session session, MPI_Request *request)
18
19
int MPI_Ssend_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
               int dest, int tag, MPI_Comm comm)
20
21
int MPI_Ssend_init_c(const void *buf, MPI_Count count, MPI_Datatype datatype,
                    int dest, int tag, MPI_Comm comm, MPI_Request *request)
22
23
int MPI_Ssend_init(const void *buf, int count, MPI_Datatype datatype, int dest,
                  int tag, MPI_Comm comm, MPI_Request *request)
24
25
int MPI_Ssend(const void *buf, int count, MPI_Datatype datatype, int dest,
              int tag, MPI_Comm comm)
26
27
int MPI_Startall(int count, MPI_Request array_of_requests[])
28
29
int MPI_Start(MPI_Request *request)
30
31
int MPI_Status_get_error(const MPI_Status *status, int *err)
32
33
int MPI_Status_get_source(const MPI_Status *status, int *source)
34
35
int MPI_Status_get_tag(const MPI_Status *status, int *tag)
36
37
int MPI_Test_cancelled(const MPI_Status *status, int *flag)
38
39
int MPI_Testall(int count, MPI_Request array_of_requests[], int *flag,
               MPI_Status array_of_statuses[])
40
41
int MPI_Testany(int count, MPI_Request array_of_requests[], int *index,
               int *flag, MPI_Status *status)
42
43
int MPI_Testsome(int incount, MPI_Request array_of_requests[], int *outcount,
                int array_of_indices[], MPI_Status array_of_statuses[])
44
45
int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)
46
47
int MPI_Waitall(int count, MPI_Request array_of_requests[],
               MPI_Status array_of_statuses[])
48

```

```

1  int MPI_Waitany(int count, MPI_Request array_of_requests[], int *index,
2      MPI_Status *status)
3
4  int MPI_Waitsome(int incount, MPI_Request array_of_requests[], int *outcount,
5      int array_of_indices[], MPI_Status array_of_statuses[])
6
7  int MPI_Wait(MPI_Request *request, MPI_Status *status)

```

A.3.2 Partitioned Communication C Bindings

```

10 int MPI_Parrived(MPI_Request request, int partition, int *flag)
11
12 int MPI_Pready_list(int length, const int array_of_partitions[],
13     MPI_Request request)
14
15 int MPI_Pready_range(int partition_low, int partition_high,
16     MPI_Request request)
17
18 int MPI_Pready(int partition, MPI_Request request)
19
20 int MPI_Precv_init(void *buf, int partitions, MPI_Count count,
21     MPI_Datatype datatype, int source, int tag, MPI_Comm comm,
22     MPI_Info info, MPI_Request *request)
23
24 int MPI_Psend_init(const void *buf, int partitions, MPI_Count count,
25     MPI_Datatype datatype, int dest, int tag, MPI_Comm comm,
26     MPI_Info info, MPI_Request *request)

```

A.3.3 Datatypes C Bindings

```

27 int MPI_Get_address(const void *location, MPI_Aint *address)
28
29 int MPI_Get_elements_c(const MPI_Status *status, MPI_Datatype datatype,
30     MPI_Count *count)
31
32 int MPI_Get_elements(const MPI_Status *status, MPI_Datatype datatype,
33     int *count)
34
35 int MPI_Pack_c(const void *inbuf, MPI_Count incount, MPI_Datatype datatype,
36     void *outbuf, MPI_Count outsize, MPI_Count *position,
37     MPI_Comm comm)
38
39 int MPI_Pack_external_c(const char datarep[], const void *inbuf,
40     MPI_Count incount, MPI_Datatype datatype, void *outbuf,
41     MPI_Count outsize, MPI_Count *position)
42
43 int MPI_Pack_external_size_c(const char datarep[], MPI_Count incount,
44     MPI_Datatype datatype, MPI_Count *size)
45
46 int MPI_Pack_external_size(const char datarep[], int incount,
47     MPI_Datatype datatype, MPI_Aint *size)
48
49 int MPI_Pack_external(const char datarep[], const void *inbuf, int incount,
50     MPI_Datatype datatype, void *outbuf, MPI_Aint outsize,
51     MPI_Aint *position)

```

```

int MPI_Pack_size_c(MPI_Count incount, MPI_Datatype datatype, MPI_Comm comm,
                    MPI_Count *size)
int MPI_Pack_size(int incount, MPI_Datatype datatype, MPI_Comm comm, int *size)
int MPI_Pack(const void *inbuf, int incount, MPI_Datatype datatype,
             void *outbuf, int outsize, int *position, MPI_Comm comm)
int MPI_Type_commit(MPI_Datatype *datatype)
int MPI_Type_contiguous_c(MPI_Count count, MPI_Datatype oldtype,
                          MPI_Datatype *newtype)
int MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_create_darray_c(int size, int rank, int ndims,
                             const MPI_Count array_of_gsizes[], const int array_of_distribs[],
                             const int array_of_dargs[], const int array_of_psize[],
                             int order, MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_create_darray(int size, int rank, int ndims,
                           const int array_of_gsizes[], const int array_of_distribs[],
                           const int array_of_dargs[], const int array_of_psize[],
                           int order, MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_create_hindexed_block_c(MPI_Count count, MPI_Count blocklength,
                                     const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
                                     MPI_Datatype *newtype)
int MPI_Type_create_hindexed_block(int count, int blocklength,
                                   const MPI_Aint array_of_displacements[], MPI_Datatype oldtype,
                                   MPI_Datatype *newtype)
int MPI_Type_create_hindexed_c(MPI_Count count,
                               const MPI_Count array_of_blocklengths[],
                               const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
                               MPI_Datatype *newtype)
int MPI_Type_create_hindexed(int count, const int array_of_blocklengths[],
                             const MPI_Aint array_of_displacements[], MPI_Datatype oldtype,
                             MPI_Datatype *newtype)
int MPI_Type_create_hvector_c(MPI_Count count, MPI_Count blocklength,
                              MPI_Count stride, MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_create_hvector(int count, int blocklength, MPI_Aint stride,
                             MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_create_indexed_block_c(MPI_Count count, MPI_Count blocklength,
                                    const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
                                    MPI_Datatype *newtype)
int MPI_Type_create_indexed_block(int count, int blocklength,
                                  const int array_of_displacements[], MPI_Datatype oldtype,
                                  MPI_Datatype *newtype)
int MPI_Type_create_resized_c(MPI_Datatype oldtype, MPI_Count lb,
                              MPI_Count extent, MPI_Datatype *newtype)

```



```

1  int MPI_Type_create_resized(MPI_Datatype oldtype, MPI_Aint lb, MPI_Aint extent,
2      MPI_Datatype *newtype)
3
4  int MPI_Type_create_struct_c(MPI_Count count,
5      const MPI_Count array_of_blocklengths[],
6      const MPI_Count array_of_displacements[],
7      const MPI_Datatype array_of_types[], MPI_Datatype *newtype)
8
9  int MPI_Type_create_struct(int count, const int array_of_blocklengths[],
10     const MPI_Aint array_of_displacements[],
11     const MPI_Datatype array_of_types[], MPI_Datatype *newtype)
12
13 int MPI_Type_create_subarray_c(int ndims, const MPI_Count array_of_sizes[],
14     const MPI_Count array_of_subsizes[],
15     const MPI_Count array_of_starts[], int order,
16     MPI_Datatype oldtype, MPI_Datatype *newtype)
17
18 int MPI_Type_create_subarray(int ndims, const int array_of_sizes[],
19     const int array_of_subsizes[], const int array_of_starts[],
20     int order, MPI_Datatype oldtype, MPI_Datatype *newtype)
21
22 int MPI_Type_dup(MPI_Datatype oldtype, MPI_Datatype *newtype)
23
24 int MPI_Type_free(MPI_Datatype *datatype)
25
26 int MPI_Type_get_contents_c(MPI_Datatype datatype, MPI_Count max_integers,
27     MPI_Count max_addresses, MPI_Count max_large_counts,
28     MPI_Count max_datatypes, int array_of_integers[],
29     MPI_Aint array_of_addresses[], MPI_Count array_of_large_counts[],
30     MPI_Datatype array_of_datatypes[])
31
32 int MPI_Type_get_contents(MPI_Datatype datatype, int max_integers,
33     int max_addresses, int max_datatypes, int array_of_integers[],
34     MPI_Aint array_of_addresses[], MPI_Datatype array_of_datatypes[])
35
36 int MPI_Type_get_envelope_c(MPI_Datatype datatype, MPI_Count *num_integers,
37     MPI_Count *num_addresses, MPI_Count *num_large_counts,
38     MPI_Count *num_datatypes, int *combiner)
39
40 int MPI_Type_get_envelope(MPI_Datatype datatype, int *num_integers,
41     int *num_addresses, int *num_datatypes, int *combiner)
42
43 int MPI_Type_get_extent_c(MPI_Datatype datatype, MPI_Count *lb,
44     MPI_Count *extent)
45
46 int MPI_Type_get_extent(MPI_Datatype datatype, MPI_Aint *lb, MPI_Aint *extent)
47
48 int MPI_Type_get_true_extent_c(MPI_Datatype datatype, MPI_Count *true_lb,
49     MPI_Count *true_extent)
50
51 int MPI_Type_get_true_extent(MPI_Datatype datatype, MPI_Aint *true_lb,
52     MPI_Aint *true_extent)
53
54 int MPI_Type_indexed_c(MPI_Count count,
55     const MPI_Count array_of_blocklengths[],
56     const MPI_Count array_of_displacements[], MPI_Datatype oldtype,
57     MPI_Datatype *newtype)

```



```

int MPI_Type_indexed(int count, const int array_of_blocklengths[],
                    const int array_of_displacements[], MPI_Datatype oldtype,
                    MPI_Datatype *newtype)
int MPI_Type_size_c(MPI_Datatype datatype, MPI_Count *size)
int MPI_Type_size(MPI_Datatype datatype, int *size)
int MPI_Type_vector_c(MPI_Count count, MPI_Count blocklength, MPI_Count stride,
                    MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Type_vector(int count, int blocklength, int stride,
                    MPI_Datatype oldtype, MPI_Datatype *newtype)
int MPI_Unpack_c(const void *inbuf, MPI_Count insize, MPI_Count *position,
                void *outbuf, MPI_Count outcount, MPI_Datatype datatype,
                MPI_Comm comm)
int MPI_Unpack_external_c(const char datarep[], const void *inbuf,
                        MPI_Count insize, MPI_Count *position, void *outbuf,
                        MPI_Count outcount, MPI_Datatype datatype)
int MPI_Unpack_external(const char datarep[], const void *inbuf,
                        MPI_Aint insize, MPI_Aint *position, void *outbuf, int outcount,
                        MPI_Datatype datatype)
int MPI_Unpack(const void *inbuf, int insize, int *position, void *outbuf,
                int outcount, MPI_Datatype datatype, MPI_Comm comm)

```

A.3.4 Collective Communication C Bindings

```

int MPI_Allgather_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                  MPI_Datatype recvtype, MPI_Comm comm)
int MPI_Allgather_init_c(const void *sendbuf, MPI_Count sendcount,
                        MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                        MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
                        MPI_Request *request)
int MPI_Allgather_init(const void *sendbuf, int sendcount,
                      MPI_Datatype sendtype, void *recvbuf, int recvcount,
                      MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
                      MPI_Request *request)
int MPI_Allgatherv_c(const void *sendbuf, MPI_Count sendcount,
                    MPI_Datatype sendtype, void *recvbuf,
                    const MPI_Count recvcounts[], const MPI_Aint displs[],
                    MPI_Datatype recvtype, MPI_Comm comm)
int MPI_Allgatherv_init_c(const void *sendbuf, MPI_Count sendcount,
                          MPI_Datatype sendtype, void *recvbuf,
                          const MPI_Count recvcounts[], const MPI_Aint displs[],
                          MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
                          MPI_Request *request)

```

```

1  int MPI_Allgatherv_init(const void *sendbuf, int sendcount,
2      MPI_Datatype sendtype, void *recvbuf, const int recvcnts[],
3      const int displs[], MPI_Datatype recvtpe, MPI_Comm comm,
4      MPI_Info info, MPI_Request *request)
5
6  int MPI_Allgatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
7      void *recvbuf, const int recvcnts[], const int displs[],
8      MPI_Datatype recvtpe, MPI_Comm comm)
9
10 int MPI_Allgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
11     void *recvbuf, int recvcnt, MPI_Datatype recvtpe,
12     MPI_Comm comm)
13
14 int MPI_Allreduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
15     MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
16
17 int MPI_Allreduce_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
18     MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
19     MPI_Request *request)
20
21 int MPI_Allreduce_init(const void *sendbuf, void *recvbuf, int count,
22     MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
23     MPI_Request *request)
24
25 int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count,
26     MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
27
28 int MPI_Alltoall_c(const void *sendbuf, MPI_Count sendcount,
29     MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcnt,
30     MPI_Datatype recvtpe, MPI_Comm comm)
31
32 int MPI_Alltoall_init_c(const void *sendbuf, MPI_Count sendcount,
33     MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcnt,
34     MPI_Datatype recvtpe, MPI_Comm comm, MPI_Info info,
35     MPI_Request *request)
36
37 int MPI_Alltoall_init(const void *sendbuf, int sendcount,
38     MPI_Datatype sendtype, void *recvbuf, int recvcnt,
39     MPI_Datatype recvtpe, MPI_Comm comm, MPI_Info info,
40     MPI_Request *request)
41
42 int MPI_Alltoallv_c(const void *sendbuf, const MPI_Count sendcounts[],
43     const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
44     const MPI_Count recvcnts[], const MPI_Aint rdispls[],
45     MPI_Datatype recvtpe, MPI_Comm comm)
46
47 int MPI_Alltoallv_init_c(const void *sendbuf, const MPI_Count sendcounts[],
48     const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
49     const MPI_Count recvcnts[], const MPI_Aint rdispls[],
50     MPI_Datatype recvtpe, MPI_Comm comm, MPI_Info info,
51     MPI_Request *request)
52
53 int MPI_Alltoallv_init(const void *sendbuf, const int sendcounts[],
54     const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
55     const int recvcnts[], const int rdispls[],

```

```

        MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
        MPI_Request *request)
1
2
3
int MPI_Alltoallv(const void *sendbuf, const int sendcounts[],
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
        const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
        const int recvcounst[], const int rdispls[],
        MPI_Datatype recvtype, MPI_Comm comm)
int MPI_Alltoallw_c(const void *sendbuf, const MPI_Count sendcounts[],
        const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
        void *recvbuf, const MPI_Count recvcounst[],
        const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
        MPI_Comm comm)
int MPI_Alltoallw_init_c(const void *sendbuf, const MPI_Count sendcounts[],
        const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
        void *recvbuf, const MPI_Count recvcounst[],
        const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
        MPI_Comm comm, MPI_Info info, MPI_Request *request)
int MPI_Alltoallw_init(const void *sendbuf, const int sendcounts[],
        const int sdispls[], const MPI_Datatype sendtypes[],
        void *recvbuf, const int recvcounst[], const int rdispls[],
        const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Info info,
        MPI_Request *request)
int MPI_Alltoallw(const void *sendbuf, const int sendcounts[],
        const int sdispls[], const MPI_Datatype sendtypes[],
        void *recvbuf, const int recvcounst[], const int rdispls[],
        const MPI_Datatype recvtypes[], MPI_Comm comm)
int MPI_Alltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
        void *recvbuf, int recvcount, MPI_Datatype recvtype,
        MPI_Comm comm)
int MPI_Barrier_init(MPI_Comm comm, MPI_Info info, MPI_Request *request)
int MPI_Barrier(MPI_Comm comm)
int MPI_Bcast_c(void *buffer, MPI_Count count, MPI_Datatype datatype, int root,
        MPI_Comm comm)
int MPI_Bcast_init_c(void *buffer, MPI_Count count, MPI_Datatype datatype,
        int root, MPI_Comm comm, MPI_Info info, MPI_Request *request)
int MPI_Bcast_init(void *buffer, int count, MPI_Datatype datatype, int root,
        MPI_Comm comm, MPI_Info info, MPI_Request *request)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root,
        MPI_Comm comm)
int MPI_Exscan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
        MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Exscan_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
        MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
        MPI_Request *request)

```

```

1  int MPI_Exscan_init(const void *sendbuf, void *recvbuf, int count,
2                      MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
3                      MPI_Request *request)
4
5  int MPI_Exscan(const void *sendbuf, void *recvbuf, int count,
6                MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
7
8  int MPI_Gather_c(const void *sendbuf, MPI_Count sendcount,
9                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
10                  MPI_Datatype recvtype, int root, MPI_Comm comm)
11
12 int MPI_Gather_init_c(const void *sendbuf, MPI_Count sendcount,
13                      MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
14                      MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
15                      MPI_Request *request)
16
17 int MPI_Gather_init(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
18                    void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
19                    MPI_Comm comm, MPI_Info info, MPI_Request *request)
20
21 int MPI_Gatherv_c(const void *sendbuf, MPI_Count sendcount,
22                  MPI_Datatype sendtype, void *recvbuf,
23                  const MPI_Count recvcounts[], const MPI_Aint displs[],
24                  MPI_Datatype recvtype, int root, MPI_Comm comm)
25
26 int MPI_Gatherv_init_c(const void *sendbuf, MPI_Count sendcount,
27                       MPI_Datatype sendtype, void *recvbuf,
28                       const MPI_Count recvcounts[], const MPI_Aint displs[],
29                       MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
30                       MPI_Request *request)
31
32 int MPI_Gatherv_init(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
33                     void *recvbuf, const int recvcounts[], const int displs[],
34                     MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
35                     MPI_Request *request)
36
37 int MPI_Gatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
38                void *recvbuf, const int recvcounts[], const int displs[],
39                MPI_Datatype recvtype, int root, MPI_Comm comm)
40
41 int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
42               void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
43               MPI_Comm comm)
44
45 int MPI_Iallgather_c(const void *sendbuf, MPI_Count sendcount,
46                     MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
47                     MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
48
49 int MPI_Iallgather_v_c(const void *sendbuf, MPI_Count sendcount,
50                       MPI_Datatype sendtype, void *recvbuf,
51                       const MPI_Count recvcounts[], const MPI_Aint displs[],
52                       MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
53
54 int MPI_Iallgather_v(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
55                     void *recvbuf, const int recvcounts[], const int displs[],
56                     MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)

```

```

int MPI_Iallgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                  void *recvbuf, int recvcount, MPI_Datatype recvtype,
                  MPI_Comm comm, MPI_Request *request)
int MPI_Iallreduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                    MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                    MPI_Request *request)
int MPI_Iallreduce(const void *sendbuf, void *recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                  MPI_Request *request)
int MPI_Ialltoall_c(const void *sendbuf, MPI_Count sendcount,
                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
                  MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ialltoallv_c(const void *sendbuf, const MPI_Count sendcounts[],
                  const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
                  const MPI_Count recvcounts[], const MPI_Aint rdispls[],
                  MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ialltoallv(const void *sendbuf, const int sendcounts[],
                  const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
                  const int recvcounts[], const int rdispls[],
                  MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ialltoallw_c(const void *sendbuf, const MPI_Count sendcounts[],
                  const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                  void *recvbuf, const MPI_Count recvcounts[],
                  const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
                  MPI_Comm comm, MPI_Request *request)
int MPI_Ialltoallw(const void *sendbuf, const int sendcounts[],
                  const int sdispls[], const MPI_Datatype sendtypes[],
                  void *recvbuf, const int recvcounts[], const int rdispls[],
                  const MPI_Datatype recvtypes[], MPI_Comm comm,
                  MPI_Request *request)
int MPI_Ialltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                  void *recvbuf, int recvcount, MPI_Datatype recvtype,
                  MPI_Comm comm, MPI_Request *request)
int MPI_Ibarrier(MPI_Comm comm, MPI_Request *request)
int MPI_Ibcast_c(void *buffer, MPI_Count count, MPI_Datatype datatype,
                int root, MPI_Comm comm, MPI_Request *request)
int MPI_Ibcast(void *buffer, int count, MPI_Datatype datatype, int root,
               MPI_Comm comm, MPI_Request *request)
int MPI_Iexscan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
                 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                 MPI_Request *request)
int MPI_Iexscan(const void *sendbuf, void *recvbuf, int count,
                MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
                MPI_Request *request)

```

```

1  int MPI_Igather_c(const void *sendbuf, MPI_Count sendcount,
2                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
3                  MPI_Datatype recvttype, int root, MPI_Comm comm,
4                  MPI_Request *request)
5
6  int MPI_Igather_v_c(const void *sendbuf, MPI_Count sendcount,
7                    MPI_Datatype sendtype, void *recvbuf,
8                    const MPI_Count recvcounts[], const MPI_Aint displs[],
9                    MPI_Datatype recvttype, int root, MPI_Comm comm,
10                   MPI_Request *request)
11
12 int MPI_Igather_v(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
13                 void *recvbuf, const int recvcounts[], const int displs[],
14                 MPI_Datatype recvttype, int root, MPI_Comm comm,
15                 MPI_Request *request)
16
17 int MPI_Igather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
18               void *recvbuf, int recvcount, MPI_Datatype recvttype, int root,
19               MPI_Comm comm, MPI_Request *request)
20
21 int MPI_Ireduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
22                 MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
23                 MPI_Request *request)
24
25 int MPI_Ireduce_scatter_block_c(const void *sendbuf, void *recvbuf,
26                               MPI_Count recvcount, MPI_Datatype datatype, MPI_Op op,
27                               MPI_Comm comm, MPI_Request *request)
28
29 int MPI_Ireduce_scatter_block(const void *sendbuf, void *recvbuf,
30                              int recvcount, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
31                              MPI_Request *request)
32
33 int MPI_Ireduce_scatter_c(const void *sendbuf, void *recvbuf,
34                          const MPI_Count recvcounts[], MPI_Datatype datatype, MPI_Op op,
35                          MPI_Comm comm, MPI_Request *request)
36
37 int MPI_Ireduce_scatter(const void *sendbuf, void *recvbuf,
38                        const int recvcounts[], MPI_Datatype datatype, MPI_Op op,
39                        MPI_Comm comm, MPI_Request *request)
40
41 int MPI_Ireduce(const void *sendbuf, void *recvbuf, int count,
42               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
43               MPI_Request *request)
44
45 int MPI_Iscan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
46               MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
47               MPI_Request *request)
48
49 int MPI_Iscan(const void *sendbuf, void *recvbuf, int count,
50              MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
51              MPI_Request *request)
52
53 int MPI_Iscatter_c(const void *sendbuf, MPI_Count sendcount,
54                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,

```

```

        MPI_Datatype recvtype, int root, MPI_Comm comm,
        MPI_Request *request)
1
2
3
int MPI_Iscatterv_c(const void *sendbuf, const MPI_Count sendcounts[],
4
5   const MPI_Aint displs[], MPI_Datatype sendtype, void *recvbuf,
6
7   MPI_Count recvcount, MPI_Datatype recvtype, int root,
8
9   MPI_Comm comm, MPI_Request *request)
10
11
int MPI_Iscatterv(const void *sendbuf, const int sendcounts[],
12
13   const int displs[], MPI_Datatype sendtype, void *recvbuf,
14
15   int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm,
16
17   MPI_Request *request)
18
19
int MPI_Iscatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
20
21   void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
22
23   MPI_Comm comm, MPI_Request *request)
24
25
int MPI_Op_commutative(MPI_Op op, int *commute)
26
27
int MPI_Op_create_c(MPI_User_function_c *user_fn, int commute, MPI_Op *op)
28
29
int MPI_Op_create(MPI_User_function *user_fn, int commute, MPI_Op *op)
30
31
int MPI_Op_free(MPI_Op *op)
32
33
int MPI_Reduce_c(const void *sendbuf, void *recvbuf, MPI_Count count,
34
35   MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
36
37
int MPI_Reduce_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
38
39   MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
40
41   MPI_Info info, MPI_Request *request)
42
43
int MPI_Reduce_init(const void *sendbuf, void *recvbuf, int count,
44
45   MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,
46
47   MPI_Info info, MPI_Request *request)
48
49
int MPI_Reduce_local_c(const void *inbuf, void *inoutbuf, MPI_Count count,
50
51   MPI_Datatype datatype, MPI_Op op)
52
53
int MPI_Reduce_local(const void *inbuf, void *inoutbuf, int count,
54
55   MPI_Datatype datatype, MPI_Op op)
56
57
int MPI_Reduce_scatter_block_c(const void *sendbuf, void *recvbuf,
58
59   MPI_Count recvcount, MPI_Datatype datatype, MPI_Op op,
60
61   MPI_Comm comm)
62
63
int MPI_Reduce_scatter_block_init_c(const void *sendbuf, void *recvbuf,
64
65   MPI_Count recvcount, MPI_Datatype datatype, MPI_Op op,
66
67   MPI_Comm comm, MPI_Info info, MPI_Request *request)
68
69
int MPI_Reduce_scatter_block_init(const void *sendbuf, void *recvbuf,
70
71   int recvcount, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,
72
73   MPI_Info info, MPI_Request *request)
74
75
int MPI_Reduce_scatter_block(const void *sendbuf, void *recvbuf, int recvcount,
76
77   MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
78
79

```



```

1  int MPI_Reduce_scatter_c(const void *sendbuf, void *recvbuf,
2                          const MPI_Count recvcounts[], MPI_Datatype datatype, MPI_Op op,
3                          MPI_Comm comm)
4
5  int MPI_Reduce_scatter_init_c(const void *sendbuf, void *recvbuf,
6                               const MPI_Count recvcounts[], MPI_Datatype datatype, MPI_Op op,
7                               MPI_Comm comm, MPI_Info info, MPI_Request *request)
8
9  int MPI_Reduce_scatter_init(const void *sendbuf, void *recvbuf,
10                             const int recvcounts[], MPI_Datatype datatype, MPI_Op op,
11                             MPI_Comm comm, MPI_Info info, MPI_Request *request)
12
13 int MPI_Reduce_scatter(const void *sendbuf, void *recvbuf,
14                       const int recvcounts[], MPI_Datatype datatype, MPI_Op op,
15                       MPI_Comm comm)
16
17 int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,
18               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
19
20 int MPI_Scan_c(const void *sendbuf, void *recvbuf, MPI_Count count,
21               MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
22
23 int MPI_Scan_init_c(const void *sendbuf, void *recvbuf, MPI_Count count,
24                    MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
25                    MPI_Request *request)
26
27 int MPI_Scan_init(const void *sendbuf, void *recvbuf, int count,
28                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Info info,
29                  MPI_Request *request)
30
31 int MPI_Scan(const void *sendbuf, void *recvbuf, int count,
32              MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
33
34 int MPI_Scatter_c(const void *sendbuf, MPI_Count sendcount,
35                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
36                  MPI_Datatype recvtype, int root, MPI_Comm comm)
37
38 int MPI_Scatter_init_c(const void *sendbuf, MPI_Count sendcount,
39                       MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
40                       MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Info info,
41                       MPI_Request *request)
42
43 int MPI_Scatter_init(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
44                     void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
45                     MPI_Comm comm, MPI_Info info, MPI_Request *request)
46
47 int MPI_Scatterv_c(const void *sendbuf, const MPI_Count sendcounts[],
48                   const MPI_Aint displs[], MPI_Datatype sendtype, void *recvbuf,
49                   MPI_Count recvcount, MPI_Datatype recvtype, int root,
50                   MPI_Comm comm)
51
52 int MPI_Scatterv_init_c(const void *sendbuf, const MPI_Count sendcounts[],
53                        const MPI_Aint displs[], MPI_Datatype sendtype, void *recvbuf,
54                        MPI_Count recvcount, MPI_Datatype recvtype, int root,
55                        MPI_Comm comm, MPI_Info info, MPI_Request *request)

```



```

int MPI_Scatterv_init(const void *sendbuf, const int sendcounts[],
                    const int displs[], MPI_Datatype sendtype, void *recvbuf,
                    int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm,
                    MPI_Info info, MPI_Request *request)
int MPI_Scatterv(const void *sendbuf, const int sendcounts[],
                const int displs[], MPI_Datatype sendtype, void *recvbuf,
                int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm)
int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                void *recvbuf, int recvcnt, MPI_Datatype recvtpe, int root,
                MPI_Comm comm)
int MPI_Type_get_value_index(MPI_Datatype value_type, MPI_Datatype index_type,
                             MPI_Datatype *pair_type)

```

A.3.5 Groups, Contexts, Communicators, and Caching C Bindings

```

int MPI_Comm_compare(MPI_Comm comm1, MPI_Comm comm2, int *result)
int MPI_Comm_create_from_group(MPI_Group group, const char *stringtag,
                              MPI_Info info, MPI_Errhandler errhandler, MPI_Comm *newcomm)
int MPI_Comm_create_group(MPI_Comm comm, MPI_Group group, int tag,
                          MPI_Comm *newcomm)
int MPI_Comm_create_keyval(MPI_Comm_copy_attr_function *comm_copy_attr_fn,
                          MPI_Comm_delete_attr_function *comm_delete_attr_fn,
                          int *comm_keyval, void *extra_state)
int MPI_Comm_create(MPI_Comm comm, MPI_Group group, MPI_Comm *newcomm)
int MPI_Comm_delete_attr(MPI_Comm comm, int comm_keyval)
int MPI_COMM_DUP_FN(MPI_Comm oldcomm, int comm_keyval, void *extra_state,
                    void *attribute_val_in, void *attribute_val_out, int *flag)
int MPI_Comm_dup_with_info(MPI_Comm comm, MPI_Info info, MPI_Comm *newcomm)
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm)
int MPI_Comm_free_keyval(int *comm_keyval)
int MPI_Comm_free(MPI_Comm *comm)
int MPI_Comm_get_attr(MPI_Comm comm, int comm_keyval, void *attribute_val,
                      int *flag)
int MPI_Comm_get_info(MPI_Comm comm, MPI_Info *info_used)
int MPI_Comm_get_name(MPI_Comm comm, char *comm_name, int *resultlen)
int MPI_Comm_group(MPI_Comm comm, MPI_Group *group)
int MPI_Comm_idup_with_info(MPI_Comm comm, MPI_Info info, MPI_Comm *newcomm,
                           MPI_Request *request)
int MPI_Comm_idup(MPI_Comm comm, MPI_Comm *newcomm, MPI_Request *request)

```

```

1  int MPI_COMM_NULL_COPY_FN(MPI_Comm oldcomm, int comm_keyval, void *extra_state,
2      void *attribute_val_in, void *attribute_val_out, int *flag)
3
4  int MPI_COMM_NULL_DELETE_FN(MPI_Comm comm, int comm_keyval,
5      void *attribute_val, void *extra_state)
6
7  int MPI_Comm_rank(MPI_Comm comm, int *rank)
8
9  int MPI_Comm_remote_group(MPI_Comm comm, MPI_Group *group)
10
11 int MPI_Comm_remote_size(MPI_Comm comm, int *size)
12
13 int MPI_Comm_set_attr(MPI_Comm comm, int comm_keyval, void *attribute_val)
14
15 int MPI_Comm_set_info(MPI_Comm comm, MPI_Info info)
16
17 int MPI_Comm_set_name(MPI_Comm comm, const char *comm_name)
18
19 int MPI_Comm_size(MPI_Comm comm, int *size)
20
21 int MPI_Comm_split_type(MPI_Comm comm, int split_type, int key, MPI_Info info,
22     MPI_Comm *newcomm)
23
24 int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm *newcomm)
25
26 int MPI_Comm_test_inter(MPI_Comm comm, int *flag)
27
28 int MPI_Group_compare(MPI_Group group1, MPI_Group group2, int *result)
29
30 int MPI_Group_difference(MPI_Group group1, MPI_Group group2,
31     MPI_Group *newgroup)
32
33 int MPI_Group_excl(MPI_Group group, int n, const int ranks[],
34     MPI_Group *newgroup)
35
36 int MPI_Group_free(MPI_Group *group)
37
38 int MPI_Group_from_session_pset(MPI_Session session, const char *pset_name,
39     MPI_Group *newgroup)
40
41 int MPI_Group_incl(MPI_Group group, int n, const int ranks[],
42     MPI_Group *newgroup)
43
44 int MPI_Group_intersection(MPI_Group group1, MPI_Group group2,
45     MPI_Group *newgroup)
46
47 int MPI_Group_range_excl(MPI_Group group, int n, int ranges[][3],
48     MPI_Group *newgroup)
49
50 int MPI_Group_range_incl(MPI_Group group, int n, int ranges[][3],
51     MPI_Group *newgroup)
52
53 int MPI_Group_rank(MPI_Group group, int *rank)
54
55 int MPI_Group_size(MPI_Group group, int *size)
56
57 int MPI_Group_translate_ranks(MPI_Group group1, int n, const int ranks1[],
58     MPI_Group group2, int ranks2[])
59
60 int MPI_Group_union(MPI_Group group1, MPI_Group group2, MPI_Group *newgroup)

```

```

int MPI_Intercomm_create_from_groups(MPI_Group local_group, int local_leader,
                                     MPI_Group remote_group, int remote_leader, const char *stringtag,
                                     MPI_Info info, MPI_Errhandler errhandler, MPI_Comm *newintercomm)
int MPI_Intercomm_create(MPI_Comm local_comm, int local_leader,
                         MPI_Comm peer_comm, int remote_leader, int tag,
                         MPI_Comm *newintercomm)
int MPI_Intercomm_merge(MPI_Comm intercomm, int high, MPI_Comm *newintracomm)
int MPI_Type_create_keyval(MPI_Type_copy_attr_function *type_copy_attr_fn,
                           MPI_Type_delete_attr_function *type_delete_attr_fn,
                           int *type_keyval, void *extra_state)
int MPI_Type_delete_attr(MPI_Datatype datatype, int type_keyval)
int MPI_TYPE_DUP_FN(MPI_Datatype oldtype, int type_keyval, void *extra_state,
                    void *attribute_val_in, void *attribute_val_out, int *flag)
int MPI_Type_free_keyval(int *type_keyval)
int MPI_Type_get_attr(MPI_Datatype datatype, int type_keyval,
                      void *attribute_val, int *flag)
int MPI_Type_get_name(MPI_Datatype datatype, char *type_name, int *resultlen)
int MPI_TYPE_NULL_COPY_FN(MPI_Datatype oldtype, int type_keyval,
                           void *extra_state, void *attribute_val_in,
                           void *attribute_val_out, int *flag)
int MPI_TYPE_NULL_DELETE_FN(MPI_Datatype datatype, int type_keyval,
                             void *attribute_val, void *extra_state)
int MPI_Type_set_attr(MPI_Datatype datatype, int type_keyval,
                      void *attribute_val)
int MPI_Type_set_name(MPI_Datatype datatype, const char *type_name)
int MPI_Win_create_keyval(MPI_Win_copy_attr_function *win_copy_attr_fn,
                           MPI_Win_delete_attr_function *win_delete_attr_fn,
                           int *win_keyval, void *extra_state)
int MPI_Win_delete_attr(MPI_Win win, int win_keyval)
int MPI_WIN_DUP_FN(MPI_Win oldwin, int win_keyval, void *extra_state,
                   void *attribute_val_in, void *attribute_val_out, int *flag)
int MPI_Win_free_keyval(int *win_keyval)
int MPI_Win_get_attr(MPI_Win win, int win_keyval, void *attribute_val,
                     int *flag)
int MPI_Win_get_name(MPI_Win win, char *win_name, int *resultlen)
int MPI_WIN_NULL_COPY_FN(MPI_Win oldwin, int win_keyval, void *extra_state,
                          void *attribute_val_in, void *attribute_val_out, int *flag)
int MPI_WIN_NULL_DELETE_FN(MPI_Win win, int win_keyval, void *attribute_val,
                            void *extra_state)
int MPI_Win_set_attr(MPI_Win win, int win_keyval, void *attribute_val)
int MPI_Win_set_name(MPI_Win win, const char *win_name)

```

A.3.6 Virtual Topologies for MPI Processes C Bindings

```

1  int MPI_Cart_coords(MPI_Comm comm, int rank, int maxdims, int coords[])
2
3  int MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[],
4                      const int periods[], int reorder, MPI_Comm *comm_cart)
5
6  int MPI_Cart_get(MPI_Comm comm, int maxdims, int dims[], int periods[],
7                  int coords[])
8
9  int MPI_Cart_map(MPI_Comm comm, int ndims, const int dims[],
10                  const int periods[], int *newrank)
11
12 int MPI_Cart_rank(MPI_Comm comm, const int coords[], int *rank)
13
14 int MPI_Cart_shift(MPI_Comm comm, int direction, int disp, int *rank_source,
15                  int *rank_dest)
16
17 int MPI_Cart_sub(MPI_Comm comm, const int remain_dims[], MPI_Comm *newcomm)
18
19 int MPI_Cartdim_get(MPI_Comm comm, int *ndims)
20
21 int MPI_Dims_create(int nnodes, int ndims, int dims[])
22
23 int MPI_Dist_graph_create_adjacent(MPI_Comm comm_old, int indegree,
24                                   const int sources[], const int sourceweights[], int outdegree,
25                                   const int destinations[], const int destweights[], MPI_Info info,
26                                   int reorder, MPI_Comm *comm_dist_graph)
27
28 int MPI_Dist_graph_create(MPI_Comm comm_old, int n, const int sources[],
29                           const int degrees[], const int destinations[],
30                           const int weights[], MPI_Info info, int reorder,
31                           MPI_Comm *comm_dist_graph)
32
33 int MPI_Dist_graph_neighbors_count(MPI_Comm comm, int *indegree,
34                                   int *outdegree, int *weighted)
35
36 int MPI_Dist_graph_neighbors(MPI_Comm comm, int maxindegree, int sources[],
37                              int sourceweights[], int maxoutdegree, int destinations[],
38                              int destweights[])
39
40 int MPI_Graph_create(MPI_Comm comm_old, int nnodes, const int index[],
41                    const int edges[], int reorder, MPI_Comm *comm_graph)
42
43 int MPI_Graph_get(MPI_Comm comm, int maxindex, int maxedges, int index[],
44                  int edges[])
45
46 int MPI_Graph_map(MPI_Comm comm, int nnodes, const int index[],
47                  const int edges[], int *newrank)
48
49 int MPI_Graph_neighbors_count(MPI_Comm comm, int rank, int *nneighbors)
50
51 int MPI_Graph_neighbors(MPI_Comm comm, int rank, int maxneighbors,
52                          int neighbors[])
53
54 int MPI_Graphdims_get(MPI_Comm comm, int *nnodes, int *nedges)
55
56 int MPI_Ineighbor_allgather_c(const void *sendbuf, MPI_Count sendcount,
57                               MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcnt,
58                               MPI_Datatype recvttype, MPI_Comm comm, MPI_Request *request)

```

```

int MPI_Ineighbor_allgather_c(const void *sendbuf, MPI_Count sendcount,
                             MPI_Datatype sendtype, void *recvbuf,
                             const MPI_Count recvcnts[], const MPI_Aint displs[],
                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ineighbor_allgatherv(const void *sendbuf, int sendcount,
                             MPI_Datatype sendtype, void *recvbuf, const int recvcnts[],
                             const int displs[], MPI_Datatype recvtype, MPI_Comm comm,
                             MPI_Request *request)
int MPI_Ineighbor_allgather(const void *sendbuf, int sendcount,
                             MPI_Datatype sendtype, void *recvbuf, int recvcnt,
                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ineighbor_alltoall_c(const void *sendbuf, MPI_Count sendcount,
                             MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcnt,
                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ineighbor_alltoallv_c(const void *sendbuf,
                             const MPI_Count sendcounts[], const MPI_Aint sdispls[],
                             MPI_Datatype sendtype, void *recvbuf,
                             const MPI_Count recvcnts[], const MPI_Aint rdispls[],
                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ineighbor_alltoallv(const void *sendbuf, const int sendcounts[],
                             const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
                             const int recvcnts[], const int rdispls[],
                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Ineighbor_alltoallw_c(const void *sendbuf,
                             const MPI_Count sendcounts[], const MPI_Aint sdispls[],
                             const MPI_Datatype sendtypes[], void *recvbuf,
                             const MPI_Count recvcnts[], const MPI_Aint rdispls[],
                             const MPI_Datatype recvtypes[], MPI_Comm comm,
                             MPI_Request *request)
int MPI_Ineighbor_alltoallw(const void *sendbuf, const int sendcounts[],
                             const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                             void *recvbuf, const int recvcnts[], const MPI_Aint rdispls[],
                             const MPI_Datatype recvtypes[], MPI_Comm comm,
                             MPI_Request *request)
int MPI_Ineighbor_alltoall(const void *sendbuf, int sendcount,
                             MPI_Datatype sendtype, void *recvbuf, int recvcnt,
                             MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
int MPI_Neighbor_allgather_c(const void *sendbuf, MPI_Count sendcount,
                             MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcnt,
                             MPI_Datatype recvtype, MPI_Comm comm)
int MPI_Neighbor_allgather_init_c(const void *sendbuf, MPI_Count sendcount,
                                  MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcnt,
                                  MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
                                  MPI_Request *request)

```

```

1  int MPI_Neighbor_allgather_init(const void *sendbuf, int sendcount,
2      MPI_Datatype sendtype, void *recvbuf, int recvcount,
3      MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
4      MPI_Request *request)
5
6  int MPI_Neighbor_allgatherv_c(const void *sendbuf, MPI_Count sendcount,
7      MPI_Datatype sendtype, void *recvbuf,
8      const MPI_Count recvcounts[], const MPI_Aint displs[],
9      MPI_Datatype recvtype, MPI_Comm comm)
10
11 int MPI_Neighbor_allgatherv_init_c(const void *sendbuf, MPI_Count sendcount,
12     MPI_Datatype sendtype, void *recvbuf,
13     const MPI_Count recvcounts[], const MPI_Aint displs[],
14     MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
15     MPI_Request *request)
16
17 int MPI_Neighbor_allgatherv_init(const void *sendbuf, int sendcount,
18     MPI_Datatype sendtype, void *recvbuf, const int recvcounts[],
19     const int displs[], MPI_Datatype recvtype, MPI_Comm comm,
20     MPI_Info info, MPI_Request *request)
21
22 int MPI_Neighbor_allgatherv(const void *sendbuf, int sendcount,
23     MPI_Datatype sendtype, void *recvbuf, const int recvcounts[],
24     const int displs[], MPI_Datatype recvtype, MPI_Comm comm)
25
26 int MPI_Neighbor_allgather(const void *sendbuf, int sendcount,
27     MPI_Datatype sendtype, void *recvbuf, int recvcount,
28     MPI_Datatype recvtype, MPI_Comm comm)
29
30 int MPI_Neighbor_alltoall_c(const void *sendbuf, MPI_Count sendcount,
31     MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
32     MPI_Datatype recvtype, MPI_Comm comm)
33
34 int MPI_Neighbor_alltoall_init_c(const void *sendbuf, MPI_Count sendcount,
35     MPI_Datatype sendtype, void *recvbuf, MPI_Count recvcount,
36     MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
37     MPI_Request *request)
38
39 int MPI_Neighbor_alltoall_init(const void *sendbuf, int sendcount,
40     MPI_Datatype sendtype, void *recvbuf, int recvcount,
41     MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
42     MPI_Request *request)
43
44 int MPI_Neighbor_alltoallv_c(const void *sendbuf, const MPI_Count sendcounts[],
45     const MPI_Aint sdispls[], MPI_Datatype sendtype, void *recvbuf,
46     const MPI_Count recvcounts[], const MPI_Aint rdispls[],
47     MPI_Datatype recvtype, MPI_Comm comm)
48
49 int MPI_Neighbor_alltoallv_init_c(const void *sendbuf,
50     const MPI_Count sendcounts[], const MPI_Aint sdispls[],
51     MPI_Datatype sendtype, void *recvbuf,
52     const MPI_Count recvcounts[], const MPI_Aint rdispls[],
53     MPI_Datatype recvtype, MPI_Comm comm, MPI_Info info,
54     MPI_Request *request)

```

```

int MPI_Neighbor_alltoallv_init(const void *sendbuf, const int sendcounts[],
                                const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
                                const int recvcounts[], const int rdispls[],
                                MPI_Datatype recvttype, MPI_Comm comm, MPI_Info info,
                                MPI_Request *request)
int MPI_Neighbor_alltoallv(const void *sendbuf, const int sendcounts[],
                            const int sdispls[], MPI_Datatype sendtype, void *recvbuf,
                            const int recvcounts[], const int rdispls[],
                            MPI_Datatype recvttype, MPI_Comm comm)
int MPI_Neighbor_alltoallw_c(const void *sendbuf, const MPI_Count sendcounts[],
                             const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                             void *recvbuf, const MPI_Count recvcounts[],
                             const MPI_Aint rdispls[], const MPI_Datatype recvtypes[],
                             MPI_Comm comm)
int MPI_Neighbor_alltoallw_init_c(const void *sendbuf,
                                   const MPI_Count sendcounts[], const MPI_Aint sdispls[],
                                   const MPI_Datatype sendtypes[], void *recvbuf,
                                   const MPI_Count recvcounts[], const MPI_Aint rdispls[],
                                   const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Info info,
                                   MPI_Request *request)
int MPI_Neighbor_alltoallw_init(const void *sendbuf, const int sendcounts[],
                                 const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                                 void *recvbuf, const int recvcounts[], const MPI_Aint rdispls[],
                                 const MPI_Datatype recvtypes[], MPI_Comm comm, MPI_Info info,
                                 MPI_Request *request)
int MPI_Neighbor_alltoallw(const void *sendbuf, const int sendcounts[],
                           const MPI_Aint sdispls[], const MPI_Datatype sendtypes[],
                           void *recvbuf, const int recvcounts[], const MPI_Aint rdispls[],
                           const MPI_Datatype recvtypes[], MPI_Comm comm)
int MPI_Neighbor_alltoall(const void *sendbuf, int sendcount,
                          MPI_Datatype sendtype, void *recvbuf, int recvcount,
                          MPI_Datatype recvttype, MPI_Comm comm)
int MPI_Topo_test(MPI_Comm comm, int *status)

```

A.3.7 MPI Environmental Management C Bindings

```

int MPI_Add_error_class(int *errorclass)
int MPI_Add_error_code(int errorclass, int *errorcode)
int MPI_Add_error_string(int errorcode, const char *string)
int MPI_Alloc_mem(MPI_Aint size, MPI_Info info, void *baseptr)
int MPI_Comm_call_errhandler(MPI_Comm comm, int errorcode)
int MPI_Comm_create_errhandler(
    MPI_Comm_errhandler_function *comm_errhandler_fn,
    MPI_Errhandler *errhandler)

```



```

1  int MPI_Comm_get_errhandler(MPI_Comm comm, MPI_Errhandler *errhandler)
2  int MPI_Comm_set_errhandler(MPI_Comm comm, MPI_Errhandler errhandler)
3
4  int MPI_Errhandler_free(MPI_Errhandler *errhandler)
5
6  int MPI_Error_class(int errorcode, int *errorclass)
7
8  int MPI_Error_string(int errorcode, char *string, int *resultlen)
9
10 int MPI_File_call_errhandler(MPI_File fh, int errorcode)
11
12 int MPI_File_create_errhandler(
13     MPI_File_errhandler_function *file_errhandler_fn,
14     MPI_Errhandler *errhandler)
15
16 int MPI_File_get_errhandler(MPI_File file, MPI_Errhandler *errhandler)
17
18 int MPI_File_set_errhandler(MPI_File file, MPI_Errhandler errhandler)
19
20 int MPI_Free_mem(void *base)
21
22 int MPI_Get_hw_resource_info(MPI_Info *hw_info)
23
24 int MPI_Get_library_version(char *version, int *resultlen)
25
26 int MPI_Get_processor_name(char *name, int *resultlen)
27
28 int MPI_Get_version(int *version, int *subversion)
29
30 int MPI_Remove_error_class(int errorclass)
31
32 int MPI_Remove_error_code(int errorcode)
33
34 int MPI_Remove_error_string(int errorcode)
35
36 int MPI_Session_call_errhandler(MPI_Session session, int errorcode)
37
38 int MPI_Session_create_errhandler(
39     MPI_Session_errhandler_function *session_errhandler_fn,
40     MPI_Errhandler *errhandler)
41
42 int MPI_Session_get_errhandler(MPI_Session session, MPI_Errhandler *errhandler)
43
44 int MPI_Session_set_errhandler(MPI_Session session, MPI_Errhandler errhandler)
45
46 int MPI_Win_call_errhandler(MPI_Win win, int errorcode)
47
48 int MPI_Win_create_errhandler(MPI_Win_errhandler_function *win_errhandler_fn,
49     MPI_Errhandler *errhandler)
50
51 int MPI_Win_get_errhandler(MPI_Win win, MPI_Errhandler *errhandler)
52
53 int MPI_Win_set_errhandler(MPI_Win win, MPI_Errhandler errhandler)
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

A.3.8 The Info Object C Bindings

```

1  int MPI_Info_create_env(int argc, char *argv[], MPI_Info *info)
2
3  int MPI_Info_create(MPI_Info *info)
4
5  int MPI_Info_delete(MPI_Info info, const char *key)

```



```

int MPI_Info_dup(MPI_Info info, MPI_Info *newinfo)
int MPI_Info_free(MPI_Info *info)
int MPI_Info_get_nkeys(MPI_Info info, int *nkeys)
int MPI_Info_get_nthkey(MPI_Info info, int n, char *key)
int MPI_Info_get_string(MPI_Info info, const char *key, int *buflen,
                        char *value, int *flag)
int MPI_Info_set(MPI_Info info, const char *key, const char *value)

```

A.3.9 Process Creation and Management C Bindings

```

int MPI_Abort(MPI_Comm comm, int errorcode)
int MPI_Close_port(const char *port_name)
int MPI_Comm_accept(const char *port_name, MPI_Info info, int root,
                   MPI_Comm comm, MPI_Comm *newcomm)
int MPI_Comm_connect(const char *port_name, MPI_Info info, int root,
                    MPI_Comm comm, MPI_Comm *newcomm)
int MPI_Comm_disconnect(MPI_Comm *comm)
int MPI_Comm_get_parent(MPI_Comm *parent)
int MPI_Comm_join(int fd, MPI_Comm *intercomm)
int MPI_Comm_spawn_multiple(int count, char *array_of_commands[],
                           char **array_of_argv[], const int array_of_maxprocs[],
                           const MPI_Info array_of_info[], int root, MPI_Comm comm,
                           MPI_Comm *intercomm, int array_of_errcodes[])
int MPI_Comm_spawn(const char *command, char *argv[], int maxprocs,
                  MPI_Info info, int root, MPI_Comm comm, MPI_Comm *intercomm,
                  int array_of_errcodes[])
int MPI_Finalized(int *flag)
int MPI_Finalize(void)
int MPI_Init_thread(int *argc, char ***argv, int required, int *provided)
int MPI_Initialized(int *flag)
int MPI_Init(int *argc, char ***argv)
int MPI_Is_thread_main(int *flag)
int MPI_Lookup_name(const char *service_name, MPI_Info info, char *port_name)
int MPI_Open_port(MPI_Info info, char *port_name)
int MPI_Publish_name(const char *service_name, MPI_Info info,
                    const char *port_name)
int MPI_Query_thread(int *provided)
int MPI_Session_finalize(MPI_Session *session)

```

```

1  int MPI_Session_get_info(MPI_Session session, MPI_Info *info_used)
2
3  int MPI_Session_get_nth_pset(MPI_Session session, MPI_Info info, int n,
4                               int *pset_len, char *pset_name)
5
6  int MPI_Session_get_num_psets(MPI_Session session, MPI_Info info,
7                               int *npset_names)
8
9  int MPI_Session_get_pset_info(MPI_Session session, const char *pset_name,
10                               MPI_Info *info)
11
12 int MPI_Session_init(MPI_Info info, MPI_Errhandler errhandler,
13                     MPI_Session *session)
14
15 int MPI_Unpublish_name(const char *service_name, MPI_Info info,
16                       const char *port_name)

```

A.3.10 One-Sided Communications C Bindings

```

17 int MPI_Accumulate_c(const void *origin_addr, MPI_Count origin_count,
18                    MPI_Datatype origin_datatype, int target_rank,
19                    MPI_Aint target_disp, MPI_Count target_count,
20                    MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
21
22 int MPI_Accumulate(const void *origin_addr, int origin_count,
23                  MPI_Datatype origin_datatype, int target_rank,
24                  MPI_Aint target_disp, int target_count,
25                  MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
26
27 int MPI_Compare_and_swap(const void *origin_addr, const void *compare_addr,
28                          void *result_addr, MPI_Datatype datatype, int target_rank,
29                          MPI_Aint target_disp, MPI_Win win)
30
31 int MPI_Fetch_and_op(const void *origin_addr, void *result_addr,
32                    MPI_Datatype datatype, int target_rank, MPI_Aint target_disp,
33                    MPI_Op op, MPI_Win win)
34
35 int MPI_Get_accumulate_c(const void *origin_addr, MPI_Count origin_count,
36                        MPI_Datatype origin_datatype, void *result_addr,
37                        MPI_Count result_count, MPI_Datatype result_datatype,
38                        int target_rank, MPI_Aint target_disp, MPI_Count target_count,
39                        MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
40
41 int MPI_Get_accumulate(const void *origin_addr, int origin_count,
42                      MPI_Datatype origin_datatype, void *result_addr,
43                      int result_count, MPI_Datatype result_datatype, int target_rank,
44                      MPI_Aint target_disp, int target_count,
45                      MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
46
47 int MPI_Get_c(void *origin_addr, MPI_Count origin_count,
48              MPI_Datatype origin_datatype, int target_rank,
49              MPI_Aint target_disp, MPI_Count target_count,
50              MPI_Datatype target_datatype, MPI_Win win)

```

```

int MPI_Get(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, 1
            int target_rank, MPI_Aint target_disp, int target_count, 2
            MPI_Datatype target_datatype, MPI_Win win) 3
4
int MPI_Put_c(const void *origin_addr, MPI_Count origin_count, 5
            MPI_Datatype origin_datatype, int target_rank, 6
            MPI_Aint target_disp, MPI_Count target_count, 7
            MPI_Datatype target_datatype, MPI_Win win) 8
9
int MPI_Put(const void *origin_addr, int origin_count, 10
            MPI_Datatype origin_datatype, int target_rank, 11
            MPI_Aint target_disp, int target_count, 12
            MPI_Datatype target_datatype, MPI_Win win) 13
14
int MPI_Raccumulate_c(const void *origin_addr, MPI_Count origin_count, 15
            MPI_Datatype origin_datatype, int target_rank, 16
            MPI_Aint target_disp, MPI_Count target_count, 17
            MPI_Datatype target_datatype, MPI_Op op, MPI_Win win, 18
            MPI_Request *request) 19
20
int MPI_Raccumulate(const void *origin_addr, int origin_count, 21
            MPI_Datatype origin_datatype, int target_rank, 22
            MPI_Aint target_disp, int target_count, 23
            MPI_Datatype target_datatype, MPI_Op op, MPI_Win win, 24
            MPI_Request *request) 25
26
int MPI_Rget_accumulate_c(const void *origin_addr, MPI_Count origin_count, 27
            MPI_Datatype origin_datatype, void *result_addr, 28
            MPI_Count result_count, MPI_Datatype result_datatype, 29
            int target_rank, MPI_Aint target_disp, MPI_Count target_count, 30
            MPI_Datatype target_datatype, MPI_Op op, MPI_Win win, 31
            MPI_Request *request) 32
33
int MPI_Rget_accumulate(const void *origin_addr, int origin_count, 34
            MPI_Datatype origin_datatype, void *result_addr, 35
            int result_count, MPI_Datatype result_datatype, int target_rank, 36
            MPI_Aint target_disp, int target_count, 37
            MPI_Datatype target_datatype, MPI_Op op, MPI_Win win, 38
            MPI_Request *request) 39
40
int MPI_Rget_c(void *origin_addr, MPI_Count origin_count, 41
            MPI_Datatype origin_datatype, int target_rank, 42
            MPI_Aint target_disp, MPI_Count target_count, 43
            MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request) 44
45
int MPI_Rput_c(const void *origin_addr, MPI_Count origin_count, 46
            MPI_Datatype origin_datatype, int target_rank, 47
            MPI_Aint target_disp, MPI_Count target_count, 48
            MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)

```

```

1  int MPI_Rput(const void *origin_addr, int origin_count,
2              MPI_Datatype origin_datatype, int target_rank,
3              MPI_Aint target_disp, int target_count,
4              MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)
5
6  int MPI_Win_allocate_c(MPI_Aint size, MPI_Aint disp_unit, MPI_Info info,
7                        MPI_Comm comm, void *baseptr, MPI_Win *win)
8
9  int MPI_Win_allocate_shared_c(MPI_Aint size, MPI_Aint disp_unit, MPI_Info info,
10                              MPI_Comm comm, void *baseptr, MPI_Win *win)
11
12 int MPI_Win_allocate_shared(MPI_Aint size, int disp_unit, MPI_Info info,
13                             MPI_Comm comm, void *baseptr, MPI_Win *win)
14
15 int MPI_Win_allocate(MPI_Aint size, int disp_unit, MPI_Info info,
16                     MPI_Comm comm, void *baseptr, MPI_Win *win)
17
18 int MPI_Win_attach(MPI_Win win, void *base, MPI_Aint size)
19
20 int MPI_Win_complete(MPI_Win win)
21
22 int MPI_Win_create_c(void *base, MPI_Aint size, MPI_Aint disp_unit,
23                    MPI_Info info, MPI_Comm comm, MPI_Win *win)
24
25 int MPI_Win_create_dynamic(MPI_Info info, MPI_Comm comm, MPI_Win *win)
26
27 int MPI_Win_create(void *base, MPI_Aint size, int disp_unit, MPI_Info info,
28                  MPI_Comm comm, MPI_Win *win)
29
30 int MPI_Win_detach(MPI_Win win, const void *base)
31
32 int MPI_Win_fence(int assert, MPI_Win win)
33
34 int MPI_Win_flush_all(MPI_Win win)
35
36 int MPI_Win_flush_local_all(MPI_Win win)
37
38 int MPI_Win_flush_local(int rank, MPI_Win win)
39
40 int MPI_Win_flush(int rank, MPI_Win win)
41
42 int MPI_Win_free(MPI_Win *win)
43
44 int MPI_Win_get_group(MPI_Win win, MPI_Group *group)
45
46 int MPI_Win_get_info(MPI_Win win, MPI_Info *info_used)
47
48 int MPI_Win_lock_all(int assert, MPI_Win win)
49
50 int MPI_Win_lock(int lock_type, int rank, int assert, MPI_Win win)
51
52 int MPI_Win_post(MPI_Group group, int assert, MPI_Win win)
53
54 int MPI_Win_set_info(MPI_Win win, MPI_Info info)
55
56 int MPI_Win_shared_query_c(MPI_Win win, int rank, MPI_Aint *size,
57                           MPI_Aint *disp_unit, void *baseptr)
58
59 int MPI_Win_shared_query(MPI_Win win, int rank, MPI_Aint *size, int *disp_unit,
60                         void *baseptr)
61
62 int MPI_Win_start(MPI_Group group, int assert, MPI_Win win)

```

```

int MPI_Win_sync(MPI_Win win)
int MPI_Win_test(MPI_Win win, int *flag)
int MPI_Win_unlock_all(MPI_Win win)
int MPI_Win_unlock(int rank, MPI_Win win)
int MPI_Win_wait(MPI_Win win)

```

A.3.11 External Interfaces C Bindings

```

int MPI_Grequest_complete(MPI_Request request)
int MPI_Grequest_start(MPI_Grequest_query_function *query_fn,
                      MPI_Grequest_free_function *free_fn,
                      MPI_Grequest_cancel_function *cancel_fn, void *extra_state,
                      MPI_Request *request)
int MPI_Status_set_cancelled(MPI_Status *status, int flag)
int MPI_Status_set_elements_c(MPI_Status *status, MPI_Datatype datatype,
                             MPI_Count count)
int MPI_Status_set_elements(MPI_Status *status, MPI_Datatype datatype,
                             int count)
int MPI_Status_set_error(MPI_Status *status, int err)
int MPI_Status_set_source(MPI_Status *status, int source)
int MPI_Status_set_tag(MPI_Status *status, int tag)

```

A.3.12 I/O C Bindings

```

int MPI_CONVERSION_FN_NULL_C(void *userbuf, MPI_Datatype datatype,
                             MPI_Count count, void *filebuf, MPI_Offset position,
                             void *extra_state)
int MPI_CONVERSION_FN_NULL(void *userbuf, MPI_Datatype datatype, int count,
                             void *filebuf, MPI_Offset position, void *extra_state)
int MPI_File_close(MPI_File *fh)
int MPI_File_delete(const char *filename, MPI_Info info)
int MPI_File_get_amode(MPI_File fh, int *amode)
int MPI_File_get_atomicity(MPI_File fh, int *flag)
int MPI_File_get_byte_offset(MPI_File fh, MPI_Offset offset, MPI_Offset *disp)
int MPI_File_get_group(MPI_File fh, MPI_Group *group)
int MPI_File_get_info(MPI_File fh, MPI_Info *info_used)
int MPI_File_get_position_shared(MPI_File fh, MPI_Offset *offset)
int MPI_File_get_position(MPI_File fh, MPI_Offset *offset)

```

```

1  int MPI_File_get_size(MPI_File fh, MPI_Offset *size)
2
3  int MPI_File_get_type_extent_c(MPI_File fh, MPI_Datatype datatype,
4                                MPI_Count *extent)
5
6  int MPI_File_get_type_extent(MPI_File fh, MPI_Datatype datatype,
7                                MPI_Aint *extent)
8
9  int MPI_File_get_view(MPI_File fh, MPI_Offset *disp, MPI_Datatype *etype,
10                        MPI_Datatype *filetype, char *datarep)
11
12 int MPI_File_iread_all_c(MPI_File fh, void *buf, MPI_Count count,
13                          MPI_Datatype datatype, MPI_Request *request)
14
15 int MPI_File_iread_all(MPI_File fh, void *buf, int count,
16                        MPI_Datatype datatype, MPI_Request *request)
17
18 int MPI_File_iread_at_all_c(MPI_File fh, MPI_Offset offset, void *buf,
19                             MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
20
21 int MPI_File_iread_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count,
22                           MPI_Datatype datatype, MPI_Request *request)
23
24 int MPI_File_iread_at_c(MPI_File fh, MPI_Offset offset, void *buf,
25                         MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
26
27 int MPI_File_iread_at(MPI_File fh, MPI_Offset offset, void *buf, int count,
28                       MPI_Datatype datatype, MPI_Request *request)
29
30 int MPI_File_iread_c(MPI_File fh, void *buf, MPI_Count count,
31                     MPI_Datatype datatype, MPI_Request *request)
32
33 int MPI_File_iread_shared_c(MPI_File fh, void *buf, MPI_Count count,
34                             MPI_Datatype datatype, MPI_Request *request)
35
36 int MPI_File_iread_shared(MPI_File fh, void *buf, int count,
37                           MPI_Datatype datatype, MPI_Request *request)
38
39 int MPI_File_iread(MPI_File fh, void *buf, int count, MPI_Datatype datatype,
40                   MPI_Request *request)
41
42 int MPI_File_iwrite_all_c(MPI_File fh, const void *buf, MPI_Count count,
43                           MPI_Datatype datatype, MPI_Request *request)
44
45 int MPI_File_iwrite_all(MPI_File fh, const void *buf, int count,
46                         MPI_Datatype datatype, MPI_Request *request)
47
48 int MPI_File_iwrite_at_all_c(MPI_File fh, MPI_Offset offset, const void *buf,
49                             MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
50
51 int MPI_File_iwrite_at_all(MPI_File fh, MPI_Offset offset, const void *buf,
52                             int count, MPI_Datatype datatype, MPI_Request *request)
53
54 int MPI_File_iwrite_at_c(MPI_File fh, MPI_Offset offset, const void *buf,
55                         MPI_Count count, MPI_Datatype datatype, MPI_Request *request)
56
57 int MPI_File_iwrite_at(MPI_File fh, MPI_Offset offset, const void *buf,
58                       int count, MPI_Datatype datatype, MPI_Request *request)

```

```

int MPI_File_iwrite_c(MPI_File fh, const void *buf, MPI_Count count,      1
                      MPI_Datatype datatype, MPI_Request *request)        2
int MPI_File_iwrite_shared_c(MPI_File fh, const void *buf, MPI_Count count, 3
                             MPI_Datatype datatype, MPI_Request *request) 4
int MPI_File_iwrite_shared(MPI_File fh, const void *buf, int count,        5
                             MPI_Datatype datatype, MPI_Request *request) 6
int MPI_File_iwrite(MPI_File fh, const void *buf, int count,              7
                     MPI_Datatype datatype, MPI_Request *request)          8
int MPI_File_open(MPI_Comm comm, const char *filename, int amode,          9
                  MPI_Info info, MPI_File *fh)                           10
int MPI_File_preallocate(MPI_File fh, MPI_Offset size)                   11
int MPI_File_read_all_begin_c(MPI_File fh, void *buf, MPI_Count count,    12
                              MPI_Datatype datatype)                     13
int MPI_File_read_all_begin(MPI_File fh, void *buf, int count,            14
                              MPI_Datatype datatype)                     15
int MPI_File_read_all_c(MPI_File fh, void *buf, MPI_Count count,          16
                        MPI_Datatype datatype, MPI_Status *status)        17
int MPI_File_read_all_end(MPI_File fh, void *buf, MPI_Status *status)     18
int MPI_File_read_all(MPI_File fh, void *buf, int count, MPI_Datatype datatype, 19
                      MPI_Status *status)                                 20
int MPI_File_read_at_all_begin_c(MPI_File fh, MPI_Offset offset, void *buf, 21
                                 MPI_Count count, MPI_Datatype datatype) 22
int MPI_File_read_at_all_begin(MPI_File fh, MPI_Offset offset, void *buf, 23
                                int count, MPI_Datatype datatype)         24
int MPI_File_read_at_all_c(MPI_File fh, MPI_Offset offset, void *buf,     25
                           MPI_Count count, MPI_Datatype datatype, MPI_Status *status) 26
int MPI_File_read_at_all_end(MPI_File fh, void *buf, MPI_Status *status) 27
int MPI_File_read_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count, 28
                          MPI_Datatype datatype, MPI_Status *status)      29
int MPI_File_read_at_c(MPI_File fh, MPI_Offset offset, void *buf,         30
                       MPI_Count count, MPI_Datatype datatype, MPI_Status *status) 31
int MPI_File_read_at(MPI_File fh, MPI_Offset offset, void *buf, int count, 32
                     MPI_Datatype datatype, MPI_Status *status)           33
int MPI_File_read_c(MPI_File fh, void *buf, MPI_Count count,             34
                    MPI_Datatype datatype, MPI_Status *status)            35
int MPI_File_read(MPI_File fh, void *buf, MPI_Count count, MPI_Datatype datatype, 36
                  MPI_Status *status)                                     37
int MPI_File_read_ordered_begin_c(MPI_File fh, void *buf, MPI_Count count, 38
                                  MPI_Datatype datatype)                 39
int MPI_File_read_ordered_begin(MPI_File fh, void *buf, int count,        40
                                 MPI_Datatype datatype)                   41
int MPI_File_read_ordered_c(MPI_File fh, void *buf, MPI_Count count,      42
                             MPI_Datatype datatype, MPI_Status *status)   43
int MPI_File_read_ordered(MPI_File fh, void *buf, MPI_Count count, MPI_Datatype datatype, 44
                           MPI_Status *status)                           45
int MPI_File_read_ordered_c(MPI_File fh, void *buf, MPI_Count count,      46
                             MPI_Datatype datatype, MPI_Status *status)   47
int MPI_File_read_ordered(MPI_File fh, void *buf, int count, MPI_Datatype datatype) 48

```



```

1  int MPI_File_read_ordered_c(MPI_File fh, void *buf, MPI_Count count,
2      MPI_Datatype datatype, MPI_Status *status)
3
4  int MPI_File_read_ordered_end(MPI_File fh, void *buf, MPI_Status *status)
5
6  int MPI_File_read_ordered(MPI_File fh, void *buf, int count,
7      MPI_Datatype datatype, MPI_Status *status)
8
9  int MPI_File_read_shared_c(MPI_File fh, void *buf, MPI_Count count,
10     MPI_Datatype datatype, MPI_Status *status)
11
12 int MPI_File_read_shared(MPI_File fh, void *buf, int count,
13     MPI_Datatype datatype, MPI_Status *status)
14
15 int MPI_File_read(MPI_File fh, void *buf, int count, MPI_Datatype datatype,
16     MPI_Status *status)
17
18 int MPI_File_seek_shared(MPI_File fh, MPI_Offset offset, int whence)
19
20 int MPI_File_seek(MPI_File fh, MPI_Offset offset, int whence)
21
22 int MPI_File_set_atomicity(MPI_File fh, int flag)
23
24 int MPI_File_set_info(MPI_File fh, MPI_Info info)
25
26 int MPI_File_set_size(MPI_File fh, MPI_Offset size)
27
28 int MPI_File_set_view(MPI_File fh, MPI_Offset disp, MPI_Datatype etype,
29     MPI_Datatype filetype, const char *datarep, MPI_Info info)
30
31 int MPI_File_sync(MPI_File fh)
32
33 int MPI_File_write_all_begin_c(MPI_File fh, const void *buf, MPI_Count count,
34     MPI_Datatype datatype)
35
36 int MPI_File_write_all_begin(MPI_File fh, const void *buf, int count,
37     MPI_Datatype datatype)
38
39 int MPI_File_write_all_c(MPI_File fh, const void *buf, MPI_Count count,
40     MPI_Datatype datatype, MPI_Status *status)
41
42 int MPI_File_write_all_end(MPI_File fh, const void *buf, MPI_Status *status)
43
44 int MPI_File_write_all(MPI_File fh, const void *buf, int count,
45     MPI_Datatype datatype, MPI_Status *status)
46
47 int MPI_File_write_at_all_begin_c(MPI_File fh, MPI_Offset offset,
48     const void *buf, MPI_Count count, MPI_Datatype datatype)
49
50 int MPI_File_write_at_all_begin(MPI_File fh, MPI_Offset offset,
51     const void *buf, int count, MPI_Datatype datatype)
52
53 int MPI_File_write_at_all_c(MPI_File fh, MPI_Offset offset, const void *buf,
54     MPI_Count count, MPI_Datatype datatype, MPI_Status *status)
55
56 int MPI_File_write_at_all_end(MPI_File fh, const void *buf, MPI_Status *status)
57
58 int MPI_File_write_at_all(MPI_File fh, MPI_Offset offset, const void *buf,
59     int count, MPI_Datatype datatype, MPI_Status *status)
60
61 int MPI_File_write_at_c(MPI_File fh, MPI_Offset offset, const void *buf,
62     MPI_Count count, MPI_Datatype datatype, MPI_Status *status)

```



```

int MPI_File_write_at(MPI_File fh, MPI_Offset offset, const void *buf,
    int count, MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write_c(MPI_File fh, const void *buf, MPI_Count count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write_ordered_begin_c(MPI_File fh, const void *buf,
    MPI_Count count, MPI_Datatype datatype)
int MPI_File_write_ordered_begin(MPI_File fh, const void *buf, int count,
    MPI_Datatype datatype)
int MPI_File_write_ordered_c(MPI_File fh, const void *buf, MPI_Count count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write_ordered_end(MPI_File fh, const void *buf,
    MPI_Status *status)
int MPI_File_write_ordered(MPI_File fh, const void *buf, int count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write_shared_c(MPI_File fh, const void *buf, MPI_Count count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write_shared(MPI_File fh, const void *buf, int count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_File_write(MPI_File fh, const void *buf, int count,
    MPI_Datatype datatype, MPI_Status *status)
int MPI_Register_datarep_c(const char *datarep,
    MPI_Datarep_conversion_function_c *read_conversion_fn,
    MPI_Datarep_conversion_function_c *write_conversion_fn,
    MPI_Datarep_extent_function *dtype_file_extent_fn,
    void *extra_state)
int MPI_Register_datarep(const char *datarep,
    MPI_Datarep_conversion_function *read_conversion_fn,
    MPI_Datarep_conversion_function *write_conversion_fn,
    MPI_Datarep_extent_function *dtype_file_extent_fn,
    void *extra_state)

```

A.3.13 Language Bindings C Bindings

```

MPI_Fint MPI_Comm_c2f(MPI_Comm comm)
MPI_Comm MPI_Comm_f2c(MPI_Fint comm)
MPI_Fint MPI_Errhandler_c2f(MPI_Errhandler errhandler)
MPI_Errhandler MPI_Errhandler_f2c(MPI_Fint errhandler)
MPI_Fint MPI_File_c2f(MPI_File file)
MPI_File MPI_File_f2c(MPI_Fint file)
MPI_Fint MPI_Group_c2f(MPI_Group group)

```

```

1 MPI_Group MPI_Group_f2c(MPI_Fint group)
2 MPI_Fint MPI_Info_c2f(MPI_Info info)
3 MPI_Info MPI_Info_f2c(MPI_Fint info)
4 MPI_Fint MPI_Message_c2f(MPI_Message message)
5 MPI_Message MPI_Message_f2c(MPI_Fint message)
6 MPI_Fint MPI_Op_c2f(MPI_Op op)
7 MPI_Op MPI_Op_f2c(MPI_Fint op)
8 MPI_Fint MPI_Request_c2f(MPI_Request request)
9 MPI_Request MPI_Request_f2c(MPI_Fint request)
10 MPI_Fint MPI_Session_c2f(MPI_Session session)
11 MPI_Session MPI_Session_f2c(MPI_Fint session)
12
13 int MPI_Status_c2f08(const MPI_Status *c_status, MPI_F08_status *f08_status)
14 int MPI_Status_c2f(const MPI_Status *c_status, MPI_Fint *f_status)
15 int MPI_Status_f082c(const MPI_F08_status *f08_status, MPI_Status *c_status)
16 int MPI_Status_f082f(const MPI_F08_status *f08_status, MPI_Fint *f_status)
17 int MPI_Status_f2c(const MPI_Fint *f_status, MPI_Status *c_status)
18 int MPI_Status_f2f08(const MPI_Fint *f_status, MPI_F08_status *f08_status)
19 MPI_Fint MPI_Type_c2f(MPI_Datatype datatype)
20 int MPI_Type_create_f90_complex(int p, int r, MPI_Datatype *newtype)
21 int MPI_Type_create_f90_integer(int r, MPI_Datatype *newtype)
22 int MPI_Type_create_f90_real(int p, int r, MPI_Datatype *newtype)
23 MPI_Datatype MPI_Type_f2c(MPI_Fint datatype)
24 int MPI_Type_match_size(int typeclass, int size, MPI_Datatype *datatype)
25 MPI_Fint MPI_Win_c2f(MPI_Win win)
26 MPI_Win MPI_Win_f2c(MPI_Fint win)
27

```

A.3.14 Application Binary Interface (ABI) C Bindings

```

39 int MPI_Abi_get_fortran_booleans(int logical_size, void *logical_true,
40 void *logical_false, int *is_set)
41
42 int MPI_Abi_get_fortran_info(MPI_Info *info)
43 int MPI_Abi_get_info(MPI_Info *info)
44 int MPI_Abi_get_version(int *abi_major, int *abi_minor)
45 int MPI_Abi_set_fortran_booleans(int logical_size, void *logical_true,
46 void *logical_false)
47

```

```

int MPI_Abi_set_fortran_info(MPI_Info info) 1
MPI_Comm MPI_Comm_fromint(int comm) 2
int MPI_Comm_toint(MPI_Comm comm) 3
MPI_Errhandler MPI_Errhandler_fromint(int errhandler) 4
int MPI_Errhandler_toint(MPI_Errhandler errhandler) 5
MPI_File MPI_File_fromint(int file) 6
int MPI_File_toint(MPI_File file) 7
MPI_Group MPI_Group_fromint(int group) 8
int MPI_Group_toint(MPI_Group group) 9
MPI_Info MPI_Info_fromint(int info) 10
int MPI_Info_toint(MPI_Info info) 11
MPI_Message MPI_Message_fromint(int message) 12
int MPI_Message_toint(MPI_Message message) 13
MPI_Op MPI_Op_fromint(int op) 14
int MPI_Op_toint(MPI_Op op) 15
MPI_Queue MPI_Queue_fromint(int queue) 16
int MPI_Queue_toint(MPI_Queue queue) 17
MPI_Request MPI_Request_fromint(int request) 18
int MPI_Request_toint(MPI_Request request) 19
MPI_Session MPI_Session_fromint(int session) 20
int MPI_Session_toint(MPI_Session session) 21
MPI_Datatype MPI_Type_fromint(int datatype) 22
int MPI_Type_toint(MPI_Datatype datatype) 23
MPI_Win MPI_Win_fromint(int win) 24
int MPI_Win_toint(MPI_Win win) 25

```

A.3.15 Tools / Profiling Interface C Bindings

```
int MPI_Pcontrol(const int level, . . . )
```

A.3.16 Tools / MPI Tool Information Interface C Bindings

```

int MPI_T_category_changed(int *update_number)
int MPI_T_category_get_categories(int cat_index, int len, int indices[])
int MPI_T_category_get_cvars(int cat_index, int len, int indices[])
int MPI_T_category_get_events(int cat_index, int len, int indices[])

```

```

1  int MPI_T_category_get_index(const char *name, int *cat_index)
2
3  int MPI_T_category_get_info(int cat_index, char *name, int *name_len,
4                             char *desc, int *desc_len, int *num_cvars, int *num_pvars,
5                             int *num_categories)
6
7  int MPI_T_category_get_num_events(int cat_index, int *num_events)
8
9  int MPI_T_category_get_num(int *num_cat)
10
11 int MPI_T_category_get_pvars(int cat_index, int len, int indices[])
12
13 int MPI_T_cvar_get_index(const char *name, int *cvar_index)
14
15 int MPI_T_cvar_get_info(int cvar_index, char *name, int *name_len,
16                         int *verbosity, MPI_Datatype *datatype, MPI_T_enum *enumtype,
17                         char *desc, int *desc_len, int *bind, int *scope)
18
19 int MPI_T_cvar_get_num(int *num_cvar)
20
21 int MPI_T_cvar_handle_alloc(int cvar_index, void *obj_handle,
22                             MPI_T_cvar_handle *handle, int *count)
23
24 int MPI_T_cvar_handle_free(MPI_T_cvar_handle *handle)
25
26 int MPI_T_cvar_read(MPI_T_cvar_handle handle, void *buf)
27
28 int MPI_T_cvar_write(MPI_T_cvar_handle handle, const void *buf)
29
30 int MPI_T_enum_get_info(MPI_T_enum enumtype, int *num, char *name,
31                         int *name_len)
32
33 int MPI_T_enum_get_item(MPI_T_enum enumtype, int index, int *value, char *name,
34                         int *name_len)
35
36 int MPI_T_event_callback_get_info(MPI_T_event_registration event_registration,
37                                   MPI_T_cb_safety cb_safety, MPI_Info *info_used)
38
39 int MPI_T_event_callback_set_info(MPI_T_event_registration event_registration,
40                                   MPI_T_cb_safety cb_safety, MPI_Info info)
41
42 int MPI_T_event_copy(MPI_T_event_instance event_instance, void *buffer)
43
44 int MPI_T_event_get_index(const char *name, int *event_index)
45
46 int MPI_T_event_get_info(int event_index, char *name, int *name_len,
47                         int *verbosity, MPI_Datatype array_of_datatypes[],
48                         MPI_Aint array_of_displacements[], int *num_elements,
49                         MPI_T_enum *enumtype, MPI_Info *info, char *desc, int *desc_len,
50                         int *bind)
51
52 int MPI_T_event_get_num(int *num_events)
53
54 int MPI_T_event_get_source(MPI_T_event_instance event_instance,
55                             int *source_index)
56
57 int MPI_T_event_get_timestamp(MPI_T_event_instance event_instance,
58                               MPI_Count *event_timestamp)
59
60 int MPI_T_event_handle_alloc(int event_index, void *obj_handle, MPI_Info info,
61                               MPI_T_event_registration *event_registration)

```

```

int MPI_T_event_handle_free(MPI_T_event_registration event_registration, 1
                           void *user_data, MPI_T_event_free_cb_function free_cb_function) 2
int MPI_T_event_handle_get_info(MPI_T_event_registration event_registration, 3
                               MPI_Info *info_used) 4
int MPI_T_event_handle_set_info(MPI_T_event_registration event_registration, 5
                               MPI_Info info) 6
int MPI_T_event_read(MPI_T_event_instance event_instance, int element_index, 7
                    void *buffer) 8
int MPI_T_event_register_callback(MPI_T_event_registration event_registration, 9
                                  MPI_T_cb_safety cb_safety, MPI_Info info, void *user_data, 10
                                  MPI_T_event_cb_function event_cb_function) 11
int MPI_T_event_set_dropped_handler( 12
    MPI_T_event_registration event_registration, 13
    MPI_T_event_dropped_cb_function dropped_cb_function) 14
int MPI_T_finalize(void) 15
int MPI_T_init_thread(int required, int *provided) 16
int MPI_T_pvar_get_index(const char *name, int var_class, int *pvar_index) 17
int MPI_T_pvar_get_info(int pvar_index, char *name, int *name_len, 18
                        int *verbosity, int *var_class, MPI_Datatype *datatype, 19
                        MPI_T_enum *enumtype, char *desc, int *desc_len, int *bind, 20
                        int *readonly, int *continuous, int *atomic) 21
int MPI_T_pvar_get_num(int *num_pvar) 22
int MPI_T_pvar_handle_alloc(MPI_T_pvar_session pe_session, int pvar_index, 23
                            void *obj_handle, MPI_T_pvar_handle *handle, int *count) 24
int MPI_T_pvar_handle_free(MPI_T_pvar_session pe_session, 25
                           MPI_T_pvar_handle *handle) 26
int MPI_T_pvar_readreset(MPI_T_pvar_session pe_session, 27
                        MPI_T_pvar_handle handle, void *buf) 28
int MPI_T_pvar_read(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle, 29
                   void *buf) 30
int MPI_T_pvar_reset(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle) 31
int MPI_T_pvar_session_create(MPI_T_pvar_session *pe_session) 32
int MPI_T_pvar_session_free(MPI_T_pvar_session *pe_session) 33
int MPI_T_pvar_start(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle) 34
int MPI_T_pvar_stop(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle) 35
int MPI_T_pvar_write(MPI_T_pvar_session pe_session, MPI_T_pvar_handle handle, 36
                    const void *buf) 37
int MPI_T_source_get_info(int source_index, char *name, int *name_len, 38
                          char *desc, int *desc_len, MPI_T_source_order *ordering, 39
                          40
                          41
                          42
                          43
                          44
                          45
                          46
                          47
                          48

```

```

1         MPI_Count *ticks_per_second, MPI_Count *max_ticks,
2         MPI_Info *info)
3
4     int MPI_T_source_get_num(int *num_sources)
5
6     int MPI_T_source_get_timestamp(int source_index, MPI_Count *timestamp)
7
8

```

A.3.17 Deprecated C Bindings

```

9     int MPI_Attr_delete(MPI_Comm comm, int keyval)
10
11     int MPI_Attr_get(MPI_Comm comm, int keyval, void *attribute_val, int *flag)
12
13     int MPI_Attr_put(MPI_Comm comm, int keyval, void *attribute_val)
14
15     int MPI_DUP_FN(MPI_Comm oldcomm, int keyval, void *extra_state,
16                   void *attribute_val_in, void *attribute_val_out, int *flag)
17
18     int MPI_Get_elements_x(const MPI_Status *status, MPI_Datatype datatype,
19                           MPI_Count *count)
20
21     int MPI_Info_get_valuelen(MPI_Info info, const char *key, int *valuelen,
22                               int *flag)
23
24     int MPI_Info_get(MPI_Info info, const char *key, int valuelen, char *value,
25                       int *flag)
26
27     int MPI_Keyval_create(MPI_Copy_function *copy_fn,
28                           MPI_Delete_function *delete_fn, int *keyval, void *extra_state)
29
30     int MPI_Keyval_free(int *keyval)
31
32     int MPI_NULL_COPY_FN(MPI_Comm oldcomm, int keyval, void *extra_state,
33                           void *attribute_val_in, void *attribute_val_out, int *flag)
34
35     int MPI_NULL_DELETE_FN(MPI_Comm comm, int keyval, void *attribute_val,
36                             void *extra_state)
37
38     int MPI_Status_set_elements_x(MPI_Status *status, MPI_Datatype datatype,
39                                   MPI_Count count)
40
41     int MPI_Type_get_extent_x(MPI_Datatype datatype, MPI_Count *lb,
42                               MPI_Count *extent)
43
44     int MPI_Type_get_true_extent_x(MPI_Datatype datatype, MPI_Count *true_lb,
45                                    MPI_Count *true_extent)
46
47     int MPI_Type_size_x(MPI_Datatype datatype, MPI_Count *size)
48

```

A.4 Fortran 2008 Bindings with the mpi_f08 Module

A.4.1 Point-to-Point Communication Fortran 2008 Bindings

```

MPI_Bsend(buf, count, datatype, dest, tag, comm, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER, INTENT(IN) :: count, dest, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Bsend(buf, count, datatype, dest, tag, comm, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Bsend_init(buf, count, datatype, dest, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, dest, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Bsend_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Buffer_attach(buffer, size, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
  INTEGER, INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Buffer_attach(buffer, size, ierror) !(_c)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Buffer_detach(buffer_addr, size, ierror)
  USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
  TYPE(C_PTR), INTENT(OUT) :: buffer_addr
  INTEGER, INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Buffer_detach(buffer_addr, size, ierror) !(_c)
2      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
3      TYPE(C_PTR), INTENT(OUT) :: buffer_addr
4      INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
5      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
6
7  MPI_Buffer_flush(ierror)
8      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
9
10 MPI_Buffer_iflush(request, ierror)
11     TYPE(MPI_Request), INTENT(OUT) :: request
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14 MPI_Cancel(request, ierror)
15     TYPE(MPI_Request), INTENT(IN) :: request
16     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18 MPI_Comm_attach_buffer(comm, buffer, size, ierror)
19     TYPE(MPI_Comm), INTENT(IN) :: comm
20     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
21     INTEGER, INTENT(IN) :: size
22     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24 MPI_Comm_attach_buffer(comm, buffer, size, ierror) !(_c)
25     TYPE(MPI_Comm), INTENT(IN) :: comm
26     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
27     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: size
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30 MPI_Comm_detach_buffer(comm, buffer_addr, size, ierror)
31     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
32     TYPE(MPI_Comm), INTENT(IN) :: comm
33     TYPE(C_PTR), INTENT(OUT) :: buffer_addr
34     INTEGER, INTENT(OUT) :: size
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37 MPI_Comm_detach_buffer(comm, buffer_addr, size, ierror) !(_c)
38     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
39     TYPE(MPI_Comm), INTENT(IN) :: comm
40     TYPE(C_PTR), INTENT(OUT) :: buffer_addr
41     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
42     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
43
44 MPI_Comm_flush_buffer(comm, ierror)
45     TYPE(MPI_Comm), INTENT(IN) :: comm
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
47
48 MPI_Comm_iflush_buffer(comm, request, ierror)
49     TYPE(MPI_Comm), INTENT(IN) :: comm
50     TYPE(MPI_Request), INTENT(OUT) :: request
51     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
52
53 MPI_Get_count(status, datatype, count, ierror)
54     TYPE(MPI_Status), INTENT(IN) :: status

```



```

TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(OUT) :: count
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get_count(status, datatype, count, ierror) !(_c)
TYPE(MPI_Status), INTENT(IN) :: status
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: count
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ibsend(buf, count, datatype, dest, tag, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count, dest, tag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ibsend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: dest, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Improbe(source, tag, comm, flag, message, status, ierror)
INTEGER, INTENT(IN) :: source, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
LOGICAL, INTENT(OUT) :: flag
TYPE(MPI_Message), INTENT(OUT) :: message
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Imrecv(buf, count, datatype, message, request, ierror)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Message), INTENT(INOUT) :: message
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Imrecv(buf, count, datatype, message, request, ierror) !(_c)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Message), INTENT(INOUT) :: message
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Iprobe(source, tag, comm, flag, status, ierror)
2      INTEGER, INTENT(IN) :: source, tag
3      TYPE(MPI_Comm), INTENT(IN) :: comm
4      LOGICAL, INTENT(OUT) :: flag
5      TYPE(MPI_Status) :: status
6      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
7
8  MPI_Irecv(buf, count, datatype, source, tag, comm, request, ierror)
9      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
10     INTEGER, INTENT(IN) :: count, source, tag
11     TYPE(MPI_Datatype), INTENT(IN) :: datatype
12     TYPE(MPI_Comm), INTENT(IN) :: comm
13     TYPE(MPI_Request), INTENT(OUT) :: request
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_Irecv(buf, count, datatype, source, tag, comm, request, ierror) !(_c)
17     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
18     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
19     TYPE(MPI_Datatype), INTENT(IN) :: datatype
20     INTEGER, INTENT(IN) :: source, tag
21     TYPE(MPI_Comm), INTENT(IN) :: comm
22     TYPE(MPI_Request), INTENT(OUT) :: request
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24
25 MPI_Irsend(buf, count, datatype, dest, tag, comm, request, ierror)
26     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
27     INTEGER, INTENT(IN) :: count, dest, tag
28     TYPE(MPI_Datatype), INTENT(IN) :: datatype
29     TYPE(MPI_Comm), INTENT(IN) :: comm
30     TYPE(MPI_Request), INTENT(OUT) :: request
31     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
32
33 MPI_Irsend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
34     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
35     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
36     TYPE(MPI_Datatype), INTENT(IN) :: datatype
37     INTEGER, INTENT(IN) :: dest, tag
38     TYPE(MPI_Comm), INTENT(IN) :: comm
39     TYPE(MPI_Request), INTENT(OUT) :: request
40     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
41
42 MPI_Isend(buf, count, datatype, dest, tag, comm, request, ierror)
43     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
44     INTEGER, INTENT(IN) :: count, dest, tag
45     TYPE(MPI_Datatype), INTENT(IN) :: datatype
46     TYPE(MPI_Comm), INTENT(IN) :: comm
47     TYPE(MPI_Request), INTENT(OUT) :: request
48     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
49
50 MPI_Isend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
51     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf

```

```

INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: dest, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Isendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
              recvtype, source, recvtag, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, dest, sendtag, recvcount, source, recvtag
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Isendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
              recvtype, source, recvtag, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Isendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                     comm, request, ierror)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count, dest, sendtag, source, recvtag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Isendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                     comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Issend(buf, count, datatype, dest, tag, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count, dest, tag
TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

1      TYPE(MPI_Comm), INTENT(IN) :: comm
2      TYPE(MPI_Request), INTENT(OUT) :: request
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
4
5      MPI_Issend(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
6      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
7      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
8      TYPE(MPI_Datatype), INTENT(IN) :: datatype
9      INTEGER, INTENT(IN) :: dest, tag
10     TYPE(MPI_Comm), INTENT(IN) :: comm
11     TYPE(MPI_Request), INTENT(OUT) :: request
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14     MPI_Mprobe(source, tag, comm, message, status, ierror)
15     INTEGER, INTENT(IN) :: source, tag
16     TYPE(MPI_Comm), INTENT(IN) :: comm
17     TYPE(MPI_Message), INTENT(OUT) :: message
18     TYPE(MPI_Status) :: status
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21     MPI_Mrecv(buf, count, datatype, message, status, ierror)
22     TYPE(*), DIMENSION(..) :: buf
23     INTEGER, INTENT(IN) :: count
24     TYPE(MPI_Datatype), INTENT(IN) :: datatype
25     TYPE(MPI_Message), INTENT(INOUT) :: message
26     TYPE(MPI_Status) :: status
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
28
29     MPI_Mrecv(buf, count, datatype, message, status, ierror) !(_c)
30     TYPE(*), DIMENSION(..) :: buf
31     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
32     TYPE(MPI_Datatype), INTENT(IN) :: datatype
33     TYPE(MPI_Message), INTENT(INOUT) :: message
34     TYPE(MPI_Status) :: status
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37     MPI_Probe(source, tag, comm, status, ierror)
38     INTEGER, INTENT(IN) :: source, tag
39     TYPE(MPI_Comm), INTENT(IN) :: comm
40     TYPE(MPI_Status) :: status
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43     MPI_Recv(buf, count, datatype, source, tag, comm, status, ierror)
44     TYPE(*), DIMENSION(..) :: buf
45     INTEGER, INTENT(IN) :: count, source, tag
46     TYPE(MPI_Datatype), INTENT(IN) :: datatype
47     TYPE(MPI_Comm), INTENT(IN) :: comm
48     TYPE(MPI_Status) :: status
49     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
50
51     MPI_Recv(buf, count, datatype, source, tag, comm, status, ierror) !(_c)
52     TYPE(*), DIMENSION(..) :: buf

```

```

INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: source, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Recv_init(buf, count, datatype, source, tag, comm, request, ierror)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count, source, tag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Recv_init(buf, count, datatype, source, tag, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: source, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Request_free(request, ierror)
TYPE(MPI_Request), INTENT(INOUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Request_get_status(request, flag, status, ierror)
TYPE(MPI_Request), INTENT(IN) :: request
LOGICAL, INTENT(OUT) :: flag
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Request_get_status_all(count, array_of_requests, flag, array_of_statuses,
                           ierror)
INTEGER, INTENT(IN) :: count
TYPE(MPI_Request), INTENT(IN) :: array_of_requests(count)
LOGICAL, INTENT(OUT) :: flag
TYPE(MPI_Status) :: array_of_statuses(*)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Request_get_status_any(count, array_of_requests, index, flag, status,
                           ierror)
INTEGER, INTENT(IN) :: count
TYPE(MPI_Request), INTENT(IN) :: array_of_requests(count)
INTEGER, INTENT(OUT) :: index
LOGICAL, INTENT(OUT) :: flag
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Request_get_status_some(incount, array_of_requests, outcount,
2      array_of_indices, array_of_statuses, ierror)
3      INTEGER, INTENT(IN) :: incount
4      TYPE(MPI_Request), INTENT(IN) :: array_of_requests(incount)
5      INTEGER, INTENT(OUT) :: outcount, array_of_indices(*)
6      TYPE(MPI_Status) :: array_of_statuses(*)
7      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9  MPI_Rsend(buf, count, datatype, dest, tag, comm, ierror)
10     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
11     INTEGER, INTENT(IN) :: count, dest, tag
12     TYPE(MPI_Datatype), INTENT(IN) :: datatype
13     TYPE(MPI_Comm), INTENT(IN) :: comm
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_Rsend(buf, count, datatype, dest, tag, comm, ierror) !(_c)
17     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
18     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
19     TYPE(MPI_Datatype), INTENT(IN) :: datatype
20     INTEGER, INTENT(IN) :: dest, tag
21     TYPE(MPI_Comm), INTENT(IN) :: comm
22     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24 MPI_Rsend_init(buf, count, datatype, dest, tag, comm, request, ierror)
25     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
26     INTEGER, INTENT(IN) :: count, dest, tag
27     TYPE(MPI_Datatype), INTENT(IN) :: datatype
28     TYPE(MPI_Comm), INTENT(IN) :: comm
29     TYPE(MPI_Request), INTENT(OUT) :: request
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_Rsend_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
33     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
34     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
35     TYPE(MPI_Datatype), INTENT(IN) :: datatype
36     INTEGER, INTENT(IN) :: dest, tag
37     TYPE(MPI_Comm), INTENT(IN) :: comm
38     TYPE(MPI_Request), INTENT(OUT) :: request
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41 MPI_Send(buf, count, datatype, dest, tag, comm, ierror)
42     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
43     INTEGER, INTENT(IN) :: count, dest, tag
44     TYPE(MPI_Datatype), INTENT(IN) :: datatype
45     TYPE(MPI_Comm), INTENT(IN) :: comm
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
47
48 MPI_Send(buf, count, datatype, dest, tag, comm, ierror) !(_c)
49     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
50     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
51     TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

INTEGER, INTENT(IN) :: dest, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Send_init(buf, count, datatype, dest, tag, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count, dest, tag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Send_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(IN) :: dest, tag
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Sendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
              recvtype, source, recvtag, comm, status, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, dest, sendtag, recvcount, source, recvtag
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Sendrecv(sendbuf, sendcount, sendtype, dest, sendtag, recvbuf, recvcount,
              recvtype, source, recvtag, comm, status, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Sendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
                     comm, status, ierror)
TYPE(*), DIMENSION(..) :: buf
INTEGER, INTENT(IN) :: count, dest, sendtag, source, recvtag
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

1  MPI_Sendrecv_replace(buf, count, datatype, dest, sendtag, source, recvtag,
2      comm, status, ierror) !(_c)
3      TYPE(*), DIMENSION(..) :: buf
4      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
5      TYPE(MPI_Datatype), INTENT(IN) :: datatype
6      INTEGER, INTENT(IN) :: dest, sendtag, source, recvtag
7      TYPE(MPI_Comm), INTENT(IN) :: comm
8      TYPE(MPI_Status) :: status
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_Session_attach_buffer(session, buffer, size, ierror)
12     TYPE(MPI_Session), INTENT(IN) :: session
13     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
14     INTEGER, INTENT(IN) :: size
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17 MPI_Session_attach_buffer(session, buffer, size, ierror) !(_c)
18     TYPE(MPI_Session), INTENT(IN) :: session
19     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
20     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: size
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
22
23 MPI_Session_detach_buffer(session, buffer_addr, size, ierror)
24     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
25     TYPE(MPI_Session), INTENT(IN) :: session
26     TYPE(C_PTR), INTENT(OUT) :: buffer_addr
27     INTEGER, INTENT(OUT) :: size
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30 MPI_Session_detach_buffer(session, buffer_addr, size, ierror) !(_c)
31     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
32     TYPE(MPI_Session), INTENT(IN) :: session
33     TYPE(C_PTR), INTENT(OUT) :: buffer_addr
34     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37 MPI_Session_flush_buffer(session, ierror)
38     TYPE(MPI_Session), INTENT(IN) :: session
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41 MPI_Session_iflush_buffer(session, request, ierror)
42     TYPE(MPI_Session), INTENT(IN) :: session
43     TYPE(MPI_Request), INTENT(OUT) :: request
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46 MPI_Ssend(buf, count, datatype, dest, tag, comm, ierror)
47     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
48     INTEGER, INTENT(IN) :: count, dest, tag
49     TYPE(MPI_Datatype), INTENT(IN) :: datatype
50     TYPE(MPI_Comm), INTENT(IN) :: comm
51     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

MPI_Ssend(buf, count, datatype, dest, tag, comm, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ssend_init(buf, count, datatype, dest, tag, comm, request, ierror)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count, dest, tag
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ssend_init(buf, count, datatype, dest, tag, comm, request, ierror) !(_c)
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  INTEGER, INTENT(IN) :: dest, tag
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Start(request, ierror)
  TYPE(MPI_Request), INTENT(INOUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Startall(count, array_of_requests, ierror)
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Status_get_error(status, err, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
  INTEGER, INTENT(OUT) :: err
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Status_get_source(status, source, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
  INTEGER, INTENT(OUT) :: source
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Status_get_tag(status, tag, ierror)
  TYPE(MPI_Status), INTENT(IN) :: status
  INTEGER, INTENT(OUT) :: tag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Test(request, flag, status, ierror)
  TYPE(MPI_Request), INTENT(INOUT) :: request
  LOGICAL, INTENT(OUT) :: flag

```

```

1      TYPE(MPI_Status) :: status
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4      MPI_Test_cancelled(status, flag, ierror)
5          TYPE(MPI_Status), INTENT(IN) :: status
6          LOGICAL, INTENT(OUT) :: flag
7          INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9      MPI_Testall(count, array_of_requests, flag, array_of_statuses, ierror)
10         INTEGER, INTENT(IN) :: count
11         TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
12         LOGICAL, INTENT(OUT) :: flag
13         TYPE(MPI_Status) :: array_of_statuses(*)
14         INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16     MPI_Testany(count, array_of_requests, index, flag, status, ierror)
17         INTEGER, INTENT(IN) :: count
18         TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
19         INTEGER, INTENT(OUT) :: index
20         LOGICAL, INTENT(OUT) :: flag
21         TYPE(MPI_Status) :: status
22         INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24     MPI_Testsome(incount, array_of_requests, outcount, array_of_indices,
25                  array_of_statuses, ierror)
26         INTEGER, INTENT(IN) :: incount
27         TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(incount)
28         INTEGER, INTENT(OUT) :: outcount, array_of_indices(*)
29         TYPE(MPI_Status) :: array_of_statuses(*)
30         INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32     MPI_Wait(request, status, ierror)
33         TYPE(MPI_Request), INTENT(INOUT) :: request
34         TYPE(MPI_Status) :: status
35         INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37     MPI_Waitall(count, array_of_requests, array_of_statuses, ierror)
38         INTEGER, INTENT(IN) :: count
39         TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
40         TYPE(MPI_Status) :: array_of_statuses(*)
41         INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43     MPI_Waitany(count, array_of_requests, index, status, ierror)
44         INTEGER, INTENT(IN) :: count
45         TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(count)
46         INTEGER, INTENT(OUT) :: index
47         TYPE(MPI_Status) :: status
48         INTEGER, OPTIONAL, INTENT(OUT) :: ierror
49
50     MPI_Waitsome(incount, array_of_requests, outcount, array_of_indices,
51                  array_of_statuses, ierror)
52         INTEGER, INTENT(IN) :: incount

```

```

TYPE(MPI_Request), INTENT(INOUT) :: array_of_requests(incount)
INTEGER, INTENT(OUT) :: outcount, array_of_indices(*)
TYPE(MPI_Status) :: array_of_statuses(*)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.2 Partitioned Communication Fortran 2008 Bindings

MPI_Parrived(request, partition, flag, ierror)

```

TYPE(MPI_Request), INTENT(IN) :: request
INTEGER, INTENT(IN) :: partition
LOGICAL, INTENT(OUT) :: flag
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

MPI_Pready(partition, request, ierror)

```

INTEGER, INTENT(IN) :: partition
TYPE(MPI_Request), INTENT(IN) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

MPI_Pready_list(length, array_of_partitions, request, ierror)

```

INTEGER, INTENT(IN) :: length, array_of_partitions(length)
TYPE(MPI_Request), INTENT(IN) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

MPI_Pready_range(partition_low, partition_high, request, ierror)

```

INTEGER, INTENT(IN) :: partition_low, partition_high
TYPE(MPI_Request), INTENT(IN) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

MPI_Precv_init(buf, partitions, count, datatype, source, tag, comm, info, request, ierror)

```

TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER, INTENT(IN) :: partitions, source, tag
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

MPI_Psend_init(buf, partitions, count, datatype, dest, tag, comm, info, request, ierror)

```

TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER, INTENT(IN) :: partitions, dest, tag
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.3 Datatypes Fortran 2008 Bindings

```

1  INTEGER(KIND=MPI_ADDRESS_KIND) MPI_Aint_add(base, disp)
2      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: base, disp
3
4  INTEGER(KIND=MPI_ADDRESS_KIND) MPI_Aint_diff(addr1, addr2)
5      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: addr1, addr2
6
7  MPI_Get_address(location, address, ierror)
8      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: location
9      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: address
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12 MPI_Get_elements(status, datatype, count, ierror)
13     TYPE(MPI_Status), INTENT(IN) :: status
14     TYPE(MPI_Datatype), INTENT(IN) :: datatype
15     INTEGER, INTENT(OUT) :: count
16     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18 MPI_Get_elements(status, datatype, count, ierror) !(_c)
19     TYPE(MPI_Status), INTENT(IN) :: status
20     TYPE(MPI_Datatype), INTENT(IN) :: datatype
21     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: count
22     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24 MPI_Pack(inbuf, incout, datatype, outbuf, outsize, position, comm, ierror)
25     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
26     INTEGER, INTENT(IN) :: incout, outsize
27     TYPE(MPI_Datatype), INTENT(IN) :: datatype
28     TYPE(*), DIMENSION(..) :: outbuf
29     INTEGER, INTENT(INOUT) :: position
30     TYPE(MPI_Comm), INTENT(IN) :: comm
31     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
32
33 MPI_Pack(inbuf, incout, datatype, outbuf, outsize, position, comm, ierror)
34     !(_c)
35     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
36     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incout, outsize
37     TYPE(MPI_Datatype), INTENT(IN) :: datatype
38     TYPE(*), DIMENSION(..) :: outbuf
39     INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
40     TYPE(MPI_Comm), INTENT(IN) :: comm
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_Pack_external(datatype, inbuf, incout, datatype, outbuf, outsize, position,
44     ierror)
45     CHARACTER(LEN=*) , INTENT(IN) :: datatype
46     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
47     INTEGER, INTENT(IN) :: incout
48     TYPE(MPI_Datatype), INTENT(IN) :: datatype
49     TYPE(*), DIMENSION(..) :: outbuf
50     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: outsize

```

```

INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(INOUT) :: position
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Pack_external(datarep, inbuf, incout, datatype, outbuf, outsize, position,
    ierror) !(_c)
CHARACTER(LEN=*), INTENT(IN) :: datarep
TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incout, outsize
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(*), DIMENSION(..) :: outbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Pack_external_size(datarep, incout, datatype, size, ierror)
CHARACTER(LEN=*), INTENT(IN) :: datarep
INTEGER, INTENT(IN) :: incout
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Pack_external_size(datarep, incout, datatype, size, ierror) !(_c)
CHARACTER(LEN=*), INTENT(IN) :: datarep
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incout
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Pack_size(incout, datatype, comm, size, ierror)
INTEGER, INTENT(IN) :: incout
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Pack_size(incout, datatype, comm, size, ierror) !(_c)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: incout
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Type_commit(datatype, ierror)
TYPE(MPI_Datatype), INTENT(INOUT) :: datatype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Type_contiguous(count, oldtype, newtype, ierror)
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Type_contiguous(count, oldtype, newtype, ierror) !(_c)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count

```

```

1      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
2      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
4
5      MPI_Type_create_darray(size, rank, ndims, array_of_gsizes, array_of_distribs,
6                          array_of_dargs, array_of_psize, order, oldtype, newtype, ierror)
7      INTEGER, INTENT(IN) :: size, rank, ndims, array_of_gsizes(ndims),
8                          array_of_distribs(ndims), array_of_dargs(ndims),
9                          array_of_psize(ndims), order
10     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
11     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14     MPI_Type_create_darray(size, rank, ndims, array_of_gsizes, array_of_distribs,
15                         array_of_dargs, array_of_psize, order, oldtype, newtype, ierror)
16     !(_c)
17     INTEGER, INTENT(IN) :: size, rank, ndims, array_of_distribs(ndims),
18                         array_of_dargs(ndims), array_of_psize(ndims), order
19     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: array_of_gsizes(ndims)
20     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
21     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
22     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
23
24     MPI_Type_create_hindexed(count, array_of_blocklengths, array_of_displacements,
25                             oldtype, newtype, ierror)
26     INTEGER, INTENT(IN) :: count, array_of_blocklengths(count)
27     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: array_of_displacements(count)
28     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
29     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32     MPI_Type_create_hindexed(count, array_of_blocklengths, array_of_displacements,
33                             oldtype, newtype, ierror) !(_c)
34     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count,
35                         array_of_blocklengths(count), array_of_displacements(count)
36     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
37     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
39
40     MPI_Type_create_hindexed_block(count, blocklength, array_of_displacements,
41                                   oldtype, newtype, ierror)
42     INTEGER, INTENT(IN) :: count, blocklength
43     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: array_of_displacements(count)
44     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
45     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
47
48     MPI_Type_create_hindexed_block(count, blocklength, array_of_displacements,
49                                   oldtype, newtype, ierror) !(_c)
50     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength,
51                         array_of_displacements(count)

```

```

TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_hvector(count, blocklength, stride, oldtype, newtype, ierror)
INTEGER, INTENT(IN) :: count, blocklength
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: stride
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_hvector(count, blocklength, stride, oldtype, newtype, ierror)
!(c)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength, stride
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_indexed_block(count, blocklength, array_of_displacements,
oldtype, newtype, ierror)
INTEGER, INTENT(IN) :: count, blocklength, array_of_displacements(count)
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_indexed_block(count, blocklength, array_of_displacements,
oldtype, newtype, ierror)!(c)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength,
array_of_displacements(count)
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_resized(oldtype, lb, extent, newtype, ierror)
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: lb, extent
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_resized(oldtype, lb, extent, newtype, ierror)!(c)
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: lb, extent
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_struct(count, array_of_blocklengths, array_of_displacements,
array_of_types, newtype, ierror)
INTEGER, INTENT(IN) :: count, array_of_blocklengths(count)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: array_of_displacements(count)
TYPE(MPI_Datatype), INTENT(IN) :: array_of_types(count)
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

1  MPI_Type_create_struct(count, array_of_blocklengths, array_of_displacements,
2      array_of_types, newtype, ierror) !(_c)
3      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count,
4      array_of_blocklengths(count), array_of_displacements(count)
5      TYPE(MPI_Datatype), INTENT(IN) :: array_of_types(count)
6      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
7      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9  MPI_Type_create_subarray(ndims, array_of_sizes, array_of_subsizes,
10      array_of_starts, order, oldtype, newtype, ierror)
11      INTEGER, INTENT(IN) :: ndims, array_of_sizes(ndims),
12      array_of_subsizes(ndims), array_of_starts(ndims), order
13      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
14      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
15      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17  MPI_Type_create_subarray(ndims, array_of_sizes, array_of_subsizes,
18      array_of_starts, order, oldtype, newtype, ierror) !(_c)
19      INTEGER, INTENT(IN) :: ndims, order
20      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: array_of_sizes(ndims),
21      array_of_subsizes(ndims), array_of_starts(ndims)
22      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
23      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
24      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26  MPI_Type_dup(oldtype, newtype, ierror)
27      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
28      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
29      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
30
31  MPI_Type_free(datatype, ierror)
32      TYPE(MPI_Datatype), INTENT(INOUT) :: datatype
33      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
34
35  MPI_Type_get_contents(datatype, max_integers, max_addresses, max_datatypes,
36      array_of_integers, array_of_addresses, array_of_datatypes,
37      ierror)
38      TYPE(MPI_Datatype), INTENT(IN) :: datatype
39      INTEGER, INTENT(IN) :: max_integers, max_addresses, max_datatypes
40      INTEGER, INTENT(OUT) :: array_of_integers(max_integers)
41      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) ::
42      array_of_addresses(max_addresses)
43      TYPE(MPI_Datatype), INTENT(OUT) :: array_of_datatypes(max_datatypes)
44      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46  MPI_Type_get_contents(datatype, max_integers, max_addresses, max_large_counts,
47      max_datatypes, array_of_integers, array_of_addresses,
48      array_of_large_counts, array_of_datatypes, ierror) !(_c)
49      TYPE(MPI_Datatype), INTENT(IN) :: datatype
50      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: max_integers, max_addresses,
51      max_large_counts, max_datatypes

```



```

INTEGER, INTENT(OUT) :: array_of_integers(max_integers)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) ::
    array_of_addresses(max_addresses)
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) ::
    array_of_large_counts(max_large_counts)
TYPE(MPI_Datatype), INTENT(OUT) :: array_of_datatypes(max_datatypes)
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_envelope(datatype, num_integers, num_addresses, num_datatypes,
    combiner, ierror)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, INTENT(OUT) :: num_integers, num_addresses, num_datatypes,
    combiner
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_envelope(datatype, num_integers, num_addresses, num_large_counts,
    num_datatypes, combiner, ierror) !(_c)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: num_integers, num_addresses,
    num_large_counts, num_datatypes
INTEGER, INTENT(OUT) :: combiner
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_extent(datatype, lb, extent, ierror)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: lb, extent
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_extent(datatype, lb, extent, ierror) !(_c)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: lb, extent
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_true_extent(datatype, true_lb, true_extent, ierror)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: true_lb, true_extent
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_get_true_extent(datatype, true_lb, true_extent, ierror) !(_c)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: true_lb, true_extent
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_indexed(count, array_of_blocklengths, array_of_displacements, oldtype,
    newtype, ierror)
INTEGER, INTENT(IN) :: count, array_of_blocklengths(count),
    array_of_displacements(count)
TYPE(MPI_Datatype), INTENT(IN) :: oldtype
TYPE(MPI_Datatype), INTENT(OUT) :: newtype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Type_indexed(count, array_of_blocklengths, array_of_displacements, oldtype,
2      newtype, ierror) !(_c)
3      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count,
4      array_of_blocklengths(count), array_of_displacements(count)
5      TYPE(MPI_Datatype), INTENT(IN) :: oldtype
6      TYPE(MPI_Datatype), INTENT(OUT) :: newtype
7      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9  MPI_Type_size(datatype, size, ierror)
10     TYPE(MPI_Datatype), INTENT(IN) :: datatype
11     INTEGER, INTENT(OUT) :: size
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14  MPI_Type_size(datatype, size, ierror) !(_c)
15     TYPE(MPI_Datatype), INTENT(IN) :: datatype
16     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
17     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
18
19  MPI_Type_vector(count, blocklength, stride, oldtype, newtype, ierror)
20     INTEGER, INTENT(IN) :: count, blocklength, stride
21     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
22     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24
25  MPI_Type_vector(count, blocklength, stride, oldtype, newtype, ierror) !(_c)
26     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count, blocklength, stride
27     TYPE(MPI_Datatype), INTENT(IN) :: oldtype
28     TYPE(MPI_Datatype), INTENT(OUT) :: newtype
29     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
30
31  MPI_Unpack(inbuf, insize, position, outbuf, outcount, datatype, comm, ierror)
32     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
33     INTEGER, INTENT(IN) :: insize, outcount
34     INTEGER, INTENT(INOUT) :: position
35     TYPE(*), DIMENSION(..) :: outbuf
36     TYPE(MPI_Datatype), INTENT(IN) :: datatype
37     TYPE(MPI_Comm), INTENT(IN) :: comm
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
39
40  MPI_Unpack(inbuf, insize, position, outbuf, outcount, datatype, comm, ierror)
41     !(_c)
42     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
43     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: insize, outcount
44     INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
45     TYPE(*), DIMENSION(..) :: outbuf
46     TYPE(MPI_Datatype), INTENT(IN) :: datatype
47     TYPE(MPI_Comm), INTENT(IN) :: comm
48     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
49
50  MPI_Unpack_external(datarep, inbuf, insize, position, outbuf, outcount,
51     datatype, ierror)
52     CHARACTER(LEN=*), INTENT(IN) :: datarep

```

```

TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: insize
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(INOUT) :: position
TYPE(*), DIMENSION(..) :: outbuf
INTEGER, INTENT(IN) :: outcount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Unpack_external(datarep, inbuf, insize, position, outbuf, outcount,
                    datatype, ierror) !(_c)
CHARACTER(LEN=*), INTENT(IN) :: datarep
TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: insize, outcount
INTEGER(KIND=MPI_COUNT_KIND), INTENT(INOUT) :: position
TYPE(*), DIMENSION(..) :: outbuf
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

A.4.4 Collective Communication Fortran 2008 Bindings

MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
              ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
              ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                   comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtpe,
2                      comm, info, request, ierror) !(_c)
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
5      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
6      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
7      TYPE(MPI_Comm), INTENT(IN) :: comm
8      TYPE(MPI_Info), INTENT(IN) :: info
9      TYPE(MPI_Request), INTENT(OUT) :: request
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12  MPI_Allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
13                recvtpe, comm, ierror)
14      TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
15      INTEGER, INTENT(IN) :: sendcount, recvcounts(*), displs(*)
16      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
17      TYPE(*), DIMENSION(..) :: recvbuf
18      TYPE(MPI_Comm), INTENT(IN) :: comm
19      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21  MPI_Allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
22                recvtpe, comm, ierror) !(_c)
23      TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
24      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcounts(*)
25      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
26      TYPE(*), DIMENSION(..) :: recvbuf
27      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
28      TYPE(MPI_Comm), INTENT(IN) :: comm
29      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
30
31  MPI_Allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
32                      recvtpe, comm, info, request, ierror)
33      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
34      INTEGER, INTENT(IN) :: sendcount
35      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
36      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
37      INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*), displs(*)
38      TYPE(MPI_Comm), INTENT(IN) :: comm
39      TYPE(MPI_Info), INTENT(IN) :: info
40      TYPE(MPI_Request), INTENT(OUT) :: request
41      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43  MPI_Allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
44                      recvtpe, comm, info, request, ierror) !(_c)
45      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
46      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
47      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtpe
48      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
49      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
50      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)

```

```

TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Allreduce_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
                   ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Allreduce_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
                   ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtpe, comm,
             ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcnt

```

```

1      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
2      TYPE(*), DIMENSION(..) :: recvbuf
3      TYPE(MPI_Comm), INTENT(IN) :: comm
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6      MPI_Alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
7                  ierror) !(_c)
8      TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
9      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
10     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
11     TYPE(*), DIMENSION(..) :: recvbuf
12     TYPE(MPI_Comm), INTENT(IN) :: comm
13     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
14
15     MPI_Alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
16                      comm, info, request, ierror)
17     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
18     INTEGER, INTENT(IN) :: sendcount, recvcount
19     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
20     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
21     TYPE(MPI_Comm), INTENT(IN) :: comm
22     TYPE(MPI_Info), INTENT(IN) :: info
23     TYPE(MPI_Request), INTENT(OUT) :: request
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26     MPI_Alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
27                      comm, info, request, ierror) !(_c)
28     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
29     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
30     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
31     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
32     TYPE(MPI_Comm), INTENT(IN) :: comm
33     TYPE(MPI_Info), INTENT(IN) :: info
34     TYPE(MPI_Request), INTENT(OUT) :: request
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37     MPI_Alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
38                  rdispls, recvtype, comm, ierror)
39     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
40     INTEGER, INTENT(IN) :: sendcounts(*), sdispls(*), recvcounts(*), rdispls(*)
41     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
42     TYPE(*), DIMENSION(..) :: recvbuf
43     TYPE(MPI_Comm), INTENT(IN) :: comm
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46     MPI_Alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
47                  rdispls, recvtype, comm, ierror) !(_c)
48     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
49     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcounts(*)
50     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
51     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype

```

```

TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcoun-
    rdispls, recvtype, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
    recvcoun-
    rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcoun-
    rdispls, recvtype, comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
    recvcoun-
    rdispls(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
    rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun-
    rdispls, recvtypes, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcounts(*), sdispls(*), recvcoun-
    rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun-
    rdispls, recvtypes, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcoun-
    rdispls(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

1  MPI_Alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
2      recvcounts, rdispls, recvtypes, comm, info, request, ierror)
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4      INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
5      recvcounts(*), rdispls(*)
6      TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
7      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
8      TYPE(MPI_Comm), INTENT(IN) :: comm
9      TYPE(MPI_Info), INTENT(IN) :: info
10     TYPE(MPI_Request), INTENT(OUT) :: request
11     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
12
13 MPI_Alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
14     recvcounts, rdispls, recvtypes, comm, info, request, ierror)
15     !(_c)
16     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
17     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
18     recvcounts(*)
19     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
20     rdispls(*)
21     TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
22     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
23     TYPE(MPI_Comm), INTENT(IN) :: comm
24     TYPE(MPI_Info), INTENT(IN) :: info
25     TYPE(MPI_Request), INTENT(OUT) :: request
26     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
27
28 MPI_Barrier(comm, ierror)
29     TYPE(MPI_Comm), INTENT(IN) :: comm
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_Barrier_init(comm, info, request, ierror)
33     TYPE(MPI_Comm), INTENT(IN) :: comm
34     TYPE(MPI_Info), INTENT(IN) :: info
35     TYPE(MPI_Request), INTENT(OUT) :: request
36     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Bcast(buffer, count, datatype, root, comm, ierror)
39     TYPE(*), DIMENSION(..) :: buffer
40     INTEGER, INTENT(IN) :: count, root
41     TYPE(MPI_Datatype), INTENT(IN) :: datatype
42     TYPE(MPI_Comm), INTENT(IN) :: comm
43     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
44
45 MPI_Bcast(buffer, count, datatype, root, comm, ierror) !(_c)
46     TYPE(*), DIMENSION(..) :: buffer
47     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
48     TYPE(MPI_Datatype), INTENT(IN) :: datatype
49     INTEGER, INTENT(IN) :: root
50     TYPE(MPI_Comm), INTENT(IN) :: comm
51     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

MPI_Bcast_init(buffer, count, datatype, root, comm, info, request, ierror)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
    INTEGER, INTENT(IN) :: count, root
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Bcast_init(buffer, count, datatype, root, comm, info, request, ierror)
    !(_c)
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, INTENT(IN) :: root
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Exscan(sendbuf, recvbuf, count, datatype, op, comm, ierror)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    TYPE(*), DIMENSION(..) :: recvbuf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Exscan(sendbuf, recvbuf, count, datatype, op, comm, ierror) !(_c)
    TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
    TYPE(*), DIMENSION(..) :: recvbuf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Exscan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
    ierror)
    TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Op), INTENT(IN) :: op
    TYPE(MPI_Comm), INTENT(IN) :: comm
    TYPE(MPI_Info), INTENT(IN) :: info
    TYPE(MPI_Request), INTENT(OUT) :: request
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Exscan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
2      ierror) !(_c)
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
5      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
6      TYPE(MPI_Datatype), INTENT(IN) :: datatype
7      TYPE(MPI_Op), INTENT(IN) :: op
8      TYPE(MPI_Comm), INTENT(IN) :: comm
9      TYPE(MPI_Info), INTENT(IN) :: info
10     TYPE(MPI_Request), INTENT(OUT) :: request
11     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
12
13 MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
14     comm, ierror)
15     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
16     INTEGER, INTENT(IN) :: sendcount, recvcount, root
17     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
18     TYPE(*), DIMENSION(..) :: recvbuf
19     TYPE(MPI_Comm), INTENT(IN) :: comm
20     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
21
22 MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
23     comm, ierror) !(_c)
24     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
25     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
26     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
27     TYPE(*), DIMENSION(..) :: recvbuf
28     INTEGER, INTENT(IN) :: root
29     TYPE(MPI_Comm), INTENT(IN) :: comm
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_Gather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
33     root, comm, info, request, ierror)
34     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
35     INTEGER, INTENT(IN) :: sendcount, recvcount, root
36     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
37     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
38     TYPE(MPI_Comm), INTENT(IN) :: comm
39     TYPE(MPI_Info), INTENT(IN) :: info
40     TYPE(MPI_Request), INTENT(OUT) :: request
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_Gather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
44     root, comm, info, request, ierror) !(_c)
45     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
46     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
47     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
48     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
49     INTEGER, INTENT(IN) :: root
50     TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Gatherv(sendbuf, sendcount, sendtype, recvbuf, recvcnt, displs,
            recvtype, root, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcnt(*), displs(*), root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Gatherv(sendbuf, sendcount, sendtype, recvbuf, recvcnt, displs,
            recvtype, root, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcnt(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Gatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcnt, displs,
                recvtype, root, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcnt(*), displs(*)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Gatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcnt, displs,
                recvtype, root, comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcnt(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Iallgather(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype,
2                comm, request, ierror)
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4      INTEGER, INTENT(IN) :: sendcount, recvcnt
5      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
6      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
7      TYPE(MPI_Comm), INTENT(IN) :: comm
8      TYPE(MPI_Request), INTENT(OUT) :: request
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_Iallgather(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype,
12               comm, request, ierror) !(_c)
13     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
14     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcnt
15     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
16     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
17     TYPE(MPI_Comm), INTENT(IN) :: comm
18     TYPE(MPI_Request), INTENT(OUT) :: request
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21 MPI_Iallgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcnts, displs,
22                recvtype, comm, request, ierror)
23     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
24     INTEGER, INTENT(IN) :: sendcount
25     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
26     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
27     INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcnts(*), displs(*)
28     TYPE(MPI_Comm), INTENT(IN) :: comm
29     TYPE(MPI_Request), INTENT(OUT) :: request
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_Iallgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcnts, displs,
33                recvtype, comm, request, ierror) !(_c)
34     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
35     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
36     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
37     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
38     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcnts(*)
39     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
40     TYPE(MPI_Comm), INTENT(IN) :: comm
41     TYPE(MPI_Request), INTENT(OUT) :: request
42     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
43
44 MPI_Iallreduce(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
45     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
46     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
47     INTEGER, INTENT(IN) :: count
48     TYPE(MPI_Datatype), INTENT(IN) :: datatype
49     TYPE(MPI_Op), INTENT(IN) :: op
50     TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iallreduce(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
    !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ialltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
    request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ialltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm,
    request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ialltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
    rdispls, recvtype, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
    recvcounts(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ialltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf, recvcounts,
    rdispls, recvtype, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
    recvcounts(*)

```

```

1      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
2          rdispls(*)
3      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
4      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
5      TYPE(MPI_Comm), INTENT(IN) :: comm
6      TYPE(MPI_Request), INTENT(OUT) :: request
7      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9  MPI_Ialltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun
10      rdispls, recvtypes, comm, request, ierror)
11      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
12      INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
13          recvcoun
14      TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
15      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
16      TYPE(MPI_Comm), INTENT(IN) :: comm
17      TYPE(MPI_Request), INTENT(OUT) :: request
18      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
19
20 MPI_Ialltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf, recvcoun
21      rdispls, recvtypes, comm, request, ierror) !(_c)
22      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
23      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
24          recvcoun
25      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
26          rdispls(*)
27      TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
28      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
29      TYPE(MPI_Comm), INTENT(IN) :: comm
30      TYPE(MPI_Request), INTENT(OUT) :: request
31      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
32
33 MPI_Ibarrier(comm, request, ierror)
34      TYPE(MPI_Comm), INTENT(IN) :: comm
35      TYPE(MPI_Request), INTENT(OUT) :: request
36      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Ibcast(buffer, count, datatype, root, comm, request, ierror)
39      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
40      INTEGER, INTENT(IN) :: count, root
41      TYPE(MPI_Datatype), INTENT(IN) :: datatype
42      TYPE(MPI_Comm), INTENT(IN) :: comm
43      TYPE(MPI_Request), INTENT(OUT) :: request
44      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46 MPI_Ibcast(buffer, count, datatype, root, comm, request, ierror) !(_c)
47      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buffer
48      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
49      TYPE(MPI_Datatype), INTENT(IN) :: datatype
50      INTEGER, INTENT(IN) :: root
51      TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iexscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iexscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Igather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Igather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Igatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs,
            recvtype, root, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*), displs(*)

```



```

1      TYPE(MPI_Comm), INTENT(IN) :: comm
2      TYPE(MPI_Request), INTENT(OUT) :: request
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
4
5      MPI_Igatherv(sendbuf, sendcount, sendtype, recvbuf, recvcunts, displs,
6                  recvtype, root, comm, request, ierror) !(_c)
7      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
8      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
9      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
10     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
11     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcunts(*)
12     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
13     INTEGER, INTENT(IN) :: root
14     TYPE(MPI_Comm), INTENT(IN) :: comm
15     TYPE(MPI_Request), INTENT(OUT) :: request
16     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18     MPI_Ireduce(sendbuf, recvbuf, count, datatype, op, root, comm, request, ierror)
19     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
20     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
21     INTEGER, INTENT(IN) :: count, root
22     TYPE(MPI_Datatype), INTENT(IN) :: datatype
23     TYPE(MPI_Op), INTENT(IN) :: op
24     TYPE(MPI_Comm), INTENT(IN) :: comm
25     TYPE(MPI_Request), INTENT(OUT) :: request
26     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
27
28     MPI_Ireduce(sendbuf, recvbuf, count, datatype, op, root, comm, request, ierror)
29     !(_c)
30     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
31     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
32     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
33     TYPE(MPI_Datatype), INTENT(IN) :: datatype
34     TYPE(MPI_Op), INTENT(IN) :: op
35     INTEGER, INTENT(IN) :: root
36     TYPE(MPI_Comm), INTENT(IN) :: comm
37     TYPE(MPI_Request), INTENT(OUT) :: request
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
39
40     MPI_Ireduce_scatter(sendbuf, recvbuf, recvcunts, datatype, op, comm, request,
41                        ierror)
42     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
43     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
44     INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcunts(*)
45     TYPE(MPI_Datatype), INTENT(IN) :: datatype
46     TYPE(MPI_Op), INTENT(IN) :: op
47     TYPE(MPI_Comm), INTENT(IN) :: comm
48     TYPE(MPI_Request), INTENT(OUT) :: request
49     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

MPI_Ireduce_scatter(sendbuf, recvbuf, recvcoun, datatype, op, comm, request,
                    ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcoun(*)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ireduce_scatter_block(sendbuf, recvbuf, recvcoun, datatype, op, comm,
                          request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: recvcoun
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ireduce_scatter_block(sendbuf, recvbuf, recvcoun, datatype, op, comm,
                          request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcoun
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Iscan(sendbuf, recvbuf, count, datatype, op, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

1      TYPE(MPI_Request), INTENT(OUT) :: request
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4      MPI_Iscatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
5                  comm, request, ierror)
6      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
7      INTEGER, INTENT(IN) :: sendcount, recvcount, root
8      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
9      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
10     TYPE(MPI_Comm), INTENT(IN) :: comm
11     TYPE(MPI_Request), INTENT(OUT) :: request
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14     MPI_Iscatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
15                 comm, request, ierror) !(_c)
16     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
17     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
18     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
19     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
20     INTEGER, INTENT(IN) :: root
21     TYPE(MPI_Comm), INTENT(IN) :: comm
22     TYPE(MPI_Request), INTENT(OUT) :: request
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24
25     MPI_Iscatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
26                  recvtype, root, comm, request, ierror)
27     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
28     INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), displs(*)
29     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
30     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
31     INTEGER, INTENT(IN) :: recvcount, root
32     TYPE(MPI_Comm), INTENT(IN) :: comm
33     TYPE(MPI_Request), INTENT(OUT) :: request
34     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
35
36     MPI_Iscatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
37                  recvtype, root, comm, request, ierror) !(_c)
38     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
39     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*)
40     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
41     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
42     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
43     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcount
44     INTEGER, INTENT(IN) :: root
45     TYPE(MPI_Comm), INTENT(IN) :: comm
46     TYPE(MPI_Request), INTENT(OUT) :: request
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48
49     MPI_Op_commutative(op, commute, ierror)
50     TYPE(MPI_Op), INTENT(IN) :: op

```

```

LOGICAL, INTENT(OUT) :: commute
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Op_create(user_fn, commute, op, ierror)
PROCEDURE(MPI_User_function) :: user_fn
LOGICAL, INTENT(IN) :: commute
TYPE(MPI_Op), INTENT(OUT) :: op
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Op_create_c(user_fn, commute, op, ierror) !(_c)
PROCEDURE(MPI_User_function_c) :: user_fn
LOGICAL, INTENT(IN) :: commute
TYPE(MPI_Op), INTENT(OUT) :: op
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Op_free(op, ierror)
TYPE(MPI_Op), INTENT(INOUT) :: op
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: count, root
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Reduce_init(sendbuf, recvbuf, count, datatype, op, root, comm, info,
                request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: count, root
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Reduce_init(sendbuf, recvbuf, count, datatype, op, root, comm, info,
2      request, ierror) !(_c)
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
4      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
5      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
6      TYPE(MPI_Datatype), INTENT(IN) :: datatype
7      TYPE(MPI_Op), INTENT(IN) :: op
8      INTEGER, INTENT(IN) :: root
9      TYPE(MPI_Comm), INTENT(IN) :: comm
10     TYPE(MPI_Info), INTENT(IN) :: info
11     TYPE(MPI_Request), INTENT(OUT) :: request
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14  MPI_Reduce_local(inbuf, inoutbuf, count, datatype, op, ierror)
15     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
16     TYPE(*), DIMENSION(..) :: inoutbuf
17     INTEGER, INTENT(IN) :: count
18     TYPE(MPI_Datatype), INTENT(IN) :: datatype
19     TYPE(MPI_Op), INTENT(IN) :: op
20     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
21
22  MPI_Reduce_local(inbuf, inoutbuf, count, datatype, op, ierror) !(_c)
23     TYPE(*), DIMENSION(..), INTENT(IN) :: inbuf
24     TYPE(*), DIMENSION(..) :: inoutbuf
25     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
26     TYPE(MPI_Datatype), INTENT(IN) :: datatype
27     TYPE(MPI_Op), INTENT(IN) :: op
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30  MPI_Reduce_scatter(sendbuf, recvbuf, recvcounts, datatype, op, comm, ierror)
31     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
32     TYPE(*), DIMENSION(..) :: recvbuf
33     INTEGER, INTENT(IN) :: recvcounts(*)
34     TYPE(MPI_Datatype), INTENT(IN) :: datatype
35     TYPE(MPI_Op), INTENT(IN) :: op
36     TYPE(MPI_Comm), INTENT(IN) :: comm
37     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
38
39  MPI_Reduce_scatter(sendbuf, recvbuf, recvcounts, datatype, op, comm, ierror)
40     !(_c)
41     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
42     TYPE(*), DIMENSION(..) :: recvbuf
43     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcounts(*)
44     TYPE(MPI_Datatype), INTENT(IN) :: datatype
45     TYPE(MPI_Op), INTENT(IN) :: op
46     TYPE(MPI_Comm), INTENT(IN) :: comm
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48
49  MPI_Reduce_scatter_block(sendbuf, recvbuf, recvcount, datatype, op, comm,
50      ierror)
51     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf

```

```

TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: recvcount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_scatter_block(sendbuf, recvbuf, recvcount, datatype, op, comm,
    ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_scatter_block_init(sendbuf, recvbuf, recvcount, datatype, op, comm,
    info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: recvcount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_scatter_block_init(sendbuf, recvbuf, recvcount, datatype, op, comm,
    info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcount
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Reduce_scatter_init(sendbuf, recvbuf, recvcounts, datatype, op, comm, info,
    request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info

```

```

1      TYPE(MPI_Request), INTENT(OUT) :: request
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_Reduce_scatter_init(sendbuf, recvbuf, recvcounts, datatype, op, comm, info,
5                          request, ierror) !(_c)
6      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
7      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
8      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
9      TYPE(MPI_Datatype), INTENT(IN) :: datatype
10     TYPE(MPI_Op), INTENT(IN) :: op
11     TYPE(MPI_Comm), INTENT(IN) :: comm
12     TYPE(MPI_Info), INTENT(IN) :: info
13     TYPE(MPI_Request), INTENT(OUT) :: request
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_Scan(sendbuf, recvbuf, count, datatype, op, comm, ierror)
17     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
18     TYPE(*), DIMENSION(..) :: recvbuf
19     INTEGER, INTENT(IN) :: count
20     TYPE(MPI_Datatype), INTENT(IN) :: datatype
21     TYPE(MPI_Op), INTENT(IN) :: op
22     TYPE(MPI_Comm), INTENT(IN) :: comm
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24
25 MPI_Scan(sendbuf, recvbuf, count, datatype, op, comm, ierror) !(_c)
26     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
27     TYPE(*), DIMENSION(..) :: recvbuf
28     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
29     TYPE(MPI_Datatype), INTENT(IN) :: datatype
30     TYPE(MPI_Op), INTENT(IN) :: op
31     TYPE(MPI_Comm), INTENT(IN) :: comm
32     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
33
34 MPI_Scan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
35               ierror)
36     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
37     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
38     INTEGER, INTENT(IN) :: count
39     TYPE(MPI_Datatype), INTENT(IN) :: datatype
40     TYPE(MPI_Op), INTENT(IN) :: op
41     TYPE(MPI_Comm), INTENT(IN) :: comm
42     TYPE(MPI_Info), INTENT(IN) :: info
43     TYPE(MPI_Request), INTENT(OUT) :: request
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46 MPI_Scan_init(sendbuf, recvbuf, count, datatype, op, comm, info, request,
47               ierror) !(_c)
48     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
49     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
50     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
51     TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root,
            comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scatter_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                 root, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount, root
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scatter_init(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype,
                 root, comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN) :: root
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Scatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcount,
             recvtype, root, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf

```



```

1      INTEGER, INTENT(IN) :: sendcounts(*), displs(*), recvcnt, root
2      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
3      TYPE(*), DIMENSION(..) :: recvbuf
4      TYPE(MPI_Comm), INTENT(IN) :: comm
5      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
6
7      MPI_Scatterv(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcnt,
8                  recvtype, root, comm, ierror) !(_c)
9      TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
10     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcnt
11     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
12     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
13     TYPE(*), DIMENSION(..) :: recvbuf
14     INTEGER, INTENT(IN) :: root
15     TYPE(MPI_Comm), INTENT(IN) :: comm
16     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
17
18     MPI_Scatterv_init(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcnt,
19                     recvtype, root, comm, info, request, ierror)
20     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
21     INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), displs(*)
22     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
23     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
24     INTEGER, INTENT(IN) :: recvcnt, root
25     TYPE(MPI_Comm), INTENT(IN) :: comm
26     TYPE(MPI_Info), INTENT(IN) :: info
27     TYPE(MPI_Request), INTENT(OUT) :: request
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30     MPI_Scatterv_init(sendbuf, sendcounts, displs, sendtype, recvbuf, recvcnt,
31                     recvtype, root, comm, info, request, ierror) !(_c)
32     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
33     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*)
34     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
35     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
36     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
37     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: recvcnt
38     INTEGER, INTENT(IN) :: root
39     TYPE(MPI_Comm), INTENT(IN) :: comm
40     TYPE(MPI_Info), INTENT(IN) :: info
41     TYPE(MPI_Request), INTENT(OUT) :: request
42     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
43
44     MPI_Type_get_value_index(value_type, index_type, pair_type, ierror)
45     TYPE(MPI_Datatype), INTENT(IN) :: value_type, index_type
46     TYPE(MPI_Datatype), INTENT(OUT) :: pair_type
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48

```


A.4.5 Groups, Contexts, Communicators, and Caching Fortran 2008 Bindings

```

MPI_Comm_compare(comm1, comm2, result, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm1, comm2
  INTEGER, INTENT(OUT) :: result
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_create(comm, group, newcomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Group), INTENT(IN) :: group
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_create_from_group(group, stringtag, info, errhandler, newcomm, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  CHARACTER(LEN=*), INTENT(IN) :: stringtag
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_create_group(comm, group, tag, newcomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: tag
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_create_keyval(comm_copy_attr_fn, comm_delete_attr_fn, comm_keyval,
  extra_state, ierror)
  PROCEDURE(MPI_Comm_copy_attr_function) :: comm_copy_attr_fn
  PROCEDURE(MPI_Comm_delete_attr_function) :: comm_delete_attr_fn
  INTEGER, INTENT(OUT) :: comm_keyval
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_delete_attr(comm, comm_keyval, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: comm_keyval
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_dup(comm, newcomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_COMM_DUP_FN(oldcomm, comm_keyval, extra_state, attribute_val_in,
  attribute_val_out, flag, ierror)
  TYPE(MPI_Comm) :: oldcomm
  INTEGER :: comm_keyval, ierror
  INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
  attribute_val_out
  LOGICAL :: flag

```

```

1  MPI_Comm_dup_with_info(comm, info, newcomm, ierror)
2      TYPE(MPI_Comm), INTENT(IN) :: comm
3      TYPE(MPI_Info), INTENT(IN) :: info
4      TYPE(MPI_Comm), INTENT(OUT) :: newcomm
5      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
6
7  MPI_Comm_free(comm, ierror)
8      TYPE(MPI_Comm), INTENT(INOUT) :: comm
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_Comm_free_keyval(comm_keyval, ierror)
12     INTEGER, INTENT(INOUT) :: comm_keyval
13     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
14
15 MPI_Comm_get_attr(comm, comm_keyval, attribute_val, flag, ierror)
16     TYPE(MPI_Comm), INTENT(IN) :: comm
17     INTEGER, INTENT(IN) :: comm_keyval
18     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: attribute_val
19     LOGICAL, INTENT(OUT) :: flag
20     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
21
22 MPI_Comm_get_info(comm, info_used, ierror)
23     TYPE(MPI_Comm), INTENT(IN) :: comm
24     TYPE(MPI_Info), INTENT(OUT) :: info_used
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26
27 MPI_Comm_get_name(comm, comm_name, resultlen, ierror)
28     TYPE(MPI_Comm), INTENT(IN) :: comm
29     CHARACTER(LEN=MPI_MAX_OBJECT_NAME), INTENT(OUT) :: comm_name
30     INTEGER, INTENT(OUT) :: resultlen
31     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
32
33 MPI_Comm_group(comm, group, ierror)
34     TYPE(MPI_Comm), INTENT(IN) :: comm
35     TYPE(MPI_Group), INTENT(OUT) :: group
36     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Comm_idup(comm, newcomm, request, ierror)
39     TYPE(MPI_Comm), INTENT(IN) :: comm
40     TYPE(MPI_Comm), INTENT(OUT), ASYNCHRONOUS :: newcomm
41     TYPE(MPI_Request), INTENT(OUT) :: request
42     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
43
44 MPI_Comm_idup_with_info(comm, info, newcomm, request, ierror)
45     TYPE(MPI_Comm), INTENT(IN) :: comm
46     TYPE(MPI_Info), INTENT(IN) :: info
47     TYPE(MPI_Comm), INTENT(OUT), ASYNCHRONOUS :: newcomm
48     TYPE(MPI_Request), INTENT(OUT) :: request
49     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
50
51 MPI_COMM_NULL_COPY_FN(oldcomm, comm_keyval, extra_state, attribute_val_in,
52     attribute_val_out, flag, ierror)
53     TYPE(MPI_Comm) :: oldcomm

```

```

INTEGER :: comm_keyval, ierror
INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
    attribute_val_out
LOGICAL :: flag

MPI_COMM_NULL_DELETE_FN(comm, comm_keyval, attribute_val, extra_state, ierror)

TYPE(MPI_Comm) :: comm
INTEGER :: comm_keyval, ierror
INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state

MPI_Comm_rank(comm, rank, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(OUT) :: rank
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_remote_group(comm, group, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Group), INTENT(OUT) :: group
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_remote_size(comm, size, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_set_attr(comm, comm_keyval, attribute_val, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(IN) :: comm_keyval
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: attribute_val
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_set_info(comm, info, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_set_name(comm, comm_name, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
CHARACTER(LEN=*), INTENT(IN) :: comm_name
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_size(comm, size, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_split(comm, color, key, newcomm, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(IN) :: color, key
TYPE(MPI_Comm), INTENT(OUT) :: newcomm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_split_type(comm, split_type, key, info, newcomm, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

1      INTEGER, INTENT(IN) :: split_type, key
2      TYPE(MPI_Info), INTENT(IN) :: info
3      TYPE(MPI_Comm), INTENT(OUT) :: newcomm
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6      MPI_Comm_test_inter(comm, flag, ierror)
7      TYPE(MPI_Comm), INTENT(IN) :: comm
8      LOGICAL, INTENT(OUT) :: flag
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11     MPI_Group_compare(group1, group2, result, ierror)
12     TYPE(MPI_Group), INTENT(IN) :: group1, group2
13     INTEGER, INTENT(OUT) :: result
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16     MPI_Group_difference(group1, group2, newgroup, ierror)
17     TYPE(MPI_Group), INTENT(IN) :: group1, group2
18     TYPE(MPI_Group), INTENT(OUT) :: newgroup
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21     MPI_Group_excl(group, n, ranks, newgroup, ierror)
22     TYPE(MPI_Group), INTENT(IN) :: group
23     INTEGER, INTENT(IN) :: n, ranks(n)
24     TYPE(MPI_Group), INTENT(OUT) :: newgroup
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26
27     MPI_Group_free(group, ierror)
28     TYPE(MPI_Group), INTENT(INOUT) :: group
29     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
30
31     MPI_Group_from_session_pset(session, pset_name, newgroup, ierror)
32     TYPE(MPI_Session), INTENT(IN) :: session
33     CHARACTER(LEN=*), INTENT(IN) :: pset_name
34     TYPE(MPI_Group), INTENT(OUT) :: newgroup
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37     MPI_Group_incl(group, n, ranks, newgroup, ierror)
38     TYPE(MPI_Group), INTENT(IN) :: group
39     INTEGER, INTENT(IN) :: n, ranks(n)
40     TYPE(MPI_Group), INTENT(OUT) :: newgroup
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43     MPI_Group_intersection(group1, group2, newgroup, ierror)
44     TYPE(MPI_Group), INTENT(IN) :: group1, group2
45     TYPE(MPI_Group), INTENT(OUT) :: newgroup
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
47
48     MPI_Group_range_excl(group, n, ranges, newgroup, ierror)
49     TYPE(MPI_Group), INTENT(IN) :: group
50     INTEGER, INTENT(IN) :: n, ranges(3, n)
51     TYPE(MPI_Group), INTENT(OUT) :: newgroup
52     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Group_range_incl(group, n, ranges, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(IN) :: n, ranges(3, n)
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Group_rank(group, rank, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(OUT) :: rank
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Group_size(group, size, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group
  INTEGER, INTENT(OUT) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Group_translate_ranks(group1, n, ranks1, group2, ranks2, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group1, group2
  INTEGER, INTENT(IN) :: n, ranks1(n)
  INTEGER, INTENT(OUT) :: ranks2(n)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Group_union(group1, group2, newgroup, ierror)
  TYPE(MPI_Group), INTENT(IN) :: group1, group2
  TYPE(MPI_Group), INTENT(OUT) :: newgroup
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Intercomm_create(local_comm, local_leader, peer_comm, remote_leader, tag,
  newintercomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: local_comm, peer_comm
  INTEGER, INTENT(IN) :: local_leader, remote_leader, tag
  TYPE(MPI_Comm), INTENT(OUT) :: newintercomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Intercomm_create_from_groups(local_group, local_leader, remote_group,
  remote_leader, stringtag, info, errhandler, newintercomm, ierror)
  TYPE(MPI_Group), INTENT(IN) :: local_group, remote_group
  INTEGER, INTENT(IN) :: local_leader, remote_leader
  CHARACTER(LEN=*), INTENT(IN) :: stringtag
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
  TYPE(MPI_Comm), INTENT(OUT) :: newintercomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Intercomm_merge(intercomm, high, newintracomm, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: intercomm
  LOGICAL, INTENT(IN) :: high
  TYPE(MPI_Comm), INTENT(OUT) :: newintracomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_keyval(type_copy_attr_fn, type_delete_attr_fn, type_keyval,
  extra_state, ierror)
  PROCEDURE(MPI_Type_copy_attr_function) :: type_copy_attr_fn

```

```

1      PROCEDURE(MPI_Type_delete_attr_function) :: type_delete_attr_fn
2      INTEGER, INTENT(OUT) :: type_keyval
3      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6      MPI_Type_delete_attr(datatype, type_keyval, ierror)
7      TYPE(MPI_Datatype), INTENT(IN) :: datatype
8      INTEGER, INTENT(IN) :: type_keyval
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11     MPI_TYPE_DUP_FN(oldtype, type_keyval, extra_state, attribute_val_in,
12                     attribute_val_out, flag, ierror)
13     TYPE(MPI_Datatype) :: oldtype
14     INTEGER :: type_keyval, ierror
15     INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
16         attribute_val_out
17     LOGICAL :: flag
18
19     MPI_Type_free_keyval(type_keyval, ierror)
20     INTEGER, INTENT(INOUT) :: type_keyval
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
22
23     MPI_Type_get_attr(datatype, type_keyval, attribute_val, flag, ierror)
24     TYPE(MPI_Datatype), INTENT(IN) :: datatype
25     INTEGER, INTENT(IN) :: type_keyval
26     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: attribute_val
27     LOGICAL, INTENT(OUT) :: flag
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30     MPI_Type_get_name(datatype, type_name, resultlen, ierror)
31     TYPE(MPI_Datatype), INTENT(IN) :: datatype
32     CHARACTER(LEN=MPI_MAX_OBJECT_NAME), INTENT(OUT) :: type_name
33     INTEGER, INTENT(OUT) :: resultlen
34     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
35
36     MPI_TYPE_NULL_COPY_FN(oldtype, type_keyval, extra_state, attribute_val_in,
37                           attribute_val_out, flag, ierror)
38     TYPE(MPI_Datatype) :: oldtype
39     INTEGER :: type_keyval, ierror
40     INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
41         attribute_val_out
42     LOGICAL :: flag
43
44     MPI_TYPE_NULL_DELETE_FN(datatype, type_keyval, attribute_val, extra_state,
45                             ierror)
46     TYPE(MPI_Datatype) :: datatype
47     INTEGER :: type_keyval
48     INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
49     INTEGER, INTENT(OUT) :: ierror
50
51     MPI_Type_set_attr(datatype, type_keyval, attribute_val, ierror)
52     TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

INTEGER, INTENT(IN) :: type_keyval
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: attribute_val
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_set_name(datatype, type_name, ierror)
TYPE(MPI_Datatype), INTENT(IN) :: datatype
CHARACTER(LEN=*), INTENT(IN) :: type_name
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_create_keyval(win_copy_attr_fn, win_delete_attr_fn, win_keyval,
                      extra_state, ierror)
PROCEDURE(MPI_Win_copy_attr_function) :: win_copy_attr_fn
PROCEDURE(MPI_Win_delete_attr_function) :: win_delete_attr_fn
INTEGER, INTENT(OUT) :: win_keyval
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_delete_attr(win, win_keyval, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, INTENT(IN) :: win_keyval
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_WIN_DUP_FN(oldwin, win_keyval, extra_state, attribute_val_in,
               attribute_val_out, flag, ierror)
TYPE(MPI_Win) :: oldwin
INTEGER :: win_keyval, ierror
INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
               attribute_val_out
LOGICAL :: flag

MPI_Win_free_keyval(win_keyval, ierror)
INTEGER, INTENT(INOUT) :: win_keyval
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_get_attr(win, win_keyval, attribute_val, flag, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, INTENT(IN) :: win_keyval
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: attribute_val
LOGICAL, INTENT(OUT) :: flag
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_get_name(win, win_name, resultlen, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
CHARACTER(LEN=MPI_MAX_OBJECT_NAME), INTENT(OUT) :: win_name
INTEGER, INTENT(OUT) :: resultlen
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_WIN_NULL_COPY_FN(oldwin, win_keyval, extra_state, attribute_val_in,
                    attribute_val_out, flag, ierror)
TYPE(MPI_Win) :: oldwin
INTEGER :: win_keyval, ierror

```

```

1      INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state, attribute_val_in,
2          attribute_val_out
3      LOGICAL :: flag
4
5      MPI_WIN_NULL_DELETE_FN(win, win_keyval, attribute_val, extra_state, ierror)
6      TYPE(MPI_Win) :: win
7      INTEGER :: win_keyval, ierror
8      INTEGER(KIND=MPI_ADDRESS_KIND) :: attribute_val, extra_state
9
10     MPI_Win_set_attr(win, win_keyval, attribute_val, ierror)
11     TYPE(MPI_Win), INTENT(IN) :: win
12     INTEGER, INTENT(IN) :: win_keyval
13     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: attribute_val
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16     MPI_Win_set_name(win, win_name, ierror)
17     TYPE(MPI_Win), INTENT(IN) :: win
18     CHARACTER(LEN=*), INTENT(IN) :: win_name
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21 A.4.6 Virtual Topologies for MPI Processes Fortran 2008 Bindings
22
23 MPI_Cart_coords(comm, rank, maxdims, coords, ierror)
24 TYPE(MPI_Comm), INTENT(IN) :: comm
25 INTEGER, INTENT(IN) :: rank, maxdims
26 INTEGER, INTENT(OUT) :: coords(maxdims)
27 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
28
29 MPI_Cart_create(comm_old, ndims, dims, periods, reorder, comm_cart, ierror)
30 TYPE(MPI_Comm), INTENT(IN) :: comm_old
31 INTEGER, INTENT(IN) :: ndims, dims(ndims)
32 LOGICAL, INTENT(IN) :: periods(ndims), reorder
33 TYPE(MPI_Comm), INTENT(OUT) :: comm_cart
34 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
35
36 MPI_Cart_get(comm, maxdims, dims, periods, coords, ierror)
37 TYPE(MPI_Comm), INTENT(IN) :: comm
38 INTEGER, INTENT(IN) :: maxdims
39 INTEGER, INTENT(OUT) :: dims(maxdims), coords(maxdims)
40 LOGICAL, INTENT(OUT) :: periods(maxdims)
41 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_Cart_map(comm, ndims, dims, periods, newrank, ierror)
44 TYPE(MPI_Comm), INTENT(IN) :: comm
45 INTEGER, INTENT(IN) :: ndims, dims(ndims)
46 LOGICAL, INTENT(IN) :: periods(ndims)
47 INTEGER, INTENT(OUT) :: newrank
48 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
49
50 MPI_Cart_rank(comm, coords, rank, ierror)
51 TYPE(MPI_Comm), INTENT(IN) :: comm
52 INTEGER, INTENT(IN) :: coords(*)

```



```

    INTEGER, INTENT(OUT) :: rank
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Cart_shift(comm, direction, disp, rank_source, rank_dest, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(IN) :: direction, disp
    INTEGER, INTENT(OUT) :: rank_source, rank_dest
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Cart_sub(comm, remain_dims, newcomm, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    LOGICAL, INTENT(IN) :: remain_dims(*)
    TYPE(MPI_Comm), INTENT(OUT) :: newcomm
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Cartdim_get(comm, ndims, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(OUT) :: ndims
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Dims_create(nnodes, ndims, dims, ierror)
    INTEGER, INTENT(IN) :: nnodes, ndims
    INTEGER, INTENT(INOUT) :: dims(ndims)
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Dist_graph_create(comm_old, n, sources, degrees, destinations, weights,
    info, reorder, comm_dist_graph, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm_old
    INTEGER, INTENT(IN) :: n, sources(n), degrees(n), destinations(*),
    weights(*)
    TYPE(MPI_Info), INTENT(IN) :: info
    LOGICAL, INTENT(IN) :: reorder
    TYPE(MPI_Comm), INTENT(OUT) :: comm_dist_graph
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Dist_graph_create_adjacent(comm_old, indegree, sources, sourceweights,
    outdegree, destinations, destweights, info, reorder,
    comm_dist_graph, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm_old
    INTEGER, INTENT(IN) :: indegree, sources(indegree), sourceweights(*),
    outdegree, destinations(outdegree), destweights(*)
    TYPE(MPI_Info), INTENT(IN) :: info
    LOGICAL, INTENT(IN) :: reorder
    TYPE(MPI_Comm), INTENT(OUT) :: comm_dist_graph
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Dist_graph_neighbors(comm, maxindegree, sources, sourceweights,
    maxoutdegree, destinations, destweights, ierror)
    TYPE(MPI_Comm), INTENT(IN) :: comm
    INTEGER, INTENT(IN) :: maxindegree, maxoutdegree
    INTEGER, INTENT(OUT) :: sources(maxindegree), destinations(maxoutdegree)

```

```

1      INTEGER :: sourceweights(*), destweights(*)
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4      MPI_Dist_graph_neighbors_count(comm, indegree, outdegree, weighted, ierror)
5      TYPE(MPI_Comm), INTENT(IN) :: comm
6      INTEGER, INTENT(OUT) :: indegree, outdegree
7      LOGICAL, INTENT(OUT) :: weighted
8      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
9
10     MPI_Graph_create(comm_old, nnodes, index, edges, reorder, comm_graph, ierror)
11     TYPE(MPI_Comm), INTENT(IN) :: comm_old
12     INTEGER, INTENT(IN) :: nnodes, index(nnodes), edges(*)
13     LOGICAL, INTENT(IN) :: reorder
14     TYPE(MPI_Comm), INTENT(OUT) :: comm_graph
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17     MPI_Graph_get(comm, maxindex, maxedges, index, edges, ierror)
18     TYPE(MPI_Comm), INTENT(IN) :: comm
19     INTEGER, INTENT(IN) :: maxindex, maxedges
20     INTEGER, INTENT(OUT) :: index(maxindex), edges(maxedges)
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
22
23     MPI_Graph_map(comm, nnodes, index, edges, newrank, ierror)
24     TYPE(MPI_Comm), INTENT(IN) :: comm
25     INTEGER, INTENT(IN) :: nnodes, index(nnodes), edges(*)
26     INTEGER, INTENT(OUT) :: newrank
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
28
29     MPI_Graph_neighbors(comm, rank, maxneighbors, neighbors, ierror)
30     TYPE(MPI_Comm), INTENT(IN) :: comm
31     INTEGER, INTENT(IN) :: rank, maxneighbors
32     INTEGER, INTENT(OUT) :: neighbors(maxneighbors)
33     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
34
35     MPI_Graph_neighbors_count(comm, rank, nneighbors, ierror)
36     TYPE(MPI_Comm), INTENT(IN) :: comm
37     INTEGER, INTENT(IN) :: rank
38     INTEGER, INTENT(OUT) :: nneighbors
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41     MPI_Graphdims_get(comm, nnodes, nedges, ierror)
42     TYPE(MPI_Comm), INTENT(IN) :: comm
43     INTEGER, INTENT(OUT) :: nnodes, nedges
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46     MPI_Ineighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
47                             recvtype, comm, request, ierror)
48     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
49     INTEGER, INTENT(IN) :: sendcount, recvcount
50     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
51     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
52     TYPE(MPI_Comm), INTENT(IN) :: comm

```

```

TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ineighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ineighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts,
    displs, recvtype, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcounts(*), displs(*)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ineighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcounts,
    displs, recvtype, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcounts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: displs(*)
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ineighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Ineighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype

```

```

1      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
2      TYPE(MPI_Comm), INTENT(IN) :: comm
3      TYPE(MPI_Request), INTENT(OUT) :: request
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6      MPI_Ineighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
7                               recvcounts, rdispls, recvtype, comm, request, ierror)
8      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
9      INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
10         recvcounts(*), rdispls(*)
11      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
12      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
13      TYPE(MPI_Comm), INTENT(IN) :: comm
14      TYPE(MPI_Request), INTENT(OUT) :: request
15      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17      MPI_Ineighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
18                               recvcounts, rdispls, recvtype, comm, request, ierror) !(_c)
19      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
20      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
21         recvcounts(*)
22      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
23         rdispls(*)
24      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
25      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
26      TYPE(MPI_Comm), INTENT(IN) :: comm
27      TYPE(MPI_Request), INTENT(OUT) :: request
28      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30      MPI_Ineighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
31                               recvcounts, rdispls, recvtypes, comm, request, ierror)
32      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
33      INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), recvcounts(*)
34      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
35         rdispls(*)
36      TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
37      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
38      TYPE(MPI_Comm), INTENT(IN) :: comm
39      TYPE(MPI_Request), INTENT(OUT) :: request
40      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
41
42      MPI_Ineighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
43                               recvcounts, rdispls, recvtypes, comm, request, ierror) !(_c)
44      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
45      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
46         recvcounts(*)
47      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
48         rdispls(*)
49      TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
50      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf

```

```

TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgather_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcoun
    displs, recvtype, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcoun(*), displs(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf

```

```

1      TYPE(MPI_Comm), INTENT(IN) :: comm
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_Neighbor_allgatherv(sendbuf, sendcount, sendtype, recvbuf, recvcoun
5      displs, recvtype, comm, ierror) !(_c)
6      TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
7      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcoun
8      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
9      TYPE(*), DIMENSION(..) :: recvbuf
10     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
11     TYPE(MPI_Comm), INTENT(IN) :: comm
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14 MPI_Neighbor_allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcoun
15     displs, recvtype, comm, info, request, ierror)
16     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
17     INTEGER, INTENT(IN) :: sendcount, displs(*)
18     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
19     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
20     INTEGER, INTENT(IN), ASYNCHRONOUS :: recvcoun
21     TYPE(MPI_Comm), INTENT(IN) :: comm
22     TYPE(MPI_Info), INTENT(IN) :: info
23     TYPE(MPI_Request), INTENT(OUT) :: request
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26 MPI_Neighbor_allgatherv_init(sendbuf, sendcount, sendtype, recvbuf, recvcoun
27     displs, recvtype, comm, info, request, ierror) !(_c)
28     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
29     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount
30     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
31     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
32     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: recvcoun
33     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: displs(*)
34     TYPE(MPI_Comm), INTENT(IN) :: comm
35     TYPE(MPI_Info), INTENT(IN) :: info
36     TYPE(MPI_Request), INTENT(OUT) :: request
37     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
38
39 MPI_Neighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcoun
40     recvtype, comm, ierror)
41     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
42     INTEGER, INTENT(IN) :: sendcount, recvcoun
43     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
44     TYPE(*), DIMENSION(..) :: recvbuf
45     TYPE(MPI_Comm), INTENT(IN) :: comm
46     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
47
48 MPI_Neighbor_alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcoun
49     recvtype, comm, ierror) !(_c)
50     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
51     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcoun

```

```

TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoall_init(sendbuf, sendcount, sendtype, recvbuf, recvcount,
    recvtype, comm, info, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcount, recvcount
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
    recvcounts, rdispls, recvtype, comm, ierror)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER, INTENT(IN) :: sendcounts(*), sdispls(*), recvcounts(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoallv(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
    recvcounts, rdispls, recvtype, comm, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcounts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
TYPE(*), DIMENSION(..) :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
    recvcounts, rdispls, recvtype, comm, info, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), sdispls(*),
    recvcounts(*), rdispls(*)

```



```

1      TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
2      TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
3      TYPE(MPI_Comm), INTENT(IN) :: comm
4      TYPE(MPI_Info), INTENT(IN) :: info
5      TYPE(MPI_Request), INTENT(OUT) :: request
6      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
7
8      MPI_Neighbor_alltoallv_init(sendbuf, sendcounts, sdispls, sendtype, recvbuf,
9                                  recvcounsts, rdispls, recvtype, comm, info, request, ierror) !(_c)
10     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
11     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
12         recvcounsts(*)
13     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
14         rdispls(*)
15     TYPE(MPI_Datatype), INTENT(IN) :: sendtype, recvtype
16     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
17     TYPE(MPI_Comm), INTENT(IN) :: comm
18     TYPE(MPI_Info), INTENT(IN) :: info
19     TYPE(MPI_Request), INTENT(OUT) :: request
20     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
21
22     MPI_Neighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
23                             recvcounsts, rdispls, recvtypes, comm, ierror)
24     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
25     INTEGER, INTENT(IN) :: sendcounts(*), recvcounsts(*)
26     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
27     TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
28     TYPE(*), DIMENSION(..) :: recvbuf
29     TYPE(MPI_Comm), INTENT(IN) :: comm
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32     MPI_Neighbor_alltoallw(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
33                             recvcounsts, rdispls, recvtypes, comm, ierror) !(_c)
34     TYPE(*), DIMENSION(..), INTENT(IN) :: sendbuf
35     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: sendcounts(*), recvcounsts(*)
36     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: sdispls(*), rdispls(*)
37     TYPE(MPI_Datatype), INTENT(IN) :: sendtypes(*), recvtypes(*)
38     TYPE(*), DIMENSION(..) :: recvbuf
39     TYPE(MPI_Comm), INTENT(IN) :: comm
40     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
41
42     MPI_Neighbor_alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
43                                 recvcounsts, rdispls, recvtypes, comm, info, request, ierror)
44     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
45     INTEGER, INTENT(IN), ASYNCHRONOUS :: sendcounts(*), recvcounsts(*)
46     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
47         rdispls(*)
48     TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
49     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
50     TYPE(MPI_Comm), INTENT(IN) :: comm

```



```

TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Neighbor_alltoallw_init(sendbuf, sendcounts, sdispls, sendtypes, recvbuf,
    recvcounts, rdispls, recvtypes, comm, info, request, ierror)
    !(_c)

TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: sendbuf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN), ASYNCHRONOUS :: sendcounts(*),
    recvcounts(*)
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN), ASYNCHRONOUS :: sdispls(*),
    rdispls(*)
TYPE(MPI_Datatype), INTENT(IN), ASYNCHRONOUS :: sendtypes(*), recvtypes(*)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: recvbuf
TYPE(MPI_Comm), INTENT(IN) :: comm
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Topo_test(comm, status, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(OUT) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.7 MPI Environmental Management Fortran 2008 Bindings

```

MPI_Add_error_class(errorclass, ierror)
INTEGER, INTENT(OUT) :: errorclass
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Add_error_code(errorclass, errorcode, ierror)
INTEGER, INTENT(IN) :: errorclass
INTEGER, INTENT(OUT) :: errorcode
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Add_error_string(errorcode, string, ierror)
INTEGER, INTENT(IN) :: errorcode
CHARACTER(LEN=*), INTENT(IN) :: string
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Alloc_mem(size, info, baseptr, ierror)
USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
TYPE(MPI_Info), INTENT(IN) :: info
TYPE(C_PTR), INTENT(OUT) :: baseptr
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Comm_call_errhandler(comm, errorcode, ierror)
TYPE(MPI_Comm), INTENT(IN) :: comm
INTEGER, INTENT(IN) :: errorcode
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Comm_create_errhandler(comm_errhandler_fn, errhandler, ierror)
2      PROCEDURE(MPI_Comm_errhandler_function) :: comm_errhandler_fn
3      TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
4      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
5
6  MPI_Comm_get_errhandler(comm, errhandler, ierror)
7      TYPE(MPI_Comm), INTENT(IN) :: comm
8      TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_Comm_set_errhandler(comm, errhandler, ierror)
12     TYPE(MPI_Comm), INTENT(IN) :: comm
13     TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_Errhandler_free(errhandler, ierror)
17     TYPE(MPI_Errhandler), INTENT(INOUT) :: errhandler
18     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
19
20 MPI_Error_class(errorcode, errorclass, ierror)
21     INTEGER, INTENT(IN) :: errorcode
22     INTEGER, INTENT(OUT) :: errorclass
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24
25 MPI_Error_string(errorcode, string, resultlen, ierror)
26     INTEGER, INTENT(IN) :: errorcode
27     CHARACTER(LEN=MPI_MAX_ERROR_STRING), INTENT(OUT) :: string
28     INTEGER, INTENT(OUT) :: resultlen
29     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
30
31 MPI_File_call_errhandler(fh, errorcode, ierror)
32     TYPE(MPI_File), INTENT(IN) :: fh
33     INTEGER, INTENT(IN) :: errorcode
34     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
35
36 MPI_File_create_errhandler(file_errhandler_fn, errhandler, ierror)
37     PROCEDURE(MPI_File_errhandler_function) :: file_errhandler_fn
38     TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41 MPI_File_get_errhandler(file, errhandler, ierror)
42     TYPE(MPI_File), INTENT(IN) :: file
43     TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46 MPI_File_set_errhandler(file, errhandler, ierror)
47     TYPE(MPI_File), INTENT(IN) :: file
48     TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
49     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
50
51 MPI_Free_mem(base, ierror)
52     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: base
53     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Get_hw_resource_info(hw_info, ierror)
    TYPE(MPI_Info), INTENT(OUT) :: hw_info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get_library_version(version, resultlen, ierror)
    CHARACTER(LEN=MPI_MAX_LIBRARY_VERSION_STRING), INTENT(OUT) :: version
    INTEGER, INTENT(OUT) :: resultlen
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get_processor_name(name, resultlen, ierror)
    CHARACTER(LEN=MPI_MAX_PROCESSOR_NAME), INTENT(OUT) :: name
    INTEGER, INTENT(OUT) :: resultlen
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get_version(version, subversion, ierror)
    INTEGER, INTENT(OUT) :: version, subversion
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Remove_error_class(errorclass, ierror)
    INTEGER, INTENT(IN) :: errorclass
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Remove_error_code(errorcode, ierror)
    INTEGER, INTENT(IN) :: errorcode
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Remove_error_string(errorcode, ierror)
    INTEGER, INTENT(IN) :: errorcode
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Session_call_errhandler(session, errorcode, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    INTEGER, INTENT(IN) :: errorcode
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Session_create_errhandler(session_errhandler_fn, errhandler, ierror)
    PROCEDURE(MPI_Session_errhandler_function) :: session_errhandler_fn
    TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Session_get_errhandler(session, errhandler, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Session_set_errhandler(session, errhandler, ierror)
    TYPE(MPI_Session), INTENT(IN) :: session
    TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_call_errhandler(win, errorcode, ierror)
    TYPE(MPI_Win), INTENT(IN) :: win
    INTEGER, INTENT(IN) :: errorcode
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1 MPI_Win_create_errhandler(win_errhandler_fn, errhandler, ierror)
2   PROCEDURE(MPI_Win_errhandler_function) :: win_errhandler_fn
3   TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
4   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

5 MPI_Win_get_errhandler(win, errhandler, ierror)
6   TYPE(MPI_Win), INTENT(IN) :: win
7   TYPE(MPI_Errhandler), INTENT(OUT) :: errhandler
8   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

10 MPI_Win_set_errhandler(win, errhandler, ierror)
11   TYPE(MPI_Win), INTENT(IN) :: win
12   TYPE(MPI_Errhandler), INTENT(IN) :: errhandler
13   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

14 DOUBLE PRECISION MPI_Wtick()

```

```

16 DOUBLE PRECISION MPI_Wtime()

```

A.4.8 The Info Object Fortran 2008 Bindings

```

20 MPI_Info_create(info, ierror)
21   TYPE(MPI_Info), INTENT(OUT) :: info
22   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

23 MPI_Info_create_env(info, ierror)
24   TYPE(MPI_Info), INTENT(OUT) :: info
25   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

27 MPI_Info_delete(info, key, ierror)
28   TYPE(MPI_Info), INTENT(IN) :: info
29   CHARACTER(LEN=*), INTENT(IN) :: key
30   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

31 MPI_Info_dup(info, newinfo, ierror)
32   TYPE(MPI_Info), INTENT(IN) :: info
33   TYPE(MPI_Info), INTENT(OUT) :: newinfo
34   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

36 MPI_Info_free(info, ierror)
37   TYPE(MPI_Info), INTENT(INOUT) :: info
38   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

39 MPI_Info_get_nkeys(info, nkeys, ierror)
40   TYPE(MPI_Info), INTENT(IN) :: info
41   INTEGER, INTENT(OUT) :: nkeys
42   INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

43 MPI_Info_get_nthkey(info, n, key, ierror)
44   TYPE(MPI_Info), INTENT(IN) :: info
45   INTEGER, INTENT(IN) :: n
46   CHARACTER(LEN=*), INTENT(OUT) :: key
47   INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48

```

```
MPI_Info_get_string(info, key, buflen, value, flag, ierror)
```

```
  TYPE(MPI_Info), INTENT(IN) :: info
  CHARACTER(LEN=*), INTENT(IN) :: key
  INTEGER, INTENT(INOUT) :: buflen
  CHARACTER(LEN=*), INTENT(OUT) :: value
  LOGICAL, INTENT(OUT) :: flag
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Info_set(info, key, value, ierror)
```

```
  TYPE(MPI_Info), INTENT(IN) :: info
  CHARACTER(LEN=*), INTENT(IN) :: key, value
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

A.4.9 Process Creation and Management Fortran 2008 Bindings

```
MPI_Abort(comm, errorcode, ierror)
```

```
  TYPE(MPI_Comm), INTENT(IN) :: comm
  INTEGER, INTENT(IN) :: errorcode
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Close_port(port_name, ierror)
```

```
  CHARACTER(LEN=*), INTENT(IN) :: port_name
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Comm_accept(port_name, info, root, comm, newcomm, ierror)
```

```
  CHARACTER(LEN=*), INTENT(IN) :: port_name
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Comm_connect(port_name, info, root, comm, newcomm, ierror)
```

```
  CHARACTER(LEN=*), INTENT(IN) :: port_name
  TYPE(MPI_Info), INTENT(IN) :: info
  INTEGER, INTENT(IN) :: root
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Comm), INTENT(OUT) :: newcomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Comm_disconnect(comm, ierror)
```

```
  TYPE(MPI_Comm), INTENT(INOUT) :: comm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Comm_get_parent(parent, ierror)
```

```
  TYPE(MPI_Comm), INTENT(OUT) :: parent
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```
MPI_Comm_join(fd, intercomm, ierror)
```

```
  INTEGER, INTENT(IN) :: fd
  TYPE(MPI_Comm), INTENT(OUT) :: intercomm
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

```

1  MPI_Comm_spawn(command, argv, maxprocs, info, root, comm, intercomm,
2      array_of_errcodes, ierror)
3      CHARACTER(LEN=*), INTENT(IN) :: command, argv(*)
4      INTEGER, INTENT(IN) :: maxprocs, root
5      TYPE(MPI_Info), INTENT(IN) :: info
6      TYPE(MPI_Comm), INTENT(IN) :: comm
7      TYPE(MPI_Comm), INTENT(OUT) :: intercomm
8      INTEGER :: array_of_errcodes(*)
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_Comm_spawn_multiple(count, array_of_commands, array_of_argv,
12     array_of_maxprocs, array_of_info, root, comm, intercomm,
13     array_of_errcodes, ierror)
14     INTEGER, INTENT(IN) :: count, array_of_maxprocs(*), root
15     CHARACTER(LEN=*), INTENT(IN) :: array_of_commands(*),
16         array_of_argv(count, *)
17     TYPE(MPI_Info), INTENT(IN) :: array_of_info(*)
18     TYPE(MPI_Comm), INTENT(IN) :: comm
19     TYPE(MPI_Comm), INTENT(OUT) :: intercomm
20     INTEGER :: array_of_errcodes(*)
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
22
23 MPI_Finalize(ierror)
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26 MPI_Finalized(flag, ierror)
27     LOGICAL, INTENT(OUT) :: flag
28     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
29
30 MPI_Init(ierror)
31     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
32
33 MPI_Init_thread(required, provided, ierror)
34     INTEGER, INTENT(IN) :: required
35     INTEGER, INTENT(OUT) :: provided
36     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Initialized(flag, ierror)
39     LOGICAL, INTENT(OUT) :: flag
40     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
41
42 MPI_Is_thread_main(flag, ierror)
43     LOGICAL, INTENT(OUT) :: flag
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46 MPI_Lookup_name(service_name, info, port_name, ierror)
47     CHARACTER(LEN=*), INTENT(IN) :: service_name
48     TYPE(MPI_Info), INTENT(IN) :: info
49     CHARACTER(LEN=MPI_MAX_PORT_NAME), INTENT(OUT) :: port_name
50     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
51
52 MPI_Open_port(info, port_name, ierror)
53     TYPE(MPI_Info), INTENT(IN) :: info

```

```

CHARACTER(LEN=MPI_MAX_PORT_NAME), INTENT(OUT) :: port_name      1
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                        2
                                                                    3
MPI_Publish_name(service_name, info, port_name, ierror)        4
CHARACTER(LEN=*), INTENT(IN) :: service_name, port_name        5
TYPE(MPI_Info), INTENT(IN) :: info                             6
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                        7
                                                                    8
MPI_Query_thread(provided, ierror)                              9
INTEGER, INTENT(OUT) :: provided                               10
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       11
                                                                    12
MPI_Session_finalize(session, ierror)                           13
TYPE(MPI_Session), INTENT(INOUT) :: session                   14
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       15
                                                                    16
MPI_Session_get_info(session, info_used, ierror)               17
TYPE(MPI_Session), INTENT(IN) :: session                       18
TYPE(MPI_Info), INTENT(OUT) :: info_used                       19
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       20
                                                                    21
MPI_Session_get_nth_pset(session, info, n, pset_len, pset_name, ierror) 22
TYPE(MPI_Session), INTENT(IN) :: session                       23
TYPE(MPI_Info), INTENT(IN) :: info                             24
INTEGER, INTENT(IN) :: n                                       25
INTEGER, INTENT(INOUT) :: pset_len                             26
CHARACTER(LEN=*), INTENT(OUT) :: pset_name                     27
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       28
                                                                    29
MPI_Session_get_num_psets(session, info, npset_names, ierror)  30
TYPE(MPI_Session), INTENT(IN) :: session                       31
TYPE(MPI_Info), INTENT(IN) :: info                             32
INTEGER, INTENT(OUT) :: npset_names                           33
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       34
                                                                    35
MPI_Session_get_pset_info(session, pset_name, info, ierror)    36
TYPE(MPI_Session), INTENT(IN) :: session                       37
CHARACTER(LEN=*), INTENT(IN) :: pset_name                     38
TYPE(MPI_Info), INTENT(OUT) :: info                             39
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       40
                                                                    41
MPI_Session_init(info, errhandler, session, ierror)            42
TYPE(MPI_Info), INTENT(IN) :: info                             43
TYPE(MPI_Errhandler), INTENT(IN) :: errhandler                44
TYPE(MPI_Session), INTENT(OUT) :: session                       45
INTEGER, OPTIONAL, INTENT(OUT) :: ierror                       46
                                                                    47
MPI_Unpublish_name(service_name, info, port_name, ierror)      48
CHARACTER(LEN=*), INTENT(IN) :: service_name, port_name
TYPE(MPI_Info), INTENT(IN) :: info
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.10 One-Sided Communications Fortran 2008 Bindings

```

1  MPI_Accumulate(origin_addr, origin_count, origin_datatype, target_rank,
2
3  target_disp, target_count, target_datatype, op, win, ierror)
4
5  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
6  INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
7  TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
8  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
9  TYPE(MPI_Op), INTENT(IN) :: op
10 TYPE(MPI_Win), INTENT(IN) :: win
11 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
12
13 MPI_Accumulate(origin_addr, origin_count, origin_datatype, target_rank,
14
15 target_disp, target_count, target_datatype, op, win, ierror)
16
17 !(_c)
18
19 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
20 INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
21 TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
22 INTEGER, INTENT(IN) :: target_rank
23 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
24 TYPE(MPI_Op), INTENT(IN) :: op
25 TYPE(MPI_Win), INTENT(IN) :: win
26 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
27
28 MPI_Compare_and_swap(origin_addr, compare_addr, result_addr, datatype,
29
30 target_rank, target_disp, win, ierror)
31
32 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr,
33
34 compare_addr
35
36 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
37 TYPE(MPI_Datatype), INTENT(IN) :: datatype
38 INTEGER, INTENT(IN) :: target_rank
39 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
40 TYPE(MPI_Win), INTENT(IN) :: win
41 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_Fetch_and_op(origin_addr, result_addr, datatype, target_rank, target_disp,
44
45 op, win, ierror)
46
47 TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
48 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
49 TYPE(MPI_Datatype), INTENT(IN) :: datatype
50 INTEGER, INTENT(IN) :: target_rank
51 INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
52 TYPE(MPI_Op), INTENT(IN) :: op
53 TYPE(MPI_Win), INTENT(IN) :: win
54 INTEGER, OPTIONAL, INTENT(OUT) :: ierror
55
56 MPI_Get(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
57
58 target_count, target_datatype, win, ierror)
59
60 TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
61 INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
62 TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype

```



```

INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, ierror) !(_c)
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
        result_count, result_datatype, target_rank, target_disp,
        target_count, target_datatype, op, win, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, result_count, target_rank,
        target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
        target_datatype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Get_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
        result_count, result_datatype, target_rank, target_disp,
        target_count, target_datatype, op, win, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, result_count,
        target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
        target_datatype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Put(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
        target_count, target_datatype, win, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp

```

```

1      TYPE(MPI_Win), INTENT(IN) :: win
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_Put(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
5          target_count, target_datatype, win, ierror) !(_c)
6      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
7      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
8      TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
9      INTEGER, INTENT(IN) :: target_rank
10     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
11     TYPE(MPI_Win), INTENT(IN) :: win
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
13
14 MPI_Raccumulate(origin_addr, origin_count, origin_datatype, target_rank,
15                target_disp, target_count, target_datatype, op, win, request,
16                ierror)
17     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
18     INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
19     TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
20     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
21     TYPE(MPI_Op), INTENT(IN) :: op
22     TYPE(MPI_Win), INTENT(IN) :: win
23     TYPE(MPI_Request), INTENT(OUT) :: request
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26 MPI_Raccumulate(origin_addr, origin_count, origin_datatype, target_rank,
27                target_disp, target_count, target_datatype, op, win, request,
28                ierror) !(_c)
29     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
30     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
31     TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
32     INTEGER, INTENT(IN) :: target_rank
33     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
34     TYPE(MPI_Op), INTENT(IN) :: op
35     TYPE(MPI_Win), INTENT(IN) :: win
36     TYPE(MPI_Request), INTENT(OUT) :: request
37     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
38
39 MPI_Rget(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
40          target_count, target_datatype, win, request, ierror)
41     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr
42     INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
43     TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
44     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
45     TYPE(MPI_Win), INTENT(IN) :: win
46     TYPE(MPI_Request), INTENT(OUT) :: request
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48
49 MPI_Rget(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
50          target_count, target_datatype, win, request, ierror) !(_c)
51     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: origin_addr

```

```

INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rget_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
    result_count, result_datatype, target_rank, target_disp,
    target_count, target_datatype, op, win, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, result_count, target_rank,
    target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
    target_datatype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rget_accumulate(origin_addr, origin_count, origin_datatype, result_addr,
    result_count, result_datatype, target_rank, target_disp,
    target_count, target_datatype, op, win, request, ierror) !(_c)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, result_count,
    target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, result_datatype,
    target_datatype
TYPE(*), DIMENSION(..), ASYNCHRONOUS :: result_addr
INTEGER, INTENT(IN) :: target_rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Op), INTENT(IN) :: op
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Rput(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
    target_count, target_datatype, win, request, ierror)
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
INTEGER, INTENT(IN) :: origin_count, target_rank, target_count
TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
TYPE(MPI_Win), INTENT(IN) :: win
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Rput(origin_addr, origin_count, origin_datatype, target_rank, target_disp,
2          target_count, target_datatype, win, request, ierror) !(_c)
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: origin_addr
4      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: origin_count, target_count
5      TYPE(MPI_Datatype), INTENT(IN) :: origin_datatype, target_datatype
6      INTEGER, INTENT(IN) :: target_rank
7      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: target_disp
8      TYPE(MPI_Win), INTENT(IN) :: win
9      TYPE(MPI_Request), INTENT(OUT) :: request
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12  MPI_Win_allocate(size, disp_unit, info, comm, baseptr, win, ierror)
13      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
14      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
15      INTEGER, INTENT(IN) :: disp_unit
16      TYPE(MPI_Info), INTENT(IN) :: info
17      TYPE(MPI_Comm), INTENT(IN) :: comm
18      TYPE(C_PTR), INTENT(OUT) :: baseptr
19      TYPE(MPI_Win), INTENT(OUT) :: win
20      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
21
22  MPI_Win_allocate(size, disp_unit, info, comm, baseptr, win, ierror) !(_c)
23      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
24      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size, disp_unit
25      TYPE(MPI_Info), INTENT(IN) :: info
26      TYPE(MPI_Comm), INTENT(IN) :: comm
27      TYPE(C_PTR), INTENT(OUT) :: baseptr
28      TYPE(MPI_Win), INTENT(OUT) :: win
29      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
30
31  MPI_Win_allocate_shared(size, disp_unit, info, comm, baseptr, win, ierror)
32      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
33      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
34      INTEGER, INTENT(IN) :: disp_unit
35      TYPE(MPI_Info), INTENT(IN) :: info
36      TYPE(MPI_Comm), INTENT(IN) :: comm
37      TYPE(C_PTR), INTENT(OUT) :: baseptr
38      TYPE(MPI_Win), INTENT(OUT) :: win
39      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41  MPI_Win_allocate_shared(size, disp_unit, info, comm, baseptr, win, ierror)
42      !(_c)
43      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
44      INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size, disp_unit
45      TYPE(MPI_Info), INTENT(IN) :: info
46      TYPE(MPI_Comm), INTENT(IN) :: comm
47      TYPE(C_PTR), INTENT(OUT) :: baseptr
48      TYPE(MPI_Win), INTENT(OUT) :: win
49      INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_Win_attach(win, base, size, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_complete(win, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_create(base, size, disp_unit, info, comm, win, ierror)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size
  INTEGER, INTENT(IN) :: disp_unit
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Win), INTENT(OUT) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_create(base, size, disp_unit, info, comm, win, ierror) !(_c)
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: size, disp_unit
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Win), INTENT(OUT) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_create_dynamic(info, comm, win, ierror)
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_Comm), INTENT(IN) :: comm
  TYPE(MPI_Win), INTENT(OUT) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_detach(win, base, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: base
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_fence(assert, win, ierror)
  INTEGER, INTENT(IN) :: assert
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_flush(rank, win, ierror)
  INTEGER, INTENT(IN) :: rank
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_flush_all(win, ierror)
  TYPE(MPI_Win), INTENT(IN) :: win
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Win_flush_local(rank, win, ierror)
  INTEGER, INTENT(IN) :: rank

```

```

1      TYPE(MPI_Win), INTENT(IN) :: win
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_Win_flush_local_all(win, ierror)
5      TYPE(MPI_Win), INTENT(IN) :: win
6      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
7
8  MPI_Win_free(win, ierror)
9      TYPE(MPI_Win), INTENT(INOUT) :: win
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12 MPI_Win_get_group(win, group, ierror)
13     TYPE(MPI_Win), INTENT(IN) :: win
14     TYPE(MPI_Group), INTENT(OUT) :: group
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17 MPI_Win_get_info(win, info_used, ierror)
18     TYPE(MPI_Win), INTENT(IN) :: win
19     TYPE(MPI_Info), INTENT(OUT) :: info_used
20     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
21
22 MPI_Win_lock(lock_type, rank, assert, win, ierror)
23     INTEGER, INTENT(IN) :: lock_type, rank, assert
24     TYPE(MPI_Win), INTENT(IN) :: win
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26
27 MPI_Win_lock_all(assert, win, ierror)
28     INTEGER, INTENT(IN) :: assert
29     TYPE(MPI_Win), INTENT(IN) :: win
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_Win_post(group, assert, win, ierror)
33     TYPE(MPI_Group), INTENT(IN) :: group
34     INTEGER, INTENT(IN) :: assert
35     TYPE(MPI_Win), INTENT(IN) :: win
36     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
37
38 MPI_Win_set_info(win, info, ierror)
39     TYPE(MPI_Win), INTENT(IN) :: win
40     TYPE(MPI_Info), INTENT(IN) :: info
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_Win_shared_query(win, rank, size, disp_unit, baseptr, ierror)
44     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
45     TYPE(MPI_Win), INTENT(IN) :: win
46     INTEGER, INTENT(IN) :: rank
47     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: size
48     INTEGER, INTENT(OUT) :: disp_unit
49     TYPE(C_PTR), INTENT(OUT) :: baseptr
50     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
51
52 MPI_Win_shared_query(win, rank, size, disp_unit, baseptr, ierror) !(_c)
53     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
54     TYPE(MPI_Win), INTENT(IN) :: win

```

```

INTEGER, INTENT(IN) :: rank
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: size, disp_unit
TYPE(C_PTR), INTENT(OUT) :: baseptr
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Win_start(group, assert, win, ierror)
TYPE(MPI_Group), INTENT(IN) :: group
INTEGER, INTENT(IN) :: assert
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Win_sync(win, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Win_test(win, flag, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
LOGICAL, INTENT(OUT) :: flag
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Win_unlock(rank, win, ierror)
INTEGER, INTENT(IN) :: rank
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Win_unlock_all(win, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Win_wait(win, ierror)
TYPE(MPI_Win), INTENT(IN) :: win
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

A.4.11 External Interfaces Fortran 2008 Bindings
MPI_Grequest_complete(request, ierror)
TYPE(MPI_Request), INTENT(IN) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Grequest_start(query_fn, free_fn, cancel_fn, extra_state, request, ierror)
PROCEDURE(MPI_Grequest_query_function) :: query_fn
PROCEDURE(MPI_Grequest_free_function) :: free_fn
PROCEDURE(MPI_Grequest_cancel_function) :: cancel_fn
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
TYPE(MPI_Request), INTENT(OUT) :: request
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Status_set_cancelled(status, flag, ierror)
TYPE(MPI_Status), INTENT(INOUT) :: status
LOGICAL, INTENT(IN) :: flag
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_Status_set_elements(status, datatype, count, ierror)
TYPE(MPI_Status), INTENT(INOUT) :: status

```



```

1      TYPE(MPI_Datatype), INTENT(IN) :: datatype
2      INTEGER, INTENT(IN) :: count
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
4
5      MPI_Status_set_elements(status, datatype, count, ierror) !(_c)
6      TYPE(MPI_Status), INTENT(INOUT) :: status
7      TYPE(MPI_Datatype), INTENT(IN) :: datatype
8      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11     MPI_Status_set_error(status, err, ierror)
12     TYPE(MPI_Status), INTENT(INOUT) :: status
13     INTEGER, INTENT(IN) :: err
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16     MPI_Status_set_source(status, source, ierror)
17     TYPE(MPI_Status), INTENT(INOUT) :: status
18     INTEGER, INTENT(IN) :: source
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21     MPI_Status_set_tag(status, tag, ierror)
22     TYPE(MPI_Status), INTENT(INOUT) :: status
23     INTEGER, INTENT(IN) :: tag
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

```

A.4.12 I/O Fortran 2008 Bindings

```

26     MPI_CONVERSION_FN_NULL(userbuf, datatype, count, filebuf, position,
27                             extra_state, ierror)
28     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
29     TYPE(C_PTR), VALUE :: userbuf, filebuf
30     TYPE(MPI_Datatype) :: datatype
31     INTEGER :: count, ierror
32     INTEGER(KIND=MPI_OFFSET_KIND) :: position
33     INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
34
35     MPI_CONVERSION_FN_NULL_C(userbuf, datatype, count, filebuf, position,
36                              extra_state, ierror) !(_c)
37     USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
38     TYPE(C_PTR), VALUE :: userbuf, filebuf
39     TYPE(MPI_Datatype) :: datatype
40     INTEGER(KIND=MPI_COUNT_KIND) :: count
41     INTEGER(KIND=MPI_OFFSET_KIND) :: position
42     INTEGER(KIND=MPI_ADDRESS_KIND) :: extra_state
43     INTEGER :: ierror
44
45     MPI_File_close(fh, ierror)
46     TYPE(MPI_File), INTENT(INOUT) :: fh
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48
49     MPI_File_delete(filename, info, ierror)
50     CHARACTER(LEN=*) , INTENT(IN) :: filename

```



```

TYPE(MPI_Info), INTENT(IN) :: info
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_amode(fh, amode, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER, INTENT(OUT) :: amode
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_atomicsity(fh, flag, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
LOGICAL, INTENT(OUT) :: flag
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_byte_offset(fh, offset, disp, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: disp
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_group(fh, group, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(MPI_Group), INTENT(OUT) :: group
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_info(fh, info_used, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(MPI_Info), INTENT(OUT) :: info_used
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_position(fh, offset, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: offset
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_position_shared(fh, offset, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: offset
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_size(fh, size, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: size
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_type_extent(fh, datatype, extent, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(OUT) :: extent
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_get_type_extent(fh, datatype, extent, ierror) !(_c)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

1      INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: extent
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_File_get_view(fh, disp, etype, filetype, datarep, ierror)
5      TYPE(MPI_File), INTENT(IN) :: fh
6      INTEGER(KIND=MPI_OFFSET_KIND), INTENT(OUT) :: disp
7      TYPE(MPI_Datatype), INTENT(OUT) :: etype, filetype
8      CHARACTER(LEN=*), INTENT(OUT) :: datarep
9      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
10
11 MPI_File_iread(fh, buf, count, datatype, request, ierror)
12     TYPE(MPI_File), INTENT(IN) :: fh
13     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
14     INTEGER, INTENT(IN) :: count
15     TYPE(MPI_Datatype), INTENT(IN) :: datatype
16     TYPE(MPI_Request), INTENT(OUT) :: request
17     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
18
19 MPI_File_iread(fh, buf, count, datatype, request, ierror) !(_c)
20     TYPE(MPI_File), INTENT(IN) :: fh
21     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
22     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     TYPE(MPI_Request), INTENT(OUT) :: request
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26
27 MPI_File_iread_all(fh, buf, count, datatype, request, ierror)
28     TYPE(MPI_File), INTENT(IN) :: fh
29     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
30     INTEGER, INTENT(IN) :: count
31     TYPE(MPI_Datatype), INTENT(IN) :: datatype
32     TYPE(MPI_Request), INTENT(OUT) :: request
33     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
34
35 MPI_File_iread_all(fh, buf, count, datatype, request, ierror) !(_c)
36     TYPE(MPI_File), INTENT(IN) :: fh
37     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
38     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
39     TYPE(MPI_Datatype), INTENT(IN) :: datatype
40     TYPE(MPI_Request), INTENT(OUT) :: request
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_File_iread_at(fh, offset, buf, count, datatype, request, ierror)
44     TYPE(MPI_File), INTENT(IN) :: fh
45     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
46     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
47     INTEGER, INTENT(IN) :: count
48     TYPE(MPI_Datatype), INTENT(IN) :: datatype
49     TYPE(MPI_Request), INTENT(OUT) :: request
50     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_File_iread_at(fh, offset, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iread_at_all(fh, offset, buf, count, datatype, request, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iread_at_all(fh, offset, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iread_shared(fh, buf, count, datatype, request, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iread_shared(fh, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iwrite(fh, buf, count, datatype, request, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_File_iwrite(fh, buf, count, datatype, request, ierror) !(_c)
2      TYPE(MPI_File), INTENT(IN) :: fh
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
4      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
5      TYPE(MPI_Datatype), INTENT(IN) :: datatype
6      TYPE(MPI_Request), INTENT(OUT) :: request
7      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9  MPI_File_iwrite_all(fh, buf, count, datatype, request, ierror)
10     TYPE(MPI_File), INTENT(IN) :: fh
11     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
12     INTEGER, INTENT(IN) :: count
13     TYPE(MPI_Datatype), INTENT(IN) :: datatype
14     TYPE(MPI_Request), INTENT(OUT) :: request
15     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
16
17 MPI_File_iwrite_all(fh, buf, count, datatype, request, ierror) !(_c)
18     TYPE(MPI_File), INTENT(IN) :: fh
19     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
20     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
21     TYPE(MPI_Datatype), INTENT(IN) :: datatype
22     TYPE(MPI_Request), INTENT(OUT) :: request
23     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
24
25 MPI_File_iwrite_at(fh, offset, buf, count, datatype, request, ierror)
26     TYPE(MPI_File), INTENT(IN) :: fh
27     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
28     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
29     INTEGER, INTENT(IN) :: count
30     TYPE(MPI_Datatype), INTENT(IN) :: datatype
31     TYPE(MPI_Request), INTENT(OUT) :: request
32     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
33
34 MPI_File_iwrite_at(fh, offset, buf, count, datatype, request, ierror) !(_c)
35     TYPE(MPI_File), INTENT(IN) :: fh
36     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
37     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
38     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
39     TYPE(MPI_Datatype), INTENT(IN) :: datatype
40     TYPE(MPI_Request), INTENT(OUT) :: request
41     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
42
43 MPI_File_iwrite_at_all(fh, offset, buf, count, datatype, request, ierror)
44     TYPE(MPI_File), INTENT(IN) :: fh
45     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
46     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
47     INTEGER, INTENT(IN) :: count
48     TYPE(MPI_Datatype), INTENT(IN) :: datatype
49     TYPE(MPI_Request), INTENT(OUT) :: request
50     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

MPI_File_iwrite_at_all(fh, offset, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iwrite_shared(fh, buf, count, datatype, request, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_iwrite_shared(fh, buf, count, datatype, request, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Request), INTENT(OUT) :: request
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_open(comm, filename, amode, info, fh, ierror)
  TYPE(MPI_Comm), INTENT(IN) :: comm
  CHARACTER(LEN=*), INTENT(IN) :: filename
  INTEGER, INTENT(IN) :: amode
  TYPE(MPI_Info), INTENT(IN) :: info
  TYPE(MPI_File), INTENT(OUT) :: fh
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_preallocate(fh, size, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: size
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read(fh, buf, count, datatype, status, ierror)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..) :: buf
  INTEGER, INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype
  TYPE(MPI_Status) :: status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read(fh, buf, count, datatype, status, ierror) !(_c)
  TYPE(MPI_File), INTENT(IN) :: fh
  TYPE(*), DIMENSION(..) :: buf
  INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
  TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

1      TYPE(MPI_Status) :: status
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_File_read_all(fh, buf, count, datatype, status, ierror)
5      TYPE(MPI_File), INTENT(IN) :: fh
6      TYPE(*), DIMENSION(..) :: buf
7      INTEGER, INTENT(IN) :: count
8      TYPE(MPI_Datatype), INTENT(IN) :: datatype
9      TYPE(MPI_Status) :: status
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12 MPI_File_read_all(fh, buf, count, datatype, status, ierror) !(_c)
13     TYPE(MPI_File), INTENT(IN) :: fh
14     TYPE(*), DIMENSION(..) :: buf
15     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
16     TYPE(MPI_Datatype), INTENT(IN) :: datatype
17     TYPE(MPI_Status) :: status
18     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
19
20 MPI_File_read_all_begin(fh, buf, count, datatype, ierror)
21     TYPE(MPI_File), INTENT(IN) :: fh
22     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
23     INTEGER, INTENT(IN) :: count
24     TYPE(MPI_Datatype), INTENT(IN) :: datatype
25     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
26
27 MPI_File_read_all_begin(fh, buf, count, datatype, ierror) !(_c)
28     TYPE(MPI_File), INTENT(IN) :: fh
29     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
30     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
31     TYPE(MPI_Datatype), INTENT(IN) :: datatype
32     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
33
34 MPI_File_read_all_end(fh, buf, status, ierror)
35     TYPE(MPI_File), INTENT(IN) :: fh
36     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
37     TYPE(MPI_Status) :: status
38     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
39
40 MPI_File_read_at(fh, offset, buf, count, datatype, status, ierror)
41     TYPE(MPI_File), INTENT(IN) :: fh
42     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
43     TYPE(*), DIMENSION(..) :: buf
44     INTEGER, INTENT(IN) :: count
45     TYPE(MPI_Datatype), INTENT(IN) :: datatype
46     TYPE(MPI_Status) :: status
47     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
48
49 MPI_File_read_at(fh, offset, buf, count, datatype, status, ierror) !(_c)
50     TYPE(MPI_File), INTENT(IN) :: fh
51     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
52     TYPE(*), DIMENSION(..) :: buf

```

```

    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_at_all(fh, offset, buf, count, datatype, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..) :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_at_all(fh, offset, buf, count, datatype, status, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..) :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_at_all_begin(fh, offset, buf, count, datatype, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_at_all_begin(fh, offset, buf, count, datatype, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_at_all_end(fh, buf, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_read_ordered(fh, buf, count, datatype, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..) :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```



```

1  MPI_File_read_ordered(fh, buf, count, datatype, status, ierror) !(_c)
2      TYPE(MPI_File), INTENT(IN) :: fh
3      TYPE(*), DIMENSION(..) :: buf
4      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
5      TYPE(MPI_Datatype), INTENT(IN) :: datatype
6      TYPE(MPI_Status) :: status
7      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
8
9  MPI_File_read_ordered_begin(fh, buf, count, datatype, ierror)
10     TYPE(MPI_File), INTENT(IN) :: fh
11     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
12     INTEGER, INTENT(IN) :: count
13     TYPE(MPI_Datatype), INTENT(IN) :: datatype
14     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
15
16 MPI_File_read_ordered_begin(fh, buf, count, datatype, ierror) !(_c)
17     TYPE(MPI_File), INTENT(IN) :: fh
18     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
19     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
20     TYPE(MPI_Datatype), INTENT(IN) :: datatype
21     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
22
23 MPI_File_read_ordered_end(fh, buf, status, ierror)
24     TYPE(MPI_File), INTENT(IN) :: fh
25     TYPE(*), DIMENSION(..), ASYNCHRONOUS :: buf
26     TYPE(MPI_Status) :: status
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
28
29 MPI_File_read_shared(fh, buf, count, datatype, status, ierror)
30     TYPE(MPI_File), INTENT(IN) :: fh
31     TYPE(*), DIMENSION(..) :: buf
32     INTEGER, INTENT(IN) :: count
33     TYPE(MPI_Datatype), INTENT(IN) :: datatype
34     TYPE(MPI_Status) :: status
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37 MPI_File_read_shared(fh, buf, count, datatype, status, ierror) !(_c)
38     TYPE(MPI_File), INTENT(IN) :: fh
39     TYPE(*), DIMENSION(..) :: buf
40     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
41     TYPE(MPI_Datatype), INTENT(IN) :: datatype
42     TYPE(MPI_Status) :: status
43     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
44
45 MPI_File_seek(fh, offset, whence, ierror)
46     TYPE(MPI_File), INTENT(IN) :: fh
47     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
48     INTEGER, INTENT(IN) :: whence
49     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
50
51 MPI_File_seek_shared(fh, offset, whence, ierror)
52     TYPE(MPI_File), INTENT(IN) :: fh

```



```

    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
    INTEGER, INTENT(IN) :: whence
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_set_atomicity(fh, flag, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    LOGICAL, INTENT(IN) :: flag
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_set_info(fh, info, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_set_size(fh, size, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_set_view(fh, disp, etype, filetype, datarep, info, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: disp
    TYPE(MPI_Datatype), INTENT(IN) :: etype, filetype
    CHARACTER(LEN=*), INTENT(IN) :: datarep
    TYPE(MPI_Info), INTENT(IN) :: info
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_sync(fh, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write(fh, buf, count, datatype, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write(fh, buf, count, datatype, status, ierror) !(_c)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    TYPE(MPI_Status) :: status
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_File_write_all(fh, buf, count, datatype, status, ierror)
    TYPE(MPI_File), INTENT(IN) :: fh
    TYPE(*), DIMENSION(..), INTENT(IN) :: buf
    INTEGER, INTENT(IN) :: count
    TYPE(MPI_Datatype), INTENT(IN) :: datatype

```

```

1      TYPE(MPI_Status) :: status
2      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
3
4  MPI_File_write_all(fh, buf, count, datatype, status, ierror) !(_c)
5      TYPE(MPI_File), INTENT(IN) :: fh
6      TYPE(*), DIMENSION(..), INTENT(IN) :: buf
7      INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
8      TYPE(MPI_Datatype), INTENT(IN) :: datatype
9      TYPE(MPI_Status) :: status
10     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
11
12 MPI_File_write_all_begin(fh, buf, count, datatype, ierror)
13     TYPE(MPI_File), INTENT(IN) :: fh
14     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
15     INTEGER, INTENT(IN) :: count
16     TYPE(MPI_Datatype), INTENT(IN) :: datatype
17     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
18
19 MPI_File_write_all_begin(fh, buf, count, datatype, ierror) !(_c)
20     TYPE(MPI_File), INTENT(IN) :: fh
21     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
22     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
23     TYPE(MPI_Datatype), INTENT(IN) :: datatype
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26 MPI_File_write_all_end(fh, buf, status, ierror)
27     TYPE(MPI_File), INTENT(IN) :: fh
28     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
29     TYPE(MPI_Status) :: status
30     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
31
32 MPI_File_write_at(fh, offset, buf, count, datatype, status, ierror)
33     TYPE(MPI_File), INTENT(IN) :: fh
34     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
35     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
36     INTEGER, INTENT(IN) :: count
37     TYPE(MPI_Datatype), INTENT(IN) :: datatype
38     TYPE(MPI_Status) :: status
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41 MPI_File_write_at(fh, offset, buf, count, datatype, status, ierror) !(_c)
42     TYPE(MPI_File), INTENT(IN) :: fh
43     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
44     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
45     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
46     TYPE(MPI_Datatype), INTENT(IN) :: datatype
47     TYPE(MPI_Status) :: status
48     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
49
50 MPI_File_write_at_all(fh, offset, buf, count, datatype, status, ierror)
51     TYPE(MPI_File), INTENT(IN) :: fh
52     INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset

```

```

TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_at_all(fh, offset, buf, count, datatype, status, ierror) !(_c)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_at_all_begin(fh, offset, buf, count, datatype, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_at_all_begin(fh, offset, buf, count, datatype, ierror) !(_c)
TYPE(MPI_File), INTENT(IN) :: fh
INTEGER(KIND=MPI_OFFSET_KIND), INTENT(IN) :: offset
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_at_all_end(fh, buf, status, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_ordered(fh, buf, count, datatype, status, ierror)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER, INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror
MPI_File_write_ordered(fh, buf, count, datatype, status, ierror) !(_c)
TYPE(MPI_File), INTENT(IN) :: fh
TYPE(*), DIMENSION(..), INTENT(IN) :: buf
INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
TYPE(MPI_Datatype), INTENT(IN) :: datatype
TYPE(MPI_Status) :: status
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_File_write_ordered_begin(fh, buf, count, datatype, ierror)
2      TYPE(MPI_File), INTENT(IN) :: fh
3      TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
4      INTEGER, INTENT(IN) :: count
5      TYPE(MPI_Datatype), INTENT(IN) :: datatype
6      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
7
8  MPI_File_write_ordered_begin(fh, buf, count, datatype, ierror) !(_c)
9      TYPE(MPI_File), INTENT(IN) :: fh
10     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
11     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
12     TYPE(MPI_Datatype), INTENT(IN) :: datatype
13     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
14
15  MPI_File_write_ordered_end(fh, buf, status, ierror)
16     TYPE(MPI_File), INTENT(IN) :: fh
17     TYPE(*), DIMENSION(..), INTENT(IN), ASYNCHRONOUS :: buf
18     TYPE(MPI_Status) :: status
19     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
20
21  MPI_File_write_shared(fh, buf, count, datatype, status, ierror)
22     TYPE(MPI_File), INTENT(IN) :: fh
23     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
24     INTEGER, INTENT(IN) :: count
25     TYPE(MPI_Datatype), INTENT(IN) :: datatype
26     TYPE(MPI_Status) :: status
27     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
28
29  MPI_File_write_shared(fh, buf, count, datatype, status, ierror) !(_c)
30     TYPE(MPI_File), INTENT(IN) :: fh
31     TYPE(*), DIMENSION(..), INTENT(IN) :: buf
32     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
33     TYPE(MPI_Datatype), INTENT(IN) :: datatype
34     TYPE(MPI_Status) :: status
35     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
36
37  MPI_Register_datarep(datarep, read_conversion_fn, write_conversion_fn,
38      dtype_file_extent_fn, extra_state, ierror)
39     CHARACTER(LEN=*), INTENT(IN) :: datarep
40     PROCEDURE(MPI_Datarep_conversion_function) :: read_conversion_fn,
41         write_conversion_fn
42     PROCEDURE(MPI_Datarep_extent_function) :: dtype_file_extent_fn
43     INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
44     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
45
46  MPI_Register_datarep_c(datarep, read_conversion_fn, write_conversion_fn,
47      dtype_file_extent_fn, extra_state, ierror) !(_c)
48     CHARACTER(LEN=*), INTENT(IN) :: datarep
49     PROCEDURE(MPI_Datarep_conversion_function_c) :: read_conversion_fn,
50         write_conversion_fn
51     PROCEDURE(MPI_Datarep_extent_function) :: dtype_file_extent_fn

```

```

INTEGER(KIND=MPI_ADDRESS_KIND), INTENT(IN) :: extra_state
INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.13 Language Bindings Fortran 2008 Bindings

```

MPI_Status_f082f(f08_status, f_status, ierror)
  TYPE(MPI_Status), INTENT(IN) :: f08_status
  INTEGER, INTENT(OUT) :: f_status(MPI_STATUS_SIZE)
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Status_f2f08(f_status, f08_status, ierror)
  INTEGER, INTENT(IN) :: f_status(MPI_STATUS_SIZE)
  TYPE(MPI_Status), INTENT(OUT) :: f08_status
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_f90_complex(p, r, newtype, ierror)
  INTEGER, INTENT(IN) :: p, r
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_f90_integer(r, newtype, ierror)
  INTEGER, INTENT(IN) :: r
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_create_f90_real(p, r, newtype, ierror)
  INTEGER, INTENT(IN) :: p, r
  TYPE(MPI_Datatype), INTENT(OUT) :: newtype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_match_size(typeclass, size, datatype, ierror)
  INTEGER, INTENT(IN) :: typeclass, size
  TYPE(MPI_Datatype), INTENT(OUT) :: datatype
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.14 Application Binary Interface (ABI) Fortran 2008 Bindings

```

MPI_Abi_get_fortran_booleans(logical_size, logical_true, logical_false, is_set,
                             ierror)
  INTEGER, INTENT(IN) :: logical_size
  LOGICAL, INTENT(OUT) :: logical_true, logical_false, is_set
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Abi_get_fortran_info(info, ierror)
  TYPE(MPI_Info), INTENT(OUT) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Abi_get_info(info, ierror)
  TYPE(MPI_Info), INTENT(OUT) :: info
  INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```

1  MPI_Abi_get_version(abi_major, abi_minor, ierror)
2      INTEGER, INTENT(OUT) :: abi_major, abi_minor
3      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
4
5  MPI_Abi_set_fortran_booleans(logical_size, logical_true, logical_false, ierror)
6      INTEGER, INTENT(IN) :: logical_size
7      LOGICAL, INTENT(IN) :: logical_true, logical_false
8      INTEGER, OPTIONAL, INTENT(OUT) :: ierror
9
10 MPI_Abi_set_fortran_info(info, ierror)
11     TYPE(MPI_Info), INTENT(IN) :: info
12     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

A.4.15 Tools / Profiling Interface Fortran 2008 Bindings

```

15 MPI_Pcontrol(level)
16     INTEGER, INTENT(IN) :: level

```

A.4.16 Deprecated Fortran 2008 Bindings

```

20 MPI_Get_elements_x(status, datatype, count, ierror)
21     TYPE(MPI_Status), INTENT(IN) :: status
22     TYPE(MPI_Datatype), INTENT(IN) :: datatype
23     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: count
24     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
25
26 MPI_Info_get(info, key, valuelen, value, flag, ierror)
27     TYPE(MPI_Info), INTENT(IN) :: info
28     CHARACTER(LEN=*), INTENT(IN) :: key
29     INTEGER, INTENT(IN) :: valuelen
30     CHARACTER(LEN=valuelen), INTENT(OUT) :: value
31     LOGICAL, INTENT(OUT) :: flag
32     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
33
34 MPI_Info_get_valuelen(info, key, valuelen, flag, ierror)
35     TYPE(MPI_Info), INTENT(IN) :: info
36     CHARACTER(LEN=*), INTENT(IN) :: key
37     INTEGER, INTENT(OUT) :: valuelen
38     LOGICAL, INTENT(OUT) :: flag
39     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
40
41 MPI_Status_set_elements_x(status, datatype, count, ierror)
42     TYPE(MPI_Status), INTENT(INOUT) :: status
43     TYPE(MPI_Datatype), INTENT(IN) :: datatype
44     INTEGER(KIND=MPI_COUNT_KIND), INTENT(IN) :: count
45     INTEGER, OPTIONAL, INTENT(OUT) :: ierror
46
47 MPI_Type_get_extent_x(datatype, lb, extent, ierror)
48     TYPE(MPI_Datatype), INTENT(IN) :: datatype
49     INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: lb, extent
50     INTEGER, OPTIONAL, INTENT(OUT) :: ierror

```

```
MPI_Type_get_true_extent_x(datatype, true_lb, true_extent, ierror)
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: true_lb, true_extent
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror

MPI_Type_size_x(datatype, size, ierror)
    TYPE(MPI_Datatype), INTENT(IN) :: datatype
    INTEGER(KIND=MPI_COUNT_KIND), INTENT(OUT) :: size
    INTEGER, OPTIONAL, INTENT(OUT) :: ierror
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

A.5 Fortran Bindings with `mpif.h` or the `mpi` Module

A.5.1 Point-to-Point Communication Fortran Bindings

```

1  MPI_BSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
2
3  A.5.1 Point-to-Point Communication Fortran Bindings
4
5  MPI_BSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
6      <type> BUF(*)
7      INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR
8
9  MPI_BSEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
10     <type> BUF(*)
11     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
12
13 MPI_BUFFER_ATTACH(BUFFER, SIZE, IERROR)
14     <type> BUFFER(*)
15     INTEGER SIZE, IERROR
16
17 MPI_BUFFER_DETACH(BUFFER_ADDR, SIZE, IERROR)
18     <type> BUFFER_ADDR(*)
19     INTEGER SIZE, IERROR
20
21 MPI_BUFFER_FLUSH(IERROR)
22     INTEGER IERROR
23
24 MPI_BUFFER_IFLUSH(REQUEST, IERROR)
25     INTEGER REQUEST, IERROR
26
27 MPI_CANCEL(REQUEST, IERROR)
28     INTEGER REQUEST, IERROR
29
30 MPI_COMM_ATTACH_BUFFER(COMM, BUFFER, SIZE, IERROR)
31     INTEGER COMM, SIZE, IERROR
32     <type> BUFFER(*)
33
34 MPI_COMM_DETACH_BUFFER(COMM, BUFFER_ADDR, SIZE, IERROR)
35     INTEGER COMM, SIZE, IERROR
36     <type> BUFFER_ADDR(*)
37
38 MPI_COMM_FLUSH_BUFFER(COMM, IERROR)
39     INTEGER COMM, IERROR
40
41 MPI_COMM_IFLUSH_BUFFER(COMM, REQUEST, IERROR)
42     INTEGER COMM, REQUEST, IERROR
43
44 MPI_GET_COUNT(STATUS, DATATYPE, COUNT, IERROR)
45     INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, COUNT, IERROR
46
47 MPI_IBSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
48     <type> BUF(*)
49     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
50
51 MPI_IMPROBE(SOURCE, TAG, COMM, FLAG, MESSAGE, STATUS, IERROR)
52     INTEGER SOURCE, TAG, COMM, MESSAGE, STATUS(MPI_STATUS_SIZE), IERROR
53     LOGICAL FLAG
54
55 MPI_IMRECV(BUF, COUNT, DATATYPE, MESSAGE, REQUEST, IERROR)
56     <type> BUF(*)
57     INTEGER COUNT, DATATYPE, MESSAGE, REQUEST, IERROR

```


MPI_IPROBE (SOURCE, TAG, COMM, FLAG, STATUS, IERROR)	1
INTEGER SOURCE, TAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR	2
LOGICAL FLAG	3
MPI_Irecv (BUF, COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR)	4
<type> BUF(*)	5
INTEGER COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR	6
MPI_Irsend (BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)	7
<type> BUF(*)	8
INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR	9
MPI_Isend (BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)	10
<type> BUF(*)	11
INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR	12
MPI_Isendrecv (SENDBUF, SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVBUF, RECVCOUNT,	13
RECVMODE, SOURCE, RECVTAG, COMM, REQUEST, IERROR)	14
<type> SENDBUF(*), RECVBUF(*)	15
INTEGER SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVCOUNT, RECVTYPE, SOURCE,	16
RECVTAG, COMM, REQUEST, IERROR	17
MPI_Isendrecv_replace (BUF, COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG,	18
COMM, REQUEST, IERROR)	19
<type> BUF(*)	20
INTEGER COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG, COMM, REQUEST,	21
IERROR	22
MPI_issend (BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)	23
<type> BUF(*)	24
INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR	25
MPI_Mprobe (SOURCE, TAG, COMM, MESSAGE, STATUS, IERROR)	26
INTEGER SOURCE, TAG, COMM, MESSAGE, STATUS(MPI_STATUS_SIZE), IERROR	27
MPI_Mrecv (BUF, COUNT, DATATYPE, MESSAGE, STATUS, IERROR)	28
<type> BUF(*)	29
INTEGER COUNT, DATATYPE, MESSAGE, STATUS(MPI_STATUS_SIZE), IERROR	30
MPI_Probe (SOURCE, TAG, COMM, STATUS, IERROR)	31
INTEGER SOURCE, TAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR	32
MPI_Recv (BUF, COUNT, DATATYPE, SOURCE, TAG, COMM, STATUS, IERROR)	33
<type> BUF(*)	34
INTEGER COUNT, DATATYPE, SOURCE, TAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR	35
MPI_Recv_init (BUF, COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR)	36
<type> BUF(*)	37
INTEGER COUNT, DATATYPE, SOURCE, TAG, COMM, REQUEST, IERROR	38
MPI_Request_free (REQUEST, IERROR)	39
INTEGER REQUEST, IERROR	40
MPI_Request_get_status (REQUEST, FLAG, STATUS, IERROR)	41
INTEGER REQUEST, STATUS(MPI_STATUS_SIZE), IERROR	42
LOGICAL FLAG	43
	44
	45
	46
	47
	48

```

1  MPI_REQUEST_GET_STATUS_ALL(COUNT, ARRAY_OF_REQUESTS, FLAG, ARRAY_OF_STATUSES,
2      IERROR)
3      INTEGER COUNT, ARRAY_OF_REQUESTS(*), ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *),
4      IERROR
5      LOGICAL FLAG
6
7  MPI_REQUEST_GET_STATUS_ANY(COUNT, ARRAY_OF_REQUESTS, INDEX, FLAG, STATUS,
8      IERROR)
9      INTEGER COUNT, ARRAY_OF_REQUESTS(*), INDEX, STATUS(MPI_STATUS_SIZE), IERROR
10     LOGICAL FLAG
11
12 MPI_REQUEST_GET_STATUS_SOME(INCOUNT, ARRAY_OF_REQUESTS, OUTCOUNT,
13     ARRAY_OF_INDICES, ARRAY_OF_STATUSES, IERROR)
14     INTEGER INCOUNT, ARRAY_OF_REQUESTS(*), OUTCOUNT, ARRAY_OF_INDICES(*),
15     ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *), IERROR
16
17 MPI_RSEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
18     <type> BUF(*)
19     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR
20
21 MPI_RSEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
22     <type> BUF(*)
23     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
24
25 MPI_SEND(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)
26     <type> BUF(*)
27     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR
28
29 MPI_SEND_INIT(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)
30     <type> BUF(*)
31     INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR
32
33 MPI_SENDRECV(SENDBUF, SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVBUF, RECVCOUNT,
34     RECVTYPE, SOURCE, RECVTAG, COMM, STATUS, IERROR)
35     <type> SENDBUF(*), RECVBUF(*)
36     INTEGER SENDCOUNT, SENDTYPE, DEST, SENDTAG, RECVCOUNT, RECVTYPE, SOURCE,
37     RECVTAG, COMM, STATUS(MPI_STATUS_SIZE), IERROR
38
39 MPI_SENDRECV_REPLACE(BUF, COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG,
40     COMM, STATUS, IERROR)
41     <type> BUF(*)
42     INTEGER COUNT, DATATYPE, DEST, SENDTAG, SOURCE, RECVTAG, COMM,
43     STATUS(MPI_STATUS_SIZE), IERROR
44
45 MPI_SESSION_ATTACH_BUFFER(SESSION, BUFFER, SIZE, IERROR)
46     INTEGER SESSION, SIZE, IERROR
47     <type> BUFFER(*)
48
49 MPI_SESSION_DETACH_BUFFER(SESSION, BUFFER_ADDR, SIZE, IERROR)
50     INTEGER SESSION, SIZE, IERROR
51     <type> BUFFER_ADDR(*)
52
53 MPI_SESSION_FLUSH_BUFFER(SESSION, IERROR)
54     INTEGER SESSION, IERROR

```

<code>MPI_SESSION_FLUSH_BUFFER</code>	(SESSION, REQUEST, IERROR)	1
	INTEGER SESSION, REQUEST, IERROR	2
<code>MPI_SSEND</code>	(BUF, COUNT, DATATYPE, DEST, TAG, COMM, IERROR)	3
	<type> BUF(*)	4
	INTEGER COUNT, DATATYPE, DEST, TAG, COMM, IERROR	5
<code>MPI_SSEND_INIT</code>	(BUF, COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR)	6
	<type> BUF(*)	7
	INTEGER COUNT, DATATYPE, DEST, TAG, COMM, REQUEST, IERROR	8
<code>MPI_START</code>	(REQUEST, IERROR)	9
	INTEGER REQUEST, IERROR	10
<code>MPI_STARTALL</code>	(COUNT, ARRAY_OF_REQUESTS, IERROR)	11
	INTEGER COUNT, ARRAY_OF_REQUESTS(*), IERROR	12
<code>MPI_STATUS_GET_ERROR</code>	(STATUS, ERR, IERROR)	13
	INTEGER STATUS(MPI_STATUS_SIZE), ERR, IERROR	14
<code>MPI_STATUS_GET_SOURCE</code>	(STATUS, SOURCE, IERROR)	15
	INTEGER STATUS(MPI_STATUS_SIZE), SOURCE, IERROR	16
<code>MPI_STATUS_GET_TAG</code>	(STATUS, TAG, IERROR)	17
	INTEGER STATUS(MPI_STATUS_SIZE), TAG, IERROR	18
<code>MPI_TEST</code>	(REQUEST, FLAG, STATUS, IERROR)	19
	INTEGER REQUEST, STATUS(MPI_STATUS_SIZE), IERROR	20
	LOGICAL FLAG	21
<code>MPI_TEST_CANCELLED</code>	(STATUS, FLAG, IERROR)	22
	INTEGER STATUS(MPI_STATUS_SIZE), IERROR	23
	LOGICAL FLAG	24
<code>MPI_TESTALL</code>	(COUNT, ARRAY_OF_REQUESTS, FLAG, ARRAY_OF_STATUSES, IERROR)	25
	INTEGER COUNT, ARRAY_OF_REQUESTS(*), ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *),	26
	IERROR	27
	LOGICAL FLAG	28
<code>MPI_TESTANY</code>	(COUNT, ARRAY_OF_REQUESTS, INDEX, FLAG, STATUS, IERROR)	29
	INTEGER COUNT, ARRAY_OF_REQUESTS(*), INDEX, STATUS(MPI_STATUS_SIZE), IERROR	30
	LOGICAL FLAG	31
<code>MPI_TESTSOME</code>	(INCOUNT, ARRAY_OF_REQUESTS, OUTCOUNT, ARRAY_OF_INDICES,	32
	ARRAY_OF_STATUSES, IERROR)	33
	INTEGER INCOUNT, ARRAY_OF_REQUESTS(*), OUTCOUNT, ARRAY_OF_INDICES(*),	34
	ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *), IERROR	35
<code>MPI_WAIT</code>	(REQUEST, STATUS, IERROR)	36
	INTEGER REQUEST, STATUS(MPI_STATUS_SIZE), IERROR	37
<code>MPI_WAITALL</code>	(COUNT, ARRAY_OF_REQUESTS, ARRAY_OF_STATUSES, IERROR)	38
	INTEGER COUNT, ARRAY_OF_REQUESTS(*), ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *),	39
	IERROR	40
<code>MPI_WAITANY</code>	(COUNT, ARRAY_OF_REQUESTS, INDEX, STATUS, IERROR)	41
	INTEGER COUNT, ARRAY_OF_REQUESTS(*), INDEX, STATUS(MPI_STATUS_SIZE), IERROR	42

```

1  MPI_WAIT_SOME(INCOUNT, ARRAY_OF_REQUESTS, OUTCOUNT, ARRAY_OF_INDICES,
2                ARRAY_OF_STATUSES, IERROR)
3      INTEGER INCOUNT, ARRAY_OF_REQUESTS(*), OUTCOUNT, ARRAY_OF_INDICES(*),
4                ARRAY_OF_STATUSES(MPI_STATUS_SIZE, *), IERROR

```

A.5.2 Partitioned Communication Fortran Bindings

```

8  MPI_PARRIVED(REQUEST, PARTITION, FLAG, IERROR)
9      INTEGER REQUEST, PARTITION, IERROR
10     LOGICAL FLAG
11
12  MPI_PREADY(PARTITION, REQUEST, IERROR)
13     INTEGER PARTITION, REQUEST, IERROR
14
15  MPI_PREADY_LIST(LENGTH, ARRAY_OF_PARTITIONS, REQUEST, IERROR)
16     INTEGER LENGTH, ARRAY_OF_PARTITIONS(*), REQUEST, IERROR
17
18  MPI_PREADY_RANGE(PARTITION_LOW, PARTITION_HIGH, REQUEST, IERROR)
19     INTEGER PARTITION_LOW, PARTITION_HIGH, REQUEST, IERROR
20
21  MPI_PRECV_INIT(BUF, PARTITIONS, COUNT, DATATYPE, SOURCE, TAG, COMM, INFO,
22                REQUEST, IERROR)
23     <type> BUF(*)
24     INTEGER PARTITIONS, DATATYPE, SOURCE, TAG, COMM, INFO, REQUEST, IERROR
25     INTEGER(KIND=MPI_COUNT_KIND) COUNT
26
27  MPI_PSEND_INIT(BUF, PARTITIONS, COUNT, DATATYPE, DEST, TAG, COMM, INFO,
28                REQUEST, IERROR)
29     <type> BUF(*)
30     INTEGER PARTITIONS, DATATYPE, DEST, TAG, COMM, INFO, REQUEST, IERROR
31     INTEGER(KIND=MPI_COUNT_KIND) COUNT

```

A.5.3 Datatypes Fortran Bindings

```

32  INTEGER(KIND=MPI_ADDRESS_KIND) MPI_AINT_ADD(BASE, DISP)
33     INTEGER(KIND=MPI_ADDRESS_KIND) BASE, DISP
34
35  INTEGER(KIND=MPI_ADDRESS_KIND) MPI_AINT_DIFF(ADDR1, ADDR2)
36     INTEGER(KIND=MPI_ADDRESS_KIND) ADDR1, ADDR2
37
38  MPI_GET_ADDRESS(LOCATION, ADDRESS, IERROR)
39     <type> LOCATION(*)
40     INTEGER(KIND=MPI_ADDRESS_KIND) ADDRESS
41     INTEGER IERROR
42
43  MPI_GET_ELEMENTS(STATUS, DATATYPE, COUNT, IERROR)
44     INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, COUNT, IERROR
45
46  MPI_PACK(INBUF, INCOUNT, DATATYPE, OUTBUF, OUTSIZE, POSITION, COMM, IERROR)
47     <type> INBUF(*), OUTBUF(*)
48     INTEGER INCOUNT, DATATYPE, OUTSIZE, POSITION, COMM, IERROR

```

```

MPI_PACK_EXTERNAL(DATAREP, INBUF, INCOUNT, DATATYPE, OUTBUF, OUTSIZE, POSITION, 1
                  IERROR) 2
    CHARACTER*(*) DATAREP 3
    <type> INBUF(*), OUTBUF(*) 4
    INTEGER INCOUNT, DATATYPE, IERROR 5
    INTEGER(KIND=MPI_ADDRESS_KIND) OUTSIZE, POSITION 6
MPI_PACK_EXTERNAL_SIZE(DATAREP, INCOUNT, DATATYPE, SIZE, IERROR) 7
    CHARACTER*(*) DATAREP 8
    INTEGER INCOUNT, DATATYPE, IERROR 9
    INTEGER(KIND=MPI_ADDRESS_KIND) SIZE 10
MPI_PACK_SIZE(INCOUNT, DATATYPE, COMM, SIZE, IERROR) 11
    INTEGER INCOUNT, DATATYPE, COMM, SIZE, IERROR 12
MPI_TYPE_COMMIT(DATATYPE, IERROR) 13
    INTEGER DATATYPE, IERROR 14
MPI_TYPE_CONTIGUOUS(COUNT, OLDTYPE, NEWTYPE, IERROR) 15
    INTEGER COUNT, OLDTYPE, NEWTYPE, IERROR 16
MPI_TYPE_CREATE_DARRAY(SIZE, RANK, NDIMS, ARRAY_OF_GSIZES, ARRAY_OF_DISTRIBS, 17
                      ARRAY_OF_DARGS, ARRAY_OF_PSIZEs, ORDER, OLDTYPE, NEWTYPE, IERROR) 18
    INTEGER SIZE, RANK, NDIMS, ARRAY_OF_GSIZES(*), ARRAY_OF_DISTRIBS(*), 19
                      ARRAY_OF_DARGS(*), ARRAY_OF_PSIZEs(*), ORDER, OLDTYPE, NEWTYPE, 20
                      IERROR 21
MPI_TYPE_CREATE_HINDEXED(COUNT, ARRAY_OF_BLOCKLENGTHS, ARRAY_OF_DISPLACEMENTS, 22
                        OLDTYPE, NEWTYPE, IERROR) 23
    INTEGER COUNT, ARRAY_OF_BLOCKLENGTHS(*), OLDTYPE, NEWTYPE, IERROR 24
    INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_DISPLACEMENTS(*) 25
MPI_TYPE_CREATE_HINDEXED_BLOCK(COUNT, BLOCKLENGTH, ARRAY_OF_DISPLACEMENTS, 26
                              OLDTYPE, NEWTYPE, IERROR) 27
    INTEGER COUNT, BLOCKLENGTH, OLDTYPE, NEWTYPE, IERROR 28
    INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_DISPLACEMENTS(*) 29
MPI_TYPE_CREATE_HVECTOR(COUNT, BLOCKLENGTH, STRIDE, OLDTYPE, NEWTYPE, IERROR) 30
    INTEGER COUNT, BLOCKLENGTH, OLDTYPE, NEWTYPE, IERROR 31
    INTEGER(KIND=MPI_ADDRESS_KIND) STRIDE 32
MPI_TYPE_CREATE_INDEXED_BLOCK(COUNT, BLOCKLENGTH, ARRAY_OF_DISPLACEMENTS, 33
                              OLDTYPE, NEWTYPE, IERROR) 34
    INTEGER COUNT, BLOCKLENGTH, ARRAY_OF_DISPLACEMENTS(*), OLDTYPE, NEWTYPE, 35
    IERROR 36
MPI_TYPE_CREATE_RESIZED(OLDTYPE, LB, EXTENT, NEWTYPE, IERROR) 37
    INTEGER OLDTYPE, NEWTYPE, IERROR 38
    INTEGER(KIND=MPI_ADDRESS_KIND) LB, EXTENT 39
MPI_TYPE_CREATE_STRUCT(COUNT, ARRAY_OF_BLOCKLENGTHS, ARRAY_OF_DISPLACEMENTS, 40
                      ARRAY_OF_TYPES, NEWTYPE, IERROR) 41
    INTEGER COUNT, ARRAY_OF_BLOCKLENGTHS(*), ARRAY_OF_TYPES(*), NEWTYPE, IERROR 42
    INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_DISPLACEMENTS(*) 43

```

```

1  MPI_TYPE_CREATE_SUBARRAY(NDIMS, ARRAY_OF_SIZES, ARRAY_OF_SUBSIZES,
2      ARRAY_OF_STARTS, ORDER, OLDTYPE, NEWTYPE, IERROR)
3      INTEGER NDIMS, ARRAY_OF_SIZES(*), ARRAY_OF_SUBSIZES(*), ARRAY_OF_STARTS(*),
4      ORDER, OLDTYPE, NEWTYPE, IERROR
5
6  MPI_TYPE_DUP(OLDTYPE, NEWTYPE, IERROR)
7      INTEGER OLDTYPE, NEWTYPE, IERROR
8
9  MPI_TYPE_FREE(DATATYPE, IERROR)
10     INTEGER DATATYPE, IERROR
11
12 MPI_TYPE_GET_CONTENTS(DATATYPE, MAX_INTEGERS, MAX_ADDRESSES, MAX_DATATYPES,
13     ARRAY_OF_INTEGERS, ARRAY_OF_ADDRESSES, ARRAY_OF_DATATYPES,
14     IERROR)
15     INTEGER DATATYPE, MAX_INTEGERS, MAX_ADDRESSES, MAX_DATATYPES,
16     ARRAY_OF_INTEGERS(*), ARRAY_OF_DATATYPES(*), IERROR
17     INTEGER(KIND=MPI_ADDRESS_KIND) ARRAY_OF_ADDRESSES(*)
18
19 MPI_TYPE_GET_ENVELOPE(DATATYPE, NUM_INTEGERS, NUM_ADDRESSES, NUM_DATATYPES,
20     COMBINER, IERROR)
21     INTEGER DATATYPE, NUM_INTEGERS, NUM_ADDRESSES, NUM_DATATYPES, COMBINER,
22     IERROR
23
24 MPI_TYPE_GET_EXTENT(DATATYPE, LB, EXTENT, IERROR)
25     INTEGER DATATYPE, IERROR
26     INTEGER(KIND=MPI_ADDRESS_KIND) LB, EXTENT
27
28 MPI_TYPE_GET_TRUE_EXTENT(DATATYPE, TRUE_LB, TRUE_EXTENT, IERROR)
29     INTEGER DATATYPE, IERROR
30     INTEGER(KIND=MPI_ADDRESS_KIND) TRUE_LB, TRUE_EXTENT
31
32 MPI_TYPE_INDEXED(COUNT, ARRAY_OF_BLOCKLENGTHS, ARRAY_OF_DISPLACEMENTS, OLDTYPE,
33     NEWTYPE, IERROR)
34     INTEGER COUNT, ARRAY_OF_BLOCKLENGTHS(*), ARRAY_OF_DISPLACEMENTS(*),
35     OLDTYPE, NEWTYPE, IERROR
36
37 MPI_TYPE_SIZE(DATATYPE, SIZE, IERROR)
38     INTEGER DATATYPE, SIZE, IERROR
39
40 MPI_TYPE_VECTOR(COUNT, BLOCKLENGTH, STRIDE, OLDTYPE, NEWTYPE, IERROR)
41     INTEGER COUNT, BLOCKLENGTH, STRIDE, OLDTYPE, NEWTYPE, IERROR
42
43 MPI_UNPACK(INBUF, INSIZE, POSITION, OUTBUF, OUTCOUNT, DATATYPE, COMM, IERROR)
44     <type> INBUF(*), OUTBUF(*)
45     INTEGER INSIZE, POSITION, OUTCOUNT, DATATYPE, COMM, IERROR
46
47 MPI_UNPACK_EXTERNAL(DATAREP, INBUF, INSIZE, POSITION, OUTBUF, OUTCOUNT,
48     DATATYPE, IERROR)
49     CHARACTER*(*) DATAREP
50     <type> INBUF(*), OUTBUF(*)
51     INTEGER(KIND=MPI_ADDRESS_KIND) INSIZE, POSITION
52     INTEGER OUTCOUNT, DATATYPE, IERROR

```

A.5.4 Collective Communication Fortran Bindings

```

MPI_ALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, COMM,
              IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, IERROR

MPI_ALLGATHER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE,
                  COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
    IERROR

MPI_ALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNTS, DISPLS,
              RECVTYPE, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
    IERROR

MPI_ALLGATHERV_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNTS, DISPLS,
                   RECVTYPE, COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
    INFO, REQUEST, IERROR

MPI_ALLREDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, IERROR

MPI_ALLREDUCE_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, INFO, REQUEST,
                  IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR

MPI_ALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, COMM,
            IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, IERROR

MPI_ALLTOALL_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE,
                  COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
    IERROR

MPI_ALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF, REVCOUNTS,
              RDISPLS, RECVTYPE, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, REVCOUNTS(*), RDISPLS(*),
    RECVTYPE, COMM, IERROR

```

```

1  MPI_ALLTOALLV_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF, REVCOUNTS,
2      RDISPLS, RECVTYPE, COMM, INFO, REQUEST, IERROR)
3      <type> SENDBUF(*), RECVBUF(*)
4      INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, REVCOUNTS(*), RDISPLS(*),
5      RECVTYPE, COMM, INFO, REQUEST, IERROR
6
7  MPI_ALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF, REVCOUNTS,
8      RDISPLS, RECVTYPES, COMM, IERROR)
9      <type> SENDBUF(*), RECVBUF(*)
10     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPES(*), REVCOUNTS(*), RDISPLS(*),
11     RECVTYPES(*), COMM, IERROR
12
13 MPI_ALLTOALLW_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
14     REVCOUNTS, RDISPLS, RECVTYPES, COMM, INFO, REQUEST, IERROR)
15     <type> SENDBUF(*), RECVBUF(*)
16     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPES(*), REVCOUNTS(*), RDISPLS(*),
17     RECVTYPES(*), COMM, INFO, REQUEST, IERROR
18
19 MPI_BARRIER(COMM, IERROR)
20     INTEGER COMM, IERROR
21
22 MPI_BARRIER_INIT(COMM, INFO, REQUEST, IERROR)
23     INTEGER COMM, INFO, REQUEST, IERROR
24
25 MPI_BCAST(BUFFER, COUNT, DATATYPE, ROOT, COMM, IERROR)
26     <type> BUFFER(*)
27     INTEGER COUNT, DATATYPE, ROOT, COMM, IERROR
28
29 MPI_BCAST_INIT(BUFFER, COUNT, DATATYPE, ROOT, COMM, INFO, REQUEST, IERROR)
30     <type> BUFFER(*)
31     INTEGER COUNT, DATATYPE, ROOT, COMM, INFO, REQUEST, IERROR
32
33 MPI_EXSCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, IERROR)
34     <type> SENDBUF(*), RECVBUF(*)
35     INTEGER COUNT, DATATYPE, OP, COMM, IERROR
36
37 MPI_EXSCAN_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, INFO, REQUEST,
38     IERROR)
39     <type> SENDBUF(*), RECVBUF(*)
40     INTEGER COUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR
41
42 MPI_GATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, ROOT,
43     COMM, IERROR)
44     <type> SENDBUF(*), RECVBUF(*)
45     INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, IERROR
46
47 MPI_GATHER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE,
48     ROOT, COMM, INFO, REQUEST, IERROR)
49     <type> SENDBUF(*), RECVBUF(*)
50     INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, ROOT, COMM, INFO,
51     REQUEST, IERROR

```



```

MPI_GATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNTS, DISPLS,
            RECVTYPE, ROOT, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, ROOT,
        COMM, IERROR

MPI_GATHERV_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNTS, DISPLS,
                RECVTYPE, ROOT, COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, ROOT,
        COMM, INFO, REQUEST, IERROR

MPI_IALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE,
              COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, REQUEST, IERROR

MPI_IALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNTS, DISPLS,
               RECVTYPE, COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
        REQUEST, IERROR

MPI_IALLREDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, REQUEST, IERROR

MPI_IALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNT, RECVTYPE, COMM,
              REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, REVCOUNT, RECVTYPE, COMM, REQUEST, IERROR

MPI_IALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF, REVCOUNTS,
               RDISPLS, RECVTYPE, COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, REVCOUNTS(*), RDISPLS(*),
        RECVTYPE, COMM, REQUEST, IERROR

MPI_IALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF, REVCOUNTS,
               RDISPLS, RECVTYPES, COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPES(*), REVCOUNTS(*), RDISPLS(*),
        RECVTYPES(*), COMM, REQUEST, IERROR

MPI_IBARRIER(COMM, REQUEST, IERROR)
    INTEGER COMM, REQUEST, IERROR

MPI_IBCAST(BUFFER, COUNT, DATATYPE, ROOT, COMM, REQUEST, IERROR)
    <type> BUFFER(*)
    INTEGER COUNT, DATATYPE, ROOT, COMM, REQUEST, IERROR

MPI_IXSCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, REQUEST, IERROR

```

```

1  MPI_IGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE, ROOT,
2             COMM, REQUEST, IERROR)
3      <type> SENDBUF(*), RECVBUF(*)
4      INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, ROOT, COMM, REQUEST,
5             IERROR
6
7  MPI_IGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS, DISPLS,
8              RECVTYPE, ROOT, COMM, REQUEST, IERROR)
9      <type> SENDBUF(*), RECVBUF(*)
10     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, ROOT,
11           COMM, REQUEST, IERROR
12
13 MPI_IREDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, ROOT, COMM, REQUEST, IERROR)
14     <type> SENDBUF(*), RECVBUF(*)
15     INTEGER COUNT, DATATYPE, OP, ROOT, COMM, REQUEST, IERROR
16
17 MPI_IREDUCE_SCATTER(SENDBUF, RECVBUF, RECVCOUNTS, DATATYPE, OP, COMM, REQUEST,
18                     IERROR)
19     <type> SENDBUF(*), RECVBUF(*)
20     INTEGER RECVCOUNTS(*), DATATYPE, OP, COMM, REQUEST, IERROR
21
22 MPI_IREDUCE_SCATTER_BLOCK(SENDBUF, RECVBUF, RECVCOUNT, DATATYPE, OP, COMM,
23                           REQUEST, IERROR)
24     <type> SENDBUF(*), RECVBUF(*)
25     INTEGER RECVCOUNT, DATATYPE, OP, COMM, REQUEST, IERROR
26
27 MPI_ISCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, REQUEST, IERROR)
28     <type> SENDBUF(*), RECVBUF(*)
29     INTEGER COUNT, DATATYPE, OP, COMM, REQUEST, IERROR
30
31 MPI_ISCATTER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE, ROOT,
32             COMM, REQUEST, IERROR)
33     <type> SENDBUF(*), RECVBUF(*)
34     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, ROOT, COMM, REQUEST,
35           IERROR
36
37 MPI_ISCATTERV(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE, RECVBUF, RECVCOUNT,
38              RECVTYPE, ROOT, COMM, REQUEST, IERROR)
39     <type> SENDBUF(*), RECVBUF(*)
40     INTEGER SENDCOUNTS(*), DISPLS(*), SENDTYPE, RECVCOUNT, RECVTYPE, ROOT,
41           COMM, REQUEST, IERROR
42
43 MPI_OP_COMMUTATIVE(OP, COMMUTE, IERROR)
44     INTEGER OP, IERROR
45     LOGICAL COMMUTE
46
47 MPI_OP_CREATE(USER_FN, COMMUTE, OP, IERROR)
48     EXTERNAL USER_FN
49     LOGICAL COMMUTE
50     INTEGER OP, IERROR
51
52 MPI_OP_FREE(OP, IERROR)
53     INTEGER OP, IERROR

```

```

MPI_REDUCE(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, ROOT, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, ROOT, COMM, IERROR

MPI_REDUCE_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, ROOT, COMM, INFO,
    REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, ROOT, COMM, INFO, REQUEST, IERROR

MPI_REDUCE_LOCAL(INBUF, INOUTBUF, COUNT, DATATYPE, OP, IERROR)
    <type> INBUF(*), INOUTBUF(*)
    INTEGER COUNT, DATATYPE, OP, IERROR

MPI_REDUCE_SCATTER(SENDBUF, RECVBUF, RECVCOUNTS, DATATYPE, OP, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER RECVCOUNTS(*), DATATYPE, OP, COMM, IERROR

MPI_REDUCE_SCATTER_BLOCK(SENDBUF, RECVBUF, RECVCOUNT, DATATYPE, OP, COMM,
    IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER RECVCOUNT, DATATYPE, OP, COMM, IERROR

MPI_REDUCE_SCATTER_BLOCK_INIT(SENDBUF, RECVBUF, RECVCOUNT, DATATYPE, OP, COMM,
    INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER RECVCOUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR

MPI_REDUCE_SCATTER_INIT(SENDBUF, RECVBUF, RECVCOUNTS, DATATYPE, OP, COMM, INFO,
    REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER RECVCOUNTS(*), DATATYPE, OP, COMM, INFO, REQUEST, IERROR

MPI_SCAN(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, IERROR

MPI_SCAN_INIT(SENDBUF, RECVBUF, COUNT, DATATYPE, OP, COMM, INFO, REQUEST,
    IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER COUNT, DATATYPE, OP, COMM, INFO, REQUEST, IERROR

MPI_SCATTER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE, ROOT,
    COMM, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, ROOT, COMM, IERROR

MPI_SCATTER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT, RECVTYPE,
    ROOT, COMM, INFO, REQUEST, IERROR)
    <type> SENDBUF(*), RECVBUF(*)
    INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, ROOT, COMM, INFO,
    REQUEST, IERROR

```

```

1 MPI_SCATTERV(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE, RECVBUF, RECVCOUNT,
2             RECVTYPE, ROOT, COMM, IERROR)
3     <type> SENDBUF(*), RECVBUF(*)
4     INTEGER SENDCOUNTS(*), DISPLS(*), SENDTYPE, RECVCOUNT, RECVTYPE, ROOT,
5             COMM, IERROR

```

```

6 MPI_SCATTERV_INIT(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE, RECVBUF, RECVCOUNT,
7                  RECVTYPE, ROOT, COMM, INFO, REQUEST, IERROR)
8     <type> SENDBUF(*), RECVBUF(*)
9     INTEGER SENDCOUNTS(*), DISPLS(*), SENDTYPE, RECVCOUNT, RECVTYPE, ROOT,
10            COMM, INFO, REQUEST, IERROR

```

```

12 MPI_TYPE_GET_VALUE_INDEX(VALUE_TYPE, INDEX_TYPE, PAIR_TYPE, IERROR)
13     INTEGER VALUE_TYPE, INDEX_TYPE, PAIR_TYPE, IERROR

```

A.5.5 Groups, Contexts, Communicators, and Caching Fortran Bindings

```

17 MPI_COMM_COMPARE(COMM1, COMM2, RESULT, IERROR)
18     INTEGER COMM1, COMM2, RESULT, IERROR
19 MPI_COMM_CREATE(COMM, GROUP, NEWCOMM, IERROR)
20     INTEGER COMM, GROUP, NEWCOMM, IERROR
21 MPI_COMM_CREATE_FROM_GROUP(GROUP, STRINGTAG, INFO, ERRHANDLER, NEWCOMM, IERROR)
22     INTEGER GROUP, INFO, ERRHANDLER, NEWCOMM, IERROR
23     CHARACTER*(*) STRINGTAG
24 MPI_COMM_CREATE_GROUP(COMM, GROUP, TAG, NEWCOMM, IERROR)
25     INTEGER COMM, GROUP, TAG, NEWCOMM, IERROR
26 MPI_COMM_CREATE_KEYVAL(COMM_COPY_ATTR_FN, COMM_DELETE_ATTR_FN, COMM_KEYVAL,
27                        EXTRA_STATE, IERROR)
28     EXTERNAL COMM_COPY_ATTR_FN, COMM_DELETE_ATTR_FN
29     INTEGER COMM_KEYVAL, IERROR
30     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
31 MPI_COMM_DELETE_ATTR(COMM, COMM_KEYVAL, IERROR)
32     INTEGER COMM, COMM_KEYVAL, IERROR
33 MPI_COMM_DUP(COMM, NEWCOMM, IERROR)
34     INTEGER COMM, NEWCOMM, IERROR
35 MPI_COMM_DUP_FN(OLDCOMM, COMM_KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,
36                ATTRIBUTE_VAL_OUT, FLAG, IERROR)
37     INTEGER OLDCOMM, COMM_KEYVAL, IERROR
38     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
39            ATTRIBUTE_VAL_OUT
40     LOGICAL FLAG
41 MPI_COMM_DUP_WITH_INFO(COMM, INFO, NEWCOMM, IERROR)
42     INTEGER COMM, INFO, NEWCOMM, IERROR
43 MPI_COMM_FREE(COMM, IERROR)
44     INTEGER COMM, IERROR

```

<code>MPI_COMM_FREE_KEYVAL</code>	<code>(COMM_KEYVAL, IERROR)</code>	1
	INTEGER <code>COMM_KEYVAL</code> , IERROR	2
<code>MPI_COMM_GET_ATTR</code>	<code>(COMM, COMM_KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)</code>	3
	INTEGER <code>COMM</code> , <code>COMM_KEYVAL</code> , IERROR	4
	INTEGER(KIND=MPI_ADDRESS_KIND) <code>ATTRIBUTE_VAL</code>	5
	LOGICAL <code>FLAG</code>	6
		7
<code>MPI_COMM_GET_INFO</code>	<code>(COMM, INFO_USED, IERROR)</code>	8
	INTEGER <code>COMM</code> , <code>INFO_USED</code> , IERROR	9
		10
<code>MPI_COMM_GET_NAME</code>	<code>(COMM, COMM_NAME, RESULTLEN, IERROR)</code>	11
	INTEGER <code>COMM</code> , <code>RESULTLEN</code> , IERROR	12
	CHARACTER*(*) <code>COMM_NAME</code>	13
		14
<code>MPI_COMM_GROUP</code>	<code>(COMM, GROUP, IERROR)</code>	15
	INTEGER <code>COMM</code> , <code>GROUP</code> , IERROR	16
<code>MPI_COMM_IDUP</code>	<code>(COMM, NEWCOMM, REQUEST, IERROR)</code>	17
	INTEGER <code>COMM</code> , <code>NEWCOMM</code> , <code>REQUEST</code> , IERROR	18
<code>MPI_COMM_IDUP_WITH_INFO</code>	<code>(COMM, INFO, NEWCOMM, REQUEST, IERROR)</code>	19
	INTEGER <code>COMM</code> , <code>INFO</code> , <code>NEWCOMM</code> , <code>REQUEST</code> , IERROR	20
<code>MPI_COMM_NULL_COPY_FN</code>	<code>(OLDCOMM, COMM_KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,</code>	21
	<code>ATTRIBUTE_VAL_OUT, FLAG, IERROR)</code>	22
	INTEGER <code>OLDCOMM</code> , <code>COMM_KEYVAL</code> , IERROR	23
	INTEGER(KIND=MPI_ADDRESS_KIND) <code>EXTRA_STATE</code> , <code>ATTRIBUTE_VAL_IN</code> ,	24
	<code>ATTRIBUTE_VAL_OUT</code>	25
	LOGICAL <code>FLAG</code>	26
		27
<code>MPI_COMM_NULL_DELETE_FN</code>	<code>(COMM, COMM_KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERROR)</code>	28
	INTEGER <code>COMM</code> , <code>COMM_KEYVAL</code> , IERROR	29
	INTEGER(KIND=MPI_ADDRESS_KIND) <code>ATTRIBUTE_VAL</code> , <code>EXTRA_STATE</code>	30
		31
<code>MPI_COMM_RANK</code>	<code>(COMM, RANK, IERROR)</code>	32
	INTEGER <code>COMM</code> , <code>RANK</code> , IERROR	33
<code>MPI_COMM_REMOTE_GROUP</code>	<code>(COMM, GROUP, IERROR)</code>	34
	INTEGER <code>COMM</code> , <code>GROUP</code> , IERROR	35
<code>MPI_COMM_REMOTE_SIZE</code>	<code>(COMM, SIZE, IERROR)</code>	36
	INTEGER <code>COMM</code> , <code>SIZE</code> , IERROR	37
<code>MPI_COMM_SET_ATTR</code>	<code>(COMM, COMM_KEYVAL, ATTRIBUTE_VAL, IERROR)</code>	38
	INTEGER <code>COMM</code> , <code>COMM_KEYVAL</code> , IERROR	39
	INTEGER(KIND=MPI_ADDRESS_KIND) <code>ATTRIBUTE_VAL</code>	40
		41
<code>MPI_COMM_SET_INFO</code>	<code>(COMM, INFO, IERROR)</code>	42
	INTEGER <code>COMM</code> , <code>INFO</code> , IERROR	43
<code>MPI_COMM_SET_NAME</code>	<code>(COMM, COMM_NAME, IERROR)</code>	44
	INTEGER <code>COMM</code> , IERROR	45
	CHARACTER*(*) <code>COMM_NAME</code>	46
		47
<code>MPI_COMM_SIZE</code>	<code>(COMM, SIZE, IERROR)</code>	48
	INTEGER <code>COMM</code> , <code>SIZE</code> , IERROR	

```

1 MPI_COMM_SPLIT(COMM, COLOR, KEY, NEWCOMM, IERROR)
2     INTEGER COMM, COLOR, KEY, NEWCOMM, IERROR
3 MPI_COMM_SPLIT_TYPE(COMM, SPLIT_TYPE, KEY, INFO, NEWCOMM, IERROR)
4     INTEGER COMM, SPLIT_TYPE, KEY, INFO, NEWCOMM, IERROR
5 MPI_COMM_TEST_INTER(COMM, FLAG, IERROR)
6     INTEGER COMM, IERROR
7     LOGICAL FLAG
8 MPI_GROUP_COMPARE(GROUP1, GROUP2, RESULT, IERROR)
9     INTEGER GROUP1, GROUP2, RESULT, IERROR
10 MPI_GROUP_DIFFERENCE(GROUP1, GROUP2, NEWGROUP, IERROR)
11     INTEGER GROUP1, GROUP2, NEWGROUP, IERROR
12 MPI_GROUP_EXCL(GROUP, N, RANKS, NEWGROUP, IERROR)
13     INTEGER GROUP, N, RANKS(*), NEWGROUP, IERROR
14 MPI_GROUP_FREE(GROUP, IERROR)
15     INTEGER GROUP, IERROR
16 MPI_GROUP_FROM_SESSION_PSET(SESSION, PSET_NAME, NEWGROUP, IERROR)
17     INTEGER SESSION, NEWGROUP, IERROR
18     CHARACTER*(*) PSET_NAME
19 MPI_GROUP_INCL(GROUP, N, RANKS, NEWGROUP, IERROR)
20     INTEGER GROUP, N, RANKS(*), NEWGROUP, IERROR
21 MPI_GROUP_INTERSECTION(GROUP1, GROUP2, NEWGROUP, IERROR)
22     INTEGER GROUP1, GROUP2, NEWGROUP, IERROR
23 MPI_GROUP_RANGE_EXCL(GROUP, N, RANGES, NEWGROUP, IERROR)
24     INTEGER GROUP, N, RANGES(3, *), NEWGROUP, IERROR
25 MPI_GROUP_RANGE_INCL(GROUP, N, RANGES, NEWGROUP, IERROR)
26     INTEGER GROUP, N, RANGES(3, *), NEWGROUP, IERROR
27 MPI_GROUP_RANK(GROUP, RANK, IERROR)
28     INTEGER GROUP, RANK, IERROR
29 MPI_GROUP_SIZE(GROUP, SIZE, IERROR)
30     INTEGER GROUP, SIZE, IERROR
31 MPI_GROUP_TRANSLATE_RANKS(GROUP1, N, RANKS1, GROUP2, RANKS2, IERROR)
32     INTEGER GROUP1, N, RANKS1(*), GROUP2, RANKS2(*), IERROR
33 MPI_GROUP_UNION(GROUP1, GROUP2, NEWGROUP, IERROR)
34     INTEGER GROUP1, GROUP2, NEWGROUP, IERROR
35 MPI_INTERCOMM_CREATE(LOCAL_COMM, LOCAL_LEADER, PEER_COMM, REMOTE_LEADER, TAG,
36     NEWINTERCOMM, IERROR)
37     INTEGER LOCAL_COMM, LOCAL_LEADER, PEER_COMM, REMOTE_LEADER, TAG,
38     NEWINTERCOMM, IERROR
39 MPI_INTERCOMM_CREATE_FROM_GROUPS(LOCAL_GROUP, LOCAL_LEADER, REMOTE_GROUP,
40     REMOTE_LEADER, STRINGTAG, INFO, ERRHANDLER, NEWINTERCOMM, IERROR)
41     INTEGER LOCAL_GROUP, LOCAL_LEADER, REMOTE_GROUP, REMOTE_LEADER, INFO,
42     ERRHANDLER, NEWINTERCOMM, IERROR
43     CHARACTER*(*) STRINGTAG

```

```

MPI_INTERCOMM_MERGE(INTERCOMM, HIGH, NEWINTRACOMM, IERROR)
    INTEGER INTERCOMM, NEWINTRACOMM, IERROR
    LOGICAL HIGH

MPI_TYPE_CREATE_KEYVAL(TYPE_COPY_ATTR_FN, TYPE_DELETE_ATTR_FN, TYPE_KEYVAL,
    EXTRA_STATE, IERROR)
    EXTERNAL TYPE_COPY_ATTR_FN, TYPE_DELETE_ATTR_FN
    INTEGER TYPE_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE

MPI_TYPE_DELETE_ATTR(DATATYPE, TYPE_KEYVAL, IERROR)
    INTEGER DATATYPE, TYPE_KEYVAL, IERROR

MPI_TYPE_DUP_FN(OLDTYPE, TYPE_KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,
    ATTRIBUTE_VAL_OUT, FLAG, IERROR)
    INTEGER OLDTYPE, TYPE_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
    ATTRIBUTE_VAL_OUT
    LOGICAL FLAG

MPI_TYPE_FREE_KEYVAL(TYPE_KEYVAL, IERROR)
    INTEGER TYPE_KEYVAL, IERROR

MPI_TYPE_GET_ATTR(DATATYPE, TYPE_KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
    INTEGER DATATYPE, TYPE_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
    LOGICAL FLAG

MPI_TYPE_GET_NAME(DATATYPE, TYPE_NAME, RESULTLEN, IERROR)
    INTEGER DATATYPE, RESULTLEN, IERROR
    CHARACTER(*) TYPE_NAME

MPI_TYPE_NULL_COPY_FN(OLDTYPE, TYPE_KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,
    ATTRIBUTE_VAL_OUT, FLAG, IERROR)
    INTEGER OLDTYPE, TYPE_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
    ATTRIBUTE_VAL_OUT
    LOGICAL FLAG

MPI_TYPE_NULL_DELETE_FN(DATATYPE, TYPE_KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE,
    IERROR)
    INTEGER DATATYPE, TYPE_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE

MPI_TYPE_SET_ATTR(DATATYPE, TYPE_KEYVAL, ATTRIBUTE_VAL, IERROR)
    INTEGER DATATYPE, TYPE_KEYVAL, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL

MPI_TYPE_SET_NAME(DATATYPE, TYPE_NAME, IERROR)
    INTEGER DATATYPE, IERROR
    CHARACTER(*) TYPE_NAME

MPI_WIN_CREATE_KEYVAL(WIN_COPY_ATTR_FN, WIN_DELETE_ATTR_FN, WIN_KEYVAL,
    EXTRA_STATE, IERROR)
    EXTERNAL WIN_COPY_ATTR_FN, WIN_DELETE_ATTR_FN

```

```

1      INTEGER WIN_KEYVAL, IERROR
2      INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
3
4      MPI_WIN_DELETE_ATTR(WIN, WIN_KEYVAL, IERROR)
5      INTEGER WIN, WIN_KEYVAL, IERROR
6
7      MPI_WIN_DUP_FN(OLDWIN, WIN_KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,
8                    ATTRIBUTE_VAL_OUT, FLAG, IERROR)
9      INTEGER OLDWIN, WIN_KEYVAL, IERROR
10     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
11           ATTRIBUTE_VAL_OUT
12     LOGICAL FLAG
13
14     MPI_WIN_FREE_KEYVAL(WIN_KEYVAL, IERROR)
15     INTEGER WIN_KEYVAL, IERROR
16
17     MPI_WIN_GET_ATTR(WIN, WIN_KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
18     INTEGER WIN, WIN_KEYVAL, IERROR
19     INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
20     LOGICAL FLAG
21
22     MPI_WIN_GET_NAME(WIN, WIN_NAME, RESULTLEN, IERROR)
23     INTEGER WIN, RESULTLEN, IERROR
24     CHARACTER*(*) WIN_NAME
25
26     MPI_WIN_NULL_COPY_FN(OLDWIN, WIN_KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,
27                          ATTRIBUTE_VAL_OUT, FLAG, IERROR)
28     INTEGER OLDWIN, WIN_KEYVAL, IERROR
29     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE, ATTRIBUTE_VAL_IN,
30           ATTRIBUTE_VAL_OUT
31     LOGICAL FLAG
32
33     MPI_WIN_NULL_DELETE_FN(WIN, WIN_KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERROR)
34     INTEGER WIN, WIN_KEYVAL, IERROR
35     INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL, EXTRA_STATE
36
37     MPI_WIN_SET_ATTR(WIN, WIN_KEYVAL, ATTRIBUTE_VAL, IERROR)
38     INTEGER WIN, WIN_KEYVAL, IERROR
39     INTEGER(KIND=MPI_ADDRESS_KIND) ATTRIBUTE_VAL
40
41     MPI_WIN_SET_NAME(WIN, WIN_NAME, IERROR)
42     INTEGER WIN, IERROR
43     CHARACTER*(*) WIN_NAME
44
45
46
47
48

```

A.5.6 Virtual Topologies for MPI Processes Fortran Bindings

```

42     MPI_CART_COORDS(COMM, RANK, MAXDIMS, COORDS, IERROR)
43     INTEGER COMM, RANK, MAXDIMS, COORDS(*), IERROR
44
45     MPI_CART_CREATE(COMM_OLD, NDIMS, DIMS, PERIODS, REORDER, COMM_CART, IERROR)
46     INTEGER COMM_OLD, NDIMS, DIMS(*), COMM_CART, IERROR
47     LOGICAL PERIODS(*), REORDER
48

```


<code>MPI_CART_GET</code>	(COMM, MAXDIMS, DIMS, PERIODS, COORDS, IERROR)	1
	INTEGER COMM, MAXDIMS, DIMS(*), COORDS(*), IERROR	2
	LOGICAL PERIODS(*)	3
<code>MPI_CART_MAP</code>	(COMM, NDIMS, DIMS, PERIODS, NEWRANK, IERROR)	4
	INTEGER COMM, NDIMS, DIMS(*), NEWRANK, IERROR	5
	LOGICAL PERIODS(*)	6
<code>MPI_CART_RANK</code>	(COMM, COORDS, RANK, IERROR)	7
	INTEGER COMM, COORDS(*), RANK, IERROR	8
<code>MPI_CART_SHIFT</code>	(COMM, DIRECTION, DISP, RANK_SOURCE, RANK_DEST, IERROR)	9
	INTEGER COMM, DIRECTION, DISP, RANK_SOURCE, RANK_DEST, IERROR	10
<code>MPI_CART_SUB</code>	(COMM, REMAIN_DIMS, NEWCOMM, IERROR)	11
	INTEGER COMM, NEWCOMM, IERROR	12
	LOGICAL REMAIN_DIMS(*)	13
<code>MPI_CARTDIM_GET</code>	(COMM, NDIMS, IERROR)	14
	INTEGER COMM, NDIMS, IERROR	15
<code>MPI_DIMS_CREATE</code>	(NNODES, NDIMS, DIMS, IERROR)	16
	INTEGER NNODES, NDIMS, DIMS(*), IERROR	17
<code>MPI_DIST_GRAPH_CREATE</code>	(COMM_OLD, N, SOURCES, DEGREES, DESTINATIONS, WEIGHTS, INFO, REORDER, COMM_DIST_GRAPH, IERROR)	18
	INTEGER COMM_OLD, N, SOURCES(*), DEGREES(*), DESTINATIONS(*), WEIGHTS(*), INFO, COMM_DIST_GRAPH, IERROR	19
	LOGICAL REORDER	20
<code>MPI_DIST_GRAPH_CREATE_ADJACENT</code>	(COMM_OLD, INDEGREE, SOURCES, SOURCEWEIGHTS, OUTDEGREE, DESTINATIONS, DESTWEIGHTS, INFO, REORDER, COMM_DIST_GRAPH, IERROR)	21
	INTEGER COMM_OLD, INDEGREE, SOURCES(*), SOURCEWEIGHTS(*), OUTDEGREE, DESTINATIONS(*), DESTWEIGHTS(*), INFO, COMM_DIST_GRAPH, IERROR	22
	LOGICAL REORDER	23
<code>MPI_DIST_GRAPH_NEIGHBORS</code>	(COMM, MAXINDEGREE, SOURCES, SOURCEWEIGHTS, MAXOUTDEGREE, DESTINATIONS, DESTWEIGHTS, IERROR)	24
	INTEGER COMM, MAXINDEGREE, SOURCES(*), SOURCEWEIGHTS(*), MAXOUTDEGREE, DESTINATIONS(*), DESTWEIGHTS(*), IERROR	25
<code>MPI_DIST_GRAPH_NEIGHBORS_COUNT</code>	(COMM, INDEGREE, OUTDEGREE, WEIGHTED, IERROR)	26
	INTEGER COMM, INDEGREE, OUTDEGREE, IERROR	27
	LOGICAL WEIGHTED	28
<code>MPI_GRAPH_CREATE</code>	(COMM_OLD, NNODES, INDEX, EDGES, REORDER, COMM_GRAPH, IERROR)	29
	INTEGER COMM_OLD, NNODES, INDEX(*), EDGES(*), COMM_GRAPH, IERROR	30
	LOGICAL REORDER	31
<code>MPI_GRAPH_GET</code>	(COMM, MAXINDEX, MAXEDGES, INDEX, EDGES, IERROR)	32
	INTEGER COMM, MAXINDEX, MAXEDGES, INDEX(*), EDGES(*), IERROR	33
<code>MPI_GRAPH_MAP</code>	(COMM, NNODES, INDEX, EDGES, NEWRANK, IERROR)	34
	INTEGER COMM, NNODES, INDEX(*), EDGES(*), NEWRANK, IERROR	35

```

1  MPI_GRAPH_NEIGHBORS(COMM, RANK, MAXNEIGHBORS, NEIGHBORS, IERROR)
2      INTEGER COMM, RANK, MAXNEIGHBORS, NEIGHBORS(*), IERROR
3
4  MPI_GRAPH_NEIGHBORS_COUNT(COMM, RANK, NNEIGHBORS, IERROR)
5      INTEGER COMM, RANK, NNEIGHBORS, IERROR
6
7  MPI_GRAPHDIMS_GET(COMM, NNODES, NEDGES, IERROR)
8      INTEGER COMM, NNODES, NEDGES, IERROR
9
10 MPI_INEIGHBOR_ALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
11     RECVTYPE, COMM, REQUEST, IERROR)
12     <type> SENDBUF(*), RECVBUF(*)
13     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, REQUEST, IERROR
14
15 MPI_INEIGHBOR_ALLGATHERV(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, REVCOUNTS,
16     DISPLS, RECVTYPE, COMM, REQUEST, IERROR)
17     <type> SENDBUF(*), RECVBUF(*)
18     INTEGER SENDCOUNT, SENDTYPE, REVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
19     REQUEST, IERROR
20
21 MPI_INEIGHBOR_ALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
22     RECVTYPE, COMM, REQUEST, IERROR)
23     <type> SENDBUF(*), RECVBUF(*)
24     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, REQUEST, IERROR
25
26 MPI_INEIGHBOR_ALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF,
27     REVCOUNTS, RDISPLS, RECVTYPE, COMM, REQUEST, IERROR)
28     <type> SENDBUF(*), RECVBUF(*)
29     INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, REVCOUNTS(*), RDISPLS(*),
30     RECVTYPE, COMM, REQUEST, IERROR
31
32 MPI_INEIGHBOR_ALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
33     REVCOUNTS, RDISPLS, RECVTYPES, COMM, REQUEST, IERROR)
34     <type> SENDBUF(*), RECVBUF(*)
35     INTEGER SENDCOUNTS(*), SENDTYPES(*), REVCOUNTS(*), RECVTYPES(*), COMM,
36     REQUEST, IERROR
37     INTEGER(KIND=MPI_ADDRESS_KIND) SDISPLS(*), RDISPLS(*)
38
39 MPI_NEIGHBOR_ALLGATHER(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
40     RECVTYPE, COMM, IERROR)
41     <type> SENDBUF(*), RECVBUF(*)
42     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, IERROR
43
44 MPI_NEIGHBOR_ALLGATHER_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
45     RECVTYPE, COMM, INFO, REQUEST, IERROR)
46     <type> SENDBUF(*), RECVBUF(*)
47     INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
48     IERROR

```

```

MPI_NEIGHBOR_ALLGATHERV_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNTS,
    DISPLS, RECVTYPE, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*), RECVTYPE, COMM,
    INFO, REQUEST, IERROR
MPI_NEIGHBOR_ALLTOALL(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
    RECVTYPE, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, IERROR
MPI_NEIGHBOR_ALLTOALL_INIT(SENDBUF, SENDCOUNT, SENDTYPE, RECVBUF, RECVCOUNT,
    RECVTYPE, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNT, SENDTYPE, RECVCOUNT, RECVTYPE, COMM, INFO, REQUEST,
    IERROR
MPI_NEIGHBOR_ALLTOALLV(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF,
    RECVCOUNTS, RDISPLS, RECVTYPE, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, RECVCOUNTS(*), RDISPLS(*),
    RECVTYPE, COMM, IERROR
MPI_NEIGHBOR_ALLTOALLV_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPE, RECVBUF,
    RECVCOUNTS, RDISPLS, RECVTYPE, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SDISPLS(*), SENDTYPE, RECVCOUNTS(*), RDISPLS(*),
    RECVTYPE, COMM, INFO, REQUEST, IERROR
MPI_NEIGHBOR_ALLTOALLW(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
    RECVCOUNTS, RDISPLS, RECVTYPES, COMM, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SENDTYPES(*), RECVCOUNTS(*), RECVTYPES(*), COMM,
    IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) SDISPLS(*), RDISPLS(*)
MPI_NEIGHBOR_ALLTOALLW_INIT(SENDBUF, SENDCOUNTS, SDISPLS, SENDTYPES, RECVBUF,
    RECVCOUNTS, RDISPLS, RECVTYPES, COMM, INFO, REQUEST, IERROR)
<type> SENDBUF(*), RECVBUF(*)
INTEGER SENDCOUNTS(*), SENDTYPES(*), RECVCOUNTS(*), RECVTYPES(*), COMM,
    INFO, REQUEST, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) SDISPLS(*), RDISPLS(*)
MPI_TOPO_TEST(COMM, STATUS, IERROR)
INTEGER COMM, STATUS, IERROR

```

A.5.7 MPI Environmental Management Fortran Bindings

```

MPI_ADD_ERROR_CLASS(ERRORCLASS, IERROR)
INTEGER ERRORCLASS, IERROR
MPI_ADD_ERROR_CODE(ERRORCLASS, ERRORCODE, IERROR)
INTEGER ERRORCLASS, ERRORCODE, IERROR

```

```

1  MPI_ADD_ERROR_STRING(ERRORCODE, STRING, IERROR)
2      INTEGER ERRORCODE, IERROR
3      CHARACTER*(*) STRING
4
5  MPI_ALLOC_MEM(SIZE, INFO, BASEPTR, IERROR)
6      INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR
7      INTEGER INFO, IERROR
8
9  If the Fortran compiler provides TYPE(C_PTR), then the above is overloaded by:
10
11  INTERFACE MPI_ALLOC_MEM
12      SUBROUTINE MPI_ALLOC_MEM(SIZE, INFO, BASEPTR, IERROR)
13          IMPORT :: MPI_ADDRESS_KIND
14          INTEGER :: INFO, IERROR
15          INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
16      END SUBROUTINE
17
18  SUBROUTINE MPI_ALLOC_MEM_CPTR(SIZE, INFO, BASEPTR, IERROR)
19      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
20      IMPORT :: MPI_ADDRESS_KIND
21      INTEGER :: INFO, IERROR
22      INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
23      TYPE(C_PTR) :: BASEPTR
24  END SUBROUTINE
25
26  END INTERFACE
27
28  MPI_COMM_CALL_ERRHANDLER(COMM, ERRORCODE, IERROR)
29      INTEGER COMM, ERRORCODE, IERROR
30
31  MPI_COMM_CREATE_ERRHANDLER(COMM, ERRHANDLER_FN, ERRHANDLER, IERROR)
32      EXTERNAL COMM_ERRHANDLER_FN
33      INTEGER ERRHANDLER, IERROR
34
35  MPI_COMM_GET_ERRHANDLER(COMM, ERRHANDLER, IERROR)
36      INTEGER COMM, ERRHANDLER, IERROR
37
38  MPI_COMM_SET_ERRHANDLER(COMM, ERRHANDLER, IERROR)
39      INTEGER COMM, ERRHANDLER, IERROR
40
41  MPI_ERRHANDLER_FREE(ERRHANDLER, IERROR)
42      INTEGER ERRHANDLER, IERROR
43
44  MPI_ERROR_CLASS(ERRORCODE, ERRORCLASS, IERROR)
45      INTEGER ERRORCODE, ERRORCLASS, IERROR
46
47  MPI_ERROR_STRING(ERRORCODE, STRING, RESULTLEN, IERROR)
48      INTEGER ERRORCODE, RESULTLEN, IERROR
49      CHARACTER*(*) STRING
50
51  MPI_FILE_CALL_ERRHANDLER(FH, ERRORCODE, IERROR)
52      INTEGER FH, ERRORCODE, IERROR
53
54  MPI_FILE_CREATE_ERRHANDLER(FILE_ERRHANDLER_FN, ERRHANDLER, IERROR)
55      EXTERNAL FILE_ERRHANDLER_FN
56      INTEGER ERRHANDLER, IERROR

```

<code>MPI_FILE_GET_ERRHANDLER(FILE, ERRHANDLER, IERROR)</code>	1
INTEGER FILE, ERRHANDLER, IERROR	2
<code>MPI_FILE_SET_ERRHANDLER(FILE, ERRHANDLER, IERROR)</code>	3
INTEGER FILE, ERRHANDLER, IERROR	4
<code>MPI_FREE_MEM(BASE, IERROR)</code>	5
<type> BASE(*)	6
INTEGER IERROR	7
<code>MPI_GET_HW_RESOURCE_INFO(HW_INFO, IERROR)</code>	8
INTEGER HW_INFO, IERROR	9
<code>MPI_GET_LIBRARY_VERSION(VERSION, RESULTLEN, IERROR)</code>	10
CHARACTER*(*) VERSION	11
INTEGER RESULTLEN, IERROR	12
<code>MPI_GET_PROCESSOR_NAME(NAME, RESULTLEN, IERROR)</code>	13
CHARACTER*(*) NAME	14
INTEGER RESULTLEN, IERROR	15
<code>MPI_GET_VERSION(VERSION, SUBVERSION, IERROR)</code>	16
INTEGER VERSION, SUBVERSION, IERROR	17
<code>MPI_REMOVE_ERROR_CLASS(ERRORCLASS, IERROR)</code>	18
INTEGER ERRORCLASS, IERROR	19
<code>MPI_REMOVE_ERROR_CODE(ERRORCODE, IERROR)</code>	20
INTEGER ERRORCODE, IERROR	21
<code>MPI_REMOVE_ERROR_STRING(ERRORCODE, IERROR)</code>	22
INTEGER ERRORCODE, IERROR	23
<code>MPI_SESSION_CALL_ERRHANDLER(SESSION, ERRORCODE, IERROR)</code>	24
INTEGER SESSION, ERRORCODE, IERROR	25
<code>MPI_SESSION_CREATE_ERRHANDLER(SESSION_ERRHANDLER_FN, ERRHANDLER, IERROR)</code>	26
EXTERNAL SESSION_ERRHANDLER_FN	27
INTEGER ERRHANDLER, IERROR	28
<code>MPI_SESSION_GET_ERRHANDLER(SESSION, ERRHANDLER, IERROR)</code>	29
INTEGER SESSION, ERRHANDLER, IERROR	30
<code>MPI_SESSION_SET_ERRHANDLER(SESSION, ERRHANDLER, IERROR)</code>	31
INTEGER SESSION, ERRHANDLER, IERROR	32
<code>MPI_WIN_CALL_ERRHANDLER(WIN, ERRORCODE, IERROR)</code>	33
INTEGER WIN, ERRORCODE, IERROR	34
<code>MPI_WIN_CREATE_ERRHANDLER(WIN_ERRHANDLER_FN, ERRHANDLER, IERROR)</code>	35
EXTERNAL WIN_ERRHANDLER_FN	36
INTEGER ERRHANDLER, IERROR	37
<code>MPI_WIN_GET_ERRHANDLER(WIN, ERRHANDLER, IERROR)</code>	38
INTEGER WIN, ERRHANDLER, IERROR	39
<code>MPI_WIN_SET_ERRHANDLER(WIN, ERRHANDLER, IERROR)</code>	40
INTEGER WIN, ERRHANDLER, IERROR	41
	42
	43
	44
	45
	46
	47
	48

1 DOUBLE PRECISION MPI_WTICK()

2 DOUBLE PRECISION MPI_WTIME()

5 A.5.8 The Info Object Fortran Bindings

6 MPI_INFO_CREATE(INFO, IERROR)

7 INTEGER INFO, IERROR

9 MPI_INFO_CREATE_ENV(INFO, IERROR)

10 INTEGER INFO, IERROR

11 MPI_INFO_DELETE(INFO, KEY, IERROR)

12 INTEGER INFO, IERROR

13 CHARACTER*(*) KEY

15 MPI_INFO_DUP(INFO, NEWINFO, IERROR)

16 INTEGER INFO, NEWINFO, IERROR

17 MPI_INFO_FREE(INFO, IERROR)

18 INTEGER INFO, IERROR

20 MPI_INFO_GET_NKEYS(INFO, NKEYS, IERROR)

21 INTEGER INFO, NKEYS, IERROR

22 MPI_INFO_GET_NTHKEY(INFO, N, KEY, IERROR)

23 INTEGER INFO, N, IERROR

24 CHARACTER*(*) KEY

26 MPI_INFO_GET_STRING(INFO, KEY, BUFLen, VALUE, FLAG, IERROR)

27 INTEGER INFO, BUFLen, IERROR

28 CHARACTER*(*) KEY, VALUE

29 LOGICAL FLAG

30 MPI_INFO_SET(INFO, KEY, VALUE, IERROR)

31 INTEGER INFO, IERROR

32 CHARACTER*(*) KEY, VALUE

35 A.5.9 Process Creation and Management Fortran Bindings

36 MPI_ABORT(COMM, ERRORCODE, IERROR)

37 INTEGER COMM, ERRORCODE, IERROR

39 MPI_CLOSE_PORT(PORT_NAME, IERROR)

40 CHARACTER*(*) PORT_NAME

41 INTEGER IERROR

42 MPI_COMM_ACCEPT(PORT_NAME, INFO, ROOT, COMM, NEWCOMM, IERROR)

43 CHARACTER*(*) PORT_NAME

44 INTEGER INFO, ROOT, COMM, NEWCOMM, IERROR

46 MPI_COMM_CONNECT(PORT_NAME, INFO, ROOT, COMM, NEWCOMM, IERROR)

47 CHARACTER*(*) PORT_NAME

48 INTEGER INFO, ROOT, COMM, NEWCOMM, IERROR

<code>MPI_COMM_DISCONNECT</code>	(<code>COMM</code> , <code>IERROR</code>)	1
	INTEGER <code>COMM</code> , <code>IERROR</code>	2
<code>MPI_COMM_GET_PARENT</code>	(<code>PARENT</code> , <code>IERROR</code>)	3
	INTEGER <code>PARENT</code> , <code>IERROR</code>	4
<code>MPI_COMM_JOIN</code>	(<code>FD</code> , <code>INTERCOMM</code> , <code>IERROR</code>)	5
	INTEGER <code>FD</code> , <code>INTERCOMM</code> , <code>IERROR</code>	6
<code>MPI_COMM_SPAWN</code>	(<code>COMMAND</code> , <code>ARGV</code> , <code>MAXPROCS</code> , <code>INFO</code> , <code>ROOT</code> , <code>COMM</code> , <code>INTERCOMM</code> , ARRAY_OF_ERRCODES, <code>IERROR</code>)	7
	CHARACTER*(*) <code>COMMAND</code> , <code>ARGV</code> (*)	8
	INTEGER <code>MAXPROCS</code> , <code>INFO</code> , <code>ROOT</code> , <code>COMM</code> , <code>INTERCOMM</code> , ARRAY_OF_ERRCODES(*), <code>IERROR</code>	9
<code>MPI_COMM_SPAWN_MULTIPLE</code>	(<code>COUNT</code> , ARRAY_OF_COMMANDS, ARRAY_OF_ARGV, ARRAY_OF_MAXPROCS, ARRAY_OF_INFO, <code>ROOT</code> , <code>COMM</code> , <code>INTERCOMM</code> , ARRAY_OF_ERRCODES, <code>IERROR</code>)	10
	INTEGER <code>COUNT</code> , ARRAY_OF_MAXPROCS(*), ARRAY_OF_INFO(*), <code>ROOT</code> , <code>COMM</code> , INTERCOMM, ARRAY_OF_ERRCODES(*), <code>IERROR</code>	11
	CHARACTER*(*) ARRAY_OF_COMMANDS(*), ARRAY_OF_ARGV(<code>COUNT</code> , *)	12
<code>MPI_FINALIZE</code>	(<code>IERROR</code>)	13
	INTEGER <code>IERROR</code>	14
<code>MPI_FINALIZED</code>	(<code>FLAG</code> , <code>IERROR</code>)	15
	LOGICAL <code>FLAG</code>	16
	INTEGER <code>IERROR</code>	17
<code>MPI_INIT</code>	(<code>IERROR</code>)	18
	INTEGER <code>IERROR</code>	19
<code>MPI_INIT_THREAD</code>	(<code>REQUIRED</code> , <code>PROVIDED</code> , <code>IERROR</code>)	20
	INTEGER <code>REQUIRED</code> , <code>PROVIDED</code> , <code>IERROR</code>	21
<code>MPI_INITIALIZED</code>	(<code>FLAG</code> , <code>IERROR</code>)	22
	LOGICAL <code>FLAG</code>	23
	INTEGER <code>IERROR</code>	24
<code>MPI_IS_THREAD_MAIN</code>	(<code>FLAG</code> , <code>IERROR</code>)	25
	LOGICAL <code>FLAG</code>	26
	INTEGER <code>IERROR</code>	27
<code>MPI_LOOKUP_NAME</code>	(<code>SERVICE_NAME</code> , <code>INFO</code> , <code>PORT_NAME</code> , <code>IERROR</code>)	28
	CHARACTER*(*) <code>SERVICE_NAME</code> , <code>PORT_NAME</code>	29
	INTEGER <code>INFO</code> , <code>IERROR</code>	30
<code>MPI_OPEN_PORT</code>	(<code>INFO</code> , <code>PORT_NAME</code> , <code>IERROR</code>)	31
	INTEGER <code>INFO</code> , <code>IERROR</code>	32
	CHARACTER*(*) <code>PORT_NAME</code>	33
<code>MPI_PUBLISH_NAME</code>	(<code>SERVICE_NAME</code> , <code>INFO</code> , <code>PORT_NAME</code> , <code>IERROR</code>)	34
	CHARACTER*(*) <code>SERVICE_NAME</code> , <code>PORT_NAME</code>	35
	INTEGER <code>INFO</code> , <code>IERROR</code>	36
<code>MPI_QUERY_THREAD</code>	(<code>PROVIDED</code> , <code>IERROR</code>)	37
	INTEGER <code>PROVIDED</code> , <code>IERROR</code>	38

```

1  MPI_SESSION_FINALIZE(SESSION, IERROR)
2      INTEGER SESSION, IERROR
3
4  MPI_SESSION_GET_INFO(SESSION, INFO_USED, IERROR)
5      INTEGER SESSION, INFO_USED, IERROR
6
7  MPI_SESSION_GET_NTH_PSET(SESSION, INFO, N, PSET_LEN, PSET_NAME, IERROR)
8      INTEGER SESSION, INFO, N, PSET_LEN, IERROR
9      CHARACTER*(*) PSET_NAME
10
11 MPI_SESSION_GET_NUM_PSETS(SESSION, INFO, NPSET_NAMES, IERROR)
12     INTEGER SESSION, INFO, NPSET_NAMES, IERROR
13
14 MPI_SESSION_GET_PSET_INFO(SESSION, PSET_NAME, INFO, IERROR)
15     INTEGER SESSION, INFO, IERROR
16     CHARACTER*(*) PSET_NAME
17
18 MPI_SESSION_INIT(INFO, ERRHANDLER, SESSION, IERROR)
19     INTEGER INFO, ERRHANDLER, SESSION, IERROR
20
21 MPI_UNPUBLISH_NAME(SERVICE_NAME, INFO, PORT_NAME, IERROR)
22     CHARACTER*(*) SERVICE_NAME, PORT_NAME
23     INTEGER INFO, IERROR
24
25 A.5.10 One-Sided Communications Fortran Bindings
26
27 MPI_ACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK,
28     TARGET_DISP, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, IERROR)
29     <type> ORIGIN_ADDR(*)
30     INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
31     TARGET_DATATYPE, OP, WIN, IERROR
32     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP
33
34 MPI_COMPARE_AND_SWAP(ORIGIN_ADDR, COMPARE_ADDR, RESULT_ADDR, DATATYPE,
35     TARGET_RANK, TARGET_DISP, WIN, IERROR)
36     <type> ORIGIN_ADDR(*), COMPARE_ADDR(*), RESULT_ADDR(*)
37     INTEGER DATATYPE, TARGET_RANK, WIN, IERROR
38     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP
39
40 MPI_FETCH_AND_OP(ORIGIN_ADDR, RESULT_ADDR, DATATYPE, TARGET_RANK, TARGET_DISP,
41     OP, WIN, IERROR)
42     <type> ORIGIN_ADDR(*), RESULT_ADDR(*)
43     INTEGER DATATYPE, TARGET_RANK, OP, WIN, IERROR
44     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP
45
46 MPI_GET(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
47     TARGET_COUNT, TARGET_DATATYPE, WIN, IERROR)
48     <type> ORIGIN_ADDR(*)
49     INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
50     TARGET_DATATYPE, WIN, IERROR
51     INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

```



```

MPI_GET_ACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_ADDR,
                  RESULT_COUNT, RESULT_DATATYPE, TARGET_RANK, TARGET_DISP,
                  TARGET_COUNT, TARGET_DATATYPE, OP, WIN, IERROR)
<type> ORIGIN_ADDR(*), RESULT_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_COUNT, RESULT_DATATYPE,
      TARGET_RANK, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

MPI_PUT(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
      TARGET_COUNT, TARGET_DATATYPE, WIN, IERROR)
<type> ORIGIN_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
      TARGET_DATATYPE, WIN, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

MPI_RACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK,
      TARGET_DISP, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, REQUEST,
      IERROR)
<type> ORIGIN_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
      TARGET_DATATYPE, OP, WIN, REQUEST, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

MPI_RGET(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
      TARGET_COUNT, TARGET_DATATYPE, WIN, REQUEST, IERROR)
<type> ORIGIN_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
      TARGET_DATATYPE, WIN, REQUEST, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

MPI_RGET_ACCUMULATE(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_ADDR,
      RESULT_COUNT, RESULT_DATATYPE, TARGET_RANK, TARGET_DISP,
      TARGET_COUNT, TARGET_DATATYPE, OP, WIN, REQUEST, IERROR)
<type> ORIGIN_ADDR(*), RESULT_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, RESULT_COUNT, RESULT_DATATYPE,
      TARGET_RANK, TARGET_COUNT, TARGET_DATATYPE, OP, WIN, REQUEST,
      IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

MPI_RPUT(ORIGIN_ADDR, ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_DISP,
      TARGET_COUNT, TARGET_DATATYPE, WIN, REQUEST, IERROR)
<type> ORIGIN_ADDR(*)
INTEGER ORIGIN_COUNT, ORIGIN_DATATYPE, TARGET_RANK, TARGET_COUNT,
      TARGET_DATATYPE, WIN, REQUEST, IERROR
INTEGER(KIND=MPI_ADDRESS_KIND) TARGET_DISP

MPI_WIN_ALLOCATE(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, WIN, IERROR)
INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR
INTEGER DISP_UNIT, INFO, COMM, WIN, IERROR

If the Fortran compiler provides TYPE(C_PTR), then the above is overloaded by:
INTERFACE MPI_WIN_ALLOCATE

```

```

1      SUBROUTINE MPI_WIN_ALLOCATE(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, &
2          WIN, IERROR)
3          IMPORT :: MPI_ADDRESS_KIND
4          INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
5          INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
6      END SUBROUTINE
7      SUBROUTINE MPI_WIN_ALLOCATE_CPTR(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, &
8          WIN, IERROR)
9          USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
10         IMPORT :: MPI_ADDRESS_KIND
11         INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
12         INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
13         TYPE(C_PTR) :: BASEPTR
14     END SUBROUTINE
15 END INTERFACE
16
17 MPI_WIN_ALLOCATE_SHARED(SIZE, DISP_UNIT, INFO, COMM, BASEPTR, WIN, IERROR)
18     INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR
19     INTEGER DISP_UNIT, INFO, COMM, WIN, IERROR
20
21 If the Fortran compiler provides TYPE(C_PTR), then the above is overloaded by:
22
23 INTERFACE MPI_WIN_ALLOCATE_SHARED
24     SUBROUTINE MPI_WIN_ALLOCATE_SHARED(SIZE, DISP_UNIT, INFO, COMM, &
25         BASEPTR, WIN, IERROR)
26         IMPORT :: MPI_ADDRESS_KIND
27         INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
28         INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
29     END SUBROUTINE
30     SUBROUTINE MPI_WIN_ALLOCATE_SHARED_CPTR(SIZE, DISP_UNIT, INFO, COMM, &
31         BASEPTR, WIN, IERROR)
32         USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
33         IMPORT :: MPI_ADDRESS_KIND
34         INTEGER :: DISP_UNIT, INFO, COMM, WIN, IERROR
35         INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
36         TYPE(C_PTR) :: BASEPTR
37     END SUBROUTINE
38 END INTERFACE
39
40 MPI_WIN_ATTACH(WIN, BASE, SIZE, IERROR)
41     INTEGER WIN, IERROR
42     <type> BASE(*)
43     INTEGER(KIND=MPI_ADDRESS_KIND) SIZE
44
45 MPI_WIN_COMPLETE(WIN, IERROR)
46     INTEGER WIN, IERROR
47
48 MPI_WIN_CREATE(BASE, SIZE, DISP_UNIT, INFO, COMM, WIN, IERROR)
49     <type> BASE(*)
50     INTEGER(KIND=MPI_ADDRESS_KIND) SIZE
51     INTEGER DISP_UNIT, INFO, COMM, WIN, IERROR

```

```

MPI_WIN_CREATE_DYNAMIC(INFO, COMM, WIN, IERROR)           1
    INTEGER INFO, COMM, WIN, IERROR                       2
                                                            3
MPI_WIN_DETACH(WIN, BASE, IERROR)                         4
    INTEGER WIN, IERROR                                   5
    <type> BASE(*)                                       6
                                                            7
MPI_WIN_FENCE(ASSERT, WIN, IERROR)                       8
    INTEGER ASSERT, WIN, IERROR                          9
                                                            10
MPI_WIN_FLUSH(RANK, WIN, IERROR)                         11
    INTEGER RANK, WIN, IERROR                            12
                                                            13
MPI_WIN_FLUSH_ALL(WIN, IERROR)                           14
    INTEGER WIN, IERROR                                  15
                                                            16
MPI_WIN_FLUSH_LOCAL(RANK, WIN, IERROR)                   17
    INTEGER RANK, WIN, IERROR                            18
                                                            19
MPI_WIN_FLUSH_LOCAL_ALL(WIN, IERROR)                    20
    INTEGER WIN, IERROR                                  21
                                                            22
MPI_WIN_FREE(WIN, IERROR)                                23
    INTEGER WIN, IERROR                                  24
                                                            25
MPI_WIN_GET_GROUP(WIN, GROUP, IERROR)                   26
    INTEGER WIN, GROUP, IERROR                           27
                                                            28
MPI_WIN_GET_INFO(WIN, INFO_USED, IERROR)                29
    INTEGER WIN, INFO_USED, IERROR                      30
                                                            31
MPI_WIN_LOCK(LOCK_TYPE, RANK, ASSERT, WIN, IERROR)      32
    INTEGER LOCK_TYPE, RANK, ASSERT, WIN, IERROR        33
                                                            34
MPI_WIN_LOCK_ALL(ASSERT, WIN, IERROR)                   35
    INTEGER ASSERT, WIN, IERROR                         36
                                                            37
MPI_WIN_POST(GROUP, ASSERT, WIN, IERROR)                38
    INTEGER GROUP, ASSERT, WIN, IERROR                  39
                                                            40
MPI_WIN_SET_INFO(WIN, INFO, IERROR)                    41
    INTEGER WIN, INFO, IERROR                          42
                                                            43
MPI_WIN_SHARED_QUERY(WIN, RANK, SIZE, DISP_UNIT, BASEPTR, IERROR) 44
    INTEGER WIN, RANK, DISP_UNIT, IERROR                45
    INTEGER(KIND=MPI_ADDRESS_KIND) SIZE, BASEPTR        46
                                                            47
If the Fortran compiler provides TYPE(C_PTR), then the above is overloaded by: 48
    INTERFACE MPI_WIN_SHARED_QUERY
    SUBROUTINE MPI_WIN_SHARED_QUERY(WIN, RANK, SIZE, DISP_UNIT, &
        BASEPTR, IERROR)
    IMPORT :: MPI_ADDRESS_KIND
    INTEGER :: WIN, RANK, DISP_UNIT, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE, BASEPTR
    END SUBROUTINE
    SUBROUTINE MPI_WIN_SHARED_QUERY_CPTR(WIN, RANK, SIZE, DISP_UNIT, &
        BASEPTR, IERROR)

```

```

1      USE, INTRINSIC :: ISO_C_BINDING, ONLY : C_PTR
2      IMPORT :: MPI_ADDRESS_KIND
3      INTEGER :: WIN, RANK, DISP_UNIT, IERROR
4      INTEGER(KIND=MPI_ADDRESS_KIND) :: SIZE
5      TYPE(C_PTR) :: BASEPTR
6      END SUBROUTINE
7      END INTERFACE
8
9      MPI_WIN_START(GROUP, ASSERT, WIN, IERROR)
10     INTEGER GROUP, ASSERT, WIN, IERROR
11
12     MPI_WIN_SYNC(WIN, IERROR)
13     INTEGER WIN, IERROR
14
15     MPI_WIN_TEST(WIN, FLAG, IERROR)
16     INTEGER WIN, IERROR
17     LOGICAL FLAG
18
19     MPI_WIN_UNLOCK(RANK, WIN, IERROR)
20     INTEGER RANK, WIN, IERROR
21
22     MPI_WIN_UNLOCK_ALL(WIN, IERROR)
23     INTEGER WIN, IERROR
24
25     MPI_WIN_WAIT(WIN, IERROR)
26     INTEGER WIN, IERROR
27
28
29     A.5.11 External Interfaces Fortran Bindings
30
31     MPI_GREQUEST_COMPLETE(REQUEST, IERROR)
32     INTEGER REQUEST, IERROR
33
34     MPI_GREQUEST_START(QUERY_FN, FREE_FN, CANCEL_FN, EXTRA_STATE, REQUEST, IERROR)
35     EXTERNAL QUERY_FN, FREE_FN, CANCEL_FN
36     INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
37     INTEGER REQUEST, IERROR
38
39     MPI_STATUS_SET_CANCELLED(STATUS, FLAG, IERROR)
40     INTEGER STATUS(MPI_STATUS_SIZE), IERROR
41     LOGICAL FLAG
42
43     MPI_STATUS_SET_ELEMENTS(STATUS, DATATYPE, COUNT, IERROR)
44     INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, COUNT, IERROR
45
46     MPI_STATUS_SET_ERROR(STATUS, ERR, IERROR)
47     INTEGER STATUS(MPI_STATUS_SIZE), ERR, IERROR
48
49     MPI_STATUS_SET_SOURCE(STATUS, SOURCE, IERROR)
50     INTEGER STATUS(MPI_STATUS_SIZE), SOURCE, IERROR
51
52     MPI_STATUS_SET_TAG(STATUS, TAG, IERROR)
53     INTEGER STATUS(MPI_STATUS_SIZE), TAG, IERROR

```

A.5.12 I/O Fortran Bindings

```

MPI_CONVERSION_FN_NULL(USERBUF, DATATYPE, COUNT, FILEBUF, POSITION,
    EXTRA_STATE, IERROR)
    <TYPE> USERBUF(*), FILEBUF(*)
    INTEGER DATATYPE, COUNT, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) POSITION
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE

MPI_FILE_CLOSE(FH, IERROR)
    INTEGER FH, IERROR

MPI_FILE_DELETE(FILENAME, INFO, IERROR)
    CHARACTER*(*) FILENAME
    INTEGER INFO, IERROR

MPI_FILE_GET_AMODE(FH, AMODE, IERROR)
    INTEGER FH, AMODE, IERROR

MPI_FILE_GET_ATOMICITY(FH, FLAG, IERROR)
    INTEGER FH, IERROR
    LOGICAL FLAG

MPI_FILE_GET_BYTE_OFFSET(FH, OFFSET, DISP, IERROR)
    INTEGER FH, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET, DISP

MPI_FILE_GET_GROUP(FH, GROUP, IERROR)
    INTEGER FH, GROUP, IERROR

MPI_FILE_GET_INFO(FH, INFO_USED, IERROR)
    INTEGER FH, INFO_USED, IERROR

MPI_FILE_GET_POSITION(FH, OFFSET, IERROR)
    INTEGER FH, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET

MPI_FILE_GET_POSITION_SHARED(FH, OFFSET, IERROR)
    INTEGER FH, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET

MPI_FILE_GET_SIZE(FH, SIZE, IERROR)
    INTEGER FH, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) SIZE

MPI_FILE_GET_TYPE_EXTENT(FH, DATATYPE, EXTENT, IERROR)
    INTEGER FH, DATATYPE, IERROR
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTENT

MPI_FILE_GET_VIEW(FH, DISP, ETYPE, FILETYPE, DATAREP, IERROR)
    INTEGER FH, ETYPE, FILETYPE, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) DISP
    CHARACTER*(*) DATAREP

MPI_FILE_IREAD(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
    INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
    <type> BUF(*)

```

```

1  MPI_FILE_IREAD_ALL(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
2      INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
3      <type> BUF(*)
4
5  MPI_FILE_IREAD_AT(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
6      INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
7      INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
8      <type> BUF(*)
9
10 MPI_FILE_IREAD_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
11     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
12     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
13     <type> BUF(*)
14
15 MPI_FILE_IREAD_SHARED(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
16     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
17     <type> BUF(*)
18
19 MPI_FILE_IWRITE(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
20     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
21     <type> BUF(*)
22
23 MPI_FILE_IWRITE_ALL(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
24     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
25     <type> BUF(*)
26
27 MPI_FILE_IWRITE_AT(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
28     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
29     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
30     <type> BUF(*)
31
32 MPI_FILE_IWRITE_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, REQUEST, IERROR)
33     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
34     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
35     <type> BUF(*)
36
37 MPI_FILE_IWRITE_SHARED(FH, BUF, COUNT, DATATYPE, REQUEST, IERROR)
38     INTEGER FH, COUNT, DATATYPE, REQUEST, IERROR
39     <type> BUF(*)
40
41 MPI_FILE_OPEN(COMM, FILENAME, AMODE, INFO, FH, IERROR)
42     INTEGER COMM, AMODE, INFO, FH, IERROR
43     CHARACTER*(*) FILENAME
44
45 MPI_FILE_PREALLOCATE(FH, SIZE, IERROR)
46     INTEGER FH, IERROR
47     INTEGER(KIND=MPI_OFFSET_KIND) SIZE
48
49 MPI_FILE_READ(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
50     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
51     <type> BUF(*)
52
53 MPI_FILE_READ_ALL(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
54     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
55     <type> BUF(*)

```

```

MPI_FILE_READ_ALL_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    <type> BUF(*)

MPI_FILE_READ_ALL_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_FILE_READ_AT(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
    <type> BUF(*)

MPI_FILE_READ_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
    <type> BUF(*)

MPI_FILE_READ_AT_ALL_BEGIN(FH, OFFSET, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
    <type> BUF(*)

MPI_FILE_READ_AT_ALL_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_FILE_READ_ORDERED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_FILE_READ_ORDERED_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
    INTEGER FH, COUNT, DATATYPE, IERROR
    <type> BUF(*)

MPI_FILE_READ_ORDERED_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_FILE_READ_SHARED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_FILE_SEEK(FH, OFFSET, WHENCE, IERROR)
    INTEGER FH, WHENCE, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET

MPI_FILE_SEEK_SHARED(FH, OFFSET, WHENCE, IERROR)
    INTEGER FH, WHENCE, IERROR
    INTEGER(KIND=MPI_OFFSET_KIND) OFFSET

MPI_FILE_SET_ATOMICITY(FH, FLAG, IERROR)
    INTEGER FH, IERROR
    LOGICAL FLAG

```

```

1  MPI_FILE_SET_INFO(FH, INFO, IERROR)
2      INTEGER FH, INFO, IERROR
3
4  MPI_FILE_SET_SIZE(FH, SIZE, IERROR)
5      INTEGER FH, IERROR
6      INTEGER(KIND=MPI_OFFSET_KIND) SIZE
7
8  MPI_FILE_SET_VIEW(FH, DISP, ETYPE, FILETYPE, DATAREP, INFO, IERROR)
9      INTEGER FH, ETYPE, FILETYPE, INFO, IERROR
10     INTEGER(KIND=MPI_OFFSET_KIND) DISP
11     CHARACTER*(*) DATAREP
12
13 MPI_FILE_SYNC(FH, IERROR)
14     INTEGER FH, IERROR
15
16 MPI_FILE_WRITE(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
17     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
18     <type> BUF(*)
19
20 MPI_FILE_WRITE_ALL(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
21     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
22     <type> BUF(*)
23
24 MPI_FILE_WRITE_ALL_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
25     INTEGER FH, COUNT, DATATYPE, IERROR
26     <type> BUF(*)
27
28 MPI_FILE_WRITE_ALL_END(FH, BUF, STATUS, IERROR)
29     INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
30     <type> BUF(*)
31
32 MPI_FILE_WRITE_AT(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
33     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
34     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
35     <type> BUF(*)
36
37 MPI_FILE_WRITE_AT_ALL(FH, OFFSET, BUF, COUNT, DATATYPE, STATUS, IERROR)
38     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
39     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
40     <type> BUF(*)
41
42 MPI_FILE_WRITE_AT_ALL_BEGIN(FH, OFFSET, BUF, COUNT, DATATYPE, IERROR)
43     INTEGER FH, COUNT, DATATYPE, IERROR
44     INTEGER(KIND=MPI_OFFSET_KIND) OFFSET
45     <type> BUF(*)
46
47 MPI_FILE_WRITE_AT_ALL_END(FH, BUF, STATUS, IERROR)
48     INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
49     <type> BUF(*)
50
51 MPI_FILE_WRITE_ORDERED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
52     INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
53     <type> BUF(*)
54
55 MPI_FILE_WRITE_ORDERED_BEGIN(FH, BUF, COUNT, DATATYPE, IERROR)
56     INTEGER FH, COUNT, DATATYPE, IERROR
57     <type> BUF(*)

```



```

MPI_FILE_WRITE_ORDERED_END(FH, BUF, STATUS, IERROR)
    INTEGER FH, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_FILE_WRITE_SHARED(FH, BUF, COUNT, DATATYPE, STATUS, IERROR)
    INTEGER FH, COUNT, DATATYPE, STATUS(MPI_STATUS_SIZE), IERROR
    <type> BUF(*)

MPI_REGISTER_DATAREP(DATAREP, READ_CONVERSION_FN, WRITE_CONVERSION_FN,
    DTYPE_FILE_EXTENT_FN, EXTRA_STATE, IERROR)
    CHARACTER*(*) DATAREP
    EXTERNAL READ_CONVERSION_FN, WRITE_CONVERSION_FN, DTYPE_FILE_EXTENT_FN
    INTEGER(KIND=MPI_ADDRESS_KIND) EXTRA_STATE
    INTEGER IERROR

```

A.5.13 Language Bindings Fortran Bindings

```

MPI_F_SYNC_REG(BUF)
    <type> BUF(*)

```

The following procedure is not available with mpif.h:

```

MPI_STATUS_F082F(F08_STATUS, F_STATUS, IERROR)
    TYPE(MPI_Status) :: F08_STATUS
    INTEGER :: F_STATUS(MPI_STATUS_SIZE), IERROR

```

The following procedure is not available with mpif.h:

```

MPI_STATUS_F2F08(F_STATUS, F08_STATUS, IERROR)
    INTEGER :: F_STATUS(MPI_STATUS_SIZE), IERROR
    TYPE(MPI_Status) :: F08_STATUS

```

```

MPI_TYPE_CREATE_F90_COMPLEX(P, R, NEWTYPE, IERROR)
    INTEGER P, R, NEWTYPE, IERROR

```

```

MPI_TYPE_CREATE_F90_INTEGER(R, NEWTYPE, IERROR)
    INTEGER R, NEWTYPE, IERROR

```

```

MPI_TYPE_CREATE_F90_REAL(P, R, NEWTYPE, IERROR)
    INTEGER P, R, NEWTYPE, IERROR

```

```

MPI_TYPE_MATCH_SIZE(TYPECLASS, SIZE, DATATYPE, IERROR)
    INTEGER TYPECLASS, SIZE, DATATYPE, IERROR

```

A.5.14 Application Binary Interface (ABI) Fortran Bindings

```

MPI_ABI_GET_FORTRAN_BOOLEANS(LOGICAL_SIZE, LOGICAL_TRUE, LOGICAL_FALSE, IS_SET,
    IERROR)
    INTEGER LOGICAL_SIZE, IERROR
    LOGICAL LOGICAL_TRUE, LOGICAL_FALSE, IS_SET

```

```

MPI_ABI_GET_FORTRAN_INFO(INFO, IERROR)
    INTEGER INFO, IERROR

```

```

MPI_ABI_GET_INFO(INFO, IERROR)
    INTEGER INFO, IERROR

```

```

1  MPI_ABI_GET_VERSION(ABI_MAJOR, ABI_MINOR, IERROR)
2      INTEGER ABI_MAJOR, ABI_MINOR, IERROR
3
4  MPI_ABI_SET_FORTRAN_BOOLEANS(LOGICAL_SIZE, LOGICAL_TRUE, LOGICAL_FALSE, IERROR)
5      INTEGER LOGICAL_SIZE, IERROR
6      LOGICAL LOGICAL_TRUE, LOGICAL_FALSE
7
8  MPI_ABI_SET_FORTRAN_INFO(INFO, IERROR)
9      INTEGER INFO, IERROR

```

A.5.15 Tools / Profiling Interface Fortran Bindings

```

12 MPI_PCONTROL(LEVEL)
13     INTEGER LEVEL

```

A.5.16 Deprecated Fortran Bindings

```

17 MPI_ATTR_DELETE(COMM, KEYVAL, IERROR)
18     INTEGER COMM, KEYVAL, IERROR
19
20 MPI_ATTR_GET(COMM, KEYVAL, ATTRIBUTE_VAL, FLAG, IERROR)
21     INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, IERROR
22     LOGICAL FLAG
23
24 MPI_ATTR_PUT(COMM, KEYVAL, ATTRIBUTE_VAL, IERROR)
25     INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, IERROR
26
27 MPI_DUP_FN(OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT,
28     FLAG, IERR)
29     INTEGER OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT,
30     IERR
31     LOGICAL FLAG
32
33 MPI_GET_ELEMENTS_X(STATUS, DATATYPE, COUNT, IERROR)
34     INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, IERROR
35     INTEGER(KIND=MPI_COUNT_KIND) COUNT
36
37 MPI_INFO_GET(INFO, KEY, VALUEN, VALUE, FLAG, IERROR)
38     INTEGER INFO, VALUEN, IERROR
39     CHARACTER*(*) KEY, VALUE
40     LOGICAL FLAG
41
42 MPI_INFO_GET_VALUEN(INFO, KEY, VALUEN, FLAG, IERROR)
43     INTEGER INFO, VALUEN, IERROR
44     CHARACTER*(*) KEY
45     LOGICAL FLAG
46
47 MPI_KEYVAL_CREATE(COPY_FN, DELETE_FN, KEYVAL, EXTRA_STATE, IERROR)
48     EXTERNAL COPY_FN, DELETE_FN
49     INTEGER KEYVAL, EXTRA_STATE, IERROR
50
51 MPI_KEYVAL_FREE(KEYVAL, IERROR)
52     INTEGER KEYVAL, IERROR

```

```

MPI_NULL_COPY_FN(OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN,
                  ATTRIBUTE_VAL_OUT, FLAG, IERR)
    INTEGER OLDCOMM, KEYVAL, EXTRA_STATE, ATTRIBUTE_VAL_IN, ATTRIBUTE_VAL_OUT,
    IERR
    LOGICAL FLAG

MPI_NULL_DELETE_FN(COMM, KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERROR)
    INTEGER COMM, KEYVAL, ATTRIBUTE_VAL, EXTRA_STATE, IERROR

MPI_SIZEOF(X, SIZE, IERROR)
    <type> X
    INTEGER SIZE, IERROR

MPI_STATUS_SET_ELEMENTS_X(STATUS, DATATYPE, COUNT, IERROR)
    INTEGER STATUS(MPI_STATUS_SIZE), DATATYPE, IERROR
    INTEGER(KIND=MPI_COUNT_KIND) COUNT

MPI_TYPE_GET_EXTENT_X(DATATYPE, LB, EXTENT, IERROR)
    INTEGER DATATYPE, IERROR
    INTEGER(KIND=MPI_COUNT_KIND) LB, EXTENT

MPI_TYPE_GET_TRUE_EXTENT_X(DATATYPE, TRUE_LB, TRUE_EXTENT, IERROR)
    INTEGER DATATYPE, IERROR
    INTEGER(KIND=MPI_COUNT_KIND) TRUE_LB, TRUE_EXTENT

MPI_TYPE_SIZE_X(DATATYPE, SIZE, IERROR)
    INTEGER DATATYPE, IERROR
    INTEGER(KIND=MPI_COUNT_KIND) SIZE

```

DRAFT

Appendix B

Change-Log

Annex B.1 summarizes changes from the previous version of the MPI standard to the version presented by this document. Only significant changes (i.e., clarifications and new features) that might either require implementation effort in the MPI libraries or change the understanding of MPI from a user's perspective are presented. Editorial modifications, formatting, typo corrections and minor clarifications are not shown. If not otherwise noted, the section and page references refer to the locations of the change or new functionality in this version of the standard. Changes in Annexes B.2–B.8 were already introduced in the corresponding sections in previous versions of this standard.

B.1 Changes from Version 5.0 to Version 5.1

1. Chapter 17, Sections 2.8, 9.1.2, 9.3, and 9.4, on pages 787, 28, 440, 448, and 459
Added operations, procedures, constants, and semantics to handle MPI process failures.
2. Chapter 13, Section 9.1.2, on pages 625 and 440
Added the MPI_Queue object, matched persistent requests, and the MPI_QUEUE_CREATE, MPI_QUEUE_FREE, MPI_MATCH, MPI_IMATCH, MPI_MATCHALL, MPI_IMATCHALL, MPI_ENQUEUE_START, MPI_ENQUEUE_STARTALL, MPI_ENQUEUE_WAIT, MPI_ENQUEUE_WAITALL, MPI_QUEUE_TEST, and MPI_QUEUE_WAIT procedures for ordering persistent communication operations with respect to external asynchronous execution contexts such as GPU streams, along with the new error class MPI_ERR_QUEUE.

B.1.1 Fixes to Errata in Previous Versions of MPI

B.1.2 Changes in MPI-5.1

B.2 Changes from Version 4.1 to Version 5.0

B.2.1 Fixes to Errata in Previous Versions of MPI

1. Section 3.2.5 on page 40, and MPI-4.1 Section 3.2.5 on page 39.
Mark status argument as const in MPI_STATUS_GET_SOURCE, MPI_STATUS_GET_TAG, and MPI_STATUS_GET_ERROR.
2. Sections 11.2.1 and 11.4.3 on pages 480 and 506, and MPI-4.1 Sections 11.2.1 and 11.4.3 on pages 484 and 508.
Clarified that, in the World Model, the value of the "mpi_memory_alloc_kinds" info key in MPI_INFO_ENV indicates the requested memory allocation kinds. The provided

memory allocation kinds can be queried through the value of the "mpi_memory_alloc_kinds" info key in the info object returned by a call to `MPI_COMM_GET_INFO` on `MPI_COMM_WORLD` or `MPI_COMM_SELF`.

3. Section 11.4.3 on page 506, and MPI-4.1 Section 11.4.3 on page 508. Clarified the requirements of the "mpi_memory_alloc_kinds" info key's value when defaulted.
4. Section 12.3.5 page 577 and MPI-4.1 Section 12.3.5 on page 580. The text contained contradicting statements about the use of `MPI_REQUEST_FREE` on RMA requests. A statement that suggested the use of `MPI_REQUEST_FREE` to release RMA requests after the window has been flushed was removed.

B.2.2 Changes in MPI-5.0

1. Fixed-size Fortran logical datatypes were added to Section 3.2.2, Section 6.9.2, Table 15.3, Section 21.1.9, and Appendix Section A.
2. Section 7.3.2 on page 312. A restriction on the provenance of the process set name supplied to `MPI_GROUP_FROM_SESSION_PSET` was removed.
3. Chapter 22 was added, which defines new functions, `MPI_ABI_GET_VERSION` and `MPI_ABI_GET_INFO`, the layout of the MPI status object, and the type of MPI object handles (e.g. `MPI_Comm`). Extensive changes and additions were made in Section A.1.1 to define literal values for many MPI constants. In Chapter 10, `MPI_MAX_INFO_KEY` was modified by one to be consistent with other constants. `MPI_ABI_GET_VERSION` and `MPI_ABI_GET_INFO` were added to Table 11.1.
4. Section 22.4.1 was added to define new functions, `MPI_ABI_SET_FORTRAN_INFO`, `MPI_ABI_GET_FORTRAN_INFO`, and `MPI_ABI_SET_FORTRAN_BOOLEANS`, along with the associated info keys and an error code `MPI_ERR_ABI`, in order to expose Fortran environment settings to the MPI library. Section 22.4.2 and Section 22.4.3 were added to define basic properties of the MPI Fortran ABI. Section 22.4.5 was added to obviate the need for `MPI_Fint`.

B.3 Changes from Version 4.0 to Version 4.1

B.3.1 Fixes to Errata in Previous Versions of MPI

1. Sections 2.4.1, 3.5, 3.7, 3.8, 6.12, 7.1.2, 7.2.2, 7.4.2, 11.6.2, 15.2.5, 15.6.5, 21.1.17, 21.1.20, on pages 13, 57, 71, 95, 250, 305, 308, 321, 514, 660, 720, 846, 856 and MPI-4.0 Sections 2.4.1, 3.5, 3.7, 3.8, 6.12, 7.1.2, 7.2.2, 7.4.2, 11.6.2, 14.2.5, 14.6.5, 19.1.17, 19.1.20 on pages 13, 54, 60, 84, 250, 312, 314, 327, 518, 650, 713, 826, and 834. The term *pending* communication or I/O operation is defined as *in the active operation state*. If the phrase *pending communication operation* in MPI-1.1 to MPI-4.0 additionally includes *decoupled MPI activities*, then this has been added explicitly. If this phrase had a different meaning, it was replaced accordingly, see item 4 in this list and item 15 in Section B.3.2.

2. Sections 2.5.4 and A.1.1 on pages 21 and 891, and MPI-4.0 Sections 2.5.4 and A.1.1 on pages 20 and 857.
The implementation of named MPI *constants* in C and Fortran and implied usage restrictions were clarified.
3. Section 2.5.4 on page 21, and MPI-4.0 Section 2.5.4 on page 20.
Add `MPI_MAX_PSET_NAME_LEN` and `MPI_MAX_STRINGTAG_LEN` to list of named constants.
4. Sections 3.9, 11.2.2, 11.3.1 and 11.10.4 on pages 105, 486, 492 and 540 and MPI-4.0 Sections 3.9, 11.2.2, 11.3.1 and 11.10.4 on pages 94, 494, 501 and 546.
The requirements for calling `MPI_FINALIZE`, `MPI_SESSION_FINALIZE`, and `MPI_COMM_DISCONNECT` and the outcome of `MPI_COMM_DISCONNECT`, especially for related inactive persistent request handles, were clarified.
5. Section 4.3 on page 121, and MPI-4.0 Section 4.3.3. on page 115. Example 4.4 on page 125, and MPI-4.0 Example 4.4 on page 115.
Fixed and simplified erroneous MPI-4.0 Example 4.4. The example could deadlock due to incorrect use of the `flag` variable in multiple MPI test procedure calls or thread concurrent access. The example was also simplified by removing unnecessary code and updated according to current best practice in OpenMP.
6. Section 4.3.1 on page 122, MPI-4 Section 4.3.1 on page 112.
Example 4.2 was corrected.
7. Section 5.1.9 on page 156 and MPI-4.0 Section 5.1.9 on page 150.
The relationship between `MPI_TYPE_COMMIT` and initialization/finalization of a session (or the World Model) with `MPI_INIT`, `MPI_INIT_THREAD`, `MPI_FINALIZE`, `MPI_SESSION_INIT`, and `MPI_SESSION_FINALIZE` was clarified.
8. Sections 7.4.2 on page 321 and 7.6.2 on page 352, and MPI-4.0 Section 7.4.2 on page 343 and Section 7.6.2 on page 360.
Use of the `errhandler` argument to `MPI_COMM_CREATE_FROM_GROUP` and `MPI_INTERCOMM_CREATE_FROM_GROUPS` is clarified. The error handler invoked when an error is encountered during invocation of these functions is also clarified.
9. Section 7.4.2 on page 321, MPI-3 Section 6.4.2 on page 237, MPI-3.1 Section 6.4.2 on page 237, and MPI-4 Section 7.4.2 on page 327.
The description of `MPI_COMM_DUP` now clarifies that error handlers are also copied in the output communicator produced when this procedure is called.
10. Section 7.7.2 on page 360, MPI-3 Section 6.7.2 on page 267, MPI-3.1 Section 6.7.2 on page 267, and MPI-4 Section 7.7.2 on page 366.
`MPI_COMM_DISCONNECT` was explicitly stated as a procedure that deletes attributes.
11. Section 8.5.5 on page 394 and MPI-4.0 Section 8.5.5 on page 403.
The unintended change in the specification of argument `coords` in `MPI_CART_COORDS` in MPI-4.0 is reverted to the original meaning in MPI-1.1 to MPI-3.1. It is clarified that the outcome of `MPI_CART_GET` and `MPI_CART_COORDS` is unspecified for the case that `maxdims` is less than `ndims`.

12. Section 9.3 on page 448 and MPI-4.0 Section 9.3 on page 458.
The fallback error-handler for the Sessions Model was clarified.
13. Section 9.5 on page 463 and MPI-4.0 Section 9.5 on page 473.
It was clarified that `MPI_LASTUSED` is only available in the World Model.
14. Sections 11.1 and 11.7 on pages 479 and 516, and MPI-4.0 Sections 11.1 and 11.7 on pages 487 and 521.
It was clarified that the usage of the Dynamic Process Model requires the World Model to be initialized.
15. Section 12.5.2 on page 590 and MPI-4 Section 12.5.2 on page 598.
The definition of `MPI_WIN_TEST` was clarified.
16. Section 12.5.4 on page 598, MPI-3 Section 11.5.4 on page 449, MPI-3.1 Section 11.5.4 on page 448, and MPI-4 Section 12.5.4 on page 605.
The description of `MPI_WIN_SYNC` was clarified to include its use for ordering load/store accesses to shared memory. A statement was added to highlight that a call to `MPI_WIN_SYNC` does not complete pending RMA operations and that a call to `MPI_WIN_SYNC` does not guarantee any progress of MPI operations.
17. Section 14.3 on page 649, and MPI-4 Section 13.3 on page 640.
Large count interface of `MPI_STATUS_SET_ELEMENTS` had been missing and was added.
18. Sections 16.3.6 and 16.3.7 on pages 745 and 752, MPI-3 Sections 14.3.6 and 14.3.7 on pages 567 and 573, MPI-3.1 Sections 14.3.6 and 14.3.7 on pages 573 and 580, and MPI-4 Sections 15.3.6 and 15.3.7 on pages 738 and 744.
The intent of handle arguments of the language independent definition of `MPI_T_CVAR_WRITE`, `MPI_T_PVAR_HANDLE_ALLOC`, `MPI_T_PVAR_HANDLE_FREE`, `MPI_T_PVAR_START`, `MPI_T_PVAR_STOP`, `MPI_T_PVAR_WRITE`, and `MPI_T_PVAR_RESET` were marked as INOUT in accordance with the special rule for handles described in Section 2.3.

B.3.2 Changes in MPI-4.1

1. Section 1.15 on page 7.
Introduced the concept of side documents.
2. Sections 2.4.1 and 2.4.2 on pages 13 and 16.
Added the definition of the *enabled operation state*.
3. Sections 2.5.5, 2.6.2, and 21.1 on pages 22, 25, and 811.
The MPI standard now reflects that TS 29113 was superseded by Fortran 2018.
4. Sections 2.6.1, 5.1.5, 5.1.7, 5.1.8, 5.1.11, 14.3, and 18.4 on pages 24, 148, 153, 154, 158, 649, and 802.
`MPI_TYPE_SIZE_X`, `MPI_TYPE_GET_EXTENT_X`, `MPI_TYPE_GET_TRUE_EXTENT_X`, `MPI_GET_ELEMENTS_X`, and `MPI_STATUS_SET_ELEMENTS_X` were deprecated and may be removed in a future version of the MPI specification.

5. Sections 2.6.1, 7.2.4 and 9.1.2 on pages 24, 309 and 440.
 MPI_HOST has been deprecated, and a mention to host process has been removed.
6. Sections 2.6.1, 21.1.1, 21.1.4, 18.4 on pages 24, 811, 817, 802.
 Deprecated the use of mpif.h.
7. Section 2.6.4 on page 27.
 Removed the functions MPI_WTIME, PMPI_WTIME, MPI_WTICK, and PMPI_WTICK from the list of functions that may be implemented as a macro.
8. Section 2.6.4 on page 27 and Section 21.3.4 on page 860.
 Removed the ability to implement MPI handle conversion functions as a macro.
9. Sections 2.9, 3.7.6, 12.5.2, and Example 12.13 on pages 29, 92, 590, and 614.
 The *progress rules* were clarified in general and for MPI_REQUEST_GET_STATUS and MPI_WIN_TEST. The terms *strong* and *weak progress* were introduced. An example showing restrictions on the use of MPI shared memory for synchronizing purposes was introduced.
10. Sections 3.2.5 and 14.3 on pages 40 and 649.
 Added procedures MPI_STATUS_GET_SOURCE, MPI_STATUS_GET_TAG, and MPI_STATUS_GET_ERROR to query MPI status fields and procedures MPI_STATUS_SET_SOURCE, MPI_STATUS_SET_TAG, and MPI_STATUS_SET_ERROR to set MPI status fields. Direct access to these fields remains available.
11. Section 3.6 on page 60.
 Automatic (unlimited) buffering is added, which can be enabled by using MPI_BUFFER_AUTOMATIC in any of the buffer attach procedures. New procedures MPI_COMM_ATTACH_BUFFER, MPI_SESSION_ATTACH_BUFFER, MPI_COMM_DETACH_BUFFER and MPI_SESSION_DETACH_BUFFER have been added. The buffers attached with the existing functions MPI_BUFFER_ATTACH and MPI_BUFFER_DETACH now only apply to communicators that have no buffer attached at the communicator or session level. New procedures MPI_COMM_FLUSH_BUFFER, MPI_SESSION_FLUSH_BUFFER, and MPI_BUFFER_FLUSH were added as a combination of detach and attach, as well as the corresponding nonblocking variants MPI_COMM_IFLUSH_BUFFER, MPI_SESSION_IFLUSH_BUFFER, and MPI_BUFFER_IFLUSH.
12. Subsection 3.7.6 on page 92
 Added new procedures MPI_REQUEST_GET_STATUS_ANY, MPI_REQUEST_GET_STATUS_SOME, and MPI_REQUEST_GET_STATUS_ALL to query the statuses of multiple requests without freeing them.
13. Section 5.1.13 and 6.9.4 on pages 162 and 229.
 Added procedure MPI_TYPE_GET_VALUE_INDEX to query predefined datatype handles for pairs of value and index types to be usable in conjunction with MPI_MINLOC and MPI_MAXLOC. Added combiner MPI_COMBINER_VALUE_INDEX for unnamed type handles returned by MPI_TYPE_GET_VALUE_INDEX.

- 1 14. Section 7.4.2 on page 321.
2 `MPI_COMM_TYPE_RESOURCE_GUIDED` was added as a new possible value for the
3 `split_type` parameter of the `MPI_COMM_SPLIT_TYPE` procedure, as well as a new
4 info key `"mpi_pset_name"`.
5
- 6 15. Section 7.4.3 on page 338.
7 The definition of `MPI_COMM_FREE` was clarified.
8
- 9 16. Section 7.4.4 on page 339.
10 A new info key was added, namely `"mpi_assert_strict_persistent_collective_ordering"`.
11
- 12 17. Sections 7.4.4, 11.2.1, 11.3, 11.4.3, 11.8.4, 12.2.1, and 15.2.8 on pages 339, 480, 491,
13 506, 526, 546, and 663.
14 Added the ability to request support for, query support of, and assert usage of memory
15 allocation kinds via two new info keys, `"mpi_memory_alloc_kinds"` and
16 `"mpi_assert_memory_alloc_kinds"`.
17
- 18 18. Section 7.8 on page 374 was amended to allow `MPI_COMM_NULL`,
19 `MPI_DATATYPE_NULL`, and `MPI_WIN_NULL` to be passed to
20 `MPI_COMM_GET_NAME`, `MPI_TYPE_GET_NAME` and `MPI_WIN_GET_NAME`,
21 respectively.
22
- 23 19. Section 9.1.2 on page 443.
24 Added new procedure `MPI_GET_HW_RESOURCE_INFO`.
25
- 26 20. Section 9.4 on page 459.
27 Added new error class `MPI_ERR_ERRHANDLER`.
28
- 29 21. Section 9.5 on page 463.
30 Add procedures `MPI_REMOVE_ERROR_CLASS`, `MPI_REMOVE_ERROR_CODE`,
31 `MPI_REMOVE_ERROR_STRING` to complement the procedures adding error classes/codes/strings. ■
32
- 33 22. Section 12.2 on page 546.
34 Relaxed the constraints on the windows for which shared memory can be queried
35 using `MPI_WIN_SHARED_QUERY` to allow windows with flavor
36 `MPI_WIN_FLAVOR_CREATE` and `MPI_WIN_FLAVOR_ALLOCATE`.
37
- 38 23. Section 12.2.1 on page 546.
39 Added new info key `"mpi_accumulate_granularity"` to specify a desired synchronization
40 granularity of accumulate operations.
41
- 42 24. Section 12.2.5 on page 560.
43 Implementations may avoid synchronization of processes in `MPI_WIN_FREE` if the
44 `"no_locks"` info key is set to `"true"`.
45
- 46 25. Section 12.5.2 on page 590.
47 In Example 12.4, `MPI_PUT` has been removed from the list of procedures that may
48 delay their return waiting for the call to `MPI_WIN_POST` to occur at the target.
49 MPI RMA communication procedures are generally not intended to delay their return
50 waiting for synchronization procedure calls to occur at the target.
- 51 26. Section 12.7 on page 602 and Section 12.8 on page 615.
52 Clarified the use of `MPI_WIN_SYNC` for memory synchronization on shared memory.

27. Section 16.3.2 on page 740.
The text specifying when entities of the MPI Tool Information Interface can be bound to objects during the object's lifetime was clarified.
28. Section 16.3.8 on page 775 and Section 16.3.9 on page 783.
Behavior specified when the count of dropped events or category changes overflow, respectively.
29. Section 21.3.3 on page 859.
Language interoperability when using `MPI_SESSION_INIT` and `MPI_SESSION_FINALIZE` was clarified.
30. Annex A.2 on page 918.
The annex has been completed with the operation-related MPI procedures for one-sided communication and some other rarely used scenarios. It is now integrated into the MPI standard.

B.4 Changes from Version 3.1 to Version 4.0

B.4.1 Fixes to Errata in Previous Versions of MPI

1. Sections 8.6.1, 8.6.2 and 8.9 on pages 408, 412 and 433, and MPI-3.1 Sections 7.6.1, 7.6.2 and 7.8 on pages 315, 318 and 329.
`MPI_NEIGHBOR_ALLTOALL{|V|W}` and `MPI_NEIGHBOR_ALLGATHER{|V}` for Cartesian virtual grids were clarified. An advice to implementors was added to illustrate a correct implementation for the case of `periods[d]=1` or `.TRUE.` and `dims[d]=1` or `2` in a direction `d`.
2. Section 21.3.5 on page 862, and MPI-3.1 Section 17.2.5 on page 657 line 11.
Clarified that the `MPI_STATUS_F2F08` and `MPI_STATUS_F082F` routines and the declaration for `TYPE(MPI_Status)` are not supposed to appear with `mpif.h`.
3. Sections 2.5.4, 21.3.5, and A.1.1 on pages 22, 862, and 898, and MPI-3.1 Sections 2.5.4, 17.2.5, and A.1.1 on pages 15, 656, and 669.
Define the C constants `MPI_F_STATUS_SIZE`, `MPI_F_SOURCE`, `MPI_F_TAG`, and `MPI_F_ERROR`.
4. Section 21.3.5 on page 863, and MPI-3.1 Section 17.2.5 on page 658.
Added missing `const` to `IN` parameters for `MPI_STATUS_F2F08` and `MPI_STATUS_F082F`.

B.4.2 Changes in MPI-4.0

1. Sections 2.2, 20.2.2, and 21.1.5 on pages 11, 810, and 818.
The limit for the maximum length of MPI identifiers was removed.
2. Section 2.4, 3.4, 3.7.2, 3.7.3, 3.8.1, 3.8.2, 6.13, 15.4.5, and Annex A.2 on pages 13, 52, 73, 80, 96, 99, 273, 696, and 918.
The semantic terms were updated.

3. Sections 2.5.8 and 21.2 on pages 23 and 856, and throughout the entire document.
New large count functions `MPI_{...}_c` in C and through function overloading in the Fortran `mpi_f08` module, (with the exception of the explicit Fortran procedures `MPI_Op_create_c` and `MPI_Register_datarep_c`) and the new large count callbacks `MPI_User_function_c` and `MPI_Datarep_conversion_function_c` together with the pre-defined function `MPI_CONVERSION_FN_NULL_C` were introduced to accomodate large buffers and/or datatypes.
Clarifications were added to the behavior of INOUT/OUT parameters that cannot represent the value to be returned for the `MPI_BUFFER_DETACH` and `MPI_FILE_GET_TYPE_EXTENT` functions.
A new error class `MPI_ERR_VALUE_TOO_LARGE` was introduced.
4. Sections 2.8, 9.3, 9.5, and 11.2.1 on pages 28, 448, 463, and 480.
MPI calls that are not related to any objects are considered to be attached to the communicator `MPI_COMM_SELF` instead of `MPI_COMM_WORLD`. The definition of `MPI_ERRORS_ARE_FATAL` was clarified to cover all connected processes, and a new error handler, `MPI_ERRORS_ABORT`, was created to limit the scope of aborting.
5. Section 3.7 on page 71.
The introduction of MPI nonblocking communication was changed to describe correctness and performance reasons for the use of nonblocking communication.
6. Section 3.7.2 on page 73.
Addition of `MPI_ISENDRECV` and `MPI_ISENDRECV_REPLACE`.
7. Sections 3.7.3, 3.9, 6.13, 8.8, and 8.9 on pages 80, 105, 273, 426, and 433.
Persistent collective communication `MPI_{ALLGATHER|...}_INIT` including persistent collective neighborhood communication `MPI_NEIGHBOR_{ALLGATHER|...}_INIT` was added to the standard.
8. Sections 3.8.4 and 18.3 on pages 103 and 800.
Cancelling a send request by calling `MPI_CANCEL` has been deprecated and may be removed in a future version of the MPI specification.
9. Chapter 4 on page 113.
A new chapter on partitioned communication with the new MPI procedures `MPI_{PARRIVED|PREADY}{...}` and `MPI_{PRECV|PSEND}_INIT` was added.
10. Section 7.4.2 on page 321.
`MPI_COMM_TYPE_HW_UNGUIDED` was added as a new possible value for the `split_type` parameter of the `MPI_COMM_SPLIT_TYPE` function.
11. Section 7.4.2 on page 321.
`MPI_COMM_TYPE_HW_GUIDED` was added as a new possible value for the `split_type` parameter of the `MPI_COMM_SPLIT_TYPE` function, as well as a new info key "mpi_hw_resource_type". A specific value associated with this new info key is also defined: "mpi_shared_memory".
12. Section 7.4.2 on page 321.
The functions `MPI_COMM_DUP` and `MPI_COMM_IDUP` were updated to no longer

- propagate info hints.
This change may affect backward compatibility.
13. Section 7.4.2 on page 321.
The `MPI_COMM_IDUP_WITH_INFO` function was added.
 14. Sections 7.4.4, 12.2.7, and 15.2.8 on pages 339, 562, and 663.
The definition of info hints was updated to allow applications to provide assertions regarding their usage of MPI objects and operations.
 15. Section 7.4.4 on page 339.
The new info hints "mpi_assert_no_any_tag", "mpi_assert_no_any_source", "mpi_assert_exact_length", and "mpi_assert_allow_overtaking" were added for use with communicators.
 16. Sections 7.4.4, 12.2.7, and 15.2.8 on pages 339, 562, and 663.
The semantics of the `MPI_COMM_SET_INFO`, `MPI_COMM_GET_INFO`, `MPI_WIN_SET_INFO`, `MPI_WIN_GET_INFO`, `MPI_FILE_SET_INFO`, and `MPI_FILE_GET_INFO` were clarified.
 17. Section 8.5 on page 384.
`MPI_DIMS_CREATE` is now guaranteed to return `MPI_SUCCESS` if the number of dimensions passed to the routine is set to 0 and the number of nodes is set to 1.
 18. Sections 9.2, 12.2.2, and 12.2.3 on pages 445, 550, and 552.
Introduced alignment requirements for memory allocated through `MPI_ALLOC_MEM`, `MPI_WIN_ALLOCATE`, and `MPI_WIN_ALLOCATE_SHARED` and added a new info key "mpi_minimum_memory_alignment" to specify a desired alternative minimum alignment.
 19. Sections 9.3 and 9.4 on pages 448 and 459.
Clarified definition of errors to say that MPI should continue whenever possible and allow the user to recover from errors.
 20. Section 9.4 on page 459.
Added text to clarify what is implied about the status of MPI and user visible buffers when MPI functions return `MPI_SUCCESS` or other error codes.
 21. Section 9.4 on page 461.
The error class `MPI_ERR_PROC_ABORTED` has been added.
 22. Section 10 on page 471.
Added a new function `MPI_INFO_GET_STRING` that takes a buffer length argument for returning info value strings. This function returns the required buffer length for the requested string and guarantees null termination for C strings where buffer size is greater than 0.
 23. Section 10 on page 471 and Section 18.3 on page 800.
`MPI_INFO_GET` and `MPI_INFO_GET_VALUELEN` were deprecated.
 24. Chapter 11, 3.2.3, 7.2.4, 7.3.2, 7.4.2, 7.6.2, 9.1.1, 9.1.2, 9.3, 9.3.4, 9.5, 11.6, 15.2.1, 15.2.7, 15.7, 16.3.4, 21.3.4, 21.3.6, and Annex A on pages 479, 37, 309, 312, 321, 352,

439, 440, 448, 457, 463, 513, 655, 661, 727, 742, 860, 864, and 891.

The Sessions Model was added to the standard. New MPI procedures are `MPI_SESSION_{INIT|FINALIZE}`, `MPI_SESSION_GET_{...}`, `MPI_SESSION_{...}_ERRHANDLER`, `MPI_GROUP_FROM_SESSION_PSET`, `MPI_COMM_CREATE_FROM_GROUP`, `MPI_INTERCOMM_CREATE_FROM_GROUPS`, and new conversion functions are `MPI_SESSION_{C2F|F2C}`. New declarations are `MPI_Session` in C and `TYPE(MPI_Session)` together with the related overloaded operators `.EQ.`, `.NE.`, `==` and `/=` in the Fortran `mpi_f08` and `mpi` modules, and the callback function prototype `MPI_Session_errhandler_function`. New constants are `MPI_SESSION_NULL`, `MPI_ERR_SESSION`, `MPI_MAX_PSET_NAME_LEN`, `MPI_MAX_STRINGTAG_LEN`, `MPI_T_BIND_MPI_SESSION` and the predefined info key `"mpi_size"`.

25. Section 11.2.1 on page 480.

A new function `MPI_INFO_CREATE_ENV` was added.

26. Sections 11.2.1 and 11.10.4 on pages 480 and 540.

Clarified the semantic of failure and error reporting before (and during) `MPI_INIT` and after `MPI_FINALIZE`.

27. Section 11.8.4 on page 526.

Added the `"mpi_initial_errhandler"` reserved info key with the reserved values `"mpi_errors_abort"`, `"mpi_errors_are_fatal"`, and `"mpi_errors_return"` to the launch keys in `MPI_COMM_SPAWN`, `MPI_COMM_SPAWN_MULTIPLE`, and `mpiexec`.

28. Section 12.5.3 on page 595.

RMA passive target synchronization using locks can now be used portably in memory allocated via `MPI_WIN_ALLOCATE_SHARED`.

29. Section 14.3 on page 649.

The `mpi_f08` binding incorrectly had the dummy parameter flag in the MPI F08 binding for `MPI_STATUS_SET_CANCELLED` marked as `INTENT(OUT)`. It has been fixed to be `INTENT(IN)`.

30. Sections 16.3 and 16.3.8 on pages 738 and 764.

A callback-driven event interface with the `MPI_T_{SOURCE|EVENT}_{...}` and `MPI_T_CATEGORY_{GET|GET_NUM}_EVENTS` routines, the declaration types `MPI_T_cb_safety`, `MPI_T_event_{instance|registration}`, `MPI_T_source_order`, and the callback function prototypes `MPI_T_event_{cb|dropped_cb|free_cb}_function`, were added to the MPI tool information interface.

31. Section 16.3.9 on page 783.

The argument `stamp` (previously described as a virtual time stamp) from `MPI_T_CATEGORY_CHANGED` was renamed to `update_number` and its intended implementation and use was clarified.

32. Section 16.3.10, Table 16.7, and Section 18.3 on pages 783, 785, and 800.

`MPI_T_ERR_INVALID_ITEM` is deprecated. MPI routines should return `MPI_T_ERR_INVALID_INDEX` instead of `MPI_T_ERR_INVALID_ITEM`.

33. Section 18.3 on page 802.

`MPI_SIZEOF` was deprecated.

34. Section 21.1.5 on page 818.

An exception was added for the specific Fortran names in the case of TS 29113 interface specifications in `mpif.h` for `MPI_NEIGHBOR_ALLTOALLW_INIT`, `MPI_NEIGHBOR_ALLTOALLV_INIT`, and `MPI_NEIGHBOR_ALLGATHERV_INIT`.

B.5 Changes from Version 3.0 to Version 3.1

B.5.1 Fixes to Errata in Previous Versions of MPI

1. Chapters 3–21, Annex A.4 on page 963, and Example 6.22 on page 239, and MPI-3.0 Chapters 3–17, Annex A.3 on page 707, and Example 5.21 on page 187.
Within the `mpi_f08` Fortran support method, `BIND(C)` was removed from all SUBROUTINE, FUNCTION, and ABSTRACT INTERFACE definitions.
2. Section 3.2.5 on page 40, and MPI-3.0 Section 3.2.5 on page 30.
The three public fields `MPI_SOURCE`, `MPI_TAG`, and `MPI_ERROR` of the Fortran derived type `TYPE(MPI_Status)` must be of type `INTEGER`.
3. Section 3.8.2 on page 99, and MPI-3.0 Section 3.8.2 on page 67.
The flag arguments of the Fortran interfaces of `MPI_IMPROBE` were originally incorrectly defined as `INTEGER` (instead as `LOGICAL`).
4. Section 7.4.2 on page 321, and MPI-3.0 Section 6.4.2 on page 237.
In the `mpi_f08` binding of `MPI_COMM_IDUP`, the output argument `newcomm` is declared as `ASYNCHRONOUS`.
5. Section 7.4.4 on page 339, and MPI-3.0 Section 6.4.4 on page 248.
In the `mpi_f08` binding of `MPI_COMM_SET_INFO`, the `INTENT` of `comm` is `IN`, and the optional output argument `ierror` was missing.
6. Section 8.6 on page 407, and MPI-3.0 Sections 7.6, on pages 314.
In the case of virtual general graph topologies (created with `MPI_CART_CREATE`), the use of neighborhood collective communication is restricted to adjacency matrices with the number of edges between any two processes is defined to be the same for both processes (i.e., with a symmetric adjacency matrix).
7. Section 9.1.1 on page 439, and MPI-3.0 Section 8.1.1 on page 335.
In the `mpi_f08` binding of `MPI_GET_LIBRARY_VERSION`, a typo in the `resultlen` argument was corrected.
8. Sections 9.2 (`MPI_ALLOC_MEM` and `MPI_ALLOC_MEM_CPTR`),
12.2.2 (`MPI_WIN_ALLOCATE` and `MPI_WIN_ALLOCATE_CPTR`),
12.2.3 (`MPI_WIN_ALLOCATE_SHARED` and
`MPI_WIN_ALLOCATE_SHARED_CPTR`),
12.2.3 (`MPI_WIN_SHARED_QUERY` and `MPI_WIN_SHARED_QUERY_CPTR`),
16.2.1 and 16.2.5 (Profiling interface), and corresponding sections in MPI-3.0.
The linker name concept was substituted by defining specific procedure names.
9. Section 12.2.1 on page 546, and MPI-3.0 Section 11.2.2 on page 407.
The "same_size" info key can be used with all window flavors, and requires that all

- processes in the process group of the communicator have provided this info key with the same value.
10. Section 12.3.4 on page 570, and MPI-3.0 Section 11.3.4 on page 424.
Origin buffer arguments to `MPI_GET_ACCUMULATE` are ignored when the `MPI_NO_OP` operation is used.
 11. Section 12.3.4 on page 570, and MPI-3.0 Section 11.3.4 on page 424.
Clarify the roles of origin, result, and target communication parameters in `MPI_GET_ACCUMULATE`.
 12. Section 16.3 on page 738, and MPI-3.0 Section 14.3 on page 561
New paragraph and advice to users clarifying intent of variable names in the tools information interface.
 13. Section 16.3.3 on page 741, and MPI-3.0 Section 14.3.3 on page 563.
New paragraph clarifying variable name equivalence in the tools information interface.
 14. Sections 16.3.6, 16.3.7, and 16.3.9 on pages 745, 752, and 779, and
MPI-3.0 Sections 14.3.6, 14.3.7, and 14.3.8 on pages 567, 573, and 584.
In functions `MPI_T_CVAR_GET_INFO`, `MPI_T_PVAR_GET_INFO`, and
`MPI_T_CATEGORY_GET_INFO`, clarification of parameters that must be identical
for equivalent control variable / performance variable / category names across con-
nected processes.
 15. Section 16.3.7 on page 752, and MPI-3.0 Section 14.3.7 on page 573.
Clarify return code of `MPI_T_PVAR_{START,STOP,RESET}` routines.
 16. Section 16.3.7 on page 752, and MPI-3.0 Section 14.3.7 on page 579, line 7.
Clarify the return code when bad handle is passed to an `MPI_T_PVAR_*` routine.
 17. Section 21.1.4 on page 817, and MPI-3.0 Section 17.1.4 on page 603.
The advice to implementors at the end of the section was rewritten and moved into
the following section.
 18. Section 21.1.5 on page 818, and MPI-3.0 Section 17.1.5 on page 605.
The section was fully rewritten. The linker name concept was substituted by defining
specific procedure names.
 19. Section 21.1.6 on page 823, and MPI-3.0 Section 17.1.6 on page 611.
The requirements on `BIND(C)` procedure interfaces were removed.
 20. Annexes A.3, A.4, and A.5 on pages 926, 963, and 1052, and
MPI-3.0 Annexes A.2, A.3, and A.4 on pages 685, 707, and 756.
The predefined callback `MPI_CONVERSION_FN_NULL` was added to all three an-
nexes.
 21. Annex A.4.5 on page 1005, and MPI-3.0 Annex A.3.4 on page 724.
In the `mpi_f08` binding of
`MPI_{COMM|TYPE|WIN}_{DUP|NULL_COPY|NULL_DELETE}_FN`, all
`INTENT(...)` information was removed.

B.5.2 Changes in MPI-3.1

1. Sections 2.6.4 and 5.1.5 on pages 27 and 148.
The use of the intrinsic operators “+” and “-” for absolute addresses is substituted by `MPI_AINT_ADD` and `MPI_AINT_DIFF`. In C, they can be implemented as macros.
2. Sections 9.1.1, 11.2.1, and 11.6 on pages 439, 480, and 513.
The routines `MPI_INITIALIZED`, `MPI_FINALIZED`, `MPI_QUERY_THREAD`, `MPI_IS_THREAD_MAIN`, `MPI_GET_VERSION`, and `MPI_GET_LIBRARY_VERSION` are callable from threads without restriction (in the sense of `MPI_THREAD_MULTIPLE`), irrespective of the actual level of thread support provided, in the case where the implementation supports threads.
3. Section 12.2.1 on page 546.
The “same_disp_unit” info key was added for use in RMA window creation routines.
4. Sections 15.4.2 and 15.4.3 on pages 672 and 679.
Added `MPI_FILE_IREAD_AT_ALL`, `MPI_FILE_IWRITE_AT_ALL`, `MPI_FILE_IREAD_ALL`, and `MPI_FILE_IWRITE_ALL`.
5. Sections 16.3.6, 16.3.7, and 16.3.9 on pages 745, 752, and 779.
Clarified that NULL parameters can be provided in `MPI_T_{CVAR|PVAR|CATEGORY}_GET_INFO` routines.
6. Sections 16.3.6, 16.3.7, 16.3.9, and 16.3.10 on pages 745, 752, 779, and 783.
New routines `MPI_T_CVAR_GET_INDEX`, `MPI_T_PVAR_GET_INDEX`, `MPI_T_CATEGORY_GET_INDEX`, were added to support retrieving indices of variables and categories. The error codes `MPI_T_ERR_INVALID` and `MPI_T_ERR_INVALID_NAME` were added to indicate invalid uses of the interface.

B.6 Changes from Version 2.2 to Version 3.0

B.6.1 Fixes to Errata in Previous Versions of MPI

1. Sections 2.6.2 and 2.6.3 on pages 25 and 26, and MPI-2.2 Section 2.6.2 on page 17, lines 41–42, Section 2.6.3 on page 18, lines 15–16, and Section 2.6.4 on page 18, lines 40–41.
This is an MPI-2 erratum: The scope for the reserved prefix `MPI_` and the C++ namespace `MPI` is now any name as originally intended in MPI-1.
2. Sections 3.2.2, 6.9.2, 15.5.2 Table 15.2, and Annex A.1.1 on pages 35, 226, 710, and 891, and MPI-2.2 Sections 3.2.2, 5.9.2, 13.5.2 Table 13.2, 16.1.16 Table 16.1, and Annex A.1.1 on pages 27, 164, 433, 472 and 513
This is an MPI-2.2 erratum: New named predefined datatypes `MPI_CXX_BOOL`, `MPI_CXX_FLOAT_COMPLEX`, `MPI_CXX_DOUBLE_COMPLEX`, and `MPI_CXX_LONG_DOUBLE_COMPLEX` were added in C and Fortran corresponding to the C++ types `bool`, `std::complex<float>`, `std::complex<double>`, and `std::complex<long double>`. These datatypes also correspond to the deprecated C++ predefined datatypes `MPI::BOOL`, `MPI::COMPLEX`, `MPI::DOUBLE_COMPLEX`, and `MPI::LONG_DOUBLE_COMPLEX`, which were removed in MPI-3.0. The nonstandard C++ types `Complex<...>` were substituted by the standard types `std::complex<...>`.

3. Sections 6.9.2 on pages 226 and MPI-2.2 Section 5.9.2, page 165, line 47.
This is an MPI-2.2 erratum: `MPI_C_COMPLEX` was added to the “Complex” reduction group.
4. Section 8.5.5 on page 394, and MPI-2.2, Section 7.5.5 on page 257, C++ interface on page 264, line 3.
This is an MPI-2.2 erratum: The argument `rank` was removed and `in/outdegree` are now defined as `int& indegree` and `int& outdegree` in the C++ interface of `MPI_DIST_GRAPH_NEIGHBORS_COUNT`.
5. Section 15.5.2, Table 15.2 on page 710, and MPI-2.2, Section 13.5.3, Table 13.2 on page 433.
This was an MPI-2.2 erratum: The `MPI_C_BOOL` “external32” representation is corrected to a 1-byte size.
6. MPI-2.2 Section 16.1.16 on page 471, line 45.
This is an MPI-2.2 erratum: The constant `MPI::_LONG_LONG` should be `MPI::LONG_LONG`.
7. Annex A.1.1 on page 891, Table “Optional datatypes (Fortran),” and MPI-2.2, Annex A.1.1, Table on page 517, lines 34, and 37–41.
This is an MPI-2.2 erratum: The C++ datatype handles `MPI::INTEGER16`, `MPI::REAL16`, `MPI::F_COMPLEX4`, `MPI::F_COMPLEX8`, `MPI::F_COMPLEX16`, `MPI::F_COMPLEX32` were added to the table.

B.6.2 Changes in MPI-3.0

1. Section 2.6.1 on page 24, Section 19.2 on page 808 and all other chapters.
The C++ bindings were removed from the standard. See errata in Section B.6.1 on page 1101 for the latest changes to the MPI C++ binding defined in MPI-2.2.
This change may affect backward compatibility.
2. Section 2.6.1 on page 24, Section 18.1 on page 797 and Section 19.1 on page 807.
The deprecated functions `MPI_TYPE_HVECTOR`, `MPI_TYPE_HINDEXED`, `MPI_TYPE_STRUCT`, `MPI_ADDRESS`, `MPI_TYPE_EXTENT`, `MPI_TYPE_LB`, `MPI_TYPE_UB`, `MPI_ERRHANDLER_CREATE` (and its callback function prototype `MPI_Handler_function`), `MPI_ERRHANDLER_SET`, `MPI_ERRHANDLER_GET`, the deprecated special datatype handles `MPI_LB`, `MPI_UB`, and the constants `MPI_COMBINER_HINDEXED_INTEGER`, `MPI_COMBINER_HVECTOR_INTEGER`, `MPI_COMBINER_STRUCT_INTEGER` were removed from the standard.
This change may affect backward compatibility.
3. Section 2.3 on page 12.
Clarified parameter usage for `IN` parameters. C bindings are now `const`-correct where backward compatibility is preserved.
4. Section 2.5.4 on page 21 and Section 8.5.4 on page 388.
The recommended C implementation value for `MPI_UNWEIGHTED` changed from `NULL` to `non-NULL`. An additional weight array constant (`MPI_WEIGHTS_EMPTY`) was introduced.

5. Section 2.5.4 on page 21 and Section 9.1.1 on page 439.
Added the new routine `MPI_GET_LIBRARY_VERSION` to query library specific versions, and the new constant `MPI_MAX_LIBRARY_VERSION_STRING`.
6. Sections 2.5.8, 3.2.2, 3.3, 6.9.2, on pages 23, 35, 37, 226, Sections 5.1, 5.1.7, 5.1.8, 5.1.11, 18.4 on pages 127, 153, 154, 158, 805, and Annex A.1.1 on page 891.
New inquiry functions, `MPI_TYPE_SIZE_X`, `MPI_TYPE_GET_EXTENT_X`, `MPI_TYPE_GET_TRUE_EXTENT_X`, and `MPI_GET_ELEMENTS_X`, return their results as an `MPI_Count` value, which is a new type large enough to represent element counts in memory, file views, etc. A new function, `MPI_STATUS_SET_ELEMENTS_X`, modifies the opaque part of an `MPI_Status` object so that a call to `MPI_GET_ELEMENTS_X` returns the provided `MPI_Count` value (in Fortran, `INTEGER(KIND=MPI_COUNT_KIND)`). The corresponding predefined datatype is `MPI_COUNT`.
7. Chapter 3 on page 33 through Chapter 21 on page 811.
In the C language bindings, the array-arguments' interfaces were modified to consistently use `[]` instead of `*`.
Exceptions are `MPI_INIT`, which continues to use `char ***argv` (correct because of subtle rules regarding the use of the `&` operator with `char *argv[]`), and `MPI_INIT_THREAD`, which is changed to be consistent with `MPI_INIT`.
8. Sections 3.2.5, 5.1.5, 5.1.11, 5.2 on pages 40, 148, 158, 177.
The functions `MPI_GET_COUNT` and `MPI_GET_ELEMENTS` were defined to set the count argument to `MPI_UNDEFINED` when that argument would overflow. The functions `MPI_PACK_SIZE` and `MPI_TYPE_SIZE` were defined to set the size argument to `MPI_UNDEFINED` when that argument would overflow. In all other MPI-2.2 routines, the type and semantics of the count arguments remain unchanged, i.e., `int` or `INTEGER`.
9. Section 3.2.6 on page 44, and Section 3.8 on page 95.
`MPI_STATUS_IGNORE` can also be used in `MPI_IPROBE`, `MPI_PROBE`, `MPI_IMPROBE`, and `MPI_MPROBE`.
10. Section 3.8 on page 95 and Section 3.10 on page 112.
The use of `MPI_PROC_NULL` in probe operations was clarified. A special predefined message `MPI_MESSAGE_NO_PROC` was defined for the use of matching probe (i.e., the new `MPI_MPROBE` and `MPI_IMPROBE`) with `MPI_PROC_NULL`.
11. Sections 3.8.2, 3.8.3, 21.3.4, A.1.1 on pages 99, 101, 860, 891.
Like `MPI_PROBE` and `MPI_IPROBE`, the new `MPI_MPROBE` and `MPI_IMPROBE` operations allow incoming messages to be queried without actually receiving them, except that `MPI_MPROBE` and `MPI_IMPROBE` provide a mechanism to receive the specific message with the new routines `MPI_MRECV` and `MPI_IMRECV` regardless of other intervening probe or receive operations. The opaque object `MPI_Message`, the null handle `MPI_MESSAGE_NULL`, and the conversion functions `MPI_Message_c2f` and `MPI_Message_f2c` were defined.
12. Section 5.1.2 on page 129 and Section 5.1.13 on page 162.
The routine `MPI_TYPE_CREATE_HINDEXED_BLOCK` and constant `MPI_COMBINER_HINDEXED_BLOCK` were added.

- 1 13. Chapter 6 on page 189 and Section 6.12 on page 250.
2 Added nonblocking interfaces to all collective operations.
- 3
- 4 14. Sections 7.4.2, 7.4.4, 12.2.7, on pages 321, 339, 562.
5 The new routines `MPI_COMM_DUP_WITH_INFO`, `MPI_COMM_SET_INFO`,
6 `MPI_COMM_GET_INFO`, `MPI_WIN_SET_INFO`, and `MPI_WIN_GET_INFO` were
7 added. The routine `MPI_COMM_DUP` must also duplicate info hints.
- 8
- 9 15. Section 7.4.2 on page 321.
10 Added `MPI_COMM_IDUP`.
- 11
- 12 16. Section 7.4.2 on page 321.
13 Added the new communicator construction routine `MPI_COMM_CREATE_GROUP`,
14 which is invoked only by the processes in the group of the new communicator being
15 constructed.
- 16
- 17 17. Section 7.4.2 on page 321.
18 Added the `MPI_COMM_SPLIT_TYPE` routine and the communicator split type con-
19 stant `MPI_COMM_TYPE_SHARED`.
- 20
- 21 18. Section 7.6.2 on page 352.
22 In MPI-2.2, communication involved in an `MPI_INTERCOMM_CREATE` operation
23 could interfere with point-to-point communication on the parent communicator with
24 the same tag or `MPI_ANY_TAG`. This interference has been removed in MPI-3.0.
- 25
- 26 19. Section 7.8 on page 374.
27 Section 6.8 on page 238. The constant `MPI_MAX_OBJECT_NAME` also applies for type
28 and window names.
- 29
- 30 20. Section 8.5.8 on page 405.
31 `MPI_CART_MAP` can also be used for a zero-dimensional topologies.
- 32
- 33 21. Section 8.6 on page 407 and Section 8.7 on page 419.
34 The following neighborhood collective communication routines were added to sup-
35 port sparse communication on virtual topology grids: `MPI_NEIGHBOR_ALLGATHER`,
36 `MPI_NEIGHBOR_ALLGATHERV`, `MPI_NEIGHBOR_ALLTOALL`,
37 `MPI_NEIGHBOR_ALLTOALLV`, `MPI_NEIGHBOR_ALLTOALLW` and the nonblocking
38 variants `MPI_INEIGHBOR_ALLGATHER`, `MPI_INEIGHBOR_ALLGATHERV`,
39 `MPI_INEIGHBOR_ALLTOALL`, `MPI_INEIGHBOR_ALLTOALLV`, and
40 `MPI_INEIGHBOR_ALLTOALLW`. The displacement arguments in
41 `MPI_NEIGHBOR_ALLTOALLW` and `MPI_INEIGHBOR_ALLTOALLW` were defined as
42 address size integers. In `MPI_DIST_GRAPH_NEIGHBORS`, an ordering rule was
43 added for communicators created with `MPI_DIST_GRAPH_CREATE_ADJACENT`.
- 44
- 45 22. Section 11.2.1 on page 480 and Section 11.2.1 on page 483.
46 The use of `MPI_INIT`, `MPI_INIT_THREAD` and `MPI_FINALIZE` was clarified. After
47 MPI is initialized, the application can access information about the execution envi-
48 ronment by querying the new predefined info object `MPI_INFO_ENV`.
23. Section 11.2.1 on page 480.
Allow calls to `MPI_T` routines before `MPI_INIT` and after `MPI_FINALIZE`.

24. Chapter 12 on page 545.

Substantial revision of the entire One-sided chapter, with new routines for window creation, additional synchronization methods in passive target communication, new one-sided communication routines, a new memory model, and other changes.
25. Section 16.3 on page 738.

A new MPI Tool Information Interface was added.

The following changes are related to the Fortran language support.
26. Section 2.3 on page 12, and Sections 21.1.1, 21.1.2, 21.1.7 on pages 811, 812, and 826.

The new `mpi_f08` Fortran module was introduced.
27. Section 2.5.1 on page 19, and Sections 21.1.2, 21.1.3, 21.1.7 on pages 812, 815, and 826.

Handles to opaque objects were defined as named types within the `mpi_f08` Fortran module. The operators `.EQ.`, `.NE.`, `==`, and `/=` were overloaded to allow the comparison of these handles. The handle types and the overloaded operators are also available through the `mpi` Fortran module.
28. Sections 2.5.4, 2.5.5 on pages 21, 22, Sections 21.1.1, 21.1.10, 21.1.11, 21.1.12, 21.1.13 on pages 811, 837, 838, 839, 842, and Sections 21.1.2, 21.1.3, 21.1.7 on pages 812, 815, 826.

Within the `mpi_f08` Fortran module, choice buffers were defined as assumed-type and assumed-rank according to Fortran 2008 with TS 29113 [47], and the compile-time constant `MPI_SUBARRAYS_SUPPORTED` was set to `.TRUE.`. With this, Fortran subscript triplets can be used in nonblocking MPI operations; vector subscripts are not supported in nonblocking operations. If the compiler does not support this Fortran TS 29113 feature, the constant is set to `.FALSE.`.
29. Section 2.6.2 on page 25, Section 21.1.2 on page 812, and Section 21.1.7 on page 826.

The `ierror` dummy arguments are `OPTIONAL` within the `mpi_f08` Fortran module.
30. Section 3.2.5 on page 40, Sections 21.1.2, 21.1.3, 21.1.7, on pages 812, 815, 826, and Section 21.3.5 on page 861.

Within the `mpi_f08` Fortran module, the status was defined as `TYPE(MPI_Status)`. Additionally, within both the `mpi` and the `mpi_f08` modules, the constants `MPI_STATUS_SIZE`, `MPI_SOURCE`, `MPI_TAG`, `MPI_ERROR`, and `TYPE(MPI_Status)` are defined. New conversion routines were added: `MPI_STATUS_F2F08`, `MPI_STATUS_F082F`, `MPI_Status_c2f08`, and `MPI_Status_f082c`. In `mpi.h`, the new type `MPI_F08_status`, and the external variables `MPI_F08_STATUS_IGNORE` and `MPI_F08_STATUSES_IGNORE` were added.
31. Section 3.6 on page 60.

In Fortran with the `mpi` module or `mpif.h`, the type of the `buffer_addr` argument of `MPI_BUFFER_DETACH` is incorrectly defined and the argument is therefore unused.
32. Section 5.1 on page 127, Section 5.1.6 on page 151, and Section 21.1.15 on page 843.

The Fortran alignments of basic datatypes within Fortran derived types are implementation dependent; therefore it is recommended to use the `BIND(C)` attribute for derived types in MPI communication buffers. If an array of structures (in C/C++)

or derived types (in Fortran) is to be used in MPI communication buffers, it is recommended that the user creates a portable datatype handle and additionally applies `MPI_TYPE_CREATE_RESIZED` to this datatype handle.

33. Sections 5.1.10, 6.9.5, 6.9.7, 7.7.4, 7.8, 9.3.1, 9.3.2, 9.3.3, 18.1, 21.1.9 on pages 157, 235, 241, 369, 374, 451, 453, 455, 797, and 829. In some routines, the dummy argument names were changed because they were identical to the Fortran keywords `TYPE` and `FUNCTION`. The new dummy argument names must be used because the `mpi` and `mpi_f08` modules guarantee keyword-based actual argument lists. The argument name `type` was changed in `MPI_TYPE_DUP`, the Fortran `USER_FUNCTION` of `MPI_OP_CREATE`, `MPI_TYPE_SET_ATTR`, `MPI_TYPE_GET_ATTR`, `MPI_TYPE_DELETE_ATTR`, `MPI_TYPE_SET_NAME`, `MPI_TYPE_GET_NAME`, `MPI_TYPE_MATCH_SIZE`, the callback prototype definition `MPI_Type_delete_attr_function`, and the predefined callback function `MPI_TYPE_NULL_DELETE_FN`; function was changed in `MPI_OP_CREATE`, `MPI_COMM_CREATE_ERRHANDLER`, `MPI_WIN_CREATE_ERRHANDLER`, `MPI_FILE_CREATE_ERRHANDLER`, and `MPI_ERRHANDLER_CREATE`. For consistency reasons, `INOUBUF` was changed to `INOUTBUF` in `MPI_REDUCE_LOCAL`, and `intracomm` to `newintracomm` in `MPI_INTERCOMM_MERGE`.
34. Section 7.7.2 on page 360.
It was clarified that in Fortran, the flag values returned by a `comm_copy_attr_fn` callback, including `MPI_COMM_NULL_COPY_FN` and `MPI_COMM_DUP_FN`, are `.FALSE.` and `.TRUE.`; see `MPI_COMM_CREATE_KEYVAL`.
35. Section 9.2 on page 445.
With the `mpi` and `mpi_f08` Fortran modules, `MPI_ALLOC_MEM` now also supports `TYPE(C_PTR)` C-pointers instead of only returning an address-sized integer that may be usable together with a nonstandard Cray-pointer.
36. Section 21.1.15 on page 843, and Section 21.1.7 on page 826.
Fortran `SEQUENCE` and `BIND(C)` derived application types can now be used as buffers in MPI operations.
37. Section 21.1.16 on page 844 to Section 21.1.19 on page 855, Section 21.1.7 on page 826, and Section 21.1.8 on page 828.
The sections about Fortran optimization problems and their solutions were partially rewritten and new methods are added, e.g., the use of the `ASYNCHRONOUS` attribute. The constant `MPI_ASYNC_PROTECTS_NONBLOCKING` tells whether the semantics of the `ASYNCHRONOUS` attribute is extended to protect nonblocking operations. The Fortran routine `MPI_F_SYNC_REG` is added. MPI-3.0 compliance for an MPI library together with a Fortran compiler is defined in Section 21.1.7.
38. Section 21.1.2 on page 812.
Within the `mpi_f08` Fortran module, dummy arguments are now declared with `INTENT=IN`, `OUT`, or `INOUT` as defined in the `mpi_f08` interfaces.
39. Section 21.1.3 on page 815, and Section 21.1.7 on page 826.
The existing `mpi` Fortran module must implement compile-time argument checking.

40. Section 21.1.4 on page 817.
The use of the `mpif.h` Fortran include file is now strongly discouraged.
41. Section A.1.1, Table **Predefined functions** on page 903, Section A.1.3 on page 908, and Section A.4.5 on page 1005.
Within the new `mpi_f08` module, all callback prototype definitions are now defined with explicit interfaces `PROCEDURE(MPI_...)` that have the `BIND(C)` attribute; user-written callbacks must be modified if the `mpi_f08` module is used.
42. Section A.1.3 on page 908.
In some routines, the Fortran callback prototype names were changed from `..._FN` to `..._FUNCTION` to be consistent with the other language bindings.

B.7 Changes from Version 2.1 to Version 2.2

1. Section 2.5.4 on page 21.
It is now guaranteed that predefined named constant handles (as other constants) can be used in initialization expressions or assignments, i.e., also before the call to `MPI_INIT`.
2. Section 2.6 on page 24, and Section 19.2 on page 808.
The C++ language bindings have been deprecated and may be removed in a future version of the MPI specification.
3. Section 3.2.2 on page 35.
`MPI_CHAR` for printable characters is now defined for C type `char` (instead of signed `char`). This change should not have any impact on applications nor on MPI libraries (except some comment lines), because printable characters could and can be stored in any of the C types `char`, signed `char`, and unsigned `char`, and `MPI_CHAR` is not allowed for predefined reduction operations.
4. Section 3.2.2 on page 35.
`MPI_(U)INT{8,16,32,64}_T`, `MPI_AINT`, `MPI_OFFSET`, `MPI_C_BOOL`, `MPI_C_COMPLEX`, `MPI_C_FLOAT_COMPLEX`, `MPI_C_DOUBLE_COMPLEX`, and `MPI_C_LONG_DOUBLE_COMPLEX` are now valid predefined MPI datatypes.
5. Section 3.4 on page 52, Section 3.7.2 on page 73, Section 3.9 on page 105, and Section 6.1 on page 189.
The read access restriction on the send buffer for blocking, non blocking and collective API has been lifted. It is permitted to access for read the send buffer while the operation is in progress.
6. Section 3.7 on page 71.
The Advice to users for `IBSEND` and `IRSEND` was slightly changed.
7. Section 3.7.3 on page 80.
The advice to free an active request was removed in the Advice to users for `MPI_REQUEST_FREE`.
8. Section 3.7.6 on page 92.
`MPI_REQUEST_GET_STATUS` changed to permit inactive or null requests as input.

- 1 9. Section 6.8 on page 218.
2 “In place” option is added to `MPI_ALLTOALL`, `MPI_ALLTOALLV`, and
3 `MPI_ALLTOALLW` for intra-communicators.
4
- 5 10. Section 6.9.2 on page 226.
6 Predefined parameterized datatypes (e.g., returned by
7 `MPI_TYPE_CREATE_F90_REAL`) and optional named predefined datatypes (e.g.
8 `MPI_REAL8`) have been added to the list of valid datatypes in reduction operations.
9
- 10 11. Section 6.9.2 on page 226.
11 `MPI_(U)INT{8,16,32,64}_T` are all considered C integer types for the purposes of the
12 predefined reduction operators. `MPI_AINT` and `MPI_OFFSET` are considered Fortran
13 integer types. `MPI_C_BOOL` is considered a Logical type.
14 `MPI_C_COMPLEX`, `MPI_C_FLOAT_COMPLEX`, `MPI_C_DOUBLE_COMPLEX`, and
15 `MPI_C_LONG_DOUBLE_COMPLEX` are considered Complex types.
16
- 17 12. Section 6.9.7 on page 241.
18 The local routines `MPI_REDUCE_LOCAL` and `MPI_OP_COMMUTATIVE` have been
19 added.
20
- 21 13. Section 6.10.1 on page 243.
22 The collective function `MPI_REDUCE_SCATTER_BLOCK` is added to the MPI stan-
23 dard.
24
- 25 14. Section 6.11.2 on page 248.
26 Added in place argument to `MPI_EXSCAN`.
27
- 28 15. Section 7.4.2 on page 321, and Section 7.6 on page 348.
29 Implementations that did not implement `MPI_COMM_CREATE` on inter-communi-
30 cators will need to add that functionality. As the standard described the behav-
31 ior of this operation on inter-communicators, it is believed that most implementa-
32 tions already provide this functionality. Note also that the C++ binding for both
33 `MPI_COMM_CREATE` and `MPI_COMM_SPLIT` explicitly allow Intercomms.
34
- 35 16. Section 7.4.2 on page 321.
36 `MPI_COMM_CREATE` is extended to allow several disjoint subgroups as input if
37 comm is an intra-communicator. If comm is an inter-communicator it was clarified
38 that all processes in the same local group of comm must specify the same value for
39 group.
40
- 41 17. Section 8.5.4 on page 388.
42 New functions for a scalable distributed graph topology interface has been added. In
43 this section, the functions `MPI_DIST_GRAPH_CREATE_ADJACENT` and
44 `MPI_DIST_GRAPH_CREATE`, the constants `MPI_UNWEIGHTED`, and the derived C++
45 class `Distgraphcomm` were added.
46
- 47 18. Section 8.5.5 on page 394.
48 For the scalable distributed graph topology interface, the functions
`MPI_DIST_GRAPH_NEIGHBORS_COUNT` and `MPI_DIST_GRAPH_NEIGHBORS`
and the constant `MPI_DIST_GRAPH` were added.

19. Section 8.5.5 on page 394.
Remove ambiguity regarding duplicated neighbors with `MPI_GRAPH_NEIGHBORS` and `MPI_GRAPH_NEIGHBORS_COUNT`.
20. Section 9.1.1 on page 439.
The subversion number changed from 1 to 2.
21. Section 9.3 on page 448, Section 18.2 on page 800, and Annex A.1.3 on page 908.
Changed function pointer typedef names `MPI_{Comm,File,Win}_errhandler_fn` to `MPI_{Comm,File,Win}_errhandler_function`. Deprecated old “_fn” names.
22. Section 11.2.4 on page 490.
Attribute deletion callbacks on `MPI_COMM_SELF` are now called in LIFO order. Implementors must now also register all implementation-internal attribute deletion callbacks on `MPI_COMM_SELF` before returning from `MPI_INIT/MPI_INIT_THREAD`.
23. Section 12.3.4 on page 570.
The restriction added in MPI 2.1 that the operation `MPI_REPLACE` in `MPI_ACCUMULATE` can be used only with predefined datatypes has been removed. `MPI_REPLACE` can now be used even with derived datatypes, as it was in MPI 2.0. Also, a clarification has been made that `MPI_REPLACE` can be used only in `MPI_ACCUMULATE`, not in collective operations that do reductions, such as `MPI_REDUCE` and others.
24. Section 14.2 on page 643.
Add “*” to the `query_fn`, `free_fn`, and `cancel_fn` arguments to the C++ binding for `MPI::Grequest::Start()` for consistency with the rest of MPI functions that take function pointer arguments.
25. Section 15.5.2 on page 708, and Table 15.2 on page 710.
`MPI_(U)INT{8,16,32,64}_T`, `MPI_AINT`, `MPI_OFFSET`, `MPI_C_COMPLEX`, `MPI_C_FLOAT_COMPLEX`, `MPI_C_DOUBLE_COMPLEX`, `MPI_C_LONG_DOUBLE_COMPLEX`, and `MPI_C_BOOL` are added as predefined datatypes in the “external32” representation.
26. Section 21.3.7 on page 867.
The description was modified that it only describes how an MPI implementation behaves, but not how MPI stores attributes internally. The erroneous MPI-2.1 Example 16.17 was replaced with three new examples 21.28, 21.29, and 21.30 on pages 867–869 explicitly detailing cross-language attribute behavior. Implementations that matched the behavior of the old example will need to be updated.
27. Annex A.1.1 on page 891.
Removed type `MPI::Fint` (compare `MPI_Fint` in Section A.1.2 on page 907).
28. Annex A.1.1 on page 891. Table **Named Predefined Datatypes**.
Added `MPI_(U)INT{8,16,32,64}_T`, `MPI_AINT`, `MPI_OFFSET`, `MPI_C_BOOL`, `MPI_C_FLOAT_COMPLEX`, `MPI_C_COMPLEX`, `MPI_C_DOUBLE_COMPLEX`, and `MPI_C_LONG_DOUBLE_COMPLEX` are added as predefined datatypes.

B.8 Changes from Version 2.0 to Version 2.1

1. Section 3.2.2 on page 35, and Annex A.1 on page 891.
In addition, the `MPI_LONG_LONG` should be added as an optional type; it is a synonym for `MPI_LONG_LONG_INT`.
2. Section 3.2.2 on page 35, and Annex A.1 on page 891.
`MPI_LONG_LONG_INT`, `MPI_LONG_LONG` (as synonym),
`MPI_UNSIGNED_LONG_LONG`, `MPI_SIGNED_CHAR`, and `MPI_WCHAR` are moved from optional to official and they are therefore defined for all three language bindings.
3. Section 3.2.5 on page 40.
`MPI_GET_COUNT` with zero-length datatypes: The value returned as the count argument of `MPI_GET_COUNT` for a datatype of length zero where zero bytes have been transferred is zero. If the number of bytes transferred is greater than zero, `MPI_UNDEFINED` is returned.
4. Section 5.1 on page 127.
General rule about derived datatypes: Most datatype constructors have replication count or block length arguments. Allowed values are nonnegative integers. If the value is zero, no elements are generated in the type map and there is no effect on datatype bounds or extent.
5. Section 5.3 on page 184.
`MPI_BYTE` should be used to send and receive data that is packed using `MPI_PACK_EXTERNAL`.
6. Section 6.9.6 on page 240.
If `comm` is an inter-communicator in `MPI_ALLREDUCE`, then both groups should provide count and datatype arguments that specify the same type signature (i.e., it is not necessary that both groups provide the same count value).
7. Section 7.3.1 on page 310.
`MPI_GROUP_TRANSLATE_RANKS` and `MPI_PROC_NULL`:
`MPI_PROC_NULL` is a valid rank for input to `MPI_GROUP_TRANSLATE_RANKS`, which returns `MPI_PROC_NULL` as the translated rank.
8. Section 7.7 on page 358.
About the attribute caching functions:
Advice to implementors. High-quality implementations should raise an error when a keyval that was created by a call to `MPI_XXX_CREATE_KEYVAL` is used with an object of the wrong type with a call to `MPI_YYY_GET_ATTR`, `MPI_YYY_SET_ATTR`, `MPI_YYY_DELETE_ATTR`, or `MPI_YYY_FREE_KEYVAL`. To do so, it is necessary to maintain, with each keyval, information on the type of the associated user function. (*End of advice to implementors.*)
9. Section 7.8 on page 374.
In `MPI_COMM_GET_NAME`: In C, a null character is additionally stored at

name[resultlen]. resultlen cannot be larger than `MPI_MAX_OBJECT_NAME-1`. In Fortran, name is padded on the right with blank characters. resultlen cannot be larger than `MPI_MAX_OBJECT_NAME`.

10. Section 8.4 on page 383.

About `MPI_GRAPH_CREATE` and `MPI_CART_CREATE`: All input arguments must have identical values on all processes of the group of `comm_old`.

11. Section 8.5.1 on page 384.

In `MPI_CART_CREATE`: If `ndims` is zero then a zero-dimensional Cartesian topology is created. The call is erroneous if it specifies a grid that is larger than the group size or if `ndims` is negative.

12. Section 8.5.3 on page 386.

In `MPI_GRAPH_CREATE`: If the graph is empty, i.e., `nnodes = 0`, then `MPI_COMM_NULL` is returned in all processes.

13. Section 8.5.3 on page 386.

In `MPI_GRAPH_CREATE`: A single process is allowed to be defined multiple times in the list of neighbors of a process (i.e., there may be multiple edges between two processes). A process is also allowed to be a neighbor to itself (i.e., a self loop in the graph). The adjacency matrix is allowed to be nonsymmetric.

Advice to users. Performance implications of using multiple edges or a nonsymmetric adjacency matrix are not defined. The definition of a node-neighbor edge does not imply a direction of the communication. (*End of advice to users.*)

14. Section 8.5.5 on page 394.

In `MPI_CARTDIM_GET` and `MPI_CART_GET`: If `comm` is associated with a zero-dimensional Cartesian topology, `MPI_CARTDIM_GET` returns `ndims=0` and `MPI_CART_GET` will keep all output arguments unchanged.

15. Section 8.5.5 on page 394.

In `MPI_CART_RANK`: If `comm` is associated with a zero-dimensional Cartesian topology, `coord` is not significant and 0 is returned in `rank`.

16. Section 8.5.5 on page 394.

In `MPI_CART_COORDS`: If `comm` is associated with a zero-dimensional Cartesian topology, `coords` will be unchanged.

17. Section 8.5.6 on page 403.

In `MPI_CART_SHIFT`: It is erroneous to call `MPI_CART_SHIFT` with a direction that is either negative or greater than or equal to the number of dimensions in the Cartesian communicator. This implies that it is erroneous to call `MPI_CART_SHIFT` with a `comm` that is associated with a zero-dimensional Cartesian topology.

18. Section 8.5.7 on page 404.

In `MPI_CART_SUB`: If all entries in `remain_dims` are false or `comm` is already associated with a zero-dimensional Cartesian topology then `newcomm` is associated with a zero-dimensional Cartesian topology.

- 18.1. Section 9.1.1 on page 439.

The subversion number changed from 0 to 1.

- 1 19. Section 9.1.2 on page 440.
 2 In `MPI_GET_PROCESSOR_NAME`: In C, a null character is additionally stored at
 3 `name[resultlen]`. `resultlen` cannot be larger than `MPI_MAX_PROCESSOR_NAME-1`. In
 4 Fortran, `name` is padded on the right with blank characters. `resultlen` cannot be larger
 5 than `MPI_MAX_PROCESSOR_NAME`.
 6
- 7 20. Section 9.3 on page 448.
 8 `MPI_{COMM,WIN,FILE}_GET_ERRHANDLER` behave as if a new error handler ob-
 9 ject is created. That is, once the error handler is no longer needed,
 10 `MPI_ERRHANDLER_FREE` should be called with the error handler returned from
 11 `MPI_ERRHANDLER_GET` or `MPI_{COMM,WIN,FILE}_GET_ERRHANDLER` to mark
 12 the error handler for deallocation. This provides behavior similar to that of
 13 `MPI_COMM_GROUP` and `MPI_GROUP_FREE`.
 14
- 15 21. Section 11.2.1 on page 480, see explanations to `MPI_FINALIZE`.
 16 `MPI_FINALIZE` is collective over all connected processes. If no processes were spawned,
 17 accepted or connected then this means over `MPI_COMM_WORLD`; otherwise it is col-
 18 lective over the union of all processes that have been and continue to be connected,
 19 as explained in Section 11.10.4 on page 540.
 20
- 21 22. Section 11.2.1 on page 480.
 22 About `MPI_ABORT`:
 23 *Advice to users.* Whether the errorcode is returned from the executable or from
 24 the MPI process startup mechanism (e.g., `mpiexec`), is an aspect of quality of the
 25 MPI library but not mandatory. (*End of advice to users.*)
 26 *Advice to implementors.* Where possible, a high-quality implementation will try
 27 to return the errorcode from the MPI process startup mechanism (e.g. `mpiexec`
 28 or singleton `init`). (*End of advice to implementors.*)
 29
- 30 23. Section 10 on page 471.
 31 An implementation must support info objects as caches for arbitrary (key, value) pairs,
 32 regardless of whether it recognizes the key. Each function that takes hints in the form
 33 of an `MPI_Info` must be prepared to ignore any key it does not recognize. This de-
 34 scription of info objects does not attempt to define how a particular function should
 35 react if it recognizes a key but not the associated value. `MPI_INFO_GET_NKEYS`,
 36 `MPI_INFO_GET_NTHKEY`, `MPI_INFO_GET_VALUELEN`, and
 37 `MPI_INFO_GET` must retain all (key,value) pairs so that layered functionality can
 38 also use the `Info` object.
 39
- 40 24. Section 12.3 on page 564.
 41 `MPI_PROC_NULL` is a valid target rank in the MPI RMA calls `MPI_ACCUMULATE`,
 42 `MPI_GET`, and `MPI_PUT`. The effect is the same as for `MPI_PROC_NULL` in MPI
 43 point-to-point communication. See also item 25 in this list.
 44
- 45 25. Section 12.3 on page 564.
 46 After any RMA operation with rank `MPI_PROC_NULL`, it is still necessary to finish
 47 the RMA epoch with the synchronization method that started the epoch. See also
 48 item 24 in this list.

26. Section 12.3.4 on page 570.
`MPI_REPLACE` in `MPI_ACCUMULATE`, like the other predefined operations, is defined only for the predefined MPI datatypes.
27. Section 15.2.8 on page 663.
 About `MPI_FILE_SET_VIEW` and `MPI_FILE_SET_INFO`: When an info object that specifies a subset of valid hints is passed to `MPI_FILE_SET_VIEW` or `MPI_FILE_SET_INFO`, there will be no effect on previously set or defaulted hints that the info does not specify.
28. Section 15.2.8 on page 663.
 About `MPI_FILE_GET_INFO`: If no hint exists for the file associated with `fh`, a handle to a newly created info object is returned that contains no key/value pair.
29. Section 15.3 on page 666.
 If a file does not have the mode `MPI_MODE_SEQUENTIAL`, then `MPI_DISPLACEMENT_CURRENT` is invalid as `disp` in `MPI_FILE_SET_VIEW`.
30. Section 15.5.2 on page 708.
 The bias of 16 byte doubles was defined with 10383. The correct value is 16383.
31. MPI-2.2, Section 16.1.4 (Section was removed in MPI-3.0).
 In the example in this section, the buffer should be declared as `const void* buf`.
32. Section 21.1.9 on page 829.
 About `MPI_TYPE_CREATE_F90_XXX`:
Advice to implementors. An application may often repeat a call to `MPI_TYPE_CREATE_F90_XXX` with the same combination of `(XXX,p,r)`. The application is not allowed to free the returned predefined, unnamed datatype handles. To prevent the creation of a potentially huge amount of handles, the MPI implementation should return the same datatype handle for the same `(REAL/COMPLEX/INTEGER,p,r)` combination. Checking for the combination `(p,r)` in the preceding call to `MPI_TYPE_CREATE_F90_XXX` and using a hash-table to find formerly generated handles should limit the overhead of finding a previously generated datatype with same combination of `(XXX,p,r)`. (*End of advice to implementors.*)
33. Section A.1.1 on page 891.
`MPI_BOTTOM` is defined as `void * const MPI::BOTTOM`.

DRAFT

Bibliography

- [1] MPI Forum Documents. <https://www.mpi-forum.org/docs/>. Citation on page 7.
- [2] Reverse domain name notation convention. <https://docs.oracle.com/javase/tutorial/java/package/namingpkgs.html>. Citation on page 338.
- [3] V. Bala and S. Kipnis. Process groups: a mechanism for the coordination of and communication among processes in the Venus collective communication library. Technical report, IBM T. J. Watson Research Center, October 1992. Preprint. Citation on page 2.
- [4] V. Bala, S. Kipnis, L. Rudolph, and Marc Snir. Designing efficient, scalable, and portable collective communication libraries. Technical report, IBM T. J. Watson Research Center, October 1992. Preprint. Citation on page 2.
- [5] Purushotham V. Bangalore, Nathan E. Doss, and Anthony Skjellum. MPI++: Issues and features. In *OON-SKI '94*, page in press, 1994. Citation on page 305.
- [6] A. Beguelin, J. Dongarra, A. Geist, R. Manchek, and V. Sunderam. Visualization and debugging in a heterogeneous environment. *IEEE Computer*, 26(6):88–95, June 1993. Citation on page 2.
- [7] Luc Bomans and Rolf Hempel. The Argonne/GMD macros in FORTRAN for portable parallel programming and their implementation on the Intel iPSC/2. *Parallel Computing*, 15:119–132, 1990. Citation on page 2.
- [8] Dan Bonachea and Jason Duell. Problems with using MPI 1.1 and 2.0 as compilation targets for parallel language implementations. *IJHPCN*, 1(1/2/3):91–99, 2004. Citation on page 604.
- [9] Rajesh Bordawekar, Juan Miguel del Rosario, and Alok Choudhary. Design and evaluation of primitives for parallel I/O. In *Proceedings of Supercomputing '93*, pages 452–461, 1993. Citation on page 653.
- [10] R. Butler and E. Lusk. User’s guide to the p4 programming system. Technical Report TM-ANL-92/17, Argonne National Laboratory, 1992. Citation on page 2.
- [11] Ralph Butler and Ewing Lusk. Monitors, messages, and clusters: The p4 parallel programming system. *Parallel Computing*, 20(4):547–564, April 1994. Also Argonne National Laboratory Mathematics and Computer Science Division preprint P362-0493. Citation on page 2.
- [12] Robin Calkin, Rolf Hempel, Hans-Christian Hoppe, and Peter Wypior. Portable programming with the PARMACS message-passing library. *Parallel Computing*, 20(4):615–632, April 1994. Citation on page 2.

- [13] S. Chittor and R. J. Enbody. Performance evaluation of mesh-connected wormhole-routed networks for interprocessor communication in multicomputers. In *Proceedings of the 1990 Supercomputing Conference*, pages 647–656, 1990. Citation on page 381.
- [14] S. Chittor and R. J. Enbody. Predicting the effect of mapping on the communication performance of large multicomputers. In *Proceedings of the 1991 International Conference on Parallel Processing, vol. II (Software)*, pages II-1–II-4, 1991. Citation on page 381.
- [15] Parasoftware Corporation. Express version 1.0: A communication environment for parallel computers, 1988. Citation on page 2.
- [16] Yiannis Cotronis, Anthony Danalis, Dimitrios S. Nikolopoulos, and Jack Dongarra, editors. *Recent Advances in the Message Passing Interface - 18th European MPI Users' Group Meeting, EuroMPI 2011, Santorini, Greece, September 18-21, 2011. Proceedings*, volume 6960 of *Lecture Notes in Computer Science*. Springer, 2011. Citations on pages 1116 and 1118.
- [17] Juan Miguel del Rosario, Rajesh Bordawekar, and Alok Choudhary. Improved parallel I/O via a two-phase run-time access strategy. In *IPPS '93 Workshop on Input/Output in Parallel Computer Systems*, pages 56–70, 1993. Also published in *Computer Architecture News* 21(5), December 1993, pages 31–38. Citation on page 653.
- [18] James Dinan, Sriram Krishnamoorthy, Pavan Balaji, Jeff R. Hammond, Manojkumar Krishnan, Vinod Tipparaju, and Abhinav Vishnu. Noncollective communicator creation in MPI. In Cotronis et al. [16], pages 282–291. Citation on page 328.
- [19] J. Dongarra, A. Geist, R. Manchek, and V. Sunderam. Integrated PVM framework supports heterogeneous network computing. *Computers in Physics*, 7(2):166–75, April 1993. Citation on page 2.
- [20] J. J. Dongarra, R. Hempel, A. J. G. Hey, and D. W. Walker. A proposal for a user-level, message passing interface in a distributed memory environment. Technical Report TM-12231, Oak Ridge National Laboratory, February 1993. Citation on page 2.
- [21] Edinburgh Parallel Computing Centre, University of Edinburgh. *CHIMP Concepts*, June 1991. Citation on page 2.
- [22] Edinburgh Parallel Computing Centre, University of Edinburgh. *CHIMP Version 1.0 Interface*, May 1992. Citation on page 2.
- [23] D. Feitelson. Communicators: Object-based multiparty interactions for parallel programming. Technical Report 91-12, Dept. Computer Science, The Hebrew University of Jerusalem, November 1991. Citation on page 306.
- [24] Message Passing Interface Forum. MPI: A message-passing interface standard. *The International Journal of Supercomputer Applications and High Performance Computing*, 8, 1994. Citation on page 2.
- [25] Message Passing Interface Forum. MPI: A message-passing interface standard (version 1.1). Technical report, 1995. <http://www.mpi-forum.org>. Citation on page 2.

- [26] Al Geist, Adam Beguelin, Jack Dongarra, Weicheng Jiang, Bob Manchek, and Vaidy Sunderam. *PVM: Parallel Virtual Machine—A User’s Guide and Tutorial for Network Parallel Computing*. MIT Press, 1994. Citation on page 479.
- [27] G. A. Geist, M. T. Heath, B. W. Peyton, and P. H. Worley. PICL: A portable instrumented communications library, C reference manual. Technical Report TM-11130, Oak Ridge National Laboratory, Oak Ridge, TN, July 1990. Citation on page 2.
- [28] Brice Goglin, Emmanuel Jeannot, Farouk Mansouri, and Guillaume Mercier. Hardware topology management in MPI applications through hierarchical communicators. *Parallel Computing*, 76:70–90, 2018. Citation on page 337.
- [29] Ryan E. Grant, Matthew G. F. Dosanjh, Michael J. Levenhagen, Ron Brightwell, and Anthony Skjellum. Finepoints: Partitioned multithreaded MPI communication. In *ISC High Performance Conference (ISC)*, 2019. Citation on page 113.
- [30] Ryan E. Grant, Anthony Skjellum, and Purushotham V. Bangalore. Lightweight threading with MPI using persistent communications semantics. In *Workshop on Exascale MPI (ExaMPI)*. Held in conjunction with the 2015 International Conference for High Performance Computing, Networking, Storage and Analysis (SC15), 2015. Citation on page 113.
- [31] D. Gregor, T. Hoefer, B. Barrett, and A. Lumsdaine. Fixing probe for multi-threaded MPI applications. Technical Report 674, Indiana University, Jan. 2009. Citation on page 99.
- [32] William D. Gropp and Barry Smith. Chameleon parallel programming tools users manual. Technical Report ANL-93/23, Argonne National Laboratory, March 1993. Citation on page 2.
- [33] Michael Hennecke. A Fortran 90 interface to MPI version 1.1. Technical Report Internal Report 63/96, Rechenzentrum, Universität Karlsruhe, D-76128 Karlsruhe, Germany, June 1996. Citation on page 816.
- [34] T. Hoefer, G. Bronevetsky, B. Barrett, B. R. de Supinski, and A. Lumsdaine. Efficient MPI support for advanced hybrid programming models. In *Recent Advances in the Message Passing Interface (EuroMPI’10)*, volume LNCS 6305, pages 50–61. Springer, Sep. 2010. Citations on pages 98 and 99.
- [35] T. Hoefer, P. Gottschling, A. Lumsdaine, and W. Rehm. Optimizing a conjugate gradient solver with non-blocking collective operations. *Elsevier Journal of Parallel Computing (PARCO)*, 33(9):624–633, Sep. 2007. Citation on page 250.
- [36] T. Hoefer, F. Lorenzen, and A. Lumsdaine. Sparse non-blocking collectives in quantum mechanical calculations. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface, 15th European PVM/MPI Users’ Group Meeting*, volume LNCS 5205, pages 55–63. Springer, Sep. 2008. Citation on page 407.
- [37] T. Hoefer and A. Lumsdaine. Message progression in parallel computing — to thread or not to thread? In *Proceedings of the 2008 IEEE International Conference on Cluster Computing*. IEEE Computer Society, Oct. 2008. Citation on page 250.

- [38] T. Hoefer, A. Lumsdaine, and W. Rehm. Implementation and performance analysis of non-blocking collective operations for MPI. In *Proceedings of the 2007 International Conference on High Performance Computing, Networking, Storage and Analysis, SC07*. IEEE Computer Society/ACM, Nov. 2007. Citation on page 252.
- [39] T. Hoefer, M. Schellmann, S. Gorlatch, and A. Lumsdaine. Communication optimization for medical image reconstruction algorithms. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface, 15th European PVM/MPI Users' Group Meeting*, volume LNCS 5205, pages 75–83. Springer, Sep. 2008. Citation on page 250.
- [40] T. Hoefer and J. L. Träff. Sparse collective operations for MPI. In *Proceedings of the 23rd IEEE International Parallel & Distributed Processing Symposium, HIPS'09 Workshop*, May 2009. Citation on page 407.
- [41] Torsten Hoefer and Marc Snir. Writing parallel libraries with MPI — common practice, issues, and extensions. In Cotronis et al. [16], pages 345–355. Citation on page 324.
- [42] Daniel J. Holmes, Bradley Morgan, Anthony Skjellum, Purushotham V. Bangalore, and Srinivas Sridharan. Planning for performance: Enhancing achievable performance for MPI through persistent collective operations. *Parallel Computing*, 81:32 – 57, 2019. Citation on page 273.
- [43] Institute of Electrical and Electronics Engineers, New York. *IEEE Standard for Binary Floating-Point Arithmetic, IEEE Standard 754-2008*, 2008. Citations on pages 708 and 709.
- [44] International Organization for Standardization, Geneva. *ISO 8859-1:1987: Information processing — 8-bit single-byte coded graphic character sets — Part 1: Latin alphabet No. 1*, February 1987. Citation on page 709.
- [45] International Organization for Standardization, Geneva. *ISO/IEC 9945-1:1996: Information technology — Portable Operating System Interface (POSIX) — Part 1: System Application Program Interface (API) [C Language]*, December 1996. Citations on pages 513 and 657.
- [46] International Organization for Standardization, Geneva. *ISO/IEC 1539-1:2010: Information technology — Programming languages — Fortran — Part 1: Base language*, November 2010. Citations on pages 811 and 814.
- [47] International Organization for Standardization, Geneva. *ISO/IEC TS 29113:2012: Information technology — Further interoperability of Fortran with C*, December 2012. Citations on pages 811, 814, 827, 828, and 1105.
- [48] International Organization for Standardization, Geneva. *ISO/IEC 1539-1:2018: Information technology — Programming languages — Fortran — Part 1: Base language*, November 2018. Citations on pages 811, 812, and 814.
- [49] Charles H. Koelbel, David B. Loveman, Robert S. Schreiber, Guy L. Steele Jr., and Mary E. Zosel. *The High Performance Fortran Handbook*. MIT Press, 1993. Citation on page 143.

- [50] David Kotz. Disk-directed I/O for MIMD multiprocessors. In *Proceedings of the 1994 Symposium on Operating Systems Design and Implementation*, pages 61–74, November 1994. Updated as Dartmouth TR PCS-TR94-226 on November 8, 1994. Citation on page 653.
- [51] O. Krämer and H. Mühlenbein. Mapping strategies in message-based multiprocessor systems. *Parallel Computing*, 9:213–225, 1989. Citation on page 381.
- [52] S. J. Lefflet, R. S. Fabry, W. N. Joy, P. Lapsley, S. Miller, and C. Torek. An advanced 4.4BSD interprocess communication tutorial, Unix programmer’s supplementary documents (PSD) 21. Technical report, Computer Systems Research Group, Department of Electrical Engineering and Computer Science, University of California, Berkeley, 1993. Available online: <https://docs.freebsd.org/44doc/psd/21.ipc/paper.pdf>. Citation on page 543.
- [53] Bradley Morgan, Daniel J. Holmes, Anthony Skjellum, Purushotham Bangalore, and Srinivas Sridharan. Planning for performance: Persistent collective operations for MPI. In *Proceedings of the 24th European MPI Users’ Group Meeting, EuroMPI ’17*, pages 4:1–4:11, New York, NY, USA, 2017. ACM. Citation on page 273.
- [54] nCUBE Corporation. *nCUBE 2 Programmers Guide, v2.0*, December 1990. Citation on page 2.
- [55] Bill Nitzberg. Performance of the iPSC/860 Concurrent File System. Technical Report RND-92-020, NAS Systems Division, NASA Ames, December 1992. Citation on page 653.
- [56] William J. Nitzberg. *Collective Parallel I/O*. PhD thesis, Department of Computer and Information Science, University of Oregon, December 1995. Citation on page 653.
- [57] *4.4BSD Programmer’s Supplementary Documents (PSD)*. O’Reilly and Associates, 1994. Citation on page 543.
- [58] Paul Pierce. The NX/2 operating system. In *Proceedings of the Third Conference on Hypercube Concurrent Computers and Applications*, pages 384–390. ACM Press, 1988. Citation on page 2.
- [59] Martin Schulz and Bronis R. de Supinski. P^NMPI tools: A whole lot greater than the sum of their parts. In *ACM/IEEE Supercomputing Conference (SC)*, pages 1–10. ACM, 2007. Citation on page 737.
- [60] K. E. Seamons, Y. Chen, P. Jones, J. Jozwiak, and M. Winslett. Server-directed collective I/O in Panda. In *Proceedings of Supercomputing ’95*, December 1995. Citation on page 653.
- [61] A. Skjellum, N.E. Doss, K. Viswanathan, A. Chowdappa, and P.V. Bangalore. Extending the message passing interface (MPI). In *Proceedings Scalable Parallel Libraries Conference*, pages 106,107,108,109,110,111,112,113,114,115,116,117,118, Los Alamitos, CA, USA, October 1994. IEEE Computer Society. Citation on page 193.
- [62] A. Skjellum and A. Leung. Zipcode: a portable multicomputer communication library atop the reactive kernel. In D. W. Walker and Q. F. Stout, editors, *Proceedings of the*

- Fifth Distributed Memory Concurrent Computing Conference*, pages 767–776. IEEE Press, 1990. Citations on pages 2 and 306.
- [63] A. Skjellum, S. Smith, C. Still, A. Leung, and M. Morari. The Zipcode message passing system. Technical report, Lawrence Livermore National Laboratory, September 1992. Citation on page 2.
- [64] Anthony Skjellum, Nathan E. Doss, and Purushotham V. Bangalore. Writing libraries in MPI. In Anthony Skjellum and Donna S. Reese, editors, *Proceedings of the Scalable Parallel Libraries Conference*, pages 166–173. IEEE Computer Society Press, October 1993. Citation on page 305.
- [65] Anthony Skjellum, Steven G. Smith, Nathan E. Doss, Alvin P. Leung, and Manfred Morari. The design and evolution of Zipcode. *Parallel Computing*, 20(4):565–596, April 1994. Citations on pages 306 and 348.
- [66] The Internet Society. XDR: External data representation standard, May 2006. <http://www.rfc-editor.org/pdf/rfc4506.txt.pdf>. Citation on page 709.
- [67] Rajeev Thakur and Alok Choudhary. An extended two-phase method for accessing sections of out-of-core arrays. *Scientific Programming*, 5(4):301–317, Winter 1996. Citation on page 653.
- [68] Jesper Larsson Träff. SMP-aware message passing programming. In *Eighth International Workshop on High-level Parallel Programming Models and Supportive Environments (HIPS), 17th International Parallel and Distributed Processing Symposium (IPDPS)*, pages 56–65, 2003. Citation on page 337.
- [69] *The Unicode Standard, Version 2.0*. Addison-Wesley, 1996. ISBN 0-201-48345-9. Citation on page 709.
- [70] D. Walker. Standards for message passing in a distributed memory environment. Technical Report TM-12147, Oak Ridge National Laboratory, August 1992. Citation on page 2.

General Index

This index lists mainly terms of the MPI specification. The underlined page numbers refer to the definitions or parts of the definition of the terms. Bold face page numbers are used for entries appearing in a section or subsection title.

- !(_c), [857](#)
- _c, [23](#), [24](#), [235](#), [857](#)
- a priori enabled, [15](#)
- ABI, **873**
 - interoperability, **873**
- absolute addresses, [23](#), [148](#), [847](#)
- access epoch, [585](#)
- action
 - in function names, [12](#)
- active, [80](#), [81–83](#), [104](#), [110](#), [111](#), [379](#), [626](#), [628](#), [633](#), [637](#), [638](#)
 - operation state, [14](#), [1090](#)
- active target communication, [585](#)
- addresses, [161](#)
 - absolute, [23](#), [24](#), [148](#), [847](#)
 - correct use, [161](#)
 - relative displacement, [23](#), [148](#)
- alignment, [446](#), [551](#), [553](#), [1097](#)
- all-gather, **214**
 - nonblocking, **259**
 - persistent, **281**
- all-reduce, **240**
 - nonblocking, **267**
 - persistent, **290**
- all-to-all, **218**
 - nonblocking, **262**
 - persistent, **284**
- array arguments, **21**
- assertions, **600**
- ASYNCHRONOUS
 - Fortran attribute, **849**
- attribute, [306](#), [358](#), **867**
 - caching, [306](#)
- automatic buffering
 - buffer allocation, **60**
- barrier synchronization, **196**
 - nonblocking, **252**
 - persistent, **274**
- basic datatypes, [35](#)
 - additional host language, [36](#)
 - byte, [35](#)
 - C, [36](#)
 - C and Fortran, [37](#)
 - C++, [37](#)
 - Fortran, [35](#)
 - packed, [35](#)
- blocked
 - procedure invocation, [29](#)
- blocking, [17](#), [52](#), [56](#), [101](#), [670](#)
 - I/O, [670](#)
 - point-to-point, [34](#)
- blocking I/O, [670](#)
- blocking operation, [14](#)
- blocking procedure, [17](#)
- bound queue, [626](#), [628](#), [629](#), [639](#)
- bounds of datatypes, **153**
- broadcast, **196**
 - nonblocking, **253**
 - persistent, **275**
- buffered mode send, [53](#), [56](#), [104](#), [107](#)
 - buffer allocation, **60**
 - nonblocking, [72](#), [73](#), [75](#)
- C
 - language binding, **26**
- caching, [305](#), [306](#), **358**
- callback functions
 - language interoperability, **866**
 - prototype definitions, **908**
 - deprecated, **914**
- cancel, [26](#), [81](#), [95](#), [95](#), [96](#), [103](#), **103**, [104](#), [105](#), [488](#), [800](#), [1096](#)
- cancelled, [95](#), [96](#), [103–105](#)
- canonical pack and unpack, **184**
- Cartesian
 - topology, [383](#), **384**
- Chameleon, **2**
- change-log, **1089**
- Chimp, **2**
- choice, **22**
- class
 - in function names, [12](#)
- clock synchronization, **442**
- collective, [17](#), [670](#)
 - split, [717](#)
- collective communication, **189**

- correctness, [296](#)
- file data access operations, [692](#)
- neighborhood, [407](#)
- nonblocking, [250](#)
 - progress, [250](#), [252](#), [300](#), [302](#)
- collective operation, [15](#)
- collective procedure, [17](#)
- commit, [156](#)
- COMMON blocks, [852](#)
- communication, [545](#)
 - collective, [189](#)
 - modes, [52](#)
 - one-sided, [545](#)
 - overlap with communication, [60](#), [71](#)
 - overlap with computation, [60](#), [71](#)
 - partitioned point-to-point, [113](#)
 - pending, [15](#), [20](#), [58](#), [72](#), [73](#), [85](#), [91](#), [95](#), [104](#), [251](#), [306–308](#), [322](#), [514](#), [850](#), [856](#), [1090](#)
 - point-to-point, [33](#)
 - queued, [625](#)
 - RMA, [545](#)
 - typed, untyped, and packed, [48](#)
- communication modes, [52](#)
- communicator, [38](#), [305](#), [306](#)
 - hidden, [192](#), [298](#), [594](#)
- complete operation, [13](#)
- complete operation state, [14](#)
- completing, [17](#)
- completing procedure, [17](#)
- completion, [52](#), [71](#), [80](#), [80](#), [81](#), [82](#), [84](#), [105](#)
 - multiple, [85](#), [86–89](#), [93](#), [94](#)
- completion stage, [13](#)
- concurrent, [717](#), [718](#)
- conflict, [717](#)
- connected, [541](#)
- constants, [21](#), [871](#), [891](#), [1091](#)
 - C integer constant expressions, [22](#)
- context, [305](#), [306](#), [308](#)
- control variable
 - tool information interface, [745](#)
- conversion, [51](#)
 - representation, [51](#)
 - type, [51](#)
- coordination, [669](#)
- counts, [23](#)
- Cray-pointers, [447](#)
- create
 - in function names, [11](#)
- data, [35](#)
- data conversion, [51](#)
- datatype, [18](#)
 - derived, [18](#)
 - equivalent, [19](#)
 - named, [18](#)
 - portable, [18](#)
 - predefined, [18](#)
 - unnamed, [18](#)
- datatypes, [127](#), [864](#)
- deadlock, [31](#), [614](#)
- deadlock avoidance
 - cyclic shift, [45](#)
 - lack of buffer space, [60](#)
 - nonblocking communication, [71](#), [85](#)
 - send modes, [60](#)
- decoupled MPI activities, [20](#), [29](#), [58](#), [306–308](#), [322](#), [542](#), [1090](#)
- default file error, [727](#)
- delete
 - in function names, [11](#)
- deprecated interfaces, [24](#), [797](#)
- derived datatype, [18](#), [127](#), [843](#)
- disconnected, [541](#)
- displacement, [653](#), [667](#)
- distributed graph
 - topology, [383](#), [388](#)
- Dynamic Process Model, [479](#)
- dynamically attached memory, [556](#)
- elementary datatype, [653](#), [667](#)
- empty, [80](#), [80](#), [81](#), [82](#), [86](#)
- enabled operation state, [15](#), [18](#), [81](#), [1092](#)
 - a priori, [15](#)
- end of file, [654](#)
- enqueue count, [626](#), [628–630](#), [633](#), [637](#), [639](#)
- enqueued, [626](#), [628](#), [633](#), [637](#), [638](#)
- enqueued operation state, [626](#)
- envelope, [33](#), [34](#), [37](#), [37](#), [38](#), [40](#), [42](#), [51](#), [52](#), [71](#)
 - data representation conversion, [51](#)
- environmental inquiries, [440](#)
- equivalent datatypes, [19](#)
- erroneous program, [40](#)
 - freeing active request, [83](#)
 - invalid matched receive, [102](#), [103](#)
 - lack of buffer space, [58](#)
 - ready mode before receive, [53](#)
 - type matching, [48](#)
- error classes, [460](#)
- error handler, [638](#)
- error handling, [28](#), [448](#)
 - default file error handler, [655](#), [659](#), [727](#)
 - error codes and classes, [459](#), [463](#)
 - error handlers, [451](#), [463](#), [866](#)
 - fatal after request free, [83](#)
 - fault tolerance, [450](#), [787](#)
 - ack, [793](#)
 - communicator, [789](#), [791](#)
 - fault tolerance error, [28](#), [450](#), [787](#), [794](#)

- mitigation, **791**
- notification, **788**, **794**
- revoked, **788**, **788**, **788**, **791**
- finalize, **28**, **488**, **489**, **541**
- I/O, **727**
- initial error handler, **28**, **449**, **450**, **481**, **488**, **526**
- multiple completions, **88**, **89**
- one-sided communication, **602**
- process failure, **28**, **541**, **787**
- program error, **28**
- resource error, **28**
- startup, **28**, **481**, **526**
- transmission failure, **28**
- establishing communication, **528**
- etype, **653**, **667**
- event
 - tool information interface, **752**, **764**, **764**
- event type
 - tool information interface, **764**
- examples, **32**
- exclusive lock, **586**, **595**, **607**, **609**
- exclusive scan
 - nonblocking, **272**
 - persistent, **294**
- explicit offsets, **669**, **672**
- exposure epoch, **585**
- extent of datatypes, **128**, **152**, **153**
 - true extent, **154**
- external32
 - file data representation, **706**
- extra-state, **870**
- fairness
 - not guaranteed, **58**
 - requirement, **90**, **95**
- file
 - data access, **669**
 - collective operations, **692**
 - explicit offsets, **672**
 - individual file pointers, **679**
 - seek, **695**
 - shared file pointers, **689**
 - split collective, **696**
 - end of file, **654**
 - filetype, **653**
 - handle, **655**
 - interoperability, **705**
 - manipulation, **655**
 - nonblocking collective
 - progress, **721**
 - offset, **23**, **654**
 - pointer, **655**
 - progress, **720**
 - size, **654**
 - view, **653**, **654**, **666**
- file access
 - concurrent, **717**
 - conflict, **717**
 - overlap, **717**
- file size, **722**
- finalize, **486**
- finished, **489**
- Fortran
 - language binding, **25**, **811**
- Fortran 2018, **22**, **25**, **811–816**, **818**, **820**, **821**, **824**, **855**, **1092**
- Fortran support, **811**
- freed operation state, **14**
- freeing procedure, **17**
- freeing stage, **14**, **83**, **92**, **629**
- gather, **198**
 - nonblocking, **254**
 - persistent, **276**
- general datatype, **127**
- generalized requests, **643**, **643**
 - progress, **643**
- get
 - in function names, **11**
- graph
 - topology, **383**, **386**
- group, **38**, **305**, **306**, **308**, **350**
- group objects, **308**
- guarantee progress, **30**
- handle serialization, **882**
- handles, **19**, **860**
- hardware resource type, **332**, **333**, **335**
- host rank, **803**
- I/O
 - blocking, **670**
 - nonblocking, **670**
- immediate, **17**, **72**, **73**, **87**, **89**, **90**, **93**, **96**, **104**
- inactive, **80**, **80**, **81–83**, **86**, **87**, **104**, **110**, **111**, **630**, **632**, **634**, **635**, **642**
 - operation state, **14**
- inclusive scan, **246**
 - nonblocking, **271**
 - persistent, **293**
- incomplete, **17**, **73**, **101**
- incomplete procedure, **17**
- independent, **541**
- individual file pointers, **669**, **679**
- info object, **471**
 - file info, **663**
 - keys, **916**
 - values, **917**

- initial error handler, [28](#)
- initialization, [100](#), [105](#), [106](#), [110](#)
- initialization procedure, [16](#)
- initialization stage, [13](#)
- initialized operation state, [14](#)
- initiation, [13](#), [71](#), [73](#), [80](#), [84](#), [97](#)
- initiation procedure, [17](#)
- inter-communication, [307](#), [349](#)
- inter-communicator, [192](#), [307](#), [349](#)
 - collective operations, [193](#), [194](#)
 - point-to-point, [38](#), [40](#)
- interlanguage communication, [871](#)
- internal
 - file data representation, [706](#)
- interoperability, [705](#)
- intra-communication, [307](#), [348](#)
- intra-communicator, [192](#), [306](#), [348](#)
 - collective operations, [192](#)
- intra-communicator objects, [309](#)
- I/O, [653](#)
- IO rank, [441](#)
- is
 - in function names, [11](#)
- language binding, [24](#), [811](#)
 - interoperability, [858](#)
 - summary, [891](#)
- large
 - count, [24](#), [856](#)
 - displacement, [856](#)
- large count, [24](#)
- lb_marker, [142](#), [146](#), [151](#), [151](#), [155](#)
 - erased, [154](#)
- local, [16](#), [16](#), [53](#), [72](#), [81–83](#), [88](#), [90](#), [96](#), [100](#), [104](#), [105](#), [111](#), [495](#), [592](#), [627–631](#), [633](#), [635](#), [637](#), [641](#)
- local group, [321](#)
- local procedure, [16](#), [29](#)
 - call, under constraint, [18](#)
- logical resource type, [333](#)
- logically concurrent, [57](#)
- loosely synchronous model, [379](#)
- lower bound, [151](#)
- lower-bound markers, [151](#)
- macros, [27](#)
- main thread, [514](#)
- matched, [625](#), [632](#), [633](#), [634](#), [636](#), [640](#)
- matched persistent request, [626](#), [628](#), [630](#), [632](#), [632](#), [633–635](#), [639](#), [641](#)
- matched probe
 - progress, [101](#)
- matched receive, [100](#), [101](#), [101](#)
- matching
 - type, [158](#), [721](#)
- matching probe, [97](#), [99](#), [99](#), [100–102](#)
- matching receive, [100](#)
- matching rules
 - blocking with nonblocking, [72](#)
 - cancel, [104](#)
 - envelope, [40](#)
 - null process, [112](#)
 - ordering, [57](#)
 - persistent collective, [274](#)
 - persistent point-to-point, [112](#)
 - probe, [96](#), [98](#)
 - send modes, [56](#)
 - type, [48](#), [48](#)
 - wildcard, [40](#)
- memory
 - alignment, [446](#), [551](#), [553](#), [1097](#)
 - allocation, [445](#), [550](#), [552](#)
 - system, [19](#)
- memory model, [545](#), [584](#)
 - separate, [545](#), [556](#)
 - unified, [545](#), [556](#)
- message, [33](#), [99](#)
 - buffer, [54](#)
 - cancel, [81](#), [95](#), [95](#), [96](#), [103–105](#)
 - data, [33](#), [34](#), [35](#), [52](#), [54](#)
 - envelope, [33](#), [34](#), [37](#), [37](#), [38](#), [40](#), [42](#), [51](#), [52](#), [71](#)
 - handle, [98](#), [100](#), [100](#), [102](#), [103](#)
 - intermediate buffering, [48](#), [52](#)
 - invalid handle, [100](#)
 - predefined handle, [100](#)
 - wildcard, [40](#)
- message handle, [98](#), [100](#), [100](#), [102](#), [103](#)
 - invalid, [100](#)
 - predefined, [100](#)
- modes, [52](#)
 - buffered, [53](#), [56](#)
 - ready, [53](#), [56](#)
 - standard, [52](#), [56](#)
 - synchronous, [53](#), [56](#)
- module variables, [852](#)
- MPI datatype, [18](#)
- MPI file, [653](#)
- mpi module
 - Fortran support, [815](#)
- MPI operation, [13](#)
- MPI procedure, [16](#)
- MPI process initialization, [479](#)
 - Dynamic Process Model, [479](#)
 - Sessions Model, [309](#), [312](#), [441](#), [479](#), [514](#), [656](#)
 - World Model, [309](#), [312](#), [440](#), [479](#), [514](#), [655](#), [727](#)

- MPI Session handle, [491](#), [491](#)
- mpi_f08 module
 - Fortran support, **812**
- MPI_SIZEOF and storage_size(), [26](#), **802**,
[835–837](#)
- mpiexec, [481](#), [482](#), [484](#), [506](#), [511](#), [511](#), [539](#), [1098](#)
- mpif.h include file
 - Fortran support, **817**
- mpirun, [511](#), [511](#)
- multiple completions, **85**, [86–89](#), [93](#), [94](#)
 - error handling, [88](#), [89](#)
- named datatype, [18](#)
- names, **529**
 - name publishing, **533**
- naming objects, **374**
- native
 - file data representation, [706](#)
- neighborhood collective communication, **407**
 - nonblocking, **419**
 - periodic and dims=1 or 2, [1095](#)
- nonblocking, [17](#), **71**, [83](#), [100](#), [104](#), [564](#), [628](#),
[634–636](#), [670](#)
 - communication, [71](#)
 - completion, **80**, [82](#)
 - Fortran problems, **846**
 - I/O, [670](#)
 - initiation, **73**
 - persistent
 - partitioned completion, **120**
 - request objects, **72**
- nonblocking I/O, [670](#)
- nonblocking operation, [14](#)
- nonblocking procedure, [17](#)
- noncollective operation, [15](#)
- noncollective procedure, [18](#)
- nonlocal, [16](#), [16](#), [52](#), [53](#), [72](#), [81](#), [97](#), [101](#),
[632–637](#), [641](#)
- nonlocal procedure, [16](#), [29](#)
 - call, under constraint, [18](#)
- nonovertaking, [57](#), [84](#)
- nonpersistent, [634–636](#)
- null handle, [80](#), [86–88](#), [92–94](#)
- null processes, **112**
- offset, **23**, [654](#)
- one-sided communication, **545**
 - Fortran problems, **847**
 - progress, **612**, [613](#), [614](#)
- opaque objects, **19**, **864**
- operation, **13**
 - blocking, [14](#)
 - collective, [15](#)
 - complete, [13](#)
 - nonblocking, [14](#)
 - noncollective, [15](#)
 - operation-related, [16](#)
 - partitioned receive, [14](#)
 - partitioned send, [14](#)
 - pending, [15](#), [20](#), [58](#), [72](#), [73](#), [85](#), [91](#), [95](#), [104](#),
[251](#), [306–308](#), [322](#), [514](#), [721](#), [850](#), [856](#),
[1090](#)
 - persistent, [14](#)
 - semantics, **13**
 - stage, [13](#)
 - completion, [13](#)
 - freeing, [14](#)
 - initialization, [13](#)
 - starting, [13](#)
 - state, [14](#)
 - active, [14](#)
 - complete, [14](#)
 - enabled, [15](#), [18](#), [81](#), [1092](#)
 - enabled a priori, [15](#)
 - freed, [14](#)
 - inactive, [14](#)
 - initialized, [14](#)
 - started, [14](#)
- operation states, [626](#)
- ordered, [57](#), [84](#)
- origin, [546](#)
- overlap, [717](#)
- pack, **177**
 - canonical, **184**
- packing unit, [180](#)
- parallel procedure, [379](#)
- Partitioned, [633](#), [637](#)
- partitioned, [625](#), [626](#), [628](#), [633](#), [634](#), [639](#)
- partitioned completion, **120**
- partitioned point-to-point communication, **113**
- passive target communication, [585](#)
- pending (communication) operation, [15](#), [20](#),
[58](#), [72](#), [73](#), [85](#), [91](#), [95](#), [104](#), [251](#),
[306–308](#), [322](#), [514](#), [850](#), [856](#), [1090](#)
- pending communication affector (Fortran),
[849](#), [850](#)
- pending I/O operation, [15](#), [514](#), [721](#), [1090](#)
- performance variable
 - tool information interface, **752**
- persistent, [632](#)
- persistent communication request, [80–83](#),
[86–89](#), [104](#), [105](#), **105**, [106](#), [106](#),
[107–111](#), [625](#), [626](#), [633](#), [637](#), [639](#)
 - active, [80](#)
 - completion, [82](#), [105](#)
 - inactive, [80](#)
 - freeing, [111](#)

- starting, [105](#), [110](#)
- persistent communication requests
 - collective persistent, [273](#), [426](#)
 - Fortran problems, [847](#)
- persistent operation, [14](#)
- persistent request, [628](#), [629](#), [632–635](#), [641](#)
- PICL, [2](#)
- PMPI_, [733](#)
- point-to-point communication, [33](#)
 - blocking, [34](#)
 - buffer allocation, [60](#)
 - cancel, [103](#)
 - matched receive, [101](#)
 - matching probe, [99](#)
 - nonblocking, [71](#)
 - persistent, [105](#)
 - probe, [96](#)
 - receive operation, [38](#)
 - send modes, [52](#), [56](#)
 - send operation, [34](#)
 - send-receive operation, [45](#)
 - status, [41](#)
- portable datatype, [18](#)
- ports, [529](#)
- positioning, [669](#)
- POSIX
 - environment, [469](#)
 - FORTTRAN, [20](#)
 - I/O, [656](#), [657](#), [669](#), [705](#), [717](#)
 - model, [653](#)
- predefined datatype, [18](#)
- predefined reduction operations, [226](#)
- private window copy, [584](#)
- probe, [95](#), [96](#), [96](#), [100](#)
 - matching, [99](#)
 - progress, [97](#)
- procedure, [16](#)
 - blocking, [17](#)
 - collective, [17](#)
 - completing, [17](#)
 - freeing, [17](#)
 - immediate, [17](#)
 - incomplete, [17](#), [17](#)
 - initialization, [16](#)
 - initiation, [17](#)
 - local, [16](#), [29](#)
 - call, under constraint, [18](#)
 - nonblocking, [17](#)
 - noncollective, [18](#)
 - nonlocal, [16](#), [29](#)
 - call, under constraint, [18](#)
 - operation-related, [16](#)
 - semantics, [16](#)
 - specification, [12](#)
 - starting, [16](#)
 - synchronizing, [18](#)
- procedure specification, [12](#)
- process creation, [479](#)
- process failures, [28](#)
- process set names, [915](#)
- processes, [27](#)
- processor name, [442](#)
- profiling interface, [733](#)
 - Fortran, [820](#)
- program error, [28](#)
- progress, [29](#), [720](#), [1093](#)
 - cancellation, [105](#)
 - file, [720](#)
 - nonblocking collective, [721](#)
 - finalize, [488](#)
 - buffered data, [488](#)
 - session, [495](#)
 - generalized requests, [643](#)
 - guarantee, [30](#)
 - load/store access synchronization, [600](#), [1092](#)
 - nonblocking collective communication, [250](#), [252](#), [300](#), [302](#)
 - one-sided communication, [600](#), [612](#), [613](#), [614](#)
 - partitioned point-to-point communication, [121](#)
 - point-to-point communication, [57](#), [84](#), [92](#), [94](#), [95](#), [97](#), [101](#), [1093](#)
 - strong, [30](#), [1093](#)
 - threads, [514](#)
 - weak, [30](#), [1093](#)
- prototype definitions, [908](#)
 - deprecated, [914](#)
- public window copy, [584](#)
- PVM, [2](#)
- queue, [625](#), [626](#), [626](#), [627](#), [628](#), [630](#), [632](#), [637–642](#)
- queue object, [627](#), [627](#)
- queued communication, [625](#)
- raise an event, [764](#)
- rank, [308](#)
- ready mode send, [53](#), [56](#), [109](#), [110](#)
 - as standard mode send, [56](#)
 - nonblocking, [72](#), [73](#), [76](#)
 - persistent, [109](#)
- receive, [33](#), [34](#)
 - blocking, [38](#), [38](#)
 - buffer, [34](#), [102](#), [110](#)
 - complete, [71](#)
 - context, [350](#)

- matched, [100](#), [101](#), [101](#)
 - nonblocking, [71](#), [77](#)
 - persistent, [110](#)
 - start, [71](#), [77](#)
- reduce, [224](#)
 - nonblocking, [266](#)
 - persistent, [288](#)
- reduce-scatter, [243](#)
 - nonblocking, [268](#), [269](#)
 - persistent, [291](#), [292](#)
- reduction operations, [224](#), [866](#)
 - predefined, [226](#)
 - process-local, [241](#)
 - scan, [246](#)
 - user-defined, [235](#)
- registration handle, [771](#)
- related, [180](#)
- relative displacement, [23](#), [148](#)
- remote group, [321](#)
- Remote Memory Access
 - see RMA, [545](#)
- removed interfaces, [24](#), [807](#)
- representation
 - conversion, [51](#)
- request, [638](#)
- request complete
 - I/O, [670](#)
- request objects, [72](#)
 - completion, [84](#)
 - freeing, [82](#), [111](#)
 - initiation, [84](#)
 - multiple completions, [85](#)
 - null handle, [80](#), [86–88](#), [92–94](#)
 - started, [52](#), [84](#), [100](#), [105](#), [110](#), [112](#)
- resource error, [28](#)
- return code
 - tool information interface, [783](#)
- RMA, [545](#)
 - communication calls, [564](#)
 - request-based, [577](#)
 - dynamic window, [557](#)
 - memory model, [584](#)
 - synchronization calls, [585](#)
- root, [190](#)
- scan, [246](#)
 - inclusive, [246](#)
 - segmented, [249](#)
- scatter, [208](#)
 - nonblocking, [257](#)
 - persistent, [278](#)
- seek, [695](#)
- segmented scan, [249](#)
- semantic changes, [809](#)
- semantics, [13](#), [1095](#)
 - collective communications, [190](#)
 - nonblocking progress, [252](#), [300](#), [302](#)
 - data conversion, [51](#)
 - deadlock
 - buffer space, [59](#)
 - cyclic shift, [45](#)
 - send to self, [40](#)
 - erroneous program
 - freeing active request, [83](#)
 - invalid matched receive, [102](#), [103](#)
 - lack of buffer space, [58](#)
 - ready mode before receive, [53](#)
 - type matching, [48](#)
 - exceptions
 - completing and local, [54](#)
 - incomplete and nonlocal, [101](#)
 - file
 - collective, [720](#)
 - collective access, [692](#)
 - conflicting access, [717](#)
 - consistency, [716](#)
 - explicit offsets, [672](#)
 - nonblocking collective, [720](#)
 - nonblocking collective progress, [721](#)
 - progress, [720](#)
 - shared file pointer, [689](#)
 - split collective, [717](#)
 - finalize
 - buffered data, [488](#)
 - generalized requests
 - progress, [643](#)
 - inter-communicator, [321](#)
 - MPI_COMM_IDUP, [323](#)
 - nonblocking communication operations, [84](#)
 - nonblocking completion, [80](#)
 - nonblocking partitioned communications, [121](#)
 - operation, [13](#)
 - overlap, [71](#)
 - partitioned point-to-point communication, [114](#)
 - progress, [121](#)
 - point-to-point communication, [57](#)
 - buffered mode send, [69](#)
 - buffered mode send automatic buffering, [69](#)
 - cancel, [81](#), [95](#), [95](#), [96](#), [103](#), [103](#), [104](#), [105](#)
 - deterministic, [57](#)
 - fairness not guaranteed, [58](#)
 - fairness requirement, [90](#), [95](#)
 - logically concurrent, [57](#)
 - matched receive, [101](#)

- 1 matching probe, [99](#)
- 2 nonovertaking, [57](#), [84](#)
- 3 ordered, [57](#), [84](#)
- 4 overflow error on truncation, [39](#)
- 5 persistent, [105](#)
- 6 probe, [96](#)
- 7 progress, [57](#), [84](#), [92](#), [94](#), [95](#), [97](#), [101](#),
8 [1093](#)
- 9 resource limitations, [58](#)
- 10 send modes, [52](#), [56](#)
- 11 send-receive concurrency, [47](#)
- 12 wildcard receive, [40](#)
- 13 procedure, [16](#)
- 14 process failure, [541](#)
- 15 terms, [13](#)
- 16 threads
- 17 progress, [514](#)
- 18 semantics and correctness
- 19 one-sided communication, [602](#)
- 20 progress, [612](#), [613](#), [614](#)
- 21 send, [33](#)
- 22 blocking, [34](#), [34](#)
- 23 buffer, [33](#)
- 24 complete, [71](#)
- 25 context, [350](#)
- 26 nonblocking, [71](#)
- 27 persistent, [106](#)
- 28 start, [71](#), [74–76](#)
- 29 send-receive
- 30 blocking, [45](#), [45](#)
- 31 nonblocking, [78](#), [80](#)
- 32 start, [78](#), [80](#)
- 33 separate memory model, [545](#), [556](#), [584](#)
- 34 sequential storage, [161](#)
- 35 serialization, [71](#)
- 36 lack of buffer space, [60](#)
- 37 Sessions Model, [309](#), [312](#), [441](#), [479](#), [514](#), [656](#)
- 38 set
- 39 in function names, [11](#)
- 40 shared file pointers, [669](#), [689](#)
- 41 shared lock, [586](#), [595](#), [607](#)
- 42 shared memory, [551](#), [555](#), [556](#), [597](#), [603](#), [606](#),
43 [620](#)
- 44 allocation, [553](#)
- 45 contiguous, [551](#), [553](#), [554](#)
- 46 domain, [27](#), [332](#), [549](#), [551](#), [553](#), [555](#), [597](#)
- 47 loads/stores for synchronizing, [614](#), [1093](#)
- 48 noncontiguous, [553](#), [554](#)
- segment, [27](#), [332](#), [549](#), [551](#), [553](#)
- window, [552](#), [556](#), [620](#)
- shared memory allocation, [552](#)
- signals, [31](#)
- singleton init, [539](#)
- size changing
- I/O, [722](#)
- source, [350](#)
- split collective, [670](#), [696](#)
- stage, [13](#)
- completion, [13](#)
- freeing, [14](#)
- initialization, [13](#)
- starting, [13](#)
- standard mode send, [52](#), [52](#), [56](#), [106](#)
- as synchronous mode send, [56](#)
- nonblocking, [72](#), [74](#)
- start, [626](#)
- started, [489](#)
- request objects, [52](#), [84](#), [100](#), [105](#), [110](#), [112](#)
- started operation state, [14](#)
- starting procedure, [16](#), [105](#)
- starting processes, [517](#), [518](#)
- starting stage, [13](#)
- startup, [480](#)
- portable, [511](#)
- state
- operation, [14](#)
- active, [14](#)
- complete, [14](#)
- enabled, [15](#), [18](#), [81](#), [1092](#)
- enabled a priori, [15](#)
- freed, [14](#)
- inactive, [14](#)
- initialized, [14](#)
- started, [14](#)
- type, [21](#)
- status, [40](#), [861](#)
- array in Fortran, [41](#)
- associating information, [649](#)
- derived type in Fortran 2008, [41](#)
- empty, [80](#), [81](#), [82](#), [86–88](#), [92–94](#)
- error in status, [41](#)
- for send operation, [80](#)
- ignore, [44](#)
- message length, [41](#)
- structure in C, [41](#)
- test, [92](#)
- strong progress, [30](#), [614](#), [1093](#)
- strong synchronization, [587](#)
- synchronism, [669](#)
- synchronization, [545](#), [564](#)
- synchronization calls
- RMA, [585](#)
- synchronizing procedure, [18](#)
- synchronous mode send, [53](#), [56](#), [80](#), [100](#), [108](#)
- nonblocking, [72](#), [73](#), [75](#)
- persistent, [108](#)
- system memory, [19](#)

- tag values, [441](#)
- target, [546](#)
- target nodes, [665](#)
- thread compliant, [484](#), [513](#)
- thread-safe, [513](#)
- threads, [513](#)
 - progress, [514](#)
 - thread-safe, [2](#), [440](#), [459](#), [463](#), [513](#), [767](#), [874](#)
- timers and synchronization, [469](#)
- tool information interface, [738](#)
 - control variable, [745](#)
 - event, [752](#), [764](#), [764](#)
 - event type, [764](#)
 - performance variable, [752](#)
 - return code, [783](#)
 - verbosity level, [740](#)
- tool support, [733](#)
- topologies, [381](#)
- topology
 - Cartesian, [383](#), [384](#)
 - distributed graph, [383](#), [388](#)
 - graph, [383](#), [386](#)
 - virtual, [381](#), [382](#)
- transmission failures, [28](#)
- true extent of datatypes, [154](#)
- TS 29113, [25](#), [811–815](#), [818](#), [821](#), [824–828](#), [837](#), [838](#), [844](#), [849](#), [852](#), [855](#), [1092](#), [1105](#)
- type
 - conversion, [51](#)
 - matching rules, [48](#)
- type map, [128](#)
- type matching, [158](#)
- type signature, [128](#)
- types, [907](#)
- ub_marker, [142](#), [146](#), [147](#), [151](#), [151](#), [155](#)
 - erased, [154](#)
- unified memory model, [545](#), [556](#), [584](#)
- universe size, [538](#)
- unnamed datatype, [18](#)
- unpack, [177](#)
 - canonical, [184](#)
- upper bound, [151](#)
- upper-bound markers, [151](#)
- user functions at process termination, [490](#)
- user-defined data representations, [709](#)
- user-defined reduction operations, [235](#)
- verbosity level
 - tool information interface, [740](#)
- version inquiries, [439](#)
- view, [653](#), [654](#), [666](#)
- virtual topology, [306](#), [381](#), [382](#)
- wait, [626](#)
- weak progress, [30](#), [1093](#)
- weak synchronization, [587](#)
- wildcard, [40](#)
- window
 - allocation, [550](#)
 - creation, [546](#)
 - dynamically attached memory, [556](#)
 - shared memory
 - window, [552](#)
 - shared memory allocation, [552](#)
- World Model, [309](#), [312](#), [440](#), [479](#), [514](#), [655](#), [727](#)
- XDR, [52](#)
- Zipcode, [2](#)

Examples Index

This index lists code examples throughout the text. Some examples are referred to by content; others are listed by the major MPI procedure name that they are demonstrating. MPI procedure names listed in all capital letter are Fortran examples; MPI procedure names listed in mixed case are C examples.

- Accumulate in RMA
 - MPI_ACCUMULATE, 572
 - MPI_TYPE_GET_EXTENT, 572
 - MPI_WIN_CREATE, 572
 - MPI_WIN_FENCE, 572
 - MPI_WIN_FREE, 572
- Actions after Finalize
 - MPI_Finalize, 489
- Active target and local reads in RMA
 - MPI_Barrier, 610
 - MPI_Put, 610
 - MPI_Win_complete, 610
 - MPI_Win_lock, 610
 - MPI_Win_post, 610
 - MPI_Win_start, 610
 - MPI_Win_unlock, 610
 - MPI_Win_wait, 610
- Allgather
 - MPI_Allgather, 217
- Allreduce of a vector
 - MPI_ALLREDUCE, 241
- argv in C and Fortran
 - MPI_COMM_SPAWN, 520
 - MPI_Comm_spawn, 520
- Array of argv in C and Fortran
 - MPI_COMM_SPAWN_MULTIPLE, 525
 - MPI_Comm_spawn_multiple, 525
- ASYNCHRONOUS, 620, 848, 855
- Attach and detach buffer
 - MPI_Buffer_attach, 68
 - MPI_Buffer_detach, 68
- Attach and detach communicator-specific buffer
 - MPI_Comm_attach_buffer, 68
 - MPI_Comm_detach_buffer, 68
- Attributes between languages, 867
 - set in C
 - MPI_Comm_get_attr, 867
 - MPI_Comm_set_attr, 867
 - set in Fortran
 - MPI_ATTR_GET, 868, 869
 - MPI_ATTR_PUT, 868
- MPI_COMM_GET_ATTR, 868, 869
- MPI_COMM_SET_ATTR, 869
- Basic usage of performance variables
 - MPI_T_finalize, 762
 - MPI_T_init_thread, 762
 - MPI_T_pvar_get_info, 762
 - MPI_T_pvar_handle_alloc, 762
 - MPI_T_pvar_handle_free, 762
 - MPI_T_pvar_read, 762
 - MPI_T_pvar_session_create, 762
 - MPI_T_pvar_start, 762
- Blocking/Nonblocking collectives do not match, 300
- Broadcast
 - MPI_Bcast, 198
- C/Fortran handle conversion, 861
- C/Fortran handle conversion and absolute addresses
 - MPI_GET_ADDRESS, 865
- Cartesian virtual topologies, 434
- Client server code with waitsome
 - MPI_Irecv, 91
 - MPI_Isend, 91
 - MPI_WAIT, 91
 - MPI_WAITsome, 91
- Client-server
 - MPI_Comm_accept, 536, 537
 - MPI_Comm_connect, 536, 537
 - MPI_Irecv, 91
 - MPI_Isend, 91
 - MPI_Open_port, 536, 537
 - MPI_WAIT, 91
 - MPI_WAITANY, 91
- Client-server (with error)
 - MPI_PROBE, 98
 - MPI_RECV, 98
 - MPI_SEND, 98
- Client-server code, 91
 - with probe, 97
 - with probe (wrong), 98
- Client-server model

MPI_Comm_remote_size, 330	MPI_Get_accumulate, 619	1
MPI_Comm_split, 330	MPI_Win_flush, 619	2
Client-server with probe	MPI_Win_flush_all, 619	3
MPI_PROBE, 97	MPI_Win_sync, 619	4
MPI_RECV, 97		5
MPI_SEND, 97	Datatype	6
Collective communication	3D array, 170	7
MPI_Bcast, 343	absolute addresses, 174	8
Communication safety	array of structures, 172	9
MPI_Comm_create, 344	elaborate example, 182, 183	10
MPI_Group_free, 344	matching type, 159	11
MPI_Group_incl, 344	matrix transpose, 171	12
Consistency by setting atomic mode	union, 175	13
MPI_File_read_at, 723	Datatype matching	14
MPI_File_set_atomicity, 723	MPI_RECV, 49	15
MPI_File_set_view, 723	MPI_SEND, 49	16
MPI_File_write_at, 723	Datatypes	17
Consistency for writing and reading files	matching, 49	18
asynchronously	MPI_BYTE, 49	19
MPI_File, 725	MPI_RECV, 49	20
MPI_File_iread_at, 725	MPI_SEND, 49	21
MPI_File_iwrite_at, 725	not matching, 49	22
MPI_File_read_all_begin, 725	untyped, 49	23
MPI_File_read_all_end, 725	Datatypes for distributed arrays	24
MPI_File_set_atomicity, 725	MPI_TYPE_CREATE_DARRAY, 147	25
MPI_File_write_all_begin, 725	Deadlock	26
MPI_File_write_all_end, 725	if not buffered, 59	27
Consistency using “sync-barrier-sync”	with MPI_Bcast, 296, 297	28
MPI_File_read_at, 723	wrong message exchange, 59	29
MPI_File_set_view, 723	Deadlock due to synchronization through	30
MPI_File_sync, 723	shared memory	31
MPI_File_write_at, 723	MPI_Bsend, 614	32
Counting semaphore (non-scalable)	MPI_Buffer_attach, 614	33
MPI_Accumulate, 618	MPI_Buffer_detach, 614	34
MPI_Barrier, 618	MPI_Recv, 614	35
MPI_Get_accumulate, 618	MPI_Win_fence, 614	36
MPI_Win_flush, 618	MPI_Win_shared_query, 614	37
MPI_Win_sync, 618	Decoding a datatype	38
Creating a communicator using the Sessions	MPI_Type_get_contents, 176	39
Model	MPI_Type_get_envelope, 176	40
MPI_Comm_create_from_group, 499	Decoding amode in Fortran 77	41
MPI_Group_from_session_pset, 499	MPI_FILE_GET_AMODE, 662	42
MPI_Info_create, 499	Defining a user function	43
MPI_Info_set, 499	MPI_OP_CREATE, 239	44
MPI_Session_finalize, 499	MPI_REDUCE, 239	45
MPI_Session_init, 499	Dims create	46
Critical region with Compare-and-Swap	MPI_DIMS_CREATE, 386	47
MPI_Barrier, 619	Dist graph creation	48
MPI_Compare_and_swap, 619	MPI_DIST_GRAPH_CREATE, 393	
MPI_Win_flush, 619	MPI_DIST_GRAPH_CREATE_ADJACENT,	
MPI_Win_sync, 619	393	
Critical region with RMA	Dist_graph_create	
MPI_Accumulate, 619	MPI_Dist_graph_create, 394	
MPI_Barrier, 619	Double buffer in RMA	

- 1 MPI_Barrier, 617
- 2 MPI_Get, 617
- 3 MPI_Win_complete, 617
- 4 MPI_Win_post, 617
- 5 MPI_Win_start, 617
- 6 MPI_Win_wait, 617
- 7 Errant message exchange
- 8 MPI_RECV, 59
- 9 MPI_SEND, 59
- 10 Erroneous attempt to achieve consistency
- 11 MPI_File_read_at, 724
- 12 MPI_File_set_view, 724
- 13 MPI_File_sync, 724
- 14 MPI_File_write_at, 724
- 15 Erroneous example fragment of concurrent
- 16 split collective access on a file handle
- 17 MPI_File_read_all, 697
- 18 MPI_File_read_all_begin, 697
- 19 MPI_File_read_all_end, 697
- 20 Erroneous matching of blocking and
- 21 nonblocking collectives
- 22 MPI_Alltoall, 300
- 23 MPI_lalltoall, 300
- 24 MPI_Wait, 300
- 25 Erroneous matching of collectives
- 26 MPI_Bcast, 299
- 27 MPI_lbarrier, 299
- 28 MPI_Wait, 299
- 29 Erroneous use of Bcast
- 30 MPI_Bcast, 296, 297
- 31 Exchange relies on buffering
- 32 MPI_RECV, 59
- 33 MPI_SEND, 59
- 34 False matching of collective operations, 299
- 35 Fault Tolerant Manager/Worker
- 36 MPI_Comm_ack_failed, 794
- 37 MPI_Comm_get_failed, 794
- 38 File pointer update semantics
- 39 MPI_FILE_CLOSE, 684
- 40 MPI_FILE_IREAD, 684
- 41 MPI_FILE_OPEN, 684
- 42 MPI_FILE_SET_VIEW, 684
- 43 MPI_WAIT, 684
- 44 Finalize and buffer attach
- 45 MPI_Buffer_attach, 488
- 46 MPI_Finalize, 488
- 47 Finalize and cancel
- 48 MPI_Barrier, 488
- MPI_Cancel, 488
- MPI_Finalize, 488
- MPI_lprobe, 488
- MPI_Test_cancelled, 488
- Finalize and request free
- MPI_Finalize, 487
- MPI_Request_free, 487
- Finalize in the Sessions Model
- MPI_SESSION_FINALIZE, 495
- Fortran 2008 measurement wrapper
- MPI_ISEND, 822
- Fortran 90
- a scalar is not an array
- MPI_CART_CREATE, 839
- copying and sequence problem, 839, 841, 842
- derived types
- MPI_GET_ADDRESS, 843
- MPI_TYPE_COMMIT, 843
- MPI_TYPE_CREATE_RESIZED, 843
- MPI_TYPE_CREATE_STRUCT, 843
- heterogeneous communication (unsafe)
- MPI_TYPE_MATCH_SIZE, 836
- heterogeneous MPI I/O (unsafe)
- MPI_FILE_SET_VIEW, 836
- MPI_TYPE_MATCH_SIZE, 836
- overlapping communication and
- computation, 853, 856
- register optimization, 846, 847
- scalars as buffer
- MPI_IRECV, 842
- selected KIND
- MPI_TYPE_CREATE_F90_INTEGER, 832
- MPI_TYPE_CREATE_F90_REAL, 832
- subarray as buffer
- MPI_IRECV, 839, 841
- MPI_ISEND, 839
- MPI_SEND, 839
- vector subscripts
- MPI_SEND, 842
- Fortran 90 register optimization in RMA
- MPI_F_SYNC_REG, 851
- MPI_PUT, 851
- MPI_WIN_FENCE, 851
- Fortran CHARACTER
- MPI_CHARACTER, 50
- MPI_RECV, 50
- MPI_SEND, 50
- Fortran language bindings
- MPI_COMM_RANK, 821
- Gather
- MPI_Gather, 202
- Gather and Gatherv
- MPI_Gather, 207
- MPI_Gatherv, 207
- MPI_Type_commit, 207

MPI_Type_create_struct, 207	
Gather with allocation at the root	
MPI_Gather, 202	
Gather with datatype	
MPI_Gather, 202	
MPI_Type_commit, 202	
MPI_Type_contiguous, 202	
Gatherv	
MPI_Gatherv, 203	
Gatherv with datatype	
MPI_Gatherv, 203, 204	
MPI_Type_commit, 203, 204	
MPI_Type_vector, 203, 204	
Gatherv with struct datatype	
MPI_Gatherv, 205	
MPI_Type_commit, 205	
MPI_Type_create_struct, 205	
Gatherv with vector datatype	
MPI_Gatherv, 206	
MPI_Type_commit, 206	
MPI_Type_vector, 206	
Get with fence	
MPI_Get, 616	
MPI_Win_fence, 616	
Get with PSCW	
MPI_Get, 617	
MPI_Win_complete, 617	
MPI_Win_post, 617	
MPI_Win_start, 617	
MPI_Win_wait, 617	
Get_address	
MPI_GET_ADDRESS, 149	
Graph create	
MPI_GRAPH_CREATE, 387	
Graph creation	
MPI_GRAPH_CREATE, 400	
Graph creation and neighbors count	
MPI_GRAPH_CREATE, 400	
MPI_GRAPH_NEIGHBORS, 400	
MPI_GRAPH_NEIGHBORS_COUNT, 400	
Hello world	
MPI_Comm_rank, 33	
MPI_Init, 33	
MPI_Recv, 33	
MPI_Send, 33	
Ibcast	
MPI_Ibcast, 254	
Independence of nonblocking operations, 302	
MPI_Ibcast, 302	
Initializing MPI	
MPI_Init, 481	
Inter-communicator, 326, 330	
Inter-communicator creation	1
MPI_Comm_create, 326	2
MPI_Comm_group, 326	3
MPI_Group_free, 326	4
MPI_Group_incl, 326	5
Interlanguage communication, 871	6
MPI_GET_ADDRESS, 871	7
MPI_TYPE_CREATE_STRUCT, 871	8
Intertwined matching pairs, 58	9
Library Example #1	10
MPI_Comm_dup, 345	11
MPI_Reduce, 345	12
Library Example #2	13
MPI_Comm_create, 346	14
MPI_Comm_group, 346	15
MPI_Group_free, 346	16
MPI_Group_incl, 346	17
Linked list in RMA	18
MPI_Accumulate, 621	19
MPI_Aint_add, 621	20
MPI_Alloc_mem, 621	21
MPI_Compare_and_swap, 621	22
MPI_Free_mem, 621	23
MPI_Get_accumulate, 621	24
MPI_Win_attach, 621	25
MPI_Win_create_dynamic, 621	26
MPI_Win_detach, 621	27
MPI_Win_flush, 621	28
MPI_Win_lock_all, 621	29
MPI_Win_unlock_all, 621	30
Linking libraries when using the profiling interface, 736	31
Listing names of control variables	32
MPI_T_cvar_get_info, 748	33
Local subarray for file output	34
MPI_Type_create_subarray, 730	35
Local subarray for file output in Fortran 90	36
MPI_TYPE_CREATE_SUBARRAY, 730	37
Manager-worker with Comm_spawn	38
MPI_Comm_get_parent, 527	39
MPI_Comm_spawn, 527	40
Matching type with datatypes	41
MPI_RECV, 159	42
MPI_SEND, 159	43
MPI_TYPE_CONTIGUOUS, 159	44
Measurement wrapper	45
MPI_Send, 735	46
MPI_Type_size, 735	47
MPI_Wtime, 735	48
Message exchange	
MPI_RECV, 59	
MPI_SEND, 59	

- 1 Message exchange (ping-pong), 59
- 2 Message matching
 - 3 MPI_BSEND, 58
 - 4 MPI_RECV, 58
 - 5 MPI_SSEND, 58
- 6 Message ordering for nonblocking operations
 - 7 MPI_IRECV, 84
 - 8 MPI_ISEND, 84
 - 9 MPI_WAIT, 84
- 10 Mixing blocking and nonblocking collective
 - 11 operations, 299
 - 12 MPI_Bcast, 299
 - 13 MPI_lbarrier, 299
 - 14 MPI_Wait, 299
- 15 Mixing collective and point-to-point
 - 16 MPI_lbarrier, 301
 - 17 MPI_irecv, 301
 - 18 MPI_Send, 301
 - 19 MPI_Wait, 301
 - 20 MPI_Waitall, 301
- 21 Mixing collective and point-to-point requests,
 - 22 301
- 23 MPI_ACCUMULATE
 - 24 Accumulate in RMA, 572
- 25 MPI_Accumulate
 - 26 Counting semaphore (nonscalable), 618
 - 27 Critical region with RMA, 619
 - 28 Linked list in RMA, 621
- 29 MPI_Aint
 - 30 Using datatypes with array of structures,
 - 31 172
- 32 MPI_Aint_add
 - 33 Linked list in RMA, 621
- 34 MPI_Allgather
 - 35 Allgather, 217
- 36 MPI_ALLOC_MEM
 - 37 Use of Alloc_mem in Fortran, 447
 - 38 Use of Alloc_mem in Fortran with Cray
 - 39 pointers, 447
- 40 MPI_Alloc_mem
 - 41 Linked list in RMA, 621
 - 42 Using Alloc_mem, 448
- 43 MPI_ALLREDUCE
 - 44 Allreduce of a vector, 241
- 45 MPI_Alltoall
 - 46 Erroneous matching of blocking and
 - 47 nonblocking collectives, 300
- 48 "mpi_assert_memory_alloc_kinds"
 - Requesting and querying memory
 - allocation kinds in the Sessions
 - Model, 509
- MPI_ASYNC_PROTECTS_NONBLOCKING,
 - 620
- MPI_ATTR_GET
 - Attributes between languages
 - set in Fortran, 868, 869
- MPI_ATTR_PUT
 - Attributes between languages
 - set in Fortran, 868
- MPI_BARRIER
 - Using process set query in group creation,
 - 503
- MPI_Barrier
 - Active target and local reads in RMA, 610
 - Counting semaphore (nonscalable), 618
 - Critical region with Compare-and-Swap,
 - 619
 - Critical region with RMA, 619
 - Double buffer in RMA, 617
 - Finalize and cancel, 488
 - Public and private memory in RMA, 609,
 - 610
 - Read data in RMA, 608
 - Read data in RMA (unsafe), 609
 - Update location in separate memory
 - model, 607
 - Update location in unified memory model,
 - 608
- MPI_Bcast
 - Broadcast, 198
 - Collective communication, 343
 - Erroneous matching of collectives, 299
 - Erroneous use of Bcast, 296, 297
 - Mixing blocking and nonblocking
 - collective operations, 299
 - Nondeterministic use of Bcast, 297
- MPI_BSEND
 - Message matching, 58
 - Nonovertaking messages, 57
- MPI_Bsend
 - Deadlock due to synchronization through
 - shared memory, 614
- MPI_Buffer_attach
 - Attach and detach buffer, 68
 - Deadlock due to synchronization through
 - shared memory, 614
 - Finalize and buffer attach, 488
- MPI_Buffer_detach
 - Attach and detach buffer, 68
 - Deadlock due to synchronization through
 - shared memory, 614
- MPI_BYTE
 - Datatypes, 49
- MPI_Cancel
 - Finalize and cancel, 488
- MPI_CART_COORDS
 - Using Cart_shift, 404
- MPI_CART_CREATE

Fortran 90		
a scalar is not an array, 839		
Neighborhood collective communication,		
434		
MPI_CART_GET		
Neighborhood collective communication,		
434		
MPI_CART_RANK		
Using Cart_shift, 404		
MPI_CART_SHIFT		
Neighbor alltoall, 414		
Neighborhood collective communication,		
434		
Using Cart_shift, 404		
MPI_CART_SUB		
Subgroup cart process topology, 405		
MPI_CARTDIM_GET		
Neighbor alltoall, 414		
MPI_CHARACTER		
Fortran CHARACTER, 50		
MPI_Comm_accept		
Client-server, 536 , 537		
Name publishing, 537		
MPI_Comm_ack_failed		
Fault Tolerant Manager/Worker, 794		
MPI_Comm_attach_buffer		
Attach and detach communicator-specific		
buffer, 68		
MPI_Comm_connect		
Client-server, 536 , 537		
Name publishing, 537		
MPI_Comm_create		
Communication safety, 344		
Inter-communicator creation, 326		
Library Example #2, 346		
Using Group_excl, 343		
MPI_COMM_CREATE_FROM_GROUP		
Using process set query in group creation,		
503		
MPI_Comm_create_from_group		
Creating a communicator using the		
Sessions Model, 499		
MPI_Comm_detach_buffer		
Attach and detach communicator-specific		
buffer, 68		
MPI_Comm_dup		
Library Example #1, 345		
MPI_COMM_GET_ATTR		
Attributes between languages		
set in Fortran, 868 , 869		
MPI_Comm_get_attr		
Attributes between languages		
set in C, 867		
MPI_Comm_get_failed		
Fault Tolerant Manager/Worker, 794		
MPI_Comm_get_parent		
Manager-worker with Comm_spawn, 527		
MPI_Comm_group		
Inter-communicator creation, 326		
Library Example #2, 346		
MPI_COMM_RANK		
Fortran language bindings, 821		
MPI_Comm_rank		
Hello world, 33		
MPI_Comm_remote_size		
Client-server model, 330		
MPI_COMM_SET_ATTR		
Attributes between languages		
set in Fortran, 869		
MPI_Comm_set_attr		
Attributes between languages		
set in C, 867		
MPI_COMM_SPAWN		
argv in C and Fortran, 520		
MPI_Comm_spawn		
argv in C and Fortran, 520		
Manager-worker with Comm_spawn, 527		
MPI_COMM_SPAWN_MULTIPLE		
Array of argv in C and Fortran, 525		
MPI_Comm_spawn_multiple		
Array of argv in C and Fortran, 525		
MPI_Comm_split		
Client-server model, 330		
Three-Group “Pipeline”, 356		
Three-Group “Ring”, 357		
MPI_Comm_split_type		
Recursive splitting of COMM_WORLD,		
337		
Splitting into NUMANode		
subcommunicators, 333 , 444		
MPI_Compare_and_swap		
Critical region with Compare-and-Swap,		
619		
Linked list in RMA, 621		
MPI_DIMS_CREATE		
Dims create, 386		
Neighborhood collective communication,		
434		
MPI_DIST_GRAPH_CREATE		
Dist graph creation, 393		
MPI_Dist_graph_create		
Dist_graph_create, 394		
MPI_DIST_GRAPH_CREATE_ADJACENT		
Dist graph creation, 393		
MPI_Enqueue_startall		
Ring exchange with queued		
communication, 640		

- 1 MPI_Enqueue_waitall
- 2 Ring exchange with queued
- 3 communication, 640
- 4 MPI_F_SYNC_REG
- 5 Fortran 90 register optimization in RMA,
- 6 851
- 7 Shared memory windows, 620
- 8 MPI_File
- 9 Consistency for writing and reading files
- 10 asynchronously, 725
- 11 MPI_FILE_CLOSE
- 12 File pointer update semantics, 684
- 13 Read to end of file, 680
- 14 MPI_FILE_GET_AMODE
- 15 Decoding amode in Fortran 77, 662
- 16 MPI_FILE_IREAD
- 17 File pointer update semantics, 684
- 18 MPI_File_iread_at
- 19 Consistency for writing and reading files
- 20 asynchronously, 725
- 21 MPI_File_iwrite_at
- 22 Consistency for writing and reading files
- 23 asynchronously, 725
- 24 MPI_FILE_OPEN
- 25 File pointer update semantics, 684
- 26 Read to end of file, 680
- 27 MPI_FILE_READ
- 28 Read to end of file, 680
- 29 MPI_File_read_all
- 30 Erroneous example fragment of concurrent
- 31 split collective access on a file handle,
- 32 697
- 33 MPI_File_read_all_begin
- 34 Consistency for writing and reading files
- 35 asynchronously, 725
- 36 Erroneous example fragment of concurrent
- 37 split collective access on a file handle,
- 38 697
- 39 MPI_File_read_all_end
- 40 Consistency for writing and reading files
- 41 asynchronously, 725
- 42 Erroneous example fragment of concurrent
- 43 split collective access on a file handle,
- 44 697
- 45 MPI_File_read_at
- 46 Consistency by setting atomic mode, 723
- 47 Consistency using “sync-barrier-sync”, 723
- 48 Erroneous attempt to achieve consistency,
- 724
- File pointer update semantics, 684
- Fortran 90
- heterogeneous MPI I/O (unsafe), 836
- Read to end of file, 680
- MPI_File_set_view
- Consistency by setting atomic mode, 723
- Consistency using “sync-barrier-sync”, 723
- Erroneous attempt to achieve consistency,
- 724
- MPI_File_sync
- Consistency using “sync-barrier-sync”, 723
- Erroneous attempt to achieve consistency,
- 724
- MPI_File_write_all_begin
- Consistency for writing and reading files
- asynchronously, 725
- Overlap computation and output, 728
- MPI_File_write_all_end
- Consistency for writing and reading files
- asynchronously, 725
- Overlap computation and output, 728
- MPI_File_write_at
- Consistency by setting atomic mode, 723
- Consistency using “sync-barrier-sync”, 723
- Erroneous attempt to achieve consistency,
- 724
- MPI_Finalize
- Actions after Finalize, 489
- Finalize and buffer attach, 488
- Finalize and cancel, 488
- Finalize and request free, 487
- Rules for finalize, 487
- MPI_FREE_MEM
- Use of Alloc_mem in Fortran, 447
- Use of Alloc_mem in Fortran with Cray
- pointers, 447
- MPI_Free_mem
- Linked list in RMA, 621
- MPI_Gather
- Gather, 202
- Gather and Gatherv, 207
- Gather with allocation at the root, 202
- Gather with datatype, 202
- Pack and Pack_size, 183
- MPI_Gatherv
- Gather and Gatherv, 207
- Gatherv, 203
- Gatherv with datatype, 203, 204
- Gatherv with struct datatype, 205
- Gatherv with vector datatype, 206
- Pack and Pack_size, 183
- MPI_GET
- Using Get, 570
- Using Get with indexed datatype, 568

MPI_Get	
Double buffer in RMA, 617	
Get with fence, 616	
Get with PSCW, 617	
Public and private memory in RMA, 609, 610	
Update location in separate memory model, 607	
Update location in unified memory model, 608	
MPI_Get_accumulate	
Counting semaphore (non-scalable), 618	
Critical region with RMA, 619	
Linked list in RMA, 621	
MPI_GET_ADDRESS	
C/Fortran handle conversion and absolute addresses, 865	
Fortran 90	
derived types, 843	
Get_address, 149	
Interlanguage communication, 871	
MPI_Get_address	
Pack/Unpack with struct datatype, 182	
Using datatypes with array of structures, 172	
Using datatypes with array of structures with absolute addresses, 174	
Using datatypes with unions, 175	
MPI_GET_COUNT	
Using Get_count and Get_elements, 160	
MPI_GET_ELEMENTS	
Using Get_count and Get_elements, 160	
MPI_Get_hw_resource_info	
Splitting into NUMANode subcommunicators, 444	
MPI_GRAPH_CREATE	
Graph create, 387	
Graph creation, 400	
Graph creation and neighbors count, 400	
MPI_GRAPH_NEIGHBORS	
Graph creation and neighbors count, 400	
MPI_GRAPH_NEIGHBORS_COUNT	
Graph creation and neighbors count, 400	
MPI_Grequest_complete	
User-defined reduce, 647	
MPI_Grequest_start	
User-defined reduce, 647	
MPI_Group_excl	
Using Group_excl, 343	
MPI_Group_free	
Communication safety, 344	
Inter-communicator creation, 326	
Library Example #2, 346	
Using Group_excl, 343	
MPI_GROUP_FROM_SESSION_PSET	1
Using process set query in group creation, 503	2
MPI_Group_from_session_pset	3
Creating a communicator using the Sessions Model, 499	4
Using process set query in group creation, 501	5
MPI_Group_incl	6
Communication safety, 344	7
Inter-communicator creation, 326	8
Library Example #2, 346	9
MPI_lallreduce	10
Overlapping communicators and collectives, 301	11
MPI_lalltoall	12
Erroneous matching of blocking and nonblocking collectives, 300	13
MPI_lbarrier	14
Erroneous matching of collectives, 299	15
Mixing blocking and nonblocking collective operations, 299	16
Mixing collective and point-to-point, 301	17
Progress of nonblocking collectives, 300	18
MPI_lbroadcast	19
Ibcast, 254	20
Independence of nonblocking operations, 302	21
Pipelining nonblocking collectives, 301	22
MPI_Info_create	23
Creating a communicator using the Sessions Model, 499	24
MPI_INFO_ENV	25
mpiexec and environment variables, 482	26
MPI_Info_set	27
Creating a communicator using the Sessions Model, 499	28
MPI_Init	29
Hello world, 33	30
Initializing MPI, 481	31
MPI_Intercomm_create	32
Three-Group “Pipeline”, 356	33
Three-Group “Ring”, 357	34
MPI_lprobe	35
Finalize and cancel, 488	36
MPI_IRECV	37
Client server code with waitsome, 91	38
Client-server, 91	39
Fortran 90	40
scalars as buffer, 842	41
subarray as buffer, 839, 841	42
Message ordering for nonblocking operations, 84	43
Nonblocking point-to-point, 82	44

- 1 Nonblocking send and receive with request
- 2 free, [83](#)
- 3 Progress semantics, [84](#)
- 4 **MPI_Irecv**
- 5 Mixing collective and point-to-point, [301](#)
- 6 **MPI_ISEND**
- 7 Client server code with waitsome, [91](#)
- 8 Client-server, [91](#)
- 9 Fortran 2008 measurement wrapper, [822](#)
- 10 Fortran 90
- 11 subarray as buffer, [839](#)
- 12 Message ordering for nonblocking
- 13 operations, [84](#)
- 14 Nonblocking point-to-point, [82](#)
- 15 Nonblocking send and receive with request
- 16 free, [83](#)
- 17 **MPI_Matchall**
- 18 Ring exchange with queued
- 19 communication, [640](#)
- 20 "mpi_memory_alloc_kinds"
- 21 Requesting and querying memory
- 22 allocation kinds in the Sessions
- 23 Model, [509](#)
- 24 **MPI_NEIGHBOR_ALLGATHER**
- 25 Neighbor allgather, [410](#)
- 26 **MPI_NEIGHBOR_ALLTOALL**
- 27 Neighbor alltoall, [414](#)
- 28 Neighborhood collective communication,
- 29 [434](#)
- 30 **MPI_Neighbor_alltoall**, [414](#)
- 31 **MPI_NEIGHBOR_ALLTOALLW**
- 32 Neighborhood collective communication,
- 33 [434](#)
- 34 **MPI_NEIGHBOR_ALLTOALLW_INIT**
- 35 Neighborhood collective communication,
- 36 [434](#)
- 37 **MPI_OP_CREATE**
- 38 Defining a user function, [239](#)
- 39 **MPI_Op_create**
- 40 Reduction with user-defined op, [238](#)
- 41 User-defined operation with Scan, [249](#)
- 42 **MPI_Open_port**
- 43 Client-server, [536](#), [537](#)
- 44 Name publishing, [537](#)
- 45 **MPI_Pack**
- 46 Pack and Pack_size, [183](#)
- 47 Pack/Unpack with struct datatype, [182](#)
- 48 Using Pack, [182](#)
- MPI_Pack_size**
- Pack and Pack_size, [183](#)
- MPI_Parrived**
- Partitioned communication with partial
- completion, [125](#)
- MPI_Pready**
- Partitioned communication, [114](#)
- Partitioned communication using threads,
- [122](#)
- Partitioned communication with partial
- completion, [125](#)
- Partitioned communication with tasks,
- [123](#)
- MPI_Precv_init**
- Partitioned communication, [114](#)
- Partitioned communication using threads,
- [122](#)
- Partitioned communication with partial
- completion, [125](#)
- Partitioned communication with tasks,
- [123](#)
- MPI_PROBE**
- Client-server (with error), [98](#)
- Client-server with probe, [97](#)
- MPI_Psend_init**
- Partitioned communication, [114](#)
- Partitioned communication using threads,
- [122](#)
- Partitioned communication with partial
- completion, [125](#)
- Partitioned communication with tasks,
- [123](#)
- MPI_Publish_name**
- Name publishing, [537](#)
- MPI_PUT**
- Fortran 90 register optimization in RMA,
- [851](#)
- MPI_Put**
- Active target and local reads in RMA, [610](#)
- Put with fence, [616](#)
- Put with PSCW, [616](#)
- Read data in RMA, [608](#)
- Read data in RMA (unsafe), [609](#)
- Register and Compiler Optimization, [615](#)
- RMA Lock and unlock, [597](#)
- RMA Start and complete, [591](#)
- MPI_Queue_create**
- Ring exchange with queued
- communication, [640](#)
- MPI_Queue_free**
- Ring exchange with queued
- communication, [640](#)
- MPI_Queue_wait**
- Ring exchange with queued
- communication, [640](#)
- MPI_RECV**
- Client-server (with error), [98](#)
- Client-server with probe, [97](#)
- Datatype matching, [49](#)
- Datatypes, [49](#)

Errant message exchange, 59	1
Exchange relies on buffering, 59	2
Fortran CHARACTER, 50	3
Matching type with datatypes, 159	4
Message exchange, 59	5
Message matching, 58	6
Nonovertaking messages, 57	7
Progress semantics, 84	8
MPI_Recv	9
Deadlock due to synchronization through shared memory, 614	10
Hello world, 33	11
Progress of nonblocking collectives, 300	12
MPI_Recv_init	13
Ring exchange with queued communication, 640	14
MPI_REDUCE	15
Defining a user function, 239	16
Reduction, 228	17
Reduction of a vector, 229	18
Reduction with maxloc, 233	19
MPI_Reduce	20
Library Example #1, 345	21
Reduction with maxloc, 233	22
Reduction with minloc, 234	23
Reduction with user-defined op, 238	24
MPI_REQUEST_FREE	25
Nonblocking send and receive with request free, 83	26
MPI_Request_free	27
Finalize and request free, 487	28
MPI_Rget	29
Requests in RMA, 620	30
MPI_Rput	31
Requests in RMA, 620	32
MPI_Scan	33
User-defined operation with Scan, 249	34
MPI_Scatter	35
Scatter, 212	36
MPI_Scatterv	37
Scatterv, 212	38
Scatterv with vector datatype, 213	39
MPI_SEND	40
Client-server (with error), 98	41
Client-server with probe, 97	42
Datatype matching, 49	43
Datatypes, 49	44
Errant message exchange, 59	45
Exchange relies on buffering, 59	46
Fortran 90	47
subarray as buffer, 839	48
vector subscripts, 842	
Fortran CHARACTER, 50	
Matching type with datatypes, 159	
Message exchange, 59	
Progress semantics, 84	
MPI_Send	
Hello world, 33	
Measurement wrapper, 735	
Mixing collective and point-to-point, 301	
Pack/Unpack with struct datatype, 182	
Progress of nonblocking collectives, 300	
Using datatypes with array of structures, 172	
Using datatypes with array of structures with absolute addresses, 174	
Using datatypes with unions, 175	
MPI_Send_init	
Ring exchange with queued communication, 640	
MPI_SENDECV	
Neighbor alltoall, 414	
Nested vector datatypes, 171	
Send/receive of a 3D array, 170	
Transpose with datatypes, 171	
Using indexed datatype, 170	
MPI_SENDECV_REPLACE	
Using Cart_shift, 404	
MPI_SESSION_FINALIZE	
Finalize in the Sessions Model, 495	
Using process set query in group creation, 503	
MPI_Session_finalize	
Creating a communicator using the Sessions Model, 499	
Using process set query in group creation, 501	
MPI_SESSION_GET_NTH_PSET	
Using process set query in group creation, 503	
MPI_Session_get_nth_pset	
Using process set query in group creation, 501	
MPI_SESSION_GET_NUM_PSETS	
Using process set query in group creation, 503	
MPI_Session_get_num_psets	
Using process set query in group creation, 501	
MPI_SESSION_INIT	
Using process set query in group creation, 503	
MPI_Session_init	
Creating a communicator using the Sessions Model, 499	
Using process set query in group creation, 501	
MPI_SSEND	

- 1 Message matching, 58
- 2 Progress semantics, 84
- 3 MPI_T_cvar_get_info
- 4 Listing names of control variables, 748
- 5 MPI_T_cvar_handle_alloc
- 6 Reading a control variable, 751
- 7 MPI_T_cvar_handle_free
- 8 Reading a control variable, 751
- 9 MPI_T_cvar_read
- 10 Reading a control variable, 751
- 11 MPI_T_finalize
- 12 Basic usage of performance variables, 762
- 13 MPI_T_init_thread
- 14 Basic usage of performance variables, 762
- 15 MPI_T_pvar_get_info
- 16 Basic usage of performance variables, 762
- 17 MPI_T_pvar_handle_alloc
- 18 Basic usage of performance variables, 762
- 19 MPI_T_pvar_handle_free
- 20 Basic usage of performance variables, 762
- 21 MPI_T_pvar_read
- 22 Basic usage of performance variables, 762
- 23 MPI_T_pvar_session_create
- 24 Basic usage of performance variables, 762
- 25 MPI_T_pvar_start
- 26 Basic usage of performance variables, 762
- 27 MPI_Test_cancelled
- 28 Finalize and cancel, 488
- 29 MPI_TYPE_COMMIT
- 30 Fortran 90
- 31 derived types, 843
- 32 Nested vector datatypes, 171
- 33 Send/receive of a 3D array, 170
- 34 Transpose with datatypes, 171
- 35 Type_commit, 156
- 36 Using Get with indexed datatype, 568
- 37 Using indexed datatype, 170
- 38 MPI_Type_commit
- 39 Gather and Gatherv, 207
- 40 Gather with datatype, 202
- 41 Gatherv with datatype, 203, 204
- 42 Gatherv with struct datatype, 205
- 43 Gatherv with vector datatype, 206
- 44 Pack/Unpack with struct datatype, 182
- 45 Scatterv with vector datatype, 213
- 46 User-defined operation with Scan, 249
- 47 Using datatypes with array of structures,
- 48 172
- Using datatypes with array of structures
- with absolute addresses, 174
- Using datatypes with unions, 175
- MPI_TYPE_CONTIGUOUS
- Matching type with datatypes, 159
- Typemap for contiguous, 130
- Typemap of nested datatypes, 151
- Using Get_count and Get_elements, 160
- MPI_Type_contiguous
- Gather with datatype, 202
- MPI_TYPE_CREATE_DARRAY
- Datatypes for distributed arrays, 147
- MPI_TYPE_CREATE_F90_INTEGER
- Fortran 90
- selected KIND, 832
- MPI_TYPE_CREATE_F90_REAL
- Fortran 90
- selected KIND, 832
- MPI_TYPE_CREATE_HVECTOR
- Nested vector datatypes, 171
- Send/receive of a 3D array, 170
- MPI_Type_create_hvector
- Using datatypes with array of structures,
- 172
- Using datatypes with array of structures
- with absolute addresses, 174
- MPI_TYPE_CREATE_INDEXED_BLOCK
- Using Get with indexed datatype, 568
- MPI_TYPE_CREATE_RESIZED
- Fortran 90
- derived types, 843
- MPI_Type_create_resized
- Using datatypes with unions, 175
- MPI_TYPE_CREATE_STRUCT
- Fortran 90
- derived types, 843
- Interlanguage communication, 871
- Transpose with datatypes, 171
- Typemap for create struct, 139
- Typemap of nested datatypes, 151
- MPI_Type_create_struct
- Gather and Gatherv, 207
- Gatherv with struct datatype, 205
- Pack/Unpack with struct datatype, 182
- User-defined operation with Scan, 249
- Using datatypes with array of structures,
- 172
- Using datatypes with array of structures
- with absolute addresses, 174
- MPI_TYPE_CREATE_SUBARRAY
- Local subarray for file output in
- Fortran 90, 730
- MPI_Type_create_subarray
- Local subarray for file output, 730
- MPI_TYPE_EXTENT
- Using Get with indexed datatype, 568
- MPI_TYPE_FREE
- Using Get with indexed datatype, 568
- MPI_Type_get_contents
- Decoding a datatype, 176

MPI_Type_get_envelope	1
Decoding a datatype, 176	2
MPI_TYPE_GET_EXTENT	3
Accumulate in RMA, 572	
Nested vector datatypes, 171	
Send/receive of a 3D array, 170	
Transpose with datatypes, 171	
Using Get, 570	
MPI_Type_get_extent	
Using datatypes with array of structures, 172	
MPI_TYPE_GET_VALUE_INDEX	
Retrieving an unnamed predefined value-index handle, 232	
MPI_TYPE_INDEXED	
Typemap for indexed, 134	
Using indexed datatype, 170	
MPI_Type_indexed	
Using datatypes with array of structures, 172	
Using datatypes with array of structures with absolute addresses, 174	
MPI_TYPE_MATCH_SIZE	
Fortran 90	
heterogeneous communication (unsafe), 836	
heterogeneous MPI I/O (unsafe), 836	
MPI_TYPE_MATCH_SIZE	
implementation, 835	
MPI_TYPE_MATCH_SIZE implementation	
MPI_TYPE_MATCH_SIZE, 835	
MPI_Type_size	
Measurement wrapper, 735	
MPI_TYPE_VECTOR	
Nested vector datatypes, 171	
Send/receive of a 3D array, 170	
Transpose with datatypes, 171	
Typemap for vector, 131	
MPI_Type_vector	
Gatherv with datatype, 203, 204	
Gatherv with vector datatype, 206	
Scatterv with vector datatype, 213	
MPI_Unpack	
Pack and Pack_size, 183	
Pack/Unpack with struct datatype, 182	
MPI_Unpublish_name	
Name publishing, 537	
MPI_WAIT	
Client server code with waitsome, 91	
Client-server, 91	
File pointer update semantics, 684	
Message ordering for nonblocking operations, 84	
Nonblocking point-to-point, 82	
Nonblocking send and receive with request free, 83	
Progress semantics, 84	
MPI_Wait	4
Erroneous matching of blocking and nonblocking collectives, 300	5
Erroneous matching of collectives, 299	6
Mixing blocking and nonblocking collective operations, 299	7
Mixing collective and point-to-point, 301	8
Progress of nonblocking collectives, 300	9
MPI_Waitall	10
Mixing collective and point-to-point, 301	11
Overlapping communicators and collectives, 301	12
Pipelining nonblocking collectives, 301	13
Requests in RMA, 620	14
MPI_WAITANY	15
Client-server, 91	16
MPI_Waitany	17
Requests in RMA, 620	18
MPI_WAITSOME	19
Client server code with waitsome, 91	20
MPI_Win_attach	21
Linked list in RMA, 621	22
MPI_Win_complete	23
Active target and local reads in RMA, 610	24
Double buffer in RMA, 617	25
Get with PSCW, 617	26
Public and private memory in RMA, 610	27
Put with PSCW, 616	28
RMA Start and complete, 591	29
MPI_WIN_CREATE	30
Accumulate in RMA, 572	31
Using Get, 570	32
Using Get with indexed datatype, 568	33
MPI_Win_create_dynamic	34
Linked list in RMA, 621	35
MPI_Win_detach	36
Linked list in RMA, 621	37
MPI_WIN_FENCE	38
Accumulate in RMA, 572	39
Fortran 90 register optimization in RMA, 851	40
Using Get, 570	41
Using Get with indexed datatype, 568	42
MPI_Win_fence	43
Deadlock due to synchronization through shared memory, 614	44
Get with fence, 616	45
Put with fence, 616	46
Register and Compiler Optimization, 615	47
MPI_Win_flush	48
Counting semaphore (nonscalable), 618	

- 1 Critical region with Compare-and-Swap, 619
- 2
- 3 Critical region with RMA, 619
- 4 Linked list in RMA, 621
- 5 Read data in RMA (unsafe), 609
- 6 MPI_Win_flush_all
- 7 Critical region with RMA, 619
- 8 MPI_Win_flush_local
- 9 Update location in unified memory model, 608
- 10 MPI_WIN_FREE
- 11 Accumulate in RMA, 572
- 12 Using Get, 570
- 13 MPI_Win_lock
- 14 Active target and local reads in RMA, 610
- 15 Public and private memory in RMA, 609, 610
- 16 Read data in RMA, 608
- 17 RMA Lock and unlock, 597
- 18 Update location in separate memory model, 607
- 19 MPI_Win_lock_all
- 20 Linked list in RMA, 621
- 21 Read data in RMA (unsafe), 609
- 22 Requests in RMA, 620
- 23 Shared memory windows, 620
- 24 MPI_Win_post
- 25 Active target and local reads in RMA, 610
- 26 Double buffer in RMA, 617
- 27 Get with PSCW, 617
- 28 Public and private memory in RMA, 610
- 29 Put with PSCW, 616
- 30 MPI_Win_shared_query
- 31 Deadlock due to synchronization through shared memory, 614
- 32 MPI_Win_start
- 33 Active target and local reads in RMA, 610
- 34 Double buffer in RMA, 617
- 35 Get with PSCW, 617
- 36 Public and private memory in RMA, 610
- 37 Put with PSCW, 616
- 38 RMA Start and complete, 591
- 39 MPI_Win_sync
- 40 shared memory windows, 620
- 41 MPI_Win_sync
- 42 Counting semaphore (nonscalable), 618
- 43 Critical region with Compare-and-Swap, 619
- 44 Critical region with RMA, 619
- 45 Update location in unified memory model, 608
- 46 MPI_Win_unlock
- 47 Active target and local reads in RMA, 610
- 48
- Public and private memory in RMA, 609, 610
- Read data in RMA, 608
- RMA Lock and unlock, 597
- Update location in separate memory model, 607
- MPI_Win_unlock_all
- Linked list in RMA, 621
- Read data in RMA (unsafe), 609
- Requests in RMA, 620
- MPI_Win_wait
- Active target and local reads in RMA, 610
- Double buffer in RMA, 617
- Get with PSCW, 617
- Public and private memory in RMA, 610
- Put with PSCW, 616
- MPI_WTIME
- Using MPI_Wtime, 469
- MPI_Wtime
- Measurement wrapper, 735
- mpiexec, 512
- mpiexec and environment variables
- MPI_INFO_ENV, 482
- Name publishing
- MPI_Comm_accept, 537
- MPI_Comm_connect, 537
- MPI_Open_port, 537
- MPI_Publish_name, 537
- MPI_Unpublish_name, 537
- Neighbor allgather
- MPI_NEIGHBOR_ALLGATHER, 410
- Neighbor alltoall
- MPI_CART_SHIFT, 414
- MPI_CARTDIM_GET, 414
- MPI_NEIGHBOR_ALLTOALL, 414
- MPI_SENDRECV, 414
- Neighborhood collective communication, 434
- MPI_CART_CREATE, 434
- MPI_CART_GET, 434
- MPI_CART_SHIFT, 434
- MPI_DIMS_CREATE, 434
- MPI_NEIGHBOR_ALLTOALL, 434
- MPI_NEIGHBOR_ALLTOALLW, 434
- MPI_NEIGHBOR_ALLTOALLW_INIT, 434
- Nested vector datatypes
- MPI_SENDRECV, 171
- MPI_TYPE_COMMIT, 171
- MPI_TYPE_CREATE_HVECTOR, 171
- MPI_TYPE_GET_EXTENT, 171
- MPI_TYPE_VECTOR, 171
- Nonblocking operations, 82, 83
- message ordering, 84
- progress, 84

Nonblocking point-to-point		
MPI_Irecv, 82		
MPI_Isend, 82		
MPI_Wait, 82		
Nonblocking send and receive with request free		
MPI_Irecv, 83		
MPI_Isend, 83		
MPI_REQUEST_FREE, 83		
MPI_Wait, 83		
Nondeterministic program with MPI_Bcast,		
297		
Nondeterministic use of Bcast		
MPI_Bcast, 297		
Nonovertaking messages, 57		
MPI_BSEND, 57		
MPI_RECV, 57		
Overlap computation and output		
MPI_File_write_all_begin, 728		
MPI_File_write_all_end, 728		
Overlapping communicators, 301		
Overlapping communicators and collectives		
MPI_allreduce, 301		
MPI_Waitall, 301		
Pack and Pack_size		
MPI_Gather, 183		
MPI_Gatherv, 183		
MPI_Pack, 183		
MPI_Pack_size, 183		
MPI_Unpack, 183		
Pack/Unpack with struct datatype		
MPI_Get_address, 182		
MPI_Pack, 182		
MPI_Send, 182		
MPI_Type_commit, 182		
MPI_Type_create_struct, 182		
MPI_Unpack, 182		
Partitioned communication		
Equal send/rcv partitioning, 122		
MPI_Pready, 114		
MPI_Precv_init, 114		
MPI_Psend_init, 114		
Partial completion notification, 125		
Send with tasks, 123		
Simple example, 114		
Partitioned communication using threads		
MPI_Pready, 122		
MPI_Precv_init, 122		
MPI_Psend_init, 122		
Partitioned communication with partial completion		
MPI_Parrived, 125		
MPI_Pready, 125		
MPI_Precv_init, 125		
MPI_Psend_init, 125		
Partitioned communication with tasks		
MPI_Pready, 123		
MPI_Precv_init, 123		
MPI_Psend_init, 123		
Pipelining nonblocking collective operations,		
301		
Pipelining nonblocking collectives		
MPI_Ibcast, 301		
MPI_Waitall, 301		
Point-to-point		
Hello world, 33		
Profiling interface		
Fortran 2008 measurement wrapper, 822		
measurement wrapper, 735		
Progress of matching pairs, 58		
Progress of nonblocking collective operations,		
300		
Progress of nonblocking collectives		
MPI_Ibarrier, 300		
MPI_Recv, 300		
MPI_Send, 300		
MPI_Wait, 300		
Progress semantics		
MPI_Irecv, 84		
MPI_RECV, 84		
MPI_SEND, 84		
MPI_SSEND, 84		
MPI_WAIT, 84		
Public and private memory in RMA		
MPI_Barrier, 609, 610		
MPI_Get, 609, 610		
MPI_Win_complete, 610		
MPI_Win_lock, 609, 610		
MPI_Win_post, 610		
MPI_Win_start, 610		
MPI_Win_unlock, 609, 610		
MPI_Win_wait, 610		
Put with fence		
MPI_Put, 616		
MPI_Win_fence, 616		
Put with PSCW		
MPI_Put, 616		
MPI_Win_complete, 616		
MPI_Win_post, 616		
MPI_Win_start, 616		
MPI_Win_wait, 616		
queued communication		
ring exchange, 640		
Read data in RMA		
MPI_Barrier, 608		

- 1 MPI_Put, 608
- 2 MPI_Win_lock, 608
- 3 MPI_Win_unlock, 608
- 4 Read data in RMA (unsafe)
 - 5 MPI_Barrier, 609
 - 6 MPI_Put, 609
 - 7 MPI_Win_flush, 609
 - 8 MPI_Win_lock_all, 609
 - 9 MPI_Win_unlock_all, 609
- 10 Read to end of file
 - 11 MPI_FILE_CLOSE, 680
 - 12 MPI_FILE_OPEN, 680
 - 13 MPI_FILE_READ, 680
 - 14 MPI_FILE_SET_VIEW, 680
- 15 Reading a control variable
 - 16 MPI_T_cvar_handle_alloc, 751
 - 17 MPI_T_cvar_handle_free, 751
 - 18 MPI_T_cvar_read, 751
- 19 Recursive splitting of COMM_WORLD
 - 20 MPI_Comm_split_type, 337
- 21 Reduction
 - 22 MPI_REDUCE, 228
- 23 Reduction of a vector
 - 24 MPI_REDUCE, 229
- 25 Reduction with maxloc
 - 26 MPI_REDUCE, 233
 - 27 MPI_Reduce, 233
- 28 Reduction with minloc
 - 29 MPI_Reduce, 234
- 30 Reduction with user-defined op
 - 31 MPI_Op_create, 238
 - 32 MPI_Reduce, 238
- 33 Register and Compiler Optimization
 - 34 MPI_Put, 615
 - 35 MPI_Win_fence, 615
- 36 Requesting and querying memory allocation
 - 37 kinds in the Sessions Model
 - 38 "mpi_assert_memory_alloc_kinds", 509
 - 39 "mpi_memory_alloc_kinds", 509
- 40 Requests in RMA
 - 41 MPI_Rget, 620
 - 42 MPI_Rput, 620
 - 43 MPI_Waitall, 620
 - 44 MPI_Waitany, 620
 - 45 MPI_Win_lock_all, 620
 - 46 MPI_Win_unlock_all, 620
- 47 Retrieving an unnamed predefined value-index
 - 48 handle
 - MPI_TYPE_GET_VALUE_INDEX, 232
- Ring exchange with queued communication
 - MPI_Enqueue_startall, 640
 - MPI_Enqueue_waitall, 640
 - MPI_Matchall, 640
 - MPI_Queue_create, 640
 - MPI_Queue_free, 640
 - MPI_Queue_wait, 640
 - MPI_Recv_init, 640
 - MPI_Send_init, 640
- RMA Lock and unlock
 - MPI_Put, 597
 - MPI_Win_lock, 597
 - MPI_Win_unlock, 597
- RMA Start and complete
 - MPI_Put, 591
 - MPI_Win_complete, 591
 - MPI_Win_start, 591
- Rules for finalize
 - MPI_Finalize, 487
- Scatter
 - MPI_Scatter, 212
- Scatterv
 - MPI_Scatterv, 212
- Scatterv with vector datatype
 - MPI_Scatterv, 213
 - MPI_Type_commit, 213
 - MPI_Type_vector, 213
- Send/receive of a 3D array
 - MPI_SENDRCV, 170
 - MPI_TYPE_COMMIT, 170
 - MPI_TYPE_CREATE_HVECTOR, 170
 - MPI_TYPE_GET_EXTENT, 170
 - MPI_TYPE_VECTOR, 170
- Shared memory windows
 - MPI_F_SYNC_REG, 620
 - MPI_Win_lock_all, 620
 - MPI_Win_sync, 620
- Splitting into NUMANode subcommunicators
 - MPI_Comm_split_type, 333, 444
 - MPI_Get_hw_resource_info, 444
- Subgroup cart process topology
 - MPI_CART_SUB, 405
- Threads and MPI, 514
- Three-Group "Pipeline"
 - MPI_Comm_split, 356
 - MPI_Intercomm_create, 356
- Three-Group "Ring"
 - MPI_Comm_split, 357
 - MPI_Intercomm_create, 357
- Tool information interface
 - basic usage of performance variables, 762
 - listing names of all control variables, 748
 - reading the value of a control variable, 751
- Topologies, 434
- Transpose with datatypes
 - MPI_SENDRCV, 171
 - MPI_TYPE_COMMIT, 171

MPI_TYPE_CREATE_STRUCT, 171	
MPI_TYPE_GET_EXTENT, 171	
MPI_TYPE_VECTOR, 171	
Type_commit	
MPI_TYPE_COMMIT, 156	
Typemap, 129–131, 134, 139, 147	
Typemap for contiguous	
MPI_TYPE_CONTIGUOUS, 130	
Typemap for create struct	
MPI_TYPE_CREATE_STRUCT, 139	
Typemap for indexed	
MPI_TYPE_INDEXED, 134	
Typemap for vector	
MPI_TYPE_VECTOR, 131	
Typemap of nested datatypes	
MPI_TYPE_CONTIGUOUS, 151	
MPI_TYPE_CREATE_STRUCT, 151	
Update location in separate memory model	
MPI_Barrier, 607	
MPI_Get, 607	
MPI_Win_lock, 607	
MPI_Win_unlock, 607	
Update location in unified memory model	
MPI_Barrier, 608	
MPI_Get, 608	
MPI_Win_flush_local, 608	
MPI_Win_sync, 608	
Use of Alloc_mem in Fortran	
MPI_ALLOC_MEM, 447	
MPI_FREE_MEM, 447	
Use of Alloc_mem in Fortran with Cray pointers	
MPI_ALLOC_MEM, 447	
MPI_FREE_MEM, 447	
User-defined operation with Scan	
MPI_Op_create, 249	
MPI_Scan, 249	
MPI_Type_commit, 249	
MPI_Type_create_struct, 249	
User-defined reduce	
MPI_Grequest_complete, 647	
MPI_Grequest_start, 647	
Using Alloc_mem	
MPI_Alloc_mem, 448	
Using Cart_shift	
MPI_CART_COORDS, 404	
MPI_CART_RANK, 404	
MPI_CART_SHIFT, 404	
MPI_SENDRECV_REPLACE, 404	
Using datatypes with array of structures	
MPI_Aint, 172	
MPI_Get_address, 172	
MPI_Send, 172	
MPI_Type_commit, 172	
MPI_Type_create_hvector, 172	
MPI_Type_create_struct, 172	
MPI_Type_get_extent, 172	
MPI_Type_indexed, 172	
Using datatypes with array of structures with absolute addresses	
MPI_Get_address, 174	
MPI_Send, 174	
MPI_Type_commit, 174	
MPI_Type_create_hvector, 174	
MPI_Type_create_struct, 174	
MPI_Type_indexed, 174	
Using datatypes with unions	
MPI_Get_address, 175	
MPI_Send, 175	
MPI_Type_commit, 175	
MPI_Type_create_resized, 175	
Using Get	
MPI_GET, 570	
MPI_TYPE_GET_EXTENT, 570	
MPI_WIN_CREATE, 570	
MPI_WIN_FENCE, 570	
MPI_WIN_FREE, 570	
Using Get with indexed datatype	
MPI_GET, 568	
MPI_TYPE_COMMIT, 568	
MPI_TYPE_CREATE_INDEXED_BLOCK, 568	
MPI_TYPE_EXTENT, 568	
MPI_TYPE_FREE, 568	
MPI_WIN_CREATE, 568	
MPI_WIN_FENCE, 568	
Using Get_count and Get_elements	
MPI_GET_COUNT, 160	
MPI_GET_ELEMENTS, 160	
MPI_TYPE_CONTIGUOUS, 160	
Using Group_excl	
MPI_Comm_create, 343	
MPI_Group_excl, 343	
MPI_Group_free, 343	
Using indexed datatype	
MPI_SENDRECV, 170	
MPI_TYPE_COMMIT, 170	
MPI_TYPE_INDEXED, 170	
Using MPI_Wtime	
MPI_WTIME, 469	
Using Pack	
MPI_Pack, 182	
Using process set query in group creation	
MPI_BARRIER, 503	
MPI_COMM_CREATE_FROM_GROUP, 503	

MPI_GROUP_FROM_SESSION_PSET,
503
MPI_Group_from_session_pset, 501
MPI_SESSION_FINALIZE, 503
MPI_Session_finalize, 501
MPI_SESSION_GET_NTH_PSET, 503
MPI_Session_get_nth_pset, 501
MPI_SESSION_GET_NUM_PSETS, 503
MPI_Session_get_num_psets, 501
MPI_SESSION_INIT, 503
MPI_Session_init, 501

Virtual topologies, 434

MPI Constant and Predefined Handle Index

This index lists predefined MPI constants and handles, including info keys and values. Underlined page numbers give the location of the primary definition or use of the indexed term.

- "access_style", 664, 664, 916
- "accumulate_ops", 548, 605, 606, 611, 916
- "accumulate_ordering", 548, 611, 612, 916
- "alloc_mem", 509, 917
- "alloc_shared_noncontig", 553, 553, 554, 916
- "appnum", 540, 916
- "arch", 482, 526, 916
- "argv", 482, 482, 916

- "cb_block_size", 665, 665, 916
- "cb_buffer_size", 665, 665, 916
- "cb_nodes", 665, 665, 916
- "chunked", 665, 916
- "chunked_item", 665, 916
- "chunked_size", 665, 916
- "coll", 790
- "collective_buffering", 665, 916
- "command", 482, 916
- "construct", 790

- "default", 507

- "external32", 184, 705, 706, 706, 707–711, 716, 831–833, 836, 915, 1102, 1109

- "false", 339, 340, 443, 472, 548, 549, 665, 917
- "file", 482, 526, 916
- "file_perm", 664, 665, 916
- "filename", 665, 916

- "group", 788

- "host", 482, 526, 916

- "internal", 705, 706, 706, 707, 716, 915
- "io_node_list", 665, 916
- "ip_address", 536, 916
- "ip_port", 536, 916

- "local", 790

- "maxprocs", 482, 483, 916
- "mpi", 509, 917
- "mpi://", 443, 495, 915
- "mpi://SELF", 495, 496, 656, 915
- "mpi://WORLD", 495, 496, 501, 511, 539, 915
- MPI_2DOUBLE_PRECISION, 231, 231, 886, 901
- MPI_2INT, 231, 231, 886, 901
- MPI_2INTEGER, 231, 231, 886, 901
- MPI_2REAL, 231, 231, 886, 901
- MPI_ABI_SUBVERSION, 874, 905
- MPI_ABI_VERSION, 874, 905
- "mpi_accumulate_granularity", 548, 916, 1094
- MPI_ADDRESS_KIND, 23, 23, 37, 129, 148, 166, 359, 446, 551, 554, 556, 825, 838, 857, 858, 867, 898, 900
- MPI_AINT, 37, 37, 129, 228, 558, 710, 857, 884, 899, 900, 1107–1109
- "mpi_aint_size", 875, 916
- MPI_ANY_SOURCE, 39, 40, 40, 46, 47, 57, 77–80, 96–100, 109, 340, 380, 441, 633, 788, 789, 793, 794, 877, 895
- MPI_ANY_TAG, 21, 39, 40, 40, 42, 46, 47, 77–80, 96–100, 102, 103, 109, 112, 339, 633, 895, 1104
- MPI_APPNUM, 540, 540, 898
- MPI_ARGV_NULL, 22, 520, 521, 524, 838, 894
- MPI_ARGVS_NULL, 22, 524, 524, 838, 894
- "mpi_assert_allow_overtaking", 340, 340, 916, 1097
- "mpi_assert_exact_length", 340, 916, 1097
- "mpi_assert_memory_alloc_kinds", 340, 507, 508, 549, 666, 916, 1094
- C example usage, 509
- "mpi_assert_no_any_source", 340, 916, 1097
- "mpi_assert_no_any_tag", 339, 916, 1097
- "mpi_assert_strict_persistent_collective_ordering", 340, 340, 1094
- MPI_ASYNC_PROTECTS_NONBLOCKING, 620, 812, 813, 815, 817, 820, 827, 829, 849, 898, 1106
- MPI_BAND, 227, 228, 889, 902
- MPI_BOR, 227, 228, 889, 902
- MPI_BOTTOM, 12, 22, 23, 44, 61, 62, 121, 148, 148, 161, 162, 193, 390, 392, 522,

1	557, 561, 813, 816, 823, 838, 842,	MPI_COMBINER_SUBARRAY, 163, <u>169</u> , 896
2	845–848, 850–852, 855, 864–866, 871,	MPI_COMBINER_VALUE_INDEX, 163, <u>169</u> ,
3	893, 1113	232, 896, 1093
4	MPI_BSEND_OVERHEAD, <u>71</u> , 895	MPI_COMBINER_VECTOR, 163, <u>168</u> , 896
5	MPI_BUFFER_AUTOMATIC, 22, 61, <u>61</u> , 62,	MPI_COMM_DUP_FN, 26, 362, <u>362</u> , 363, 818,
6	65, 69, 893, 1093	904, 1100, 1106
7	MPI_BXOR, <u>227</u> , 228, 889, 902	MPI_COMM_NULL, 309, 325, 326, 328–330,
8	MPI_BYTE, 35, <u>35</u> , 36, 48–51, 184, 228, 654,	332, 334, 335, 338, 339, 354, 376, 385,
9	706, 710, 721, 871, 887, 899, 900, 1110	386, 522, 542, 543, 790, 860, 890, 898,
10	Fortran example usage, 49	1094, 1111
11	MPI_C_BOOL, <u>36</u> , 227, 710, 886, 899, 1102,	"MPI_COMM_NULL", <u>376</u> , 915
12	1107–1109	MPI_COMM_NULL_COPY_FN, 25, 26, 362,
13	MPI_C_COMPLEX, <u>36</u> , 228, 710, 899, 1102,	<u>362</u> , 363, 814, 866, 904, 1100, 1106
14	1107–1109	MPI_COMM_NULL_DELETE_FN, 26, 362,
15	MPI_C_DOUBLE_COMPLEX, <u>36</u> , 228, 710,	<u>362</u> , 363, 904, 1100
16	885, 899, 1107–1109	"MPI_COMM_PARENT", <u>376</u> , 915
17	MPI_C_FLOAT_COMPLEX, <u>36</u> , 228, 710, 885,	MPI_COMM_SELF, 28, 309, 309, 328, 341,
18	899, 1107–1109	358, 376, 449, 453, 480, 481, 486, 488,
19	MPI_C_LONG_DOUBLE_COMPLEX, <u>36</u> , 228,	490, 491, 507, 508, 542, 656, 809, 890,
20	710, 886, 899, 1107–1109	898, 1090, 1096, 1109
21	MPI_CART, 395, 896	"MPI_COMM_SELF", <u>376</u> , 915
22	MPI_CHAR, <u>36</u> , 51, 140, 229, 710, 743, 744,	MPI_COMM_TYPE_HW_GUIDED, <u>332</u> , 333,
23	887, 899, 1107	444, 897, 1096
24	MPI_CHARACTER, <u>35</u> , 50, 51, 229, 710, 885,	MPI_COMM_TYPE_HW_UNGUIDED, 335,
25	900	<u>335</u> , <u>336</u> , 897, 1096
26	Fortran example usage, 50	MPI_COMM_TYPE_RESOURCE_GUIDED,
27	MPI_COMBINER_CONTIGUOUS, 163, <u>167</u> ,	<u>333</u> , 334, 444, 897, 1094
28	896	MPI_COMM_TYPE_SHARED, 332, <u>332</u> , 333,
29	MPI_COMBINER_DARRAY, 163, <u>169</u> , 896	334, 614, 897, 1104
30	MPI_COMBINER_DUP, 163, <u>167</u> , 896	MPI_COMM_WORLD, 21, 31, 33, 34, 38, 307,
31	MPI_COMBINER_F90_COMPLEX, 163, <u>169</u> ,	309, <u>309</u> , 311, 312, 319, 321, 333,
32	896	335–337, 341, 344, 352, 376, 440–442,
33	MPI_COMBINER_F90_INTEGER, 163, <u>169</u> ,	444, 453, 464, 479–481, 483, 485–487,
34	896	489, 491, 506–508, 511, 512, 517–519,
35	MPI_COMBINER_F90_REAL, 163, <u>169</u> , 896	523, 525, 538–542, 705, 727, 750, 758,
36	MPI_COMBINER_HINDEXED, 26, 163, <u>168</u> ,	802, 803, 809, 859, 871, 876, 890, 891,
37	896	898, 923, 1090, 1096, 1112
38	MPI_COMBINER_HINDEXED_BLOCK, 163,	"MPI_COMM_WORLD", <u>376</u> , 915
39	<u>168</u> , 896, 1103	MPI_COMPLEX, <u>35</u> , 228, 709, 710, 829, 885,
40	MPI_COMBINER_HINDEXED_INTEGER, 26,	900
41	<u>808</u> , 1102	MPI_COMPLEX16, <u>37</u> , 228, 711, 834, 880, 888,
42	MPI_COMBINER_HVECTOR, 26, 163, <u>168</u> ,	901
43	896	"mpi_complex16_supported", 880, 916
44	MPI_COMBINER_HVECTOR_INTEGER, 26,	MPI_COMPLEX32, <u>37</u> , 228, 711, 834, 880, 888,
45	<u>808</u> , 1102	901
46	MPI_COMBINER_INDEXED, 163, <u>168</u> , 896	"mpi_complex32_supported", 880, 916
47	MPI_COMBINER_INDEXED_BLOCK, 163,	MPI_COMPLEX4, <u>37</u> , 228, 711, 880, 888, 901
48	<u>168</u> , 896	"mpi_complex4_supported", 880, 916
	MPI_COMBINER_NAMED, 163, 164, <u>167</u> , 232,	MPI_COMPLEX8, <u>37</u> , 228, 711, 834, 880, 888,
	896	901
	MPI_COMBINER_RESIZED, 163, <u>169</u> , 896	"mpi_complex8_supported", 880, 916
	MPI_COMBINER_STRUCT, 26, 163, <u>168</u> , 896	MPI_CONGRUENT, <u>320</u> , 351, 896
	MPI_COMBINER_STRUCT_INTEGER, 26,	MPI_CONVERSION_FN_NULL, <u>715</u> , 904, 1100
	<u>808</u> , 1102	MPI_CONVERSION_FN_NULL_C, <u>715</u> , 857,

904, 1096	
MPI_COUNT, 37, <u>37</u> , 129, 228, 237, 710, 743,	
857, 884, 899, 900, 1103	
MPI_COUNT_KIND, <u>23</u> , 24, 37, 857, 858, <u>898</u> ,	
900, 1103	
"mpi_count_size", <u>875</u> , 916	
MPI_CXX_BOOL, <u>37</u> , 227, 710, 711, 886, 900,	
1101	
MPI_CXX_DOUBLE_COMPLEX, <u>37</u> , 228, 710,	
711, 885, 900, 1101	
MPI_CXX_FLOAT_COMPLEX, <u>37</u> , 228, 710,	
711, 885, 900, 1101	
MPI_CXX_LONG_DOUBLE_COMPLEX, <u>37</u> ,	
228, 710, 711, 886, 900, 1101	
MPI_DATATYPE_NULL, <u>157</u> , 232, 378, 884,	
899, 1094	
"MPI_DATATYPE_NULL", <u>378</u> , 915	
MPI_DISPLACEMENT_CURRENT, <u>667</u> , <u>667</u> ,	
897, 1113	
MPI_DIST_GRAPH, <u>395</u> , 896, 1108	
MPI_DISTRIBUTE_BLOCK, <u>144</u> , 145, 895	
MPI_DISTRIBUTE_CYCLIC, <u>144</u> , 145, 895	
MPI_DISTRIBUTE_DFLT_DARG, 145, <u>145</u> ,	
895	
MPI_DISTRIBUTE_NONE, 145, <u>145</u> , 895	
MPI_DOUBLE, <u>36</u> , 227, 710, 743, 753, 754,	
829, 885, 899	
MPI_DOUBLE_COMPLEX, <u>36</u> , 228, 709, 710,	
829, 880, 885, 901	
"mpi_double_complex_supported", <u>880</u> , 916	
MPI_DOUBLE_INT, <u>231</u> , <u>231</u> , 886, 901	
MPI_DOUBLE_PRECISION, <u>35</u> , 227, 710, <u>829</u> ,	
885, 900	
"mpi_double_precision_size", <u>879</u> , 916	
MPI_DUP_FN, <u>26</u> , <u>362</u> , <u>798</u> , 905	
MPI_ERR_ABI, <u>879</u> , 881, 892, 1090	
MPI_ERR_ACCESS, <u>461</u> , <u>659</u> , 728, 892	
MPI_ERR_AMODE, <u>461</u> , <u>657</u> , 728, 892	
MPI_ERR_ARG, <u>461</u> , 891	
MPI_ERR_ASSERT, <u>461</u> , <u>603</u> , 892	
MPI_ERR_BAD_FILE, <u>461</u> , 728, 892	
MPI_ERR_BASE, <u>447</u> , <u>461</u> , <u>603</u> , 892	
MPI_ERR_BUFFER, <u>461</u> , 891	
MPI_ERR_COMM, <u>461</u> , 891	
MPI_ERR_CONVERSION, <u>461</u> , <u>716</u> , 728, 892	
MPI_ERR_COUNT, <u>461</u> , 891	
MPI_ERR_DIMS, <u>461</u> , 891	
MPI_ERR_DISP, <u>461</u> , <u>603</u> , 892	
MPI_ERR_DUP_DATAREP, <u>461</u> , <u>712</u> , 728, 892	
MPI_ERR_ERRHANDLER, <u>461</u> , 892, 1094	
MPI_ERR_FILE, <u>461</u> , 728, 892	
MPI_ERR_FILE_EXISTS, <u>461</u> , 728, 892	
MPI_ERR_FILE_IN_USE, <u>461</u> , <u>659</u> , 728, 892	
MPI_ERR_GROUP, <u>461</u> , 891	
MPI_ERR_IN_STATUS, <u>41</u> , 45, 80, 87, 89, 452,	1
460, 461, 646, 672, 892	2
MPI_ERR_INFO, <u>461</u> , 892	3
MPI_ERR_INFO_KEY, <u>461</u> , <u>473</u> , 892	4
MPI_ERR_INFO_NOKEY, <u>461</u> , <u>473</u> , 892	5
MPI_ERR_INFO_VALUE, <u>461</u> , <u>473</u> , 892	6
MPI_ERR_INTERN, <u>450</u> , <u>461</u> , 891	7
MPI_ERR_IO, <u>461</u> , 728, 892	8
MPI_ERR_KEYVAL, <u>372</u> , <u>461</u> , 892	9
MPI_ERR_LASTCODE, <u>460</u> , <u>462</u> , <u>464</u> ,	10
466, 783, 893	
MPI_ERR_LOCKTYPE, <u>461</u> , <u>603</u> , 892	11
MPI_ERR_NAME, <u>461</u> , <u>536</u> , 892	12
MPI_ERR_NO_MEM, <u>446</u> , <u>461</u> , 892	13
MPI_ERR_NO_SPACE, <u>461</u> , 728, 892	14
MPI_ERR_NO_SUCH_FILE, <u>461</u> , <u>658</u> , 728, 892	15
MPI_ERR_NOT_SAME, <u>461</u> , 728, 892	16
MPI_ERR_OP, <u>462</u> , <u>602</u> , 891	17
MPI_ERR_OTHER, <u>460</u> , <u>462</u> , 891	18
MPI_ERR_PENDING, <u>88</u> , <u>462</u> , 891	19
MPI_ERR_PORT, <u>462</u> , <u>533</u> , <u>533</u> , 892	20
MPI_ERR_PROC_ABORTED, <u>462</u> , <u>506</u> , 892,	21
1097	22
MPI_ERR_PROC_FAILED, <u>462</u> , 789, 790, 794,	23
892	24
MPI_ERR_PROC_FAILED_PENDING, <u>462</u> ,	25
789, 793, 794, 892	26
MPI_ERR_QUEUE, <u>462</u> , <u>628</u> , 629, 639, 892,	27
1089	28
MPI_ERR_QUOTA, <u>462</u> , 728, 892	29
MPI_ERR_RANK, <u>462</u> , <u>602</u> , 891	30
MPI_ERR_READ_ONLY, <u>462</u> , 728, 892	31
MPI_ERR_REQUEST, <u>462</u> , 891	32
MPI_ERR_REVOKED, <u>462</u> , 788-792, 794, 892	33
MPI_ERR_RMA_ATTACH, <u>462</u> , <u>603</u> , 892	34
MPI_ERR_RMA_CONFLICT, <u>462</u> , <u>603</u> , 892	35
MPI_ERR_RMA_FLAVOR, <u>462</u> , <u>603</u> , 892	36
MPI_ERR_RMA_RANGE, <u>462</u> , <u>603</u> , 892	37
MPI_ERR_RMA_SHARED, <u>462</u> , <u>603</u> , 892	38
MPI_ERR_RMA_SYNC, <u>462</u> , <u>603</u> , 892	39
MPI_ERR_ROOT, <u>462</u> , 891	40
MPI_ERR_SERVICE, <u>462</u> , <u>535</u> , 892	41
MPI_ERR_SESSION, <u>462</u> , 892, 1098	42
MPI_ERR_SIZE, <u>462</u> , <u>603</u> , 892	43
MPI_ERR_SPAWN, <u>462</u> , <u>521</u> , <u>521</u> , 522, 892	44
MPI_ERR_TAG, <u>462</u> , 891	45
MPI_ERR_TOPOLOGY, <u>462</u> , 891	46
MPI_ERR_TRUNCATE, <u>462</u> , 891	47
MPI_ERR_TYPE, <u>164</u> , <u>165</u> , <u>462</u> , 727, 891	48
MPI_ERR_UNKNOWN, <u>460</u> , <u>462</u> , 891	
MPI_ERR_UNSUPPORTED_DATAREP, <u>462</u> ,	
728, 892	
MPI_ERR_UNSUPPORTED_OPERATION,	
462, 728, 892	

MPI_ERR_VALUE_TOO_LARGE, 462, [713](#),
 [892](#), [1096](#)
 MPI_ERR_WIN, 462, [603](#), [892](#)
 MPI_ERRCODES_IGNORE, 22, [522](#), [522](#), [838](#),
 [894](#)
 MPI_ERRHANDLER_NULL, [459](#), [890](#), [902](#)
 MPI_ERROR, 41, [41](#), [43](#), [44](#), [80](#), [251](#), [577](#), [651](#),
 [652](#), [862](#), [898](#), [1099](#), [1105](#)
 "mpi_error_range", [788](#), [789](#)
 "mpi_error_uniform", [790](#)
 MPI_ERRORS_ABORT, [449](#), [450](#), [481](#), [526](#),
 [890](#), [902](#), [1096](#)
 "mpi_errors_abort", [526](#), [917](#), [1098](#)
 MPI_ERRORS_ARE_FATAL, [449](#), [450](#), [451](#),
 [468](#), [526](#), [602](#), [727](#), [788](#), [890](#), [902](#), [1096](#)
 "mpi_errors_are_fatal", [526](#), [917](#), [1098](#)
 MPI_ERRORS_RETURN, [450](#), [450](#), [451](#), [468](#),
 [506](#), [526](#), [727](#), [871](#), [890](#), [902](#)
 "mpi_errors_return", [526](#), [917](#), [1098](#)
 MPI_F08_STATUS_IGNORE, [863](#), [882](#), [905](#),
 [1105](#)
 MPI_F08_STATUSES_IGNORE, [863](#), [882](#), [905](#),
 [1105](#)
 MPI_F_ERROR, [862](#), [898](#), [1095](#)
 MPI_F_SOURCE, [862](#), [898](#), [1095](#)
 MPI_F_STATUS_IGNORE, [862](#), [882](#), [905](#)
 MPI_F_STATUS_SIZE, [862](#), [898](#), [1095](#)
 MPI_F_STATUSES_IGNORE, [862](#), [882](#), [905](#)
 MPI_F_TAG, [862](#), [898](#), [1095](#)
 MPI_FILE_NULL, [658](#), [727](#), [727](#), [890](#), [903](#)
 MPI_FLOAT, [36](#), [140](#), [225](#), [227](#), [707](#), [710](#), [885](#),
 [899](#)
 MPI_FLOAT_INT, 18, [231](#), [231](#), [886](#), [901](#)
 MPI_GRAPH, [395](#), [896](#)
 MPI_GROUP_EMPTY, 308, [308](#), [314](#), [315](#),
 [325](#)–[328](#), [338](#), [354](#), [792](#), [890](#), [902](#)
 MPI_GROUP_NULL, 308, [308](#), [318](#), [890](#), [902](#)
 MPI_HOST, 5, 26, [802](#), [802](#), [803](#), [898](#), [1093](#)
 "mpi_hw_resource_type", [332](#), [333](#), [333](#), [334](#),
 [337](#), [444](#), [916](#), [1096](#)
 MPI_IDENT, [312](#), [320](#), [896](#)
 MPI_IN_PLACE, 22, [192](#), [221](#), [816](#), [838](#), [893](#)
 MPI_INFO_ENV, [476](#), [482](#), [482](#), [483](#), [491](#), [890](#),
 [902](#), [1089](#), [1104](#)
 Language-independent example usage, [482](#)
 MPI_INFO_NULL, [332](#), [334](#), [392](#), [446](#), [476](#),
 [521](#), [531](#), [627](#), [634](#), [657](#), [659](#), [668](#), [879](#),
 [890](#), [902](#)
 "mpi_initial_errhandler", [481](#), [482](#), [526](#), [916](#),
 [1098](#)
 MPI_INT, 18, [36](#), [128](#), [227](#), [707](#), [709](#), [710](#), [743](#),
 [744](#), [747](#), [752](#), [756](#), [829](#), [871](#), [872](#), [884](#),
 [899](#)
 MPI_INT16_T, [36](#), [227](#), [710](#), [887](#), [899](#),
 [1107](#)–[1109](#)
 MPI_INT32_T, [36](#), [227](#), [710](#), [743](#), [887](#), [899](#),
 [1107](#)–[1109](#)
 MPI_INT64_T, [36](#), [227](#), [710](#), [743](#), [887](#), [899](#),
 [1107](#)–[1109](#)
 MPI_INT8_T, [36](#), [227](#), [710](#), [887](#), [899](#),
 [1107](#)–[1109](#)
 MPI_INTEGER, [35](#), [48](#), [227](#), [710](#), [829](#), [872](#), [878](#),
 [885](#), [900](#)
 MPI_INTEGER1, [37](#), [227](#), [711](#), [834](#), [880](#), [888](#),
 [901](#)
 MPI_INTEGER16, [37](#), [227](#), [711](#), [834](#), [880](#), [888](#),
 [901](#)
 "mpi_integer16_supported", [880](#), [916](#)
 "mpi_integer1_supported", [880](#), [916](#)
 MPI_INTEGER2, [37](#), [227](#), [708](#), [711](#), [834](#), [880](#),
 [888](#), [901](#)
 "mpi_integer2_supported", [880](#), [916](#)
 MPI_INTEGER4, [37](#), [227](#), [711](#), [834](#), [880](#), [888](#),
 [901](#)
 "mpi_integer4_supported", [880](#), [916](#)
 MPI_INTEGER8, [37](#), [227](#), [711](#), [833](#), [834](#), [880](#),
 [888](#), [901](#)
 "mpi_integer8_supported", [880](#), [916](#)
 "mpi_integer_size", [879](#), [916](#)
 MPI_IO, [441](#), [441](#), [898](#)
 MPI_KEYVAL_INVALID, [362](#), [363](#), [364](#), [898](#)
 MPI_LAND, [227](#), [228](#), [889](#), [902](#)
 MPI_LASTUSED_CODE, [464](#), [464](#), [898](#), [1092](#)
 MPI_LB, 26, [808](#), [1102](#)
 MPI_LOCK_EXCLUSIVE, [595](#), [595](#), [897](#)
 MPI_LOCK_SHARED, [595](#), [595](#), [596](#), [897](#)
 MPI_LOGICAL, [35](#), [227](#), [710](#), [885](#), [900](#)
 MPI_LOGICAL1, [37](#), [228](#), [711](#), [834](#), [879](#), [888](#),
 [901](#)
 MPI_LOGICAL16, [37](#), [228](#), [711](#), [834](#), [879](#), [888](#),
 [901](#)
 "mpi_logical16_supported", [879](#), [917](#)
 "mpi_logical1_supported", [879](#), [917](#)
 MPI_LOGICAL2, [37](#), [228](#), [711](#), [834](#), [879](#), [888](#),
 [901](#)
 "mpi_logical2_supported", [879](#), [917](#)
 MPI_LOGICAL4, [37](#), [228](#), [711](#), [834](#), [879](#), [888](#),
 [901](#)
 "mpi_logical4_supported", [879](#), [917](#)
 MPI_LOGICAL8, [37](#), [228](#), [711](#), [834](#), [879](#), [888](#),
 [901](#)
 "mpi_logical8_supported", [879](#), [917](#)
 "mpi_logical_size", [879](#), [917](#)
 MPI_LONG, [36](#), [227](#), [710](#), [884](#), [899](#)
 MPI_LONG_DOUBLE, [36](#), [227](#), [710](#), [886](#), [899](#)
 MPI_LONG_DOUBLE_INT, [231](#), [886](#), [901](#)
 MPI_LONG_INT, [231](#), [231](#), [886](#), [901](#)
 MPI_LONG_LONG, [36](#), [227](#), [884](#), [899](#), [1110](#)

MPI_LONG_LONG_INT, 36 , 227 , 710 , 899 , 1110	1100	1
MPI_LOR, 227 , 228 , 889 , 902	MPI_NULL_COPY_FN, 26 , 362 , 798 , 905	2
MPI_LXOR, 227 , 228 , 889 , 902	MPI_NULL_DELETE_FN, 26 , 362 , 798 , 905	3
MPI_MAX, 225 , 227 , 228 , 232 , 249 , 889 , 902	MPI_OFFSET, 37 , 37 , 228 , 710 , 884 , 899 , 900 , 1107 – 1109	4
MPI_MAX_DATAREP_STRING, 669 , 712 , 894	MPI_OFFSET_KIND, 23 , 23 , 37 , 722 , 838 , 857 , 859 , 897 , 898 , 900	5
MPI_MAX_ERROR_STRING, 459 , 466 , 894	"mpi_offset_size", 875 , 917	6
MPI_MAX_INFO_KEY, 461 , 471 , 471 , 474 , 801 , 894 , 1090	MPI_OP_NULL, 238 , 889 , 902	7
MPI_MAX_INFO_VAL, 461 , 471 , 471 , 894	MPI_ORDER_C, 21 , 142 , 142 , 145 , 895	8
MPI_MAX_LIBRARY_VERSION_STRING, 440 , 440 , 894 , 1103	MPI_ORDER_FORTRAN, 21 , 142 , 142 , 145 , 895	9
MPI_MAX_OBJECT_NAME, 375 , 375 , 376 – 378 , 894 , 1104 , 1111	MPI_PACKED, 18 , 35 , 36 , 48 , 49 , 179 , 180 , 180 , 184 , 709 , 710 , 871 , 884 , 899 , 900	10
MPI_MAX_PORT_NAME, 530 , 894	MPI_PROC_NULL, 34 , 38 , 40 , 97 , 100 , 102 , 103 , 112 , 112 , 194 , 197 , 200 , 202 , 209 , 211 , 226 , 311 , 403 , 408 , 441 , 554 , 555 , 564 , 802 , 803 , 895 , 1103 , 1110 , 1112	11
MPI_MAX_PROCESSOR_NAME, 442 , 442 , 443 , 894 , 1112	MPI_PROD, 227 , 228 , 889 , 902	12
MPI_MAX_PSET_NAME_LEN, 498 , 498 , 894 , 1091 , 1098	"mpi_pset_name", 333 , 334 , 334 , 335 , 917 , 1094	13
MPI_MAX_STRINGTAG_LEN, 338 , 338 , 354 , 894 , 1091 , 1098	MPI_QUEUE_NULL, 628 , 902	14
MPI_MAXLOC, 227 , 227 , 229 , 230 , 232 , 234 , 889 , 902 , 1093	MPI_QUEUE_TYPE_DEFAULT, 627 , 632	15
"mpi_memory_alloc_kinds", 482 , 493 , 507 , 507 , 508 , 526 , 917 , 1089 , 1090 , 1094	MPI_REAL, 35 , 48 , 227 , 709 , 710 , 829 , 830 , 836 , 878 , 885 , 900	16
C example usage, 509	MPI_REAL16, 37 , 227 , 711 , 834 , 880 , 888 , 901	17
MPI_MESSAGE_NO_PROC, 100 , 100 , 102 , 103 , 112 , 890 , 901 , 1103	"mpi_real16_supported", 880 , 917	18
MPI_MESSAGE_NULL, 100 , 102 , 103 , 890 , 901 , 1103	MPI_REAL2, 37 , 227 , 711 , 880 , 888 , 901	19
MPI_MIN, 227 , 228 , 232 , 889 , 902	"mpi_real2_supported", 880 , 917	20
"mpi_minimum_memory_alignment", 446 , 551 , 553 , 554 , 917 , 1097	MPI_REAL4, 37 , 227 , 711 , 829 , 833 , 834 , 880 , 888 , 901	21
MPI_MINLOC, 227 , 227 , 229 , 230 , 232 , 234 , 889 , 902 , 1093	"mpi_real4_supported", 880 , 917	22
MPI_MODE_APPEND, 656 , 657 , 895	MPI_REAL8, 37 , 227 , 711 , 829 , 834 , 880 , 888 , 901 , 1108	23
MPI_MODE_CREATE, 656 , 657 , 665 , 895	"mpi_real8_supported", 880 , 917	24
MPI_MODE_DELETE_ON_CLOSE, 656 , 657 , 658 , 895	"mpi_real_size", 879 , 917	25
MPI_MODE_EXCL, 656 , 657 , 895	MPI_REPLACE, 572 , 572 , 573 , 575 , 576 , 619 , 889 , 902 , 1109 , 1113	26
MPI_MODE_NOCHECK, 596 , 601 , 601 , 602 , 618 , 895	MPI_REQUEST_NULL, 80 , 81 – 83 , 86 – 89 , 94 , 95 , 629 , 646 , 876 , 890 , 903	27
MPI_MODE_NOPRECEDE, 590 , 601 , 601 , 602 , 895	MPI_ROOT, 194 , 895	28
MPI_MODE_NOPUT, 601 , 601 , 602 , 895	MPI_SEEK_CUR, 688 , 695 , 897	29
MPI_MODE_NOSTORE, 601 , 601 , 602 , 895	MPI_SEEK_END, 688 , 695 , 897	30
MPI_MODE_NOSUCCEED, 601 , 602 , 895	MPI_SEEK_SET, 688 , 688 , 695 , 696 , 897	31
MPI_MODE_RDONLY, 656 , 657 , 662 , 895	MPI_SESSION_NULL, 494 , 890 , 903 , 1098	32
MPI_MODE_RDWR, 656 , 657 , 895	"mpi_shared_memory", 333 , 333 , 334 , 917 , 1096	33
MPI_MODE_SEQUENTIAL, 656 , 657 , 659 , 660 , 667 , 672 , 679 , 695 , 720 , 895 , 1113	MPI_SHORT, 36 , 227 , 710 , 884 , 899	34
MPI_MODE_UNIQUE_OPEN, 656 , 657 , 895	MPI_SHORT_INT, 231 , 231 , 886 , 901	35
MPI_MODE_WRONLY, 656 , 657 , 895	MPI_SIGNED_CHAR, 36 , 227 , 229 , 710 , 887 , 899 , 1110	36
MPI_NO_OP, 548 , 575 , 575 , 576 , 889 , 902 ,	MPI_SIMILAR, 312 , 320 , 351 , 896	37
	"mpi_size", 496 , 499 , 499 , 503 , 917 , 1098	38
		39
		40
		41
		42
		43
		44
		45
		46
		47
		48

MPI_SOURCE, [41](#), [41](#), [43](#), [251](#), [651](#), [862](#), [898](#),
[1099](#), [1105](#)
 MPI_STATUS_IGNORE, [12](#), [22](#), [44](#), [44](#), [45](#),
[630](#), [645](#), [672](#), [816](#), [838](#), [862](#), [863](#), [871](#),
[894](#), [905](#), [1103](#)
 MPI_STATUS_SIZE, [41](#), [41](#), [817](#), [898](#), [1105](#)
 MPI_STATUSES_IGNORE, [21](#), [22](#), [44](#), [44](#), [45](#),
[631](#), [645](#), [646](#), [838](#), [862](#), [863](#), [894](#), [905](#)
 MPI_SUBARRAYS_SUPPORTED, [812](#), [813](#),
[816–820](#), [824–827](#), [839–841](#), [898](#), [1105](#)
 MPI_SUBVERSION, [440](#), [905](#)
 MPI_SUCCESS, [25](#), [27](#), [80](#), [88](#), [89](#), [232](#), [361](#),
[362](#), [364–367](#), [370](#), [371](#), [385](#), [459](#), [460](#),
[461](#), [467–469](#), [481](#), [522](#), [716](#), [739](#), [748](#),
[757](#), [759](#), [761](#), [766](#), [770](#), [774](#), [781](#), [783](#),
[785](#), [790](#), [798](#), [891](#), [1097](#)
 MPI_SUM, [227](#), [228](#), [572](#), [871](#), [889](#), [902](#)
 MPI_T_BIND_MPI_COMM, [741](#), [906](#)
 MPI_T_BIND_MPI_DATATYPE, [741](#), [906](#)
 MPI_T_BIND_MPI_ERRHANDLER, [741](#), [906](#)
 MPI_T_BIND_MPI_FILE, [741](#), [906](#)
 MPI_T_BIND_MPI_GROUP, [741](#), [906](#)
 MPI_T_BIND_MPI_INFO, [741](#), [906](#)
 MPI_T_BIND_MPI_MESSAGE, [741](#), [906](#)
 MPI_T_BIND_MPI_OP, [741](#), [906](#)
 MPI_T_BIND_MPI_REQUEST, [741](#), [906](#)
 MPI_T_BIND_MPI_SESSION, [741](#), [906](#), [1098](#)
 MPI_T_BIND_MPI_WIN, [741](#), [906](#)
 MPI_T_BIND_NO_OBJECT, [741](#), [747](#), [750](#),
[756](#), [758](#), [770](#), [771](#), [906](#)
 MPI_T_CB_REQUIRE_ASYNC_SIGNAL_SAFE, [767](#), [767](#), [907](#)
 MPI_T_CB_REQUIRE_MPI_RESTRICTED, [767](#), [767](#), [907](#)
 MPI_T_CB_REQUIRE_NONE, [766](#), [767](#), [907](#)
 MPI_T_CB_REQUIRE_THREAD_SAFE, [767](#), [907](#)
 MPI_T_CVAR_HANDLE_NULL, [750](#), [905](#)
 MPI_T_ENUM_NULL, [747](#), [756](#), [770](#), [905](#)
 MPI_T_ERR_CANNOT_INIT, [785](#), [893](#)
 MPI_T_ERR_CVAR_SET_NEVER, [751](#), [785](#),
[893](#)
 MPI_T_ERR_CVAR_SET_NOT_NOW, [751](#),
[785](#), [893](#)
 MPI_T_ERR_INVALID, [785](#), [893](#), [1101](#)
 MPI_T_ERR_INVALID_HANDLE, [758](#), [774](#),
[785](#), [893](#)
 MPI_T_ERR_INVALID_INDEX, [26](#), [766](#), [770](#),
[785](#), [785](#), [802](#), [893](#), [1098](#)
 MPI_T_ERR_INVALID_ITEM, [26](#), [802](#), [893](#),
[1098](#)
 MPI_T_ERR_INVALID_NAME, [748](#), [757](#), [770](#),
[781](#), [785](#), [893](#), [1101](#)
 MPI_T_ERR_INVALID_SESSION, [785](#), [785](#),
[893](#)
 MPI_T_ERR_MEMORY, [785](#), [893](#)
 MPI_T_ERR_NOT_ACCESSIBLE, [767](#), [767](#),
[768](#), [785](#), [893](#)
 MPI_T_ERR_NOT_INITIALIZED, [785](#), [893](#)
 MPI_T_ERR_NOT_SUPPORTED, [766](#), [785](#),
[893](#)
 MPI_T_ERR_OUT_OF_HANDLES, [785](#), [893](#)
 MPI_T_ERR_OUT_OF_SESSIONS, [785](#), [893](#)
 MPI_T_ERR_PVAR_NO_ATOMIC, [761](#), [785](#),
[893](#)
 MPI_T_ERR_PVAR_NO_STARTSTOP, [759](#),
[759](#), [785](#), [893](#)
 MPI_T_ERR_PVAR_NO_WRITE, [760](#), [761](#),
[785](#), [893](#)
 MPI_T_PVAR_ALL_HANDLES, [759](#), [759](#),
[760–762](#), [905](#)
 MPI_T_PVAR_CLASS_AGGREGATE, [754](#),
[754](#), [907](#)
 MPI_T_PVAR_CLASS_COUNTER, [753](#), [907](#)
 MPI_T_PVAR_CLASS_GENERIC, [754](#), [907](#)
 MPI_T_PVAR_CLASS_HIGHWATERMARK,
[753](#), [907](#)
 MPI_T_PVAR_CLASS_LEVEL, [753](#), [907](#)
 MPI_T_PVAR_CLASS_LOWWATERMARK,
[753](#), [907](#)
 MPI_T_PVAR_CLASS_PERCENTAGE, [753](#),
[907](#)
 MPI_T_PVAR_CLASS_SIZE, [753](#), [907](#)
 MPI_T_PVAR_CLASS_STATE, [752](#), [907](#)
 MPI_T_PVAR_CLASS_TIMER, [754](#), [907](#)
 MPI_T_PVAR_HANDLE_NULL, [759](#), [905](#)
 MPI_T_PVAR_SESSION_NULL, [757](#), [905](#)
 MPI_T_SCOPE_ALL, [748](#), [906](#)
 MPI_T_SCOPE_ALL_EQ, [748](#), [751](#), [906](#)
 MPI_T_SCOPE_CONSTANT, [748](#), [906](#)
 MPI_T_SCOPE_GROUP, [748](#), [906](#)
 MPI_T_SCOPE_GROUP_EQ, [748](#), [751](#), [906](#)
 MPI_T_SCOPE_LOCAL, [748](#), [906](#)
 MPI_T_SCOPE_READONLY, [748](#), [906](#)
 MPI_T_SOURCE_ORDERED, [766](#), [907](#)
 MPI_T_SOURCE_UNORDERED, [766](#), [907](#)
 MPI_T_VERBOSITY_MPIDEV_ALL, [740](#), [740](#),
[906](#)
 MPI_T_VERBOSITY_MPIDEV_BASIC, [740](#),
[906](#)
 MPI_T_VERBOSITY_MPIDEV_DETAIL, [740](#),
[906](#)
 MPI_T_VERBOSITY_TUNER_ALL, [740](#), [906](#)
 MPI_T_VERBOSITY_TUNER_BASIC, [740](#),
[906](#)
 MPI_T_VERBOSITY_TUNER_DETAIL, [740](#),
[906](#)
 MPI_T_VERBOSITY_USER_ALL, [740](#), [906](#)

MPI_T_VERBOSE_USER_BASIC, 740 , 740 , 906	MPI_VERSION, 440 , 905	1
MPI_T_VERBOSE_USER_DETAIL, 740 , 740 , 906	MPI_WCHAR, 36 , 229 , 378 , 709 , 710 , 886 , 899 , 1110	2
MPI_TAG, 41 , 41 , 43 , 44 , 251 , 651 , 652 , 862 , 898 , 1099 , 1105	MPI_WEIGHTS_EMPTY, 22 , 390 , 390 , 392 , 838 , 894 , 1102	3
MPI_TAG_UB, 38 , 441 , 441 , 867 , 870 , 898	MPI_WIN_BASE, 560 , 561 , 870 , 898	4
MPI_THREAD_FUNNELED, 483 , 483 , 895	MPI_WIN_CREATE_FLAVOR, 560 , 561 , 898	5
"MPI_THREAD_FUNNELED", 493 , 917	MPI_WIN_DISP_UNIT, 560 , 561 , 898	6
MPI_THREAD_MULTIPLE, 483 , 483 , 484 , 486 , 638 , 895 , 1101	MPI_WIN_DUP_FN, 366 , 366 , 904 , 1100	7
"MPI_THREAD_MULTIPLE", 493 , 917	MPI_WIN_FLAVOR_ALLOCATE, 561 , 897 , 1094	8
MPI_THREAD_SERIALIZED, 483 , 483 , 895	MPI_WIN_FLAVOR_CREATE, 561 , 897 , 1094	9
"MPI_THREAD_SERIALIZED", 493 , 917	MPI_WIN_FLAVOR_DYNAMIC, 561 , 897	10
MPI_THREAD_SINGLE, 483 , 483 , 484 , 485 , 493 , 895	MPI_WIN_FLAVOR_SHARED, 561 , 897	11
"MPI_THREAD_SINGLE", 493 , 917	MPI_WIN_MODEL, 560 , 561 , 584 , 898	12
MPI_TYPE_DUP_FN, 370 , 370 , 904 , 1100	MPI_WIN_NULL, 379 , 560 , 890 , 903 , 1094	13
MPI_TYPE_NULL_COPY_FN, 370 , 370 , 904 , 1100	"MPI_WIN_NULL", 379 , 915	14
MPI_TYPE_NULL_DELETE_FN, 370 , 904 , 1100 , 1106	MPI_WIN_NULL_COPY_FN, 366 , 366 , 904 , 1100	15
MPI_TYPECLASS_COMPLEX, 835 , 896	MPI_WIN_NULL_DELETE_FN, 366 , 904 , 1100	16
MPI_TYPECLASS_INTEGER, 835 , 896	MPI_WIN_SEPARATE, 561 , 584 , 585 , 605 , 897	17
MPI_TYPECLASS_REAL, 835 , 896	MPI_WIN_SIZE, 560 , 561 , 898	18
MPI_UB, 4 , 26 , 808 , 1102	MPI_WIN_UNIFIED, 561 , 584 , 605 , 615 , 897	19
MPI_UINT16_T, 36 , 227 , 710 , 887 , 899 , 1107 – 1109	MPI_WTIME_IS_GLOBAL, 441 , 442 , 470 , 867 , 898	20
MPI_UINT32_T, 36 , 227 , 710 , 743 , 887 , 899 , 1107 – 1109	"native", 705 , 706 , 707 , 716 , 915	21
MPI_UINT64_T, 36 , 227 , 548 , 710 , 743 , 887 , 899 , 1107 – 1109	"nb_proc", 665 , 917	22
MPI_UINT8_T, 36 , 227 , 710 , 887 , 899 , 1107 – 1109	"no_locks", 548 , 560 , 917 , 1094	23
MPI_UNDEFINED, 24 , 24 , 42 , 65 , 86 , 87 , 89 , 90 , 92 , 93 , 95 , 151 , 153 , 155 , 160 , 182 , 310 , 311 , 329 , 330 , 332 , 395 , 406 , 407 , 708 , 713 , 830 , 831 , 878 , 895 , 1103 , 1110	"none", 611 , 917	24
MPI_UNEQUAL, 312 , 320 , 351 , 896	"num_io_nodes", 666 , 917	25
MPI_UNIVERSE_SIZE, 517 , 538 , 539 , 898	"operation", 788 , 789	26
MPI_UNSIGNED, 36 , 227 , 710 , 743 , 753 , 754 , 884 , 899	"path", 526 , 526 , 917	27
MPI_UNSIGNED_CHAR, 36 , 227 , 229 , 710 , 887 , 899	"random", 665 , 917	28
MPI_UNSIGNED_LONG, 36 , 227 , 710 , 743 , 753 , 754 , 884 , 899	"rar", 612 , 917	29
MPI_UNSIGNED_LONG_LONG, 36 , 227 , 710 , 743 , 753 , 754 , 884 , 899 , 1110	"raw", 612 , 612 , 917	30
MPI_UNSIGNED_SHORT, 36 , 227 , 710 , 884 , 899	"read_mostly", 665 , 917	31
MPI_UNWEIGHTED, 22 , 390 , 390 , 392 , 393 , 401 , 402 , 838 , 894 , 1102 , 1108	"read_once", 665 , 917	32
MPI_VAL, 19 , 860	"reverse_sequential", 665 , 918	33
	"same_disp_unit", 549 , 917 , 1101	34
	"same_op", 548 , 918	35
	"same_op_no_op", 548 , 548 , 918	36
	"same_size", 549 , 917 , 1099	37
	"sequential", 665 , 918	38
	"soft", 482 , 521 , 526 , 917	39
	"striping_factor", 666 , 917	40
	"striping_unit", 666 , 917	41
	"system", 508 , 918	42
	"thread_level", 482 , 493 , 917	43
		44
		45
		46
		47
		48

"true", 339, 340, 443, [472](#), 548, 549, 553, 554,
560, 665, 918, 1094
"universe", 789
"war", [612](#), 918
"waw", [612](#), 918
"wdir", [482](#), 526, 917
"win_allocate", [509](#), 918
"win_allocate_shared", [509](#), 918
"write_mostly", [665](#), 918
"write_once", [665](#), 918

MPI Declarations Index

This index refers to declarations needed in C and Fortran, such as address kind integers, handles, etc. The underlined page numbers is the “main” reference (sometimes there are more than one when key concepts are discussed in multiple areas). Fortran defined types are shown as `TYPE(name)`.

- MPI_Aint, 23, 23, 37, 129, 129, 132, 135, 138,
148–150, 153, 154, 164, 184–186, 546,
550, 552, 557, 561, 565, 567, 570, 573,
575, 576, 578–580, 582, 707, 713, 766,
803, 804, 838, 857, 867, 875, 877, 899,
907
 - C example usage, 172
- MPI_Comm, 19, 34, 312, 319–324, 327, 328,
331, 338, 340, 341, 350, 351, 353, 353,
355, 360, 361, 363–365, 497, 791, 792,
876, 898, 907, 1090
- TYPE(MPI_Comm), 34, 327, 353, 824, 898, 908
- MPI_Count, 23, 23, 24, 37, 764, 766, 857,
875–877, 899, 907, 1103
- MPI_Datatype, 129, 370, 847, 899–901, 907
- TYPE(MPI_Datatype), 129, 899–901, 908
- MPI_Errhandler, 451, 452–458, 860, 883, 902,
907
- TYPE(MPI_Errhandler), 451, 902, 908
- MPI_F08_Status, 878
- MPI_F08_status, 862, 905, 907, 1105
- MPI_File, 455, 456, 655, 657, 659–661, 663,
664, 666, 668, 672–679, 681–683,
685–695, 697–704, 707, 718, 719, 860,
883, 903, 907
 - C example usage, 725
- TYPE(MPI_File), 655, 903, 908
- MPI_Fint, 860, 860, 877, 878, 882, 905, 907,
1090, 1109
- MPI_Group, 310, 310, 311–316, 318, 351, 497,
561, 590, 592, 661, 792, 860, 883, 902,
907
- TYPE(MPI_Group), 310, 902, 908
- MPI_Info, 445, 471, 471, 472–475, 496, 499,
518, 521, 523, 530, 533, 535, 542, 562,
563, 655, 658, 663, 664, 666, 800, 860,
883, 902, 907, 1112
- TYPE(MPI_Info), 471, 902, 908
- MPI_Message, 99, 860, 883, 901, 907, 1103
- TYPE(MPI_Message), 99, 901, 908
- MPI_Offset, 23, 23, 37, 659, 660, 666, 668,
672–678, 687, 688, 695, 697, 699, 713,
721, 721, 722, 766, 857, 859, 875, 877,
897, 899, 907
- MPI_Op, 224, 235, 238, 240, 242, 243, 245,
246, 248, 249, 266–269, 271, 272, 288,
290–294, 570, 573, 575, 580, 582, 860,
883, 889, 902, 907
- TYPE(MPI_Op), 235, 902, 908
- MPI_Queue, 625, 626, 627–631, 883, 902, 908,
1089
- TYPE(MPI_Queue), 626, 902, 908
- MPI_Request, 73–78, 80, 81, 82, 85–89, 92–94,
103, 104, 106–111, 116–120, 460,
628–630, 634–636, 643, 647, 675–678,
683, 685, 686, 691, 692, 840, 860, 876,
883, 903, 908
- TYPE(MPI_Request), 80, 903, 908
- MPI_Session, 457, 458, 492, 861, 883, 903, 908,
1098
- TYPE(MPI_Session), 492, 903, 908, 1098
- MPI_Status, 38, 41, 43–45, 47, 80, 81, 85–89,
92–94, 96, 97, 99–101, 104, 159, 644,
650–652, 672–675, 679, 681, 682, 689,
690, 693, 694, 698, 700–704, 804, 805,
813, 815, 861–864, 894, 907, 1099,
1103, 1105
- TYPE(MPI_Status), 38, 41, 824, 825, 862–864,
894, 908, 1095, 1099, 1105
- MPI_T_cb_safety, 766, 772–774, 907, 908,
1098
- MPI_T_cvar_handle, 749, 749, 750, 751, 905,
908
- MPI_T_enum, 744, 744, 745, 746, 755, 768,
905, 908
- MPI_T_event_instance, 773, 777, 778, 908,
1098
- MPI_T_event_registration, 771, 771, 772–775,
908, 1098
- MPI_T_pvar_handle, 757, 757, 758–761, 905,
908
- MPI_T_pvar_session, 757, 757, 758–761, 905,
908
- MPI_T_source_order, 765, 765, 907, 908, 1098
- MPI_Win, 366–368, 454, 546, 550, 552, 557,
560–563, 565, 567, 570, 573, 575, 576,

1 578–580, 582, 588, 590–593, 595, 596,
2 598–600, 860, 883, 903, 908
3 TYPE(MPI_Win), [546](#), [550](#), [552](#), [557](#), 903, 908
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

DRAFT

MPI Callback Function Prototype Index

This index lists the C typedef names for callback routines, such as those used with attribute caching or user-defined reduction operations. Fortran example prototypes are given near the text of the C name.

COMM_COPY_ATTR_FUNCTION, 25, 26, 360, 361, 814, 904, 913
COMM_DELETE_ATTR_FUNCTION, 26, 360, 361, 904, 913
COMM_ERRHANDLER_FUNCTION, 451, 452, 913
COPY_FUNCTION, 26, 797, 797, 798, 905, 915
DATAREP_CONVERSION_FUNCTION, 712, 714, 904, 914
DATAREP_EXTENT_FUNCTION, 712, 713, 914
DELETE_FUNCTION, 26, 797, 798, 905, 915
FILE_ERRHANDLER_FUNCTION, 455, 456, 914
GREQUEST_CANCEL_FUNCTION, 644, 646, 914
GREQUEST_FREE_FUNCTION, 644, 645, 914
GREQUEST_QUERY_FUNCTION, 644, 645, 914
MPI_Comm_copy_attr_function, 25, 26, 360, 361, 814, 904, 909, 910
MPI_Comm_delete_attr_function, 26, 360, 361, 904, 909, 910
MPI_Comm_errhandler_fn, 800, 1109
MPI_Comm_errhandler_function, 26, 451, 452, 800, 808, 909, 911, 1109
MPI_Copy_function, 26, 797, 797, 905, 914
MPI_Datarep_conversion_function, 709, 711, 713, 713, 857, 904, 909, 912
MPI_Datarep_conversion_function_c, 712, 713, 714, 857, 904, 909, 912, 1096
MPI_Datarep_extent_function, 709, 711, 712, 713, 909, 912
MPI_Delete_function, 26, 797, 798, 905, 914
MPI_File_errhandler_fn, 800, 1109
MPI_File_errhandler_function, 455, 455, 456, 800, 909, 911, 1109
MPI_Grequest_cancel_function, 644, 646, 909, 912
MPI_Grequest_free_function, 644, 645, 909, 912
MPI_Grequest_query_function, 644, 644, 909, 912
MPI_Handler_function, 808, 1102
MPI_Session_errhandler_function, 457, 457, 909, 911, 1098
MPI_T_event_cb_function, 772, 773, 909, 1098
MPI_T_event_dropped_cb_function, 775, 776, 909, 1098
MPI_T_event_free_cb_function, 775, 909, 1098
MPI_Type_copy_attr_function, 369, 370, 904, 909, 911
MPI_Type_delete_attr_function, 369, 370, 904, 909, 911, 1106
MPI_User_function, 235, 236, 239, 857, 908, 910
MPI_User_function_c, 235, 236, 857, 908, 910, 1096
MPI_Win_copy_attr_function, 366, 366, 367, 904, 909, 910
MPI_Win_delete_attr_function, 366, 366, 367, 904, 909, 911
MPI_Win_errhandler_fn, 800, 1109
MPI_Win_errhandler_function, 453, 454, 800, 909, 911, 1109
SESSION_ERRHANDLER_FUNCTION, 457, 457, 914, 1098
TYPE_COPY_ATTR_FUNCTION, 369, 370, 904, 913
TYPE_DELETE_ATTR_FUNCTION, 369, 371, 904, 913
USER_FUNCTION, 235, 236, 913
WIN_COPY_ATTR_FUNCTION, 366, 367, 904, 913
WIN_DELETE_ATTR_FUNCTION, 366, 367, 904, 913
WIN_ERRHANDLER_FUNCTION, 453, 454, 914

MPI Function Index

The underlined page numbers refer to the function definitions.

- MPI_ABI_GET_FORTTRAN_BOOLEANS, [881](#), [881](#)
- MPI_ABI_GET_FORTTRAN_INFO, [879](#), [879](#), [1090](#)
- MPI_ABI_GET_INFO, [505](#), [874](#), [874](#), [875](#), [1090](#)
- MPI_ABI_GET_VERSION, [505](#), [874](#), [874](#), [1090](#)
- MPI_ABI_SET_FORTTRAN_BOOLEANS, [880](#), [881](#), [1090](#)
- MPI_ABI_SET_FORTTRAN_INFO, [878](#), [879](#), [881](#), [1090](#)
- MPI_ABORT, [237](#), [449](#), [450](#), [481](#), [487](#), [505](#), [506](#), [541](#), [743](#), [859](#), [1112](#)
- MPI_ACCUMULATE, [545](#), [564](#), [570](#), [572](#), [573](#), [575](#), [582](#), [585](#), [611](#), [618](#), [619](#), [924](#), [1109](#), [1112](#), [1113](#)
 - C example usage, [621](#)
 - Fortran example usage, [572](#)
 - Language-independent example usage, [618](#), [619](#)
 - MPI_Accumulate_c, [571](#), [1096](#)
- MPI_ADD_ERROR_CLASS, [463](#), [464](#), [465](#), [505](#)
- MPI_ADD_ERROR_CODE, [465](#), [465](#), [466](#), [505](#)
- MPI_ADD_ERROR_STRING, [466](#), [466](#), [467](#), [505](#)
- MPI_ADDRESS, [26](#), [807](#), [823](#), [1102](#)
- MPI_AINT_ADD, [27](#), [148](#), [149](#), [149](#), [557](#), [1101](#)
 - C example usage, [621](#)
- MPI_AINT_DIFF, [27](#), [148–150](#), [150](#), [557](#), [1101](#)
- MPI_ALLGATHER, [189](#), [193](#), [194](#), [214](#), [215](#), [217](#), [219](#), [260](#), [919](#), [922](#)
 - C example usage, [217](#)
 - MPI_Allgather_c, [214](#), [1096](#)
- MPI_ALLGATHER_INIT, [189](#), [193](#), [194](#), [281](#), [922](#), [1096](#)
 - MPI_Allgather_init_c, [282](#), [1096](#)
- MPI_ALLGATHERV, [189](#), [193](#), [194](#), [216](#), [217](#), [262](#), [922](#)
 - MPI_Allgatherv_c, [216](#), [1096](#)
- MPI_ALLGATHERV_INIT, [189](#), [193](#), [194](#), [282](#), [922](#), [1096](#)
 - MPI_Allgatherv_init_c, [283](#), [1096](#)
- MPI_ALLOC_MEM, [445](#), [446](#), [447](#), [461](#), [507](#), [509](#), [549–551](#), [553](#), [555](#), [559](#), [566](#), [597](#), [824–826](#), [838](#), [1097](#), [1099](#), [1106](#)
 - C example usage, [448](#), [621](#)
 - Fortran example usage, [447](#)
- MPI_ALLOC_MEM_CPTR, [446](#), [1099](#)
- MPI_ALLREDUCE, [189](#), [193](#), [194](#), [226](#), [235](#), [240](#), [241](#), [268](#), [919](#), [922](#), [1110](#)
 - Fortran example usage, [241](#)
 - MPI_Allreduce_c, [240](#), [1096](#)
- MPI_ALLREDUCE_INIT, [189](#), [193](#), [194](#), [290](#), [922](#), [1096](#)
 - MPI_Allreduce_init_c, [290](#), [1096](#)
- MPI_ALLTOALL, [189](#), [193](#), [194](#), [218](#), [219](#), [221](#), [222](#), [263](#), [919](#), [922](#), [1108](#)
 - C example usage, [300](#)
 - MPI_Alltoall_c, [218](#), [1096](#)
- MPI_ALLTOALL_INIT, [189](#), [193](#), [194](#), [284](#), [922](#), [1096](#)
 - MPI_Alltoall_init_c, [284](#), [1096](#)
- MPI_ALLTOALLV, [189](#), [193](#), [194](#), [220](#), [221](#), [222](#), [224](#), [264](#), [922](#), [1108](#)
 - MPI_Alltoallv_c, [220](#), [1096](#)
- MPI_ALLTOALLV_INIT, [189](#), [193](#), [194](#), [285](#), [922](#), [1096](#)
 - MPI_Alltoallv_init_c, [286](#), [1096](#)
- MPI_ALLTOALLW, [189](#), [193](#), [194](#), [222](#), [223](#), [224](#), [266](#), [922](#), [1108](#)
 - MPI_Alltoallw_c, [222](#), [1096](#)
- MPI_ALLTOALLW_INIT, [189](#), [193](#), [194](#), [287](#), [922](#), [1096](#)
 - MPI_Alltoallw_init_c, [287](#), [1096](#)
- MPI_ATTR_DELETE, [26](#), [372](#), [798](#), [799](#)
- MPI_ATTR_GET, [26](#), [372](#), [799](#), [867](#)
 - Fortran example usage, [868](#), [869](#)
- MPI_ATTR_PUT, [26](#), [372](#), [798](#), [867](#), [870](#)
 - Fortran example usage, [868](#)
- MPI_BARRIER, [189](#), [193](#), [194](#), [196](#), [196](#), [253](#), [504](#), [608–610](#), [723](#), [919](#), [922](#)
 - C example usage, [488](#), [617](#)
 - Fortran example usage, [503](#)
 - Language-independent example usage, [607–610](#), [618](#), [619](#)
- MPI_BARRIER_INIT, [189](#), [193](#), [194](#), [274](#), [922](#), [1096](#)
- MPI_BCAST, [17](#), [189](#), [193](#), [194](#), [196](#), [197](#), [226](#), [254](#), [299](#), [789](#), [922](#)
 - C example usage, [198](#), [296](#), [297](#), [299](#), [343](#)
 - MPI_Bcast_c, [197](#), [1096](#)

MPI_BCAST_INIT, 17, 189, 193, 194, 275 , 922, 1096	1
MPI_Bcast_init_c, 275 , 1096	2
MPI_BSEND, 17, 30, 53 , 54, 71, 615, 919, 921	3
C example usage, 614	4
Fortran example usage, 57, 58	5
MPI_Bsend_c, 53 , 1096	6
MPI_BSEND_INIT, 107 , 111, 921	7
MPI_Bsend_init_c, 107 , 1096	8
MPI_BUFFER_ATTACH, 62 , 69, 81, 615, 921, 1093	9
C example usage, 68, 488, 614	10
MPI_Buffer_attach_c, 62 , 1096	11
MPI_BUFFER_DETACH, 64 , 69, 615, 921, 1093, 1096, 1105	12
C example usage, 68, 614	13
MPI_Buffer_detach_c, 64 , 1096	14
MPI_BUFFER_FLUSH, 66 , 69, 1093	15
MPI_BUFFER_IFLUSH, 67 , 1093	16
MPI_CANCEL, 26, 57, 81, 95, 103, 103 , 104, 105, 251, 274, 488, 577, 629, 638, 643, 646, 647, 800, 921, 1096	17
C example usage, 488	18
MPI_CART_COORDS, 383, 398, 398 , 1091, 1111	19
Fortran example usage, 404	20
MPI_CART_CREATE, 349, 383, 384, 384 , 385, 386, 406, 408, 839, 923, 1099, 1111	21
Fortran example usage, 434, 839	22
MPI_CART_GET, 383, 396, 397, 397 , 1091, 1111	23
Fortran example usage, 434	24
MPI_CART_MAP, 384 , 405 , 406, 1104	25
MPI_CART_RANK, 383, 397 , 398, 1111	26
Fortran example usage, 404	27
MPI_CART_SHIFT, 383, 403, 403 , 404, 407, 1111	28
Fortran example usage, 404, 434	29
Language-independent example usage, 414	30
MPI_CART_SUB, 383, 404 , 405, 406, 923, 1111	31
Language-independent example usage, 405	32
MPI_CARTDIM_GET, 383, 396, 396 , 1111	33
Language-independent example usage, 414	34
MPI_CLOSE_PORT, 531 , 531 , 534	35
MPI_COMM_ACCEPT, 529, 530, 531 , 532, 533, 539, 541, 923	36
C example usage, 536, 537	37
MPI_COMM_ACK_FAILED, 793, 793	38
C example usage, 794	39
MPI_COMM_ATTACH_BUFFER, 60 , 69, 81, 1093	40
C example usage, 68	41
MPI_Comm_attach_buffer_c, 60	42
MPI_COMM_C2F, 860	43
MPI_COMM_CALL_ERRHANDLER, 467 , 469	44
MPI_COMM_COMPARE, 320 , 351	45
MPI_COMM_CONNECT, 462, 532 , 533, 539, 540, 923	46
C example usage, 536, 537	47
MPI_COMM_CREATE, 318, 321, 324, 324 , 325–330, 383, 922, 1108	48
C example usage, 326, 343, 344, 346	
MPI_COMM_CREATE_ERRHANDLER, 26, 450, 451, 451 , 452, 807, 911, 913, 1106	
MPI_COMM_CREATE_FROM_GROUP, 318, 321, 337 , 338, 354, 441, 1091, 1098	
C example usage, 499	
Fortran example usage, 503	
MPI_COMM_CREATE_GROUP, 318, 321, 327, 327, 328, 329, 338, 922, 1104	
MPI_COMM_CREATE_KEYVAL, 26, 359, 360 , 362, 372, 797, 866, 867, 910, 913, 1106, 1110	
MPI_COMM_DELETE_ATTR, 26, 359, 362–364, 365 , 372, 799	
MPI_COMM_DETACH_BUFFER, 63 , 69, 1093	
C example usage, 68	
MPI_Comm_detach_buffer_c, 63	
MPI_COMM_DISCONNECT, 111, 112, 362, 373, 487, 494, 495, 504, 523, 541, 541 , 542, 919, 922, 1091	
MPI_COMM_DUP, 312, 318, 321, 321 , 322, 323, 326, 352, 355, 359, 361, 365, 372, 380, 789, 790, 797, 809, 922, 1091, 1096, 1104	
C example usage, 345	
MPI_COMM_DUP_FN, 26, 362, 362 , 363, 818, 904, 1100, 1106	
MPI_COMM_DUP_WITH_INFO, 318, 321, 322, 322 , 323, 324, 339, 359, 361, 365, 373, 809, 922, 1104	
MPI_COMM_F2C, 860	
MPI_COMM_FLUSH_BUFFER, 66 , 69, 1093	
MPI_COMM_FREE, 112, 318, 322, 338 , 352, 355, 362, 363, 365, 373, 487, 490, 494, 504, 523, 541, 542, 790, 798, 919, 922, 1094	
MPI_COMM_FREE_KEYVAL, 26, 359, 363 , 372, 798	
MPI_COMM_FROMINT, 883	
MPI_COMM_GET_ATTR, 26, 359, 364, 364 , 372, 440, 799, 820, 867, 870	
C example usage, 867	
Fortran example usage, 868, 869	
MPI_COMM_GET_ERRHANDLER, 26, 451, 453 , 807, 1112	
MPI_COMM_GET_FAILED, 792, 792 , 793	
C example usage, 794	

- MPI_COMM_GET_INFO, 339, 341, [341](#), 507,
 508, 1090, 1097, 1104
 MPI_COMM_GET_NAME, 375, 376, [376](#), 377,
 1094, 1110
 MPI_COMM_GET_PARENT, 376, 481, 487,
 492, 519, 522, [522](#)
 C example usage, 527
 MPI_COMM_GROUP, 21, 309, 312, [312](#),
 318–320, 351, 451, 1112
 C example usage, 326, 346
 MPI_COMM_IDUP, 318, 321, 323, [323](#), 324,
 349, 359, 361, 365, 372, 809, 922,
 1096, 1099, 1104
 MPI_COMM_IDUP_WITH_INFO, 318, 321,
 323, 324, 339, 359, 361, 365, 373, 809,
 1097
 MPI_COMM_IFLUSH_BUFFER, [67](#), 1093
 MPI_COMM_IS_REVOKED, [791](#), 792
 MPI_COMM_JOIN, 492, [542](#), 543, 544
 MPI_COMM_NULL_COPY_FN, 25, 26, 362,
 362, 363, 814, 866, 904, 1100, 1106
 MPI_COMM_NULL_DELETE_FN, 26, 362,
 362, 363, 904, 1100
 MPI_COMM_RANK, 17, 319, [319](#), 351
 C example usage, 33
 Fortran example usage, 821
 MPI_COMM_REMOTE_GROUP, [352](#)
 MPI_COMM_REMOTE_SIZE, [351](#), 352
 C example usage, 330
 MPI_COMM_REVOKE, 788, 789, [791](#)
 MPI_COMM_SET_ATTR, 26, 359, 362, [363](#),
 372, 798, 820, 867, 870
 C example usage, 867
 Fortran example usage, 869
 MPI_COMM_SET_ERRHANDLER, 26, 451,
 452, 807
 MPI_COMM_SET_INFO, 274, 339, [340](#), 341,
 922, 1097, 1099, 1104
 MPI_COMM_SET_NAME, 375, [375](#)
 MPI_COMM_SIZE, 319, [319](#), 320, 351, 618
 MPI_COMM_SPAWN, 481, 482, 507, 508, 511,
 512, 517, 518, [518](#), 519–527, 539, 540,
 923, 1098
 C example usage, 520, 527
 Fortran example usage, 520
 MPI_COMM_SPAWN_MULTIPLE, 481, 482,
 507, 508, 512, 517, 522, [523](#), 524, 525,
 540, 923, 1098
 C example usage, 525
 Fortran example usage, 525
 MPI_COMM_SPLIT, 318, 321, 325, 326, [328](#),
 329, 330, 380, 383, 385, 386, 405–407,
 789, 922, 1108
 C example usage, 330, 356, 357
 MPI_COMM_SPLIT_TYPE, 318, 321, [331](#),
 334, 336, 339, 444, 549, 551, 556, 614,
 877, 922, 1094, 1096, 1104
 C example usage, 333, 337, 444
 MPI_COMM_TEST_INTER, 349, [350](#)
 MPI_COMM_TOINT, [882](#)
 MPI_COMPARE_AND_SWAP, 545, 564, [576](#),
 618, 924
 C example usage, 621
 Language-independent example usage, 619
 MPI_CONVERSION_FN_NULL, [715](#), 904, 1100
 MPI_CONVERSION_FN_NULL_C, [715](#), 857,
 904, 1096
 MPI_DIMS_CREATE, 383, 385, [385](#), 386, 1097
 Fortran example usage, 434
 Language-independent example usage, 386
 MPI_DIST_GRAPH_CREATE, 339, 383, 388,
 390, 391–394, 402, 403, 407, 923, 1108
 C example usage, 394
 Language-independent example usage, 393
 MPI_DIST_GRAPH_CREATE_ADJACENT,
 339, 383, 388, [388](#), 389, 390, 393, 402,
 407, 923, 1104, 1108
 Language-independent example usage, 393
 MPI_DIST_GRAPH_NEIGHBORS, 383, 401,
 401, 402, 408, 1104, 1108
 MPI_DIST_GRAPH_NEIGHBORS_COUNT,
 383, 401, [401](#), 402, 1102, 1108
 MPI_DUP_FN, 26, 362, [798](#), 905
 MPI_ENQUEUE_START, 626, 628, [628](#), 629,
 630, 632, 634, 635, 637–639, 1089
 MPI_ENQUEUE_STARTALL, 626, 629, [629](#),
 632, 637–639, 641, 1089
 C example usage, 640
 MPI_ENQUEUE_WAIT, 626, 628, [629](#), 630,
 630, 631, 632, 638, 1089
 MPI_ENQUEUE_WAITALL, 628, 630, 631, [631](#),
 632, 637, 638, 641, 642, 1089
 C example usage, 640
 MPI_ERRHANDLER_C2F, 505, [860](#)
 MPI_ERRHANDLER_CREATE, 26, [807](#), 1102,
 1106
 MPI_ERRHANDLER_F2C, 505, [860](#)
 MPI_ERRHANDLER_FREE, 451, [458](#), 487, 494,
 505, 1112
 MPI_ERRHANDLER_FROMINT, [883](#)
 MPI_ERRHANDLER_GET, 26, [807](#), 1102, 1112
 MPI_ERRHANDLER_SET, 26, [807](#), 1102
 MPI_ERRHANDLER_TOINT, [883](#)
 MPI_ERROR_CLASS, 460, [460](#), 463, 505
 MPI_ERROR_STRING, 459, [459](#), 460, 463,
 466, 505
 MPI_EXSCAN, 190, 193, 227, 235, 248, [248](#),
 272, 922, 1108

- MPI_Exscan_c, [248](#), [1096](#)
- MPI_EXSCAN_INIT, [190](#), [193](#), [294](#), [922](#), [1096](#)
 - MPI_Exscan_init_c, [295](#), [1096](#)
- MPI_F_SYNC_REG, [149](#), [620](#), [812](#), [827](#), [828](#), [829](#), [848–852](#), [854](#), [1106](#)
 - Fortran example usage, [851](#)
 - Language-independent example usage, [620](#)
- MPI_FETCH_AND_OP, [545](#), [564](#), [572](#), [575](#), [575](#), [924](#)
- MPI_FILE_C2F, [860](#)
- MPI_FILE_CALL_ERRHANDLER, [468](#), [469](#)
- MPI_FILE_CLOSE, [487](#), [494](#), [542](#), [655](#), [656](#), [657](#), [658](#), [925](#)
 - Fortran example usage, [680](#), [684](#)
- MPI_FILE_CREATE_ERRHANDLER, [450](#), [451](#), [455](#), [456](#), [911](#), [914](#), [1106](#)
- MPI_FILE_DELETE, [657](#), [658](#), [658](#), [663](#), [665](#), [727](#), [925](#)
- MPI_FILE_F2C, [860](#)
- MPI_FILE_FROMINT, [883](#)
- MPI_FILE_GET_AMODE, [661](#), [662](#)
 - Fortran example usage, [662](#)
- MPI_FILE_GET_ATOMICITY, [719](#), [719](#)
- MPI_FILE_GET_BYTE_OFFSET, [679](#), [688](#), [688](#), [689](#), [696](#)
- MPI_FILE_GET_ERRHANDLER, [451](#), [456](#), [727](#), [1112](#)
- MPI_FILE_GET_GROUP, [661](#), [661](#)
- MPI_FILE_GET_INFO, [508](#), [663](#), [664](#), [664](#), [665](#), [1097](#), [1113](#)
- MPI_FILE_GET_POSITION, [688](#), [688](#)
- MPI_FILE_GET_POSITION_SHARED, [695](#), [695](#), [696](#), [720](#)
- MPI_FILE_GET_SIZE, [660](#), [661](#), [722](#)
- MPI_FILE_GET_TYPE_EXTENT, [707](#), [707](#), [715](#), [1096](#)
 - MPI_File_get_type_extent_c, [708](#), [1096](#)
- MPI_FILE_GET_VIEW, [668](#), [669](#), [925](#)
- MPI_FILE_IREAD, [670](#), [683](#), [684](#), [696](#), [717](#), [925](#)
 - Fortran example usage, [684](#)
 - MPI_File_iread_c, [684](#), [1096](#)
- MPI_FILE_IREAD_ALL, [670](#), [685](#), [685](#), [925](#), [1101](#)
 - MPI_File_iread_all_c, [685](#), [1096](#)
- MPI_FILE_IREAD_AT, [670](#), [676](#), [676](#), [925](#)
 - C example usage, [725](#)
 - MPI_File_iread_at_c, [676](#), [1096](#)
- MPI_FILE_IREAD_AT_ALL, [670](#), [676](#), [677](#), [925](#), [1101](#)
 - MPI_File_iread_at_all_c, [677](#), [1096](#)
- MPI_FILE_IREAD_SHARED, [670](#), [691](#), [692](#), [925](#)
 - MPI_File_iread_shared_c, [691](#), [1096](#)
- MPI_FILE_IWRITE, [670](#), [686](#), [686](#), [925](#)
 - MPI_File_iwrite_c, [686](#), [1096](#)
- MPI_FILE_IWRITE_ALL, [670](#), [686](#), [687](#), [925](#), [1101](#)
 - MPI_File_iwrite_all_c, [687](#), [1096](#)
- MPI_FILE_IWRITE_AT, [670](#), [677](#), [678](#), [925](#)
 - C example usage, [725](#)
 - MPI_File_iwrite_at_c, [678](#), [1096](#)
- MPI_FILE_IWRITE_AT_ALL, [670](#), [678](#), [679](#), [925](#), [1101](#)
 - MPI_File_iwrite_at_all_c, [678](#), [1096](#)
- MPI_FILE_IWRITE_SHARED, [670](#), [692](#), [692](#), [925](#)
 - MPI_File_iwrite_shared_c, [692](#), [1096](#)
- MPI_FILE_OPEN, [461](#), [515](#), [655](#), [655](#), [656](#), [657](#), [663](#), [665](#), [667](#), [689](#), [721](#), [722](#), [727](#), [728](#), [925](#)
 - Fortran example usage, [680](#), [684](#)
- MPI_FILE_PREALLOCATE, [659](#), [660](#), [660](#), [717](#), [722](#), [925](#)
- MPI_FILE_READ, [669](#), [670](#), [679](#), [680](#), [682](#), [684](#), [721](#), [722](#), [925](#)
 - Fortran example usage, [680](#)
 - MPI_File_read_c, [680](#), [1096](#)
- MPI_FILE_READ_ALL, [670](#), [681](#), [682](#), [685](#), [697](#), [925](#)
 - C example usage, [697](#)
 - MPI_File_read_all_c, [681](#), [1096](#)
- MPI_FILE_READ_ALL_BEGIN, [17](#), [670](#), [696](#), [697](#), [700](#), [717](#), [854](#), [925](#)
 - C example usage, [697](#), [725](#)
 - MPI_File_read_all_begin_c, [700](#), [1096](#)
- MPI_FILE_READ_ALL_END, [670](#), [696](#), [697](#), [701](#), [717](#), [854](#), [925](#)
 - C example usage, [697](#), [725](#)
- MPI_FILE_READ_AT, [670](#), [672](#), [673](#), [674](#), [676](#), [925](#)
 - C example usage, [723](#), [724](#)
 - MPI_File_read_at_c, [672](#), [1096](#)
- MPI_FILE_READ_AT_ALL, [670](#), [673](#), [674](#), [677](#), [925](#)
 - MPI_File_read_at_all_c, [673](#), [1096](#)
- MPI_FILE_READ_AT_ALL_BEGIN, [17](#), [670](#), [698](#), [854](#), [925](#)
 - MPI_File_read_at_all_begin_c, [698](#), [1096](#)
- MPI_FILE_READ_AT_ALL_END, [670](#), [698](#), [854](#), [925](#)
- MPI_FILE_READ_ORDERED, [670](#), [693](#), [694](#), [925](#)
 - MPI_File_read_ordered_c, [693](#), [1096](#)
- MPI_FILE_READ_ORDERED_BEGIN, [17](#), [670](#), [702](#), [854](#), [925](#)
 - MPI_File_read_ordered_begin_c, [702](#), [1096](#)
- MPI_FILE_READ_ORDERED_END, [670](#), [703](#),

- 854, 925
- MPI_FILE_READ_SHARED, 670, 689, 690, 692, 694, 925
- MPI_File_read_shared_c, 690, 1096
- MPI_FILE_SEEK, 687, 688, 925
- MPI_FILE_SEEK_SHARED, 695, 695, 696, 720, 925
- MPI_FILE_SET_ATOMICITY, 657, 717, 718, 718, 925
- C example usage, 723, 725
- MPI_FILE_SET_ERRHANDLER, 451, 456, 727
- MPI_FILE_SET_INFO, 663, 663, 664, 665, 925, 1097, 1113
- MPI_FILE_SET_SIZE, 659, 659, 660, 717, 720, 722, 925
- MPI_FILE_SET_VIEW, 143, 462, 656, 663–666, 666, 667, 668, 688, 696, 705, 712, 721, 728, 925, 1113
- C example usage, 723, 724
- Fortran example usage, 680, 684, 836
- MPI_FILE_SYNC, 658, 669, 716–719, 719, 725, 925
- C example usage, 723, 724
- MPI_FILE_TOINT, 883
- MPI_FILE_WRITE, 669, 670, 682, 682, 683, 686, 721, 925
- MPI_File_write_c, 682, 1096
- MPI_FILE_WRITE_ALL, 670, 683, 683, 687, 925
- MPI_File_write_all_c, 683, 1096
- MPI_FILE_WRITE_ALL_BEGIN, 17, 670, 701, 840, 854, 925
- C example usage, 725, 728
- MPI_File_write_all_begin_c, 701, 1096
- MPI_FILE_WRITE_ALL_END, 670, 702, 854, 925
- C example usage, 725, 728
- MPI_FILE_WRITE_AT, 669, 670, 674, 675, 678, 925
- C example usage, 723, 724
- MPI_File_write_at_c, 674, 1096
- MPI_FILE_WRITE_AT_ALL, 670, 675, 675, 679, 925
- MPI_File_write_at_all_c, 675, 1096
- MPI_FILE_WRITE_AT_ALL_BEGIN, 17, 670, 699, 854, 925
- MPI_File_write_at_all_begin_c, 699, 1096
- MPI_FILE_WRITE_AT_ALL_END, 670, 700, 854, 925
- MPI_FILE_WRITE_ORDERED, 670, 693, 694, 695, 925
- MPI_File_write_ordered_c, 694, 1096
- MPI_FILE_WRITE_ORDERED_BEGIN, 17, 670, 703, 854, 925
- MPI_File_write_ordered_begin_c, 704, 1096
- MPI_FILE_WRITE_ORDERED_END, 670, 704, 854, 925
- MPI_FILE_WRITE_SHARED, 669, 670, 690, 691–693, 695, 925
- MPI_File_write_shared_c, 690, 1096
- MPI_FINALIZE, 22, 28, 29, 31, 111, 441, 449, 479, 486, 487–491, 495, 507, 514, 541, 542, 638, 656, 739, 749, 762, 764, 859, 862, 863, 923, 1091, 1098, 1104, 1112
- C example usage, 487–489
- MPI_FINALIZED, 489, 490, 490, 491, 505, 859, 1101
- MPI_FREE_MEM, 446, 447, 461, 551, 553
- C example usage, 621
- Fortran example usage, 447
- MPI_GATHER, 189, 193, 194, 198, 201, 202, 209, 215, 226, 255, 922
- C example usage, 183, 202, 207
- MPI_Gather_c, 198, 1096
- MPI_GATHER_INIT, 189, 193, 194, 276, 922, 1096
- MPI_Gather_init_c, 276, 1096
- MPI_GATHERV, 189, 193, 194, 200, 201–203, 211, 217, 257, 922
- C example usage, 183, 203–207
- MPI_Gatherv_c, 200, 1096
- MPI_GATHERV_INIT, 189, 193, 194, 277, 922, 1096
- MPI_Gatherv_init_c, 277, 1096
- MPI_GET, 545, 564, 567, 568, 573, 575, 580, 585, 602, 608, 620, 851, 924, 1112
- C example usage, 616, 617
- Fortran example usage, 568, 570
- Language-independent example usage, 607–610
- MPI_Get_c, 567, 1096
- MPI_GET_ACCUMULATE, 545, 564, 572, 573, 575, 583, 611, 618, 924, 1100
- C example usage, 621
- Language-independent example usage, 618, 619
- MPI_Get_accumulate_c, 574, 1096
- MPI_GET_ADDRESS, 26, 129, 148, 148, 149, 150, 161, 557, 807, 823, 842, 847, 864–866
- C example usage, 172, 174, 175, 182
- Fortran example usage, 149, 843, 865, 871
- MPI_GET_COUNT, 41, 42, 42, 43, 80, 160, 577, 650, 672, 1103, 1110
- Fortran example usage, 160
- MPI_Get_count_c, 42, 1096
- MPI_GET_ELEMENTS, 80, 159, 159, 160, 161,

- 650, 652, 672, 805, 1103
- Fortran example usage, 160
- MPI_Get_elements_c, 26, [159](#), 1096
- MPI_GET_ELEMENTS_X, 26, [804](#), 1092, 1103
- MPI_GET_HW_RESOURCE_INFO, 443, [443](#),
444, 1094
- C example usage, 444
- MPI_GET_LIBRARY_VERSION, 440, [440](#), 505,
1099, 1101, 1103
- MPI_GET_PROCESSOR_NAME, 442, [442](#),
443, 1112
- MPI_GET_VERSION, [439](#), 440, 505, 827, 874,
1101
- MPI_GRAPH_CREATE, 383, 386, [386](#), 387,
388, 393, 396, 399, 400, 406–408, 923,
1111
- Language-independent example usage,
387, 400
- MPI_GRAPH_GET, 383, 395, [396](#)
- MPI_GRAPH_MAP, 384, [406](#), 407
- MPI_GRAPH_NEIGHBORS, 383, 399, [399](#), 400,
408, 1109
- Language-independent example usage, 400
- MPI_GRAPH_NEIGHBORS_COUNT, 383, [398](#),
399, 400, 1109
- Language-independent example usage, 400
- MPI_GRAPHDIMS_GET, 383, 395, [395](#)
- MPI_GREQUEST_COMPLETE, 643, 645–647,
[647](#)
- C example usage, 647
- MPI_GREQUEST_START, [644](#), [644](#), [911](#), 914,
1109
- C example usage, 647
- MPI_GROUP_C2F, [860](#)
- MPI_GROUP_COMPARE, [311](#), 315
- MPI_GROUP_DIFFERENCE, [313](#)
- MPI_GROUP_EXCL, [315](#), [315](#), 317
- C example usage, 343
- MPI_GROUP_F2C, [860](#)
- MPI_GROUP_FREE, 318, [318](#), 319, 320, 451,
487, 494, 1112
- C example usage, 326, 343, 344, 346
- MPI_GROUP_FROM_SESSION_PSET, [317](#),
318, 1090, 1098
- C example usage, 499, 501
- Fortran example usage, 503
- MPI_GROUP_FROMINT, [883](#)
- MPI_GROUP_INCL, [314](#), 315, 316
- C example usage, 326, 344, 346
- MPI_GROUP_INTERSECTION, [313](#)
- MPI_GROUP_RANGE_EXCL, [316](#), 317
- MPI_GROUP_RANGE_INCL, [315](#), 316
- MPI_GROUP_RANK, [310](#), 320
- MPI_GROUP_SIZE, [310](#), 319
- MPI_GROUP_TOINT, [883](#)
- MPI_GROUP_TRANSLATE_RANKS, 311, [311](#),
1110
- MPI_GROUP_UNION, [313](#)
- MPI_IALLGATHER, 189, 193, 194, [259](#), 922
- MPI_iallgather_c, [260](#), 1096
- MPI_IALLGATHERV, 189, 193, 194, [261](#), 922
- MPI_iallgatherv_c, [261](#), 1096
- MPI_IALLREDUCE, 189, 193, 194, [267](#), 922
- C example usage, 301
- MPI_iallreduce_c, [268](#), 1096
- MPI_IALLTOALL, 189, 193, 194, [262](#), 922
- C example usage, 300
- MPI_ialltoall_c, [262](#), 1096
- MPI_IALLTOALLV, 189, 193, 194, [263](#), 922
- MPI_ialltoallv_c, [263](#), 1096
- MPI_IALLTOALLW, 189, 193, 194, [264](#), 922
- MPI_ialltoallw_c, [265](#), 1096
- MPI_IBARRIER, 189, 193, 194, 251, [252](#), 253,
299, 919, 922
- C example usage, 299–301
- MPI_IBCAST, 17, 189, 193, 194, [253](#), 254, 303,
919, 922
- C example usage, 254, 301, 302
- MPI_ibcast_c, [253](#), 1096
- MPI_IBSEND, [74](#), 81, 111, 921
- MPI_ibsend_c, [74](#), 1096
- MPI_IEXSCAN, 190, 193, [272](#), 922
- MPI_lexscan_c, [272](#), 1096
- MPI_IGATHER, 189, 193, 194, [254](#), 922
- MPI_igather_c, [255](#), 1096
- MPI_IGATHERV, 189, 193, 194, [255](#), 922
- MPI_igatherv_c, [256](#), 1096
- MPI_IMATCH, 628, 633, [634](#), 635, [635](#), 636,
637, 1089
- MPI_IMATCHALL, 628, 633, [636](#), 637, 1089
- MPI_IMPROBE, 17, 30, 95, 98, 99, [99](#), 100,
101, 103, 515, 919, 921, 1099, 1103
- MPI_IMRECV, 98, 100, 101, [102](#), 103, 487, 494,
919, 921, 1103
- MPI_imrecv_c, [102](#), 1096
- MPI_INEIGHBOR_ALLGATHER, 384, [419](#), 923,
1104
- MPI_ineighbor_allgather_c, [420](#), 1096
- MPI_INEIGHBOR_ALLGATHERV, 384, [421](#),
923, 1104
- MPI_ineighbor_allgatherv_c, [421](#), 1096
- MPI_INEIGHBOR_ALLTOALL, 384, [422](#), 923,
1104
- MPI_ineighbor_alltoall_c, [422](#), 1096
- MPI_INEIGHBOR_ALLTOALLV, 384, [423](#), 923,
1104
- MPI_ineighbor_alltoallv_c, [424](#), 1096

- 1 MPI_INEIGHBOR_ALLTOALLW, 384, [424](#), 923,
- 2 [1104](#)
- 3 MPI_Ineighbor_alltoallw_c, [425](#), 1096
- 4 MPI_INFO_C2F, 505, [860](#)
- 5 MPI_INFO_CREATE, 472, [472](#), 505
- 6 C example usage, 499
- 7 MPI_INFO_CREATE_ENV, [476](#), 505, 1098
- 8 MPI_INFO_DELETE, 461, 473, [473](#), 475, 505
- 9 MPI_INFO_DUP, 475, [475](#), 505
- 10 MPI_INFO_F2C, 505, [860](#)
- 11 MPI_INFO_FREE, 342, 443, 476, [476](#), 487, 494,
- 12 [499](#), 505, 563, 664, 766, 770, 772, 774
- 13 MPI_INFO_FROMINT, [883](#)
- 14 MPI_INFO_GET, 26, [800](#), 801, 1097, 1112
- 15 MPI_INFO_GET_NKEYS, 471, [474](#), 475, 505,
- 16 [1112](#)
- 17 MPI_INFO_GET_NTHKEY, 471, [475](#), 505,
- 18 [1112](#)
- 19 MPI_INFO_GET_STRING, 26, 471, [473](#), 474,
- 20 [505](#), 800, 801, 1097
- 21 MPI_INFO_GET_VALUELEN, 26, [801](#), 802,
- 22 [1097](#), 1112
- 23 MPI_INFO_SET, [472](#), 473, 475, 505, 801
- 24 C example usage, 499
- 25 MPI_INFO_TOINT, [883](#)
- 26 MPI_INIT, 22, 28, 31, 309, 441, 449, 476, 479,
- 27 [480](#), [480](#), 481, 483–487, 489–491, 504,
- 28 [507](#), 515, 516, 519–522, 539, 739, 742,
- 29 [749](#), 762, 859, 862, 863, 919, 923,
- 30 [1091](#), 1098, 1103, 1104, 1107, 1109
- 31 C example usage, 33, [481](#)
- 32 MPI_INIT_THREAD, 28, 309, 449, 476, 479,
- 33 [480](#), [483](#), 484–486, 489–491, 493, 515,
- 34 [516](#), 539, 739, 742, 749, 859, 923,
- 35 [1091](#), 1103, 1104, 1109
- 36 MPI_INITIALIZED, 489, [489](#), 490, 505, 859,
- 37 [1101](#)
- 38 MPI_INTERCOMM_CREATE, 318, 321, 328,
- 39 [352](#), [353](#), [354](#), [355](#), 922, 1104
- 40 C example usage, 356, 357
- 41 MPI_INTERCOMM_CREATE_FROM_GROUPS,
- 42 [318](#), 321, [352](#), [353](#), [354](#), 441, 1091,
- 43 [1098](#)
- 44 MPI_INTERCOMM_MERGE, 321, 328, 349,
- 45 [352](#), [353](#), [355](#), [355](#), 922, 1106
- 46 MPI_IProbe, 17, 30, 42, 95, 96, [96](#), 97–101,
- 47 [103](#), 515, 919, 921, 1103
- 48 C example usage, 488
- MPI_IRECV, 17, 30, [76](#), 103, 841, 842, 844,
- [846](#), 921
- C example usage, 301
- Fortran example usage, 82–84, 91, 839,
- [841](#), [842](#)
- MPI_Irecv_c, [77](#), 1096
- MPI_IREDUCE, 189, 193, 194, [266](#), 267, 922
- MPI_Ireduce_c, [266](#), 1096
- MPI_IREDUCE_SCATTER, 190, 193, 194, [269](#),
- [922](#)
- MPI_Ireduce_scatter_c, [270](#), 1096
- MPI_IREDUCE_SCATTER_BLOCK, 189, 193,
- [194](#), [268](#), 922
- MPI_Ireduce_scatter_block_c, [269](#), 1096
- MPI_IRSEND, [76](#), 921
- MPI_Irsend_c, [76](#), 1096
- MPI_IS_THREAD_MAIN, 483, [485](#), 1101
- MPI_ISCAN, 190, 193, [271](#), 922
- MPI_Iscan_c, [271](#), 1096
- MPI_ISCATTER, 189, 193, 194, [257](#), 922
- MPI_Iscatter_c, [257](#), 1096
- MPI_ISCATTERV, 189, 193, 194, [258](#), 922
- MPI_Iscatterv_c, [259](#), 1096
- MPI_ISEND, 17, 30, [73](#), 111, 819, 820, 823,
- [840](#), [841](#), [846](#), 921
- Fortran example usage, 82–84, 91, 822, 839
- MPI_Isend_c, [73](#), 1096
- MPI_ISENDRECV, [77](#), 921, 1096
- MPI_Isendrecv_c, [78](#), 1096
- MPI_ISENDRECV_REPLACE, [79](#), 921, 1096
- MPI_Isendrecv_replace_c, [79](#), 1096
- MPI_ISSEND, [75](#), 921
- MPI_Issend_c, [75](#), 1096
- MPI_KEYVAL_CREATE, 26, [797](#), 798, 915
- MPI_KEYVAL_FREE, 26, 372, [798](#)
- MPI_LOOKUP_NAME, 461, 529, 534, [535](#), 536
- MPI_MATCH, 628, 632–634, [634](#), 635–637,
- [639](#), 1089
- MPI_MATCHALL, 628, 633, [635](#), 636, [636](#),
- [640](#)–642, 1089
- C example usage, 640
- MPI_MESSAGE_C2F, [861](#), 1103
- MPI_MESSAGE_F2C, [860](#), 1103
- MPI_MESSAGE_FROMINT, [883](#)
- MPI_MESSAGE_TOINT, [883](#)
- MPI_MPROBE, 17, 95, 98–100, [100](#), 101, 103,
- [515](#), 919, 921, 1103
- MPI_MRECV, 17, 98, 100, 101, [101](#), 102, 103,
- [487](#), 494, 919, 921, 1103
- MPI_Mrecv_c, [101](#), 1096
- MPI_NEIGHBOR_ALLGATHER, 384, [408](#),
- [410](#)–412, 420, 923, 1095, 1104
- Language-independent example usage, 410
- MPI_Neighbor_allgather_c, [408](#), 1096
- MPI_NEIGHBOR_ALLGATHER_INIT, [426](#),
- [923](#), 1096
- MPI_Neighbor_allgather_init_c, [427](#), 1096
- MPI_NEIGHBOR_ALLGATHERV, 384, [411](#),
- [422](#), 923, 1095, 1104
- MPI_Neighbor_allgatherv_c, [411](#), 1096

- MPI_NEIGHBOR_ALLGATHERV_INIT, [427](#),
 [819](#), [923](#), [1096](#), [1099](#)
 MPI_Neighbor_allgatherv_init_c, [428](#),
 [1096](#)
 MPI_NEIGHBOR_ALLTOALL, [384](#), [413](#), [414](#),
 [415](#), [423](#), [923](#), [1095](#), [1104](#)
 Fortran example usage, [434](#)
 Language-independent example usage, [414](#)
 MPI_Neighbor_alltoall_c, [413](#), [1096](#)
 MPI_NEIGHBOR_ALLTOALL_INIT, [429](#), [923](#),
 [1096](#)
 MPI_Neighbor_alltoall_init_c, [429](#), [1096](#)
 MPI_NEIGHBOR_ALLTOALLV, [384](#), [415](#), [424](#),
 [923](#), [1095](#), [1104](#)
 MPI_Neighbor_alltoallv_c, [416](#), [1096](#)
 MPI_NEIGHBOR_ALLTOALLV_INIT, [430](#), [819](#),
 [923](#), [1096](#), [1099](#)
 MPI_Neighbor_alltoallv_init_c, [431](#), [1096](#)
 MPI_NEIGHBOR_ALLTOALLW, [384](#), [417](#), [417](#),
 [426](#), [923](#), [1095](#), [1104](#)
 Fortran example usage, [434](#)
 MPI_Neighbor_alltoallw_c, [418](#), [1096](#)
 MPI_NEIGHBOR_ALLTOALLW_INIT, [432](#),
 [819](#), [923](#), [1096](#), [1099](#)
 Fortran example usage, [434](#)
 MPI_Neighbor_alltoallw_init_c, [432](#), [1096](#)
 MPI_NULL_COPY_FN, [26](#), [362](#), [798](#), [905](#)
 MPI_NULL_DELETE_FN, [26](#), [362](#), [798](#), [905](#)
 MPI_OP_C2F, [860](#)
 MPI_OP_COMMUTATIVE, [243](#), [243](#), [1108](#)
 MPI_OP_CREATE, [235](#), [235](#), [238](#), [818](#), [910](#),
 [912](#), [1106](#)
 C example usage, [238](#), [249](#)
 Fortran example usage, [239](#)
 MPI_Op_create_c, [235](#), [235](#), [858](#), [910](#),
 [1096](#)
 MPI_OP_F2C, [860](#)
 MPI_OP_FREE, [238](#), [487](#), [494](#)
 MPI_OP_FROMINT, [883](#)
 MPI_OP_TOINT, [883](#)
 MPI_OPEN_PORT, [529](#), [530](#), [530](#), [532](#)–[534](#),
 [536](#)
 C example usage, [536](#), [537](#)
 MPI_PACK, [71](#), [178](#), [180](#)–[182](#), [184](#), [709](#), [714](#)
 C example usage, [182](#), [183](#)
 MPI_Pack_c, [178](#), [1096](#)
 MPI_PACK_EXTERNAL, [9](#), [184](#), [185](#), [833](#), [1110](#)
 MPI_Pack_external_c, [185](#), [1096](#)
 MPI_PACK_EXTERNAL_SIZE, [187](#)
 MPI_Pack_external_size_c, [187](#), [1096](#)
 MPI_PACK_SIZE, [71](#), [181](#), [181](#), [182](#), [1103](#)
 C example usage, [183](#)
 MPI_Pack_size_c, [181](#), [1096](#)
 MPI_PARRIVED, [30](#), [114](#), [120](#), [120](#), [121](#), [628](#),
 [637](#), [920](#), [921](#), [1096](#)
 C example usage, [125](#)
 MPI_PCONTROL, [734](#), [738](#), [738](#)
 MPI_PREADY, [114](#)–[116](#), [118](#), [118](#), [119](#)–[121](#),
 [628](#), [637](#), [920](#), [921](#), [1096](#)
 C example usage, [114](#), [122](#), [123](#), [125](#)
 MPI_PREADY_LIST, [119](#), [120](#), [1096](#)
 MPI_PREADY_RANGE, [119](#), [119](#), [120](#), [1096](#)
 MPI_PRECV_INIT, [114](#), [117](#), [117](#), [118](#), [122](#),
 [628](#), [637](#), [921](#), [1096](#)
 C example usage, [114](#), [122](#), [123](#), [125](#)
 MPI_PROBE, [17](#), [40](#), [42](#), [45](#), [95](#), [97](#), [97](#), [98](#), [99](#),
 [101](#)–[103](#), [515](#), [919](#), [921](#), [1103](#)
 Fortran example usage, [97](#), [98](#)
 MPI_PSEND_INIT, [114](#), [116](#), [117](#)–[119](#), [122](#),
 [628](#), [637](#), [921](#), [1096](#)
 C example usage, [114](#), [122](#), [123](#), [125](#)
 MPI_PUBLISH_NAME, [529](#), [530](#), [533](#), [534](#)–[536](#)
 C example usage, [537](#)
 MPI_PUT, [545](#), [564](#), [565](#), [568](#), [572](#), [579](#), [585](#),
 [602](#), [609](#), [620](#), [840](#), [851](#), [924](#), [1094](#),
 [1112](#)
 C example usage, [591](#), [597](#), [616](#)
 Fortran example usage, [851](#)
 Language-independent example usage,
 [608](#)–[610](#), [615](#)
 MPI_Put_c, [565](#), [1096](#)
 MPI_QUERY_THREAD, [485](#), [486](#), [515](#), [1101](#)
 MPI_QUEUE_CREATE, [626](#), [627](#), [627](#), [1089](#)
 C example usage, [640](#)
 MPI_QUEUE_FREE, [627](#), [628](#), [1089](#)
 C example usage, [640](#)
 MPI_QUEUE_FROMINT, [883](#)
 MPI_QUEUE_TEST, [630](#), [631](#), [631](#), [632](#), [637](#),
 [638](#), [1089](#)
 MPI_QUEUE_TOINT, [883](#)
 MPI_QUEUE_WAIT, [630](#), [632](#), [632](#), [637](#), [638](#),
 [640](#), [642](#), [1089](#)
 C example usage, [640](#)
 MPI_RACCUMULATE, [545](#), [564](#), [572](#), [575](#), [580](#),
 [582](#), [924](#)
 MPI_Raccumulate_c, [581](#), [1096](#)
 MPI_RECV, [17](#), [30](#), [34](#), [39](#), [41](#)–[45](#), [96](#), [98](#)–[100](#),
 [102](#), [128](#), [158](#), [159](#), [180](#), [190](#), [199](#), [300](#),
 [615](#), [652](#), [723](#), [762](#), [847](#), [851](#), [921](#)
 C example usage, [33](#), [300](#), [614](#)
 Fortran example usage, [49](#), [50](#), [57](#)–[59](#), [84](#),
 [97](#), [98](#), [159](#)
 MPI_Recv_c, [39](#), [1096](#)
 MPI_RECV_INIT, [17](#), [109](#), [110](#), [118](#), [921](#)
 C example usage, [640](#)
 MPI_Recv_init_c, [109](#), [1096](#)
 MPI_REDUCE, [189](#), [193](#), [194](#), [224](#), [225](#), [226](#),

- 235–238, 241, 244, 246, 247, 249, 267,
572, 575, 576, 922, 1109
- C example usage, 233, 234, 238, 345
- Fortran example usage, 228, 229, 233, 239
- MPI_Reduce_c, [225](#), 1096
- MPI_REDUCE_INIT, 189, 193, 194, [288](#), 922,
1096
- MPI_Reduce_init_c, [289](#), 1096
- MPI_REDUCE_LOCAL, 226, 227, 235, [242](#),
1106, 1108
- MPI_Reduce_local_c, [242](#), 1096
- MPI_REDUCE_SCATTER, 189, 193, 194, 226,
235, 245, [245](#), 246, 270, 919, 922
- MPI_Reduce_scatter_c, [245](#), 1096
- MPI_REDUCE_SCATTER_BLOCK, 189, 193,
194, 226, 235, [243](#), [243](#), 244, 245, 269,
919, 922, 1108
- MPI_Reduce_scatter_block_c, [243](#), 1096
- MPI_REDUCE_SCATTER_BLOCK_INIT, 189,
193, 194, [291](#), 922, 1096
- MPI_Reduce_scatter_block_init_c, [291](#),
1096
- MPI_REDUCE_SCATTER_INIT, 190, 193, 194,
[292](#), 922, 1096
- MPI_Reduce_scatter_init_c, [292](#), 1096
- MPI_REGISTER_DATAREP, 461, [711](#),
712–715, 728, 912, 914
- MPI_Register_datarep_c, [712](#), 715, 858,
912, 1096
- MPI_REMOVE_ERROR_CLASS, [464](#), 465, 505,
1094
- MPI_REMOVE_ERROR_CODE, [465](#), [465](#), 466,
505, 1094
- MPI_REMOVE_ERROR_STRING, [466](#), 467,
505, 1094
- MPI_REQUEST_C2F, [860](#)
- MPI_REQUEST_F2C, [860](#)
- MPI_REQUEST_FREE, 83, [83](#), 104, 111, 112,
251, 273, 274, 487, 494, 542, 577,
628–630, 633, [642](#), 645–647, 919–923,
1090, 1107
- C example usage, 487
- Fortran example usage, 83
- MPI_REQUEST_FROMINT, [883](#)
- MPI_REQUEST_GET_STATUS, 30, 45, 92, [92](#),
93, 628, 645, 921, 1093, 1107
- MPI_REQUEST_GET_STATUS_ALL, [93](#), 94,
95, 1093
- MPI_REQUEST_GET_STATUS_ANY, [92](#), 93,
95, 1093
- MPI_REQUEST_GET_STATUS_SOME, [94](#),
95, 1093
- MPI_REQUEST_TOINT, [883](#)
- MPI_RGET, 545, 564, [579](#), 580, 924
- C example usage, 620
- MPI_Rget_c, [579](#), 1096
- MPI_RGET_ACCUMULATE, 545, 564, 572,
[575](#), [582](#), 583, 924
- MPI_Rget_accumulate_c, [582](#), 1096
- MPI_RPUT, 545, 564, [578](#), 579, 924
- C example usage, 620
- MPI_Rput_c, [578](#), 1096
- MPI_RSEND, 17, 30, [55](#), 919, 921
- MPI_Rsend_c, [55](#), 1096
- MPI_RSEND_INIT, [108](#), 921
- MPI_Rsend_init_c, [109](#), 1096
- MPI_SCAN, 190, 193, 227, 235, [246](#), 247, 249,
271, 922
- C example usage, 249
- MPI_Scan_c, [247](#), 1096
- MPI_SCAN_INIT, 190, 193, [293](#), 922, 1096
- MPI_Scan_init_c, [294](#), 1096
- MPI_SCATTER, 189, 193, 194, [208](#), 209, 211,
212, 244, 258, 922
- C example usage, 212
- MPI_Scatter_c, [208](#), 1096
- MPI_SCATTER_INIT, 189, 193, 194, [278](#), 922,
1096
- MPI_Scatter_init_c, [279](#), 1096
- MPI_SCATTERV, 189, 193, 194, [210](#), 211, 212,
224, 246, 259, 922
- C example usage, 212, 213
- MPI_Scatterv_c, [210](#), 1096
- MPI_SCATTERV_INIT, 189, 193, 194, [280](#),
922, 1096
- MPI_Scatterv_init_c, [280](#), 1096
- MPI_SEND, 17, 30, 33, [34](#), 35, 43, 49, 128, 158,
179, 300, 656, 723, 735, 847, 848, 851,
921
- C example usage, 33, 172, 174, 175, 182,
300, 301, 735
- Fortran example usage, 49, 50, 59, 84, 97,
98, 159, 839, 842
- MPI_Send_c, [34](#), 1096
- MPI_SEND_INIT, 17, [106](#), 111, 117, 634, 921
- C example usage, 640
- MPI_Send_init_c, [106](#), 1096
- MPI_SENDRECV, [45](#), 112, 403, 921
- Fortran example usage, 170, 171
- Language-independent example usage, 414
- MPI_Sendrecv_c, [46](#), 1096
- MPI_SENDRECV_REPLACE, [47](#), 112, 921
- Fortran example usage, 404
- MPI_Sendrecv_replace_c, [47](#), 1096
- MPI_SESSION_ATTACH_BUFFER, [61](#), 69, 81,
1093
- MPI_Session_attach_buffer_c, [61](#)
- MPI_SESSION_C2F, [861](#), 1098

MPI_SESSION_CALL_ERRHANDLER, 468 , 469 , 505 , 1098		
MPI_SESSION_CREATE_ERRHANDLER, 450 , 451 , 457 , 458 , 493 , 505 , 911 , 914 , 1098		
MPI_SESSION_DETACH_BUFFER, 64 , 69 , 1093		
MPI_Session_detach_buffer_c, 64		
MPI_SESSION_F2C, 861 , 1098		
MPI_SESSION_FINALIZE, 29 , 112 , 493 , 494 , 495 , 504 , 638 , 859 , 923 , 1091 , 1095 , 1098		
C example usage, 499 , 501		
Fortran example usage, 503		
Language-independent example usage, 495		
MPI_SESSION_FLUSH_BUFFER, 66 , 69 , 1093		
MPI_SESSION_FROMINT, 883		
MPI_SESSION_GET_ERRHANDLER, 451 , 458 , 1098		
MPI_SESSION_GET_INFO, 493 , 498 , 499 , 507 , 508 , 1098		
MPI_SESSION_GET_NTH_PSET, 318 , 497 , 498 , 1098		
C example usage, 501		
Fortran example usage, 503		
MPI_SESSION_GET_NUM_PSETS, 496 , 496 , 497 , 1098		
C example usage, 501		
Fortran example usage, 503		
MPI_SESSION_GET_PSET_INFO, 499 , 1098		
MPI_SESSION_IFLUSH_BUFFER, 67 , 1093		
MPI_SESSION_INIT, 451 , 487 , 492 , 493 , 504 , 507 , 514 , 515 , 539 , 742 , 859 , 923 , 1091 , 1095 , 1098		
C example usage, 499 , 501		
Fortran example usage, 503		
MPI_SESSION_SET_ERRHANDLER, 451 , 457 , 1098		
MPI_SESSION_TOINT, 883		
MPI_SIZEOF, 26 , 802 , 802 , 1098		
MPI_SSEND, 30 , 54 , 921		
Fortran example usage, 58 , 84		
MPI_Ssend_c, 54 , 1096		
MPI_SSEND_INIT, 107 , 921		
MPI_Ssend_init_c, 108 , 1096		
MPI_START, 110 , 110 , 111 – 113 , 116 , 119 , 120 , 273 , 632 , 639 , 847 , 919 – 923		
MPI_STARTALL, 111 , 111 , 113 , 116 , 119 , 120 , 273 , 632 , 633 , 639 , 847 , 919 , 921 – 923		
MPI_STATUS_C2F, 862		
MPI_STATUS_C2F08, 863 , 1105		
MPI_STATUS_F082C, 862 , 1105		
MPI_STATUS_F082F, 864 , 1095 , 1105		
MPI_STATUS_F2C, 862		
MPI_STATUS_F2F08, 863 , 1095 , 1105		
MPI_STATUS_GET_ERROR, 44 , 1089 , 1093		1
MPI_STATUS_GET_SOURCE, 43 , 1089 , 1093		2
MPI_STATUS_GET_TAG, 43 , 1089 , 1093		3
MPI_STATUS_SET_CANCELLED, 651 , 1098		4
MPI_STATUS_SET_ELEMENTS, 650 , 650 , 805 , 1092		5
MPI_Status_set_elements_c, 26 , 650		6
MPI_STATUS_SET_ELEMENTS_X, 26 , 805 , 1092 , 1103		7
MPI_STATUS_SET_ERROR, 652 , 1093		9
MPI_STATUS_SET_SOURCE, 651 , 1093		10
MPI_STATUS_SET_TAG, 652 , 1093		11
MPI_T_CATEGORY_CHANGED, 783 , 1098		12
MPI_T_CATEGORY_GET_CATEGORIES, 782 , 782 , 783 , 785		13
MPI_T_CATEGORY_GET_CVARS, 781 , 782 , 783 , 785		14
MPI_T_CATEGORY_GET_EVENTS, 782 , 782 , 783 , 1098		15
MPI_T_CATEGORY_GET_INDEX, 781 , 781 , 785 , 1101		16
MPI_T_CATEGORY_GET_INFO, 780 , 780 , 783 , 785 , 1100 , 1101		17
MPI_T_CATEGORY_GET_NUM, 780		18
MPI_T_CATEGORY_GET_NUM_EVENTS, 781 , 781 , 1098		19
MPI_T_CATEGORY_GET_PVARS, 782 , 782 , 783 , 785		20
MPI_T_CVAR_GET_INDEX, 747 , 748 , 785 , 1101		21
MPI_T_CVAR_GET_INFO, 744 , 746 , 746 , 747 , 750 , 751 , 783 , 785 , 1100 , 1101		22
C example usage, 748		23
MPI_T_CVAR_GET_NUM, 746 , 750		24
MPI_T_CVAR_HANDLE_ALLOC, 744 , 749 , 750 , 751 , 785		25
C example usage, 751		26
MPI_T_CVAR_HANDLE_FREE, 750 , 750 , 785		27
C example usage, 751		28
MPI_T_CVAR_READ, 750 , 785		29
C example usage, 751		30
MPI_T_CVAR_WRITE, 751 , 785 , 1092		31
MPI_T_ENUM_GET_INFO, 744 , 744 , 785		32
MPI_T_ENUM_GET_ITEM, 745 , 745 , 785		33
MPI_T_EVENT_CALLBACK_GET_INFO, 774 , 774 , 1098		34
MPI_T_EVENT_CALLBACK_SET_INFO, 773 , 774 , 1098		35
MPI_T_EVENT_COPY, 768 , 769 , 777 , 777 , 778 , 1098		36
MPI_T_EVENT_GET_INDEX, 770 , 770 , 785 , 1098		37
MPI_T_EVENT_GET_INFO, 744 , 768 , 768 , 769 , 771 , 777 , 783 , 785 , 1098		38

- MPI_T_EVENT_GET_NUM, [768](#), [1098](#)
 MPI_T_EVENT_GET_SOURCE, [768](#), [779](#),
[1098](#)
 MPI_T_EVENT_GET_TIMESTAMP, [766](#), [768](#),
[778](#), [778](#), [1098](#)
 MPI_T_EVENT_HANDLE_ALLOC, [771](#), [771](#),
[773](#), [785](#), [1098](#)
 MPI_T_EVENT_HANDLE_FREE, [774](#), [774](#),
[775](#), [785](#), [1098](#)
 MPI_T_EVENT_HANDLE_GET_INFO, [772](#),
[772](#), [1098](#)
 MPI_T_EVENT_HANDLE_SET_INFO, [771](#),
[771](#), [772](#), [1098](#)
 MPI_T_EVENT_READ, [768](#), [769](#), [777](#), [777](#),
[1098](#)
 MPI_T_EVENT_REGISTER_CALLBACK, [772](#),
[772](#), [773](#), [1098](#)
 MPI_T_EVENT_SET_DROPPED_HANDLER,
[775](#), [775](#), [776](#), [1098](#)
 MPI_T_FINALIZE, [743](#), [743](#)
 C example usage, [762](#)
 MPI_T_INIT_THREAD, [742](#), [742](#), [743](#)
 C example usage, [762](#)
 MPI_T_PVAR_GET_INDEX, [756](#), [756](#), [785](#),
[1101](#)
 MPI_T_PVAR_GET_INFO, [744](#), [754](#), [755](#), [755](#),
[758](#), [760](#), [761](#), [783](#), [785](#), [1100](#), [1101](#)
 C example usage, [762](#)
 MPI_T_PVAR_GET_NUM, [754](#), [758](#)
 MPI_T_PVAR_HANDLE_ALLOC, [744](#), [758](#),
[758](#), [760](#), [785](#), [1092](#)
 C example usage, [762](#)
 MPI_T_PVAR_HANDLE_FREE, [758](#), [759](#), [785](#),
[1092](#), [1100](#)
 C example usage, [762](#)
 MPI_T_PVAR_READ, [760](#), [760](#), [761](#), [768](#), [785](#),
[1100](#)
 C example usage, [762](#)
 MPI_T_PVAR_READRESET, [756](#), [761](#), [761](#),
[768](#), [785](#), [1100](#)
 MPI_T_PVAR_RESET, [761](#), [761](#), [768](#), [785](#),
[1092](#), [1100](#)
 MPI_T_PVAR_SESSION_CREATE, [757](#), [785](#)
 C example usage, [762](#)
 MPI_T_PVAR_SESSION_FREE, [757](#), [785](#)
 MPI_T_PVAR_START, [759](#), [768](#), [785](#), [1092](#),
[1100](#)
 C example usage, [762](#)
 MPI_T_PVAR_STOP, [759](#), [768](#), [785](#), [1092](#),
[1100](#)
 MPI_T_PVAR_WRITE, [760](#), [760](#), [768](#), [785](#),
[1092](#), [1100](#)
 MPI_T_SOURCE_GET_INFO, [765](#), [765](#), [779](#),
[785](#), [1098](#)
 MPI_T_SOURCE_GET_NUM, [765](#), [765](#), [785](#),
[1098](#)
 MPI_T_SOURCE_GET_TIMESTAMP, [766](#),
[766](#), [768](#), [785](#), [1098](#)
 MPI_TEST, [30](#), [45](#), [80](#), [81](#), [82](#), [83](#), [85](#), [87](#), [92](#),
[94](#), [95](#), [104](#), [111](#), [113](#), [114](#), [120](#), [121](#),
[274](#), [487](#), [494](#), [542](#), [629](#), [645](#)–[647](#), [649](#),
[670](#), [671](#), [919](#)–[924](#)
 MPI_TEST_CANCELLED, [80](#)–[82](#), [104](#), [105](#),
[645](#), [651](#), [672](#), [920](#), [921](#)
 C example usage, [488](#)
 MPI_TESTALL, [30](#), [85](#), [88](#), [88](#), [487](#), [494](#), [515](#),
[542](#), [645](#), [646](#), [649](#)
 MPI_TESTANY, [30](#), [85](#), [86](#), [87](#), [90](#), [487](#), [494](#),
[515](#), [542](#), [645](#), [646](#), [649](#)
 MPI_TESTSOME, [30](#), [85](#), [90](#), [90](#), [91](#), [95](#), [487](#),
[494](#), [515](#), [542](#), [645](#), [646](#), [649](#)
 MPI_TOPO_TEST, [383](#), [394](#), [395](#)
 MPI_TYPE_C2F, [860](#)
 MPI_TYPE_COMMIT, [156](#), [156](#), [861](#), [1091](#)
 C example usage, [172](#), [174](#), [175](#), [182](#),
[202](#)–[207](#), [213](#), [249](#)
 Fortran example usage, [156](#), [170](#), [171](#), [568](#),
[843](#)
 MPI_TYPE_CONTIGUOUS, [18](#), [129](#), [129](#), [132](#),
[151](#), [163](#), [654](#), [707](#)
 C example usage, [202](#)
 Fortran example usage, [159](#), [160](#)
 Language-independent example usage,
[130](#), [151](#)
 MPI_Type_contiguous_c, [129](#), [1096](#)
 MPI_TYPE_CREATE_DARRAY, [18](#), [43](#), [143](#),
[144](#), [163](#)
 Fortran example usage, [147](#)
 MPI_Type_create_darray_c, [144](#), [1096](#)
 MPI_TYPE_CREATE_F90_COMPLEX, [18](#),
[163](#), [166](#), [228](#), [709](#), [812](#), [831](#), [833](#)
 MPI_TYPE_CREATE_F90_INTEGER, [18](#), [163](#),
[166](#), [227](#), [709](#), [812](#), [831](#), [833](#)
 Fortran example usage, [832](#)
 MPI_TYPE_CREATE_F90_REAL, [18](#), [163](#),
[166](#), [227](#), [709](#), [812](#), [830](#), [831](#)–[833](#), [1108](#)
 Fortran example usage, [832](#)
 MPI_TYPE_CREATE_HINDEXED, [19](#), [26](#), [129](#),
[135](#), [135](#), [138](#), [140](#), [163](#), [807](#), [857](#)
 MPI_Type_create_hindexed_c, [136](#), [1096](#)
 MPI_TYPE_CREATE_HINDEXED_BLOCK,
[19](#), [129](#), [137](#), [138](#), [163](#), [857](#), [1103](#)
 MPI_Type_create_hindexed_block_c, [138](#),
[1096](#)
 MPI_TYPE_CREATE_HVECTOR, [19](#), [26](#), [129](#),
[132](#), [132](#), [163](#), [807](#)
 C example usage, [172](#), [174](#)
 Fortran example usage, [170](#), [171](#)

- MPI_Type_create_hvector_c, [132](#), [1096](#)
- MPI_TYPE_CREATE_INDEXED_BLOCK, [18](#), [137](#), [137](#), [163](#)
 - Fortran example usage, [568](#)
 - MPI_Type_create_indexed_block_c, [137](#), [1096](#)
- MPI_TYPE_CREATE_KEYVAL, [359](#), [369](#), [372](#), [867](#), [911](#), [913](#), [1110](#)
- MPI_TYPE_CREATE_RESIZED, [26](#), [129](#), [151](#), [154](#), [154](#), [163](#), [707](#), [808](#), [1106](#)
 - C example usage, [175](#)
 - Fortran example usage, [843](#)
 - MPI_Type_create_resized_c, [154](#), [1096](#)
- MPI_TYPE_CREATE_STRUCT, [19](#), [26](#), [129](#), [138](#), [139](#), [139](#), [140](#), [152](#), [163](#), [223](#), [807](#), [857](#)
 - C example usage, [172](#), [174](#), [182](#), [205](#), [207](#), [249](#)
 - Fortran example usage, [171](#), [843](#), [871](#)
 - Language-independent example usage, [139](#), [151](#)
 - MPI_Type_create_struct_c, [139](#), [1096](#)
- MPI_TYPE_CREATE_SUBARRAY, [18](#), [21](#), [140](#), [143](#), [145](#), [163](#)
 - C example usage, [730](#)
 - Fortran example usage, [730](#)
 - MPI_Type_create_subarray_c, [141](#), [1096](#)
- MPI_TYPE_DELETE_ATTR, [359](#), [372](#), [372](#), [1106](#)
- MPI_TYPE_DUP, [18](#), [157](#), [157](#), [163](#), [1106](#)
- MPI_TYPE_DUP_FN, [370](#), [370](#), [904](#), [1100](#)
- MPI_TYPE_EXTENT, [26](#), [807](#), [1102](#)
 - Fortran example usage, [568](#)
- MPI_TYPE_F2C, [860](#)
- MPI_TYPE_FREE, [157](#), [166](#), [371](#), [487](#), [494](#)
 - Fortran example usage, [568](#)
- MPI_TYPE_FREE_KEYVAL, [359](#), [371](#), [372](#)
- MPI_TYPE_FROMINT, [883](#)
- MPI_TYPE_GET_ATTR, [359](#), [371](#), [372](#), [820](#), [867](#), [1106](#)
- MPI_TYPE_GET_CONTENTS, [163](#), [164](#), [164](#), [165](#)–[167](#), [857](#)
 - C example usage, [176](#)
 - MPI_Type_get_contents_c, [164](#), [1096](#)
- MPI_TYPE_GET_ENVELOPE, [162](#), [163](#)–[165](#), [167](#), [232](#), [832](#), [857](#)
 - C example usage, [176](#)
 - MPI_Type_get_envelope_c, [162](#), [1096](#)
- MPI_TYPE_GET_EXTENT, [26](#), [153](#), [155](#), [804](#), [807](#), [835](#), [864](#)
 - C example usage, [172](#)
 - Fortran example usage, [170](#), [171](#), [570](#), [572](#)
 - MPI_Type_get_extent_c, [26](#), [153](#), [1096](#)
- MPI_TYPE_GET_EXTENT_X, [26](#), [803](#), [1092](#), [1103](#)
- MPI_TYPE_GET_NAME, [377](#), [1094](#), [1106](#)
- MPI_TYPE_GET_TRUE_EXTENT, [154](#), [155](#), [804](#)
 - MPI_Type_get_true_extent_c, [26](#), [155](#), [1096](#)
- MPI_TYPE_GET_TRUE_EXTENT_X, [26](#), [804](#), [1092](#), [1103](#)
- MPI_TYPE_GET_VALUE_INDEX, [163](#), [230](#), [231](#), [232](#), [1093](#)
 - C example usage, [232](#)
- MPI_TYPE_HINDEXED, [26](#), [807](#), [1102](#)
- MPI_TYPE_HVECTOR, [26](#), [807](#), [1102](#)
- MPI_TYPE_INDEXED, [18](#), [133](#), [134](#), [134](#), [135](#), [136](#), [163](#)
 - C example usage, [172](#), [174](#)
 - Fortran example usage, [170](#)
 - Language-independent example usage, [134](#)
 - MPI_Type_indexed_c, [134](#), [1096](#)
- MPI_TYPE_LB, [26](#), [807](#), [1102](#)
- MPI_TYPE_MATCH_SIZE, [812](#), [834](#), [835](#), [1106](#)
 - C example usage, [835](#)
 - Fortran example usage, [836](#)
- MPI_TYPE_NULL_COPY_FN, [370](#), [370](#), [904](#), [1100](#)
- MPI_TYPE_NULL_DELETE_FN, [370](#), [904](#), [1100](#), [1106](#)
- MPI_TYPE_SET_ATTR, [359](#), [371](#), [372](#), [820](#), [867](#), [870](#), [1106](#)
- MPI_TYPE_SET_NAME, [377](#), [1106](#)
- MPI_TYPE_SIZE, [150](#), [151](#), [803](#), [878](#), [1103](#)
 - C example usage, [735](#)
 - MPI_Type_size_c, [26](#), [150](#), [1096](#)
- MPI_TYPE_SIZE_X, [26](#), [803](#), [1092](#), [1103](#)
- MPI_TYPE_STRUCT, [26](#), [807](#), [1102](#)
- MPI_TYPE_TOINT, [883](#)
- MPI_TYPE_UB, [26](#), [807](#), [1102](#)
- MPI_TYPE_VECTOR, [18](#), [130](#), [131](#), [131](#), [132](#), [135](#), [163](#)
 - C example usage, [203](#), [204](#), [206](#), [213](#)
 - Fortran example usage, [170](#), [171](#)
 - Language-independent example usage, [131](#)
 - MPI_Type_vector_c, [131](#), [1096](#)
- MPI_UNPACK, [179](#), [180](#), [714](#)
 - C example usage, [182](#), [183](#)
 - MPI_Unpack_c, [179](#), [1096](#)
- MPI_UNPACK_EXTERNAL, [9](#), [186](#), [833](#)
 - MPI_Unpack_external_c, [186](#), [1096](#)
- MPI_UNPUBLISH_NAME, [462](#), [535](#), [535](#)
 - C example usage, [537](#)
- MPI_WAIT, [41](#), [45](#), [80](#), [81](#), [81](#), [82](#)–[84](#), [86](#), [87](#), [104](#), [111](#), [113](#), [114](#), [120](#), [121](#), [251](#), [274](#)

- 1 300, 487, 494, 515, 542, 629–632, 643,
- 2 645–647, 649, 670, 671, 696, 717, 718,
- 3 840, 846, 847, 850, 918–924
- 4 C example usage, 299–301
- 5 Fortran example usage, 82–84, 91, 684
- 6 MPI_WAITALL, 85, 87, 87, 88, 251, 301, 487,
- 7 494, 515, 542, 577, 632, 633, 645, 646,
- 8 649
- 9 C example usage, 301, 620
- 10 MPI_WAITANY, 57, 85, 85, 86, 91, 487, 494,
- 11 515, 542, 645, 646, 649
- 12 C example usage, 620
- 13 Fortran example usage, 91
- 14 MPI_WAIT SOME, 85, 89, 89, 90, 91, 95, 487,
- 15 494, 515, 542, 645, 646, 649
- 16 Fortran example usage, 91
- 17 MPI_WIN_ALLOCATE, 507, 509, 546, 549,
- 18 550, 551, 553, 555, 556, 560, 561, 566,
- 19 597, 824–826, 924, 1097, 1099
- 20 MPI_Win_allocate_c, 550, 1096
- 21 MPI_WIN_ALLOCATE_CPTR, 552, 1099
- 22 MPI_WIN_ALLOCATE_SHARED, 332, 509,
- 23 546, 549, 552, 553, 555, 560, 561, 597,
- 24 825, 826, 924, 1097–1099
- 25 MPI_Win_allocate_shared_c, 552, 1096
- 26 MPI_WIN_ALLOCATE_SHARED_CPTR, 554,
- 27 1099
- 28 MPI_WIN_ATTACH, 549, 557, 558, 558, 559,
- 29 597
- 30 C example usage, 621
- 31 MPI_WIN_C2F, 860
- 32 MPI_WIN_CALL_ERRHANDLER, 467, 469
- 33 MPI_WIN_COMPLETE, 560, 586, 591, 591,
- 34 592, 594, 603, 610, 924
- 35 C example usage, 591, 616, 617
- 36 Language-independent example usage, 610
- 37 MPI_WIN_CREATE, 515, 546, 546, 549–551,
- 38 553, 555–557, 559–561, 602, 924
- 39 Fortran example usage, 568, 570, 572
- 40 MPI_Win_create_c, 547, 1096
- 41 MPI_WIN_CREATE_DYNAMIC, 462, 546, 556,
- 42 557, 557, 558–561, 603, 924
- 43 C example usage, 621
- 44 MPI_WIN_CREATE_ERRHANDLER, 450, 451,
- 45 453, 454, 911, 913, 1106
- 46 MPI_WIN_CREATE_KEYVAL, 359, 366, 372,
- 47 867, 910, 913, 1110
- 48 MPI_WIN_DELETE_ATTR, 359, 369, 372
- MPI_WIN_DETACH, 557, 559, 559, 560
- C example usage, 621
- MPI_WIN_DUP_FN, 366, 366, 904, 1100
- MPI_WIN_F2C, 860
- MPI_WIN_FENCE, 560, 568, 586, 588, 589,
- 590, 600, 601, 603, 604, 607, 612, 616,
- 851, 924
- C example usage, 614, 616
- Fortran example usage, 568, 570, 572, 851
- Language-independent example usage, 615
- MPI_WIN_FLUSH, 556, 577, 579, 598, 598,
- 603, 618, 619, 924
- C example usage, 621
- Language-independent example usage,
- 609, 618, 619
- MPI_WIN_FLUSH_ALL, 577, 579, 599, 599,
- 603, 924
- Language-independent example usage, 619
- MPI_WIN_FLUSH_LOCAL, 577, 599, 599, 603,
- 924
- Language-independent example usage, 608
- MPI_WIN_FLUSH_LOCAL_ALL, 577, 599,
- 600, 603, 924
- MPI_WIN_FREE, 367, 487, 494, 542, 560, 560,
- 924, 1094
- Fortran example usage, 570, 572
- MPI_WIN_FREE_KEYVAL, 359, 367, 372
- MPI_WIN_FROMINT, 883
- MPI_WIN_GET_ATTR, 359, 368, 372, 561,
- 820, 867, 870
- MPI_WIN_GET_ERRHANDLER, 451, 454,
- 1112
- MPI_WIN_GET_GROUP, 562, 562
- MPI_WIN_GET_INFO, 508, 562, 563, 563,
- 1097, 1104
- MPI_WIN_GET_NAME, 378, 1094
- MPI_WIN_LOCK, 548, 560, 586, 595, 595,
- 596–598, 600, 602, 604, 607–609, 920,
- 924
- C example usage, 597
- Language-independent example usage,
- 607–610
- MPI_WIN_LOCK_ALL, 548, 586, 595, 596,
- 597, 600, 602, 604, 609, 618, 619, 924
- C example usage, 620, 621
- Language-independent example usage,
- 609, 620
- MPI_WIN_NULL_COPY_FN, 366, 366, 904,
- 1100
- MPI_WIN_NULL_DELETE_FN, 366, 904, 1100
- MPI_WIN_POST, 560, 586, 590–592, 592, 593,
- 594, 597, 600, 601, 603, 610, 612, 920,
- 924, 1094
- C example usage, 616, 617
- Language-independent example usage, 610
- MPI_WIN_SET_ATTR, 359, 368, 372, 561,
- 820, 867, 870
- MPI_WIN_SET_ERRHANDLER, 451, 454
- MPI_WIN_SET_INFO, 562, 562, 563, 924,
- 1097, 1104

MPI_WIN_SET_NAME, 378	1
MPI_WIN_SHARED_QUERY, 549 , 551 , 553 , 554 , 554 , 555 , 556 , 598 , 825 , 826 , 1094 , 1099	2 3 4
C example usage, 614	5
MPI_Win_shared_query_c, 555 , 1096	6
MPI_WIN_SHARED_QUERY_CPTR, 556 , 1099	7
MPI_WIN_START, 560 , 586 , 590 , 591 , 592 , 594 , 600 , 601 , 610 , 618 , 920 , 924	8 9
C example usage, 591 , 616 , 617	10
Language-independent example usage, 610	11
MPI_WIN_SYNC, 600 , 600 , 603–607 , 610 , 618–620 , 1092 , 1094	12 13
Language-independent example usage, 608 , 618 , 619	14
MPI_WIN_TEST, 30 , 593 , 593 , 594 , 924 , 1092 , 1093	15 16
MPI_WIN_TOINT, 883	17
MPI_WIN_UNLOCK, 560 , 579 , 586 , 596 , 597 , 598 , 603 , 604 , 607 , 608 , 924	18 19
C example usage, 597	20
Language-independent example usage, 607–610	21
MPI_WIN_UNLOCK_ALL, 579 , 586 , 596 , 596 , 603 , 604 , 607 , 619 , 924	22 23
C example usage, 620 , 621	24
Language-independent example usage, 609	25
MPI_WIN_WAIT, 560 , 586 , 592 , 592 , 594 , 597 , 603 , 604 , 607 , 610 , 611 , 618 , 924	26 27
C example usage, 616 , 617	28
Language-independent example usage, 610	29
MPI_WTICK, 470 , 470 , 767 , 809 , 1093	30
MPI_WTIME, 17 , 442 , 469 , 469 , 470 , 754 , 767 , 809 , 1093	31
C example usage, 469 , 735	32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48